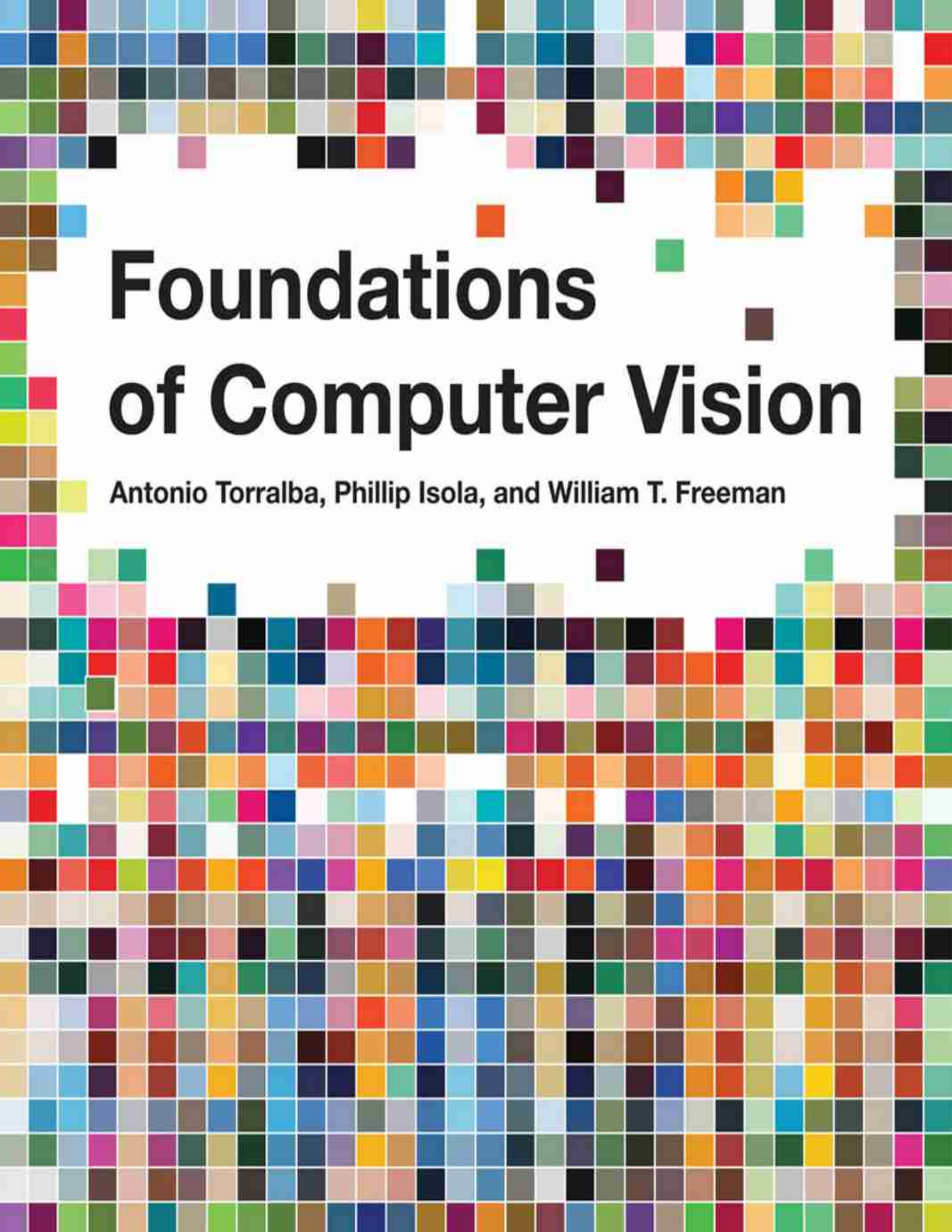




Foundations of Computer Vision

Antonio Torralba, Phillip Isola, and William T. Freeman

Foundations of Computer Vision

Adaptive Computation and Machine Learning series

Francis Bach, editor

A complete list of books published in the Adaptive Computation and Machine Learning series appears at the back of this book.

Foundations of Computer Vision

Antonio Torralba
Phillip Isola
William T. Freeman

The MIT Press
Cambridge, Massachusetts
London, England

© 2024 Antonio Torralba, Phillip Isola, and William T. Freeman

This work is subject to a Creative Commons CC-BY-NC-ND license.

This license applies only to the work in full and not to any components included with permission. Subject to such license, all rights are reserved.



The MIT Press would like to thank the anonymous peer reviewers who provided comments on drafts of this book. The generous work of academic experts is essential for establishing the authority and quality of our publications. We acknowledge with gratitude the contributions of these otherwise uncredited readers.

This book was set in Times New Roman by the authors.

Library of Congress Cataloging-in-Publication Data is available.

ISBN: 978-0-262-04897-2

10 9 8 7 6 5 4 3 2 1

d_r0

Dedicated to all the pixels.

Contents

[Preface](#)

[Notation](#)

[1 The Challenge of Vision](#)

[I FOUNDATIONS](#)

[2 A Simple Vision System](#)

[3 Looking at Images](#)

[4 Computer Vision and Society](#)

[II IMAGE FORMATION](#)

[5 Imaging](#)

[6 Lenses](#)

[7 Cameras as Linear Systems](#)

[8 Color](#)

[III FOUNDATIONS OF LEARNING](#)

[9 Introduction to Learning](#)

[10 Gradient-Based Learning Algorithms](#)

[11 The Problem of Generalization](#)

[12 Neural Networks](#)

[13 Neural Networks as Distribution Transformers](#)

[14 Backpropagation](#)

[IV FOUNDATIONS OF IMAGE PROCESSING](#)

[15 Linear Image Filtering](#)

[16 Fourier Analysis](#)

[V LINEAR FILTERS](#)

[17 Blur Filters](#)

[18 Image Derivatives](#)

[19 Temporal Filters](#)

[VI SAMPLING AND MULTISCALE IMAGE REPRESENTATIONS](#)

[20 Image Sampling and Aliasing](#)

[21 Downsampling and Upsampling Images](#)

[22 Filter Banks](#)

[23 Image Pyramids](#)

[VII NEURAL ARCHITECTURES FOR VISION](#)

[24 Convolutional Neural Nets](#)

[25 Recurrent Neural Nets](#)

[26 Transformers](#)

[VIII PROBABILISTIC MODELS OF IMAGES](#)

[27 Statistical Image Models](#)

[28 Textures](#)

[29 Probabilistic Graphical Models](#)

[IX GENERATIVE IMAGE MODELS AND REPRESENTATION LEARNING](#)

[30 Representation Learning](#)

[31 Perceptual Grouping](#)

[32 Generative Models](#)

[33 Generative Modeling Meets Representation Learning](#)

34 Conditional Generative Models

X CHALLENGES IN LEARNING-BASED VISION

35 Data Bias and Shift

36 Training for Robustness and Generality

37 Transfer Learning and Adaptation

XI UNDERSTANDING GEOMETRY

38 Representing Images and Geometry

39 Camera Modeling and Calibration

40 Stereo Vision

41 Homographies

42 Single View Metrology

43 Learning to Estimate Depth from a Single Image

44 Multiview Geometry and Structure from Motion

45 Radiance Fields

XII UNDERSTANDING MOTION

46 Motion Estimation

47 3D Motion and Its 2D Projection

48 Optical Flow Estimation

49 Learning to Estimate Motion

XIII UNDERSTANDING VISION WITH LANGUAGE

50 Object Recognition

51 Vision and Language

XIV ON RESEARCH, WRITING AND SPEAKING

52 How to Do Research

53 How to Write Papers

54 How to Give Talks

XV CLOSING REMARKS

55 A Simple Vision System—Revisited

Bibliography

Index

Preface

About this Book

This book covers foundational topics within computer vision, with an image processing and machine learning perspective. We want to build the reader's intuition and so we include many visualizations. The audience is undergraduate and graduate students who are entering the field, but we hope experienced practitioners will find the book valuable as well.

Our initial goal was to write a large book that provided a good coverage of the field. Unfortunately, the field of computer vision is just too large for that. So, we decided to write a small book instead, limiting each chapter to no more than five pages. Such a goal forced us to really focus on the important concepts necessary to understand each topic. Writing a short book was perfect because we did not have time to write a long book and you did not have time to read it. Unfortunately, we have failed at that goal, too.

Writing this Book

To appreciate the path we took to write this book, let's look at some data first. [Figure 0.1](#) shows the number of pages written as a function of time since we mentioned the idea to MIT press for the first time on November 24, 2010.

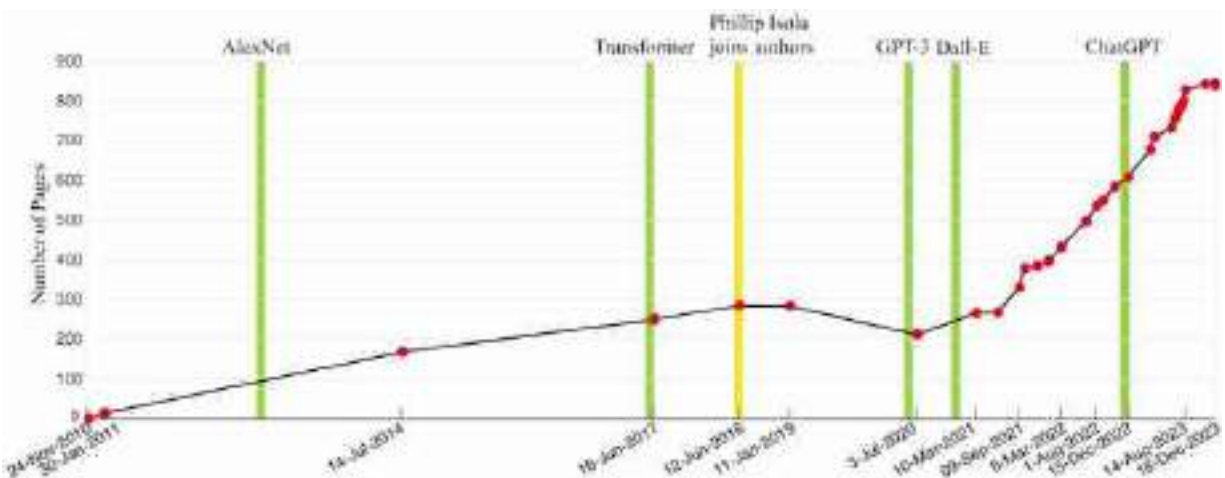


Figure 0.1: Evolution of the number of pages written as a function of time.

Writing this book has not been a linear process. As the plot shows, the evolution of the manuscript length is non-monotonic, with a period when the book shrank before growing again. Lots of things have happened since we started thinking about this book in November 2010; yes, it has taken us more than 10 years to write this book. If we knew on the first day all the work that is involved in writing a book like this one there is no way we would have started. However, from today's vantage point, with most of the work behind us, we feel happy we started this journey. We learned a lot by writing and working out the many examples we show in this book, and we hope you will too by reading and reproducing the examples yourself.

When we started writing the book, the field was moving ahead steadily, but unaware of the revolution that was about to unfold in less than 2 years. Fortunately, the deep learning revolution in 2012 made the foundations of the field more solid, providing tools to build working implementations of many of the original ideas that were introduced in the field since it began. During the first years after 2012, some of the early ideas were forgotten due to the popularity of the new approaches, but over time many of them returned. We find it interesting to look at the process of writing this book with the perspective of the changes that were happening in the field. Figure 0.1 shows some important events in the field of artificial intelligence (AI) that took place while writing this book.

Starting to write this book was like entering this cave.



We had no idea what we were getting into.

Structure of the Book

Computer vision has undergone a revolution over the last decade. It may seem like the methods we use now bear little relationship to the methods of 10 years ago. But that's not the case. The names have changed, yes, and some ideas are genuinely new, but the methods of today in fact have deep roots in the history of computer vision and AI. Throughout this book we will emphasize the unifying themes behind the concepts we present. Some chapters revisit concepts presented earlier from different perspectives.

One of the central metaphors of vision is that of **multiple views**. There is a true physical scene out there and we view it from different angles, with different sensors, and at different times. Through the collection of views we come to understand the underlying reality. This book also presents a collection of views, and our goal will be to identify the underlying foundations.

The book is organized in multiple parts, of a few chapters each, devoted to a coherent topic within computer vision. It is preferable to read them in that order as most of the chapters assume familiarity with the topics covered before them. The parts are as follows:

Part I discusses some motivational topics to introduce the problem of vision and to place it in its societal context. We will introduce a simple

vision system that will let us present concepts that will be useful throughout the book, and to refresh some of the basic mathematical tools.

Part II covers the image formation process.

Part III covers the foundations of learning using vision examples to introduce concepts of broad applicability.

Part IV provides an introduction to signal and image processing, which is foundational to computer vision.

Part V describes a collection of useful linear filters (Gaussian kernels, binomial filters, image derivatives, Laplacian filter, and temporal filters) and some of their applications.

Part VI describes multiscale image representations.

Part VII describes neural networks for vision, including convolutional neural networks, recurrent neural networks, and transformers. Those chapters will focus on the main principles without going into describing specific architectures.

Part VIII introduces statistical models of images and graphical models.

Part IX focuses on two powerful modeling approaches in the age of neural nets: generative modeling and representation learning. Generative image models are *statistical image models* that create synthetic images that follow the rules of natural image formation and proper geometry. Representation learning seeks to find useful abstract representations of images, such as vector embeddings.

Part X is composed of brief chapters that discuss some of the challenges that arise from building learning-based vision systems.

Part XI introduces geometry tools and their use in computer vision to reconstruct the 3D world structure from 2D images.

Part XII focuses on processing sequences and how to measure motion.

Part XIII deals with scene understanding and object detection.

Part XIV is a collection of chapters with advice for junior researchers on effective methods of giving presentations, writing papers, and the mentality of an effective researcher.

Part XV returns to the simple visual system and applies some of the techniques presented in the book to solve the toy problem introduced in Part I.

What Do We Not Cover?

This should be a long section, but we will keep it short. We do not provide a review on the current state of the art of computer vision; we focus instead on the foundational concepts. We do not cover in depth the many applications of computer vision such as shape analysis, object tracking, person pose analysis, or face recognition. Many of those topics are better studied by reading the latest publications from computer vision conferences and specialized monographs.

Related Books

We want to mention a number of related books that we've had the pleasure to learn from. For a number of years, we taught our computer vision class from the *Computer Vision: A Modern Approach* by Forsyth and Ponce [135], and have also used Rick Szeliski's book, *Computer Vision: Algorithms and Applications* [462]. These are excellent general texts. *Robot Vision*, by Horn [216] is an older textbook, but covers physics-based fundamentals very well. The book that enticed one of us into computer vision is still in print: *Vision*, by David Marr [318]. The intuitions are timeless and the writing is wonderful.

The geometry of vision through multiple cameras is covered thoroughly in Hartley and Zisserman's classic, *Multiple View Geometry in Computer Vision* [187]. *Solid Shape* [270], by Koenderink, offers a general treatment of three-dimensional (3D) geometry. Useful and related books include *Three-Dimensional Computer Vision*, by Faugeras [122], and *Introductory Techniques for 3D Computer Vision* [477], by Trucco and Verri.

A number of recent textbooks focus on learning. Our favorites are by Mackay [313], Bishop [52], Murphy [347], and Goodfellow, Bengio, and Courville [167]. Probabilistic models for vision are well covered in the textbook of Simon Prince [393].

Vision Science: Photons to Phenomenology, by Steve Palmer [372], is a wonderful book covering human visual perception. It includes some chapters discussing connections between studies in visual cognition and computer vision. This is an indispensable book if you are interested in the science of vision.

Signal Processing for Computer Vision, by Granlund and Knutsson [172], covers many basics of low-level vision. Ullman insightfully addresses *High-level Vision* in his book of that title, [483].

Finally, a favorite book of ours, about light and vision, is *Light and Color in the Outdoors*, by Minnaert [338], a delightful treatment of optical effects in nature.

Acknowledgments

We thank our teachers, students, and colleagues all over the world who have taught us so much and have brought us so much joy in conversations about research. This book also builds on many computer vision courses taught around the world that helped us decide which topics should be included. We thank everyone that made their slides and syllabus available. A lot of the material in this book has been created while preparing the MIT course, “Advances in Computer Vision.”

We thank our colleagues who gave us comments on the book: Ted Adelson, David Brainard, Fredo Durand, David Fouhey, Agata Lapedriza, Pietro Perona, Olga Russakovsky, Rick Szeliski, Greg Wornell, Jose María Llauradó, and Alyosha Efros. A special thanks goes to David Fouhey and Rick Szeliski for all the help and advice they provided. We also thank Rob Fergus and Yusuf Aytar for early contributions to this manuscript. Many colleagues and students have helped proof reading the book and with some of the experiments. Special thanks to Manel Baradad, Sarah Schwettmann, Krishna Murthy Jatavallabhula, Wei-Chiu Ma, Kabir Swain, Adrian Rodriguez Muñoz, Tongzhou Wang, Jacob Huh, Yen-Chen Lin, Pratyusha Sharma, Joanna Materzynska, and Shuang Li. Thanks to Manel Baradad for his help on the experiments in chapter 55, to Krishna Murthy Jatavallabhula for helping with the code for chapter 44, and Aina Torralba for help designing the book cover and several figures.

Antonio Torralba thanks Juan, Idoia, Ade, Sergio, Aina, Alberto, and Agata for all their support over many years.

Phillip Isola thanks Pam, John, Justine, Anna, DeDe, and Daryl for being a wonderful source of support along this journey.

William Freeman thanks Franny, Roz, Taylor, Maddie, Michael, and Joseph for their love and support.

Notation

We will use notes inside boxes to bring attention to important concepts, or to add additional comments without breaking the flow of the main text.

This book deals with many different fields and each has its own notation. We will stick to the following conventions throughout most of this book, and indicate when we deviate from these rules. To define the conventions we give examples of usage, from which you can infer the pattern.

General Notation

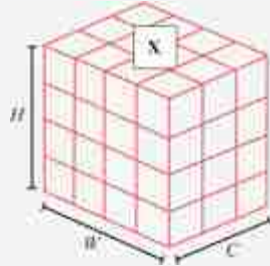
- Scalar: x, y, z .
- Vector: $\mathbf{x}, \mathbf{y}, \mathbf{z}$. We use bold letters to represent vectors, matrices, and tensors.
- Index of a vector: x_i, x_j, y_i , or $x[i], x[j], y[i]$.
- Matrix: $\mathbf{X}, \mathbf{Y}, \mathbf{Z}$. We use bold letters to represent vectors, matrices, and tensors.
- Index of a matrix: X_{ij}, Y_{jk}, Z_{ii} , or $X[i, j], Y[j, k], Z[i, i]$.
- For an indexed matrix X_{ij} or $X[i, j]$, i indexes rows and j indexes columns. We use non-bold font because X_{ij} and $X[i, j]$ are scalars.
- Slice of a matrix: $\mathbf{X}_i$ or $\mathbf{X}[i, :]; \mathbf{X}[:, j]$. Here is one example, using zero-based indexing:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \qquad \mathbf{X}[2, :] = [5 \quad 6]$$

- Tensor (i.e., multidimensional arrays): Typically, we will use lowercase bold variables to represent tensors, for example, $\mathbf{x}$. This is because tensors can have any number of dimensions (they can be one-dimensional, two-dimensional, three-dimensional, and so on). Furthermore, we will often define operators that are agnostic to the dimensionality of the tensor (they apply to N -dimensional arrays, for

any N). However, in some sections, we use uppercase to make a distinction between tensors of different shapes, and we will specify when this is the case.

A 3D tensor that could represent a $C \times H \times W$ color image:



- Index or slice of a tensor: $x[c, i, j, k]$, $\mathbf{x}[:, :, k]$; if $\mathbf{x}$ is a multidimensional tensor and we wish to slice by the first dimension, we may use $\mathbf{x}_t$ or $\mathbf{x}[t]$ or $\mathbf{x}[t, :]$, all of which have the same meaning.
- A set of N datapoints: $\{x^{(i)}\}_{i=1}^N$, $\{\mathbf{x}^{(i)}\}_{i=1}^N$, $\{\mathbf{X}^{(i)}\}_{i=1}^N$
- Dot product: $\mathbf{x}^\top \mathbf{y}$
- Matrix product: $\mathbf{AB}$
- Hadamard product (i.e., element-wise product): $\mathbf{x} \odot \mathbf{y}$, $\mathbf{A} \odot \mathbf{B}$
- Product of two scalars: ab or $a * b$

In the geometry chapters, and in some other chapters, the variables x , y , z will denote location and will have a special treatment.

Signal Processing

- Discrete signals (and images) are vectors $\mathbf{x}$, $\mathbf{y}$, $\mathbf{z}$.
- When indexing a discrete signal we will use brackets: $x[n]$, where n is a discrete index.
- Continuous signals are denoted $x(t)$, where t is a continuous variable.
- Convolution operator: $\circ$
- Cross-correlation operator: $\star$
- Discrete Fourier transforms: $X[u]$, where u is a discrete frequency index.

Images

Images are a very special signal and, whenever is convenient, we will use the following notation:

We will use the letter ℓ , from *light*, when the signal is an image. We use this letter because it is very distinct from all the other notation. But sometimes we will use other notation if it is more convenient.

- Image as a discrete 2D signal: $\ell[n, m]$ where n indexes columns and m indexes rows. The origin, $n = m = 0$, corresponds to the bottom-left side of the image unless we specify a different convention. An image has a size $N \times M$, N columns and M rows.
- Image as a matrix: $\boldsymbol{\ell}$ of size $M \times N$, M rows and N columns, which can be indexed as ℓ_{ij} .
- The Fourier transform on an image: $\mathcal{L}[u, v]$, where u and v are spatial frequencies.

Note that the way matrices and images are indexed is transposed. However, we will rarely represent images as arrays. If we do, we will use the general notation and the images will be just generic signals, $\mathbf{x}$, $\mathbf{y}$, $\mathbf{z}$.

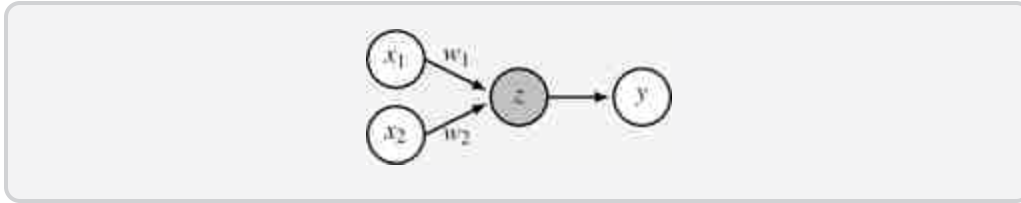
Machine Learning

- Total loss function / cost function / objective function: J
- Per-datapoint loss function: L
- Generic learnable parameters: θ

Neural Networks

- *Parameters*: θ . These include weights, $\mathbf{W}$, and biases, $\mathbf{b}$, as well as any other learnable parameters of a network.
- *Data*: $\mathbf{x}$. Data can refer to inputs to the network, activations on hidden layers, outputs from the network, and so on. Any representation of the signal being processed is considered to be data. Sometimes we will wish to distinguish between the raw inputs, hidden units, and outputs to a network, in which case we will use $\mathbf{x}$, $\mathbf{h}$, and $\mathbf{y}$ respectively. When we

need to distinguish between pre-activation hidden units and postactivation, we will use $\mathbf{z}$ for pre-activation and $\mathbf{h}$ for postactivation.



- When describing batches of tensors (as is commonly encountered in code), you may encounter $x_l[b, c, n, m]$ to represent an activation on layer l of some network, with b indexing batch element, n and m as spatial coordinates, and c indexing channels.
- Neuron values on layer l of a deep net: $\mathbf{x}_l$
- $x_l[n]$ refers to the n -th neuron on layer l . For neural networks with spatial feature maps (such as convolutional neural networks), each layer is an array of neurons, and we will use notation such as $x_l[n, m]$ to index the neuron at location n, m .
- The layer l neural representation of the i -th datapoint in a dataset is written as $\mathbf{x}_l^{(i)}$.
- We will also use $\mathbf{x}_{\text{in}}$ and $\mathbf{x}_{\text{out}}$ when we are describing a particular layer or module in a neural net and wish to speak of its inputs and outputs without having to keep track of layer indices.
- For signals with multiple channels, including neural network feature maps, the first dimension of the tensor indexes over channel. For example, in $\mathbf{x} \in \mathbb{R}^{C \times N \times M \times \dots}$, where C is the number of channels of the signal.
- For transformers, we deviate from the previous point slightly, in order to match standard notation: a set of tokens (which will be defined in the transformers chapter) is represented by a $[N \times d]$ matrix, where d is the token dimensionality.

Probabilities

We will sometimes not distinguish between random variables and realizations of those variables; which we mean should be clear from context. When it is important to make a distinction, we will use nonbold

capital letters to refer to random variables and lowercase to refer to realizations.

Suppose X, Y are discrete random variables and $\mathbf{x}, \mathbf{y}$ are realizations of those variables. X and Y may take on values in the sets $\mathcal{X}$ and $\mathcal{Y}$, respectively.

- $a = p(X = \mathbf{x} | \dots)$ is the probability of the realization $X = \mathbf{x}$, possibly conditioned on some observations (a is a scalar).
- $f = p(X | \dots)$ is the probability distribution over X , possibly conditioned on some observations (f is a function: $f: \mathcal{X} \rightarrow \mathbb{R}$). If X is discrete, f is the *probability mass function*. If X is continuous, f is the *probability density function*.
- $p(\mathbf{x} | \dots)$ is shorthand for $p(X = \mathbf{x} | \dots)$. and so forth, following these patterns.
- Suppose we have defined a named distribution, for example, p_θ , for some random variable X ; then referring to p_θ on its own is shorthand for $p_\theta(X)$.
- We will write $Z \sim p$ to indicate a random variable whose distribution is p , and we will write $z \sim p$ to indicate a sampled realization of such a variable.

For continuous random variables, all the above notations hold except that they refer to probability densities and probability density functions rather than probabilities and probability distributions. We will sometimes also use the term “probability distribution” for continuous random variables, and in those cases it should be understood that we are referring to a probability density function.

Matrix Calculus Conventions

In this book, we adopt the following conventions for matrix calculus. These conventions make the equations simpler, and that also means simpler implementations when it comes to actually writing these equations in code. Everything in this section is just definitions. There is no right or wrong to it. We could have used other conventions but we will see that these are useful ones.

Vectors are represented as column vectors with shape $[N \times 1]$:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \quad (0.1)$$

If y is a scalar and $\mathbf{x}$ is an N -dimensional vector, then the gradient $\frac{\partial y}{\partial \mathbf{x}}$ is a row vector of shape $[1 \times N]$:

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1} \quad \frac{\partial y}{\partial x_2} \quad \dots \quad \frac{\partial y}{\partial x_N} \right] \quad (0.2)$$

If $\mathbf{y}$ is an M -dimensional vector and $\mathbf{x}$ is a N -dimensional vector then the gradient (also called the Jacobian in this case) is shaped as $[M \times N]$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \dots & \frac{\partial y_1}{\partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_M}{\partial x_1} & \frac{\partial y_M}{\partial x_2} & \dots & \frac{\partial y_M}{\partial x_N} \end{bmatrix} \quad (0.3)$$

Finally, if $\mathbf{W}$ is an $[N \times M]$ dimensional matrix, and L is a scalar, then the gradient $\frac{\partial L}{\partial \mathbf{W}}$ is represented as an $[M \times N]$ dimensional matrix (note that the dimensions are transposed from what you might have expected; this makes the math simpler later):

$$\frac{\partial L}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial L}{\partial W_{11}} & \dots & \frac{\partial L}{\partial W_{N1}} \\ \vdots & \ddots & \vdots \\ \frac{\partial L}{\partial W_{1M}} & \dots & \frac{\partial L}{\partial W_{NM}} \end{bmatrix} \quad (0.4)$$

We will often draw matrices and vectors to help visualize the operations. For example, if $N = 3$ and $M = 4$:



then, the gradient will have the form:



Conventions That Will Not Be Strictly Observed

- We will often use $\mathbf{x}$ as the input to a function, and $\mathbf{y}$ as the output.
- f , g , and h are typically functions. The corresponding function spaces are F , G , H .
- Basis functions are ϕ and ψ .

Miscellaneous

- The word “dimension” has two usages in the computational sciences. The first usage is as a coordinate in a multivariate data structure, for example, “the i -th dimension of a vector” or “a 128-dimensional feature space.” The second usage is as the shape of a multidimensional array, as in “a 4D tensor.” We will use both these meanings in this book and we hope the usage will be clear from context.

1 The Challenge of Vision

1.1 Introduction

Let's start with a simple observation: every day, when you wake up, you open your eyes and see. You see without any effort; you do not get tired after a few hours from seeing too much. The same happens with all the other senses. With them you perceive the world around you: you hear the birds or the car noises, you smell the breakfast and feel the touch of the sheets, and that keeps going on and on until the end of the day. But the most surprising fact is that your brain is continuously solving very complex tasks still unmatched by any artificial system.

The apparent simplicity of perceiving the world around us produces the false intuition that it will be easy to build a machine capable of seeing as humans do. Let's ignore the senses of hearing, touch, taste, and smell, and let's focus on the visual sense. Human vision is capable of extracting information about the world around us using only the light that reflects off surfaces in the direction of our eyes. The light that reaches our eyes does not tell us what object we are looking at. It only give us information about the amount of light reaching our eye from each direction in space. Our brains have to translate the information collected by millions of photoreceptors in our retinas into an interpretation of the world in front of us. What we see is different than the light that reaches our eyes, as visual illusions prove to us.

This book will focus on the visual sense. Nonetheless, the techniques introduced aren't exclusive to images and possess the versatility to be adapted for the processing of various other signal types.

Computer vision studies how to reproduce in a computer the ability to see. Since its origins, the study of vision has been an interdisciplinary study involving many disciplines (physics, psychology, biology, neuroscience, arts, and computer science).

The goal of a vision scientist is twofold: to understand how perception works and to build systems that can interpret the world around them using

images (or image sequences) as input. In this chapter, we want to provide a broad perspective on **vision science** and the multiple disciplines that contribute to it.

1.2 Vision

David Marr [[317](#)] defines vision as “to know what is where by looking,” and adds “vision is the process of discovering from images what is present in the world, and where it is.”

It is difficult to say exactly what makes understanding the mechanisms of vision hard as we do not have a full solution yet [[71](#)]. In this section we will mention two aspects that make vision hard: the structure of the input and the structure of the desired output.

What is the goal of vision? Our eyes are sensors, and for an agent that navigates and solves tasks in the world, the role of the sensor is to provide relevant information for solving the task. In the case of computer vision, we mostly study visual perception from a disembodied perspective. There is no agent and there is no task. This perspective makes the study of vision difficult.

1.2.1 The Input: The Structure of Ambient Light

From a light source, a dense array of light rays emerges in all directions. Before these light rays reach the eye, the light interacts with the objects in the world. Let's consider a single ray emerging from the light source (we will use here a simple geometric interpretation of the structure of light). If the light ray is not directed toward the eye, this ray will strike some surface in the world and, as result, a new set of rays will be emitted in many directions. This process will continue producing more and more interactions and filling the space. In the middle of the space the observer will sense only a subset of the light rays that will strike the eye. As a result of this complex pattern of interactions even a single ray will form a complex image in the eye of the observer.

[Figure 1.1](#) shows some pictures taken when illuminating a complex scene with a laser pointer that illuminate the scene with a narrow beam of light (this is the best approximation to the image produced by a single light ray that we could do at home).

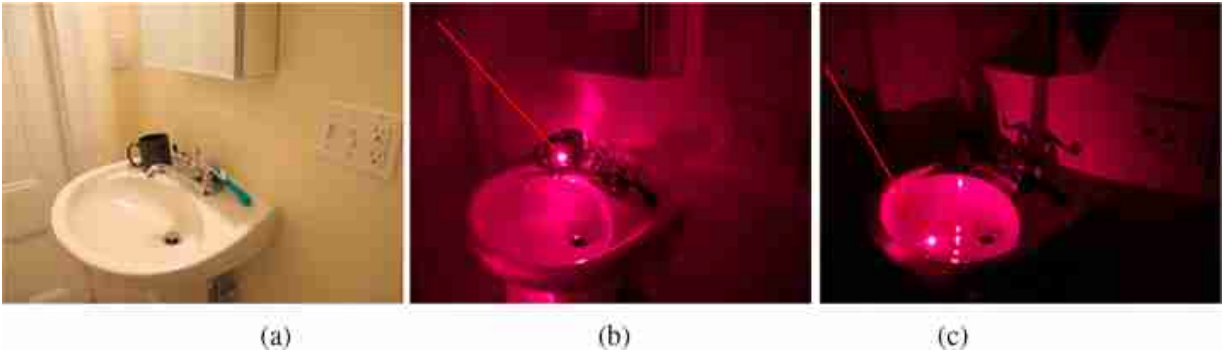


Figure 1.1: (a) Scene illuminated with a ceiling lamp. (b-c) Two images obtained by illuminating a scene with a laser pointer (the red line indicates the direction of the ray).

The resulting images show the complexity of the interactions between the different surfaces. The sum of the contribution of all the light rays emitted by the light source will give rise to a natural looking picture. On each image, the approximate direction of the light beam produced by pointer is indicated by the red arrow in [figure 1.1](#)(b-c).

Despite the complexity of the structure of the ambient light (a term coined by James J. Gibson [[161](#)]) with multiple reflections, shadows, and specular surfaces (which provide incorrect disparity information to our two eyes), our visual system has no problem in interacting with this scene, even if it is among the first things that we see just after waking up.

The pattern of light filling the space can be described by the function:

$$P(\theta, \Phi, \lambda, t, X, Y, Z) \quad (1.1)$$

where P is the light intensity of a ray passing by the world location (X, Y, Z) in the direction given by the angle (θ, Φ) , and with wavelength λ (we will study color in chapter [[8](#)]) at an instant in time t . This function, called the **plenoptic function** Edward H. Adelson and James R. Bergen [[12](#)], contains all the information needed to describe the complete pattern of light rays that fills the space.

Plenoptic function: Edward H. Adelson was going to call it the *holoscopic function*, but a wellknown holographer told him that he would punch him in the nose if he called it that.

The plenoptic function does not include information about the observer. The observer does not have access to the entire plenoptic function, only to a

small slice of it. In chapter 5 we will describe how different mechanisms can produce images by sampling the ambient light in different ways.

For a given observer, most of the light rays are occluded. Without occlusion, vision would be a lot simpler. Occlusion is the best example of how hard vision can get. Many times, properly interpreting an image will require understanding what part is occluded (e.g., we know that a person is not floating just because the legs are occluded behind a table). Unfortunately, occlusions are common. In fact, there are more occluded surfaces than visible surfaces for any given observer.

Although recovering the entire plenoptic function would have many applications, fortunately, the goal of vision is not to recover this function.

1.2.2 The Output: Measuring Light Versus Measuring Scene Properties

Vision is not a deterministic process that analyzes the input images independently of our internal state. Even when two people look at the same thing they will have different **visual awareness**. Our visual experience is greatly influenced by what we know, what we are doing, and what we expect to see.

If the goal vision were simply to measure the light intensity coming from a particular direction of space (like a photometer) then things would be easy. However, the goal of vision is to provide an interpretation of the world in terms of “meaningful” surfaces, objects, materials, etc., in order to extract all the different elements that compose the scene (anything that will be relevant to the observer). This problem is hard because most of the information is lost and the visual system needs to make a number of assumptions about the structure of the visual world in order to be able to recover the desired information from a small sample of the plenoptic function. It is also hard because our understanding about what is relevant for the observer is incomplete.

The goal of vision is not to measure light intensities, but to extract scene properties relevant for the observer. [Figure 1.2](#) shows different images that illustrate that the human visual system is trying to recover the scenes that are the cause of those images.

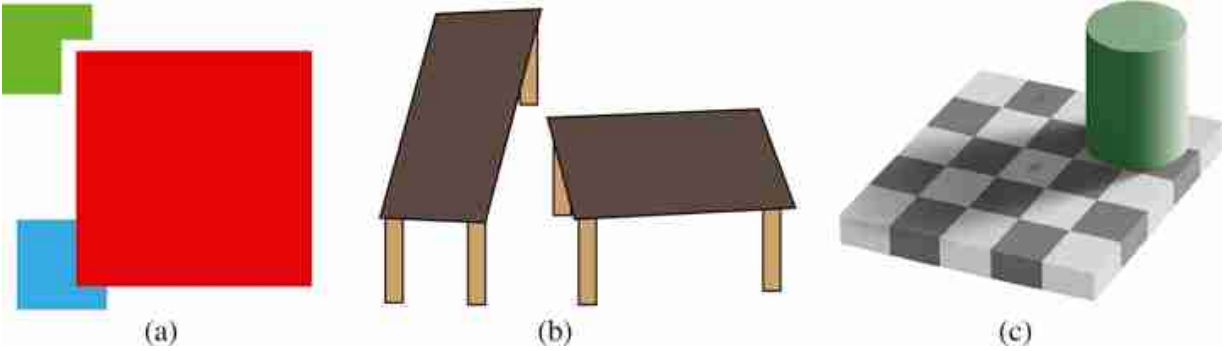


Figure 1.2: We cannot shut down the automatic mechanisms that interpret these images not as light patterns but as pictures of real three-dimensional (3D) scenes. (a) Occluding figures, (b) R. Shepard's turning tables illusion, and (c) E. Adelson's Checkershadow illusion.

In [figure 1.2\(a\)](#), we see a red square occluding a smaller blue square. But why do we see a blue square and not an L-shaped figure like the green shape on the top? If we assume that squares are typical in the world and that occlusions are common, then perceiving an occluded blue square is the most natural interpretation of the scene. Even though the green figure shows us that L-shaped figures are possible, we interpret the blue L-shaped figure as an occluded square. The next image, [figure 1.2\(b\)](#), shows the “turning tables illusion” by Roger Shepard [431]. In this illusion, both table tops have the same shape and size but one is rotated with respect to the other. The visual system insists in interpreting these objects as 3D objects giving the impression that the left table is longer than the table of the right. And this perception can not be shut down even if we know exactly how this image has been generated. The third example, [figure 1.2\(c\)](#), shows the “checkershadow illusion” by Adelson [11]. In this figure, the squares marked with A and B have the exact same intensity values. But the visual system is trying to measure the surface reflectance of the squares, removing the effect of the shadow to infer the true gray value of each square. As a result, the square in the shadow is perceived as being lighter than the square outside the shadow region, even though both have the same gray levels.

When looking at a two-dimensional (2D) picture, we automatically interpret it as a 3D scene if the right cues are present.

The goal of vision is to provide the observer with information relevant to understanding the outside world and to enable them to solve other tasks such as navigating the world, interacting with other agents, and finding

food. Tasks within computer vision include the following: detecting changes in the environment; motion estimation; object recognition and localization; recognizing materials; reading text and visual symbols; building 3D models from images; finding free space to move; finding other people; deciding if food is in good state; and understanding the behavior of animals. Not all of those tasks are at the same level. Some seem to require a lot of external knowledge while others seem solvable from the images alone.

1.3 Theories of Vision

In this section we want to describe, with a few strokes, some of the theories of vision that have contributed to and shaped modern approaches. Think of this section as a travel brochure that will show you a few snapshots of a trip, but that's not intended to be a replacement for traveling yourself. You should read the books and papers that we will mention in this section, as they are the foundations on which this fascinating field has been built.

1.3.1 The Origins of the Science of Perception

How do we know what is in the world by looking at it? Understanding how an image of the world is getting into our minds has a long history that required contributions from many scientific disciplines (art, philosophy, physics, optics, biology, psychology, neuroscience, etc.).

Ancient humans probably knew that their image of the world originated in the eyes. It just takes closing your eyes or putting one hand in front of one eye to see the corresponding image disappear. In fact, chimpanzees and orangutans might also know that the eyes are the source of visual stimuli as they seem to do **eye-gaze following** to understand what their companions are paying attention to. Humans learned to create astonishing images in caves with a degree of realism that indicates an understanding of colors, forms, shadows, and motion. They did not think that in order to create an image of a bison on the wall of the cave the only option was to attach a dead bison to the wall. They knew that a dynamic scene could be represented by static painting. Paintings are a potent visual illusion that show that the image of the object does not need the object itself.

If you have never read the works of the Greek philosophers, please do. You will be astounded by how much they knew about math and physics even though they lacked devices to confirm many of their hypotheses.

How is information about the world being picked up by our eyes? The Greeks had two competing theories: intromission theories and extramission (or emission) theories.

Early **intromission** theories (425 BC) believed that objects emitted copies of themselves (eidola or simulacra) that entered the eyes. This theory was defended by philosophers such as Demokritos, Epicurus, and Lucretius. However, it was unclear how objects could be sending copies when there were multiple observers, and how the copies did not interfere with each other. It was also unclear how copies of large objects could fit into the eye.

Extramission theory, started with Empedocles [247] and was later followed by Plato and Euclid among others. Empedocles (approx. 494–434 BC) provided one of the first theories of vision in his poem “On Nature”, and introduced many other influential ideas such that all things are composed of four elements: fire, earth, air, and water. He said that the eye contained the four elements and that the fire was responsible of creating **rays** that emanated from the eyes, like fingers that sensed the world. The extramission theory explained why sometimes eyes shined at night, in particular cat's eyes, which were assumed to contain so much fire that even humans could see it occasionally. Of course, now we know that when cats' eyes appear to shine, they are reflecting light from other sources, but without a theory of light and reflection it made sense to believe that one was observing rays emitted by the eyes.

The extramission theory attributed the glow observed in cat eyes to the presence of fire inside them.



Plato's theory of vision (427–347 BC) was very influential. Plato's theory contained both intromission and extramission elements. The following is a fragment of Plato's dialog *Timaeus* [388], which provides a description of

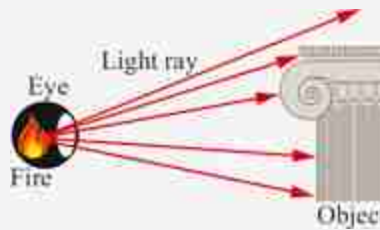
how the flow produced by the internal fire in the eyes interacts with the external fire to produce sight:

And of the organs they first contrived the eyes to give light, and the principle according to which they were inserted was as follows: So much of fire as would not burn, but gave a gentle light, they formed into a substance akin to the light of every-day life; and the pure fire which is within us and related thereto they made to flow through the eyes in a stream smooth and dense, compressing the whole eye, and especially the centre part, so that it kept out everything of a coarser nature, and allowed to pass only this pure element. When the light of day surrounds the stream of vision, then like falls upon like, and they coalesce, and one body is formed by natural affinity in the line of vision, wherever the light that falls from within meets with an external object. And the whole stream of vision, being similarly affected in virtue of similarity, diffuses the motions of what it touches or what touches it over the whole body, until they reach the soul, causing that perception which we call sight. But when night comes on and the external and kindred fire departs, then the stream of vision is cut off; for going forth to an unlike element it is changed and extinguished, being no longer of one nature with the surrounding atmosphere which is now deprived of fire: and so the eye no longer sees, and we feel disposed to sleep.

Plato considered the sense of vision as being worse than touch because vision could only sense the part that was facing the observer. Therefore, according to Plato, one could not trust the sense of vision. However, Aristotle (384–322 BC), Plato's student, was critical of the extramission theory and pointed out that the stars were too far away for rays from the eyes to reach them (an argument also used by Euclid). Aristotle went further and suggested that only some objects are light sources (e.g., fire) and the other objects reflect the rays that hit the eyes [20]. He also criticized the belief that the eye had fire inside. Instead, Aristotle defended the idea that the element of perception had to be water as vision needed a transparent element.

Euclid (325 BC), a Greek mathematician, provided the first mathematical theory of vision, giving a mathematical description of how the emitted rays by the eye traveled on **straight lines** and formed a cone that reached the scene.

Modeling light as traveling along straight lines was a major discovery. This simple model of light sets the foundation upon which image formation, scene geometry, and computer vision rests.



He listed the following seven axioms of light, extracted from [67]:

1. Let it be assumed that lines draw directly from the eye pass through a space of great extent;
2. and that the form of the space included within our vision is a cone, with its apex in the eye and its base at the limits of our vision;
3. and that those things upon which vision falls are seen, and that those things upon which vision does not fall are not seen;
4. and that those things seen within a larger angle appear larger, and that those seen within a smaller angle appear smaller, and those seen within equal angles appear to be of the same size;
5. and that things seen within the higher visual range appear higher, while those within the lower range appear lower;
6. and, similarly, that those seen within the visual range on the right appear on the right, while those within that on the left appear on the left;
7. but that things seen within several angles appear to be more clear.

Starting from those axioms, in his paper “Optics” [67], Euclid describes many different ways to use geometric reasoning to measure the size of objects in the world using the properties of light and vision. One example of the application of his theory is shown in [figure 1.3](#). The description of the figure in the original text reads as follows: “To know how great is a given elevation (AB) when the sun is shining. Let the eye be D, and let GA be a ray of the sun falling upon the end of line AB, and let it be prolonged as far as the eye D. And let DB be the shadow of AB. And let there be a second line, EZ, meeting the ray, but not at all illuminated by it below the end of line at Z. So, into the triangle ABD has been fitted a second triangle, EZD.

Thus, as DE is to ZE, so is DB to AB. But the ratio of ED to ZE is known. Moreover, DB is known; so, AB is also known” [67].

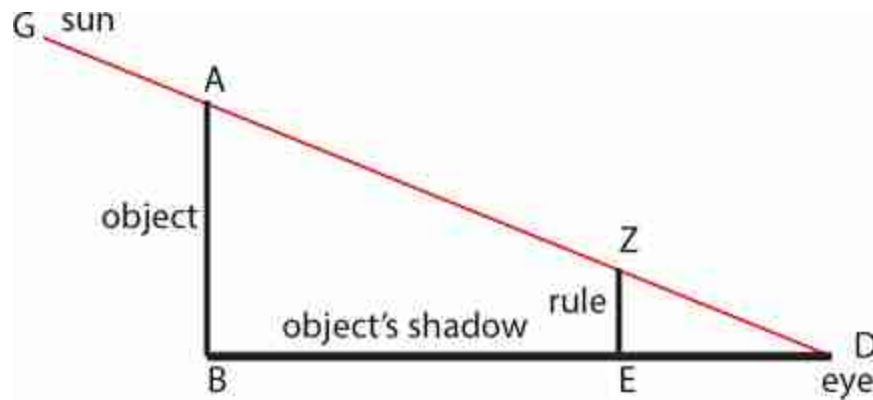


Figure 1.3: Adapted from Euclid (325 BC). The color and text, added here for clarity, is not part of the original figure [67].

Euclid's work set the basis of **perspective**, and his discoveries influenced thinkers and artists in the following centuries.

Many other Greek philosophers and mathematicians contributed to a deeper understanding of light. Hero of Alexandria (10-70), in the work **Catoptrica**, postulated that light propagated in straight lines and also described an early version of the **law of reflection**: light will follow the shortest path between two points, which means that the angle of incidence is the same as the angle of departure. Ptolemy described how light changed direction (i.e., **refraction**) when changing medium (e.g., from air to water). Few other discoveries about vision between the first and tenth centuries survive.

Credit assignment is unclear. For instance, early versions of the law of reflection are credited to Heron, Ptolemy, Archimedes, Plato, and was potentially known to others before [58].

In the late tenth century Hasan Ibn al-Haytham's work transformed the initial Greek theories into a true scientific discipline, inspiring many of the works that followed, even through Johannes Kepler. Ibn al-Haytham (965–1040 AD), known as Alhacen in the Latin world, published the *Book of Optics* [446] between the years 1028 and 1038. This book is considered by

many the beginning of the scientific method. In his Book of Optics, Ibn al-Haytham describes how light rays bounce off objects in all directions and that the rays only become visible when they reach the eye perpendicularly. He described the pinhole camera and invented the **camera obscura**.

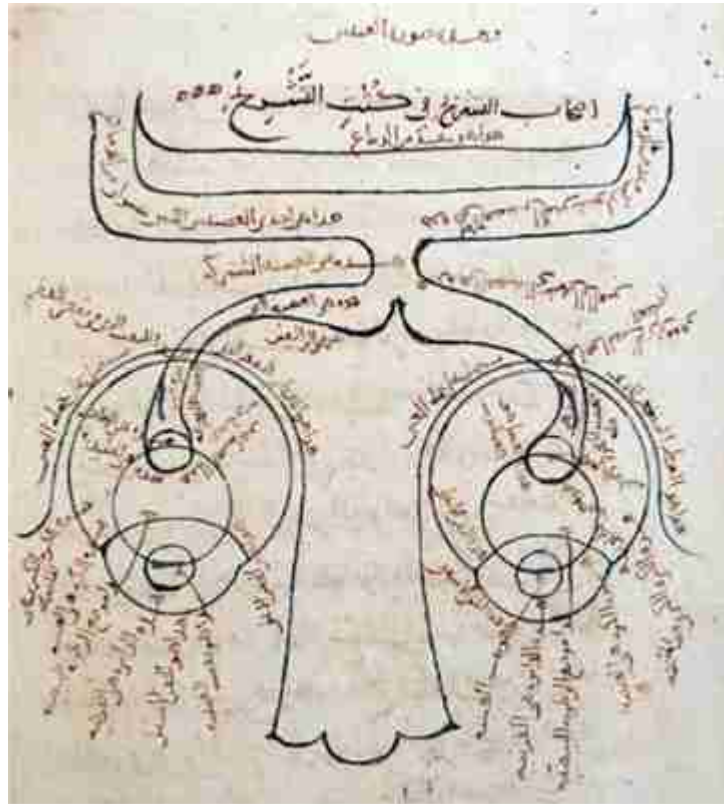


Figure 1.4: Alhacen's diagram of the eyes and the optic nerve (*Book of Optics*), drawn around the year 1027. It is one of the first drawings of the visual system. *Source:* MS Fatih 3212, vol. 1, fol. 81b, Süleimaniye Mosque Library, Istanbul.

Ibn al-Haytham rejected the extramission theory. He argued that sight could be explained by the light rays emitted by objects and that it was not necessary to assume that the eye emitted rays. Ibn al-Haytham's work built upon the work of Ptolemy, Euclid, Galen, and Aristotle, although little is known about the exact sources of inspiration as manuscripts at that time did not include citations to previous work and it was rare when other scientists were cited by name. Ptolemy believed that distance between the eye and an object could be measured by feeling the length of a ray of light (thus

requiring a single eye), while Ibn al-Haytham showed experimentally that both eyes were needed to perceive depth [446].

Kepler (1604 AD), building on Alhacen's work, provided the first complete description of how images are formed in the retina in *Astronomiae Pars Optica* [321]. Kepler understood **lenses** and how the eye projected a reverse picture into the retina. Kepler understood the role of the crystalline lens as the focusing element of images in the eye, in contrast with previous theories that believed that the crystalline lens was the sensitive element selecting only the perpendicular rays that reached the eye.

1.3.2 Helmholtz: Perception as Inference

Hermann von Helmholtz (1821–1894) was a German scientist and philosopher. While he wanted to study physics, he trained as a physician at the urging of his father [430], then went on to make important contributions in philosophy, physics, audition, color theory, and visual perception.

With Thomas Young, he codeveloped the theory of **trichromacy**, that the eye has three classes of receptors, each sensitive to different wavelengths of light.



He measured the speed of transmission of nerves (24.6–38.4 m/s [502], previously thought to be unmeasurably fast [430]). He also developed the ophthalmoscope, shown in [figure 1.5](#), an instrument to observe the retina of the human eye. The key of the ophthalmoscope was to see the need for colinear viewing and illumination directions. [Figure 1.5\(a\)](#) shows a schematic illustration of Helmholtz's ophthalmoscope, viewed from above. The glass plates along the diagonal, labeled *a* in the figure, allow for viewing the eye under study by the ophthalmologist while reflecting illumination from a light source into the eye. [Figure 1.5\(b\)](#) shows a drawing by Helmholtz of an image from the ophthalmoscope, showing blood vessels

(in background) and branches of the retinal artery and vein over the optic nerve (center).

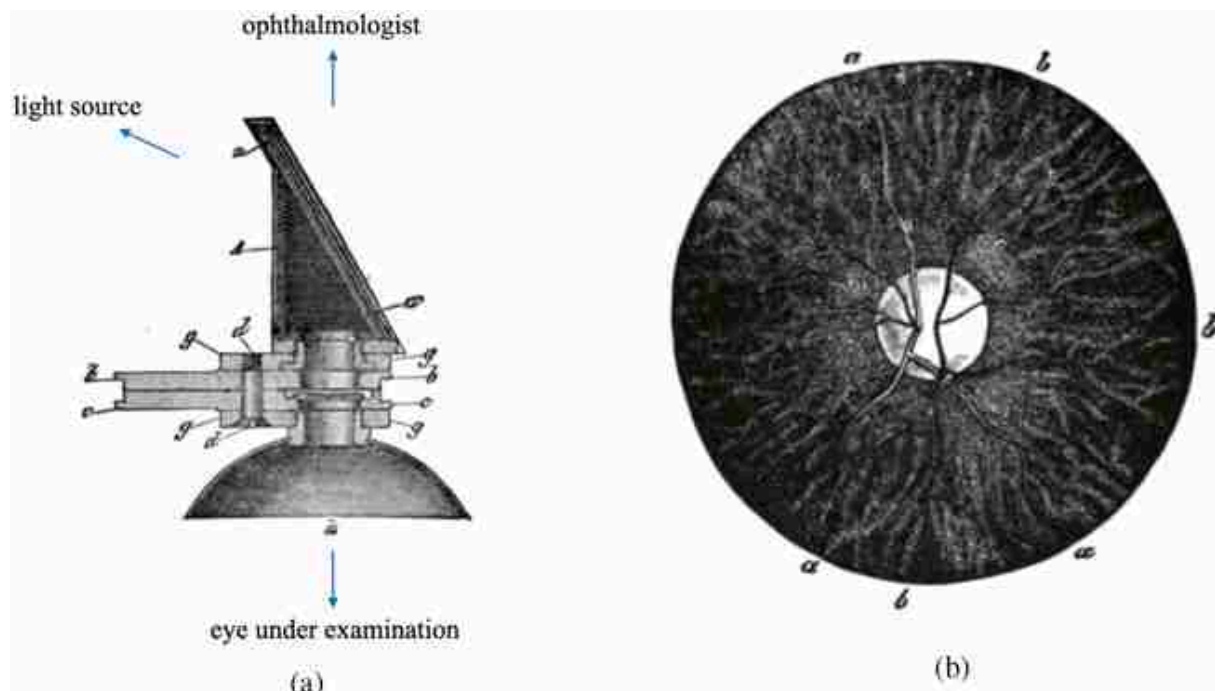


Figure 1.5: Hermann von Helmholtz invented the ophthalmoscope at age 29. Images from [204].

Helmholtz was also an important theorist in visual perception. He wrote [203],

The general rule determining the ideas of vision that are formed whenever an impression is made on the eye, is that *such objects are always imagined as being present in the field of vision as would have to be there in order to produce the same impression on the nervous mechanism, the eyes being used under ordinary normal conditions.*

Thus, the mind makes perceptions out of sensations [430], finding representations of the object most likely to explain the sensory input [491]. This viewpoint relates to **Bayesian methods** for computer vision inference.

In his introduction, “Concerning the Perceptions in General,” he continued [203]:

Still, it may be permissible to speak of the psychic acts of ordinary perception as *unconscious conclusions*, thereby making a distinction of some sort between them and the common so-called conscious conclusions.

He added that when an astronomer computes the position of stars in space, based on observations, this is a conscious conclusion. When you press on the eye and see light, that is an unconscious conclusion about what is in the world, given the responses of your eye. This unconscious inference is the topic of computer vision.

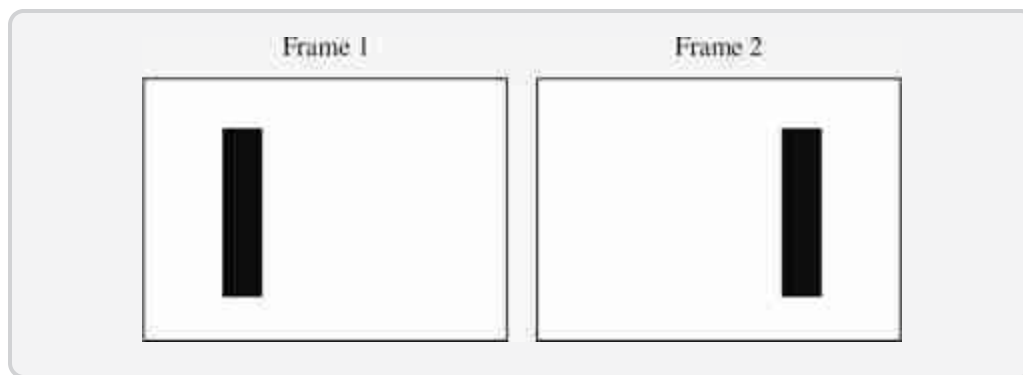
Helmholtz emphasized the active role of the viewer in perception [203],

If the objects had simply been passed in review before our eyes by some foreign force without our being able to do anything about them, probably we should never have found our way about amid such an optical phantasmagoria ... But when we notice that we can get various images of a table in front of us simply by changing our position; and that we can sometimes have one view and sometimes another, just as we like at any time ... Thus by our movements we find out that it is the stationary form of the table in space which is the cause of the changing image in our eyes.

This foreshadows some unsupervised learning methods common in computer vision today.

1.3.3 Gestalt Psychology and Perceptual Organization

Gestalt psychology started around 1912, with the publication of “Experimental Studies of the Perception of Movement” by Max Wertheimer [500]. In this paper Wertheimer introduced the **phi phenomenon**, a visual illusion that consists in presenting two images with a vertical bar at different locations in rapid succession giving the impression that there is continuous motion between them [454].



Wertheimer, together with Kurt Koffka and Wolfgang Köhler, postulated that perceived motion was a new phenomenon that is not present in the individual stimuli (the two flashing frames) and that what we perceive is the whole event as a single unit (continuous motion). Gestalt psychology extended this interpretation to explain many other visual phenomena and emerged as a reaction of the existing trend that said that for psychology to

be a science it had to decompose stimuli into its constituent elements. Gestalt theory argued that perception is about wholes more than it is about parts.

Wertheimer introduced the problem of **perceptual organization**. Perceptual organization studies how our visual system organizes individual visual features (i.e., colors, lines, ...) into coherent objects and wholes. Wertheimer proposed a set of rules used by the visual system to organize elements on a simple display. He showed sets of dots and lines disposed in different arrangements, as shown in [figure 1.6](#), to find out when those elements were grouped together into larger elements.

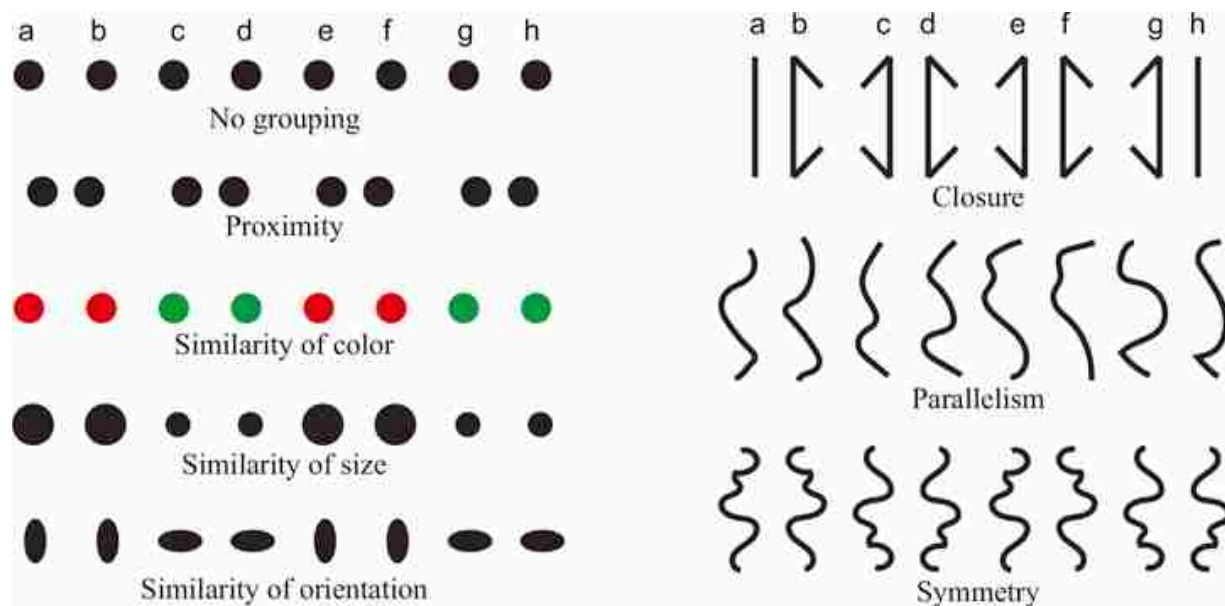


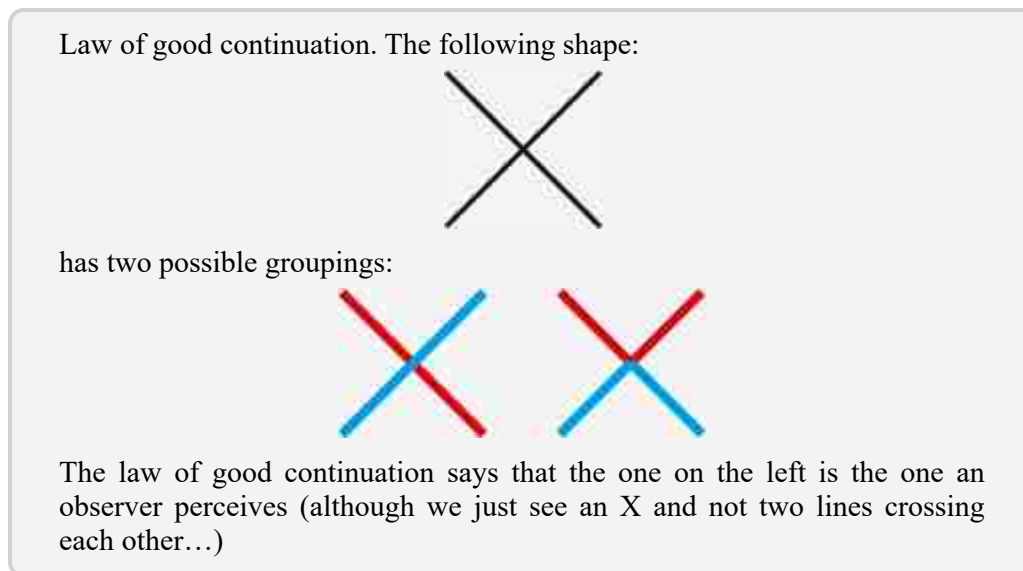
Figure 1.6: Gestalt grouping rules for perceptual organization. Most of the examples are recreated from [372]. The closure pattern is from Koffka [272], who modified it from Kohler.

Gestalt psychologists called **grouping laws** the visual cues they uncovered. However, they are not strictly laws and not all grouping cues are equally strong. Some of the most important laws are as follows:

- Law of **proximity**: Items that are nearby are more likely to group together. From [figure 1.6](#), we can see that groupings a-b, c-d, e-f, and g-h are stronger than b-c, d-e, and f-g. With some effort one can group the items according to the second arrangement, but it is hard to sustain that

grouping perceptually. The law of proximity is affected by the similarity of the items.

- Law of **similarity**: Elements that have similar features (color, size, orientation) also group together, even when they are all equally distant.
- Law of **closure**: We are very familiar with the fact that when a line forms a closed figure, we do not simply see a line, we see a shape. The example shown in [figure 1.6](#) shows how closure wins over proximity. The vertical lines are closer for the grouping a-b. However, we group them as b-c, d-e, and f-g.
- Law of **good continuation**: Edges are likely to be smooth. Lines that follow each other pointing in the same direction are likely to be grouped together. For instance, an X-shape is perceived as two lines that cross each other instead of perceiving it as a V-shape on top of an inverted V.



- Laws of **parallelism**, and **symmetry**: There are many other grouping cues with different strengths that induce clustering between visual elements.
- **Common fate**: Motion is a very important grouping cue. When two elements have a common motion (accelerations, direction, rate of change) they tend to group together.
- **Past experience**: Items grouped in the past are more likely to be perceived as a group in the future.

In complex displays, one will expect that multiple of these grouping cues will be present. Gestalt theory not did quantitatively address how grouping

cues will balance each other when they were in conflict. Stephen Palmer [370] measured the strength of these grouping cues and introduced new ones.

Gestalists also studied the problem of **lightness perception**.

Lightness is a perceptual quantity that is influenced by both the perceived reflectance and the perceived illumination of a surface [7].

For a wonderful book on lightness perception, we refer the reader to [163], and the work of Edward Adelson [7]. Adelson defines lightness as “the visual system's attempt to extract reflectance based on the luminances in the scene.”

Gestalt psychologists said that the perception of lightness on an image patch is a function of the context. One has to take into account the grouping laws (perceptual organization) in order to explain the perceived lightness on a display. Koffka [272] introduced the **Koffka ring**, a beautiful illusion to illustrate the power of perceptual grouping to explain lightness perception (figure 1.7).

Kurt Koffka, born in Berlin in 1886, published his *Principles of Gestalt Psychology* in 1935, after moving to the USA in 1924.

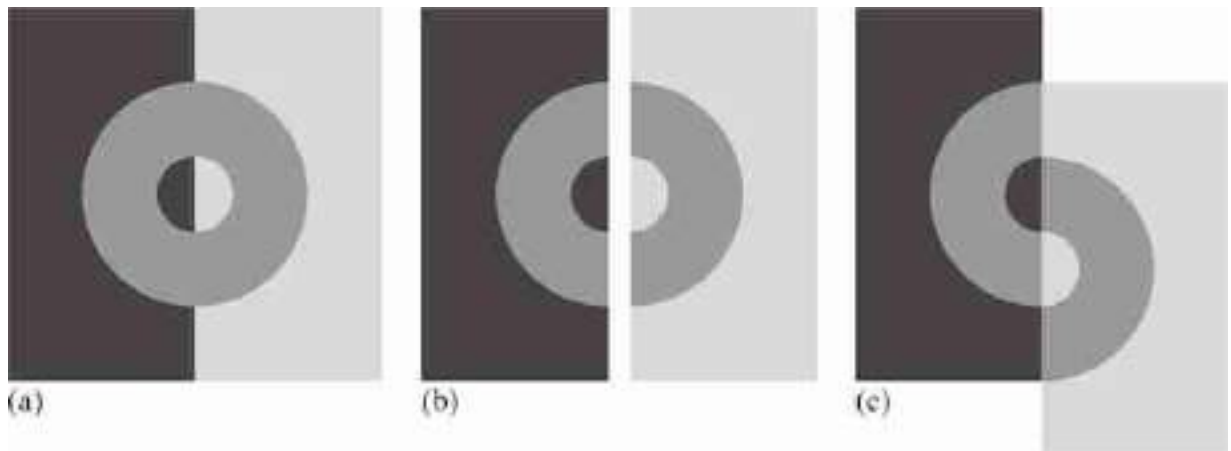


Figure 1.7: (a) A ring of uniform gray. (b) Koffka ring and (c) a variant introduced by Adelson [7]. In all the examples, the two sides of the ring are identical.

In each display, the two sides of the ring have the same gray level. Depending on the spatial configuration of the two sides, their lightnesses

appear very different. In [figure 1.7\(a\)](#) the ring appears as solid gray. In [figure 1.7\(b\)](#) splitting the figure into two halves by an interleaving white space makes the two sides of the ring look different. [Figure 1.7\(c\)](#) shows a variant of the Koffka ring proposed by Adelson [7] where the grouping cues create a stronger effect. One can conclude from this experiment that lightness perception is not a local process and that it involves considering the context and the principles of grouping.

Another striking proof of the importance of perceptual organization is the visual phenomenon of **amodal completion**. As discussed in the introduction, in [figure 1.2\(a\)](#) we see a red square on top of a blue square. Even though the blue square is occluded by the red square, we see it as a square. The green geometric figure is identical to the blue one but the gap breaks the illusion of occlusion and we do not perceive it as a square anymore. This phenomenon of perceiving a whole object when only a part is visible is called **amodal completion**. The word *amodal* means that we have perception without using any direct perceptual modality.

Amodal completion is something we do all the time when we explore a scene. Most of the objects that we see are partially occluded and we use amodal completion to extend the object behind the occlusion.

Modal completion is a related visual phenomenon that happens when we see an induced object appear in front of others. [Figure 1.8](#) shows several examples of modal and amodal completion. A beautiful visual illusion that illustrates modal completion is the **Kanizsa triangle**, shown in [figure 1.8\(d\)](#). In this case, we see a triangle that is not really there.

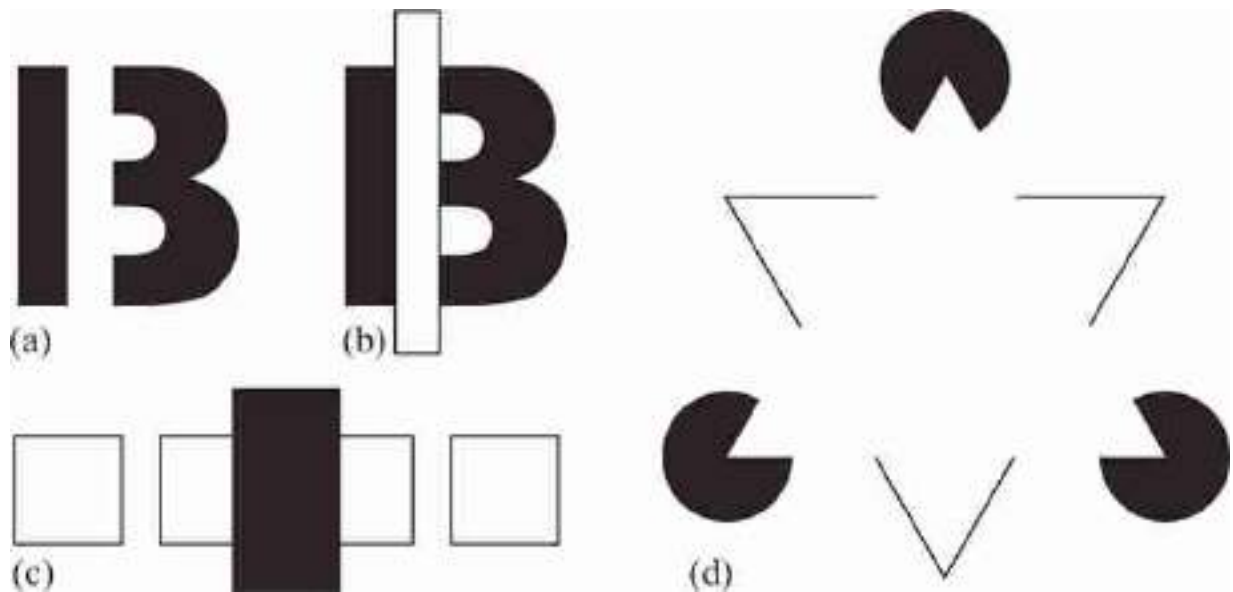


Figure 1.8: Amodal and modal completion. (a) is this 13 or an occluded B? (b) By adding boundaries to the occluder, the letter B becomes clearer. (c) How many squares are there? Behind the occluder are there two squares or just one rectangle? (d) Kanizsa triangle. Illusory contours appear at the triangle sides.

The Kanizsa triangle shows both amodal and modal completion. The circles are seen by amodal completion and the triangle appears as modal completion. Modal completion produces **illusory contours** to appear in the image. Some observers report that the triangle on top is brighter than the surrounding background and that illusory contours appear at the triangle sides.

Gaetano Kanizsa [249] studied how perceptual organization could be used to see what is not there.

Perceptual organization remains an intriguing and rich area of research with many visual mechanisms still poorly understood. The list of principles of perceptual organization discovered by the gestalt theory have impacted a lot of modern research in computer vision, in particular in the domain of **image segmentation** [315].

1.3.4 Gibson's Ecological Approach to Visual Perception

While previous theories of perception considered that the simplest scenario to study vision was assuming a static camera taking a snapshot in a controlled lab setting, James J. Gibson postulated that, to understand vision,

one should take an **ecological approach to visual perception** [[160](#)]. The study of the eye should be done in the context of the body that supports it and the world it lives in.

Experimental science based in simple visual stimuli allowed for measuring quantities, reporting results, and reproducing findings, but the simple stimuli were very limiting. Dealing with real-world stimuli is messy, and no one had good theories on how to use them experimentally. Gibson argued that, to understand perception, the experimental lab should be like real life. Many of the displays used by gestalt psychologists were too simple and unlikely to occur in the real world. They provide useful knowledge, but fall short of unraveling the true nature of the visual system.

The environment should be studied by means of ecological properties, and not by using physical laws. That is, by understanding what quantities and processes of the environment are relevant for the observer. An ecological description of the environment describes it in terms of medium (air or water), substances (rocks, metal, wood, etc.), and surfaces (which separate medium from substances), together with the notions of change and persistence. Gibson considers those concepts more relevant to the study of perception than the notions of space, matter, and time, which have their origin in classical physics but have little relevance for the observer. The dominant role of the observer in Gibson's theory is illustrated in how the environment is understood: water is substance for terrestrial animals, and it is the medium for aquatic animals. The **ground plane** is the most important surface for terrestrial animals, it is the center of their perception and behavior. The ground provides the support for action and a reference for perception. Other notions of ecological importance are the concepts of *layout*, *place*, and *enclosure*.

Drawing by Gibson showing how few lines induce a strong 3D percept of a ground plane.



Gibson also made a distinction between the light studied by optical physics and the one that is relevant for perception, which he called **ecological optics**. According to Gibson, physicists are interested in studying light in simplified settings such as the light emitted by point sources that send rays into an infinite space, or that interact with surfaces or lenses once. Ecological optics instead studies the light that converges into the observer, which is the result of countless interactions with all the surfaces in a messy environment. The **ambient optic array** is the set of light rays that converge into a point of observation. And this point of observation can be occupied by an observer who will move around the world. This moving ambient optic array will contain information about the environment and also about the observer themselves ([figure 1.9](#)).

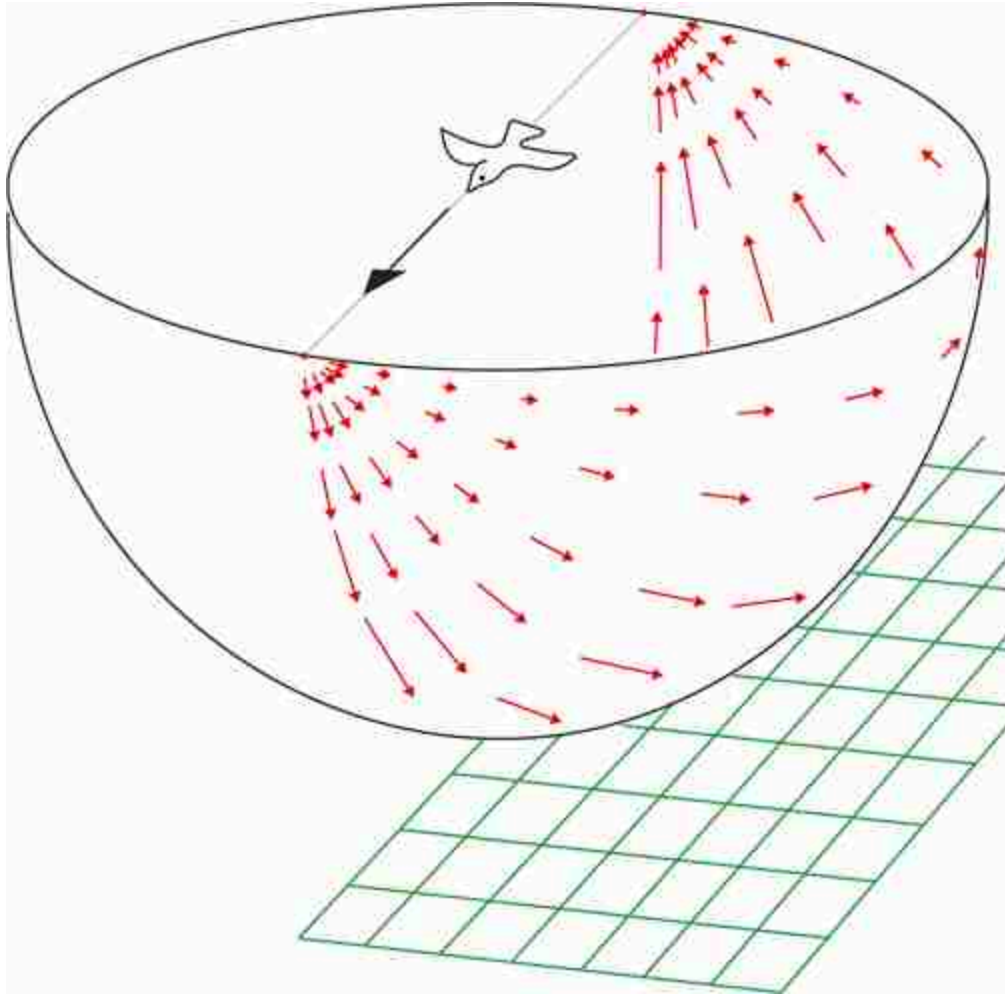


Figure 1.9: The flow pattern in the optic array of a bird flying over the earth provides information about the environment and also about the bird. Figure recreated from [161].

Another key concept in Gibson's theory is the notion of **affordance** introduced by Gibson in 1966 [161]. A *path* affords locomotion from one point to another, an *obstacle* affords collision, and a *stairway* affords both descent and ascent. An *object* is a substance enclosed by a surface. The affordance of an object is generally a direct consequence of its properties: a hollow object can contain substances, a detached object affords carrying, an object with a sharp edge affords cutting, and so on. Animate objects are controlled by internal forces, and they afford social interaction.

Gibson's theory of visual perception postulates that the ambient optic array is very rich and contains all the information needed to perceive the environment. **Direct perception** is the process of **information pickup**

directly from the ambient array. This is in contrast with other theories of visual perception that assume that the input stimuli

Gibson did not like the term **stimuli** because it implied that the environment stimulates the observer. For Gibson, it is important to stress that the observer is only picking up information.

is very impoverished and that perception is **indirect** as it has to be complemented by additional processes. The indirect theory of perception is the most common one. However, the important message to learn here is that, when building a perceptual system, the richer the input is, one might hope that the solution to the perception problem will be simpler.

1.3.5 The Neural Mechanisms of Visual Perception

Neuroscience has greatly contributed to our understanding of how vision works and has inspired a large number of computer vision approaches: red-green-blue (RGB) encoding of images, filter based image representations, neural networks, and attention modulation. Studies on the neural mechanisms of visual perception had a lasting impact. In this section will only review some of the many discoveries made over many centuries trying to explain how the brain works. One premise in the computational neuroscience community is that one can understand how vision works by reverse engineering how the brain solves the task.

For a long time it was known that the brain was the center of reason, but the mechanisms and anatomy of the brain were not well understood. In fact, researchers believed that the brain was not composed of individualized cells as the rest of the body. It was in 1890 that Santiago Ramón y Cajal, using Golgi's stain,

Santiago Ramón y Cajal was born in 1852 in Navarra, Spain.

isolated individual neurons and proved that the brain is composed by **networks of interconnected cells** as shown in his work on the anatomy of the retina [399]. This was the first time that networks were observed. Ramón y Cajal then mapped most of the brain and his drawings are still a reference. Both Santiago Ramón y Cajal and Camillo Golgi received the Nobel prize in 1906 [166].

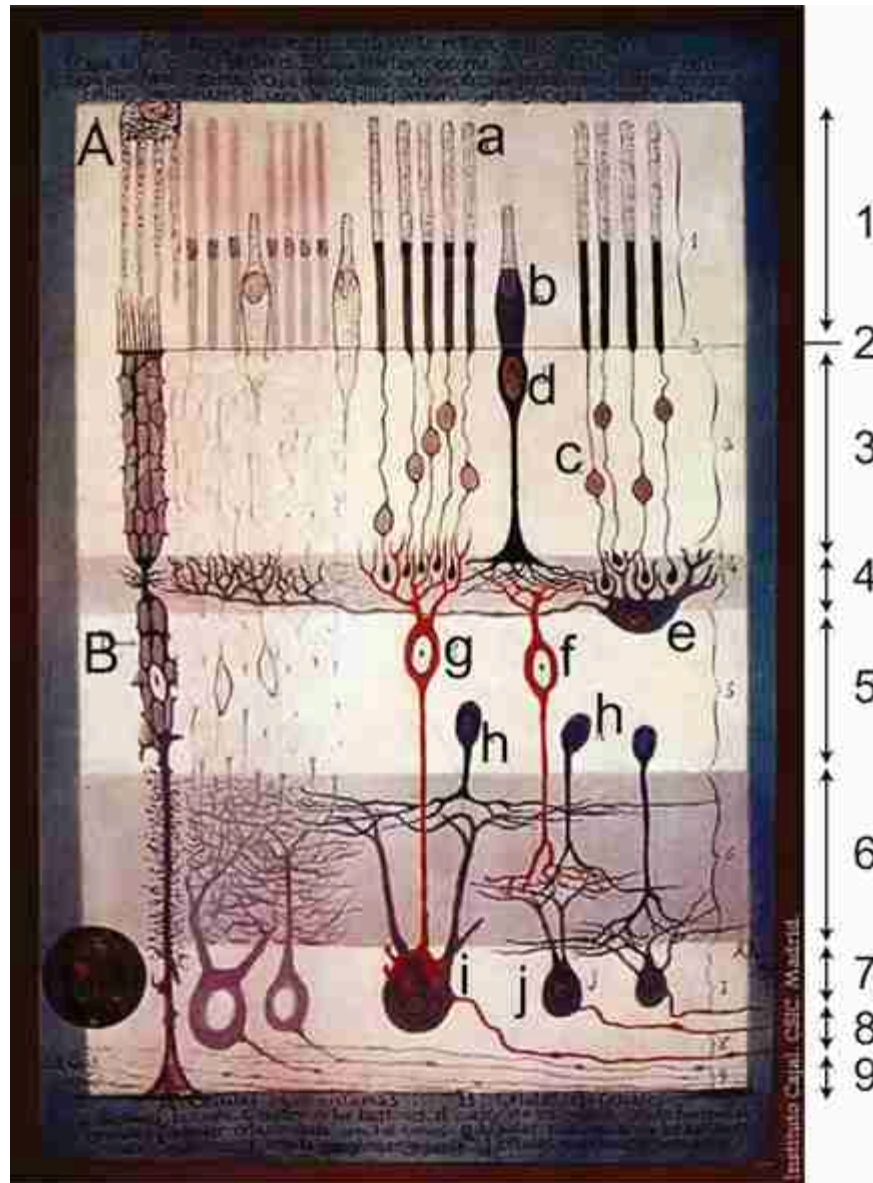


Figure 1.10: Drawing of the retina by Santiago Ramón y Cajal. In this drawing photoreceptors are in the top and light comes from the bottom: 1. Rod and cone layer. 2. External limiting membrane. 3. Outer granular layer. 4. Outer plexiform layer. 5. Inner granular layer. 6. Inner plexiform layer. 7. Ganglion cell layer. 8. Optic nerve fibre layer. 9. Internal limiting membrane. A. Pigmented cells. B. Epithelial cells. a. Rods. b. Cones. c. Rod nucleus. d. Cone nucleus. e. Large horizontal cell f. Cone-associated bipolar cell. g. Rod-associated bipolar cell. h. Amacrine cells. i. Giant ganglion cell. j. Small ganglion cells. *Source:* “Structure of the Mammalian Retina” c.1900 By Santiago Ramon y Cajal.

The drawing of the retina ([figure 1.10](#)) shows the first layers of neurons that process the visual input. The retina is an amazing piece of the brain that transforms light into impulses. Light is first transformed into electric signal

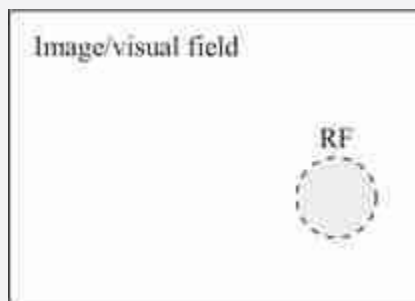
by the photoreceptors (rods and cones) and is then processed by a few layers formed by several types of neurons (amacrine, bipolar, and ganglion cells). Finally, ganglion cells transmit the output of the retina through the optic nerve to the rest of the brain. The studies of Ramón y Cajal revealed the circuitry of many parts of the brain but told us little about their functional behavior or how the circuits were processing information.

The **retina** was one of the first visual structures that was studied from a functional perspective. Haldan Keffer Hartline [[188](#)], in 1938, using an innovative method to record the response of single optic nerve fibers (which corresponds to the axons of ganglion cells), studied retinal ganglion cells and popularized the concept of **receptive field**, previously introduced by Charles Scott Sherrington [[433](#)] in 1906 when studying the tactile domain.

A review of how the concept of **receptive field** evolved can be found in [[452](#)].

The receptive field of a neuron corresponds to the region of the input stimulus space (the retina in the case of a visual neuron) that has to be stimulated in order to produce a response in the neuron. Most neurons in the visual processing stream are activated only when light shines on a precise portion of the retina.

Only stimulation within the **receptive field** (RF) of a neuron produces a response.



Steven Kuffler (1953) studied the organization of the retina and showed that ganglion cells have concentric receptive fields with a circular **center-surround organization** [[280](#)]. He did this by shining a small spot of light on different parts of the retina and recording the response of a ganglion cell as he changed the location of the spot of light. First Hartline, and then Kuffler, showed that there were two types of ganglion cells; a first group of cells, called **on-center**, would get activated when the spot of light was

inside a small region in the middle of the receptive field and would get deactivated (firing below their average rate) when the spot light was projected inside a ring around the center. A second group had the opposite behavior, called **off-center**. These results are fascinating because they reveal that the retina seems to be performing some sort of contrast enhancing operation (as we will discuss in other chapters) and their work motivated a large number of studies in **computational neuroscience** and **neuromorphic engineering** [330] that tried to reproduce the operations made in the retina. Haldan Keffer Hartline received the Nobel Prize in 1967 for his discoveries on how the eye processes visual information.

The axons of ganglion cells from both eyes project to another structure called the **lateral geniculate nucleus** (LGN). The LGN is composed of six layers of neurons and its output goes to the visual cortex and other visual areas. This structure receives inputs from both eyes but the signals are not mixed and the neurons in this area remain monocular. LGN neurons have concentric (center-surround) receptive fields. The role of the LGN is not completely understood but it may be involved in temporal decorrelation, attention modulation and saccadic suppression. Interestingly, only 5 percent of the input connections come from the retina while 95 percent of the LGN inputs are **feedback connections** from the primary visual cortex and other areas.

Concentric receptive fields are very common in the early layers of visual processing, but things become more interesting when studying cells in the **visual cortex**. One challenge was that the techniques used to record the responses of cells in the optic nerve could not be applied to record the activity of cells in the cortex. [Figure 1.11](#) shows some of the receptive fields found in the LGN. When a cell with an **ON-center Receptive Field (RF)** is illuminated with a small spot of light projected on the inner part of the RF, the firing rate increases, as shown in [figure 1.11\(a\)](#). The gray rectangle under the graph represents the duration of the stimulation. When the outer part of the RF is stimulated there is an inhibition of the cells response. [Figure 1.11\(a\)](#) shows an **OFF-center RF**.

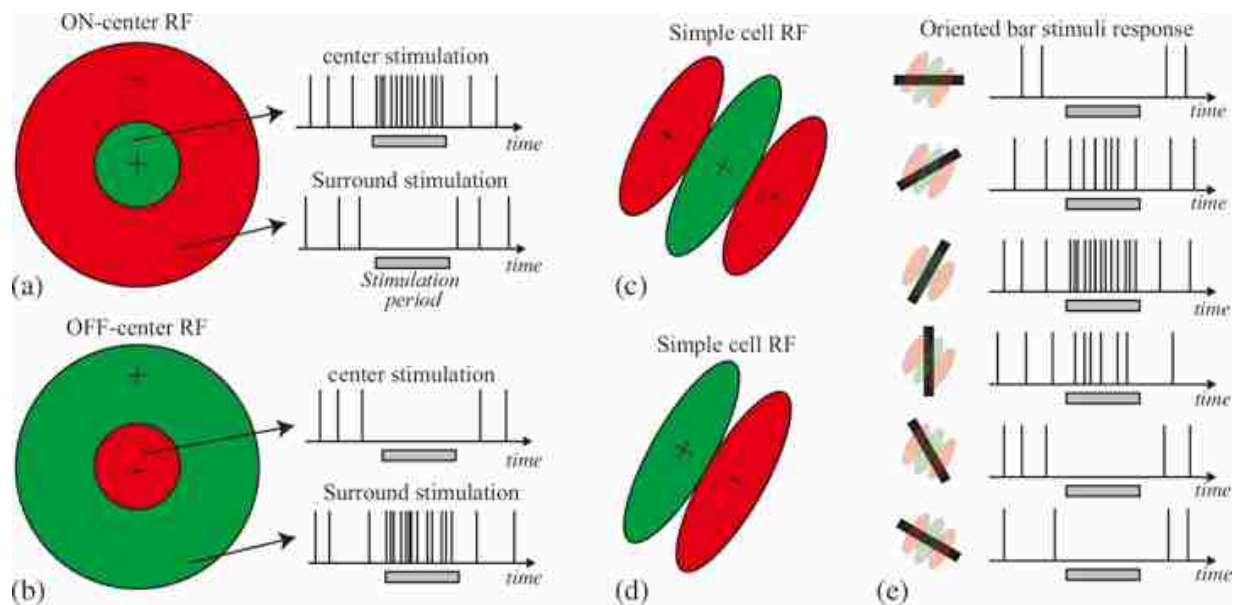


Figure 1.11: Receptive fields (RF) for retinal ganglion cells and oriented receptive fields from the primary visual system. (a) ON-center RF. (b) OFF-center RF. (c-d) Two types of oriented simple cell. (e) Sketch of the response of the cell from (c) to an oriented bar with different orientations in the center of its RF. The gray bar corresponds to the moment the bar is visible.

Between 1955 and 1958, David H. Hubel invented a microelectrode that allowed him to record the activity of individual cells in the cortex. He then went to work with Torsten N. Wiesel in Kuffler's lab. They used Hubel's technique to record the response of cells in the visual cortex of cats; this work resulted in them receiving the Nobel prize in 1981.

In 1959, Hubel and Wiesel published the study that marked the beginning in understanding how visual information is processed in the brain [226]. They studied 45 neurons for a period of two to nine hours from the striate cortex on an anesthetized cat. They exposed each neuron to a diverse set of visual stimuli containing spots of various sizes and locations, and **oriented bars**. For most of the neurons they could locate a region of the retina that produced a firing of the neuron when stimulated with light. However, not all types of visual stimuli were effective in driving neurons at the cortical level.

Hubel and Wiesel discovered that some neurons in the visual cortex were best stimulated by an oriented bar at an specific orientation when it appeared at the center of the neuron's receptive field. They found different types of cells in the primary visual cortex, some that were similar to those found in the retina and LGN (center-surround), others selective to oriented

bars, and more complex ones. They classified each cell according to its attributes: spatial selectivity, orientation preference, eye preference (left or right), and the type (simple, complex, or hypercomplex).

[Figure 1.11\(c\)](#) shows an **oriented simple cell**, while [figure 1.11\(d\)](#) shows another type of simple oriented cell found in the visual cortex. An oriented cell shows increased firing rate when stimulated with an oriented bar in the positive region of the RF with the preferred orientation.

[Figure 1.12](#) shows a schematic of the visual pathways from the retina up to the primary visual cortex. The first visual area in the visual cortex is called **V1**. It is a sheet of neurons arranged along several layers as shown in [figure 1.12](#). Hubel and Wiesel found that neurons in V1 are organized along **columns**. When probing with an electrode, as the electrode went from the surface to the bottom of V1, all the cells found were selective to the same orientation. When moving the electrode along one horizontal direction, they found that the preferred orientation changed smoothly (orientation columns), and when moving the electrode along the perpendicular horizontal direction, the cells changed the eye preference, alternating between the two eyes with some binocular cells between them (ocular dominance columns). A section of around 1×3 mm (called a **hypercolumn**) covers all orientations and both eyes for a small portion of the visual field.

Margaret Livingston, in collaboration with David Hubel, discovered color sensitive neurons [\[304\]](#) organized in blobs surrounded by orientation-sensitive neurons in the visual cortex.

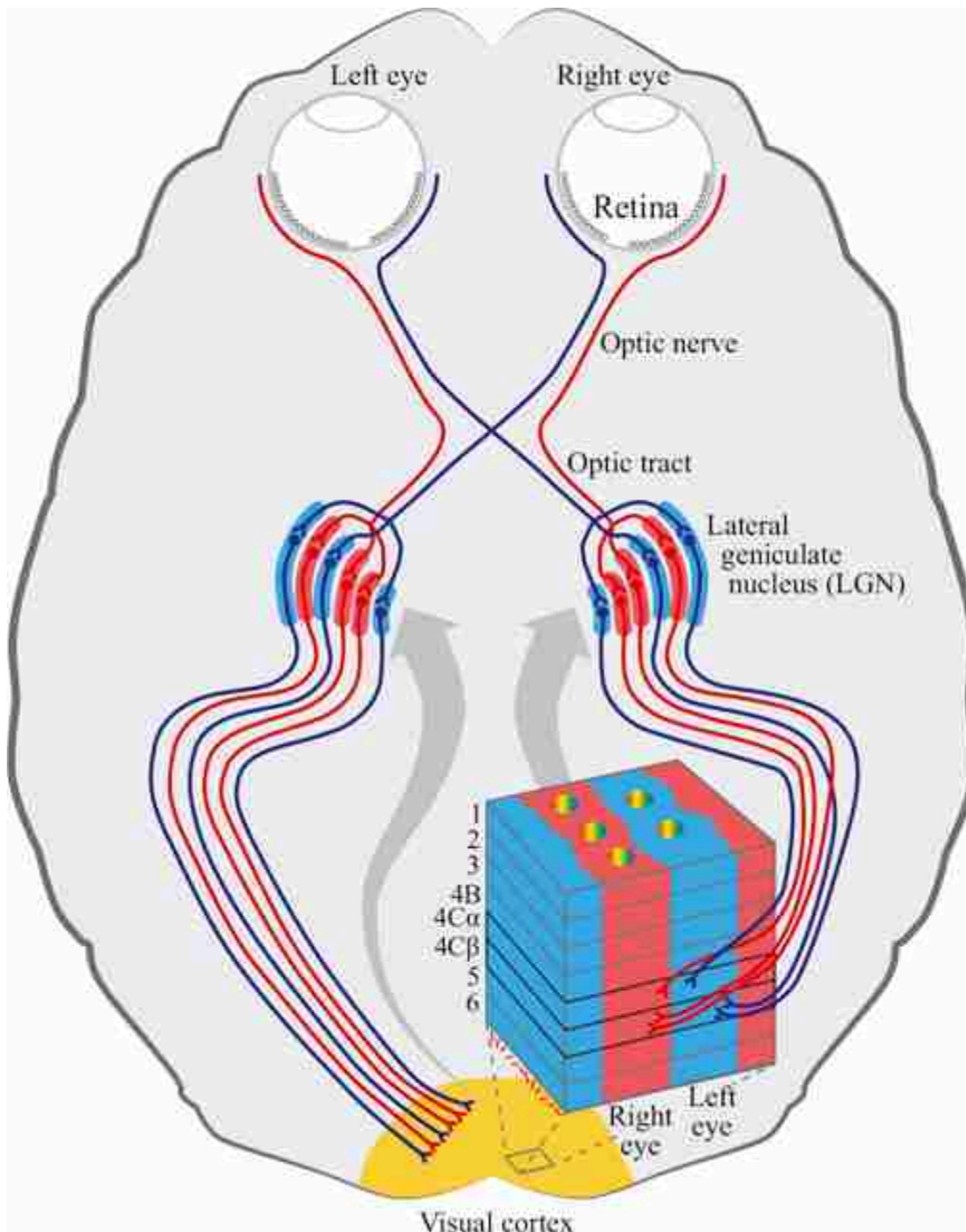


Figure 1.12: Visual pathways. The retina projects the axons of the ganglion cells into the lateral geniculate nucleus (LGN). In each hemisphere, the LGN gets inputs from both eyes (the LGN on the right gets the right side of each retina, which corresponds to the left side of the visual field). The output of the LGN is divided into three channels; the parvocellular, magnocellular, and koniocellular (not shown). Those channels project into the primary visual cortex following a very regular architecture organized along hypercolumns. The koniocellular pathway projects into the color blobs in the upper layer of the visual cortex. Figure modified from [248].

Compare the drawing from [figure 1.12](#) with the one made a thousand years earlier by Ibn al-Haytham, shown in [figure 1.4](#).

Cells in the visual cortex project their outputs to a diverse set of other visual areas following two main streams (the ventral stream and the dorsal stream) and project into specialized areas performing motion processing (area MT), face processing (fusiform face area [FFA]; see [\[250\]](#)), and object recognition (IT).

As a result of all those discoveries in neuroscience, a large community of interdisciplinary researchers developed models trying to explain the function of visual neurons. Many of the neurons in the early layer of visual processing (retina, LGN, and visual cortex) can be approximated by linear filters with rectifying nonlinearities and contrast control mechanisms, which are the basis of many computer vision algorithms as we will see throughout this book.

1.3.6 Marr's Computational Theory of Vision

David's Marr influential book *Vision* was published posthumously in 1982 [\[317\]](#). Marr's philosophy was influenced by advances in computer vision, and studies in cognitive psychology and neurophysiology. Studies in psychology at the time suggested that visual information was processed by independent modules of perception, each one devoted to a specific task (stereo vision, motion perception, color processing), and that visual images were analyzed by independent spatial-frequency-tuned channels. Progress in neurophysiological experiments using single cell recordings showed a strong connection between perception and neural activity. In the 1970's, computer vision was already a well-established field. Some of the advances that inspired Marr's philosophy were the work by David L. Waltz [\[490\]](#) on the interpretation of block worlds, Edwin H. Land and John J. McCann's Retinex theory about color perception [\[283\]](#), and Berthold Horn's theory of shape from shading [\[219\]](#), among many other works.

David Marr, together with Tomaso Poggio and others, postulated that it was important to separate the analysis of the task being solved from the particular mechanism used to solve it [\[317\]](#). Marr said that the vision problem should be understood at different levels, and that what was missing in previous theories of vision was to understand vision as an **information-processing task**. Marr proposed that an information-processing device should be described at three levels:

- **Computational theory.** This level of description should answer the following questions: what is the task to be performed and why should it be carried out? In the case of vision, the task is to derive properties of the world from images. This level of description should specify what the mapping is between the input and output, and why this mapping is appropriate for the task that the device needs to solve. This level does not specify how that mapping should be constructed or which algorithm will implement it.
- **Representation.** We should define which representation will be used for the input and output. The representation is a system of making explicit certain types of information present on a signal. For instance, an image could be represented by the sequence of values that encode the color of each pixel, but it could be also represented using a Fourier decomposition. The choice of representation will make explicit certain type of information while hiding other information, which will become harder to access. The algorithm used to implement the mapping between input and output will be strongly dependent on the representation used. The importance of the choice of representation is very present in today's computer vision community. Examples such as using position encoding to represent location illustrate the dramatic effect that the choice of representation and algorithm can have in the final system performance. We will discuss the importance of representations many times throughout this book.
- **Hardware.** Finally, the representation and algorithm will have to be implemented in a physical device.

Marr noted that when trying to explain visual phenomena, one needs to find what is the right level to use to describe it. For instance, the trichromatic color theory of human perception is explained by the hardware level (i.e., by the three color receptors in our retina) while the bistable nature of the Necker cube is likely to be related to the 3D representation. The illusions shown in [figure 1.2](#), are also likely due to the representation and algorithm used and they are rather independent on the particular hardware implementation. Marr considered that Gibson was asking the right questions, getting very close to a computational theory of vision, but that he had a naive view on how difficult it would be to solve them. We will return to this point in chapter 2 when we will build a simple visual system.

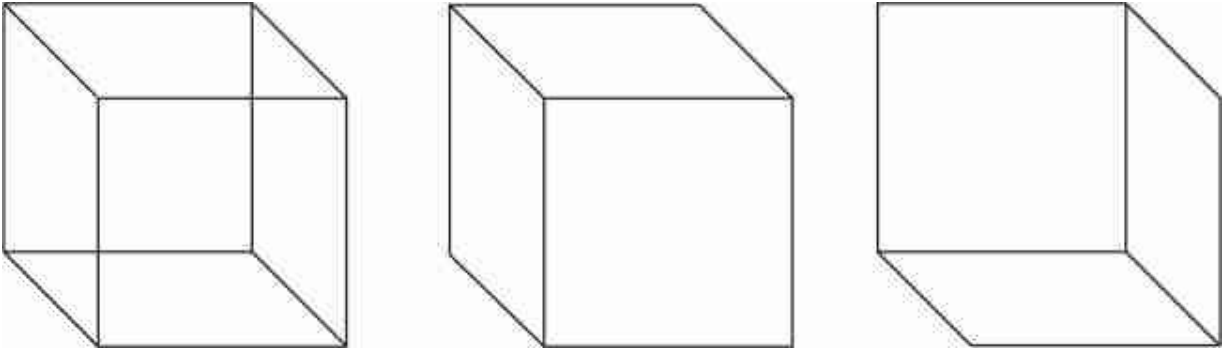


Figure 1.13: Necker cube illusion, introduced by Louis Albert Necker [351]. Figure recreated from [317].

Marr postulated that these three levels of understanding can be studied relatively independently, despite their loose coupling, with each problem needing description at the appropriate level. This message, although simple, sometimes is often ignored in today's computer vision where the task being solved is tightly connected to an architecture (usually a neural net) without a clear understanding of what is being computed, what the representations mean, or how something is being solved.

In David Marr's approach, vision was implemented as a sequence of representations, starting from the image represented as a sequence of pixel intensities, and then transforming each representation into another one that extracts more explicit information about the geometry and objects present in the world. The sequence of representations proposed by Marr was the following:

- Image: The collection of pixel intensities.
- Primal sketch: Represents changes in the image (edges, junctions, zero-crossings) and their spatial organization.
- 2.5D sketch: Contains an estimation of the distance between the observer and the visible surfaces in the world at each pixel, also called a **depth map**. It also contains information about depth discontinuities and surface orientations.
- 3D model representation: Objects are represented as 3D shapes together with their spatial organization.

You will notice a strong parallelism between the sequence of representations proposed by Marr and the particular algorithm that we will discuss in chapter 2 when building the simple visual system.

[Figure 1.14](#) shows the image of a cube, and its 2.5D sketch as proposed by Marr [[317](#)]. The arrows (**gauge-figures**) represent surface orientation. The dashed lines are surface orientation discontinuities and the black lines are occlusion boundaries. The pixel intensities represent distance between the observer and the visible surfaces of the cube. Depth is represented by gray levels. Intensity increases with depth.

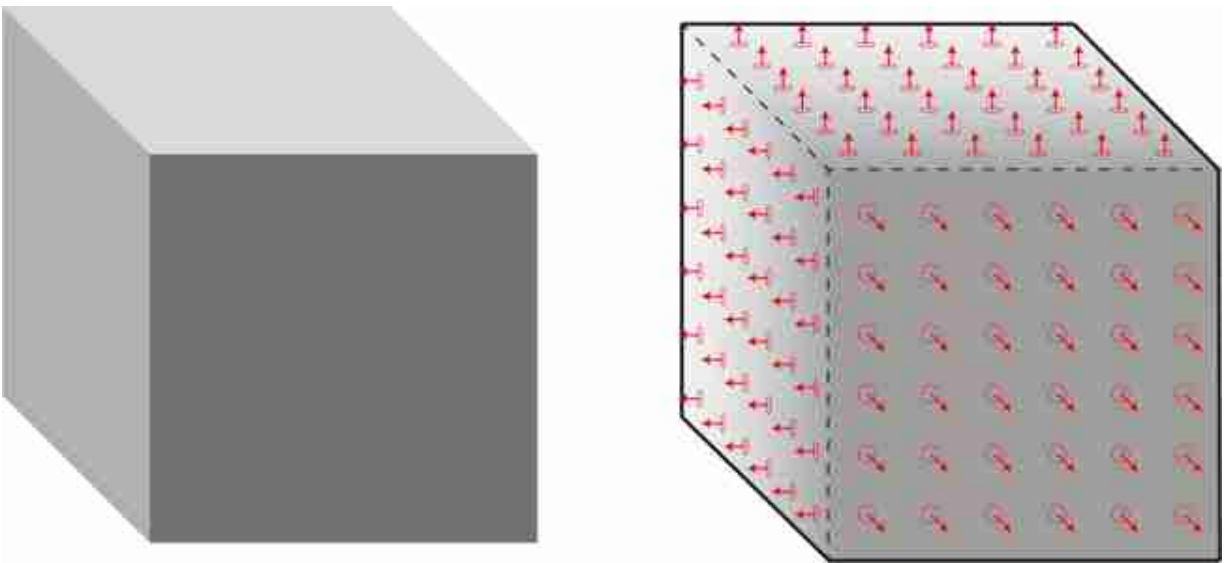


Figure 1.14: Image of a cube, and its 2.5D sketch. Figure inspired from figure 4-2 from [[317](#)].

It is hard to overstate the influence that Marr's book had in shaping the thinking of several generations of computer vision scientists, directly or indirectly. Perhaps Marr's shortcoming was underestimating the role of learning in constructing representations. A learning-based architecture could determine the sequence of representations that lead to the desired behavior, which is the premise of deep learning. Current approaches specify vision tasks in terms on training and test sets, loss function, representation, architecture, and optimization as we will discuss later.

1.3.7 Computer Vision

Computer vision has become an exciting and fast evolving area of research. We believe it is important to understand the field of computer vision within the scientific context that has been laid out in the previous sections. Computer vision is not just an engineering discipline that tries to build systems that see. Instead computer vision is another aspect of the interdisciplinary scientific quest that has focused on understanding how natural intelligence and perception works.

Doing a review of the field of computer vision and being fair to all of the people that have contributed to the field in a short subsection such as this one is not a reasonable task. It has the challenge that recent history is difficult to summarize as it requires fine-grained temporal resolution, and in many cases, we lack the perspective to identify the crucial discoveries. We will include those references throughout the book and we will just paint here, with a few coarse strokes, the main directions in the field.

In 1963, at MIT, Larry Roberts became the first computer vision Ph.D. entitled *Machine Perception of Three-dimensional Solids* [406].

Larry Roberts did his Ph.D. under the supervision of Peter Elias, and expert in the field of information theory that introduced convolutional codes, a type of error-correcting code, still in use in communication systems.

The period of 1960–1990 was dominated by the geometric perspective of computer vision and included concepts such as stereo vision [306], shape from shading [217], multiview geometry [122, 187], motion estimation [218], and model driven object recognition [129, 343]. These methods transformed old hypothesis into concrete algorithms capable of extracting information from images. During this period there were also many advances in building image representations such as edge-based representations [69, 186, 269, 384], filter-based image pyramids [173, 66, 271, 315, 140], and Thomas O. Binford's generalized cylinders for 3D object representation [50], as well as rule-based vision systems.

The decade of 1990–2000 experienced an acceleration of computer vision, in part because digital cameras became common. There was a change toward measuring ecological image properties [160], shifting away from rigid model-driven approaches, and focusing on human-driven perceptual properties such as image segmentation [434], and texture modeling [46, 315]. Object detection using learning-based approaches

started gaining attention and there were significant advances in face detection [414, 290, 340]. The field of computer vision got broken into specialized subareas targeting different tasks and grew with a strong solid foundation based on first principles.

The decade of 2000–2010 focused on image classification and object recognition [489, 126, 93, 124] and the creation of medium size benchmarks, with tens of thousands of images, started driving progress (Caltech 101 [294], PASCAL [118], LabelMe [420], CIFAR [278]). The introduction of benchmarks popularized new metrics and guidelines for reproducible research in the field. There was an explosion of image descriptors (BoW [86], SIFT [309], HOG [93], GIST [359]) that, combined with machine learning (decision trees [289], boosting [469], support vector machines, and graphical models and belief propagation), provided the basis to address many vision tasks. The first very large image databases, with millions of images, (Tiny Images [473], ImageNet [419], COCO [298]) appeared with the hypothesis that most of the tasks on vision need to be learned. Although learning played an important role, most of the computer vision architectures and features were designed manually.

Some example images from the COIL dataset [352]:



The decade of 2010–2020 has been clearly dominated by learning-based methods using deep learning as the main framework in all areas of computer vision.

1.3.8 Learning-Based Vision

While many previous perspectives on perception began with models about how perception might work, learning-based approaches take a different path. They rely on flexible architectures that incorporate as few hypothesis about perception as possible, and instead focus on data to learn to build the right model for perception.

Learning-based vision has its roots in the philosophical tradition of empiricism, which holds that knowledge about the world should be

acquired by experience, rather than solely through reason (an approach known as rationalism). The modern day term for experience is “data,” and learning is really all about data.

Learning derives vision algorithms by fitting functions to data. One way to think about it is that our goal is to “program” a vision algorithm and there are a few ways to do so ([figure 1.15](#)). One way is to write the algorithm directly in Python code: first we will compare the intensity of this pixel and that pixel, then we will threshold about some such value, and so on. Another approach is to derive the algorithm as the rational thing to do given some underlying model of the world, for example, because overlapping objects generically produce contrast edges in the image, the optimal way to detect boundaries between objects is to look for such and such particular kind of contrast. The third approach — the learning approach — is to program algorithms via data: first collect a bunch of examples of inputs and the correct output, then fit a function that replicates these exemplar mappings from input to output.

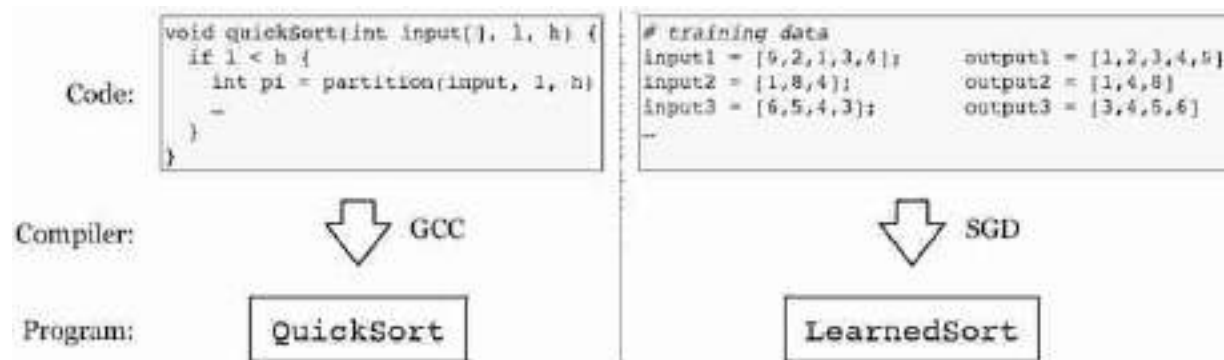


Figure 1.15: Two ways of programming an algorithm. Left: traditional way, Right: the machine learning way. You will learn about the machine learning “compiler”, called Stochastic Gradient Descent (SGD), in chapter 14.

Learning algorithms have been a part of computer vision since the field's origins. In 1958, Frank Rosenblatt published one of the first formal learning algorithms, called the **perceptron** learning algorithm, and showed that it can, in principle, learn to recognize certain simple visual patterns [413]. The perceptron was also one of the first artificial neural nets, a computational model of biological brains. The ability of perceptrons to learn to solve recognition problems created an explosion of interest in neural nets.

Rosenblatt published the perceptron algorithm in the journal *Psychological Review* in 1958. This is a nice example of the interdisciplinary nature of the AI field in its origins.

Interest in neural nets has since gone through a series of ebbs and flows ([figure 1.16](#)). Note that these cycles have partially lined up with interest in learning algorithms more generally, but there have also been phases where interest in neural nets was low but interest in other learning algorithms was high.

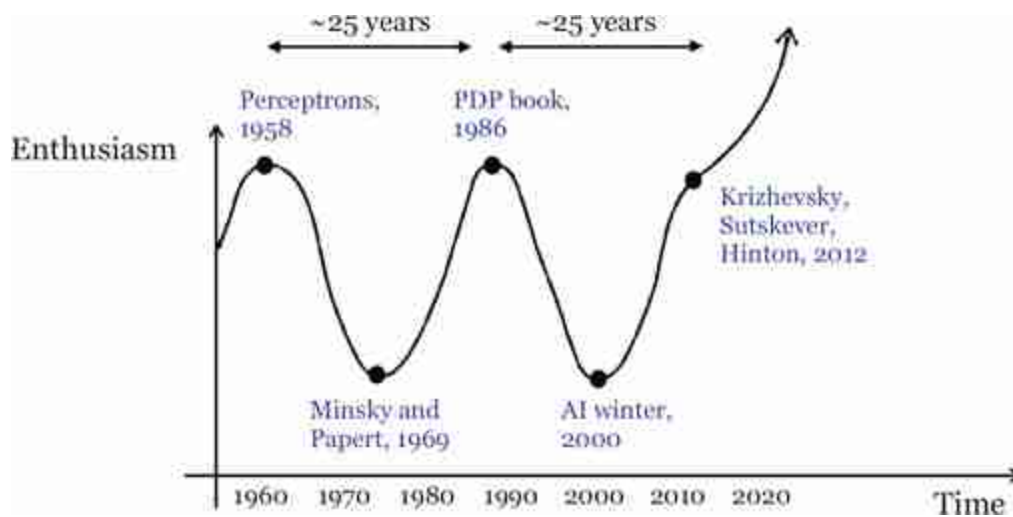


Figure 1.16: Enthusiasm for neural nets has gone up and down over time, with a period of roughly 25 years. What will happen next?

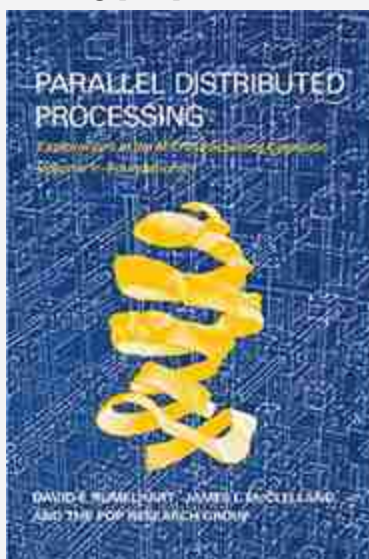
In 1969, Marvin Minsky and Seymour Papert published a book titled *Perceptrons: an introduction to computational geometry* [339]. You might think this would be a book that would increase the excitement around perceptrons, but in fact it largely killed it. Minsky and Papert showed that perceptrons are incapable of learning basic logical operations. Clearly then, perceptrons could not be the basis of intelligence.

It wasn't until the 1980s that interest in learning algorithms, and neural nets in particular, began to surge again. The reason was because researchers showed that perceptrons could be stacked to create systems capable of complex operations, including logic. These systems were the precursors to the **deep learning** systems that dominate today. In this decade, several architectures specific to vision were also introduced, including Kunihiro

Fukushima's *neocognitron*, which was a precursor to modern convolutional neural networks [148]. In 1989, Yann LeCun et al. showed how these kinds of networks can be efficiently trained [287] and their system was put into use for recognizing handwritten digits by the US Postal Service.

One of the peaks of neural network research in the 1980s was the publication of *Parallel Distributed Processing* by David Rumelhart and Jay McClelland [417]. This book mixed the neuroscience of the brain with computational models of distributed processing in artificial networks.

Parallel Distributed Processing [417] cover:



Unfortunately, however, at that time we did not have the compute and data necessary to make these systems really work, and interest waned during the 1990s. By the year 2000, the field of AI as a whole was firmly in a “winter”; the promises of the 1980s had failed to materialize and the study of learning algorithms continued on without much fanfare in just a few corners of academia. Nonetheless, several major advances did occur in the relative quiet of the 1990s and early 2000s. For example, in 2001, Paul Viola and Michael Jones published a method that used a simple learning algorithm called **boosting** to detect faces in photos [489].

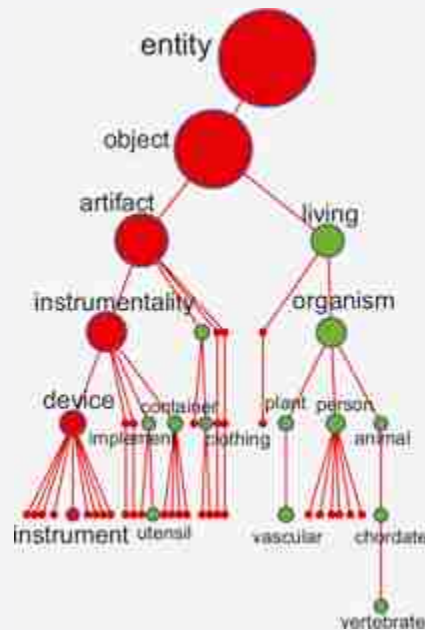
Face detection example from [\[489\]](#):



This method outperformed the complicated hand-engineered face detectors of the time in terms of computational efficiency and detection rate, and even became a popular algorithm used in consumer cameras.

The 2000s were a decade where learning returned to the foreground. One common pattern was to take the best hand-engineered features and feed them as input to a shallow learning machine, such as a support vector machine [\[83\]](#). Gradually more and more of the vision pipeline was replaced with learned modules. Finally, in 2012, a deep neural net called AlexNet dramatically outperformed all existing methods for image classification, and did so with a system that used learning **end-to-end** (for each processing stage from the input pixels to the output predictions) [\[279\]](#). This event has come to be called the “ImageNet Moment” since AlexNet's dramatic performance was first demonstrated at the 2012 ImageNet competition [\[419\]](#).

WordNet is an electronic dictionary used to define word hierarchies [123]:



The ImageNet moment was a watershed for learning-based vision and for neural networks. In the few years after 2012, learning became the key ingredient in almost all research on computer vision. As of the 2020s, it is hard to find any computer vision system that does not use deep learning somewhere in its pipeline.

It may seem like learning-based vision is very different than other approaches, but actually it is similar in many ways. The study of vision consists of the structure of the input, the structure of the output, and the mapping between the two. Learning-based vision does not fundamentally change any of these: the input and target output are the same as before, and the mapping is remarkably similar as in other approaches. The big difference with learning-based vision is how you *find* the mapping. In this sense, learners are just like a new kind of vision scientist: they play the same role as the humans who, in previous eras, sat down and thought deeply to determine the way vision works.

In this book we will see that very often machine learners come up with similar vision algorithms as were previously designed by humans. But in many ways machine learners are also more powerful than human designers, and machine learned algorithms have now far exceeded those designed by humans in terms of raw performance.

1.4 What's Next?

In a span of more than 2000 years, from the Greeks' extramission theories of vision to the understanding of the neural mechanisms and the engineering of computer vision systems, we have come a long way in our understanding of the nature of perception. Each step during the evolution of vision theories revealed important aspects of perception ignored by the previous theories. What theories will come next? What is missing now? Certainly, many things. For instance, we are missing visual common sense (now powered by large language models), integration with other perceptual systems, and, most importantly, a theory of embodied perception. And those are just the most obvious. There might be many other steps missing that we are not smart enough to see from where we are standing right now.

1.5 Concluding Remarks

The goal of this chapter was to define the topic of computer vision by placing it inside its historical context. We believe that any researcher of computer vision would benefit by studying vision from an interdisciplinary perspective as it will provide a solid basis for creativity, technical depth, and an understanding of its societal impact. We hope this chapter also inspires students to learn more about this fascinating field and to get excited by its potential.

I

FOUNDATIONS

This first block of three chapters defines the problem that computer vision tries to solve, introduces some of the mathematical tools one needs to be familiar with, and poses the study of computer vision as a scientific and engineering effort with an enormous societal impact that the reader should be familiar with initially and not as an after thought.

- **Chapter 2** will make you think about the math foundations needed to understand the rest of the book and it will also introduce some important general concepts in vision.
- **Chapter 3** will make you think about the vision problem from the perspective of an observer.
- **Chapter 4** will make you think about the social implications of this area of work.

Readers might want to skip this part and directly jump into the latest and most exciting techniques in computer vision (deep learning, image generative models) that will be presented later in the book, but it is important to slow down and be patient. We believe that being patient will result in rewards in the long run. Let's first focus on foundational concepts.

2 A Simple Vision System

2.1 Introduction

The goal of this chapter is to embrace the optimism of the 1960s and to hand-design an end-to-end visual system. During this process, we will cover some of the main concepts that will be developed in the rest of the book.

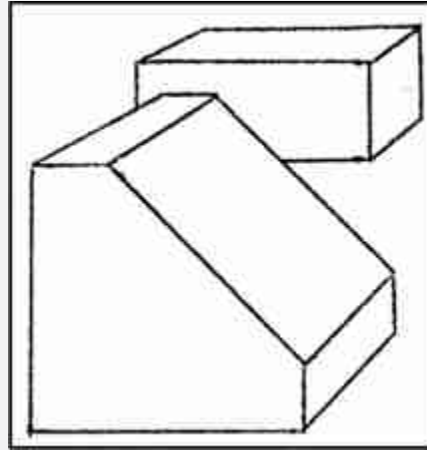
In 1966, Seymour Papert wrote a proposal for building a vision system as a summer project [374]. The abstract of the proposal starts by stating a simple goal: “The summer vision project is an attempt to use our summer workers effectively in the construction of a significant part of a visual system.” The report then continues dividing all the tasks (most of which also are common parts of modern computer vision approaches) among a group of MIT students. This project was a reflection of the optimism existing in the early days of computer vision. However, the task proved to be harder than anybody expected.

In this first chapter, we will discuss several of the main topics that we will cover in this book. We will do this in the framework of a real, although a bit artificial, vision problem. Vision has many different goals (e.g., object recognition, scene interpretation, three-dimensional [3D] interpretation), but in this chapter we're just focusing on the task of 3D interpretation.

2.2 A Simple World: The Blocks World

As the visual world is too complex, we will start by simplifying it enough that we will be able to build a simple visual system right away. This was the strategy used by some of the first scene interpretation systems. Larry G. Roberts [406] introduced the **Block world**, a world composed of simple 3D geometrical figures.

Blocks world from Larry Roberts' Ph.D. in June 1963.



For the purposes of this chapter, let's think of a world composed of a very simple (yet varied) set of objects. These simple objects are composed of flat surfaces that can be horizontal or vertical. These objects will be resting on a white horizontal ground plane. We can build these objects by cutting, folding, and gluing together some pieces of colored paper as shown in [figure 2.1](#). Here, we will not assume that we know the exact geometry of these objects in advance.

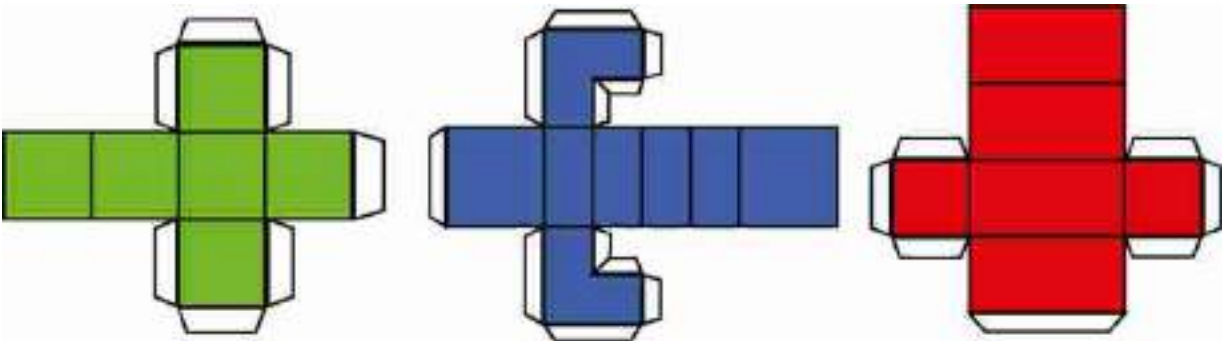


Figure 2.1: A world of simple objects. Print, cut, and build your own blocks world!

2.3 A Simple Image Formation Model

One of the simplest forms of projection is **parallel** (or **orthographic**) **projection**. In this image formation model, the light rays travel parallel to each other and perpendicular to the camera plane. This type of projection produces images in which objects do not change size as they move closer or

farther from the camera, and parallel lines in 3D remain parallel in the 2D image. This is different from the perspective projection, to be discussed in section 5.3, where the image is formed by the convergence of the light rays into a single point (focal point). If we do not take special care, most pictures taken with a camera will be better described by **perspective projection**, as shown in [figure 2.2\(a\)](#).

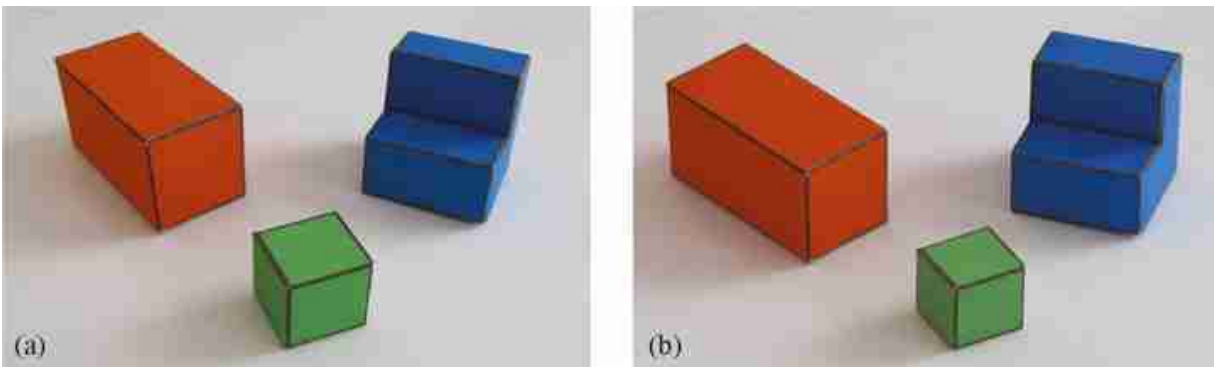


Figure 2.2: (a) Close up picture without zoom. Note that near edges are larger than far edges, and parallel lines in 3D are not parallel. (b) Picture taken from far away using zoom resulting in an image that can be described by parallel projection.

One way of generating images that can be described by parallel projection is to use the camera zoom. If we increase the distance between the camera and the object while zooming, we can keep the same approximate image size of the objects, but with reduced perspective effects, as shown in [figure 2.2\(b\)](#). Note how, in [figure 2.2\(b\)](#), 3D parallel lines in the world are almost parallel in the image (some weak perspective effects remain).

Changing the zoom to compensate a moving camera can be used to create movie effects. This method was introduced by Alfred Hitchcock to create a sequence, centered in a subject, where the background moves away or closer to the subject.

The first step is to characterize how a point in world coordinates (X , Y , Z) projects into the image plane. [Figure 2.3\(a\)](#) shows our parameterization of the world and the camera coordinate systems. The camera center is inside the 3D plane $X = 0$, and the horizontal axis of the camera (x) is parallel to the ground plane ($Y = 0$). The camera is tilted so that the line connecting the

origin of the world coordinates system and the image center is perpendicular to the image plane. The angle θ is the angle between this line and the Z -axis. The image is parameterized by coordinates (x, y) .



Figure 2.3: A simple projection model. (a) World axis and camera plane. (b) Visualization of the world axis projected into the camera plane with parallel projection. The Z -axis is identical to the Y -axis up to a sign change and a scaling.

In this simple projection model, the origin of the world coordinates projects on the origin of the image coordinates. Therefore, the world point $(0, 0, 0)$ projects into $(0, 0)$. The resolution of the image (the number of pixels) will also affect the transformation from world coordinates to image coordinates via a constant factor α (for now we assume that pixels are square and we will see a more general form in section 39) and that this constant is $\alpha = 1$. Taking into account all these assumptions, the transformation between world coordinates and image coordinates can be written as follows:

$$x = X \quad (2.1)$$

$$y = \cos(\theta)Y - \sin(\theta)Z \quad (2.2)$$

With this particular parametrization of the world and camera coordinate systems, the world coordinates Y and Z are mixed after projection. From the camera, a point moving parallel to the Z -axis will be indistinguishable from a point moving parallel to the Y -axis.

2.4 A Simple Goal

Part of the simplification of the vision problem resides in simplifying its goals. In this chapter we will focus on recovering the world coordinates of all the pixels seen by the camera.

Besides recovering the 3D structure of the scene, there are many other possible goals that we will not consider in this chapter. For instance, one goal (which might seem simpler but is not) is to recover the actual color of the surface seen by each pixel (x, y) . This will require discounting for illumination effects as the color of the pixel is a combination of the surface albedo and illumination (color of the light sources and interreflections).

2.5 From Images to Edges and Useful Features

The observed image is a function,

$$I(x, y) \tag{2.3}$$

that takes as input location, (x, y) , and it outputs the intensity at that location. In this **representation**, the image is an array of intensity values (color values) indexed by location ([figure 2.4](#)).

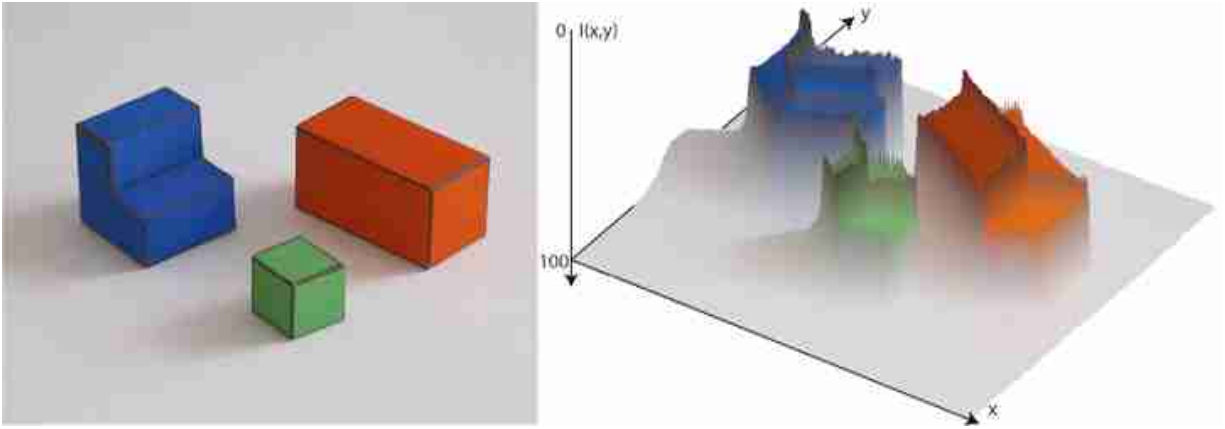


Figure 2.4: Image as a surface. The vertical axis corresponds to image intensity. For clarity here, we have reversed the vertical axis. Dark values are shown higher than lighter values.

This representation is ideal for determining the light intensity originating from different directions in space and striking the camera plane, as it provides explicit representation of this information. The array of pixel intensities, $l(x, y)$, is a reasonable representation as input to the early stages of visual processing because, although we do not know the distance of surfaces in the world, the direction of each light ray in the world is well defined. However, other initial representations could be used by the visual system (e.g., images could be coded in the Fourier domain, or pixels could combine light coming in different directions).

However, in the simple visual system of this chapter we are interested in interpreting the 3D structure of the scene and the objects within. Therefore, it will be useful to transform the image into a representation that makes more explicit some of the important parts of the image that carry information about the boundaries between objects and changes in the surface orientation.

There are several representations that can be used as an initial step for scene interpretation. Images can be represented as collections of small image patches, regions of uniform properties, and edges.

Question: Are there animal eyes that produce a different initial representations than ours? **Answer:** Yes! One example is the Gekko's eye. Their pupil has a four-diamond-shaped pinhole aperture that could allow them to encode distance to a target in the retinal image [\[345\]](#). Photo credit: [\[1\]](#).



2.5.1 A Catalog of Edges

Edges denote image regions where there are strong changes of the image with respect to location. Those variations can be due to a multitude of scene factors (e.g., occlusion boundaries, changes in surface orientation, changes in surface albedo, shadows).

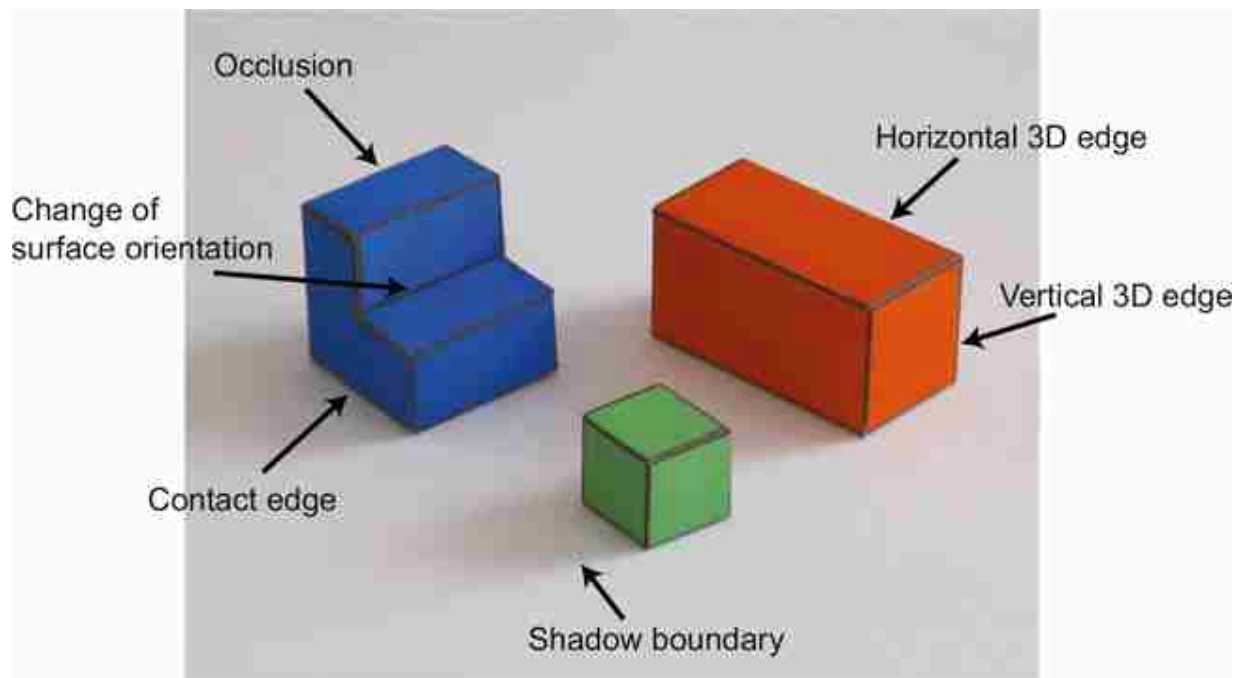


Figure 2.5: Edges denote image regions where there are sharp changes of the image intensities. Those variations can be due to a multitude of scene factors (e.g., occlusion boundaries, changes in surface orientation, changes in surface albedo, and shadows).

One of the tasks that we will solve first is to classify **image edges** according to their most probable cause. We will use the following classification of image boundaries ([figure 2.5](#)):

- **Object boundaries:** These indicate pixels that delineate the boundaries of any object. Boundaries between objects generally correspond to changes in surface color, texture, and orientation.
- **Changes in surface orientation:** These indicate locations where there are strong image variations due to changes in the surface orientations. A change in surface orientation produces changes in the image intensity because intensity is a function of the angle between the surface and the incident light.
- **Shadow edges:** This can be harder than it seems. In this simple world, shadows are soft, creating slow transitions between dark and light.

We will also consider two types of object boundaries:

- **Contact edges:** This is a boundary between two objects that are in physical contact. Therefore, there is no depth discontinuity.

- **Occlusion boundaries:** Occlusion boundaries happen when an object is partially in front of another. Occlusion boundaries generally produce depth discontinuities. In this simple world, we will position the objects in such a way that objects do not occlude each other but they will occlude the background.

Despite the apparent simplicity of this task, in most natural scenes, this classification is very hard and requires the interpretation of the scene at different levels. In other chapters we will see how to make better edge classifiers (i.e., by propagating information along boundaries, junction analysis, inferring light sources).

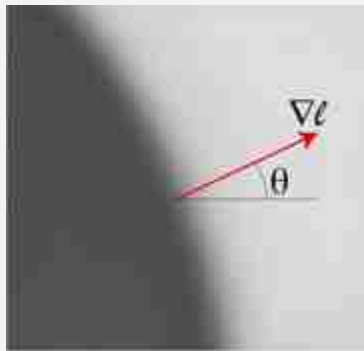
2.5.2 Extracting Edges from Images

The first step will consist in detecting candidate edges in the image. Here we will start by making use of some notions from differential geometry. If we think of the image $\ell(x, y)$ as a function of two (continuous) variables ([figure 2.4](#)), we can measure the degree of variation using the gradient:

$$\nabla \ell = \left(\frac{\partial \ell}{\partial x}, \frac{\partial \ell}{\partial y} \right) \quad (2.4)$$

The direction of the gradient indicates the direction in which the variation of intensities is larger. If we are on top of an edge, the direction of larger variation will be in the direction perpendicular to the edge.

Gradient of an image at one location:



However, the image is not a continuous function as we only know the values of the $\ell(x, y)$ at discrete locations (pixels). Therefore, we will approximate the partial derivatives by:

$$\frac{\partial \ell}{\partial x} \simeq \ell(x, y) - \ell(x-1, y) \quad (2.5)$$

$$\frac{\partial \ell}{\partial y} \simeq \ell(x, y) - \ell(x, y-1) \quad (2.6)$$

A better behaved approximation of the partial image derivative can be computed by combining the image pixels around (x, y) with the weights:

$$\frac{1}{4} \times \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

We will discuss these approximations in detail in chapter 18.

From the image gradient, we can extract a number of interesting quantities:

$$e(x, y) = \|\nabla \ell(x, y)\| \quad \leftarrow \text{edge strength} \quad (2.7)$$

$$\theta(x, y) = \angle \nabla \ell = \arctan \left(\frac{\partial \ell / \partial y}{\partial \ell / \partial x} \right) \quad \leftarrow \text{edge orientation} \quad (2.8)$$

The edge strength is the gradient magnitude, and the edge orientation is perpendicular to the gradient direction.

The unit norm vector perpendicular to an edge is:

$$\mathbf{n} = \frac{\nabla \ell}{\|\nabla \ell\|} \quad (2.9)$$

The first decision that we will perform is to decide which pixels correspond to edges (regions of the image with sharp intensity variations) and which ones belong to uniform regions (flat surfaces). We will do this by simply thresholding the edge strength $e(x, y)$. In the pixels with edges, we can also measure the edge orientation $\theta(x, y)$. [Figure 2.6](#) visualizes the edges and the normal vector on each edge.

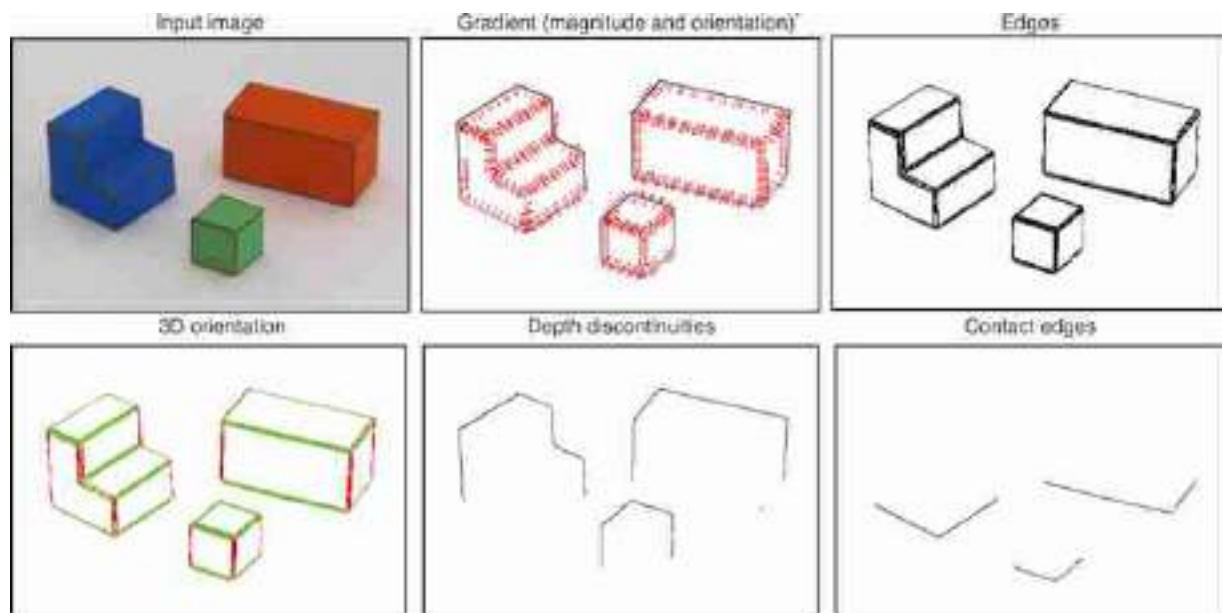


Figure 2.6: Gradient and edge types.

2.6 From Edges to Surfaces

We want to recover world coordinates $X(x, y)$, $Y(x, y)$, and $Z(x, y)$ for each image location (x, y) . Given the simple image formation model described before, recovering the X world coordinates is trivial as they are directly observed: for each pixel with image coordinates (x, y) , the corresponding world coordinate is $X(x, y) = x$. Recovering Y and Z will be harder as we only observe a mixture of the two world coordinates (one dimension is lost due to the projection from the 3D world into the image plane). Here we have written the world coordinates as functions of image location (x, y) to make explicit that we want to recover the 3D locations of the visible points.

In this simple world, we will formulate this problem as a set of linear equations.

2.6.1 Figure/Ground Segmentation

Segmentation of an image into figure and ground is a classical problem in human perception and computer vision that was introduced by **Gestalt psychology**.

The classical visual illusion “two faces or a vase” is an example of **figure-ground** segmentation problem.



In this simple world deciding when a pixel belongs to one of the **foreground** objects or to the **background** can be decided by simply looking at the color values of each pixel. Bright pixels that have low saturation (similar values of the red-blue-green [RBG] components) correspond to the white ground plane, and the rest of the pixels are likely to belong to the colored blocks that compose our simple world. In general, the problem of **image segmentation** into distinct objects is a very challenging task.

Once we have classified pixels as ground or figure, if we assume that the background corresponds to a horizontal ground plane, then for all pixels that belong to the ground we can set $Y(x, y) = 0$. For pixels that belong to objects we will have to measure additional image properties before we can deduce any geometric scene constraints.

2.6.2 Occlusion Edges

An occlusion boundary separates two different surfaces at different distances from the observer. Along an occlusion edge, it is also important to know which object is in front as this will be the one owning the boundary. Knowing who owns the boundary is important as an edge provides cues about the 3D geometry, but those cues only apply to the surface that owns the boundary.

Border ownership: The foreground object is the one that owns the common edge.



The dotted lines belong to the object in front.

In this simple world, we will assume that objects do not occlude each other (this can be relaxed) and that the only occlusion boundaries are the boundaries between the objects and the ground. However, as we describe subsequently, not all boundaries between the objects and the ground correspond to depth gradients.

2.6.3 Contact Edges

Contact edges are boundaries between two distinct objects but where there exists no depth discontinuity. Despite that there is not a depth discontinuity, there is an occlusion here (as one surface is hidden behind another), and the edge shape is only owned by one of the two surfaces.

In this simple world, if we assume that all the objects rest on the ground plane, then we can set $Y(x, y) = 0$ on the contact edges. Contact edges can be detected as transitions between the object (above) and ground (below). In our simple world only horizontal edges can be contact edges. We will discuss next how to classify edges according to their 3D orientation.

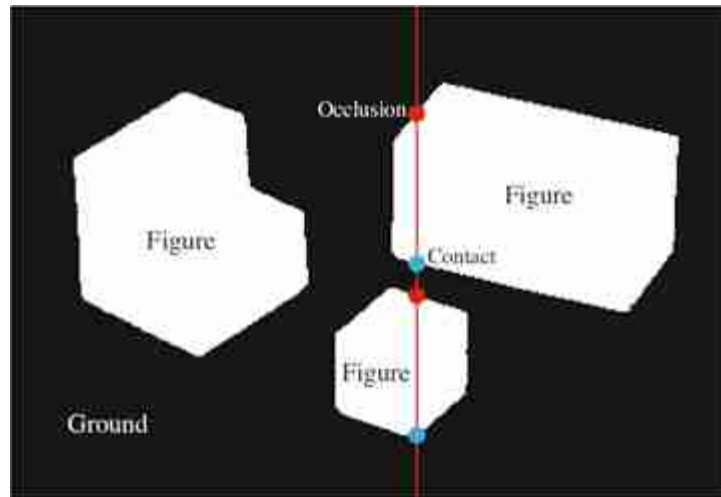
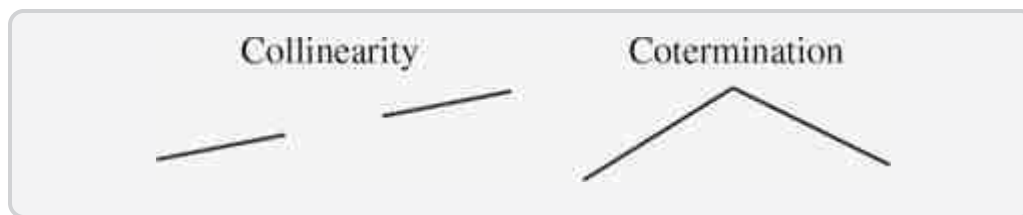


Figure 2.7: For each vertical line (shown in red), scanning from top to bottom, transitions from ground to figure are occlusion boundaries, and transitions from figure to ground are contact edges. This heuristic will fail when an object occludes another.

2.6.4 Generic View and Nonaccidental Scene Properties

Despite that in the projection of world coordinates to image coordinates we have lost a great deal of information, there are a number of properties that will remain invariant and can help us in interpreting the image. Here is a list of some of those invariant properties:

- **Collinearity:** a straight 3D line will project into a straight line in the image.
- **Cotermination:** if two or more 3D lines terminate at the same point, the corresponding projections will also terminate at a common point.
- **Smoothness:** a smooth 3D curve will project into a smooth 2D curve.



Note that those invariances refer to the process of going from world coordinates to image coordinates. The opposite might not be true. For instance, a straight line in the image could correspond to a curved line in the 3D world but that happens to be precisely aligned with respect to the viewer's point of view to appear as a straight line. Also, two lines that intersect in the image plane could be disjointed in the 3D space.

However, some of these properties (not all), while not always true, can nonetheless be used to reliably infer something about the 3D world using a single 2D image as input. For instance, if two lines coterminate in the image, then, one can conclude that it is very likely that they also touch each other in 3D. If the 3D lines do not touch each other, then it will require a very specific alignment between the observer and the lines for them to appear to coterminate in the image. Therefore, one can safely conclude that the lines might also touch in 3D.

These properties are called **nonaccidental properties** [310] because they will only be observed in the image if they also exist in the world or by accidental alignments between the observer and scene structures. Under a **generic view**, nonaccidental properties will be shared by the image and the 3D world [138].

Let's see how this idea applies to our simple world. In this simple world all 3D edges are either vertical or horizontal. Under parallel projection and with the camera having its horizontal axis parallel to the ground, we know that vertical 3D lines will project into vertical 2D lines in the image. Conversely, horizontal lines will, in general, project into oblique lines. Therefore, we can assume that any vertical line in the image is also a vertical line in the world.

However, the assumption that vertical 2D lines are also 3D vertical lines will not always work. As shown in [figure 2.8](#), in the case of the cube, there is a particular viewpoint that will make an horizontal line project into a vertical line, but this will require an accidental alignment between the cube and the line of sight of the observer. Nevertheless, this is a weak property and accidental alignments such as this one can occur, and a more general algorithm will need to account for that. But for the purposes of this chapter we will consider images with **generic views** only.



Figure 2.8: The generic-view assumption breaks down for the central image, where the cube is precisely aligned with the camera axis.

In [figure 2.6](#) we show the edges classified as vertical or horizontal using the edge angle. Anything that deviates from 2D verticality by more than 15 degrees is labeled as 3D horizontal.

We can now translate the inferred 3D edge orientation into linear constraints on the global 3D structure. We will formulate these constraints in terms of $Y(x, y)$. Once $Y(x, y)$ is recovered we can also recover $Z(x, y)$ from equation (2.2).

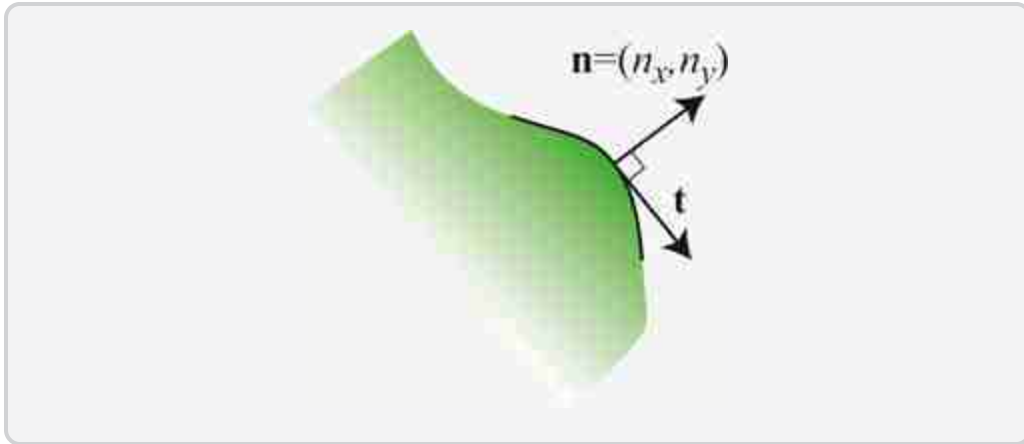
In a 3D vertical edge, using the projection equations, the derivative of Y along the edge will be

$$\partial Y / \partial y = 1 / \cos(\theta) \quad (2.10)$$

In a 3D horizontal edge, the coordinate Y will not change. Therefore, the derivative along the edge should be zero:

$$\partial Y / \partial \mathbf{t} = 0 \quad (2.11)$$

where the vector $\mathbf{t}$ denotes direction tangent to the edge, $\mathbf{t} = (-n_y, n_x)$.



We can write this derivative as a function of derivatives along the x and y image coordinates:

$$\partial Y / \partial \mathbf{t} = \nabla Y \cdot \mathbf{t} = -n_x \partial Y / \partial x + n_y \partial Y / \partial y \quad (2.12)$$

When the edges coincide with occlusion edges, special care should be taken so that these constraints are only applied to the surface that owns the boundary.

In this chapter, we represent the world coordinates $X(x, y)$, $Y(x, y)$, and $Z(x, y)$ as images where the coordinates x, y correspond to pixel locations. Therefore, it is useful to approximate the partial derivatives in the same way that we approximated the image partial derivatives in equations (2.6). Using this approximation, equation (2.10) can be written as follows:

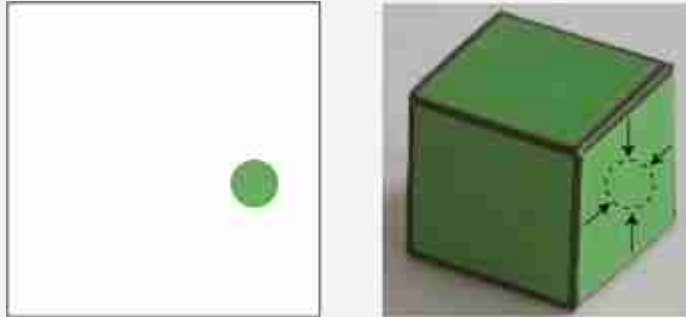
$$Y(x, y) - Y(x, y - 1) = l / \cos(\theta) \quad (2.13)$$

Similar relationships can be obtained from equations (2.11) and (2.12).

2.6.5 Constraint Propagation

Most of the image consists of flat regions where we do not have such edge constraints and we thus don't have enough local information to infer the surface orientation. Therefore, we need some criteria in order to propagate information from the boundaries, where we do have information about the 3D structure, into the interior of flat image regions. This problem is common in many visual domains.

As shown on the left image bellow, an image patch without context is not enough to infer its 3D shape. The same patch shown in the original image (right):



Information about its 3D orientation is propagated from the surrounding edges.

In this case we will assume that the object faces are planar. Thus, flat image regions impose the following constraints on the local 3D structure:

$$\partial^2 Y / \partial x^2 = 0 \quad (2.14)$$

$$\partial^2 Y / \partial y^2 = 0 \quad (2.15)$$

$$\partial^2 Y / \partial y \partial x = 0 \quad (2.16)$$

That is, the second order derivative of Y should be zero. As before, we want to approximate the continuous partial derivatives. The approximation to the second derivative can be obtained by applying twice the first order derivative approximated by equations (2.6). The result is $\partial^2 Y / \partial x^2 \approx 2Y(x, y) - Y(x + 1, y) - Y(x - 1, y)$, and similarly for $\partial^2 X / \partial x^2$.

2.6.6 A Simple Inference Scheme

All the different constraints described previously can be written as an overdetermined system of linear equations. Each equation will have the form:

$$\mathbf{a}_i \mathbf{Y} = b_i \quad (2.17)$$

where $\mathbf{Y}$ is a vectorized version of the image Y (i.e., all rows of pixels have been concatenated into a flat vector). Note that there might be many more equations than there are image pixels.

We can translate all the constraints described in the previous sections into this form. For instance, if the index i corresponds to one of the pixels inside one of the planar faces of a foreground object, then there will be three equations. One of the planarity constraint can be written as $\mathbf{a}_i = [0, \dots, 0, -1, 2, -1, 0, \dots, 0]$, $b_i = 0$, and analogous equations can be written for the other two.

We can solve the system of equations by minimizing the following cost function:

$$J = \sum_i (\mathbf{a}_i \mathbf{Y} - b_i)^2 \quad (2.18)$$

where the sum is over all the constraints.

If some constraints are more important than others, it is possible to also add a weight w_i .

$$J = \sum_i w_i (\mathbf{a}_i \mathbf{Y} - b_i)^2 \quad (2.19)$$

Our formulation has resulted on a big system of linear constraints (there are more equations than there are pixels in the image). It is convenient to write the system of equations in matrix form:

$$\mathbf{A}\mathbf{Y}=\mathbf{b} \quad (2.20)$$

where row i of the matrix $\mathbf{A}$ contains the constraint coefficients $\mathbf{a}_i$. The system of equations is overdetermined ($\mathbf{A}$ has more rows than columns). We can use the pseudoinverse to compute the solution:

$$\mathbf{Y}=(\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{b} \quad (2.21)$$

This problem can be solved efficiently as the matrix $\mathbf{A}$ is very sparse (most of the elements are zero).

2.6.7 Results

[Figure 2.9](#) shows the resulting world coordinates $X(x, y)$, $Y(x, y)$, $Z(x, y)$ for each pixel. The world coordinates are shown here as images with the gray level coding the value of each coordinate (black corresponds to the value 0).

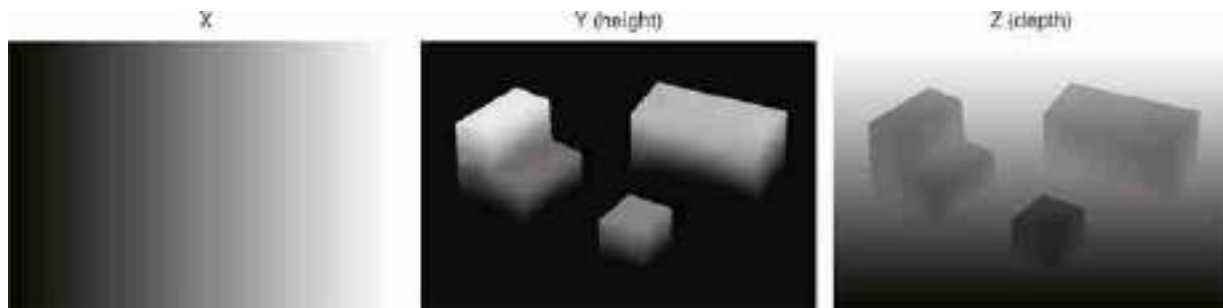


Figure 2.9: The solution to our vision problem for the input image of [figure 2.5](#). World coordinates X , Y , and Z are shown as images.

There are a few things to reflect on:

- It works. At least it seems to work pretty well. Knowing how well it works will require having some way of evaluating performance. This will be important.
- But it cannot possibly work all the time. We have made lots of assumptions that will work only in this simple world. The rest of the book will involve upgrading this approach to apply to more general input images.

Despite that this approach will not work on general images, many of the general ideas will carry over to more sophisticated solutions (e.g., gather and propagate local evidence). One thing to think about is if information

about 3D orientation is given in the edges, how does that information propagate to the flat areas of the image in order to produce the correct solution in those regions?

Evaluation of performance is a very important topic. Here, one simple way to visually verify that the solution is correct is to render the objects under new view points. [Figure 2.10](#) shows the reconstructed 3D scene rendered under different view points to show that the 3D structure has been accurately captured.

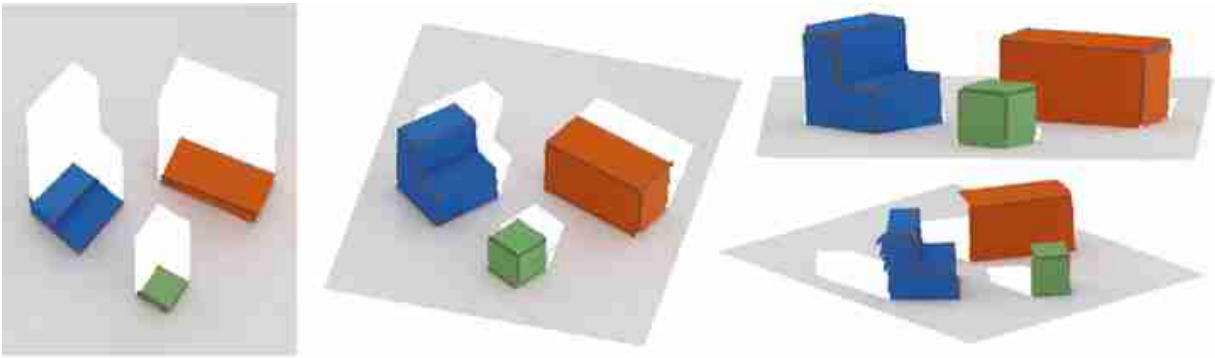


Figure 2.10: To show that the algorithm for 3D interpretation gives reasonable results we can re-render the inferred 3D structure from different viewpoints.

2.7 Generalization

One desired property of any vision system is its ability to **generalize** outside of the domain for which it was designed to operate. In the approach presented in this chapter, the domain of images the algorithm is expected to work is defined by the assumptions described in section 2.5. Later in the book, when we describe learning-based approaches, the training dataset will specify the domain.

Out of domain generalization refers to the ability of a system to operate outside the **domain** for which it was designed.

What happens when assumptions are violated? Or when the images contain configurations that we did not consider while designing the algorithm?

Let's run the algorithm with shapes that do not satisfy the assumptions that we have made for the simple world. [Figure 2.11](#) shows the result when the input image violates several assumptions we made (shadows are not soft

and the green cube occludes the red one) and when it has one object on top of the other, a configuration that we did not consider while designing the algorithm. In this case the result happens to be good, but it could have gone wrong.

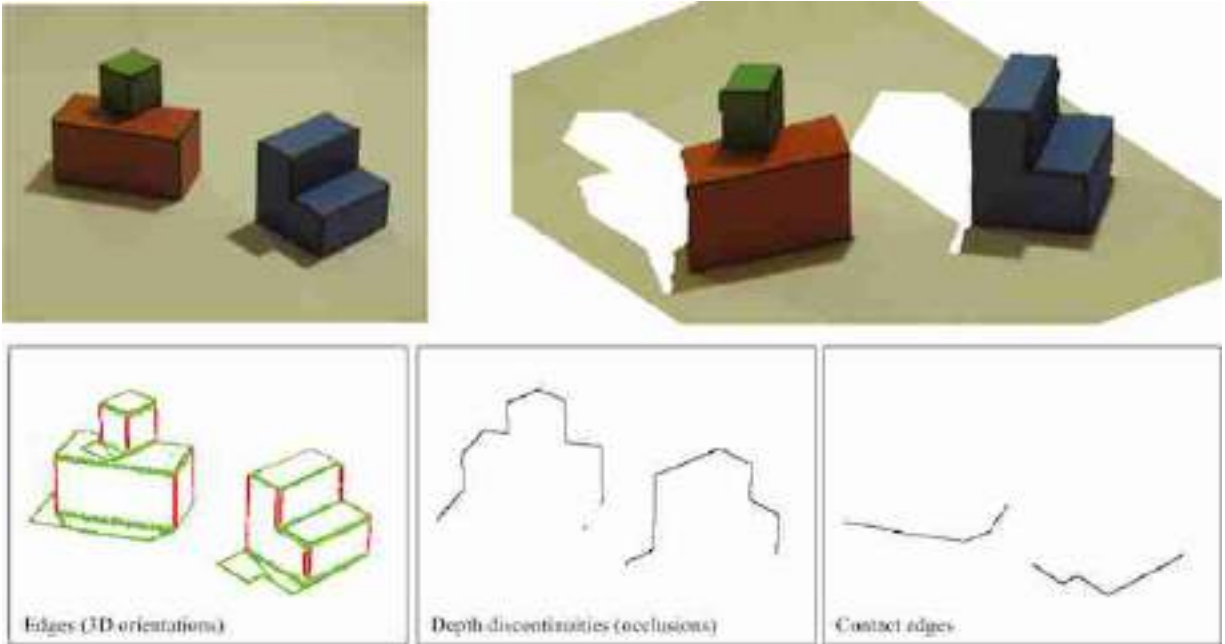


Figure 2.11: Reconstruction of an out-of-domain example. The input image violates several assumptions we made (shadows are not soft and the green cube occludes the red one) and it has one object on top of the other, a configuration that we did not consider while designing the algorithm.

[Figure 2.12](#) shows the *impossible steps* inspired from [7]. On the left side, this shape looks rectangular and the stripes appear to be painted on the surface. On the right side, the shape looks as though it has steps, with the stripes corresponding to shading due to the surface orientation. In the middle, the shape is ambiguous. [Figure 2.12](#) shows the reconstructed 3D scene for this unlikely image. The system has tried to approximate the constraints, but for this shape it is not possible to exactly satisfy all the constraints.

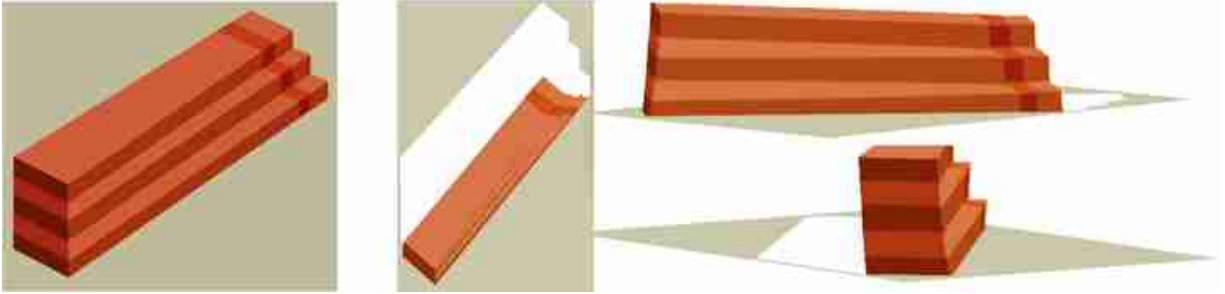


Figure 2.12: Reconstruction of an impossible figure (inspired from [7]): the reconstruction agrees with our 3D perception.

2.8 Concluding Remarks

Despite having a 3D representation of the structure of the scene, the system is still unaware of the fact that the scene is composed of a set of distinct objects. For instance, as the system lacks a representation of which objects are actually present in the scene, we cannot visualize the occluded parts. The system cannot do simple tasks like counting the number of cubes.

A different approach to the one discussed here is model-based scene interpretation, where we could have a set of predefined models of the objects that can be present in the scene and the system can try to decide if they are present or not in the image, and recover their parameters (i.e., pose, color).

Recognition allows indexing properties that are not directly available in the image. For instance, we can infer the shape of the invisible surfaces. Recognizing each geometric figure also implies extracting the parameters of each figure (i.e., pose, size).

Given a detailed representation of the world, we could render the image back, or at least some aspects of it. We can check that we are understanding things correctly and judge if we can make verifiable predictions, such as what we would see if we looked behind the object. Closing the loop between interpretation and input will be productive at some point.

In this chapter, besides introducing some useful vision concepts, we also made use of mathematical tools from algebra and calculus that you should be familiar with.

3 Looking at Images

3.1 Introduction

Vision is about solving an amazing problem: Can we build a system capable of perceiving its surrounding world using only the information available in the small number of electromagnetic waves that fall into the observer's sensor (eye or camera), after accidentally bounding off the objects surrounding the observer? How is it possible to sense the world and its three-dimensional (3D) structure without touching it? It seems quite likely that if humans did not have the sense of vision, we would think that solving this problem is impossible.

The goal of this chapter is to present vision not as an engineering problem that we want to solve, but as a set of scientific questions that we want to answer. We want to understand how it is that our visual system can interpret the world the way it does. We want to explain what components of an image can be used to infer certain properties of the world. We want to know what questions about the world can be answered by just looking at object with the light that reaches our eyes.

In this chapter we will not answer any of those questions, instead, we will use images to motivate a set of questions that vision scientists and computer vision engineers have been studying for many years. The goal of this chapter is to help you ask questions about how do you perceive the world around you.

3.2 Looking at Individual Pixels

Before we build systems that can see, it is good to learn to look at images and to pay attention to individual pixels. The image in [figure 3.1](#) is a tiny image with just 32×32 pixels. It is so small that we can actually look at each single pixel and think about how it is that we can make sense of it. For instance, let's look at the pixel in row 21 counting from the top, and column 20 counting from the left. That pixel corresponds to a dark gray pixel, which is near a couple of other pixels with similar intensity, surrounded by

a set of white pixels. That pixel seems to be a pen, but how do we know what it is? The pen is barely three pixels long and one pixel wide. How can that provide enough information?



Figure 3.1: A tiny image with 32×32 color pixels. Despite the very low resolution, we can still recognize most of its content. *Source:* Original image created with Dall-E, and downsampled with Photoshop.

Same image as in [figure 3.1](#) but shown small.



Reducing the size helps better recognizing the objects despite containing the same visual information.

Recognition of the meaning of that pixel can not come just from the intensity of that pixel or the few pixels nearby. There are way too many possible situations in which different objects in the world could map to a similar set of pixel intensities. Instead, is the whole image and the context of the pen that provides the necessary information to recognize the pen. Taken in isolation, those pixels are impossible to recognize. Here we show, from left to right, the hand, the pen, and the blue mug.

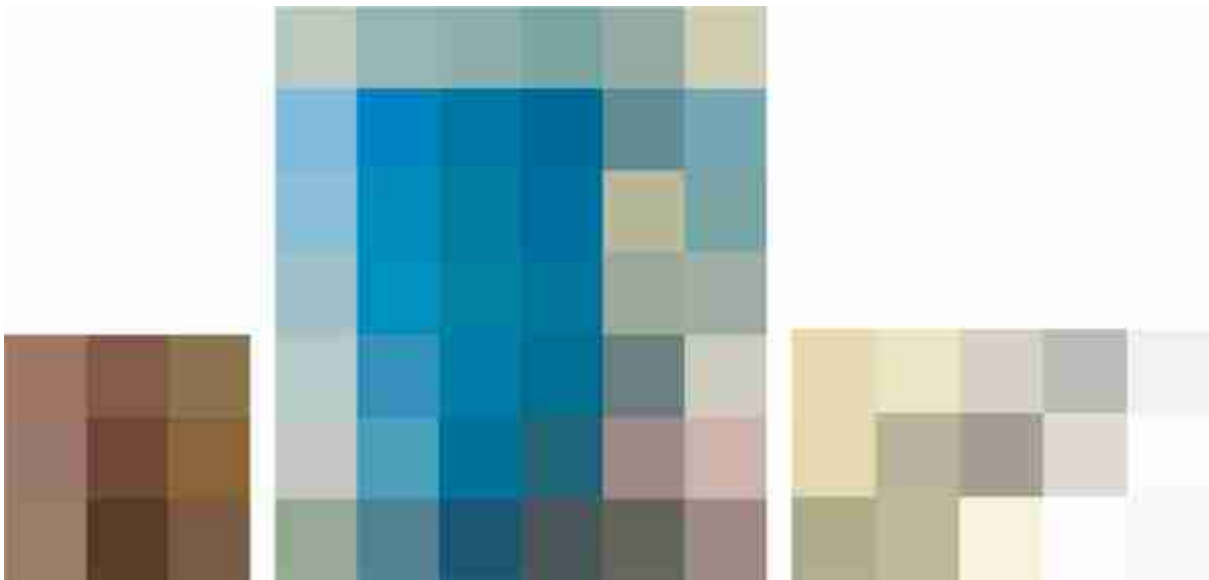


Figure 3.2: Small image patches from [figure 3.1](#) in isolation. It is difficult to recognize image patches outside their natural context.

What is the minimum number of pixels needed to form a recognizable image? The answer depends on the image. It is possible to create visually recognizable images with very few pixels [\[472\]](#).

3.3 The More You Look, the More You See

Visual perception cannot be formulated as a simple input-output function over predefined domains. Vision is a dynamical system, even when looking at a static image. The longer you look at an image, the more details you see and the better you understand the scene. Just look at the image shown in [figure 3.3](#) and try to write down everything you see down to the smallest detail.



Figure 3.3: A street in Paris. Is everything fine in this picture?

But this feeling of unlimited and effortless understanding of a picture is also an illusion. Do you truly understand everything you see? For instance, are you sure you can make sense of all the chair legs in [figure 3.3](#)? In fact, this image has been manipulated so that some of legs do not correspond to any chair.

When solving a task, as the **visual cognitive load** increases we are more likely to make judgment mistakes.

As you look around the image, notice that each patch you look at is actually a small photo in its own right. In fact, a big image like this contains thousands of tiny images within it ([figure 3.4](#)). A single photo can be *big data*; if you chop up this photo you get a big dataset of tiny images. A common trick in computer vision is to take a method that was developed for processing a dataset and instead apply it to the set of patches in an image, or vice versa.



[Figure 3.4](#): Each picture contains lots of images.

3.4 The Eye of the Artist

Learning to paint is a great way of learning to see. You start from a white piece of canvas and a finite set of acrylic paints, and your goal is to paint an object that should look real, with shadows, 3D shape, and reflections. What mixtures of paint and what strokes can accomplish that? Artists learn to see the world not by what it represents, but as why it looks the way it does.

The following sequence ([figure 3.5](#)), painted by Agata Lapedriza, shows different steps in the painting process of a strawberry. Layer after layer, the artist replicates the light that will fool your eye into seeing a strawberry instead of acrylic paint. It is interesting to see that the realism does not increase monotonically as the painting progresses. For instance, from step 3 to 4, the right side of the strawberry has lost realism. However, those new strokes will become the shading that will make the strawberry pop out from the canvas.

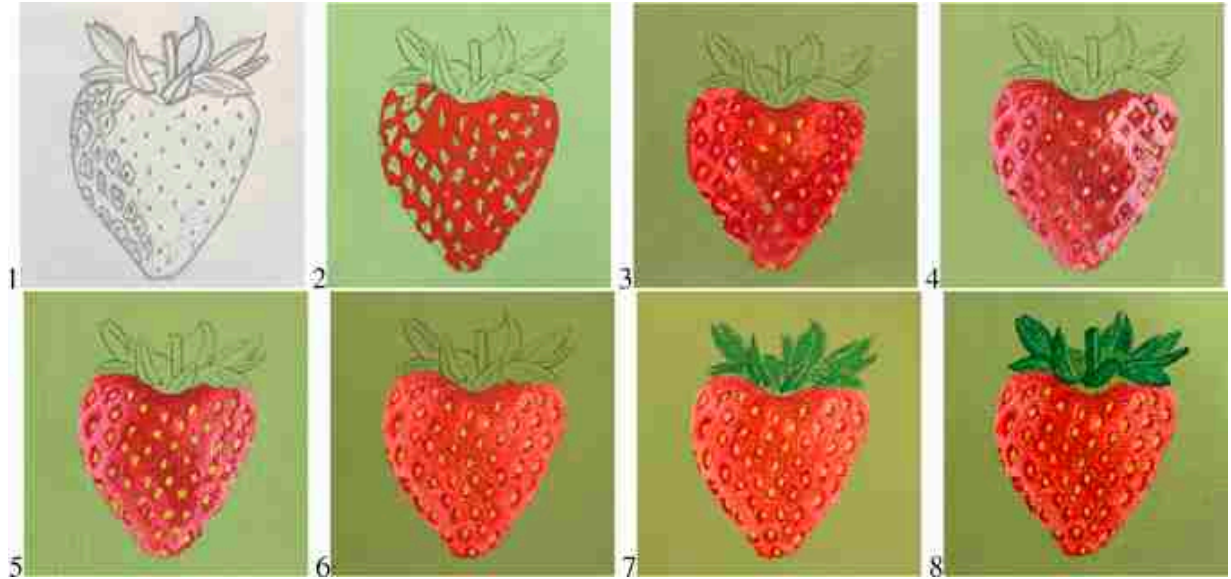


Figure 3.5: The following sequence shows different steps in the process of painting a strawberry.
Source: Original images by Agata Lapedriza.

3.5 Tree Shadows and Image Formation

When walking under the shadow of a tree on a bright sunny day when the sun is right above us ([figure 3.6](#)[left]), we can see how some light rays get through the holes left between leaves, creating spots on the floor ([figure 3.6](#)[right]).



Figure 3.6: The shadow of this tree on a normal day projects dozens of pictures of the sun on the ground.

But something is a bit odd about the shape of the bright spots projected on the floor. We would expect that the spots of light would have irregular shapes as the openings between leaves will have all kinds of irregular profiles ([figure 3.6](#)[left]). Instead, as shown in [figure 3.6](#)(right), the spots appear to be circular and of similar size. This observation already puzzled Aristotle. What could be happening? As a hint, [figure 3.7](#) shows the shadow of leaves during a solar eclipse.



Figure 3.7: During an eclipse, the shadow of the tree contains many copies of the crescent sun.

One of the most common situations that we often encounter is the **pinhole cameras** formed by the spacing between the leaves of a tree. The tiny holes between the leaves of a tree create a multitude of pinholes. The pinholes created by the leaves project different copies of the sun on the floor. When the openings between leaves are small enough, the shape of the projected spot light is dominated by the shape of the sun. This is something we see often but rarely think about the origin of the spotlights that we see on the floor.

In fact, the leaves of a tree create pinholes that produce images in many other situations. We will talk about pinhole cameras in chapter 5.

3.6 Horizontal or Vertical

Looking at the previous picture of the shadow of the tree, another interesting question comes to mind: How do you know you are looking at a picture of the ground? How do you know the orientation of the camera?

Look at the two drawings in [figure 3.8](#). The lines in the drawing on the left correspond to tiles in a floor, the line drawing on the right is the same drawing rotated 90 degrees. Which one seems to be an horizontal surface and which one seems vertical? Why? The differences in the line orientations are subtle and yet it produces a very different perception.

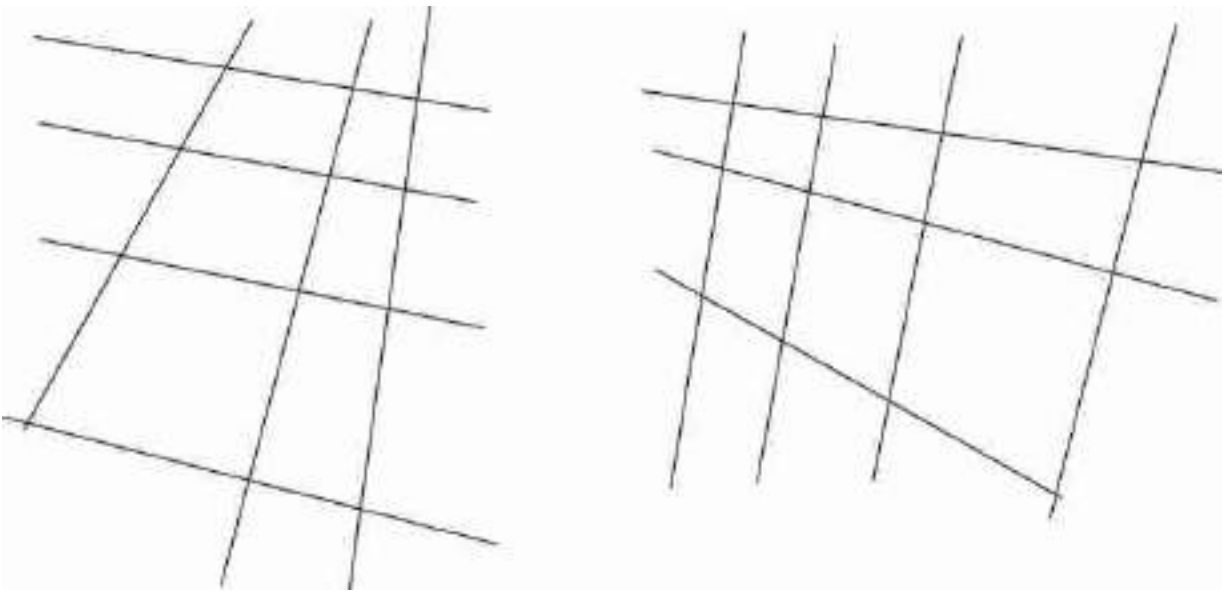


Figure 3.8: Line drawings. The one on the left seems to correspond to an horizontal surface while the one on the right looks like a vertical surface.

In the arrangement in the left, there are two **vanishing points** (the points in which parallel lines intersect) and both are above the drawing, compatible with the horizon line. However, on the drawing on the right, one of the vanishing points is below the figure and gives the impression that the camera is looking at a vertical wall (the camera seems to be higher than the visible part of the wall). The horizon line of the plane (the line that connects both vanishing points) is horizontal in the left drawing and vertical on the right one.

3.7 Motion Blur

When we are in a moving car and we take a picture from the passenger window, often images will appear blurring, specially when taken at night ([figure 3.9](#)). Blur is due to the fact that the camera sensor is collecting light during a period of time while the car is moving and the picture that results is the average of several translated copies of the same image. However, note that not all of the objects are equally blurry. Things that are far away might still look sharp while things nearby appear very blurry.

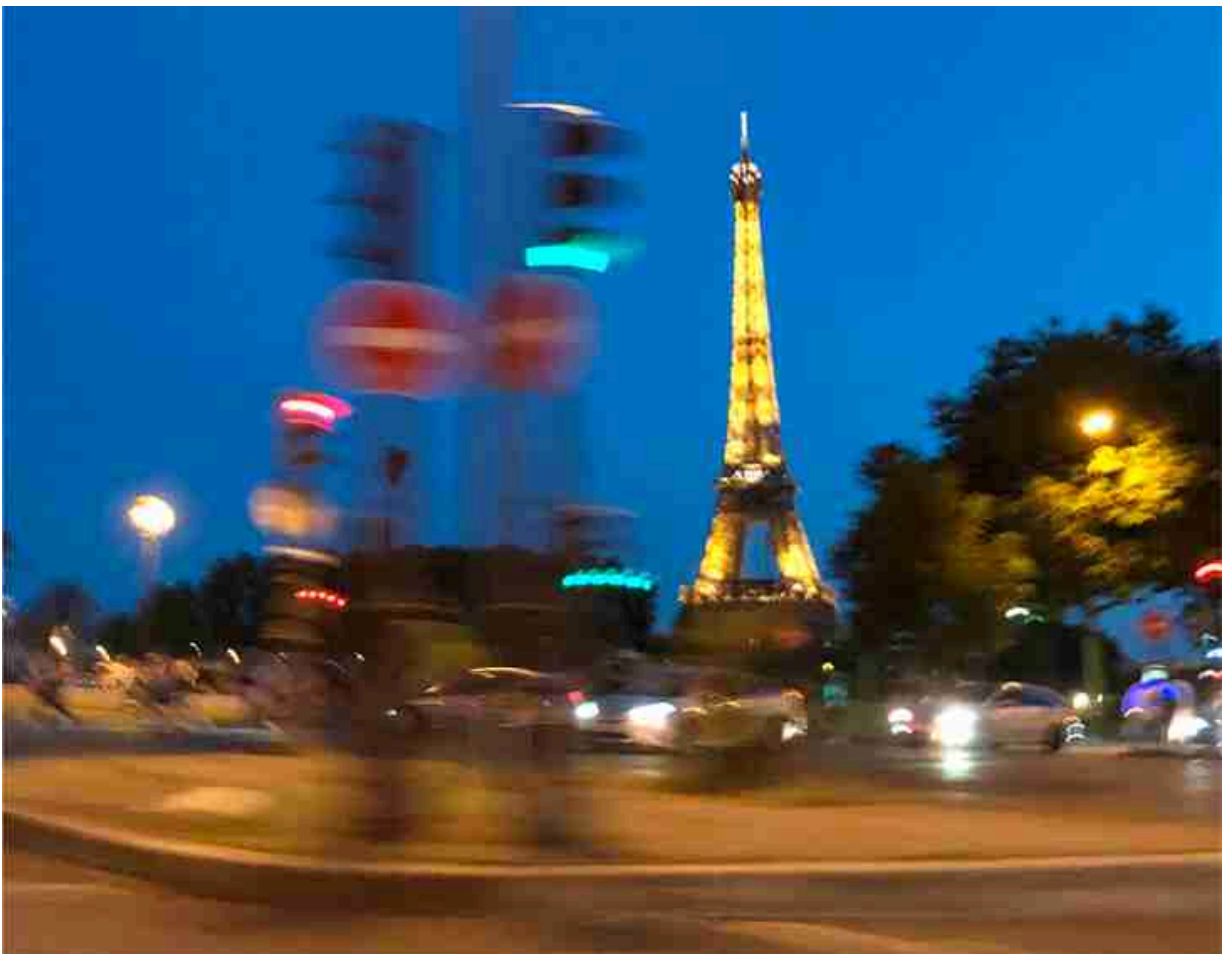
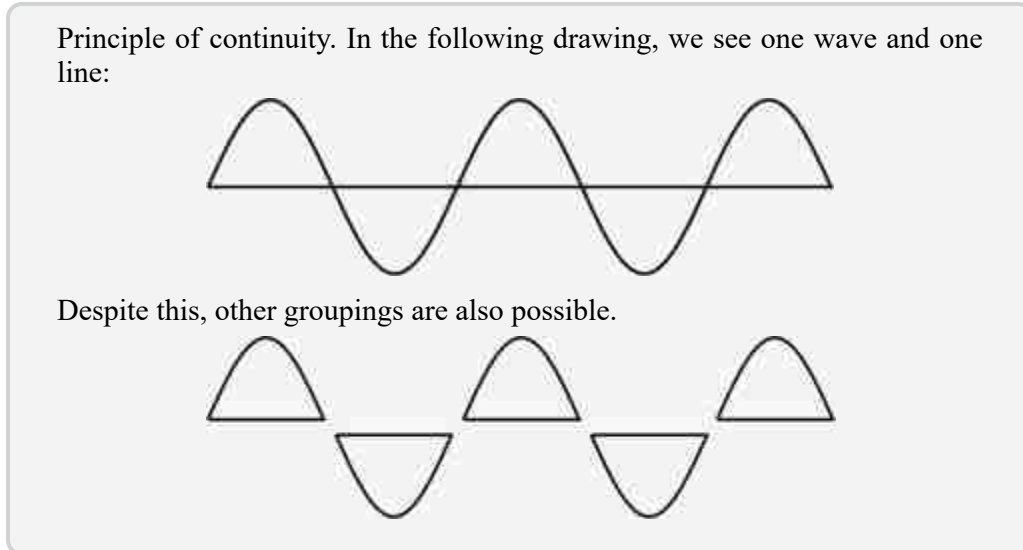


Figure 3.9: Picture of Paris taken from a moving car.

The amount of blur is related to the car speed, the exposure time, and the distance between the camera and each object in the scene. We will talk

more about how camera motion relates to distance in chapter 47.

3.8 Accidents Happen



We learned about the generic view principle in chapter 2, and this idea comes up a lot in computer vision. But it turns out that in most photographs it's easy to find accidents. This is because photos contain so many things, thousands of tiny images, and there are so many coincidences that could occur. [Figure 3.10](#) shows a famous photo and we will look for some coincidences in it. The coincidences are of a specific variety: generically, if you see smooth, continuous line segments you assume that they lie on a single object because it's quite a coincidence for two objects to line up just so their contours perfectly align. The gestalt psychologists called this the **principle of continuity**. The insets for [figure 3.10](#) show several locations in the photo “Migrant Mother” by Dorothea Lange where exactly this happens. Can you find more?



Figure 3.10: Source: Migrant Mother (1936), by Dorothea Lange. On the right are four examples of coincidences that violate the Gestalt principle of continuity. From top-left clockwise: finger boundary aligns with lip contour, cloth aligns with eye, creases align on two different shirts, sleeve boundary is tangent to line on a different shirt.

You have to be careful about this when developing vision algorithms. If you assume *no* accidents will happen, then you will be in for some trouble. This is one reason why older, rule-based methods failed, because they did not handle exceptions to the rules.

3.9 Cues for Support

When can we tell by looking that one object supports another? Consider the ball over a surface depicted in [figure 3.11](#). The drop shadow provides a strong cue that the ball is above the surface in [figures 3.11\(a\)](#) and 3.11(c), while the slack string provides evidence for the ball being on the surface in [figures 3.11\(b\)](#) and 3.11(c). Only [figure 3.11\(c\)](#) is a doctored photograph.

In which of these figures does the ball appear to be in contact with the ground plane?

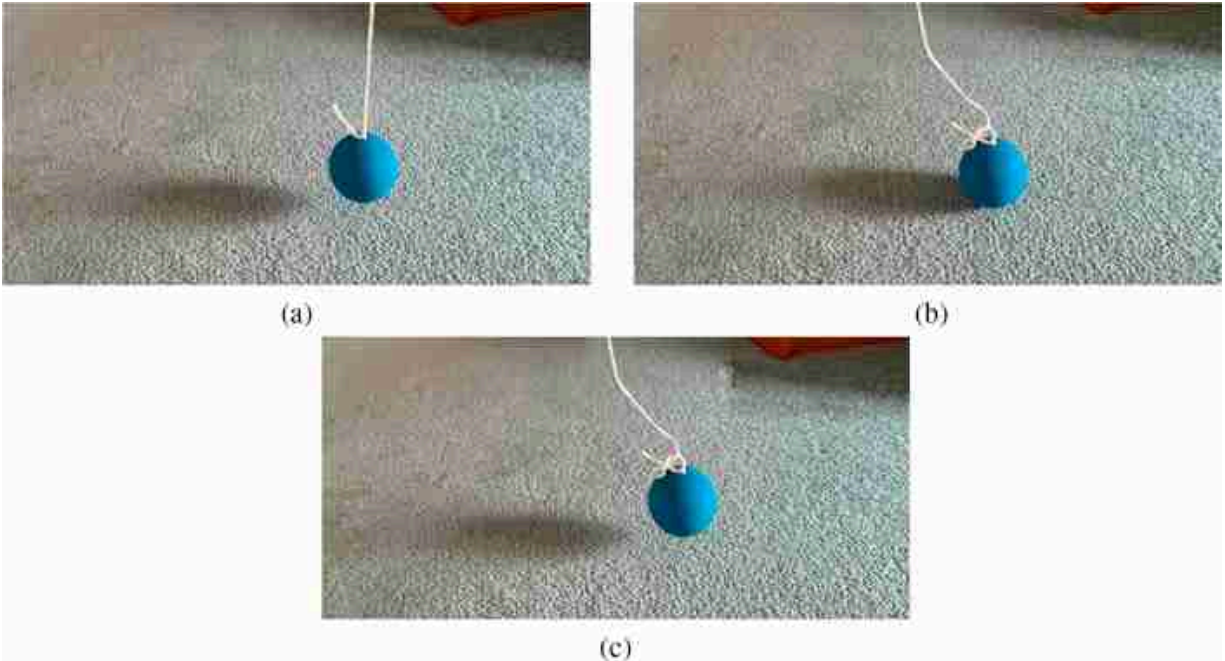


Figure 3.11: Cues for support. The relative location between an object, and its shadow on the ground is a powerful cue for contact.

3.10 Looking at Raindrops

When it rains you get to see a rare sight. Every raindrop refracts a panorama of the scene around it ([figure 3.12](#)). What you are seeing is not just one photo but hundreds of images of the scene, one in each raindrop, all from slightly different angles. It turns out specially designed cameras have been made that do the same thing, and they are called **light field cameras**. These cameras use an array of tiny lenses rather than raindrops, but the result is similar: they capture what the scene looks like from hundreds of different viewing angles, and from that density of information they can achieve many interesting tricks, such as triangulating the depth of objects in the scene and synthetically refocusing the image after it has been taken.



Figure 3.12: A naturally occurring light field camera. This picture of raindrops on a window contains hundreds of tiny images of the scene, one in each raindrop. The images to the right are individual zoomed in raindrops from the photo on the left.

3.11 Plato's Cave

In the *Allegory of the Cave*, Plato imagined a group of prisoners chained up in a cave whose only experience of the outside world is through shadows cast on the cave's wall. When we look at images, we think we are directly seeing reality, but actually the situation is much more like Plato's cave. An image is a crude projection of the underlying physical state. It collapses all the 3D geometry into a flat 2D grid of pixels. From any given viewing angle, most of the scene is occluded. Materials are only depicted in terms of how photons bounce off their surfaces; what's below the surface may be completely omitted. When you look at a picture, think about all the things you cannot see. There's more you don't see than you do. One job of a vision system is to infer everything you can't see given everything you can.

The allegory of the cave has several layers of indirection: the shadows are cast by puppets that are controlled by puppeteers who act out interpretations of the broader reality outside.

3.12 How Do You Know Something Is Wet?

Our visual system allows us to recognize not just objects, but also material. We can tell if something will feel hot or cold, soft or hard, smooth or rugged. We can also recognize if something is wet or not without touching it, as shown in [figure 3.13](#).



Figure 3.13: Why does wet sand look dark? The answer to this question is interesting, but not as much as the question itself.

Why does sand look darker when it's wet? Does everything wet look darker than when it is dry? How can we tell which parts of the sand are dark because they are wet and which ones are dark because they are under a shadow?

3.13 Concluding Remarks

This chapter was about learning to ask questions about what we see, why things look the way they do, and how do we recognize them. There are so many things that we see every day that we recognize by the way they look, and yet we seldom ask ourselves why they look that way. Why does the sky appear blue? Why does the moon always look the same? Why do hot things become redder? There are many other sources you can consult that explore why things look the way they do in the world. A beautiful book on this topic is *Light and Color in the Outdoors* by Marcel Minnaert [[337](#)], first published in 1937.

After reading this chapter, you will not know how to implement anything new, nor will you have learned new math or new algorithms. The goal of this chapter was to show you some of the questions you should ask about the visual world that you experience every day. The exercise to do now is to go around with your camera and take pictures of interesting visual phenomena.

Your brain is like a detective looking at all the clues that reveal what the world is made of.

4 Computer Vision and Society

4.1 Introduction

The success of computer vision has led to vision algorithms playing an ever-larger role in society. Face recognition algorithms unlock our phones and find our friends and family members in photographs; cameras identify swimmers in trouble in swimming pools and assist people in driving cars.

With such a large role in society, one may expect that computer vision mistakes can have a large impact and have the potential to cause harm, and unfortunately this is true. While self-driving cars may save tens of thousands of lives each year, mistakes in those algorithms can also lead to human deaths. A mistaken computer match with a surveillance photograph may implicate the wrong person in a crime. Furthermore, the performance of computer vision algorithms can vary from person to person, sometimes as a function of protected attributes of the person, such as gender, race, or age. These are potential outcomes that the computer vision community needs to be aware of and to mitigate.

The field of **Algorithmic Fairness** studies the performance and fairness of algorithms across people, groups, and cultures. It seeks to address algorithmic biases, and to develop strategies to ensure that computer vision algorithms don't reflect or create biases against different groups of people or cultures. Entry points to this literature include a number of text and video sources, including [[254](#), [155](#), [181](#), [154](#), [228](#), [112](#), [36](#), [184](#)].

We review two topics: fairness and ethics. In section 4.2, we describe some current techniques. This is just a small sampling of work in this quickly evolving area, selected to intersect well with the material in the rest of this book. There is more literature and more subtleties than can be covered here. Even if our algorithms and datasets were completely free of bias, there are still many ethical concerns regarding the use of computer vision technology. In section 4.3, we pose ethics questions that we encourage computer vision researchers to think about.

4.2 Fairness

Modern computer vision algorithms are trained from data, and thus are influenced by the statistical make-up of the training data. Correlations between the expressed gender of individuals and their depicted activities recorded within a dataset can perpetuate or even amplify societal gender biases captured in the dataset [527, 92, 356]. Gender classification systems in 2018 showed performance that depended on skin tone [65].

There is a need for datasets, and the algorithms trained from them, to minimize spurious relations between protected attributes of people, such as skin color, age, gender, and the image labels or recognition outputs, such as occupation or other qualities that should not depend on the protected attributes. Researchers have begun to develop methods to identify and mitigate such biases in either computer vision databases or algorithms.

4.2.1 Facial Analysis

Facial detection asks whether there is a face in some image region, while facial analysis refers to measuring additional attributes of the face, including pose and identity. It is important to characterize the performance of facial analysis systems across demographic groups, as certain studies have done [265]. A large Face Recognition Vendor Test by the National Institute of Standards and Technology (NIST) [176] examined the accuracy of face recognition algorithms for different demographic groups defined by sex, age, and race or country of birth. For one dataset, they found false positive rates were highest in West and East African and East Asian people, and lowest in Eastern European individuals. For the systems offered by many vendors, this effect was large, with differences of up to a factor of 100 in the recognition false positive rates between different countries of birth. In addition, the more accurate face recognition systems showed less bias and some developers supplied highly accurate identification algorithms for which false positive differentials were undetectable. Certain studies [176] have stressed the importance of reporting both false negative and false positive rates for each demographic group at that threshold. Others [72] have provided a checklist for measuring bias in face recognition algorithms. It is clearly important to understand the biases of any facial analysis system that is put into use, as these characteristics can vary greatly from system to system.

4.2.2 Dataset Biases

One cause of algorithmic bias can be a bias in the contents of the training dataset. There are many subtleties in the creation and labeling of computer vision datasets [395]. Any dataset represents only one slice of the possible images and labels that one could capture from the world [471]. Furthermore, the samples taken may well be influenced by the background of the researchers gathering the dataset. Figure 4.1 shows examples of common objects from four different households across the globe. From left to right, the photographs are shown in decreasing order of the average household income of the region from which the photo was taken. The object recognition results for six different commercially available image recognition systems are listed below each photograph, showing that the performance is much better for the images from first-world households [102], possibly due to a better representation of imagery from such households in the training datasets of the image recognition systems.



Figure 4.1: Images of household items, and their recognized classes by five object recognition systems [102]. The systems tend to perform worse for non-Western countries and for lower-income households, such as those of the right two photographs.

4.2.3 Generative Adversarial Networks to Create Unbiased Datasets and Algorithms

One way to mitigate a biased dataset is to synthetically produce the images needed to provide the necessary balance or coverage. Generative adversarial networks (GANs), described in chapter 32, can operate on a dataset to generate sets of images similar to those of the dataset, but

differing from each other in controlled ways, for example, with changes in the apparent gender, age, or race of depicted people. Such GANs have been used to develop debiased datasets and algorithms.

Sattigeri et al. [422] proposed an algorithm they called Fairness GAN, which included a classifier trained to perform *as poorly as possible* on predicting the classification result based on a protected attribute. For example, the algorithm was designed to predict an attractiveness label while gaining no benefit from knowing the gender or skin tone. The resulting debiased dataset showed some improvement in fairness metrics.

Another study [396] took an alternative approach to the problem of removing correlations between a protected attribute and a target label. The authors used GANs to generate pairs of realistic looking images that were balanced with respect to each protected attribute. Figure 4.2 shows an example of this. If “wearing hat” is deemed to be the protected attribute, it is desired to remove its correlation in the data with the attribute, “wearing glasses.” If no debiasing steps were taken, the algorithm would learn to associate wearing hats with wearing glasses. The GAN is used to augment the dataset with paired examples of images of people wearing hats both with and without glasses. When this augmented dataset is combined with the original dataset, performance in a variety of fairness measures is improved.

Still another approach to dataset fairness, by [527], injects constraints to require that the model predictions follow the distributions observed within the training data, assuming that the training data has the desired distribution. See also [495] for a benchmark and comparison of techniques for dataset bias mitigation.

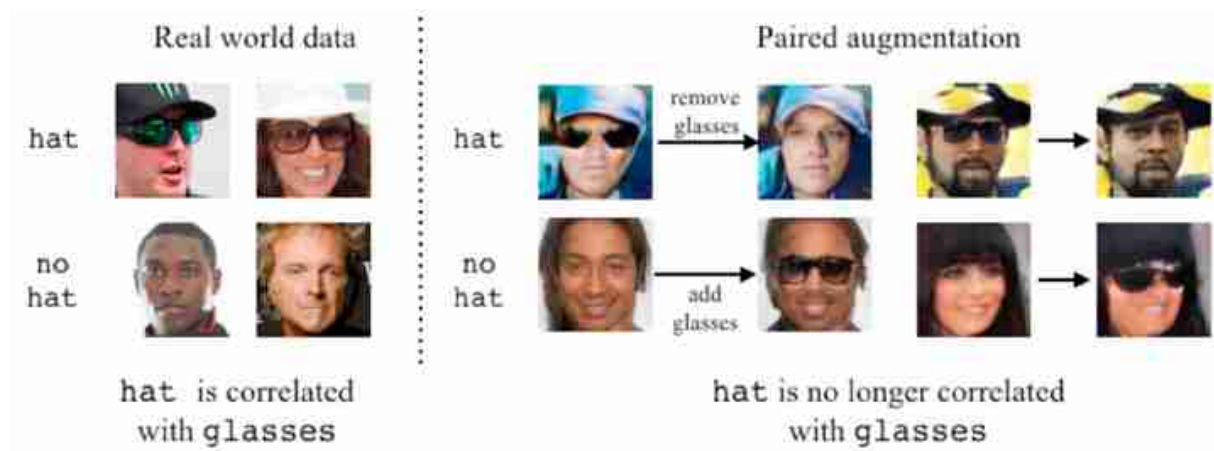


Figure 4.2: In this example “wears hat” is deemed to be a protected attribute, but it is correlated with another attribute, which in this example is “wears glasses” [396]. The GAN-based method generates sets of images where wearing hats is not correlated with wearing glasses.

4.2.4 Counterfactuals for Analyzing Algorithmic Biases

It may be difficult to distinguish whether a given biased result is caused by algorithmic bias or by biases in the testing dataset [30]. One way to disentangle those is through experimentation, that is, modifying variables of a probe image and examining the algorithm's decision. GAN models, when coupled with human labeling to learn directions in the latent space corresponding to changes in the desired attributes, allow for such experimental intervention. Researchers [30] have shown that such counterfactual studies may lead to very different conclusions than observational studies alone, which can be influenced by biased correlations in the testing dataset. (See also related work in counterfactual reasoning by [100].) Figure 4.3 shows several **transects** from [30], paths through the GAN's latent space where only one face variable is modified. The variation in an algorithm's classification results along the images of the transect to provide a clean assessment of any bias related to the variable being modified.

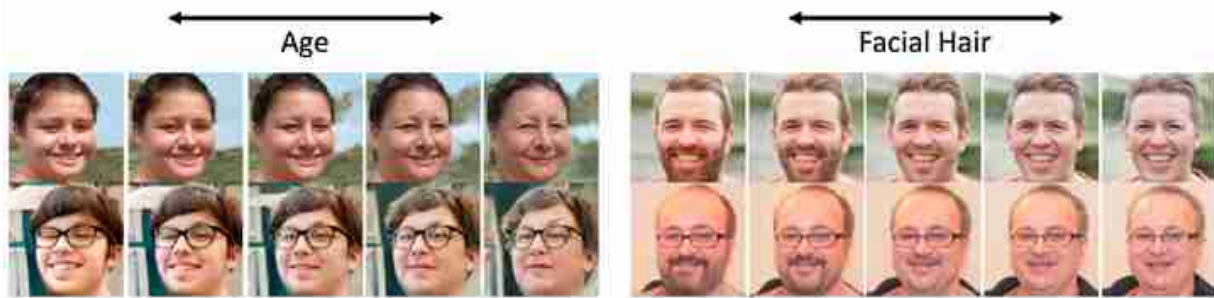


Figure 4.3: A GAN creates sequences of faces, called **transects**, where only one attribute changes [30].

4.2.5 Privacy and Differential Privacy

Since progress in computer vision often comes via new datasets, which typically show images and activities of people, issues of privacy are very important in computer vision research. There are many ways in which people can be harmed by either intentional or unintentional participation in a dataset. Datasets of medical results, financial transactions, views of public places, all can contain information that must remain private. Conversely, there is a benefit to society of releasing datasets for researchers to study: associations can be found that will benefit public health or safety. Consequently, subjects want researchers to work with anonymized versions of their datasets. Unfortunately, a number of well-known examples have shown that seemingly cleaned data, when combined with another apparently innocuous dataset, can reveal information that was desired to be private. For example, in 1997, the medical records of the governor of Massachusetts were identified by matching anonymized medical data with publicly available voter registration records [254, 111]. The combination of the two datasets allowed seemingly de-identified private data to be re-identified.

A very successful theoretical privacy framework, called differential privacy, addresses these issues. Following the techniques of differential privacy [111] researchers can guarantee to the participants of a study that they will not be affected by allowing their data to be used in a study or analysis, no matter what other studies, datasets, or information sources are available.

Differential privacy allows extracting aggregated information about a population from a database without revealing information about any single individual.

Algorithms can be designed for which it can be shown that the guarantee of differential privacy is met [254, 111]. One approach is to inject enough randomness into each recorded observation to guarantee that no individual's data can be reconstructed (with probabilistic guarantees that can be made as stringent as desired, at the cost of data efficiency), while still allowing scientific conclusions to be formed by averaging over the many data samples. A simple example of this approach, described in [254], is the following procedure to query a set of adults to determine what fraction of them have cheated on their partner. Each surveyed adult is instructed to flip a coin. If the coin shows heads, they are instructed to answer the question, “Have you cheated on your partner?”, truthfully. If the coin shows tails, they are asked to answer “yes” or “no” at random, by flipping the coin again and answering “yes” if heads and “no” if tails. The resulting responses for each individual are stored.

If the stored data were hacked, or accidentally leaked, no harm would come to the surveyed individuals even though their data from this sensitive survey had been revealed. Any “yes” answer could very plausibly be explained as being a result of the subject having been instructed by the protocol to flip a coin to select the answer. Yet it is still possible to infer the true percentage of “yes” answers in the surveyed population from the stored data: the expected value of the measured percentages will be the true “yes” and “no” answers in the population. The price for this differential privacy is that more data must be collected to obtain the same accuracy guarantees in the estimate of the true population averages.

There are also risks associated with the wrong use of differential privacy [113].

4.3 Ethics

Many ethical issues are outside of the traditional training and education of scientists or engineers, but ethics are important for vision scientists to think through and grapple with. Scientists have a distinguished history of engaging with ethical and moral issues [229, 367, 254, 408].

4.3.1 Concerns beyond Algorithmic Bias

Suppose the research community developed algorithms that could recognize and analyze people or objects equally well, regardless of displayed gender, race, age, culture or other class attributes. Is the community's work toward fairness completed? Unfortunately, there are still many issues of concern, as pointed out by many authors and speakers [[155](#), [156](#), [44](#)]. To list a few of these issues:

- Face analysis for job hiring. Companies have used automated facial analysis methods (analyzing expressions and length of responses) as part of their proprietary algorithms for resume screening, although, at least for some cases, that process has stopped after criticism of the fairness of these methods for screening [[243](#)].
- Automated identification of faces can be used to compile a list of the participants at a public protest, potentially allowing retribution for attendance [[154](#), [341](#)].
- People can show a bias toward believing the output of a machine [[87](#)], making it more difficult for a person to correct an algorithm's mistake.
- There are concerns whether labeling the gender of a person from their appearance could cause distress or harm to some in the LGBTQ community [[181](#), [45](#)].

We provide a subsequent list of questions for thought and discussion. We encourage computer vision students and practitioners to engage with these questions, and, of course, to continue to ask their own questions as well. We need to work both as technologists and as citizens to ensure that computer vision technologies are used responsibly.

4.3.2 Some Ethical Issues in Computer Vision

The following are an (incomplete) set of questions that researchers and engineers may keep in mind:

- Would you prefer that a person identify someone from a photograph, with the potential biases and assumptions that the individual may have, or for an algorithm to do so, based on training from a dataset which may include biases of its own? See [[254](#), [342](#)] for related discussions.
- What privacy safeguards must be in place to accept always-on camera and voice recording inside people's homes? Inside your own home?

- Traffic fatalities in the US currently result in the tragedy of approximately 30,000 deaths annually, with most of those fatalities being caused by driver error. If every vehicle were self-driving, fatalities caused by human error could fall dramatically, yet there will surely still be fatalities caused by machine errors, albeit far fewer than had been caused by humans. Is that a tradeoff society should make? What should be our threshold of fatalities caused by machine? This moral question is a societal-scale version of “the trolley problem” [467]: A bystander can choose to divert a trolley from the main track to a side track, saving five people who are on that main track, but killing one person who is on the side track. Should the bystander actively divert the trolley, intentionally killing the person on the side track, who would otherwise have been spared if no action were taken? These issues are also present for automobile airbags, which save many lives but sometimes injure a small number of people [94], and in other public health issues.
- Algorithms will never perform exactly as well among all groups of people. Within what tolerance must the performance of human analysis algorithms be for an algorithm to be considered fair? Is algorithmic bias being less than human bias sufficient for deployment?
- Some image datasets have contained offensive material [51, 34]. How should we decide which concepts or images can be part of a training set? Is there any need for algorithms to understand offensive words or concepts?
- Could potential categorization tasks of humans in photographs (e.g., gender identity, skincolor, age, height) bring harm to individuals, and how?
- Should computer vision systems ever be used in warfare? In policing? In public surveillance?
- What is the role of machines in society? Regarding decisions of person identification, do we want people making decisions, with their own difficult-to-measure biases, or do we want machines involved, with biases that may be measurable?
- Do face recognition algorithms suppress public protests? Consider a world where any face shown in public is always recorded and identifiable. (We are almost in that world). What are the consequences of that for personal safety, speech, assembly, and liberties?

- What are the most beneficial uses of computer vision?
- Which uses have the most potential for harm, or currently cause the most harm?
- Is it ok to use computer vision to monitor workers in order to improve their productivity? In order to improve workplace safety? In order to prevent workplace harassment?
- What criteria would you use to evaluate the pros and cons of using machines or humans for a given task?
- Should datasets represent real-world disparities, or represent the world as we want it to be? How might these choices amplify existing economic disparities?
- When should consent be required (and from whom) before using an image to train a computer vision algorithm?

4.4 Concluding Remarks

Computer vision, with its expanding roles in society, can both cause harm as well as bring many benefits. It is important that every computer vision researcher be aware of the potentials for harm, as well as learn techniques to mitigate against those possibilities. It is our responsibility to bring this technology into our society with care and understanding.

II

IMAGE FORMATION

This set of chapters describe how images are formed. How can the light illuminating the space around us be captured by a device to form a picture? We will discuss pinhole cameras and lenses, which will let us introduce the geometry and formulation of the image formation process. We will then further develop the geometry of the image formation process in part XI. We will also describe how to interpret cameras as linear systems, which will allow us to understand and design new cameras. Together with computation, this can be used to form images in a wide range of situations, going beyond the pinhole and lens-based cameras. We will finalize by describing how color works.

- **Chapter 5** introduces the pinhole camera and perspective projection.
- **Chapter 6** describes how lenses can be modeled and used to address the limitations of the pinhole camera.
- **Chapter 7** gives a different perspective on how to think about cameras. In this chapter we will describe cameras as linear systems, and we will describe new types of cameras.
- **Chapter 8** will cover the foundations of color perception.

Notation

- **Images:** We will use the ℓ symbol to denote images (i.e., *light*). We will also use the same symbol when referring to light rays and other physical quantities.
- **Matrices and vectors:** We will use bold fonts to denote vectors and matrices. We will use lowercase for vectors and uppercase for matrices.

5 Imaging

5.1 Introduction

Light sources, like the sun or artificial lights, flood our world with light rays. These reflect off surfaces, generating a field of light rays heading in all directions through space. As the light rays reflect from the surfaces, they generally change in some attributes, such as their brightness or their color. It is those changes, after reflection from a surface, that let us interpret what we see. In this chapter, we describe how light interacts with surfaces and how those interactions are recorded with a camera.

5.2 Light Interacting with Surfaces

Visible light is electromagnetic radiation, exhibiting wave effects like diffraction. For many imaging models, however, it is helpful to introduce the abstraction of a **light ray**, describing the light radiation heading in a particular direction from a particular location in space ([figure 5.1](#)). A light ray is specified by its position, direction, and intensity as a function of wavelength and polarization. In this treatment, we ignore diffraction effects.

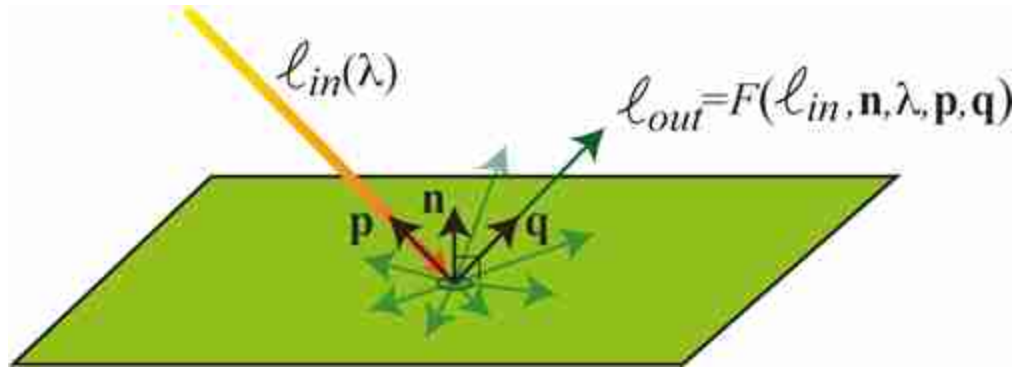


Figure 5.1: A light ray from the sun strikes a surface and generates outgoing rays of intensity and color depending on the angles of the incoming and outgoing rays relative to the surface orientation.

Let an incident light ray be from direction $\mathbf{p}$ and of power, $\ell_{in}(\lambda)$, as a function of spectral wavelength λ ([figure 5.1](#)). The power of the outgoing light, reflected in the direction, $\mathbf{q}$, is determined by what is called the **bidirectional reflection distribution function** (BRDF), F , of the surface. If the surface normal is $\mathbf{n}$, then outgoing power is some function, F , of the surface normal, the incoming and outgoing ray directions, the wavelength, and the incoming light power:

$$\ell_{out} = F(\ell_{in}, \mathbf{n}, \lambda, \mathbf{p}, \mathbf{q}) \quad (5.1)$$

5.2.1 Lambertian Surfaces

In general, BRDFs can be quite complicated (see [\[324\]](#)), describing both diffuse and specular components of reflection. Even surfaces with completely diffuse reflections can give complicated reflectance distributions [\[366\]](#). A useful approximation describing diffuse reflection is the **Lambertian model**, with a particularly simple BRDF, which we denote as F_L . The outgoing ray intensity, ℓ_{out} , is a function only of the surface orientation relative to the incoming and outgoing ray directions, the wavelength, a scalar surface reflectance, and the incoming light power:

$$\ell_{out} = F_L(\ell_{in}(\lambda), \mathbf{n}, \mathbf{p}) = a \ell_{in}(\lambda) (\mathbf{n} \cdot \mathbf{p}), \quad (5.2)$$

where a is the surface **reflectance**, or **albedo**, $\mathbf{n}$ is the surface normal vector, and $\mathbf{p}$ points toward the source of the incident light. Note that the brightness of the outgoing light ray depends on the orientation of the surface relative to the incident ray, as well as the reflectance, a , of the surface. For a Lambertian surface, the intensity of the reflected light is a

function of the direction of the incoming light ray, but not a function of the outgoing direction of the ray, $\mathbf{q}$.

Perfectly Lambertian surfaces are not common. The synthetic material called spectralon is the most perfectly Lambertian material.

5.2.2 Specular Surfaces

A widely used model of surfaces with a specular component of reflection is the **Phong reflection model** [386]. The light reflected from a surface is assumed to have three components that result in the observed reflection: (1) an ambient component, which is a constant term added to all reflections; (2) a diffuse component, which is the Lambertian reflection of equation (5.1); and (3) a specular reflection component. For a given ray direction, $\mathbf{q}$, from the surface, the Phong specular contribution, $\ell_{\text{Phong spec}}$, is:

$$\ell_{\text{Phong spec}} = k_s (\mathbf{r} \cdot \mathbf{q})^\alpha \ell_{\text{in}}, \quad (5.3)$$

where k_s is a constant, α is a parameter governing the spread of the specular reflection, and the unit vector $\mathbf{r}$ denotes the direction of maximum specular reflection, given by

$$\mathbf{r} = 2(\mathbf{p} \cdot \mathbf{n})\mathbf{n} - \mathbf{p} \quad (5.4)$$

[Figure 5.2](#) shows the ambient, Lambertian, and Phong shading components of a sphere under two-source illumination, and a comparison with a real sphere under similar real illumination.

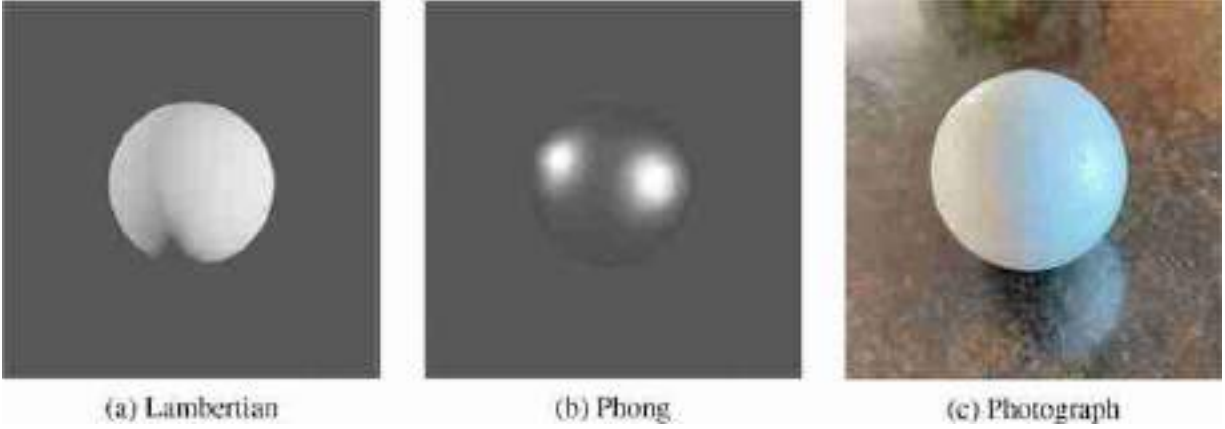


Figure 5.2: (a and b) Two different renderings of a sphere. (c) Note the many extra details required of a photorealistic rendering.

In general, surface reflection behaves linearly: the reflection from the sum of two light sources is the sum of the reflections from the individual sources.

To associate the reflected light with surfaces in the world, we need to know which light rays came from which direction in space. That requires that we form an image, which we discuss next.

5.3 The Pinhole Camera and Image Formation

Naively, we might wonder when looking at a blank wall, why we don't see an image of the scene facing that wall. The light reflected from the wall integrates light from every reflecting surface in the room, so the reflected intensities are an average of light intensities from many different directions and many different sources, as illustrated in [figure 5.3\(a\)](#). Mathematically, integrating the equation for Lambertian reflections (equation [5.2]) over all possible incoming light directions $\mathbf{p}$, we have for the intensity reflecting off a Lambertian surface, ℓ_{out} :

$$\ell_{\text{out}} = \int_{\mathbf{p}} a \ell_{\text{in}}(\mathbf{p}) \cos(\mathbf{n} \cdot \mathbf{p}) d\mathbf{p} \quad (5.5)$$

The intensity, ℓ_{out} , reflecting off a diffuse wall, thus tells us very little about the incoming light intensity $\ell_{\text{in}}(\mathbf{p})$ from any given direction $\mathbf{p}$. To

learn about $\ell_{\text{in}}(\mathbf{p})$, we need to form an image. Forming an image involves identifying which rays came from which directions. The role of a camera is to organize those rays, to convert the cacophony of light rays going everywhere to a set of measurements of intensities coming from different directions in space, and thus from different surfaces in the world.

Perhaps the simplest camera is a **pinhole camera**. A pinhole camera requires a light-tight enclosure, a small hole that lets light pass, and a projection surface where one senses or views the illumination intensity as a function of position. [Figure 5.3\(b\)](#) shows the geometry of a scene, the pinhole, and a projection surface (wall). For any given point on the projection surface, the light that falls there comes from only from one direction, along the straight line joining the surface position and the pinhole. This creates an image of what's in the world on the projection plane.

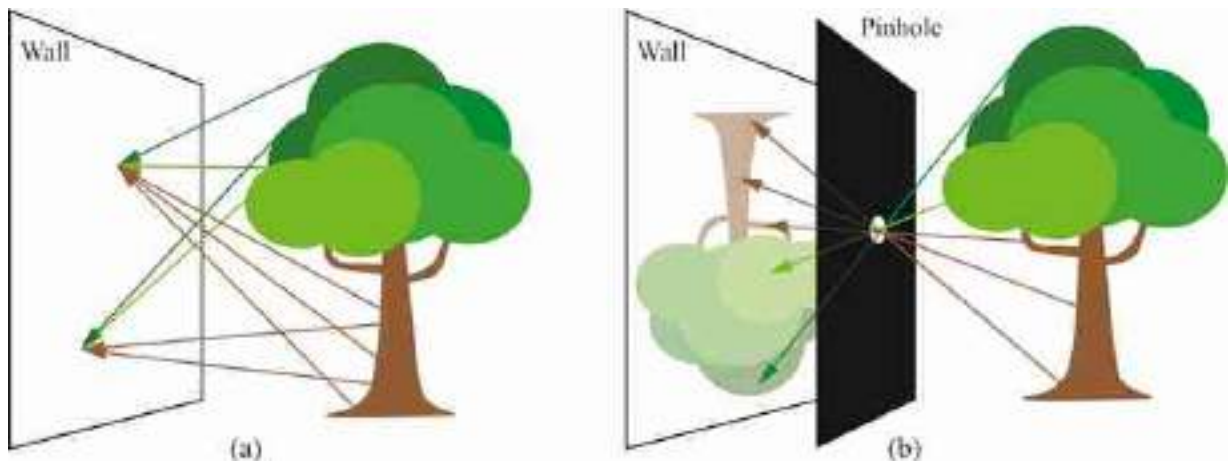


Figure 5.3: (a) Why there are no pictures appearing on the walls? (b) The pinhole camera restricts the light rays reaching the wall, producing an image to appear.

Making pinhole cameras is easy and it is a good exercise to gain an intuition of the image projection process. The picture in [figure 5.4](#) shows a very simple setup similar to the diagram from [figure 5.3\(b\)](#). This setup is formed by two pieces of paper, one with a hole on it. With the right illumination conditions you can see an image projected on the white piece of paper in front of the opening. You can use this very simple setup to see

the effect of changing the distance between the projection plane and the pinhole.



Figure 5.4: A simple setting for creating images on a white piece of paper. In front of the white piece of paper we place another piece of black paper with a hole in the middle. The black paper projects a shadow on the white paper and, in the middle of the shadow, appears a picture of the scene in front of the hole. By making the hole large you will get a brighter, but blurrier image.

[Figure 5.5](#) shows how to make a pinhole camera using a paper bag with a hole in it. One sticks their head inside the bag, which has been padded to be opaque. We encourage readers to make their own pinhole camera designs. The needed elements are an aperture to let light through, mechanisms to block stray light, a projection screen, and some method to view or record the image on the projection screen.

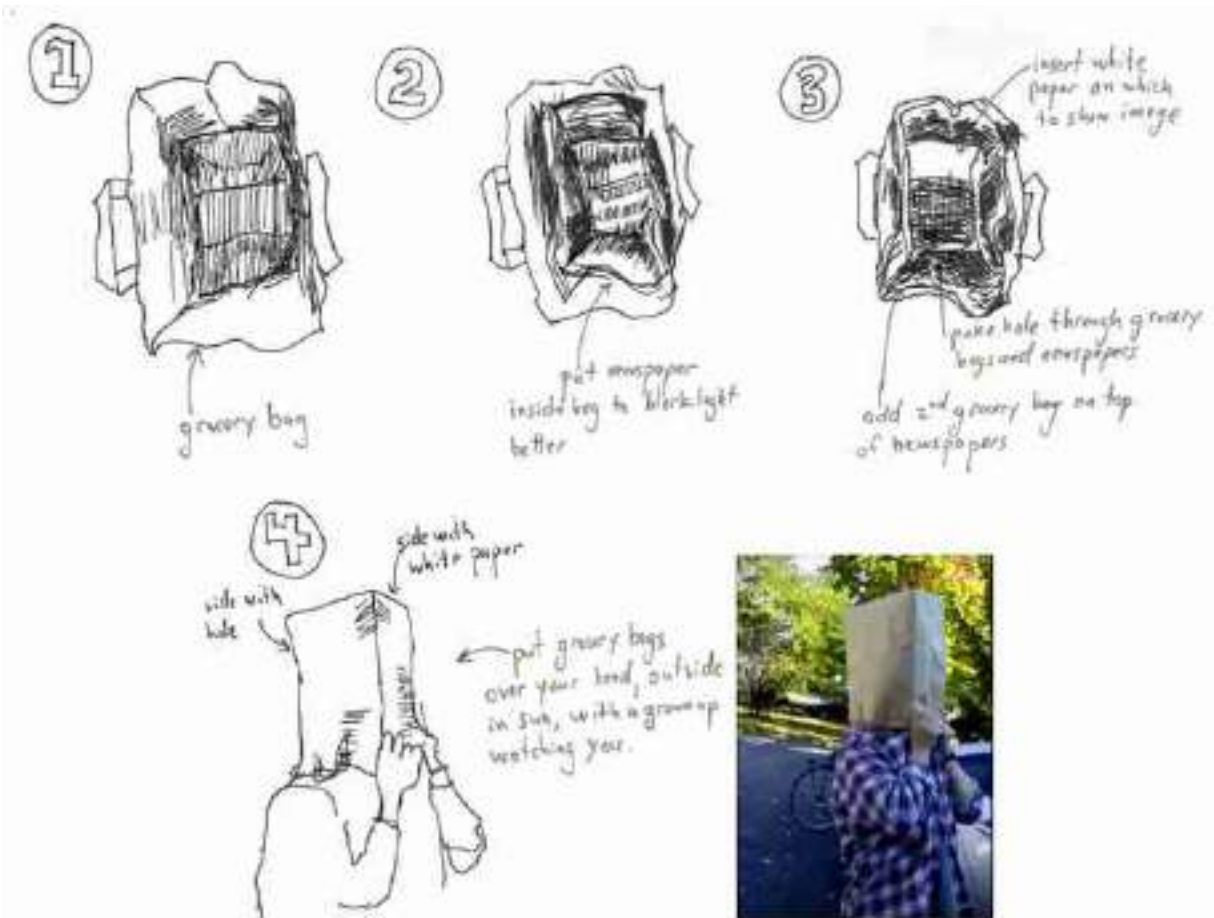


Figure 5.5: A pinhole camera made from paper bags. Following steps 1–4, you can turn a paper bag into a light-tight pinhole camera, with the viewer inside. Newspapers can be added between two layers of paper bags to make a light-tight enclosure. The last picture shows the use of the **paper bag pinhole** camera by one of the authors. Walking with this camera is challenging because you only get to see an upside-down version of what is behind you (adult supervision required).

We started the section asking why there are no pictures on regular walls and explaining that, for an image to appear, we need some way of restricting the rays that hit the wall so that each location only gets rays from different directions. The pinhole camera is one way of forming a picture. But the reality is that, in most settings, light rays are restricted by accidental surfaces present in the space (other walls, ceiling, floor, obstacles, etc.). For instance, in a room, light rays are entering the room via a window, and therefore some of the rays are blocked. In general, the lower part of a wall will see the top of the world outside the window, while the top part of the wall will see the bottom part of the world outside the window. As a

consequence, most walls will appear as having a faint blue tone on the bottom because they reflect the sky, as shown in [figure 5.6\(a\)](#).



Figure 5.6: (a) A wall might contain a picture of the world after all. (b) Turning the room into a pinhole camera by closing most of the window helps to focus the picture that appears in the wall. In this case we can see some buildings that are outside the window.

The world is full of **accidental cameras** that create faint images often ignored by the naive observer.

5.3.1 Image Formation by Perspective Projection

A pinhole camera projects 3D coordinates in the world to 2D positions on the projection plane of the camera through the straight line path of each light ray through the pinhole ([figure 5.3](#)). The simple geometry of the camera lets us identify the projection by inspection. The sketch in [figure 5.7](#) shows a pinhole camera and the relevant terminology.

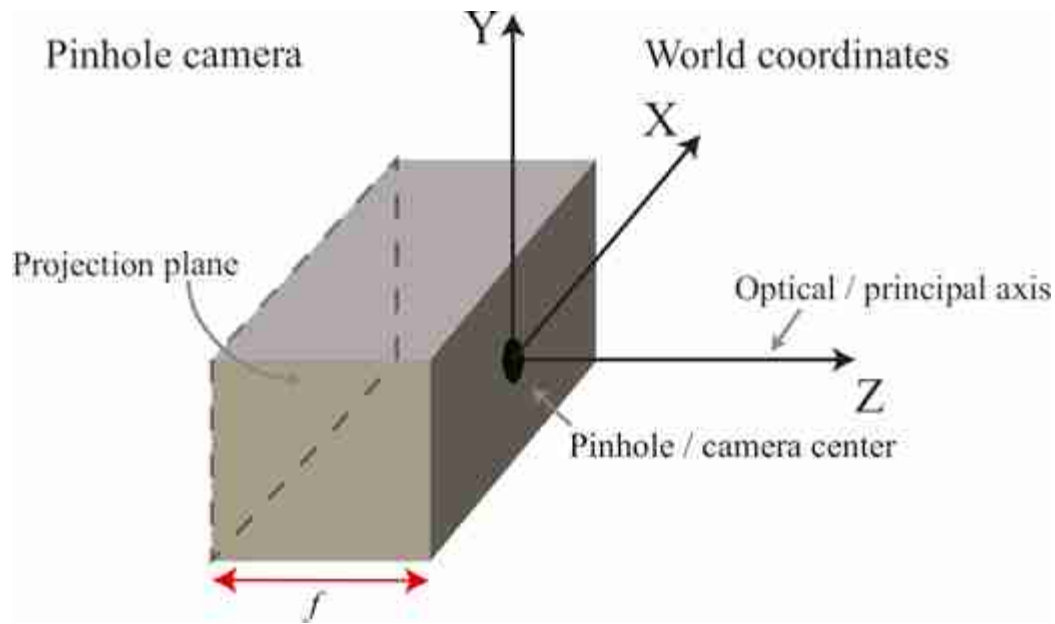


Figure 5.7: Coordinate systems. In computer vision it is common to use the **right-hand rule** for choosing the orientation of the 3D coordinate axes.

[Figure 5.8](#) shows the pinhole camera of [figure 5.7](#) but with the box removed, leaving visible the projection plane. [Figure 5.8](#) shows the definition of the three coordinate systems that we will use:

- **World coordinates.** Let the origin of a World Cartesian coordinate system be the camera's pinhole. The coordinates of 3D position of a point, $\mathbf{P}$, in the world will be $\mathbf{P} = (X, Y, Z)$, where the Z axis is perpendicular to the camera's sensing plane (projection plane).
- **Camera coordinates in the virtual camera plane.** The camera projection plane is behind the pinhole and at a distance f of the origin. Let the coordinates in the camera projection plane, x and y , be parallel to the world coordinate axes X and Y but in opposite directions, respectively. For simplicity, it is useful to create a virtual camera plane that is radially symmetrical to the projection plane with respect to the origin and it is placed in front of the camera ([figure 5.8\[a\]](#)). This virtual camera plane will create a projected image without the inversion of the image. The coordinates in the virtual camera plane will have the same sign as the world coordinates, that is, with the same x, y coordinates (i.e., both images are identical apart from a flip).

- **Image coordinates**, shown in [figure 5.8\(b\)](#), are typically measured in pixels. Both the camera coordinate system (x, y) and the image coordinate system (n, m) are related by an affine transform as we will discuss later.

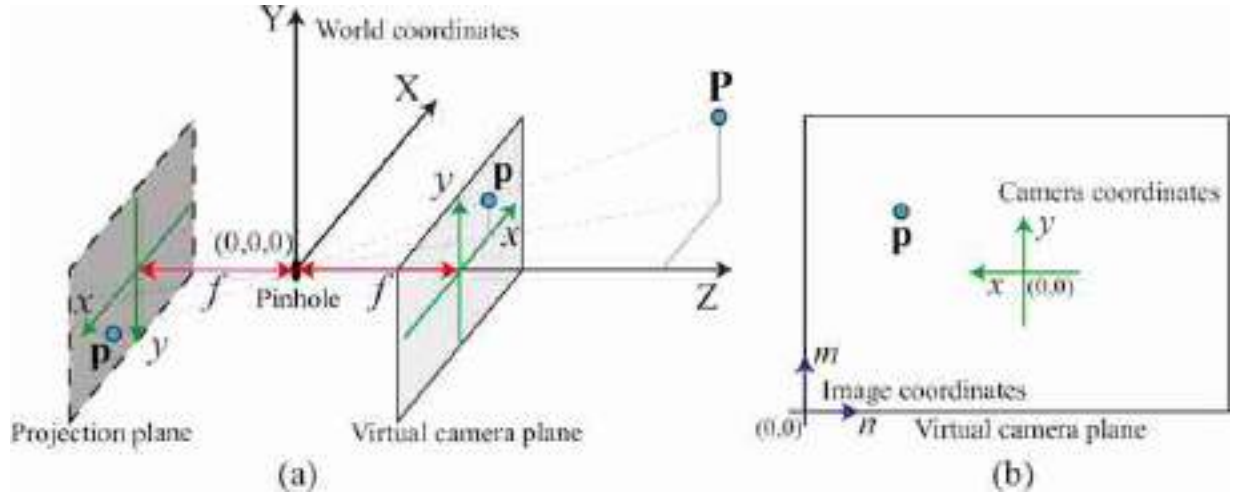


Figure 5.8: (a) Geometry of the pinhole camera. A 3D point P projects into the location p in the projection plane, located at a distance f of the pinhole. The virtual camera plane is a radially symmetric projection of the camera plane. (b) Relation between the camera, (x, y) , and the image coordinate system (n, m) .

If the distance from the sensing plane to the pinhole is f (see [figure 5.9](#)) then similar triangles gives us the following relations:

$$x = f \frac{X}{Z} \quad (5.6)$$

$$y = f \frac{Y}{Z} \quad (5.7)$$

Thales of Miletus, 624 B.C., introduced the notion of **similar triangles**. It seems he used this to measure the height of Egypt pyramids, and the distance to boats in the sea.

Equations (5.6) and (5.7) are called the **perspective projection equations**. Under perspective projection, distant objects become smaller, through the inverse scaling by Z . As we will see, the perspective projection equations apply not just to pinhole cameras but to most lens-based cameras, and human vision as well.

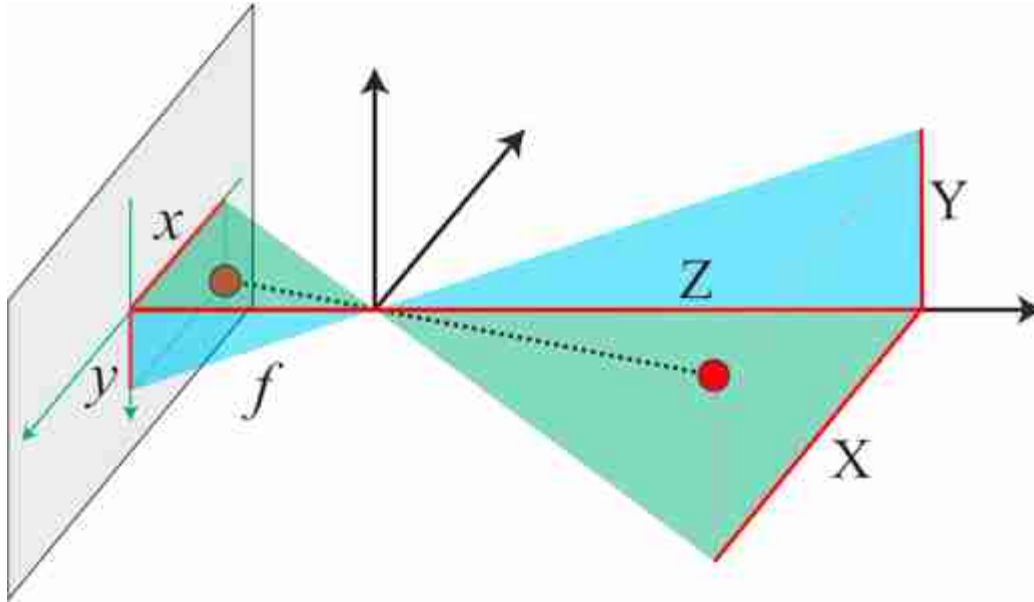


Figure 5.9: Perspective projection equations derived geometrically. From similar triangles, we have $x/f = X/Z$ and $y/f = Y/Z$. Similar triangles are indicated by the same color.

Due to the choice of coordinate systems, the coordinates in the virtual camera plane have the x coordinate in the opposite direction than the way we usually do for image coordinates (m, n) , where m indexes the pixel column and n the pixel row in an image. This is shown in [figure 5.8\(b\)](#). The relationship between camera coordinates and image coordinates is

$$n = -ax + n_0 \quad (5.8)$$

$$m = ay + m_0 \quad (5.9)$$

where a is a constant, and (n_0, m_0) is the image coordinates of the camera optical axis. Note that this is different than what we introduced a simple projection model in figure 2.3 in the framework of the simple vision system in chapter 2. In that example, we placed the world coordinate system in front of the camera, and the origin was not the location of the pinhole camera.

Biological systems rarely have eyes that are built as pinhole cameras. One exception is the Nautilus, which has evolved a pinhole eye (without any lens).

5.3.2 Image Formation by Orthographic Projection

Perspective projection is not the only feasible projection from 3D coordinates of a scene down to the 2D coordinates of the sensor plane. Different camera geometries can lead to other projections. One alternative to perspective projection is **orthographic projection**. [Figure 5.10](#) shows the geometric interpretation of orthographic (or parallel) projection.

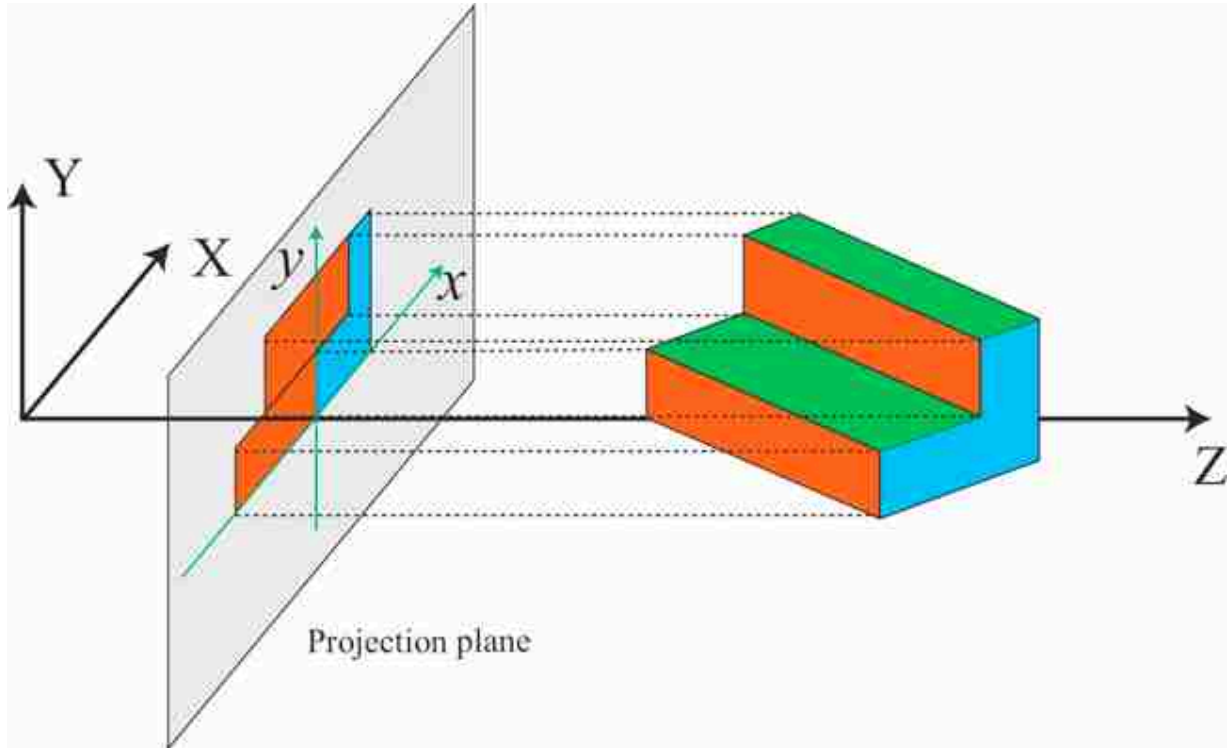


Figure 5.10: Orthographic projection. Projection is done by parallel rays orthogonal to the projection plane. In this example, we have $x = X$ and $y = Y$.

In this projection, as the rays are orthogonal to the projection plane, the size of objects is independent of the distance to the camera. The projection equations are generally written as follows:

$$\begin{aligned}x &= kX \\ y &= kY\end{aligned}\tag{5.10}$$

where the constant scaling factor k accounts for change of units and is a fixed global image scaling.

Orthographic projection is a good model for telephoto lenses, where the apparent size of objects in the image is roughly independent of their distance to the camera. We used this projection when building the simple visual system in chapter 2.

Orthographic Projection is a correct model when looking at an infinitely far away object that is infinitely zoomed in, as discussed in chapter 2.

The next section provides another example of a camera producing an orthographic projection.

5.3.3 How to Build an Orthographic Camera?

Can we build a camera that has an orthographic projection? What we need is some way of restricting the light rays so that only perpendicular rays to the plane illuminate the projection plane.

One such camera is the **soda straw camera**, shown in [figure 5.11](#), which you can easily build. The example shown in (c) used around 500 straws.

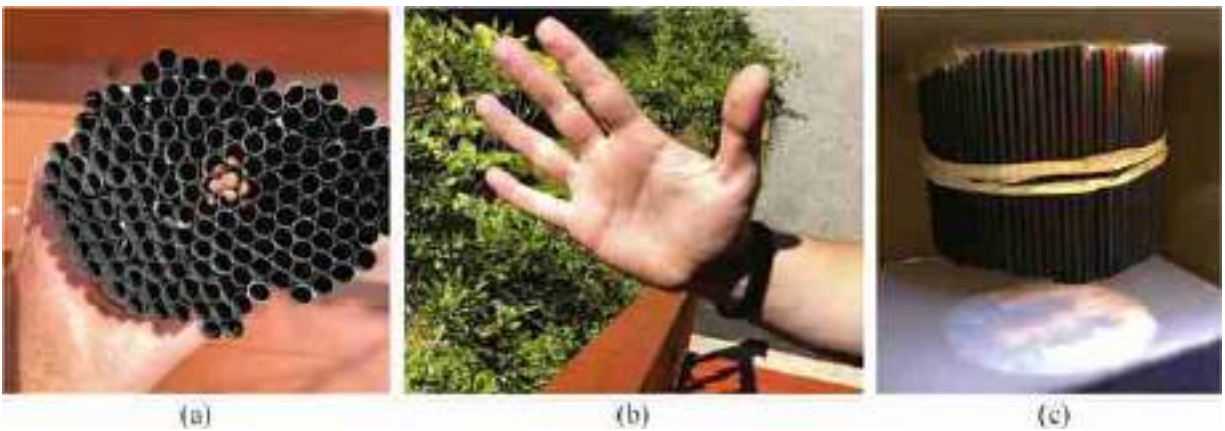


Figure 5.11: Straw camera example. (a) View through parallel straws. (b) The subject is a hand in sunlight. (c) The resulting image of the straw camera (using smaller straws than (a)). The image projection is orthographic.

A set of parallel straws allow parallel light rays to pass from the scene to the projection plane, but extinguish rays passing from all other directions. It works better if the straws are painted black to reduce internal reflections that might capture light rays not parallel to the straws, resulting in a less sharp picture.

[Figure 5.11](#)(c) shows the resulting image when imagining the scene shown in [figure 5.11](#)(b). The straw camera doesn't invert the image as the pinhole camera does and objects do not get smaller as they move away from the camera. The image projection is orthographic, with a unity scale factor; the object sizes on the projection plane are the same as those of the objects in the world.

A larger straw camera has been built by Michael Farrell and Cliff Haynes [\[120\]](#) with an impressive 32,000 drinking straws.

Another implementation of an orthographic camera is the telecentric lens that combines a pinhole with a lens.

5.4 Concluding Remarks

Light reflects off surfaces and scatters in many directions. Pinhole cameras allow only selected rays to pass to a sensing plane, resulting in a perspective projection of the 3D scene onto the sensor. Other camera configurations can give other image projections, such as orthographic projections.

At this point, it is a good exercise to build your own pinhole camera and experiment with it. Building a pinhole camera helps in acquiring a better intuition about the image formation process.

6 Lenses

6.1 Introduction

While pinhole cameras can form good images, they suffer from a serious drawback: the images are very dim because not much light passes through the small pinhole to the sensing plane of the pinhole camera. As shown in [figures 6.1 and 6.2](#), one can try to let in more light by making the pinhole aperture bigger. But that allows light from many different positions to land on a given position at the sensor plane, resulting in a bright but blurry image. Putting a lens in the larger aperture can give the best of both worlds, capturing more light while redirecting the light rays entering the camera so that each sensor plane position maps to just one surface point, creating a focused image on the sensor plane. Here we analyze how light passes through lenses under the approximation of **geometric optics**, where we ignore effects like diffraction, due to the wave nature of light.

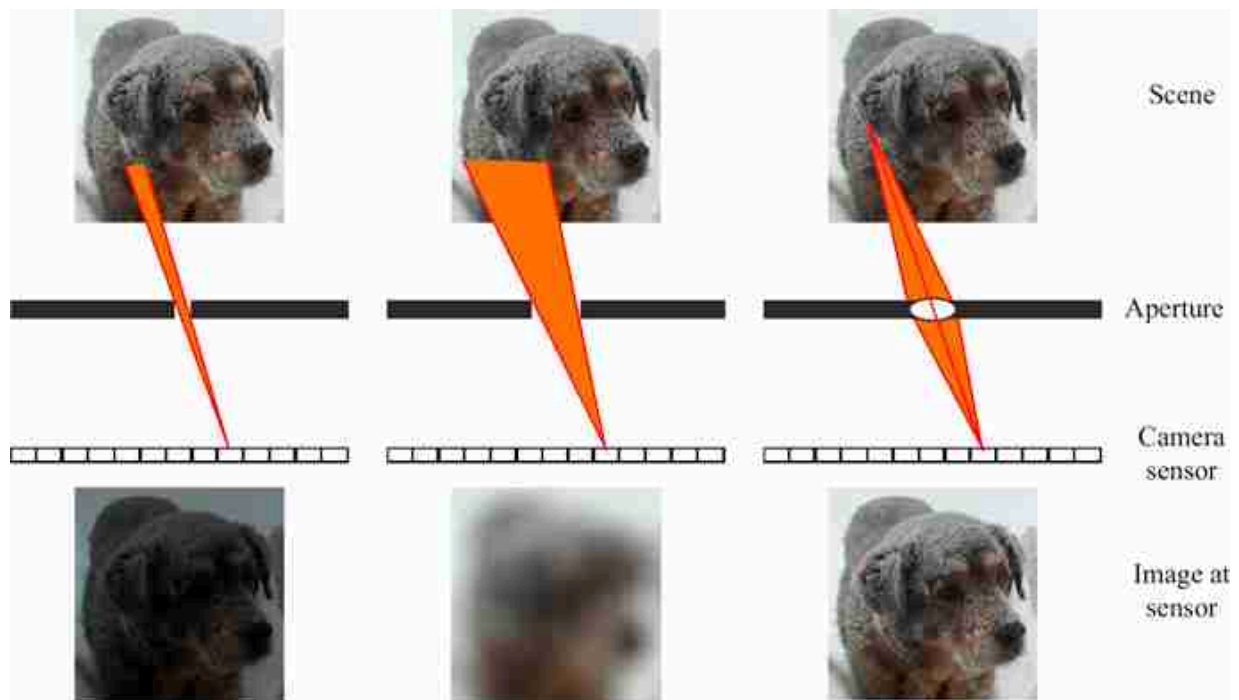


Figure 6.1: Brightness/sharpness trade-offs in image formation. From left to right a small pinhole will create a sharp image, but lets in little light, so the image may appear dark for a given exposure time. A larger pinhole lets in more light and generates a brighter image, but each sensor element records light from many different image positions, creating a blurry image. A lens can collect light reflected over many different angles from a single point, allowing a bright, sharp image.

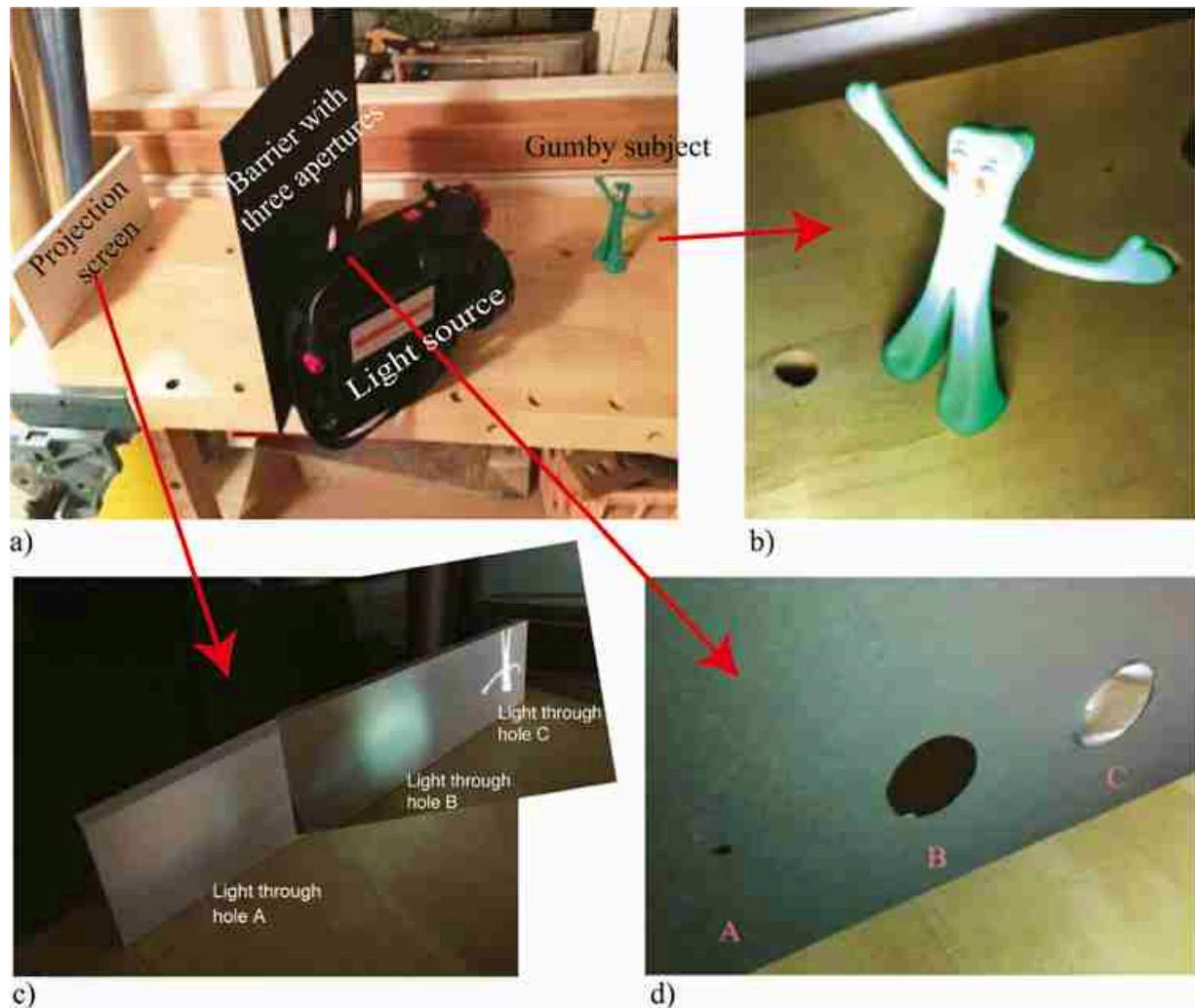


Figure 6.2: Physical demonstration of the trade-offs illustrated in [figure 6.1](#). (a) Gumby subject, illumination light, barrier with the three apertures, and white projection screen. Also, a light source is added to illuminate the subject. (b) Picture of Gumby. (c) Images formed by light through the three apertures. (d) Detail of the three apertures (small pinhole, large pinhole, and lens).

6.2 Lensmaker's Formula

In general, light changes both its wavelength and its speed as it passes from one material to another. Those changes at the material interface will cause the light to bend, an effect called **refraction**. The amount of light bending depends on the change of speed of light within each material, and the orientation of the light ray with respect to the interface surface, according to **Snell's Law** [\[199\]](#), given in the equation below. Both the wavelength and the speed of light in a medium are inversely proportional to the **index of**

refraction of that medium, denoted as n . The n for a vacuum is 1. At a material boundary, for the geometry illustrated in [figure 6.3](#), we have

$$n_1 \sin(\theta_1) = n_2 \sin(\theta_2) \quad (6.1)$$

where θ_1 and θ_2 are the angles with respect to the surface normal of the incident and outgoing light rays, and n_1 and n_2 are the indices of refraction of the materials in region 1 and region 2. Snell's law can be derived by matching the wavelength of light projected along the material interface boundary across each side of the boundary.

Snell's Law relates the bending angles of refraction, θ_1 and θ_2 , to the indices of refraction, n_1 and n_2 , at a material interface.

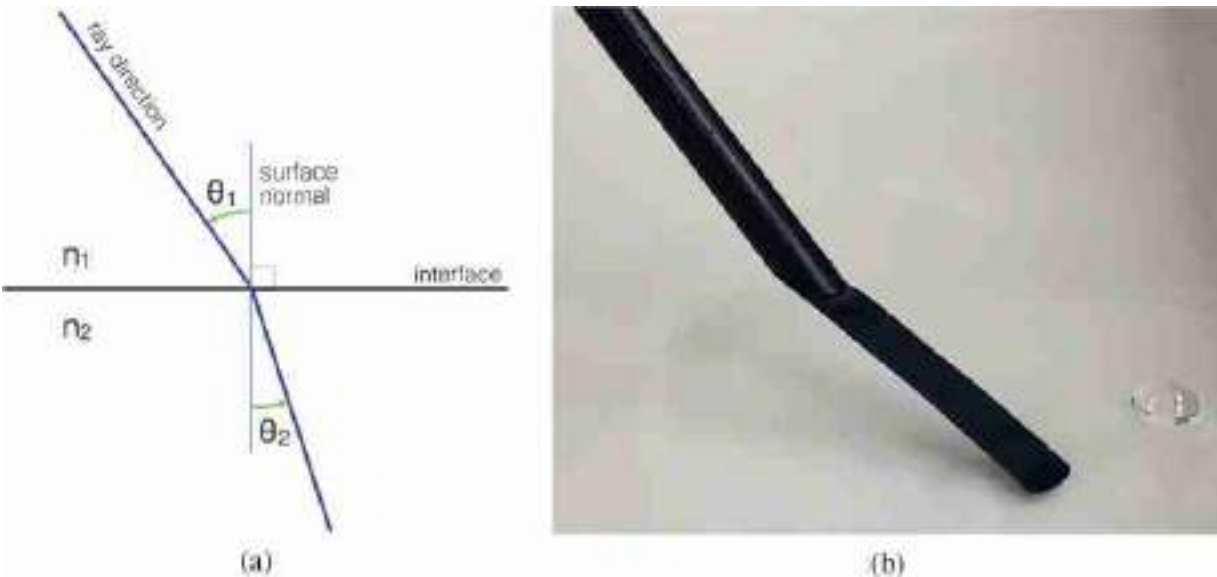


Figure 6.3: (a) Snell's law describes the bending of light at interfaces of differing indices of refraction, n_1 and n_2 , in terms of the angles, θ_1 and θ_2 , relative to the interface, or surface, normal. (b) Straw below water surface appears distorted due to refraction at the air/water boundary. Note that (a) and (b) are not trivially connected!

A lens is a specially shaped piece of transparent material, positioned to focus light from a surface point onto a sensor. In an ideal world, a lens focusing light from a surface onto a sensor plane has the property that every light ray from the surface point that passes through the lens is refracted onto a common position at the sensor, no matter what part of the lens the ray from the surface hits. This dramatically increases the light-gathering ability

of the camera system, overcoming the poor light-gathering properties of a pinhole camera system.

To achieve that property, we must find a surface shape that allows for this focusing. Modern lens surfaces are designed by numerical optimization methods, trading off engineering constraints to achieve the best design, often involving several optical elements and different materials. But to gain insight into the properties of lenses, we can analytically design a lens surface shape provided we simplify the optical system.

For small angles θ denoted in radians, $\sin(\theta) \approx \theta$. If we also assume the index of refraction of air is 1 (it is 1.0003) and denote the index of refraction of lens glass as n , then Snell's law, as shown in [figure 6.3](#), becomes

$$\theta_1 = n\theta_2, \tag{6.2}$$

for small bending angles θ_1 and θ_2 .

Consider a lens, and two points along its optical axis at a distance a and b from the lens, as shown in [figure 6.4\(a\)](#). We seek to find a surface shape for a lens which creates the light paths shown in [figure 6.4\(a\)](#): light leaving from any direction at point a will be focused to arrive at point b . [Figure 6.4\(b\)](#) shows a view of [figure 6.4\(a\)](#), with angles and distances distorted for clarity of labeling.

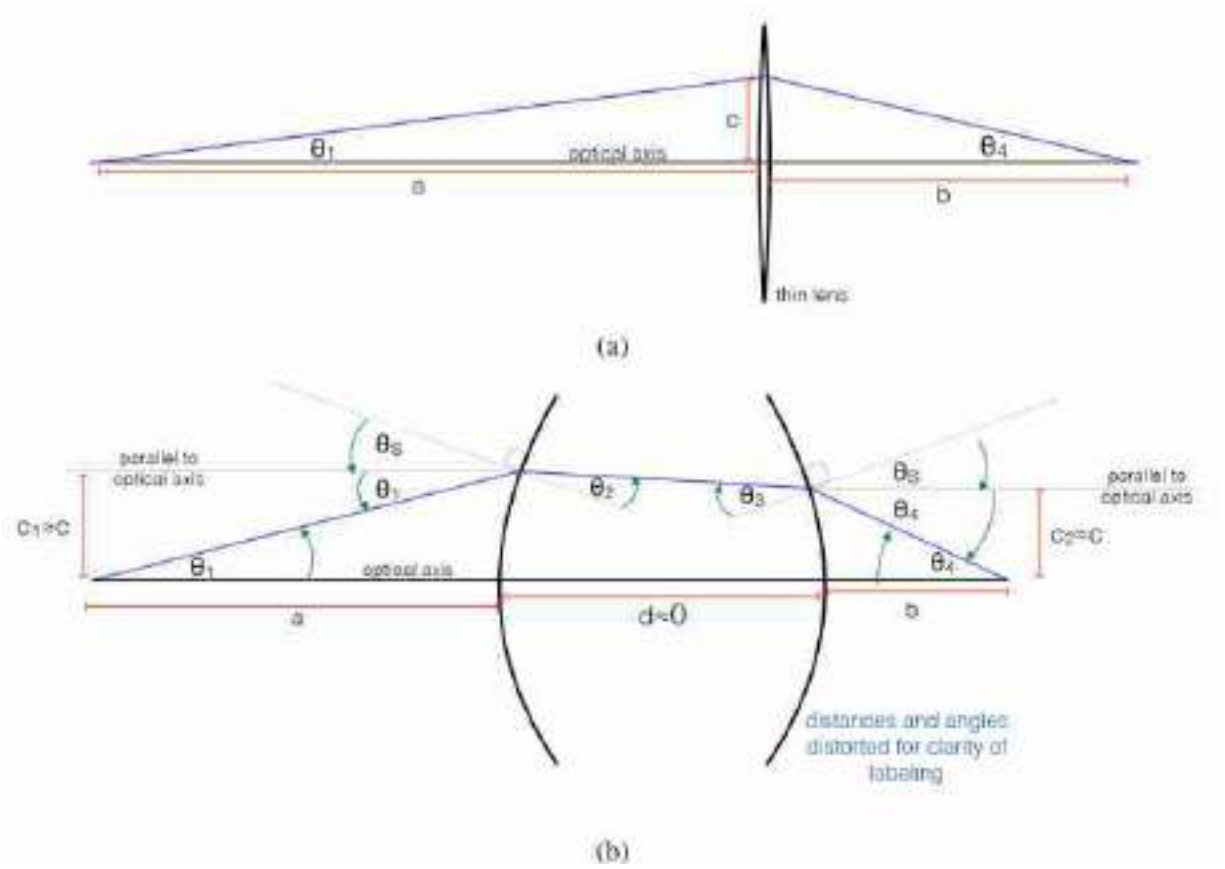


Figure 6.4: (a) The geometry of a thin lens. (b) Showing the labels of adding angles referenced in [table 6.1](#), used to describe the conditions for the lens shape, θ_S to give the desired focusing. Geometry is distorted for visibility.

We make use of several approximations that commonly hold for imaging systems: the deviations in angle from the optical axes are very small (i.e., the *paraxial approximation*) and the lens is modeled to have negligible thickness compared with other distances along the optical axis (i.e., the *thin lens approximation*). Under those approximations, we can write simple expressions for the bending angles shown in [figure 6.4\(b\)](#) as a function of θ_S , the lens surface orientation at height c . Those relations are summarized below in [table 6.1](#).

The middle row of [table 6.1](#) requires explanation. The light ray within the lens, depicted in [figure 6.4\(b\)](#), is not necessarily parallel to the optical axis. In general, it will be rotated by some angle δ away from the optical axis, giving $\theta_S = \theta_2 + \delta$, and, at the other lens surface, $\theta_S = \theta_3 - \delta$. Adding those two equations removes δ and gives the relation $2\theta_S = \theta_2 + \theta_3$

Table 6.1Relations between angles of thin lens example, [figure 6.4\(b\)](#)

Angle	Description	Relation	Reason
θ_1	Initial angle from optical axis	$\theta_1 = c/a$	Small angle approx.
θ_2	Angle of refracted ray wrt front surface normal	$n\theta_2 = \theta_1 + \theta_S$	Snell's law, small angle approx.
θ_3	Angle of refracted ray wrt back surface normal	$2\theta_S = \theta_2 + \theta_3$	Symmetry of lens, thin lens approx.
$\theta_4 + \theta_S$	Angle of ray exiting lens wrt back surface normal	$n\theta_3 = \theta_4 + \theta_S$	Snell's law, small angle approx.
θ_4	Final angle from optical axis	$\theta_4 = c/b$	Small angle approx.

If we start from the relation in [table 6.1](#) for θ_4 , and substitute for each angle using the relation in each line above, up through $\theta_1 = c/a$, we can algebraically eliminate the angles θ_1 through θ_4 to find the condition on the lens surface angle, θ_S , as a function of the distance c from the optical axis, which allows for the desired focusing to occur. The result is as follows:

$$\theta_S = \frac{c}{2(n-1)} \left(\frac{1}{a} + \frac{1}{b} \right) \quad (6.3)$$

This relationship creates the effect that every ray emanating at a small angle from point a will be focused to point b . Equation (6.3) shows that the lens surface angle, θ_S , must deviate from flat in linear proportional to the distance c from the center of the lens.

To focus, the lens surface angle, θ_S , must be a *linear* function of the distance, c , from the center of the lens.

In the thin lens approximation, both parabolic and spherical shapes satisfy that constraint on the lens surface slope. For a spherical lens surface, such as [figure 6.5](#), curving according to a radius R , we have $\sin(\theta_S) = c/R$. For small angles θ_S , this reduces to

$$\theta_S = \frac{c}{R}, \quad (6.4)$$

where R is the radius of the sphere, which has the desired property that $\theta_S \propto c$.

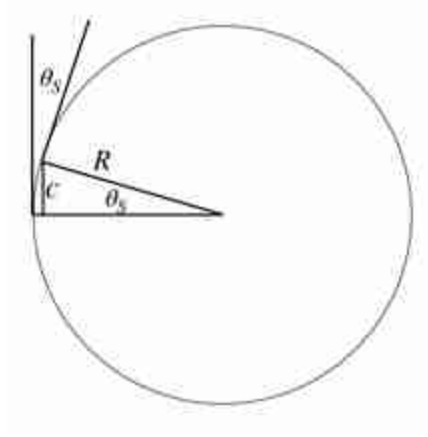


Figure 6.5: Relation between R and θ_S , in equation (6.4), for a spherical lens.

Substituting equation (6.4) into the focusing condition, equation (6.3) yields the **Lensmaker's Formula**,

$$\frac{1}{a} + \frac{1}{b} = \frac{1}{f}, \quad (6.5)$$

where the lens **focal length**, f is defined to be

$$f = \frac{R}{2(n-1)} \quad (6.6)$$

It is straightforward to show, by rotating the lens in [figure 6.4\(a\)](#) through an angle θ_R in the above derivation, thus adding θ_R to θ_S in [table 6.1](#), that the lensmaker's equation also holds for light originating off the optical axis. Thus, under the paraxial and thin lens approximations, the lens focuses light from points on a plane onto points on a second plane, both perpendicular to the optical axis, as illustrated in [figure 6.6](#).

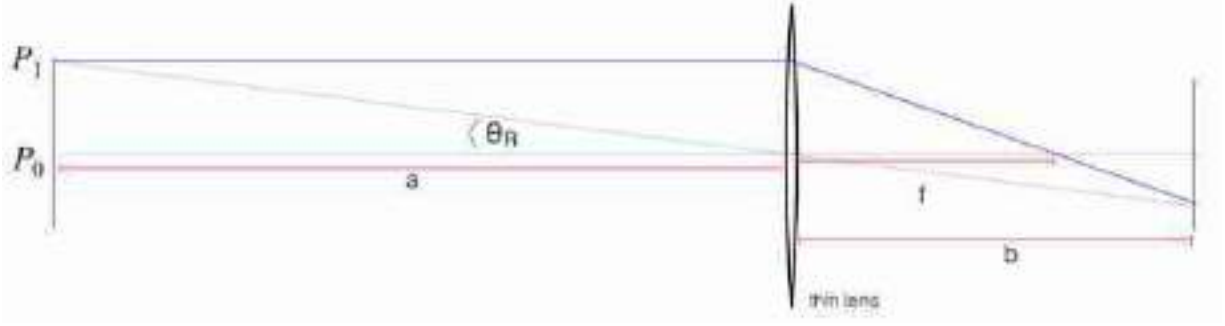


Figure 6.6: Considering light from off-axis sources is equivalent to rotating the lens surface by θ_R , for small angles θ_R . The lens focuses rays from off-axis points like P_1 , as well as from the on-axis point, P_0 .

One can also generalize the equation for the case of a lens with different radii of curvature, R_1 and R_2 on the front and back faces of the lens. Defining the left and right surface angles θ_{s1} and θ_{s2} , respectively, the middle row equation of [table 6.1](#) becomes $\theta_{s1} + \theta_{s2} = \theta_2 + \theta_3$. Then, the equation substitutions of [Table 6.1](#), combined with $\theta_{Si} = c/R_i$, for $i = 1$ and 2 , lead to

$$\frac{1}{f} = (n - 1) \left(\frac{1}{R_1} + \frac{1}{R_2} \right) \quad (6.7)$$

Note that some texts (e.g., [\[199\]](#)) adopt a sign convention where the back surface of a lens is defined to have negative curvature.

[Figure 6.7](#) shows a demonstration of the focusing property for a thin lens. The hand, labeled (a), is flicking a laser pointer back and forth, sending light rays in many directions from a central point during the image exposure, as would a diffuse surface reflection. All the light rays that strike the lens, labeled (b), are focused to the same green spot, labeled (c), while the light rays passing outside the lens at (b) reveal their straight line trajectories on the wall at (c).

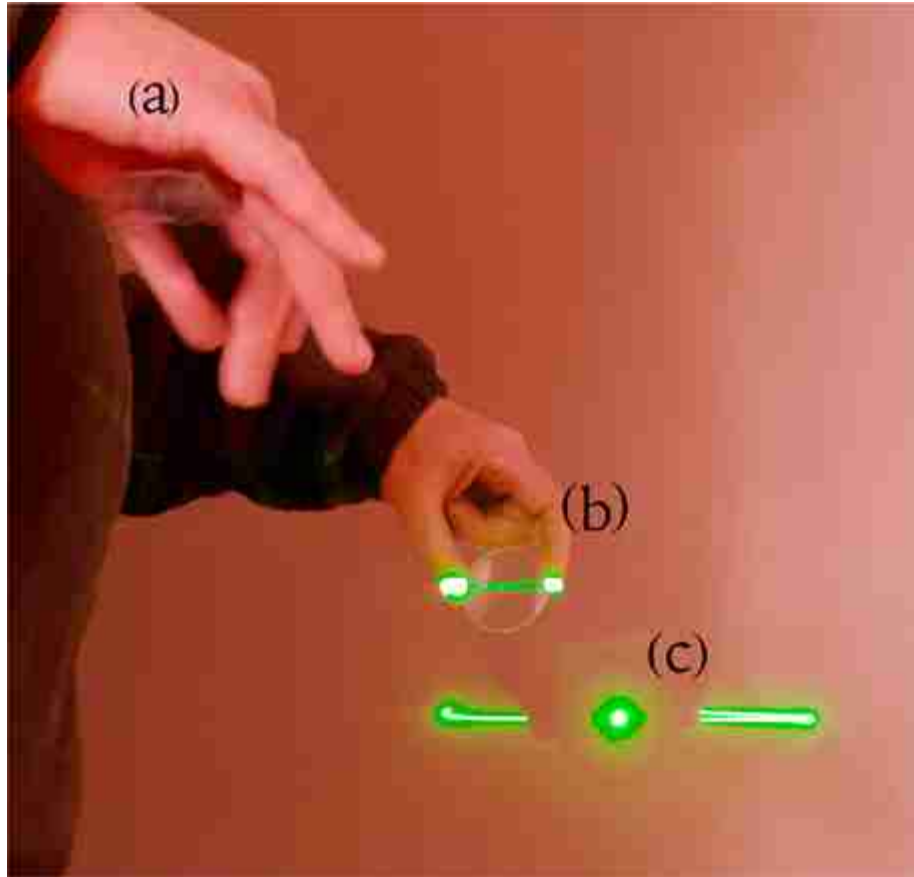


Figure 6.7: Demonstration showing that a lens focuses a fan of rays, such as those reflecting from a diffuse surface, to a point. (a) Here the right hand wiggles a laser pointer back and forth during the photographic exposure, generating a fan of light rays, approximating (in one dimension) rays reflecting from one point on a diffuse surface. (b) The fan of rays sweeps across a lens, which focuses each ray passing through the lens to the same spot at the wall, (c), regardless of where each ray had entered the lens. (c) The rays that pass outside the lens form the two line segments on either side.

6.3 Imaging with Lenses

Armed with the lensmaker's formula and the observation of [figure 6.8](#), we can analyze how rays travel through lenses in the approximation of geometrical optics. Light passing through the center of a thin lens, where the front and back surfaces are parallel, proceeds without bending. Thus a convex thin lens images as a pinhole camera does, creating a perspective projection.

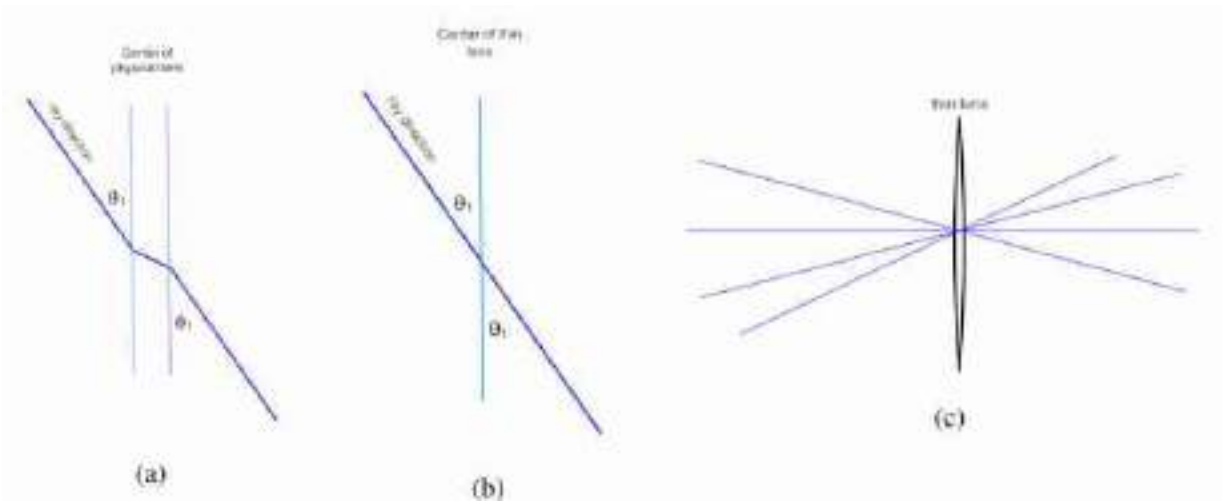


Figure 6.8: Light travels straight through the center of a thin lens, where front and back surfaces are parallel. (a) Lens with non-zero thickness. (b) Idealized thin lens. (c) Rays passing through the center of the lens behave like rays passing through a pinhole camera, and thus lenses impose a *perspective projection*.

Two points on opposite sides of the lens at distances a and b from the lens that satisfy equation (6.5) are known as conjugate points. Light from one conjugate point, traveling through the lens, focuses to the other conjugate point, and vice versa (see [figure 6.9](#)). The rules for tracing light travel through a lens include the following:

1. Every ray passing through one conjugate point and passing through the lens then passes through the conjugate point. This is the fundamental focusing property of a lens.
2. Parallel rays entering the lens all focus to a point at distance f behind the lens, per equation (6.5). In this case, one of the focal points is at infinity.
3. Any ray passing through the center of the thin lens proceeds in a straight line, as if through a pinhole at the center of the lens (see [figure 6.8](#)). Thus, lenses, like pinholes, render the world in *perspective projection*.
4. Because focused rays render positions onto a plane as does a line passing through the center of the lens, then the **magnification** of a lens is simply a/b , where a is the distance of an infocus object to the lens, and b is the distance from the lens to a sensor plane.

The above properties allow us to analyze the light paths through simple configurations of lenses.

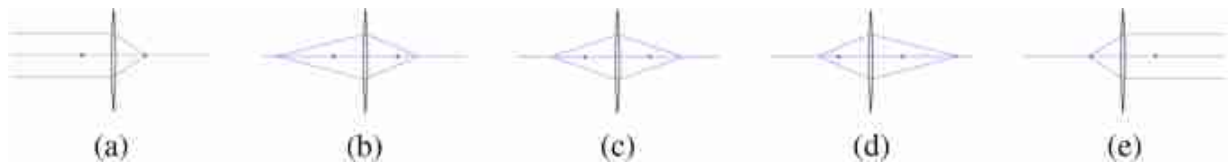


Figure 6.9: Showing some conjugate points for a convex thin lens.

Referring to [figure 6.9](#), the two dots are at distance f from the lens. (a) In [figure 6.9](#)(a), parallel rays from infinity focus at a distance f from the lens. [Figure 6.9](#)(b–d) show that as a source of light rays moves closer to the lens, they focus further away on the other side, while [figure 6.9](#)(e) shows how rays emanating from a distance f from the lens are parallel after the lens, that is, they focus at infinity.

6.3.1 Depth of Field

Assume that the lens is part of a camera, focusing light from the world onto the camera's sensor. If a plane outside of the camera, called the **focal plane**, and the camera's sensor plane are at conjugate positions, then an object in the focal plane is focussed sharply onto the sensor plane. All rays from a point on the object's surface will come into focus at a point in the sensor plane. This will image a point in the focal plane to a **circle of confusion** at the sensor plane, as illustrated in [figure 6.10](#).

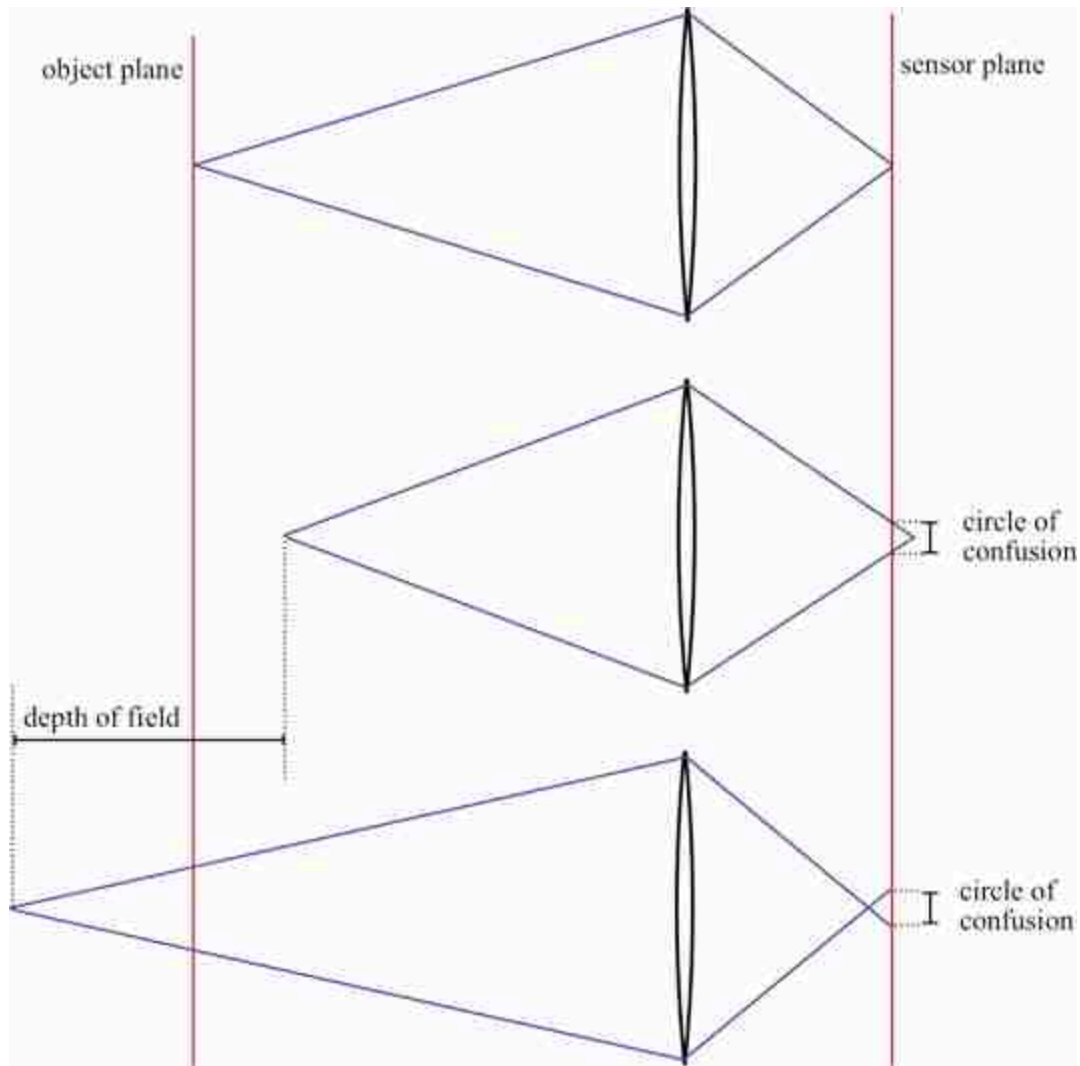


Figure 6.10: Circle of confusion and depth of field. Referring to [figure 6.6](#), if the object of interest and the camera's sensor plane are related as conjugate points, that is, with distances a and b from the camera lens respecting the lensmaker's formula (equation (6.5)), then the object will form an image in sharp focus in the sensor plane. But objects in front of the plane of focus by δ_a will, in general, come to focus some distance δ_b behind the sensor plane [\[292\]](#).

Objects closer or further from the focal plane will come into focus behind or in front of the sensor plane, respectively, resulting in a blur circle at the sensor plane, as illustrated in [figures 6.10 and 6.11](#).

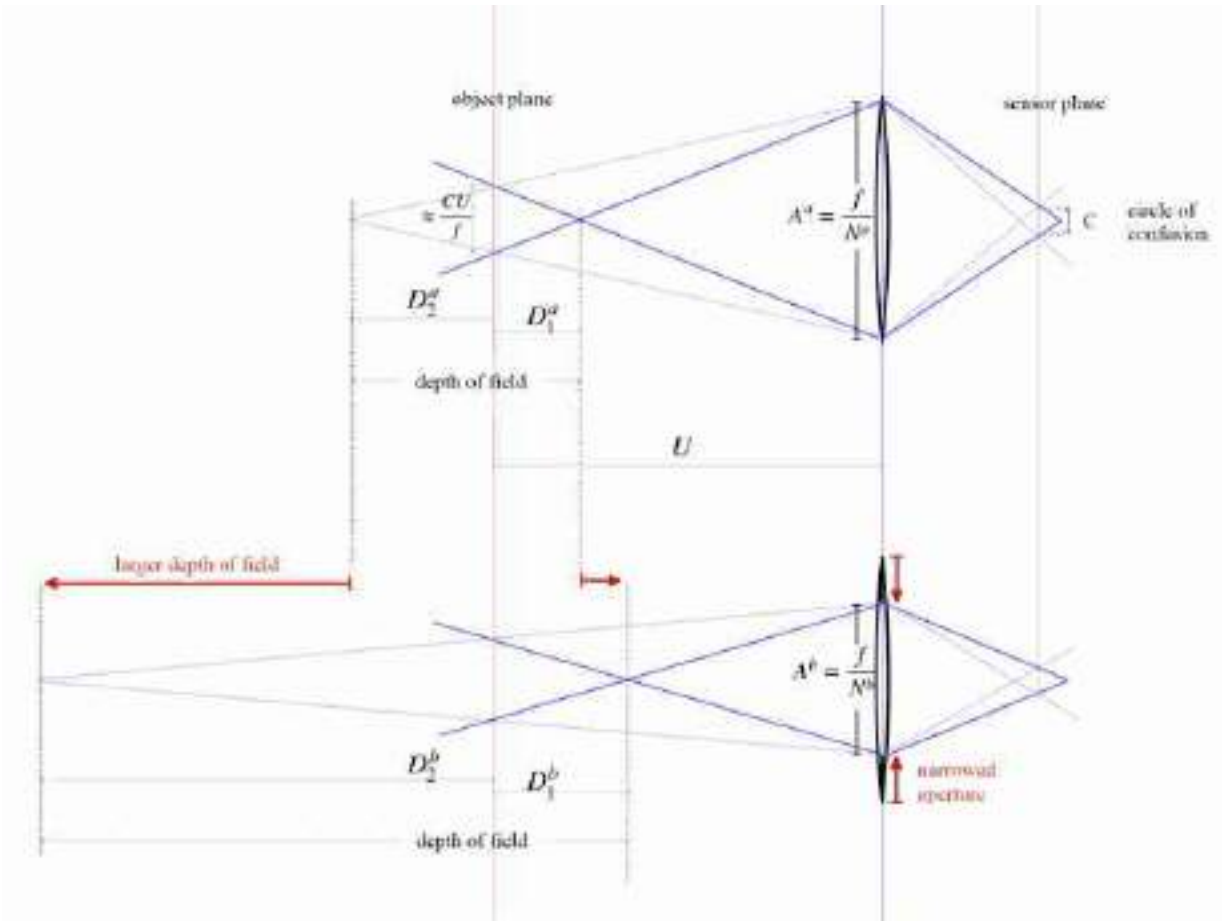


Figure 6.11: Circle of confusion and depth of field. (Top) Variables used in the computation of the depth of field of a lens. (Bottom) The elongation of the depth of field size for a given tolerable circle of confusion results from narrowing the lens aperture [292]. The catch is that narrowing the aperture results in less light reaching the sensor plane.

The region around the plane of focus that results in a blur at the sensor plane that is smaller than some tolerance is called the **depth of field**. We can calculate the depth of field from geometric considerations. This derivation follows that of Marc Levoy [292].

A camera's **f-number**, written N , is the ratio of the focal length of the lens divided by the diameter of the lens that light passes through, called the aperture, A : $N = f/A$. Let the diameter of the circle of confusion be C . If the distance, U , of the focal plane to the camera is much larger than the focal length of the lens ($U \gg f$) then the diameter of the focal plane that is imaged onto the circle of confusion is C times the magnification of the imaging system, or approximately CU/f , again, for $U \gg f$. Referring to the top of [figure 6.10](#), by similar triangles, we have

$$\frac{D_1 f}{CU} = \frac{U - D_1}{\frac{f}{N}} \quad (6.8)$$

Also by similar triangles, we have

$$\frac{D_2 f}{CU} = \frac{D_2 + U}{\frac{f}{N}} \quad (6.9)$$

Defining the depth of field, D , to be $D = D_1 + D_2$, and combining terms, we have

$$D = \frac{2NCU^2 f^2}{f^4 - N^2 C^2 U^2} \quad (6.10)$$

When the circle of confusion, C , is much smaller than the lens aperture, f/N , then the term $N^2 C^2 D^2$ can be ignored relative to f^4 , and we have

$$D \approx \frac{2NCU^2}{f^2} \quad (6.11)$$

Note that for smaller camera f-numbers, N (larger apertures), the depth of field decreases in proportion. This effect can be seen in the bottom of [figure 6.11](#), and in [figure 6.12](#).

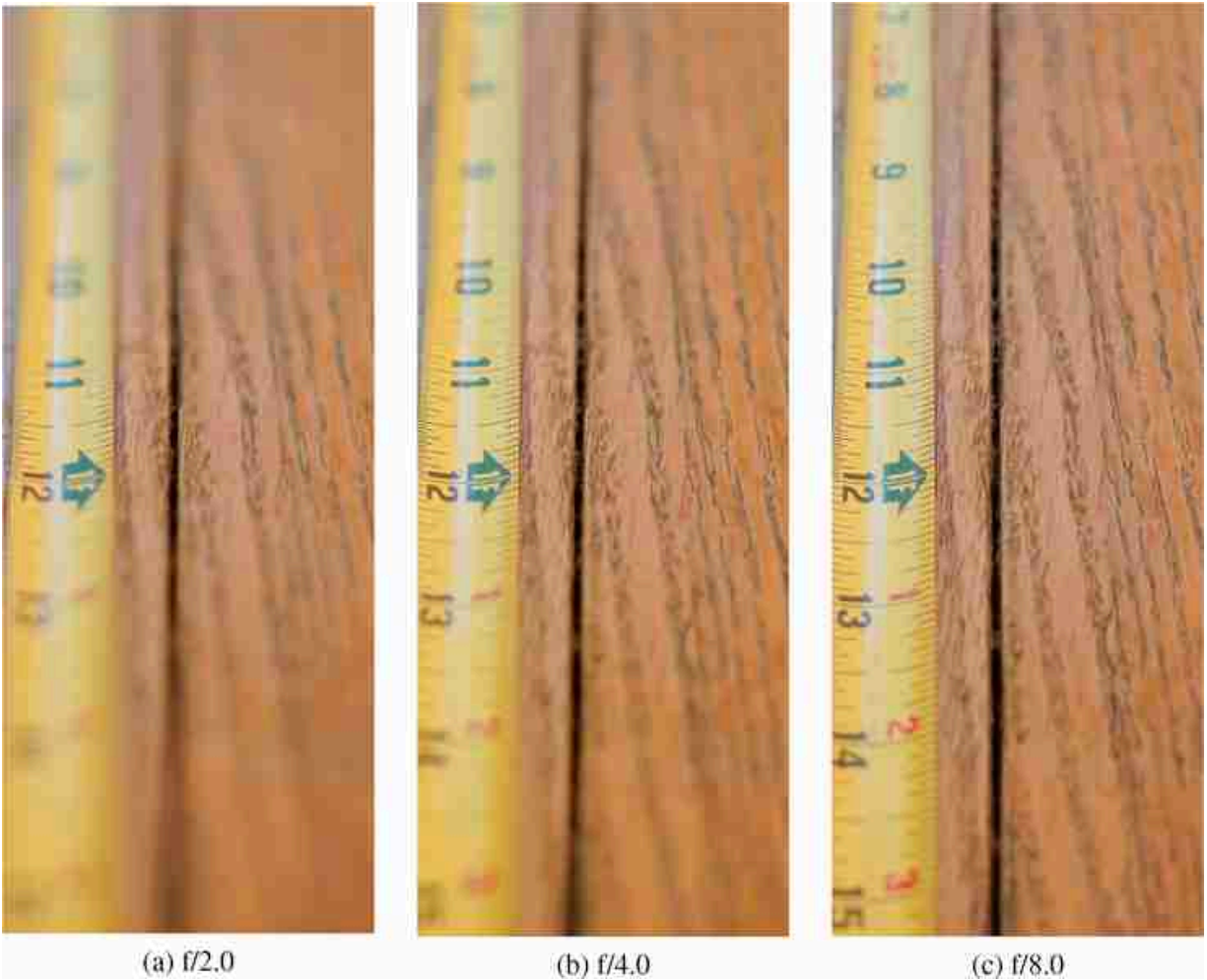


Figure 6.12: Showing the linear relationship between f-number and depth of field. From the same camera position, a ruler was photographed, in (a), (b), and (c), using f/2.0, f/4.0, and f/8.0, respectively. Note the region of sharp focus doubles from (a) to (b), and from (b) to (c).

6.3.2 Concave Lenses

The lens we designed in section 6.2 was convex. Also of interest are **concave** lenses, with the lens surface curve bowing inward toward the center of the lens. For example, consider the concave lens with the surface orientation, $\theta_{sx} = -\theta_s$ in equation (6.5). By the reflection symmetry of the lens surface angles, parallel rays entering a concave lens will be bent away from the lens center by the same amount it would be bent *toward* the center axis for a convex lens of the same lens radii of curvature. This leads to a concave lens bringing parallel rays to a virtual focus, a point from which the diverging rays leaving the lens appear to be emanating. This is

illustrated in [figure 6.13\(b\)](#) for on-axis, and [figure 6.13\(c\)](#) for off-axis parallel rays.

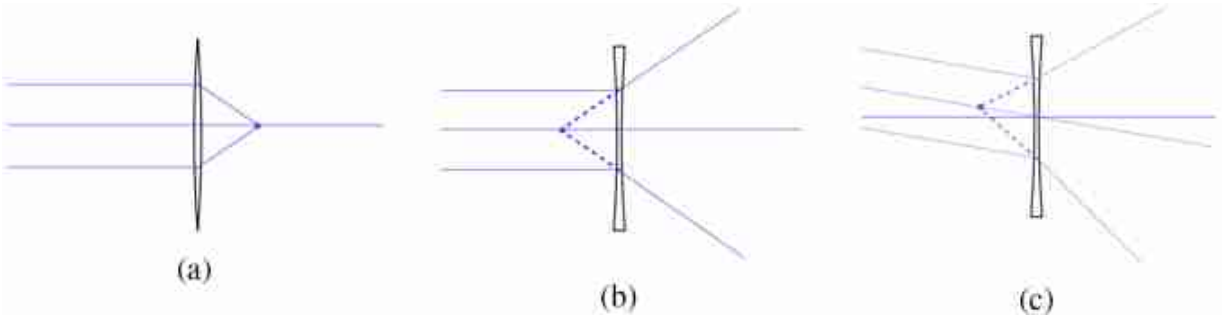


Figure 6.13: (a) A convex lens focuses rays to a point. (b) A concave lens focuses rays to a virtual point. (c) As with convex lenses, a small shift in the angle of the incoming rays causes a shift in the focus point at the focal plane.

The mathematics of the ray bending is similar to that for convex lenses, except the focal length of concave lenses has a negative value, illustrated in [figure 6.13](#). The focus point for a set of parallel rays entering a concave lens from the left is also on the *left side* of the lens. It is a *virtual focal point*, and from the right side of the lens, the emanating rays appear to be originating from the point at distance $-f$ to the left of the lens.

6.3.3 Lenses in a Telescope

The properties of convex and concave lenses can be used to make a telescope, as Galileo did in the early 1600s [\[152\]](#). We can see how the telescope magnifies the size of objects from geometric considerations. As shown in [figure 6.14\(a\)](#), we position a convex lens, lens 1, with a long focal length, f_1 , and a concave lens, lens 2, with a shorter focal length, f_2 , such that each are f_1 and f_2 away from some common point, respectively. Under those conditions, the configurations of [figures 6.13\(a\)](#) and 6.13(b) will cause parallel rays from a distant object entering lens 1 to become a compressed set of parallel rays leaving lens 2. Referring to [figure 6.14\(b\)](#), an angular deviation of the input rays of δ_i will become an angular deviation, exiting the convex lens of δ_o .

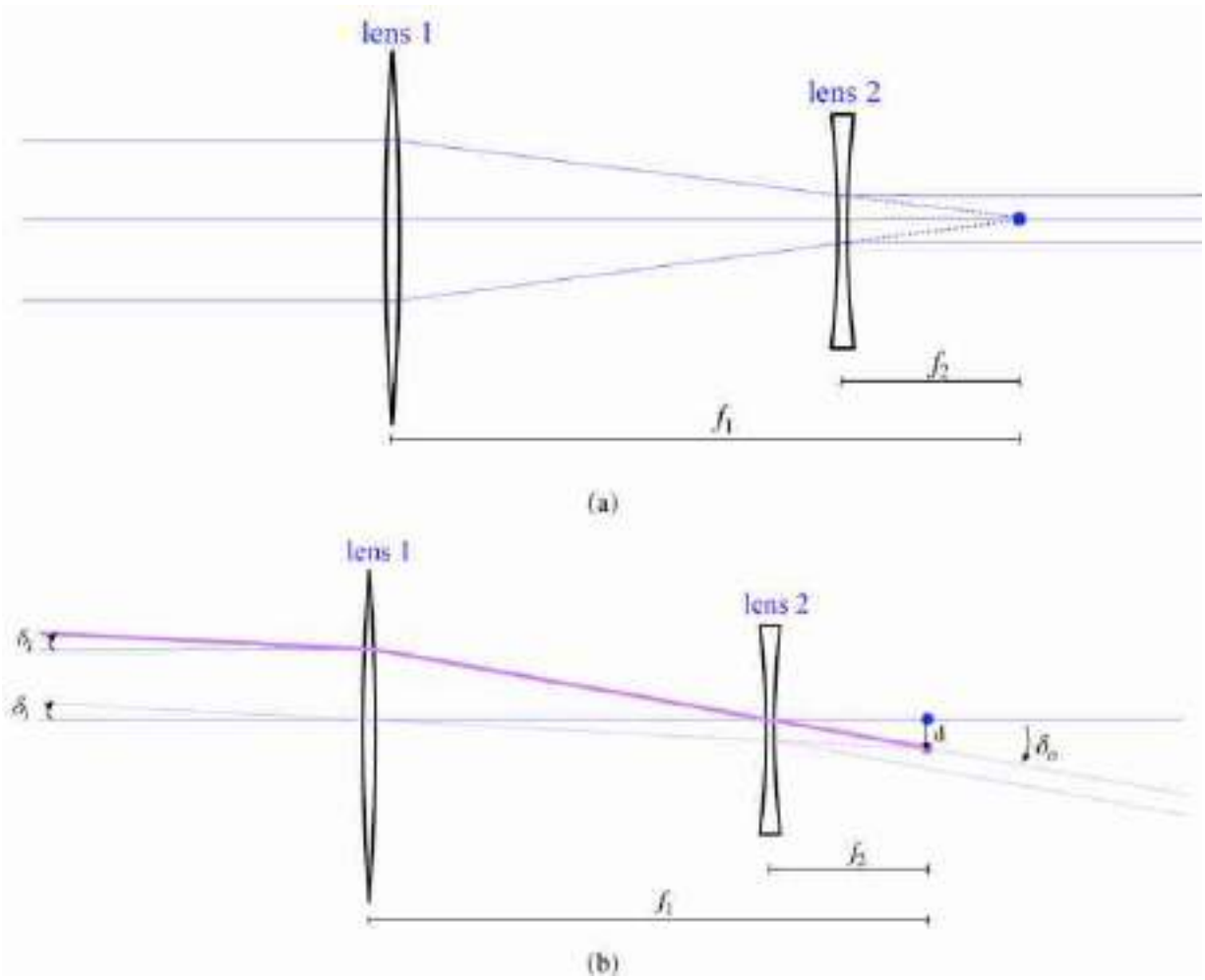


Figure 6.14: (a) Galilean telescope. The convex lens 1 and the concave lens 2 share the same focal point, marked with a dot, resulting in parallel rays in giving parallel rays out. (b) A small change in the input ray angle gives a larger change in the output ray angle. The magnification of the telescope is the ratio, M , of the angular deviation of an output ray, δ_o , to the angular deviation of an input ray, δ_i . Writing the focal point offset, d , in terms of the rays passing through the centers of lenses 1 and 2 gives the relation, $M = f_1/f_2$, where f_1 and f_2 are the respective ray focal lengths.

By property 3 of section 6.3, and the small angle approximation, we have $\delta_i f_1 = d$, because the point p is a distance f_1 from lens 1. Similarly, we have $\delta_o f_2 = d$. Substituting for d gives

$$M = \frac{\delta_o}{\delta_i} = \frac{f_1}{f_2}, \quad (6.12)$$

where we have defined the telescope magnification, M , to be the amplification of the parallel ray bending angles.

[Figure 6.15](#) shows a homemade telescope, inspired by Galileo's. The tube is a 2 in. cardboard mailing tube, with a second tube inserted within it to allow the interlens distance to be adjusted for focusing. The convex lens at the front of the telescope has a focal length of 50 cm, and the convex lens eyepiece has a focal length of 18 mm, giving a magnification of 27. This is comparable to the 30x magnification of Galileo's telescope.

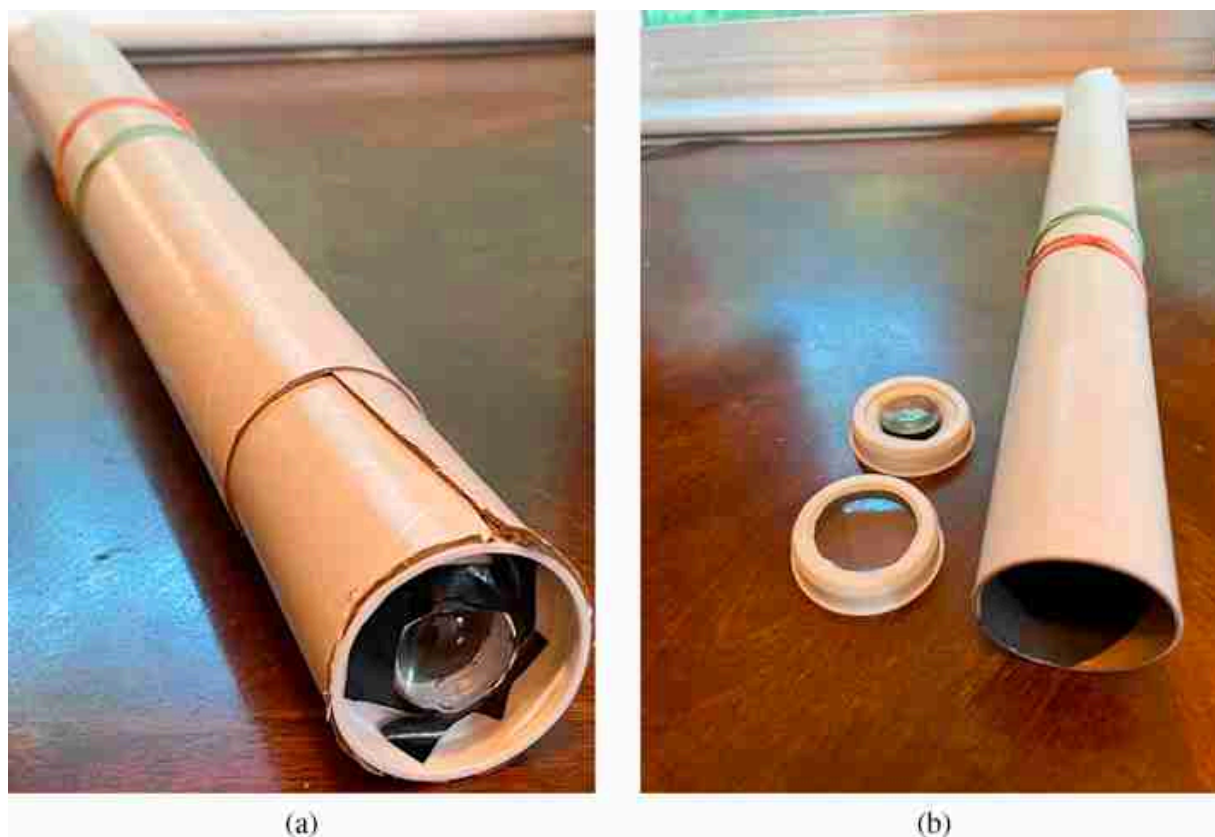


Figure 6.15: Homemade Galilean telescope. (a) The two lenses of the telescope are mounted in a cardboard mailing tube, with focusing allowed by adjustment of an inserted slightly smaller tube with the eyepiece lens. (b) Mailing tube with the lenses removed from the tube ends. The rubber bands mount the telescope to a tripod, and tube interior is lined with black paper to reduce stray light.

[Figure 6.16](#) compares images made from this homemade telescope with drawings Galileo made of what he saw through his telescope. He discovered that our moon had mountains and craters, and was thus not a perfect sphere, as some had thought. While he could not resolve the rings of Saturn, he could see that there was something different about its shape. These can all be seen from just two lenses within a tube!

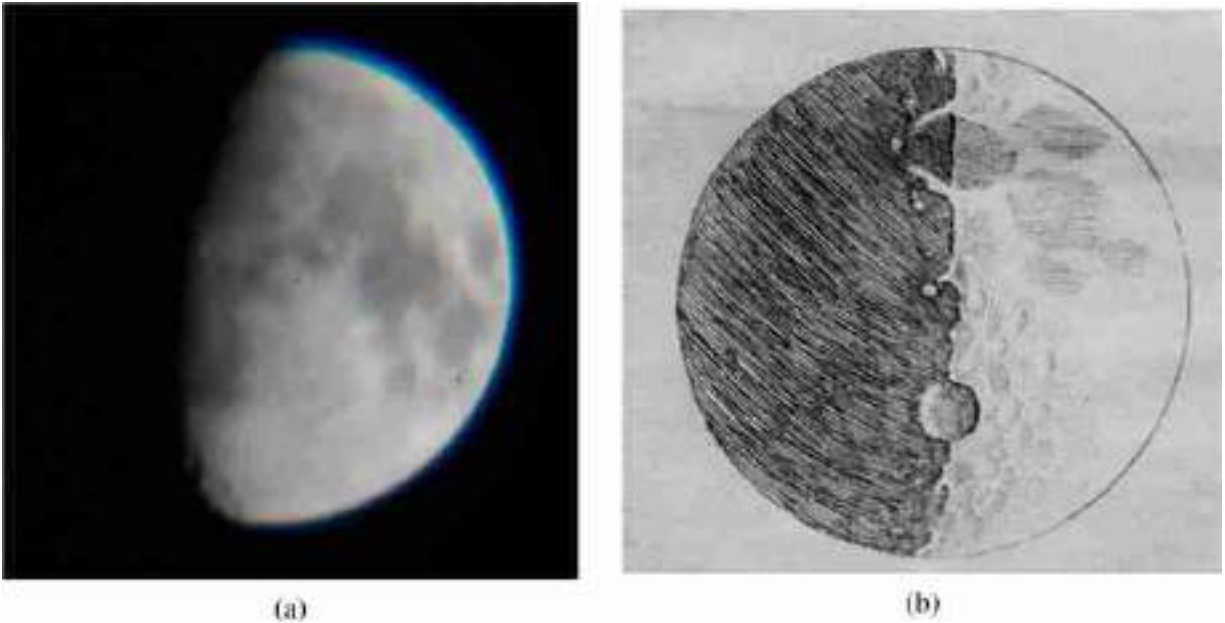


Figure 6.16: (a) Moon as viewed through homemade telescope of Galileo's design, photographed at eyepiece with a cell phone camera. The telescope magnification is 27x. (b) Drawing by Galileo of the moon, seen through his telescope, with 30x magnification.

With his telescope, Galileo discovered four of the moons of Jupiter. He called them the Medician Stars, in honor of the brothers of the Medici family, who were prospective patrons.

6.4 Concluding Remarks

Pinhole cameras can allow more light to enter the camera aperture only at the expense of image sharpness. Lenses overcome that tradeoff by focusing all the rays passing through the lens from a point on a surface to a single position on the sensor plane, creating an image that is both sharp and bright.

Using small-angle and thin-lens approximations, we designed a lens surface to have that focusing property, showing that the lens must have a parabolic or spherical shape.

Geometric considerations allow depth-of-field calculations, as well as the design of a simple telescope.

7 Cameras as Linear Systems

7.1 Introduction

Lens-based and pinhole cameras are special-case devices where the light falling on the sensors or the photographic film forms an image. For other imaging systems, such as medical or astronomical systems, the intensities recorded by the camera may look nothing like an interpretable image. Because the optical elements of imaging systems are often linear in their response to light, it is convenient and powerful to describe such general cameras using linear algebra, which then provides a powerful machinery to recover an interpretable image from the recorded data. This formulation also helps to build intuitions about how cameras work and it will allow us think about new types of cameras.

7.2 Flatland

For the simplicity of visualization, let's consider one dimensional (1D) imaging systems. As shown in [figure 7.1](#) the sensor is 1D and the scene lives in flatland.

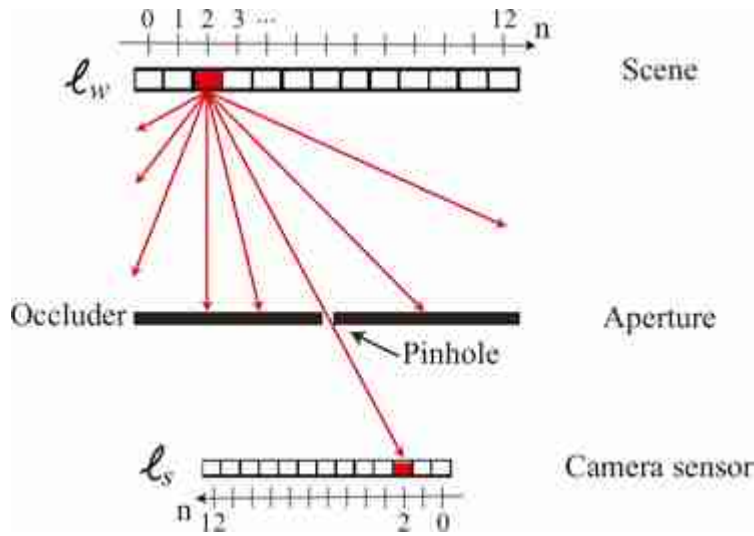


Figure 7.1: A one dimensional imaging system in flatland. As in figure 5.8, we have reversed the direction of the sensor coordinates.

Let the light intensities in the world be represented by a column vector, ℓ_w . The value of the n -th component of ℓ_w gives the intensity of the light at position n , heading in the direction of the camera. There are a few more things to notice in [figure 7.1](#). The first one is that the axis in the camera sensor is reversed, consistent with figure 5.8. The value n corresponds to image coordinates and the units are pixels. The second is that we are modeling the scene as being a line with varying albedos at a fixed depth from the camera. If the scene had a more complex spatial structure we would have to use the **light field** in order to use the same linear algebra that we will use in this chapter to model cameras. We will use light fields in chapter 45.

In this chapter we will approximate the input light by a discrete and finite set of light rays instead of a continuous wave. This approximation allow us to use linear algebra to describe the imaging process.

7.3 Cameras as Linear Systems

As shown in [figure 7.2\(a\)](#) the sensor is 1D and the scene lives in flatland. Let the sensor measurements be ℓ_s and the unknown scene be ℓ_w . If the camera sensors respond linearly to the light intensity at each sensor, their

measurements, ℓ_s , will be some linear combination, given by the matrix, $\mathbf{A}$, of the light intensities:

$$\ell_s = \mathbf{A}\ell_w \quad (7.1)$$

We represent the camera by matrix $\mathbf{A}$. For the case of a pinhole camera, and assuming 13 pixel sensor observations, the camera matrix is just a 13×13 identity matrix, depicted in [figure 7.2\(b\)](#), as each sensor depends only on the value of a single scene element. For the case of conventional lens and pinhole cameras, where the observed intensities, ℓ_s , are an image of the reflected intensities in the scene, ℓ_w , then $\mathbf{A}$ is approximately an identity matrix. For more general cameras, $\mathbf{A}$ may be very different from an identity matrix, and we will need to estimate ℓ_w from ℓ_s .

In the presence of noise, there may not be a solution ℓ_w that exactly satisfies equation (7.1), so we often seek to satisfy it in a least squares sense. In most cases, $\mathbf{A}$ is either not invertible, or is poorly conditioned. It is often useful to introduce a regularizer, an additional term in the objective function to be minimized [\[389\]](#). If the regularization term favors a small ℓ_{world} , then the objective term to minimize, E , could be

$$E = \|\ell_s - \mathbf{A}\ell_w\|^2 + \lambda \|\ell_w\|^2 \quad (7.2)$$

The regularization parameter, λ , determines the trade-off between explaining the observations and satisfying the regularization term.

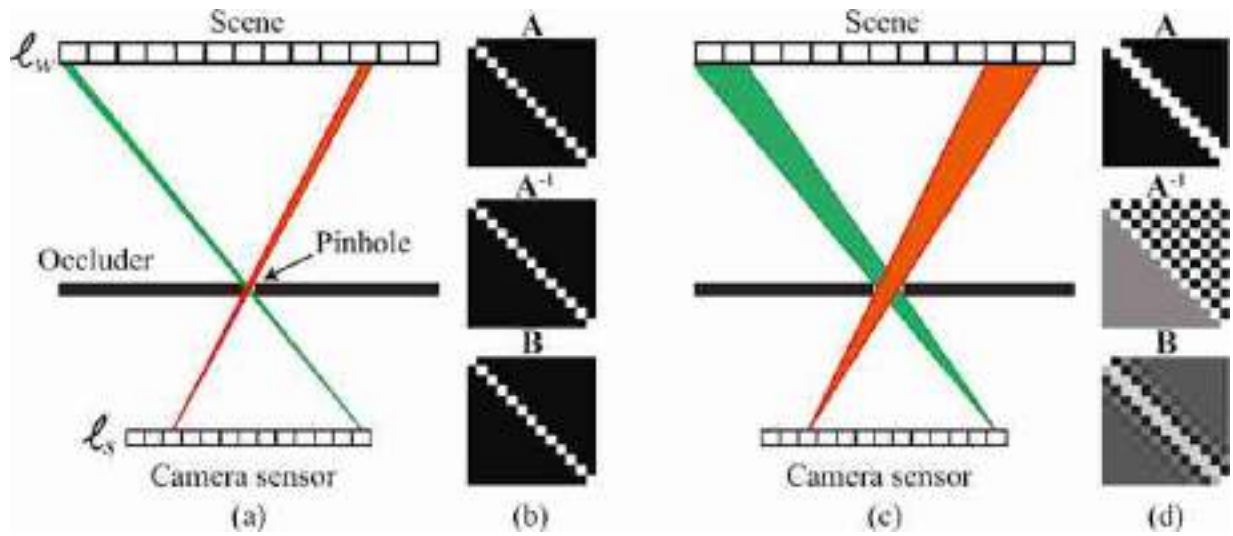


Figure 7.2: (a) Schematic drawing of a small-hole 1D pinhole camera, and (b) the visualization of its imaging matrices $\mathbf{A}$, $\mathbf{A}^{-1}$, and the regularized inverse $\mathbf{B}$. For the small-pin-hole imager, all three matrices are identity matrices. (c) Large-hole 1D pinhole camera, and (d) the visualization of its imaging matrices.

Setting the derivative of equation (7.2) with respect to the elements of the vector ℓ_w equal to zero, we have

$$0 = \nabla_{\ell_w} \|\ell_s - \mathbf{A}\ell_w\|^2 + \nabla_{\ell_w} \lambda \|\ell_w\|^2 \quad (7.3)$$

$$= \mathbf{A}^T \mathbf{A} \ell_w - \mathbf{A}^T \ell_s + \lambda \ell_w \quad (7.4)$$

or

$$\ell_w = (\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I})^{-1} \mathbf{A}^T \ell_s \quad (7.5)$$

Where the matrix $\mathbf{B} = (\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I})^{-1} \mathbf{A}^T$ is the regularized inverse of the imaging matrix $\mathbf{A}$.

1D and the scene lives in flatland. Let the sensor measurements be ℓ_s and the unknown scene be ℓ_w . We represent the camera by matrix $\mathbf{A}$. For the case of a pinhole camera, and assuming 13 pixel sensor observations, the camera matrix is just a 13x13 identity matrix, depicted in [figure 7.2\(b\)](#), as each sensor depends only on the value of a single scene element.

Next, consider the case of a wide aperture pinhole camera, shown in [figure 7.2\(c\)](#). If a single pixel in the sensor plane covers exactly two positions of the scene intensities, then the geometry is as shown in [figure 7.2\(c\)](#). [Figure 7.2\(d\)](#) also shows the imaging matrix, $\mathbf{A}$, its inverse, $\mathbf{A}^{-1}$, and the regularized inverse of the imaging matrix, which will usually give

image reconstructions with fewer artifacts as it is less sensitive to noise. Note that $\mathbf{A}^{-1}$ contains lots of fast changes that will make the result very sensitive to noise in the observation. The regularized matrix, $\mathbf{B}$, shown in the bottom row of [figure 7.2\(d\)](#), is better behaved.

The following plot ([figure 7.3](#)) shows an example of an input 1D signal, ℓ_w , and the output of the 2-pixel wide pinhole camera, ℓ_s .

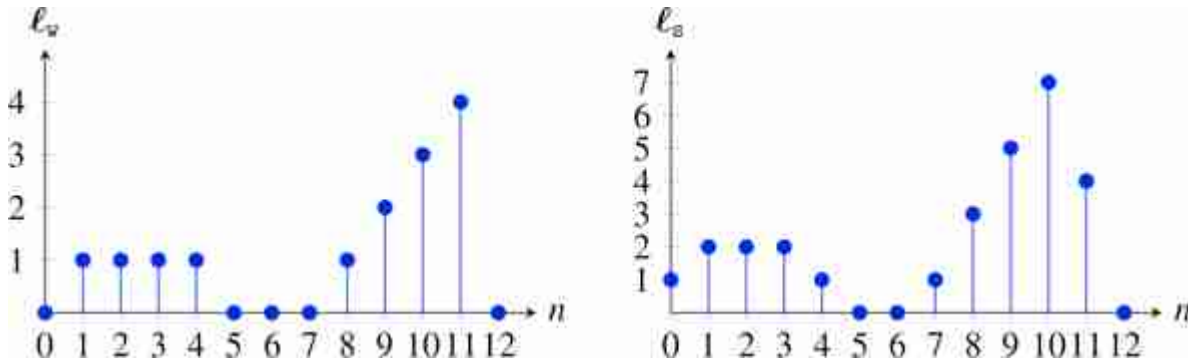


Figure 7.3: Pinhole camera. (left) Input 1D signal, ℓ_w . (right) The output of the two-pixel wide pinhole camera, ℓ_s .

The output is obtained by multiplying the input vector by the matrix $\mathbf{A}$ shown in [figure 7.2\(d\)](#). Each output value is the results of adding up two consecutive input values. The output is nearly identical to the input signal but with a larger magnitude and a bit smoother.

7.4 More General Imagers

Many different optical systems can form cameras and the linear analysis described before can be used to characterize the imaging process. Even a simple edge will do. [Figure 7.4](#) shows two non traditional imaging systems that we will analyze in this section.

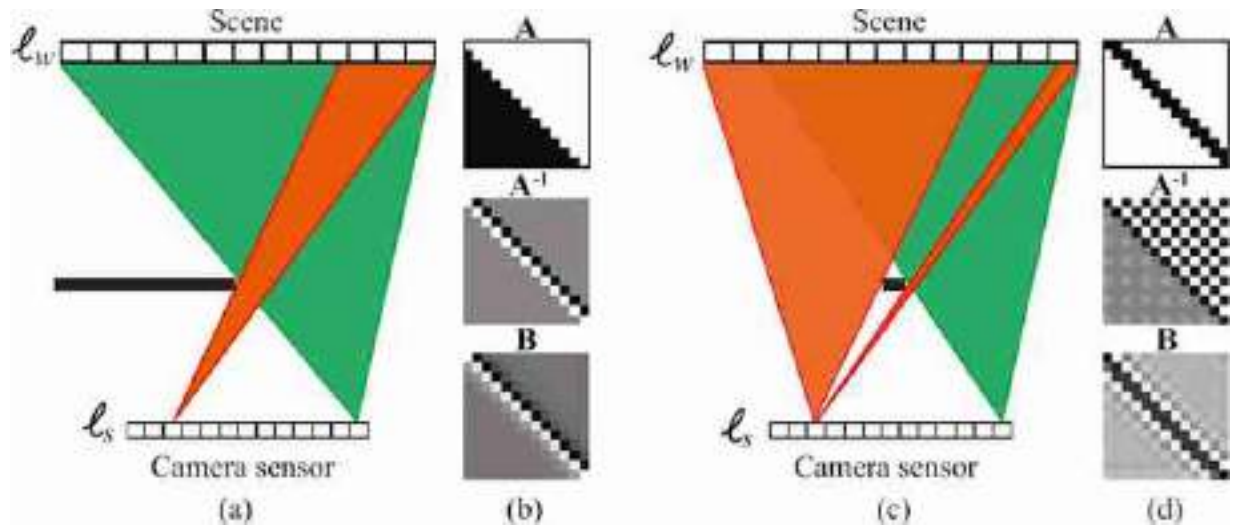


Figure 7.4: (a) Schematic drawing of an edge camera, and (b) its imaging matrices. (c) A pinspeck camera (an occluder that blocks two of the values on the scene), and (d) its imaging matrices.

7.4.1 Edge Camera

Consider the example of [figure 7.4\(a\)](#). This camera corresponds to an **edge camera**. This is not a traditional pinhole camera; instead light is blocked only on one side.

For this camera, the imaging matrix and its inverse are as follows:

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 0 & 1 & 1 & 1 & & 1 \\ 0 & 0 & 1 & 1 & & 1 \\ 0 & 0 & 0 & 1 & & 1 \\ \vdots & & & & \ddots & \\ 0 & 0 & 0 & 0 & & 1 \end{bmatrix}, A^{-1} = \begin{bmatrix} 1 & -1 & 0 & 0 & \dots & 0 \\ 0 & 1 & -1 & 0 & & 0 \\ 0 & 0 & 1 & -1 & & 0 \\ 0 & 0 & 0 & 1 & & 0 \\ \vdots & & & & \ddots & \\ 0 & 0 & 0 & 0 & & 1 \end{bmatrix} \quad (7.6)$$

If we consider the same input as in the pinhole camera example in the previous section, the output signal for the corner camera will look like ([figure 7.5](#)):

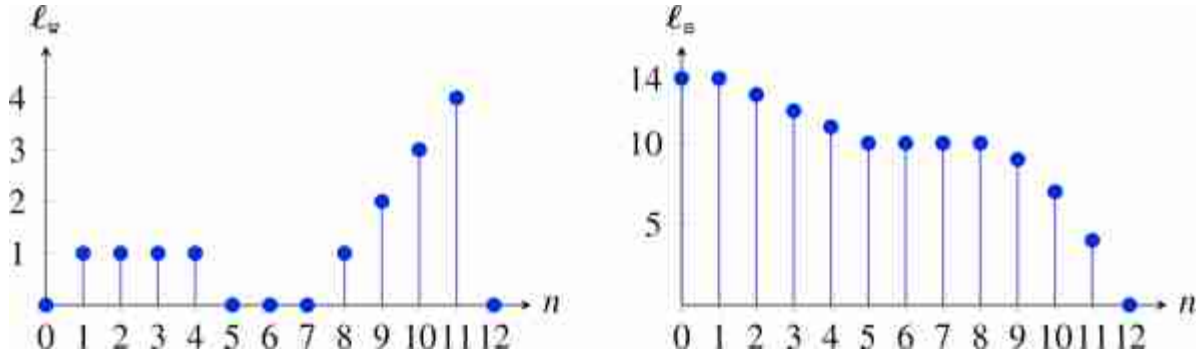


Figure 7.5: Edge camera. (left) Input 1D signal, ℓ_w . (right) The output of an edge camera, ℓ_s .

The output now looks very different than that of a pinhole camera. In the pinhole camera, the output is very similar to the input. This is not the case here where the output looks like the integral of the input (reversed along the horizontal axis). Note that the first value of ℓ_s is equal to the sum of all the values of ℓ_w :

$$\ell_s[0] = \sum_{n=0}^{12} \ell_w[n] \quad (7.7)$$

Figure 7.4(b) illustrates the imaging matrix, $\mathbf{A}$, and reconstruction matrices for the imager of equation (7.6). You can think of this imager as computing an integral of the input signal. Therefore, its inverse looks like a derivative. The regularized inverse looks like a blurred derivative (we will talk more about blurred derivatives in chapter 18).

7.4.2 Pinspeck Camera

Figure 7.4(c) shows another nontraditional imager. Now, instead of a pinhole we have an occluder. The occluder blocks part of the light (complementary to the large-hole pinhole camera shown in figure 7.2(c)). What the camera sensor records is the shadow of the occluder.

The shadow of an object is related to the negative of a picture taken of the environment around the object.

We can write the imaging matrix $\mathbf{A}$, which corresponds to $1 - \mathbf{A}_{\text{pinhole}}$ as shown in figure 7.2(d). Figure 7.4(d) also shows its inverse and the regularized inverse. This camera is called a **pinspeck camera** [81, 474] and has also been used in practice.

In the example of [figure 7.4\(c\)](#), the occluder has the size of the wide pinhole from [figure 7.2\(c\)](#). The following plots shows, [figure 7.6](#)(left), the input scene (same as in the previous examples) and, [figure 7.6](#)(right), the output recorded by the camera sensor of the pinspeck camera.

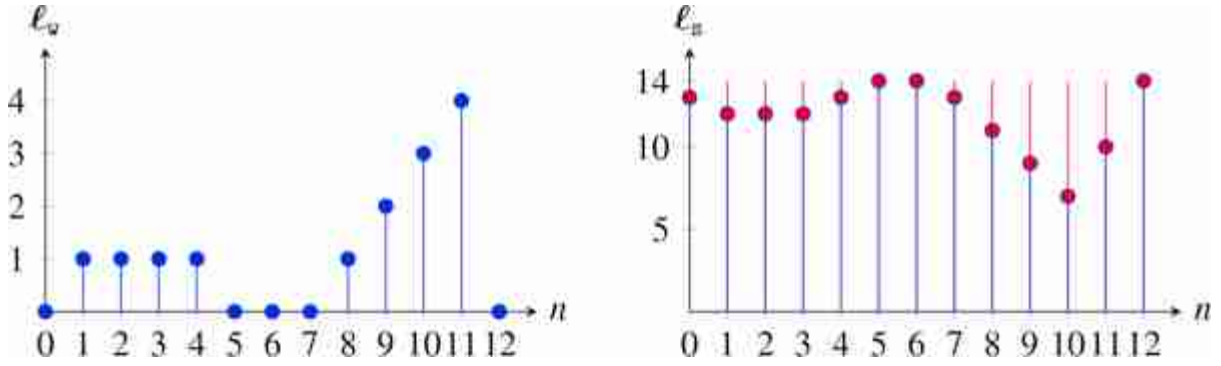


Figure 7.6: Pinspeck camera. (left) Input 1D signal, ℓ_w . (right) The output of a pinspeck camera, ℓ_s .

The output is now a signal with a wide dynamic range (a max value of 14, which correspond to the sum of the values in ℓ_w) and with fluctuations due to the shadow of the occluder on the camera sensor. If there was no occluder, then the output would be a constant signal of value 14. In red we show the effect of the shadow, which is the light missing because of the presence of the occluder. You can see how the missing signal is identical to the output of the two-pixel wide pinhole camera but reversed in sign.

7.4.3 Corner Camera

To show a real-world example of a more general imager, let us consider the **corner camera** [56]. This is similar to the edge camera of equation (7.6) and [figure 7.4](#), but with slightly more complicated geometry. As shown in [figure 7.7\(a\)](#), a vertical edge partially blocks a scene from view, creating intensity variations on the ground, observable by viewing those intensity variations from around the corner.

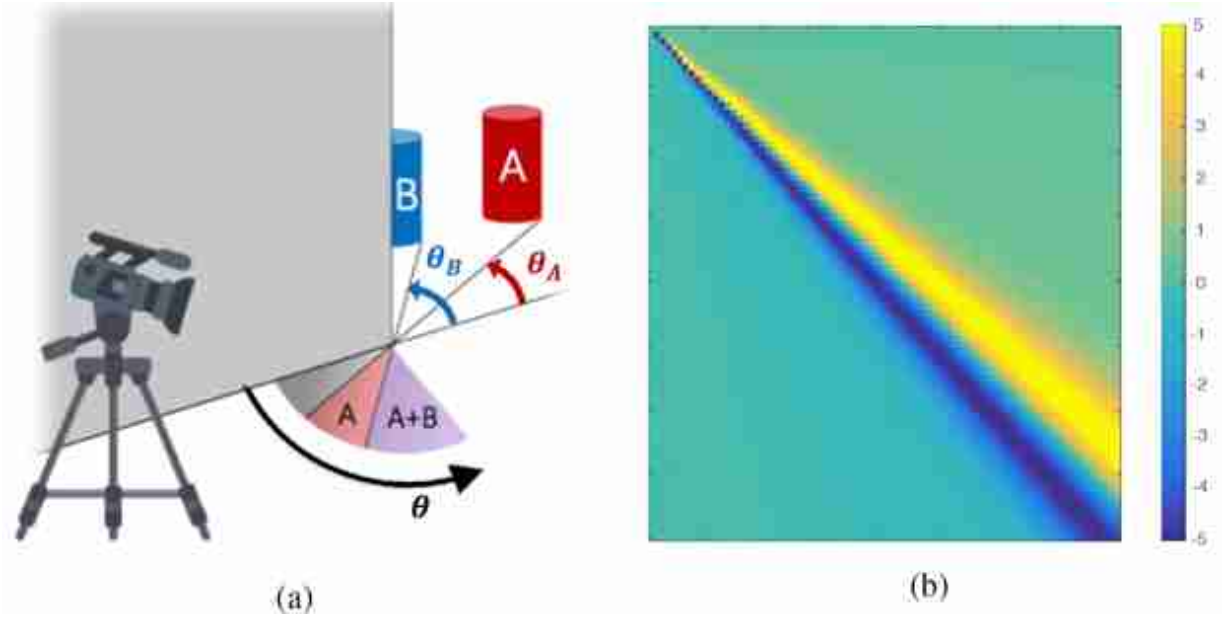


Figure 7.7: The corner camera [56]. (a) Objects, such as the cylinders labeled A and B, hidden behind a corner from a camera nonetheless cause a very small intensity differences in the ambient illumination falling on the ground. (b) the spatial mask is multiplied by the observed reflected ground image.

In practice, we will subtract a mean image from our observations of the ground plane, so in the rendering equation below, we will only consider components of the scene that may change over time, under the assumption that what we want to image behind the corner (e.g., a person) is moving. We will call these intensities $S(\phi, \theta)$ (S for the subject), where ϕ measures vertical inclination and θ measures azimuthal angle, relative to position where the vertical edge intersects the ground plane. Integrating the light intensities falling on the ground plane, the observed intensities on the ground will be $\ell_{\text{ground}}(r, \theta)$, where the polar coordinates r and θ are measured with respect to the corner. Assuming Lambertian diffuse reflection from the ground plane, we have, for the observed intensities $\ell_{\text{ground}}(r, \theta)$,

$$\ell_{\text{ground}}(r, \theta) = \int_{\phi=0}^{\phi=\pi} \int_{\xi=0}^{\xi=\theta} \cos(\phi) S(\phi, \xi) d\phi d\xi, \quad (7.8)$$

where the $\cos(\phi)$ term in equation (7.8) follows from the equation for surface reflection, equation (5.2).

The dependence of the observation, ℓ_{ground} , on vertical variations in $S(\phi, \theta)$ is very weak, just through the $\cos(\phi)$ term. We can integrate over ϕ first, to form the 1D signal, $\ell_w(\xi)$:

$$\ell_w(\xi) = \int_{\phi=0}^{\phi=\pi} \cos(\phi) S(\phi, \xi) d\phi \quad (7.9)$$

Then, to a good approximation, equation (7.8) has the form,

$$\ell_{\text{ground}}(r, \theta) \approx \int_{\xi=0}^{\xi=\theta} \ell_w(\xi) d\xi, \quad (7.10)$$

where $\ell_w(\xi)$ is a 1D image of the scene around the corner from the vertical edge.

[Figure 7.7](#) shows the corner camera geometry for a three-dimensional (3D) scene. [Figure 7.7\(a\)](#) shows two objects, such as the cylinders labeled A and B, hidden behind a corner from a camera. Despite being behind the corner they cause a very small intensity differences in the ambient illumination falling on the ground, observable from the camera as a very small change in the light intensity reflecting from the ground. Can we reconstruct the scene hidden behind the corner using just the intensities observed on the ground? The image reconstruction operation is analogous to that for the edge camera of [figure 7.4](#): we subtract the reflected intensities observed at one orientation angle, say θ_A from those observed at another, say θ_B .

If we sample equation (7.10) in its continuous variables, we can write it in the form $\ell_{\text{ground}} = \mathbf{A} \ell_w$. Solving equation (7.5) for the multiplier to apply to ℓ_{ground} to estimate ℓ_w yields the form shown in [figure 7.7\(b\)](#). [Figure 7.7\(b\)](#) shows the spatial mask to be multiplied by the observed reflected ground image, with the result summed over all spatial pixels in order to estimate the light coming from around the corner at one particular orientation angle relative to the corner, in this case approximately 45 degrees. We see that the way to read the 1D signal from the ground plane is to take a derivative with respect to angle. This makes intuitive sense, as the light intensities on the ground integrate all the light up to the angle of the vertical edge. To find the 1D signal at the angle of the edge, we ask, “What does one pie-shaped ray from the wall see that the pie-shaped ray next to it doesn't see?”

It can be shown [56] that the image intensities from around-the-corner scenes introduce a perturbation of about $\frac{1}{1,000}$ to the light reflected from the ground from all sources. By averaging image intensities over the appropriate pie-shaped regions on the ground at the corner (figure 7.7[b]), one can extract a 1D image as a function of time from the scene around the corner. Figure 7.8 shows two 1D videos reconstructed from one (figure 7.8[a]) and two (figure 7.8[b]) people walking around the corner. By processing videos of very faint intensity changes on the ground, we can infer a 1D video of the scene around the corner. The image inversion formulas were derived using inversion methods very similar to equation (7.5). The corner camera is just one of the many possible instantiations of a computational imaging system.

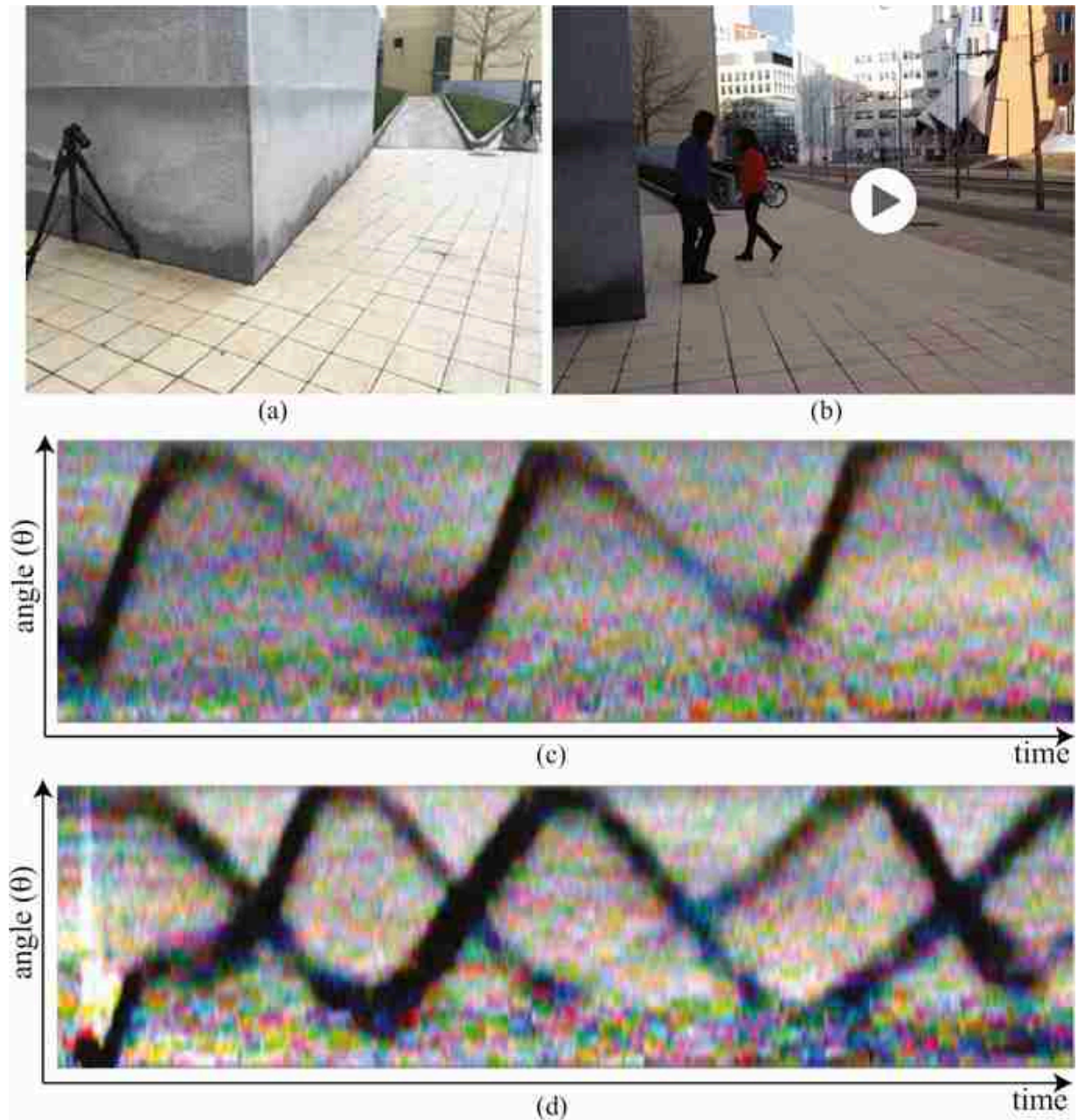


Figure 7.8: Outdoor corner camera [56] experiments. (a) Camera recording ground plane intensities. (b) Two people walking around the corner, hidden from direct view of the camera. (c) Corner camera trace with one person moving. (d) Corner camera trace with two people moving. Angle from the corner is plotted vertically, and time is plotted horizontally.

People moving out of view around a corner cause small, invisible changes in the light reflecting from the ground. These faint changes occur at nearly every corner and can be used to read-out 1D images of the people around the corner.

7.5 Concluding Remarks

Treating cameras as general linear systems allows for the machinery of linear algebra to be applied to camera design and processing. We reviewed several simple camera systems, including cameras utilizing pinholes, pinspecks, and edges to form images.

8 Color

8.1 Introduction

There are many benefits to sensing color. Color differences let us check whether fruit is ripe, tell whether a child is sick by looking at small changes in the color of the skin, and find objects in clutter.

We'll begin our study of color by describing the physical properties of light that lead to the perception of different colors. Then we'll describe how humans and machines sense colors, and how to build a system to match the colors perceived by an observer. We'll briefly describe how to represent color—different color coordinate systems. Finally, we'll describe how spatial resolution and color interact.

8.2 Color Physics

Isaac Newton revealed several intrinsic properties of light in experiments summarized by his drawing in [figure 8.1](#). A pinhole of sunlight enters through the window shade, and a lens focuses the light onto a prism. The prism then divides the white light into many different colors. These colors are elemental: if one of the component colors is passed through a second prism, it doesn't split into further components.

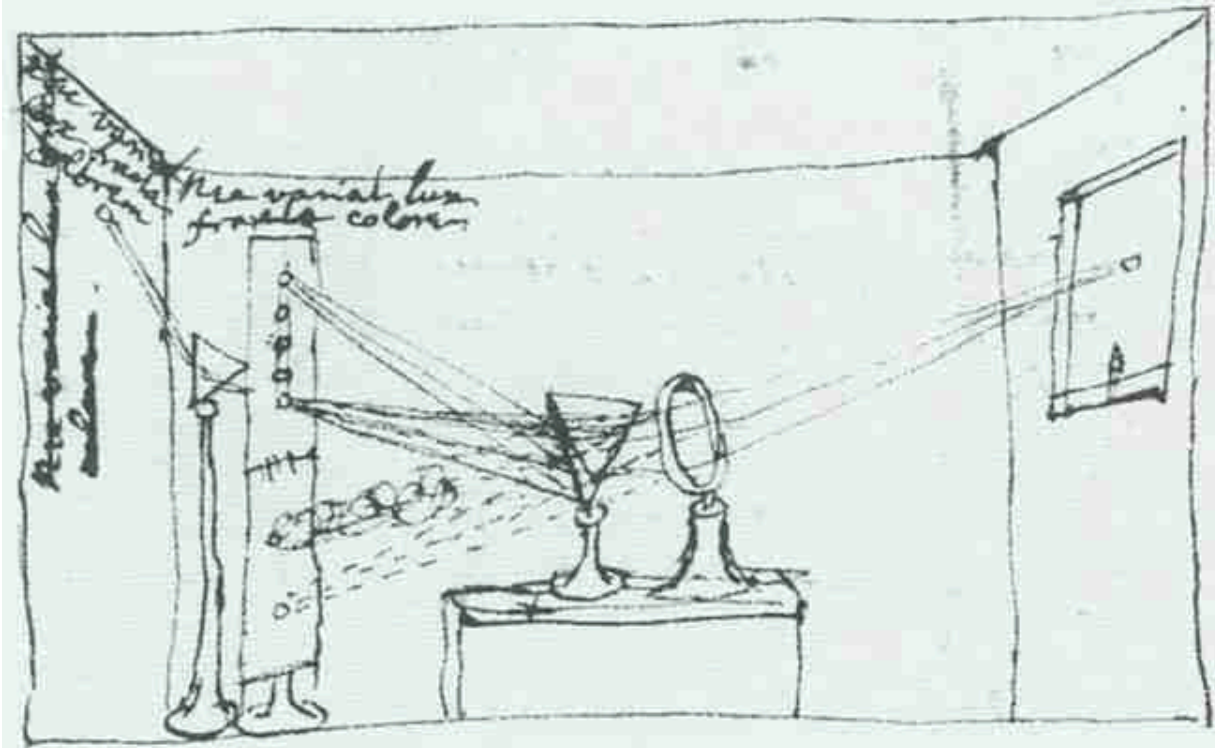


Figure 8.1: Isaac Newton's illustration of experiments with light [119]. White light enters from a hole in the window shade at the right, where it is focused with a lens and then passes through the first prism. The prism separates the white light into different colors by bending each color a different amount. The second prism in the drawing demonstrates that those colors are elemental: as an individual color passes through the second prism, the light doesn't break into other colors.

Our understanding of light and color explains such experiments. Light is a mixture of electromagnetic waves of different wavelengths. Sunlight has a broad distribution of light of the visible wavelengths. At an air/glass interface, light bends in a wavelength-dependent manner, so a prism disperses the different wavelength components of sunlight into different angles, and we see different wavelengths of light as different colors. Our eyes are sensitive to only the narrow band of that electromagnetic spectrum, the visible light, from approximately 400 nm to 700 nm, which appears blue to deep red, respectively.

The bending of light at a material boundary is called **refraction**, and its wavelength dependence lets the prism separate white light into its component colors. A second way to separate light into its spectral components is through **diffraction**, where constructive interference of

scattered light occurs in different directions for different wavelengths of light.

Newton understood that white light could be decomposed into different colors and invented the term **spectrum**.

[Figure 8.2](#)(a) shows a simple spectrograph, an apparatus to reveal the spectrum of light, based on diffraction from a compact disk (CD) [[105](#)]. Light passes through the slit at the right, and strikes a CD (with a track pitch of about 1,600 nm). Constructive interference from the light waves striking the CD tracks will occur at a different angle for each wavelength of the light, yielding a separation of the different wavelengths of light according to their angle of diffraction. The diffracted light can be viewed, [figure 8.2](#)(b), or photographed through the hole at the bottom left of the photograph. The spectrograph gives an immediate visual representation of the spectral components of colors in the world. Some examples are shown in [figure 8.3](#).

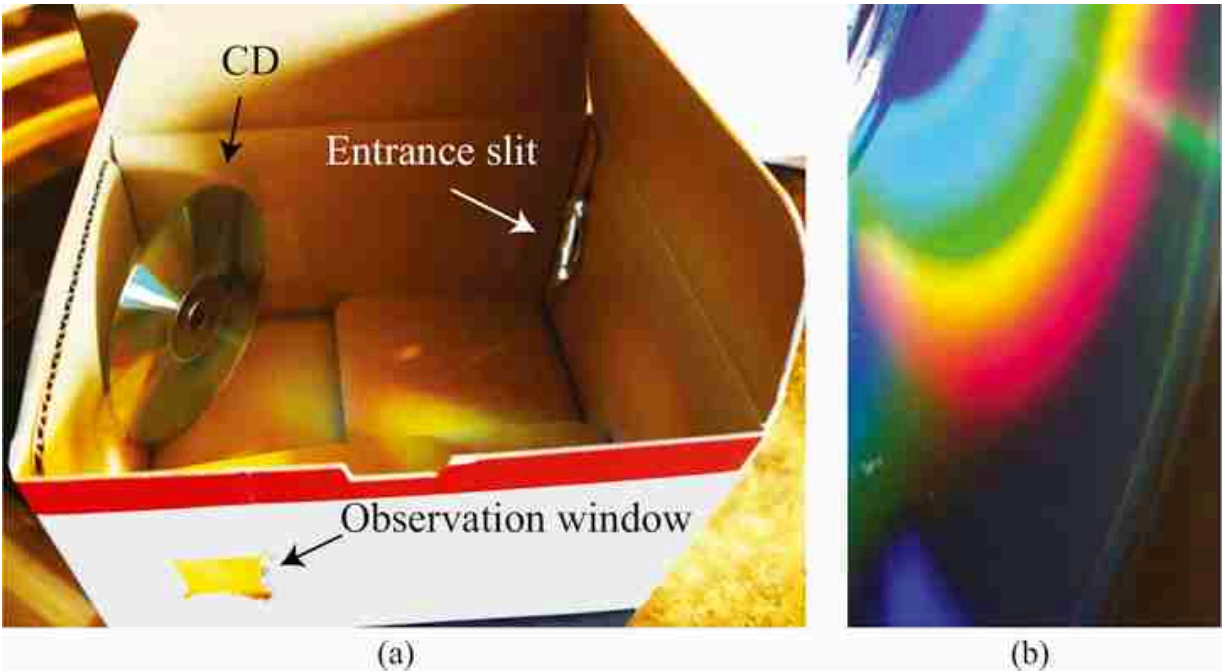


Figure 8.2: (a) A simple spectrograph. Slit at right accepts light from primarily one object in the world. Light diffracted by the CD is viewed from the hole at the bottom left. (b) The bending by diffraction is wavelength dependent, and the light from a given direction is broken into its spectral components. We indicate the location of the CD in the picture just in case our youngest readers have not seen one.

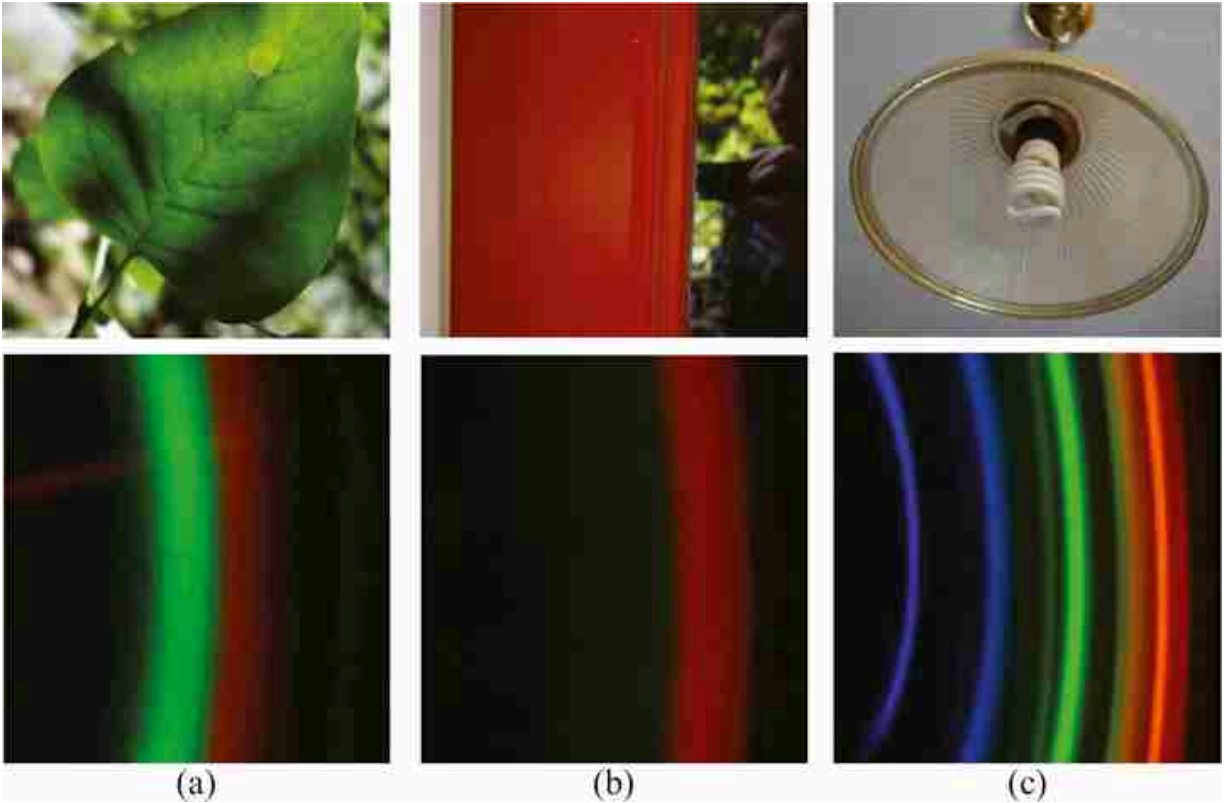


Figure 8.3: The light spectra from some everyday objects, analyzed by the spectrograph of [figure 8.2](#). (a) A leaf, with some yellowish highlights, shows primarily green, with a little red (red and green can combine to appear yellow). (b) A red door. (c) A fluorescent light (when turned on) shows the discrete spectral wavelengths at which the gas fluoresces.

8.2.1 Light Power Spectra

The light intensity at each wavelength is called the **power spectrum** of the light. The color appearance of light is determined by many factors, including the image context in which the light is viewed; but a very important factor in determining color appearance is the power spectrum. In this initial discussion of color, we will assume that the power spectrum of light determines its appearance, although you should remember that this is true only within a fixed visual context.

Why does the sky look blue? And why does it look orange during a sunset?



[Figure 8.3](#) shows a spectrograph visualization of some light power spectra (the right image of each row) along with the image that the spectrograph was pointed toward (left images). [Figure 8.4](#) shows the spectrum of a blue sky, plotted as intensity as a function of wavelength.

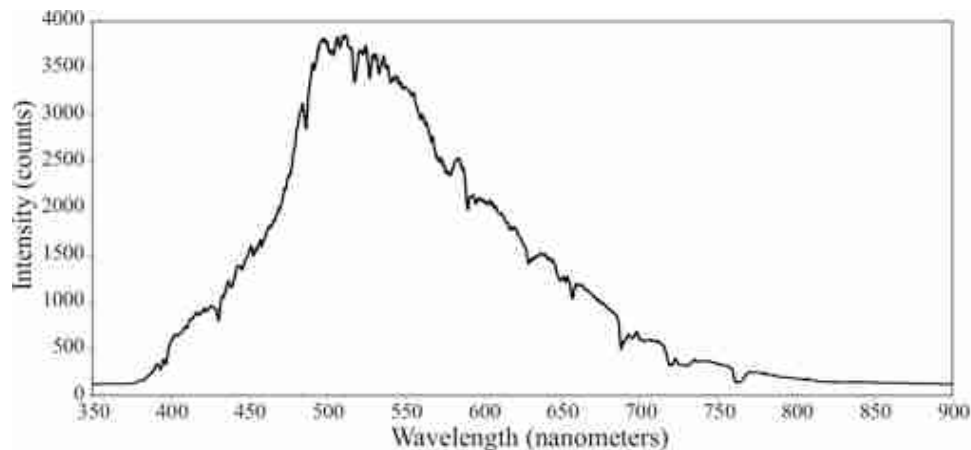


Figure 8.4: Power spectrum of blue skylight [99].

8.2.2 The Color Appearance of Different Spectral Bands

It is helpful to develop a feel for the approximate color appearance of different light spectra. Again, we say the approximate appearance because the subjective appearance can change according to other factors than just the spectrum.

The visible spectrum lies roughly in the range between 400 and 700 nm, see [figure 8.5](#). We can divide the visible spectrum into three 100 nm bands, and study the appearance of light power spectra where power is present or

absent from each of those three bands, in all of the eight (2^3) possible combinations.

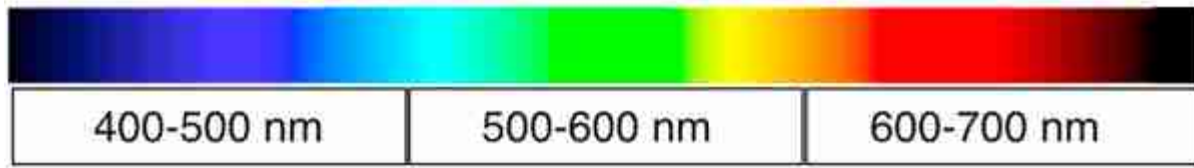


Figure 8.5: The approximate color appearance of light over different spectral regions.

Light with spectral power distributed in just the 400–500 nm wavelength band will look some shade of blue, the exact hue depending on the precise distribution. Light in the 500–600 nm band will appear greenish. Most distributions within the 600–700 nm band will look red.

White light is a mixture of all spectral colors. A spectrum of light containing power evenly distributed over 400–700 nm would appear approximately white. Light with no power in any of those three bands, that is, darkness, appears black.

There are three other spectral classes left in this simplified grouping of spectra: spectral power present in two of the spectral bands, but missing in the third. Cyan is a combination of both blue and green, or roughly spectral power between 400 and 600 nm. In printing and color film applications, this is sometimes called *minus red*, since it is the spectrum of white light, minus the spectrum of red light. The blue and red color blocks, or light in the 400–500 nm band, and in the 600–700 nm band, is called magenta, or minus green. Red and green together, with spectral power from 500–700 nm, make yellow, or minus blue ([figure 8.6](#)).

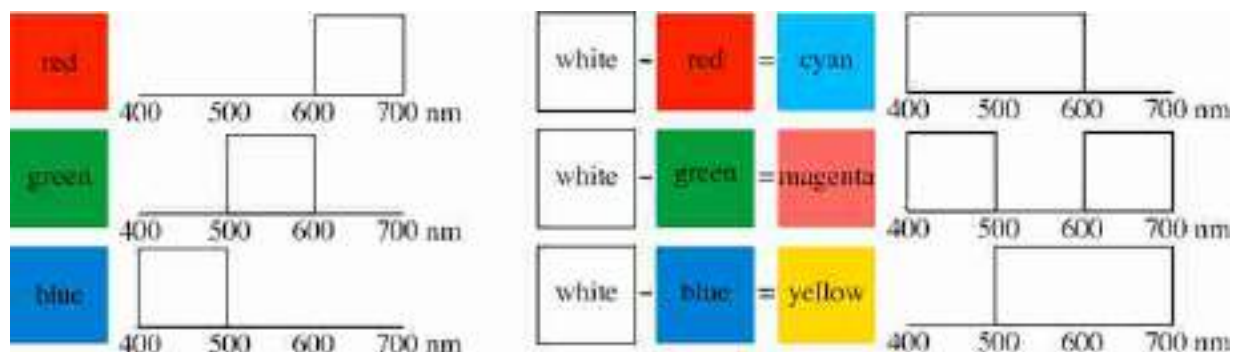


Figure 8.6: For coarse orientation only, this cartoon model gives the color appearance of different spectral bands of the spectrum of visible light.

8.2.3 Light Reflecting from Surfaces

When light reflects off a surface, the power spectrum alters in ways that depend on the surface's characteristics and geometry. These changes in light allow us to perceive objects and surfaces by observing their influence on the reflected light.

The interaction between light and a surface can be quite complex. Reflections can be specular or diffuse, and the reflected power spectrum may depend on the relative orientations of the incident light, surface, and the observed reflected ray. In its full generality, the reflection of light from a surface is described by the bidirectional reflectance distribution function (BRDF) [355, 324]. For this discussion, we will focus on diffuse surface reflections, where the power spectrum of the reflected light, $r(\lambda)$, is proportional to the wavelength-by-wavelength product of the power spectrum of the incident light, $\ell_{\text{in}}(\lambda)$, and a **reflectance spectrum**, $s(\lambda)$, of the surface:

$$r(\lambda) = k \ell_{\text{in}}(\lambda) s(\lambda), \quad (8.1)$$

where the proportionality constant k depends on the reflection geometry. This diffuse reflection model characterizes most matte reflections. Such wavelength-by-wavelength scaling is also a good model for the spectral changes to light caused by transmission through an attenuating filter. The incident power spectrum is then multiplied at each wavelength by the **transmittance spectrum** of the attenuating filter.

Some reflectance spectra of real-world surfaces are plotted in [figure 8.7](#). The flower *Hepatica Nobilis* (solid line) is blue, while *Pyrostegia venusta* (dotted line) is orange.

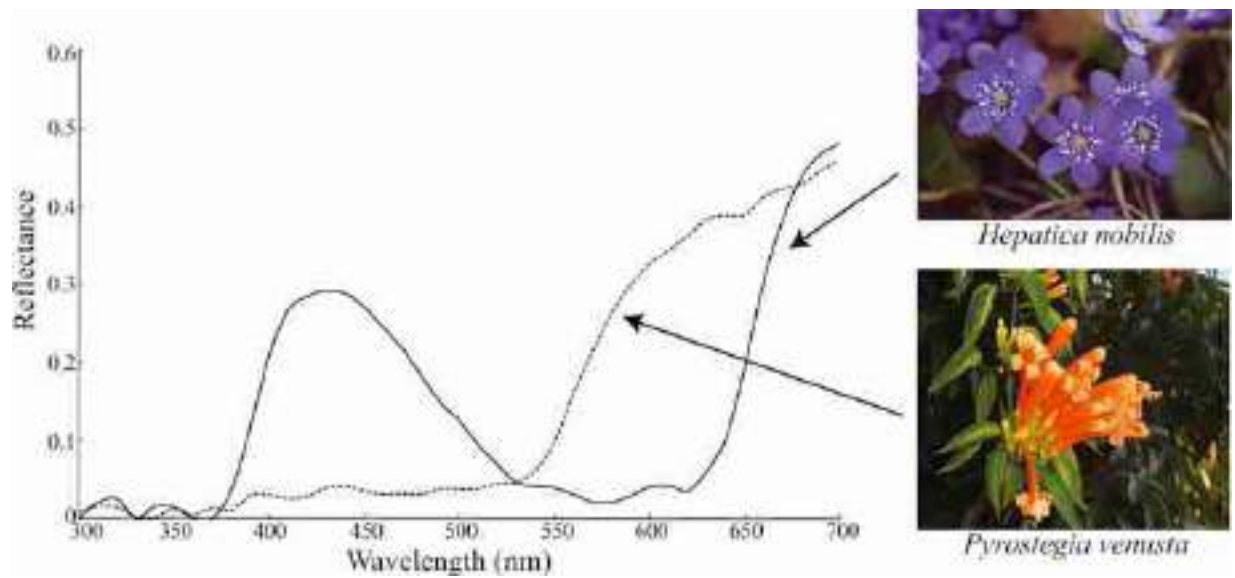


Figure 8.7: Reflectance spectra from two flowers [21]. The blue *Hepatica nobilis* flower [153] and the orange flower, *Pyrostegia venusta* [349].

The reflectance spectrum of a surface often carries valuable information about the material being viewed, such as whether a fruit is ripe, whether a human's skin is healthy, or whether the material differs from another viewed material. It may be the case that a low-dimensional version of the surface reflectance spectrum is sufficient for many visual tasks, and as we see subsequently, the human visual system represents color with only three numbers. So it is an important visual task to estimate either the surface reflectance spectrum, or a low-dimensional summary of it.

To estimate surface colors by looking, a vision system has the task of estimating the illumination spectrum and the surface reflectance spectra, from observations of only their products. When the illumination is white light with equal power in all spectral bands, the observed reflected spectrum is proportional to the reflectance spectrum of the material itself. However, under the more general condition of unknown illumination color, a visual system will need to estimate the surface reflectance spectrum, or projections of it, by taking the context of nearby image information into account.

There are many proposed algorithms to do that, ranging from heuristics, for example, assuming the color of the brightest observed object is white [328], to statistical methods [59] and neural network based approaches [37]. Even humans don't solve the problem perfectly or consistently, revealed

especially through the internet meme of #TheDress [54], shown in [figure 8.8](#).

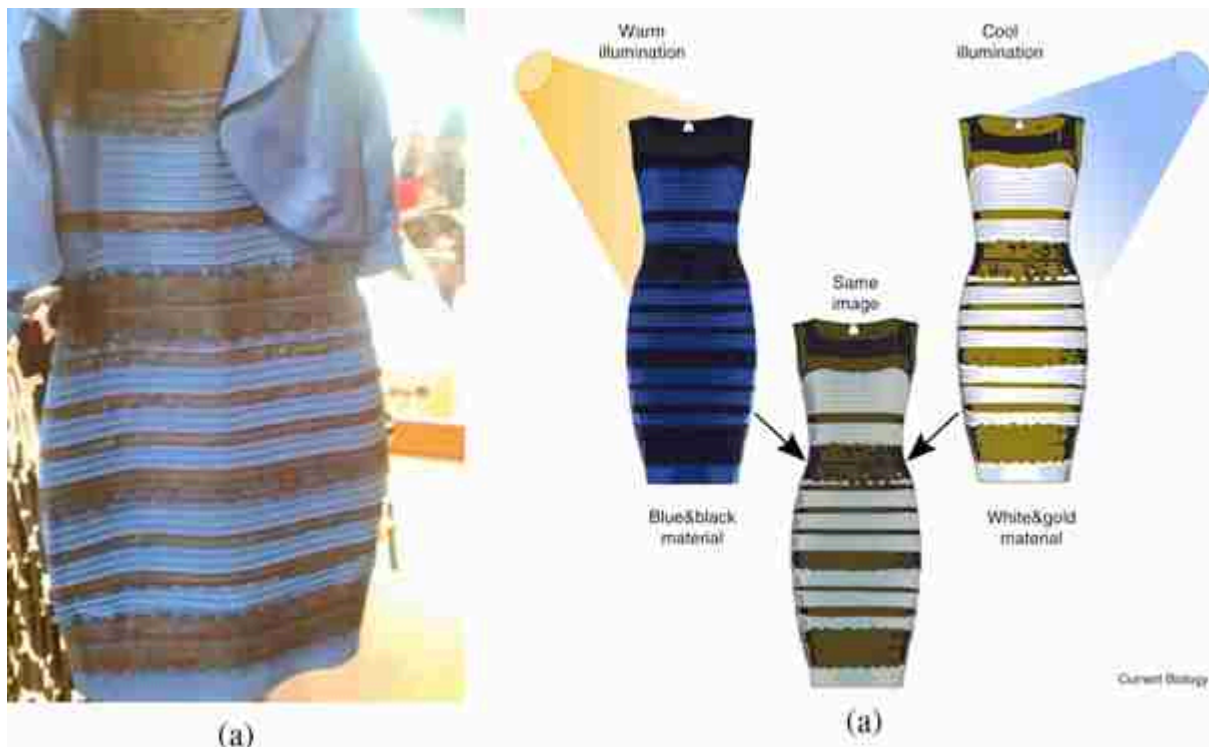


Figure 8.8: (a) #The Dress, photographed and posted by [54], reprinted with permission. (b) The spectra of the light we see is the product of an assumed illumination spectrum and the inferred surface reflectance spectra. If people make different assumptions about the illumination, they can perceive different colors within the same image, as shown in this illustration [60]. Reprinted with permission from Elsevier.

The perceived colors of the dress depend on the assumed color of the illumination, and people can disagree significantly about the colors they see [60]. The assumption of a warm, yellowish, illumination color leads to a perception of blue and black material. The assumption of a cool, blueish illumination leads to a perception of white and gold material. Both perceptions are consistent with the observed product of illumination and reflectance seen in the dress image.

Did you perceive the dress, [figure 8.8](#), to be black on blue, or gold on white?
Can you make the perception change?

While such context-dependent effects are important in color perception, as we quantify color perception we will assume that our perception of color depends only on the spectrum of light entering the eye. This will serve us well for many industrial applications.

8.3 Color Perception

Now we turn to our perception of color. We first describe the machinery of the eye, then describe some methods to measure color appearance. These methods to measure color appearance implicitly assume a white illumination spectrum, but they often serve the needs of industry and science.

8.3.1 The Machinery of the Eye

An instrument such as a spectrograph ([figure 8.2](#) shows a simple one) can measure the light power spectrum at hundreds of different wavelengths within the visible band, yielding hundreds of numbers to describe the light power spectrum. Despite this, a useful description of the visual world can be obtained from a much lower dimensional description of the light power spectrum.

The retina contains photoreceptors called the rod and cones. Rods are used in low-light levels, and the cones are used in color vision. In low-light, only the rods operate and our vision becomes black and white.

The human visual system analyzes the incident light power spectrum with only three different classes photoreceptors, called the L, M, and S cones because they sample at the long, medium, and short wavelengths. This gives the human visual system a three-dimensional (3D) description of light, with each photoreceptor class taking a different weighted average of the incident light power spectrum.

[Figure 8.9\(a\)](#) shows the spatial sampling pattern, for each of the three cone classes, measured from a subject's eye [\[210\]](#). The L cones are colored red, the M cones green, and the S cones are colored blue. Note the hexagonal packing of the cones in the retina, and the stochastic assignment of L, M, and S cones over space. Note also the much sparser spatial sampling of the S cones than of either the L or M cones. [Figure 8.9\(b\)](#) shows the spectral sensitivity curves for the L, M, and S cones. This

sampling pattern was measured from near the **fovea**, where there are no rods, and the cones are close-packed.

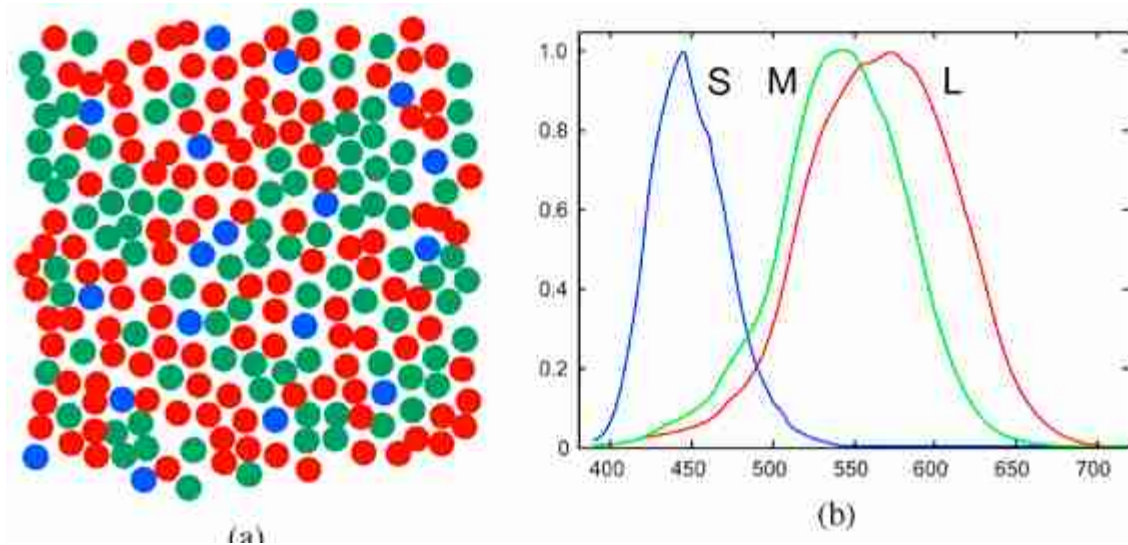


Figure 8.9: (a) Measured cone receptor classes and positions in a human retina, with cone receptor classes shown as red, green, and blue (subject YY, redrawn from [210]). (b) Photoreceptor sensitivities as a function of light wavelength [486].

We can describe the photoreceptor responses using matrix algebra. If the matrix C_{eye} consists of the spectral sensitivities of the L, M, and S cones in three rows, and the vector $\mathbf{t}$ is a column vector of the spectrum of light incident on the eye, then the L, M, and S cone responses will be the product,

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = C_{\text{eye}} \mathbf{t} \quad (8.2)$$

The fact that our eyes have three different classes of photoreceptors has many consequences for color science. It determines that there are three primary colors, three color layers in photographic film, and three colors of dots needed to render color on a display screen. In the next section, we describe how to build a color reproduction system that matches the colors seen in the world by a human observer.

8.3.2 Color Matching

Color science tells us how to analyze and reproduce color. We seek to build image displays so that the output colors match those of some desired target, and to manufacture items with the same colors over time. Much of the color industry revolves around the ability to repeatably control colors. Colors can be trademarked (Kodak Yellow, IBM Blue) and we have color standards for foods. [Figure 8.10](#) shows French fry color standards.



Figure 8.10: The USDA color standards for French fried potatoes, one of many color standards [373].

One of the tasks of color science is to predict when a person will perceive that two colors match. For example, we want to know how to adjust a display to match the color reflecting off some colored surface. Even though the spectra may be very different, the colors can almost always be made to match.

It is possible to infer human color matching performance by examining the spectral sensitivity curves of the receptors shown in [figure 8.9\(b\)](#). We attempt to match a color using a combination of reference colors, typically called primary colors. Through experimentation, it has been discovered that the appearance of any color can be matched through a linear combination of three primary colors. This is due to the presence of three classes of

photoreceptors in our eyes. It has also been found that these color matches are transitive—if color A matches a particular combination of primaries and color B matches the same combination of primaries, then color A will match color B. Consequently, the amount of each primary required to match a color can serve as a set of coordinates indicating color [\[491\]](#).

8.3.3 Color Metamerism

Two different spectra are metameric if they appear to be the same color to our eyes. There is a large space of metamers: any two vectors describing light power spectra that give the same projection onto a set of color matching functions will look the same to our eyes. There's a high-dimensional space of light spectra, and we're only viewing projections onto a 3D subspace in the colors we see.

Do spotlights on produce in a supermarket help good and bad fruit become metameric matches?

In practice, the three projections we observe capture much of the interesting action in images. Hyperspectral images (recorded at many different wavelengths of analysis) add some, but not a lot, to the image formed by our eyes.

8.3.4 Linear Algebraic Interpretation of Color Matching

In this chapter, we assume that color appearance is determined by the light spectrum reaching the eye. In reality, numerous factors can influence color appearance, including the eye's state of brightness adaptation, ambient illumination, and surrounding colors. However, a color matching system that relies solely on the light spectrum will still perform well.

To measure the color associated with a light spectrum $\mathbf{t}$ we need to be able to predict the eye's responses to the spectrum. From equation (8.2), the task of color measurement is to find the projection of a given light spectrum into the special 3D subspace defined by the eye's cone spectral response curves. Any basis for that 3D subspace will serve that task, so the three basis functions do not need to be the color sensitivity curves of [figure 8.9](#) themselves. They can be any linear combination of them, as well.

We can define a **color reproduction system**, [figure 8.11](#), by first specifying its three spectral sensitivity curves. We put these into the $3 \times N$ matrix, $\mathbf{C}$, where N is the number of spectral samples over the visible

illumination range. As discussed previously, the three curves, $\mathbf{C}$, should be a linear combination of the eye's spectral sensitivity curves, or

$$\mathbf{C} = \mathbf{R} \mathbf{C}_{\text{eye}}, \quad (8.3)$$

where $\mathbf{R}$ is any full-rank 3×3 matrix. We can translate between any two different color spaces by applying a general 3×3 matrix transformation to change basis vectors. Note, the basis vectors do not need to be orthogonal, and most color system basis vectors are not.

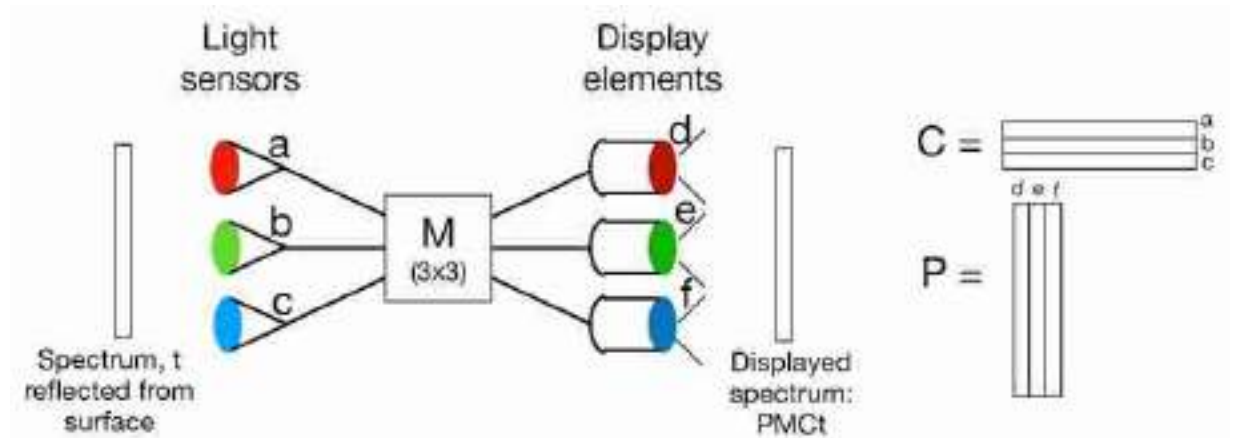


Figure 8.11: A color reproduction system. Light sensors a , b , and c respond to the input light spectrum $\mathbf{t}$, giving sensor activations $\mathbf{C}\mathbf{t}$. Combinations of those activations drive the display elements d , e , and f , producing the output spectrum $\mathbf{PM}\mathbf{C}\mathbf{t}$. Conditions on $\mathbf{P}$, $\mathbf{M}$, and $\mathbf{C}$ can ensure that the input and output colors match for an observer.

8.3.5 Color Matching Functions and Primary Lights

A color reproduction system measures an input light and produces an output light that matches the color appearance of the input light. A camera with a screen display is an example of a color reproduction system, as the colors of objects in the world are measured, then reproduced on the screen display of the camera. We can match the eye's response to a given light spectrum through the appropriate control of a sum of three lights, which we'll call the **primary lights**. For the example of a display screen, the three color elements of each pixel are the primary lights.

Because the eye's photosensors respond in linear proportion to the amount of the incoming light spectrum within its spectral sensitivity curve, the rules of linear algebra apply to color manipulation. For a given set of three primary lights, the strengths of each primary light can be adjusted to obtain a visual match to a desired color. There is one exception to this:

because primary lights can only be combined in positive values, some input colors are outside the **gamut**—the range of colors that can be produced—of a the positive combination of a given set of primary lights.

The color reproduction system is then defined by two sets of spectral curves and a 3×3 matrix, $\mathbf{M}$, see [figure 8.11](#). The first set are the spectral sensitivities as a function of wavelength for each of the three photosensors. We write each of those spectral curves as a row vector of a matrix, $\mathbf{C}$. The 3×3 matrix, $\mathbf{M}$, translates any color measurement to a set of amplitude controls for the primary lights, $\mathbf{P}$. The second set of spectral curves of a color reproduction system are the spectra of the three primary lights, which we write as column vectors of the matrix, $\mathbf{P}$.

For a given set of primary lights, $\mathbf{P}$, we seek to find a matrix of color sensitivity curves, $\mathbf{C}$, and 3×3 matrix, $\mathbf{M}$, which allow perfect reproduction of colors, as viewed by a human observer. Here, we derive the conditions that $\mathbf{P}$, $\mathbf{C}$, and $\mathbf{M}$ must satisfy to allow for perfect reproduction of colors.

Let the spectrum of the light reflecting from the surface to be matched in color be $\mathbf{t}$. The eye's response to that light is modeled as the sum of a term-by-term multiplication of the spectral sensitivities of each photoreceptor, in the rows of $\mathbf{C}_{eye}$ times the spectrum of the light. Thus, the responses of the eye to that spectrum will be the 3×1 column vector, $\mathbf{C}_{eye}\mathbf{t}$.

We can write the response of the eye to the *output* of the color matching system as the following matrix products. The light impinging on the sensors (a, b, c in [figure 8.11](#)) of the color matching system will give responses $\mathbf{Ct}$ and controls $\mathbf{MCt}$ to the primary lights. If this output modulates the corresponding primary lights (d, e, and f in [figure 8.11](#)), then the light displayed by the color matching system will be $\mathbf{PMCt}$. We want this spectrum to give the same responses in the eye as the original reflected color, thus we must have: $\mathbf{C}_{eye}\mathbf{PMCt} = \mathbf{C}_{eye}\mathbf{t}$. Because that relation must hold for *any* input vector $\mathbf{t}$, we must have

$$\mathbf{C}_{eye}\mathbf{PMC} = \mathbf{C}_{eye} \quad (8.4)$$

What conditions on $\mathbf{P}$ and $\mathbf{C}$ are required for equation (8.4) to hold? First, the subspace of the eye responses $\mathbf{C}_{eye}$ must be the same as that measured by the color sensing system, $\mathbf{C}$. If that weren't the case, then perceptibly different spectra could map onto the same color sensing system measurements. So we must have $\mathbf{C} = \mathbf{RC}_{eye}$, for some full-rank 3×3

matrix, $\mathbf{R}$ (with inverse $\mathbf{R}^{-1}$). Substituting this twice into equation (8.4) gives

$$\mathbf{R}^{-1} \mathbf{C} \mathbf{P} \mathbf{M} \mathbf{R} \mathbf{C}_{eye} = \mathbf{C}_{eye} \quad (8.5)$$

From equation (8.5) it follows that $\mathbf{C} \mathbf{P} \mathbf{M} = \mathbf{I}_3$, the 3×3 identity matrix. These conditions on the spectral sensitivities, $\mathbf{C}$, the display spectra, $\mathbf{P}$ and the sensors-to-primaries mixing matrix, $\mathbf{M}$ for a color reproduction system are summarized below:

$$\mathbf{C} = \mathbf{R} \mathbf{C}_{eye} \quad (8.6)$$

$$\mathbf{M} = (\mathbf{C} \mathbf{P})^{-1}, \quad (8.7)$$

where $\mathbf{C}_{eye}$ are the human photoreceptor sensitivities and $\mathbf{R}$ is a full-rank 3×3 matrix.

[Figure 8.12](#) displays the elements of two matrices $\mathbf{C}$ and $\mathbf{P}$ that define a valid color matching system. For a color reproduction system to be physically realizable, the elements of the primary spectra, $\mathbf{P}$, must be non-negative.

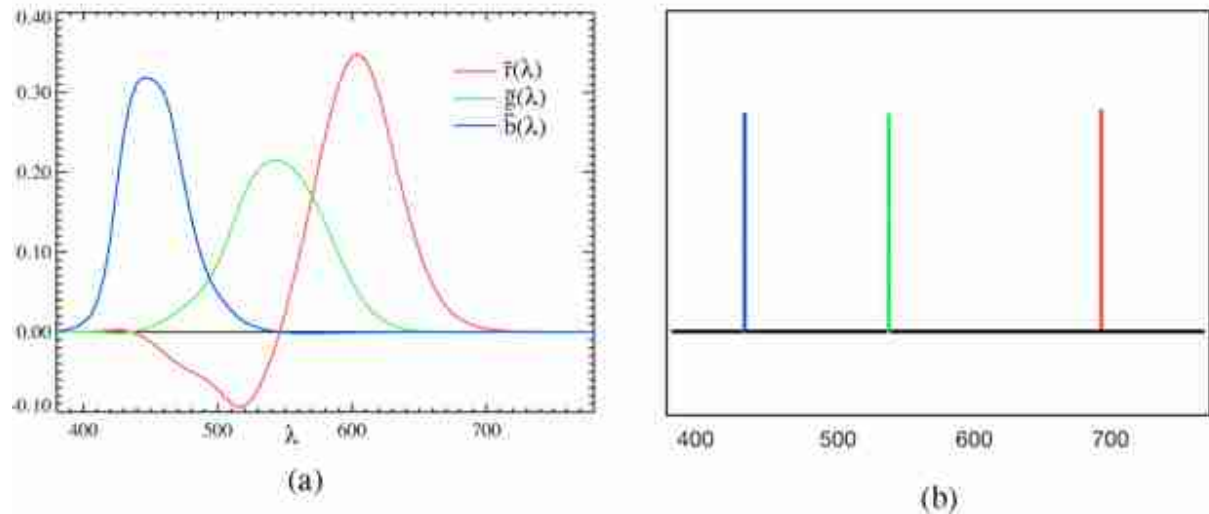


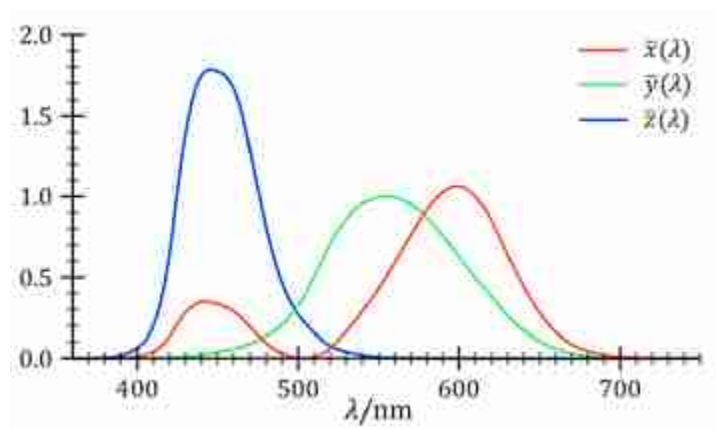
Figure 8.12: The elements of the matrices $\mathbf{C}$ and $\mathbf{P}$ for an example color matching system. (a) Spectral sensitivity curves, the rows of a color measurement matrix, $\mathbf{C}$. These should be linear combinations of the eye's photosensitivity curves, $\mathbf{C}_{eye}$. (b) Corresponding primary lights, $\mathbf{P}$ [4], satisfying $\mathbf{C} \mathbf{P} = \mathbf{I}_3$ ($\mathbf{M} = \mathbf{I}_3$ in equation [8.7]).

Because many color sensitivity matrices $\mathbf{C}$ can satisfy equation (8.6) some standardized color sensitivity matrices have been adopted to allow

common representations of colors.

8.3.6 CIE Color Space

A color space is defined by the matrix, $\mathbf{C}$, with three rows of color sensitivity functions. These three sensitivity functions, $\mathbf{C}$, must be some linear combination of the sensitivity functions of the eye, $\mathbf{C}_{\text{eye}}$. One color standard is the Commission Internationale de l'Eclairage (CIE) XYZ color space, $\mathbf{C}_{\text{CIE}}$. The CIE color matching functions, the rows of $\mathbf{C}_{\text{CIE}}$ were designed to be all-positive at every wavelength and are shown in [figure 8.13](#).



[Figure 8.13](#): CIE color matching functions [80].

An unfortunate property of the CIE color-matching functions is that no all-positive set of color primaries, $\mathbf{P}_{\text{CIE}}$ forms a color-matching system with those color-matching functions, $\mathbf{C}_{\text{CIE}}$. But $\mathbf{C}_{\text{CIE}}$ is a valid matrix with which to measure colors, even though there is no physically realizable set of corresponding color primaries, $\mathbf{P}_{\text{CIE}}$.

To find the CIE color coordinates, one projects the input spectrum onto the three color-matching functions, to find coordinates, called tristimulus values, labeled X , Y , and Z . Often, these values are normalized to remove overall intensity variations, and one calculates **CIE chromaticity coordinates** $\bar{x} = \frac{X}{X+Y+Z}$ and $\bar{y} = \frac{Y}{X+Y+Z}$.

8.4 Spatial Resolution and Color

Color interacts with our perception of spatial resolution. For some directions in color space, the eye is very sensitive to fine spatial modulations, while for other color space directions, the eye is relatively insensitive. Some color coordinate systems take advantage of this disparity to enable efficient representation of images by sampling image data sparsely along color axes where human perception is insensitive to blurring.

In a red-green-blue (RGB) representation, the eye is sensitive to high spatial frequency changes in both red and green, as shown in [figures 8.14 and 8.15](#). A rotation of the color coordinates of the image, followed by a nonlinear stretching, can put the image into a color space called L, a, b. In that space, the eye is very sensitive to any blurring of the L color component, called luminance, but is relatively insensitive to blurring of the a or b components, as demonstrated in [figure 8.16 and 8.17](#). This effect is commonly exploited in image compression, allowing some image components to be sampled more coarsely than others.

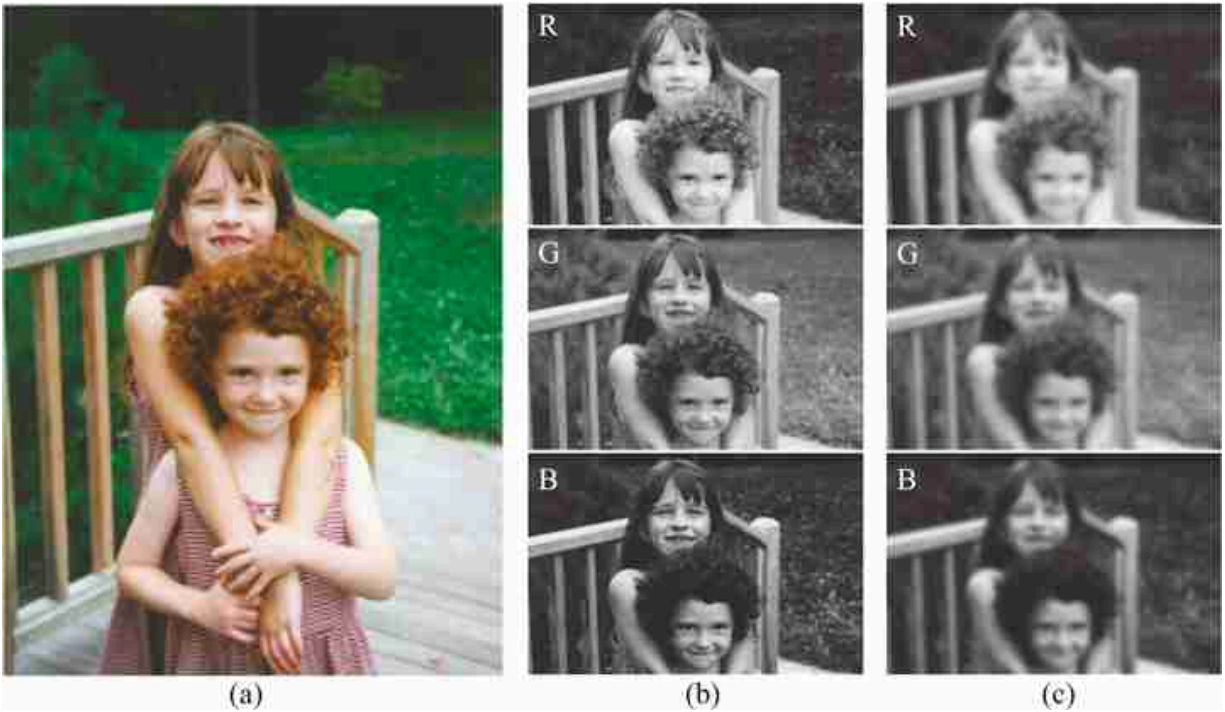


Figure 8.14: (a) Original image. (b) RGB components. (c) RGB components, each blurred. These sharp and blurred components are used in the color images of [figure 8.15](#).

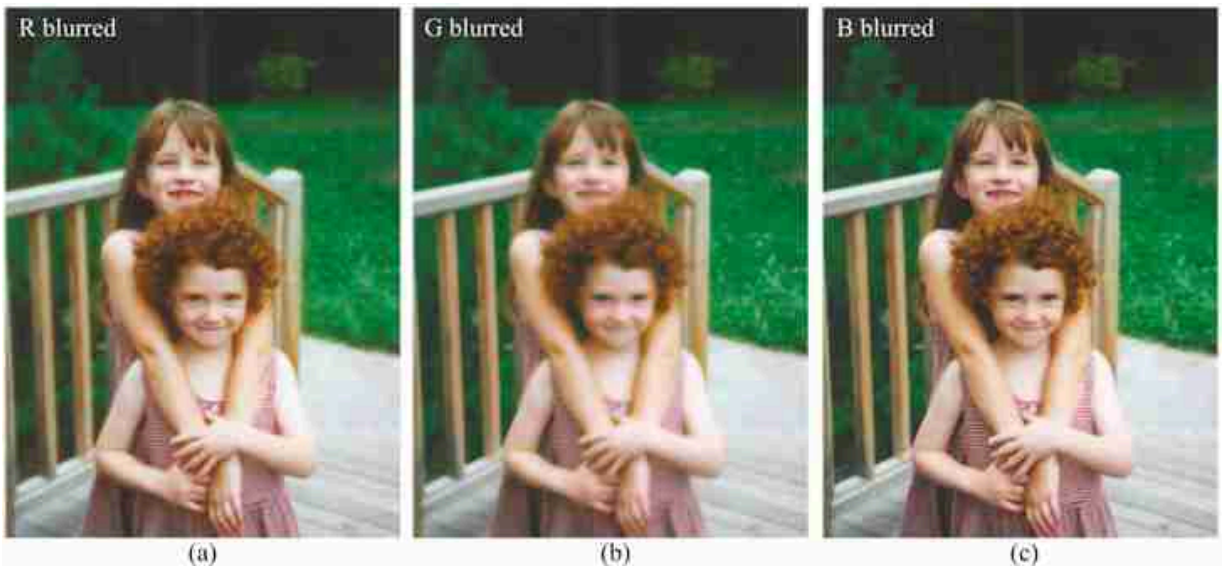


Figure 8.15: Color images composed of sharp and blurred components from [figures 8.14\(b\)](#) and [8.14\(c\)](#). (a) R component blurred, G and B components sharp. (b) G component blurred, R and B components sharp. (c) B component blurred, R and G components sharp. Blurring either the R or G components of the image leads to a blurry-looking full-color image.



Figure 8.16: (a) Original image. (b) Lab components. (c) Lab components, each blurred. These sharp and blurred components are used in the color images of [figure 8.17](#).

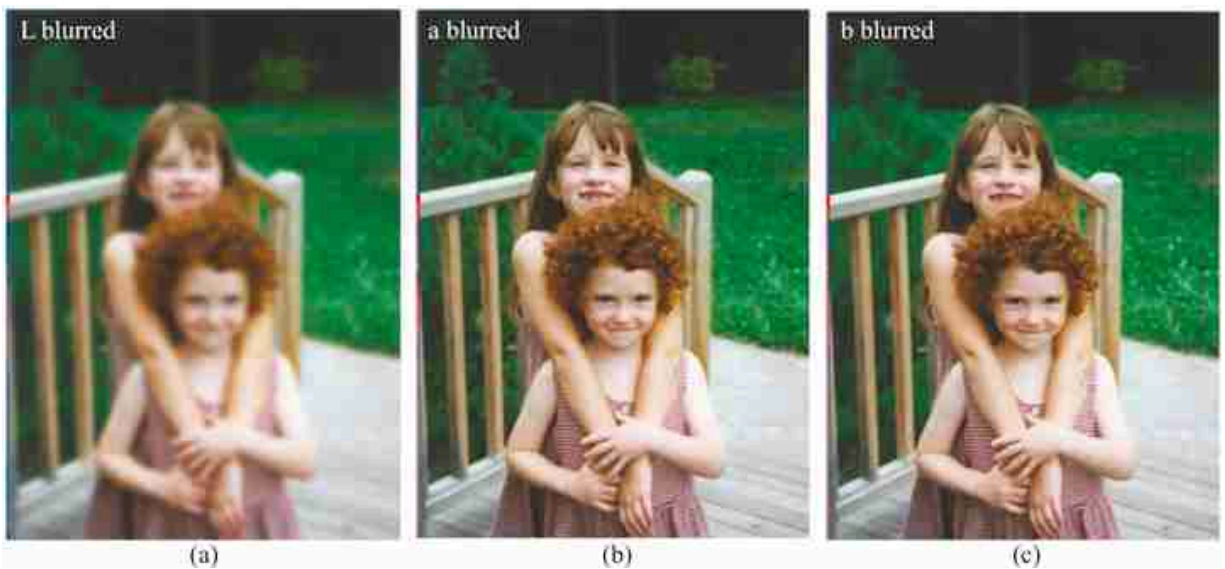


Figure 8.17: Color images composed of sharp and blurred components from [figures 8.16\(b\)](#) and [8.16\(c\)](#). (a) L component blurred, a and b components sharp. (b) a component blurred, L and b components sharp. (c) b component blurred, L and a components sharp. Blurring only either of the a or b components of the image yields a full-color image that appears sharp, provided the luminance component of the image is sharp.

8.5 Concluding Remarks

The three classes of color sensors in our eyes project light spectra into a 3-dimensional color space. While human perception of color depends on more than just the spectrum of the light being observed, most systems for color matching achieve good results by assuming that the spectrum is all that matters. Linear algebra can find the best controls for a display or printing device in order to match the color measured by a set of sensors.

As would be expected from the spatial varying pattern of color sensors in our eyes—we have few blue or short-wavelength sensors—humans observe different colors with different spatial resolutions, a fact that is exploited in image compression methods.

III

FOUNDATIONS OF LEARNING

Learning is one of the key components of a computer vision system. In these chapters we cover the foundations of machine learning from a general perspective but explore examples using vision problems.

- **Chapter 9** introduces the basic principles of machine learning.
- **Chapter 10** describes how to learn the parameters that fit a model to data.
- **Chapter 11** describes the difference between fitting to training data and generalizing to test data, and the new considerations that arise given this difference.
- **Chapter 12** introduces neural networks, a general family of models common in both biological and artificial vision systems.
- **Chapter 13** presents neural networks as functions that apply a series of geometric transformations to a data distribution.
- **Chapter 14** describes the backpropagation algorithm for calculating the gradient of a neural network with respect to its parameters.

Notation

- The algorithms we will see apply to many kinds of signals, not just images. Therefore, in this part we will use $\mathbf{x}$ to represent model inputs rather than ℓ . A model's final output will usually be represented by $\mathbf{y}$.
- Neural networks consist of a sequence of layers that perform a sequence of transformations $\mathbf{x}_0 \rightarrow \mathbf{x}_1 \rightarrow \dots \rightarrow \mathbf{y}$. When we consider a single layer in isolation, we will generically refer to its input as $\mathbf{x}_{\text{in}}$ and its output as $\mathbf{x}_{\text{out}}$. We will also use the variables $\mathbf{h}$ and $\mathbf{z}$ to represent certain kinds of intermediate representations in neural nets, which will be defined when they are first used.

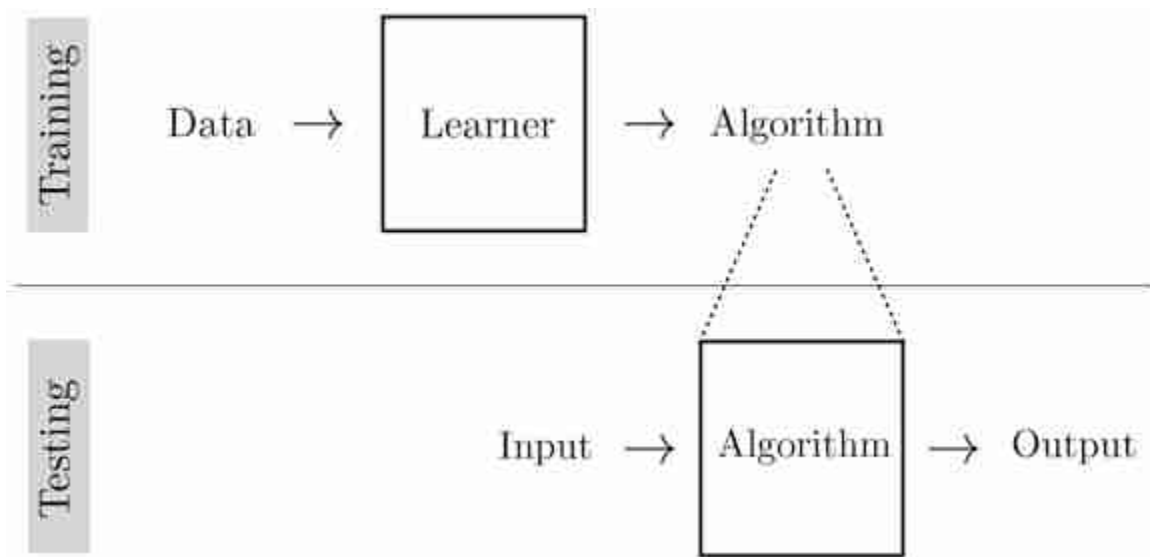
9 Introduction to Learning

9.1 Introduction

The goal of learning is to extract lessons from past experience in order to solve future problems. Typically, this involves searching for an algorithm that solves past instances of the problem. This algorithm can then be applied to future instances of the problem.

Past and future do not necessarily refer to the calendar date; instead they refer to what the *learner* has previously seen and what the learner will see next.

Because learning is itself an algorithm, it can be understood as a meta-algorithm: an algorithm that outputs algorithms ([figure 9.1](#)).



[Figure 9.1](#): Learning is an algorithm that outputs algorithms.

Learning usually consists of two phases: the **training** phase, where we search for an algorithm that performs well on past instances of the problem

(training data), and the **testing** phase, where we deploy our learned algorithm to solve new instances of the problem.

9.2 Learning from Examples

Learning from examples is also called **supervised learning**.

Imagine you find an ancient mathematics text, with marvelous looking proofs, but there is a symbol you do not recognize, “ $\star$ ”. You see it being used here and there in equations, and you note down examples of its behavior:

$$2 \star 3 = 36$$

$$7 \star 1 = 49$$

$$5 \star 2 = 100$$

$$2 \star 2 = 16$$

What do you think $\star$ represents? What function is it computing? Do you have it? Let's test your answer: what is the value of $3 \star 5$? (The answer is in the figure below.)

It may not seem like it, but you just performed learning! You *learned* what $\star$ does by looking at examples. In fact, [figure 9.2](#) shows what you did:

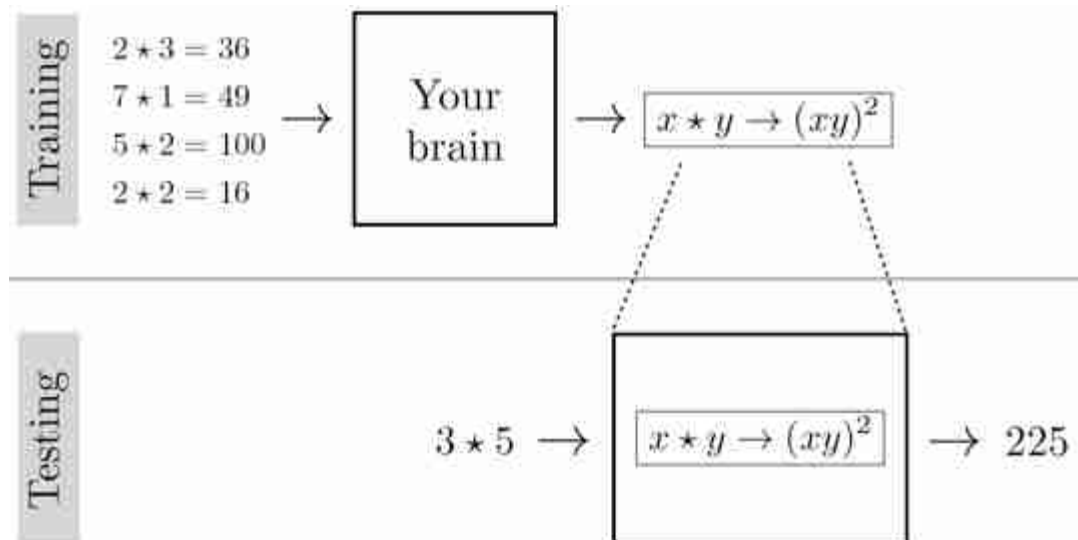


Figure 9.2: How your brain may have solved the star problem.

Nice job!

It turns out, we can learn almost anything from examples. Remember that we are learning an *algorithm*, i.e., a computable mapping from inputs to outputs. A formal definition of *example*, in this context, is an $\{\text{input}, \text{output}\}$ pair. The examples you were given for $\star$ consisted of four such pairs:

$\{\text{input}: [2, 3], \text{output}: 36\}$

$\{\text{input}: [7, 1], \text{output}: 49\}$

$\{\text{input}: [5, 2], \text{output}: 100\}$

$\{\text{input}: [2, 2], \text{output}: 16\}$

This kind of learning, where you observe example input-output behavior and infer a functional mapping that explains this behavior, is called **supervised learning**.

Another name for this kind of learning is **fitting a model** to data.

We were able to model the behavior of $\star$, on the examples we were given, with a simple algebraic equation. Let's try something rather more complicated. From the three examples in [figure 9.3](#), can you figure out what the operator F does?

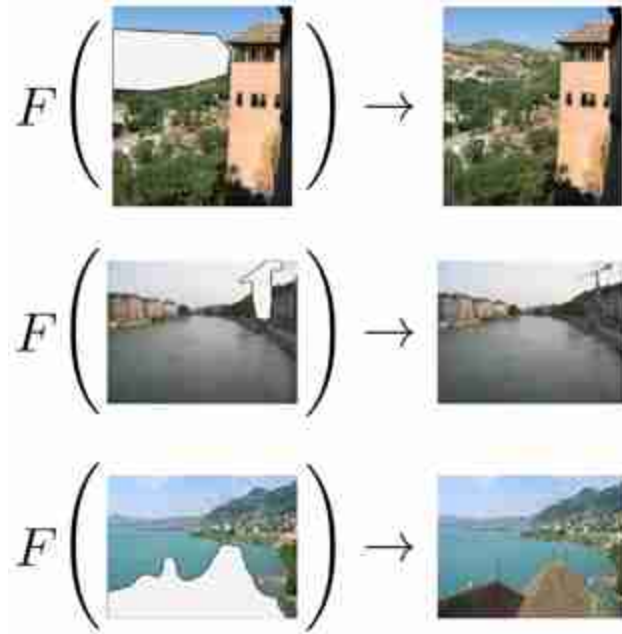


Figure 9.3: A complicated function that could be learned from examples. This example is from [191].

You probably came up with something like “it fills in the missing pixels.” That’s exactly right, but it’s sweeping a lot of details under the rug. Remember, we want to learn an *algorithm*, a procedure that is completely unambiguous. How exactly does F fill in the missing pixels?

It’s hard to say in words. We may need a very complex algorithm to specify the answer, an algorithm so complex that we could never hope to write it out by hand. This is the point of machine learning. The machine writes the algorithm for us, but it can only do so if we give it many examples, not just these three.

Some things are not learnable from examples, such as noncomputable functions. An example of a noncomputable function is a function that takes as input a program and outputs a 1 if the program will eventually finish running, and a 0 if it will run forever. It is noncomputable because there is no algorithm that can solve this task in finite time. However, it might be possible to learn a good approximation to it.

9.3 Learning without Examples

Even without examples, we can still learn. Instead of matching input-output examples, we can try to come up with an algorithm that optimizes for desirable *properties* of the input-output mapping. This class of learners includes **unsupervised learning** and **reinforcement learning**.

In unsupervised learning, we are given examples of *input data* $\{x^{(i)}\}_{i=1}^N$ but we are not told the target outputs $\{y^{(i)}\}_{i=1}^N$. Instead the learner has to come up with a model or representation of the input data that has useful properties, as measured by some **objective function**. The objective could be, for example, to compress the data into a lower dimensional format that still preserves all information about the inputs. We will encounter this kind of learning in part IX of this book, on representation learning and generative modeling.

In reinforcement learning, we suppose that we are given a **reward function** that explicitly measures the quality of the learned function's output. To be precise, a reward function is a mapping from outputs to scores: $r: Y \rightarrow \mathbb{R}$. The learner tries to come up with a function that maximizes rewards. This book will not cover reinforcement learning in detail, but this kind of learning is becoming an important part of computer vision, especially in the context of vision for robots. We direct the interested reader to [\[460\]](#) to learn more.

At first glance unsupervised learning and reinforcement learning look similar: both maximize a function that scores desirable properties of the input-output mapping. The big difference is that unsupervised learning has access to *training data* whereas reinforcement learning usually does not; instead the reinforcement learner has to collect its own training data.

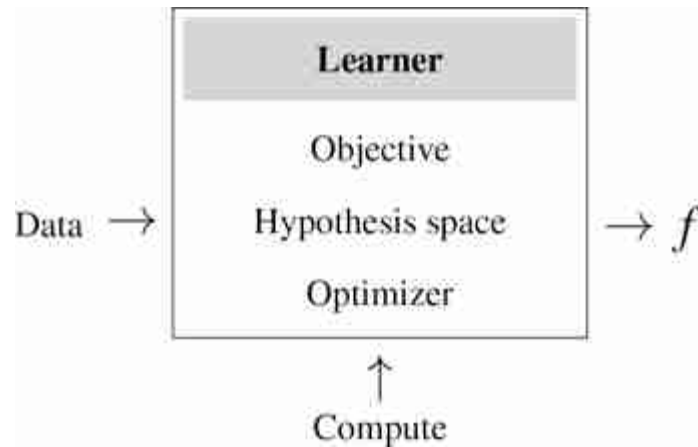
9.4 Key Ingredients

A learning algorithm consists of three key ingredients:

1. **Objective:** What does it mean for the learner to succeed, or, at least, to perform well?
2. **Hypothesis space:** What is the set of possible mappings from inputs to outputs that we will search over?

3. **Optimizer:** *How, exactly, do we search the hypothesis space for a specific mapping that maximizes the objective?*

These three ingredients, when applied to large amounts of data, and run on sufficient hardware (referred to as **compute**) can do amazing things. We will focus on the learning algorithms in this chapter, but often the data and compute turn out to be more important.



A learner outputs an algorithm, $f: X \rightarrow Y$, which maps inputs, $\mathbf{x} \in X$, to outputs, $\mathbf{y} \in Y$. Commonly, f is referred to as the **learned function**. The objective that the learner optimizes is typically a function that scores model outputs, $L: Y \rightarrow \mathbb{R}$, or compares model outputs to target answers, $L: Y \times Y \rightarrow \mathbb{R}$. We will interchangeably call this L either the **objective function**, the **loss function**, or the **loss**. A loss almost always refers to an objective we seek to *minimize*, whereas an objective function can be used to describe objectives we seek to minimize as well as those we seek to maximize.

9.4.1 Importance of Parameterization

The hypothesis space can be described by a set F of all the possible functions under consideration by the learner. For example, one hypothesis space might be “all mappings from $\mathbb{R}^2 \rightarrow \mathbb{R}$ ” and another could be “all functions $\mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}$ that satisfy the conditions of being a distance metric.” Commonly, however, we will not just specify the hypothesis space, but also how we parameterize the space. For example, we may say that our *parameterized* hypothesis space is $y = \theta_1 x + \theta_0$, where θ_0 and θ_1 are the parameters. This example corresponds to the space of affine functions from $\mathbb{R} \rightarrow \mathbb{R}$, but this is not the only way to parameterize that space. Another

choice could be $y = \theta_2\theta_1x + \theta_0$, with parameters θ_0 , θ_1 , and θ_2 . These two choices parameterize *exactly the same space*, that is, any affine functions can be represented by either parameterization and both parameterizations can only represent affine functions. However, these two parameterizations are not equivalent, because optimizers and objectives may treat different parameterizations differently. Because of this, to fully define a learning algorithm, it is important to specify how the hypothesis space is parameterized.

Overparameterized models, where you use more parameters than the minimum necessary to fit the data, are especially important in modern computer vision; most neural networks (chapter 12) are overparameterized.

9.5 Empirical Risk Minimization: A Formalization of Learning from Examples

The three ingredients from the last section can be formalized using the framework of **empirical risk minimization (ERM)**. This framework applies specifically to the supervised setting where we are learning a function that predicts $\mathbf{y}$ from $\mathbf{x}$ given many training examples $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$. The idea is to minimize the average error (i.e., risk) we incur over all the training data (i.e., empirical distribution). The ERM problem is stated as follows:

$$\arg \min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}) \triangleq \text{ERM} \quad (9.1)$$

Here, $\mathcal{F}$ is the hypothesis space, $\mathcal{L}$ is the loss function, and $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$ is the training data (example {input, output} pairs), and f is the learned function.

9.6 Learning as Probabilistic Inference

Depending on the loss function, there is often an interpretation of ERM as doing maximum likelihood probabilistic inference. In this interpretation, we are trying to infer the hypothesis f that assigns the highest probability to the data. For a model that predicts $\mathbf{y}$ given $\mathbf{x}$, the max likelihood f is:

$$\arg \max_f p(\{\mathbf{y}^{(i)}\}_{i=1}^N | \{\mathbf{x}^{(i)}\}_{i=1}^N, f) \quad \triangleleft \quad \text{Max likelihood learning} \quad (9.2)$$

The term $p(\{\mathbf{y}^{(i)}\}_{i=1}^N | \{\mathbf{x}^{(i)}\}_{i=1}^N, f)$ is called the **likelihood** of the $\mathbf{y}$ values given the model f and the observed $\mathbf{x}$ values, and maximizing this quantity is called **maximum likelihood learning**.

To fully specify this model, we have to define the form of this conditional distribution. One common choice is that the prediction errors, $(\mathbf{y} - f(\mathbf{x}))$, are Gaussian distributed, which leads to the least-squares objective (section 9.7.1).

In later chapters we will see that **priors** $p(f)$ can also be used for inferring the most probable hypothesis. When a prior is used in conjunction with a likelihood function, we arrive at **maximum a posteriori learning (MAP learning)**, which infers the most probable hypothesis given the training data:

$$\arg \max_f p(f | \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N) \quad \triangleleft \quad \text{MAP learning} \quad (9.3)$$

$$= \arg \max_f p(\{\mathbf{y}^{(i)}\}_{i=1}^N | \{\mathbf{x}^{(i)}\}_{i=1}^N, f) p(f) \quad \triangleleft \quad \text{by Bayes' rule} \quad (9.4)$$

9.7 Case Studies

The next three sections cover several case studies of particular learning problems. Examples 1 and 3 showcase the two most common workhorses of machine learning: regression and classification. Example 2, Python program induction, demonstrates that the paradigms in this chapter are not limited to simple systems but can actually apply to very general and sophisticated models.

9.7.1 Example 1: Linear Least-Squares Regression

One of the simplest learning problems is known as **linear least-squares regression**. In this setting, we aim to model the relationship between two variables, x and y , with a line.

As a concrete example, let's imagine x represents the temperature outside, and y represents the number people at the beach. As before, we train (i.e., fit) our model on many observed examples of {temperature outside, number of people at the beach} pairs, denoted as

$\{x^{(i)}, y^{(i)}\}_{i=1}^N$. At test time, this model can be applied to predict the y value of a new query x' , as shown in [figure 9.4](#).

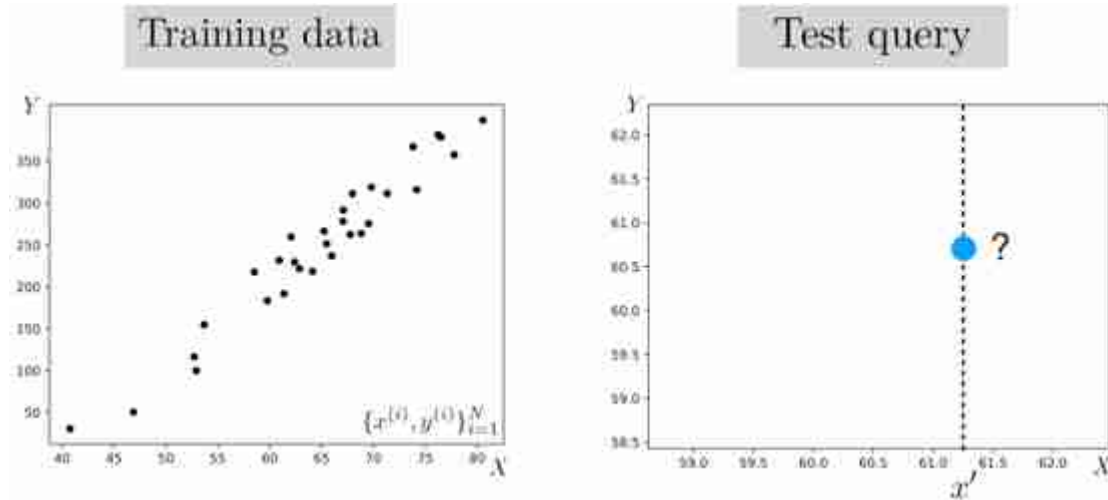


Figure 9.4: The goal of learning is to use the training data to predict the y value of the test query. In our example we find that for every 1 degree increase in temperature, we can expect ~ 10 more people to go to the beach.

Our *hypothesis space* is linear functions, that is, the relationship between x and our predictions $\hat{y}$ of y has the form $\hat{y} = f_{\theta}(x) = \theta_1 x + \theta_0$. This hypothesis space is parameterized by a two scalars, $\theta_0, \theta_1 \in \mathbb{R}$, the intercept and slope of the line. In this book, we will use θ in general to refer to any parameters that are being learned. In this case we have $\theta = [\theta_0, \theta_1]$. Learning consists of finding the value of these parameters that maximizes the objective.

Our *objective* is that predictions should be near ground truth targets in a least-squares sense, that is, $(\hat{y}^{(i)} - y^{(i)})^2$ should be small for all training examples $\{x^{(i)}, y^{(i)}\}_{i=1}^N$. We call this objective the L_2 loss:

$$J(\theta) = \sum_i \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) \quad (9.5)$$

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2 \quad \triangleleft \quad L_2 \text{ loss} \quad (9.6)$$

We will use $J(\theta)$ to denote the total objective, over all training datapoints, as a function of the parameters; we will use L to denote the loss per datapoint. That is, $J(\theta) = \sum_{i=1}^N \mathcal{L}(f_{\theta}(x^{(i)}), y^{(i)})$.

The full learning problem is as follows:

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N (\theta_1 x^{(i)} + \theta_0 - y^{(i)})^2. \quad (9.7)$$

We can choose any number of optimizers to solve this problem. A first idea might be “try a bunch of random values for θ and return the one that maximizes the objective.” In fact, this simple approach works, it just can be rather slow since we are searching for good solutions blind. A better idea can be to exploit *structure* in the search problem. For the linear least-squares problem, the tools of calculus give us clean mathematical structure that makes solving the optimization problem easy, as we show next.

From calculus, we know that at any maxima or minima of a function, $J(\theta)$, with respect to a variable θ , the derivative $\frac{\partial J(\theta)}{\partial \theta} = 0$. We are trying to find the minimum of the objective $J(\theta)$:

$$J(\theta) = \sum_{i=1}^N (\theta_1 x^{(i)} + \theta_0 - y^{(i)})^2. \quad (9.8)$$

This function can be rewritten as

$$J(\theta) = (\mathbf{y} - \mathbf{X}\theta)^T (\mathbf{y} - \mathbf{X}\theta) \quad (9.9)$$

with

$$\mathbf{X} = \begin{bmatrix} 1 & x^{(1)} \\ 1 & x^{(2)} \\ \vdots & \vdots \\ 1 & x^{(N)} \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(N)} \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}. \quad (9.10)$$

The J is a quadratic form, which has a single global minimum where the derivative is zero, and no other points where the derivative is zero. Therefore, we can solve for the θ^* that minimizes J by finding the point where the derivative is zero. The derivative is:

$$\frac{\partial J(\theta)}{\partial \theta} = 2(\mathbf{X}^T \mathbf{X} \theta - \mathbf{X}^T \mathbf{y}). \quad (9.11)$$

We set this derivative to zero and solve for θ^* :

$$2(\mathbf{X}^T \mathbf{X} \theta^* - \mathbf{X}^T \mathbf{y}) = 0 \quad (9.12)$$

$$\mathbf{X}^T \mathbf{X} \theta^* = \mathbf{X}^T \mathbf{y} \quad (9.13)$$

$$\theta^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}. \quad (9.14)$$

The θ^* defines the best fitting line to our data, and this line can be used to predict the y value of future observations of x ([figure 9.5](#)).

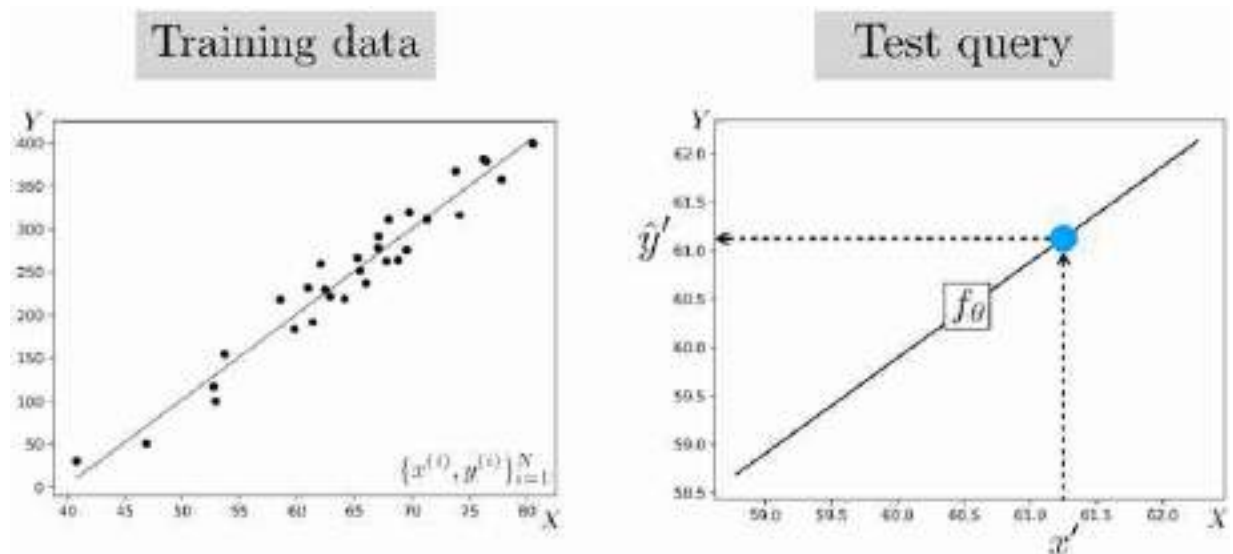
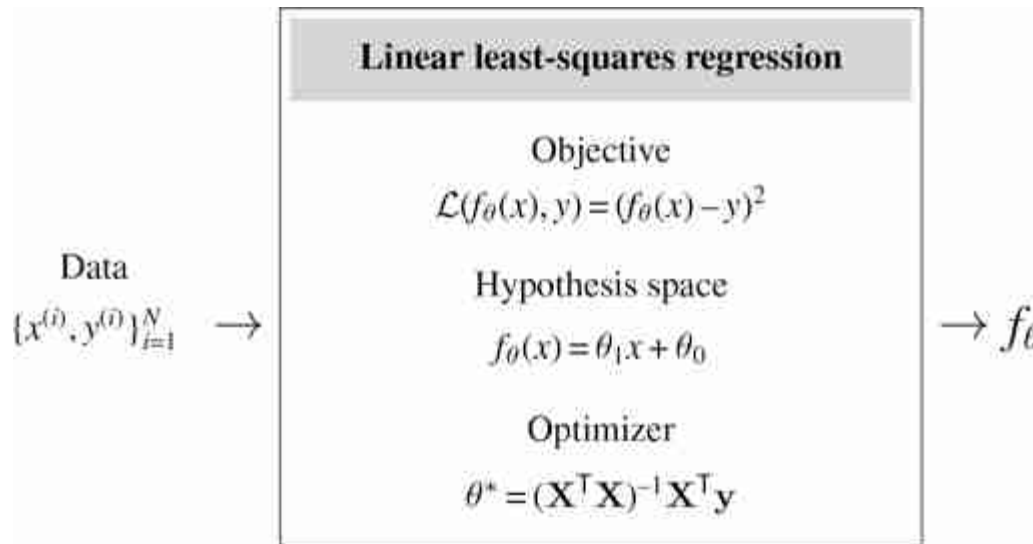


Figure 9.5: A best fit line is a visualization of a function f_{θ^*} that predicts the y -value for each input x -value.

We can now summarize the entire linear least-squares learning problem as follows:



In these diagrams, we will sometimes describe the objective just in terms of L , in which case it should be understood that this implies $J(\theta) = \sum_{i=1}^N \mathcal{L}(f_{\theta}(x^{(i)}), y^{(i)})$.

9.7.2 Example 2: Program Induction

At the other end of the spectrum we have what is known as **program induction**, which is one of the broadest classes of learning algorithm. In this setting, our hypothesis space may be all Python programs. Let's contrast linear least-squares with Python program induction. [Figure 9.6](#) shows what linear least-squares looks like.

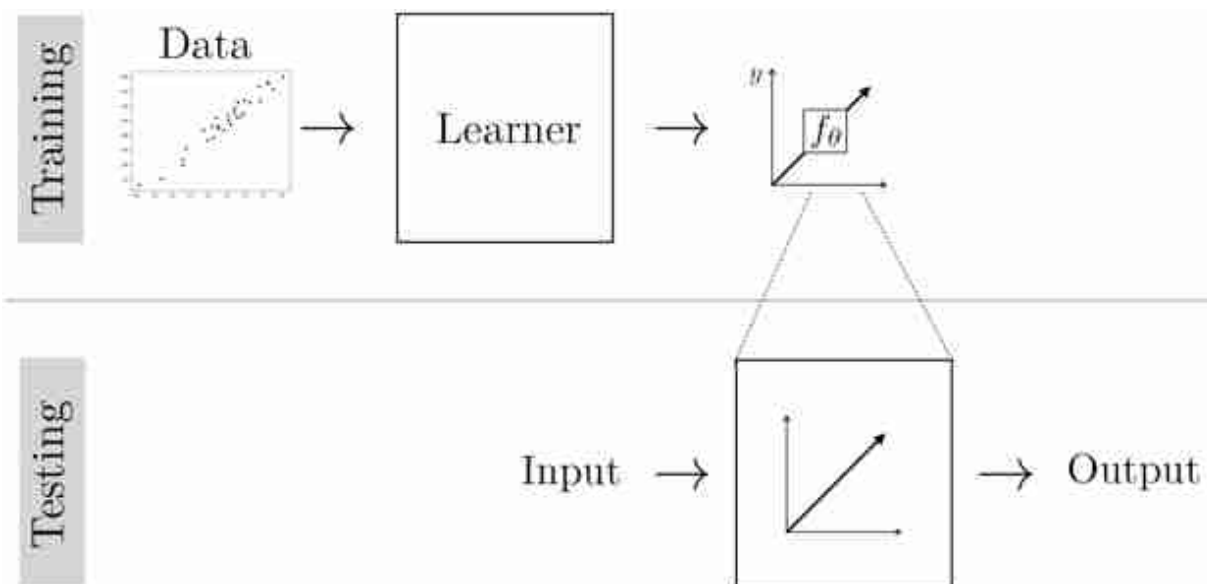


Figure 9.6: Linear regression finds a line that predicts the training data's y -values from its x -values.

The learned function is an algebraic expression that maps x to y . Learning consisting of searching over two scalar parameters, θ_0 and θ_1 .

[Figure 9.7](#) shows Python program induction solving the same problem. In this case, the learned function is a Python program that maps x to y . Learning consisted of searching over the space of all possible Python programs (within some max length). Clearly that's a much harder search problem than just finding two scalars. In chapter 11, we will see some pitfalls of using too powerful a hypothesis space when a simpler one will do.

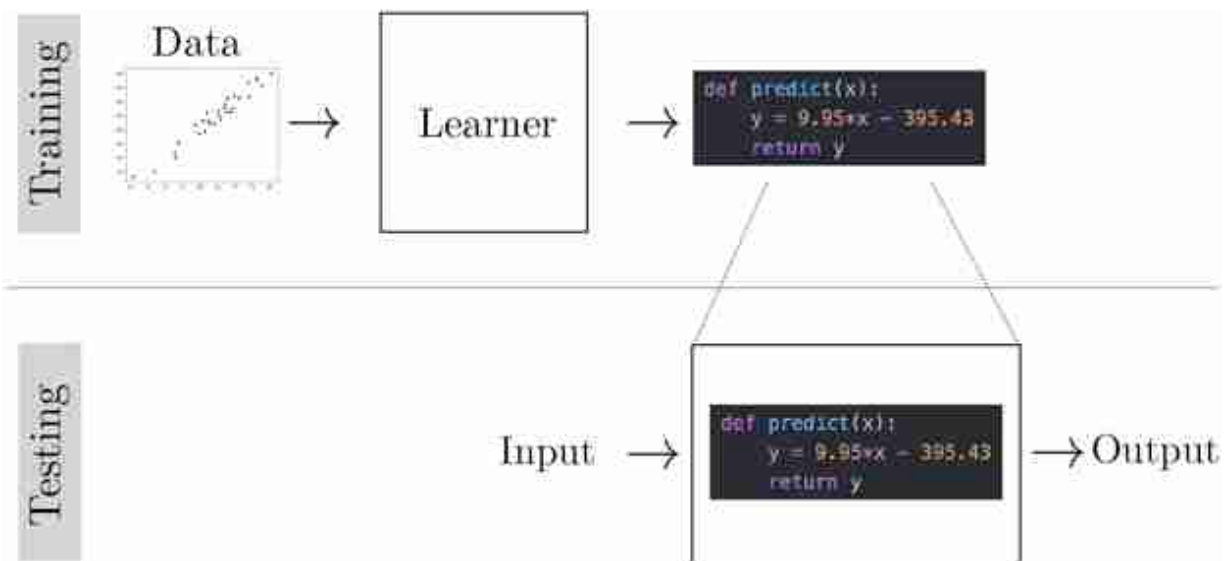


Figure 9.7: Python program induction finds a Python program that predicts the training data's y -values from its x -values.

9.7.3 Example 3: Classification and Softmax Regression

A common problem in computer vision is to recognize objects. This is a **classification** problem. Our input is an image x , and our target output is a class label y ([figure 9.8](#)).

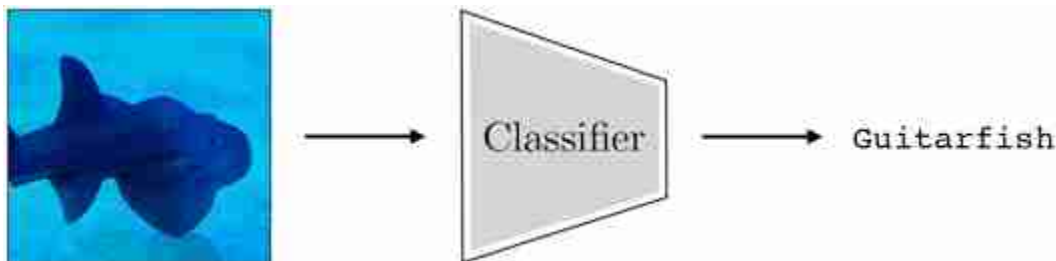


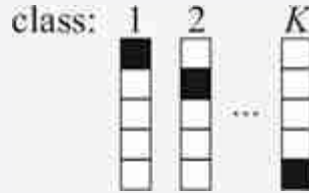
Figure 9.8: Image classification.

How should we formulate this task as a learning problem? The first question is how do we even represent the input and output? Representing images is pretty straightforward; as we have seen elsewhere in this book, they can be represented as arrays of numbers representing red-green-blue colors: $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$, where H is image height and W is image width.

How can we represent class labels? It turns out a convenient representation is to let y be a K -dimensional vector, for K possible classes,

with $y_k = 1$ if $\mathbf{y}$ represents class k , and $y_k = 0$ otherwise. This representation is called a **one-hot code**, since just one element of the vector is on (“hot”). Each class has a unique one-hot code. We will see why this representation makes sense shortly. The one-hot codes are the targets for the function we are learning. Our goal is to learn a function f_θ that output vectors $\hat{\mathbf{y}}$ that match the one-hot codes, thereby correctly classifying the input images.

An example of one-hot codes for representing $K=5$ different classes:



Next, we need to pick a loss function. Our first idea might be that we should minimize misclassifications. That would correspond to the so called **0-1 loss**:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \mathbb{1}(\hat{\mathbf{y}} \neq \mathbf{y}), \quad (9.15)$$

where $\mathbb{1}$ is the indicator function that evaluates to 1 if and only if its argument is true, and 0 otherwise. Unfortunately, minimizing this loss is a discrete optimization problem, and it is NP-hard. Instead, people commonly use the **cross-entropy loss**, which is continuous and differentiable (making it easier to optimize):

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k \quad \triangleleft \text{cross-entropy loss} \quad (9.16)$$

The way to think about this is $\hat{y}_k$ should *represent the probability* we think the image is an image of class k . Under that interpretation, minimizing cross-entropy maximizes the log likelihood of the ground truth observation $\mathbf{y}$ under our model's prediction $\hat{\mathbf{y}}$.

For that interpretation to be valid, we require that $\hat{\mathbf{y}}$ represent a **probability mass function (pmf)**. A pmf $\mathbf{p}$, over K classes, is defined as a K -dimensional vector with elements in the range $[0, 1]$ that sums to 1. In other words, $\mathbf{p}$ is a point on the $(K - 1)$ -**simplex**, which we denote as $\mathbf{p} \in \Delta^{K-1}$.

The $(K - 1)$ -simplex, Δ^{K-1} , is the set of all K -dimensional vectors whose elements sum to 1. K -dimensional one-hot codes live on the vertices of Δ^{K-1} .

To ensure that the output of our learned function f_θ has this property, i.e., $f_\theta \in \Delta^{K-1}$, we can compose two steps: (1) first apply a function $z_\theta : X \rightarrow \mathbb{R}^K$, (2) then squash the output into the range $[0, 1]$ and normalize it to sum to 1. A popular way to squash is via the **softmax** function:

Using softmax is a modeling choice; we could have used any function that squashes into a valid pmf, that is, a nonnegative vector that sums to 1.

$$\mathbf{z} = z_\theta(\mathbf{x}) \quad (9.17)$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \quad (9.18)$$

$$\hat{y}_j = \frac{e^{-z_j}}{\sum_{i=1}^K e^{-z_i}} \quad (9.19)$$

The values in $\mathbf{z}$ are called the **logits** and can be interpreted as the unnormalized log probabilities of each class. Now we have,

$$\hat{\mathbf{y}} = f_\theta(\mathbf{x}) = \text{softmax}(z_\theta(\mathbf{x})) \quad (9.20)$$

[Figure 9.9](#) shows what the variables look like for processing one photo of a fish during training.

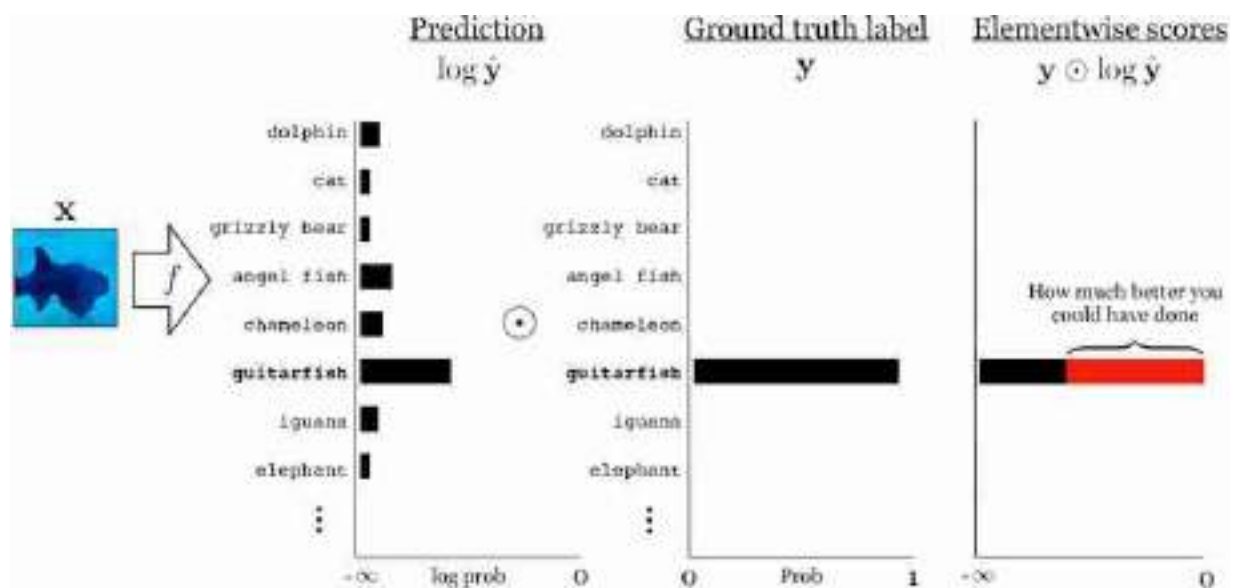
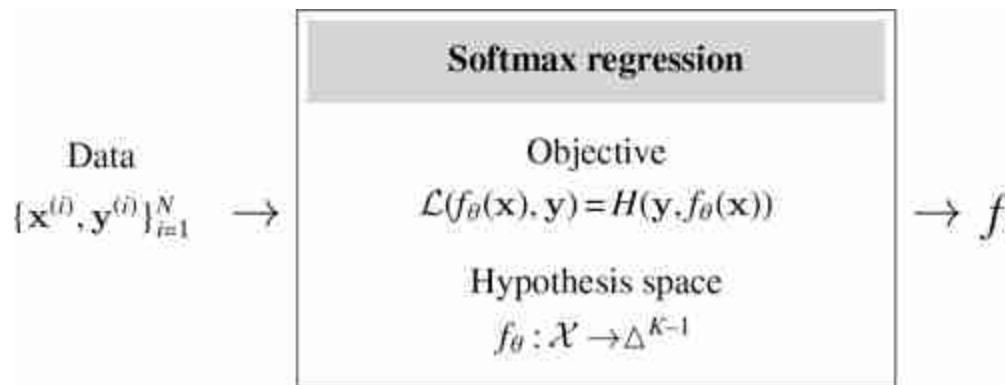


Figure 9.9: Softmax regression for image classification. The $\odot$ symbol represents an elementwise product. The cross-entropy loss is the negative of the sum over elementwise agreements between the prediction vector $\hat{y}$ and the label vector y , that is, if $s = y \odot \log \hat{y}$ is the vector of scores for how well our prediction agrees with the label, then our cross-entropy loss is $H(y, \hat{y}) = -\sum_{k=1}^K s_k$.

The prediction placed about 40 percent probability on the true class, “guitarfish,” so we are 60 percent off from an ideal prediction (indicated by the red bar; an ideal prediction would place 100 percent probability on the true class). Our loss is $-\log 0.4$.

This learning problem, which is also called **softmax regression**, can be summarized as follows:

Softmax regression is just one way to model a classification problem. We could have made other choices for how to map input data to class labels.



Notice that we have left the hypothesis space only partially specified, and we left the optimizer unspecified. This is because softmax regression refers to the whole family of learning methods that have this general form. This is one of the reasons we conceptualized the learning problem in terms of the three key ingredients described previously: you can often develop them each in isolation, then mix and match.

9.8 Learning to Learn

Learning to learn, also called **metalearning**, is a special case of learning where the hypothesis space is learning algorithms.

Recall that learners train on past instances of a problem to produce an algorithm that can solve future instances of the problem. The goal of metalearning is to handle the case where the future problem we will encounter is itself a learning problem, such as “find the least-squares line fit to these data points.” One way to train for this would be by example. Suppose that we are given the following {input, output} examples:

{input: $(x: [1, 2], y: [1, 2])$,	output: $y = x$ }
{input: $(x: [1, 2], y: [2, 4])$,	output: $y = 2x$ }
{input: $(x: [1, 2], y: [0.5, 1])$,	output: $y = \frac{x}{2}$ }

These are examples of performing least-squares regression; therefore the learner can fit these examples by learning to perform least-squares regression.

Note that least-squares regression is not the unique solution that fits these examples, and the metalearner might arrive at a different solution that fits equally well.

Since least-squares regression is itself a learning algorithm, we can say that the learner learned to learn.

We started this chapter by saying the learning is a meta-algorithm: it's an algorithm that outputs an algorithm. Metalearning is a meta-meta-algorithm and we can visualize it by just adding another outer loop on top of a learner, as shown in [figure 9.10](#).

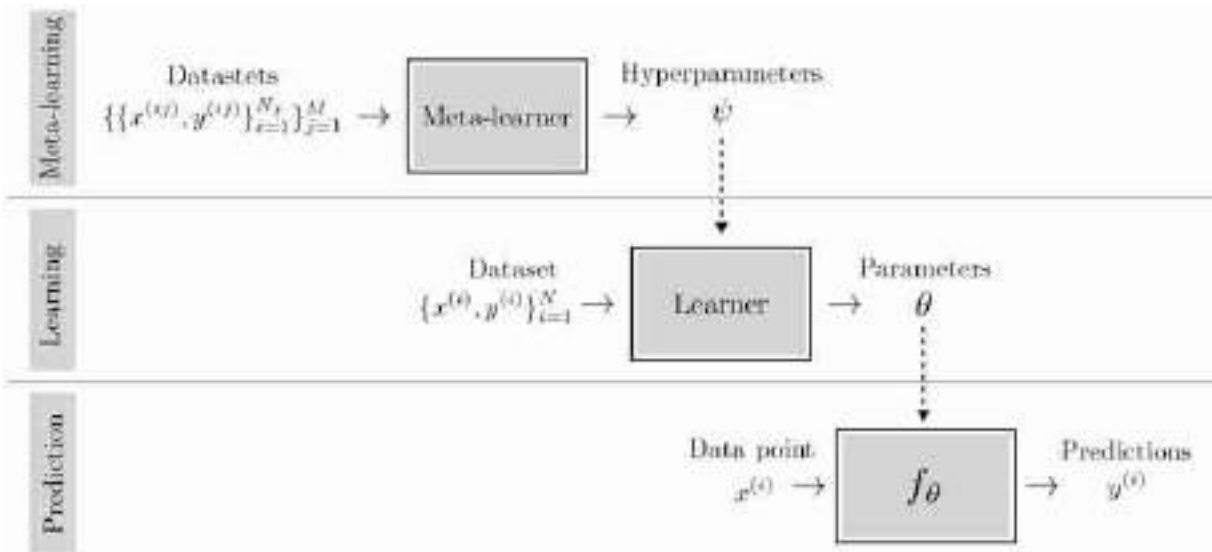


Figure 9.10: Learning is a meta-algorithm, an algorithm that outputs algorithms; metalearning is just learning applied to learning, and therefore it is a meta-meta-algorithm.

Notice that you can apply this idea recursively, constructing meta-meta-...-metalearners. Humans perform at least three levels of this process, if not more: we have *evolved* to be *taught* in school how to *learn* quickly on our own.

Evolution is a learning algorithm according to our present definition.

9.9 Concluding Remarks

Learning is an extremely general and powerful approach to problem solving. It turns data into algorithms. In this era of big data, learning is very often the preferred approach. It is a a major component of almost all modern computer vision systems.

10 Gradient-Based Learning Algorithms

10.1 Introduction

Once you have specified a learning problem (loss function, hypothesis space, parameterization), the next step is to find the parameters that minimize the loss. This is an optimization problem, and the most common optimization algorithm we will use is **gradient descent**. Gradient descent is like a skier making their way down a snowy mountain, where the shape of the mountain is the loss function.

There are many varieties of gradient descent, and we will call this whole family **gradient-based learning algorithms**. All share the same basic idea: at some operating point, calculate the direction of steepest descent, then use this direction to find a new operating point with lower loss.

We use the term **operating point** to refer to a particular point (setting of the parameters) where we are currently evaluating the loss.

10.2 Technical Setting

In this chapter, we consider the task of minimizing a cost function $J : \cdot \rightarrow \mathbb{R}$, which is a function that maps some arbitrary input to a scalar cost.

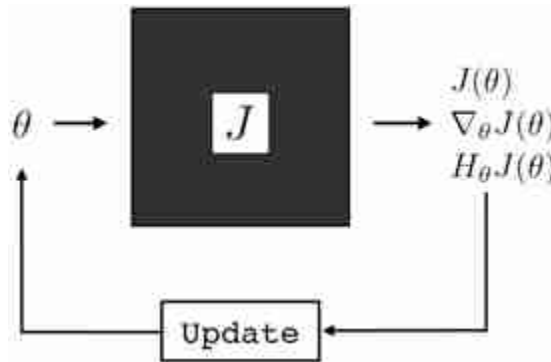
In learning problems, the domain of J is the training data and the parameters θ .

Remember from chapter 9 that in supervised learning, the training data is $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$, while in unsupervised learning the training data is $\{\mathbf{x}^{(i)}\}_{i=1}^N$.

Often, we will consider the training data to be fixed and only denote the objective as a function of the parameters, $J(\theta)$. Our goal is to solve:

$$\theta^* = \arg \min_{\theta} J(\theta) \quad (10.1)$$

Pretty much all optimizers work by some iterative process, where they update the parameters to be better and better. Different optimizers differ in how the parameter update function works. The update function gets to view some information about the loss landscape, then uses that information to update the parameters, as shown in [figure 10.1](#).



[Figure 10.1](#): General optimization loop.

In the simplest setting, called **zeroth-order optimization**, the update function only gets to observe the value $J(\theta)$. The only way, then, to find θ 's that minimize the loss is to sample different values for θ and move toward the values that are lower.

For gradient-based optimization, also called **first-order optimization**, the update function takes as input the gradient of the cost with respect to the parameters at the current operating point, $\nabla_{\theta} J(\theta)$. This reveals hugely useful information about the loss that directly tells us how to minimize it: just move in the direction of steepest descent, that is, the gradient direction.

Higher-order optimization methods observe higher-order derivatives of the loss, such as the Hessian H , which tells you how the landscape is locally curving. The Hessian is costly to compute but many methods use approximations to the Hessian, or other properties related to loss curvature, and these are growing in popularity [[320](#), [134](#)].

10.3 Basic Gradient Descent

The simplest version of gradient descent just takes a step in the gradient direction of length proportional to the gradient magnitude. This algorithm is described in algorithm 10.1.

Algorithm 10.1: Gradient descent (GD). Optimizing a cost function $J: \theta \rightarrow \mathbb{R}$ by descending the gradient $\nabla_{\theta} J$.

```
1 Input: objective function  $J$ , initial parameter vector  $\theta^0$ , learning rate  $\eta$ ,  
   number of steps  $K$   
2 Output: trained parameter vector  $\theta^* = \theta^K$   
3 for  $k = 0, \dots, K - 1$  do  
4    $\theta^{k+1} \leftarrow \theta^k - \eta \nabla_{\theta} J(\theta^k)$ 
```

This algorithm has two hyperparameters, the **learning rate** η , which controls the step size (learning rate times gradient magnitude), and the number of steps K . If the learning rate is sufficiently small and the initial parameter vector θ^0 is random, then this algorithm will almost surely converge to a local minimum of J as $K \rightarrow \infty$ [288]. However, to descend more quickly, it can be useful to set the learning rate to a higher value.

10.4 Learning Rate Schedules

A generally useful strategy is to start with a high value for η and then decay it until convergence according to a **learning rate schedule**. Researchers have come up with innumerable schedules and they generally work by calling some function $\text{lr}(\eta^0, k)$ to get the learning rate on each iteration of descent:

$$\eta^k = \text{lr}(\eta^0, k) \tag{10.2}$$

Generally, we want an update rule where $\eta^{k+1} < \eta^k$, so that we take smaller steps as we approach the minimizer. A few simple and popular approaches are given below:

$$\text{lr}(\eta^0, k) = \beta^{-k} \eta^0 \quad \triangleleft \text{exponential decay} \quad (10.3)$$

$$\text{lr}(\eta^0, k) = \beta^{-\lfloor k/M \rfloor} \eta^0 \quad \triangleleft \text{stepwise exponential decay} \quad (10.4)$$

$$\text{lr}(\eta^0, k) = \frac{(K-k)}{K} \eta^0 \quad \triangleleft \text{linear decay} \quad (10.5)$$

One downside of linear decay is that it depends on the total number of steps K . This makes it hard to compare optimization runs of different lengths. This is something to also be aware of in more advanced learning rate schedules, such as cosine decay [308], which also have different behavior for different settings of K .

The β and M are additional hyperparameters of these methods. The general approach of learning rate decay is summarized in algorithm 10.2.

Algorithm 10.2: Gradient descent with a learning rate schedule.

```

1 Input: objective function  $J$ , initial parameter vector  $\theta^0$ , initial learning
   rate  $\eta^0$ , learning rate function  $\text{lr}$ , number of steps  $K$ 
2 Output: trained parameter vector  $\theta^* = \theta^K$ 
3 for  $k = 0, \dots, K-1$  do
4    $\eta^k \leftarrow \text{lr}(\eta^0, k)$ 
5    $\theta^{k+1} \leftarrow \theta^k - \eta^k \nabla_{\theta} J(\theta^k)$ 

```

Variations on this algorithm include only decaying the learning rate when a plateau is reached (i.e., when the loss is not decreasing for many iterations in a row), or decaying the learning rate according to more complex nonlinear schedules, such as one shaped like a cosine function [308].

10.5 Momentum

Could we do a smarter update than just taking a step in the direction of the gradient? Of the countless ideas that have been proposed, one of the few that has stuck is **momentum** [391, 459]. Momentum makes the analogy to skiing even more precise: momentum is like the inertia of the skier, carrying them over the little bumps and imperfections in the ski slope and increasing their speed as they descend along a straight path. In math, momentum just means that we set the parameter update to be a direction $\mathbf{v}^{k+1}$, given by a

weighted combination of the previous update direction, $\mathbf{v}^k$, plus the current negative gradient:

$$\mathbf{v}^{k+1} = \mu \mathbf{v}^k - \eta \nabla_{\theta} J(\theta^k) \quad (10.6)$$

The weight μ in this combination is a new hyperparameter, sometimes simply called the momentum. The full algorithm is given in algorithm 10.3.

Algorithm 10.3: Gradient descent with momentum.

```

1 Input: objective function  $J$ , initial parameter vector  $\theta^0$ , learning rate  $\eta$ ,
  momentum  $\mu$ , number of steps  $K$ 
2 Output: trained parameter vector  $\theta^* = \theta^K$ 
3  $\mathbf{v}^0 = \mathbf{0}$ 
4 for  $k = 0, \dots, K - 1$  do
5    $\mathbf{v}^{k+1} = \mu \mathbf{v}^k - \eta \nabla_{\theta} J(\theta^k)$ 
6    $\theta^{k+1} \leftarrow \theta^k + \mathbf{v}^{k+1}$ 

```

[Figure 10.2](#) shows how momentum affects gradient descent for a simple objective $J = \text{abs}(\theta)$ (absolute value of θ). As can be seen in the figure, some momentum can help convergence rate ([figure 10.2](#), $\mu = 0.5$) but too much momentum will cause the trajectory to overshoot the optimum and even when the optimum loss is achieved, the trajectory might not stop ([figure 10.2](#), $\mu = 0.95$).

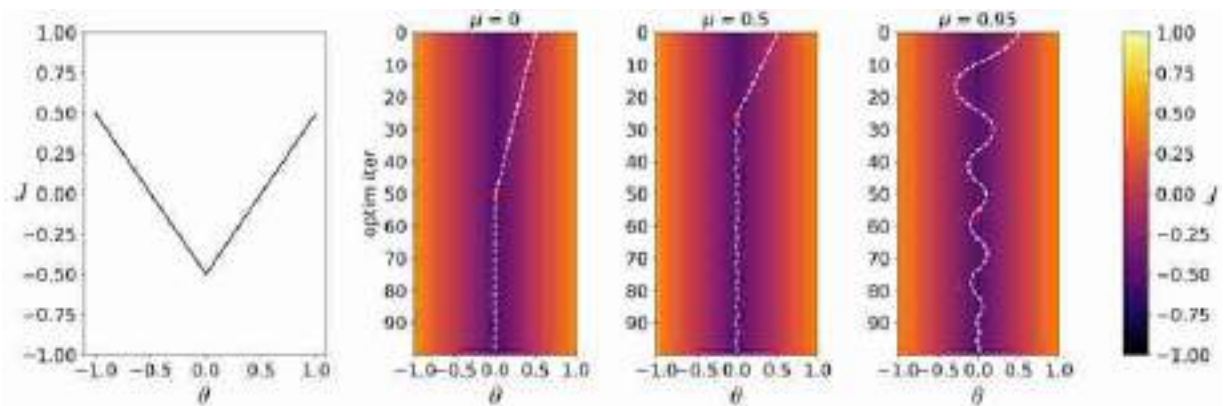


Figure 10.2: (left) A simple loss function $J = \text{abs}(\theta)$. (right) Optimization trajectory for three different settings of momentum μ . White line indicates value of the parameter at each iteration of optimization, starting at top and progressing to bottom. Color is value of the loss. Red dot is location where loss first reaches within 0.01 of optimal value.

It is also possible to come up with other kinds of momentum, which bias the update direction based on some accumulated information from previous updates. Two popular alternatives, which you can read up on elsewhere, are Nesterov's accelerated gradient [353] and Adam [262].

10.6 What Kinds of Functions Can Be Minimized with Gradient Descent?

What if a function is not differentiable? Can we still use gradient descent? Sometimes! The property we need is that we can get a meaningful signal as to how to perturb the function's parameters in order to reduce the loss. This property is *not* the same as differentiability defined in math textbooks. A function may be differentiable but not give useful gradients (e.g., if the gradient is zero everywhere), and a function may be nondifferentiable (at certain points) but still allow for meaningful gradient-based updates (e.g., abs).

[Figure 10.3](#) gives examples of different types of functions being minimized with gradient descent. [Figures 10.3\(b\)](#) and [10.3\(d\)](#) are cases where the function is discontinuous, and the analytical derivative is undefined at the discontinuity. Surprisingly, in [figure 10.3\(b\)](#), this is not a problem for gradient descent. This is because the gradient descent algorithm we are using here (the one used by Pytorch [378]) uses a **one-sided**

derivative at the discontinuity, that is, we set the gradient at the discontinuity to be equal to the gradient value an infinitesimal step away from the discontinuity in a fixed arbitrary direction. Under the hood, for each atomic discontinuous operation, Pytorch requires that we define its gradients at the discontinuities, and the one-sided gradient is a standard choice. This is why it can be fine in deep learning (chapter 12) to use functions like rectified nonlinear units (relus), which are common in deep networks and have discontinuous gradients.

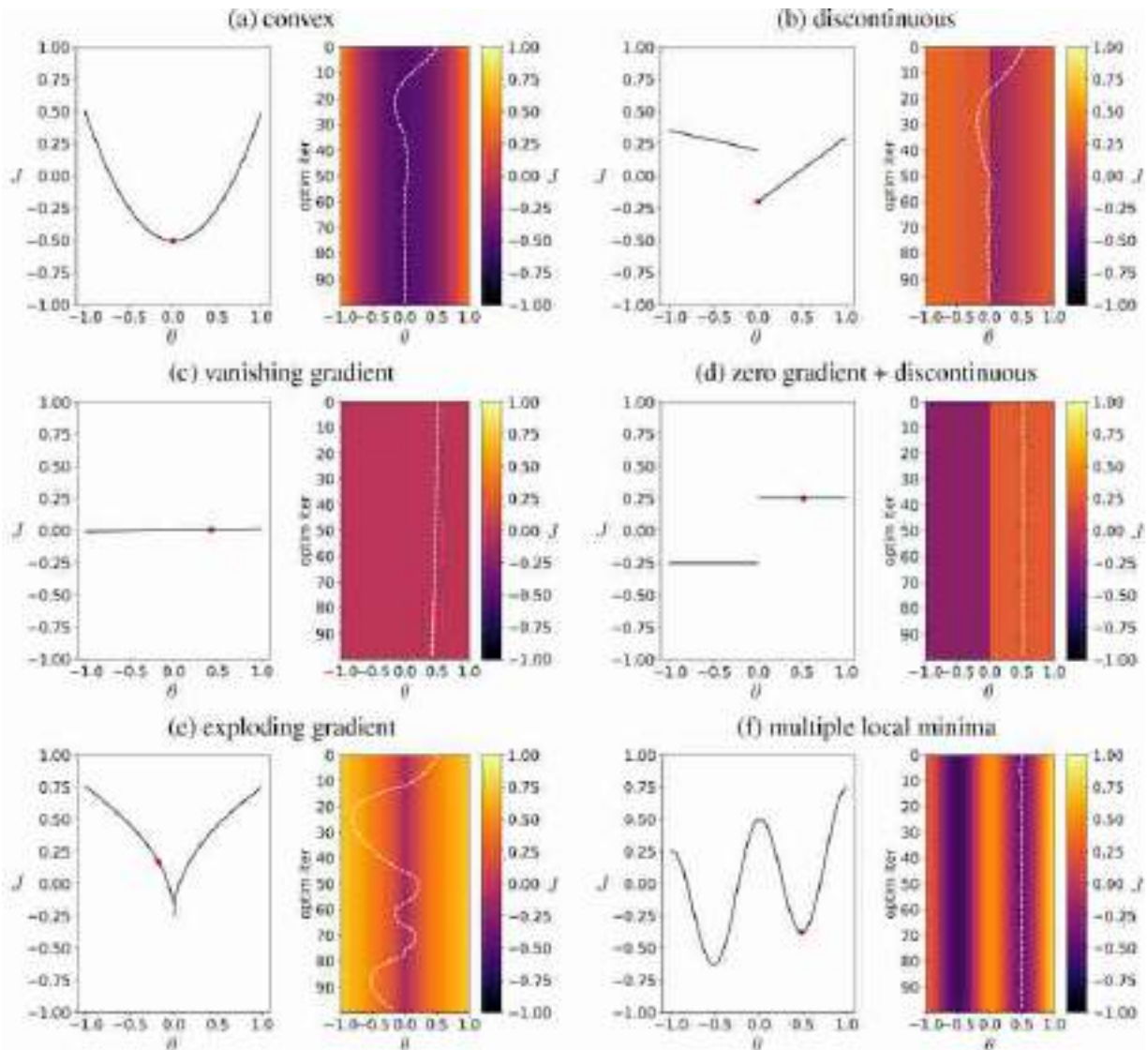


Figure 10.3: How gradient descent behaves on various functions. In each subplot, left is the function J , the red point is the solution found by GD (with $\eta = 0.01$ and $\mu = 0.9$), and right is the trajectory of x values over iterations of GD, plotted on top of J at each iteration. (a) As η goes to zero, GD converges for convex functions. (b) Discontinuities pose no essential problem, as long as the gradient is defined on either side. (c) A nearly flat function will exhibit very slow descent. (d) Piecewise constant functions are problematic because the gradient completely vanishes. (e) For the function $J = \sqrt{\text{abs}(\theta)} - 0.25$, the gradient goes to infinity at the minimizer, causing instability. (f) When J has multiple local minima, we may not find the global minimum.

[Figures 10.3\(c\)](#) and [\(e\)](#) give cases where the function is continuous, but the gradients are not well-behaved. In [figure 10.3\(c\)](#) we have a gradient that has nearly **vanished**, that is, it is near zero everywhere and gradient descent, with a fixed learning rate, will therefore be slow. [Figure 10.3\(e\)](#)

shows the opposite scenario: the gradient at the minimizer goes to infinity; we call this an **exploding gradient**, and this leads to failures of convergence.

Finally, [figure 10.3\(f\)](#) shows one more problematic case: when there are multiple minima, gradient descent can get stuck in a suboptimal minimum. Which minimum we arrive at will depend on where we initialized x .

10.6.1 Gradient-Like Optimization for Functions without Good Gradients

What about minimizing functions like [figure 10.3\(d\)](#), where the gradient is zero almost everywhere? This is a case where gradient descent truly struggles. However, it is often possible to transform such a problem into one that can be treated with gradient descent. Remember that the key property of a gradient, from the perspective of optimization, is that it is a locally loss-minimizing direction in parameter space. Most gradient-based optimizers don't really need true gradients; instead their update functions are compatible with a broader family of local loss-minimizing directions, $\mathbf{v}$.

Besides the true gradient, what are some other good choices for $\mathbf{v}$? One common idea is to set $\mathbf{v}$ to be the gradient of a **surrogate loss** function, which is a function, J_{sur} , with meaningful (non-zero) gradients, that approximates J . An example might be a smoothed version of J . Another way to get $\mathbf{v}$ is to compute it by sampling perturbations of θ , and seeing which perturbation leads to lower loss. In this strategy, we evaluate $J(\theta + \epsilon)$ for a set of perturbations ϵ , then move toward the ϵ 's that decreased the loss. Approaches of this kind are sometimes called **evolution strategies** [[47](#), [421](#)], and a basic version of this algorithm is given in algorithm 10.4.

Algorithm 10.4: Evolution strategies (ES). Optimizing a cost function $J: \theta \rightarrow \mathbb{R}$ by evolution strategies, i.e., sampling different values for θ and taking a step toward the values that work best.

```

1 Input: objective function  $J$ , initial parameter vector  $\theta^0$ , learning rate  $\eta$ ,
   sampling standard deviation  $\sigma$ , number of samples  $M$ , number of steps  $K$ 
2 Output: trained parameter vector  $\theta^* = \theta^K$ 
3 for  $k = 0, \dots, K - 1$  do
4   for  $i = 1, \dots, M$  do
5      $\epsilon_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
6      $s_i = J(\theta + \sigma \epsilon_i)$ 
7    $\theta^{k+1} \leftarrow \theta^k - \eta \frac{1}{\sigma M} \sum_{i=1}^M s_i \epsilon_i$ 

```

As shown in [figure 10.4](#), this algorithm can successfully minimize the function in [figure 10.3\(c\)](#).

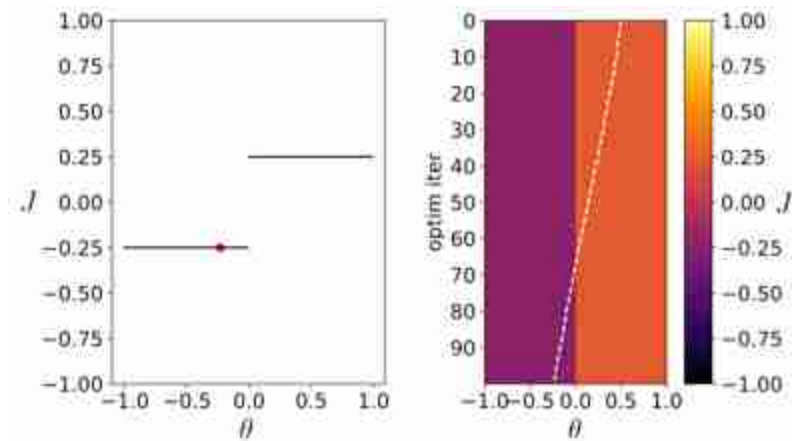


Figure 10.4: Using ES (algorithm 10.4) to minimize a nondifferentiable (zero-gradient) loss, using $\sigma = 1$, $M = 10$, and $\eta = 0.02$.

10.6.2 Gradient Clipping

What about [figure 10.3\(e\)](#), where the gradient explodes near the optimum? Is there anything we can do to improve optimization of this function? To combat exploding gradients, a useful trick is **gradient clipping**, which just means clamping the magnitude of the gradient to some maximum value. Algorithm 10.5 describes this approach.

Algorithm 10.5: Gradient descent with gradient clipping.

```
1 Input: objective function  $J$ , initial parameter vector  $\theta^0$ , learning rate  $\eta$ ,  
   number of steps  $K$ , max gradient magnitude  $m$   
2 Output: trained parameter vector  $\theta^* = \theta^K$   
3 for  $k = 0, \dots, K - 1$  do  
4    $\mathbf{v} = \nabla_{\theta} J(\theta^k)$   
5    $\theta^{k+1} \leftarrow \theta^k - \eta [\text{clip}(v_1, -m, m), \dots, \text{clip}(v_M, -m, m)]^T$ 
```

`clip` is the “clipping” function: $\text{clip}(v, -m, m) = \max(\min(v, m), -m)$

This algorithm indeed successfully minimizes our example of the exploding gradient, as can be seen in [figure 10.5](#).

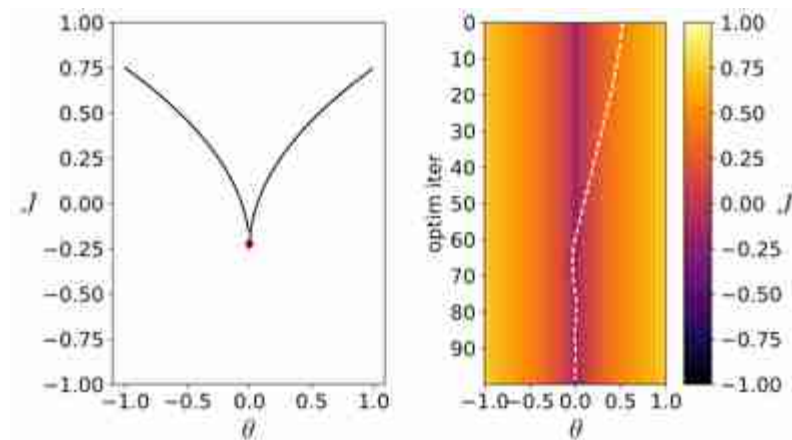


Figure 10.5: Using GD with clipping to minimize a loss with exploding gradients, using $m = 0.1$.

10.7 Stochastic Gradient Descent

One problem with the gradient-based methods we have seen so far is that the gradient may in fact be very expensive to compute, and this is often the case for learning problems. This is because learning problems typically have the form that J is the average of losses incurred on each training datapoint. Computing $\nabla_{\theta} J(\theta)$ requires computing the gradient for each element in the average, that is, the gradient of the function being learned evaluated at the location of each datapoint in the training set. If we train on

a big dataset, say 1 million training points, then to perform just *one* step of gradient descent requires computing 1 million gradients! To make this clear, we will write out J as an explicit function of the training data $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$. For typical learning problems, $\nabla_{\theta} J(\theta, \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N)$ decomposes as follows:

$$\nabla_{\theta} J(\theta, \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N) = \nabla_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}) \quad (10.7)$$

$$= \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \mathcal{L}(f_{\theta}(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}) \quad (10.8)$$

For large N , computing this sum is very expensive. Suppose instead we randomly subsample (without replacement) a *batch* of terms from this sum, $\{\mathbf{x}^{(b)}, \mathbf{y}^{(b)}\}_{b=1}^B$, where B is the **batch size**. We then compute an *estimate* of the total gradient as the average gradient over this batch as follows:

$$\tilde{\mathbf{g}} = \frac{1}{B} \sum_{b=1}^B \nabla_{\theta} \mathcal{L}(f_{\theta}(\mathbf{x}^{(b)}), \mathbf{y}^{(b)}) \quad (10.9)$$

If we sample a large batch, where B is almost as large as N , then the average over the B terms should be roughly the same as the average over all N terms. If we sample a smaller batch, then our estimate of the gradient will be less accurate but faster to compute. Therefore we have a tradeoff between accuracy and speed, and we can navigate this tradeoff with the hyperparameter B . The variant of gradient descent that uses this idea is called **stochastic gradient descent** (SGD), because each iteration of descent uses a different randomly (stochastically) sampled batch of training data to estimate the gradient. The full description of SGD is given in algorithm 10.6.

Algorithm 10.6: Stochastic gradient descent (SGD). Stochastic gradient descent estimates the gradient from a stochastic subset (batch) of the full training data, and makes an update on that basis.

```
1 Input: initial parameter vector  $\theta^0$ , data  $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$ , learning rate  $\eta$ , batch  
   size  $B$ , number of steps  $K$   
2 Output: trained parameter vector  $\theta^* = \theta^K$   
3 for  $k = 0, \dots, K - 1$  do  
4    $\{\mathbf{x}^{(b)}, \mathbf{y}^{(b)}\}_{b=1}^B \sim \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$        $\triangleleft$  sample batch of training data  
5    $\tilde{\mathbf{g}} = \frac{1}{N} \sum_{b=1}^B \nabla_{\theta} \mathcal{L}(f_{\theta}(\mathbf{x}^{(b)}), \mathbf{y}^{(b)})$   
6    $\theta^k \leftarrow \theta^{k-1} - \eta \tilde{\mathbf{g}}$ 
```

SGD has a number of useful properties beyond just being faster to compute than GD. Because each step of descent is somewhat random, SGD can jump over small bumps in the loss landscape, as long those bumps disappear for some randomly sampled batches. Another important property is that SGD can implicitly regularize the learning problem. For example, for linear problems (i.e., f_{θ} is linear), then if there are multiple parameter settings that minimize the loss, SGD will often converge to the solution with minimum parameter norm [517].

10.8 Concluding Remarks

The study of optimization can fill dozens of textbooks and thousands of academic papers. But fortunately for us, modern machine learning has converged on just a few very simple optimization methods that are used in practice. We will soon encounter deep learning, which is the main kind of machine learning used for computer vision. In deep learning, gradient-based optimization is the workhorse. Believe it or not, the handful of algorithms described above are enough to train most state-of-the-art deep learning models. Every year there are new elaborations on these ideas, and second-order methods are ever on the horizon, yet the basic concepts remain quite simple: compute a local estimate of the shape of the loss landscape, then, based on this shape, take a small step toward a lower loss.

11 The Problem of Generalization

11.1 Introduction

So far, we have described learning as an optimization problem: maximize an objective over the *training set*. But this is not our actual goal. Our goal is to maximize the objective over the *test set*. This is the key difference between learning and optimization. We do not have access to the test set, so we use optimization on the training set as a proxy for optimization on the test set.

Learning theory studies the settings under which optimization on the training set yields good results on the test set. In general it may not, since the test set may have different properties from the training set. When we fit to properties in the training data that do not exist in the test data, we call this **overfitting**. When this happens, training performance will be high, but test performance can be very low, since what we learned about the training data does not **generalize** to the test data.

11.2 Underfitting and Overfitting

A learner may perform poorly for one of two reasons: either it failed to optimize the objective on the training data, or it succeeded on the training data but in a way that does not generalize to the test setting. The former is called **underfitting** and the latter is called **overfitting**.

We will walk through a concrete example of a learner, polynomial regression, that exemplifies these two effects. We introduce this learner briefly in the next subsection before exploring what it tells us about underfitting, overfitting, and generalization.

11.2.1 Background: The Polynomial Regression Learning Problem

Polynomial regression is just like linear regression (chapter 9) except that the hypothesis space is polynomial functions rather than linear functions,

that is,

$$y = f_{\theta}(x) = \sum_{k=0}^K \theta_k x^k \quad (11.1)$$

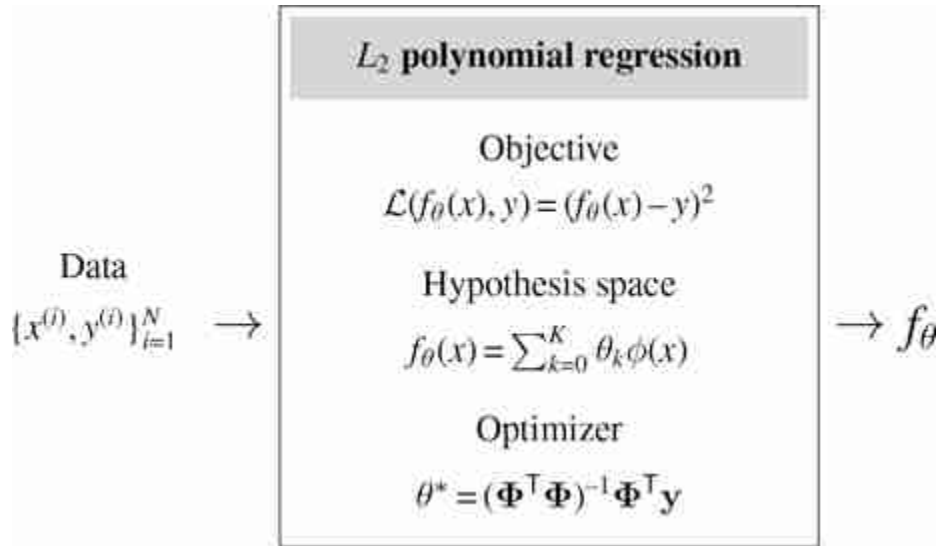
where K , the degree of the polynomial, is a hyperparameter of the hypothesis space.

Let us consider the setting where we use the least-squares (L_2) loss function. It turns out polynomial regression is highly related to linear regression; in fact, we can transform a polynomial regression problem into a linear regression problem! We can see this by rewriting the polynomial as:

$$\sum_{k=0}^K \theta_k x^k = \theta^T \phi(x) \quad (11.2)$$

$$\phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^K \end{bmatrix} \quad (11.3)$$

Now the form of f_{θ} is $f_{\theta}(x) = \theta^T \phi(x)$, which is a linear function in the parameters θ . Therefore, if we featurize x , representing each datapoint x with a feature vector $\phi(x)$, then we have arrived at a linear regression problem in this feature space. So, the learning problem, and closed form optimizer, for L_2 polynomial regression looks almost identical to that of L_2 linear regression:



where

$$\Phi = \begin{bmatrix} 1 & x^{(1)} & x^{(1)2} & \dots & x^{(1)K} \\ 1 & x^{(2)} & x^{(2)2} & \dots & x^{(2)K} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x^{(N)} & x^{(N)2} & \dots & x^{(N)K} \end{bmatrix} \quad (11.4)$$

This same trick works for any kind of function ϕ , not just polynomials. The ϕ could be an expansion into a Fourier basis, for example. The general name for this kind of regression is **basis function regression**; it is equivalent to linear regression on top of *features* of x (i.e., functions of x) rather than directly on x itself.

The matrix Φ is an array of the features (columns) for each datapoint (rows). It plays the same role as data matrix $\mathbf{X}$ did in chapter 9; in fact we often call matrices of the feature representations of each datapoint also as a **data matrix**. As an exercise, you can derive the closed form of the optimizer, given above, using the same steps as we did for linear regression in chapter 9.

When we get to the chapters on neural nets we will see that data matrices appear all over. A neural net is a sequence of transformations of an input data matrix into increasingly more powerful feature representations of the data, i.e. a sequence of better and better data matrices.

11.2.2 Polynomial Regression as a Lens into Generalization

What happens as we increase the order of the polynomial K , that is, we use $K + 1$ basis functions $x^0, \dots, x^K$? With $K = 1$, we arrive back at linear regression. With $K = 2$, we are fitting quadratic functions (i.e., parabolas) to the training data. As K increases, the hypothesis space expands to include ever more curvy fits to the data, as shown in [figure 11.1](#).

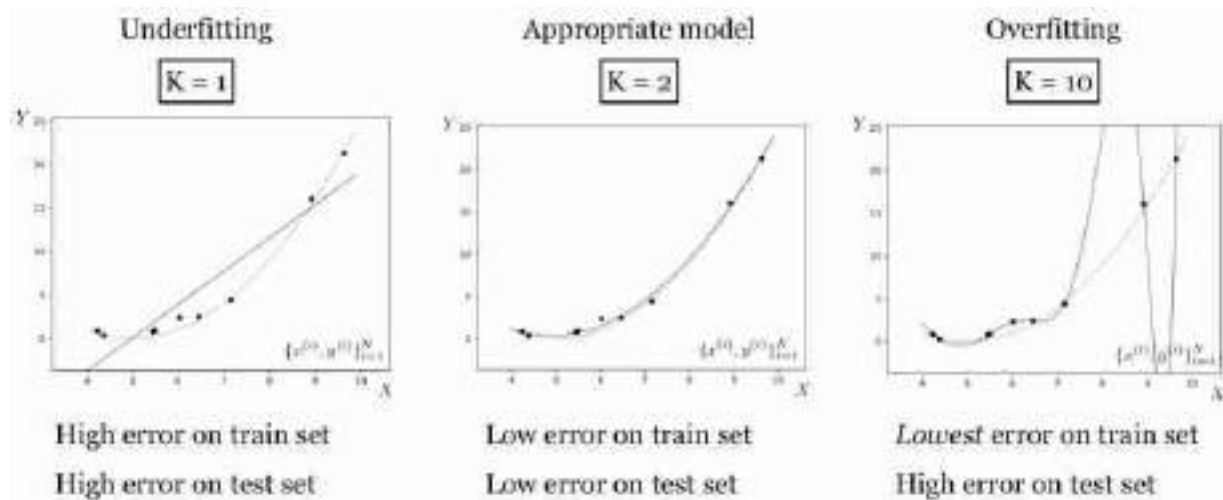


Figure 11.1: Underfitting and overfitting.

The black line is the model's fit. The green line is the ground truth relationship between random variables X and Y . The observed data $\{x^{(i)}, y^{(i)}\}_{i=1}^N$ is the black points, and this data is sampled from the green line plus observation noise. We refer to the full process that generates the data as the **data generating process**:

$$Y = X^2 + 1 \quad \triangleleft \text{true underlying relationship} \quad (11.5)$$

$$\epsilon \sim \mathcal{N}(0, 1) \quad \triangleleft \text{observation noise} \quad (11.6)$$

$$Y' = Y + \epsilon \quad \triangleleft \text{noisy observations} \quad (11.7)$$

$$x, y \sim p(X, Y') \quad \triangleleft \text{data-generating process} \quad (11.8)$$

This data generating process assumes there is no noise in our observations of x . This is a common assumption in least-squares regression problems. Other models relax this assumption; for example, the generative models we will see in chapter 32 model uncertainty in both the x and y observations jointly.

As we increase K we fit the data points better and better, but eventually start *overfitting*, where the model perfectly interpolates the data (passes through every datapoint) but deviates more and more from the true data-generating line. Why does that happen? It's because for $K = 10$ the curve can become wiggly enough to not just fit the true underlying relationship but also to *fit the noise*, the minor offsets ϵ around the green line. This noise is *a property of the training data that does not generalize to the test data*; the test data will have different observation noise. That's what we mean when we say a model is overfitting.

As K grows, a second phenomenon also occurs. For $K = 10$ there are many hypotheses (polynomial functions) that perfectly fit the data (true function + noise) – there is insufficient data for the objective to uniquely identify one of the hypotheses to be the best. Because of this, the hypothesis output by the optimizer may be an arbitrary one, or rather will be due to details of the optimization algorithm (e.g., how it is initialized), rather than selected by the objective. The optimizer we used above has a tendency to pick, among all the equally good hypotheses, one that is very curvy. This is an especially bad choice in this example, because the true function is much more smooth.

Approximation error is the gap between the black line and the training data points. Let $\{x_{(\text{train})}^{(i)}, y_{(\text{train})}^{(i)}\}_{i=1}^N$ be our training data set (the black points). Then the approximation error J_{approx} is defined as the total cost incurred on this training data:

$$J_{\text{approx}} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_{(\text{train})}^{(i)}), y_{(\text{train})}^{(i)}) \quad (11.9)$$

Notice that approximation error is the cost function we minimize in empirical risk minimization (chapter 9).

Generalization error is the gap between the black line and the green line, that is, the expected cost we would incur if we sampled a new test point at random from the true data generating process. Generalization error is often approximated by measuring performance on a heldout **validation dataset**, $\{x_{(\text{val})}^{(i)}, y_{(\text{val})}^{(i)}\}_{i=1}^N$, which can simply be a subset of the data that we don't use for training or testing:

$$J_{\text{gen}} = \mathbb{E}_{x,y \sim p_{\text{data}}} [\mathcal{L}(f_{\theta}(x), y)] \quad (11.10)$$

$$\approx \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_{(\text{val})}^{(i)}), y_{(\text{val})}^{(i)}) \quad (11.11)$$

Approximation error goes down with increasing K but all we really care about is generalization error, which measures how well we will do on test queries that are newly sampled from the true data generating process. For polynomial regression, generalization error obeys a U-shaped function with respect to K : at first it is high because we are underfitting; gradually we fit the data better and better and eventually we overfit, with generalization error becoming high again. [Figure 11.2](#) shows how it looks like for our example.

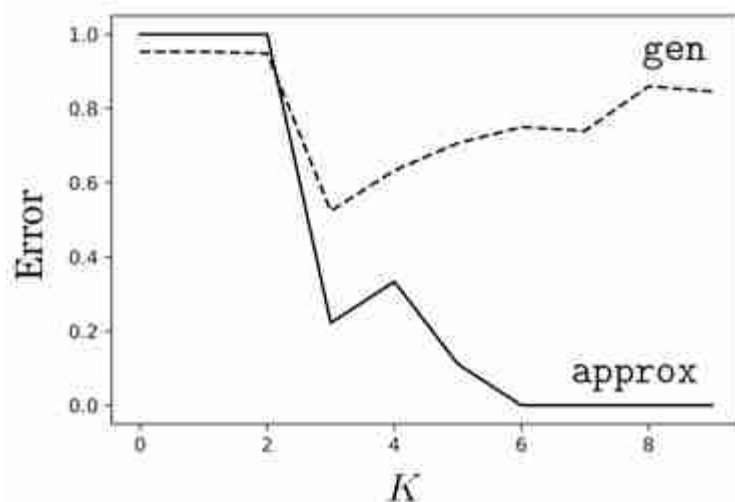


Figure 11.2: Approximation error (approx) versus generalization error (gen) for polynomial regression of order K . Here we measured error as the proportion of validation points that are mispredicted (defined as having an L_2 prediction error greater than 0.25).

This U-shaped curve is characteristic of classical learning regimes like polynomial regression, where we often find that the more parameters a model has, the more it will tend to overfit. However, this behavior does not well characterize modern learners like deep neural networks. Deep networks, which we will see in chapter 12, can indeed either underfit or overfit, but it is not the case that there is a simple relationship between the number of parameters the net has and whether that leads to underfitting

versus overfitting. In fact, bigger deep nets with more parameters may overfit less than smaller nets. We discuss this further in section 11.4.

11.3 Regularization

The previous example suggests a kind of “Goldilocks principle.” We should prefer hypotheses (functions f) that are sufficiently expressive to fit the data, but not so flexible that they can overfit the data.

Regularization refers to mechanisms that penalize function complexity so that we avoid learning too flexible a function that overfits. Typically, regularizers are terms we add to the objective that prefer simple functions in the hypothesis space, all else being equal. They therefore embody the principle of **Occam’s razor**. The general form of a regularized objective is:

$$J(\theta) = \underbrace{\frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x^{(i)}), y^{(i)})}_{\text{data fit loss}} + \underbrace{\lambda R(\theta)}_{\text{regularizer}} \quad \triangleleft \text{regularized objective function} \quad (11.12)$$

where λ is a hyperparameter that controls the strength of the regularization.

One of the most common regularizers is to penalize the L_p norm of the parameters of our model, θ :

$$R(\theta) = \|\theta\|_p, \quad (11.13)$$

The L_p -norm of $\mathbf{x}$ is $(\sum_i |x_i|^p)^{1/p}$. The L_2 -norm is the familiar least-squares objective.

An especially common choice is $p = 2$, in which case the regularizer is called **ridge regularization** (which is also known as **Tikonov regression**). In the context of neural networks, this regularizer is called **weight decay**. When $p = 1$, the regularizer, applied to regression problems, is called **LASSO regression**. For any p , the effect is to encourage most parameters to be zero, or near zero. When most parameters are zero, the function takes on a degenerate form, that is, a simpler form. For example, if we consider the quadratic hypothesis space $\theta_1 x + \theta_2 x^2$, then, if we use a strong L_p regularizer, and if a linear fit is almost perfect, then θ_2 will be forced to zero and the learned function will be linear rather than quadratic. Again, we find

that regularization is an embodiment of Occam's razor: when multiple functions can explain the data, give preference to the simplest.

11.3.1 Regularizers as Probabilistic Priors

Regularizers can be interpreted as **priors** that prefer, a priori (before looking at the data), some solutions over others. Under this interpretation, the data fit loss (e.g., L_2 loss) is a likelihood function $p(\{y^{(i)}\}_{i=1}^N | \{x^{(i)}\}_{i=1}^N, \theta)$ and the regularizer is a prior $p(\theta)$. Bayes' rule then states that the posterior $p(\theta | \{x^{(i)}, y^{(i)}\}_{i=1}^N)$ is proportional to the product of the prior and the likelihood. The log posterior is then the *sum* of the log likelihood and the log prior, plus a constant. Hence we arrive at the form of equation (11.12).

11.3.2 Revisiting the $\star$ Problem

Remember the $\star$ problem from chapter 9?

$$3 \star 2 = 36$$

$$7 \star 1 = 49$$

$$5 \star 2 = 100$$

$$2 \star 2 = 16$$

You may have figured out that $x \star y = (xy)^2$. We said that is the correct answer. But hold on, couldn't it be that $x \star y = 94.5x - 9.5x^2 + 4y^2 - 151$? That also perfectly explains these four examples (trust us, we checked). Or maybe $\star$ is the following Python program ([figure 11.3](#)).

```
def star(x,y):
    if x==2 && y==3:
        return 36
    elif x==7 && y==1:
        return 49
    elif x==5 && y==2:
        return 100
    elif x==2 && y==2:
        return 16
    else:
        return 0
```

Figure 11.3: A function written in Python that solves the $\star$ problem from chapter 9.

That also perfectly fits the observed data. Why didn't you come up with those answers? What made $x \star y = (xy)^2$ more compelling?

We suspect your reason is again Occam's razor, which states that when multiple hypotheses equally well fit the data, you should prefer the simplest. To a human, it may be that $x \star y = (xy)^2$ is the simplest. To a computer, defining a proper notion of simplicity is a hard problem, but can be made precise.

As we saw, most regularizers can be given probabilistic interpretations as priors on the hypothesis, whereas the original objective (e.g., least-squares) measures the likelihood of the data given the hypothesis. These priors are not arbitrarily chosen. The notion of the **Bayesian Occam's razor** derives such priors by noting that more complex hypothesis spaces must cover more possible hypotheses, and therefore must assign less prior mass to any single hypothesis (the prior probability of all possible hypotheses in the hypothesis space must sum to 1) [235, 313]. This is why, probabilistically, simpler hypotheses are more likely to be true.

11.4 Rethinking Generalization

A recent empirical finding is that some seemingly complex hypothesis spaces, such as deep nets (which we will see in later chapters), tend not to overfit, even though they have many more free parameters than the number of datapoints they are fit to. Exactly why this happens is an ongoing topic of research [518]. But we should not be too surprised. The number of

parameters is just a rough proxy for model capacity (i.e., the expressivity of the hypothesis space). A single parameter that has infinite numerical precision can parameterize an arbitrarily complex function. Such a parameter defines a very expressive hypothesis space and will be capable of overfitting data. Conversely, if we have a million parameters, but they are regularized so that almost all are zero, then these parameters may end up defining a simple class of functions, which does not overfit the data. So you can think of the number of parameters as a rough estimate of model capacity, but it is not at all the full story. See [\[42\]](#) for more discussion on this point.

11.5 Three Tools in the Search for Truth: Data, Priors, and Hypotheses

In this section, we will introduce a perspective on learning and generalization that puts all our tools together. We will consider learning as akin to searching for the proverbial needle in a haystack. The haystack is our search space (i.e., the hypothesis space). The needle is the “truth” we are seeking, that is, the true function that generates our observations.

We have several tools at our disposal to help us pinpoint the location of the needle, and in this section we will focus on the following three: *data*, *priors*, and *hypotheses*.

The first tool, *data*, was the topic of chapter 9. In vision, the data is observations of the world like photos and videos. Finding explanations consistent with the observed data is the centerpiece of learning-based vision.

The second tool at our disposal was introduced earlier in this chapter: priors (a.k.a. regularizers) that prefer some solutions over others a priori.

The last tool at our disposal is the set of *hypotheses* under consideration for what the true function may be. The hypothesis space constrains which solutions we can possibly find. If the true solution is not in our hypothesis space, then no amount of data or priors can help us find it. This situation is like the old joke of a drunk man looking for his lost keys under a lamppost [\[137\]](#). “Why are you looking there,” a cop asks. “Because this is where the light is.”

Together these three tools allow us to pinpoint one (or a few) good solutions in the space of all possibilities, as depicted in the cartoon in [figure 11.4](#).

In this cartoon, we are learning a mapping from some domain X to another domain Y . The hypothesis space, F (white circle; “the lamppost’s light”) places a hard constraint on the subset of possible mappings under consideration, the prior (yellow ellipse) places a soft constraint on which mappings are preferred over which others, and the data (green ellipse) also places a soft constraint on the space, preferring mappings that well fit the data.

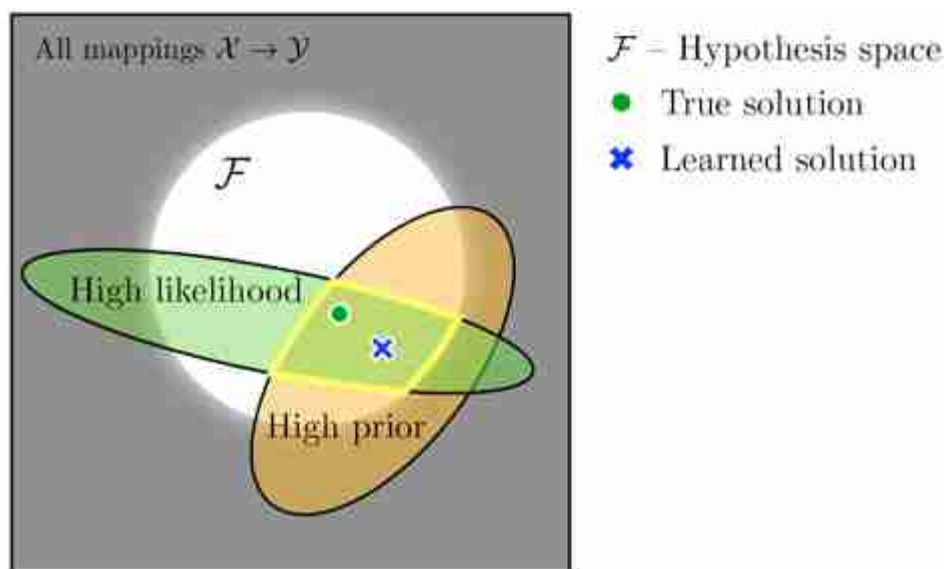


Figure 11.4: A cartoon of the tools for honing in on the truth.

Approximation error is low within the green region. If we didn't care about generalization, then it would be sufficient just to select any solution in this green region. But since we do care about generalization, we bias our picks toward the yellow region, which corresponds to a prior that selects points we believe to be closer to the true solution, even if they might not fit the data perfectly well. These tools isolate the area outlined in bright yellow as the region where we may find our needle of truth. A learning algorithm, which searches over F in order to maximize the likelihood times the prior, will find a solution somewhere in this outlined region.

In the next three sections, we will explore these three tools in more detail through the simple experiment of fitting a curve to a set of datapoints.

11.5.1 Experiment 1: Effect of Data

Training data is the main source of information for a learner, and a learner is just like a detective: as it observes more and more data, it gets more and more evidence to narrow in on the solution. Here we will look at how data shapes the objective function J for the following empirical risk minimization problem:

$$J(\theta; \{x^{(i)}, y^{(i)}\}_{i=1}^N) = \frac{1}{N} \sum_i |f_\theta(x^{(i)}) - y^{(i)}|^{0.25} \quad \triangleleft \text{ objective} \quad (11.14)$$

$$f_\theta(x) = \theta_0 x + \theta_1 \sin(x) \quad \triangleleft \text{ hypothesis space} \quad (11.15)$$

We use the exponent 0.25, rather than the more common squared error, just so that the plots in [figure 11.5](#) show more clearly the linear constraints (the dark lines) added by each datapoint to the objective J .

In [figure 11.5](#), bottom row, we plot J as a heatmap over the values obtained for different settings of θ . On the top row we plot the data being fit, $\{x^{(i)}, y^{(i)}\}_{i=1}^N$, along with the function f_θ that achieves the best fit, and a sample other settings of θ that achieve within 0.1 of the cost of the best fit. Each column corresponds to some amount of training data N . Moving to the right, we increase N .

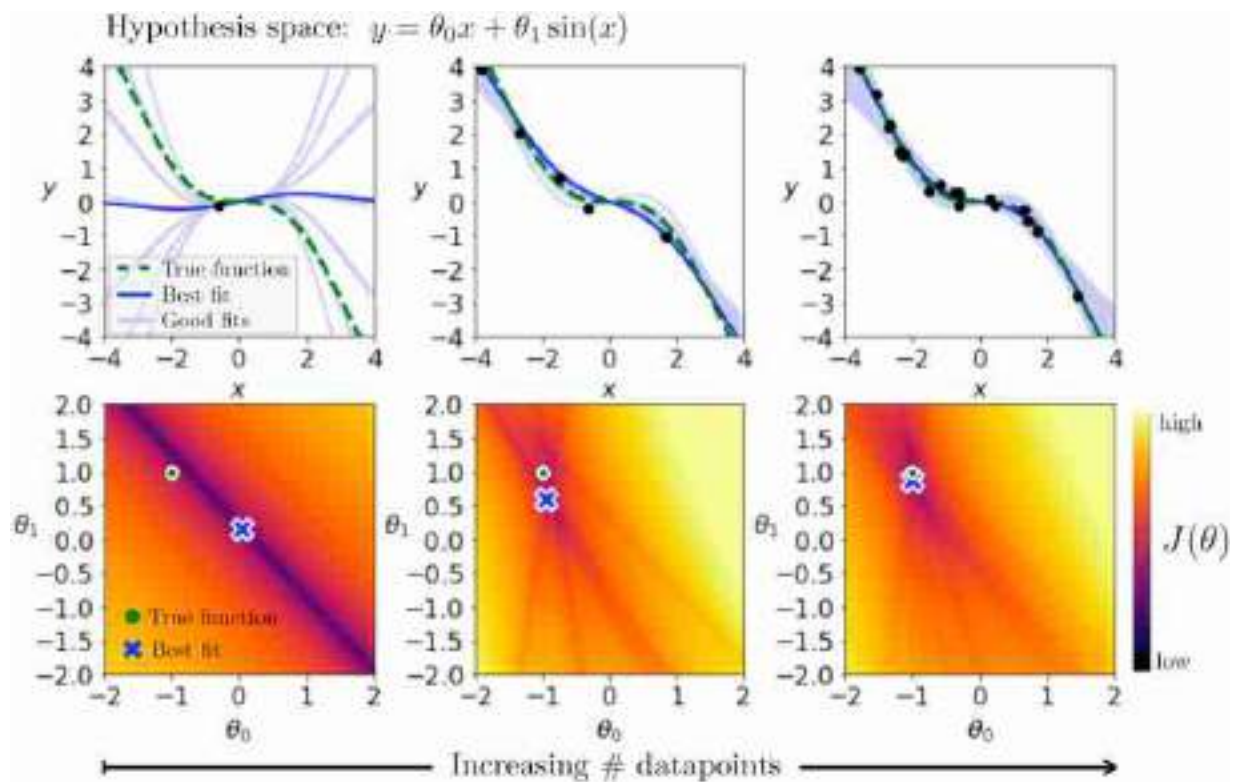


Figure 11.5: More data, more (soft) constraints.

The heatmaps here are reminiscent of a classic computer vision algorithm called the Hough transform [222, 110]. This transform can be used to find geometric primitives that fit feature points in images. In fact, the bottom row is *exactly* the generalized Hough transform [110] of the data points in the top row, using $\theta_0 x + \theta_1 \sin(x)$ as the family of geometric primitives we are fitting.

The first thing to look at is the leftmost column, where $N = 1$. For a single datapoint, there are an infinite number of functions in our hypothesis space that perfectly fit that point. This shows up in the heatmaps as a *line* of settings of θ that all achieve zero loss.

Why a line? Because we have *two* free parameters, $[\theta_0, \theta_1]$, and *one* constraint, $y^{(1)} = \theta_0 x^{(1)} + \theta_1 \sin(x^{(1)})$, so we have one more parameter than constraint yielding a one-dimensional (1D) subspace of solutions.

The learning algorithm we used in this example picks a random solution from the set of solutions that achieve zero loss. Unfortunately, here it got unlucky and picked a solution that happens to be far from the true solution.

Next, look at the second column where we have $N = 5$ training points. Ideally we want to find a curve that exactly passes through each point. Each *single* datapoint adds one constraint to this problem, and each constraint shows up as a line in the heatmap for J [the line of solutions that satisfy the constraint $y^{(i)} = \theta_0 x^{(i)} + \theta_1 \sin(x^{(i)})$ for that datapoint i]. The *intersection* of all these constraints pinpoints the setting of parameters that fits *all* that data. In this example, with five datapoints, there is no location where all the constraint lines intersect, which means there is no curve in our hypothesis space that can perfectly fit this data. Instead we settle for the curve that best approximates the data, according to our loss function. Notice that the learned solution is now pretty close to the true function that generated the data. It is not an exact match, because the data is generated by taking *noisy* samples from the true data generating function.

Finally, let's look at the third column, where we are fitting 20 datapoints. Now the intersection (or, more precisely, average) of the losses for all the datapoints gives a relatively smooth cost function $J(\theta)$, and the learned solution is almost right on top of the true solution. This illustrates a very important point:

The more data you have, the less you need other modeling tools.

With enough data, the true solution will be pinpointed by data alone. However, when we don't have enough data, or when the data is noisy or incorrectly labeled, we can turn to our two other tools, which we will see next.

11.5.2 Experiment 2: Effect of Priors

Now we will run a similar experiment to look at the effect of priors. In this experiment we will use a slightly different hypothesis space and objective function, namely

$$J(\theta; \{x^{(i)}, y^{(i)}\}_{i=1}^N) = \frac{1}{N} \sum_i \|f_\theta(x^{(i)}) - y^{(i)}\|_2^2 + \lambda \|\theta\|_2^2 \quad \triangleleft \text{objective} \quad (11.16)$$

$$f_\theta(x) = \theta_0 x + \theta_1 x \quad \triangleleft \text{hypothesis space} \quad (11.17)$$

We will look at the effect of the ridge regularizer $\|\theta\|_2^2$. We plot the energy landscape and function fit for this problem in [figure 11.6](#), fitting to a single datapoint. The ridge regularizer prefers solutions with small parameter norm, so its contribution to the energy landscape is to place a quadratic

bowl around the origin. As we increase λ (moving left to right in the subplots), the effect of the regularizer becomes stronger and pulls the learned solution closer and closer to the origin.

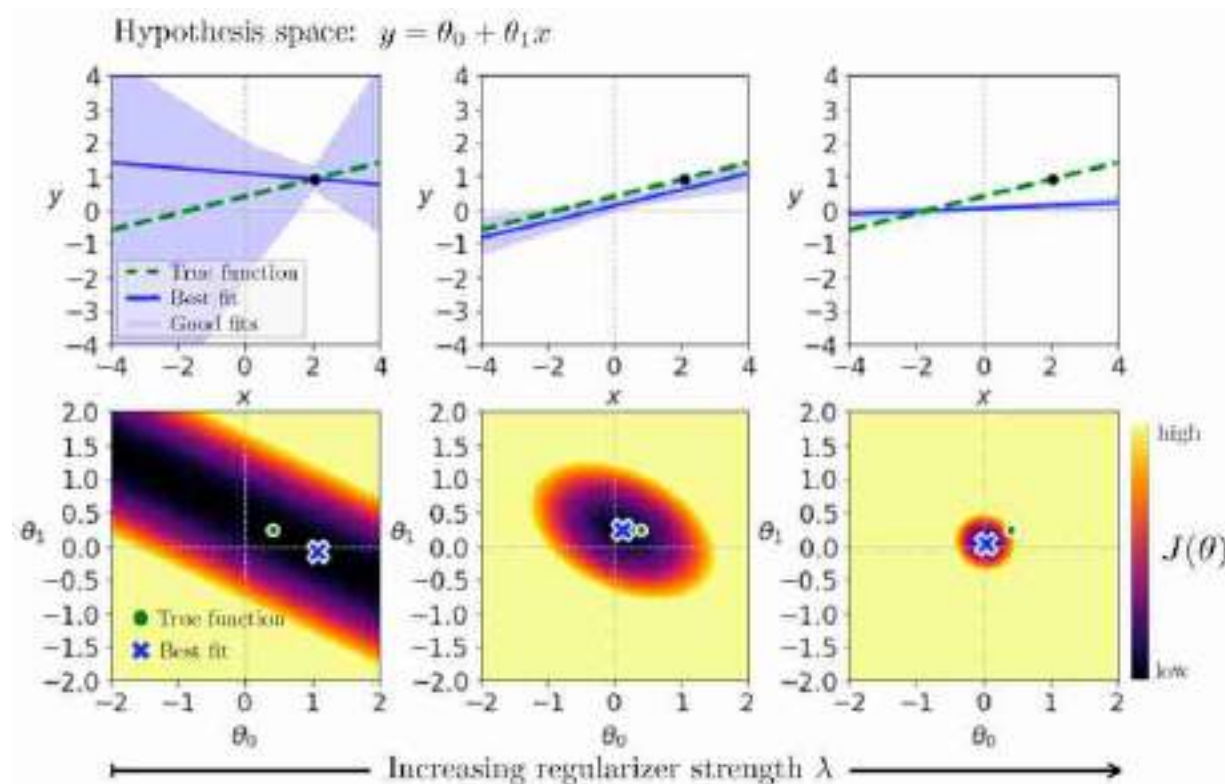


Figure 11.6: More regularization, more (soft) constraints.

In this example, the true solution lies near the origin, so adding some regularization gets us closer to the truth. But using too strong a λ overregularizes; the true solution is not *exactly* $\theta = 0$. The middle column is the Goldilocks solution, where the strength of the regularizer is just right.

You can take away a few lessons from this example:

1. Priors help only when they are good guesses as to the truth.
2. Overreliance on the prior means ignoring the data, and this is generally a bad thing.
3. For any given prior, there is a sweet spot where the strength is optimal. Sometimes this ideal strength can be derived from modeling

assumptions and other times you may need to tune it as a hyperparameter.

11.5.3 Experiment 3: Effect of the Hypothesis Space

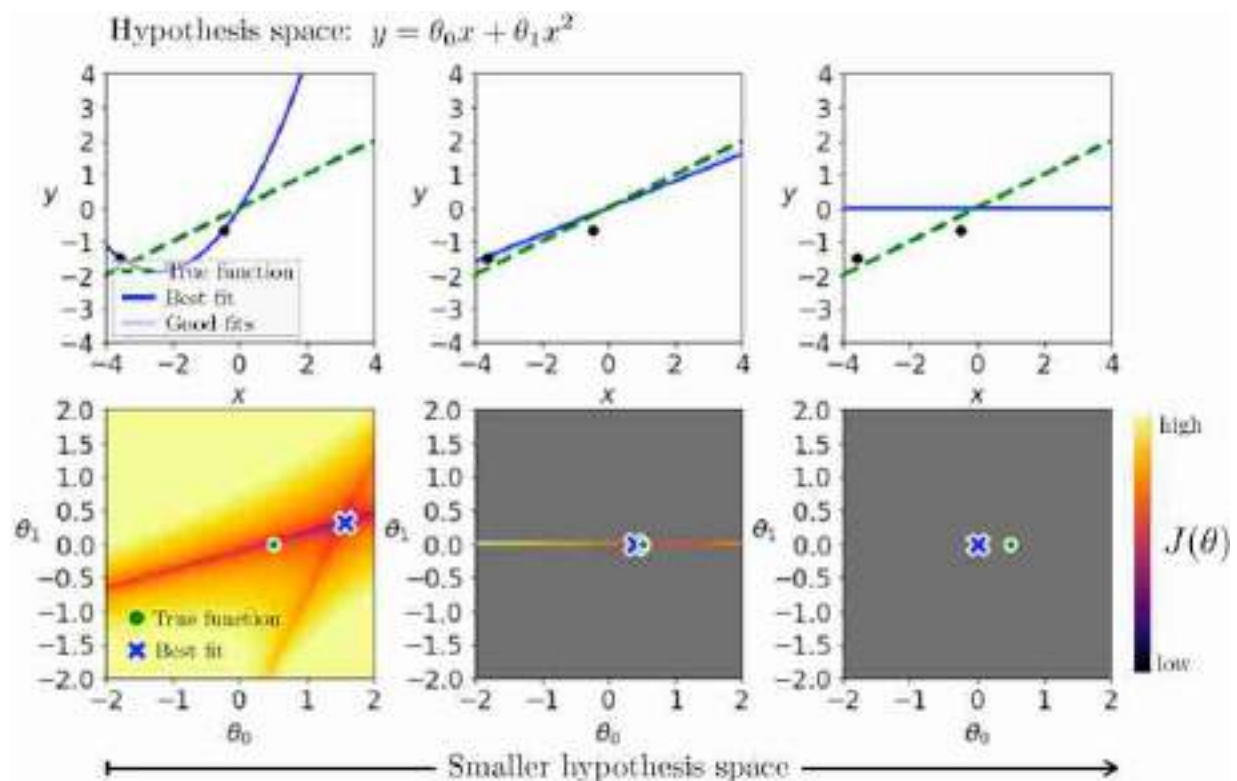
Now we turn to the last of our tools: the hypothesis space itself. For this experiment, we will use the same objective as in equation (11.14) and we will consider the following three hypothesis spaces:

$$f_{\theta}(x) = \theta_0 x + \theta_1 x^2 \quad \triangleleft \text{quadratic} \quad (11.18)$$

$$f_{\theta}(x) = \theta_0 x \quad \triangleleft \text{linear} \quad (11.19)$$

$$f_{\theta}(x) = 0 \quad \triangleleft \text{constant} \quad (11.20)$$

Our experiment on these three spaces is shown in [figure 11.7](#). We show the hypothesis spaces in order of decreasing size moving to the right.



[Figure 11.7](#): Fewer hypotheses, more (hard) constraints.

The true function is linear, and because of that both the quadratic and linear hypothesis spaces contain the true solution. However, the

linear hypothesis space is *much* smaller than the quadratic space. For the linear hypothesis space, searching for the solution (i.e., learning) only considers the slice of parameter space where $\theta_1 = 0$; all the gray region in [figure 11.7](#) can be ignored. Using a smaller hypothesis space can potentially accelerate our search.

However, clearly you can go too far, as is demonstrated by the constant hypothesis space (far right column). This hypothesis space only contains one function, namely $f_\theta(x) = 0$. Search is trivial, but the truth is not in this space.

11.5.4 Summary of the Experiments

These three experiments demonstrate that data, priors, and hypotheses can all constrain our search in similar ways. All three rule out some parts of the full space of mappings and help us focus on others.

This leads us to another general principle:

What can be achieved with any one of our tools can also be achieved with any other.^{[1](#)}

If you don't have much data, you can use strong priors and structural constraints instead. If you don't have much domain knowledge, you can collect lots of data instead. This principle was nicely articulated by Ilya Sutskever when he wrote that “methods ... are extra training data in disguise” [[458](#)].

11.6 Concluding Remarks

The goal of learning is to extract lessons from past data to help on future problem solving. Unless the future is *identical* to the past, this is a problem that requires generalization. One goal of learning algorithms is to make systems that generalize ever better, meaning they continue to work even when the test data is very different than the training data. Currently, however, the systems that generalize in the strongest sense—that work *for all* possible test data—are generally not learned but designed according to other principles. In this way, many classical algorithms still have advantages over the latest learned systems. But this gap is rapidly closing!

1. However, note that the hypothesis space places *hard* constraints on our search; we cannot violate them. Data and priors apply *soft* constraints; we can violate them but we will pay a penalty.

12 Neural Networks

12.1 Introduction

Neural networks are functions loosely modeled on the brain. In the brain, we have billions of neurons that connect to one another. Each neuron can be thought of as a node in a graph, and the edges are the connections from one neuron to the next ([figure 12.1](#)). The edges are directed; electrical signals propagate in just one direction along the wires in the brain.

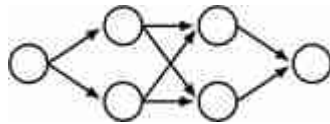


Figure 12.1: A neural network can be drawn as a directed graph.

Outgoing edges are called axons and incoming edges are called dendrites. A neuron fires, sending a pulse down its axon, when the incoming pulses, from the dendrites, exceed a threshold.

12.2 The Perceptron: A Simple Model of a Single Neuron

Let's consider a neuron, shaded in gray, that has four inputs and one output ([figure 12.2](#)).

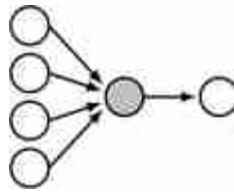


Figure 12.2: Perceptron.

A simple model for this neuron is the **perceptron**. A perceptron is a neuron with n inputs $\{x_i\}_{i=1}^n$ and one output y , that maps inputs to outputs according to the following equations:

$$z = f(\mathbf{x}) = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b \quad \triangleleft \text{linear layer} \quad (12.1)$$

$$g(z) = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{otherwise} \end{cases} \quad \triangleleft \text{activation function} \quad (12.2)$$

$$y = g(f(\mathbf{x})) \quad \triangleleft \text{perceptron} \quad (12.3)$$

In words, we take a weighted sum of the inputs and, if that sum exceeds a threshold (here 0), the neuron fires (outputs a 1). The function f is called a **linear layer** because it computes a linear function of the inputs, $\mathbf{w}^\top \mathbf{x}$, plus a **bias**, b . The function g is called the **activation function** because it decides whether the neuron activates (fires).

Mathematically, f is an affine function, but by convention we call it a “linear layer.” One way to think of it is f is a linear function of $\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}$.

12.2.1 The Perceptron as a Classifier

People got excited about perceptrons in the late 1950s because it was shown that they can learn to classify data [413]. Let's see how that works. We will consider a perceptron with two inputs, x_1 and x_2 , and one output, y . Let the incoming connection weights be $w_1 = 2$, $w_2 = 1$, and $b = 0$. The values of z and y , as a function of x_1 and x_2 , are shown in [figure 12.3](#).

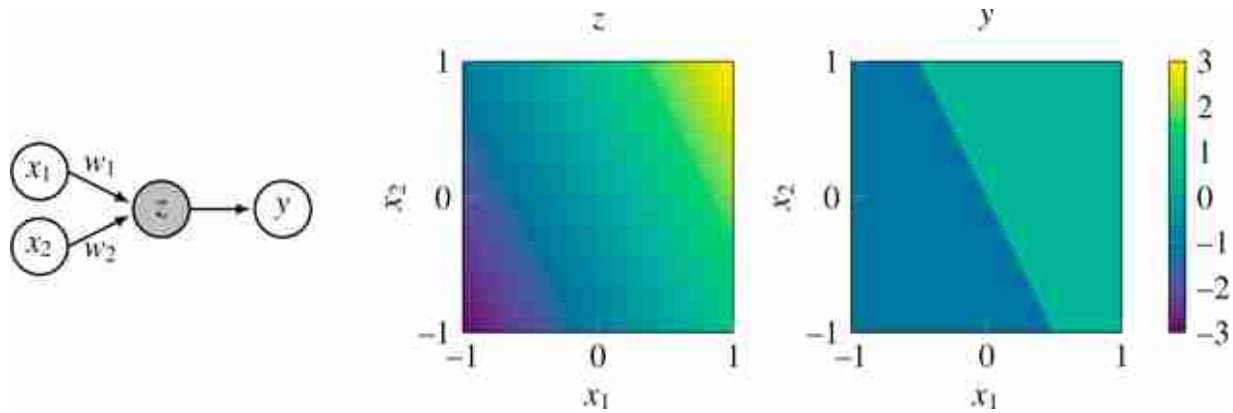


Figure 12.3: Value of hidden unit (z) and output unit (y) in a perceptron, as a function of the input data.

Notice that y takes on values 0 or 1, so you can think of this as a classifier that assigns a class label of 1 to the upper-right half of the plotted region.

12.2.2 Learning with a Perceptron

So a perceptron acts like a classifier, but how can we use it to learn? The idea is that given data, $\{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N$, we will adjust the weights $\mathbf{w}$ and the bias b , in order to minimize a classification loss, L , that scores number of misclassifications:

$$\mathbf{w}^*, b^* = \arg \min_{\mathbf{w}, b} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\mathbf{w}^T \mathbf{x}^{(i)} + b, y^{(i)}) \quad (12.4)$$

In [figure 12.4](#), this optimization process corresponds to shifting and rotating the **decision boundary**, until you find a line that separates data labeled as $y = 0$ from data labeled as $y = 1$.

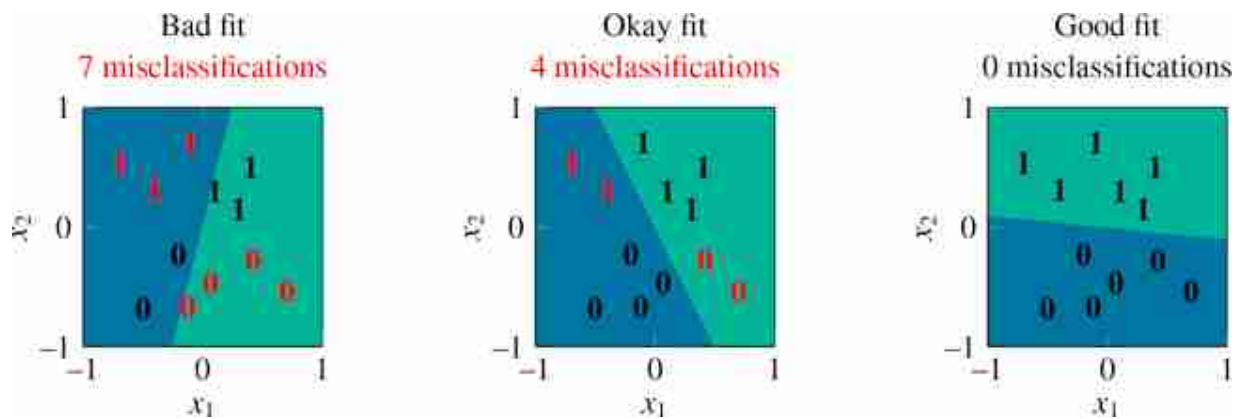


Figure 12.4: Different possible decision surfaces of a perceptron.

You might be wondering, what's the exact optimization algorithm that will find the best line that separates the classes? The original perceptron paper proposed one particular algorithm, the “perceptron learning algorithm.” This was an optimizer tailored to the specific structure of the perceptron. Older papers on neural nets are full of specific learning rules for specific architectures: the delta rule, the Rescorla-Wagner model, and so forth [405]. Nowadays we rarely use these special-purpose algorithms. Instead, we use *general-purpose* optimizers like gradient descent (for differentiable objectives) or zeroth-order methods (for nondifferentiable objectives). The next chapter will cover the backpropagation algorithm, which is a general-purpose gradient-based optimizer that applies to essentially all neural networks we will see in this book (but, note that for the perceptron objective, because it has a nondifferentiable threshold function, we would instead opt for a zeroth order optimizer).

12.3 Multilayer Perceptrons

Perceptrons can solve linearly separable binary classification problems, but they are otherwise rather limited. For one, they only produce a single output. What if we want multiple outputs? We can achieve this by adding edges that fan out after the perceptron ([figure 12.5](#)).

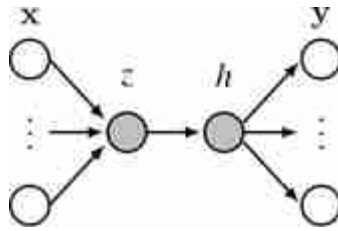


Figure 12.5: Multiple outputs fan out from a neuron.

This network maps an input **layer** of data $\mathbf{x}$ to a layer of outputs $\mathbf{y}$. The neurons in between inputs and outputs are called **hidden units**, shaded in gray. Here, z is a **preactivation** hidden unit and h is a **postactivation** hidden unit, that is, $h = g(z)$ where $g(\cdot)$ is an activation function like in equation (12.2).

More commonly we might have many hidden units in stack, which we call a **hidden layer** ([figure 12.6](#)).

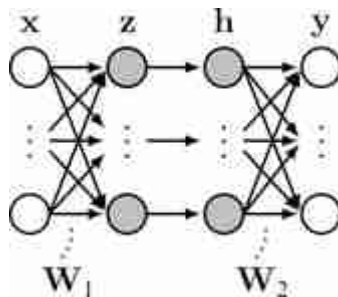


Figure 12.6: Multilayer perceptron.

How many layers does this net have? Some texts will say two [$\mathbf{W}_1$, $\mathbf{W}_2$], others three [$\mathbf{x}$, $\{\mathbf{z}, \mathbf{h}\}$, $\mathbf{y}$], others four [$\mathbf{x}$, $\mathbf{z}$, $\mathbf{h}$, $\mathbf{y}$]. We must get comfortable with the ambiguity.

Because this network has multiple layers of neurons, and because each neuron in this net acts as a perceptron, we call it a **multilayer perceptron (MLP)**. The equation for this MLP is:

$$\mathbf{z} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1 \quad \triangleleft \text{linear layer} \quad (12.5)$$

$$\mathbf{h} = g(\mathbf{z}) \quad \triangleleft \text{activation function} \quad (12.6)$$

$$\mathbf{y} = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2 \quad \triangleleft \text{linear layer} \quad (12.7)$$

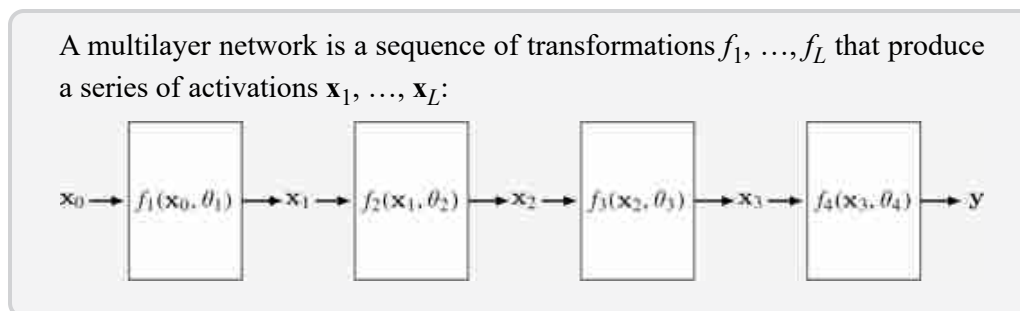
In general, MLPs can be constructed with any number of layers following this pattern: linear layer, activation function, linear layer, activation function, and so on.

The activation function g could be the threshold function like in equation (12.2), but more generally it can be any pointwise nonlinearity, that is, $g(\mathbf{h}) = [\bar{g}(h_1), \dots, \bar{g}(h_N)]$ and $\bar{g}$ is any nonlinear function that maps $\mathbb{R} \rightarrow \mathbb{R}$.

Beyond MLPs, this kind of sequence (linear layer, pointwise nonlinearity, linear layer, pointwise nonlinearity, and so on) is the prototypical motif in almost all neural networks, including most we will see later in this book.

12.4 Activations Versus Parameters

When working with deep nets it's useful to distinguish *activations* and *parameters*. The activations are the values that the neurons take on, $[\mathbf{x}, \mathbf{z}_1, \mathbf{h}_1, \dots, \mathbf{z}_{L-1}, \mathbf{h}_{L-1}, \mathbf{y}]$; slightly abusing notation, we use this term for both preactivation function neurons and postactivation function neurons. The activations are the neural representations of the data being processed. Often, we will not worry about distinguishing between inputs, hidden units, and outputs to the net, and simply refer to all data and neural activations in a network, layer by layer, as a sequence $[\mathbf{x}_0, \dots, \mathbf{x}_L]$, in which case $\mathbf{x}_0$ is the raw input data.



Conversely, parameters are the weights and biases of the network. These are the variables being learned. Both activations and parameters are tensors of variables.

Often we think of a layer as a function $\mathbf{x}_{l+1} = f_{l+1}(\mathbf{x}_l)$, but we can also make the parameters explicit and think of each layer as a function:

$$\mathbf{x}_{l+1} = f_{l+1}(\mathbf{x}_l, \theta_{l+1}) \quad (12.8)$$

That is, each layer takes the activations from the previous layer, as well as parameters of the current layer as input, and produces activations of the next layer. Varying either the input activations or the input parameters will affect the output of the layer. From this perspective, anything we can do with parameters, we can do with activations instead, and vice versa, and that is the basis for a lot of applications and tricks. For example, while normally we learn the values of the parameters, we could instead hold the parameters fixed and learn the values of the activations that achieve some objective. In fact, this is what is done in many methods such as network prompting, adversarial attacks, and network visualization, which we will see in more detail in later chapters.

12.4.1 Fast Activations and Slow Parameters

So what's different about activations versus parameters? One way to think about it is that activations are *fast* functions of a *datapoint*: they are the result of a few layers of processing this datapoint. Parameters are *also* functions of the data (they are learned from data) but they are *slow* functions of *datasets*: the parameters are arrived at via an optimization procedure over a whole dataset. So, both activations and parameters are statistics of the data, that is, information extracted about the data that organizes or summarizes it. The parameters are a kind of metasummary since they specify a functional transformation that produces activations from data, and activations themselves are a summary of the data. [Figure 12.7](#) shows how this looks.

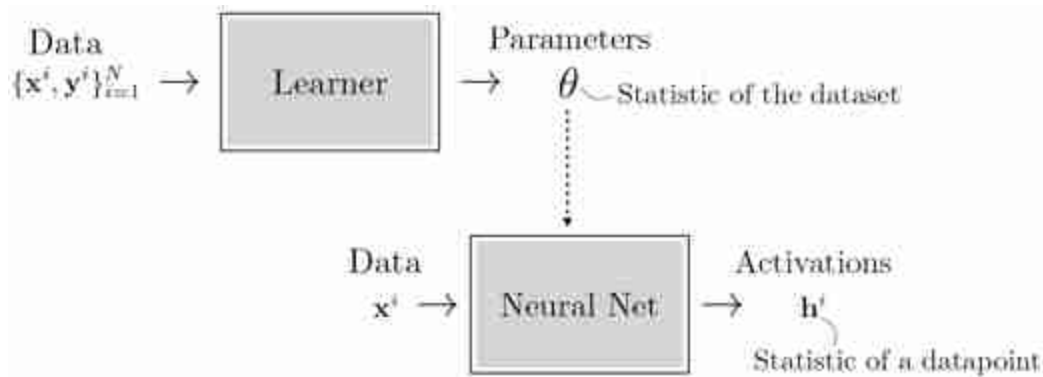


Figure 12.7: Learning is a function that maps a dataset to parameters. Inference, through a neural net, is a function that maps a datapoint to activations.

12.5 Deep Nets

Deep nets are neural nets that stack the linear-nonlinear motif many times ([figure 12.8](#)):

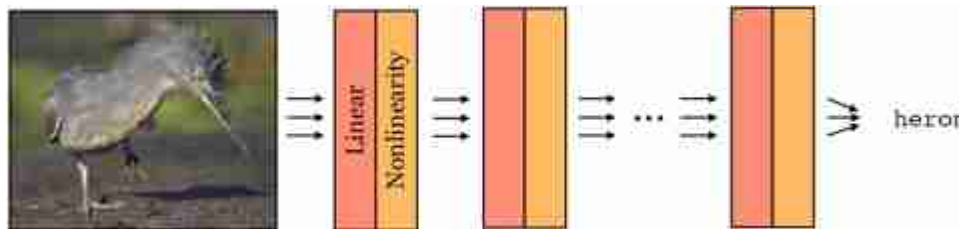


Figure 12.8: Deep nets consist of linear layers interleaved with nonlinearities.

Each layer is a function. Therefore, a deep net is a composition of many functions:

$$f(\mathbf{x}) = f_L(f_{L-1}(\dots f_2(f_1(\mathbf{x})))) \quad (12.9)$$

The L is the number of layers in the net.

These functions are parameterized by weights $[\mathbf{W}_1, \dots, \mathbf{W}_L]$ and biases $[\mathbf{b}_1, \dots, \mathbf{b}_L]$. Some layers we will see later have other parameters. Collectively, we will refer to the concatenation of all the parameters in a deep net as θ .

Deep nets are powerful because they can perform nonlinear mappings. In fact, a deep net with sufficiently many neurons can fit almost any desired

function arbitrarily closely, a property we will investigate further in section 12.5.2.

12.5.1 Deep Nets Can Perform Nonlinear Classification

Let's return to our binary classification problem shown previously, but now let's make the two classes not linearly separable. Our new dataset is shown in [figure 12.9](#).

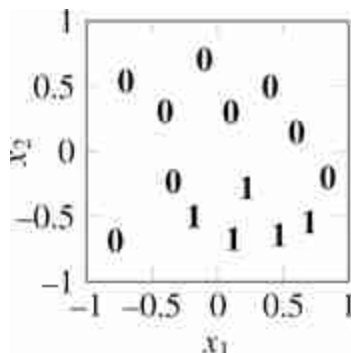


Figure 12.9: Dataset that is not linearly separable.

Here there is no line that can separate the zeros from the ones. Nonetheless, we will demonstrate a multilayer network that can solve this problem. The trick is to just add more layers! We will use the two layer MLP shown in [figure 12.10](#).

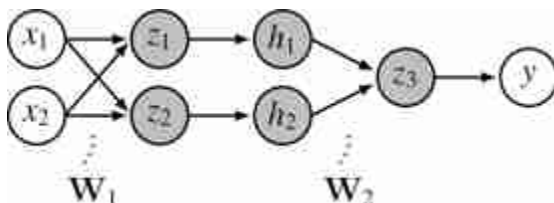


Figure 12.10: A simple MLP network.

Consider using the following settings for $\mathbf{W}_1$ and $\mathbf{W}_2$:

$$\mathbf{W}_1 = \begin{bmatrix} -1 & 1 \\ 1 & 2 \end{bmatrix}, \quad \mathbf{W}_2 = \begin{bmatrix} 1 & -1 \end{bmatrix} \quad (12.10)$$

The full net then performs the following operation:

$$z_1 = x_1 - x_2, \quad z_2 = 2x_1 + x_2 \quad \triangleleft \text{ linear} \quad (12.11)$$

$$h_1 = \max(z_1, 0), \quad h_2 = \max(z_2, 0) \quad \triangleleft \text{ relu} \quad (12.12)$$

$$z_3 = h_1 - h_2 \quad \triangleleft \text{ linear} \quad (12.13)$$

$$y = \mathbb{1}(z_3 > 0) \quad \triangleleft \text{ threshold} \quad (12.14)$$

The $\mathbb{1}$ is the indicator function, which we define as:

$$\mathbb{1}(x) = \begin{cases} 1 & \text{if } x \text{ is true} \\ 0 & \text{otherwise} \end{cases}$$

Here we have introduced a new pointwise nonlinearity, the **Rectified linear unit (relu)**, which is like a graded version of a threshold function, and has the advantage that it yields non-zero gradients over half its domain, thus being amenable to gradient-based learning.

We visualize the values that the neurons take on as a function of x_1 and x_2 in [figure 12.11](#).

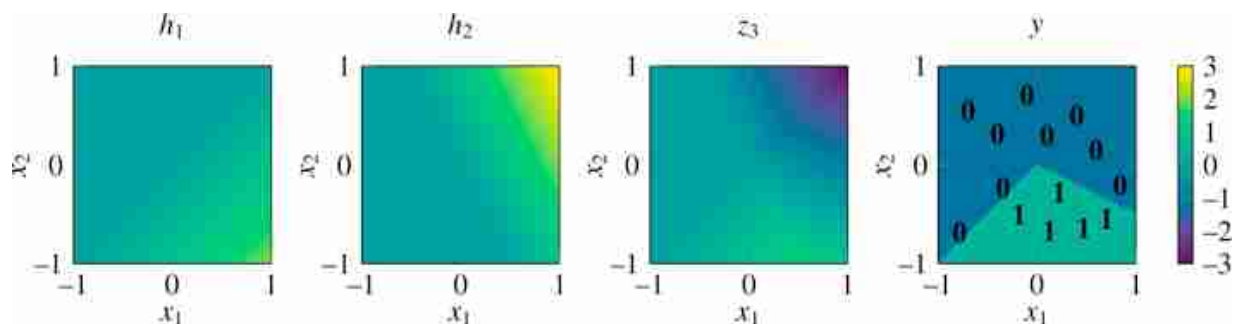


Figure 12.11: Values of hidden units and output unit for the MLP shown in [figure 12.10](#).

As can be seen in the rightmost plot, at the output y , this neural net successfully assigns a value of 1 to the region of the dataspace where the datapoints labeled as 1 live. This example demonstrates that it is possible to solve nonlinear classification problems with a deep net. In practice, we would want to *learn* the parameter settings that achieve this classification. One way to do so would be to enumerate all possible parameter settings and pick one that successfully separates the zeros from the ones. This kind of exhaustive enumeration is a slow process, but don't worry, in chapter 14 we will see how to speed things up using gradient descent. But it's worth remarking that enumeration is always a sufficient solution, at least when the possible parameter values form a finite set.

12.5.2 Deep Nets Are Universal Approximators

Not only can deep nets perform nonlinear classification, they can in principle perform *any* continuous input-output mapping. The **universal approximation theorem** [91] states that this is true even for a network with just a single hidden layer. The caveat is that the number of neurons in the hidden layers will have to be very large in order to fit complicated functions.

Technically, this theorem only holds for continuous functions on compact subsets of $\mathbb{R}^N$ – for example a neural net cannot fit noncomputable functions. We will not be rigorous in this section. We direct the reader to [466] for a formal treatment of universal approximation.

To get an intuition for why this is true, we will consider the case of approximating an arbitrary function from $\mathbb{R} \rightarrow \mathbb{R}$ with a relu-MLP (an MLP with relu nonlinearities). First observe that any function can be approximated arbitrarily well by a sum of indicator functions, that is, bumps placed at different positions:

$$f(x) \approx \sum_i w_i \mathbb{1}(\alpha_i < x < \beta_i) \quad (12.15)$$

As an example, in [figure 12.12](#) we show a curve (blue line) approximated in this way. As the width, $\beta - \alpha$, of the bumps (black lines) goes to zero, the error in the fit goes to zero.

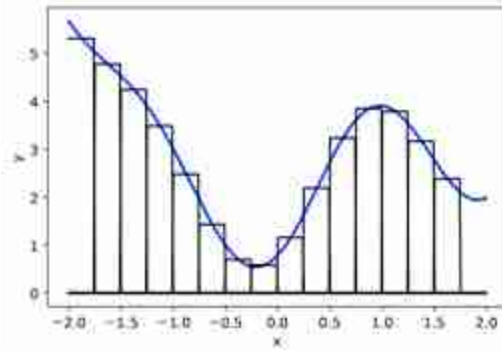


Figure 12.12: Any function from $\mathbb{R} \rightarrow \mathbb{R}$ can be approximated arbitrarily well by a sum of elementary bumps.

While we only consider scalar functions $\mathbb{R} \rightarrow \mathbb{R}$ here, a similar construction can be used to approximate general functions of the form $\mathbb{R}^n \rightarrow \mathbb{R}^m$.

Next we will show that a relu-MLP can represent equation (12.15). The weighted sum $\sum_i w_i \dots$ is the easy part: that's just a linear layer. So we just have to show that we can also write $\mathbb{I}(\alpha < x < \beta)$ using linear and relu layers. It turns out the construction is rather simple:

$$\mathbb{I}(\alpha < x < \beta) \approx \text{relu}\left(\frac{x - (\alpha - \gamma)}{\gamma}\right) - \text{relu}\left(\frac{x - \alpha}{\gamma}\right) - \text{relu}\left(\frac{x - (\beta - \gamma)}{\gamma}\right) + \text{relu}\left(\frac{x - \beta}{\gamma}\right) \quad (12.16)$$

Here we show how a neural net can represent a function as a sum of basis functions. This idea is also foundational in signal processing, where signals are often represented as a sum of sine waves (chapter 16), boxes (figure 21.18), or trapezoids (figure 21.19).

As $\gamma \rightarrow 0$, this approximation becomes exact. The input to each of the four relus in equation (12.16) is an affine function of the input x , hence these four values can be represented by a linear layer with four outputs. Then we apply a relu layer to these four values, and finally we apply a linear layer to compute the sum over these relus (a weighted sum with weights $[1, -1, -1, 1]$). Therefore equation (12.16) can be implemented as a linear-relu-linear network. In [figure 12.13](#), we show an example of constructing a bump in this way.

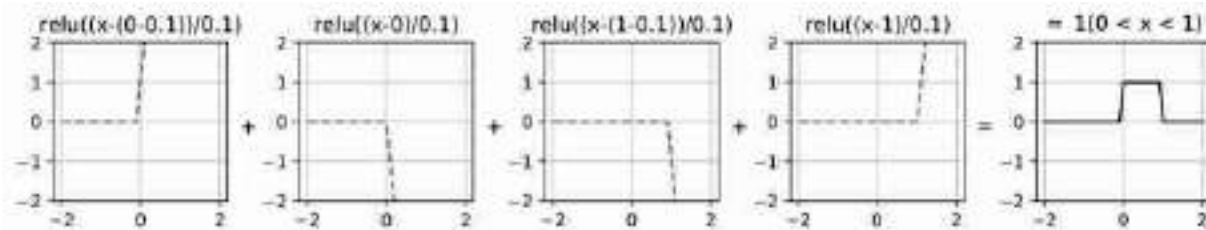


Figure 12.13: A bump can be represented as a weighted sum of shifted and scaled relu functions.

Putting everything together, we have a linear-relu-linear for each bump, followed by a linear layer for summing up all the bumps. The two linear layers in sequence can be collapsed to a single linear layer, and hence the full function can therefore be approximated, to arbitrary precision, by a linear-relu-linear net.

Most literature refers to such a net as having a single hidden layer, using the convention that we don't count pre- and postactivation neurons as separate layers.

Notice that in this approximation, we need four relu neurons for each bump we are modeling. Therefore if we want to approximate a very bumpy function, say with N bumps, we will need $4N$ relu neurons. In general, to achieve arbitrarily good approximation to a curve we may need an unbounded number of neurons in our network.

12.5.3 Depth versus Width

Above we saw that if you have a hidden layer with N neurons, you can fit a scalar function with $O(N)$ bumps. The number of neurons on a single hidden layer is called its **width**.

If different layers have different numbers of neurons, then we may specify the width per layer. Here we will assume all layers have the same width and simply speak of the width of the network.

So, as we increase the width of a network, we can fit ever more complicated functions. What if we instead increase the **depth** of a network, that is, its number of layers? It turns out that this can also be an effective way to increase the capacity of the net, but its effect is a bit different than increasing width.

Interestingly, it is sometimes the case that *deep* nets require far fewer parameters to fit data than *wide* nets. Evidence for this statement comes mostly from empiricism, where researchers have found that deeper nets just work better in practice on many popular problems. However, there is also the beginning of a mathematical theory of when and why this can happen. The basic idea of this theory is to establish that there are certain classes of function that can be represented with a polynomial number of neurons in a depth d network but require an exponential number of neurons in a depth d' network, for certain $d' < d$. Arguments along these lines are called **depth separations**, and the interested reader can refer to [\[465\]](#) to learn more about this ongoing line of research.

12.6 Deep Learning: Learning with Neural Nets

Using the formalism we defined in chapter 9, learning consists of using an *optimizer* to find a function in a *hypothesis space*, that maximizes an *objective*. From this perspective, neural nets are simply a special kind of hypothesis space (and a particular parameterization of that hypothesis space). **Deep learning** refers to learning algorithms that use this parameterized hypothesis space.

Deep learning also typically involves using gradient-based optimization to search the hypothesis space for the best fit to the data. We will investigate this approach in detail in chapter 14, where we will learn about the **backpropagation** algorithm for gradient-based learning with neural nets. However, it is certainly possible to optimize neural nets with other methods, including zeroth-order optimizers like evolution strategies (section 10.6.1; [\[421\]](#)).

One intriguing alternative to backpropagation is called **Hebbian learning** [\[198\]](#). Backpropagation is a *top-down* learning algorithm, where errors incurred at the output (top) of the net are propagated backward to inform earlier layers how to update their weights and biases to minimize the loss, which is a form of learning based on *feedback*. Hebbian learning, in contrast, is a *bottom-up* approach, where neurons wire up just based on the *feedforward* pattern of activity in the net. The canonical learning rule in Hebbian methods is **Hebb's rule**: “fire together, wire together.” That is, we

increase the weight of the connection between two neurons whenever the two neurons are active at the same time. Although this learning rule is not explicitly minimizing a loss function, it has been shown to lead to effective neural representations. For example, Hebb-like rules can learn **infomax** representations, which capture, in the neural activations, as much information as possible about the input signal [300]. Similar rules lead to networks that act like memory banks [215]. Hebbian learning is also of interest because it is considered to be more biologically plausible than backpropagation. This is because Hebb's rule can be computed *locally*—each neuron strengthens and weakens its weights based just on the activity of adjacent neurons—whereas backpropagation requires global coordination throughout the neural network. It is currently unknown how this global coordination can be achieved in biological brains.

12.6.1 Data Structures for Deep Learning: Tensors and Batches

The main data structure that we will encounter in deep learning is the **tensor**, which is just a multidimensional array. This may seem simple, but it's important to get comfortable with the conventions of tensor processing.

In general, everything in deep learning is represented as tensors—the input is one tensor, the activations are tensors, the weights are tensors, the outputs are tensors. If you have data that is not natively represented as a tensor, the first step, before feeding it to a deep net, is to convert it into a tensor format. Most often we use tensor of real numbers, that is, the elements of the tensor are in $\mathbb{R}$.

Suppose we have a dataset $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$ of images $\mathbf{x}$ and labels $\mathbf{y}$. The tensor way of thinking about such a dataset is as two tensors, $\mathbf{X} \in \mathbb{R}^{N \times C_0 \times H \times W}$ and $\mathbf{Y} \in \mathbb{R}^{N \times K}$. The first dimension of the tensor is the number of elements in our dataset. The remaining dimensions are the dimensionality of the images (C_0 color channels by height H by width W) and labels (K -way classification).

The activations in the network are also tensors. For the MLP networks we have seen so far, the activation tensors have shape $N \times C_\ell$, where C_ℓ is the number of neurons on layer ℓ , sometimes also called **channels** in analogy to the color channels of the input image. In later chapters we will encounter other architectures where the activation layers have additional

dimensions, for example, in convolutional networks we will see activation layers that are of shape $N \times C_\ell \times H_\ell \times W_\ell$.

One other important concept is **batch processing**. Normally, we don't process one image at a time through a neural net. Instead we run a *batch* of images all at once, and they are processed in parallel. A batch sampled from the training data can be denoted as $\{\mathbf{x}_{\text{batch}}^{(i)}, \mathbf{y}_{\text{batch}}^{(i)}\}_{i=1}^{N_{\text{batch}}}$, and the batch represented as a tensor has shape $\mathbf{X} \in \mathbb{R}^{N_{\text{batch}} \times C_0 \times H \times W}$ and $\mathbf{Y} \in \mathbb{R}^{N_{\text{batch}} \times K}$.

The weights and biases of the net are also usually represented as tensors. The weights and biases of a linear layer will be tensors of shape $\mathbf{W}_\ell \in \mathbb{R}^{C_{\ell+1} \times C_\ell}$ and $\mathbf{b}_\ell \in \mathbb{R}^{C_{\ell+1}}$.

As an example, in [figure 12.14](#) below, we visualize all the tensors associated with a batch of three datapoints being processed by the MLP from [figure 12.10](#). For this network, the input is not a set of images but instead a set of vectors $\mathbf{X} \in \mathbb{R}^{N_{\text{batch}} \times C_0}$. The output is one value for each input vector, so we have $\mathbf{Y} \in \mathbb{R}^{N_{\text{batch}} \times 1}$.

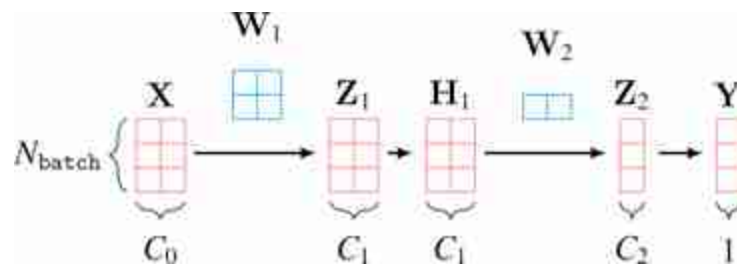


Figure 12.14: The tensors that represent one pass through the MLP in [figure 12.10](#).

where the capital letters are the batches of datapoints and activations corresponding to the lowercase names of datapoints and hidden units in [figure 12.10](#).

This example shows the basic concept of working with tensors and batches for one-dimensional data, but, in vision, most of the time we will be working with higher-dimensional tensors. For image data we typically use four-dimensional tensors: batch $\times$ channels $\times$ height $\times$ width; for videos we may use five-dimensional tensors: batch $\times$ channels $\times$ height $\times$ width $\times$ time. Three-dimensional (3D) scans have an additional *depth* spatial dimension; videos of 3D data could therefore be represented by six-dimensional tensors. As you can see, thinking in terms of two-dimensional

matrices is not quite sufficient. Instead, you should be imagining data processing as operating on N -dimensional tensors, sliced and diced in different ways. As a step in this direction, you may find it useful to visualize tensors in 3D, as shown in [figure 12.15](#).

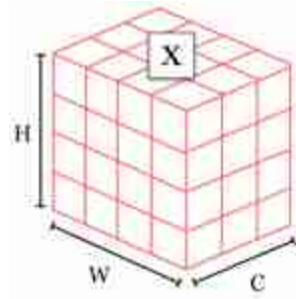


Figure 12.15: A 3D tensor that could represent an $C \times H \times W$ color image.

This is closer to the actual ND tensors vision systems work with, and many concepts can be adequately captured just by thinking in 3D. We will see some examples in later chapters.

12.7 Catalog of Layers

Below, we use the color blue to denote **parameters** and the color red to denote **data/activations** (inputs and outputs to each layer).

We color equations in this way only in this chapter, to make clear the roles of different variables. However, be on the lookout for these colors in figures later in the book. We will often draw activations in red and parameters in blue.

12.7.1 Linear Layers

Linear layers are the workhorses of deep nets. Almost all parameters of the network are contained in these layers; we call these parameters the weights and biases. We have already introduced linear layers previously. They look like this:

$$x_{out} = Wx_{in} + b \quad \triangleleft \text{linear} \quad (12.17)$$

12.7.2 Activation Layers

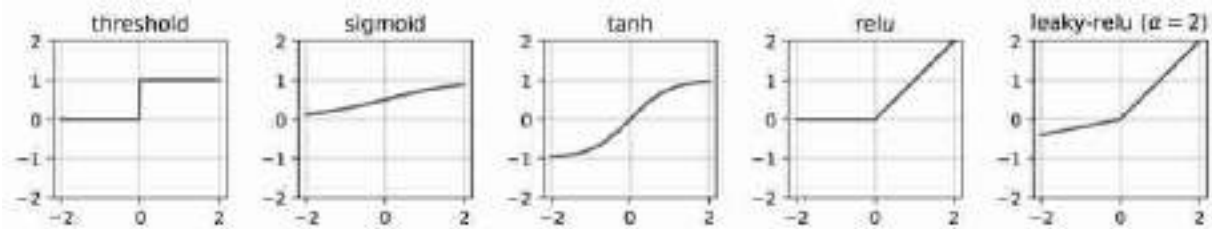


Figure 12.16: Common pointwise nonlinearities.

If a net only contained linear layers then it could only compute linear functions. This is because the composition of N linear functions is a linear function. **Activation layers** add nonlinearity. Activation layers are typically pointwise functions, applying a scalar to scalar mapping on each dimension of the input vector. Typically parameters of these layers, if any, are not learned (but they can be). Some common activation layers are defined below and are plotted in [figure 12.16](#):

$$x_{out}[i] = \begin{cases} 1, & \text{if } x_{in}[i] > 0 \\ 0, & \text{otherwise} \end{cases} \quad \triangleleft \text{threshold} \quad (12.18)$$

$$x_{out}[i] = \frac{1}{1 + e^{-x_{in}[i]}} \quad \triangleleft \text{sigmoid} \quad (12.19)$$

$$x_{out}[i] = 2 * \text{sigmoid}(2 * x_{in}[i]) - 1 \quad \triangleleft \text{tanh} \quad (12.20)$$

$$x_{out}[i] = \max(x_{in}[i], 0) \quad \triangleleft \text{relu} \quad (12.21)$$

$$x_{out}[i] = \begin{cases} \max(x_{in}[i], 0), & \text{if } x_{in}[i] \geq 0 \\ \alpha \min(x_{in}[i], 0), & \text{otherwise} \end{cases} \quad \triangleleft \text{leaky-relu} \quad (12.22)$$

12.7.3 Normalization Layers

Normalization layers add another kind of nonlinearity. Instead of being a pointwise nonlinearity, like in activation layers, they are nonlinearities that perturb each neuron based on the collective behavior of a set of neurons. Let's start with the example of **batch normalization (batchnorm)** [230].

Batchnorm **standardizes** each neural activation with respect to its mean and variance over a batch of datapoints. Mathematically,

$$x_{\text{out}}[i] = \gamma \frac{x_{\text{in}}[i] - \mathbb{E}[x_{\text{in}}[i]]}{\sqrt{\text{Var}[x_{\text{in}}[i]]}} + \beta \quad \triangleleft \text{batchnorm} \quad (12.23)$$

Recall from statistics that the standard score of a draw of a random variable is how many standard deviations it differs from the mean: $z = z = \frac{x - \mu}{\sigma}$.

where γ and β are learned parameters of this layer that maintain expressivity so that the layer can output values with non-zero mean and non-unit variance. Most commonly batchnorm is applied using training batch statistics to compute the mean and variance, which change batch to batch. At test time, aggregate statistics from the training data are used. However, using test batch statistics can be useful for achieving invariance to changes in the statistics from training data to test data [492].

There are numerous other normalization layers that have been defined over the years. Two more that we will highlight are L_2 **normalization** and **layer normalization (layernorm)** [26]. L_2 normalization projects the inputs onto the unit hypersphere, which useful for bounding the activations to unit vectors:

$$x_{\text{out}}[i] = \frac{x_{\text{in}}[i]}{\|x_{\text{in}}\|_2} \quad \triangleleft L_2\text{-norm} \quad (12.24)$$

Layernorm is similar except that it standardizes the vector of input activations:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_{\text{in}}[i] \quad (12.25)$$

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_{\text{in}}[i] - \mu)^2 \quad (12.26)$$

$$x_{\text{out}}[i] = \gamma \frac{x_{\text{in}}[i] - \mu}{\sigma} + \beta \quad \triangleleft \text{layernorm} \quad (12.27)$$

Notice that layernorm, like L_2 -normalization, maps the activation vector to the surface of a hypersphere, but it also centers the activations to have zero mean, and then scales and shifts the activations via γ and β . As an exercise, see if you can write layernorm using L_2 -normalization as one of the steps.

Notice that layernorm also looks quite similar to batchnorm. Both standardize activations but do so with respect to different statistics. Layernorm computes a mean and variance over elements of a datapoint $\mathbf{x}_{\text{in}}$, and will do so separately for each such datapoint in a batch. Batchnorm

computes the mean and variance per channel over datapoints in a batch. If we have a batch stored in the tensor $\mathbf{X} \in \mathbb{R}^{N_{\text{batch}} \times C}$, then what layernorm does looks just like a “transpose” of what batchnorm does. Batchnorm standardizes each element of the tensor by the mean and variance of its column. Layernorm standardizes each element by the mean and variance of its row:

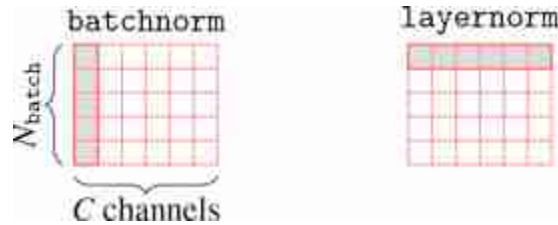


Figure 12.17: Batchnorm vs layernorm. Gray indicates the region over which mean and variance are computed. See also Figure 2 of [511] for more such visualizations.

One issue with batchnorm is that it requires processing a batch of datapoints all at once, and introduces a dependency between each datapoint in the batch. This violates the principle that datapoints should be processed independently and identically (iid), and this can lead to bugs if your method relies on the iid assumption. Layernorm does not have this problem and does indeed process each datapoint in an iid fashion.

12.7.4 Output Layers

The last piece we need is an **output layer** that maps a neural representation—a high-dimensional array of floating point numbers—to a desired output representation. In classification problems, the desired output is a class label, and the most common output operation is the softmax function, which we have already encountered in previous chapters. In image synthesis problems, the desired output is typically a 3D array with dimensions $N \times M \times 3$, and values in the range $[0, 255]$. A sigmoid multiplied by 255 is a typical output transformation for this setting. The equations for these two layers are:

$$A_{\text{out}}[i] = \frac{e^{-\tau z_{\text{out}}[i]}}{\sum_{k=1}^K e^{-\tau z_{\text{out}}[k]}} \quad \leftarrow \text{softmax} \quad (12.28)$$

$$A_{\text{out}}[i] = 255 * \text{sigmoid}(V_{\text{in}}[i]) \quad \leftarrow \text{common layer for image output problems} \quad (12.29)$$

In the softmax definition we have added a **temperature** parameter τ , which is used to scale how peaky, or confident, the predictions are.

The output layer is the input to the loss function, thus completing our specification of the deep learning problem. However, to use the outputs in practice requires translating them into actual pictures, or actions, or decisions. For a classification problem, this might mean taking the argmax of the softmax distribution, so that we can report a single class. For image prediction problems, it might mean rounding each output to an integral value since common image formats represent RGB values as integers.

There are of course many other output transformations you can try. Often, they will be very problem specific since they depend on the structure of the output space you are targeting.

12.8 Why Are Neural Networks a Good Architecture?

As you will soon learn, almost all modern computer vision algorithms involve deep nets in one way or another. So you may be wondering: why are deep nets such a good architecture? We will highlight here five reasons:

1. They are high capacity (big enough nets are universal approximators).
2. They are differentiable (the parameters can be optimized via gradient descent).
3. They have good inductive biases (neural architectures reflect real structure in the world).
4. They run efficiently on parallel hardware.
5. They build abstractions.

Let's look at reasons 1-3 in light of the discussion of searching for truth from chapter 11 (see figure 11.4). Reason 1 relates to the size of the hypothesis space. The hypothesis space can be made very big if we use a large neural network with many parameters. So we can usually be sure that

our true solution (or a close approximation to it) does indeed lie in the space spanned by the neural net architecture. Reason 2 says that searching for the solution within this space is relatively easy, since we can use gradients to direct us toward ever better fits to the data. Reason 3 is one we will only come to appreciate later in the book as we investigate more advanced neural net architectures. It turns out that these architectures impose constraints and regularizers that bias our search toward solutions that capture true structure about the visual world, and this leads to learned solutions that generalize.

Reason 4 is equally important to the first three: it says we can do all of this *efficiently* because most computations can be parallelized on modern hardware; in particular both matrix multiplies (linear layers) and pointwise operations (e.g., relu layers) are easy to parallelize on graphical processing units. Further, most operations are applied to image batches, where each item in the batch can be sent to a different parallel compute node.

Reason 5 is the perhaps the most subtle. It is related to the layered structure of neural nets. Layer by layer, neural nets build up increasingly abstracted representations of the input data, and these abstractions tend to be increasingly useful. This argument is not easy to appreciate at first glance, but it will be a major theme of the subsequent chapters in this book, especially those on representation learning. For now, just keep in mind that the *internal representations* that are built up layer by layer in deep nets are useful and important beyond just the net's overall input-output behavior.

12.9 Concluding Remarks

Neural nets are a very simple and useful parameterized hypothesis space. They are universal approximators that can be trained via gradient descent and run on parallel hardware. *Deep* nets are especially effective in computer vision; as we will soon see, deep architectures can be constructed that specifically reflect structure in the visual world, making visual processing highly efficient and performant. Artificial neural nets also have connections to the real neural nets in our brains. This connection runs deeper than merely sharing a name: the deep net architectures we will see later in this book (e.g., convolutional networks, transformers) are our best current models of computation in animal brains, in the sense that they explain brain data better than any competing models [428]. This is a class of models truly worth paying attention to.

13 Neural Networks as Distribution Transformers

13.1 Introduction

So far we have seen that deep nets are stacks of simple functions, which compose to achieve interesting mappings from inputs to outputs. This section will introduce a slightly different way of thinking about deep nets. The idea is think of each layer as a *geometric transformation of a data distribution*.

13.2 A Different Way of Plotting Functions

Each layer in a deep net is a mapping from one representation of the data to another: $f: \mathbf{x}_{\text{in}} \rightarrow \mathbf{x}_{\text{out}}$. If $\mathbf{x}_{\text{in}}$ and $\mathbf{x}_{\text{out}}$ are both one-dimensional (1D), then we can plot the mapping as a function with $\mathbf{x}_{\text{in}}$ on the x -axis and $\mathbf{x}_{\text{out}}$ on the y -axis ([figure 13.1](#)):

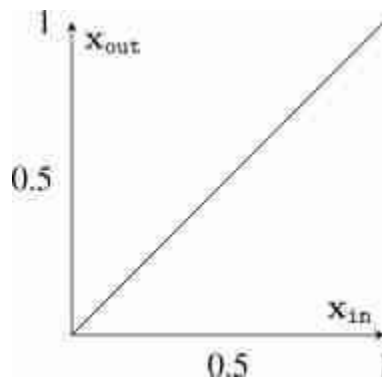


Figure 13.1: The traditional way of plotting the function $\mathbf{x}_{\text{out}} = \mathbf{x}_{\text{in}}$.

Now, we will instead consider a different way of plotting the mapping, where we simply rotate the y -axis to be horizontal rather than vertical

([figure 13.2](#)):

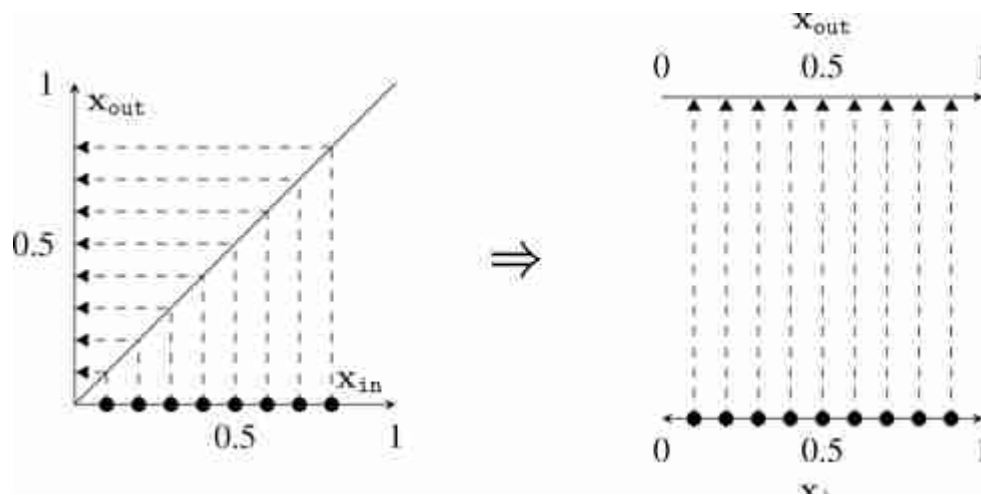


Figure 13.2: An alternative way of plotting a function (right). Functions are mappings that rearrange the input space. The identity function $x_{out} = x_{in}$, shown here, means “no rearrangement,” so the mapping is straight lines.

The depiction to the right makes it obvious that the plot $x_{out} = x_{in}$ is the identity mapping: datapoints get mapped to unchanged positions. [Figure 13.3](#) shows a few more mappings plotted in this way:

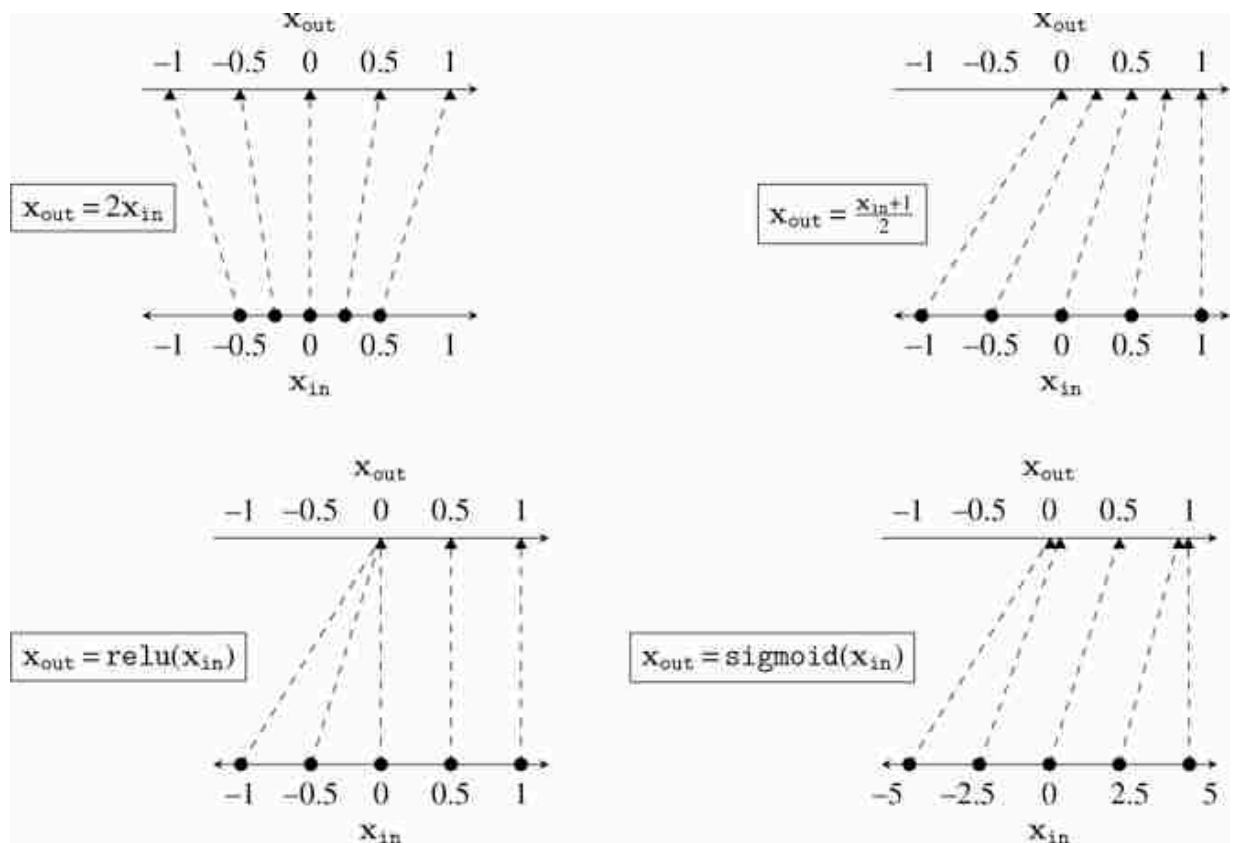


Figure 13.3: Mapping plots for several simple functions that could be neural layers.

Each of the above are layers that could be found in a deep net. Linear layers, like those in the top row above, stretch and squash the data distribution. The relu nonlinearity maps all negative data to 0, and applies an identity map to all nonnegative data. The sigmoid function pulls negative data to 0 and positive data to 1.

13.3 How Deep Nets Remap a Data Distribution

In this way, an incoming data distribution can be reshaped layer by layer into a desired configuration. The goal of a binary softmax classifier, for example, is to move the datapoints around until all the class 0 points end up moved to $[1, 0]$ on the output layer and all the class 1 points end up moved to $[0, 1]$.

This is because the one-hot code for the integer 0 is $[1, 0]$ and the one-hot code for the integer 1 is $[0, 1]$.

A deep net stacks these operations; an example is given in [figure 13.4](#).

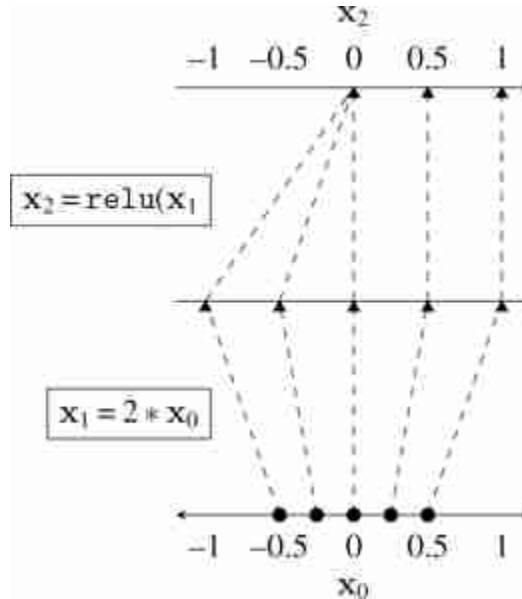


Figure 13.4: Mapping plot for a linear-relu stack.

The plots above show how a uniform grid of datapoints get mapped from layer to layer in a deep net. We can also use this plotting style to show how a nonuniform distribution of incoming datapoints gets transformed. This is the setting in which deep nets actually operate, and sometimes the real action of the network looks very different when viewed this way. We can think of a deep net as transforming an input data distribution, p_{data} , into an output data distribution, p_{out} . Each layer of activations in a network is a different representation or **embedding** of the data, and we can consider the distribution of activations on some layer ℓ to be p_{ℓ} . Then, layer by layer, a deep net transforms p_{data} into p_1 into p_2 , and so on until finally transforming the data to the distribution p_{out} . Most loss functions can also be interpreted from this angle: they penalize the divergence, in one form or another, between the output distribution p_{out} and a target distribution p_{target} .

A nice property of this way of plotting is that it also extends to visualizing two-dimensional (2D)-to-2D mappings (something that conventional x -axis/ y -axis plotting is not well equipped to do). Real deep nets perform N -dimensional (ND)-to-ND mappings, but already 2D-to-2D visualizations can give a lot of insight into the general case. In [figure 13.5](#), we show how three common neural net layers may act to transform a Gaussian blob of data centered at the origin.

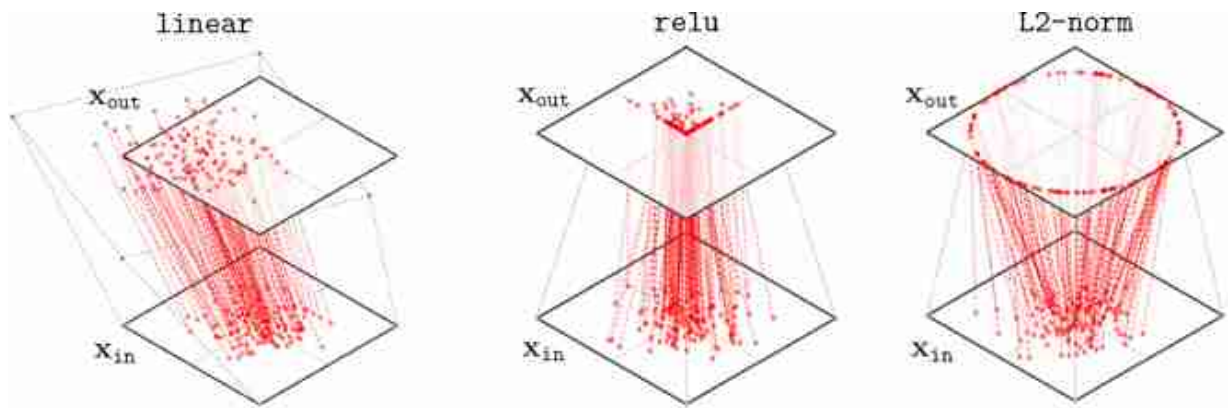


Figure 13.5: 2D mapping diagrams for several neural layers. The `linear` layer mapping will shift, stretch, and rotate depending on its weights and biases.

One interesting thing to notice here is that the `relu` layer maps many points to the axes of the positive quadrant. In general, with `relu`-nets, a lot of data density will build up along these axes, because any point *not* in the positive quadrant snaps to an axis. This effect becomes exaggerated in real networks with high-dimensional embeddings. In particular, for a width- N layer, the region that is strictly positive occupies only $\frac{1}{2^N}$ proportion of the embedding space, so almost the entire space gets mapped to the axes after a `relu` layer. The geometry of high-dimensional neural representations may become very sparse because of this, where most of the volume of representational space is not occupied by any datapoints.

13.4 Binary Classifier Example

Consider a multilayer perceptron (MLP) that performs binary classification formulated as two-way softmax regression. The input datapoints are each in

$\mathbb{R}^2$ and the target outputs are in Δ^1 (the one-simplex), with layer structure linear-relu-linear-relu-linear. This network is drawn below in [figure 13.6](#):

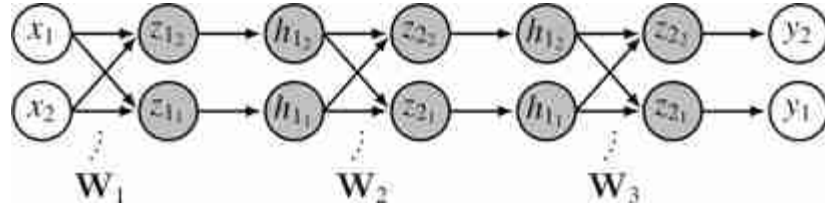


Figure 13.6: An MLP with three linear layers and two outputs, suitable for performing binary softmax regression.

Or expressed in math as follows:

$$z_1 = W_1 x + b_1 \quad \triangleleft \text{linear} \quad (13.1)$$

$$h_1 = \text{relu}(z_1) \quad \triangleleft \text{relu} \quad (13.2)$$

$$z_2 = W_2 h_1 + b_2 \quad \triangleleft \text{linear} \quad (13.3)$$

$$h_2 = \text{relu}(z_2) \quad \triangleleft \text{relu} \quad (13.4)$$

$$z_3 = W_3 h_2 + b_3 \quad \triangleleft \text{linear} \quad (13.5)$$

$$y = \text{softmax}(z_3) \quad \triangleleft \text{softmax} \quad (13.6)$$

Now we wish to train this net to classify between two data distributions. The goal is to transform the input distribution into a target distribution that separates the classes, as shown in [figure 13.7](#).

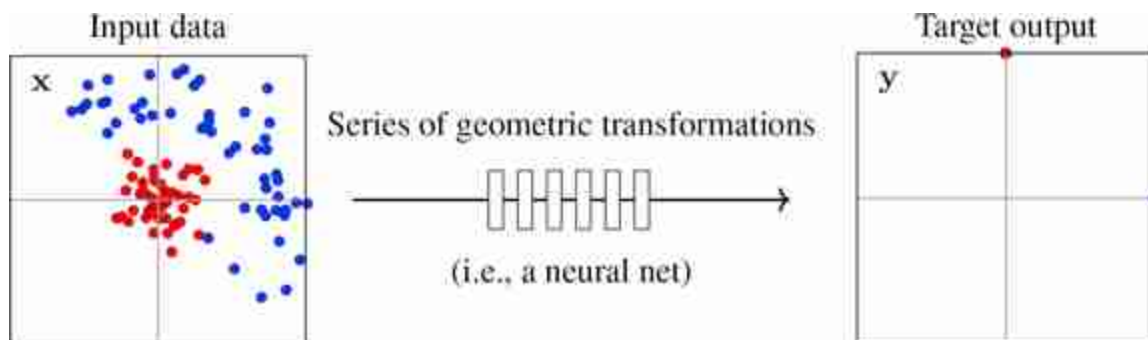


Figure 13.7: The goal of a neural net classifier is to rearrange the input data distribution to match the target label distribution. (left) Input dataset with two classes in red and blue. (right) Target output (one-hot codes).

In this example, the target output places all the red dots at $(0, 1)$ and all the blue dots at $(1, 0)$. These are the coordinates of one-hot codes for our two classes. Training the network consists of find the series of geometric transformations that rearrange the input distribution to this target output distribution.

We will visualize how the net transforms the training dataset, layer by layer, at four **checkpoints** over the course of training.

A checkpoint is a record of the parameters at some iteration of training, that is, if iterates of the parameter vector are $\theta^0, \theta^1, \dots, \theta^T$, then any θ^k can be recorded as a checkpoint.

In [figure 13.8](#), we plot this as a 3D visualization of $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ mappings. Each dotted line connects the representation of a datapoint at one layer to its representation at the next. The gray lines show, for each layer, how a square region around the origin gets stretched, rotated, or squished to map to a transformed region on the next layer.

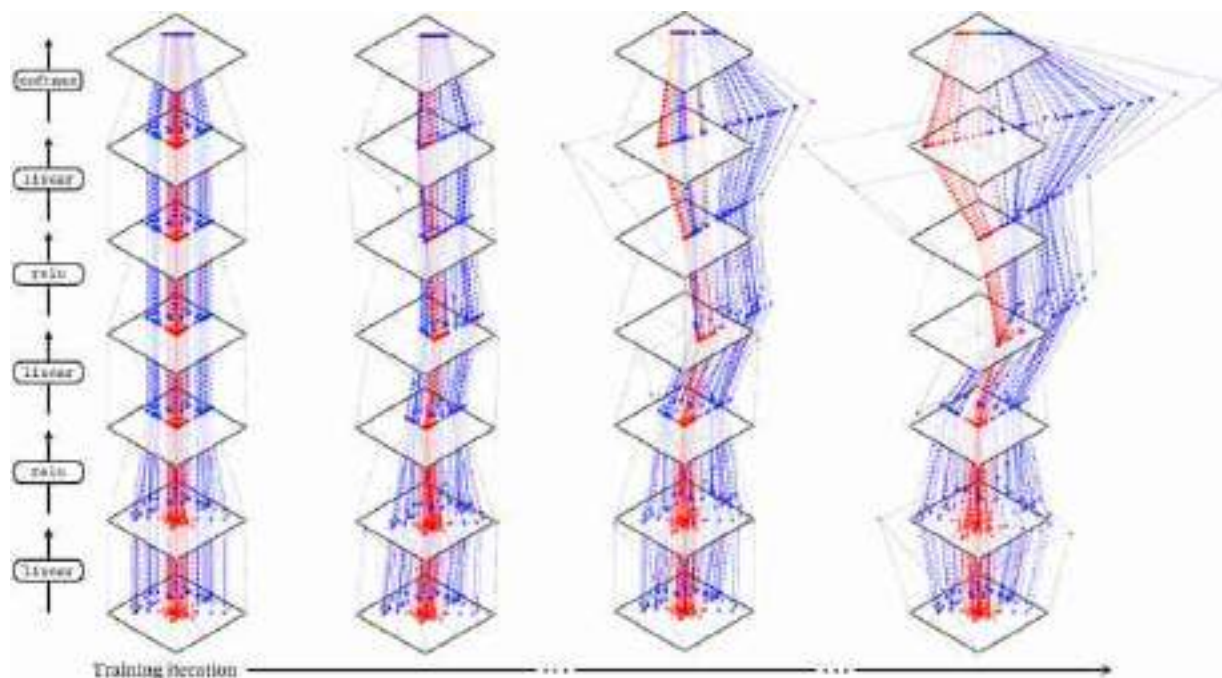


Figure 13.8: How a deep net remaps input data layer by layer. The target output is to move all the red points to $(0, 1)$ and all the blue points to $(1, 0)$ (one-hot codes for the two classes). As training progresses the network gradually achieves this separation.

Of course, “stretched, rotated, or squished” is just an intuitive way to think about it, but we can be more precise. The linear layers perform an affine transformation of the space, while the relu project all negative values to the boundaries of the cone whose dimensions are strictly positive. Layer by layer, over the course of training, the net learns to disentangle these two classes and pull the points toward vertices of the one-simplex, achieving a correct classification of the points!

13.5 How High-Dimensional Datapoints Get Remapped by Deep Net

What if our data and activations are high-dimensional? The plots above only can visualize 1D and 2D data distributions. Deep representations are typically much higher-dimensional than this, and to visualize them, we need to apply tools from **dimensionality reduction**. These tools project the high-dimensional data to a lower dimensionality, for example 2D, which can be visualized. A common objective is to perform the projection such that the

distance between two datapoints in the 2D projection is roughly proportional to their actual distance in the high-dimensional space. In the next plot, [figure 13.9](#), we use a dimensionality reduction technique called **t-Distributed Stochastic Neighbor Embedding t-SNE** [\[312\]](#) to visualize how different layers of a modern deep net represent a dataset of images of different semantic classes, where each color represents a different semantic class. The network we are visualizing is of the transformer architecture, and we will learn about all its details in chapter 26. For now, the important things to note are that (1) we are only showing a selected few of the layers (this net actually has dozen of layers) and (2) the embeddings are high-dimensional (in particular, they are 38,400-dimensional) but mapped down to 2D by t-SNE. Therefore, this visualization is just a rough view of what is going on in the net, unlike the visualizations in the previous section, which showed the exact embeddings and every single step of the layer-to-layer transformations.

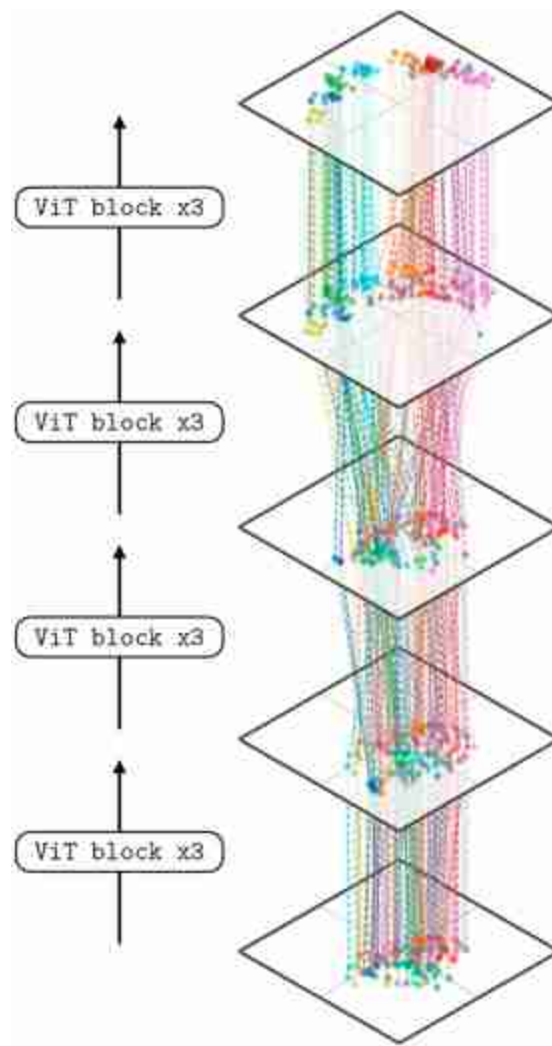


Figure 13.9: How a powerful deep net remaps input images into a disentangled representation where semantic classes (shown in different colors) are separated. This deep net is a vision transformer (ViT [109]), which we will learn about in section 26.7. It was trained using contrastive language-image pre-training (CLIP [394], see section 51.3). Each ViT block contains multiple layers of neural processing (see figure 26.14; we visualize the embeddings right after the first token norm in a block). We apply t-SNE jointly across all shown layers.

Notice that on the first layer, semantic classes are not well separated but by the last layer the representation has **disentangled** the semantic classes so that each class occupies a different part of representational space. This is expected because the final layer in this visualization is near the output of the network, and this network has been trained to output a direct representation of semantics (in particular, this net was trained with contrastive language-image pre-training [CLIP [394]], which is a method

for learning semantic representations that we will learn about in section 51.3).

13.6 Concluding Remarks

Layer by layer, deep nets transform data from its raw format to ever more abstracted and useful representations. It can be helpful to think about this process as a set of geometric transformations of a data distribution, or as a kind of disentangling where initially messy data gets reorganized so that different data classes become cleanly separated.

14 Backpropagation

14.1 Introduction

A key idea of neural nets is to decompose computation into a series of layers. In this chapter we will think of layers as modular blocks that can be chained together into a **computation graph**. [Figure 14.1](#) shows the computation graph for the two-layer multilayer perceptron (MLP) from chapter 12.

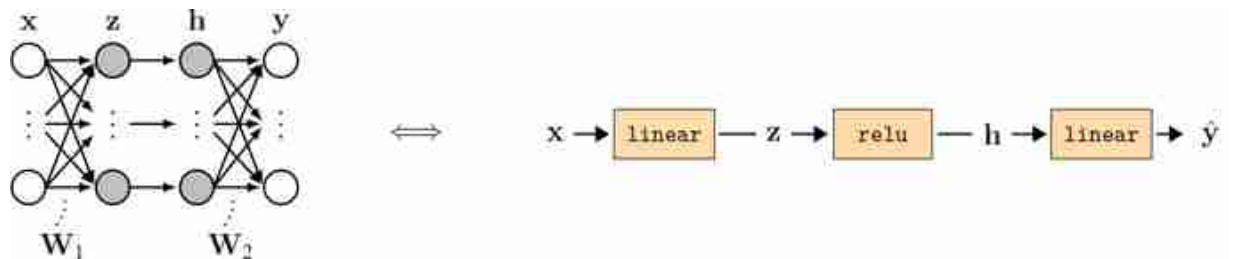


Figure 14.1: In this chapter we will visualize neural nets as a sequence of layers, which we call a computation graph.

Each **layer** takes in some inputs and transforms them into some outputs. We call this the **forward pass** through the layer. If the layer has parameters, we will consider the parameters to be an *input* to a parameter-free transformation:

$$\mathbf{x}_{\text{out}} = f(\mathbf{x}_{\text{in}}, \theta) \quad (14.1)$$

Graphically, we will depict the forward operation of a layer like shown below ([figure 14.2](#)).

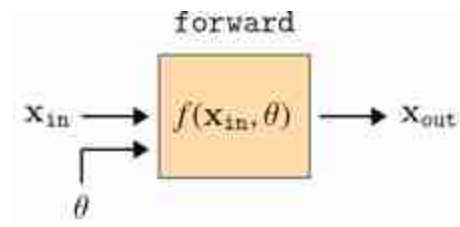


Figure 14.2: Forward operation of a neural net layer.

The learning problem is to find the parameters θ that achieve a desired mapping. Usually we will solve this problem via gradient descent. The question of this chapter is, how do we compute the gradients?

We will use the color █ to indicate *free* parameters, which are set via learning and are not the result of any other processing.

Backpropagation is an algorithm that efficiently calculates the gradient of the loss with respect to each and every parameter in a computation graph. It relies on a special new operation, called `backward` that, just like `forward`, can be defined for each layer, and acts in isolation from the rest of the graph. But first, before we get to defining `backward`, we will build up some intuition about the key trick backpropagation will exploit.

14.2 The Trick of Backpropagation: Reuse of Computation

To start, we will consider a simple computation graph that is a chain of functions $f_L \circ f_{L-1} \circ \dots \circ f_2 \circ f_1$, with each function f_l parameterized by θ_l .

Such a computation graph could represent an MLP, for example, which we will see in the next section.

We aim to optimize the parameters with respect to a loss function L . The loss can be treated as another node in our computation graph, which takes in $\mathbf{x}_L$ (the output of f_L) and outputs a scalar J , the loss. This computation graph appears as follows ([figure 14.3](#)).

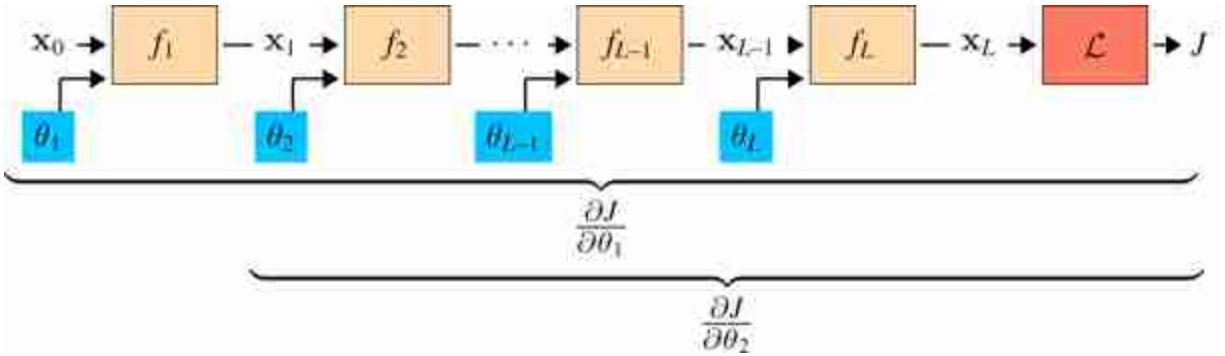
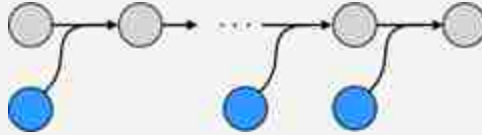


Figure 14.3: Basic sequential computation graph.

This computation graph is a narrow tree; the parameters live on branches of length 1. This can be easier to see when we plot it with data and parameters as nodes and edges as the functions:



The parameters, along with the input training data, are the leaves of the computation graph.

Our goal is to update all the values highlighted in blue: θ_1 , θ_2 , and so forth. To do so we need to compute the gradients $\frac{\partial J}{\partial \theta_1}$, $\frac{\partial J}{\partial \theta_2}$, etc. Each of these gradients can be calculated via the chain rule. Here is the chain rule written out for the gradients for θ_1 and θ_2 :

$$\frac{\partial J}{\partial \theta_1} = \frac{\partial J}{\partial \mathbf{x}_L} \frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_{L-1}} \cdots \frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2} \frac{\partial \mathbf{x}_2}{\partial \mathbf{x}_1} \frac{\partial \mathbf{x}_1}{\partial \theta_1} \quad (14.2)$$

$$\frac{\partial J}{\partial \theta_2} = \frac{\partial J}{\partial \mathbf{x}_L} \frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_{L-1}} \cdots \frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2} \frac{\partial \mathbf{x}_2}{\partial \theta_2} \quad (14.3)$$

Rather than evaluating both equations separately, we notice that all the terms in each gray box are shared. We only need to evaluate this product once, and then can use it to compute both $\frac{\partial J}{\partial \theta_1}$ and $\frac{\partial J}{\partial \theta_2}$. Now notice that this pattern of reuse can be applied in the same way for θ_3 , θ_4 , and so on. This is the whole trick of backpropagation: rather than computing each layer's gradients independently, observe that they share many of the same terms, so we might as well calculate each shared term once and reuse them.

This strategy, in general, is called **dynamic programming**.

14.3 Backward for a Generic Layer

To come up with a general algorithm for reusing all the shared computation, we will first look at one generic layer in isolation, and see what we need in order to update its parameters ([figure 14.4](#)).

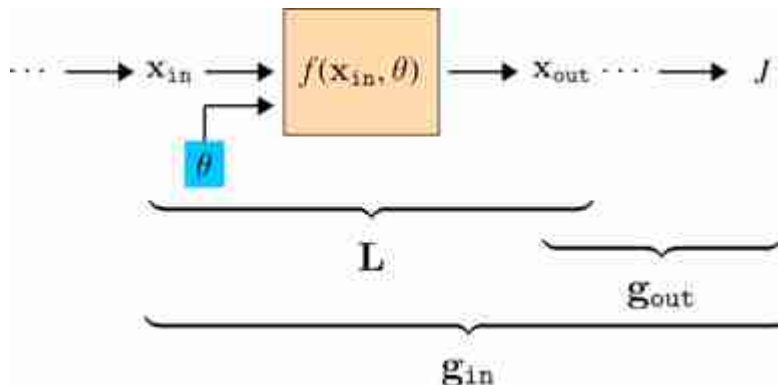


Figure 14.4: A generic layer in the computation graph. The braces represent the part of the computation graph we need to consider in order to evaluate g_{out} , L , and g_{in} .

Here we have introduced two new shorthands, L and g ; these represent arrays of partial derivatives, defined below, and they are the key arrays we need to keep track of to do backprop. They are defined as:

All these arrays represent the gradient *at a single operating point*, namely that of the current value of the data and parameters.

$$\mathbf{g}_l \triangleq \frac{\partial J}{\partial \mathbf{x}_l} \quad \triangleleft \quad \text{grad of cost with respect to } \mathbf{x}_l \quad [1 \times |\mathbf{x}_l|] \quad (14.4)$$

$$\mathbf{L} \triangleq \frac{\partial \mathbf{x}_{\text{out}}}{\partial [\mathbf{x}_{\text{in}}, \theta]} \quad \triangleleft \quad \text{grad of layer} \quad (14.5)$$

$$\mathbf{L}^x \triangleq \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{x}_{\text{in}}} \quad \triangleleft \quad \text{with respect to layer input data} \quad [|\mathbf{x}_{\text{out}}| \times |\mathbf{x}_{\text{in}}|] \quad (14.6)$$

$$\mathbf{L}^\theta \triangleq \frac{\partial \mathbf{x}_{\text{out}}}{\partial \theta} \quad \triangleleft \quad \text{with respect to layer params} \quad [|\mathbf{x}_{\text{out}}| \times |\theta|] \quad (14.7)$$

These arrays give a simple formula for computing the gradient we need, that is, $\frac{\partial J}{\partial \theta}$, in order to update θ to minimize the cost:

$$\frac{\partial J}{\partial \theta} = \underbrace{\frac{\partial J}{\partial \mathbf{x}_{\text{out}}}}_{\mathbf{g}_{\text{out}}} \underbrace{\frac{\partial \mathbf{x}_{\text{out}}}{\partial \theta}}_{\mathbf{L}^\theta} = \mathbf{g}_{\text{out}} \mathbf{L}^\theta \quad (14.8)$$

$$\theta^{i+1} \leftarrow \theta^i - \eta \left(\frac{\partial J}{\partial \theta} \right)^\top \quad \triangleleft \quad \text{update} \quad (14.9)$$

The transpose is because, by convention, θ is a column vector while $\frac{\partial J}{\partial \theta}$ is a row vector; see the Notation section prior to chapter 1.

The remaining question is clear: how do we get $\mathbf{g}_l$ and $\mathbf{L}_l^\theta$ for each layer l ?

Computing $\mathbf{L}$ is an entirely local process: for each layer, we just need to know the functional form of its derivative, f' , which we then evaluate at the operating point $[\mathbf{x}_{\text{in}}, \theta]$ to obtain $\mathbf{L} = f'(\mathbf{x}_{\text{in}}, \theta)$.

Computing $\mathbf{g}$ is a bit trickier; it requires evaluating the chain rule, and depends on all the layers between $\mathbf{x}_{\text{out}}$ and J . However, this can be computed iteratively: once we know $\mathbf{g}_l$, computing $\mathbf{g}_{l-1}$ is just one more matrix multiply! This can be summarized with the following recurrence relation:

$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \mathbf{L}^x \quad \triangleleft \quad \text{backpropagation of errors} \quad (14.10)$$

This recurrence is essence of backprop: it sends error signals (gradients) backward through the network, starting at the last layer and iteratively applying equation (14.10) to compute $\mathbf{g}$ for each previous layer.

Deep learning libraries like Pytorch have a `.grad` field associated with each variable (data, activations, parameters). This field represents $\frac{\partial J}{\partial v}$ for each variable v .

We are finally ready to define the full `backward` function promised at the beginning of this chapter! It consists of the following operation, shown in [figure 14.5](#), which has three inputs ($\mathbf{x}_{\text{in}}$, θ , $\mathbf{g}_{\text{out}}$) and two outputs ($\mathbf{g}_{\text{in}}$ and $\frac{\partial J}{\partial \theta}$).

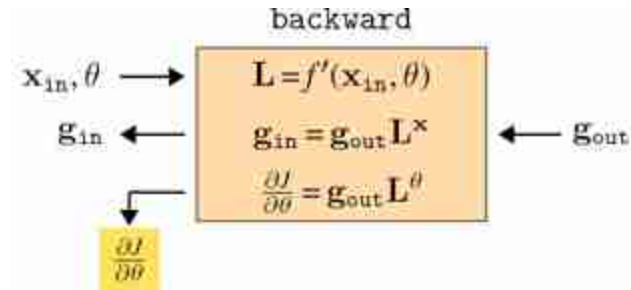


Figure 14.5: `backward` for a generic layer. We use the color to indicate parameter gradients.

14.4 The Full Algorithm: Forward, Then Backward

We are ready now to define the full backprop algorithm. In the last section we saw that we can easily compute the gradient update for θ_l once we have computed $\mathbf{L}_l$ and $\mathbf{g}_l$.

The $\mathbf{g}_l$ and $\mathbf{L}_l$ are the $\mathbf{g}$ and $\mathbf{L}$ arrays for layer l .

So, we just need to order our operations so that when we get to updating layer l we have these two arrays ready. The way to do it is to first compute a **forward pass** through the entire network, which means starting with input data $\mathbf{x}_0$ and evaluating layer by layer to produce the sequence $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_L$. [Figure 14.6](#) shows what the forward pass looks like.



Figure 14.6: Forward pass.

We use the color ■ for data/activations being passed forward through the network.

Next, we compute a **backward pass**, iteratively evaluating the $\mathbf{g}$'s and obtaining the sequence $\mathbf{g}_L, \mathbf{g}_{L-1}, \dots$, as well as the parameter gradients for each layer ([figure 14.7](#)).

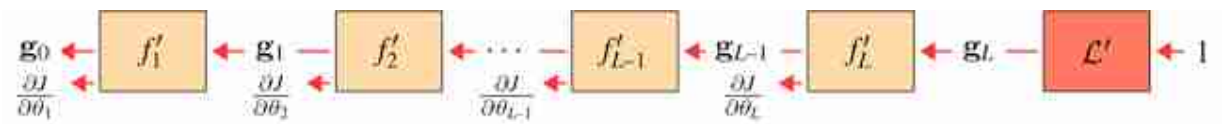


Figure 14.7: Backward pass.

We use the color ■ for data/activation gradients being passed backward through the network.

The full algorithm is summarized in algorithm 14.1.

Algorithm 14.1: Backpropagation (for chain computation graphs). A simple version of the backpropagation algorithm. This will work for the computation graphs we have seen so far, which consist of a series of layers, $f_1 \circ \dots \circ f_L$, with no merging or branching (see section 14.7 for how to handle more complicated graphs with merge and branch operations).

```

1  Input: parameter vector  $\theta = \{\theta_l\}_{l=1}^L, f_1, \dots, f_L, f'_1, \dots, f'_L$ , training datapoint  $\{\mathbf{x}_0, \mathbf{y}\}$ , Loss function  $L: \mathbb{R}^N \rightarrow \mathbb{R}$ 
2  Output: gradient direction  $\frac{\partial J}{\partial \theta} = \left\{ \frac{\partial J}{\partial \theta_l} \right\}_{l=1}^L$ 
3
4  Forward pass:
5  for  $l = 1, \dots, L$  do
6     $\mathbf{x}_l = f_l(\mathbf{x}_{l-1}, \theta_l)$ 
7
8  Backward pass:
9   $\mathbf{g}_L = L'(\mathbf{x}_L, \mathbf{y})$ 
10 for  $l = L, \dots, 1$  do
11    $\mathbf{L}_l = f'_l(\mathbf{x}_{l-1}, \theta_l)$ 
12    $\mathbf{g}_{l-1} = \mathbf{g}_l \mathbf{L}_l^x$ 
13    $\frac{\partial J}{\partial \theta_l} = \mathbf{g}_l \mathbf{L}_l^\theta$ 

```

14.5 Backpropagation Over Data Batches

So far, we have only examined computing the gradient of the loss for a single datapoint, $\mathbf{x}$. As you may recall from chapters 9 and 10, the total cost function we wish to minimize will typically be the *average* of the losses over *all* the datapoints in a training set, $\{\mathbf{x}^{(i)}\}_{i=1}^N$. However, once we know how to compute the gradient for a single datapoint, we can easily compute the gradient for the whole dataset, due to the following identity:

$$\frac{\partial \frac{1}{N} \sum_{i=1}^N J_i(\theta)}{\partial \theta} = \frac{1}{N} \sum_{i=1}^N \frac{\partial J_i(\theta)}{\partial \theta} \quad (14.11)$$

The gradient of a sum of terms is the sum of the gradients of each term.

where $J_i(\theta)$ is the loss for a single datapoint $\mathbf{x}^{(i)}$. Therefore, to compute a gradient update for an algorithm like stochastic gradient descent (section 10.7), we apply backpropagation in batch mode, that is, we run it over each datapoint in our batch (which can be done in parallel) and then average the results.

In the remaining sections, we will still focus only on the case of backpropagation for the loss at a single datapoint. As you read on, keep in mind that doing the same for batches simply requires applying equation (14.11).

14.6 Example: Backpropagation for an MLP

In order to fully describe backprop for any given architecture, we need $\mathbf{L}$ for each layer in the network. One way to do this is to define the derivative f' for all atomic functions like addition, multiplication, and so on, and then expand every layer into a computation graph that involves just these atomic operations. Backprop through the expanded computation graph will then simply make use of all the atomic f' s to compute the necessary $\mathbf{L}$ matrices. However, often there are more efficient ways of writing backward for standard layers. In this section we will derive a compact backward for linear layers and relu layers — the two main layers in MLPs.

14.6.1 Backpropagation for a Linear Layer

The definition of a linear layer, in forward direction, is as follows:

$$\mathbf{x}_{\text{out}} = \mathbf{W}\mathbf{x}_{\text{in}} + \mathbf{b} \quad (14.12)$$

We have separated the parameters into $\mathbf{W}$ and $\mathbf{b}$ for clarity, but remember that we could always rewrite the following in terms of $\theta = \text{vec}[\mathbf{W}, \mathbf{b}]$.

The `vec` is the vectorization operator, which takes numbers in some structured format and rearranges them into a vector.

Let $\mathbf{x}_{\text{in}}$ be N -dimensional and $\mathbf{x}_{\text{out}}$ be M -dimensional; then $\mathbf{W}$ is an $[M \times N]$ dimensional matrix and $\mathbf{b}$ is an M -dimensional vector.

Next we need the gradients of this function, with respect to its inputs and parameters, that is, $\mathbf{L}$. Matrix algebra typically hides the details so we will

instead first write out all the individual scalar gradients:

Here we define $\mathbf{L}^{\mathbf{W}}$ and $\mathbf{L}^{\mathbf{b}}$ as matrices that store the gradients of each output with respect to each weight and bias, respectively. The columns of $\mathbf{L}^{\mathbf{W}}$ index over all the MN weights.

$$\mathbf{L}^{\mathbf{x}}[i, j] = \frac{\partial x_{\text{out}}[i]}{\partial x_{\text{in}}[j]} = \frac{\partial \sum_l \mathbf{W}[i, l] x_{\text{in}}[l]}{\partial x_{\text{in}}[j]} = \mathbf{W}[i, j] \quad (14.13)$$

$$\mathbf{L}^{\mathbf{W}}[i, jk] = \frac{\partial x_{\text{out}}[i]}{\partial \mathbf{W}[j, k]} = \frac{\partial \sum_l \mathbf{W}[i, l] x_{\text{in}}[l]}{\partial \mathbf{W}[j, k]} = \begin{cases} x_{\text{in}}[k], & \text{if } i == j \\ 0, & \text{otherwise} \end{cases} \quad (14.14)$$

$$\mathbf{L}^{\mathbf{b}}[i, j] = \frac{\partial x_{\text{out}}[i]}{\partial \mathbf{b}[j]} = \begin{cases} 1, & \text{if } i == j \\ 0, & \text{otherwise} \end{cases} \quad (14.15)$$

Equations (14.13) and (14.15) imply:

$$\boxed{\mathbf{L}^{\mathbf{x}} = \mathbf{W}} \quad \triangleleft \quad [M \times N] \quad (14.16)$$

$$\boxed{\mathbf{L}^{\mathbf{b}} = \mathbf{I}} \quad \triangleleft \quad [M \times M] \quad (14.17)$$

There is no such simple shorthand for $\mathbf{L}^{\mathbf{W}}$, but that is no matter, as we can proceed at this point to implement backward for a linear layer by plugging our computed $\mathbf{L}^{\mathbf{x}}$ into equation (14.10), and $\mathbf{L}^{\mathbf{W}}$ and $\mathbf{L}^{\mathbf{b}}$ into equation (14.9).

$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \mathbf{L}^{\mathbf{x}} = \mathbf{g}_{\text{out}} \mathbf{W} \quad (14.18)$$

$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{g}_{\text{out}} \mathbf{L}^{\mathbf{W}} \quad (14.19)$$

$$\frac{\partial J}{\partial \mathbf{b}} = \mathbf{g}_{\text{out}} \mathbf{L}^{\mathbf{b}} = \mathbf{g}_{\text{out}} \quad (14.20)$$

To get an intuition for equation (14.18), it can help to draw the matrices being multiplied. Below, in [figure 14.8](#), on the left we have the forward operation of the layer (omitting biases) and on the right we have the backward operation in equation (14.18).



Figure 14.8: The forward and backward matrix multiples for a linear layer.

Unlike the other equations, at first glance $\frac{\partial J}{\partial \mathbf{W}}$ does not seem to have a simple form. A naive approach would be to first build out the large sparse matrix $\mathbf{L}^{\mathbf{W}}$ (which is $[M \times MN]$, with zeros wherever $i \neq k$ in $\mathbf{L}^{\mathbf{W}}[i, jk]$), then do the matrix multiply $\mathbf{g}_{out} \mathbf{L}^{\mathbf{W}}$. We can avoid all those multiplications by zero by observing the following simplification:

$$\frac{\partial J}{\partial \mathbf{W}[i, j]} = \frac{\partial J}{\partial \mathbf{x}_{out}} \frac{\partial \mathbf{x}_{out}}{\partial \mathbf{W}[i, j]} \quad \triangleleft \quad [1 \times M][M \times 1] \rightarrow [1 \times 1] \quad (14.21)$$

$$= \frac{\partial J}{\partial \mathbf{x}_{out}} \left[\frac{\partial x_{out}[0]}{\partial \mathbf{W}[i, j]}, \dots, \frac{\partial x_{out}[M-1]}{\partial \mathbf{W}[i, j]} \right]^T \quad (14.22)$$

$$= \frac{\partial J}{\partial \mathbf{x}_{out}} \left[\dots, 0, \dots, \frac{\partial x_{out}[i]}{\partial \mathbf{W}[i, j]}, \dots, 0, \dots \right]^T \quad (14.23)$$

$$= \frac{\partial J}{\partial \mathbf{x}_{out}} \left[\dots, 0, \dots, x_{in}[j], \dots, 0, \dots \right]^T \quad (14.24)$$

$$= \frac{\partial J}{\partial x_{out}[i]} x_{in}[j] \quad \triangleleft \quad [1 \times 1][1 \times 1] \rightarrow [1 \times 1] \quad (14.25)$$

In matrix equations, it's very useful to check that the dimensions all match up. To the right of some equations in this chapter, we denote the dimensionality of the matrices in the product, where $\mathbf{x}_{in}$ is $\mathbf{M}$ dimensions, $\mathbf{x}_{out}$ is $\mathbf{N}$ dimensions, and the loss J is always a scalar.

Now we can just arrange all these scalar derivatives into the matrix for $\frac{\partial J}{\partial \mathbf{W}}$, and obtain the following:

$$\frac{\partial J}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial J}{\partial \mathbf{W}[0,0]} & \cdots & \frac{\partial J}{\partial \mathbf{W}[N-1,0]} \\ \vdots & \ddots & \vdots \\ \frac{\partial J}{\partial \mathbf{W}[0,M-1]} & \cdots & \frac{\partial J}{\partial \mathbf{W}[N-1,M-1]} \end{bmatrix} \quad (14.26)$$

$$= \begin{bmatrix} \frac{\partial J}{\partial x_{\text{out}}[0]} x_{\text{in}}[0] & \cdots & \frac{\partial J}{\partial x_{\text{out}}[N-1]} x_{\text{in}}[0] \\ \vdots & \ddots & \vdots \\ \frac{\partial J}{\partial x_{\text{out}}[0]} x_{\text{in}}[M-1] & \cdots & \frac{\partial J}{\partial x_{\text{out}}[N-1]} x_{\text{in}}[M-1] \end{bmatrix} \quad (14.27)$$

$$= \mathbf{x}_{\text{in}} \frac{\partial J}{\partial \mathbf{x}_{\text{out}}} \quad (14.28)$$

$$= \mathbf{x}_{\text{in}} \mathbf{g}_{\text{out}} \quad (14.29)$$

Note, we are using the convention of zero-indexing into vectors and matrices.

So we see that in the end this gradient has the simple form of an outer product between two vectors, $\mathbf{x}_{\text{in}}$ and $\mathbf{g}_{\text{out}}$ ([figure 14.9](#)).

$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{x}_{\text{in}} \mathbf{g}_{\text{out}}$$

Figure 14.9: Matrix multiply for parameter gradient of a linear layer.

We can summarize all these operations in the forward and backward diagram for linear layer in [figure 14.10](#).

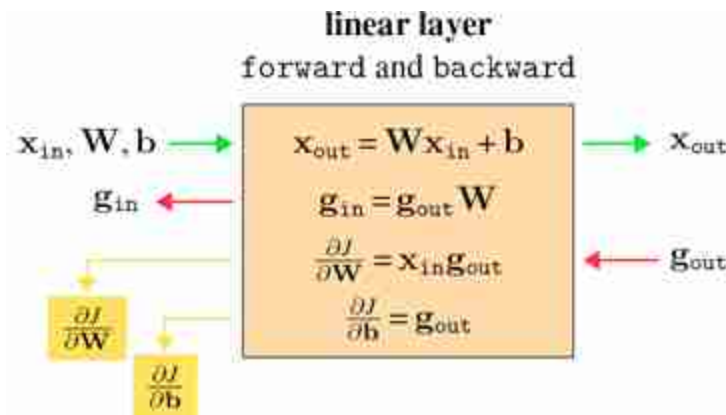


Figure 14.10: Linear layer forward and backward.

Notice that all these operations are simple expressions, mainly involving matrix multiplies. Forward and backward for a linear layer are also very easy to write in code, using any library that provides matrix multiplication (`matmul`) as a primitive. [Figure 14.11](#) gives Python pseudocode for this layer.

```
class linear():
    def __init__(self, W, b, lr):
        self.W = W
        self.b = b
        self.lr = lr # learning rate

    def forward(self, x_in):
        self.x_in = x_in
        return matmul(W, x_in) + b

    def backward(self, J_out):
        J_in = matmul(J_out, W)
        dJdW = matmul(self.x_in, J_out)
        dJdb = J_out
        return J_in, dJdW, dJdb

    def update(self, dJdW, dJdb):
        self.W -= self.lr * dJdW.transpose()
        self.b -= self.lr * dJdb
```

Figure 14.11: Pytorch-like pseudocode for a linear layer with forward and backward.

14.6.2 Backpropagation for a Pointwise Nonlinearity

Pointwise nonlinearities have very simple backward functions. Let a (parameterless) scalar nonlinearity be $h : \mathbb{R} \rightarrow \mathbb{R}$ with derivative function $h' : \mathbb{R} \rightarrow \mathbb{R}$. Define a pointwise layer using h as $f(\mathbf{x}_{\text{in}}) = [h(x_{\text{in}}[0]), \dots, h(x_{\text{in}}[N-1])]^\top$. Then we have

$$\mathbf{L}^* = f'(\mathbf{x}_{\text{in}}) = \text{diag}([h'(x_{\text{in}}[0]), \dots, h'(x_{\text{in}}[N-1])]^\top) \triangleq \mathbf{H}' \quad (14.30)$$

The `diag` is the operator that places a vector on the diagonal of a matrix, whose other entries are all zero.

There are no parameters to update, so we just have to calculate $\mathbf{g}_{\text{in}}$ in the backward operation, using equation (14.10):

$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \mathbf{H}' \quad (14.31)$$

As an example, for a `relu` layer we have:

$$h'(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (14.32)$$

As a matrix multiply, the backward operation is shown in [figure 14.12](#).

$$\begin{matrix} \mathbf{g}_{\text{in}} & = & \mathbf{g}_{\text{out}} & \mathbf{H}' \\ \begin{bmatrix} a & b & c \end{bmatrix} & = & \begin{bmatrix} a & b & c \end{bmatrix} & \begin{bmatrix} a & 0 & 0 \\ 0 & b & 0 \\ 0 & 0 & c \end{bmatrix} \end{matrix}$$

Figure 14.12: Matrix multiply for backward of a pointwise layer.

with $a = h'(x_{\text{in}}[0])$, $b = h'(x_{\text{in}}[1])$, and $c = h'(x_{\text{in}}[2])$. We can simplify this equation as follows:

$$\mathbf{g}_{\text{in}}[i] = \mathbf{g}_{\text{out}}[i] h'(\mathbf{x}_{\text{in}}[i]) \quad \forall i \quad (14.33)$$

The full set of operations for a pointwise layer is shown next in [figure 14.13](#).

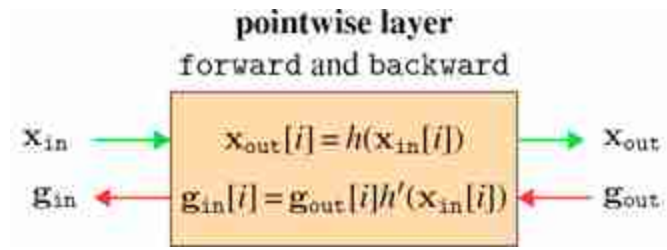


Figure 14.13: Pointwise layer forward and backward

14.6.3 Backpropagation for Loss Layers

The last layer we need to define for a complete MLP is the loss layer. As a simple example, we will derive backprop for an L_2 loss function: $\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$, where $\hat{\mathbf{y}}$ is the output of the network (prediction) and $\mathbf{y}$ is the ground truth.

This layer has no parameters so we only need to derive equation (14.10) for this layer:

$$\mathbf{L}^x = \frac{\partial \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2}{\partial \hat{\mathbf{y}}} = 2(\hat{\mathbf{y}} - \mathbf{y}) \quad \triangleleft \quad [1 \times |\mathbf{y}|] \quad (14.34)$$

$$\mathbf{g}_{in} = \mathbf{g}_{out} * 2(\hat{\mathbf{y}} - \mathbf{y}) = 2(\hat{\mathbf{y}} - \mathbf{y}) \quad (14.35)$$

Here we have made use of the fact that $\mathbf{g}_{out} = \frac{\partial J}{\partial \mathbf{x}_{out}} = \frac{\partial J}{\partial J} = 1$, since the output of the loss layer *is* the cost J .

So, the backward signal sent by the L_2 loss layer is a row vector of per-dimension errors between the prediction and the target.

This completes our derivation of forward and backward for a L_2 loss layer, yielding [figure 14.14](#).

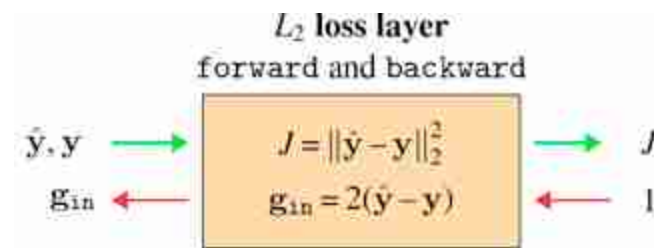


Figure 14.14: L_2 loss layer forward and backward

14.6.4 Putting It All Together: Backpropagation through an MLP

Let's see what happens when we put all these operations together in an MLP. We will start with the MLP in [figure 14.1](#). For simplicity, we will omit biases. Let $\mathbf{x}$ be four-dimensional and $\mathbf{z}$ and $\mathbf{h}$ be three-dimensional, and $\hat{\mathbf{y}}$ be two-dimensional. The forward pass for this network is shown below in [figure 14.15](#).

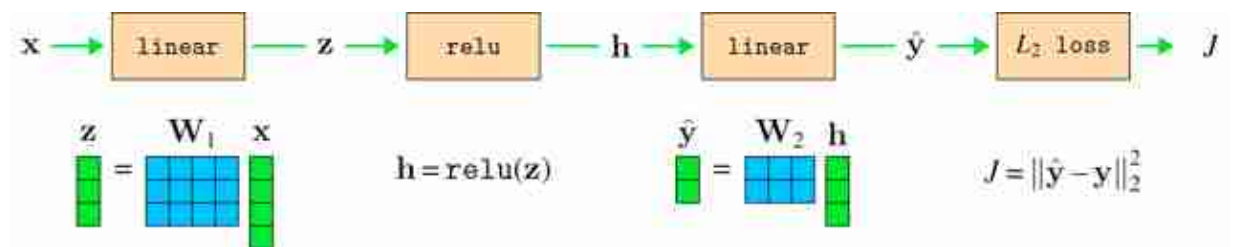


Figure 14.15: Forward pass through an MLP.

For the backward pass, we will here make a slight change in convention, which will clarify an interesting connection between the forward and backward directions. Rather than representing gradients $\mathbf{g}$ as row vectors, we will transpose them and treat them as column vectors. The backward operation for transposed vectors follows from the matrix identity that $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$:

$$\mathbf{g}_{\text{in}}^T = (\mathbf{g}_{\text{out}} \mathbf{W})^T = \mathbf{W}^T \mathbf{g}_{\text{out}}^T \quad (14.36)$$

Now we will draw the backward pass, using these transposed $\mathbf{g}$'s, in [figure 14.16](#).

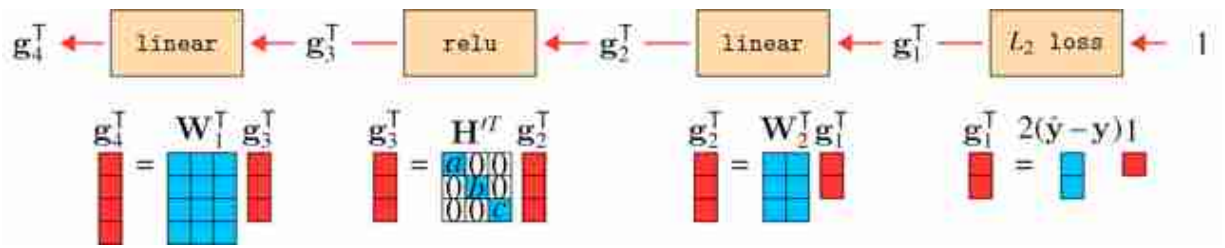


Figure 14.16: Backward pass through an MLP.

This reveals an interesting connection between forward and backward for linear layers: backward for a linear layer is the same operation as forward, just with the weights transposed! We have omitted the bias terms here, but recall from equation (14.18) that the backward pass to the activations ignores biases anyway.

In contrast, the relu layer is not a relu on the backward pass. Instead, it becomes a sort of gating matrix, parameterized by functions of the activations from the forward pass (a , b , and c). This matrix is all zeros except for ones on the diagonal where the activation was nonnegative. This layer acts to mask out gradients for variables on the negative side of the relu . Notice that this operation is a matrix multiply — in fact, *all* backward operations are matrix multiplies, no matter what the forward operation might be, as you can observe in algorithm 14.1.

In the diagrams above, we have not yet included the computation of the parameter gradients. At each step of the backward pass, these are computed as an outerproduct between the activations input to that layer, $\mathbf{x}_{\text{in}}$, and the gradients being sent back, $\mathbf{g}_{\text{out}}$ (see [figure 14.9](#)).

14.6.5 Forward-Backward Is Just a Bigger Neural Network

In the previous section we saw that the backward pass through an neural network can itself be implemented as another neural network, which is in fact a linear network with parameters determined by the weight matrices of the original network as well as by the activations in the original network. Therefore, the full backward pass is a neural network! Since the forward pass is also a neural network (the original network), the full backpropagation algorithm—a forward pass followed by a backward pass—can be viewed as just one big neural network. The parameter gradients can

be computed from this network via one additional matrix multiply (matmul) for each layer of the layer of the backward network. The full network, for a three-layer MLP, is shown in [figure 14.17](#).

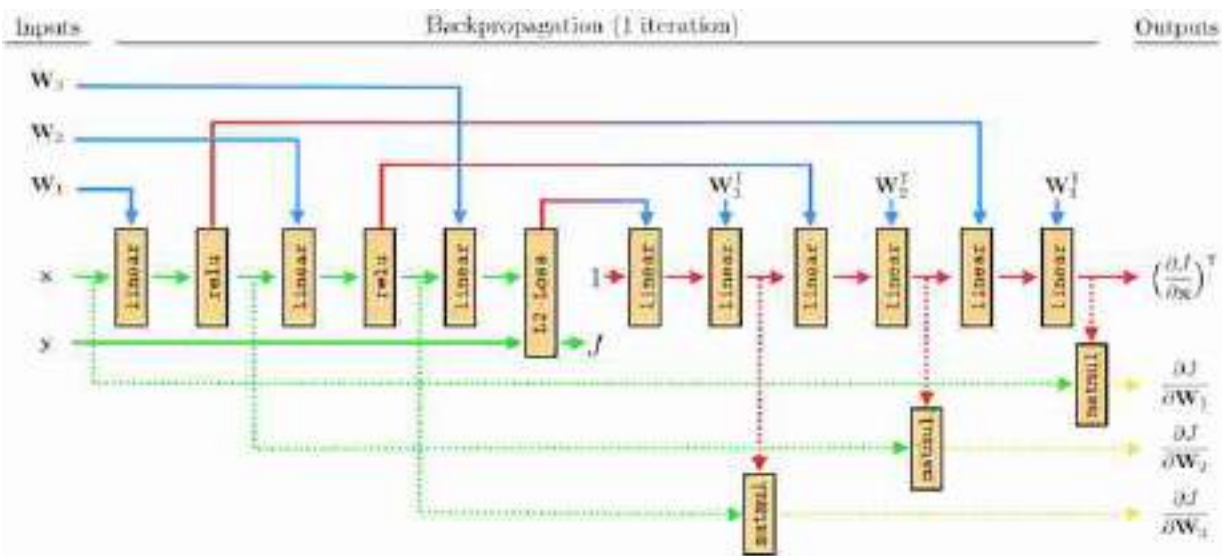


Figure 14.17: The computation graph for backpropagation through a three-layer MLP. It's just another neural net! Solid lines are involved in computing data/activation gradients and dotted lines are involved in computing parameter gradients.

—: params forward
 —: params backward
 —: data forward
 —: data backward

There are a few interesting things about this forward-backward network. One is that *activations* from the relu layers get transformed to become *parameters* of a linear layer of the backward network (see equation (14.33)). There is a general term for this setup, where one neural net outputs values that parameterize another neural net; this is called a **hypernetwork** [179]. The forward network is a hypernetwork that parameterizes the backward network.

Another interesting property, which we already pointed out previously, is that the backward network only consists of linear layers. This is true no matter what the forward network consists of (even if it is not a conventional neural network but some arbitrary computation graph). The reason why this happens is because backprop implements the chain rule, and the chain rule

is always a product of Jacobian *matrices*. Since a Jacobian is a matrix, clearly it is a linear function. But more intuitively, you can think of each Jacobian as being a locally linear approximation to the loss surface; hence each can be represented with a linear layer.

14.7 Backpropagation through DAGs: Branch and Merge

So far we have only seen chain-like graphs, $[-][-][-] \rightarrow$. Can backprop handle other graphs? It turns out the answer is *yes*. Presently we will consider **directed acyclic graphs (DAGs)**. In chapter 25, we will see that neural nets can also include cycles and still be trained with variants of backprop (e.g., backprop through time).

In a DAG, nodes can have multiple inputs and multiple outputs. In fact, we have already seen several examples of such nodes in the preceding sections. For example, a linear layer can be thought of as having two inputs, $\mathbf{x}_{\text{in}}$ and θ , and one output $\mathbf{x}_{\text{out}}$; or it can be thought of as having $N = |\mathbf{x}_{\text{in}}| + |\theta|$ inputs and $M = |\mathbf{x}_{\text{out}}|$ outputs, if we count up each dimension of the input and output vectors. So we have already seen DAG computation graphs.

However, to work with general DAGs, it helps to introduce two new special modules, which act to construct the topology of the graph. We will call these special operators *merge* and *branch* ([figure 14.18](#)).

We only consider binary branching and merging here, but branching and merging N ways can be done analogously or by repeating these operators.



Figure 14.18: The merge and branch layers.

We define them mathematically as variable concatenation and copying, respectively:

$$\text{merge}(\mathbf{x}_{\text{in}}^u, \mathbf{x}_{\text{in}}^b) \triangleq [\mathbf{x}_{\text{in}}^u, \mathbf{x}_{\text{in}}^b] \triangleq \mathbf{x}_{\text{out}} \quad (14.37)$$

$$\text{branch}(\mathbf{x}_{\text{in}}) \triangleq [\mathbf{x}_{\text{in}}, \mathbf{x}_{\text{in}}] \triangleq [\mathbf{x}_{\text{out}}^u, \mathbf{x}_{\text{out}}^b] \quad (14.38)$$

What if $\mathbf{x}_{\text{in}}^u$ and $\mathbf{x}_{\text{in}}^b$ are tensors, or other objects, with different shapes? Can we still concatenate them? The answer is yes. The shape of the data tensor has no impact on the math. We pick the shape just as a notational convenience; for example, it's natural to think about images as two-dimensional arrays.

Here, `merge` takes two inputs and concatenates them. This results in a new multidimensional variable. The backward pass equation is trivial. To compute the gradient of the cost with respect to $\mathbf{x}_{\text{in}}^u$, that is, $\mathbf{g}_{\text{in}}^u$, we have

$$\mathbf{g}_{\text{in}}^u = \mathbf{g}_{\text{out}} \mathbf{L}^{\mathbf{x}^u} = \frac{\partial J}{\partial \mathbf{x}_{\text{out}}} \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{x}_{\text{in}}^u} \quad (14.39)$$

$$= \mathbf{g}_{\text{out}} \left[\frac{\partial \mathbf{x}_{\text{in}}^u}{\partial \mathbf{x}_{\text{in}}^u}, \frac{\partial \mathbf{x}_{\text{in}}^b}{\partial \mathbf{x}_{\text{in}}^u} \right]^T \quad (14.40)$$

$$= \mathbf{g}_{\text{out}} [1, 0]^T \quad (14.41)$$

and likewise for $\mathbf{g}_{\text{in}}^b$. That is, we just pick out the first half of the $\mathbf{g}_{\text{out}}$ gradient vector for $\mathbf{g}_{\text{in}}^u$ and the second half for $\mathbf{g}_{\text{in}}^b$. There is really nothing new here. We already defined backpropagation for multidimensional variables above, and `merge` is just an explicit way of constructing multidimensional variables.

The `branch` operator is only slightly more complicated. In branching, we send *copies* of the same output to multiple downstream nodes. Therefore, we have multiple gradients coming back to the `branch` module, each from different downstream paths. So the inputs to this module on the backward pass are $\frac{\partial J}{\partial \mathbf{x}_{\text{in}}^u}, \frac{\partial J}{\partial \mathbf{x}_{\text{in}}^b}$, which we can write as the gradient vector $\mathbf{g}_{\text{out}} = \frac{\partial J}{\partial [\mathbf{x}_{\text{in}}^u, \mathbf{x}_{\text{in}}^b]} = \left[\frac{\partial J}{\partial \mathbf{x}_{\text{in}}^u}, \frac{\partial J}{\partial \mathbf{x}_{\text{in}}^b} \right] = [\mathbf{g}_{\text{out}}^u, \mathbf{g}_{\text{out}}^b]$. Let's compute the backward pass output:

$$\mathbf{g}_{in} = \mathbf{g}_{out} \mathbf{L}^x \quad (14.42)$$

$$= [\mathbf{g}_{out}^a, \mathbf{g}_{out}^b] \frac{\partial [\mathbf{x}_{out}^a, \mathbf{x}_{out}^b]}{\partial \mathbf{x}_{in}} \quad (14.43)$$

$$= [\mathbf{g}_{out}^a, \mathbf{g}_{out}^b] \frac{\partial [\mathbf{x}_{in}, \mathbf{x}_{in}]}{\partial \mathbf{x}_{in}} \quad (14.44)$$

$$= [\mathbf{g}_{out}^a, \mathbf{g}_{out}^b] [1, 1]^T \quad (14.45)$$

$$= \mathbf{g}_{out}^a + \mathbf{g}_{out}^b \quad (14.46)$$

So, branching just sums both the gradients passed backward to it.

Both merge and branch have no parameters, so there is no parameter gradient to define. Thus, we have fully specified the forward and backward behavior of these layers. The next diagrams summarize the behavior ([figure 14.19](#)).

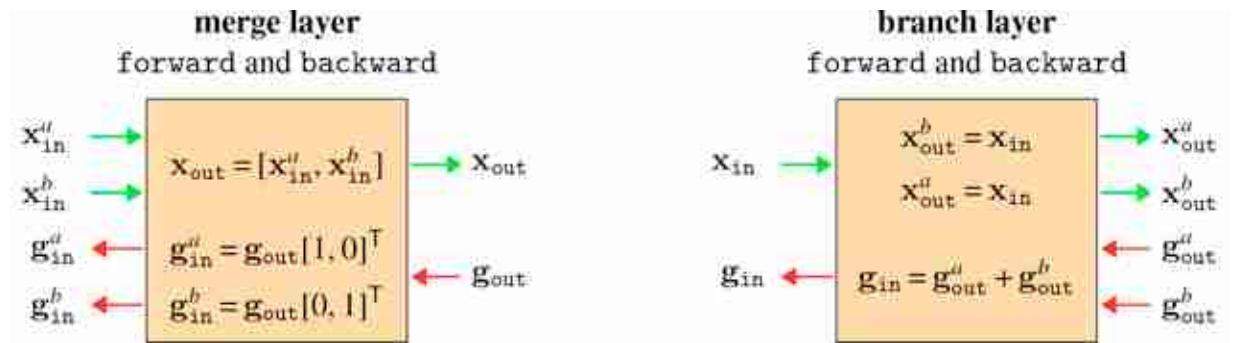


Figure 14.19: Merge and branch layers forward and backward.

With merge and branch, we can construct any DAG computation graph by simply inserting these layers wherever we want a layer to have multiple inputs or multiple outputs. An example is given in [figure 14.20](#).

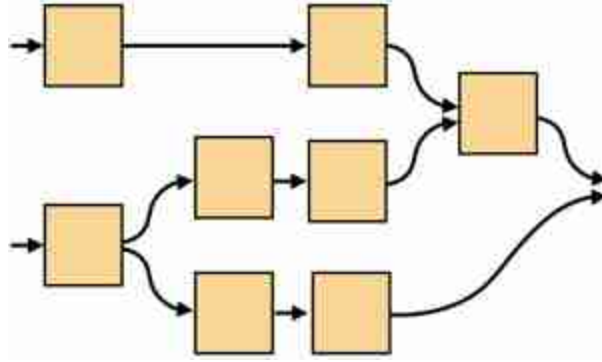


Figure 14.20: An example of a DAG computation graph that we can construct, and do backpropagation through, with the tools defined previously.

14.8 Parameter Sharing

Parameter sharing consists of a single parameter being sent as input to multiple different layers. We can consider this as a branching operation, as shown in [figure 14.21](#).

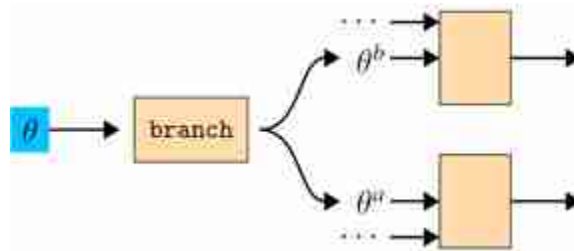


Figure 14.21: Parameter sharing is equivalent to branching a parameter in the computation graph.

Then, from the previous section, it is clear that gradients summate for shared parameters. Let $\{\theta^i\}_{i=1}^N$ be a set of variables that are all copies of one free parameter θ . Then,

$$\frac{\partial J}{\partial \theta} = \sum_i \frac{\partial J}{\partial \theta^i} \quad (14.47)$$

Neural net layers that have their parameters shared in this way are sometimes said to use **tied weights**.

14.9 Backpropagation to the Data

Backpropagation does not distinguish between parameters and data — it treats both as generic inputs to parameterless modules. Therefore, we can use backprop to optimize data inputs to the graph just like we can use backprop to optimize parameter inputs to the graph.

To see this, it helps to think about just the inputs and outputs to the full computation graph. In the forward direction, the inputs are the data and parameter settings and the output is the loss. In the backward direction, the input is the number 1 and the outputs are the gradients of the loss with respect to the data and parameters. The full computation graph, for a learning problem using neural net $F = f_L \circ \dots \circ f_1$ and loss function L , is $L(F(\mathbf{x}_0), \mathbf{y}, \theta) \triangleq J(\mathbf{x}_0, \mathbf{y}, \theta)$. This function can itself be thought of as a single computation block, with inputs and outputs as specified previously ([figure 14.22](#)).

In Pytorch you can only set *input* variables as optimization targets — these are called the *leaves* of the computation graph since, on the backward pass, they have no children. All the other variables are completely determined by the values of the input variables — they are not *free* variables.



Figure 14.22: Full forward and backward passes for a learning problem $\min J(\mathbf{x}_0, \mathbf{y}, \theta)$, collapsed into a single computation block.

From this diagram, it should be clear that data inputs and parameter inputs play symmetric roles. Just as we can optimize parameters to minimize the loss, by descending the parameter gradient given by backward, we can also optimize input data to minimize the loss by descending the data gradient.

This can be useful for lots of different applications. One example is visualizing the input image that most activates a given neuron we are probing in a neural net. To do this, we define J to be the *negative* of the value of the neuron we are probing², that is, $J(\mathbf{x}_0, \theta) = -x_l[i]$ if we are interested in the i -th neuron on layer l (notice $\mathbf{y}$ is not used for this problem). We show an example of this in [figure 14.23](#) below, where we used backprop to find the input image that most activates a node in the computation graph that scores whether or not an image is “a photo of a cat.” Do you see cat-like stuff in the optimized image? What does this tell you about how the network is working?



Figure 14.23: Visualizing the optimal image of a cat according to a particular neural net. The net we used is called Contrastive Language-Image Pre-Training (CLIP) [\[394\]](#) and here we found the image that maximizes a node in CLIP's computation graph that measures how much the image matches the text “a photo of a cat.” In chapter 51 we will cover exactly how the CLIP model works in more detail.

Visualizations like this are a useful way to figure out what visual features a given neuron is sensitive to. Researchers often combine this visualization method with a natural image prior in order to find an image that not only strongly activates the neuron in question but also looks like a natural photograph (e.g., [\[358\]](#)).

14.10 Concluding Remarks

Backprop is often presented as a method just for training neural networks, but it is actually a much more general tool than that. Backprop is an efficient way to find partial derivatives in computation graphs. It is general to a large family of computation graphs and can be used not just for learning parameters but also for optimizing data.

[2.](#) It is negative so that minimizing the loss maximizes the activation.

IV

FOUNDATIONS OF IMAGE PROCESSING

Many of the techniques in computer vision are rooted on signal processing. The goal of these collection of chapters is to introduce the most important signal processing concepts and tools that any computer vision researcher should have on its toolbox. We will present those concepts from the perspective of images and having in mind that the goal of computer vision is to build representations that are useful for downstream tasks. We do not assume any prior knowledge about signal processing.

- **Chapter 15** introduces signal processing, linear image filtering, and convolutions.
- **Chapter 16** describes the Fourier transform and some applications.

Notation

- Images: We will use ℓ symbol to denote images (i.e., *light*), but also arbitrary signals that correspond to physical quantities (such as sounds).
- Discrete signals: We will index specific values using $\ell[n]$ or $\ell[n, m]$ for 1D and 2D discrete signals/images, where n and m are discrete spatial indices. We will use bold fonts when signals are treated as vectors: $\boldsymbol{\ell}$. Vectors will be column vectors, and their transpose is $\boldsymbol{\ell}^T$.
- Continuous signals: We will use parenthesis for continuous signals, $\ell(t)$, where t is a continuous variable.
- Functions: We will mostly use f to denote functions. We will write inputs and outputs as ℓ_{in} and ℓ_{out} , analogously to the notation used in the neural network chapters. Resulting in the notation $\ell_{\text{out}} = f(\ell_{\text{in}})$.
- Convolution kernels: $h[n]$ or $h[n, m]$.

- Discrete Fourier transforms: We will use capital letters: $\mathcal{L}[u]$, where u is a discrete frequency index of the discrete Fourier transform of $\ell[n]$.
- Convolution operator, $\circ$, and cross-correlation operator, $\star$, as in $\ell_1[n]$ $\circ \ell_2[n]$.

15 Linear Image Filtering

15.1 Introduction

We want to build a vision system that operates in the real world. One such system is the human visual system. Although much remains to be understood about how our visual system processes images, we have a fairly good idea of what happens at the initial stages of visual processing, and it will turn out to be similar to some of the filtering we discuss in this chapter. While we're inspired by the biology, here we describe some mathematically simple processing that will help us to parse an image into useful tokens, or **low-level features** that will be useful later to construct visual interpretations.

We'd like for our processing to enhance image structures of use for subsequent interpretation, and to remove variability within the image that makes more difficult comparisons with previously learned visual signals. Let's proceed by invoking the simplest mathematical processing we can think of: a linear filter. Linear filters as computing structures for vision have received a lot of attention because of their surprising success in modeling some aspect of the processing carried out by early visual areas such as the retina, the lateral geniculate nucleus (LGN) and primary visual cortex (V1) [227]. In this chapter we will see how far it takes us toward these goals.

For a deeper understanding there are many books [364] devoted to **signal processing**, providing essential tools for any computer vision scientist. In this chapter we will present signal processing tools from a computer vision and image processing perspective.

15.2 Signals and Images

A **signal** is a measurement of some physical quantity (light, sound, height, temperature, etc.) as a function of another independent quantity (time, space, wavelength, etc.). A **system** is a process/function that transforms a signal into another.

15.2.1 Continuous and Discrete Signals

Although most of the signals that exist in nature are infinite **continuous signals**, when we introduce them into a computer they are sampled and transformed into a finite sequence of numbers, also called a **discrete signal**.

Sampling is the process of transforming a continuous signal into a discrete one and will be studied in detail in chapter 20.

If we consider the light that reaches one camera photo sensor, we could write it as a one dimensional function of time, $\ell(t)$, where $\ell(t)$ denotes the incident light at time t , and t is a continuous variable that can take on any real value. The signal $\ell(t)$ is then **sampled** in time (as is done in a video) by the camera where the values are only captured at discrete times (e.g., 30 times per second). In that case, the signal ℓ will be defined only on discrete time instants and we will write the sequence of measured values as: $\ell[n]$, where n can only take on integer values. The relationship between the discrete and the continuous signals is given by the sampling equation:

$$\ell[n] = \ell(n \Delta T) \quad (15.1)$$

where ΔT is the **sampling period**. For instance, $\Delta T = 1/30$ s in the case of sampling the signal 30 times per second. As is common in signal processing, we will use parenthesis to indicate continuous variables and brackets to denote discrete variables. For instance, if we sample the signal shown in [figure 15.1\(a\)](#) once every second ($\Delta T = 1$ s) we get the discrete signal shown in [figure 15.1\(b\)](#).

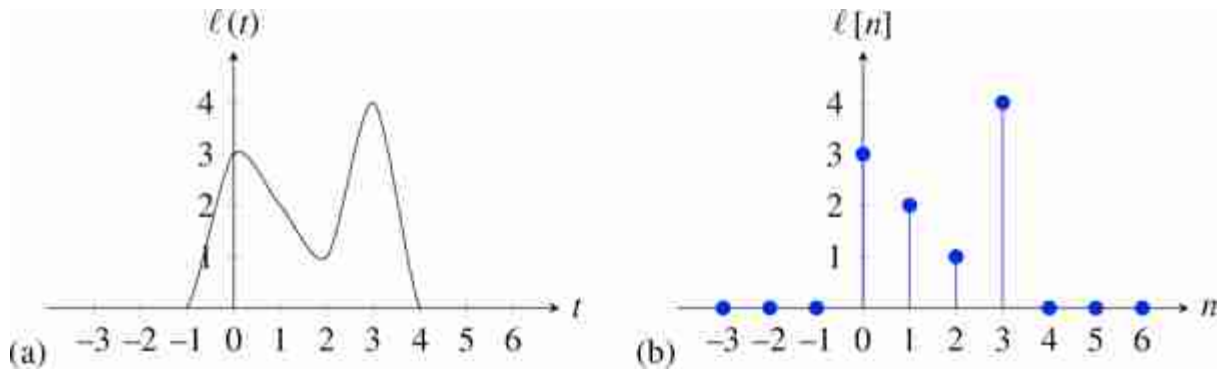


Figure 15.1: (a) A continuous signal, and (b) a discrete signal obtained by sampling the continuous signal at the times $t = n$.

The signal in [figure 15.1\(b\)](#) is a function that takes on the values $\ell[0] = 3$, $\ell[1] = 2$, $\ell[2] = 1$ and $\ell[3] = 4$ and all other values are zero, $\ell[n] = 0$. In most of the book we will work with discrete signals. In many cases it will be convenient to write discrete signals as vectors. Using vector notation we will write the previous signal, in the interval $n \in [0, 6]$, as a column vector $\boldsymbol{\ell} = [3, 2, 1, 4, 0, 0, 0]^T$, where T denotes transpose.

15.2.2 Images

An image is a two dimensional array of values: $\boldsymbol{\ell} \in \mathbb{R}^{M \times N}$, where N is the image width and M is the image height. A grayscale image is then just an array of numbers such as the following (in this example, intensity values are scaled between 0 and 256):

$$\boldsymbol{\ell} = \begin{bmatrix} 160 & 175 & 171 & 168 & 168 & 172 & 164 & 158 & 167 & 173 & 167 & 163 & 162 & 164 & 160 & 159 & 163 & 162 \\ 149 & 164 & 172 & 175 & 178 & 179 & 176 & 118 & 97 & 168 & 175 & 171 & 169 & 175 & 176 & 177 & 165 & 152 \\ 161 & 166 & 182 & 171 & 170 & 177 & 175 & 116 & 109 & 169 & 177 & 173 & 168 & 175 & 175 & 159 & 153 & 123 \\ 171 & 174 & 177 & 175 & 167 & 161 & 157 & 138 & 103 & 112 & 157 & 164 & 159 & 160 & 165 & 169 & 148 & 144 \\ 163 & 163 & 162 & 165 & 167 & 164 & 178 & 167 & 77 & 55 & 134 & 170 & 167 & 162 & 164 & 175 & 168 & 160 \\ 173 & 164 & 158 & 165 & 180 & 180 & 150 & 89 & 61 & 34 & 137 & 186 & 186 & 182 & 175 & 165 & 160 & 164 \\ 152 & 155 & 146 & 147 & 169 & 180 & 163 & 51 & 24 & 32 & 119 & 163 & 175 & 182 & 181 & 162 & 148 & 153 \\ 134 & 135 & 147 & 149 & 150 & 147 & 148 & 62 & 36 & 46 & 114 & 157 & 163 & 167 & 169 & 163 & 146 & 147 \\ 135 & 132 & 131 & 125 & 115 & 129 & 132 & 74 & 54 & 41 & 104 & 156 & 152 & 156 & 164 & 156 & 141 & 144 \\ 151 & 155 & 151 & 145 & 144 & 149 & 143 & 71 & 31 & 29 & 129 & 164 & 157 & 155 & 159 & 158 & 156 & 148 \\ 172 & 174 & 178 & 177 & 177 & 181 & 174 & 54 & 21 & 29 & 136 & 190 & 180 & 179 & 176 & 184 & 187 & 182 \\ 177 & 178 & 176 & 173 & 174 & 180 & 150 & 27 & 101 & 94 & 74 & 189 & 188 & 186 & 183 & 186 & 188 & 187 \\ 160 & 160 & 163 & 163 & 161 & 167 & 100 & 45 & 169 & 166 & 59 & 136 & 184 & 176 & 175 & 177 & 185 & 186 \\ 147 & 150 & 153 & 155 & 160 & 155 & 56 & 111 & 182 & 180 & 104 & 84 & 168 & 172 & 171 & 164 & 168 & 167 \\ 184 & 182 & 178 & 175 & 179 & 133 & 86 & 191 & 201 & 204 & 191 & 79 & 172 & 220 & 217 & 205 & 209 & 200 \\ 184 & 187 & 192 & 182 & 124 & 32 & 109 & 168 & 171 & 167 & 163 & 51 & 105 & 203 & 209 & 203 & 210 & 205 \\ 191 & 198 & 203 & 197 & 175 & 149 & 169 & 189 & 190 & 173 & 160 & 145 & 156 & 202 & 199 & 201 & 205 & 202 \\ 153 & 149 & 153 & 155 & 173 & 182 & 179 & 177 & 182 & 177 & 182 & 185 & 179 & 177 & 167 & 176 & 182 & 180 \end{bmatrix}$$

This matrix represents an image of size 18×18 pixels as encoded in a computer. Can you guess what object appears in the array? The goal of a computer vision system is to **reorganize** this array of numbers into a

meaningful representation of the scene depicted in the image. The next image shows the same array of values but visualized as gray levels.

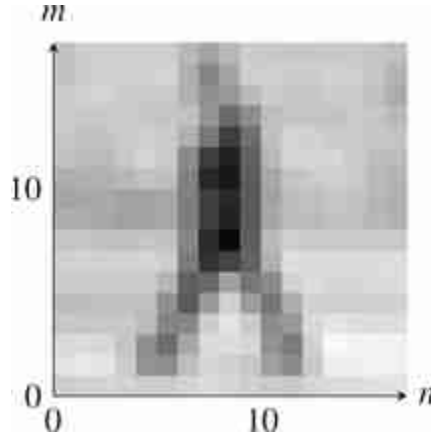


Figure 15.2: Grayscale image showing a person walking in the street. This tiny image has only 18×18 pixels.

When we want to make explicit the location of a pixel we will write $\ell[n, m]$, where $n \in [0, N - 1]$ and $m \in [0, M - 1]$ are the indices for the horizontal and vertical dimensions, respectively. Each value in the array indicates the intensity of the image in that location. For color images we will have three channels, one for each color.

Although images are discrete signals, in some cases it is useful to work on the continuous domain as it simplifies the derivation of analytical solutions. This was the case in the previous chapter where we used the image gradient in the continuous domain and then approximated it in the discrete space domain. In those cases we will write images as $\ell(x, y)$ and video sequences as $\ell(x, y, t)$.

15.2.3 Properties of a Signal

Here are some important quantities that are useful to characterize signals. As most signals represent physical quantities, a lot of the terminology used to describe them is borrowed from physics.

Both continuous and discrete signals can be classified according to its extend. **Infinite length** signals are signals that extend over the entire support $\ell[n]$ for $n \in (-\infty, \infty)$. **Finite length** signals have non-zero values on a compact time interval and they are zero outside, i.e. $\ell[n] = 0$ for $n \notin S$

where S is a finite length interval. A signal $\ell[n]$ is **periodic** if there is a value N such that $\ell[n] = \ell[n + kN]$ for all n and k . A periodic signal is an infinite length signal.

An important quantity is the mean value of a signal often called the **DC value**.

DC means **direct current** and it comes from electrical engineering. Although most signals have nothing to do with currents, the term DC is still commonly used.

In the case of an image, the DC component is the average intensity of the image. For a finite signal of length N (or a periodic signal), the **DC value** is

$$\mu = \frac{1}{N} \sum_{n=0}^{N-1} \ell[n] \quad (15.2)$$

If the signal is infinite, the average value is

$$\mu = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=-N}^N \ell[n] \quad (15.3)$$

Another important quantity to characterize a signal is its energy. Following the same analogy with physics, the **energy of a signal** is defined as the sum of squared magnitude values:

$$E = \sum_{n=-\infty}^{\infty} |\ell[n]|^2 \quad (15.4)$$

Signal energy and energy of a physical quantity are equivalent up to a normalization constant. For instance, the energy dissipated by a resistance R with voltage $v(t)$ is

$$E = \frac{1}{R} \int_{-\infty}^{\infty} v(t)^2 dt$$

Signal are further classified as **finite energy** and **infinite energy** signals. Finite length signals are finite energy and periodic signals are infinite energy signals when measuring the energy in the whole time axis.

If we want to compare two signals, we can use the squared Euclidean distance (squared L2 norm) between them:

$$D^2 = \frac{1}{N} \sum_{n=0}^{N-1} |\ell_1[n] - \ell_2[n]|^2 \quad (15.5)$$

However, the euclidean distance (L2) is a poor metric when we are interested in comparing the content of the two images and the building better metrics is an important area of research. Sometimes, the metric is L2 but in a different representation space than pixel values. We will talk about this more later in the book.

The same definitions apply also to continuous signals, where all the equations are analogous by replacing the sums with integrals. For instance, in the case of the energy of a continuous signal, we can write $E = \int_{-\infty}^{\infty} |\ell(t)|^2$, which assumes that the integral is finite. Most natural signals will have infinite energy.

15.3 Systems

The kind of systems that we will study here take a signal, ℓ_{in} , as input, perform some computation, f , and output another signal, $\ell_{\text{out}} = f(\ell_{\text{in}})$.



Figure 15.3: System processing one signal.

This transformation is very general and it can do all sorts of complex things: it could detect edges in images, recognize objects, detect motion in sequences, or apply aesthetic transformations to a picture. The function f could be specified as a mathematical operation or as an algorithm. As a consequence, it is very difficult to find a simple way of characterizing what the function f does. In the next section we will study a simple class of functions f that allow for a simple analytical characterization: linear systems.

15.3.1 Linear Systems

Linear systems represent a very small portion of all the possible systems one could implement, but we will see that they are capable of creating very interesting image transformations. A function f is linear if it satisfies the following two properties:

$$\begin{aligned} f(\ell_1 + \ell_2) &= f(\ell_1) + f(\ell_2) \\ f(a\ell) &= af(\ell) \quad \text{for any scalar } a \end{aligned} \quad (15.6)$$

Although the previous properties might seem simple, it is easy to get confused on when an operation can be or not represented by a linear function. Let's first check your intuition about what operations are linear and which ones are not. Which ones of the image transformations shown in [figure 15.4](#) can be written as linear systems?³

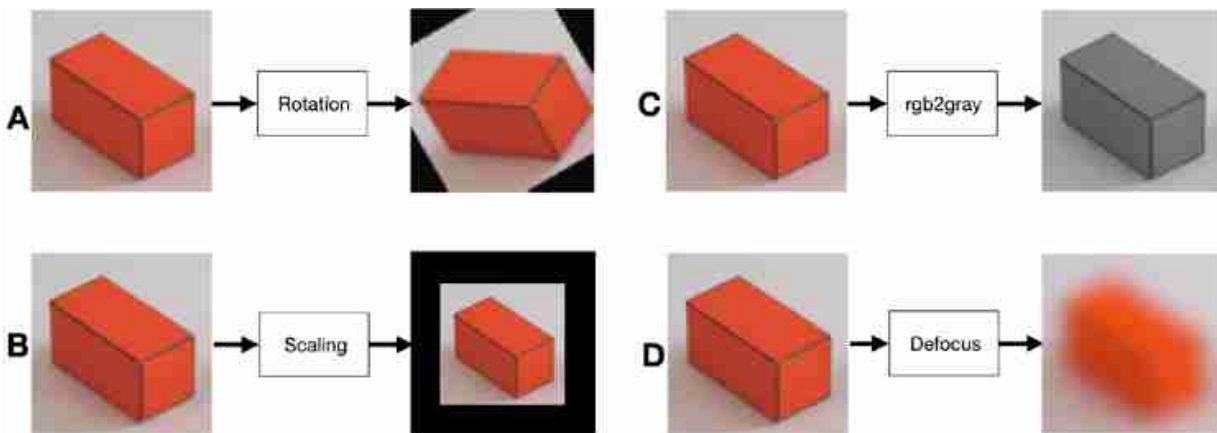


Figure 15.4: Which one of these four image transformations (rotation by 30 degrees, scaling by 1/2, color to grayscale, and defocusing) can be written as linear functions?

To make things more concrete, let's assume the input is a 1D signal with length N that we will write as $\ell_{\text{in}}[n]$, and the output is another 1D signal with length M that we will write as $\ell_{\text{out}}[n]$. Most of the times we will work with input and output pairs with the same length $M = N$. A linear system, in its most general form, can be written as follows:

$$\ell_{\text{out}}[n] = \sum_{k=0}^{N-1} h[n, k] \ell_{\text{in}}[k] \quad \text{for } n \in [0, M-1] \quad (15.7)$$

where each output value $\ell_{\text{out}}[n]$ is a linear combination of values of the input signal $\ell_{\text{in}}[n]$ with weights $h[n, k]$. The value $h[n, k]$ is the weight applied to the input sample $\ell_{\text{in}}[k]$ to compute the output sample $\ell_{\text{out}}[n]$. To help visualizing the operation performed by the linear filter it is useful to write it in matrix form:

$$\begin{bmatrix} \ell_{\text{out}}[0] \\ \ell_{\text{out}}[1] \\ \vdots \\ \ell_{\text{out}}[n] \\ \vdots \\ \ell_{\text{out}}[M-1] \end{bmatrix} = \begin{bmatrix} h[0,0] & h[0,1] & \dots & h[0,N-1] \\ h[1,0] & h[1,1] & \dots & h[1,N-1] \\ \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & h[n,k] & \vdots \\ \vdots & \vdots & \vdots & \vdots \\ h[M-1,0] & h[M-1,1] & \dots & h[M-1,N-1] \end{bmatrix} \begin{bmatrix} \ell_{\text{in}}[0] \\ \ell_{\text{in}}[1] \\ \vdots \\ \ell_{\text{in}}[k] \\ \vdots \\ \ell_{\text{in}}[N-1] \end{bmatrix}$$

which we will write in short form as $\boldsymbol{\ell}_{\text{out}} = \mathbf{H}\boldsymbol{\ell}_{\text{in}}$. The matrix $\mathbf{H}$ has size $M \times N$. We will use the matrix formulation many times in this book.

A linear function can also be drawn as a **fully connected layer** in a neural network, with weights $\mathbf{W} = \mathbf{H}$, where the output unit i , $\ell_{\text{out}}[i]$, is a linear combination of the input signal $\boldsymbol{\ell}_{\text{in}}$ with weights given by the row vectors of $\mathbf{H}$. Graphically it looks like this:

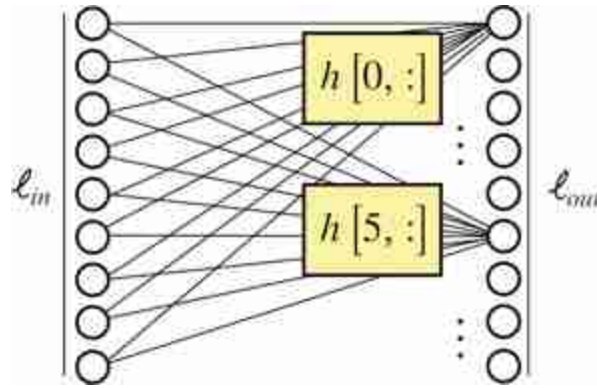


Figure 15.5: A linear function drawn as a **fully connected layer** in a neural network.

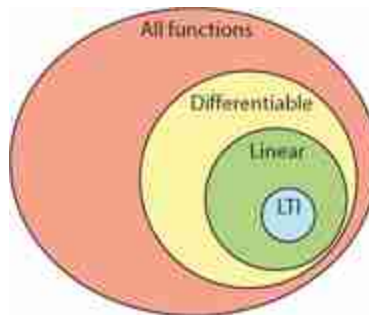
In two dimensions the equations are analogous. Each pixel of the output image, $\ell_{\text{out}}[n, m]$, is computed as a linear combination of pixels of the input image, $\ell_{\text{in}}[n, m]$:

$$\ell_{\text{out}}[n, m] = \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} h[n, m, k, l] \ell_{\text{in}}[k, l] \quad (15.8)$$

By writing the images as column vectors, concatenating all the image columns into a long vector, we can also write the previous equation using matrices and vectors: $\ell_{\text{out}} = \mathbf{H}\ell_{\text{in}}$.

15.3.2 Linear Translation Invariant Systems

Linear systems are still too general for us. So let's consider an even smaller family of systems: **linear translation invariant (LTI) systems**.



Translation invariant systems are motivated by the following observation: typically, we don't know where within the image we expect to find any given item ([figure 15.6](#)), so we often want to process the image in a spatially invariant manner, the same processing algorithm at every pixel.



Figure 15.6: A fundamental property of images is translation invariance—the same image may appear at arbitrary spatial positions within the image. *Source:* Fredo Durand.

An example of a translation invariant system is a function that takes as input an image and computes at each location a local average value of the pixels around in a window of 5×5 pixels:

$$\ell_{\text{out}}[n, m] = \frac{1}{25} \sum_{k=-2}^2 \sum_{l=-2}^2 \ell_{\text{in}}[n+k, m+l] \quad (15.9)$$

A system is an LTI system if it is linear and when we translate the input signal by n_0, m_0 , then output is also translated by n_0, m_0 :

$$\ell_{\text{out}}[n-n_0, m-m_0] = f(\ell_{\text{in}}[n-n_0, m-m_0]) \quad (15.10)$$

for any n_0, m_0 . This property is called **equivariant** with respect to translation. Translation invariance and linearity impose a strong constraint on the form of equation (15.7).

In that case, the linear system becomes a **convolution** of the image data with some **convolution kernel**. This operation is very important and we devote the rest of this chapter to study its properties.

15.4 Convolution

The convolution, denoted $\circ$, between a signal $\ell_{\text{in}}[n]$ and the **convolutional kernel** $h[n]$ is defined as follows:

$$\ell_{\text{out}}[n] = h[n] \circ \ell_{\text{in}}[n] = \sum_{k=-\infty}^{\infty} h[n-k] \ell_{\text{in}}[k] \quad (15.11)$$

The convolution computes the output values as a linear weighted sum. The weight between the input sample $\ell_{\text{in}}[k]$ and the output sample $\ell_{\text{out}}[n]$ is a function $h[n, k]$ that depends only on their relative position $n - k$. Therefore, $h[n, k] = h[n - k]$. If the signal $\ell_{\text{in}}[n]$ has a finite length, N , then the sum is only done in the interval $k \in [0, N - 1]$.

The convolution is related to the **correlation** operator. In the correlation, the weights are defined as $h[n, k] = h[k - n]$. The convolution and the correlation use the same kernel but mirrored around the origin. The correlation is also translation invariant as we will discuss at the end of the chapter.

Most of the literature in neural networks calls convolution to the correlation operator. As the kernels are learned, the distinction is not important most of the time.

One important property of the convolution is that it is commutative: $h[n] \circ \ell[n] = \ell[n] \circ h[n]$. This property is easy to prove by changing the variables, $k = n - k'$, so that:

$$h[n] \circ \ell_{\text{in}}[n] = \sum_{k=-\infty}^{\infty} h[n-k] \ell_{\text{in}}[k] = \sum_{k'=-\infty}^{\infty} h[k'] \ell_{\text{in}}[n-k'] = \ell_{\text{in}}[n] \circ h[n] \quad (15.12)$$

The convolution operation can be described in words as: first take the kernel $h[k]$ and mirror it $h[-k]$, then shift the mirrored kernel so that the origin is at location n , then multiply the input values around location n by the mirrored kernel and sum the result. Store the result in the output vector $\ell_{\text{out}}[n]$. For instance, if the convolutional kernel has three non-zero values: $h[-1] = 1$, $h[0] = 2$ and $h[1] = 3$, then the output value at location n is $\ell_{\text{out}}[n] = 3\ell_{\text{in}}[n-1] + 2\ell_{\text{in}}[n] + 1\ell_{\text{in}}[n+1]$.

For finite length signals, it helps to make explicit the structure of the convolution writing it in matrix form. For instance, if the convolutional

kernel has only three non-zero values, then:

$$\begin{bmatrix} \ell_{\text{out}}[0] \\ \ell_{\text{out}}[1] \\ \ell_{\text{out}}[2] \\ \vdots \\ \ell_{\text{out}}[N-1] \end{bmatrix} = \begin{bmatrix} h[0] & h[-1] & 0 & \dots & 0 \\ h[1] & h[0] & h[-1] & \dots & 0 \\ 0 & h[1] & h[0] & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & h[0] \end{bmatrix} \begin{bmatrix} \ell_{\text{in}}[0] \\ \ell_{\text{in}}[1] \\ \ell_{\text{in}}[2] \\ \vdots \\ \ell_{\text{in}}[N-1] \end{bmatrix} \quad (15.13)$$

This operation is much easier to understand if we draw it as a **convolutional layer** of a neural network as shown in [figure 15.7](#).

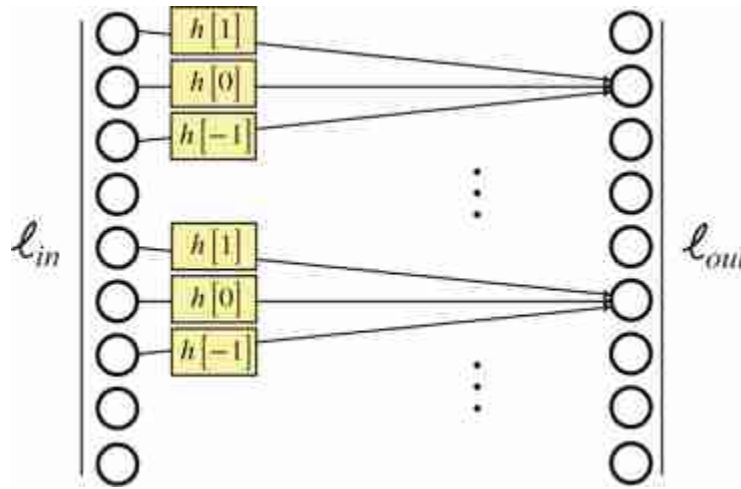


Figure 15.7: A convolution represented as a layer in a neural network. Each output unit i , $\ell_{\text{out}}[i]$, is a linear combination of the input signal values around location i always using the same set of weights given by h .

In two dimensions, the processing is analogous: the input filter is flipped vertically and horizontally, then slid over the image to record the inner product with the image everywhere. Mathematically, this is:

$$\ell_{\text{out}}[n, m] = h[n, m] \circ \ell_{\text{in}}[n, m] = \sum_{k, l} h[n-k, m-l] \ell_{\text{in}}[k, l] \quad (15.14)$$

The next figure shows the result of a 2D convolution between a 3×3 kernel with a 9×9 image. The particular kernel used in the figure averages in the vertical direction and takes differences horizontally. The output image reflects that processing, with horizontal differences accentuated and vertical changes diminished. To compute one of the pixels in the output image, we start with an input image patch of 3×3 pixels, we flip the convolutional

kernel upside-down and left-right, and then we multiply all the pixels in the image patch with the corresponding flipped kernel values and we sum the results.

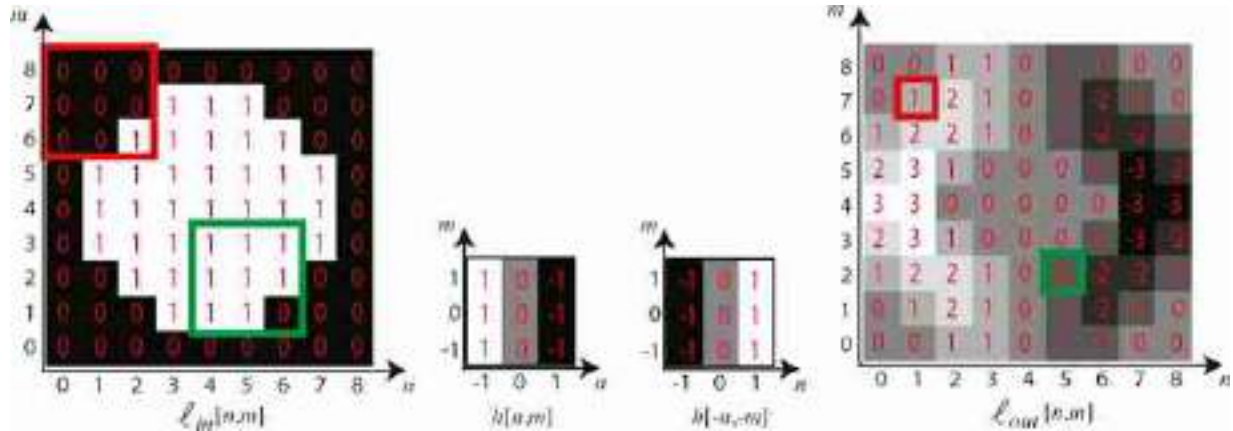


Figure 15.8: Illustration of a 2D convolution of an input image with a kernel of size 3×3 . For the pixels in the boundary we assumed that input image has zero values outside its boundary. The red and green boxes show the input pixels used to compute the corresponding output pixels.

Let's revisit the four image transformation examples from [figure 15.4](#) and ask a new question. Which ones of the image transformations shown in [figure 15.4](#) can be written as a convolution?⁴

As shown in [figure 15.9\(a\)](#), in the defocusing transformation each output pixel at location n, m is the local average of the input pixels in a local neighborhood around location n, m , and this operation is the same regardless of what pixel output we are computing. Therefore, it can be written as a convolution. The rotation transformation (for a fix rotation) is a linear operation but it is not translation invariant. As illustrated in [figure 15.9\(b\)](#), different output pixels require looking at input pixels in a way that is location specific. At the top left corner, one wants to grab a pixel from, say, five pixels up and to the right, and from the bottom right one needs to grab the pixel from about five pixels down and to the left. So this rotation operation can't be written as a spatially invariant operation.

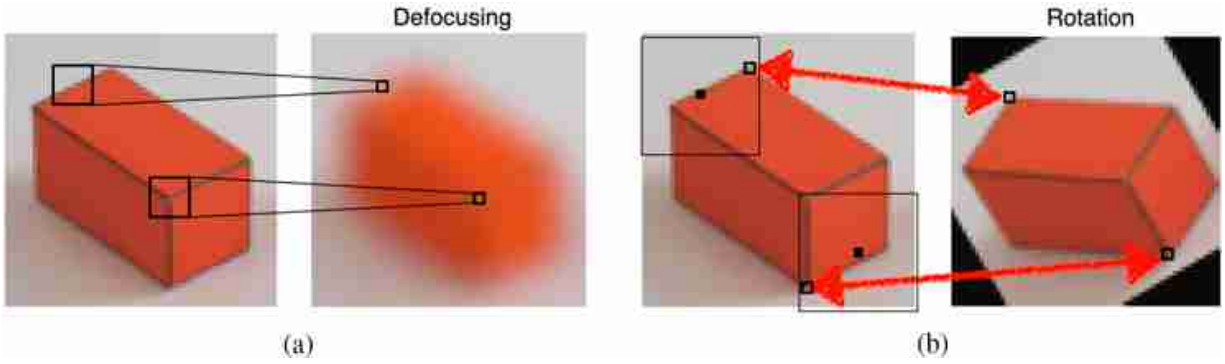


Figure 15.9: (a) Defocusing an image can be written as a convolution. (b) Rotation can't be written as a convolution.

15.4.1 Properties of the Convolution

The convolution is a linear operation that will be extensively used thorough the book and it is important to be familiar with some of its properties.

- **Commutative.** As we have already discussed before the convolution is commutative,

$$h[n] \circ \ell[n] = \ell[n] \circ h[n] \quad (15.15)$$

which means that the order of convolutions is irrelevant. This is not true for the correlation.

- **Associative.** Convolutions can be applied in any order:

$$\ell_1[n] \circ \ell_2[n] \circ \ell_3[n] = \ell_1[n] \circ (\ell_2[n] \circ \ell_3[n]) = (\ell_1[n] \circ \ell_2[n]) \circ \ell_3[n] \quad (15.16)$$

In practice, for finite length signals, the associative property might be affected by how boundary conditions are implemented.

- **Distributive** with respect to the sum:

$$\ell_1[n] \circ (\ell_2[n] + \ell_3[n]) = \ell_1[n] \circ \ell_2[n] + \ell_1[n] \circ \ell_3[n] \quad (15.17)$$

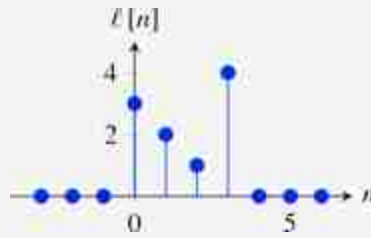
- **Shift.** Another interesting property involves shifting the two convolved functions. Let's consider the convolution $\ell_{\text{out}}[n] = h[n] \circ \ell_{\text{in}}[n]$. If the input signal, $\ell_{\text{in}}[n]$, is translated by a constant n_0 , i.e. $\ell_{\text{in}}[n - n_0]$, the result of the convolution with the same kernel, $h[n]$, is the same output as before but translated by the same constant n_0 :

$$\ell_{\text{out}}[n - n_0] = h[n] \circ \ell_{\text{in}}[n - n_0] \quad (15.18)$$

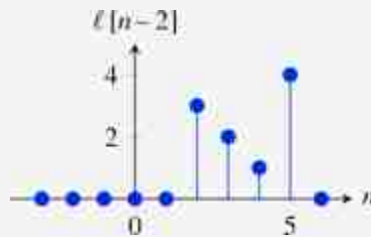
Translating the kernel is equivalent:

$$\ell_{\text{out}}[n - n_0] = h[n] \circ \ell_{\text{in}}[n - n_0] = h[n - n_0] \circ \ell_{\text{in}}[n] \quad (15.19)$$

Given the signal:



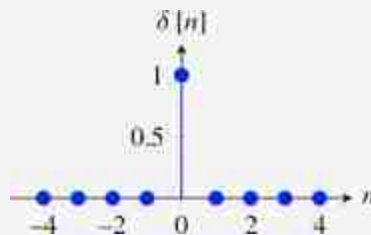
The signal $\ell[n - n_0]$ is a shifted version of $\ell[n]$. With $n_0 = 2$ this is



- **Support.** The convolution of a discrete signal of length N with another discrete signal of length M results in a discrete signal with length $L \leq M + N - 1$.
- **Identity.** The convolution also has an identity function, that is the **impulse**, $\delta[n]$, which takes the value 1 for $n = 0$ and it is zero everywhere else:

$$\delta[n] = \begin{cases} 1 & \text{if } n=0 \\ 0 & \text{if } n \neq 0 \end{cases} \quad (15.20)$$

The impulse function is:



The convolution of the impulse with any other signal, $\ell[n]$, returns the same signal:

$$\delta[n] \circ \ell[n] = \ell[n] \quad (15.21)$$

15.4.2 Some Examples

[Figure 15.10](#) shows several simple convolution examples (each color

channel is convolved with the same kernel). [Figure 15.10\(a\)](#) shows a kernel with a single central non-zero element, $\delta [n, m]$, convolved with any image, gives back that same image. [Figure 15.10\(b\)](#) shows a kernel, $\delta [n, m - 2]$, that produces a two pixel shift toward the right of the input image. The black line that appears on the left is due to the zero-boundary conditions (to be discussed in the next section) and it has a width of two pixels. [Figure 15.10\(c\)](#) shows the result of convolving the image with a kernel $0.5\delta [n - 2, m - 2] + 0.5\delta [n + 2, m + 2]$. This results in two superimposed copies of the same image shifted diagonally in opposite directions. And finally, [figure 15.10\(d\)](#) shows the result of convolving the image with a uniform kernel of all $1/25$.

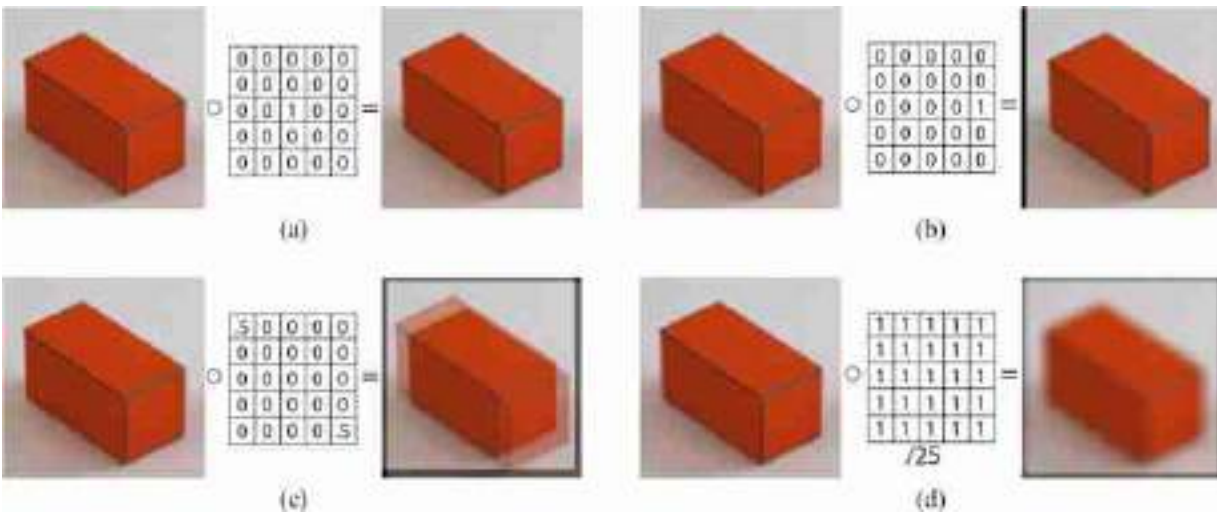
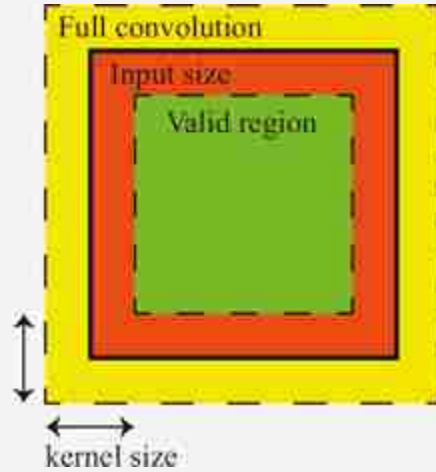


Figure 15.10: (a) An impulse convolved with the input image gives no change. (b) A shifted impulse shifts the image. (c) Sum of two shifted copies of the image. (d) Image defocusing by computing a local average over windows of 5×5 pixels. All the examples use zero padding for handling boundary conditions.

15.4.3 Handling Boundaries

When implementing a convolution, one is confronted with the question of what to do at the image boundaries. There's really no satisfactory answer for how to handle the boundaries that works well for all applications. One solution consists in omitting from the output any pixels that are affected by the input boundary. The issue with this is that the output will have a different size than the input and, for large convolutional kernels, there might be a large portion of the output image missing.

Only the green region contains values that can be computed, the rest will be affected by how we decide to handle the boundaries:



The most general approach consists in extending the input image by adding additional pixels so that the output can have the same size as the input. So, for a kernel with support $[-N, N] \times [-M, M]$, one has to add $N/2$ additional pixels left and right of the image and $M/2$ pixels at the top and bottom. Then, the output will have the same size as the original input image.

Some typical choices for how to pad the input image are (see [figure 15.11](#)):

- Zero padding: Set the pixels outside the boundary to zero (or to some other constant such as the mean image value). This is the default option in most neural networks.
- Circular padding: Extend the image by replicating the pixels from the other side. If the image has size $P \times Q$, circular padding consists of the following:

$$\ell_{\text{in}}[n, m] = \ell_{\text{in}}[(n)_P, (m)_Q] \quad (15.22)$$

where $(n)_P$ denotes the modulo operation and $(n)_P$ is the remainder of n/P .

This padding transform the finite length signal into a periodic infinite length signal. Although this will introduce many artifacts, it is a convenient extension for analytical derivations.

- Mirror padding: Reflect the valid image pixels over the boundary of valid output pixels. This is the most common approach and the one that gives the best results.

- Repeat padding: Set the value to that of the nearest output image pixel with valid mask inputs.

[Figure 15.11](#) shows different examples of boundary extensions. Boundary extensions are different ways of approximating the ground truth image that exists beyond the image boundary. The top row shows how the image is extended beyond its boundary when using different types of boundary extensions (only the central part corresponds to the input). The middle row shows the output of the convolution (cropped to match the size of the input image). We can see that the zero-padding makes a darkening appear on the boundary of the image. This is not very desirable in general. The other padding methods seem to produce better visual results.

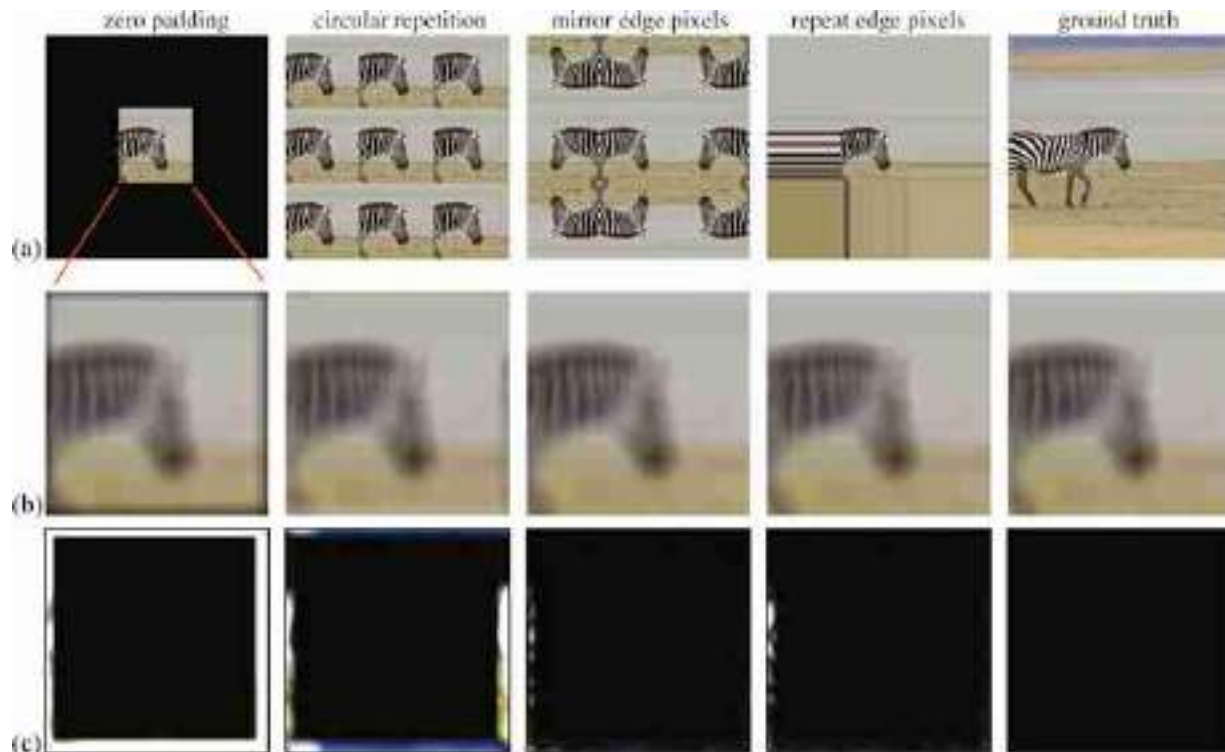


Figure 15.11: Each row shows: (a) Different types of boundary extension. The last image shows the ground truth. (b) The output of convolving the image with a uniform kernel of size 11×11 with all the values equal to $1/121$. The output only shows the central region that corresponds to the input image without boundary extension. (c) The difference between each output and the ground truth image; see last column of (b). Note that the ground truth will not be available in practice.

But which one is the best boundary extension? Ideally, we would to get a result that is as close as possible to the output we would get if we had access

to a larger image. In this case we know what the larger image looks like as shown in the last column of [figure 15.11](#). For each boundary extension method, the final row shows the absolute value of the difference between the convolution output and the result obtained with the ground truth.

For the example of [figure 15.11](#), the error seems to be smaller when extending the image using mirror padding. On the other extreme, zero padding seems to be the worst.

15.4.4 Circular Convolution

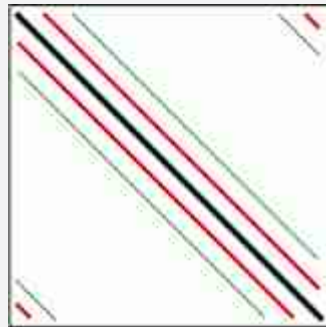
When convolving a finite length signal with a kernel, we have seen that it can be convenient to extend the signal beyond its boundary to reduce boundary artifacts. This trick is also very useful analytically and conduces to a special case of the convolution: the **circular convolution** uses circular padding. The circular convolution of two signals, h and ℓ_{in} , of length N is defined as follows:

$$\ell_{\text{out}}[n] = h[n] \circledast_N \ell_{\text{in}}[n] = \sum_{k=0}^{N-1} h[(n-k)_N] \ell_{\text{in}}[k] \quad (15.23)$$

In this formulation, the signal $h[(n)_N]$ is the infinite periodic extension, with period N , of the finite length signal $h[n]$. The output of the circular convolution is a periodic signal with period N , $\ell_{\text{out}}[n] = \ell_{\text{out}}[(n)_N]$.

The circular convolution has the same properties than the convolution (e.g., is commutative.)

Writing the circular convolution in matrix form will look like:



15.4.5 Convolution in the Continuous Domain

The convolution in the continuous domain is defined in the same way as with discrete signals but replacing the sums by integrals. Given two

continuous signals $h(t)$ and $\ell_{\text{in}}(t)$, the convolution operator is defined as follows:

$$\ell_{\text{out}}(t) = h(t) \circ \ell_{\text{in}}(t) = \int_{-\infty}^{\infty} h(t - \tau) \ell_{\text{in}}(\tau) d\tau \quad (15.24)$$

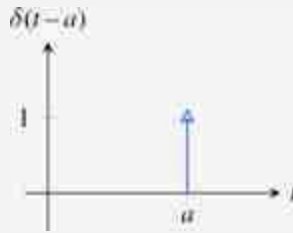
The properties of the continuous domain convolution are analogous to the properties of the discrete convolution, with the commutative, associative and distributive relationships still holding.

We can also define the impulse, $\delta(t)$, in the continuous domain such that $\ell(t) \circ \delta(t) = \ell(t)$. The impulse is defined as being zero everywhere but at the origin where it takes an infinitely high value so that its integral is equal to 1:

$$\int_{-\infty}^{\infty} \delta(t) dt = 1 \quad (15.25)$$

The impulse function is also called the impulse distribution (as it is not a function in the strict sense) or the Dirac delta function.

The delta distribution is usually represented with an arrow of height 1, indicating that it has an finite value at that point, and a finite area equal to 1:



The impulse has several important properties:

- Scaling property: $\delta(at) = \delta(t)/|a|$
- Symmetry: $\delta(-t) = \delta(t)$
- Sampling property: $\ell(t)\delta(t-a) = \ell(a)\delta(t-a)$ where a is a constant.

From this, we have the following:

$$\int_{-\infty}^{\infty} \ell(t) \delta(t-a) dt = \ell(a) \quad (15.26)$$

- As in the discrete case, the continuous impulse is also the identity for the convolution: $\ell(t) \circ \delta(t) = \ell(t)$.

The impulse will be useful in analytical derivations and we will see it again in chapter 20 when we discuss sampling.

15.5 Cross-Correlation Versus Convolution

Another form of writing a translation invariant filter is using the cross-correlation operator. The cross-correlation provides a simple technique to locate a template in an image. The cross-correlation and the convolution are closely related.

The **cross-correlation** operator (denoted by $\star$) between the image ℓ_{in} and the kernel h is written as follows:

$$\ell_{\text{out}}[n, m] = \ell_{\text{in}} \star h = \sum_{k, l=-N}^N \ell_{\text{in}}[n+k, m+l] h[k, l] \quad (15.27)$$

where the sum is done over the support of the filter h , which we assume is a square $(-N, N) \times (-N, N)$. In the convolution, the kernel is inverted left-right and up-down, while in the cross-correlation is not. Remember that the convolution between image ℓ_{in} and kernel h is written as follows:

$$\ell_{\text{out}}[n, m] = \ell_{\text{in}} \circ h = \sum_{k, l=-N}^N \ell_{\text{in}}[n-k, m-l] h[k, l] \quad (15.28)$$

[Figure 15.12](#) illustrates the difference between the correlation and the convolution. In these examples, the kernel $h[n, m]$ is a triangle pointing up (the center of the triangle is $n = 0$ and $m = 0$) as shown in [figure 15.12\(a\)](#). When the input image is black (all zeros) with only four bright pixels, [figure 15.12\(b\)](#), this is the convolution of the triangle with four impulses, which results in four triangles pointing up ([figure 15.12\(c\)](#)). However, the cross-correlation looks different; it still results in four copies of the triangles centered on each point, but the triangles are pointing downward ([figure 15.12\(d\)](#)).

For the input image containing triangles pointing up and down, [figure 15.12\(e\)](#), the convolution will have its maximum output on the triangles points down, [figure 15.12\(f\)](#), while the correlation will output the maximum values in the center of the triangles pointing up ([figure 15.12\(g\)](#)).

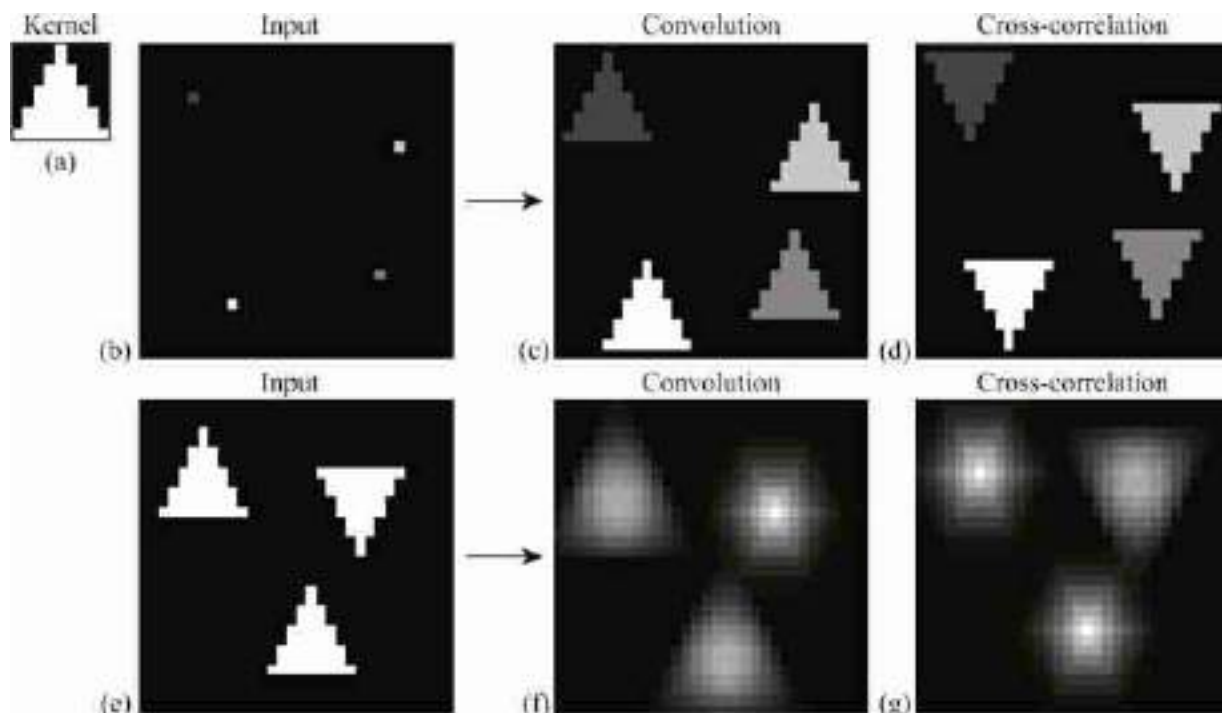


Figure 15.12: Cross-correlation versus convolution. (a) Kernel. (b) and (e) are two input images. (c) and (f) are the output convolution with the kernel (a). (d) and (g) are the cross-correlation output with the kernel (a).

Another important difference between the two operators is that the convolution is commutative and associative while the correlation is not. The correlation breaks the symmetry between the two functions h and ℓ_{in} . For instance, in the correlation, shifting h is not equivalent to shifting ℓ_{in} (the output will move in opposite directions). Note that the cross-correlation and convolution outputs are identical when the kernel h has central symmetry.

15.5.1 Template Matching and Normalized Correlation

The goal of template matching is to find a small image patch within a larger image. [Figure 15.13](#) shows how to use template matching to detect all the occurrences of the letter “a” with an image of text, shown in [figure 15.13\(b\)](#). [Figure 15.13\(a\)](#) shows the template of the letter “a”.

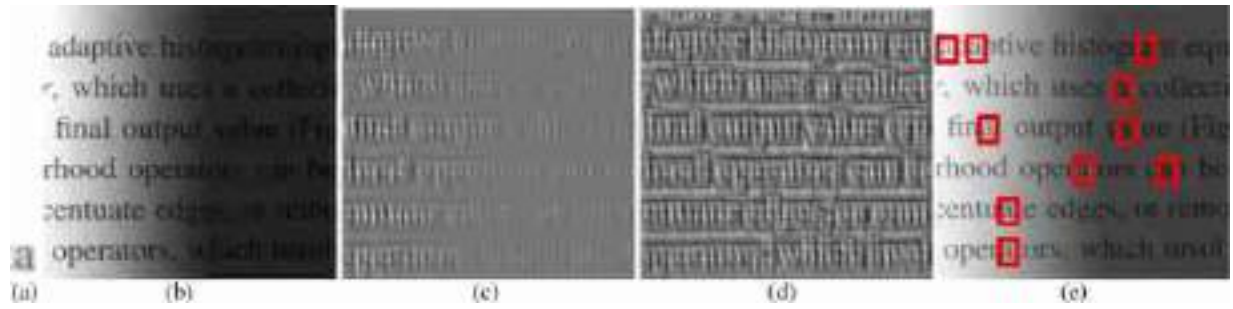


Figure 15.13: (a) Template. (b) Input image. (c) Correlation between input (b) and template (a). (d) Normalized correlation. (e) Locations with cross-correlation above 75 percent of its maximum value.

Cross-correlation is a potential method for template matching, but it has certain flaws. Consider matching the “a” template in [figure 15.13\(a\)](#) in a given input image. Just by increasing the brightness of the image in different parts we can increase the filter response since correlation is essentially a multiplication between the kernel h and any input image patch p (note that all the values are positive in p and h). In bright image regions we will have the maximum correlation values regardless of regions’ content as shown in [figure 15.13\(c\)](#).

The normalized cross-correlation, at location (n, m) , is the dot product between a template, h , (normalized to be zero mean and unit norm) and the local image patch, p , centered at that location and with the same size as the kernel, normalized to be unit norm. This can be implemented with the following steps: First, we normalize the kernel h . The normalized kernel $\hat{h}$ is the result of making the kernel h zero mean and unit norm. Then, the normalized cross-correlation is as follows ([figure 15.13\(d\)](#)):

$$\ell_{\text{out}}[n, m] = \frac{1}{\sigma[n, m]} \sum_{k, l=-N}^N \ell_{\text{in}}[n+k, m+l] \hat{h}[k, l] \quad (15.29)$$

Note that this equation is similar to the correlation function, but the output is scaled by the local standard deviation, $\sigma[n, m]$, of the image patch centered in (n, m) and with support $(-N, N) \times (-N, N)$, where $(-N, N) \times (-N, N)$ is the size of the kernel h . To compute the local standard deviation, we first compute the local mean, $\mu[n, m]$, as follows:

$$\mu[n, m] = \frac{1}{(2N+1)^2} \sum_{k, l=-N}^N \ell_{\text{in}}[n+k, m+l] \quad (15.30)$$

And then we compute:

$$\sigma^2[n, m] = \frac{1}{(2N+1)^2} \sum_{k,l=-N}^N (\ell_{\text{in}}[n+k, m+l] - \mu[n, m])^2 \quad (15.31)$$

[Figure 15.13](#)(e) shows the detected copies of the template [figure 15.13](#)(a) in the image [figure 15.13](#)(b). The red bounding boxes in [figure 15.13](#)(e) correspond to the location with local maximum values of the normalized cross-correlation ([figure 15.13](#)(d)) that are above a threshold chosen by hand.

In the example of [figure 15.13](#), all the instances of the letter ‘a’ have been correctly detected. But this method is very fragile when used to detect objects and is useful only under very limited conditions. Normalized correlation will detect the object independently of location, but it will not be robust to any other changes such as rotation, scaling, and changes in appearance. Just changing the font of the letters will certainly make the approach fail.

15.6 System Identification

One important task that we will study is that of explaining the behavior of complex systems such as neural networks. For an LTI filter with kernel $h[n]$, the output from an impulse is $y[n] = h[n]$. The output of a translated impulse $\delta[n - n_0]$ is $h[n - n_0]$. This is why the kernel that defines an LTI system, $h[n]$, is also called the impulse response of the system.

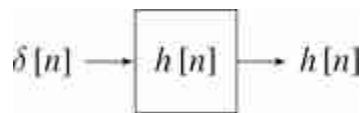


Figure 15.14: LTI system with impulse response $h[n]$.

For an unknown system, one way of finding the convolution kernel h consists in computing the output of the system when the input is an impulse.

Let's start with a beautiful example from acoustics [\[475\]](#). When you are in a room you often hear that sounds are distorted with echos ([figure 15.15](#)). The number and strength of the echos depend on the geometry of the room, whether it is full or empty and the materials in the walls and ceilings.

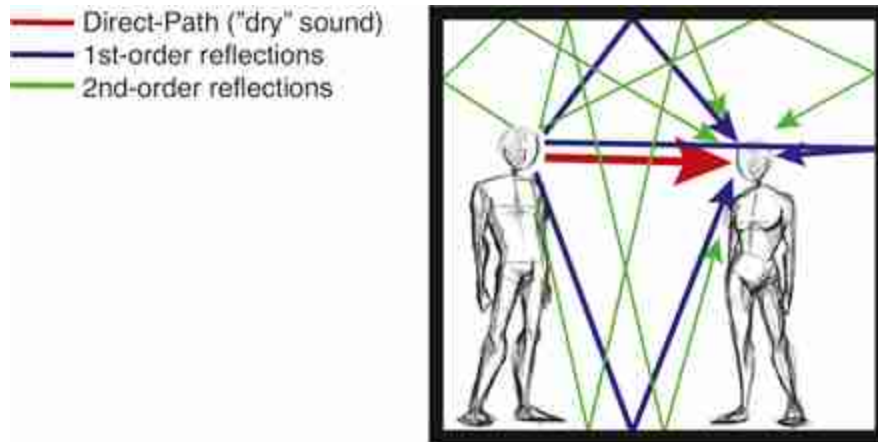


Figure 15.15: As shown in the sketch, sounds produced by one speaking person reach a listener via multiple paths. What the person hears is the superposition of all those signals. Figure modified from [475].

Interestingly the relationship between the sounds that are originated in one location and what a listener hears in another position are related by an LTI system. If you are in a room and you clap (which is a good approximation to an impulse of sound), the echoes that you hear are very close to the impulse response of the system formed by the acoustics of the room. Any sounds that originates at the location of your clap will produce a sound in the room that will be qualitatively similar to the convolution of the acoustic signal with the echoes that you hear before when you clapped. [Figure 15.16](#) shows a recording made in a restaurant in Cambridge, Massachusetts.

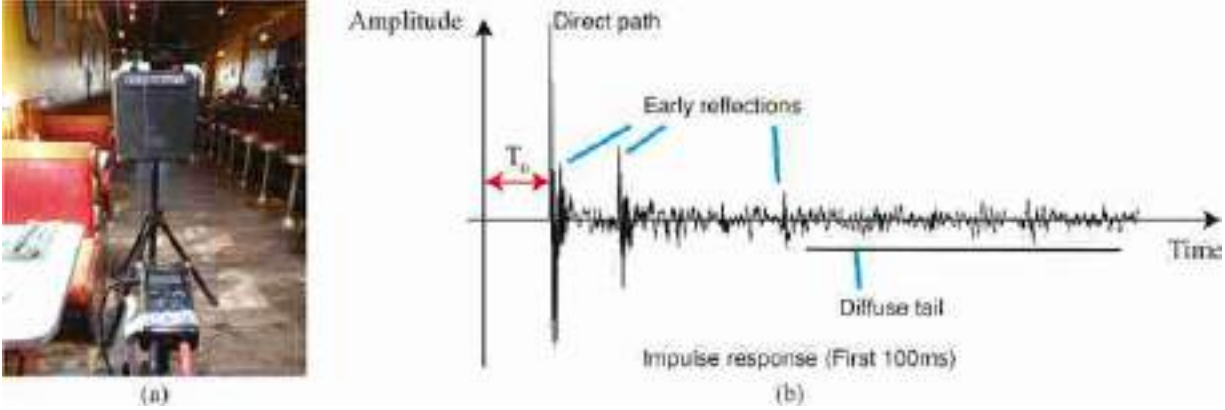


Figure 15.16: (a) Apparatus used to measure the impulse response generated by a battery-powered speaker and a portable digital recorder. (b) Recorded acoustic impulse response of a room. Figure modified from [475].

Figure 15.17, adapted from [475], shows a sketch of how a voice signal gets distorted by the impulse response of a room before it reaches the listener. The impulse response, $h(t)$, is modeled here as a sequence of four impulses, the first impulse corresponds to the direct path and the remaining three to the different paths inside the room, modeled with different delays (T_i) and amplitudes ($0 < a_i < 1$):

$$h(t) = a_0 \delta(t) + a_1 \delta(t - T_1) + a_2 \delta(t - T_2) + a_3 \delta(t - T_3) \quad (15.32)$$

The sound that reaches the listener is the superposition of four copies of the sound emitted by the speaker. This can be written as the convolution of the input sound, $\ell_{in}(t)$, with room impulse response, $h(t)$:

$$\ell_{out}(t) = \ell_{in}(t) \circ h(t) = a_0 \ell_{in}(t) + a_1 \ell_{in}(t - T_1) + a_2 \ell_{in}(t - T_2) + a_3 \ell_{in}(t - T_3) \quad (15.33)$$

which is shown graphically in figure 15.17.

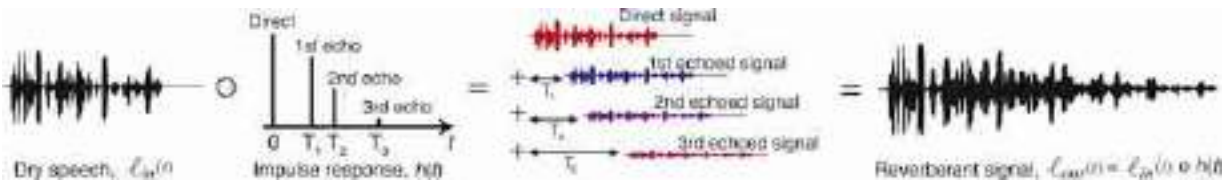


Figure 15.17: Modified from [475].

15.7 Concluding Remarks

A good foundation in signal processing is a valuable background for computer vision scientists and engineers. This short introduction should now allow you go deeper by consulting other specialized books.

But we are not done yet. In the next chapter we will study the Fourier transform, which will provide a useful characterization of signals and images.

[3.](#) Answer: All of them!

[4.](#) Only C and D from [figure 15.4](#) can be implemented by convolutions.

16 Fourier Analysis

16.1 Introduction

We need a more precise language to talk about the effect of linear filters, and the different image components, than to say “sharp” and “blurry” parts of the image. The Fourier transform provides that precision. By analogy with temporal frequencies, which describe how quickly signals vary over time, a **spatial frequency** describes how quickly a signal varies over space. The Fourier transform lets us describe a signal as a sum of complex exponentials, each of a different spatial frequency.

Fourier transforms are the basis of a number of computer vision approaches and are an important tool to understand images and how linear spatially invariant filters transform images. There are also important to understand modern representations such as positional encoding popularized by transformers [487].

16.2 Image Transforms

Sometimes it is useful to transform the image pixels into another representation that might reveal image properties that can be useful for solving vision tasks. Before we saw that linear filtering is a useful way of transforming an image. Linear image transforms with the form $\mathbf{x} = \mathbf{H}\ell_{\text{in}}$ can be thought of as a way of changing the initial pixels representation of ℓ_{in} into a different representation in $\mathbf{x}$.

We use $\mathbf{x}$ to denote an intermediate representation. The representation might not be an image.

This representation is specially interesting when it can be inverted so that the original pixels can be recovered as $\ell_{\text{in}} = \mathbf{H}^{-1}\mathbf{x}$. The Fourier transform is one of those representations. A useful representation $\mathbf{x}$ should have a

number of interesting properties not immediately available in the original pixels of ℓ_{in} .

16.3 Fourier Series

In 1822, French mathematician and engineer Joseph Fourier, as part of his work on the study on heat propagation, showed that any periodic signal could be written as an infinite sum of trigonometric functions (cosine and sine functions). His ideas were published in his famous book *Theorie de la Chaleur* [136], and the **Fourier series** has become one of the most important mathematical tools in science and engineering.

Fourier showed that any function, $\ell(t)$ defined in the interval $t \in (0, \pi)$, could be expressed as an infinite linear combination of harmonically related sinusoids,

$$\ell(t) = a_1 \sin(t) + a_2 \sin(2t) + a_3 \sin(3t) + \dots \quad (16.1)$$

and that the value of the coefficients a_n could be computed as the area of the curve $\ell(t) \sin(nt)$. Precisely,

$$a_n = \frac{2}{\pi} \int_0^\pi \ell(t) \sin(nt) dt \quad (16.2)$$

However, the sum is only guaranteed to converge to the function $\ell(t)$ within the interval $t \in (0, \pi)$. In fact, the resulting sum, for any values a_n , is a **periodic function** with period 2π and is anti-symmetric with respect to the origin, $t = 0$.

One of Fourier's original examples of **sine series** is the expansion of the ramp signal $\ell(t) = t/2$. This series was first introduced by Euler. Fourier showed that his theory explained why a ramp could be written as the following infinite sum:

$$\frac{1}{2}t = \sin(t) - \frac{1}{2} \sin(2t) + \frac{1}{3} \sin(3t) - \frac{1}{4} \sin(4t) + \dots \quad (16.3)$$

The result of this series approximates the ramp with increasing accuracy as we add more terms. The plots in [figure 16.1](#) show each of the weighted sine functions (on top) and the resulting partial sum (bottom graphs). We can see how adding **higher frequency** sines adds more details to the resulting function making it closer to the ramp. These plots show the signal in the

interval $t \in (-4\pi, 4\pi)$ to reveal the periodic nature of the result of the sine series.

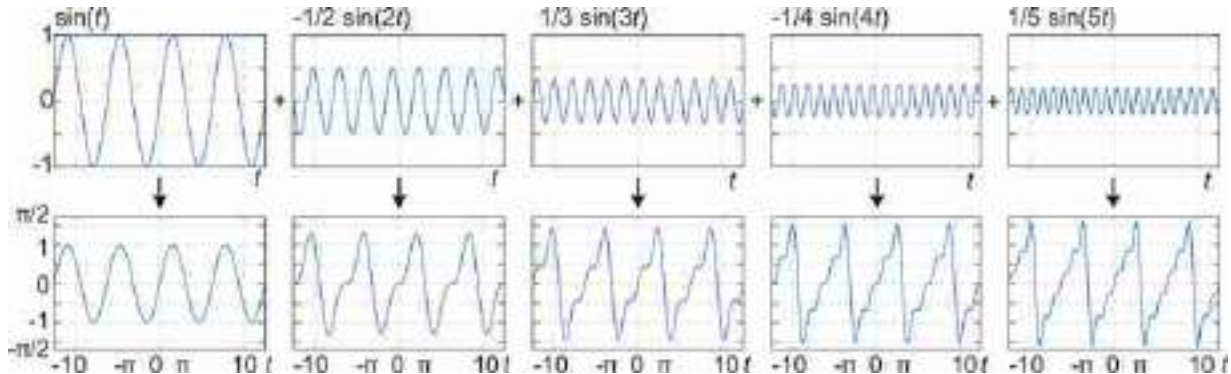


Figure 16.1: Reconstruction of a ramp with the first five sine functions.

It is useful to think of the **Fourier series** of a signal as a **change of representation** as shown in [figure 16.2](#). Instead of representing the signal by the sequence of values specified by the original function $\ell(t)$, the same function can be represented by the infinite sequence of coefficients a_n . The coefficients a_n give us an alternative description of the function $\ell(t)$ and the rest of this chapter will be devoted to understand what this **transformation** has to offer.

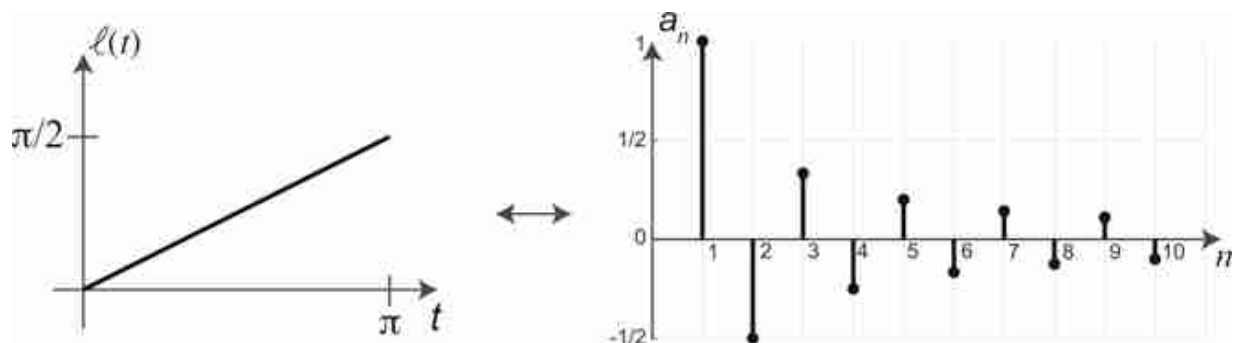


Figure 16.2: Two representations for the ramp function. (left) Time domain. (right) Fourier domain with coefficients of the sine series.

Fourier series can also be written as sums of different sets of harmonic functions. For instance, using cosine functions we can describe the ramp

function also as:

$$\frac{1}{2}t = \frac{\pi}{4} - \frac{2}{\pi} \cos(t) - \frac{2}{3^2\pi} \cos(3t) - \frac{2}{5^2\pi} \cos(5t) - \dots \quad (16.4)$$

The cosine and sine series of the same function are only equal in the interval $t \in (0, \pi)$, and result in different periodic extensions outside that interval.

The fields of signal and image processing have introduced different types of Fourier series. In the next sections we will study how these series are applied to describe discrete images using discrete sine and cosine functions and then we will focus on using complex exponentials. The representation using Fourier series is important for studying linear systems and convolutions.

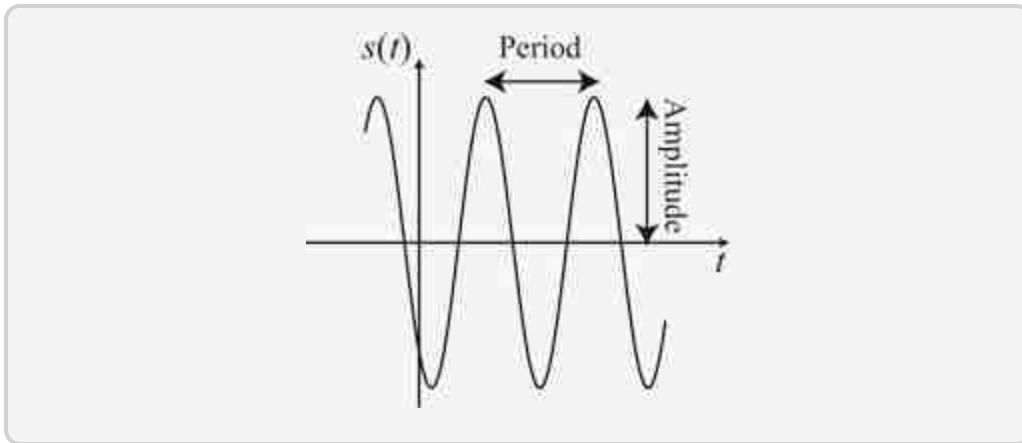
16.4 Continuous and Discrete Waves

Now that we have seen the importance of sine waves as a tool to represent signals, let's describe them in a bit more detail.

The **continuous time sine wave** is

$$s(t) = A \sin(wt - \theta) \quad (16.5)$$

where A is the **amplitude**, w is the **frequency**, and θ is the **phase**. The wave signal is periodic with a period $T = 2\pi/w$.



In discrete time, the **discrete time sine wave** is as follows

$$s[n] = A \sin(wn - \theta) \quad (16.6)$$

Note that the discrete sine wave will not be periodic for any arbitrary value of w . A discrete signal $\ell[n]$ is periodic, if there exists $T \in \mathbb{N}$ such that $\ell[n] = \ell[n + mT]$ for all $m \in \mathbb{Z}$. For the discrete sine (and cosine) wave to be

periodic the frequency has to be $w = 2\pi K/N$ for $K, N \in \mathbb{N}$. If K/N is an irreducible fraction, then the period of the wave will be $T = N$ samples. Although θ can have any value, here we will consider only the values $\theta = 0$ and $\theta = \pi/2$, which correspond to the sine and cosine waves, respectively.

In general, to make explicit the periodicity of the wave we will use the form:

$$s_k[n] = \sin\left(\frac{2\pi}{N} k n\right) \quad (16.7)$$

The same applies for the cosine:

$$c_k[n] = \cos\left(\frac{2\pi}{N} k n\right) \quad (16.8)$$

This particular notation makes sense when considering the set of periodic signals with period N , or the set of signals with finite support signals of length N with $n \in [0, N - 1]$. In such a case, $k \in [1, N/2]$ denotes the frequency (i.e., the number of wave cycles that will occur within the region of support). Note that if $k = 0$ then $s[n] = 0$ and $c[n] = 1$ for all n . One can also verify that $c_{N-k} = c_k$, and $s_{N-k} = -s_k$. Therefore, for frequencies $k > N/2$ we find the same set of waves as the ones in the interval $s \in [1, N/2]$. [Figure 16.3](#) shows some examples.

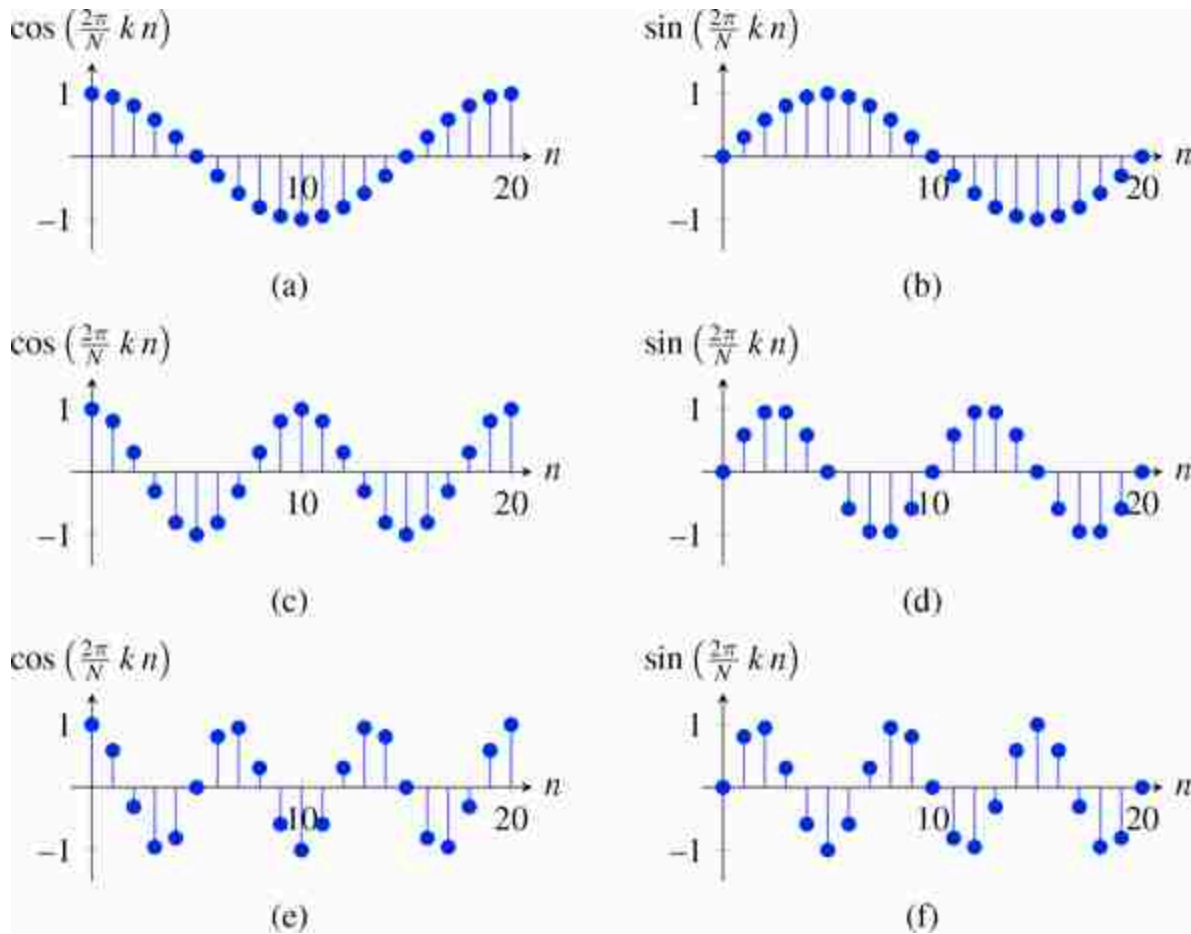


Figure 16.3: Sine and cosine waves with $A = 1$ and $N = 20$. Each row corresponds to $k = 1$, $k = 2$ and $k = 3$. Note that for $k = 3$ the waves oscillates three times in the interval $[0, N - 1]$, but the samples in each oscillation are not identical, and it is only truly periodic once every N samples. This is because $3/20$ is an irreducible fraction.

16.4.1 Sines and Cosines in 2D

The same analysis can be extended to two dimensions (2D). In 2D, the discrete sine and cosine waves are as follows:

$$s_{u,v}[n, m] = A \sin \left(2\pi \left(\frac{u n}{N} + \frac{v m}{M} \right) \right) \quad (16.9)$$

$$c_{u,v}[n, m] = A \cos \left(2\pi \left(\frac{u n}{N} + \frac{v m}{M} \right) \right) \quad (16.10)$$

where A is the amplitude and u and v are the two spatial frequencies and define how fast or slow the waves change along the spatial dimensions n and m . [Figure 16.4](#) shows some examples.

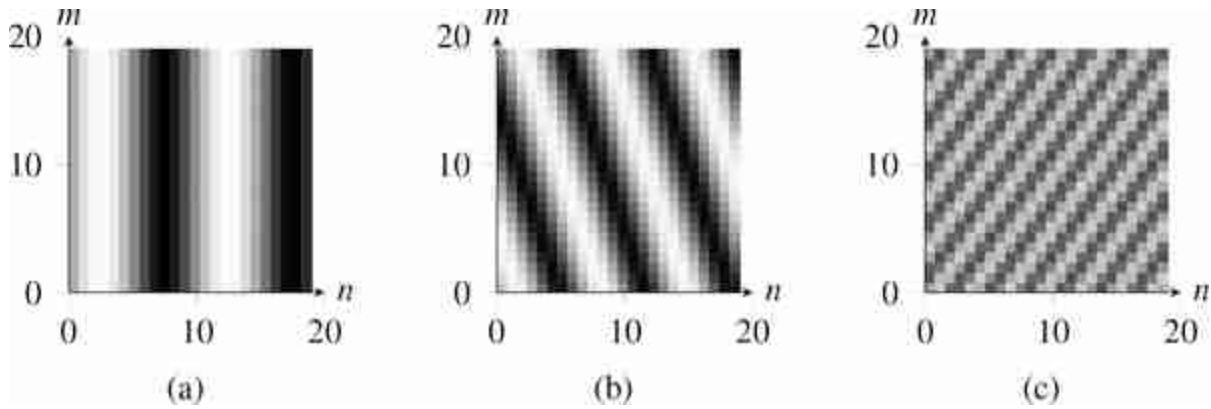


Figure 16.4: 2D sine waves with $N = M = 20$. The frequency values are (a) $u = 2$, $v = 0$; (b) $u = 3$, $v = 1$; (c) $u = 7$, $v = -5$.

In 2D, the sine and cosine waves can also be described using polar coordinates for encoding the frequency: radial frequency, $\sqrt{u^2 + v^2}$, and orientation, $\angle(u, v)$.

16.4.2 Complex Exponentials

Another important signal is the complex exponential wave:

$$e_u[n] = \exp\left(2\pi j \frac{un}{N}\right) \quad (16.11)$$

Complex exponentials are related to cosine and sine waves by Euler's formula:

$$\exp(ja) = \cos(a) + j \sin(a) \quad (16.12)$$

[Figure 16.5](#) shows the one dimensional (1D) discrete complex exponential function (for $v = 0$). As the values are complex, the plot shows in the x -axis the real component and in the y -axis the imaginary component. As n goes from 0 to $N - 1$ the function rotates along the complex circle of unit magnitude.

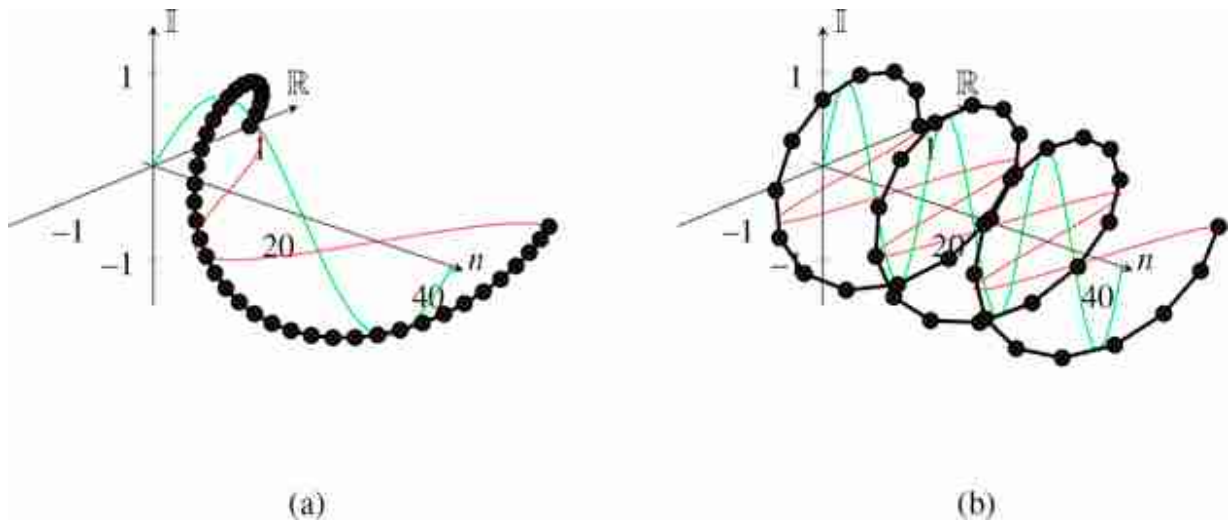


Figure 16.5: Complex exponential wave with (a) $N = 40$, $k = 1$, $A = 1$; and (b) $N = 40$, $k = 3$, $A = 1$. The red and green curves show the real and imaginary waves. The black line is the complex exponential. The dots correspond to the discrete samples.

In 2D, the complex exponential wave is

$$e_{u,v}[n, m] = \exp\left(2\pi j \left(\frac{u n}{N} + \frac{v m}{M}\right)\right) \quad (16.13)$$

where u and v are the two spatial frequencies. Note that complex exponentials in 2D are separable. This means they can be written as the product of two 1D signals:

$$e_{u,v}[n, m] = e_u[n] e_v[m] \quad (16.14)$$

A remarkable property is that the complex exponentials form an orthogonal basis for discrete signals and images of finite length. For images of size $N \times M$,

$$\langle e_{u,v}, e_{u',v'} \rangle = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} e_{u,v}[n, m] e_{u',v'}^*[n, m] = MN \delta[u - u'] \delta[v - v'] \quad (16.15)$$

Therefore, any finite length discrete image can be decomposed as a linear combination of complex exponentials.

16.5 The Discrete Fourier Transform

In this chapter we will focus on the discrete Fourier transform as it provides important tools to understand the behavior of signals and systems (e.g., sampling and convolutions). For a more detailed study of other transforms

and the foundations of Fourier series we refer the reader to other specialized books in signal and image processing .

16.5.1 Discrete Fourier Transform and Inverse Transform

The **Discrete Fourier Transform** (DFT) transforms an image $\ell [n, m]$, of finite size $N \times M$, into the complex image Fourier transform $\mathcal{L}[u, v]$ as:

$$\mathcal{L}[u, v] = \mathcal{F}\{\ell[n, m]\} = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \ell[n, m] \exp\left(-2\pi j \left(\frac{u n}{N} + \frac{v m}{M}\right)\right) \quad (16.16)$$

We will call $\mathcal{L}[u, v]$ the Fourier transform of $\ell[n, m]$. We will often represent the relationship between the signal as its transform as:

$$\ell[n, m] \xrightarrow{\mathcal{F}} \mathcal{L}[u, v] \quad (16.17)$$

By applying $\frac{1}{MN} \sum_{u=0}^{N-1} \sum_{v=0}^{M-1}$ to both sides of equation (16.16) and exploiting the orthogonality between distinct Fourier basis elements, we find the **inverse Fourier transform** relation

$$\ell[n, m] = \mathcal{F}^{-1}\{\mathcal{L}[u, v]\} = \frac{1}{NM} \sum_{u=0}^{N-1} \sum_{v=0}^{M-1} \mathcal{L}[u, v] \exp\left(+2\pi j \left(\frac{u n}{N} + \frac{v m}{M}\right)\right) \quad (16.18)$$

As we can see from the inverse transform equation, we rewrite the image, instead of as a sum of offset pixel values, as a sum of complex exponentials, each at a different frequency, called a spatial frequency for images because they describe how quickly things vary across space. From the inverse transform formula, we see that to construct an image from a Fourier transform, $\mathcal{L}[u, v]$, we just add in the corresponding amount of that particular complex exponential (conjugated).

As $\mathcal{L}[u, v]$ is obtained as a sum of complex exponential with a common period of N, M samples, the function $\mathcal{L}[u, v]$ is also periodic: $\mathcal{L}[u + aN, v + bM] = \mathcal{L}[u, v]$ for any $a, b \in \mathbb{Z}$. Also the result of the inverse DFT is a periodic image. Indeed you can verify from equation (16.18) that $\ell[n + aN, m + bM] = \ell[n, m]$ for any $a, b \in \mathbb{Z}$.

Using the fact that $e_{N-u, M-v} = e_{-u, -v}$, another equivalent way to write for the Fourier transform is to sum over the frequency interval $[-N/2, N/2]$ and $[-M/2, M/2]$. This is especially useful for the inverse that can be written as:

$$\ell[n, m] = \frac{1}{NM} \sum_{u=-N/2}^{N/2} \sum_{v=-M/2}^{M/2} \mathcal{L}[u, v] \exp\left(+2\pi j \left(\frac{u n}{N} + \frac{v m}{M}\right)\right) \quad (16.19)$$

This formulation allows us to arrange the coefficients in the complex plane so that the zero frequency, or DC, coefficient is at the center. Slow, large variations correspond to complex exponentials of frequencies near the origin. If the amplitudes of the complex conjugate exponentials are the same, then their sum will represent a cosine wave; if their amplitudes are opposite, it will be a sine wave. Frequencies further away from the origin represent faster variation with movement across space. The DFT and its inverse in 1D are defined in the same way as shown in [table 16.2](#).

One very important property is that the decomposition of a signal into a sum of complex exponentials is unique: there is a unique linear combination of the exponentials that will result in a given signal.

The DFT became very popular thanks to the **Fast Fourier Transform** (FFT) algorithm. The most common FFT algorithm is the Cooley–Tukey algorithm [\[82\]](#) that reduces the computation cost from $O(N^2)$ to $O(N \log N)$.

16.5.2 Matrix Form of the Fourier Transform

As the DFT is a linear transform we can also write the DFT in matrix form, with one basis per row. In 1D, the matrix for the DFT is as follows:

$$\mathbf{F} = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \exp(-2\pi j \frac{1}{N}) & \exp(-2\pi j \frac{2}{N}) & \dots & \exp(-2\pi j \frac{N-1}{N}) \\ 1 & \exp(-2\pi j \frac{2}{N}) & \exp(-2\pi j \frac{4}{N}) & \dots & \exp(-2\pi j \frac{2(N-1)}{N}) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \exp(-2\pi j \frac{(N-1)}{N}) & \exp(-2\pi j \frac{(N-1)2}{N}) & \dots & \exp(-2\pi j \frac{(N-1)(N-1)}{N}) \end{bmatrix} \quad (16.20)$$

Each entry in the matrix is $\mathbf{F}_{u,n} = \exp(-2\pi j \frac{u n}{N})$, with u indexing rows and n indexing columns. Note that $\mathbf{F}$ is a symmetric matrix. The inverse of the DFT is the complex conjugate: $\mathbf{F}^{-1} = \mathbf{F}^*$.

Working in 1D, as we did before, allows us to visualize the transformation matrix. [Figure 16.6](#) shows a color visualization of the complex-value matrix for the 1D DFT, which when used as a multiplicand yields the Fourier transform of 1D vectors. Many Fourier transform properties and symmetries can be observed from inspecting that matrix.

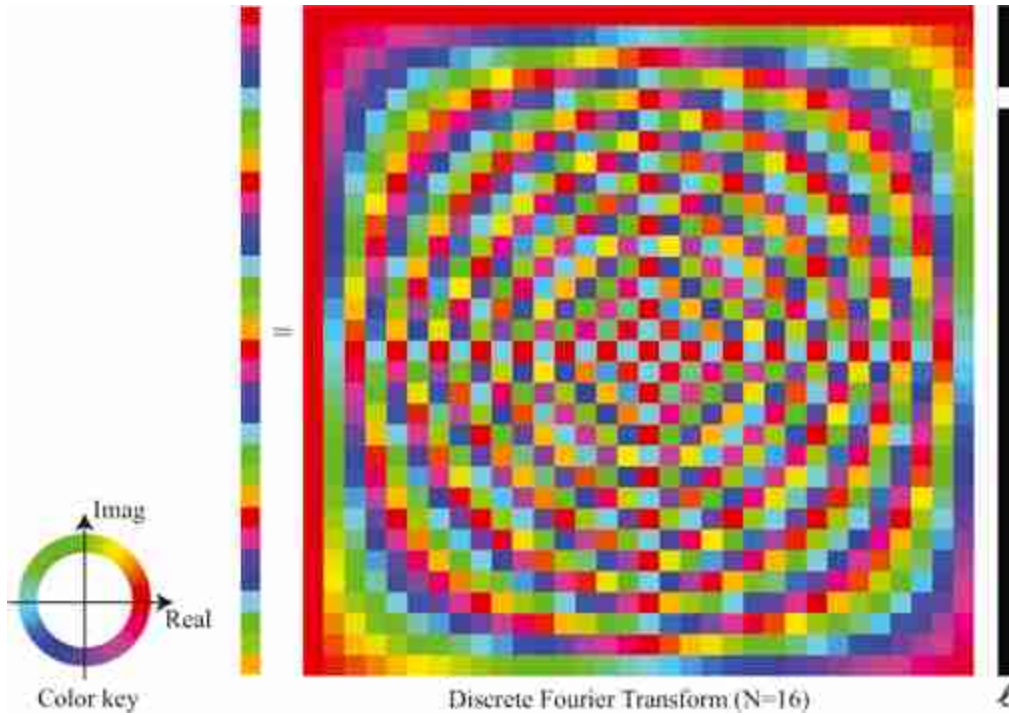


Figure 16.6: Visualization of the discrete Fourier transform as a matrix. The signal to be transformed forms the entries of the column vector at right. The complex values of the Fourier transform matrix are indicated by the color, with the key in the bottom left. In the vector at the right, black values indicate zero.

16.5.3 Visualizing the Fourier Transform

When computing the DFT of a real image, we will not be able to write the analytic form of the result, but there are a number of properties that will help us to interpret the result. [Figure 16.7](#) shows the Fourier transform of a 64×64 resolution image of a cube. As the DFT results in a complex representation, there are two possible ways of writing the result. Using the real and imaginary components:

$$\mathcal{L}[u, v] = \text{Re}\{\mathcal{L}[u, v]\} + j \text{Imag}\{\mathcal{L}[u, v]\} \quad (16.21)$$

where Re and Imag denote the real and imaginary part of each Fourier coefficient. Or using a polar decomposition:

$$\mathcal{L}[u, v] = A[u, v] \exp(j\theta[u, v]) \quad (16.22)$$

where $A[u, v] \in \mathbb{R}^+$ is the amplitude and $\theta[u, v] \in [-\pi, \pi]$ is the phase. [Figure 16.7](#) shows both decompositions of the Fourier transform.

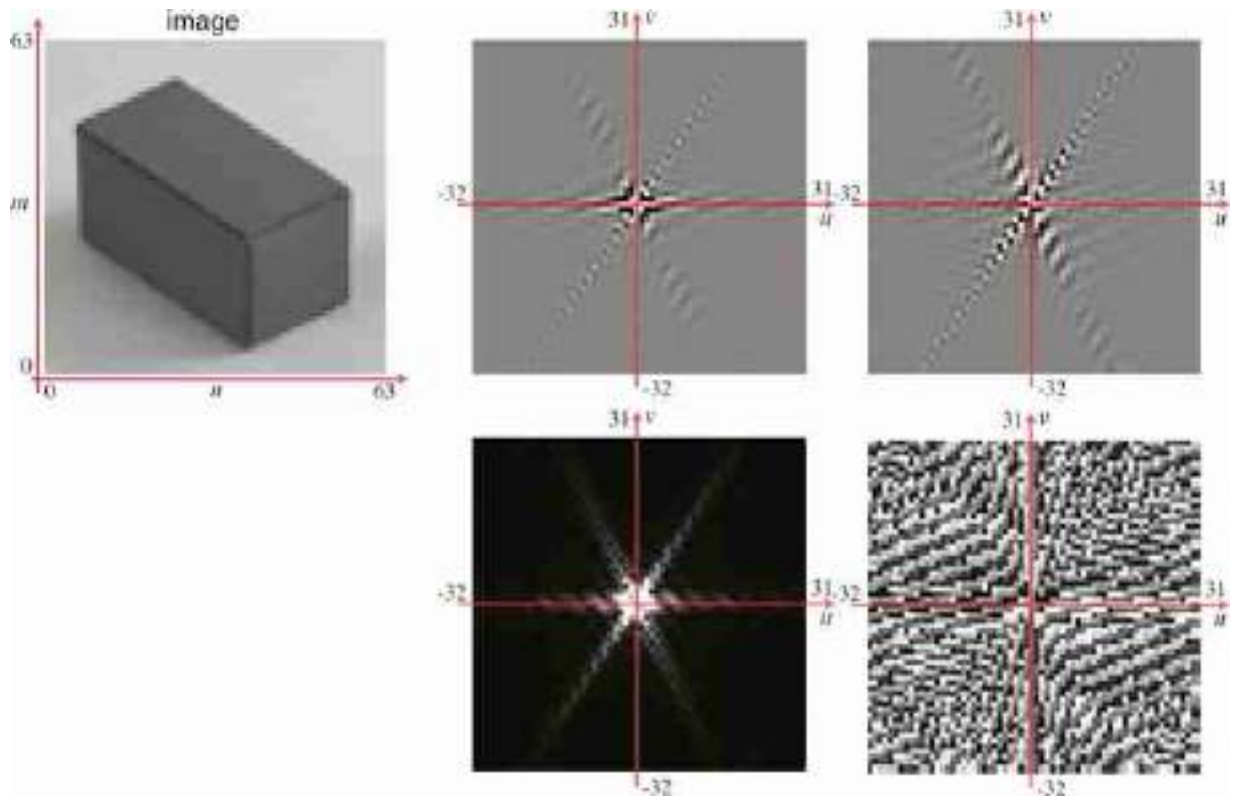


Figure 16.7: DFT of an image and visualization of (top) the real and imaginary components, and (bottom) the amplitude and phase of the Fourier transform.

Upon first learning about Fourier transforms, it may be a surprise for readers to learn that one can synthesize any image as a sum of complex exponentials (i.e., sines and cosines). To help gain insight into how that works, it is informative to show examples of partial sums of complex exponentials. [Figure 16.8](#) shows partial sums of the Fourier components of an image. In each partial sum of N components, we use the largest N components of the Fourier transform. Using the fact that the Fourier basis functions are orthonormal, it is straightforward to show that this is the best least-squares reconstruction possible from each given number of Fourier basis components. This first image shows what is reconstructed from the largest Fourier component which turns out to be $\mathcal{L}[0, 0]$. This component encodes the DC value of the image, therefore the resulting image is just a constant. The next two components correspond to two complex conjugates of a very slow varying wave. And so on. As more components get added, the figure slowly emerges. In this example, the first 127 coefficients are sufficient for recognizing this 64×64 resolution image.

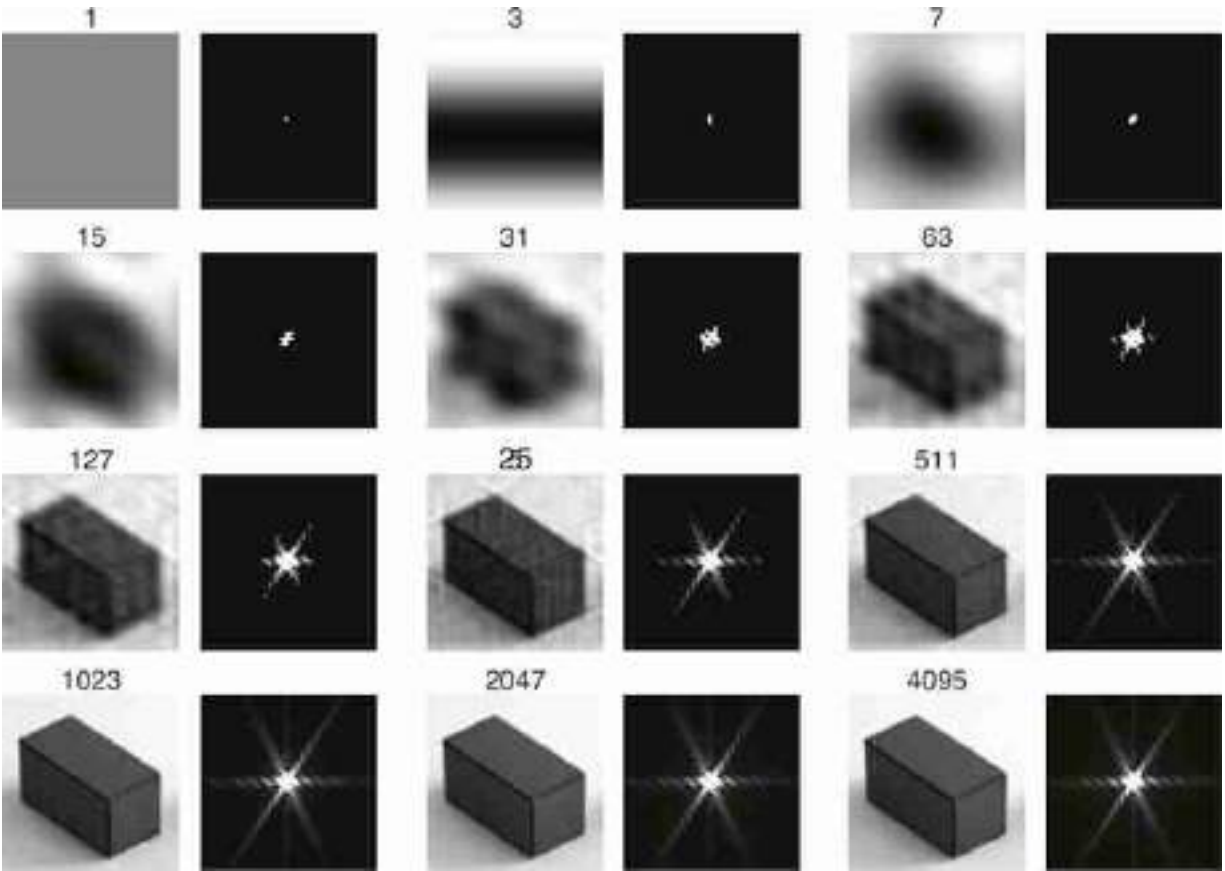


Figure 16.8: Reconstructing an image from the N Fourier coefficients of the largest amplitude. The right frame shows the location, in the Fourier domain, of the N Fourier coefficients, which when inverted, give the image at the left.

16.6 Useful Transforms

It's useful to become adept at computing and manipulating simple Fourier transforms. For some simple cases, we can compute the analytic form of the Fourier transform.

16.6.1 Delta Distribution

The Fourier transform of the delta function $\delta[n, m]$ is

$$\mathcal{F}\{\delta[n, m]\} = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \delta[n, m] \exp\left(-2\pi j \left(\frac{un}{N} + \frac{vm}{M}\right)\right) = 1 \quad (16.23)$$

where the Fourier transform of the delta signal is 1.

$$\delta[n, m] \xrightarrow{\mathcal{F}} 1 \quad (16.24)$$

If we think in terms of the inverse Fourier transform, this means that if we sum all the complex exponentials with a coefficient of 1, then all the values will cancel but the one at the origin, which results in a delta function:

$$\delta[n, m] = \frac{1}{NM} \sum_{u=-N/2}^{N/2} \sum_{v=-M/2}^{M/2} \exp\left(2\pi j \left(\frac{u n}{N} + \frac{v m}{M}\right)\right) \quad (16.25)$$

16.6.2 Cosine and Sine Waves

The Fourier transform of the cosine wave, $\cos\left(2\pi\left(\frac{u_0 n}{N} + \frac{v_0 m}{M}\right)\right)$, is:

$$\sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \cos\left(2\pi\left(\frac{u_0 n}{N} + \frac{v_0 m}{M}\right)\right) \exp\left(-2\pi j \left(\frac{u n}{N} + \frac{v m}{M}\right)\right) = \quad (16.26)$$

$$= \frac{1}{2} (\delta[u - u_0, v - v_0] + \delta[u + u_0, v + v_0]) \quad (16.27)$$

This can be easily proven using Euler's equation (16.12) and the orthogonality between complex exponentials. This results in the Fourier transform relationship:

$$\cos\left(2\pi\left(\frac{u_0 n}{N} + \frac{v_0 m}{M}\right)\right) \xrightarrow{\mathcal{F}} \frac{1}{2} (\delta[u - u_0, v - v_0] + \delta[u + u_0, v + v_0]) \quad (16.28)$$

And for the sine wave, $\sin\left(2\pi\left(\frac{u_0 n}{N} + \frac{v_0 m}{M}\right)\right)$, we have a similar relationship:

$$\sin\left(2\pi\left(\frac{u_0 n}{N} + \frac{v_0 m}{M}\right)\right) \xrightarrow{\mathcal{F}} \frac{1}{2j} (\delta[u - u_0, v - v_0] - \delta[u + u_0, v + v_0]) \quad (16.29)$$

[Figure 16.9](#) shows the DFT of several waves with different frequencies and orientations.

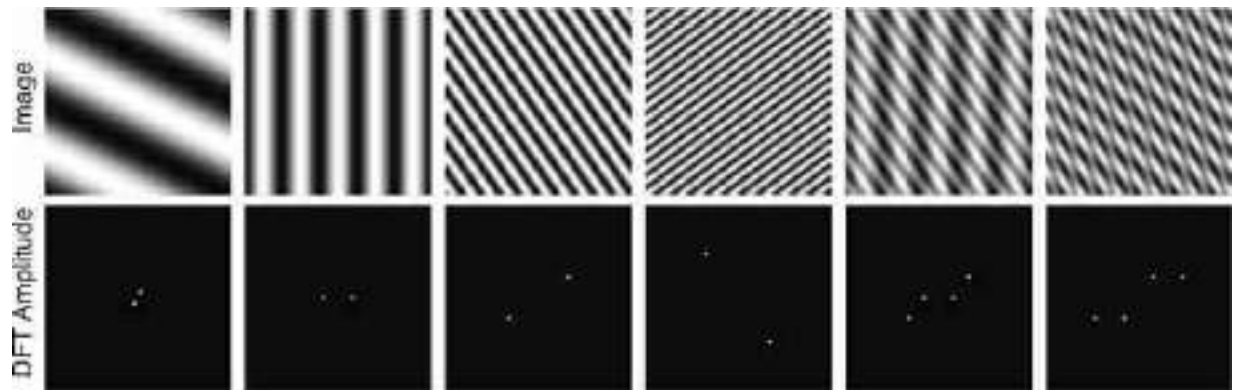


Figure 16.9: Some 2D Fourier transform pairs. Images are 64×64 pixels. The waves are cosine with frequencies $(1, 2)$, $(5, 0)$, $(10, 7)$, $(11, -15)$. The last two examples show the sum of two waves and the product.

[Figure 16.10](#) shows the 2D Fourier transforms of some periodic signals. The depicted signals all happen to be symmetric about the spatial origin. From the Fourier transform equation, one can show that real and even input signals transform to real and even outputs. So for the examples of [figure 16.10](#), we only show the magnitude of the Fourier transform, which in this case is the absolute value of the real component of the transform, and the imaginary component happens to be zero for the signals we'll examine. Also, all these images but the last one are separable (they can be written as the product of two 1D signals). Therefore, their DFT is also the product of 1D DFTs.

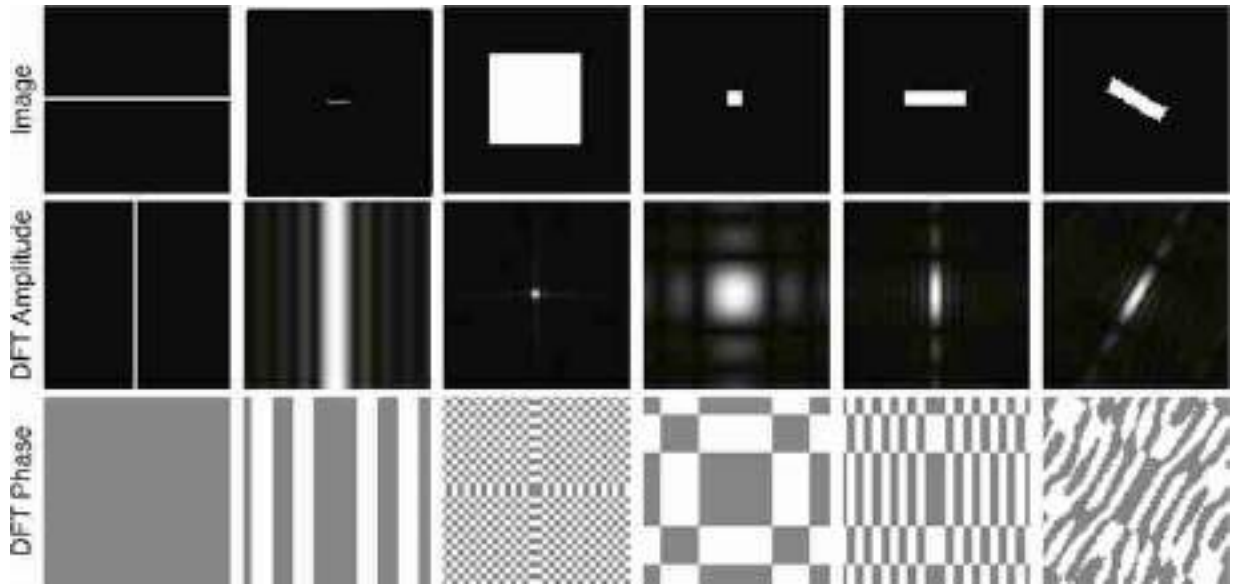


Figure 16.10: Some two-dimensional Fourier transform pairs. Note the trends visible in the collection of transform pairs: As the support of the image in one domain gets larger, the magnitude in the other domain becomes more localized. A line transforms to a line oriented perpendicularly to the first. Images are 64×64 pixels, origin is in the center.

16.6.3 Box Function

The box function is a very useful function that we will use several times in this book. It is good to be familiar with its Fourier transform and how to compute it. The box function is defined as follows:

$$\text{box}_L[n] = \begin{cases} 1 & \text{if } -L \leq n \leq L \\ 0 & \text{otherwise.} \end{cases} \quad (16.30)$$

The box function takes values of 1 inside the interval $[-L, L]$ and it is 0 outside. The duration of the box is $2L + 1$.

It is useful to think of the box function as a finite length signal of length N and to visualize its periodic extension outside of that interval. The following plot shows the box function for $L = 5$ and $N = 32$. In this plot, the box signal is defined between $-N/2 = -16$ and $N/2 - 1 = 15$ and the rest is its periodic extension.

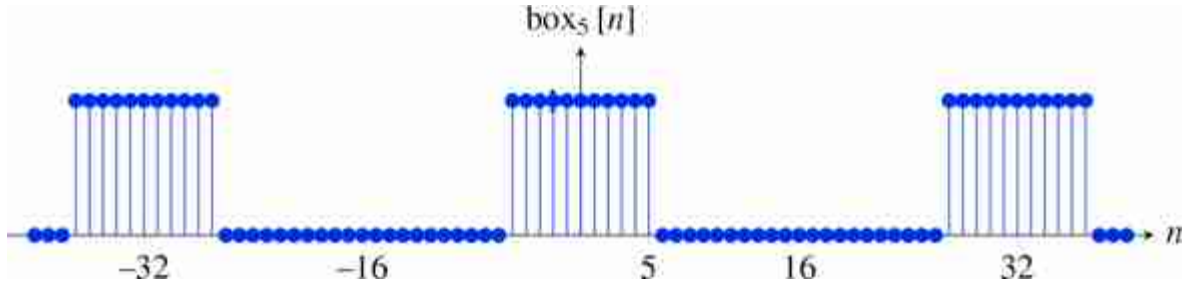


Figure 16.11: Box function for $L = 5$ and $N = 32$.

We will compute here the DFT of the finite length box function, with a length N . To compute the DFT we use the DFT definition and change the interval of the sum making use of the periodic nature of the terms inside the sum:

$$\mathcal{F}\{\text{box}_L[n]\} = \sum_{n=0}^{N-1} \text{box}_L[n] \exp\left(-2\pi j \frac{un}{N}\right) \quad (16.31)$$

$$= \sum_{n=-L}^L \exp\left(-2\pi j \frac{un}{N}\right) \quad (16.32)$$

We can use the equation of the sum of a geometric series:

$$\sum_{n=-L}^L a^n = a^{-L} \sum_{n=0}^{2L} a^n = a^{-L} \frac{1 - a^{2L+1}}{1 - a} = \frac{a^{-(2L+1)/2} - a^{(2L+1)/2}}{a^{-1/2} - a^{1/2}} \quad (16.33)$$

where a is a constant. With $a = \exp\left(-2\pi j \frac{u}{N}\right)$ we can write the sum in equation (16.32) as

$$\sum_{n=-L}^L \exp\left(-2\pi j \frac{un}{N}\right) = \frac{\exp\left(\pi j \frac{u(2L+1)}{N}\right) - \exp\left(-\pi j \frac{u(2L+1)}{N}\right)}{\exp\left(\pi j \frac{u}{N}\right) - \exp\left(-\pi j \frac{u}{N}\right)} \quad (16.34)$$

$$= \frac{\sin(\pi u(2L+1)/N)}{\sin(\pi u/N)} \quad (16.35)$$

This last function in equation (16.35) gets the special name of **discrete sinc** function.

The discrete sinc function (**sincd**) is defined as follows:

$$\text{sincd}(x; a) = \frac{\sin(\pi x)}{a \sin(\pi x/a)}$$

Where a is a constant. This is a symmetrical periodic function with a maximum value of 1.

In summary, we get that The DFT of the box function is the discrete sinc function:

$$\text{box}_L[n] \xrightarrow{\mathcal{F}} \frac{\sin \pi u(2L+1)/N}{\sin \pi u/N} \quad (16.36)$$

We will denote the DFT of the box function, $\text{box}_L[n]$, capitalizing the first letter, $\text{Box}_L[u]$. This function has its maximum value at $u = 0$. The DFT, $\text{Box}_L[u]$, is a symmetric real function. The following plot ([figure 16.12](#)) shows the DFT of the box with $L = 5$, and $N = 32$. One period of the DFT is contained in the interval $[-16, 15]$ and the rest is a periodic extension.

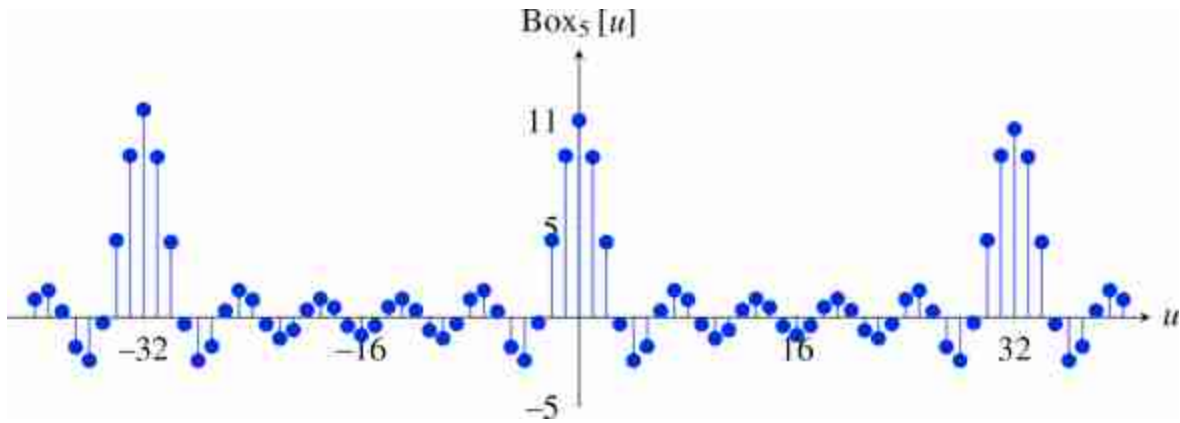


Figure 16.12: DFT of the box filter with $L = 5$, and $N = 32$

Now that we know how to compute the DFT of the 1D box, we can easily extend it to the 2D case. A 2D box is a separable function and can be written as the product of two box functions, $\text{box}_{L_u, L_v}[n, m] = \text{box}_{L_u}[n] \text{box}_{L_v}[m]$. The DFT is the product of the two DFTs,

$$\text{Box}_{L_u, L_v}[u, v] = \text{Box}_{L_u}[u] \text{Box}_{L_v}[v]. \quad (16.37)$$

[Figure 16.10](#) shows several DFTs of 2D boxes of different sizes and aspect ratios.

16.7 Discrete Fourier Transform Properties

It is important to be familiar with the properties for the DFT. [Table 16.1](#) summarizes some important transforms and properties.

Table 16.1

Table of basic DFT transforms and properties.

$\ell [n]$	$\mathcal{L} [u]$
$\ell_1 [n] \circ \ell_2 [n]$	$\mathcal{L}_1 [u] \mathcal{L}_2 [u]$
$\ell_1 [n] \ell_2 [n]$	$\frac{1}{N} \mathcal{L}_1 [u] \circ \mathcal{L}_2 [u]$
$\ell [n - n_0]$	$\mathcal{L} [u] \exp (-2\pi j \frac{un_0}{N})$
$\delta [n]$	1
$\exp (2\pi j u_0 \frac{n}{N})$	$\delta [u - u_0]$
$\cos (2\pi u_0 \frac{n}{N})$	$\frac{1}{2} (\delta [u - u_0] + \delta [u + u_0])$
$\sin (2\pi u_0 \frac{n}{N})$	$\frac{1}{2j} (\delta [u - u_0] - \delta [u + u_0])$
$\text{box}_L [n]$	$\frac{\sin \pi u (2L+1)/N}{\sin \pi u/N}$

16.7.1 Linearity

The Fourier transform and its inverse are linear transformations:

$$\alpha \ell_1 [n, m] + \beta \ell_2 [n, m] \xrightarrow{\mathcal{F}} \alpha \mathcal{L}_1 [u, v] + \beta \mathcal{L}_2 [u, v] \quad (16.38)$$

where α and β are complex numbers. This property can be easily proven from the definition of the DFT. [Figure 16.6](#) shows a color visualization of the complex-value matrix for the 1D DFT.

16.7.2 Separability

An image is separable if it can be written as the product of two 1D signals, $\ell [n, m] = \ell_1 [n] \ell_2 [m]$. If an image is separable, then its Fourier transform is separable: $\mathcal{L} [u, v] = \mathcal{L}_1 [u] \mathcal{L}_2 [v]$

Separability. Although almost all images are not separable, some can be approximated by a separable image. Here is one example. We can use the SVD decomposition to find a separable approximation to the image on the left. The result is shown on the right.



The right-hand image can be written as the product of two 1D signals.

16.7.3 Parseval's Theorem

As the DFT is a change of basis, the dot product between two signals and the norm of a vector is preserved (up to a constant factor) after the basis change. This is stated by Parseval's theorem:

$$\sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \ell_1[n, m] \ell_2^*[n, m] = \frac{1}{NM} \sum_{u=0}^{N-1} \sum_{v=0}^{M-1} \mathcal{L}_1[u, v] \mathcal{L}_2^*[u, v] \quad (16.39)$$

In particular, if $\ell_1 = \ell_2$ this reduces to the **Plancherel theorem**:

$$\sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \|\ell[n, m]\|^2 = \frac{1}{NM} \sum_{u=0}^{N-1} \sum_{v=0}^{M-1} \|\mathcal{L}[u, v]\|^2 \quad (16.40)$$

This relationship is important because it tells us that the energy of a signal can be computed as the sum of the squared magnitude of the values of its Fourier transform.

16.7.4 Convolution

Consider a signal ℓ_{out} that is the result of applying (circular) convolution of two signals, ℓ_1 and ℓ_2 , both of size $N \times M$:

$$\ell_{\text{out}} = \ell_1 \circ \ell_2 \quad (16.41)$$

The question is; how does the Fourier transform of the signal ℓ_{out} relates to the Fourier transforms of the two signals ℓ_1 and ℓ_2 ?

The relationship between the convolution and the Fourier transform is probably the most important property of the Fourier transform and you should be familiar with it.

If we take the Fourier transform of both sides, and use the definition of the Fourier transform, we obtain

$$\begin{aligned}\mathcal{L}_{\text{out}}[u, v] &= \mathcal{F} \{ \ell_1 \circ_{N,M} \ell_2 \} \\ &= \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \left\{ \sum_{k=0}^{N-1} \sum_{l=0}^{M-1} \ell_1[n-k, m-l] \ell_2[k, l] \right\} \exp \left(-2\pi j \left(\frac{nu}{N} + \frac{mv}{M} \right) \right) \quad (16.42)\end{aligned}$$

In these sums, the finite signal $\ell_1[n, m]$ of size $N \times M$ is extended periodically infinitely. The periodic extension allows us to drop the modulo operators. Changing the dummy variables in the sums (introducing $n' = n - k$ and $m' = m - l$), we have

$$\mathcal{L}_{\text{out}}[u, v] = \sum_{k=0}^{N-1} \sum_{l=0}^{M-1} \ell_2[k, l] \sum_{n'=k}^{N-k-1} \sum_{m'=l}^{M-l-1} \ell_1[n', m'] \exp \left(-2\pi j \left(\frac{(n'+k)u}{N} + \frac{(m'+l)v}{M} \right) \right) \quad (16.43)$$

Recognizing that the last two summations give the DFT of $x[n, m]$, using circular boundary conditions, gives

$$\mathcal{L}_{\text{out}}[u, v] = \sum_{k=0}^{N-1} \sum_{l=0}^{M-1} \mathcal{L}_1[u, v] \exp \left(-2\pi j \left(\frac{ku}{N} + \frac{lv}{M} \right) \right) \ell_2[k, l] \quad (16.44)$$

Performing the DFT indicated by the second two summations gives the desired result:

$$\mathcal{L}_{\text{out}}[u, v] = \mathcal{L}_1[u, v] \mathcal{L}_2[u, v] \quad (16.45)$$

Thus, the operation of a convolution, in the Fourier domain, is just a multiplication of the Fourier transform of each term in the Fourier domain:

$$\ell_1[n, m] \circ_{N,M} \ell_2[n, m] \xrightarrow{\mathcal{F}} \mathcal{L}_1[u, v] \mathcal{L}_2[u, v] \quad (16.46)$$

The Fourier transform lets us characterize images by their spatial frequency content. It's also the natural domain in which to analyze space invariant linear processes, because the Fourier bases are the eigenfunctions of all space invariant linear operators. In other words, if you start with a complex exponential, and apply any linear, space invariant operator to it, you always come out with a complex exponential of that same frequency, but, in general, with some different amplitude and phase.

This property lets us examine the operation of a filter on any image by examining how it modulates the Fourier coefficients of any image.

Note that the autocorrelation function does not have this property.

We will discuss this in more detail in section 16.10.

16.7.5 Dual Convolution

Another common operation with working with images is to do products between images. For instance, this might happen if we are applying a mask to an image (which corresponds to multiplying the image by a mask that has 0 in the pixels that we want to mask out).

It turns out that the Fourier transform of the product of two images is the (circular) convolution of their DFTs:

$$\ell_1[n, m] \ell_2[n, m] \xrightarrow{\mathcal{F}} \frac{1}{NM} \mathcal{L}_1[u, v] \circ \mathcal{L}_2[u, v] \quad (16.47)$$

We leave the proof of this property to the reader.

16.7.6 Shift Property, Translation in Space

A shift corresponds to a translation in space. This is a very common operation that has a very particular influence on the Fourier transform of the signal. It turns out that translating a signal in the spatial domain, results in multiplying its Fourier transform by a complex exponential.

To show this, consider an image $\ell[n, m]$, with Fourier transform $\mathcal{L}[u, v]$ and period N, M . When displacing the image by (n_0, m_0) pixels, we get $\ell[n - n_0, m - m_0]$ and its Fourier transform is:

$$\begin{aligned} \mathcal{F}\{\ell[n - n_0, m - m_0]\} &= \\ &= \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \ell[n - n_0, m - m_0] \exp\left(-2\pi j \left(\frac{u n}{N} + \frac{v m}{M}\right)\right) = \\ &= \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \ell[n, m] \exp\left(-2\pi j \left(\frac{u(n + n_0)}{N} + \frac{v(m + m_0)}{M}\right)\right) = \\ &= \mathcal{L}[u, v] \exp\left(-2\pi j \left(\frac{u n_0}{N} + \frac{v m_0}{M}\right)\right) \end{aligned} \quad (16.48)$$

Note that as the signal ℓ and the complex exponentials have the period N, M , we can change the sum indices over any range of size $N \times M$ samples.

In practice, if we have an image and we apply a translation there will be some boundary artifacts. So, in general, this property is only true if we apply a **circular shift**, i.e., pixels that disappear on the boundary due to the

translation reappear on the other side. This is a consequence of the periodic structure of the signal ℓ assumed by the DFT. In practice, when the translation is due to the motion of the camera, the shift will not be circular and new pixels will appear on the image boundary. Therefore, the shift property will be only an approximation.

To see how good of an approximation it is, let's look at one example of a shift due to camera motion. [Figure 16.13](#) shows two images that correspond to a translation with $n_0 = 16$ and $m_0 = -4$. Note that at the image boundaries, new pixels appear in (c) not visible in (a). As this is not a pure circular translation, the result from equation (16.48) will not apply exactly. To verify equation (16.48) let's look at the real part of the DFT of each image shown in [figure 16.13](#)(b) and (d). If equation (16.48) holds true, then the real part of the ratio between the DFTs of the two translated images should be $\cos(-2\pi j(\frac{un_0}{N} + \frac{vm_0}{M}))$ with $N = M = 128$ and $[n_0, m_0] = [16, -4]$. [Figure 16.13](#)(f) shows that the real part of the ratio is indeed very close to a cosine, despite of the boundary pixels which are responsible of the noise (the same is true for the imaginary part). In fact, [figure 16.13](#)(e) shows the inverse DFT of the ratio between DFTs, considering both real and imaginary components, which is very close to an impulse at $[16, -4]$.

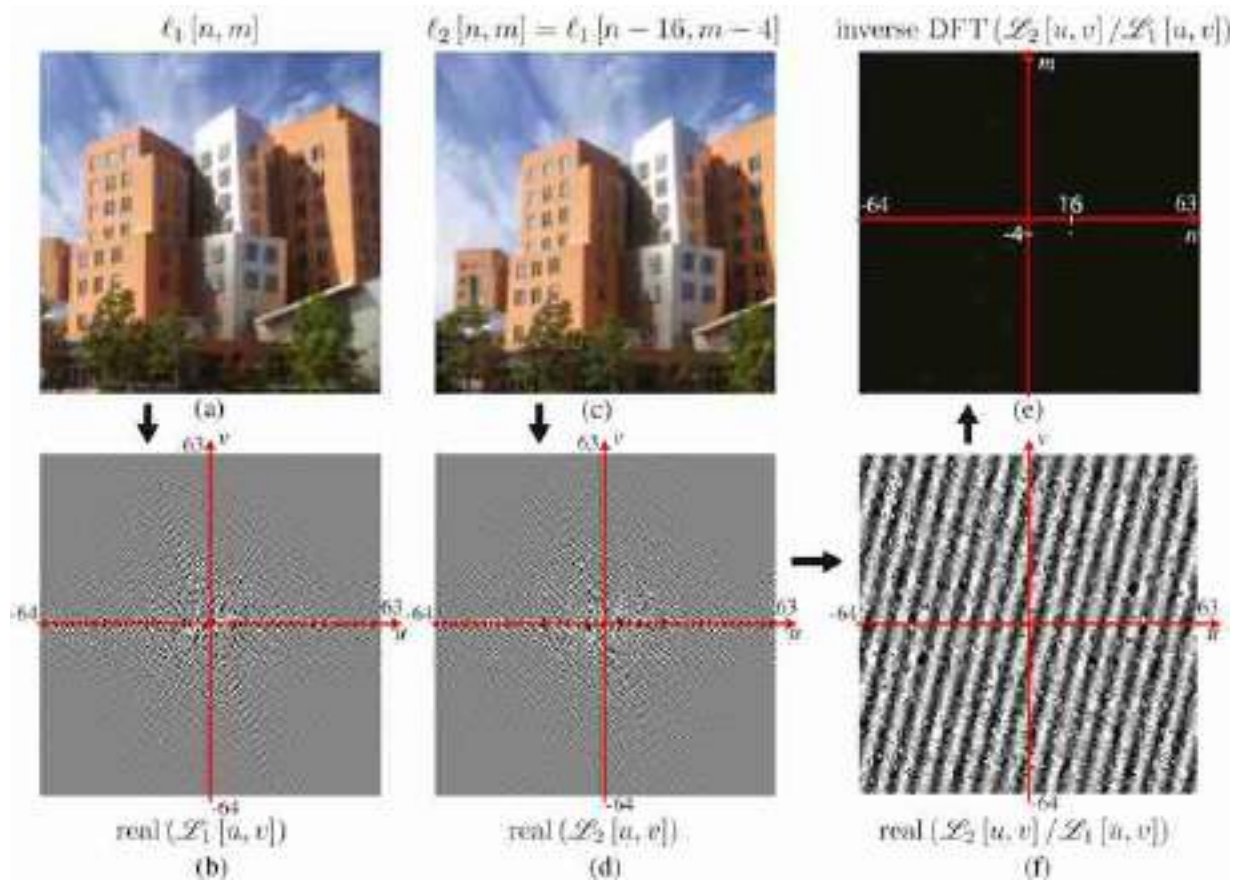


Figure 16.13: Translation in space. Image (c) corresponds to image (a) after a translation of 16 pixels to the right and four pixels down. Images (b) and (d) show the real parts of their corresponding DFTs (with $N = 128$). Image (f) shows the real part of the ratio between the two DFTs, and (e) is the inverse transform of the ratio between DFTs. The inverse is very close to an impulse located at the coordinates of the displacement vector between the two images.

Locating the maximum on [figure 16.13\(f\)](#) can be used to estimate the displacement between two images when the translation corresponds to a global translation. However, this method is not very robust and it is rarely used in practice.

16.7.7 Modulation, Translation in Frequency

If we multiply an image with a complex exponential, its Fourier transform is translated, a property related to the previous one:

$$\ell[n, m] \exp\left(-2\pi j \left(\frac{u_0 n}{N} + \frac{v_0 m}{M}\right)\right) \xrightarrow{\mathcal{F}} \mathcal{Z}[u - u_0, v - v_0] \quad (16.49)$$

This can be proven using the dual convolution property from section section 16.7.5. Note that now the result in equation (16.49) is not a real signal, and

for this reason its Fourier Transform does not have symmetries around $u, v = 0$.

A related relationship is as follows:

$$\ell[n, m] \cos\left(2\pi j\left(\frac{u_0 n}{N} + \frac{v_0 m}{M}\right)\right) \xrightarrow{\mathcal{F}} \mathcal{L}[u - u_0, v - v_0] + \mathcal{L}[u + u_0, v + v_0] \quad (16.50)$$

Multiplying a signal by a wave is called signal modulation and it is one of the basic operations in communications. It is also an important property in image analysis and we will see its use later.

[Figure 16.14](#) shows an example of modulating an image of a stop sign by multiplying by diagonal wave. The bottom row shows the corresponding Fourier transforms.

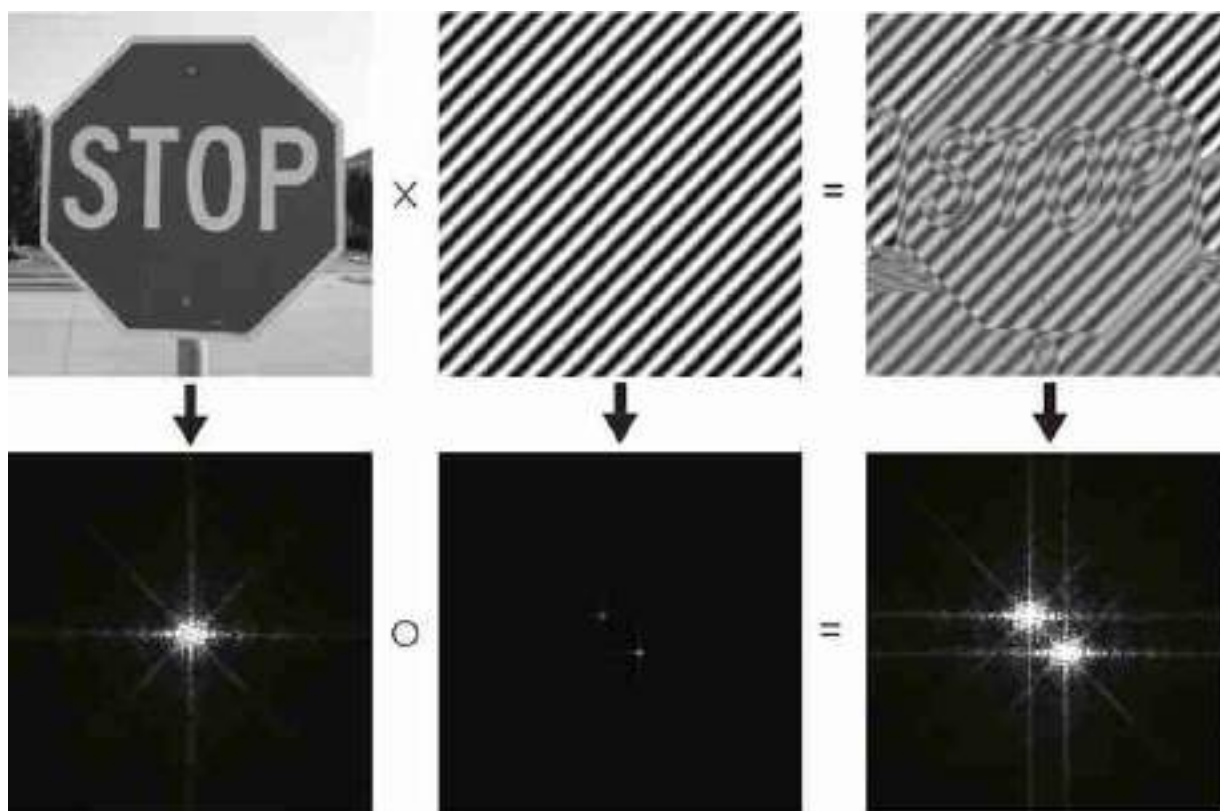


Figure 16.14: Modulation in space. Multiplying an image by a cosine wave results in a new image with a Fourier transform with two copies of the Fourier transform of the original image. Only the magnitude of the Fourier transforms are shown.

Note that a shift and a modulation are equivalent operations in different domains. A shift in space is a modulation in the frequency domain and that

a shift in frequency is a modulation in the spatial domain.

16.8 A Family of Fourier Transforms

The Fourier transform has a multitude of forms and it depends on the conventions used. When you will read papers or other books that make use of the Fourier transform, you will have to carefully check how are they defining the Fourier transform before using it in your own analysis. The important thing is to be consistent in the formulation.

The Fourier transform also will change depending on the type of signal being analyzed. Although in this book we will mostly work with finite length discrete signals and images, it is worth defining the Fourier transform for other signals such as continuous signals (which are a function of a continuous independent variable such as time), or infinite length discrete signals for which the definition of the DFT will not be convenient.

16.8.1 Fourier Transform for Continuous Signals

The continuous domain is convenient when working with functions and operations that are not defined in the discrete domain. For instance, the derivative operator is well defined in the continuous domain but we can only approximate it in the discrete domain. Another example is when using the Gaussian distribution, which is defined as a continuous function. For this reason, sometimes some formulation is easier when done in the continuous domain.

For infinite length signals defined on the continuous domain, the Fourier transform is defined as:

$$\mathcal{L}(w) = \int_{-\infty}^{\infty} \ell(t) \exp(-j\omega t) dt \quad (16.51)$$

where $\mathcal{L}(w)$ is a continuous function on the frequency w (in radians). As before, this transform also has an inverse. The inverse Fourier transform is as follows:

$$\ell(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \mathcal{L}(w) \exp(j\omega t) dw \quad (16.52)$$

Most of the properties that we have seen for the DFT also apply to the continuous domain, replacing sums with integrals.

The convolution between two continuous signals is defined as

$$\ell_{\text{out}}(t) = \ell_1 \circ \ell_2 = \int_{-\infty}^{\infty} \ell_1(t-t') \ell_2(t') dt' \quad (16.53)$$

Those definitions can be extended to 2D dimensions by making all the integrals double and integrating across the spatial variables x and y . Although, in practice, images and filters will be discrete signals, many times it is convenient to think of them as continuous signals.

16.8.2 Fourier Transform for Infinite Length Discrete Signals

Another useful tool is the Fourier transform for discrete signals with infinite length. In this case, the sum becomes infinite, and by replacing $w = 2\pi u/N$ in equation (16.16), we can write:

$$\mathcal{L}(w) = \sum_{n=-\infty}^{\infty} \ell[n] \exp(-jwn) \quad (16.54)$$

The frequency w is now a continuous variable. The Fourier transform $\mathcal{L}(w)$ is a periodic function with period 2π .

The inverse Fourier transform is

$$\ell[n] = \frac{1}{2\pi} \int_{2\pi} \mathcal{L}(w) \exp(jwn) dw \quad (16.55)$$

where the integral is done only in one of the periods.

[Table 16.2](#) summarizes the three transforms we have seen in this chapter. All of them can be easily extended to 2D. There are more variations of the Fourier transform that we will not discuss here.

Table 16.2

Fourier transforms. There are many variants of the Fourier transform. Here we describe the ones we will use in this book.

Time domain	FT	FT ⁻¹	Frequency domain
Discrete time, Finite length (N)	$\mathcal{L}[u] = \sum_{n=0}^{N-1} u[n] e^{-j2\pi n/N}$	$\ell[n] = \frac{1}{N} \sum_{k=0}^{N-1} \mathcal{L}[u] e^{j2\pi kn/N}$	Discrete frequency, Finite length (N)
Continuous time, Infinite length	$\mathcal{L}(u) = \int_{-\infty}^{\infty} u(t) e^{-j2\pi ft} dt$	$u(t) = \int_{-\infty}^{\infty} \mathcal{L}(u) e^{j2\pi ft} df$	Continuous frequency, Infinite length
Discrete time, Infinite length	$\mathcal{L}(u) = \sum_{n=-\infty}^{\infty} \ell[n] e^{-j2\pi fn}$	$\ell[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} \mathcal{L}(u) e^{j2\pi fn} df$	Continuous frequency, Finite length (2π)

16.9 Fourier Analysis as an Image Representation

The Fourier transform has been extensively used as an image representation. In this section we will discuss the information about the picture that is made explicit by this representation.

As we discussed before, the Fourier transform of an image can be written in polar form:

$$\mathcal{L}[u, v] = A[u, v] \exp(j\theta[u, v]) \quad (16.56)$$

where $A[u, v] = |\mathcal{L}[u, v]|$ and $\theta[u, v] = \angle \mathcal{L}[u, v]$.

Decomposing the Fourier transform of a signal in its amplitude and phase can be very useful. It has been a popular image representation during the early days of computer vision and image processing.

If we think in terms of the inverse of the Fourier transform, $A[u, v]$ gives the strength of the weight for each complex exponential and the phase $\theta[u, v]$ translates the complex exponential. The phase carries the information of where the image contours are by specifying how the phases of the sinusoids must line up in order to create the observed contours and edges. In fact, as shown in section 16.7, translating the image in space only modifies the phase of its Fourier transform. In short, one can think that location information goes into the phase while intensity scaling goes into the magnitude.

One might ask which is more important in determining the appearance of the image, the magnitude of the Fourier transform, or its phase. [Figure 16.15](#) shows the result of a classical experiment that consists of computing the Fourier transform for two images and building two new images by swapping their phases [[363](#)].

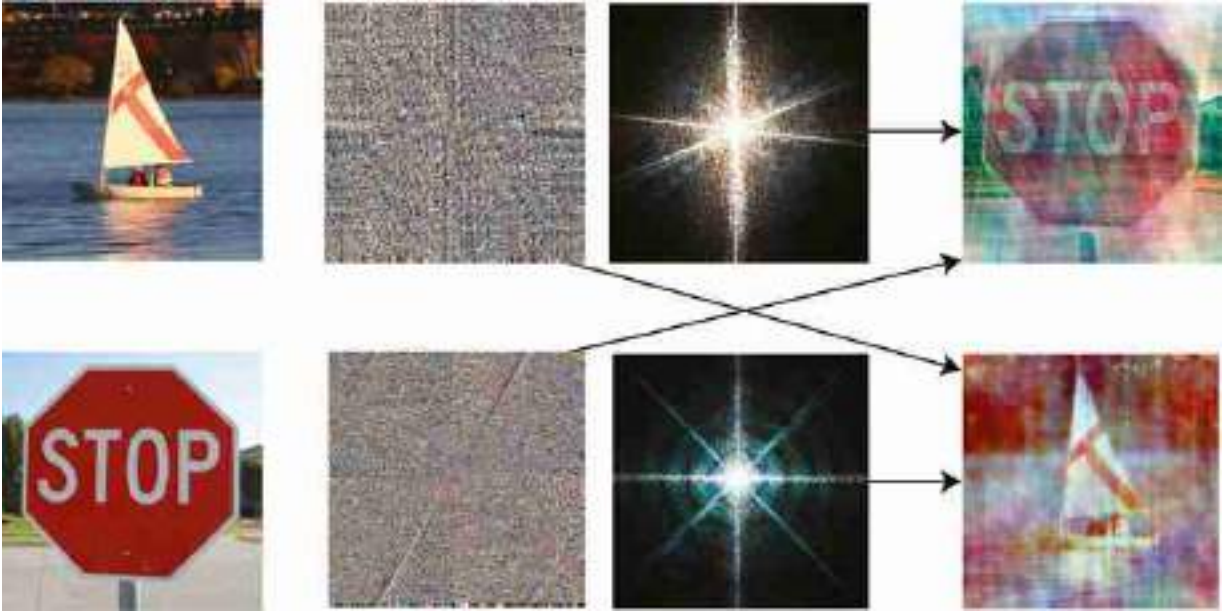


Figure 16.15: Swapping the amplitude and the phase of the Fourier Transform of two images. Each color channel is processed in the same way.

The first output image is the inverse Fourier transform of the amplitude of the first input image and the phase of the DFT of the second input image. The second output image contains the other two terms. The figure shows that the appearance of the resulting images is mostly dominated by the phase of the image they come from. The image built with the phase of the stop sign looks like the stop sign even if the amplitude comes from a different image. [Figure 16.15](#) shows the result in color by doing the same operation over each color channel (red, green, and blue) independently. The phase signal determines where the edges and colors are located in the resulting image. The final colors are altered as the amplitudes have changed.

A remarkable property of natural images: the magnitude of their DFT generally has its maximum at the origin and decays inversely proportional to the radial frequency.

One remarkable property of real images is that the magnitude of the DFT of natural images is quite similar and can be approximated by $A[u, v] = a/(u^2 + v^2)^b$ with a and b being two constants. We will talk more about this property when discussing statistical models of natural images in chapter 27.

This typical structure of the DFT magnitude of natural images has been used as a prior for many tasks such as image denoising.

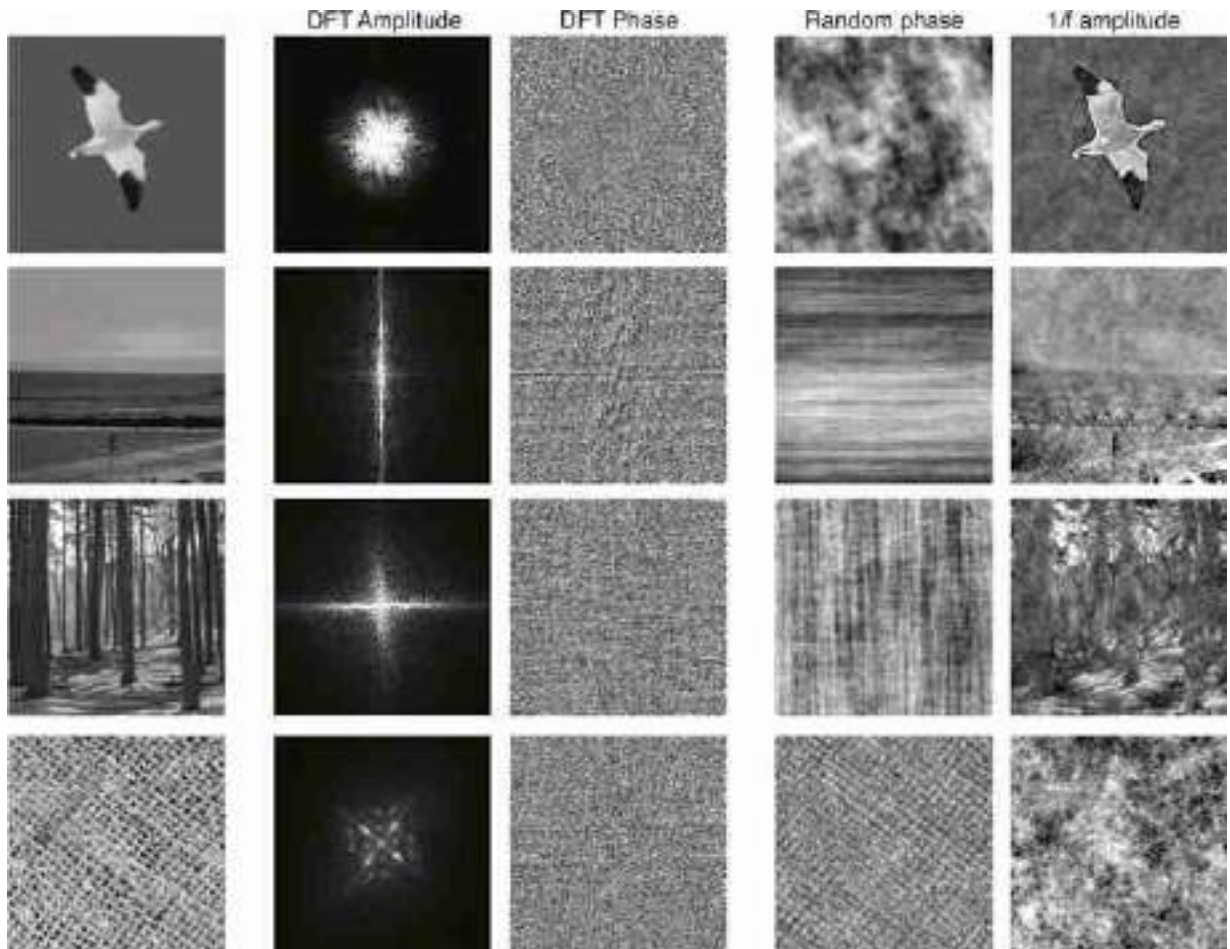


Figure 16.16: The relative importance of phase and amplitude depends on the image. Each row shows one image, its Fourier transform (amplitude and phase), and the resulting images obtained by applying the inverse Fourier transform to a signal with the original amplitude and randomized phase, and a signal with the original phase and a generic fixed $1/f$ amplitude. Note that for the first image, the phase seems to be the most important component.

However, this does not mean that all the information of the image is contained inside the phase only. The amplitude contains very useful information as shown in [figure 16.16](#). To get an intuition of the information available on the amplitude and phase let's do the following experiment: let's take an image, compute the Fourier transform, and create two images by applying the inverse Fourier transform when removing one of the components while keeping the other original component. For the amplitude

image, we will randomize the phase. For the phase image, we will replace the amplitude by a noninformative $A[u, v] = 1/(u^2 + v^2)^{1/2}$ for all images. This amplitude is better than a random amplitude because a random amplitude produces a very noisy image hiding the information available, while this generic form for the amplitude will produce a smoother image, revealing its structure while still removing any information available on the original amplitude. [Figure 16.16](#) shows different types of images and how the DFT amplitude and phase contribute to defining the image content. The top image is inline with the observation from [figure 16.15](#) where phase seems to be carrying most of the image information. However, the rest of the images do not show the same pattern. As we move down along the examples in the figure, the relative importance between the two components changes. And for the bottom image (showing a pseudo-periodic thread texture) the amplitude is the most important component.

The amplitude is great for capturing images that contain strong periodic patterns. In such cases, the amplitude can be better than the phase. This observation has been the basis for many image descriptors [[169](#), [359](#)]. The amplitude is somewhat invariant to location (although it is not invariant to the relative location between different elements in the scene). However the phase is a complex signal that does not seem to make explicit any information about the image.

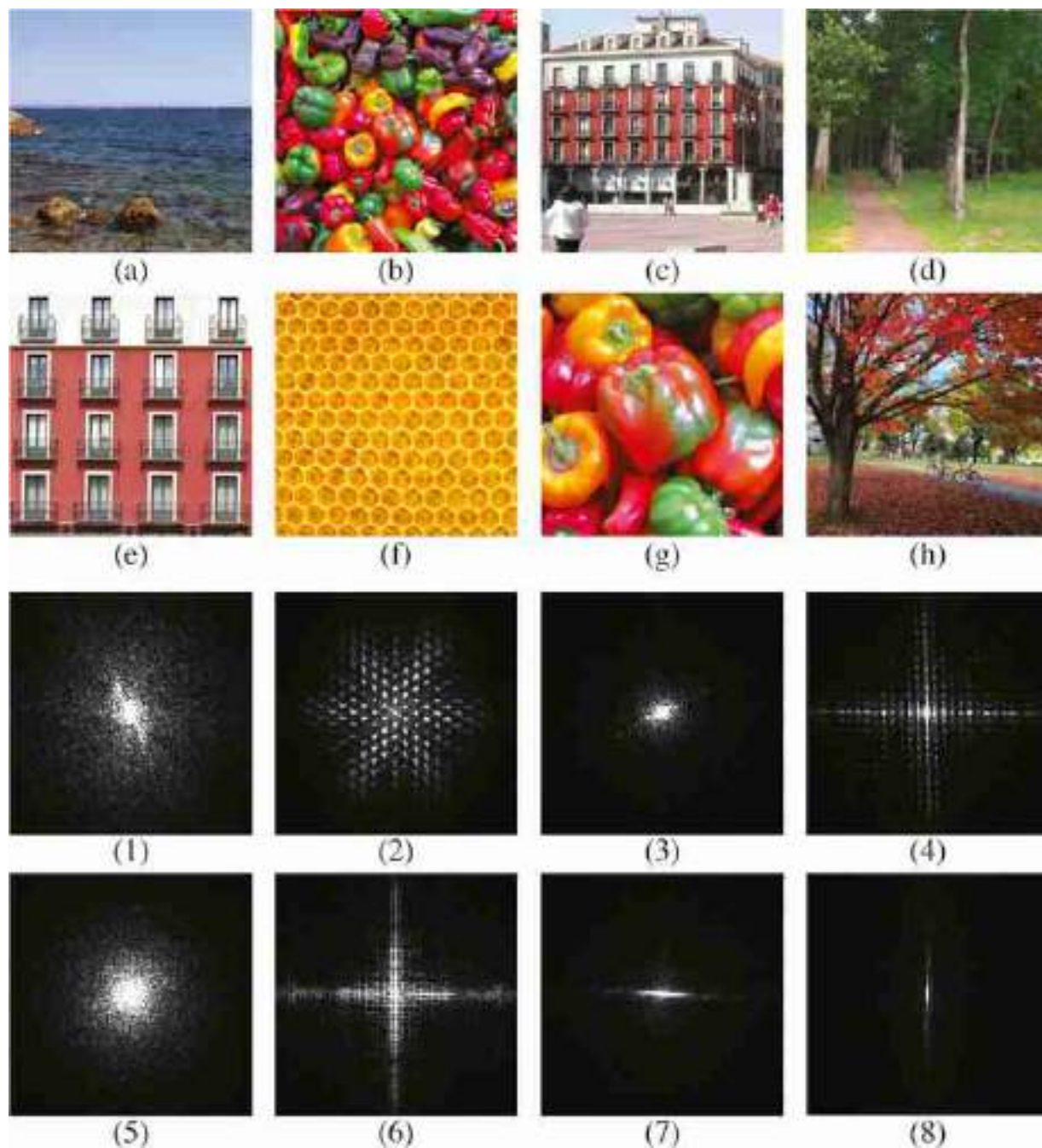


Figure 16.17: The Fourier transform matching game: Match each image (a-h) with its corresponding Fourier transform magnitude (1-8).

Take the following quiz: match the Fourier transform magnitudes with the corresponding images in [figure 16.17⁵](#). Some image patterns are easily visible in the Fourier domain. For instance, strong image contrasts produce oriented lines in the Fourier domain. Periodic patterns are also clearly

visible in the Fourier domain. A periodic pattern in the image domain produces periodic impulses in the Fourier domain. The location of the impulses will be related to the period and orientation or the repetitions.

16.10 Fourier Analysis of Linear Filters

Linear convolutions, despite their simplicity, are surprisingly useful for processing and interpreting images. It's often very useful to blur images, in preparation for subsampling or to remove noise, for example. Other useful processing includes edge enhancement and motion analysis. From the previous section we know that we can write linear filters as convolutions:

$$\ell_{\text{in}}[n, m] \longrightarrow \boxed{h[n, m]} \longrightarrow \ell_{\text{out}}[n, m] = h[n, m] \circ \ell_{\text{in}}[n, m]$$

where $h[n, m]$ is the impulse response of the system, or convolution kernel. We can also write this as a product in the Fourier domain:

$$\mathcal{L}_{\text{in}}[u, v] \longrightarrow \boxed{H[u, v]} \longrightarrow \mathcal{L}_{\text{out}}[u, v] = H[u, v] \mathcal{L}_{\text{in}}[u, v]$$

The function $H[u, v]$ is called the **transfer function** of the filter.

The transfer function of a filter is the Fourier transform of the convolution kernel.

The transfer function will allow us interpret filters by how they modify the spectral content of the input signal. As the Fourier transform of the output is just a product between the Fourier transform of the input and the transfer function, a linear filter simply reweights the spectral content of the signal. It does not create new content, it can only enhance or decrease the spectral components already present in the input.

A convolution between two signals does not create new spectral content. To create new spectral components not present in the input we need to apply nonlinearities.

If we use the polar form:

$$H[u, v] = |H[u, v]| \exp(j \angle H[u, v]) \quad (16.57)$$

The magnitude $|H[u, v]|$ is the **amplitude gain**, and the phase $\angle H[u, v]$ is the **phase shift**. The magnitude at the origin, $|H[0, 0]|$, is the **DC gain** of

the filter (the average value of the output signal is equal to the average value of the input times the DC gain).

The Fourier domain shows that, in many cases, what a filter does is to block or let pass certain frequencies. Filters are many times classified according to the frequencies that they let pass through the filter (low, medium, or high frequencies) as shown in [figure 16.18](#).

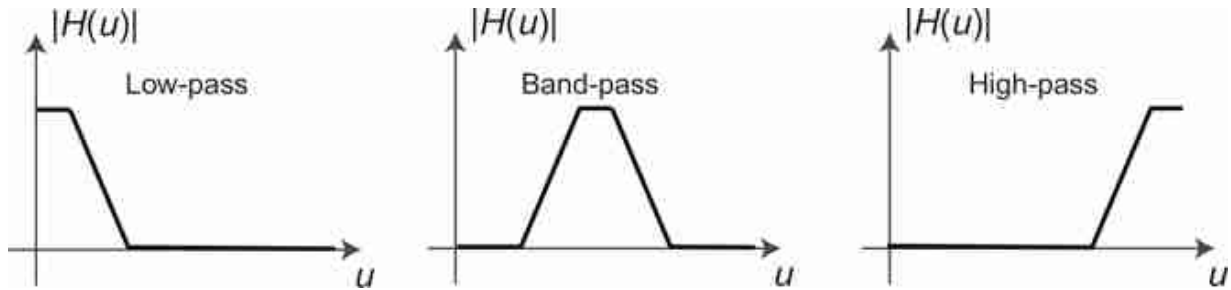


Figure 16.18: Sketch of the frequency responses of low-pass, band-pass, and high-pass filters.

When filtering images, low-pass filters will output a blurry picture encoding the **coarse** elements of the image. A band-pass filter will highlight middle size elements, while the output of a high-pass filter will show the **fine** details of the image. The three images in [figure 16.19](#) show the output image processed by three filters corresponding to the frequency responses of [figure 16.18](#).



Figure 16.19: An image filtered by (left) low-pass, (middle) band-pass, and (right) high-pass filters.

There are many other properties of a filter that can be used to classify linear systems (e.g., causality, stability, phase-changes). Some filters have their main effect over the phase of the signal and they are better understood in the spatial domain. In general, filters affect both the magnitude and the phase of the input signal.

Let's now look at two examples of the application of the Fourier analysis of linear filters.

16.10.1 Example 1: Removing the Columns from the MIT Building

[Figure 16.20\(a\)](#) shows a picture of the main MIT building. The columns produce a quasiperiodic pattern. [Figure 16.20\(b\)](#) shows the magnitude of the DFT of the MIT picture. One can see picks in the horizontal frequency axis, those picks are due to the columns.

To check this we can verify first that the location of the picks is related to the separation of the columns. The image in [figure 16.20\(a\)](#) has a size of 256×256 pixels, and the columns are repeated every 14 pixels. Therefore, the DFT, with $N = 256$, will have picks at the horizontal frequencies: $256/14 = 18.2$, which is indeed what we observe in [figure 16.20\(b\)](#). As the repeated pattern is not a pure sinusoidal function, there will be picks at all the harmonic frequencies $k \frac{256}{14}$, where k is an integer. Note also that the picks seem to produce vertical bands with decreasing amplitude with increasing vertical frequency ν . These bands occur because the columns only occupy a small vertical segment of the image. Also, as the columns only exist in a

portion of the horizontal region of the image, the picks also have some horizontal width.

We can now also check the effect of suppressing those frequencies by zeroing the magnitude of the DFT around each pick (here we zero 7 pixels in the horizontal dimension and all the pixels along the vertical dimension) as shown [figure 16.20\(d\)](#). [Figure 16.20\(c\)](#) shows the resulting image where the columns are almost gone while the rest of the image is little affected. [Figure 16.20\(e\)](#) shows the complementary image (in fact, $a = c + e$) and its DFT in [figure 16.20\(f\)](#).

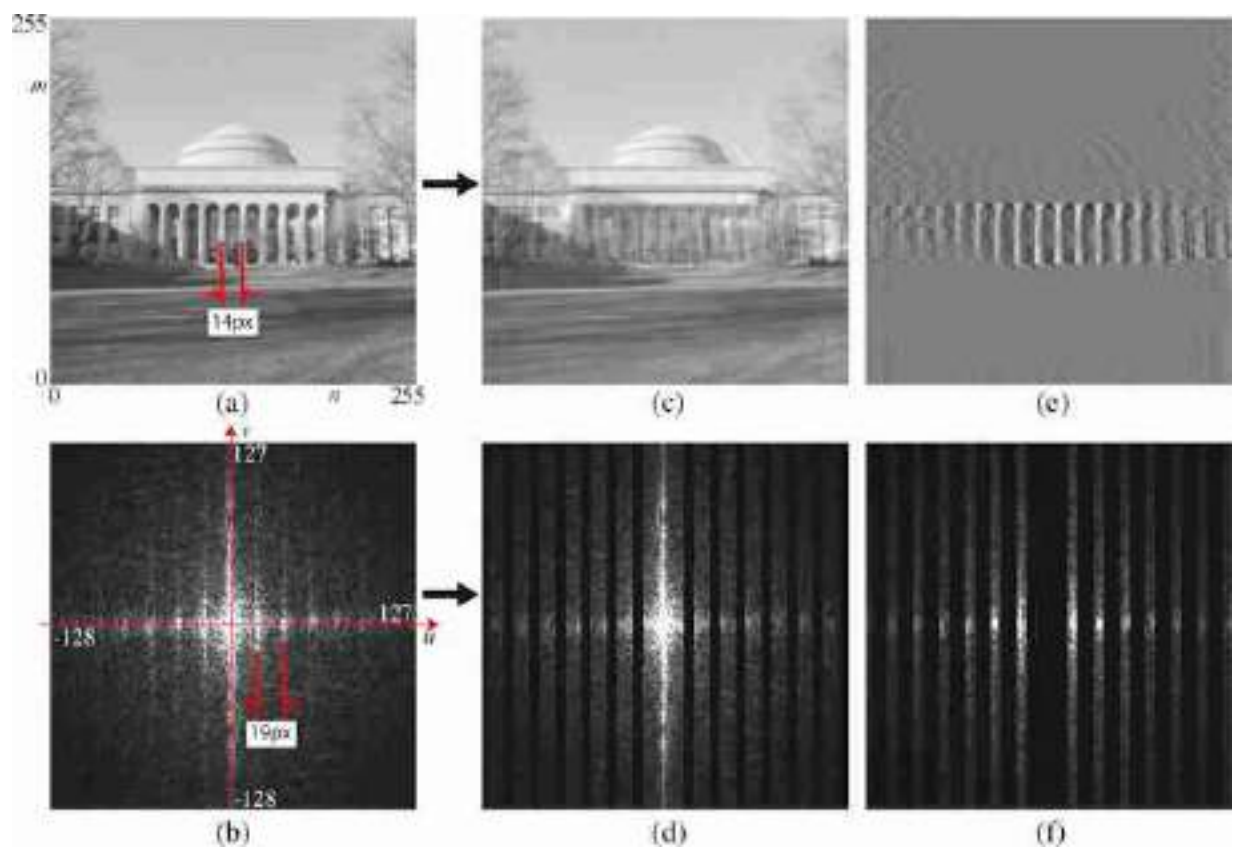


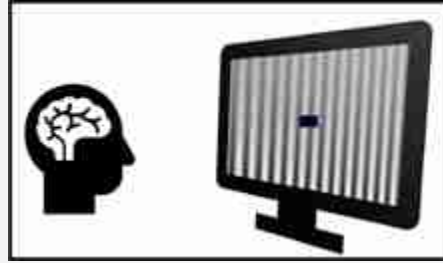
Figure 16.20: Simple filtering in the Fourier domain. (a) The repeated columns of the building of the MIT dome generate harmonics along a horizontal line in the Fourier domain shown in (b). By zeroing out those Fourier components, as done in (d), the columns of the building are substantially removed (c). We can also the complementary operation keeping only those harmonics, shown in (f), which results in keeping only the columns (e).

16.10.2 Example 2: The Human Visual System and the Contrast Sensitivity Function

Before we start describing different types of linear filters in the next chapters, let's start by gaining some subjective experience by playing with one: our own visual system. Although our visual system is clearly a nonlinear system, linear filter theory can explain some aspects of our perception. Indeed, under certain conditions, the first stages of visual computation perform of the visual system can be approximated by a linear filter.

Fourier analysis and the use of sine waves became very popular in the field of visual psychophysics. **Visual psychophysics** is an experimental science that studies the relationship between real world stimuli and our perception.

The concept of **psychophysics** was introduced by Gustav Theodor Fechner (1801–1887).



Using the theory we have studied in this chapter, we can show that when the input to a linear system is a wave of frequency u_0 and amplitude 1, the output is another wave of the same frequency as the input but with an amplitude equal to $|H[u_0]|$:

$$\exp\left(2\pi j \frac{u_0 n}{N}\right) \rightarrow \boxed{H[u]} \rightarrow |H[u_0]| \exp\left(2\pi j \frac{u_0 n}{N} + j\angle H[u]\right)$$

This means that one way of identifying the transfer function of a system is by using as input a wave and measuring the output amplitude as a function of the input frequency u_0 . This inspired a generation of psychophysicists to study how the human visual system behaved when presented with periodic signals.

To experience the transfer function of our own visual system, let's build the following $N \times M$ image:

$$\ell[n, m] = A[m] \sin(2\pi f[n] n/N) \quad (16.58)$$

with

$$A [m] = A_{min} \left(\frac{A_{max}}{A_{min}} \right)^{m/M} \quad (16.59)$$

and

$$f [n] = f_{min} \left(\frac{f_{max}}{f_{min}} \right)^{n/N} \quad (16.60)$$

This image is separable and it is composed of two factors: an amplitude, $A [m]$, that varies only along the vertical dimension, m , and a wave with a frequency, $f [n]$, that varies along the horizontal component, n . The amplitude, $A [m]$, goes from A_{max} to A_{min} on a logarithmic scale. The frequency function, $f [n]$, is defined as an increasing function that starts from $f_{min} = 1$ and grows up to $f_{max} = 60$ (with $N = 2,048$ being the number of horizontal pixels in the image). This image is shown in [figure 16.21](#) and it is also called the **Campbell and Robson chart**. It appears to an observer as a signal with a wave that oscillates slow at the left and faster toward the right and that has high contrast at the bottom and loses contrast towards the top becoming invisible.

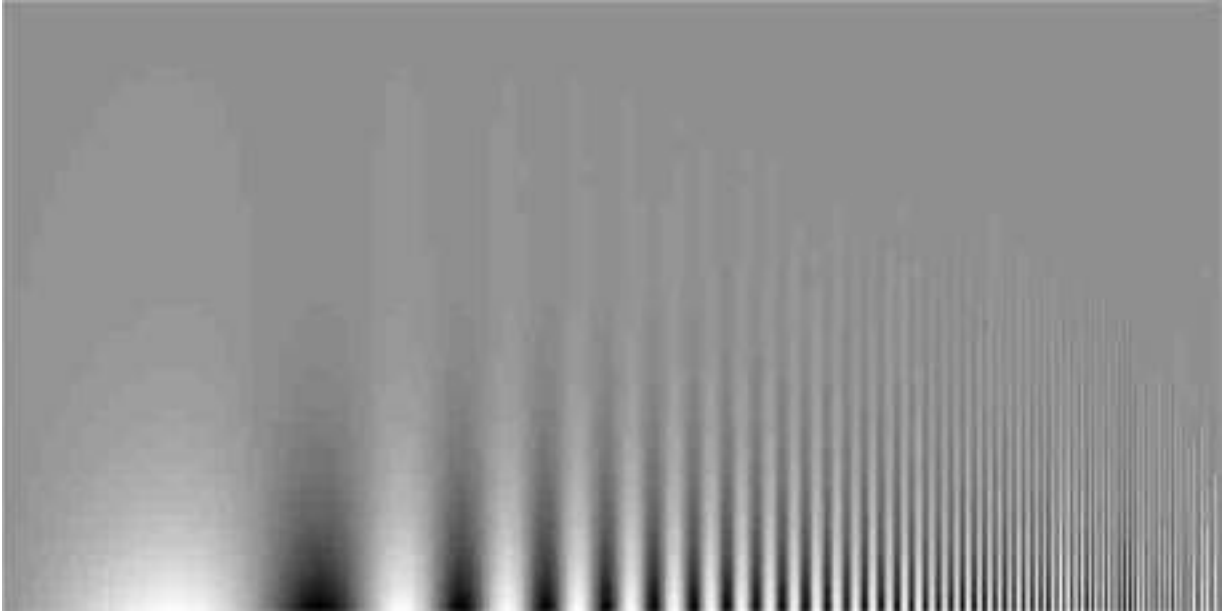


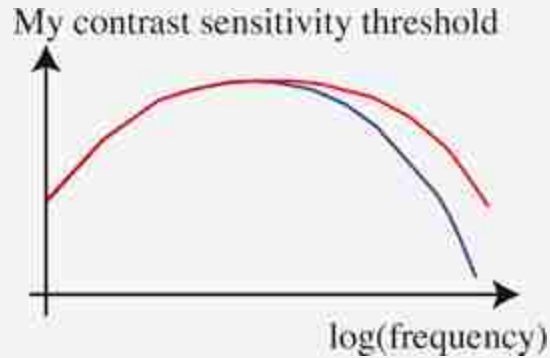
Figure 16.21: Contrast sensitivity function shown by the Campbell and Robson chart. The image shows a sine wave of increasing frequency from left to right, and increasing amplitude from top to bottom. Can you trace a curve over the chart to indicate where the sine wave becomes invisible?

The first sign that our visual system is nonlinear is that we do not perceive the amplitude as changing exponentially fast from top to bottom. It feels more linear. This is because our photo-receptors compute the log of the incoming intensity (approximately).

What is interesting is that [figure 16.21](#) is not perceived as being separable. If you trace the region where the sine wave seems to disappear you will trace a curve. In fact, your visual system acts like a band-pass filter: you are sensitive to middle spatial frequencies (peaking around 6 cycles/degree) and less sensitive to very low spatial frequencies (on the left of the image) and high spatial frequencies (on the right of the image). This curve, known as the contrast sensitivity function (CSF) in psychophysics literature, closely resembles the magnitude of the transfer function of the filter, $H[u, v]$, which approximates the human visual system [\[97\]](#).

The CSF is not a simple linear function but it can be approximated by one under certain conditions. However, the CSF changes depending of many factors such as overall intensity (the pick moves toward the left under low illumination.), adaptation (long exposure to one frequency reduces the sensitivity for that frequency), age, and so on.

Here is my own CSF, traced by hand, with and without eyeglasses. Can you guess which one is with eyeglasses?



(Answer: the red curve is my CSF wearing eyeglasses.)

16.11 Concluding Remarks

The Fourier transform is an indispensable tool for linear systems analysis, image analysis, and for efficient filter output computation. Among the benefits of the Fourier transform representation are that it's easy to analyze images according to spatial frequency, and this represents some progress in interpreting the image over merely a pixel representation. But the Fourier transform has a major drawback as an image representation: it's too global! Every sinusoidal component covers the entire image. The Fourier transform tells us a little about **what** is happening in the image (based on the spatial frequency content), but it tells us nothing about **where** it is happening.

Conversely, a pixel representation gives great spatial localization, but a pixel value by itself doesn't help us learn much about what's going on in the image. A Fourier representation tells us a bit about what's going on, but nothing about where it happens. We seek a representation that's somewhere in between those two extremes.

In the next chapters we will analyze several important linear filters. We will study spatial filters first, and then temporal filters. The filters we will study provide a foundation to understand many basic image processing operations, and to interpret learned filters by neural networks.

5. The correct answers to the quiz [figure 16.17](#) are 1-h, 2-f, 3-g, 4-c, 5-b, 6-e, 7-d, and 8-a.

V

LINEAR FILTERS

In these set of chapters we will discuss a wide family of linear filters. These filters lay the foundation of most representations for images and sequences. Even learning-based representations end up learning filters that are similar to those we will discuss here.

This part is composed of the following chapters:

- **Chapter 17** introduces low-pass filters. These are filters used to lower the resolution of an image and are a key building block of many image processing operations such as image upsampling and downsampling.
- **Chapter 18** describes a set of band-pass filters, such as image derivatives, and several applications.
- **Chapter 19** extends filtering to the temporal domain, and describes applications of spatiotemporal filters for motion estimation, a topic that will be further extended in part XII.

Notation

We will continue using the same notation as in the previous chapters.

- Images and sequences: We will use ℓ to denote images and also sequences: $\ell [n, m, t]$ or $\ell (x, y, t)$
- Convolution kernels: We will use h, g
- Derivatives: We will use subindices for partial derivatives, for example $\ell_x = \partial \ell / \partial x$.

17 Blur Filters

17.1 Introduction

Blur filters are low-pass filters. They remove the high spatial-frequency content from an image leaving only the low-frequency spatial components. The result is an image that has lost details and that looks **blurry**. Image blur has many applications in computer graphics and computer vision. It can be used to reduce noise (as shown in [figure 17.1](#)), to reveal image structures at different scales, or for upsampling and downsampling images.



Figure 17.1: Image denoising by blurring (each color channel is filtered independently). The noise is mostly gone, but also many image details are gone. Image blurring removes all the high-frequency content in the image.

Blurring is implemented by computing local averages over small neighborhoods of input pixel values. This can be done by a convolution. However, there are nonlinear approaches for removing image details such as anisotropic diffusion [[385](#)] and bilateral filtering [[376](#)]. These nonlinear

blurring techniques can be useful when we want to remove noise while preserving some image details such as contours.

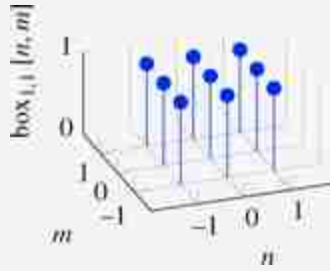
In this chapter we will focus on linear filters for image blurring. We will describe three popular families of blurring filters, discussing properties and limitations.

17.2 Box Filter

Let's start with a very simple low-pass filter, the box filter. In section 16.6.3 we presented the box function. The box filter uses a box function as the convolution kernel. The box convolution kernel can be written as:

$$\text{box}_{N,M}[n, m] = \begin{cases} 1 & \text{if } -N \leq n \leq N \text{ and } -M \leq m \leq M \\ 0 & \text{otherwise.} \end{cases} \quad (17.1)$$

The box filter with $N = M = 1$ is:



Filtering an input image, ℓ_{in} , with a box filter results in the following:

$$\begin{aligned} \ell_{\text{out}}[n, m] &= \text{box}_{N,M}[n, m] \circ \ell_{\text{in}}[n, m] \\ &= \sum_{k,l} \ell_{\text{in}}[n-k, m-l] \text{box}_{N,M}[k, l] \\ &= \sum_{k=-N}^N \sum_{l=-M}^M \ell_{\text{in}}[n-k, m-l] \end{aligned} \quad (17.2)$$

That is, the output value on each location (n, m) is the sum of the input pixels within the rectangle around that location.

Visually, filtering an image with the box filter results in blurring the picture. [Figure 17.2](#) shows some box filters and the corresponding output images (after normalizing the box filter coefficients so that they sum to 1). [Figure 17.2\(a\)](#) shows an image convolved with a squared box kernel (i.e., $N = M$).

[Figures 17.2\(b\)](#) and [17.2\(c\)](#) show the results of blurring in just one direction. Blur happens very often in real life. It happens when we look at something very far away, or at some detail inside a picture, or when we remove our eyeglasses (or wear some that are not ours).

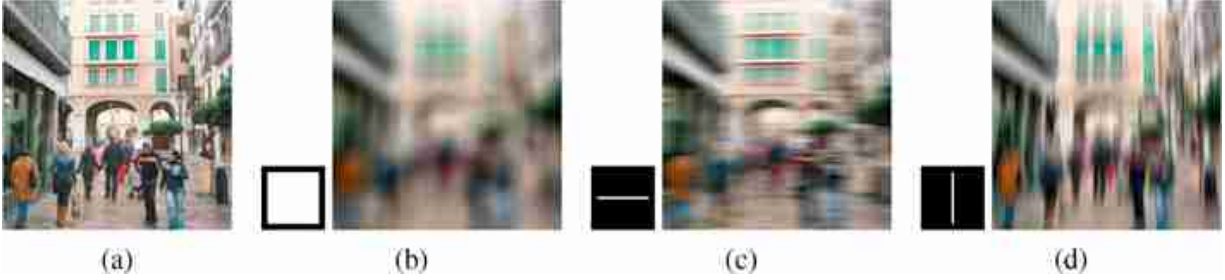


Figure 17.2: (a) Input image. (b) Blurring with a square, (c) a horizontal, and (d) a vertical line. Each color channel is filtered independently.

17.2.1 Properties

The box filter is a low-pass filter. That is, it attenuates the high spatial-frequency content of the input image.

As we already mentioned in section 16.6.3, the two-dimensional (2D) box filter is separable as it can be written as the convolution of two 1D kernels: $\text{box}_{N,M}[n, m] = \text{box}_{N,0} \circ \text{box}_{0,M}$. When the box is large, it can be implemented efficiently using the integral image [\[489\]](#).

One important property of a low-pass filter is the **DC gain**⁶, that is, the gain it has for a constant value input (i.e., the lowest possible input frequency). If the input signal is a constant, i.e., $\ell[n, m] = a$, where a is a real number, the result of convolving the image with the box filter $h_{N,M}$ is also a constant:

$$\ell_{\text{out}}[n, m] = \sum_{k,l} a \text{box}_{N,M}[k, l] = a \sum_{k,l} \text{box}_{N,M}[k, l] = a(2N+1)(2M+1) \quad (17.3)$$

In general, The DC gain of an arbitrary filter $h[n, m]$ is the sum of its kernel values,

$$\text{DC gain} = \sum_{n,m} h[n, m] \quad (17.4)$$

In the example of the box filter with $N = 1$, the DC gain is 3.

In the frequency domain, the DC gain of a filter refers to its gain at frequency of 0. This gain is represented by the value $H[0, 0]$, where H is the result of applying the Discrete Fourier Transform (DFT) to the filter's kernel h . This can be easily proven by checking the definition of the DFT and setting the spatial frequencies to zero. One can verify in [figure 17.3](#) that the value of $|\text{Box}_1[0]|$ is 3. The DC gain of a filter will change the mean value of the input signal.

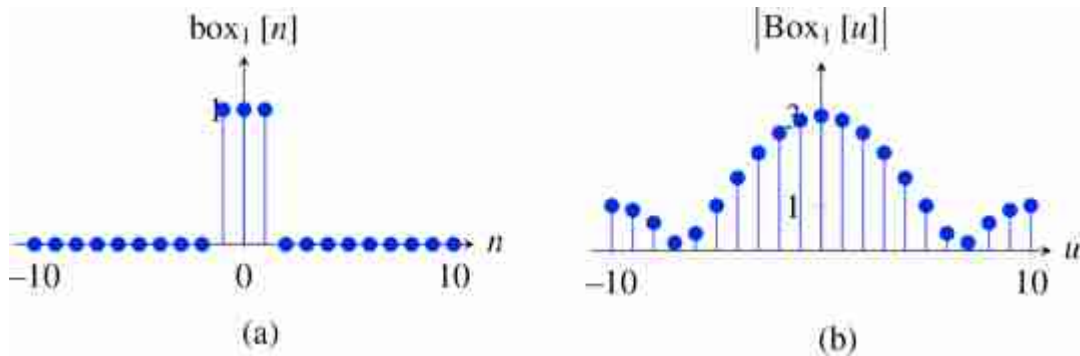


Figure 17.3: (a) A one-dimensional (1D) box filter $([1, 1, 1])$, and (b) its Fourier transform over 20 samples. Note that the frequency gain is not monotonically decreasing with spatial frequency.

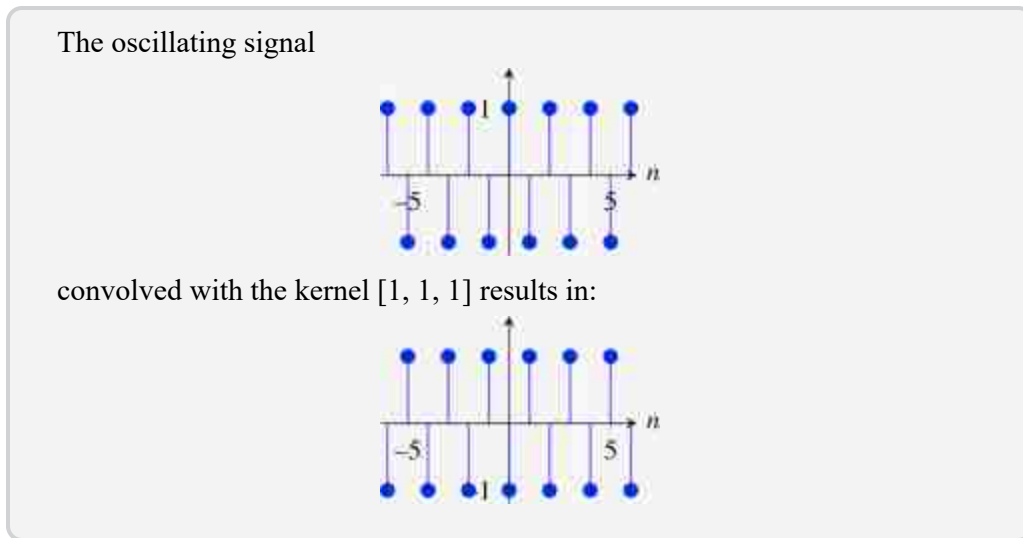
When designing blur kernels (low-pass filters), we generally want to have a DC gain of 1. The reason is that if we have an image with grayscale levels in the range of 0 to 256 and with an average around 128, we will want to preserve the same mean value in the output. For this reason, in most applications, we will normalize the kernel values so that they sum 1. In the case of the box filter this means dividing the kernel values by $(2N + 1)(2M + 1)$.

17.2.2 Limitations

The box filter is simple to implement and to understand, but it has a number of limitations:

- The box filter is not a perfect blurring filter. A blur filter should attenuate high spatial frequencies with stronger attenuation for higher spatial frequencies. However, if you consider the highest spatial frequency, which will be an oscillating signal that takes successively on the values 1 and -1 : $[\dots, 1, -1, 1, -1, 1, -1, \dots]$ when filtered with the

box filter box_1 the result is the same oscillating signal! However, if you filter a wave with lower frequency such as $[\dots, 0.5, 0.5, -1, 0.5, 0.5, -1, \dots]$ then the result is $[\dots, 0, 0, 0, 0, \dots]$. Therefore, the attenuation is not monotonic with spatial frequency as it is shown in [figure 17.3](#). This is not a desirable behavior for a blurring filter and it can cause artifacts to appear. This could be addressed using an even size box filter $[1, 1]$. However, an even box filter is not centered around the origin and the output image will have a half-pixel translation. Therefore, odd filter sizes are preferred.



- If you convolve two boxes you do not get another box. Instead you get a triangle. You can easily check this by convolving two box filters. For instance, in the simple case where $N = 1, M = 0$:

$$\text{box}_{1,0} \circ \text{box}_{1,0} = [1, 1, 1] \circ [1, 1, 1] = [1, 2, 3, 2, 1] \quad (17.5)$$

the output is a **triangular filter** with length $2 \times L - 1$, with $L = 3$ the length of the box filter. When convolving two box filters of different length the result will be a truncated triangle. Although that is not a problem at first sight, it means that if you blur an image twice with box filters, what you get is not the equivalent to blurring only once with a larger box filter.

The next low-pass filter addresses both of these issues.

17.3 Gaussian Filter

One of the important blurring (low-pass) filters in computer vision is the Gaussian filter. The Gaussian filter is important because it is a good model for many naturally occurring filters. It also has a number of properties, as we will discuss here, that make it unique.

The Gaussian distribution is defined in continuous variables. In one dimension:

$$g(x; \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right) \quad (17.6)$$

and in two dimensions:

$$g(x, y; \sigma) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \quad (17.7)$$

The parameter σ adjusts the spatial extend of the Gaussian. The normalization constant is set so that the function integrates to 1. The Gaussian kernel is positive and symmetric (a zero-phase filter).

17.3.1 Discretization

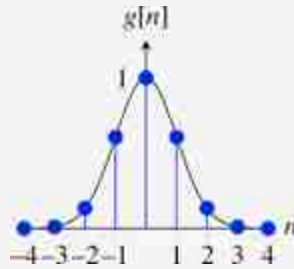
In order to use this filter in practice we need to consider discrete locations and also approximate the function by a finite support function. In practice, we only need to consider samples within three standard deviations $x \in (-3\sigma, 3\sigma)$. At 3σ the amplitude of the Gaussian filter is around 1 percent of its central value. Unfortunately, many of the properties of the Gaussian filter that we will discuss later are only true in the continuous domain and are only approximated when using its discrete form.

For a given standard deviation parameter, σ , the discretized Gaussian kernel is $g[n, m; \sigma]$:

$$g[n, m; \sigma] = \exp\left(-\frac{n^2 + m^2}{2\sigma^2}\right) \quad (17.8)$$

We have removed the normalization constant as the sum of the discrete Gaussian will be different from the integral of the continuous function. So here we prefer to define the form in which the value at the origin is 1. In practice, we should normalize the discrete Gaussian by the sum of its values to make sure that the DC gain is 1.

1D Gaussian with $\sigma = 1$ and its discretized version:



By adjusting the standard deviation, σ , of the Gaussian, it is possible to adjust the level of image detail that appears in the blurred image. [Figure 17.4](#) shows the result of narrow and wider Gaussians applied to an image.

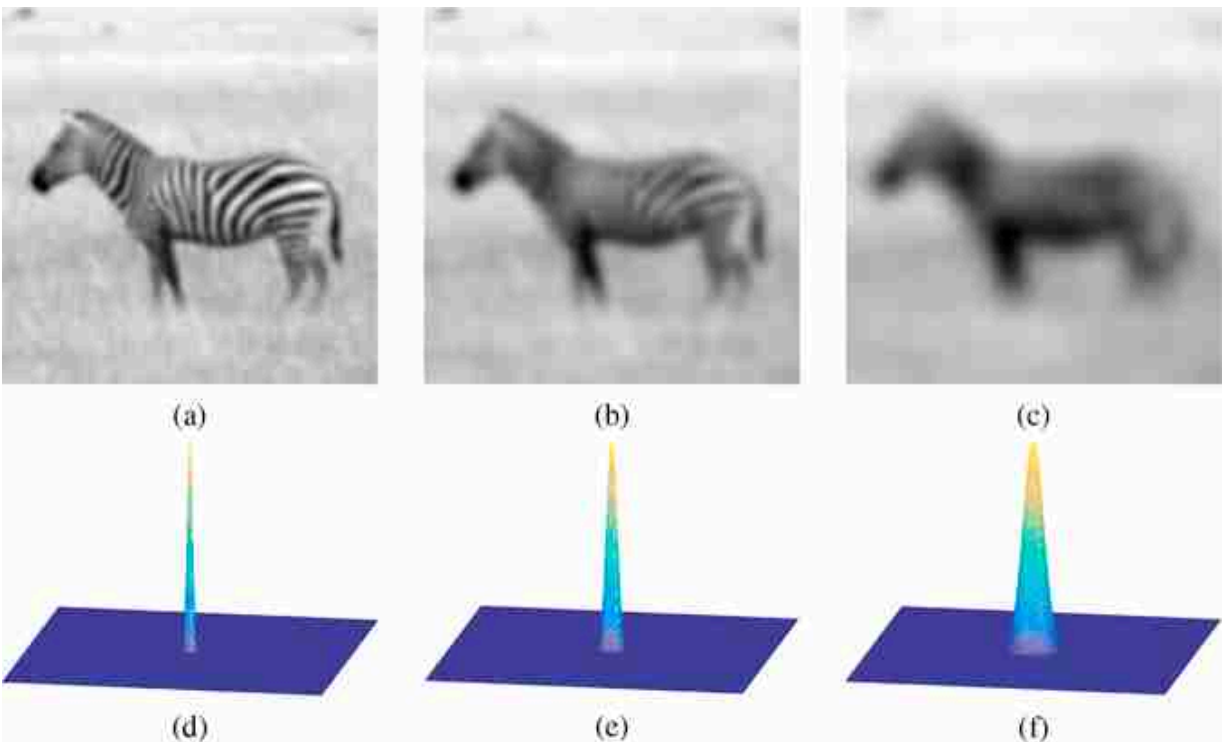


Figure 17.4: An image filtered with three Gaussians with standard deviations: (a) $\sigma = 2$, (b) $\sigma = 4$, and (c) $\sigma = 8$. Plots (d–f) show the three Gaussians over the same spatial support as the image. The discrete Gaussians are approximated by sampling the continuous Gaussian. The convolutions are performed with mirror boundary conditions.

The n -dimensional Gaussian filter has the additional computational advantage that it can be applied as a concatenation of n 1D Gaussian filters. This can be seen by writing the 2D Gaussian, equation (17.8), in the

convolution equation, equation (15.14). Letting g^x and g^y be the 1D Gaussian convolution kernels in the horizontal and vertical directions (i.e., $g^x[n] = g[n, 0]$, and $g^y[m] = g[0, m]$), we have

$$\begin{aligned}
 g[n, m] \circ \ell[n, m] &= \sum_{k, l} g[n-k, m-l] \ell[k, l] \\
 &= \sum_{k, l} \exp\left(-\frac{(n-k)^2 + (m-l)^2}{2\sigma^2}\right) \circ \ell[n, m] \\
 &= \sum_k \exp\left(-\frac{(n-k)^2}{2\sigma^2}\right) \left(\sum_l \exp\left(-\frac{(m-l)^2}{2\sigma^2}\right) \ell[k, l]\right) \\
 &= g^x \circ (g^y \circ \ell[n, m])
 \end{aligned}$$

This can save quite a bit in computation time when applying the convolution of equation (17.9). If the 2D convolution kernel is $N \times N$ samples, then a direct convolution of that 2D kernel scales in proportion to N^2 , since equation (17.9) requires one multiplication per image position per kernel sample. Using the cascade of two 1D kernels, resulting in an equivalent 2D filter of the same size, scales in proportion to $2N$.

Another application of blurring is to remove distracting high-resolution image details. [Figure 17.5](#) shows a Gaussian low-pass filter applied to remove unwanted image details (the blocky artifacts) from an image [\[185\]](#).

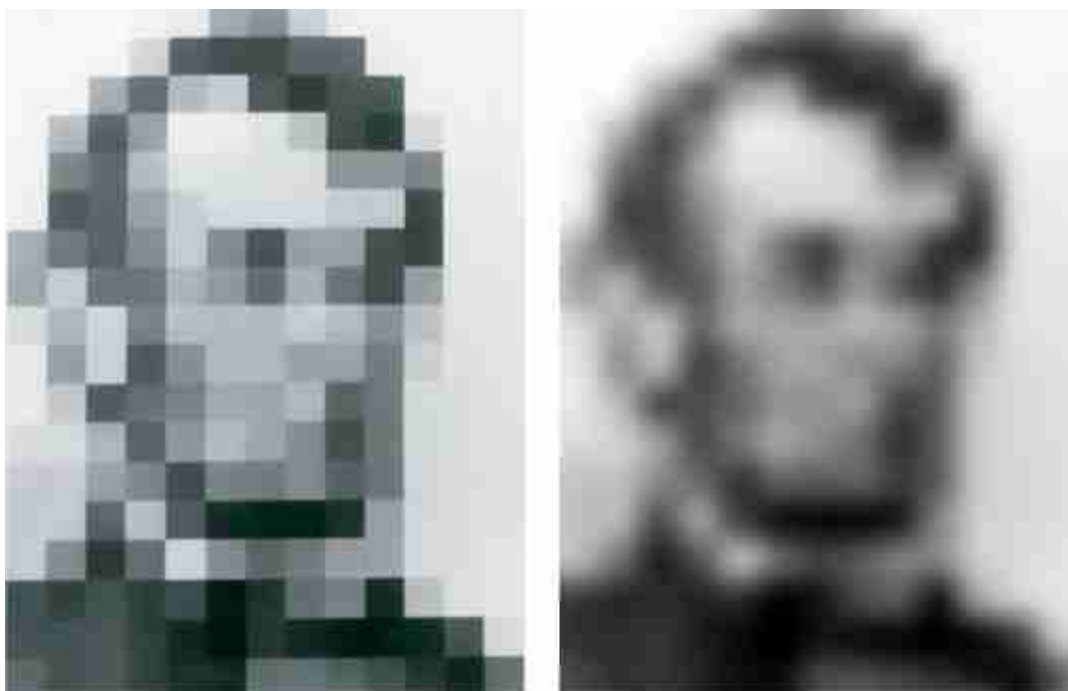


Figure 17.5: (Left) Input image. (Right) Blurred version. The left version has many spurious details introduced by the blocky style of the image. The right image has been blurred by a large Gaussian filter. *Source:* Image by Bela Julesz and Leon Harmon, 1971 [185].

17.3.2 Properties of the Continuous Gaussian

- The n -dimensional Gaussian is the only completely circularly symmetric operator that is separable.
- The continuous Fourier transform (FT) of a Gaussian is also a Gaussian. For a 1D Gaussian, its Fourier transform is:

$$G(w; \sigma) = \exp\left(-\frac{w^2 \sigma^2}{2}\right) \quad (17.9)$$

and in 2D the Fourier transform is:

$$G(w_x, w_y; \sigma) = \exp\left(-\frac{(w_x^2 + w_y^2) \sigma^2}{2}\right) \quad (17.10)$$

Note that this function is monotonically decreasing in magnitude for increasing frequencies, and it is also radially symmetric.

- The width of the Gaussian Fourier transform decreases with σ (this is the opposite behavior to the Gaussian in the spatial domain).
- The convolution of two n -dimensional Gaussians is an n -dimensional Gaussian.

$$g(x, y; \sigma_1) \circ g(x, y; \sigma_2) = g(x, y; \sigma_3) \quad (17.11)$$

where the variance of the result is the sum $\sigma_3^2 = \sigma_1^2 + \sigma_2^2$. This is a remarkable property of Gaussian filters and is the basis of the Gaussian pyramid that we will see later in chapter 23. To prove this property, one can use the Fourier transform of the Gaussian and the fact that the convolution is the product of Fourier transforms.

- The Gaussian is the solution to the heat equation.
- Repeated convolutions of any function concentrated in the origin result in a Gaussian (central limit theorem).
- In the limit $\sigma \rightarrow 0$ the Gaussian becomes an impulse. This property is shared by many other functions, but it is a useful thing to know.

17.3.3 Limitations

However, many of these properties are only true for the continuous version of the Gaussian and do not work for its discrete approximation $g[n, m; \sigma]$ obtained by directly sampling the values of the Gaussian at discrete locations. To see this, let's look at one example in 1D. Let's consider a Gaussian with variance $\sigma^2 = 1/2$. It can be approximated by five samples. We will call this approximation g_5 and it takes the values:

$$g_5[n] = [0.0183, 0.3679, 1.0000, 0.3679, 0.0183] \quad (17.12)$$

You can check that if you compute the approximation for $\sigma^2 = 1$ by discretizing the Gaussian, the result obtained is not equal to doing $g_5 \circ g_5$. Therefore, as you apply successive convolutions of discretized Gaussians the errors will accumulate. That is, the convolution of discretized Gaussians is not a Gaussian anymore.

Note that the convolution of $g_5[n]$ with the wave $[1, -1, 1, -1, \dots]$ is not zero. This is to be expected from the form of the FT of the Gaussian. This is not a strong limitation and in many applications it does not matter. However, in some cases is important to completely cancel the highest frequencies (like when applying antialiasing filters as we will see later).

The next low-pass filter addresses the limitations of the box and Gaussian filters.

17.4 Binomial Filters

In practice, there are very efficient discrete approximations to the Gaussian filter that, for certain σ values, have nicer properties than when working with discretized Gaussians. One common approximation of the Gaussian filter is to use **binomial coefficients** [77]. Binomial filters are obtained by successive convolutions of the box filter [1, 1].

The binomial coefficients use the central limit theorem to approximate a Gaussian as successive convolutions of a very simple function. The binomial coefficients form the **Pascal's triangle** as shown in [figure 17.6](#).

b_0					1						$\sigma_0^2 = 0$
b_1					1		1				$\sigma_1^2 = 1/4$
b_2					1		2		1		$\sigma_2^2 = 1/2$
b_3					1		3		3		$\sigma_3^2 = 3/4$
b_4					1		4		6		$\sigma_4^2 = 1$
b_5					1		5		10		$\sigma_5^2 = 5/4$
b_6					1		6		15		$\sigma_6^2 = 3/2$
b_7					1		7		21		$\sigma_7^2 = 7/4$
b_8					1		8		28		$\sigma_8^2 = 2$

Figure 17.6: Binomial coefficients. To build the Pascal's triangle, each number is the sum of the number above to the left and the one above to the right.

Each row of [figure 17.6](#) shows one 1D-binomial kernel. The first binomial kernel, b_0 , is the impulse.

17.4.1 Properties

- Binomial coefficients provide a compact approximation of the Gaussian coefficients using only integers. Note that the values of b_2 are different from g_5 despite that both will be used as approximations to the a Gaussian with the same variance $\sigma^2 = 1/2$. The thing to note is that the variance of g_5 is not really $\sigma^2 = 1/2$ despite of being obtained by discretizing a Gaussian with that variance.

- The sum of all the coefficients (DC gain) for each binomial filter b_n is 2^n , and their spatial variance is $\sigma^2 = n/4$.
- One remarkable property of the binomial filters is that $b_n \circ b_m = b_{n+m}$, and, therefore, $\sigma_n^2 + \sigma_m^2 = \sigma_{n+m}^2$, which is the analogous to the Gaussian property in the continuous domain. That is, the convolution of two binomial filters is another binomial filter.
- The simplest approximation to the Gaussian filter is the 3-tap binomial kernel:

$$b_2 = [1, 2, 1] \quad (17.13)$$

This filter is interesting because it is even (so it can be applied to an image without producing any translation) and its discrete Fourier transform (DFT) is:

$$B_2[u] = 2 + 2 \cos(2\pi u/N) \quad (17.14)$$

The frequency gain is shown in [figure 17.7\(b\)](#). The gain decreases monotonically (there are no ripples) with spatial frequency, u , and it becomes zero at the highest frequency, $G_3[10] = 0$.

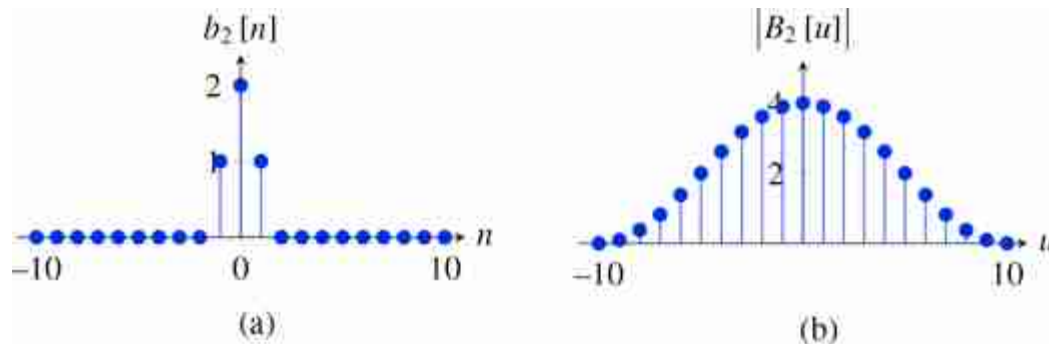


Figure 17.7: (a) A one-dimensional three-tap approximation to the Gaussian filter ($[1, 2, 1]$) and, (b) its Fourier transform for $N = 20$ samples.

- All the even binomial filters can be written as successive convolutions with the kernel $[1, 2, 1]$. Therefore, their Fourier transform is a power of the Fourier transform of the filter $[1, 2, 1]$ and therefore they are also monotonic:

$$B_{2n}[u] = (2 + 2 \cos(2\pi u/N))^n \quad (17.15)$$

The filter transfer function, B_{2n} , is real and positive. It is a **zero-phase filter**.

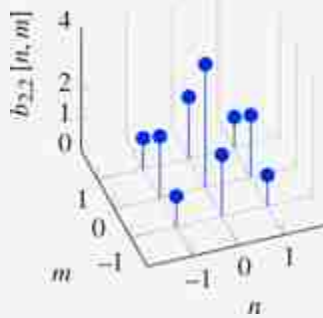
- For all the binomial filters b_n , when they are convolved with the wave $[1, -1, 1, -1, \dots]$, the result is the zero signal $[0, 0, 0, 0, \dots]$. This is a very nice property of binomial filters and will become very useful later when talking about downsampling an image (see chapter 21).

17.4.2 2D Binomial Filters

The Gaussian in 2D can be approximated, using separability, as the convolution of two binomial filters one vertical and another horizontal. For instance:

$$b_{2,2} = b_{2,0} \circ b_{0,2} = \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} \circ \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad (17.16)$$

2D binomial filter:



The filter $b_{2,2}$ has a DC gain of 16, so we will divide the convolution output by 16 so that the output image has a similar contrast to the input image.

[Figure 17.8](#) shows an image corrupted by high-frequency checkerboard-pattern noise. The next two images are the result of filtering the noisy image with a 3×3 box filter (middle) and with the $b_{2,2}$ binomial filter. The 3×3 box filter cannot completely eliminate the noise, while the binomial filter can perfectly cancel this checkerboard noise. Both the box and the binomial filter reduce the resolution of the input image.

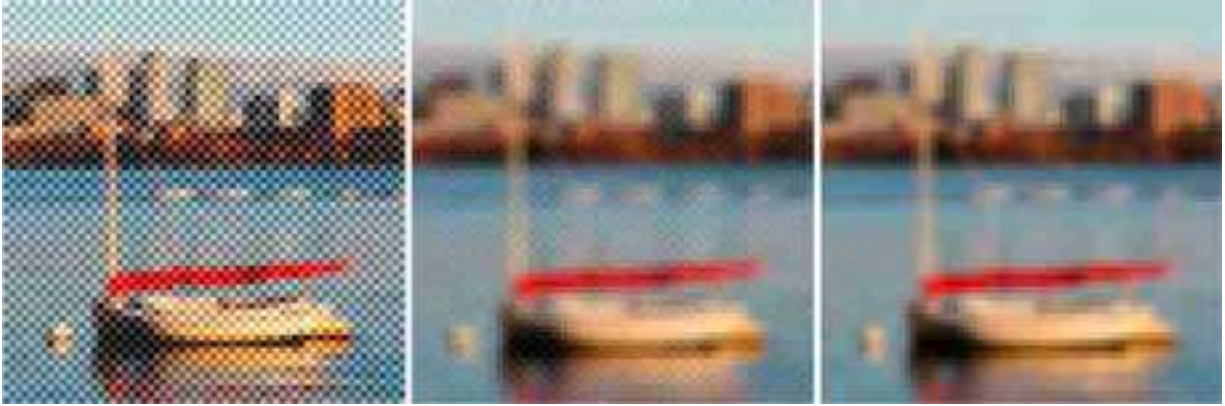


Figure 17.8: Image corrupted by a checkerboard-pattern noise (left) and its output to two different blur kernels: (middle) 3×3 box filter. (right) Binomial filter $b_{2,2}$.

17.5 Concluding Remarks

The Gaussian and the binomial filters are widely used in computer vision. In particular, the binomial filter $[1, 2, 1]/4$ (here normalized so that its DC gain is 1), and its 2D extension, are very useful kernels that you can use in many situations like, when you need to remove high-frequency noise, or downsample an image by a factor of 2. Blur kernels are useful when building image pyramids, or in neural networks when performing different pooling operations or when resizing feature maps.

Keep the 2D binomial filter close to you:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} / 16$$

6. Remember that the DC value of a signal is its mean value. DC is an old name derived from Direct Current.

18 Image Derivatives

18.1 Introduction

Computing image derivatives is an essential operator for extracting useful information from images. As we show in the previous chapter, image derivatives allowed us computing boundaries between objects and to have access to some of the three-dimensional (3D) information lost when projecting the 3D world into the camera plane. Derivatives are useful because they give us information about where are happening the changes in the image and we expect those changes to be correlated with transitions between objects.

The operator that computes the image derivatives along the spatial dimensions (x and y) is a linear system:

$$\ell(x, y) \longrightarrow \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y} \right) \longrightarrow (\ell_x(x, y), \ell_y(x, y))$$

Figure 18.1: Computing image derivatives along x - and y -dimensions.

The derivative operator is linear and translation invariant. Therefore, it can be written as a convolution. The following images show an input image and the resulting derivatives along the horizontal and vertical dimensions using one of the discrete approximations that we will discuss in this chapter.

18.2 Discretizing Image Derivatives

If we had access to the continuous image, then image derivatives could be computed as: $\partial \ell(x, y) / \partial x$, which is defined as

$$\frac{\partial \ell(x, y)}{\partial x} = \lim_{\epsilon \rightarrow 0} \frac{\ell(x + \epsilon, y) - \ell(x, y)}{\epsilon} \quad (18.1)$$

However, there are several reasons why we might not be able to apply this definition:

- We only have access to a sampled version of the input image, $\ell [n, m]$, and we can not compute the limit when ϵ goes to zero.
- The image could contains many nonderivable points and the gradient would not be defined. We will see how to address this issue later when we study Gaussian derivatives.
- In the presence of noise, the image derivative might not be meaningful as it might just be dominated by the noise and not the by image content.

For now, let's focus on the problem of approximating the continuous derivative with discrete operators. As the derivative is a linear operator, it can be approximated by a discrete linear filter. There are several ways in which image derivatives can be approximated.

Let's start with a simple approximation to the derivative operator that we have already played with $d_0 = [1, -1]$. In one dimension (1D), convolving a signal $\ell [n]$ with this filter results in

$$\ell \circ d_0 = \ell [n] - \ell [n-1] \quad (18.2)$$

this approximates the derivative by the difference between consecutive values, which is obtained when $\epsilon = 1$. [Figure 18.3\(c\)](#) shows the result of filtering a 1D signal ([figure 18.3\[a\]](#)) convolved with $d_0 [n]$ ([figure 18.3\[b\]](#)). The output is zero wherever the input signal is constant and it is large in the places where there are variations in the input values. However, note that the output is not perfectly aligned with the input. In fact there is half a sample displacement to the right. This is due to the fact that $d_0 [n]$ is not centered around the origin.

This can be addressed with a different approximation to the spatial derivative $d_1 = [1, 0, -1] / 2$. In one dimension, convolving a signal $\ell [n]$ with $d_1 [n]$ results in

$$\ell \circ d_1 = \frac{\ell [n+1] - \ell [n-1]}{2} \quad (18.3)$$

[Figure 18.3\(e\)](#) shows the result of filtering the 1D signal ([figure 18.3\[a\]](#)) convolved with $d_1 [n]$ ([figure 18.3\[d\]](#)). Now the output shows the highest

magnitude output in the midpoint where there is variation in the input signal.

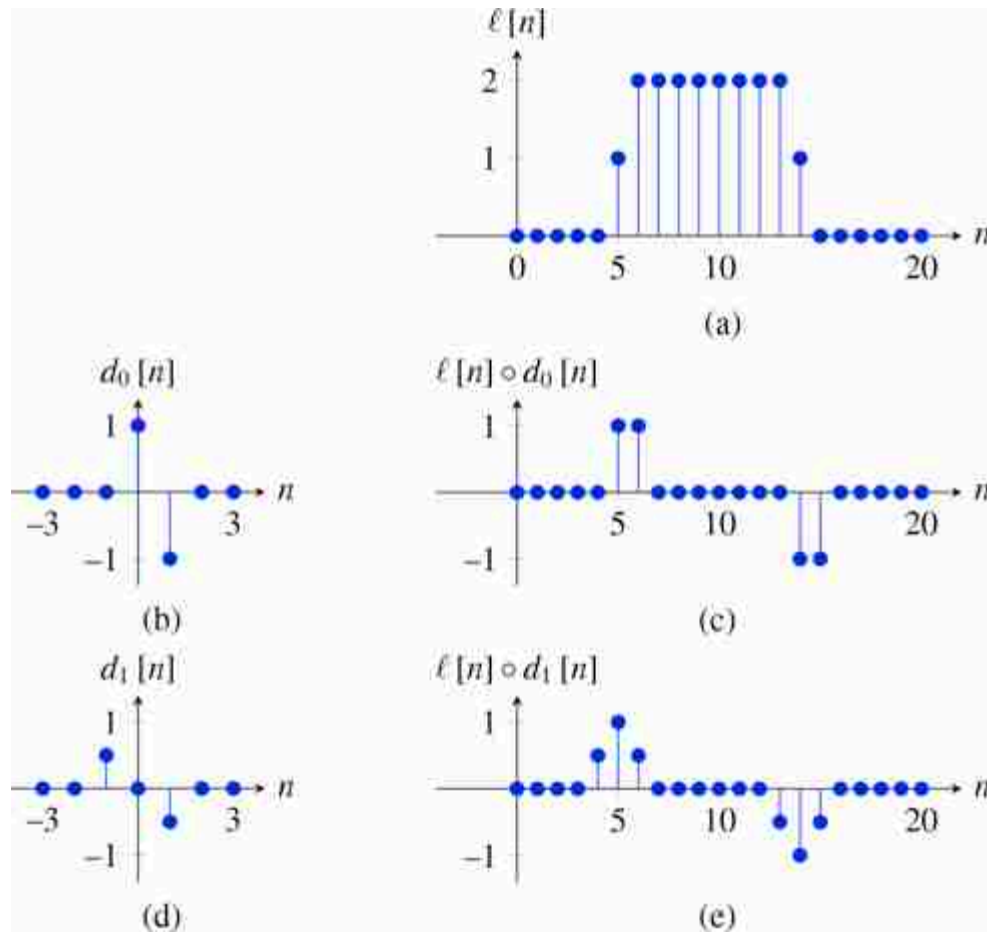


Figure 18.3: (a) Input signal, $\ell[n]$. (b) Convolutional kernel $d_0[n]$, defined as $d_0[0] = 1$ and $d_0[1] = -1$ and zero everywhere else. (c) Output of the convolution between $\ell[n]$ and $d_0[n]$. (d) Kernel $d_1[n]$, defined as $d_1[-1] = 1$ and $d_1[1] = -1$ and zero everywhere else. (e) Output of the convolution between $\ell[n]$ and $d_1[n]$.

It is also interesting to see the behavior of the derivative and its discrete approximations in the Fourier domain. In the continuous domain, the relation ship between the Fourier transform on a function and the Fourier transform of its derivative is

$$\frac{\partial \ell(x)}{\partial x} \xrightarrow{\mathcal{F}} j\omega \mathcal{L}(w) \quad (18.4)$$

In the continuous Fourier domain, derivation is equivalent to multiplying by $j\omega$. In the discrete domain, the DFT of a signal derivative will depend on the approximation used. Let's study now the DFT of the two approximations that we have discussed here, d_0 and d_1 .

The DFT of $d_0[n]$ is

$$\begin{aligned} D_0[u] &= 1 - \exp\left(-2\pi j \frac{u}{N}\right) \\ &= \exp\left(-\pi j \frac{u}{N}\right) \left(\exp\left(\pi j \frac{u}{N}\right) - \exp\left(-\pi j \frac{u}{N}\right)\right) \\ &= \exp\left(-\pi j \frac{u}{N}\right) 2j \sin(\pi u/N) \end{aligned} \quad (18.5)$$

the first term is a pure phase shift and it is responsible of the half a sample delay in the output. The second term is the amplitude gain and it can be approximated by a linear dependency on u for small u values.

The DFT of $d_1[n]$ is

$$\begin{aligned} D_1[u] &= 1/2 \exp\left(2\pi j \frac{u}{N}\right) - 1/2 \exp\left(-2\pi j \frac{u}{N}\right) \\ &= j \sin(2\pi u/N) \end{aligned} \quad (18.6)$$

[Figure 18.4](#) shows the magnitude of $D_0[u]$ and $D_1[u]$ and compares it with $2\pi u/N$, which will be the ideal approximation to the derivative. The amplitude of $D_0[u]$ provides a better approximation to the ideal derivative, but the phase of $D_0[u]$ introduces a small shift in the output. Conversely, $D_1[u]$ has no shift, but it approximates the derivative over a smaller range of frequencies. The output to $D_1[u]$ is smoother than the output to $D_0[u]$, and, in particular, $D_1[u]$ gives a zero output when the input is the signal $[1, -1, 1, -1, \dots]$. In fact, we can see that $[1, 0, -1] = [1, -1] \circ [1, 1]$, and, therefore $D_1[u] = D_0[u] B_1[u]$, where $B_1[u]$ is the DFT of the binomial filter $b_1[n]$.

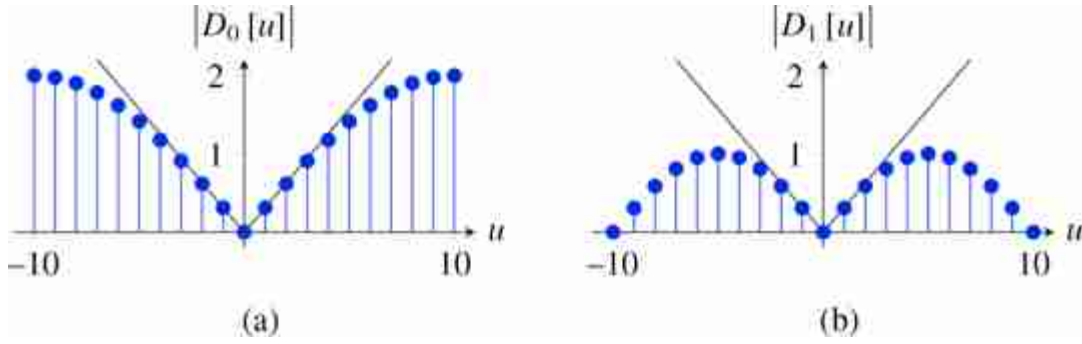


Figure 18.4: Magnitude of (a) $D_0[u]$ and (b) $D_1[u]$ and comparison with $|2\pi u/N|$, shown as a thin black line. Both DFT are computed over 20 samples.

When working with two-dimensional (2D) images there are several ways in which partial image derivatives can be approximated. For instance, we can compute derivatives along the n and m components.

$$\begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad [1 \quad -1] \quad (18.7)$$

The problem with this discretization of the image derivatives is that the outputs are spatially miss-aligned because a filter of length 2 shifts the output image by half a pixel as we discussed earlier. The spatial missalignment can be removed by using a slightly different version of the same operator: we can use a rotated reference frame as it is done in the **Roberts cross** operator, introduced in 1963 [406] in a time when reading an image of 256×256 pixels into memory took several minutes:

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \quad (18.8)$$

Now, both outputs are spatially aligned because they are shifted in the same way. Another discretization is the Sobel operator, based on the $[-1, 0, 1]$ kernel, and will be discussed in section 18.7.

While newer algorithms for edge extraction have been developed since the creation of these operators, they still hold value in cases where efficiency is important. These simple operators continue to be used as fundamental components in modern computer vision descriptors, including the Scale-invariant feature transform (SIFT) [309] and the histogram of oriented gradients (HOG) [93].

18.3 Gradient-Based Image Representation

Derivatives have become an important tool to represent images and they can be used to extract a great deal of information from the image as it was shown in the previous chapter. One thing about derivatives is that it might seem as we are losing information from the input image. An important question is if we have the derivative of a signal, can we recover the original image? What information is being lost? Intuitively, we should be able to recover the input image by integrating its derivative, but it is an interesting exercise to look in detail how this integration can be performed. We will start with a 1D signal and then we will discuss the 2D case.

A simple way of seeing that we can recover the input from its derivatives is to write the derivative in matrix form. This is the matrix that corresponds to the convolution with the kernel $[1, -1]$ that we will call $\mathbf{D}_0$. The next two matrices show the matrix $\mathbf{D}_0$ and its inverse $\mathbf{D}_0^{-1}$ for a 1D image of length five pixels using zero boundary conditions:

$$\mathbf{D}_0 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix} \quad \mathbf{D}_0^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix} \quad (18.9)$$

We can see that the inverse $\mathbf{D}_0^{-1}$ is reconstructing each pixel as a sum of all the derivative values from the left-most pixel to the right. And the inverse perfectly reconstructs the input. But, this is cheating because the first sample of the derivative gets to see the actual value of the input signal and then we can integrate back the entire signal. That matrix is assuming zero boundary conditions for the signal and the boundary gives us the needed constraint to be able to integrate back the input signal.

But what happens if you only get to see valid differences and you remove any pixel that was affected by the boundary? In this case, the derivative operator in matrix form is

$$\mathbf{D}_0 = \begin{bmatrix} -1 & 1 & 0 & 0 & 0 \\ 0 & -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & -1 & 1 \end{bmatrix} \quad (18.10)$$

Let's consider the next 1D input signal:

$$\ell = [1, 1, 2, 2, 0] \quad (18.11)$$

Then, the output of the derivative operator is

$$\mathbf{r} = \mathbf{D}_0 \ell = [0, -1, 0, 2] \quad (18.12)$$

Note that this vector has one sample less than the input. To recover the input ℓ we can not invert $\mathbf{D}_0$ as it is not a square matrix, but we can compute the pseudoinverse which turns out to be

$$\mathbf{D}_0^+ = \frac{1}{5} \begin{bmatrix} -4 & -3 & -2 & -1 \\ 1 & -3 & -2 & -1 \\ 1 & 2 & -2 & -1 \\ 1 & 2 & 3 & -1 \\ 1 & 2 & 3 & 4 \end{bmatrix} \quad (18.13)$$

The pseudoinverse has an interesting structure and it is easy to see how it can be written in the general form for signals of length N . Also note that $\mathbf{D}_0^+$ can not be written as a convolution. Another important thing is that the inversion process is trying to recover more samples than there are observations. The trade off is that the signal that it will recover will have zero mean (so it loses one degree of freedom that can not be estimated). In this example, the reconstructed input is

$$\hat{\ell} = \mathbf{D}_0^+ \mathbf{r} = [-0.2, -0.2, 0.8, 0.8, -1.2] \quad (18.14)$$

Note that $\hat{\ell}$ is a zero mean vector. In fact, the recovered input is a shifted version of the original input, $\hat{\ell} = \ell - 1.2$, where 1.2 is the mean value of samples on ℓ . Then, you still can recover the input signal up to the DC component (mean signal value).

18.4 Image Editing in the Gradient Domain

One of the interests of transforming an image into a different representation than pixels is that it allows us to manipulate aspects of the image that would be hard to control using the pixels directly. [Figure 18.5](#) shows an example of image editing using derivatives.

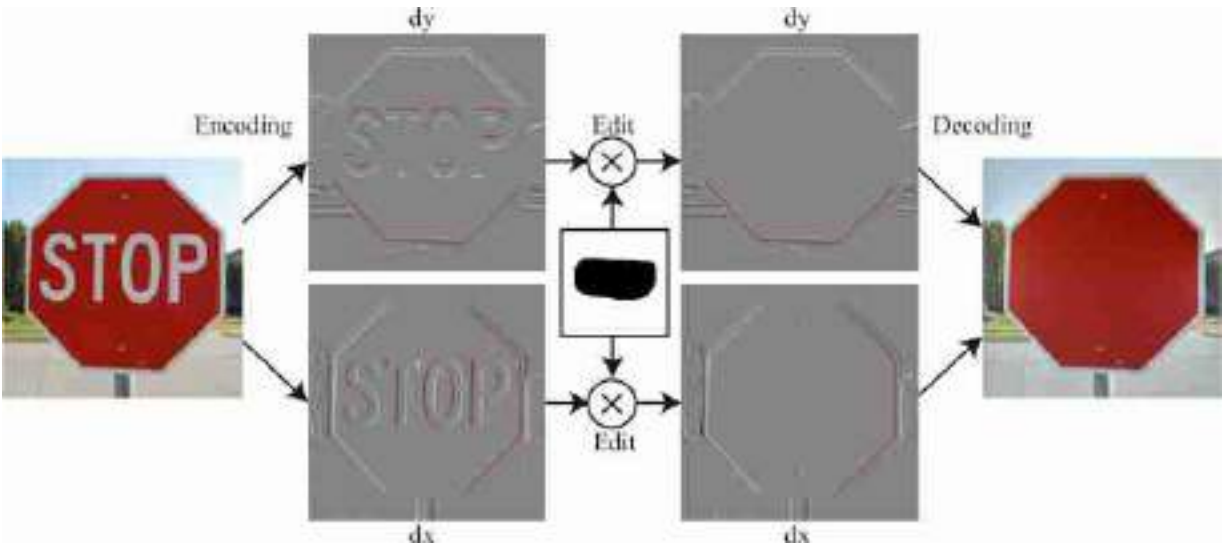


Figure 18.5: Image inpainting: Using image derivatives we delete the word “stop” by setting to zero the gradients indicated by the mask. The resulting decoded image propagates the red color inside the region that contained the word.

First, the image is **encoded** using derivatives along the x and y dimensions:

$$\mathbf{r} = \begin{bmatrix} \mathbf{D}_x \\ \mathbf{D}_y \end{bmatrix} \ell \quad (18.15)$$

The resulting representation, $\mathbf{r}$, contains the concatenation of the output of both derivative operators. $\mathbf{r}$ will have high values in the image regions that contains changes in pixel intensities and colors and will be near zero in regions with small variations. This representation can be **decoded** back into the input image by using the pseudoinverse as we did in the 1D case. The pseudoinverse can be efficiently computed in the Fourier domain [498].

We can now manipulate this representation. In the example shown in [figure 18.5](#), if we set to zero the image derivatives that correspond to the word “stop”, when we decode the representation we obtain a new image where the word “stop” has been removed. The red color fills⁷ the region with the word is present in the input image. We did not have to specify what color that region had to be, that information comes from the integration process. Therefore, deleting an object using image derivatives can be achieved by setting to zero the gradients regardless of the color of the background. Unfortunately, things get more complex when the background has texture and this simple technique will fail.

18.5 Gaussian Derivatives

In the previous sections we studied how to discretize derivatives and how to use them to represent images. However, computing derivatives in practice presents several difficulties. First, derivatives are sensitive to noise. In the presence of noise, as images tend to vary slowly, the difference between two continuous pixel values will be dominated by noise. This is illustrated in [figure 18.6](#).



Figure 18.6: Derivatives of a noisy image. (a) Input image with noise, (b) its x -derivative obtained by convolving with a kernel $[1, -1]$, and (c) its x -derivative obtained using a gaussian derivative kernel.

As shown in [figure 18.6](#), the input is an image corrupted with Gaussian noise. The result of convolving the input noisy image with the kernel $[1, -1]$ results in an output with an increased noise. This is because the noise at each pixel is independent of each other, so when computing the difference between two adjacent pixels we get increased noise (the difference between two Gaussian random variables with variance σ^2 is a new Gaussian with variance $2\sigma^2$). On the other hand, the values of the pixels of the original image are very similar and the difference is a small number (except at the object boundaries). As a consequence, the output is dominated by noise as shown in [figure 18.6\(b\)](#).

There are also situations in which the derivative of an image is not defined. For instance, consider an image in the continuous domain with the form $\ell(x, y) = 0$ if $x < 0$ and 1 otherwise. If we try to compute $\partial\ell(x, y)/\partial x$ we

will get 0 everywhere, but around $x = 0$ the value of the derivative is not defined. We avoided this issue in the previous section because for discrete images the approximation of the derivative is always defined.

Gaussian derivatives address these two issues. They were popularized by Koendering and Van Doorn [271] as a model of neurons in the visual system.

Let's start with the following observation. For two functions defined in the continuous domain $\ell(x, y)$ and $g(x, y)$, the convolution and the derivative are commutative:

$$\frac{\partial \ell(x, y)}{\partial x} \circ g(x, y) = \ell(x, y) \circ \frac{\partial g(x, y)}{\partial x} \quad (18.16)$$

This equality is easy to prove in the Fourier domain. If our goal is to compute image derivatives and then blur the output using a differentiable low-pass filter, $g(x, y)$, then instead of computing the derivative of the image we can compute the derivatives of the filter kernel and convolve it with the image. Even if the derivative of $\ell(x, y)$ is not defined in some locations (e.g., step boundaries), we can always compute the result of this convolution.

If $g(x, y)$ is a blurring kernel it will smooth the derivatives, reducing the output noise at the expense of a loss in spatial resolution ([figure 18.6\[c\]](#)). A common smoothing kernel for computing derivatives is the Gaussian kernel. The Gaussian has nice smoothing properties as we discussed in the previous chapter, and it is infinitely differentiable.

If g is a Gaussian, then the first order derivative is

$$g_x(x, y; \sigma) = \frac{\partial g(x, y; \sigma)}{\partial x} = \frac{-x}{2\pi\sigma^4} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) = \frac{-x}{\sigma^2} g(x, y; \sigma) \quad (18.17)$$

The next plots show the 2D Gaussian with $\sigma = 1$, and its x -derivative:

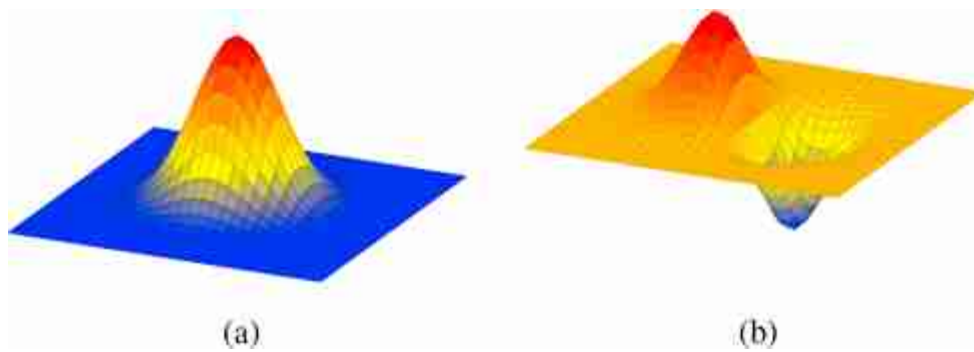


Figure 18.7: (a) 2D Gaussian with $\sigma = 1$, and (b) its x -derivative.

[Figure 18.8](#) shows an image filtered with three Gaussian derivatives with different widths, σ . Derivatives at different scales emphasize different aspects of the image. The fine-scale derivatives ([figure 18.8\[a\]](#)) highlight the bands in the zebra, while the coarse-scale derivatives ([figure 18.8\[c\]](#)) emphasize more the object boundaries. This multiscale image analysis will be studied in depth in the following chapter.

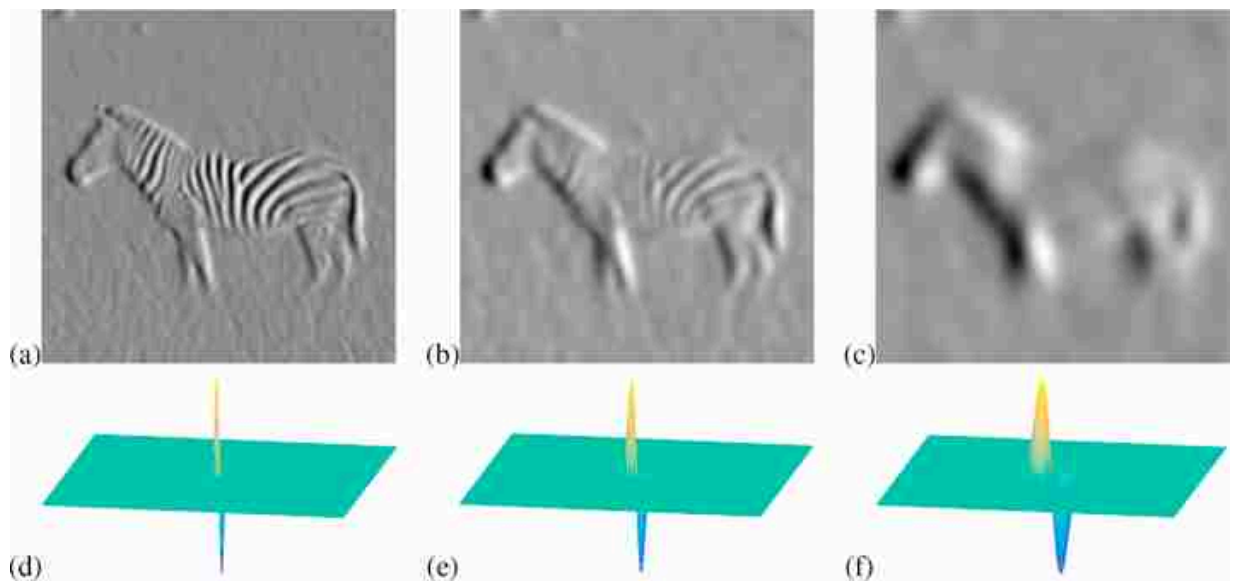


Figure 18.8: An image filtered with three Gaussian derivatives: (a) $\sigma = 2$; (b) $\sigma = 4$; and (c) $\sigma = 8$. Plots (d-f) show the three Gaussian derivatives over the same spatial support as the image. The discrete functions are approximated by sampling the continuous Gaussian derivatives. The convolutions are performed with mirror boundary extension.

18.6 High-Order Gaussian Derivatives

The second order derivative of a Gaussian is

$$g_{x^2}(x, y; \sigma) = \frac{x^2 - \sigma^2}{\sigma^4} g(x, y; \sigma) \quad (18.18)$$

The n -th order derivative of a Gaussian can be written as the product between a polynomial on x , with the same order as the derivative, times a Gaussian. The family of polynomials that result on computing Gaussian derivatives is called Hermite polynomials. The general expression for the n derivative of a Gaussian is

$$g_{x^n}(x; \sigma) = \frac{\partial^n g(x)}{\partial x^n} = \left(\frac{-1}{\sigma\sqrt{2}} \right)^n H_n \left(\frac{x}{\sigma\sqrt{2}} \right) g(x; \sigma) \quad (18.19)$$

The first Hermite polynomial is $H_0(x) = 1$. For $n = 0$ we have the original Gaussian. [Figure 18.9](#) shows the 1D Gaussian derivatives.

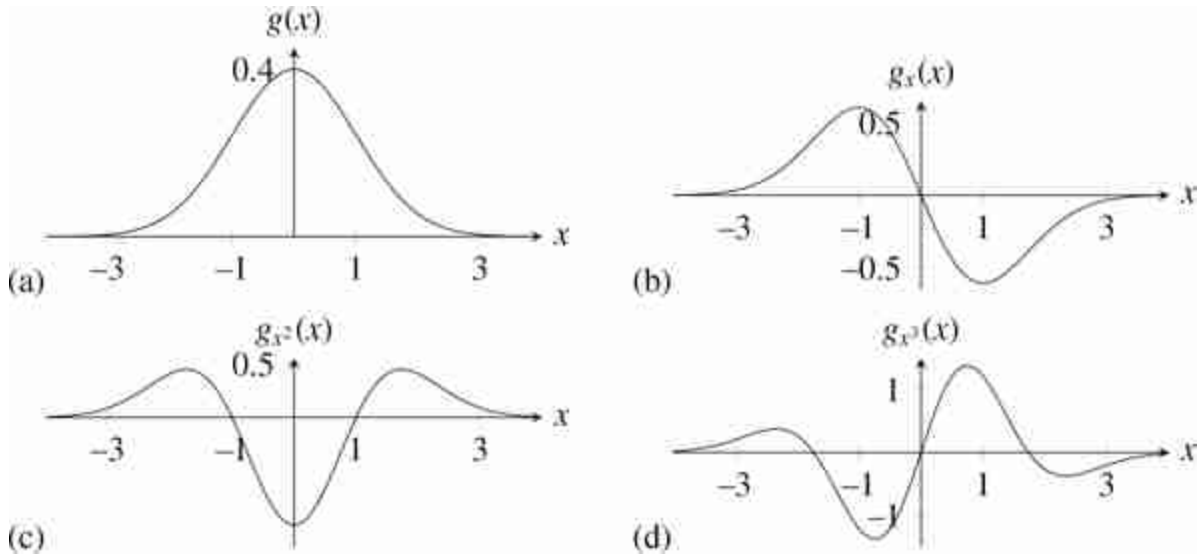


Figure 18.9: (a) 1D Gaussian with $\sigma = 1$. (b–d) Gaussian derivatives up to order 3.

In two dimensions, as the Gaussian is separable, the partial derivatives result on the product of two Hermite polynomial, one for each spatial dimension:

$$g_{x^n y^m}(x, y; \sigma) = \frac{\partial^{n+m} g(x, y)}{\partial x^n \partial y^m} = \left(\frac{-1}{\sigma\sqrt{2}} \right)^{n+m} H_n \left(\frac{x}{\sigma\sqrt{2}} \right) H_m \left(\frac{y}{\sigma\sqrt{2}} \right) g(x, y; \sigma) \quad (18.20)$$

[Figure 18.10](#) shows the 2D Gaussian derivatives, and [figure 18.11](#) shows the corresponding Fourier transforms. [Figure 18.12](#) shows the output of the

derivatives when applied to a simple input image containing a square and a circle. Different derivatives detect a diverse set of image features. However, this representation might not be very useful as it is not rotationally invariant.

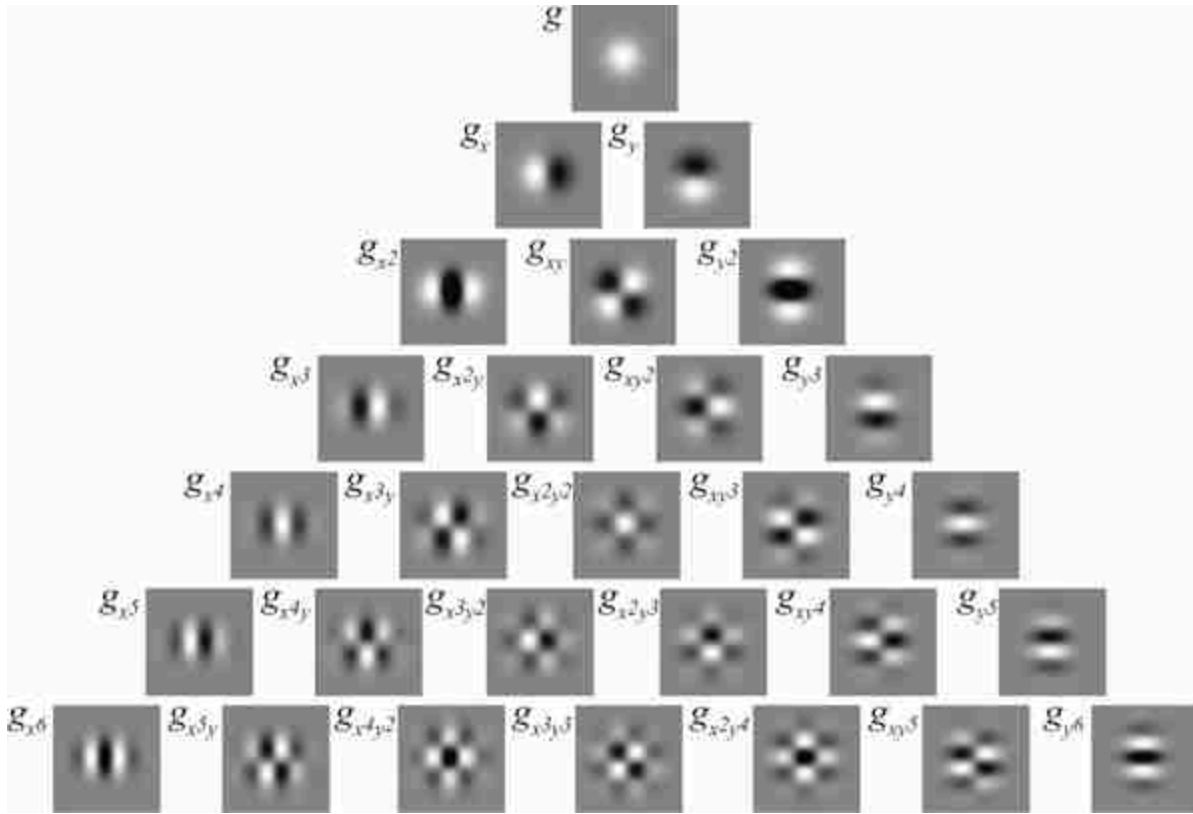


Figure 18.10: Gaussian derivatives up to order 6. All the kernels are separable. They seem similar to Fourier basis multiplied with a Gaussian window. [Figure 18.11](#) shows that they are different to sine and cosine waves, instead they look more like products of cosine and sine waves.

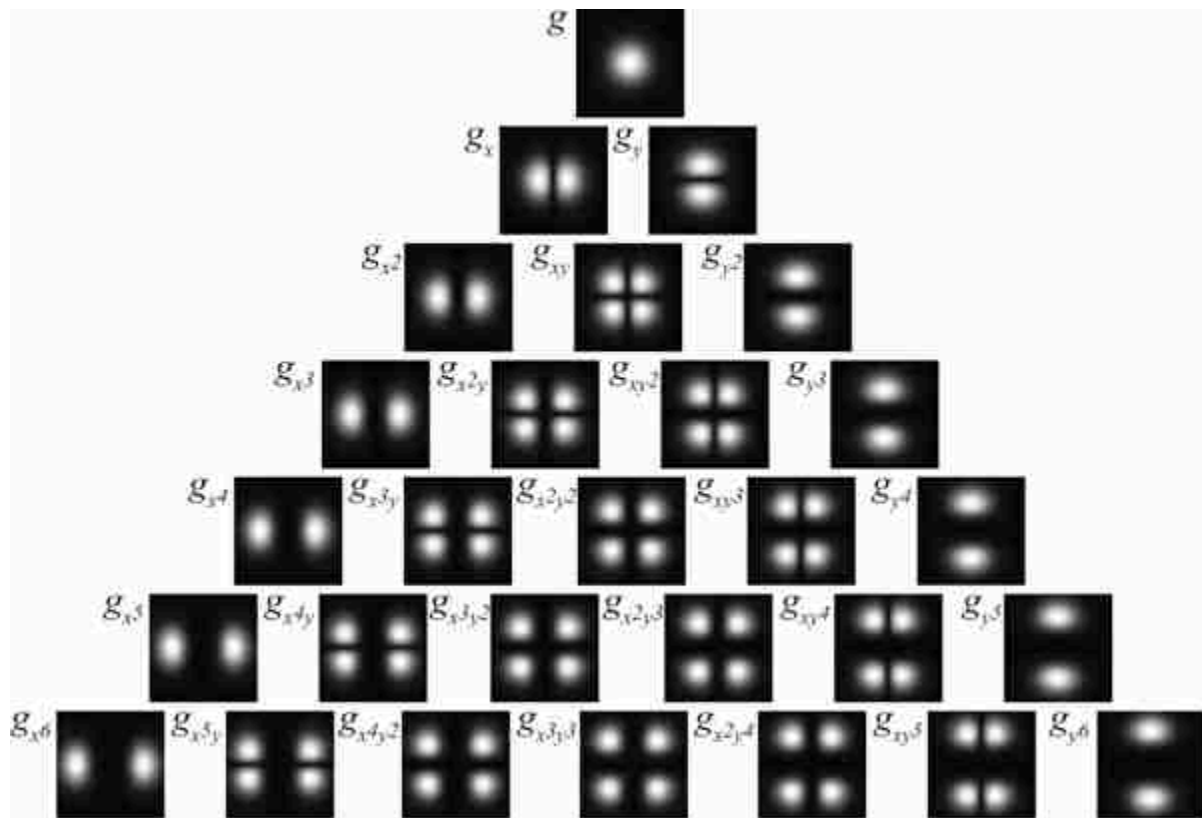


Figure 18.11: Fourier transform of the Gaussian derivatives shown in [figure 18.10](#).

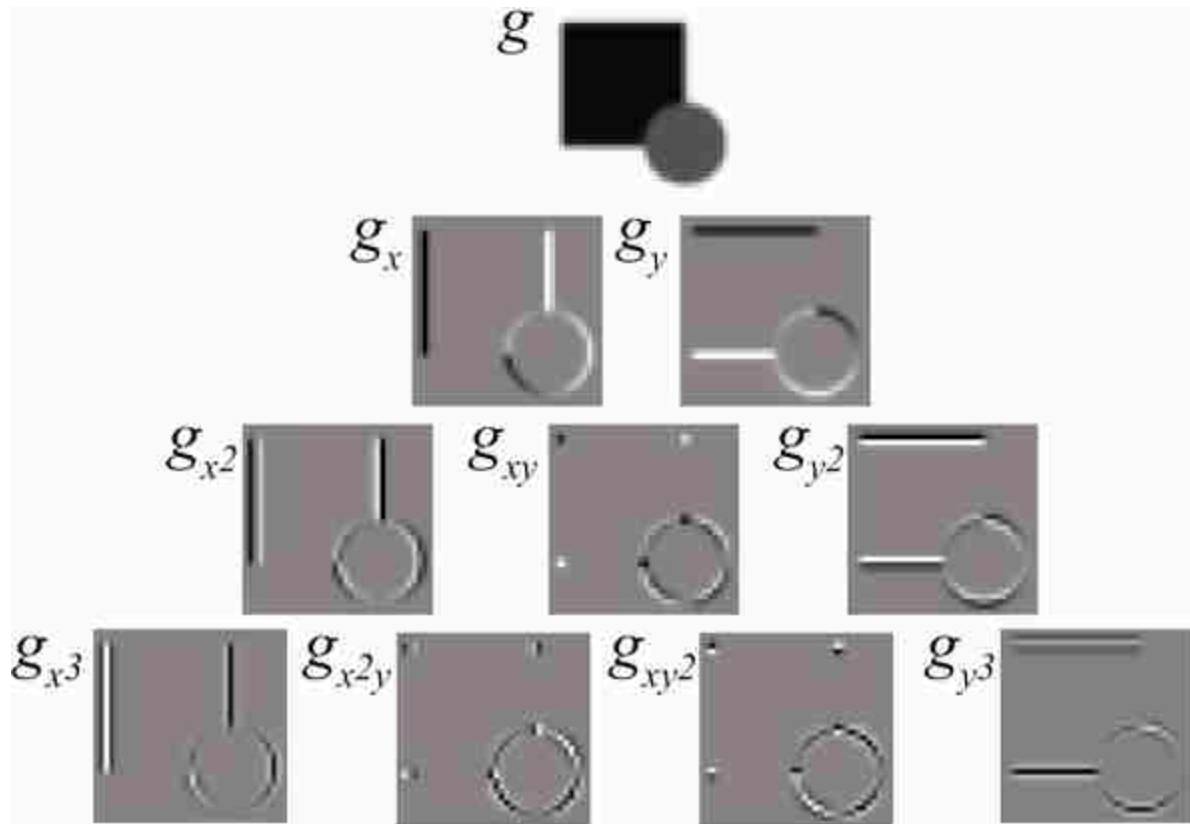


Figure 18.12: An image containing a square and a circle and its output to the Gaussian derivatives up to order 3.

The Gaussian derivatives share many of the properties of the Gaussian. The convolution of two Gaussian derivatives of order n and m and variances σ_1^2 and σ_2^2 result in another Gaussian derivative of order $n + m$ and variance $\sigma_1^2 + \sigma_2^2$. Proving this property on the spatial domain can be tedious. However, it is trivial to prove it in the Fourier domain.

18.7 Derivatives of Binomial Filters

When processing images we have to use discrete approximations for the Gaussian derivatives. After discretization, many of the properties of the continuous Gaussian will not hold exactly.

There are many discrete approximations. For instance, we can take samples of the continuous functions. In practice it is common to use the discrete approximation given by the binomial filters. [Figure 18.13](#) shows the result of convolving the binomial coefficients, b_n , with $[1, -1]$.

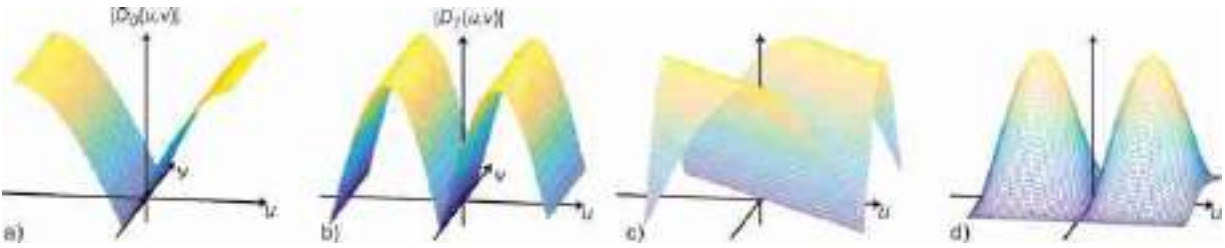


Figure 18.14: Magnitude of the DFT of four different discretization of Gaussian derivatives: (a) d_0 ; (b) d_1 ; (c) Robert cross operator; and (d) Sobel-Feldman operator.

[Figure 18.14](#) compares the DFT of the four types of approximations of the derivatives that we have discussed. These operators are still very popular. *Sobel* has the best tolerance to noise due to its band-pass nature. The kernel d_0 is the one that provides the highest resolution in the output. [Figure 18.15](#) shows the output of different derivative approximations to a simple input image containing a circle. In the next section we will discuss how to use these derivatives to extract other interesting quantities.

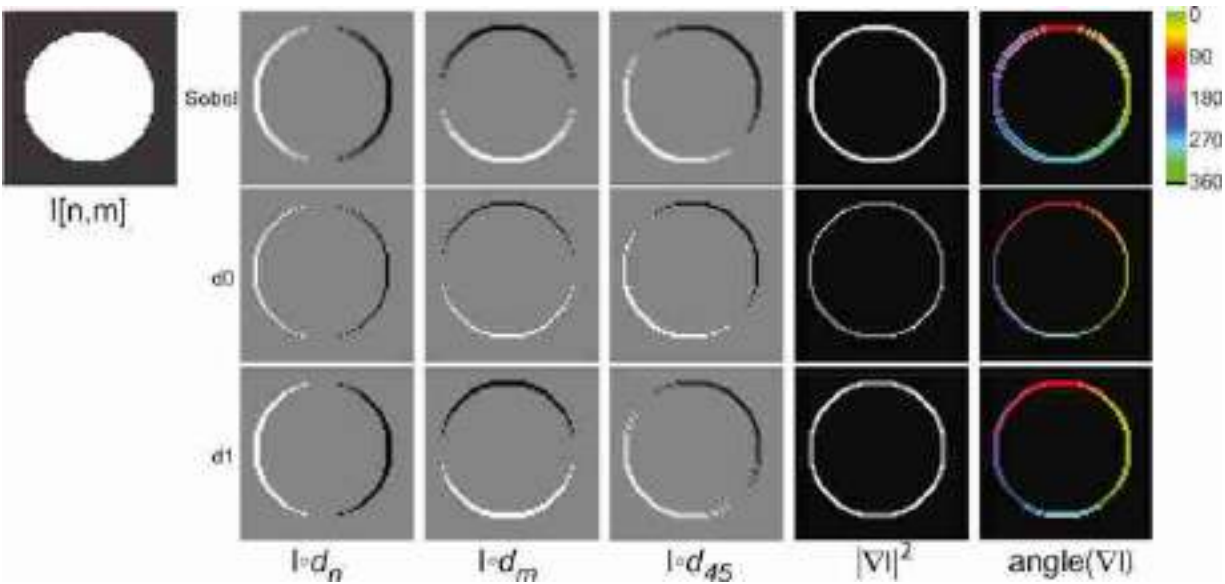


Figure 18.15: Derivatives of a circle along the directions n , m , and 45 degrees. The angle is shown only where the magnitude is > 0 . The derivative output along 45 degrees is obtained as a linear combination of the derivatives outputs along n and m . Check the differences among the different kernels. The Sobel operator gives the most rotationally invariant gradient magnitude, but it is blurrier.

18.8 Image Gradient and Directional

Derivatives

As we saw in chapter 2, an important image representation is given by the image gradient. From the image derivatives we can define also the image gradient as the vector:

$$\nabla \ell(x, y) = \left(\frac{\partial \ell(x, y)}{\partial x}, \frac{\partial \ell(x, y)}{\partial y} \right) \quad (18.24)$$

For each pixel, the output is a 2D vector. In the case of using Gaussian derivatives, we can write:

$$\nabla \ell \circ g = \nabla g \circ \ell = (g_x(x, y), g_y(x, y)) \circ \ell \quad (18.25)$$

Although we have mostly computed derivatives along the x and y variables, we can obtain the derivative on any orientation as a linear combination of the two derivatives along the main axes. With $\mathbf{t} = (\cos(\theta), \sin(\theta))$, we can write the directional derivative along the vector $\mathbf{t}$ as:

$$\frac{\partial \ell(x, y)}{\partial \mathbf{t}} = \nabla \ell \cdot \mathbf{t} = \cos(\theta) \frac{\partial \ell}{\partial x} + \sin(\theta) \frac{\partial \ell}{\partial y} \quad (18.26)$$

In the Gaussian case:

$$\frac{\partial \ell}{\partial \mathbf{t}} \circ g = (\cos(\theta) g_x(x, y) + \sin(\theta) g_y(x, y)) \circ \ell = (\nabla g \cdot \mathbf{t}) \circ \ell = g_\theta(x, y) \circ \ell \quad (18.27)$$

with $g_\theta(x, y) = \cos(\theta) g_x(x, y) + \sin(\theta) g_y(x, y)$. However, to compute the derivative along any arbitrary angle θ does not require doing new convolutions. Instead, we can compute any derivative as a linear combination of the output of convolving the image with $g_x(x, y)$ and $g_y(x, y)$:

$$\frac{\partial \ell}{\partial \mathbf{t}} \circ g = \cos(\theta) g_x(x, y) \circ \ell + \sin(\theta) g_y(x, y) \circ \ell \quad (18.28)$$

When using discrete convolutional kernels $d_n[n, m]$ and $d_m[n, m]$ to approximate the derivatives along n and m , it can be written as

$$\nabla \ell = (d_n[n, m], d_m[n, m]) \circ \ell[n, m] \quad (18.29)$$

and

$$\nabla \ell \cdot \mathbf{t} = d_\theta[n, m] \circ \ell[n, m] \quad (18.30)$$

with $d_\theta[n, m] = \cos(\theta) d_n[n, m] + \sin(\theta) d_m[n, m]$. We expect that the linear combination of these two kernels should approximate the derivative along the direction θ . The quality of this approximation will vary for the different kernels we have seen in the previous sections.

18.9 Image Laplacian

The **Laplacian filter** was made popular by Marr and Hildreth in 1980 [319] in the search for operators that locate the boundaries between objects. The Laplacian is a common operator from differential geometry to measure the divergence of the gradient and it appears frequently in modeling fields in physics. The Laplacian also has applications in graph theory and in spectral methods for image segmentation [354].

One example of application of the Laplacian is in the paper *Can One Hear the Shape of a Drum* [242] where the Laplacian is used for modeling vibrations in a drum and the sounds it produces as a function of its shape.

The Laplacian operator is defined as the sum of the second order partial derivatives of a function:

$$\nabla^2 \ell = \frac{\partial^2 \ell}{\partial x^2} + \frac{\partial^2 \ell}{\partial y^2} \quad (18.31)$$

The Laplacian is more sensitive to noise than the first order derivative. Therefore, in the presence of noise, when computing the Laplacian operator on an image $\ell(x, y)$, it is useful to smooth the output with a Gaussian kernel, $g(x, y)$. We can write,

$$\nabla^2 \ell \circ g = \nabla^2 g \circ \ell \quad (18.32)$$

Therefore, the same result can be obtained if we first compute the Laplacian of the Gaussian kernel, $g(x, y)$ and then convolve it with the input image. The Laplacian of the Gaussian is

$$\nabla^2 g = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} g(x, y) \quad (18.33)$$

The next plot ([figure 18.16](#)) shows the Gaussian Laplacian ($\sigma = 1$). Due to its shape, the Laplacian is also called the inverted **mexican hat wavelet**. Despite that visually it might seem as if the Laplacian has a negative mean value, the mean is actually zero as the positive side is wider than the negative.

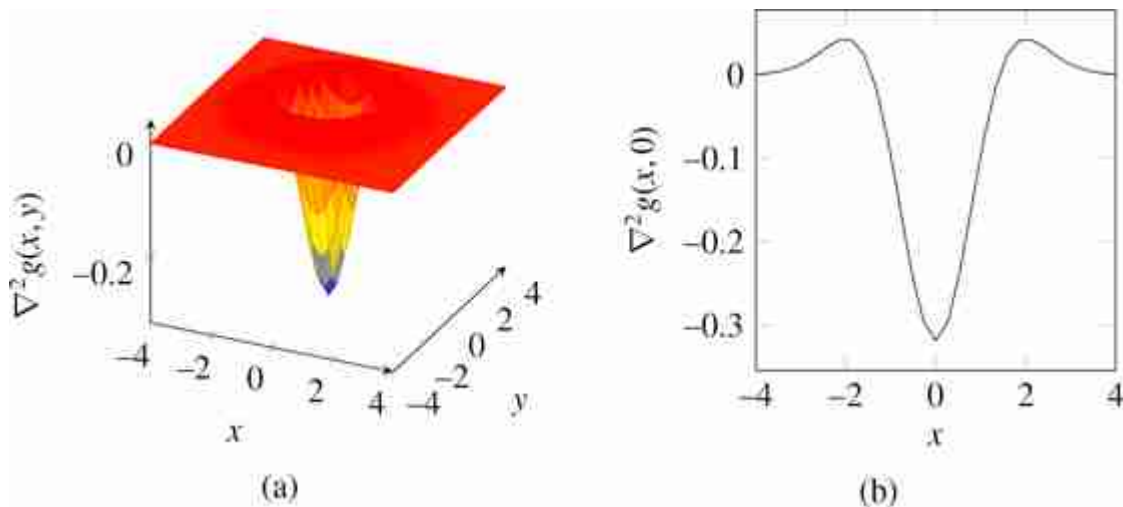


Figure 18.16: The Gaussian Laplacian ($\sigma = 1$) is also called the inverted **mexican hat wavelet**. (a) 2D plot. (b) 1D section at $y = 0$.

In the discrete domain there are several approximations to the Laplacian filter. The simplest one consists in sampling the continuous Gaussian Laplacian in discrete locations. [Figures 18.17\(a-c\)](#) shows the DFT of the Gaussian Laplacian for three different values of σ . For values $\sigma > 1$ the resulting filter is band-pass, which is the expected behavior for the Gaussian Laplacian.

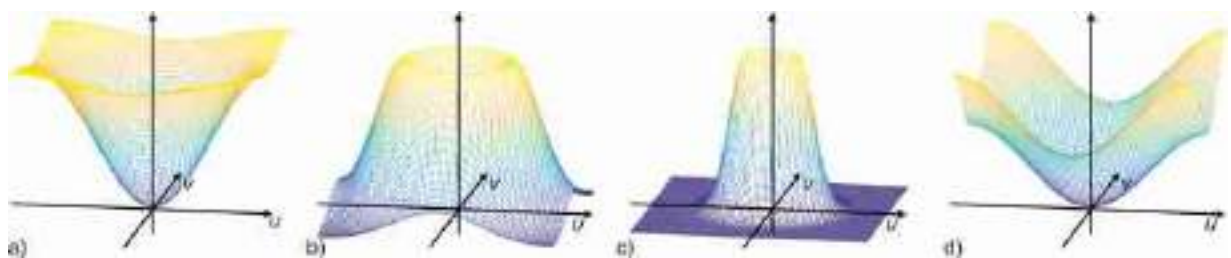


Figure 18.17: Magnitude of the DFT of the Gaussian Laplacian with (a) $\sigma = 1/2$; (b) $\sigma = 1$; (c) $\sigma = 2$; and (d) DFT of the five-point discrete approximation, equation (18.34).

In one dimension, the Laplacian can be approximated by $[1, -2, 1]$, which is the result of the convolution of two 2-tap discrete approximations of the derivative $[1, -1] \circ [1, -1]$. In two dimensions, the most popular approximation is the five-point formula, which consists in convolving the image with the kernel:

$$\nabla_5^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad (18.34)$$

It involves the central pixel and its four nearest neighbors. This is a sum-separable kernel: it corresponds to approximating the second-order derivative for each coordinate and summing the result (i.e., convolve the image with $[1, -2, 1]$ and also with its transpose and summing the two outputs). [Figure 18.17\(d\)](#) shows the DFT of this approximation. It also has a quadratic form near the origin of the frequency domain, but it has a high-pass shape similar to the one obtained for a small value of σ .

The Laplacian of the image, using the five-point formula, is:

$$\nabla_5^2 \ell[n, m] = -4\ell[n, m] + \ell[n+1, m] + \ell[n-1, m] + \ell[n, m+1] + \ell[n, m-1]$$

[Figure 18.18](#) compares the output of image derivatives and the Laplacian. Note that when summing two first-order derivatives along x and y , what we obtain is a first-order derivative along the 45 degree angle. However, when summing up the two second-order derivatives we obtain a rotationally invariant kernel, as shown in [figure 18.18\(d\)](#).

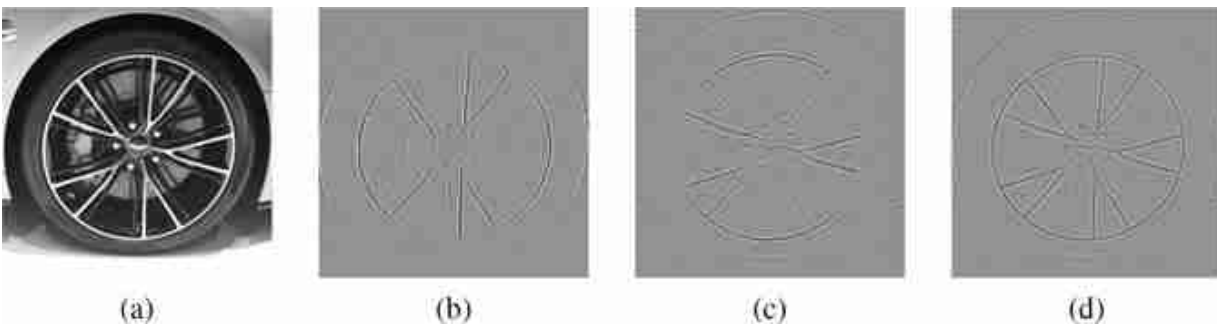


Figure 18.18: (a) Input image. (b) Second order derivative along x . (c) Second-order derivative along y . (d) The sum of (b) + (c), which gives the Laplacian.

The discrete approximation also provides a better intuition of how it works than its continuous counterpart. The Laplacian filter has a number of advantages with respect to the gradient:

- It is rotationally invariant. It is a linear operator that responds equally to edges in any orientation (this is only approximate in the discrete case).

- It measures curvature. If the image contains a linear trend the derivative will be non-zero despite having no boundaries, while the Laplacian will be zero.
- Edges can be located as the zero-crossings in the Laplacian output. However, this way of detecting edges is not very reliable.
- Zero crossings of an image form closed contours.

[Figure 18.19](#) compares the output of the first order derivative ([figure 18.19\[b\]](#)) and the Laplacian ([figure 18.19\[d\]](#)) on a simple 1D signal ([figure 18.19\[a\]](#)). The local maximum of the derivative output ([figure 18.19\[c\]](#)) and the zero crossings of the Laplacian output ([figure 18.19\[e\]](#)) are aligned with the transitions (boundaries) of the input signal ([figure 18.19\[a\]](#)).

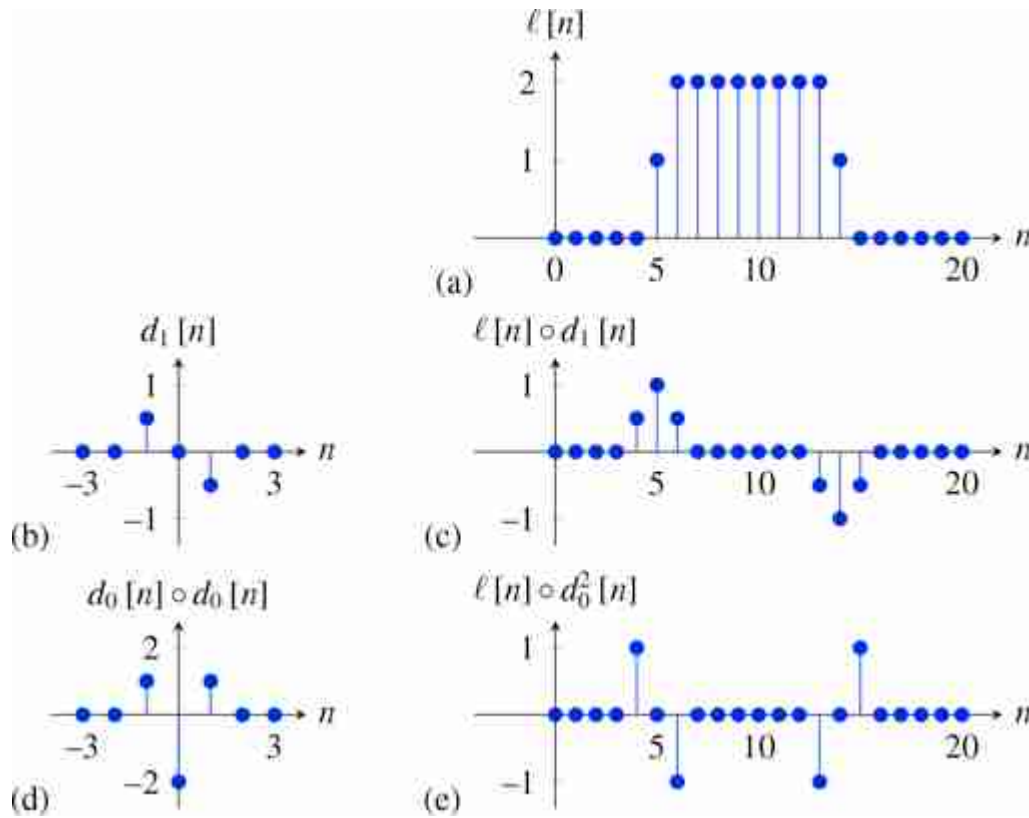


Figure 18.19: Comparison between the output of a first-order derivative and the Laplacian of 1D signal. (a) Input signal. (b) Kernel d_1 . (c) Output of the derivative, that is, convolution of (a) and (b). (d) Discrete approximation of the Laplacian. (e) Output of convolving the signal (a) with the Laplacian kernel (d).

Marr and Hildreth [319] used zero-crossings of the Laplacian output to compute edges, but this method is not used nowadays for edge detection. Instead the Laplacian filter is widely used in a number of other image representations. It is used to build image pyramids (multiscale representations, chapter 23), and to detect points of interest in images (it is the basic operator used to detect keypoints to compute SIFT descriptors [309]).

18.10 A Simple Model of the Early Visual System

The Laplacian filter can also be used as a coarse approximation of the behavior of the **early visual system**. When looking at the magnitude of the

DFT of the Laplacian with $\sigma = 1$ ([figure 18.17](#)), the shape seems reminiscent of our subjective evaluation of our own visual sensitivity to spatial frequencies when we look at the Campbell and Robson chart ([figure 16.21](#)). As the visual filter does not seem to cancel exactly the very low spatial frequencies as the Laplacian does, a better approximation is

$$h = -\nabla^2 g + \lambda g \quad (18.35)$$

where the negative sign in front of the Laplacian helps to fit our perception better. The kernel h is the approximate impulse response of the human visual system, λ is a small constant that is equal to the DC gain of the visual filter (here we have set up $\lambda = 2$ and $\sigma = 5$). This results in the profile shown in [figure 18.20](#).

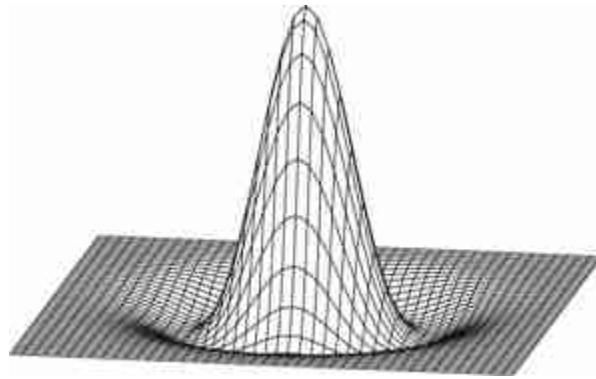


Figure 18.20: Kernel corresponding to equation (18.35) with $\lambda = 2$ and $\sigma = 5$.

The first thing worth pointing out is that the Fourier transform of the impulse response shown in [figure 18.20](#) has a shape that is qualitatively similar to the shape of the human contrast sensitivity function as discussed in chapter 16. Here we show again the Campbell and Robson chart, together with a radial section of the Fourier transform of h from equation (18.35):

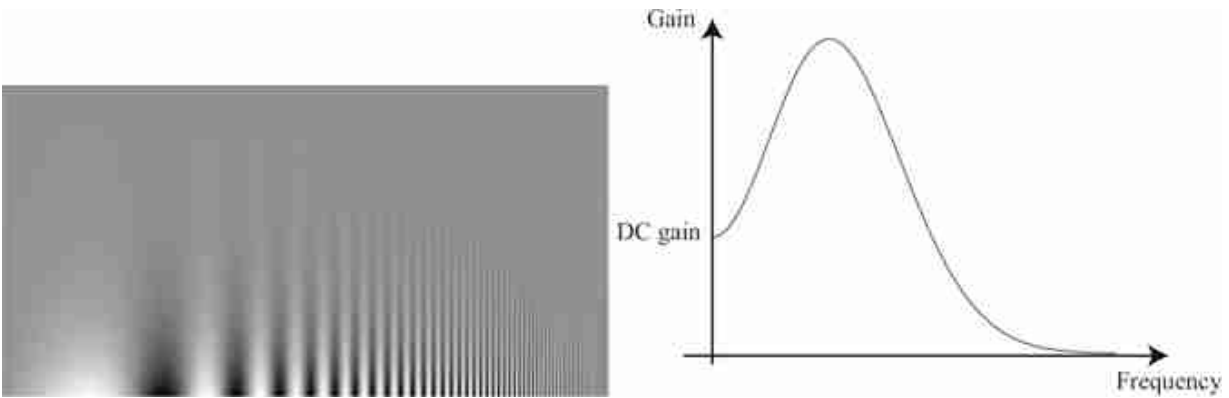


Figure 18.21: Campbell and Robson chart, and a radial section of the Fourier transform of h from equation (eq:humanmodel).

This particular form of impulse response explains some visual illusions as shown in [figure 18.22](#). This visual illusion is called the **Vasarely visual illusion**. [Figures 18.22\(a\)](#) and [18.22\(d\)](#) show two grayscale pyramids formed by superimposing squares of increasing (or decreasing) intensity. When looking at them we perceive the diagonal directions as being brighter (a) or darker (d) than their neighborhood. This is an illusion because there is not such a difference in image intensity. What mechanism is responsible of this illusion?

One explanation consists in saying that the image that we perceive is the output of a filter (as shown in equation (18.35)). [Figures 18.22\(b\)](#) and [18.22\(e\)](#) show the output of such a filter. We can see the diagonals again as being brighter or darker but now this effect is not an illusion, the brighter and darker diagonals are really part of the filtered image. In fact, [figure 18.22\(c\)](#) shows in blue a horizontal section at one quarter of [figure 18.22\(b\)](#). The red curve is the section of the input image in [figure 18.22\(a\)](#). We can see that the output really contains a pick in intensity on the diagonals of [figure 18.22\(b\)](#). The plot in [figure 18.22\(f\)](#) shows the same for [figures 18.22 \(d and e\)](#). For the results shown in the [figure 18.22](#) we have set $\lambda = 2$ and $\sigma = 5$. The result is probably overly smooth and a more appropriate value might be around $\sigma = 2$.

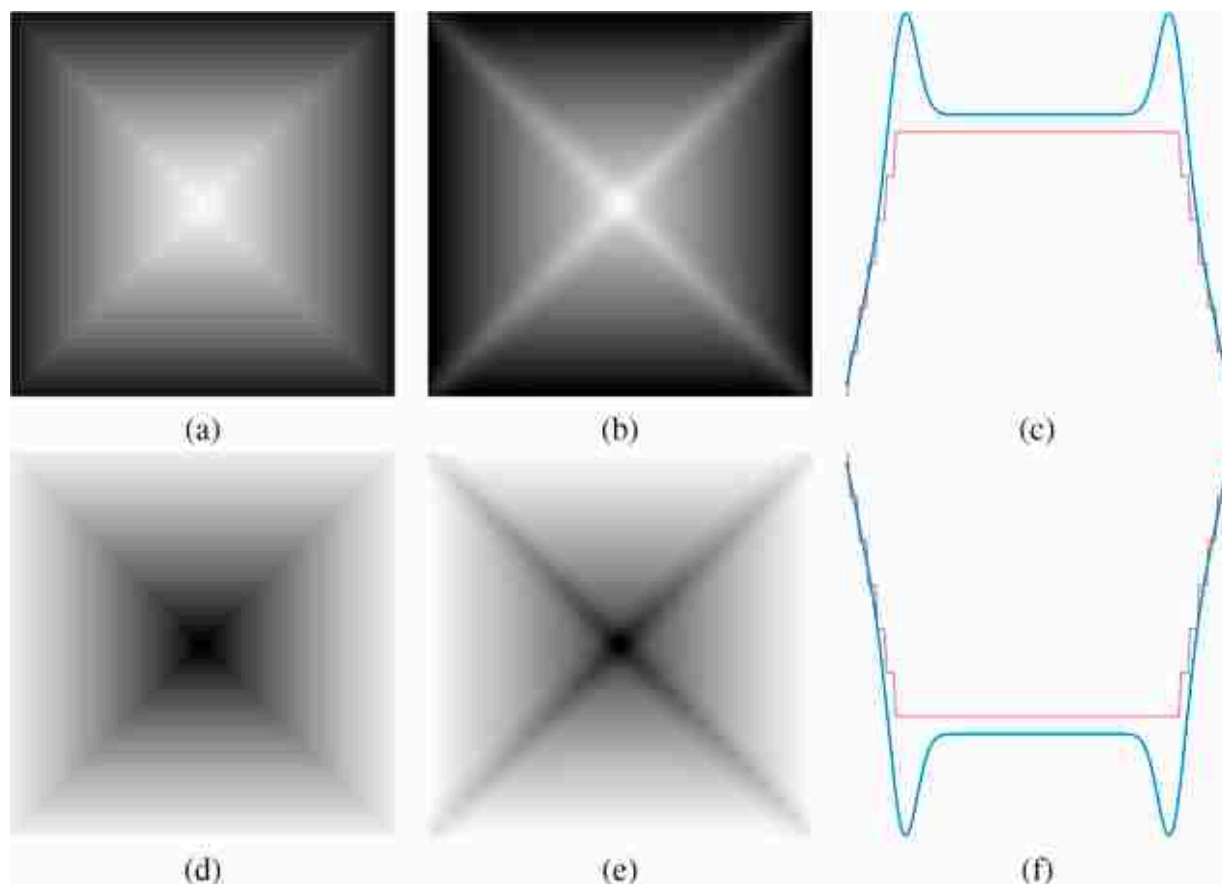


Figure 18.22: Vasarely visual illusion. Images (a) and (d), formed by nested-squares, appear as having bright diagonals in (a) and dark in (d). Images (b) and (e) show the output of the human model given by the filter from equation (18.35). Plot (c) displays the intensity profiles of images (a) and (b) as horizontal sections, with image (a) represented in red and image (b) in blue. Similarly, Plot (f) represents the intensity profiles of images (d) and (e).

18.11 Sharpening Filter

One example of a simple but very useful filter is a **sharpening filter**. The goal of a sharpening filter is to transform an image so that it appears sharper (i.e., it contains more fine details). This can be achieved by amplifying the amplitude of the high-spatial frequency content of the image. We can achieve this with a combination of filters that we have already discussed in this section.

A simple way to design a sharpening filter is to de-emphasize the blurry components of an image. By the linearity of the convolution operator, we're allowed to add and subtract kernels to make a new kernel that would give us the same filtered image as if we had added and subtracted the filtered

outputs of each of the component kernels. For this example, we start with twice the original image (sharp plus blurred parts), then subtract away the blurred components of the image:

$$\text{sharpening filter} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad (18.36)$$

Note that the DC gain of this sharpening filter is 1. That would leave one original image in there, plus an additional component of the sharp details. The perceptual result is that of a sharpened image ([figure 18.23](#)). We can apply this filter successively in order to further enhance the image details. If too much sharpening is applied we might end up enhancing noise and introducing image artifacts.

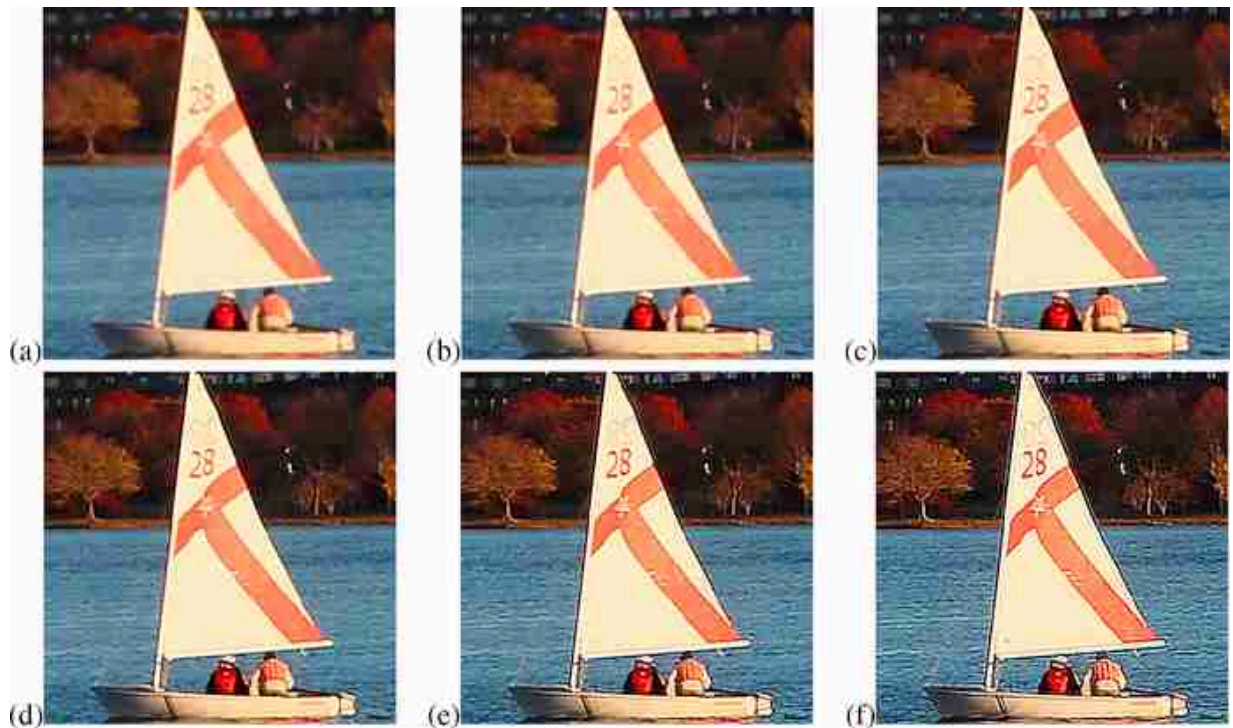


Figure 18.23: Sharpening achieved by subtraction of blurred components. (a) Original image. (b) Sharpened once by filtering with kernel from equation (18.36). Each color channel is filtered independently. (c-f) The same filter is applied successively to the previous output. In the last image the sharpening filter has been applied five times to the input image. The last image looks substantially sharper than the original image, but close inspection will reveal some artifacts.

Note that the sharpening filter described here does not introduce new fine image details, it only enhances the ones already present in the image.

It is also interesting to note that this sharpening filter is very similar to the model of the early visual system that we described before.

18.12 Retinex

How do you tell gray from white? This might seem like a simple question, but one remarkable aspect of vision is that perception of the simplest quantity might be far from trivial. As a final example to illustrate the power of using image derivatives as an image representation, we will discuss here a partial answer to this question.

To understand that answering this question is not trivial, let's consider the structure of light that reaches the eye, which is the input upon which our visual system will try to differentiate black from white. The amount of light

that reaches the eye from painted piece of paper is the result of two quantities: the amount of light reaching the piece of paper, and the reflectance of the surface (what fraction of the light is reflected back into space and what fraction is absorbed by the material).

If two patches receive the same amount of light, then we will perceive as being darker the patch that reflects less light. But what happens if we change the amount of light that reaches the scene? If we increase the amount of light projected on top of the dark patch, what will be seen now? What will happen if we have two patches on the same scene, each of them receiving different amounts of light? How does the visual system decide what white is and how does it deal with changes in the amount of light illuminating the scene? The visual system is constantly trying to estimate what the incident illumination is and what are the actual reflectances of the surfaces present in the scene.

If our goal is to estimate the shade of gray of a piece of painted paper, then the quantity that we care about is the reflectance of the surface. But, what happens if we see two patches of unknown reflectance, and each is illuminated with two different light sources of unknown identity? How does the visual system manages to infer what the illumination and reflectances are?

Visual illusions are a way of getting to feel how our own visual system processes images. To experience how this estimation process works let's start by considering a very simple image as shown in [figure 18.24](#).

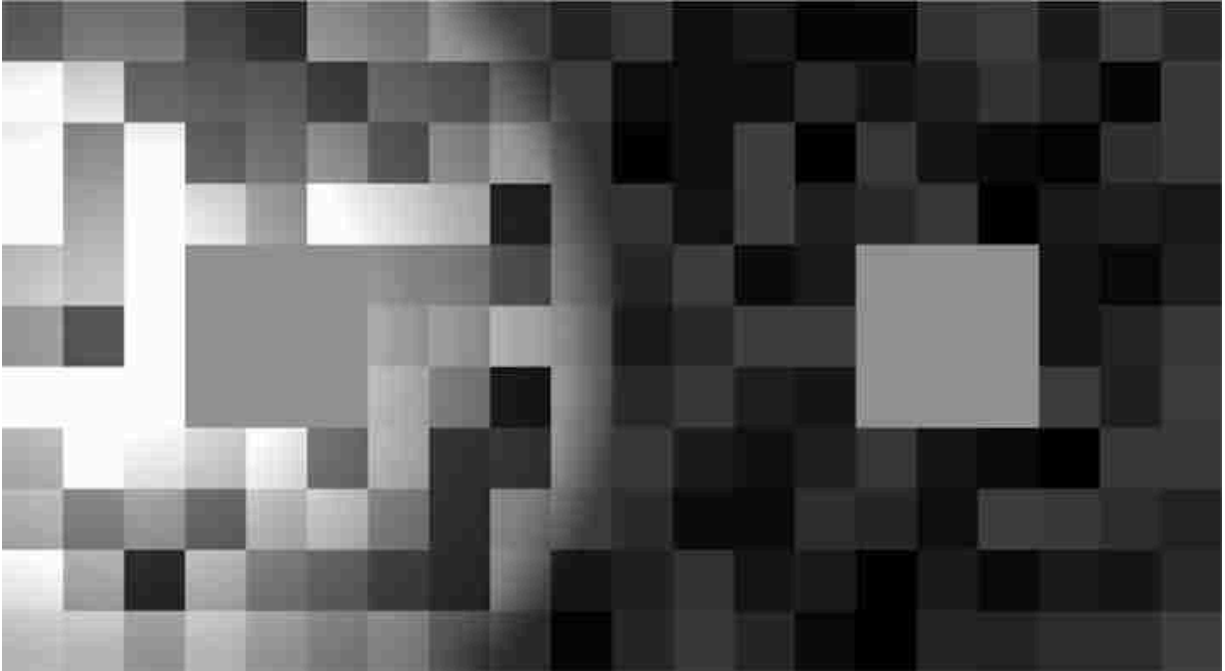


Figure 18.24: Simultaneous contrast illusion. What happens if we see two patches of unknown reflectance, and each is illuminated with two different light sources of unknown identity?

This image shows a very simple scene formed by a surface with a set of squares of different gray levels illuminated by a light spot oriented toward the left. Our visual system tries to estimate the two quantities: the reflectance of each square and the illumination intensity reaching each pixel.

Let's think of the image formation process. The surface is made of patches of different reflectances $r(x, y) \in (0, 1)$. Each location receives an illumination $l(x, y)$. The observed brightness is the product:

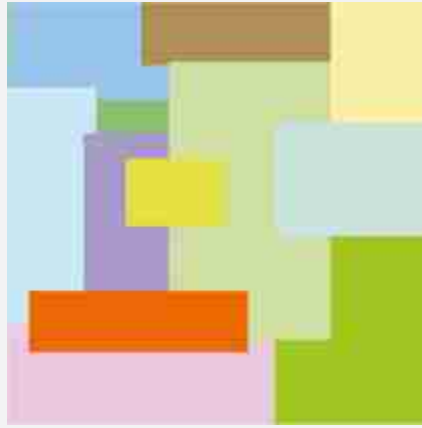
$$\ell(x, y) = r(x, y) \times l(x, y) \quad (18.37)$$

Despite that what reaches the eye is the signal $\ell(x, y)$, our perception is not the value of $\ell(x, y)$. In fact, the squares 1 and 2 in [figure 18.24](#) have the exact same values of intensity but we see them differently which is generally explained by saying that we discount (at least partially) the effects of the illumination, $l(x, y)$.

But how can we estimate $r(x, y)$ and $l(x, y)$ by only observing $\ell(x, y)$? One of the first solutions to this problem was the **Retinex** algorithm proposed by Land and McCann [[283](#)]. Land and McCann observed that it is possible to extract r and l from ℓ if one takes into account some constraints

about the expected nature of the images r and l . Land and McCann noted that if one considers two adjacent pixels inside one of the patches of uniform reflectance, the difference between the two pixel values will be very small as the illumination, even if it is nonuniform, will only produce a small change in the brightness of the two pixels. However, if the two adjacent pixels are in the boundary between two patches of different reflectances, then the intensities of these two pixels will be very different.

The Retinex algorithm, by Land and McCann [283], is based on modeling images as if they were part of a Mondrian world (images that look like the paintings of Piet Mondrian). One example of a color Mondrian is:



The Retinex algorithm works by first extracting x and y spatial derivatives of the image $\ell(x, y)$ and then thresholding the gradients. First, we transform the product into a sum using the log:

$$\log \ell(x, y) = \log r(x, y) + \log l(x, y) \quad (18.38)$$

Taking derivatives along x and y is now simple:

$$\frac{\partial \log \ell(x, y)}{\partial x} = \frac{\partial \log r(x, y)}{\partial x} + \frac{\partial \log l(x, y)}{\partial x} \quad (18.39)$$

And the same thing is done for the derivative along y .

Any derivative larger than the threshold is assigned to the derivative of the reflectance image $r(x, y)$ and the ones smaller than a threshold are assigned to the illumination image $l(x, y)$:

$$\frac{\partial \log r(x, y)}{\partial x} = \begin{cases} \frac{\partial \log \ell(x, y)}{\partial x} & \text{if } \left| \frac{\partial \log \ell(x, y)}{\partial x} \right| > T \\ 0 & \text{otherwise} \end{cases} \quad (18.40)$$

[Figure 18.25](#) shows how the image derivatives are decomposed into reflectance and illumination changes.

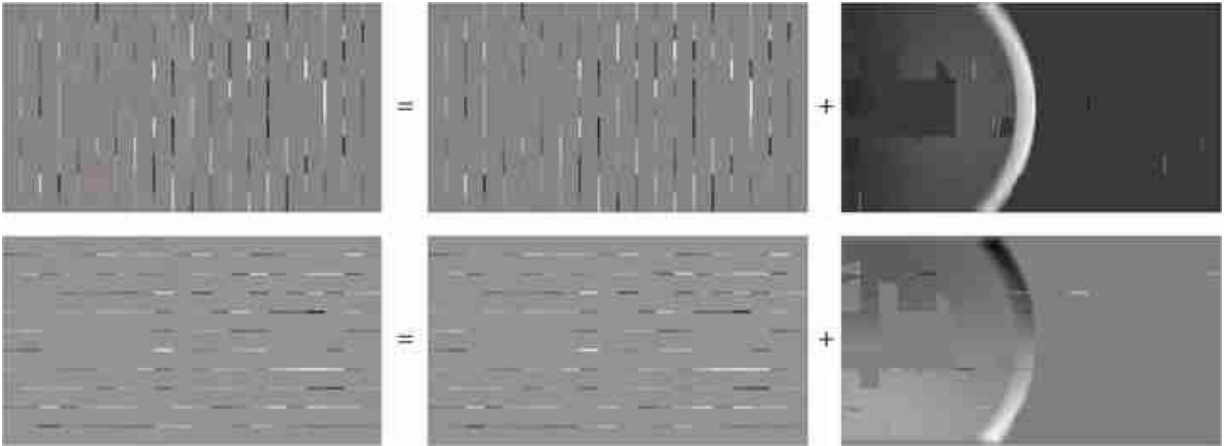


Figure 18.25: Derivatives classified into reflectance or luminance components.

Then, the image $\log r(x, y)$ is obtained by integrating the gradients, as shown in section 18.4, and exponentiating the result. Finally, the illumination can be obtained as $l(x, y) = \ell(x, y)/r(x, y)$. The results are shown in [figure 18.26](#).

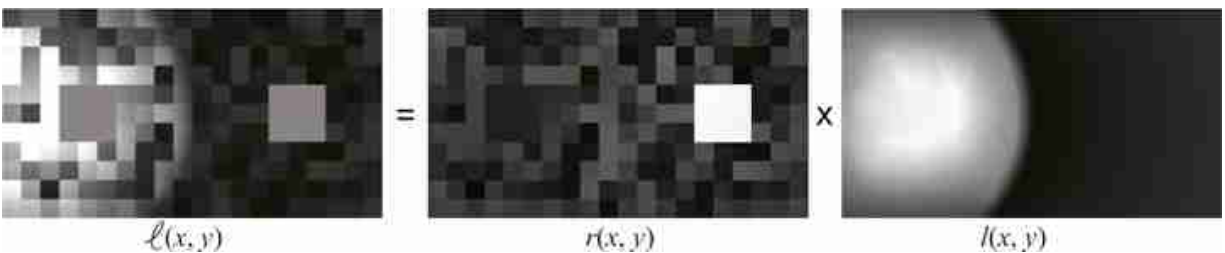


Figure 18.26: Recovered components. The estimated reflectance, $r(x, y)$, is close to what we perceive. It seems that what we perceive contains part of $l(x, y)$.

Note that the illumination derivatives have much lower magnitude than the reflectance derivatives; however they extend over much larger spatial regions, which means that once they are integrated, changes in brightness due to nonuniform illumination might be larger than changes due to variations in reflectance values.

Despite the simplicity of this approach it works remarkably well with images like the ones shown in [figure 18.24](#) and also with a number of real images. However, the algorithm is limited and it is easy to make it fail in situations where the visual system works, as we will see later in more depth.

The Retinex algorithm works by making assumptions about the structure of the images it tries to separate. The underlying assumption is that the illumination image, $l(x, y)$, varies smoothly and the reflectance image, $r(x, y)$, is composed of uniform regions separated by sharp boundaries. These assumptions are very restrictive and will limit the domain of applicability of such an approach. Understanding the structure of images in order to build models such as Retinex has been the focus of a large amount of research. This chapter will introduce some of the most important image models.

The recovered brightness is stronger than what we perceive. There are a number of reasons that weaken the perceived brightness. The effect is bigger if the display occupies the entire visual field. Right now the picture appears in the context of the page, which also affects our perception of the illumination. Also, the two larger squares do not seem to group well with the rest of the display, as if they were floating in a different plane. And finally, for such a simple display, the visual system does not fully separate the perception of both components.

Decomposing an image into different physical causes is known as **intrinsic images decomposition**. The intrinsic image decomposition, proposed by Barrow and Tenenbaum [39], aims to recover intrinsic scene characteristics from images, such as occlusions, depth, surface normals, shading, reflectance, reflections, and so on.

18.13 Concluding Remarks

In this chapter we have covered a very powerful image representation: image derivatives (first order, second order, and the Laplacian) and their discrete approximations. Despite the simplicity of this representation, we have seen that it can be used in a number of applications such as image inpainting, separation of illumination and reflectance, and it can be used to explain simple visual illusions. It is not surprising that similar filters like the ones we have seen in this chapter emerge in convolutional neural networks when trained to solve visual tasks.

[7](#). **Image inpainting** consists in filling a missing region in an image.

19 Temporal Filters

19.1 Introduction

Although adding time might seem like a trivial extension from two-dimensional (2D) signals to three-dimensional (3D) signals, and in many aspects it is, there are some properties of how the world behaves that make sequences seem different from arbitrary 3D signals. In 2D images most objects are bounded, occupying compact and well defined image regions. However, in sequences, objects do not appear and disappear instantaneously unless they get occluded behind other objects or enter or exit the scene through doors or the image boundaries. So, the behavior of an object across time t is very different than its behavior across space n, m . In time, objects move and deform defining continuous trajectories that have no beginning and never end.

19.2 Modeling Sequences

Sequences will be represented as functions $\ell(x, y, t)$, where x, y are the spatial coordinates and t is time. As before, when processing sequences we will work with the discretized version that we will represent as $\ell[n, m, t]$, where n, m are the pixel indices and t is the frame number. Discrete sequences will be bounded in space and time, and can be stored as arrays of size $N \times M \times P$.

[Figure 19.1](#) illustrates this with one sequence, as shown in [figure 19.1\(a\)](#). This sequence has 90 frames and shows people on the street walking parallel to the camera plane and at different distances from the camera. [Figure 19.1\(b\)](#) shows the space-time array $\ell[n, m, t]$. When we look at a picture we are looking at a 2D section, $t = \text{constant}$, of this cube. But it is interesting to look at sections along other orientations. [Figures 19.1\(c-d\)](#) show sections for $m = \text{constant}$ and $n = \text{constant}$, respectively. Although they are also 2D images, their structure looks very different from the images we are used to seeing. [Figure 19.1\(c\)](#) shows a horizontal section that is parallel to the direction of motion of the people walking. Here we see

straight bands with different orientations. The bands appear to occlude each other. Each band corresponds to one person and its orientation is given by the speed and the direction of motion. [Figure 19.1](#)(d) looks like a photo-finish picture like the ones used in sporting races. In both [figures 19.1](#)(c and d), static objects appear as vertical stripes in [figure 19.1](#)(b), and horizontal stripes in [figure 19.1](#)(d).

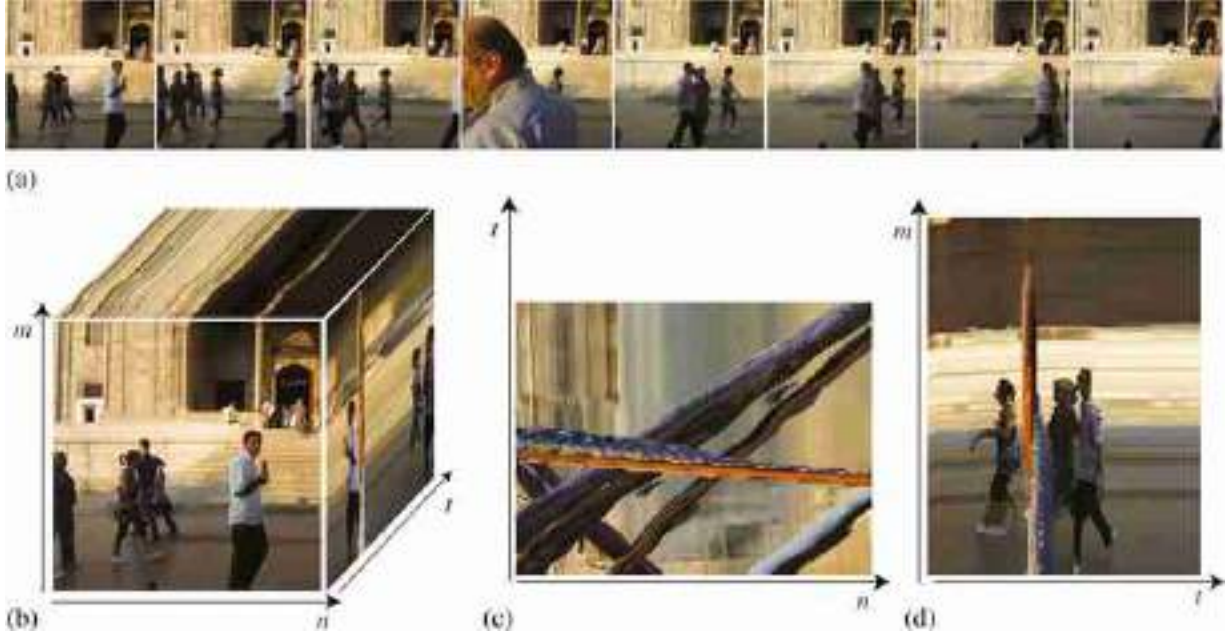


Figure 19.1: (a) Eight frames from a sequence with people walking. The frames are shown at regular time intervals. The full sequence had 90 frames (corresponding to 3 s of video). (b) Space-time array, $\ell [n, m, t]$ of size $128 \times 128 \times 90$. (c) Section for $m = 50$. (d) Section for $n = 75$. Static objects appear as straight lines.

One special sequence is when the image has a global motion with constant velocity (v_x, v_y) . In such a case we can write:

$$\ell(x, y, t) = \ell_0(x - v_x t, y - v_y t) \quad (19.1)$$

where $\ell_0(x, y) = \ell(x, y, 0)$ is the image being translated, and v_x and v_y are constants. At time t the image $\ell_0(x, y)$ is translated by the vector $(v_x t, v_y t)$ as described by equation (19.1). The pixel value at location $(x = 0, y = 0)$ at time $t = 0$ will appear at time T in location $(x = v_x T, y = v_y T)$. This is what we see in [figure 19.1](#)(c) where the bands are created by moving pixels.

We use continuous values for x , y , and t , and continuous images, $\ell_0(x, y)$, because it allows us to deal with any velocity values. This function also assumes that the brightness of the pixels does not change while the scene is moving (**constant brightness assumption**).

The constant brightness assumption is the initial hypothesis of most motion estimation algorithms. We will devote multiple chapters in part XI to study motion in depth.

In general, sequences will be more complex, but the properties of a globally moving image are helpful to understand local properties in sequences. We can also write models for more complex sequences. For instance, a sequence containing a moving object over a static background can be written as:

$$\ell(x, y, t) = b(x, y)(1 - m(x - v_x t, y - v_y t)) + o(x - v_x t, y - v_y t)m(x - v_x t, y - v_y t) \quad (19.2)$$

where $b(x, y)$ is the static background image, $o(x, y)$ is the object image moving with speed (v_x, v_y) , and $m(x, y)$ is a binary mask that moves with the object and that models the fact that the object occludes the background.

19.3 Modeling Sequences in the Fourier Domain

The Fourier transform (FT) of a globally moving image, equation (19.1), is (using the shift property):

$$\mathcal{L}(w_x, w_y, w_t) = \mathcal{L}_0(w_x, w_y) \delta(w_t + v_x w_x + v_y w_y) \quad (19.3)$$

The continuous FT of the sequence is equal to the product of the 2D FT of the static image $\ell_0(x, y)$ and a delta wall. To better understand this function let's look at a simple example on only one spatial dimension, as shown in [figure 19.2](#).

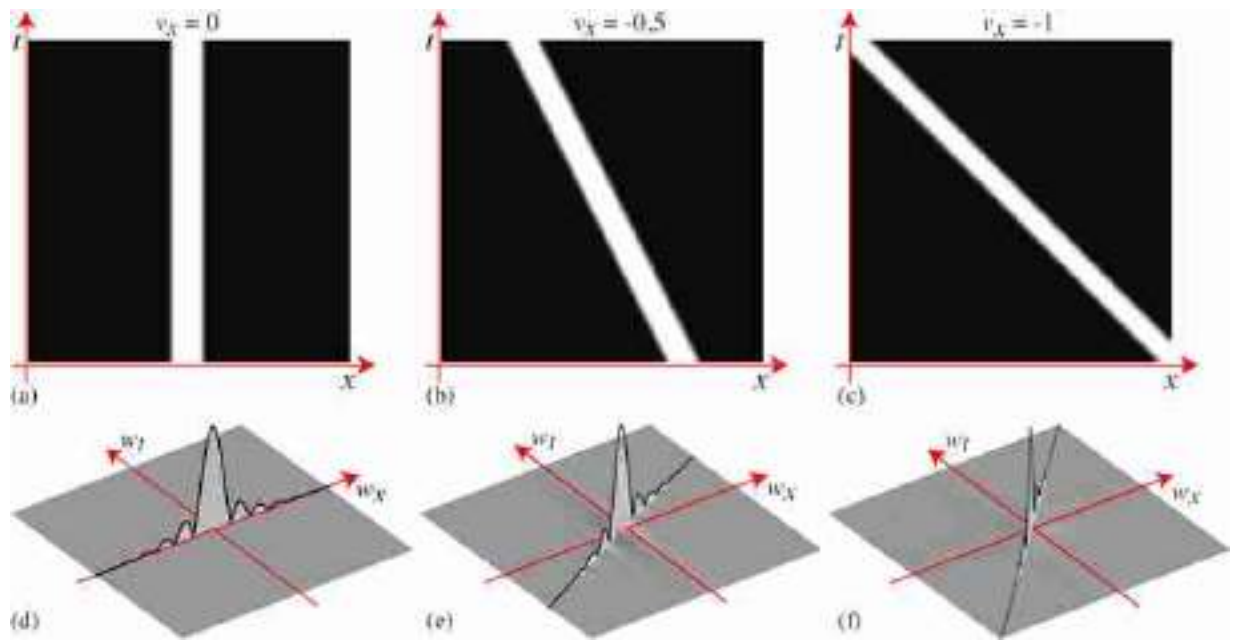


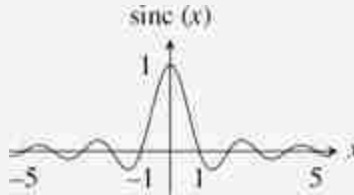
Figure 19.2: (a) A sequence with one spatial dimension showing a static rectangular pulse. (b) The rectangular pulse moves toward the left at speed $v = -0.5$, and (c) toward the left, $v = -1$. As we work with discretized signals, speed units are in pixels per frame.

[Figure 19.2](#) shows the FT for a sequence with one spatial dimension, $\ell(x, t)$, that contains a blurry rectangular pulse moving at three different speeds toward the left. [Figure 19.2\(a\)](#) shows a sequence when the pulse is static and [figure 19.2\(d\)](#) shows its FT. Across the spatial frequency w_x , the FT is approximately a sinc function. Across the temporal frequency w_t , as the signal is constant, its FT is a delta function. Therefore the FT is a sinc function contained inside a delta wall in the line $w_t = 0$. [Figure 19.2\(b\)](#) shows the same rectangular pulse moving towards the left, $v_x = 0.5$. [Figure 19.2\(e\)](#) shows the sinc function but skewed along the frequency line $w_t + 0.5w_x = 0$. Note that this is not a rotation of the sinc function from [figure 19.2\(d\)](#) as the locations of the zeros lie at the same w_x locations. [Figure 19.2\(c\)](#) shows the pulse moving at a faster speed resulting in a larger skewing of its FT, [figure 19.2\(f\)](#).

The **sinc function** is the FT of a box kernel. In the continuous domain, the sinc function has the form:

$$\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$$

This function has its maximum at $x = 0$ and then has decreasing oscillations.



We will discuss this further in section 20.5.

19.4 Temporal Filters

Linear spatiotemporal filters can be written as spatiotemporal convolutions between the input sequence and a convolutional kernel (impulse response). Discrete spatiotemporal filters have an impulse response $h[n, m, t]$, where the independent variables n , m , and t are discrete and only take on integer values. The extension from 2D filter to spatiotemporal filters does not have any additional complications. We can also classify filters as low-pass, high-pass, and so on. But in the case of time, there is another attribute used to characterize filters, namely **causality**:

- Causal filters. These are filters with output variations that only depend on the past values of the input. This puts the following constraint: $h[n, m, t] = 0$ for all $t < 0$. This means that if the input is an impulse at $t = 0$, the output will only have non-zero values for $t > 0$. If this condition is satisfied, then the filter output will only depend on the input's past for all possible inputs.
- Noncausal filters. When the output has dependency on future inputs.
- Anticausal filters. This is the opposite, when the output only depends on the future: $h[n, m, t] = 0$ for all $t > 0$.

Many filters are noncausal and have both causal and anticausal components (e.g., a Gaussian filter). Note that noncausal filters cannot be implemented in practice and, therefore, any filter with an anticausal component will have to be approximated by a purely causal filter by bounding the temporal support and shifting in time the impulse response.

In this chapter, we have written all the filters as convolutions. However, some filters are better described as difference equations (this is specially important in time). An example of a difference equation is:

$$\ell_{\text{out}}[n, m, t] = \ell_{\text{in}}[n, m, t] + \alpha \ell_{\text{out}}[n, m, t-1] \quad (19.4)$$

where the output ℓ_{out} at time t depends on the input at time t and the output at the previous time instant $t-1$ multiplied by a constant α . We can easily evaluate the impulse response, $h[n, m, t]$, of such a filter by replacing $\ell_{\text{in}}[n, m, t]$ with an impulse, $\delta[n, m, t]$.

Remember that $\delta[n, m, t]$ is 1 for $n=0, m=0, t=0$, and 0 everywhere else.

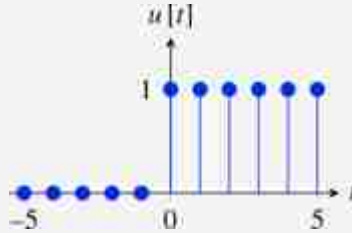
The impulse response is:

$$h[n, m, t] = \alpha^t \delta[n, m] u[t] \quad (19.5)$$

where $u[t]$, called the **Heaviside step** function, is:

$$u[t] = \begin{cases} 0 & \text{if } t < 0 \\ 1 & \text{otherwise} \end{cases} \quad (19.6)$$

The Heaviside step function, $u[t]$, is an infinite length function with the form



Most filters described by differences equations have an impulse response with infinite support. They are called Infinite Impulse Response (IIR) filters. IIR filters can be further classified as stable and unstable. **Stable** are the ones that given a bounded input, $|\ell_{\text{in}}[n, m, t]| < A$, produce a bounded output, $|\ell_{\text{out}}[n, m, t]| < B$. For this to happen, the impulse response has to be bounded. In **unstable** filters, the amplitude of the impulse response diverges to infinity. In the previous example, the filter is stable if and only if $|\alpha| < 1$.

Let's now describe some spatiotemporal filters.

19.4.1 Spatiotemporal Gaussian

As with the spatial case, we can define the same low-pass filters in the spatiotemporal domain: the box filter, triangular filters, and so on. As an example, let's focus on the Gaussian filter. The spatiotemporal Gaussian is a simple extension of the spatial Gaussian filter in section 17.3:

$$g(x, y, t; \sigma_x, \sigma_t) = \frac{1}{(2\pi)^{3/2} \sigma_x^2 \sigma_t} \exp -\frac{x^2 + y^2}{2\sigma_x^2} \exp -\frac{t^2}{2\sigma_t^2} \quad (19.7)$$

where σ_x is the width of the Gaussian along the two spatial dimensions, and σ_t is the width on the temporal domain. As the units for t and x, y are unrelated, it does not make sense to set all the σ s to have the same value.

We can discretize the continuous Gaussian by taking samples and building a 3D convolutional kernel. We can also use the binomial approximation. The 3D Gaussian is separable so it can be implemented efficiently as a convolutional cascade of three one-dimensional (1D) kernels. [Figure 19.3](#)(a) shows a spatiotemporal Gaussian. The temporal Gaussian is a noncausal filter, therefore it is not physically realizable. This is not a problem when processing a video stored in memory. However, if we are processing a streamed video, we will have to bound and shift the filter to make it causal, which will result in a delay in the output.

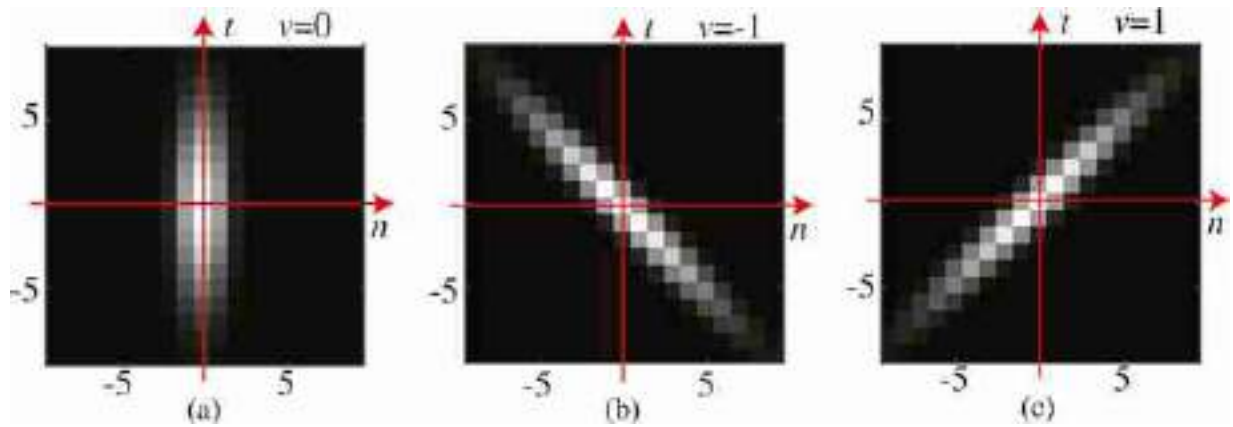


Figure 19.3: (a) Spatiotemporal Gaussian with $\sigma = 1$ and $\sigma_t = 4$. (b) Same Gaussian parameters but skewed by the velocity vector $v_x = -1$, $v_y = 0$ pixels/frame, and (c) $v_x = 1$, $v_y = 0$ pixel/frame.

[Figure 19.4](#)(a) shows one sequence and [figure 19.4](#)(b) shows the sequence filtered with the Gaussian from [figure 19.3](#)(a). This Gaussian has

a small spatial width, $\sigma = 1$, and a large temporal width, $\sigma_t = 4$ so the sequence is strongly blurred across time. The moving objects show motion blur and are strongly affected by the temporal blur, while the static background is only affected by the spatial width of the Gaussian.

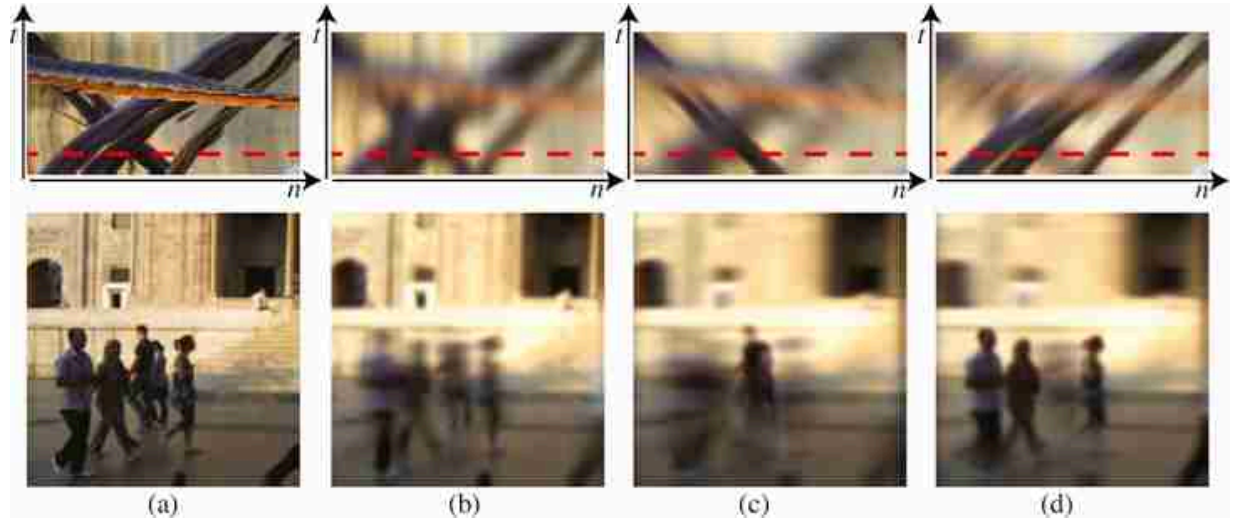


Figure 19.4: (a) One frame from the input sequence and the space-time section (on top). (b) Output when convolving the Gaussian from [figure 19.3\(a\)](#). (c) Output of the convolution with [figure 19.3\(b\)](#). (d) Output of the convolution with [figure 19.3\(c\)](#).

How could we create a filter that keeps sharp objects that move at some velocity (v_x, v_y) while blurring the rest? [Figure 19.4\(c\)](#) shows the desired output of such a filter. The bottom image shows one frame for a sequence filtered with a kernel that keeps sharp objects moving left at 1 pixel/frame while blurring the rest. This filter can be obtained by skewing the Gaussian:

$$g_{v_x, v_y}(x, y, t) = g(x - v_x t, y - v_y t, t) \quad (19.8)$$

This directional blur is not a rotation of the original Gaussian as the change of variables is not unitary, but the same effect could be obtained with a rotation. [Figure 19.4\(c\)](#) shows the effect when $v_x = -1$, $v_y = 0$. The Gaussian is shown in [figure 19.3\(b\)](#). The space-time section shows how the sequence is blurred everywhere except one oriented bar corresponding to the person walking left. [Figure 19.4\(d\)](#) shows the effect when $v_x = 1$, $v_y = 0$. The output of this filter looks as if the camera was tracking one of the objects while the shutter was open, producing a blurry image of all the other objects.

19.4.2 Temporal Derivatives

Spatial derivatives are useful to find regions of image variation such as object boundaries. Temporal derivatives can be used to locate moving objects. We can approximate a temporal derivative for discrete signals as the difference:

$$\ell[m, n, t] - \ell[m, n, t-1] \quad (19.9)$$

When implementing temporal filters it is important to use causal filters. A causal filter depends only on the present and past input samples.

As in the spatial case, it is useful to compute temporal derivatives of spatiotemporal Gaussians:

$$\frac{\partial g}{\partial t} = \frac{-t}{\sigma_t^2} g(x, y, t) \quad (19.10)$$

where $g(x, y, t)$ is the Gaussian as written in equation (19.7). We can compute the spatiotemporal gradient of a Gaussian:

$$\begin{aligned} \nabla g &= (g_x(x, y, t), g_y(x, y, t), g_t(x, y, t)) \\ &= (-x/\sigma^2, -y/\sigma^2, -t/\sigma_t^2) g(x, y, t) \end{aligned} \quad (19.11)$$

We can use the analytic form of the spatiotemporal Gaussian derivatives from equation (19.11) to discretize the filter, by taking samples at discrete locations in space and time, and use the resulting discrete spatiotemporal kernel to filter an input discrete sequence. These filters can be used for many applications. Let's look at one practical example: What should we do if we want to remove from a sequence the objects moving at a particular velocity?

To answer that question, we will first assume the sequence contains a single object moving at a constant velocity. We will then compute the derivatives along x , y , and t of the sequence and we will find out what particular linear combination of those derivatives makes the output go to zero only when the input sequence moves at a target velocity.

In the case of an image, $\ell_0(x, y)$, that moves with velocity (v_x, v_y) , the sequence is

$$\ell(x, y, t) = \ell_0(x - v_x t, y - v_y t) \quad (19.12)$$

We can compute the temporal derivative of $\ell(x, y, t)$ as:

$$\frac{\partial \ell}{\partial t} = \frac{\partial \ell_0}{\partial t} = -v_x \frac{\partial \ell_0}{\partial x} - v_y \frac{\partial \ell_0}{\partial y} \quad (19.13)$$

This result is interesting because it shows that for a moving image there is a relationship between the temporal derivative of the sequence and the spatial derivatives along the direction of motion. We can use this relationship to find a linear combination of the temporal and spatial derivatives so that the output is zero.

In fact, if we compute the gradient of the Gaussian along the vector $1, v_x, v_y$:

$$h(x, y, t; v_x, v_y) = g_t + v_x g_x + v_y g_y = \nabla g (1, v_x, v_y)^T \quad (19.14)$$

we get a kernel $h(x, y, t; v_x, v_y)$ that we can use as a spatiotemporal filter that will do exactly what we were looking for. Indeed, if we convolve it with the input sequence $\ell_0(x - v_x t, y - v_y t)$ we get a zero output (using equation [19.13]):

$$\begin{aligned} \ell_0(x - v_x t, y - v_y t) \circ h &= \ell_0(x - v_x t, y - v_y t) \circ (g_t + v_x g_x + v_y g_y) \\ &= \left(\frac{\partial \ell_0}{\partial t} + v_x \frac{\partial \ell_0}{\partial x} + v_y \frac{\partial \ell_0}{\partial y} \right) \circ g \\ &= 0 \end{aligned}$$

The filter h from equation (19.14) is shown in [figure 19.5](#). As this filter is 3D we show it as a sequence for different velocities. Each row in the figure corresponds to one particular velocity v_x, v_y . In the example shown in [figure 19.5\(a\)](#) the Gaussian has a width of $\sigma^2 = \sigma_t^2 = 1.5$, and has been discretized as a 3D array of size $7 \times 7 \times 7$. [Figures 19.5\(b,c and d\)](#) show the filter h for different velocities: $(v_x, v_y) = (0, 0), (1, 0)$ and $(-1, 0)$.

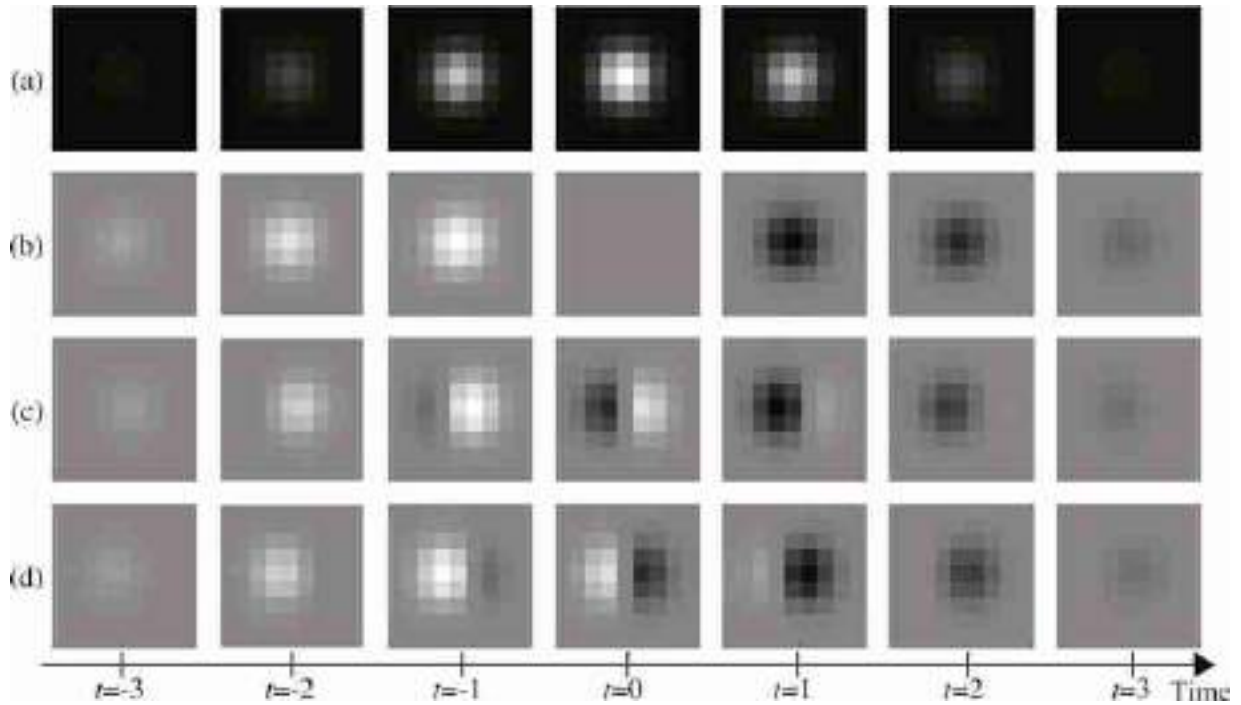


Figure 19.5: Visualization of the space-time Gaussian. The Gaussian has a width of $\sigma^2 = \sigma_t^2 = 1.5$, and has been discretized as a 3D array of size $7 \times 7 \times 7$. Images along each row show the seven frames of each spatiotemporal discrete Gaussian. (a) Gaussian. (b) The partial derivative of the Gaussian with respect to t . (c) Derivative along $v = (1, 0)$ pixels/frame. (d) $v = (-1, 0)$ pixels/frame.

[Figure 19.6](#) shows a different visualization of the same filter h from equation (19.14). Each image in [figure 19.6](#) shows a space-time section of the same spatiotemporal Gaussian derivatives as the ones shown in [figure 19.5](#). Both visualizations are equivalent and help to understand how the filter works.

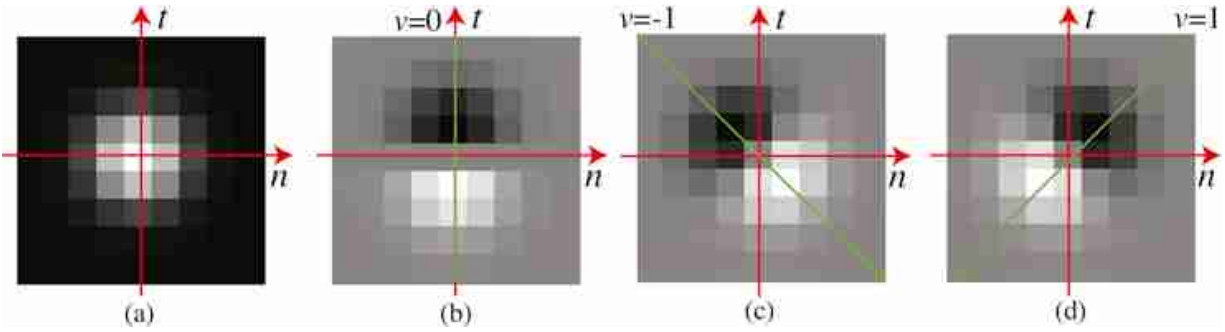


Figure 19.6: Spatiotemporal Gaussian $g[n, t]$ and derivatives. (a) Gaussian with $\sigma^2 = 1.5$. (b) Partial derivative with respect to t . (c) Partial derivative along $(1, -1)$. (d) Partial derivative along $(1, 1)$. The result of adding values along the green line is zero.

Such a filter h will cancel any objects moving at the velocity (v_x, v_y) . By using different filters, each one computing derivatives a long different space-time orientations, we can create output sequences where specific objects disappear, as shown in [figure 19.7](#). This filter is called a **nulling filter** [95].

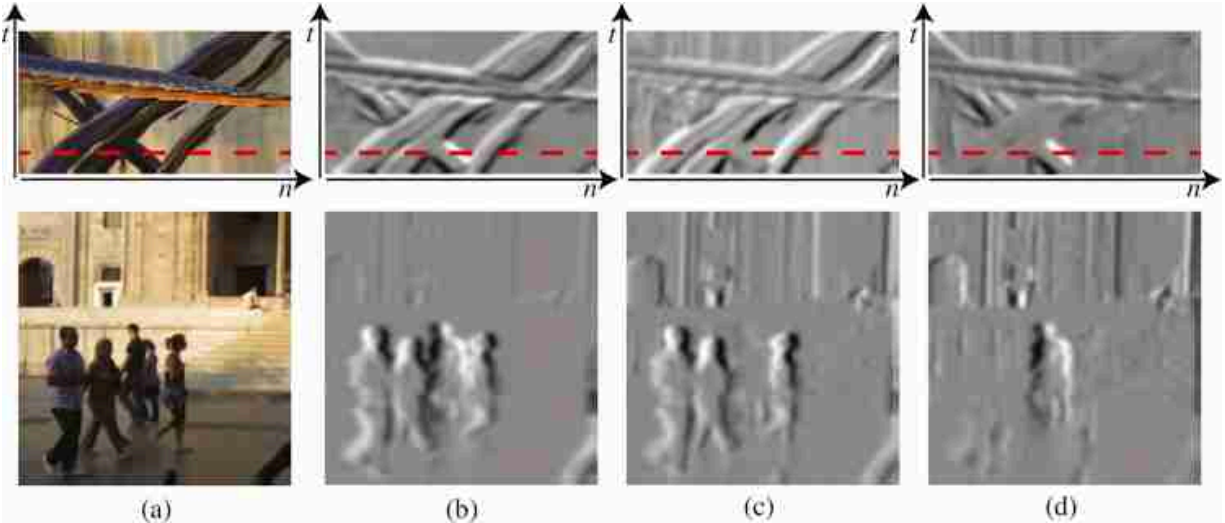


Figure 19.7: (a) Input sequence. (b) Output to h with $v_x = v_y = 0$. (c) $v_x = 1$ pixels/frame. (d) $v_x = -1$ pixel/frame.

Note that despite that we assumed that the sequence contained a single moving object when deriving the formulation for the nulling filter, the filter also works in sequences with multiple objects moving a different speeds as

shown in [figure 19.7](#). This is because the operations are local (the kernel h has a small size) and the behavior will be correct if in a local image patch there is only one velocity present. In fact, it is easy to show that the method will also work if the sequence is a sum of several moving transparent layers.

19.5 Concluding Remarks

In this chapter we have discussed different types of spatiotemporal filters used to analyze video. However, we have not presented how these filters can be used to estimate useful quantities such as velocity. We will devote several chapters in part XII to motion estimation.

VI

SAMPLING AND MULTISCALE IMAGE REPRESENTATIONS

Changing the scale of an image is one of the most important operations. It does not matter what area of computer vision you focus on. If there is one thing that you will always do, it is to resize images. This seemingly innocuous operation can have detrimental effects if the right precautions are not taken into consideration. In this part we will study image resizing in detail.

Image resizing is also a key operation to build scale-invariant representations. The last two chapters of this part will be devoted to construct multiscale image representations.

This part is composed of the following chapters:

- **Chapter 20** explains how to sample a continuous signal. Sampling is a necessary step to digitize a signal.
- **Chapter 21** describes the process of downsampling and upsampling an image.
- **Chapter 22** introduces filter banks as a type of image representation.
- **Chapter 23** describes multiscale image pyramids.

20 Image Sampling and Aliasing

20.1 Introduction

In nature, most of the signals we measure (sound, light, etc.) are defined over continuous domains (time, space, etc.). In order to process them with computers we need to transform the continuous domain into a discrete one. Sampling is the process of transforming a continuous signal into a discrete one.

We need to study the following questions: What are the possible sampling patterns to sample a signal? How can we characterize the loss of information? And how do we reduce artifacts?

Understanding how information is transformed when we convert a continuous signal into a discrete signal is important because we often have to optimize two competing factors:

- Maximize the amount of available information. This requires as many samples as possible.
- Minimize the computational cost and memory requirements. This implies that we want to minimize the number of samples we collect.

Let's consider a one-dimensional (1D) continuous time signal $\ell(t)$ and its sampled version $\ell[n] = \ell(nT_s)$, where T_s is the **sampling period**. Intuitively, it is clear that in this sampling process some information will get lost. If no information was lost, then we should be able to recover the continuous signal $\ell(t)$ from its sampled version $\ell[n]$ by doing some kind of interpolation. One way of ensuring that information is not lost is to decrease T_s , which will result in a more accurate approximation of the continuous signal $\ell(t)$ at the expense of the amount of memory needed to store $\ell[n]$. Decreasing T_s will also result in an increase of the computational cost of processing the signal $\ell[n]$. Therefore, it is important to choose the appropriate T_s . Understanding the sampling process and how to reconstruct the continuous signal is necessary as it will allow us to find the optimal sampling parameters.

In this chapter we will be dealing with continuous and discrete signals and their corresponding Fourier transforms.

20.2 Aliasing

Let's first look at one example to get a sense of the type of issues that might arise when discretizing a signal. Consider the continuous signal with the form $\ell(t) = \cos(\omega t)$ with $\omega = 18\pi$, shown in [figure 20.1](#). The period of this signal is $T = 2\pi/\omega = 1/9$ (there are nine periods in the interval $t \in [0, 1]$).

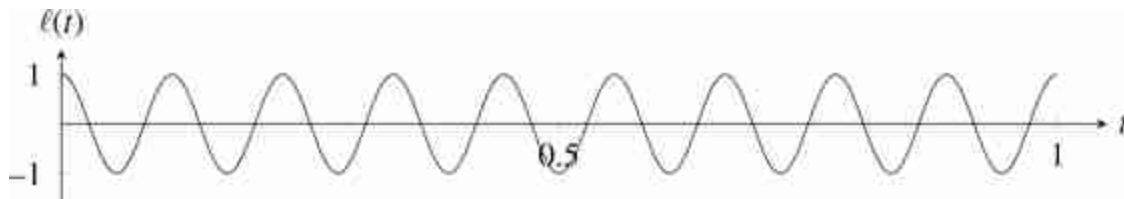


Figure 20.1: Continuous cosine wave, $\ell(t) = \cos(\omega t)$, with frequency $\omega = 18\pi$.

We now build a discrete signal by sampling with a period of $T_s = 1/11$, which results in the discrete signal $\ell[n] = \ell(nT_s)$ (there are 11 samples in the same interval $t \in [0, 1]$). This could seem enough because there are more samples than periods. However, the samples of discrete signal, $\ell[n]$ are shown here ([figure 20.2](#)):

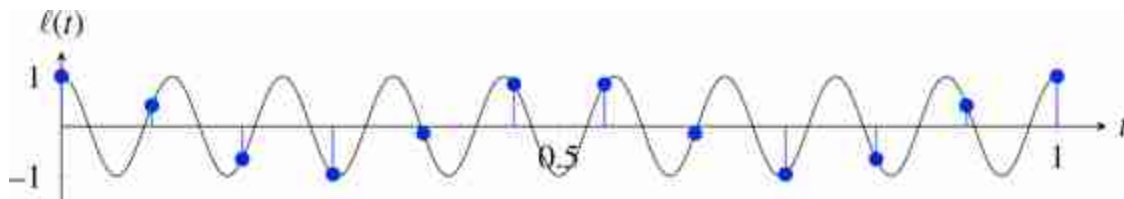


Figure 20.2: Sampling the cosine wave with a sampling period of $T_s = 1/11$.

If we now want to reconstruct the original continuous signal from its samples $\ell[n]$ there are many possibilities because the samples do not constrain what happens between samples. Therefore we will need to make some assumptions about the continuous signal. In the absence of any other

prior information, we will assume that the most likely signal is the slowest and smoothest signal (we will make this assumption more precise later). Therefore, our preferred interpolation will be the one shown in red in the following plot:

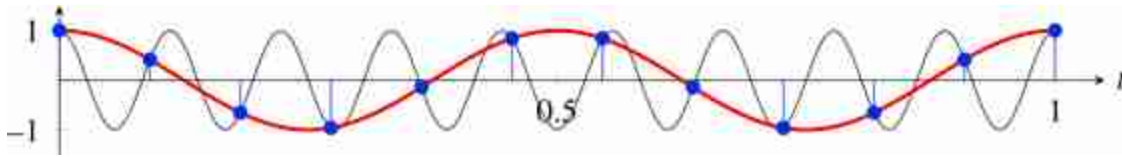


Figure 20.3: There are infinite waves (only two shown) that perfectly pass by all the samples.

The plot above shows the superposition of the original signal before sampling and the reconstructed one after sampling (in red). Both signals perfectly pass through the same samples. Under the slow and smooth prior, the samples seem to correspond to a cosine function with a lower frequency (in this example $T = 1/2$) than the input (which had $T = 1/9$).

It is important to mention that there is nothing special about how the parameters have been chosen for this example. Many different parameter choices would have yielded the same qualitative behavior. This confusion of frequencies is called **aliasing**.

Let's now look at an example in two dimensions (2D). As a concrete example, let's consider the 2D signal shown in [figure 20.4\(a\)](#). This 2D signal is a continuous function in the spatial domain defined as $\ell(x, y) = \cos(w(x + 2y))$ and with a frequency, w , which also varies with location following $w = x + 2y$. That is $\ell(x, y) = \cos(x + 2y)^2$. The image shows a set of diagonal waves with a changing frequency starting slow at the bottom left and becoming faster and faster as we approach the top-right corner.

In order to store this continuous image on a computer, we need to sample it and convert it into a discrete image, $\ell[n, m]$. [Figure 20.4\(b\)](#) shows the resulting sampled image when the sampling rate is chosen so that it results in an image size 52×52 pixels. The first thing to note is that the resulting image keeps very little similarity to the original signal. Only the bottom-left corner of the image looks anything like the continuous signal. As we move toward the right, the image changes the dominant orientation and the frequency becomes slow again.

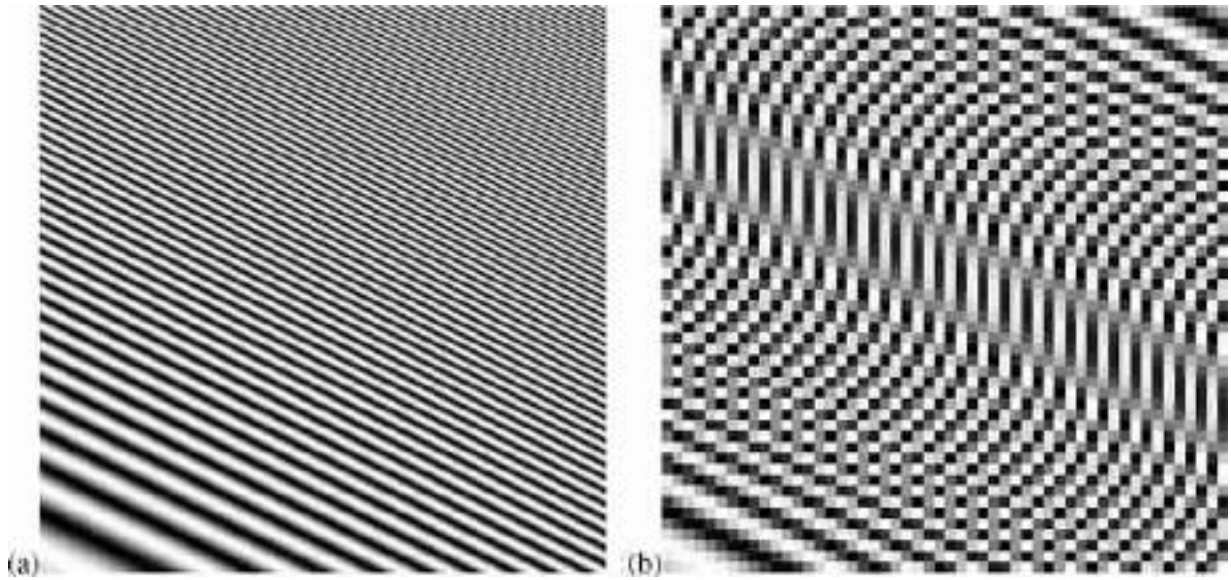


Figure 20.4: (a) A continuous image (here shown by sampling very finely). (b) Its heavily sampled version resulting in an image with size 52×52 .

Our perception seems to follow the slow and smooth assumption [499]. We have prior towards smooth and slow trajectories and textures. In the presence of noise, or missing information, our visual system will tend to follow the prior.

20.3 Sampling Theorem

Let's start by considering a band-limited signal $\ell(t)$. A band limited signal is a signal with no spectral content above a frequency w_{max} . An example of band-limited signal is a signal with the following Fourier transform:

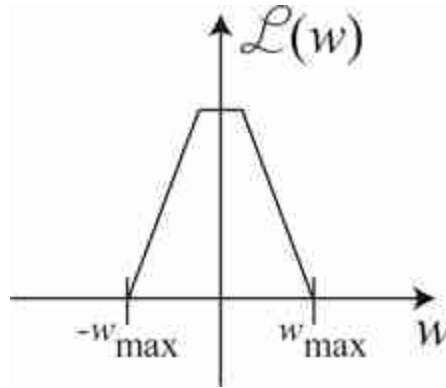


Figure 20.5: A band-limited signal with maximum frequency w_{max} .

The **sampling theorem** (also known as Nyquist theorem) states that for a signal to be perfectly reconstructed from a set of samples (under the slow and smooth prior), the sampling frequency $w_s = 2\pi/T_s$ has to be $w_s > 2w_{max}$ when w_{max} is the maximum frequency present in the input signal [429].

The same theorem can be stated in terms of periods: the sampling period, T_s , has to be $T_s < T_{min}/2$, i.e, smaller than half of period of the highest frequency component, $T_{min} = 2\pi/w_{max}$. Note that our previous example did not satisfy the **Nyquist condition**.

In this book we use the **radian frequency** for continuous signals, $w = 2\pi f$, where f is the **hertz frequency**. For a sinusoidal wave of period T seconds, its frequency is $w = 2\pi/T$.

One way of characterizing the sampling process is achieved by analyzing the relationship between the Fourier transform of the continuous and discrete signals. There are many ways of finding the relationship between the two Fourier transforms. Here we will describe the most common one.

20.3.1 Modeling the Sampling Process

First, let's use the following notation. Let's denote as $\ell(t)$ the continuous signal and $\ell_s[n] = f(nT_s)$ the discrete signal obtained by sampling the continuous signal. We are interested in knowing how the Fourier transform of $\ell_s[n]$ is related to the Fourier transform of $\ell(t)$.

Let's start writing a model of the sampling process by defining a very special signal: the **delta train** distribution (also known as the **impulse**

train, or **Dirac comb**). The delta train, $\delta_{T_s}(t)$, is a periodic signal, with period T_s , composed of delta impulses centered at times nT_s . It is defined as:

$$\delta_{T_s}(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT_s) \quad (20.1)$$

When $T_s = 1$, the delta train has the following shape:

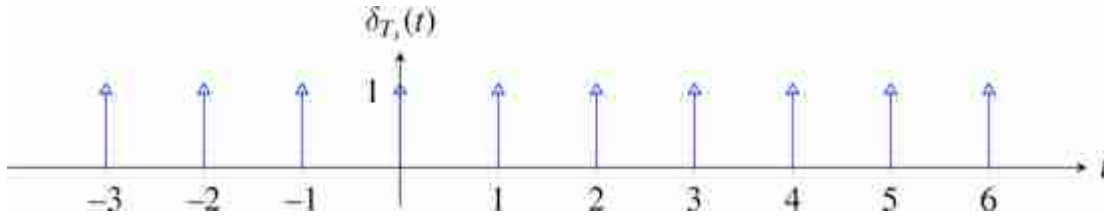


Figure 20.6: Delta train with period $T_s = 1$. The arrows show when the train's value is infinite. The height of each impulse represents its area.

The delta train can be used to model the sampling process. Remember the sampling property of the delta distribution $\ell(t)\delta(t - a) = \ell(a)$. If we multiply a function, $\ell(t)$, by the delta train, $\delta_{T_s}(t)$, we obtain its sampled version at times nT_s , which we will denote as $\ell_\delta(t)$:

$$\ell_\delta(t) = \ell(t)\delta_{T_s}(t) = \ell(t) \sum_{n=-\infty}^{\infty} \delta(t - nT_s) = \sum_{n=-\infty}^{\infty} \ell_s[n] \delta(t - nT_s) \quad (20.2)$$

The two plots in [figure 20.7](#) show a continuous function and its sampled version using the delta train.

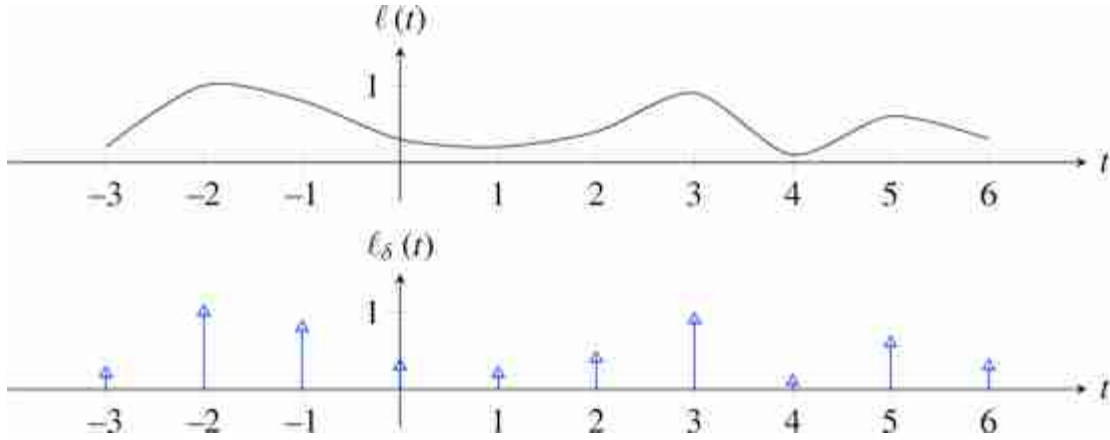


Figure 20.7: Sampling a signal using the delta train.

The delta sampled function $\ell_\delta(t)$ is a very special function that contains the same amount of information as the discrete signal $\ell_s[n]$ but that is defined over the continuous domain t . In fact, the Fourier transform of $\ell_\delta(t)$ and $\ell_s[n]$ are closely related as we will show next.

As $\ell_s[n]$ is a infinite length discrete signal, its discrete Fourier transform (DFT) is:

$$\mathcal{L}_s(w) = \sum_{n=-\infty}^{\infty} \ell_s[n] \exp(-jwn) \quad (20.3)$$

and the continuous Fourier transform of $\ell_\delta(t)$, an infinite length continuous signal, is:

$$\mathcal{L}_\delta(w) = \int_{t=-\infty}^{\infty} \ell_\delta(t) \exp(-j\omega t) dt \quad (20.4)$$

$$= \int_{t=-\infty}^{\infty} \ell(t) \sum_{n=-\infty}^{\infty} \delta(t - nT_s) \exp(-j\omega t) dt \quad (20.5)$$

$$= \sum_{n=-\infty}^{\infty} \int_{t=-\infty}^{\infty} \ell(t) \delta(t - nT_s) \exp(-j\omega t) dt \quad (20.6)$$

$$= \sum_{n=-\infty}^{\infty} \ell(nT_s) \exp(-j\omega nT_s) \quad (20.7)$$

Comparing equations 20.3 and 20.7 we see that both Fourier transforms are identical up to a scaling factor:

$$\mathcal{L}_s(w) = \mathcal{L}_\delta\left(\frac{w}{T_s}\right). \quad (20.8)$$

In practice, we will never directly work with the signal $\ell_\delta(t)$, but it is a convenient construction to understand how information is transformed during the sampling process. Understanding how the sampling rate T_s affects $\ell_\delta(t)$ is equivalent to understanding the effect that the sampling rate has in $\ell_s[n]$.

20.3.2 Sampling in the Fourier Domain

Now that we have seen that $\ell_\delta(t)$ and the discrete signal $\ell_s[n]$ have similar Fourier transforms, let's compute the Fourier transform of $\ell_\delta(t)$. Note that $\ell_\delta(t)$ is the product of two continuous signals. The first term is the continuous signal $\ell(t)$, and the second term is a function composed of impulses placed at regular time instants $\delta(t - nT_s)$. We can compute the continuous Fourier transform of $\ell_\delta(t)$ as the convolution of the Fourier transforms of $\ell(t)$ and $\delta_{T_s}(t)$.

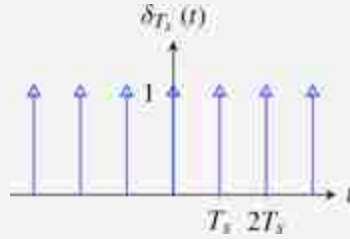
The Fourier transform of a delta comb is:

$$\begin{aligned} \Delta_{T_s}(w) &= \int_{-\infty}^{\infty} \delta_{T_s}(t) \exp(-j\omega t) dt \\ &= \sum_{n=-\infty}^{\infty} \int_{-\infty}^{\infty} \delta(t - nT_s) \exp(-j\omega t) dt \\ &= \sum_{n=-\infty}^{\infty} \exp(-j\omega nT_s) \\ &= \frac{2\pi}{T_s} \sum_{k=-\infty}^{\infty} \delta\left(w - k\frac{2\pi}{T_s}\right) \end{aligned} \quad (20.9)$$

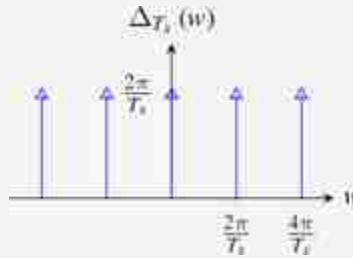
Deriving the last step is a bit involved and we invite the readers to work out the details. The result is that the Fourier transform of an impulse train is also an impulse train but with a displacement in frequency between impulses that grows when the spacing in time decreases:

$$\delta_{T_s}(t) \xrightarrow{\mathcal{F}} \delta_{\frac{2\pi}{T_s}}(w) \quad (20.10)$$

The delta train with period T_s is:



and its Fourier transform is:



Therefore, the continuous Fourier transform of $\ell_\delta(t)$ can be written as:

$$\mathcal{L}_\delta(w) = \mathcal{L}(w) \circ \frac{2\pi}{T_s} \sum_{k=-\infty}^{\infty} \delta\left(w - k \frac{2\pi}{T_s}\right) = \frac{2\pi}{T_s} \sum_{k=-\infty}^{\infty} \mathcal{L}\left(w - k \frac{2\pi}{T_s}\right) \quad (20.11)$$

where $\mathcal{L}(w)$ is the Fourier transform of $\ell(t)$. This equation shows that $\mathcal{L}_\delta(w)$ is built as an infinite sum of translated copies of $\mathcal{L}(w)$, as shown in [figure 20.8\(a\)](#). Each copy is centered on $k \frac{2\pi}{T_s}$. If T_s is small (i.e., if we sample very fast) then those copies will be far away from each other ([figure 20.8\[a\]](#)). But if we have few samples and T_s is large, those copies will get very close and will start mixing with each other ([figure 20.8\[b\]](#)). High frequency content in $\mathcal{L}(w)$ will affect the low frequency content of $\mathcal{L}_\delta(w)$, and this is exactly what produces aliasing.

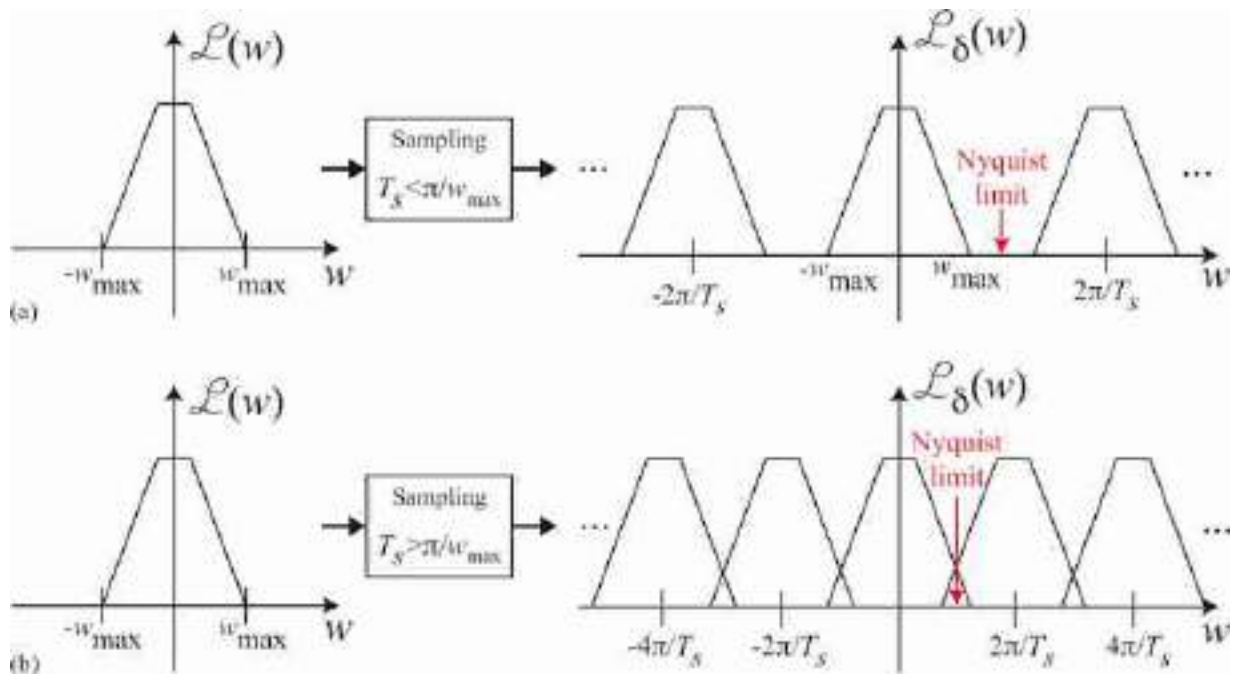


Figure 20.8: Sketch to illustrate aliasing. Example of a band-limited signal, with frequency content only inside the interval $(-w_{\max}, w_{\max})$. (a) Sampled with a sampling period such a that $T_s < \pi/w_{\max}$. (b) Sampled with a period $T_s > \pi/w_{\max}$. Aliasing is due to the overlap between the translated copies of the signal Fourier transform, $\mathcal{A}(w)$.

The Nyquist limit is reached when the maximum frequency, $w_{\max}$, with spectral content in $\mathcal{A}(w)$ overlaps with its translated copy. This overlap occurs when the sampling period, T_s , is $T_s > \pi/w_{\max}$. To avoid aliasing, the sampling period has to be shorter than $\pi/w_{\max}$.

20.4 Reconstruction

The reconstruction process consists in recovering a continuous signal from its samples by interpolation. In this section we will study what the ideal interpolation scheme is and what other practical interpolations exist.

20.5 Ideal Reconstruction

When the Nyquist condition is satisfied, we know that it is possible to reconstruct the original continuous signal from the samples. We just need to

apply a box filter that has a constant gain for all the frequencies inside $w \in [-\pi/T_s, \pi/T_s]$, and 0 outside. The phase of the filter should be zero. That is,

$$H(w) = \begin{cases} \frac{T_s}{2\pi} & \text{if } w \in [-\pi/T_s, \pi/T_s] \\ 0 & \text{otherwise} \end{cases} \quad (20.12)$$

The impulse response of such a filter is $h(t) = \text{sinc}(t/T_s)$ where the **sinc** function is:

$$\text{sinc}(t) = \frac{\sin(\pi t)}{\pi t} \quad (20.13)$$

The sinc function ([figure 20.9](#)) it is an infinite length continuous signal that has its maximum at the origin, $\text{sinc}(0) = 1$, it is symmetrical, $\text{sinc}(t) = \text{sinc}(-t)$, and it is a sinusoidal wave with a period of 2 that decays in amplitude at a rate $1/t$ as shown in this plot:

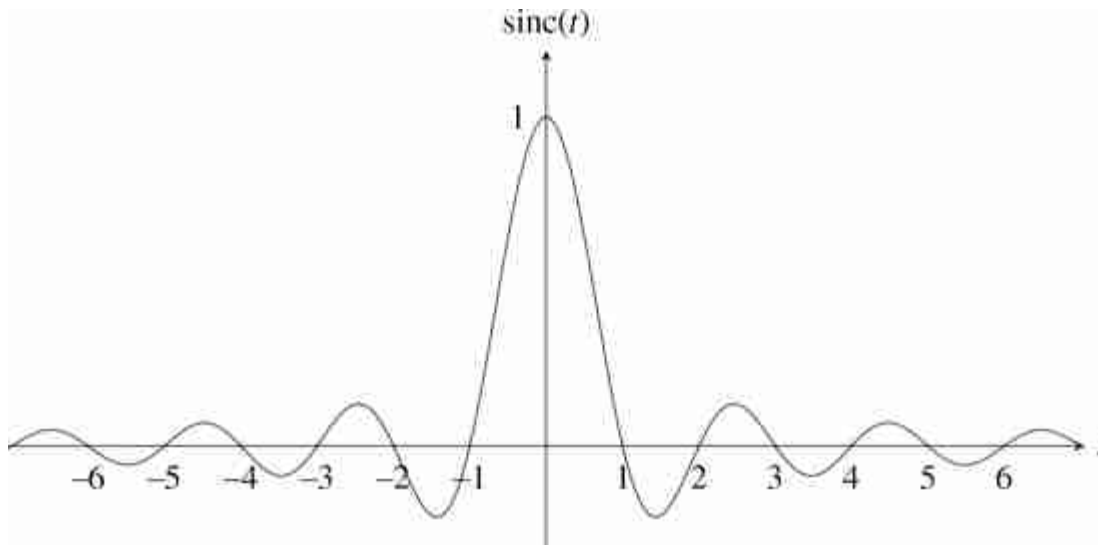


Figure 20.9: Sinc function. This signal is a modulated sine signal with an amplitude decay of $1/t$. The frequency is normalized so that the zero crossings happen at integer values.

When no other prior information is available about the function $\ell(t)$, and under the slow and smooth prior, this is the optimal reconstruction in terms of the L2 norm:

$$\text{sinc}(t/T_s) = \arg \min_{h(t)} \int (\ell(t) - \ell_\delta(t) \circ h(t))^2 dt = \arg \min_{h(t)} \int (\mathcal{L}(w) - \mathcal{L}_\delta(w)H(w))^2 dw \quad (20.14)$$

Then the function, $\ell_\delta(t)$, that best approximates the input signal from its samples is:

$$\tilde{\ell}(t) = \ell_\delta(t) \circ \text{sinc}(t) = \sum_{n=-\infty}^{\infty} \ell_s[n] \text{sinc}\left(\frac{t - nT_s}{T_s}\right) \quad (20.15)$$

where $\tilde{\ell}(t)$ is the reconstructed signal from the samples $\ell_s[n]$. The plot in [figure 20.10](#) shows how the linear superposition of all the scaled and shifted sincs produces a smooth interpolation of the sampled function. Note how the zero crossings of each sinc coincide with the sample locations.

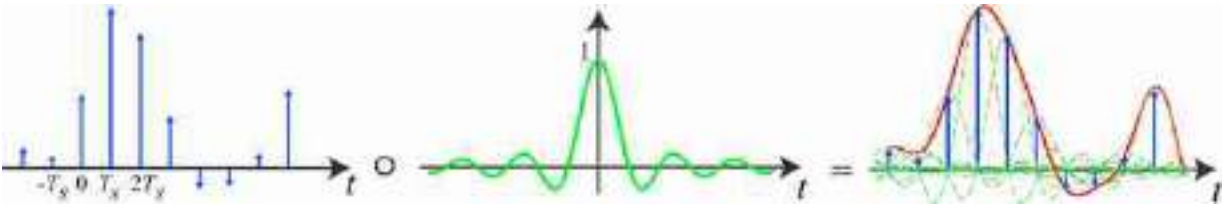


Figure 20.10: Smooth interpolation of the sampled function using sinc functions.

[Figure 20.11](#) illustrates how the reconstruction degrades as a function of the sampling rate. In this example, there is one band-limited input signal (i.e., there is a frequency, w_{max} , for which the magnitude of the Fourier transform is zero for all frequencies above w_{max}). When we sample this signal with a sampling period of $T_s = 4$ s, in its FT we see replicates of the $\mathcal{A}(w)$ centered around $\pi/2$. With $T_s = 8$ s, the replicates appear centered around $\pi/4$ and they start touching. Note that $T_s = 8$ is slightly above the Nyquist's limit and some aliasing will exist. For $T_s = 16$ aliasing is severe and information will be lost, making it impossible (without any additional prior information) to reconstruct the continuous function from its samples.

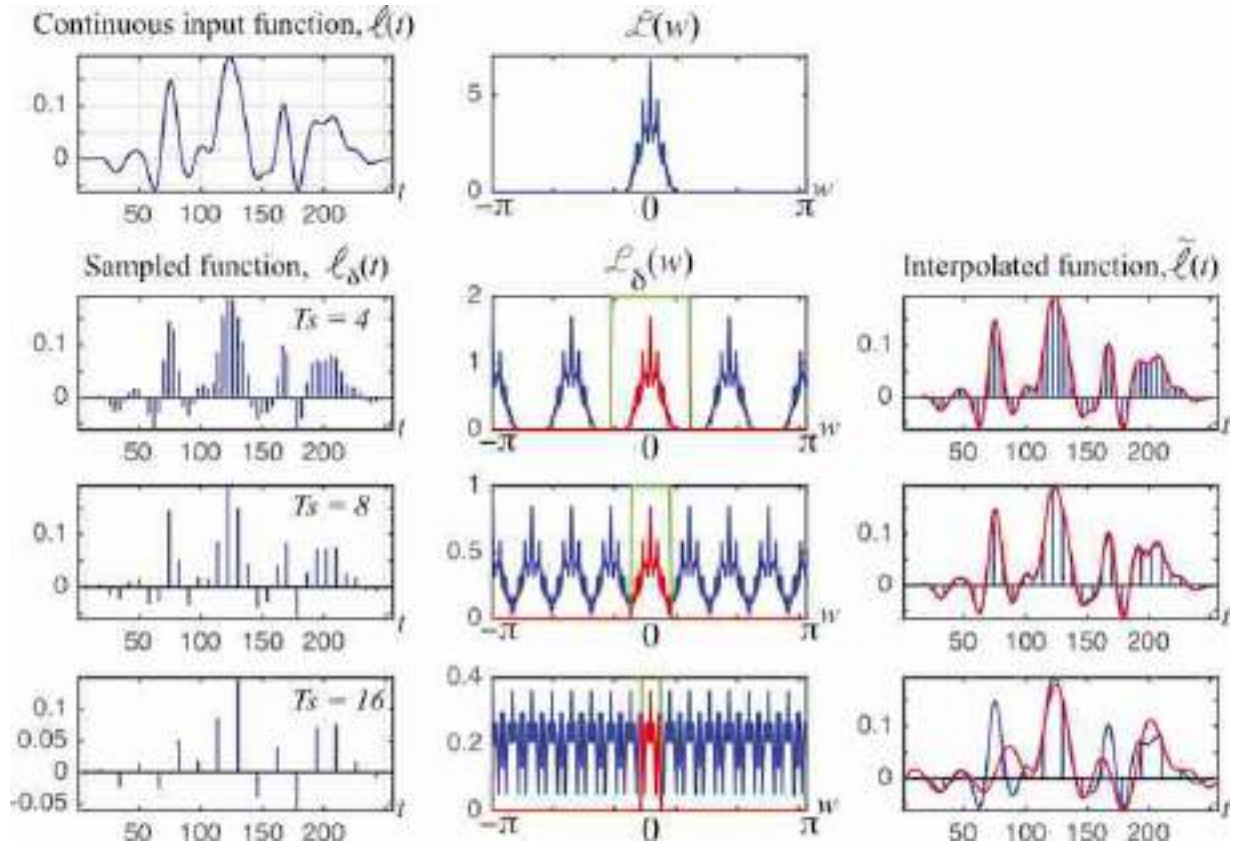


Figure 20.11: (left column) Spatial sampling pattern. (middle column) Fourier transform of that spatial pattern, revealing replication locations of the Fourier transform spectrum of the subsampled signal. (right column) Subsampled signal. Zeroing out all but the central replication of the image spectrum yields the interpolated signal shown in red.

A signal cannot be simultaneously time limited and frequency band limited. For any time limited signal (i.e., a signal that is defined inside an interval $t \in [a, b]$ and it is zero outside) the Fourier transform is not band limited.

All the derivations extend naturally to 2D because all the functions are separable along the two spatial dimensions. In two dimensions, the ideal interpolation function is:

$$\text{sinc}(x, y) = \frac{\sin(\pi x)}{\pi x} \frac{\sin(\pi y)}{\pi y} \quad (20.16)$$

We will discuss the 2D case a bit more in detail later as sampling images opens the door to different sampling strategies beyond simply adjusting the sampling rate.

20.5.1 Approximate Reconstruction with Local Kernels

One disadvantage of the ideal reconstruction is that the sinc function has infinite support which means that to interpolate each instant we need to linearly combine all the samples $\ell[n]$. Sometimes it is better to have a local reconstruction that only depends on the nearby samples.

Indeed, there are other possible reconstructions that are not optimal in terms of the L2 norm, but that only require local computations.

In 1D, some of the most popular interpolation kernels are nearest interpolation, and linear. Nearest interpolation consists in assigning to each time instant the value of the nearest sample in time. Linear interpolation consist in interpolating linearly between two consecutive values.

All of them can be written as a linear convolution with a kernel $h(t)$ as shown in [figure 20.12](#):

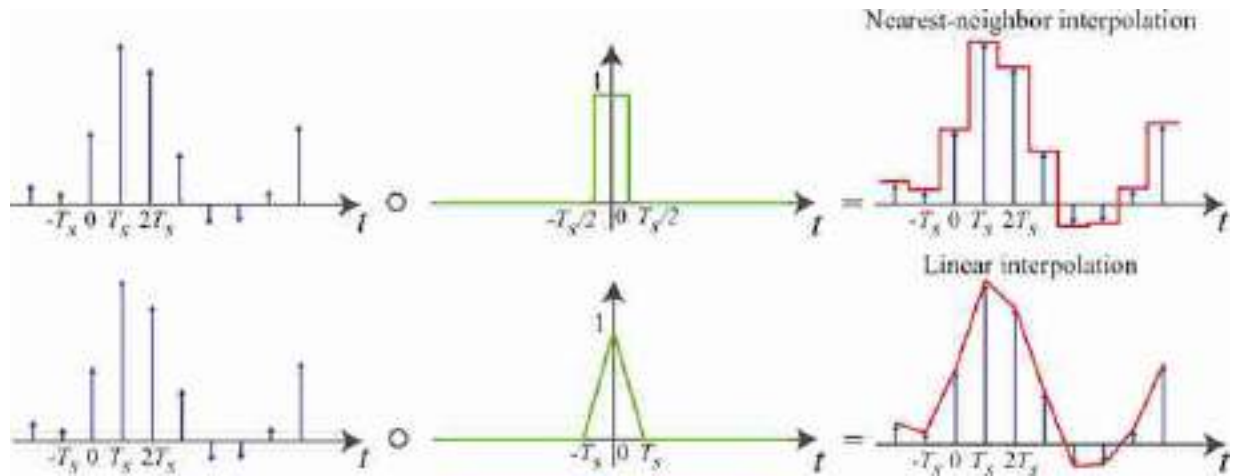


Figure 20.12: (top) Nearest interpolation. (bottom) Linear interpolation. Both interpolation methods can be modeled by a convolution with the kernel shown in the middle (a box and a triangle).

The nearest neighbor interpolation is the result of the convolution of $\ell_\delta(t)$ with a box of width T_s . The linear interpolation, the interpolation kernel $h(t)$ is a triangle of width $2T_s$.⁸ Lanczos-1 interpolation consists in using as kernel only the central lobe of the sinc function. This provides an interpolation that is smooth using only a local neighborhood. Other interpolation methods such as cubic interpolation can also be written as convolutions.

20.6 A Family of 2D Spatial Samplings

Let's now analyze what happens when sampling 2D signals to form discrete images. In 2D things get more interesting. If we have a continuous image $\ell(x, y)$ we can sample it using a rectangular grid as $\ell_s[n, m] = \ell(nT_x, mT_y)$. We can do a very similar analysis to the one we just did for the 1D case. But in 2D we can have more interesting sampling patterns. For instance, we could define the discrete image as:

$$\ell_s[n, m] = \ell(an + bm, cn + dm) \quad (20.17)$$

where a, b, c, d are constants. For instance, if $a = T, b = 0, c = 0, d = T$ then we will have a regular **rectangular sampling**. But we could have other patterns. For instance, if we set $a = T_1, b = -T_2/2, c = 0, d = T_2$ then we obtain an **hexagonal sampling**. We can also place the samples at random locations (**irregular sampling**). [Figure 20.13](#) shows different sampling patterns.

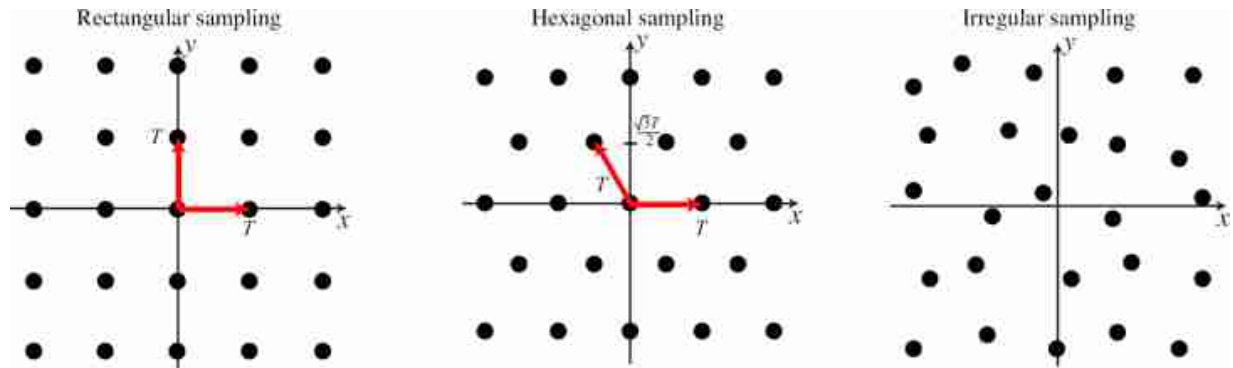


Figure 20.13: Three types of sampling: rectangular grid, hexagonal, and irregular.

Now we can ask the following question: What is the optimal 2D sample arrangement given a fixed number of samples? What we want is to choose the sample arrangement that will allow the best reconstruction of the input continuous signal from a fixed number of samples. As we did with the 1D case, we can answer this question by studying the relationship between the Fourier transform of the continuous signal and the sampled one, which will reveal how aliasing happens and what sampling pattern minimizes it.

We model sampling by multiplying the continuous image by a delta train in 2D:

$$\ell_{\delta}(x, y) = \ell(x, y) \sum_{n=-\infty}^{\infty} \sum_{m=-\infty}^{\infty} \delta(x - an - bm, y - cn - dm) \quad (20.18)$$

We can simplify the notation by encoding the position as vectors, $\mathbf{x} = (x, y)^T$, $\mathbf{n} = (n, m)^T$, and $\mathbf{A}$ is the matrix:

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad (20.19)$$

where the 2D delta train can be written using vector notation for the continuous spatial coordinates:

$$\delta_{\mathbf{A}}(\mathbf{x}) = \sum_{\mathbf{n} \in \mathbb{Z}^2} \delta(\mathbf{x} - \mathbf{A}\mathbf{n}) \quad (20.20)$$

The continuous Fourier transform of this delta train can be done by applying a change in variables and then using a similar procedure as the one followed in the 1D case. The result is:

$$\Delta_{\mathbf{A}}(\mathbf{w}) = \frac{(2\pi)^2}{|\mathbf{A}|} \sum_{\mathbf{k} \in \mathbb{Z}^2} \delta(\mathbf{w} - 2\pi \mathbf{A}^{-1} \mathbf{k}) \quad (20.21)$$

Therefore, the Fourier transform of the sampled signal $\ell_{\delta}(x, y)$ is:

$$\mathcal{L}_{\delta}(\mathbf{w}) = \frac{(2\pi)^2}{|\mathbf{A}|} \sum_{\mathbf{k} \in \mathbb{Z}^2} \mathcal{L}(\mathbf{w} - 2\pi \mathbf{A}^{-1} \mathbf{k}) \quad (20.22)$$

Remember that for 2×2 matrices the inverse is easy to write:

$$\mathbf{A}^{-1} = \frac{1}{|\mathbf{A}|} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \quad (20.23)$$

We can now check what happens with different sampling strategies. For instance, the 2D rectangular sampling, equation (20.22), simplifies to:

$$\mathcal{L}_{\delta}(w_x, w_y) = \frac{(2\pi)^2}{T^2} \sum_{k_1=-\infty}^{\infty} \sum_{k_2=-\infty}^{\infty} \mathcal{L}\left(w_x - \frac{2\pi}{T} k_1, w_y - \frac{2\pi}{T} k_2\right) \quad (20.24)$$

This is similar to the 1D case. We leave to the reader to write down the form of the hexagonal sampling.

[Figure 20.14](#) shows a sketch of the Fourier transforms of the rectangular and hexagonal samplings. The region delimited by the red polygon shows the region of valid frequencies. If the input signal only has spectral content within that region, then there will be no aliasing. The optimal sampling will be the one that makes that region as large as possible for a fixed number of samples.

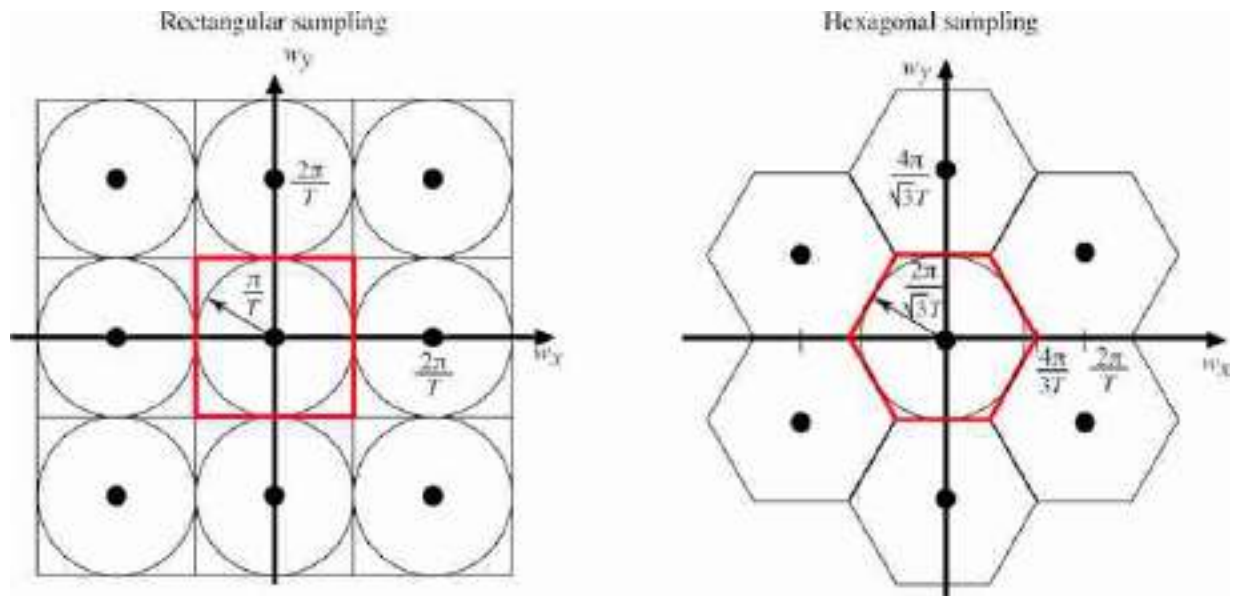


Figure 20.14: Sketch of the Fourier transforms of the rectangular and hexagonal samplings. The red boundary denotes the spectral content that gets periodically repeated.

The optimal sampling strategy is the regular hexagonal sampling. This is not the sampling used in computer vision as all images are always represented on a rectangular grid, but an hexagonal sampling achieves an increase of around 10 percent in resolution for the same amount of samples. In fact, the distribution of photoreceptors at the center of the retina [89, 88] are distributed closely resembling an hexagonal array over small patches as shown in [figure 20.15](#). Working with convolutional filters defined over an hexagonal grid is more efficient and it can achieve better radial symmetry [332, 443].

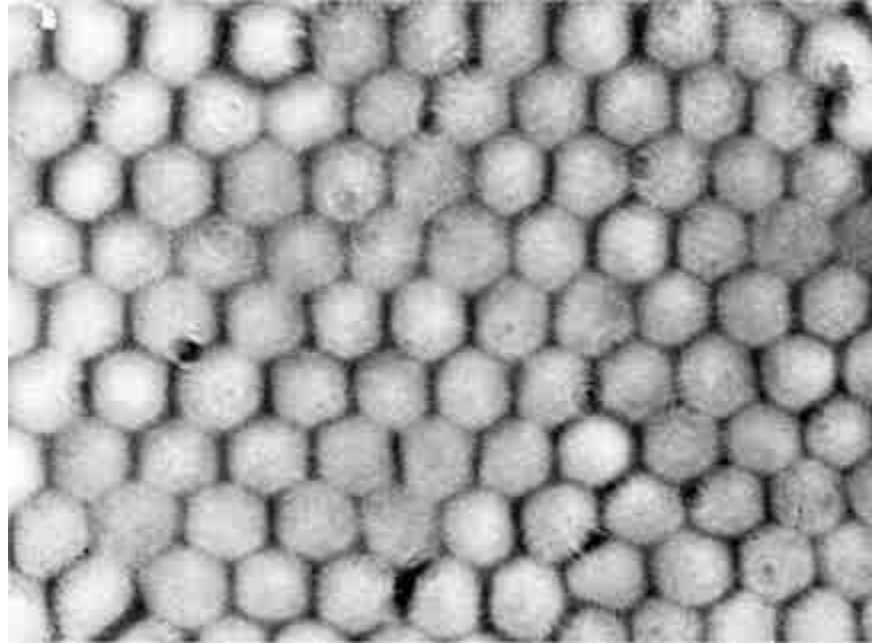


Figure 20.15: Distributions of cones in the fovea of a Monkey [88]. The image shows a cross-section of the retina at the level of the photoreceptors. *Source:* [88].

The analysis we have presented in this chapter assumes signals of infinite length. However, images will have a finite size. The same results can be applied by extending the image to infinity with zero padding.

20.7 Anti-Aliasing Filter

Sampling with the wrong frequency has interesting effects in 2D. [Figure 20.16\(a\)](#) shows an example of a picture downsampled at different resolutions (412×512 , 103×128 , 52×64 , and 26×32) and then reconstructed to the original resolution (412×512 pixels). For the figures, as we do not have access to the continuous image, we always work with sampled versions. But the original image is very high resolution and we can think of it as being the continuous image.

The images in [figure 20.16\(a\)](#) show the effects of aliasing. The stripes in the zebra's body change orientation as we downsample them. And for the lowest resolution image, it is even hard to recognize the animal as being a zebra. [Figure 20.16\(b\)](#) shows what happens with the image Fourier transform when we multiply the image with the delta train, with a one every four samples along each dimension (second column of [figure 20.16\(b\)](#)).

[Figure 20.16](#)(c) shows the magnitude of the DFT of the sampled image (it corresponds to the region inside the green square in [figure 20.16](#)(b)). The DFT changes substantially, due to aliasing, from one resolution to the next one.

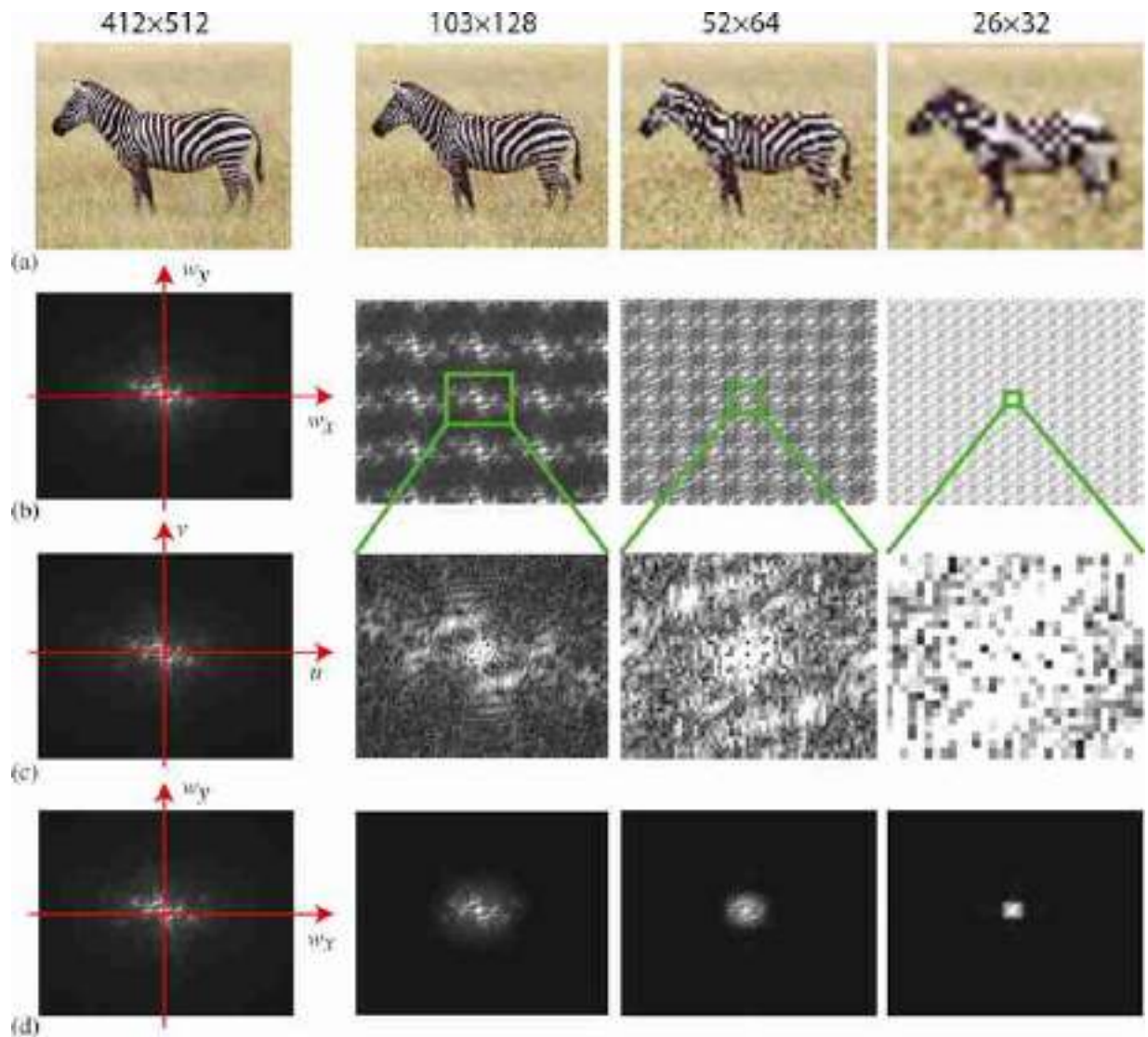


Figure 20.16: Subsampling with aliasing. (a) The zebra sampled with aliasing starts looking like a cow. (b) Fourier transform of the continuous signal $\ell(x, y)$ multiplied by delta trains: $\ell_{\delta}(x, y)$. (c) Discrete Fourier transform of the corresponding sampled signals, $\ell[n, m]$. (d) Fourier transform of the reconstructed signal with aliasing. (e) Sampled image after processing it with an anti-aliasing filter. (f) Discrete Fourier transform of the corresponding anti-aliased sampled images, $\ell[n, m]$. Note that now the central part of the Fourier transform is not changing.

In order to reduce aliasing artifacts we need to filter the continuous signal with a low-pass filter in order to make it band-limited. Then we will be able to sample it avoiding high-spatial frequencies to interfere with the low-frequency content of the image. The anti-aliasing filter will not prevent the loss of information contained in the high spatial frequencies. [Figure 20.17\(a\)](#) shows the reconstructed images at different resolutions when an

anti-aliasing filter is applied before sampling. Each resolution requires a different filter. The anti-aliasing filter can be a box filter like in equation (20.12) with the support equal to the green region in [figure 20.16\(b\)](#).

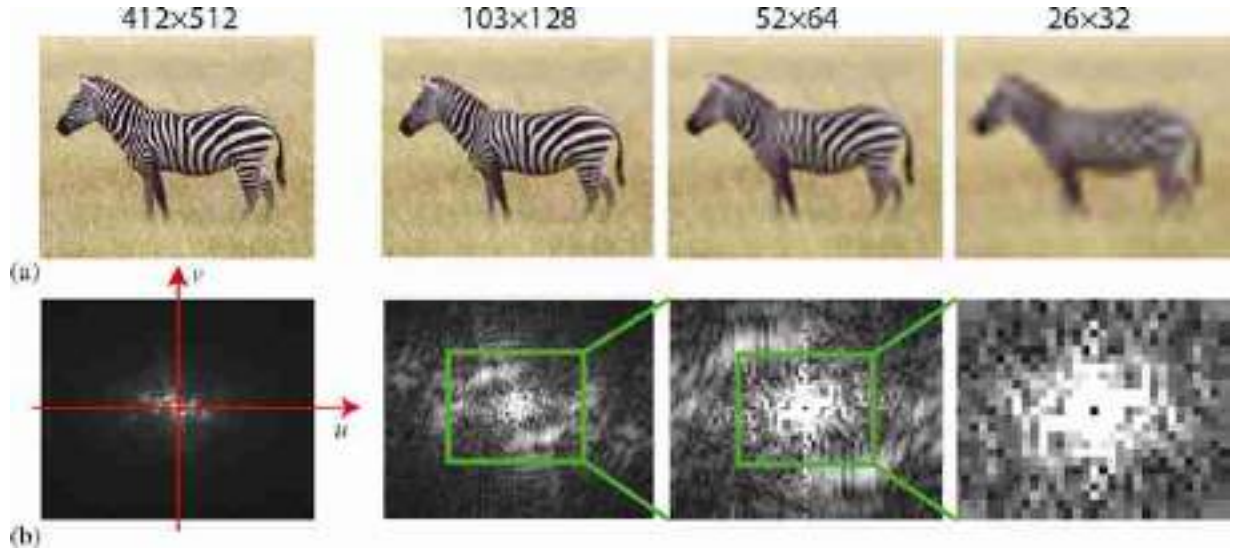


Figure 20.17: Subsampling with anti-aliasing filter. (a) Sampled image after processing it with an anti-aliasing filter. (b) Discrete Fourier transform of the corresponding anti-aliased sampled images, ℓ $[n, m]$. Note that now the central part of the Fourier transform is almost not changing.

20.8 Spatiotemporal Sampling

Spatiotemporal sampling can be studied with the tools we have already seen. In particular, equation (20.22) can explain sampling in the N -dimensional case. Temporal aliasing is responsible of the typical illusion in which we see wheels or fans changing the sense of rotation in movies. To avoid those artifacts it is important to apply an anti-aliasing filter as before.

The most common type of spatiotemporal sampling is regular sampling: here, all the pixels are sampled in a regular spatial 2D grid and at regular time instants in which all the pixels are exposed simultaneously. This is also called global shutter mode.

In many cameras (DSLRs, mobile phones, etc.) the most commonly used sampling is the rolling shutter mode. Here, every row in the image is sampled simultaneously. But different rows are sampled at different instants. This sampling mode allows for a faster sampling rate with current

hardware implementations but it can create spatial distortions in the image if the camera moves or when taking pictures of moving objects.

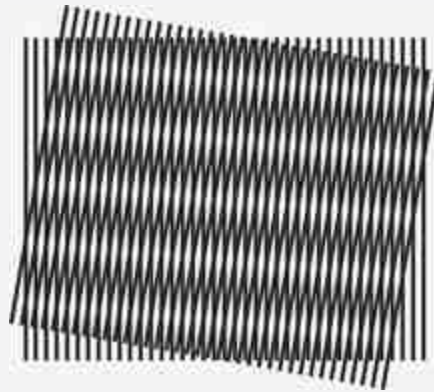
20.9 Concluding Remarks

Aliasing is a common artifact that appears whenever an image is captured by sampling a continuous light field. As we will discuss in the next chapter, aliasing also affects images whenever there is upsampling or downsampling. Therefore, understanding how aliasing introduces artifacts and the best way to avoid it is an important skill.

However, aliasing is not always bad and it might be possible to use aliasing to recover high-frequency content that would be otherwise lost. Superresolution algorithms can learn to extract, from the aliasing pattern, fine image details.

Also, when an image is encoded by multiple channels, aliasing in one channel does not mean that the information is lost because multiple channels could contain complementary information that allows the recovery of high resolution details.

Moire patterns, also related to aliasing, are interference patterns that appear when superposing semitransparent patterns on top of each other.



[8.](#) The convolution of two box filters of width $T_s/2$ is a triangle of length T_s .

21 Downsampling and Upsampling Images

21.1 Introduction

Changing the resolution of an image, or a feature map, is an operation you are likely to do many times, hence it will be worth expending some time understanding all its subtleties. Whether it is for resizing a picture, or to change the dimensionality of a representation, changing the sample-rate of a signal is a common operation. Resizing an image might seem like a trivial operation and often it is carelessly applied, producing results that are far from optimal. As discussed in the previous chapter, aliasing reduces the quality of the image. Aliasing is a visual phenomenon whereby the appearance of an image can change drastically after subsampling. In the previous chapter we discussed the case when the input is a continuous signal and the output is its discretized image.

21.2 Example: Aliasing-Based Adversarial Attack

Aliasing in a system has consequences beyond the aesthetics of the final output. To illustrate the importance of aliasing when building vision systems in practice, let's consider the following simple system [407]:

$$u[n, m] = \ell_{\text{in}}[n, m] \odot [-1, 1] \quad \triangleleft \text{conv} \quad (21.1)$$

$$h[n, m] = u[2n, 2m] \quad \triangleleft \text{decimation} \quad (21.2)$$

$$\ell_{\text{out}}[n, m] = \max(0, h[n, m]) \quad \triangleleft \text{relu} \quad (21.3)$$

This system has three steps: the first step is a vertical edge filter (convolution of the input, ℓ_{in} , with a horizontal kernel, $[1, -1]$). The next step is to reduce the resolution of the filter output by taking one pixel every two along each dimension (this operation is called decimation). Decimation takes an input, u , with size $N \times M$, and outputs, h , with half the size $N/2 \times M/2$. Finally, the third step is a relu nonlinearity.

A convolution followed by decimation is also called **strided convolution** in the neural networks literature.

As we will discuss in this chapter, the decimation step introduces **aliasing**. As illustrated in [figure 21.1](#), an attacker could use aliasing in a system to infiltrate a signal hidden in the high-spatial frequencies that could change the output in unexpected ways. In this toy example, the construction of the attack is as follows. Let $\ell_{\text{in}}[n, m]$ be the original image (e.g., a picture of a bird), and $\ell_{\text{hidden}}[n, m]$ be the hidden image (e.g., a picture of a stop sign). The attack $z[n, m]$ is given by

$$z[n, m] = (-1)^n (-1)^m \ell_{\text{hidden}}[n, m] \quad (21.4)$$

The attacked input image is $\ell_{\text{in}}[n, m] + \epsilon z[n, m]$, where ϵ is the chosen adversarial strength. In this example we used $\epsilon = 0.05$. As we can see in [figure 21.1](#), the attacked input image is indistinguishable from the original input, $\ell_{\text{in}}[n, m]$, and yet their outputs are completely different. The culprit for this bizarre effect is the strided convolution that samples the output for one pixel every two. An attacker with knowledge about it is able to construct a perturbation focused on manipulating the surviving samples (pixels at even rows and columns); the output at the discarded samples (pixels at an odd row or column) do not matter, or may even serve as extra degrees of freedom that can be used to make the attack less noticeable.

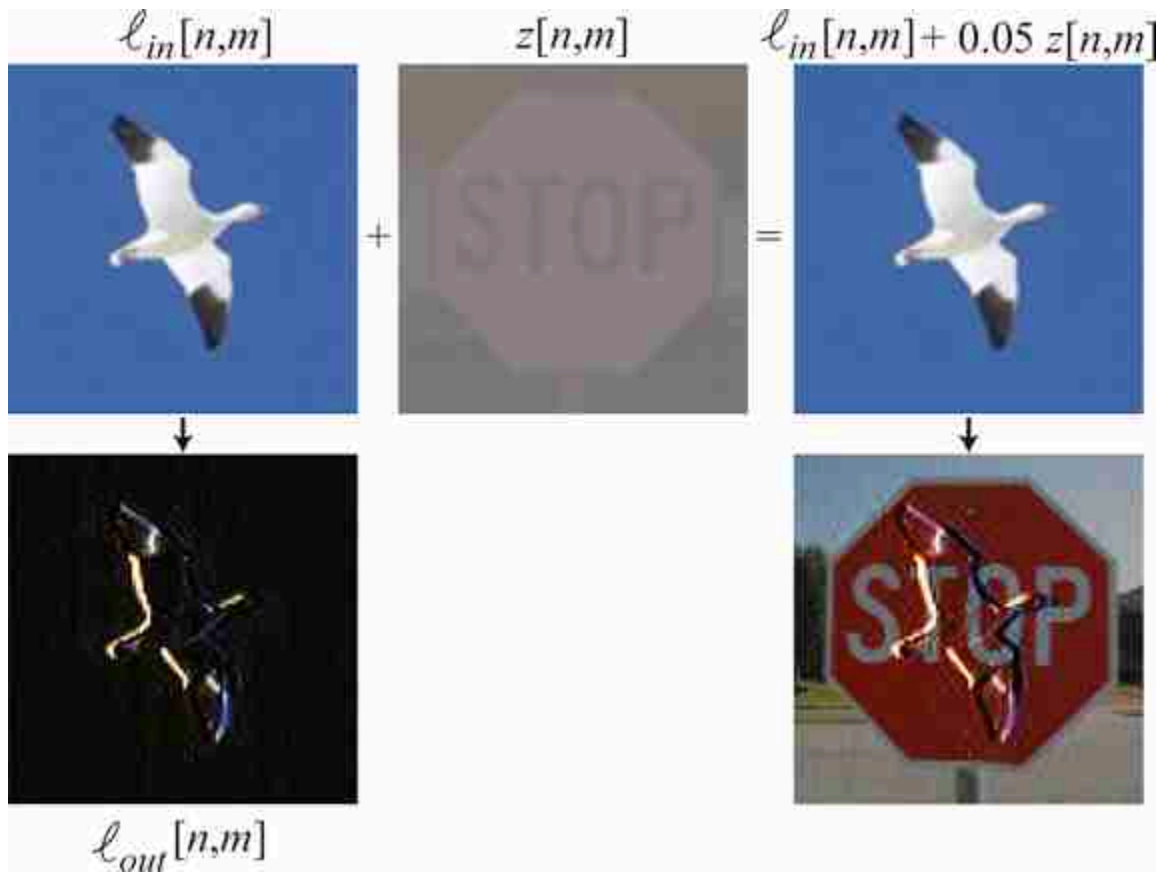


Figure 21.1: Analytic aliasing-based attack of a toy system that computes the horizontal image gradient [407]. Input image, ℓ_{in} , and output image of $\ell_{out} = \max(0, \ell_{in} \circ_2 [1, -1])$, where $\circ_2$ denotes convolution with a stride of 2. As this simple system has aliasing, an attacker can inject a high frequency pattern, z , to change the output. Attacked image, $\ell_{in} + z$, and the output after attack. It doesn't even look like a derivative!

In this chapter we will study aliasing when manipulating image resolution and how to reduce it.

21.3 Downsampling

Downsampling is an operation that consists of reducing the resolution of an image. When we reduce the resolution, some information will be lost; specifically fine image details will be lost. But carelessly removing pixels will produce a greater loss of information than necessary. The goal of this chapter will be to study how downsampling has to be done to minimize the amount of information lost.

21.3.1 Decimation

Image decimation consists of changing the resolution of an image by dropping samples. Decimation of an image of size $N \times M$ by a factor of k , where N and M are divisible by k , results in a lower resolution image of size $N/k \times M/k$:

$$\ell_{\downarrow k}[n, m] = \ell[kn, km] \quad (21.5)$$

It is common to use the notations $\ell_{\downarrow k}[n, m] = \ell[n, m] \downarrow k = \ell[kn, km]$ to denote the decimation operator by a factor k . Decimation could be different across each dimension, but for the analysis here we will assume that both dimensions are treated equally, as it is usually done.

Decimation is often drawn using the following block:



Decimation is a linear operation therefore we can write it as a matrix. In the one-dimensional (1D) case, for a signal of length 8, subsampling by a factor of 2 can be described by the following linear operator:

$$\mathbf{D}_2 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad (21.6)$$

The [figure 21.2](#) shows an example of a picture and two decimated images with factors $k = 2$ and $k = 4$. For visualization purposes, the images are scaled so that they occupy the same area.



Figure 21.2: Input image (left) and two decimated versions of the same image with factors $k = 2$ and $k = 4$.

As we downsample the image, we lose details in the texture such as the orientation of the knitting, and it becomes harder to see the overall shape. Just reducing resolution by a factor of 4 results in an almost unintelligible image. The loss of information is due to the reduced resolution, but most important, in this example, additional information is lost due to aliasing. Aliasing has introduced new texture features in the subsampled image that are not present in the original image and that could have been avoided.

How can we reduce the resolution of an image while preserving as many details as possible?

21.3.2 Decimation in the Fourier Domain

As we did before for the continuous case, analyzing how the decimation operator affects the signal in the Fourier domain can be very revealing. In this section we compute the relationship between the Fourier transform (FT) of the original image, ℓ and the sampled one, $\ell_{\downarrow k}$. We will use the finite length discrete Fourier transform (DFT). If the DFT of $\ell[n, m]$ is $\mathcal{L}[u, v]$, the DFT of the sampled image, $\ell[kn, km]$ (with size $N/k \times M/k$), is:

$$\mathcal{L}_{\downarrow k}[u, v] = \sum_{n=0}^{N/k-1} \sum_{m=0}^{M/k-1} \ell[kn, km] \exp\left(-j2\pi \frac{nu}{N/k}\right) \exp\left(-j2\pi \frac{mv}{M/k}\right) \quad (21.7)$$

as before, we can define an image, $\ell_{\delta k}[n, m]$, of same size as the input image, $N \times M$, where we set to zero the samples that will be removed in the decimation operation:

$$\ell_{\delta_k}[n, m] = \ell[n, m] \sum_{s=0}^{N/k-1} \sum_{r=0}^{M/k-1} \delta[n - sk, m - rk] \quad (21.8)$$

$$= \ell[n, m] \delta_k[n, m] \quad (21.9)$$

where $\delta_k[n, m]$ is the discrete version of the **delta train**. Using ℓ_{δ_k} we can rewrite equation (21.7), with a change of the summation indices, as:

$$\mathcal{L}_{\downarrow k}[u, v] = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} \ell_{\delta_k}[n, m] \exp\left(-j2\pi \frac{nu}{N}\right) \exp\left(-j2\pi \frac{mv}{M}\right) \quad (21.10)$$

Therefore, the DFT of $\ell[kn, km]$ is the DFT of $\ell_{\delta_k}[n, m]$, which is the product of two signals, equation (21.9). The first term is $\ell[n, m]$, and the second term is a discrete delta train (also called the Kronecker comb):

$$\delta_k[n, m] = \sum_{s=0}^{N/k-1} \sum_{r=0}^{M/k-1} \delta[n - sk, m - rk] \quad (21.11)$$

The DFT of the discrete delta train, when M and N are divisible by k , is:

$$\Delta_k[u, v] = \frac{NM}{k^2} \sum_{s=0}^{k-1} \sum_{r=0}^{k-1} \delta\left[u - s\frac{N}{k}, v - r\frac{M}{k}\right] \quad (21.12)$$

This definition is valid for frequencies inside the interval $[u, v] \in [0, N-1] \times [0, M-1]$. Outside of that interval, we will have its periodic extension.

[Figure 21.3](#) shows $\delta_k[n, m]$ and its Fourier transform, $\Delta_k[u, v]$, in the bottom, for images size of 32×32 pixels. For the FT we plot signals using an interval that puts the frequency $[0, 0]$ in the center.

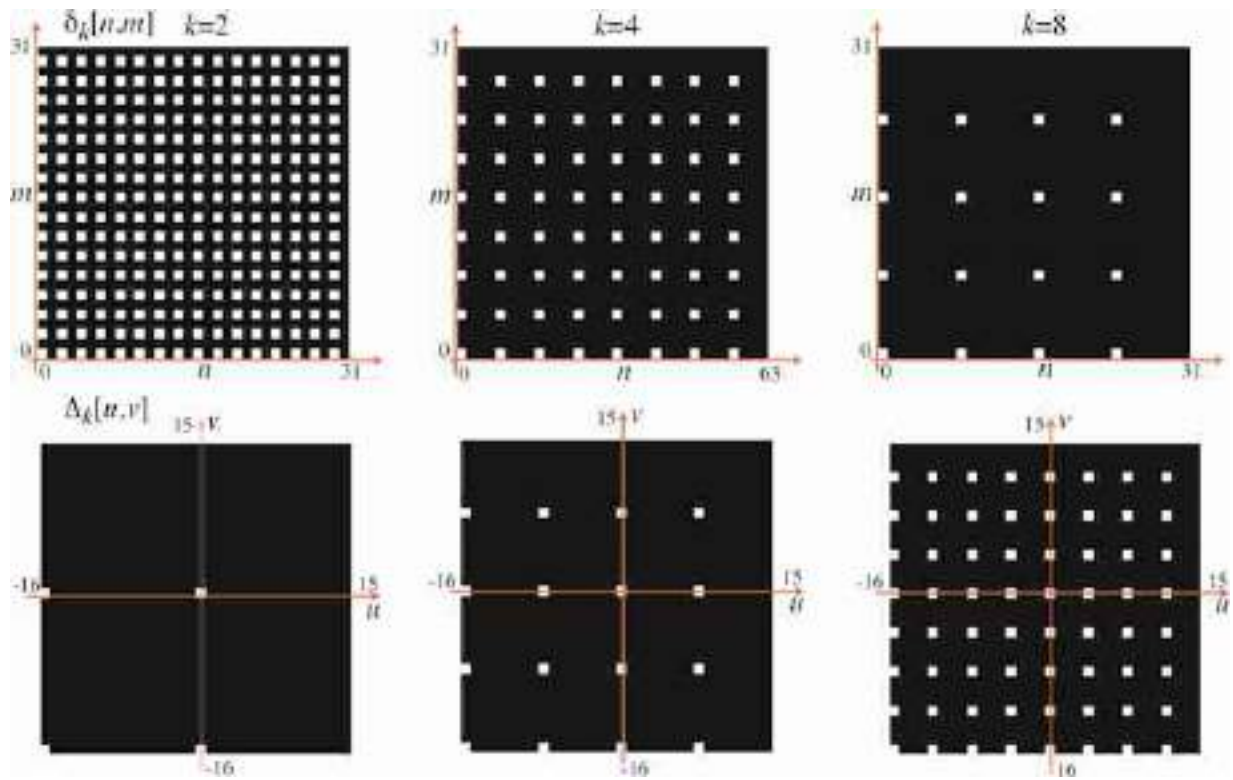


Figure 21.3: (top) Visualization of three discrete two-dimensional (2D) delta trains with $k = 2, 4, 8$. (bottom) Corresponding DFTs. The DFT of a delta train is another delta train.

Now we are ready to write the DFT of $x_{\delta_k}[n, m]$. This is the DFT of the product of two images, equation (21.9), so we can apply the dual of the circular convolution theorem for finite length discrete signals (where both signals are extended periodically):

$$\begin{aligned}\mathcal{L}_{\downarrow k}[u, v] &= \frac{1}{NM} \mathcal{L}[u, v] \circ \Delta_k[u, v] \\ &= \frac{1}{k^2} \sum_{s=0}^{k-1} \sum_{r=0}^{k-1} \mathcal{L}\left[u - s \frac{N}{k}, v - r \frac{M}{k}\right]\end{aligned}\quad (21.13)$$

The DFT of the sampled image by a factor k is the superposition of $k \times k$ shifted copies of the DFT of the original image. Each copy is centered on one of the deltas in $\Delta_k[u, v]$.

The images in [figure 21.4](#) show $\ell_{\delta_k}[n, m]$ and $\mathcal{L}_{\downarrow k}[u, v]$. The original image, $\ell[n, m]$, is 128×128 . The DFT of the sampled image, $\ell_{\downarrow k}[n, m]$, is the inner window with size $N/2k \times M/2k$ (shown as a green square).

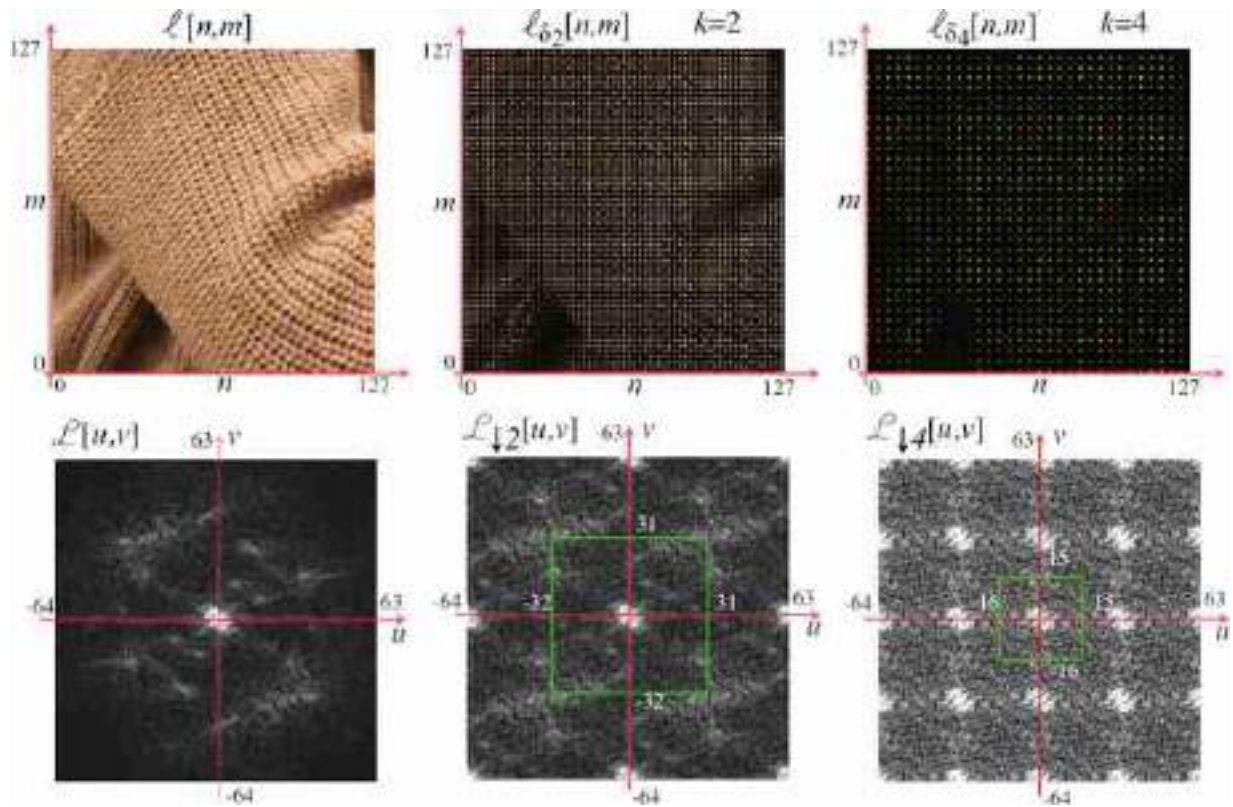


Figure 21.4: (top) Visualization of an image multiplied by delta trains, and (bottom) and their corresponding DFTs. Only the DFT magnitude is shown.

In the case of $k = 2$, it is useful to see how the spectrum of the decimated signal is obtained. As shown from equation (21.13), the resulting spectrum is a sum of shifted versions of the original spectrum, $\mathcal{L}[u, v]$, as shown in [figure 21.5](#). The image shows the four copies that are summed to obtain the decimated spectrum:

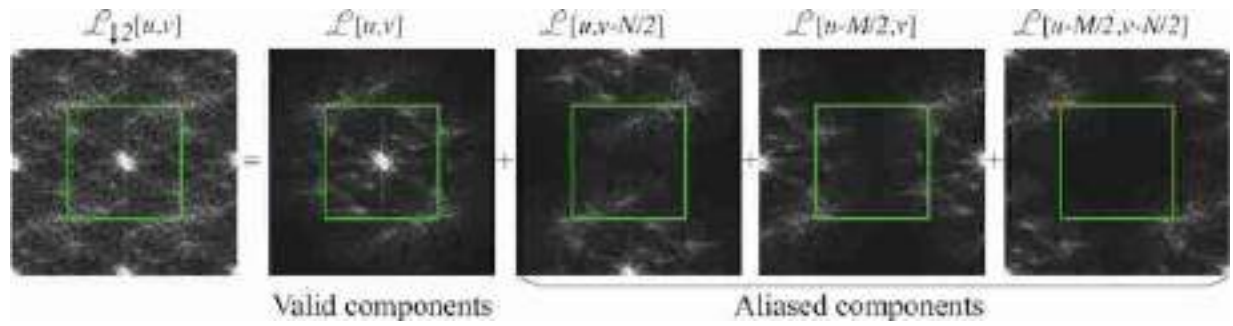


Figure 21.5: DFT of an image multiplied by a delta train and the decomposition into the four different translated copies of the DFT of the original input image.

The first component is $\mathcal{L}[u, v]$, the three other copies are shifted versions of $\mathcal{L}[u, v]$. Shifting is equivalent to a circular shift as $\mathcal{L}[u, v]$ is periodic in u and v . All the spectral content inside the green box in the shifted copies will produce aliasing after decimation.

21.3.3 Aliasing in Matrix Form

We can provide another description of decimation and aliasing using the matrix form of the decimation and DFT operators. As decimation and the Fourier transform are linear operators, we can write them as matrices. This description will provide a different visualization for the same phenomena described in the previous section. For simplicity of the visualizations we will work with 1D signals, but the same can be extended to 2D.

Using matrices, we can write decimation as $\ell_{\downarrow 2} = \mathbf{D}_2 \ell$. We use the Fourier matrices $\mathbf{F}_N$ and $1/N \mathbf{F}_N^*$ to denote the DFT and its inverse (which is the complex conjugate of the DFT) for signals of length N . Using this notation, the relationship between the original spectrum $\mathcal{L}[u]$ and the sampled version, $\mathcal{L}_{\downarrow 2}[u]$, is:

$$\mathcal{L}_{\downarrow 2} = \frac{1}{N} \mathbf{F}_{N/2} \mathbf{D}_2 \mathbf{F}_N^* \mathcal{L} \quad (21.14)$$

One interesting property of the Fourier matrix is that $\mathbf{D}_2 \mathbf{F}_N = [\mathbf{F}_{N/2} \mid \mathbf{F}_{N/2}]$. The matrix $\mathbf{D}_2$ selects all the odd rows of the matrix $\mathbf{F}_N$ and ignores the even ones. Therefore,

$$\frac{1}{N} \mathbf{F}_{N/2} \mathbf{D}_2 \mathbf{F}_N^* = \frac{1}{N} \mathbf{F}_{N/2} [\mathbf{F}_{N/2}^* \mid \mathbf{F}_{N/2}^*] \quad (21.15)$$

$$= \frac{1}{N} \frac{N}{2} [\mathbf{I}_{N/2} \mid \mathbf{I}_{N/2}] \quad (21.16)$$

$$= \frac{1}{2} [\mathbf{I}_{N/2} \mid \mathbf{I}_{N/2}] \quad (21.17)$$

where $\mathbf{I}_{N/2}$ is the identity matrix of size $N/2 \times N/2$. Replacing this result inside equation (21.14) gives the following relationship between the DFT of the original signal and the subsampled one:

$$\mathcal{L}_{\downarrow 2} = \frac{1}{2} [\mathbf{I}_{N/2} \mid \mathbf{I}_{N/2}] \mathcal{L} \quad (21.18)$$

This equation is analogous to equation (21.13) with $k = 2$ and for 1D signals.

In 1D, we can visualize these operators which helps to make the previous derivations intuitive. For 1D signals of length $N = 16$, equation (21.14) has the visual form shown in [figure 21.6](#):

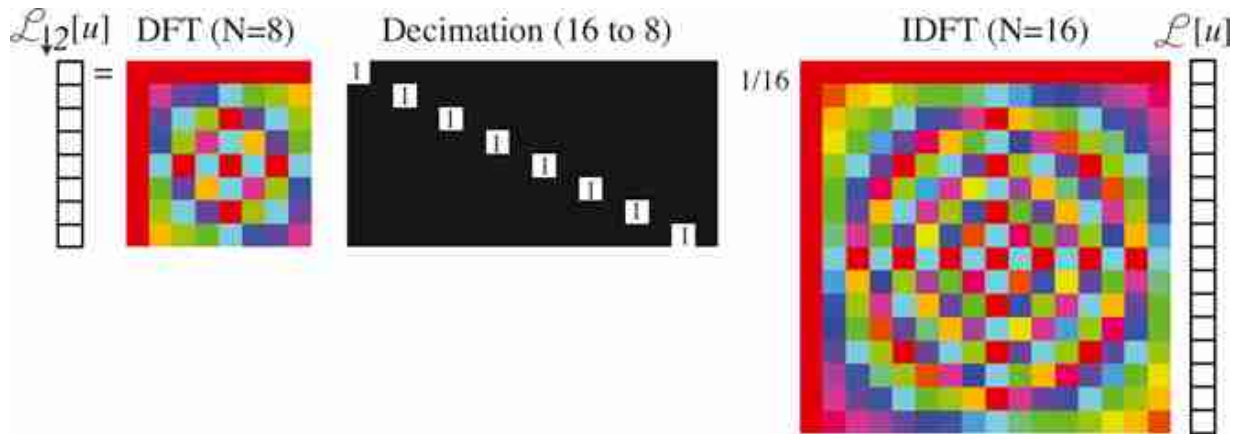


Figure 21.6: Visual representation of equation (21.14).

Note that for the matrices $\mathbf{F}_N$ used here, we are not using the convention of putting the frequency 0 in the middle as this makes visualizations in matrix form easier.

The product between the decimation matrix and the inverse DFT, $\mathbf{D}_2 \mathbf{F}_{16}^*$, is ([figure 21.7](#)):

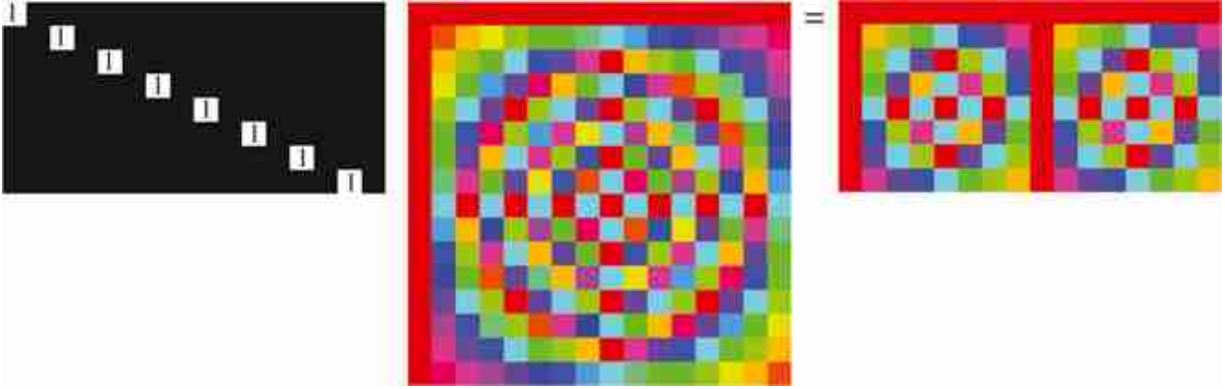


Figure 21.7: Visualization of the product $\mathbf{D}_2\mathbf{F}_{16}^*$ and its result.

In [figure 21.7](#) we can see how the odd rows of $\mathbf{F}_{16}^*$ gives a new sub matrix composed of two identical blocks. Interestingly, each of those two blocks are identical to $\mathbf{F}_8^*$. A similar structure emerges also when applying $\mathbf{D}_4$. Combining these last equations together, we get the final result:

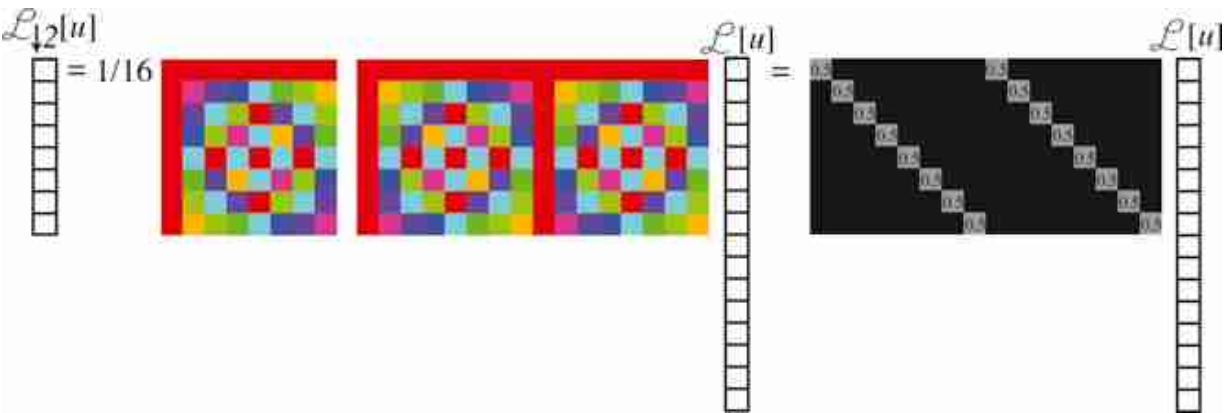


Figure 21.8: Visualization of decimation and aliasing.

This last result shows visually how aliasing happens. Each component of the final DFT, $\mathcal{L}_{\downarrow 2}[u]$, is the sum of two of the components of $\mathcal{L}[u]$. For instance, $\mathcal{L}_{\downarrow 2}[0] = 0.5\mathcal{L}[0] + 0.5\mathcal{L}[8]$. Both components are mixed in the final result.

In the next subsection we will discuss anti-aliasing filters. Anti-aliasing consists in setting to zero, before decimation, the N/k the spectral

components that produce aliasing.

21.3.4 Anti-Aliasing Filters

Anti-aliasing filtering is an essential operation whenever we manipulate the resolution of an image. When decimating a signal of size $N \times N$ with a scaling factor k , aliasing is produced by all the content at frequencies that are above $u, v > N/2k$. Therefore, to reduce aliasing, we have to remove any spectral content of the signal present at high spatial frequencies. We will do so by using an anti-aliasing filter that will blur the image before downsampling. Blurring removes high-spatial frequencies in the image.

The ideal anti-aliasing filter is a box in the Fourier domain with a cut-off frequency of $L = N/(2k)$ (using the notation introduced in chapter 20). The inverse Fourier transform of a box in the frequency domain, with width $2L + 1 = N/k + 1$, is the **discrete sinc** function of length N in the spatial domain:

$$\frac{1}{N} \frac{\sin(\pi n(N/k + 1)/N)}{\sin(\pi n/N)} \xrightarrow{\mathcal{F}} \text{box}_{N/2k}[u] \quad (21.19)$$

In 2D, the ideal anti-aliasing filter is separable and is obtained by two consecutive 1D convolutions along each dimension. The discrete sinc is very similar to the continuous sinc described in chapter 20. The discrete sinc is periodic, with period N . The following plot ([figure 21.9](#)) shows the anti-aliasing sinc for $N = 32$ and $k = 2$ and the magnitude of its Fourier transform:

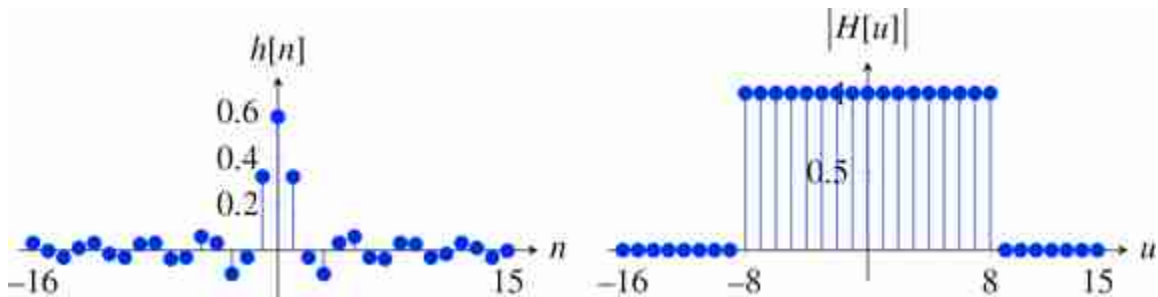


Figure 21.9: Discrete sinc function for $N = 32$ and $k = 2$ and the magnitude of its DFT.

And for $k = 4$, the impulse response becomes wider and its Fourier transform has a lower cut-off frequency, $N/2k = 4$:

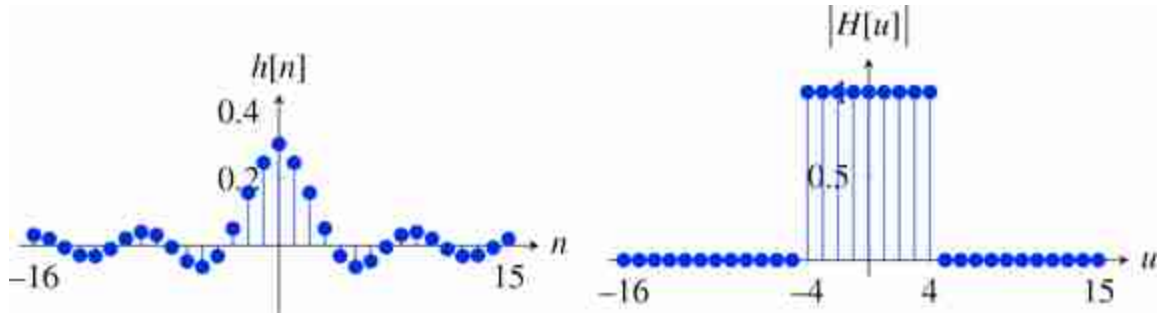


Figure 21.10: Discrete sinc function for $N = 32$ and $k = 4$ and the magnitude of its DFT.

One disadvantage of the ideal anti-aliasing filter is that it requires convolving the image with a very large filter (as large as the image). We would like to use smaller filters to reduce the computational cost. One method to design anti-aliasing filters with a finite impulse response is to use a window, $w[n]$, and to use as anti-aliasing filter the kernel $w[n]h[n]$, where $h[n]$ is the impulse response of the ideal anti-aliasing filter. Typical windows used are the rectangular, Hamming, Parzen, or Kaiser windows.

The rectangular window sets to zero all the values outside of the interval $[-L, L]$. When $L = 5$, it gives a tap-11 filter (a filter with an impulse response of length 11) with the following profile:

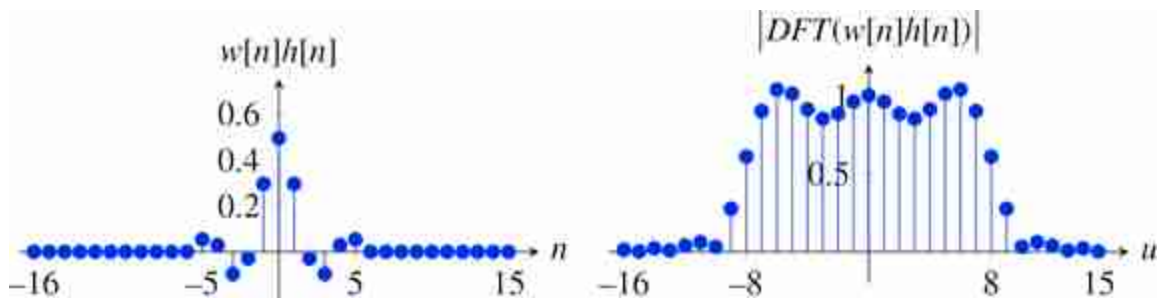


Figure 21.11: Kernel $w[n]h[n]$, where $h[n]$ is a sinc and $w[n]$ is a rectangular window of length 11.

The Fourier transform of the windowed impulse response, $w[n]h[n]$, is the convolution of Fourier transforms. The Fourier transform of the rectangular window is a sinc function. Therefore, the resulting Fourier transform is the convolution of the rectangle with the sinc function, which produces a smooth rectangle with some oscillations. Using other windows

allows trading off the smoothing of the ideal rectangular form of the ideal low-pass filter and the oscillations.

The **Hamming window** is defined as:

$$w[n] = \begin{cases} 0.54 + 0.46 \cos(\pi n/L) & \text{if } n \in [-L, L] \\ 0 & \text{otherwise} \end{cases} \quad (21.20)$$

The Hamming window ($L = 5$) produces a smooth transfer function with oscillations of smaller amplitude in the Fourier domain:

$$h = [1, 0.65, -2.45, -0.82, 8.8, 15, 8.8, -0.82, -2.45, 0.65, 1]/29.36 \quad (21.21)$$

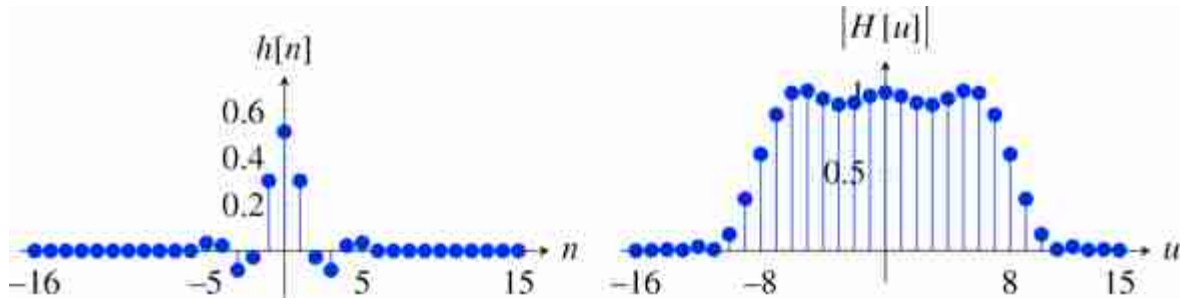


Figure 21.12: Hamming window ($L = 5$) and the magnitude of its DFT.

In practice, when we need very small anti-aliasing filters, binomial filters are a good choice. The binomial filter b_2 has a DFT (length N) with a $1 + \cos(2\pi/N)$ profile. At the frequency $N/4$ the gain is $1/2$. Therefore, the spectral content responsible for aliasing will not be completely removed, but the attenuation is enough in most cases.

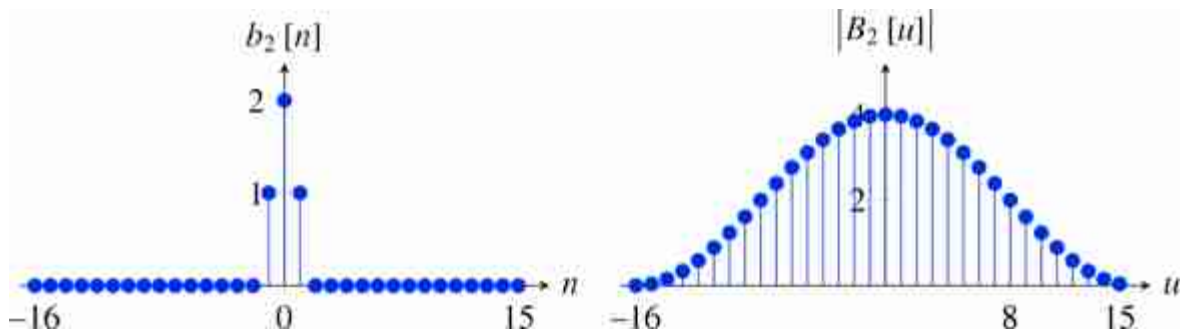


Figure 21.13: The binomial kernel $b_2 = [1, 2, 1]$, and its DFT.

Or $b_4 = [1, 4, 6, 4, 1]$, which attenuates the aliased frequencies even more, also results in an oversmoothed signal.

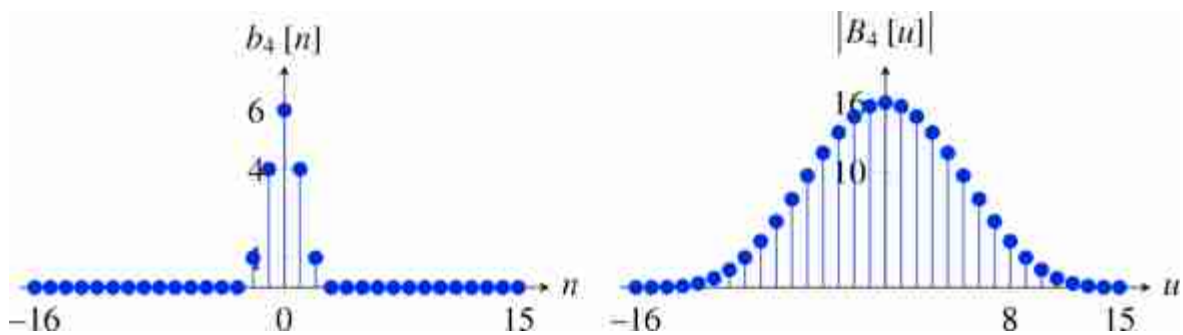


Figure 21.14: The binomial kernel $b_4 = [1, 4, 6, 4, 1]$, and its DFT.

When implementing 2D anti-aliasing filters, we apply two consecutive 1D convolutions along each dimension. For efficiency, the anti-aliasing filter only needs to be computed on the samples that will be kept after downsampling (k -strided convolution). The necessary amount of attenuation of the aliasing frequencies will have to be decided based on each application.

In 2D, we can apply the binomial filter b_2 on each dimension, $[1, 2, 1] / 4 \circ [1, 2, 1]^T / 4$.

The images in [figure 21.15](#) show the output of three different low-pass anti-aliasing filters. Note how the ideal low-pass filter introduces **ringing artifacts** in the output image ([figure 21.15\[a\]](#)). Despite that it removes

frequencies that will produce aliasing after downsampling, it does that at a high cost because it introduces undesired image structures, which is exactly what we want to prevent with the anti-aliasing filter. The Hamming windowed sinc (shown in [figure 21.15\[b\]](#)) reduces the oscillation but there is still a halo present around edges. This is because there is still one oscillation on the filter impulse response. [Figure 21.15\(c\)](#) shows the result applying an anti-aliasing binomial filter.



Figure 21.15: Output of three different low-pass anti-aliasing filters. (left) Ideal low-pass filter. (center) Hamming window. (right) Binomial filter.

For most applications using a binomial filter (b_2 or b_4) is enough because it reduces aliasing and it does not introduce undesired image structures.

21.3.5 Downsampling

We will call **downsampling** the sequence of applying an anti-aliasing filtering followed by decimation:

$$z[n, m] = \ell_{\text{in}}[n, m] \circ h_k[n, m] \quad \triangleleft \text{conv} \quad (21.22)$$

$$\ell_{\text{out}}[n, m] = z[kn, km] \quad \triangleleft \text{decimation} \quad (21.23)$$

[Figure 21.16](#) shows the results when using a binomial filter as the anti-aliasing filter followed by decimation by 2, and [figure 21.17](#) shows results for different downsampling factors.

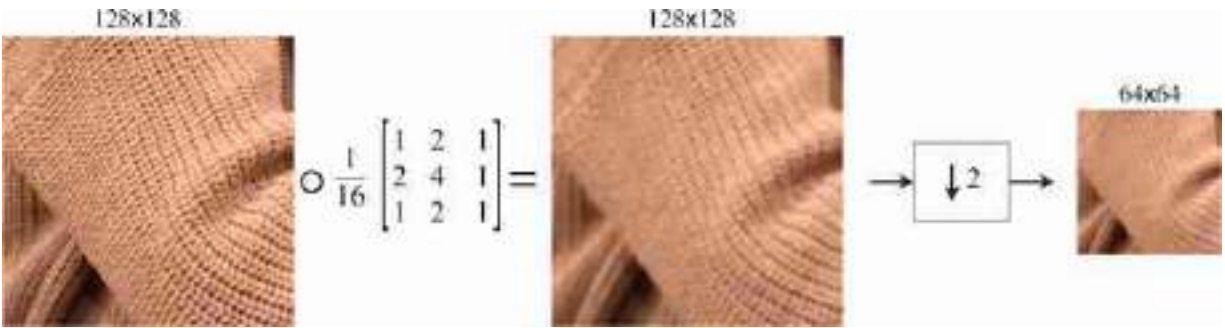


Figure 21.16: Downsampling by 2 uses a binomial filter as the anti-aliasing filter followed by decimation by 2.

In the anti-aliased downsampled images shown in [figure 21.17](#), fine details are gone while preserving the smooth details of the image. In fact, as shown in the images below, even at 32×32 , we can still see some of the orientations of the knitting pattern and the shading due to the three-dimensional shape.



Figure 21.17: Successive downsampling by a factor of 2 with anti-aliasing. Compare these results with the ones from [figure 21.2](#).

As the convolution and downsampling are linear operators, sometimes it will be useful to write it as a matrix. For instance, for a 1D signal of length $N = 8$ samples, if we downsample it by a factor $k = 2$, then, using repeat padding for handling the boundary, and the binomial kernel as an anti-aliasing filter, we can write the downsampling operator as a matrix:

$$\mathbf{D}_2 = \frac{1}{4} \begin{bmatrix} 3 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 2 & 1 \end{bmatrix} \quad (21.24)$$

Downsampling by a factor $k = 2^n$ is usually done by a sequence of n stages of binomial filtering followed by downsampling by 2. Non-integer downsampling factors require an interpolation step instead of decimation.

21.4 Upsampling

Upsampling requires increasing the resolution of an image from a low resolution image. This is a very challenging task that requires recovering information that is not available on the low resolution image. The problem of making up new details will be called super-resolution. In this section we will focus on the simpler problem of creating an image that has k times more pixels along each dimension but that contains the same amount of information than the original lower-resolution image. That is, we will not require to the upsampling procedure to predict any of the missing high-resolution details.

Upsampling requires interpolating sample values. We want to obtain a new image ℓ_{out} with more samples than the input image ℓ_{in} . In order to interpolate a new sample $\ell_{\text{out}}(a, b)$, where a and b are continuous values, we can use different interpolations functions as we will discuss next.

21.4.1 Nearest Neighbor Interpolation

In **nearest neighbor interpolation** we assign to $\ell_{\text{out}}(a, b)$ the closest value of the input ℓ_{in} :

$$\ell_{\text{out}}(a, b) = \ell_{\text{in}}[n, m] \quad (21.25)$$

where $n - 0.5 < a < n + 0.5$ and $m - 0.5 < b < m + 0.5$. Note that here we are treating $\ell_{\text{out}}(a, b)$ as a continuous image. Nearest neighbor interpolation results in an image that is piecewise-constant. [Figure 21.18](#) shows the nearest neighbor interpolation process graphically.

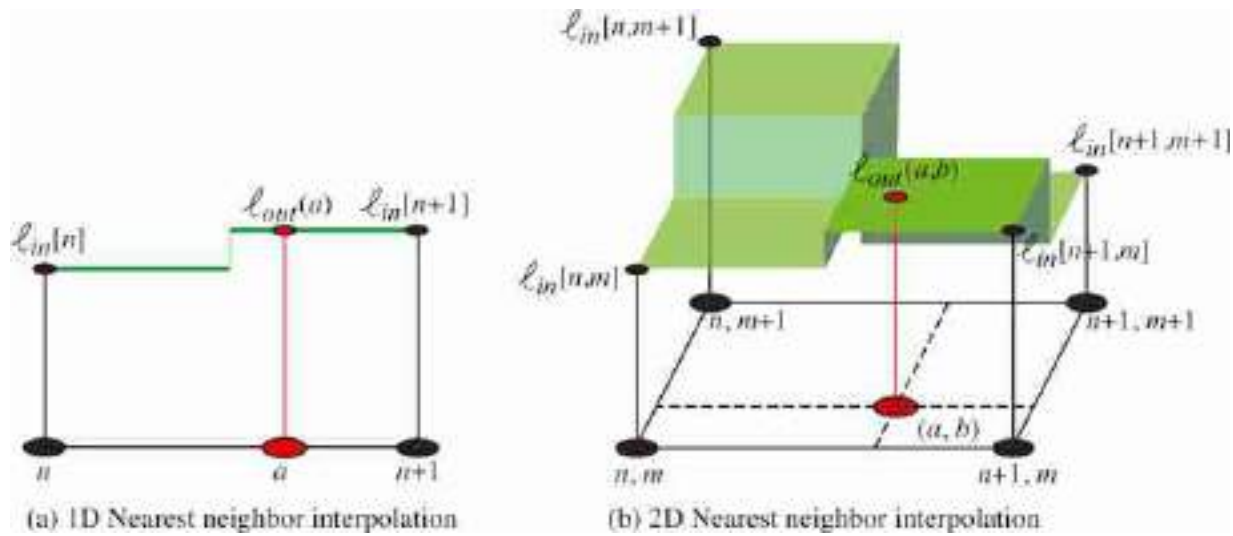


Figure 21.18: Nearest neighbor interpolation in 1D and 2D.

21.4.2 Linear and Bilinear Interpolation

Linear interpolation: For 1D signals, the value of $\ell_{\text{out}}(a)$ is a result of linear interpolation between the two closest input values $\ell_{\text{in}}[n]$ and $\ell_{\text{in}}[n+1]$, where $n < a < n+1$. For 2D signals, we have to interpolate the output value $\ell_{\text{out}}(a, b)$ using four input samples: $\ell_{\text{in}}[n, m]$, $\ell_{\text{in}}[n+1, m]$, $\ell_{\text{in}}[n, m+1]$ and $\ell_{\text{in}}[n+1, m+1]$, where $n < a < n+1$ and $m < b < m+1$.

Bilinear interpolation consists of first applying linear interpolation to obtain $\ell_{\text{out}}(a, m)$ and $\ell_{\text{out}}(a, m+1)$ and then again applying linear interpolation between those two values to obtain $\ell_{\text{out}}(a, b)$. The result is illustrated in [figure 21.19](#).

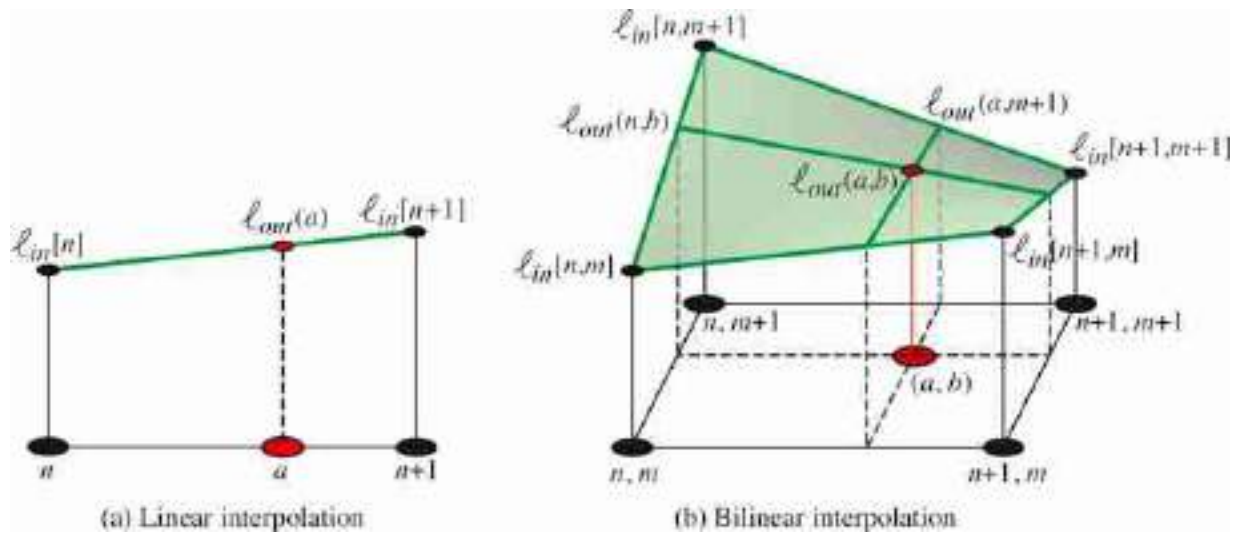


Figure 21.19: 1D linear interpolation and 2D bilinear interpolation.

The linear (and bilinear) interpolations are linear operations. In the case when the upsampling factor k is an integer value, the resampling operator can be efficiently implemented as a sequence of two operations: expansion and interpolation. The interpolation operation, as it is linear and spatially stationary, can be efficiently implemented as a convolution.

Interpolation is also a key operation when performing image warping.

21.4.3 Expansion

Expansion by a factor k consists in increasing the number of samples in the image by inserting $k - 1$ zeros between each sample on the horizontal dimension, and then on the vertical dimension. This will give us the image of size $Nk \times Mk$. We will use the notation:

$$\ell_{\uparrow k}[n, m] = \ell[n, m] \uparrow k \quad (21.26)$$

Expansion is a linear operator that is the transposed to the decimation operation and is defined as:

$$\ell_{\uparrow k}[n, m] = \begin{cases} \ell[n/k, m/k] & \text{if } \text{mod}(n, k) = 0 \text{ and } \text{mod}(m, k) = 0 \\ 0 & \text{otherwise} \end{cases} \quad (21.27)$$

In the 1D case, for a signal of length 4, expanding by a factor of $k = 2$ can be described by the following linear operator:

$$\mathbf{E}_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (21.28)$$

21.4.4 Interpolation Filters

When the upsampling factor k is an integer value, interpolation using the methods described in section 21.4.1 can be implemented using a convolution. The interpolation filters we will discuss here are separable, so the convolution kernel can be written as the product $h[n, m] = h[n] h[m]$:

As we discussed before, there are different interpolation methods: nearest neighbor, bilinear, and so on. In this section we will study them by looking at the convolution kernel needed to implement each interpolation method.

Nearest neighbor interpolation consists of assigning to each interpolated sample the value of the closest sample from the input signal. This can be done by convolving the expanded image from equation (21.27) with the kernel (if k is even):

$$h[n] = \begin{cases} 1 & \text{if } n \in [-k/2 + 1, k/2] \\ 0 & \text{otherwise} \end{cases} \quad (21.29)$$

This is very fast as it only requires replicating samples. But it gives very low quality. For $k = 2$, the convolution kernel in 1D is $h[n] = [1, 1]$

Bilinear interpolation: As we discussed before, bilinear interpolation is done in two stages: first, linearly interpolate each sample with the two nearest neighbors across rows, and then do the same for the columns. This can also be written as a convolution with the kernel (if k is even):

$$h[n] = \begin{cases} (k-n)/k & \text{if } n \in [-k, k] \\ 0 & \text{otherwise} \end{cases} \quad (21.30)$$

In the special case where $k = 2$, the 1D bilinear interpolation kernel reduces to the binomial filter $h[n] = [1/2, 1, 1/2]$. And in 2D, for $k = 2$, the kernel is $[1/2, 1, 1/2] \circ [1/2, 1, 1/2]^T$. The binomial filter is not the best interpolation

filter, but it is very efficient. When quality is important, there are better interpolation filters.

The plots in [figure 21.20](#) show the nearest neighbor and linear interpolation kernels for $k = 4$.

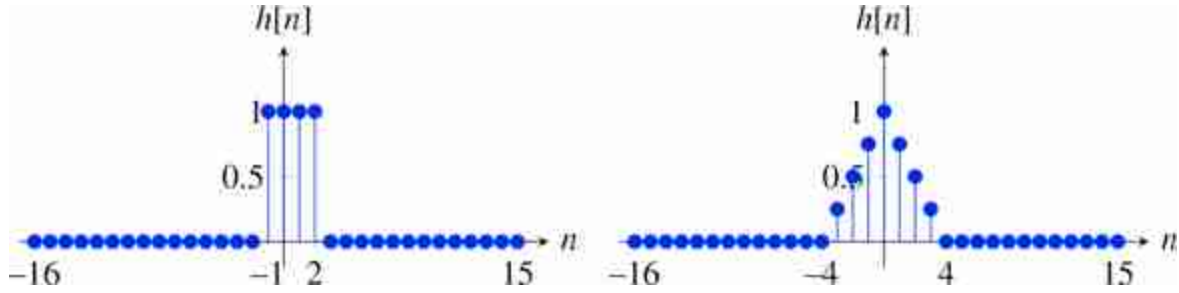


Figure 21.20: Kernels for (left) nearest neighbor, and (right) linear interpolation for $k = 4$.

Other popular interpolation methods are the **bicubic interpolation** and the **Lanczos interpolation**.

21.4.5 Upsampling

Upsampling consists in the sequence of expansion followed by interpolation:

$$z[n, m] = \ell_{in}[n, m] \uparrow k \quad \leftarrow \text{expansion} \quad (21.31)$$

$$\ell_{out}[n, m] = z[n, m] \circ h_k[n, m] \quad \leftarrow \text{conv} \quad (21.32)$$

Upsampling by a factor of 2 is done by inserting one 0 between each two samples (first for the horizontal dimension and then for the vertical one), and then interpolating between samples applying the binomial filter $[1/2, 1, 1/2] \circ [1/2, 1, 1/2]^T$.

[Figure 21.21](#) shows the upsampling process for a color image. Each color channel is upsampled by a factor of $k = 2$ independently. The top row shows the input image, the expanded image by inserting zeros between samples, the binomial kernel and the resulting interpolated image. The bottom row shows the magnitude of DFTs of each corresponding image. Note that the binomial interpolation filter does not completely suppress the copies of the original DFT that appear when inserting new samples. Remember that the DFTs have the same number of samples as the corresponding input image. The DFT of the kernel is shown by zero

padding the convolutional kernel to match the image size. Despite that this interpolation does correspond to an ideal interpolation filter (a sinc) the interpolated image looks reasonably good.

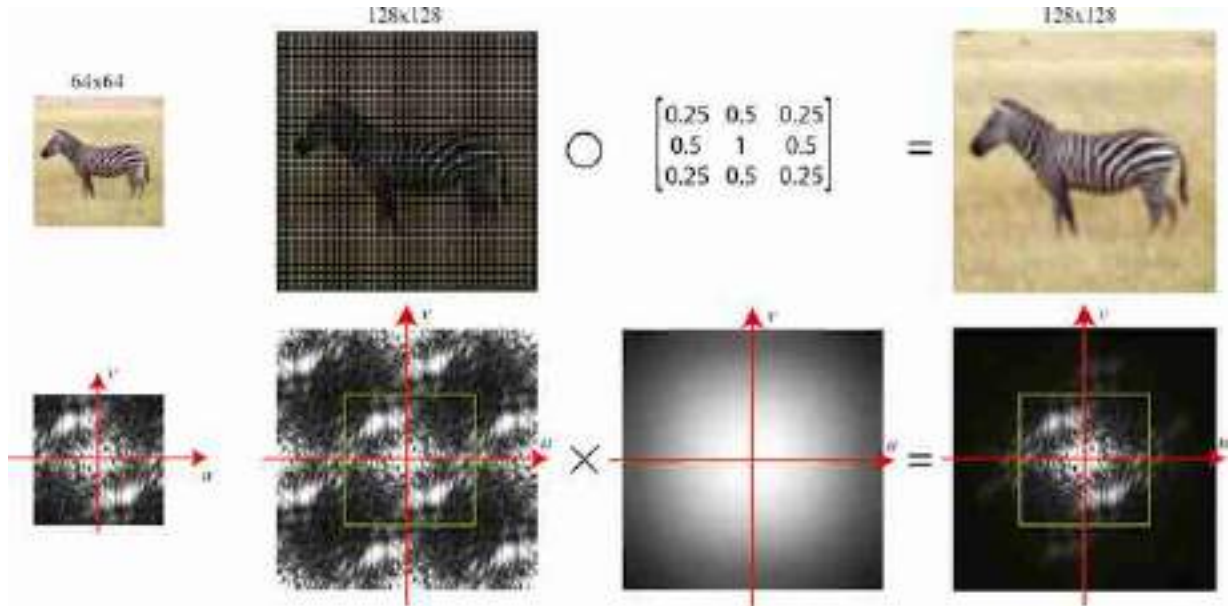


Figure 21.21: Upsampling by a factor of $k = 2$ using a bilinear interpolation filter.

The upsampling operator is also a linear operation, therefore we can also write it in matrix form. For instance, for a 1D signal of length $N = 4$ samples, if we upsample it by a factor $k = 2$, then, using repeat padding for handling the boundary and the bilinear interpolation, we can write the upsampling operator as a matrix:

$$U_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 0.5 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0.5 & 0.5 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0.5 & 0.5 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (21.33)$$

Multiplying this matrix by a signal of length 4, gives an output signal of length 8. The output samples at locations 1, 3, 5, 7, and 8 are copies of the input samples 1, 2, 3, and 4, while samples 2, 4, and 6 are computed as the

average of two consecutive input samples. The last output sample is a copy of the last input sample because it corresponds to extending the input using repeat padding.

21.5 Concluding Remarks

Aliasing is a constant source of problems in many image processing pipelines. In computer vision, aliasing can introduce noise, spurious features and textures, and lack of translation invariance. In chapter 24 we will discuss how aliasing affects the quality of some of the convolutional layers used in neural networks.

Some of the tools we discussed in this chapter will also be used in other settings such as when building image pyramids (chapter 23), and geometry (chapter 38).

Aliasing is often a source of image artifacts in generative image models. One example are generative adversarial networks (chapter 32) where images are produced as a sequence of learned convolutions and upsampling stages. When the upsampling operations are done without paying attention to aliasing, the output images show a number of artifacts as has been shown in [\[252\]](#). Those artifacts can be removed by carefully implementing each stage to avoid aliasing.

22 Filter Banks

22.1 Introduction

Although linear filters can perform a wide range of operations as we have seen in previous chapters, image understanding requires nonlinear operators. In this chapter we will study how sets of filters can be used to extract information about images. We will focus on some traditional families of filters that have been used to build image representations in the early days of computer vision and that help to understand part of the representational power of modern deep neural networks.

We will show how simple nonlinearities applied to the output of linear filters (such as the squaring nonlinearity) can be used to build useful image representations.

22.2 Gabor Filters

In chapter 16 we discussed several useful image representations: representing an image in the frequency domain and decomposing it into amplitude and phase, and also the analysis of image content across different scales and orientations. The Fourier transform is a tool that allows us to extract that information, but only globally. For this information to be useful it needs to be localized. For instance, the analysis of orientations of local image structures can be done using image gradients (chapter 18), which are localized in space. In fact, we made use of image derivatives in chapter 2 to recover the three-dimensional (3D) structure of a scene. Here we will discuss another family of filters that has a long history of being used for image analysis: Gabor filters.

[Figure 22.1](#)(a) shows a one-dimensional (1D) sine function. This function has infinite support. When used as a representation, the Fourier coefficients are obtained as the scalar product between the sine wave and the input image. Each Fourier coefficient involves a point-wise multiplication between the wave and the input image and then the sum of the result. As a consequence of the infinite support of the wave, one single

Fourier coefficient does not tell us anything about what happens locally inside the analyzed image.

A good start for a localized image analysis is to restrict the spatial support of a sinusoidal basis function. One function that has a local support (a lots of nice properties as we discussed in chapter 17) is the Gaussian function ([figure 22.1\[b\]](#)). The product of the Gaussian and the sine wave (or a cosine wave) gives the function with the profile shown in [figure 22.1\(c\)](#) called a Gabor filter. Gabor functions were introduced by Dennis Gabor in 1946 [[149](#)].

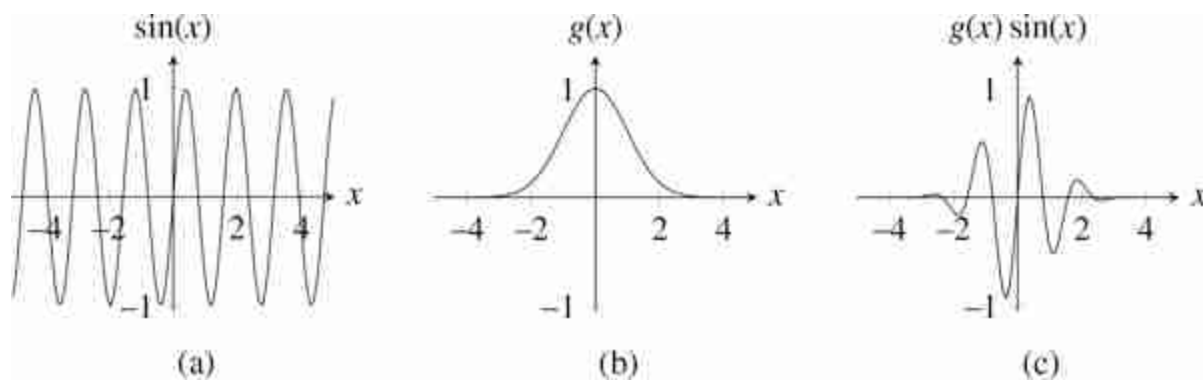


Figure 22.1: Construction of a Gabor function. (a) Sine function. (b) Gaussian function. (c) Gabor function obtained as the product of (a) and (b).

Gabor functions, originally proposed in 1D, were made popular as an operator for image analysis by Goesta Granlund in his paper *In Search of a General Picture Processing Operator* [[174](#)].

Gabor functions are localized in both the spatial and frequency domain. One remarkable property of the Gabor function is that it is the optimal function that achieves the best simultaneous localization in space and frequency.

Gabor functions can be extended to two dimensions by using 2D Gaussians and waves. We can also use complex waves (chapter 16) as they will give us a more general form of the Gabor function. We can multiply a complex Fourier basis function, $\exp(j(u_0x + v_0y))$, by a spatially localized 2D Gaussian window, $\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$ to obtain a Gabor function, $\psi(x, y)$:

$$\psi(x, y; u_0, v_0) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \exp(j(u_0x + v_0y)) \quad (22.1)$$

A Gabor function defined as in equation (22.1) is a complex-valued function of location and frequency. We can get the cosine and sine waves by looking at the real and imaginary components, $\psi(x, y; u_0, v_0) = \psi_r(x, y; u_0, v_0) + j\psi_i(x, y; u_0, v_0)$ with

$$\psi_r(x, y; u_0, v_0) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \cos(u_0x + v_0y) \quad (22.2)$$

$$\psi_i(x, y; u_0, v_0) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \sin(u_0x + v_0y) \quad (22.3)$$

Where $\psi_r(x, y; u_0, v_0)$ and $\psi_i(x, y; u_0, v_0)$ are the cosine and sine phase Gabor filters. [Figure 22.2](#) shows the Gaussian window and the real and imaginary parts of the Gabor function.

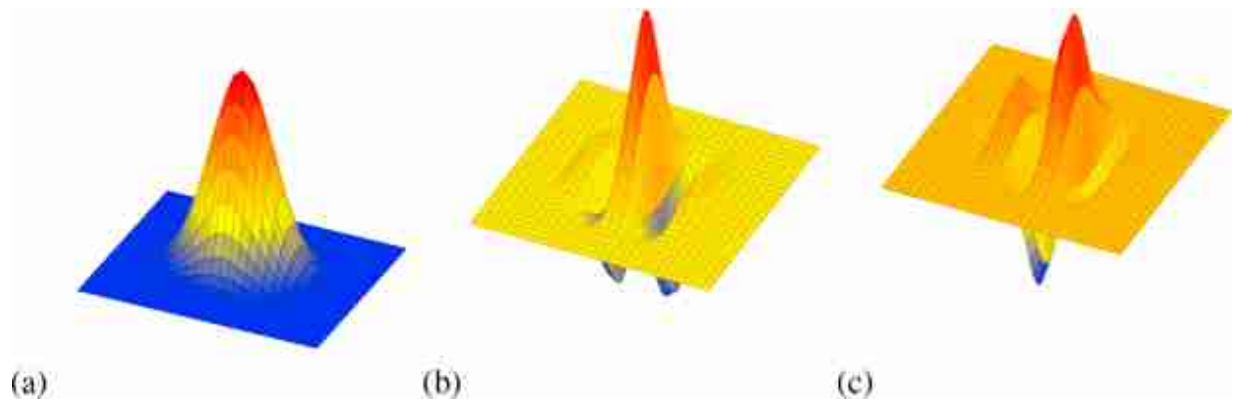


Figure 22.2: 2D Gabor functions. (a) The localizing Gaussian window ($\sigma = 1$), which can be thought of as a Gabor function for a zero frequency sinusoid. (b) Cosine, and (c) sine phase Gabor functions with central frequency $u_0 = 2\pi$ and $v_0 = 0$.

The Fourier transform of a complex Gabor function is a Gaussian in the Fourier domain shifted with respect to the origin:

$$\Psi(u, v; u_0, v_0) = \exp\left(-((u - u_0)^2 + (v - v_0)^2) \frac{\sigma^2}{2}\right) \quad (22.4)$$

The Gabor filter is an oriented band-pass filter.

Figure 22.3 shows the magnitude of the Fourier transform of the Gabor filters. Figure 22.3(a) shows the cosine phase Gabor function, $\psi_r(x, y; u_0, v_0)$, and figure 22.3(b) shows the sine phase Gabor function, $\psi_i(x, y; u_0, v_0)$. Figure 22.3(c) shows the Fourier transform of the cosine phase Gabor function, $\Psi_r(u, v; u_0, v_0)$, and figure 22.3(d) shows the Fourier transform of the complex Gabor function, $\Psi(u, v; u_0, v_0)$.

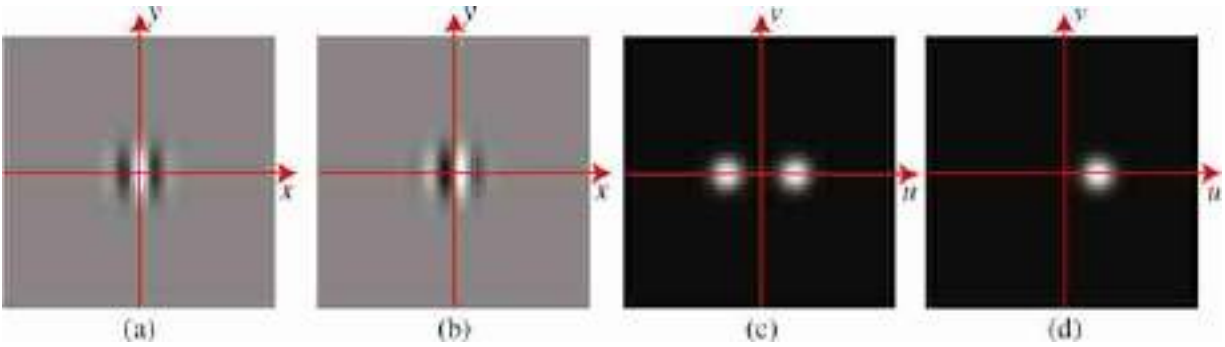


Figure 22.3: (a) Cosine and (b) sine Gabor functions. (c) Magnitude of the Fourier transform of both the cosine and sine Gabor functions (their FT only differs in the phase). (d) FT of the complex Gabor function, which is asymmetrical with a single lobe.

Figure 22.4 shows how the Gabor function changes when modifying its parameters (central frequency, (u_0, v_0) , and width, σ). The value of σ sets the locality of the window of analysis and the values of (u_0, v_0) adjust the orientation of the Gabor function and frequency. A large σ makes the spatial extend large and the frequency extend small. A small value of σ has the opposite behavior. Analyzing the image with a set of Gabor functions with a large σ is like computing the Fourier transform of the image.

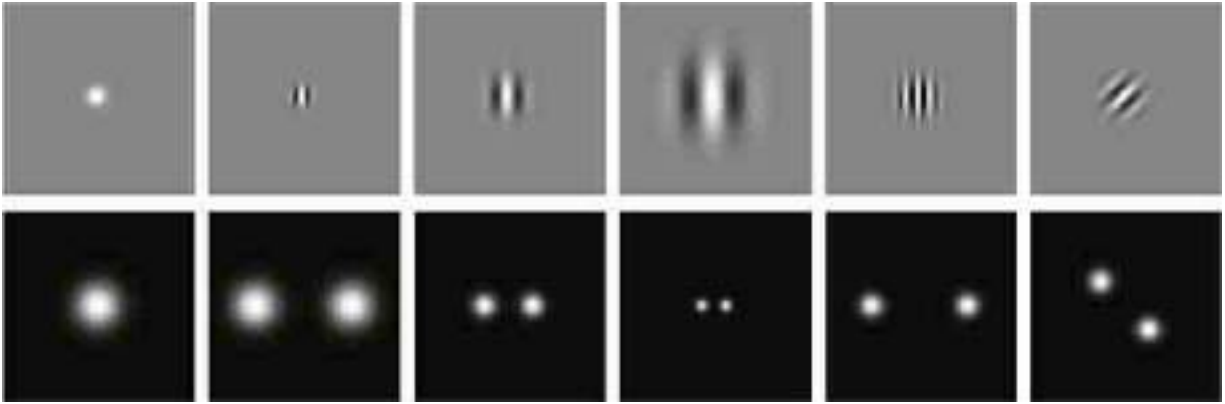


Figure 22.4: Cosine phase Gabor functions tuned to different widths, frequencies, and orientations, and their corresponding Fourier transforms (only the magnitude is shown).

The sine phase Gabor function is zero mean. However, the cosine phase Gabor function is not zero mean. The Gaussian width σ has to be sufficiently large so that the Gabor function behaves like a zero mean filter. All the previous definitions are given in the continuous domain. The discrete version of the Gabor function is obtained by sampling the continuous functions.

One important characteristic of Gabor filters is that they are very similar to the shape of some cortical receptive fields found in the mammalian visual system. This provides a hint that we're on the right track with these filters to build image representations.

Remember the description of simple and complex cells in the visual system from chapter 1.

The convolution of the Gabor function with an image, $\ell(x, y)$ results in an output image that depends on both space, (x, y) , and frequency, (u, v) :

$$\ell_{\psi}(x, y, u, v) = \ell(x, y) \circ \psi(x, y, u, v) \quad (22.5)$$

[Figure 22.5](#) shows the result of the convolution between a picture and the cosine and sine phase Gabor functions at three different scales ($\sigma = 2, 4, 8$). In this example, the Gabor filters are tuned to detect vertical edges. Gabor filters are useful for analyzing line or edge phase structures in images. But they have many other benefits when we combine them together in quadrature pairs.

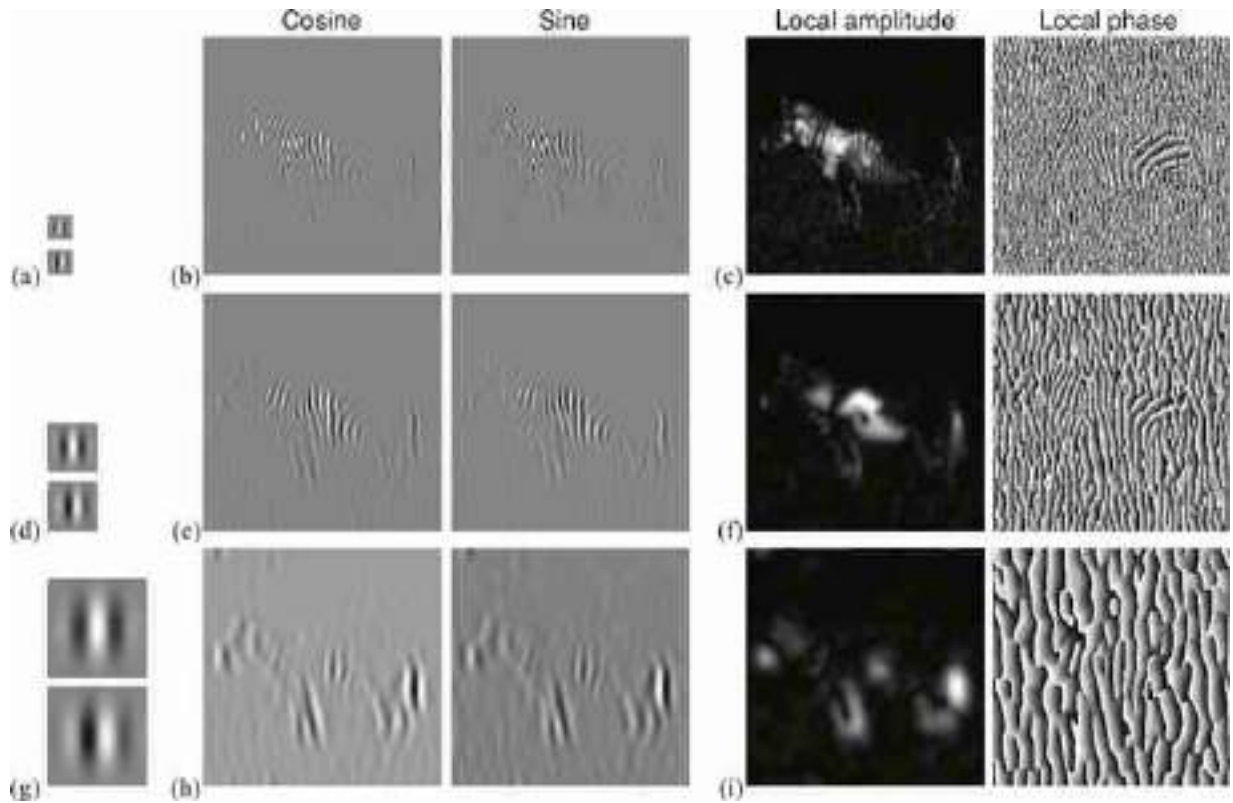


Figure 22.5: Zebra picture filtered by cosine and sine Gabor functions at three scales with $\sigma = 2, 4, 8$ and $u_0 = 1/(2\sigma)$, $v_0 = 0$. Each row shows one scale. (a) Cosine and sine Gabor filters. (b) Cosine and sine outputs. (c) Magnitude and phase of the output of the complex Gabor filter.

22.2.1 Quadrature Pairs and the Hilbert Transform

Pairs of filters can be in a relationship to each other that is called **quadrature phase**. This relationship is useful for many low-level visual processing tasks. When in quadrature phase, pairs of oriented filters can measure what is called local oriented energy, can identify contours, independently of the phase of the contour, and can measure positional changes very accurately. Let's start with the mathematical definition of quadrature.

For notational simplicity, we'll describe quadrature pair filters in the time domain, but they extend naturally to two or more dimensions. Consider an even symmetric zero-mean signal, $\ell(t)$, and with Fourier transform $\mathcal{L}(\omega)$, with $\mathcal{L}(0) = 0$ because the signal is zero-mean. In the Fourier domain, two functions, $\mathcal{L}(\omega)$ and $\mathcal{L}_h(\omega)$, are said to be Hilbert transform pairs if:

$$\mathcal{L}_h(\omega) = \begin{cases} -j\mathcal{L}(\omega) & \text{if } \omega > 0 \\ j\mathcal{L}(\omega) & \text{if } \omega < 0 \end{cases} \quad (22.6)$$

We now define the complex signal:

$$\ell_a(t) = \ell(t) + j\ell_h(t) \quad (22.7)$$

where $\ell_h(t)$ is the Hilbert transform of $\ell(t)$. The signal $\ell_a(t)$ is called the **analytic signal** and its Fourier transform has no negative frequency components. It is easy to show that

$$\mathcal{L}_a(\omega) = \begin{cases} 2\mathcal{L}(\omega) & \text{if } \omega > 0 \\ 0 & \text{if } \omega < 0 \end{cases} \quad (22.8)$$

It is interesting to write the complex signal $\ell_a(t)$ in polar form:

$$\ell_a(t) = a(t) \exp(j\theta(t)) \quad (22.9)$$

where $a(t)$ is the instantaneous amplitude (**local amplitude**) and $\theta(t)$ is the instantaneous phase (**local phase**). This particular representation is common in communications theory, and it has been used to build image representations invariant to certain image structures as we will discuss in the following sections.

The extension of the Hilbert transform to images can be done in several ways. The most common approach in image processing is to define one direction in the frequency space n and then the Hilbert transform is as follows:

$$\mathcal{L}_h(\omega_x, \omega_y) = \begin{cases} -j\mathcal{L}(\omega_x, \omega_y) & \text{if } n^T \cdot (\omega_x, \omega_y) > 0 \\ j\mathcal{L}(\omega_x, \omega_y) & \text{if } n^T \cdot (\omega_x, \omega_y) < 0 \end{cases} \quad (22.10)$$

Two band-pass filters are said to be in quadrature if the impulse responses are Hilbert transform pairs along some orientation in the frequency domain. Sine and cosine functions of the same frequency are Hilbert transform pairs, as are sine and cosine phase Gabor functions, of the same frequency and Gaussian envelope parameters. Thus, these filter pairs are also quadrature pairs. When convolving two filters in quadrature with a signal (or image) the two outputs are also in quadrature.

The quadrature in the Gabor functions is an approximation that only holds for σ sufficiently large. For small σ the filter does not form a quadrature pair. For large σ , the real and imaginary parts of the Gabor filter (cosine and sine phase local filters) are filters in quadrature with the vector

$n = (u_0, v_0)$ pointing in the direction of the central frequency of the Gabor function.

22.2.2 Local Amplitude

Let $h(x, y)$ and $q(x, y)$ be band-pass filters in quadrature, and let $\ell_h(x, y)$ and $\ell_q(x, y)$ be the result of convolving the signal $\ell(x, y)$ with $h(x, y)$ and $q(x, y)$, respectively. Then, the squared local amplitude is a measure of the image power within the frequency bandwidth of the filters in the local neighborhood of (x, y) :

$$a^2(x, y) = \ell_h(x, y)^2 + \ell_q(x, y)^2 \quad (22.11)$$

[Figure 22.6](#) shows the steps to compute the local amplitude of an input image using Gabor filters. This is like a two-layer convolutional neural network with two channels in the first layer, a squaring nonlinearity and then followed by a sum in the second layer. This very simple system can perform a number of interesting operations.

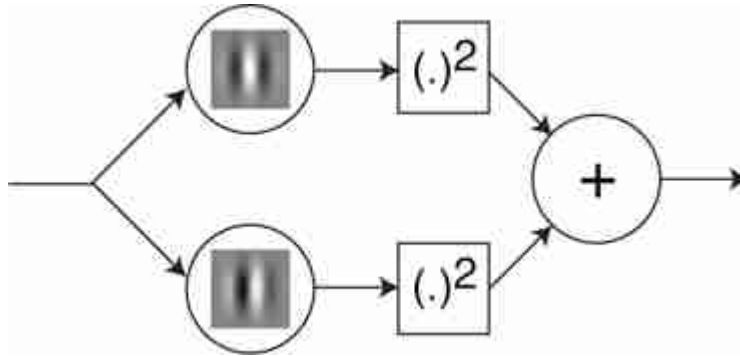


Figure 22.6: Computation of localized amplitude. The input is filtered by a pair of quadrature Gabor filters. Each filter output is squared and the result is added.

To see how useful the local amplitude is, let's start by computing it for some simple images. If the input image is a delta, $\ell(x, y) = \delta(x, y)$ (this is an image with a single bright dot on it), and the filters h and q are the cosine and sine phase Gabor filters, then, the local amplitude image is:

$$\begin{aligned}
a(x, y) &= \sqrt{\ell_h(x, y)^2 + \ell_q(x, y)^2} = \\
&= \sqrt{\psi_r^2(x, y; u_0, v_0) + \psi_l^2(x, y; u_0, v_0)} = \\
&= \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)
\end{aligned} \tag{22.12}$$

The amplitude of the delta function is the Gaussian envelope of the Gabor function. This result is independent of the contrast of the input image. For instance, if the input is $\ell(x, y) = -\delta(x, y)$, the amplitude image does not change (it is **sign invariant**). [Figure 22.7\(a\)](#) shows an image with two impulses of opposite signs. [Figure 22.7\(b\)](#) show the output of the cosine and sine filters, and [figure 22.7\(c\)](#) shows the local amplitude $a(x, y)$. The local amplitude is two Gaussians centered on each impulse. Note that the sign of each impulse does not affect the sign or shape of the local amplitude. Therefore, this system is sign invariant.

The sign invariance property is specially useful when using the local amplitude to localize edges in images. [Figure 22.7\(d\)](#) shows an image composed of several squares. Each square is defined by different polarities with respect to the background. Two of the squares are solid while the other two are only defined by lines. The local amplitude, [figure 22.7\(f\)](#), provides a detector for the square boundaries that is invariant to all those changes. The differences between all the squares are encoded in the local phase image as we will discuss in the following section. [Figure 22.5](#) shows the local amplitude signal computed on real images.

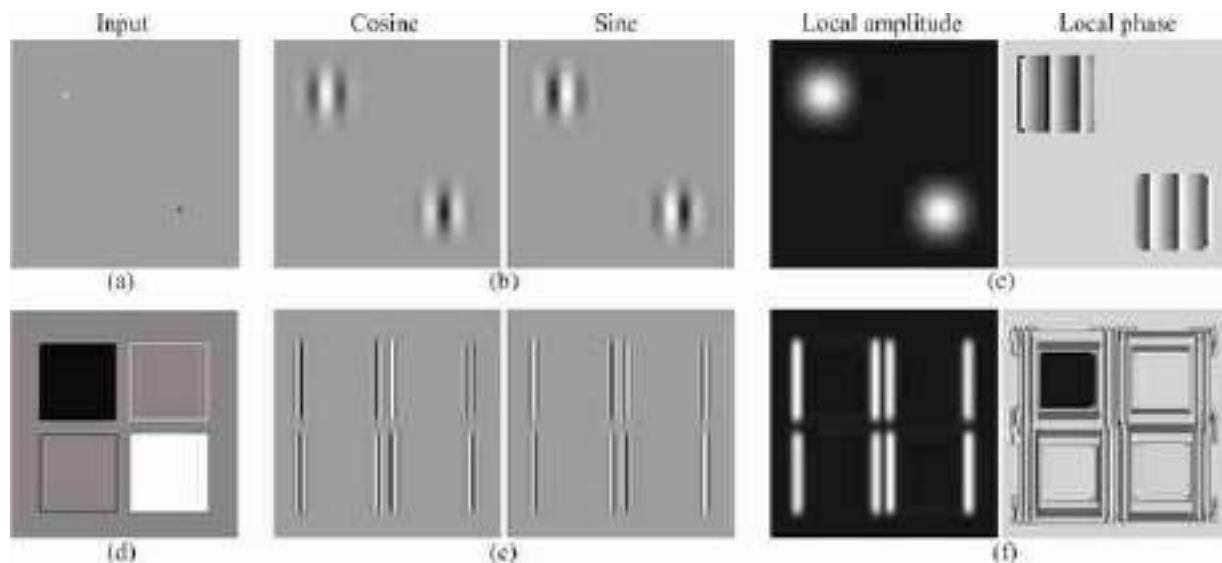


Figure 22.7: Examples of Gabor outputs to illustrate the contrast invariances present in the local amplitude. In these examples the Gabor filters are centered along the horizontal frequency axis ($\nu_0 = 0$) therefore detecting only vertical edges.

One property to notice in all these examples, is that, although the images $\ell_h(x, y)$ and $\ell_q(x, y)$ are band-pass, the amplitude $a^2(x, y)$ is a low-pass image.

22.2.3 Local Phase

The local phase is a measure of angle between the real and imaginary components (cosine and sine in the case of complex Gabor filters):

$$\theta(x, y) = \angle [\ell_h(x, y) + j\ell_q(x, y)] \quad (22.13)$$

where $\theta(x, y)$ is the instantaneous phase (local phase). This is another interesting nonlinearity although it is not commonly used in neural networks. Local image phase has been used to estimate motion [130]. This information is invariant to local changes of contrast and it is only sensitive to the local image structure.

For oriented, spatial filters, cycling through the phase of the quadrature pair of filters can generate motion along the direction of the phase change [141].

22.2.4 Gabor Filter Bank

Another useful concept is the notion of **filter bank**. A filter bank is a collection of filters, each tuned to extract a different image feature, and used to build an image representation.

As shown in [figure 22.3, 2D](#) Gabor filters are selective in spatial frequency. It is very useful to work with sets of Gabor filters, each selective to a different spatial frequency so that they cover the full space of spatial frequencies.

[Figure 22.8](#) shows two different arrangements of Gabor filters. [Figure 22.8\(a\)](#) shows a set of Gabor filters sampling the frequency domain using a rectangular grid. [Figure 22.8\(c\)](#) shows the corresponding (cosine) Gabor kernels. All the functions have the same σ . [Figure 22.8\(b\)](#) shows a polar arrangement of Gabor functions, and [figure 22.8\(d\)](#) the spatial kernels. Here, the Gaussian width σ is proportional to the distance between the central frequency and the origin. This produces filters that are rotated and scaled versions of each other.

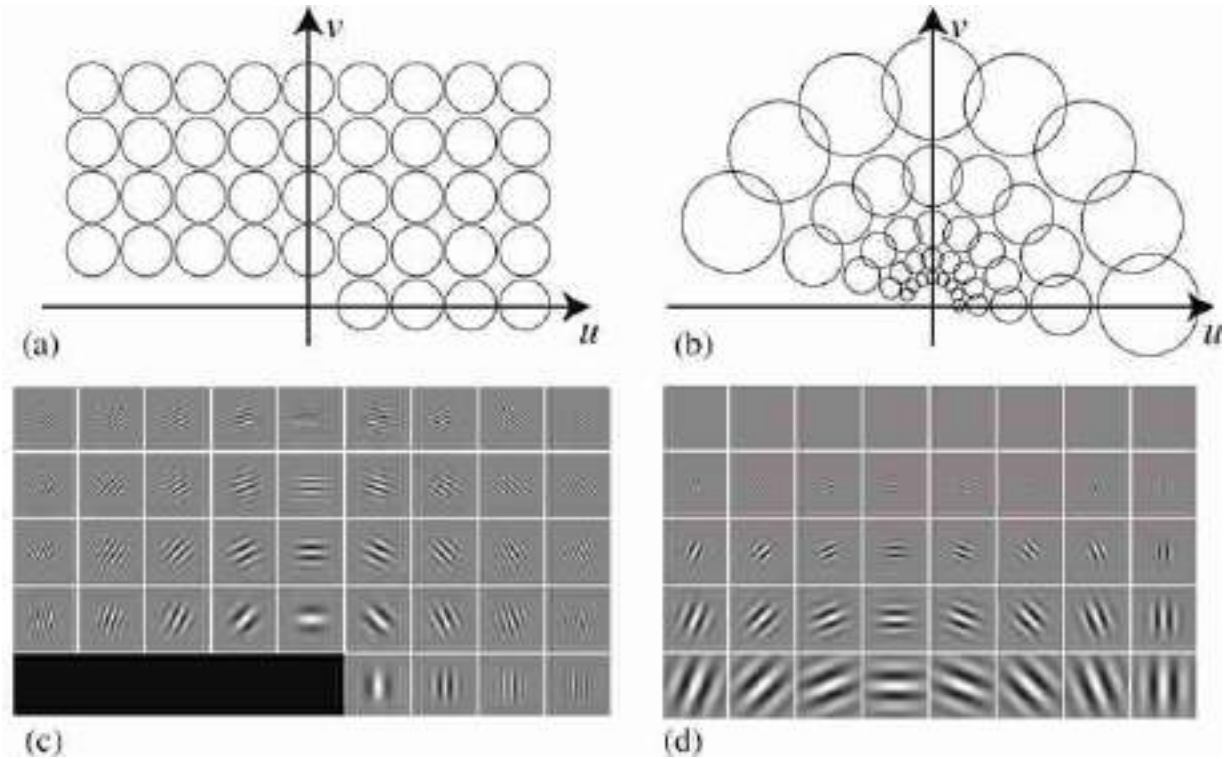


Figure 22.8: Examples of Gabor sets. Two different ways of tiling the frequency domain.

If we write in detail the convolution from equation (22.5) we can see that the convolution with a Gabor function is like doing a Fourier transform of the image after applying to it a Gaussian window:

$$\begin{aligned} \ell_{\psi}(x, y, u, v) &= \iint \ell(x', y') \psi(x-x', y-y'; u, v) dx' dy' = \\ &= \iint \ell(x', y') g(x-x', y-y') \exp(j(u(x-x') + v(y-y'))) dx' dy' = \end{aligned}$$

If we extract the term that does not depend on (x', y') , and we also use the symmetry of the Gaussian window, we get the following expression:

$$= \exp(j(ux + vy)) \iint \ell(x', y') g(x'-x, y'-y) \exp(-j(ux' + vy')) dx' dy' \quad (22.14)$$

The analogous to the Fourier transform is obtained when we vary the central frequency of the Gabor function (u, v) . This is usually called the Gabor transform. Note that the Gabor transform is not self-inverting.

22.3 Steerable Filters and Orientation Analysis

One question that arises with oriented filters is how to change their orientation. More precisely, given a filter $h(x, y)$, we want to transform it into a continuous function $h(x, y, \theta)$ of angle θ . The angle θ specifies the rotation of the original filter $h(x, y)$.

In the case of the Gabor filters, equation (22.1), each orientation requires convolving the image with a Gabor function tuned to that orientation. But do we need to create a new Gabor function for each orientation, or can we interpolate between a fixed number of predefined oriented filter outputs? How many orientations need to be sampled?

In this section we will study a generalization of the Nyquist sampling theorem but applied to dimensions different than space.

We'd like an analog in orientation for the Nyquist sampling theorem in space: given a certain number of discrete samples in orientation (or space), can one interpolate between the samples and synthesize what would have been found from having a filter (or a spatial sample) at some arbitrary, intermediate orientation? The answer is yes, and the number of filter samples needed to interpolate depends on the form of the filter.

In chapter 18, we described the simplest example of an oriented filter: a Gaussian derivative. As we discussed in equation (18.27), we can synthesize a directional derivative in any direction as a linear combination of derivatives in the horizontal and vertical directions. By the linearity of convolution, that applies to the derivative applied to any filter or image, as well. It can be seen that the **steering equation** for the first derivative of a Gaussian filter is

$$g_x(x, y, \theta) = \cos(\theta)g_x(x, y) + \sin(\theta)g_y(x, y) \quad (22.15)$$

where g_x and g_y are the Gaussian derivatives along x and y , and $g_x(x, y, \theta)$ is the derivative along the direction defined by the angle θ . The interpolation functions are $k_1(\theta) = \cos(\theta)$ and $k_2(\theta) = \sin(\theta)$.

In fact, all higher order Gaussian derivatives have the same property but the number of basis filters changes. For instance, for second-order Gaussian derivatives, the steering equation is:

$$\begin{aligned}
g_{x,x}(x, y, \theta) &= g_{xx}(\cos(\theta)x - \sin(\theta)y, \sin(\theta)x + \cos(\theta)y) = \\
&= \cos^2(\theta)g_{xx}(x, y) + \sin^2(\theta)g_{yy}(x, y) - 2\cos(\theta)\sin(\theta)g_{xy}(x, y) \quad (22.16)
\end{aligned}$$

To interpolate the derivative along any orientation requires three basis filters and the interpolation functions are $k_1(\theta) = \cos^2(\theta)$, $k_2(\theta) = \sin^2(\theta)$, and $k_3(\theta) = -2\cos(\theta)\sin(\theta)$. The minus sign in k_3 is due to the positive direction of θ to be counter-clockwise. [Figure 22.9](#) shows examples for the first- and second-order Gaussian derivatives.

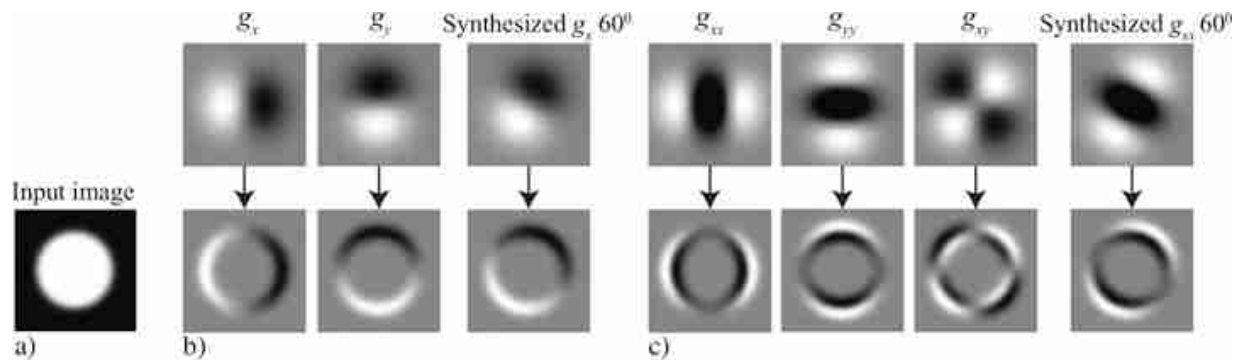


Figure 22.9: The simplest steerable filters: a first-order derivative filter of any orientation can be synthesized from a linear combination of two basis filter derivatives. The second-order derivative needs three basis.

This leads to an architecture for processing images with oriented *steerable* filters shown in [figure 22.10](#). The input images pass through a set of **basis filters**, then the outputs of those filters are modulated with a set of **gain maps** (which can be different at each pixel). Those gain maps adjust the linear combinations of the basis filters to allow the input filter to be steered to the desired orientations at each position.

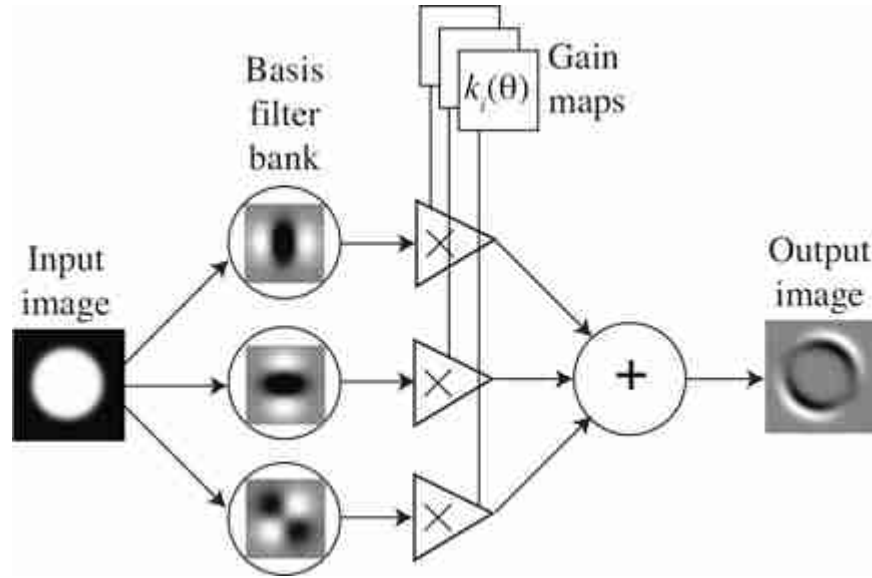


Figure 22.10: Architecture for steerable filters. This architecture computes the second-order image derivative along orientation θ .

In the case of oriented Gabor filters it is not possible to reconstruct exactly any filter orientation by interpolating filter responses. It is interesting to study the conditions in which interpolation gives exact results.

22.3.1 Steering Theorem

Let's make the previous observation more precise and general. How many basis filters does it take to steer any given filter? You could imagine that will depend on how sharply oriented the filter is. A circularly symmetric filter takes just one basis function to synthesize all other orientation responses, and a very narrow filter will take quite a few. This is quantified by steering theorems.

Let's consider a filter with impulse response $h(x, y)$. For convenience, it is better to write the filter response in polar coordinates, $h(r, \phi)$. The **steering condition** is the requirement that the rotated filter, $h(r, \phi - \theta)$, be a linear combination of a set of basis filters that are rotated versions of itself, $h(r, \phi - \theta_m)$ with $m \in (1, M)$. The steering condition is:

$$h(r, \phi - \theta) = \sum_{m=1}^M k_m(\theta) h(r, \phi - \theta_m). \quad (22.17)$$

If we express the filter to be steered as a Fourier series in angle (using complex exponentials for notational convenience), we have

$$h(r, \phi) = \sum_{n=-N}^N a_n(r) \exp(jn\phi) \quad (22.18)$$

Substituting equation (22.18) into equation (22.17), we have an equation for the interpolation functions, $k_j(\theta)$. The steering condition, equation (22.17), holds for functions expandable in the form of equation (22.18) if and only if the interpolation functions $k_j(\theta)$ are solutions of:

$$\begin{bmatrix} 1 \\ \exp(j\theta) \\ \dots \\ \exp(jN\theta) \end{bmatrix} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ \exp(j\theta_1) & \exp(j\theta_2) & \dots & \exp(j\theta_M) \\ \vdots & \vdots & \ddots & \vdots \\ \exp(jN\theta_1) & \exp(jN\theta_2) & \dots & \exp(jN\theta_M) \end{bmatrix} \begin{bmatrix} k_1(\theta) \\ k_2(\theta) \\ \vdots \\ k_M(\theta) \end{bmatrix}. \quad (22.19)$$

Let's check this for a simple example. Our derivative of a Gaussian filter (equation [18.17]) is x times a Gaussian. When changing to polar coordinates $x = r \cos(\theta)$, which gives an angular distribution times a radially symmetric Gaussian when written in polar coordinates. This requires two complex exponentials to write (to create the $\cos(\theta)$ from complex exponentials) and thus requires two basis functions to steer.

Sometimes its more convenient to think of the filters as polynomials times radially symmetric window functions (this is the case for high-order Gaussian derivatives). Then you can show that for an N -th order polynomial with even or odd symmetry $N + 1$ basis functions are sufficient [140].

Although everything has been derived in the continuous domain, steerability is a property that still holds after sampling the filter function. This is because spatial sampling and steerability are interchangeable: the weighted sum of spatially sampled function is equal to the spatial sampling of the same weighted sum of continuous basis functions.

For computational efficiency, it's more convenient to have the basis filters all be $x - y$ separable functions. In many cases, it's straightforward to find such basis functions, and where it's not, there are simple numerical methods to find the best fitting $x - y$ separable basis set.

22.3.2 Steerable Quadrature Pairs

As in the case of Gabor filters, it is useful to build quadrature pairs of steerable filters. Steerable-quadrature filters allow for arbitrary shifts both in orientation and in phase. We can design such filters. For instance, let's consider the second-order derivatives of a Gaussian with $\sigma^2 = 1/2$ and

normalized so that the integral over all the space of its squared magnitude equals 1:

$$\begin{aligned} g_{xx}(x, y) &= 0.9213(2x^2 - 1) \exp(-(x^2 + y^2)) \\ g_{xy}(x, y) &= 1.843(xy) \exp(-(x^2 + y^2)) \\ g_{yy}(x, y) &= 0.9213(2y^2 - 1) \exp(-(x^2 + y^2)) \end{aligned} \quad (22.20)$$

It is possible to get a good approximation to its Hilbert transform using a Gaussian times a third-order odd polynomial. The approximation to the Hilbert transform of $g_{xx}(x, y)$ is:

$$h_{xx}(x, y) = -0.9780(-2.254x + x^3) \exp(-(x^2 + y^2)) \quad (22.21)$$

where $g_{xx}(x, y)$ and its Hilbert transform $h_{xx}(x, y)$ have the same spectral content, but the opposite phase. [Figure 22.11](#) analyzes the quality of the approximation. The figure shows the quadrature pair, g_{xx} and h_{xx} , sampled in space and cropped into a window of 13×13 pixels, and its Fourier transform. The DFT is computed by zero padding to 256×256 pixels. In [figure 22.11\(a\)](#) g_{xx} is the even phase, and in [figure 22.11\(b\)](#) h_{xx} is the odd phase. [Figure 22.11\(c\)](#) illustrates that the sum of their squares reveals the square of their Gaussian envelopes. [Figure 22.11\(d\)](#) illustrates a section of the magnitude of their Fourier transforms along the u axis for $v = 0$. The h_{xx} is a sampled third-order polynomial approximation to the Hilbert transform of g_{xx} , so their power spectra may not be exactly the same. The blue line shows the magnitude of the analytic filter $g_{xx} + jh_{xx}$. It has double amplitude, and the content for negative frequencies is close to zero.

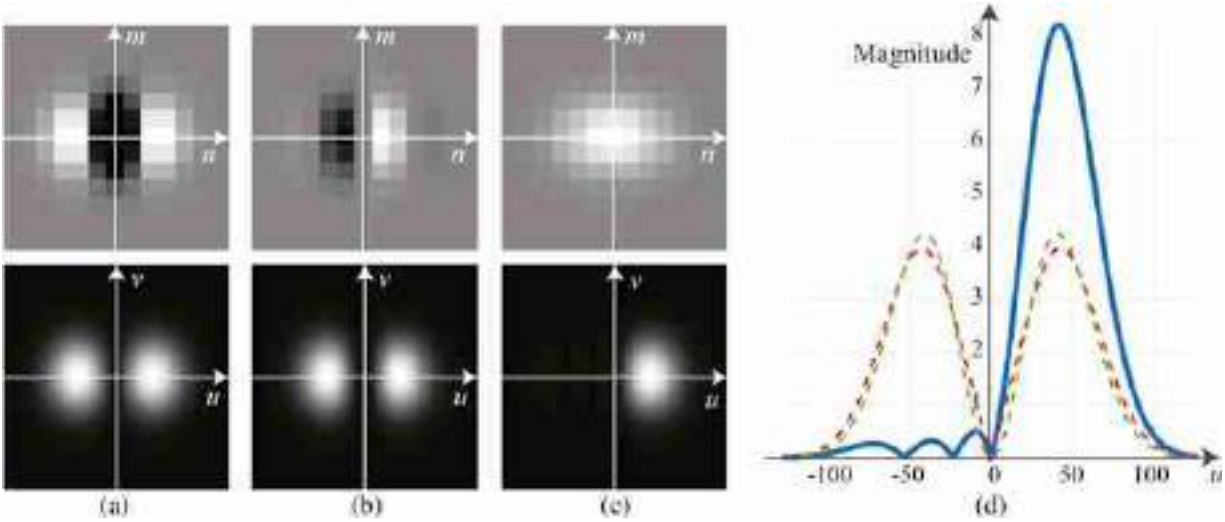


Figure 22.11: Quadrature pair, $g_{xx} [n, m]$ and $h_{xx} [n, m]$. The filters are sampled in space and cropped into a window of 13×13 pixels.

To steer $h_{xx}(x, y)$ to an angle θ , this approximation requires four basis functions, and not just three as for the second-order derivative of a Gaussian. The other three functions needed are:

$$\begin{aligned} h_2(x, y) &= -0.9780(-0.7515 + x^2)y \exp(-(x^2 + y^2)) \\ h_3(x, y) &= -0.9780(-0.7515 + y^2)x \exp(-(x^2 + y^2)) \\ h_4(x, y) &= -0.9780(-2.254y + y^3) \exp(-(x^2 + y^2)) \end{aligned} \quad (22.22)$$

These functions have been optimized in order to be $x - y$ separable. The basis function for steerability are not unique. For instance, [figures 22.12\(a and c\)](#) show two basis functions that span all rotations of g_{xx} . [Figure 22.12\(a\)](#) has separable filters corresponding to g_{xx} , g_{xy} and g_{yy} , and [figure 22.12\(c\)](#) shows three rotated versions of g_{xx} at 0, 60, and 120 degrees. [Figure 22.12\(c\)](#) is a non-separable basis spanning the same space as the filters of [figure 22.12\(a\)](#). [Figures 22.12\(b and d\)](#) show two basis functions that span all rotations on h_{xx} spanning the same space as the filters of [figure 22.12\(b\)](#). Spatial scaling of the filters will result in changing σ .

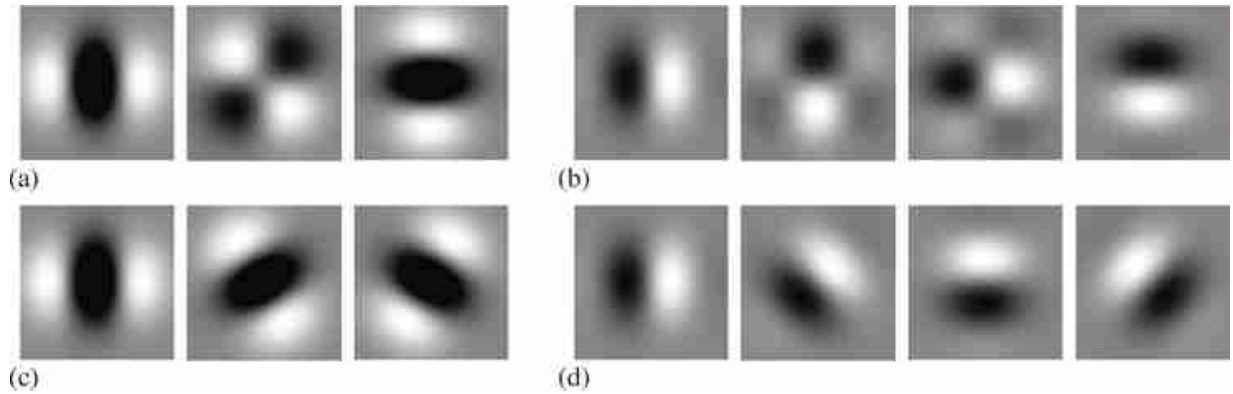


Figure 22.12: (a) Second derivative of Gaussian, $x - y$ separable steerable basis set. (b) Approximation to Hilbert transform of second derivative of Gaussian, $x - y$ steerable basis set. (c) Nonseparable basis equivalent to (a). (d) Nonseparable basis set equivalent to (b).

Putting it all together, can we compute oriented energy as a function of angle, for all angles, just from the seven basis filter responses shown in [figure 22.12](#).

$$E(x, y, \theta) = g_{xx}(x, y, \theta)^2 + h_{xx}(x, y, \theta)^2 \quad (22.23)$$

From the basis filter responses we can form polar plots of the oriented energy as a function of angle ([figure 22.13](#)). Note some strange goings on at intersections using the g_{xx} , h_{xx} filters. You might think this is a result of simply not enough angular resolution from those filters. While it's true that the fourth-order Gaussian derivatives and their quadrature pairs don't suffer from that problem, the g_{xx} , h_{xx} filters actually do have enough angular resolution. In this case, the issue is a more subtle one.

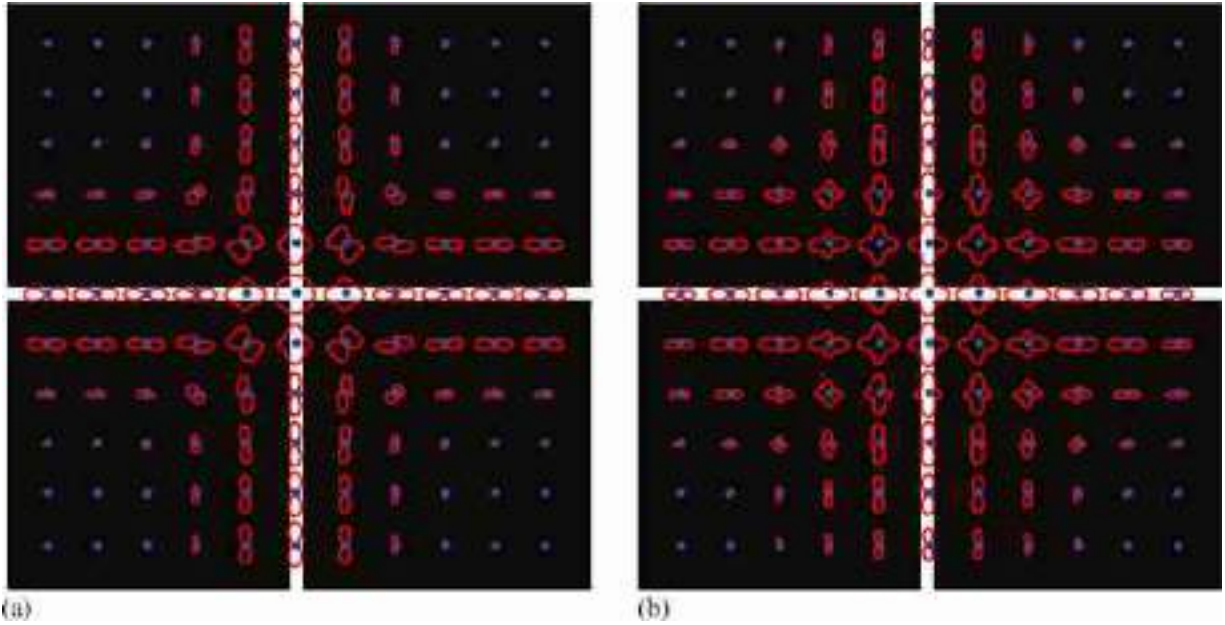


Figure 22.13: (Polar plots of orientation energy as a function of angle, computed using g_{xx} , h_{xx} filters. a) Note the non-superposition of orientated energies near the junction of the two lines. b) Spatially blurring the oriented energy components of the filters results in much improved linear superposition of the orientation plots, removing spurious interference terms, as described in the text.

When there are two oriented structures within the passband of the quadrature pair filters, the sum of the energies of the individual structures is not the same as the energy of the sum of the structures. Because we're squaring to find the energies, the combination of multiple structures isn't linear. As [figure 22.13\(a\)](#) shows, when there are two oriented structures within the passband, when the filter responses are squared, the convolution in the Fourier domain picks up extra cross-terms from the one oriented structure interacting with the other, in addition to the desired term from simply squaring all the frequency responses individually within the passband. These cross terms show up as spurious spatial frequencies in the energy term, and we can get rid of them by spatially low-pass filtering the squared oriented energy responses. Using the blurred squared basis filter responses, we get much cleaner oriented energy as a function of angle plots, even with the g_{xx} , h_{xx} filters in the junctions, [figure 22.13\(b\)](#).

[Figure 22.14](#) shows two examples on real images. At each pixel, the polar plots reveal the local image structure.

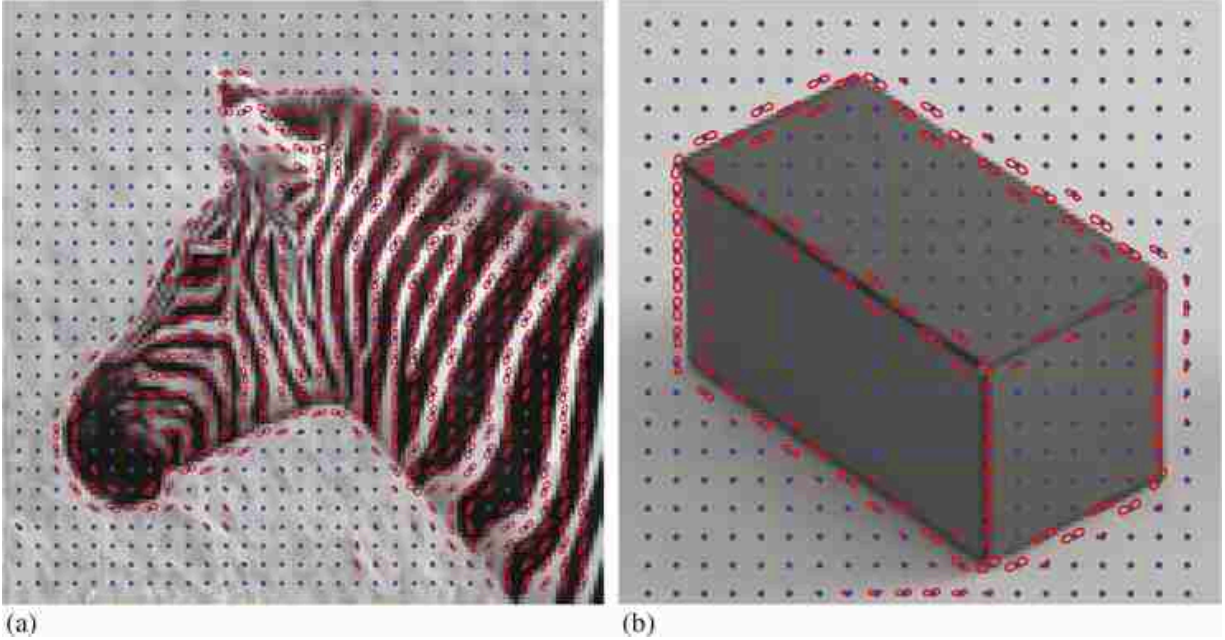


Figure 22.14: Polar plots of orientation energy as a function of angle, computed using g_{xx} , h_{xx} filters in two images.

We can also make steerable filters in three dimensions (3D), allowing us to analyze medical volumetric data, or to process spatiotemporal volumes to measure image motion.

22.4 Motion Analysis

Spatiotemporal filter banks are a basic building block to build video understanding vision systems. In this chapter we will describe some traditional filters that are similar to filters learned when using neural networks.

22.4.1 Spatiotemporal Gabor Filters

Just as we did with Gaussian derivatives, extending Gabor filter for motion analysis is a direct generalization of the $x - y$ 2D Gabor function to a $x - y - t$ 3D Gabor function. [Figure 22.15\(a\)](#) shows a $x - t$ (cosine and sine) Gabor function in one spatial dimension, and [figure 22.15\(b\)](#) shows a sketch of its Fourier transform. This function is selective to signals translating to the right with a speed $v = 1$, i.e. $\ell(x - t)$. The red line in [figure 22.15\(b\)](#) shows Dirac line that contains the energy of the moving signal.

In two spatial dimensions, [figure 22.15\(c\)](#) shows the sketch of the Gabor transfer function. Note that the $x - y - t$ Gabor filter is not selective to velocity. If we have a 2D moving signal $\ell(x - v_x t, y - v_y t)$, the Fourier transform is contained inside a Dirac plane. Therefore, there are an infinite number of planes that will pass by the frequencies of the Gabor filter. All those planes intersect the red line shown in [figure 22.15\(c\)](#). A single Gabor filter cannot disambiguate the input velocity.

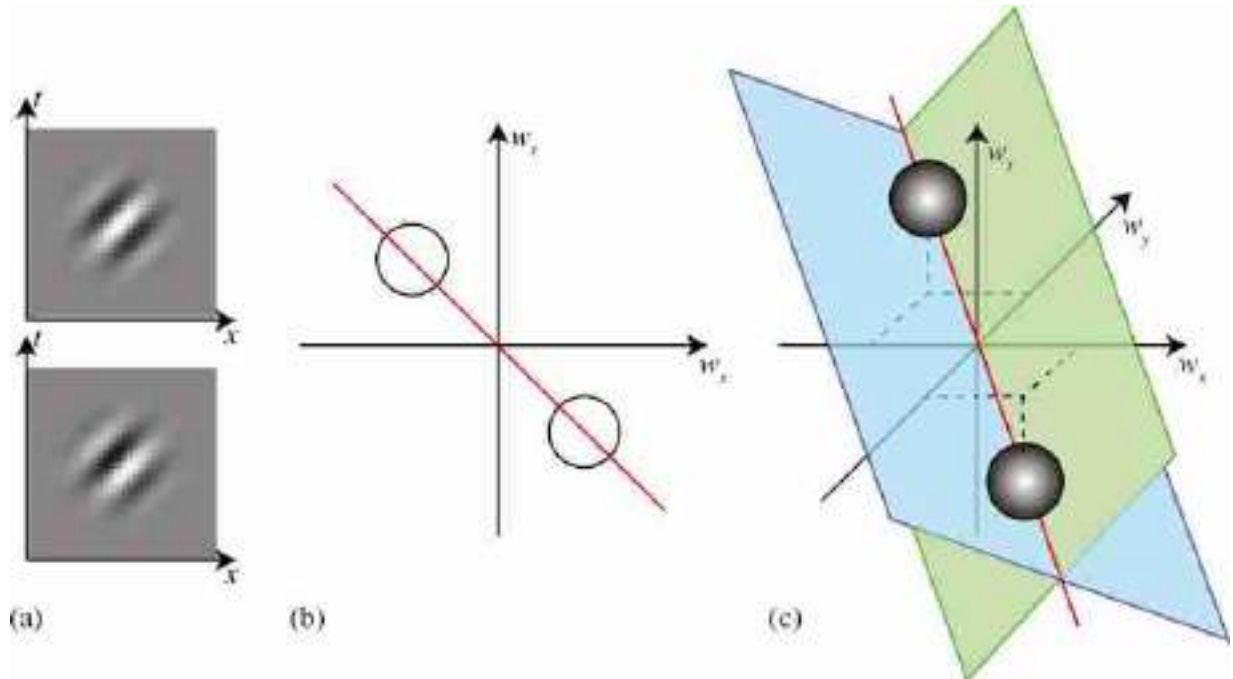


Figure 22.15: Space-time Gabor filters. (a) Cosine and sine $x-t$ Gabor filter, and (b) the sketch of its transfer function. (c) Sketch of the transfer function of a spatiotemporal Gabor filter in two spatial dimensions ($x-y-t$). The two planes show examples of spatiotemporal planes that intersect the Gabor filter in the same way.

22.4.2 Velocity-Tuned Filters

How can we measure input velocity? There are many different approaches in the computer vision community for measuring motion, and we will study them in chapter 46. Here we show that it is possible to measure motion even with the simple processing machinery that we've developed so far.

We can use quadrature pairs of oriented filters in space-time to find motion speed and direction in the video signal. We just need to find the space-time orientation of strongest response. [Figure 22.16\(a\)](#) shows a set of Gabor filters sampling the space-time frequency domain. When the input

contains a moving signal, we can use a set of filters to identify the plane in the Fourier domain that contains the input energy. [Figure 22.16](#)(b) shows the subset of filters that have the strongest output for a particular input motion.

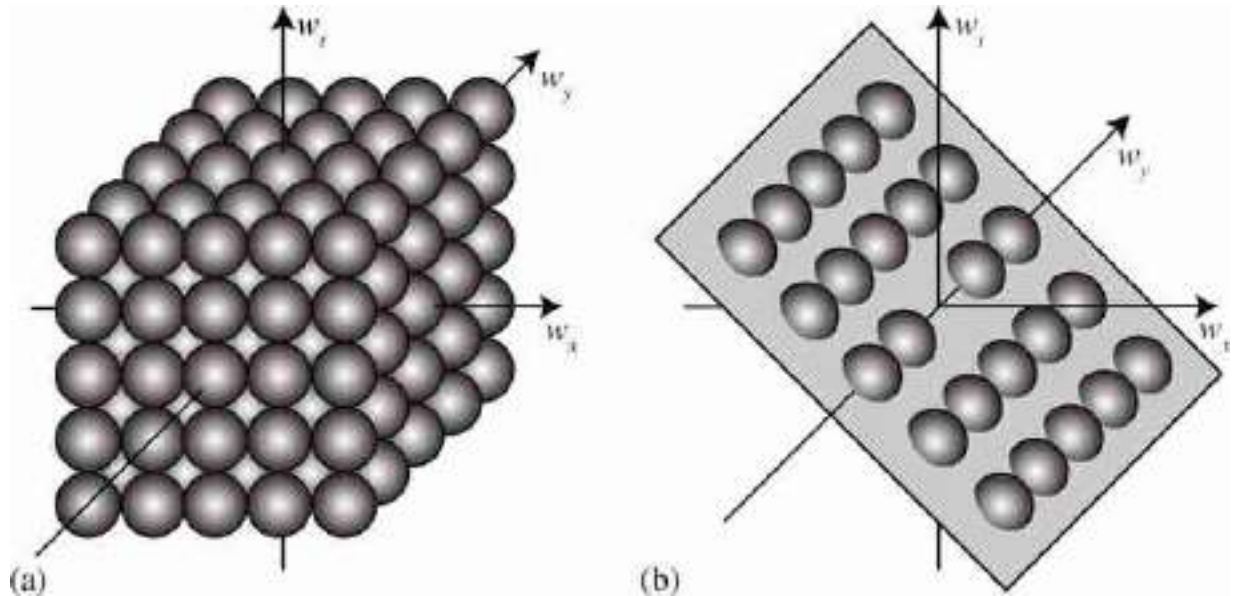


Figure 22.16: (a) Space-time Gabor filters tiling the full spatiotemporal frequency domain. (b) Subset of the Gabor filters selective to a particular velocity.

As an illustration, [figure 22.17](#) shows one possible architecture to create velocity-selective units. The first layer is composed by space-time Gabor filters (cosine and sine), which are frequency-selective units. Here we represent the impulse response of each filter by a small $x - y - t$ cube. For each quadrature pair we compute the amplitude. Then amplitude outputs are combined according to form different planes in the Fourier domain to create velocity-selective outputs. A normalization layer can be added to normalize the outputs by dividing every output by the sum of all the amplitudes (not shown). The full architecture is nonlinear.

Given an input sequence, one can estimate velocity by looking at the velocity-tuned unit with the strongest response.

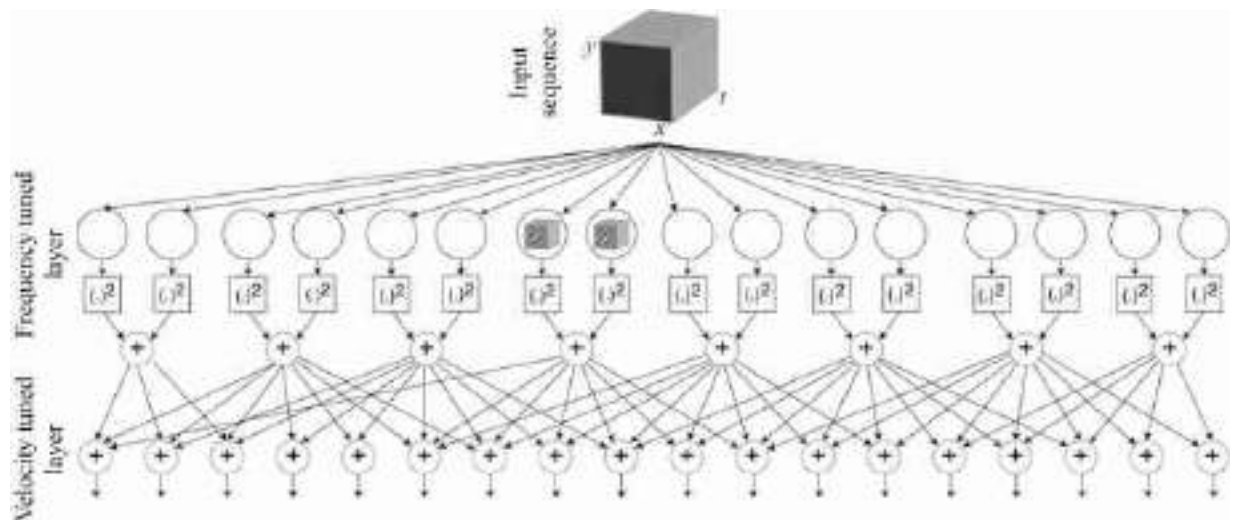


Figure 22.17: Architecture to create velocity-selective units. In the first layer, cosine and sine filters are combined to create phase invariant frequency-tuned outputs. In the second layer, the outputs of spatiotemporal Gabor filters are grouped according to different planes in the Fourier domain to create velocity-selective outputs.

Figure 22.17: Architecture to create velocity-selective units. In the first layer, cosine and sine filters are combined to create phase invariant frequency-tuned outputs. In the second layer, the outputs of spatiotemporal Gabor filters are grouped according to different planes in the Fourier domain to create velocity-selective outputs.

22.5 Concluding Remarks

In this chapter we have seen the power of using filter banks with some simple nonlinearities.

Hand-crafted filter banks started as a model of low-level vision mechanisms in humans, and became the basis to perform many visual tasks such as texture analysis [202], image segmentation [383], motion analysis [201], orientation analysis [139], image denoising [440], and many others.

These approaches had an advantage in that they were based on first principles and required no training data. However, note that performances when using hand-crafted architectures were limited. Many of the architectures we described in this chapter can be seen as precursors to many of the learning-based architectures we will discuss in subsequent chapters.

Before we dive into learning-based architectures, we will study multiscale image pyramids in the following chapter.

23 Image Pyramids

23.1 Introduction

In chapter 15 we motivated translation invariant linear filters as a way of accounting for the fact that objects in images might appear at any location. Therefore, a reasonable way of processing an image is by manipulating pixel neighborhoods in the same way independently on the image location. In addition to translation invariance, scale invariance is another fundamental property of images. Due to perspective projection, objects at different distances will appear with different sizes as shown in [figure 23.1](#). Therefore, if we want to locate all the bird instances in this image, we will have to apply an operator that is invariant in translation and in scale. Image pyramids provide an efficient representation for space-scale invariant processing.



Figure 23.1: Objects in images appear at arbitrary locations and with arbitrary image sizes.

23.2 Image Pyramids and Multiscale Image Analysis

Image information occurs over many different spatial scales. Image pyramids (i.e., multiresolution representations for images) are a useful data structure for analyzing and manipulating images over a range of spatial scales. Here we'll discuss three different ones, in a progression of complexity. The first is a Gaussian pyramid, which creates versions of the input image at multiple resolutions. This is useful for analysis across different spatial scales, but doesn't separate the image into different frequency bands. The Laplacian pyramid provides that extra level of analysis, breaking the image into different isotropic spatial frequency bands. The steerable pyramid provides a clean separation of the image into different scales and orientations. There are various other differences between these pyramids, which we'll describe below.

As a motivating example, let's assume we want to detect the birds from [figure 23.1](#). If we have a template of a bird, normalized correlation will be able to detect only the birds that have a similar image size than the template. To introduce **scale invariance**, one possible solution is to change the size of the template to cover a wide range of possible sizes and apply them to the image. Then, the ensemble of templates will be able to detect birds of different sizes. The disadvantage of this approach is that it will be computationally expensive as detecting large birds will require computing convolutions with big kernels, which is very slow. Another alternative is to change the image size as shown in [figure 23.2](#), resulting in a **multiscale image pyramid**. In this example, the original image has a resolution of 848×643 pixels. Each image in the pyramid is obtained by scaling down the image from the previous level by reducing the number of pixels by a factor of 25 percent (that is, each image in the pyramid has $3/4$ of the size of the precedent image)⁹.

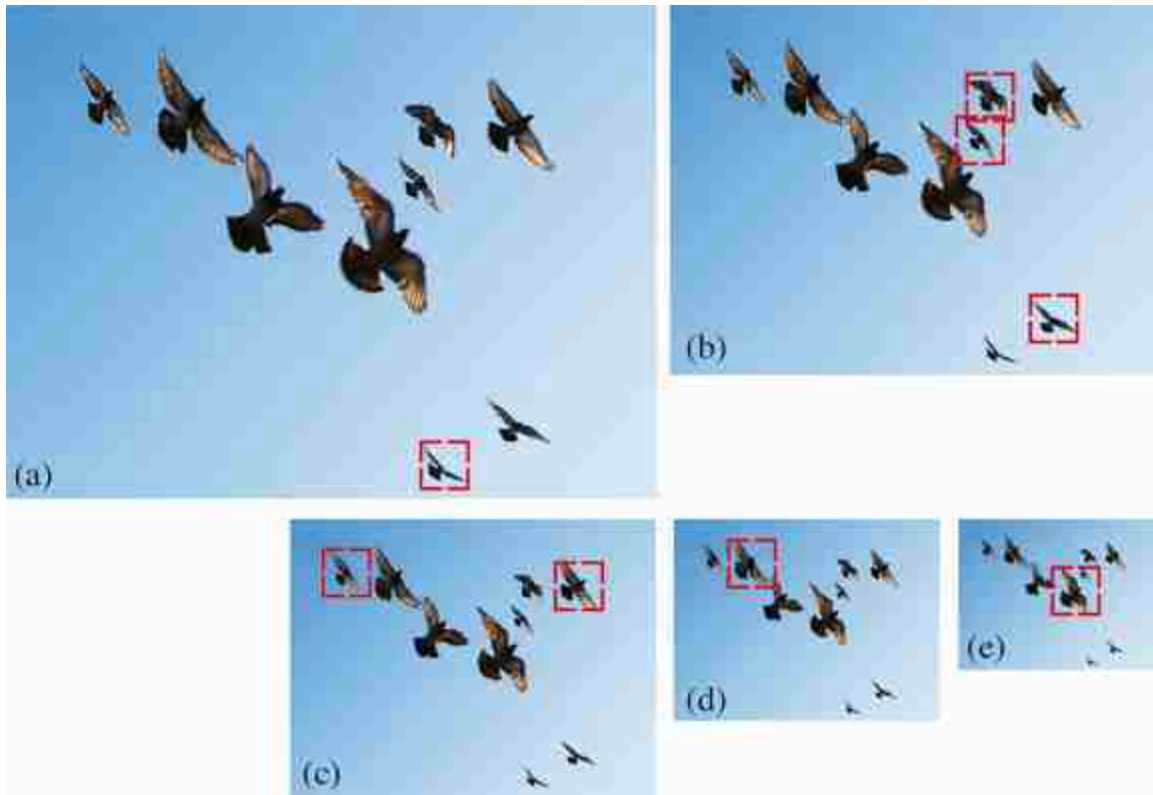


Figure 23.2: Multiscale image pyramid. Each image is 25 percent smaller than the previous one. The red box indicates the size of a template used for detecting birds. As the size of the template is fixed; it will only be able to detect the birds that tightly fit inside the box. By running the same template across many levels in this pyramid, different instances of birds are detected at different scales.

Now we can use the pyramid to detect birds at different sizes using a single template. The red box in the figure denotes the size of the template used. The figure shows how birds of different sizes become detectable at, at least, one of the levels of the pyramid. This method will be more efficient as the template can be kept small and the convolutions will remain computationally efficient.

Mutiscale image processing and image pyramids have many applications beyond scale invariant object detection. In this chapter we will describe some important image pyramids and their applications.

23.3 Linear Image Transforms

Let's first look at some general properties of linear image transforms. For an input image ℓ with N pixels, a linear transform is:

$$\mathbf{r} = \mathbf{P}^T \ell \quad (23.1)$$

where $\mathbf{r}$ is a vector of dimensionality M , and $\mathbf{P}$ is a matrix of size $N \times M$. The columns of $\mathbf{P} = [\mathbf{P}_0, \mathbf{P}_1, \dots, \mathbf{P}_{M-1}]$ are the projection vectors. The vector $\mathbf{r}$ contains the transform coefficients: $r_i = \mathbf{P}_i^T \ell$. The vector $\mathbf{r}$ corresponds to a different representation of the image ℓ than the original pixel space.

We are interested in transforms that are invertible, so that we can recover the input ℓ from the projection coefficients $\mathbf{r}$:

$$\ell = \mathbf{Q} \mathbf{r} = \sum_{i=0}^{M-1} r_i \mathbf{Q}_i \quad (23.2)$$

The columns of $\mathbf{Q} = [\mathbf{Q}_0, \mathbf{Q}_1, \dots, \mathbf{Q}_{M-1}]$ are the basis vectors. The input signal ℓ can be reconstructed as a linear combination of the basis vectors $\mathbf{Q}_i$ weighted by the representation coefficients r_i .

The transform $\mathbf{P}$ is said to be **critically sampled** when $M = N$. The transform is **oversampled** when $M > N$, and **undersampled** when $M < N$. The transform $\mathbf{P}$ is complete, that is, encoding all image structure, if it is invertible. If critically sampled (i.e., $M = N$) and the transform is complete, then $\mathbf{Q} = (\mathbf{P}^T)^{-1}$. If it is overcomplete (oversampled and complete), then the inverse can be obtained using the pseudoinverse $\mathbf{Q} = (\mathbf{P} \mathbf{P}^T)^{-1} \mathbf{P}$.

An important special case is when the transform is **self-inverting**, then $\mathbf{P} \mathbf{P}^T = \mathbf{I}$. The values of $\mathbf{P}$ can be real or complex (like in the Fourier transform). For complex transforms, we should replace the $\mathbf{P}^T$ by $\mathbf{P}^{*T}$ (i.e., complex conjugate transpose).

The quadrature mirror filter (QMF) transform is an example of self-inverting transform [10]. For 1D signals of length 4, the QMF transform can be written as:

$$\mathbf{P} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{bmatrix}$$

This is equivalent to the convolution of the input signal with two orthogonal kernels, $[1, 1]$ and $[1, -1]$, with a stride of 2.

This transform also has a multiscale version that is also self-inverting:

$$\mathbf{P} = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

You can check that in both cases $\mathbf{Q} = (\mathbf{P}^T)^{-1} = \mathbf{P}^T$. These transforms can be extended to 2D.

23.4 Gaussian Pyramid

We'd like to make a recursive algorithm for creating a multiresolution version of an image. A Gaussian filter is a natural one to use to blur out an image, since the multiple successive application of a Gaussian filter is equivalent to application of a single, wider Gaussian filter.

Here's an elegant, efficient algorithm for making a resolution-reduced version of an input image. It involves two steps: convolving the image with a low-pass filter (e.g., using the fourth binomial filter $\mathbf{b}_4 = [1, 4, 6, 4, 1] / 16$, normalized to sum to 1, separably in each dimension), and then subsampling by a factor of 2 the result. Each level is obtained by filtering the previous level with the fourth binomial filter with a stride of 2 (on each dimension). Applied recursively, this algorithm generates a sequence of images, subsequent ones being smaller, lower resolution versions of the earlier ones in the processing.

[Figure 23.3](#) shows the Gaussian pyramid of an image with six levels. Each level has half the resolution of the previous level.



Figure 23.3: Gaussian pyramid with six levels. The first level, $\mathbf{g}_0$, is the input image. The Gaussian pyramid is built for each color channel independently.

To make the filters more intuitive, it is useful to write the two steps in matrix form. The following matrix shows the recursive construction of level $k + 1$ of the Gaussian pyramid for a one-dimensional (1D) image:

$$\mathbf{g}_{k+1} = \mathbf{D}_k \mathbf{B}_k \mathbf{g}_k = \mathbf{G}_k \mathbf{g}_k \quad (23.3)$$

where $\mathbf{D}_k$ is the downsampling operator, $\mathbf{B}_k$ is the convolution with the fourth binomial filter, and $\mathbf{G}_k = \mathbf{D}_k \mathbf{B}_k$ is the blur-and-downsample operator for level k .

One **block** of the Gaussian pyramid computation.



We call the sequence of images $\mathbf{g}_0, \mathbf{g}_1, \dots, \mathbf{g}_N$ as the Gaussian pyramid. The first level of the Gaussian pyramid is the input image: $\mathbf{g}_0 = \ell$.

It is useful to check a concrete example. If $\mathbf{x}$ is a 1D signal of length 8, and if we assume zero boundary conditions, the matrices for computing $\mathbf{g}_1$ are:

$$\mathbf{G}_0 = \mathbf{D}_0 \mathbf{B}_0 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \frac{1}{16} \begin{bmatrix} 6 & 4 & 1 & 0 & 0 & 0 & 0 & 0 \\ 4 & 6 & 4 & 1 & 0 & 0 & 0 & 0 \\ 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 \\ 0 & 1 & 4 & 6 & 4 & 1 & 0 & 0 \\ 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 \\ 0 & 0 & 0 & 1 & 4 & 6 & 4 & 1 \\ 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 \\ 0 & 0 & 0 & 0 & 0 & 1 & 4 & 6 \end{bmatrix} \quad (23.4)$$

Multiplying the two matrices:

$$\mathbf{g}_1 = \mathbf{G}_0 \boldsymbol{\ell} = \frac{1}{16} \begin{bmatrix} 6 & 4 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 \end{bmatrix} \boldsymbol{\ell} \quad (23.5)$$

the first level of the Gaussian pyramid is a signal $\mathbf{g}_1$ with length 4. Applying the recursion we can write the output of each level as a function of the input $\boldsymbol{\ell}$: $\mathbf{g}_2 = \mathbf{G}_1 \mathbf{G}_0 \boldsymbol{\ell}$, $\mathbf{g}_3 = \mathbf{G}_2 \mathbf{G}_1 \mathbf{G}_0 \boldsymbol{\ell}$, and so on.

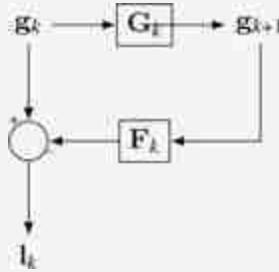
23.5 Laplacian Pyramid

In the Gaussian pyramid, each level losses some of the fine image details available in the previous level. The **Laplacian pyramid** [66] is simple: it represents, at each level, what is present in a Gaussian pyramid image of one level, but not present at the level below it. We calculate that by expanding the lower-resolution Gaussian pyramid image to the same pixel resolution as the neighboring higher-resolution Gaussian pyramid image, then subtracting the two. This calculation is made in a recursive, telescoping fashion.

Let's look at the steps for calculating a Laplacian pyramid. What we want is to compute the difference between $\mathbf{g}_k$ and $\mathbf{g}_{k+1}$. To do this first we need to upsample the image $\mathbf{g}_{k+1}$ so that it has the same size as $\mathbf{g}_k$. Let $\mathbf{F}_k = \mathbf{B}_k \mathbf{U}_k$ be the upsample-and-blur operator for pyramid level k . The operator $\mathbf{F}_k$ applies first the upsampling operator $\mathbf{U}_k$, that inserts zeros between samples, followed by blurring by the same filter $\mathbf{B}_k$ than the one we used for the Gaussian pyramid. The Laplacian pyramid coefficients, $\mathbf{l}_k$, at pyramid level k , are:

$$l_k = g_k - F_k g_{k+1} = (I_k - F_k G_k) g_k = L_k g_k \quad (23.6)$$

One **block** of the Laplacian pyramid computation.



[Figure 23.4](#) shows the resulting Laplacian pyramid for an image. To compute a Laplacian pyramid with N levels, we need to first compute the Gaussian pyramid of the input image with $N + 1$ levels. The last level of this pyramid is the smallest level of the Gaussian pyramid used to compute the Laplacian pyramid and is called the low-pass residual.



Figure 23.4: Laplacian pyramid, including the tiny low-pass residual as the last image. The Laplacian pyramid is built for each color channel independently.

Let's write down the matrices in equation (23.6) for a 1D input ℓ of length 8, and assuming zero boundary conditions. The operators to compute the first level ($k = 0$) of the Laplacian pyramid are:

$$\mathbf{F}_0 = 2\mathbf{B}_0\mathbf{U}_0 = \frac{1}{8} \begin{bmatrix} 6 & 4 & 1 & 0 & 0 & 0 & 0 & 0 \\ 4 & 6 & 4 & 1 & 0 & 0 & 0 & 0 \\ 1 & 4 & 6 & 4 & 1 & 0 & 0 & 0 \\ 0 & 1 & 4 & 6 & 4 & 1 & 0 & 0 \\ 0 & 0 & 1 & 4 & 6 & 4 & 1 & 0 \\ 0 & 0 & 0 & 1 & 4 & 6 & 4 & 1 \\ 0 & 0 & 0 & 0 & 1 & 4 & 6 & 4 \\ 0 & 0 & 0 & 0 & 0 & 1 & 4 & 6 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (23.7)$$

The factor 2 is necessary because inserting zeros decreases the average value of the signal $\mathbf{g}_{k+1}$ by a factor of 2. Multiplying the two matrices:

$$\mathbf{l}_0 = \mathbf{g}_0 - \mathbf{F}_0\mathbf{g}_1 = \mathbf{g}_0 - \frac{1}{8} \begin{bmatrix} 6 & 1 & 0 & 0 \\ 4 & 4 & 0 & 0 \\ 1 & 6 & 1 & 0 \\ 0 & 4 & 4 & 0 \\ 0 & 1 & 6 & 1 \\ 0 & 0 & 4 & 4 \\ 0 & 0 & 1 & 6 \\ 0 & 0 & 0 & 4 \end{bmatrix} \mathbf{g}_1 \quad (23.8)$$

We can also calculate the matrix that should be applied to the input $\boldsymbol{\ell} = \mathbf{g}_0$.

$$\mathbf{l}_0 = (\mathbf{I} - \mathbf{F}_0\mathbf{G}_0)\mathbf{g}_0 = \frac{1}{256} \begin{bmatrix} 182 & -56 & -24 & -8 & -2 & 0 & 0 & 0 \\ -56 & 192 & -56 & -32 & -8 & 0 & 0 & 0 \\ -24 & -56 & 180 & -56 & -24 & -8 & -2 & 0 \\ -8 & -32 & -56 & 192 & -56 & -32 & -8 & 0 \\ -2 & -8 & -24 & -56 & 180 & -56 & -24 & -8 \\ 0 & 0 & -8 & -32 & -56 & 192 & -56 & -32 \\ 0 & 0 & -2 & -8 & -24 & -56 & 182 & -48 \\ 0 & 0 & 0 & 0 & -8 & -32 & -48 & 224 \end{bmatrix} \boldsymbol{\ell} \quad (23.9)$$

Interestingly, the Laplacian pyramid is an invertible transform, but only if we keep the low-pass residual. We can reconstruct the original image from the Laplacian pyramid. Using the **low-pass residual** signal associated with the Laplacian pyramid, we can recursively reconstruct the corresponding Gaussian pyramid. Remember that in the Gaussian pyramid level 1 is just the original image itself, so we can use this to reconstruct the original image from the Laplacian pyramid. We can do it recursively applying, from $k = N - 1$ to $k = 0$:

$$\mathbf{g}_k = \mathbf{l}_k + \mathbf{F}_k \mathbf{g}_{k+1} \quad (23.10)$$

The diagram in [figure 23.5](#) shows the Gaussian pyramid, the Laplacian pyramid and the Laplacian inversion for a three-level Laplacian pyramid. The reconstruction uses the Laplacian pyramid $\mathbf{l}_0, \mathbf{l}_1, \dots, \mathbf{l}_2$ and the low pass residual $\mathbf{g}_3$ to recover the input signal $\ell = \mathbf{g}_0$.

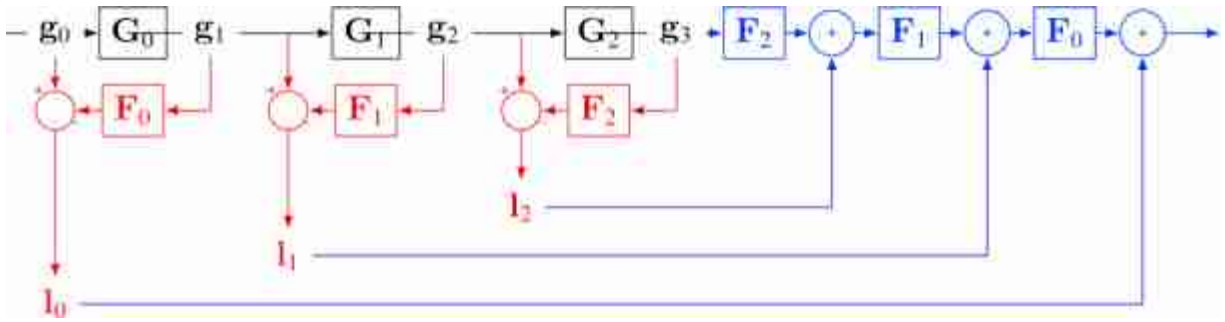


Figure 23.5: The diagram shows the Gaussian pyramid (black), the Laplacian pyramid (red) and the Laplacian inversion for a three-level Laplacian pyramid (blue).

This architecture has two parts: (1) the analysis network (or **encoder**) that transforms the input image $\mathbf{x}$ into a representation composed of $\mathbf{l}_0, \mathbf{l}_1, \dots$ and the low pass residual $\mathbf{x}_n$; and (2) the synthesis network (or **decoder**) that reconstructs the input from the representation. The Laplacian pyramid is an **overcomplete representation** (more coefficients than pixels); the dimensionality of the representation is higher than the dimensionality of the input.

Encoder and **decoder** networks are also common building blocks of deep neural networks. The Laplacian pyramid can be considered as a special type of deep neural net.

Note that the reconstruction property of the Laplacian pyramid does not depend on the filters used for subsampling and upsampling. Even if we used random filters the reconstruction property would still hold.

23.5.1 Image Blending

Image blending (or image compositing) consists in introducing elements of one image inside another. Creating convincing composite images is challenging, as the edge between the two images has to be invisible. One

way of formalizing the image-blending operation is by defining a mask, $\mathbf{m}$, that specifies how the images will be combined. The mask is used to select which pixels come from each of the two images in order to create the composite image. For example, let's blend two images using the mask as shown in [figure 23.6](#).



Figure 23.6: Two images to be blended and the blending mask.

In this example, the mask indicates that the blended image will combine the half-left of the first image with the right-half of the second image. One naive solution to this problem will be to define blending as $\ell_{\text{out}} = \ell^A * \mathbf{m} + \ell^B * (1 - \mathbf{m})$, giving as result the image in [figure 23.7](#).



Figure 23.7: Resulting blended image from [figure 23.6](#). This result is not very pleasing.

The result in [figure 23.7](#) is not very pleasing as there is a sharp transition from one image to another (see the straight edge between the two halves of the apple and orange.) We would like to be able to merge both images in a seamless way. In 1983, Burt and Adelson [[66](#)] introduced an image blending algorithm using Laplacian pyramid capable of achieve a smooth transition between the two images. This approach remains one of the best solutions to this problem.

Using the Laplacian pyramid, we can transition from one image to the next over many different spatial scales to make a gradual transition between the two images. The algorithm proceeds as follows. We first build the Laplacian pyramid for the two input images; in this example we use seven levels and we also keep the last low-pass residual as shown in [figure 23.8](#).

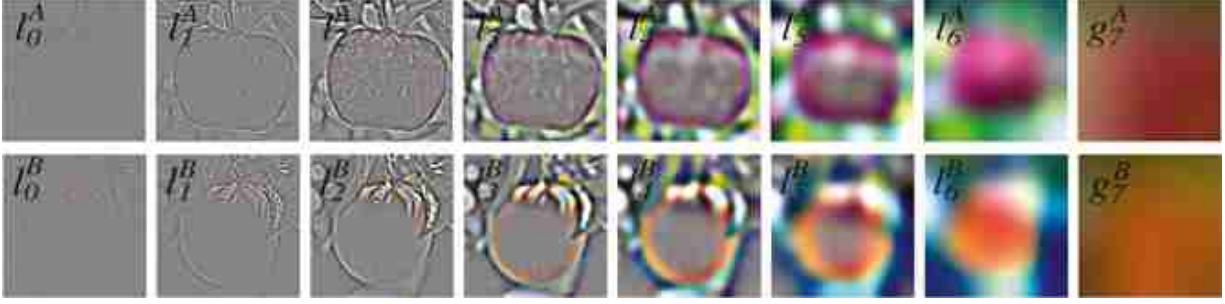


Figure 23.8: Laplacian pyramids (seven levels and Gaussian residual) for both input images.

The second step is to build the Gaussian pyramid of the mask as shown in [figure 23.9](#) (note that we use eight levels, one level more than for the Laplacian pyramid).



Figure 23.9: Gaussian pyramid of the mask.

In the third step, we combine the three pyramids to compute the Laplacian pyramid of the blended image. The Laplacian pyramid of the blended image is obtained as

$$l_k = l_k^A * m_k + l_k^B * (1 - m_k) \quad (23.11)$$

The same is done for the low-pass residual.

Finally, the fourth step consists in collapsing (i.e., decoding) the resulting pyramid to produce the blended image shown in [figure 23.10](#).



Figure 23.10: Image compositing with the Laplacian pyramid. Example inspired by [66].

The result in [figure 23.10](#) has a smooth transition between the two sides and produces a pleasing blending. The mask does not need to be rectangular. It is possible to blend arbitrary images with complex masks, but the quality of the final image will depend on how well aligned the images are and the composition of the objects in the scene. This method remains one of the most popular methods for image blending due to its simplicity and the quality of the resulting images.

This method will fail when blending requires geometric image transformations.

23.6 Steerable Pyramid

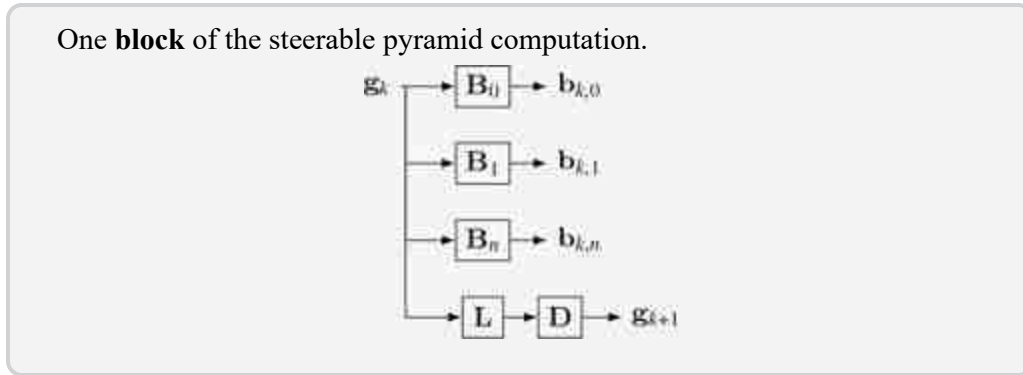
The Laplacian pyramid provides a richer representation than the Gaussian pyramid. But we would like to have an even more expressive image representation. The **steerable pyramid** [441] adds information about image orientation. Therefore, the steerable representation is a **multiscale oriented representation** that is translation-invariant. It is non-aliased and self-invertible. Ideally, we'd like to have an image transformation that was shiftable, that is, where we could perform interpolations in position, scale, and orientation using linear combinations of a set of basis coefficients.

We analyze in orientation using a steerable filter bank, shown in [figure 23.11](#). We form a decomposition in scale by introducing a low-pass filter

(designed to work with the selected bandpass filters), and recursively breaking the low-pass filtered component into angular and low-pass frequency components. Pyramid subsampling steps are preceded by sufficient low-pass filtering to remove aliasing.



Figure 23.11: Oriented filters and the low-pass kernel used in the steerable pyramid.



To ensure that the image can be reconstructed from the steerable filter transform coefficients, the filters must be designed so that their sums of squared magnitudes *tile* in the frequency domain. We reconstruct by applying each filter a second time to the steerable filter representation, and we want the final system frequency response to be flat, for perfect reconstruction.

The following block diagram ([figure 23.12](#)) shows the steps to build a two-level steerable pyramid and the reconstruction of the input. The architecture has two parts: (1) the analysis network (or encoder) that transforms the input image x into a representation composed of $r = [b_{0,0}, \dots, b_{0,n}, b_{1,0}, \dots, b_{1,n}, \dots, b_{k-1,0}, \dots, b_{k-1,n}]$ and the low pass residual g_{k-1} ; and (2) the synthesis network (or decoder) that reconstructs the input from the representation r .

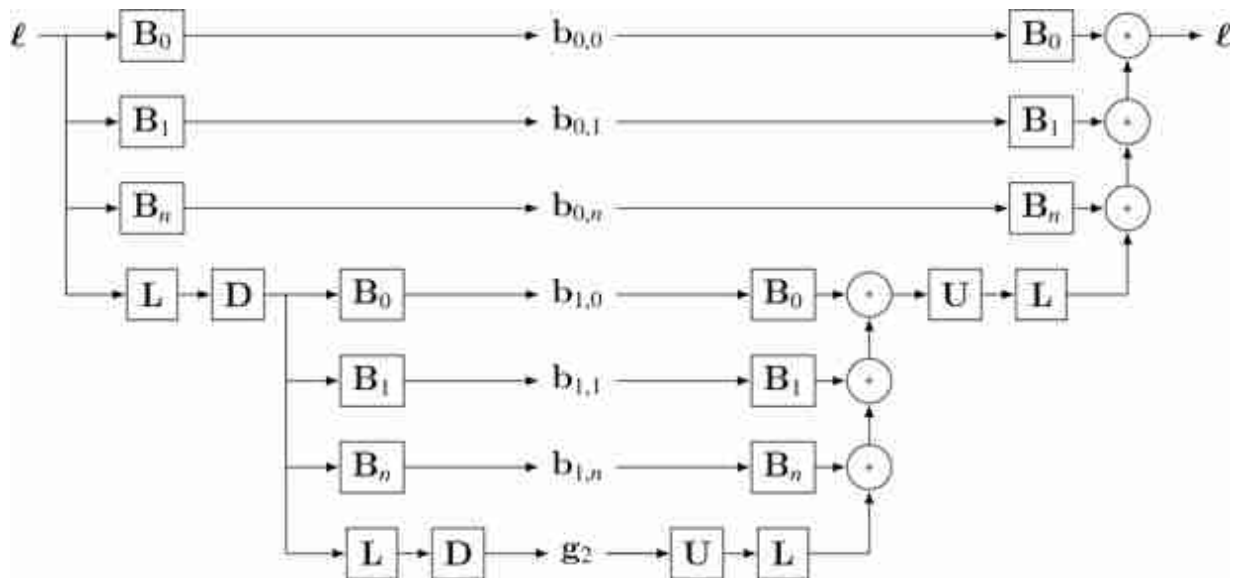


Figure 23.12: Steerable pyramid architecture.

[Figure 23.13](#) shows an example of an image and its steerable pyramid decomposition using four orientations and three scales. The steerable pyramid is a self-inverting overcomplete representation (more coefficients than pixels).

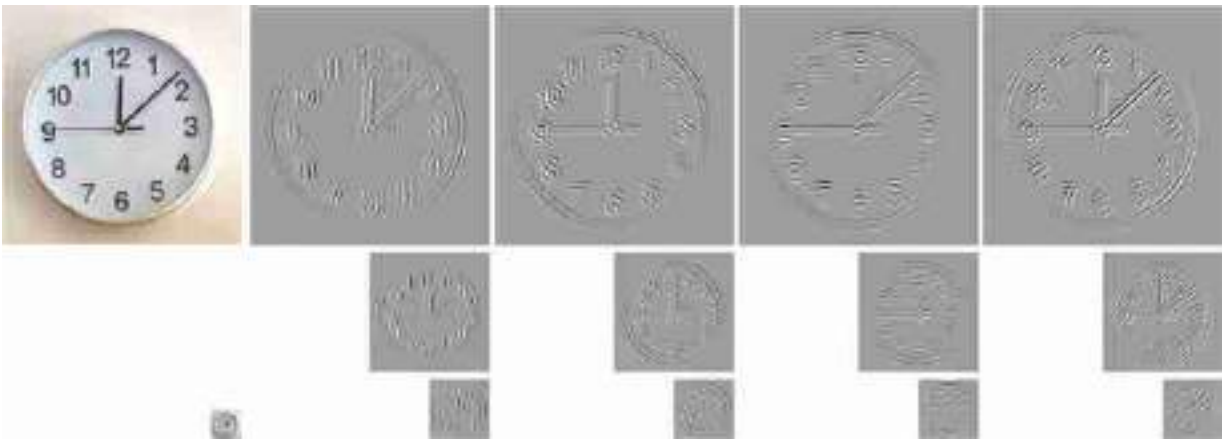


Figure 23.13: Steerable pyramid representation (three levels and four orientations). Why is that each orientation subband seems to indicate a different time?

23.7 A Pictorial Summary

[Figure 23.14](#) shows a pictorial summary of the different pyramid

representations we've discussed in this chapter. The figure shows the projection matrices $\mathbf{P}$ for each transformation, both in the 1D case and 2D case. Each projection matrix formed by stacking the projection matrices of each pyramid level.

[Figures 23.14\(a-c\)](#) start with a 1D input signal of length 16. For instance, for a three-level Gaussian pyramid, [figure 23.14\(b\)](#) shows the following projection matrix:

$$\mathbf{P} = \begin{bmatrix} \mathbf{I} \\ \mathbf{G}_0 \\ \mathbf{G}_1 \end{bmatrix} \quad (23.12)$$

The first block is the identity matrix as the first level is the input image itself.

For a two-level Laplacian pyramid with a Gaussian residual, the projection matrix $\mathbf{P}$ is ([figure 23.14\[b\]](#)):

$$\mathbf{P} = \begin{bmatrix} \mathbf{L}_0 \\ \mathbf{L}_1 \\ \mathbf{G}_1 \end{bmatrix} \quad (23.13)$$

The Fourier transform gives a complex-valued output (represented in color) and is global, that is, each output coefficient depends, in general, on every input pixel. The Gaussian pyramid is seen to be banded, showing that it is a localized transform where the output values only depend on pixels in a neighborhood. It is an overcomplete representation, shown by the transform matrix being taller than it is wide. The Laplacian pyramid is a band-passed image representation, except for the low-pass residual layer, shown in the bottom rows. For this matrix, zero is plotted as gray.

[Figures 23.14\(d-h\)](#) show the projection matrices when the input is a 2D image of size 16×16 pixels. First, the image is represented as a column vector of length $256 = 16 \times 16$ values. [Figure 23.14\(d\)](#) shows the projection matrix of the 2D Fourier transform, a square matrix of size 256×256 , composed of small blocks of size 16×16 that look like 1D Fourier transforms. [Figures 23.14\(e and f\)](#) show the Gaussian and Laplacian pyramids, respectively.

[Figures 23.14\(g-h\)](#) show the steerable pyramid representation, depending on the number of orientations ([figure 23.14\[g\]](#) is a pyramid with two orientations and [figure 23.14\[h\]](#) has four orientations per scale). The steerable pyramid representation can be very overcomplete (the matrix is much taller than wide).

The steerable pyramid is an overcomplete, multiorientation representation. We only show the steerable pyramid for 2D images as in 1D, image orientation isn't defined. The multiple orientations, and non-aliased subbands cause the representation to be very overcomplete, much taller than it is wide. The last block of the steerable pyramid projection matrices computes the low-pass residual.

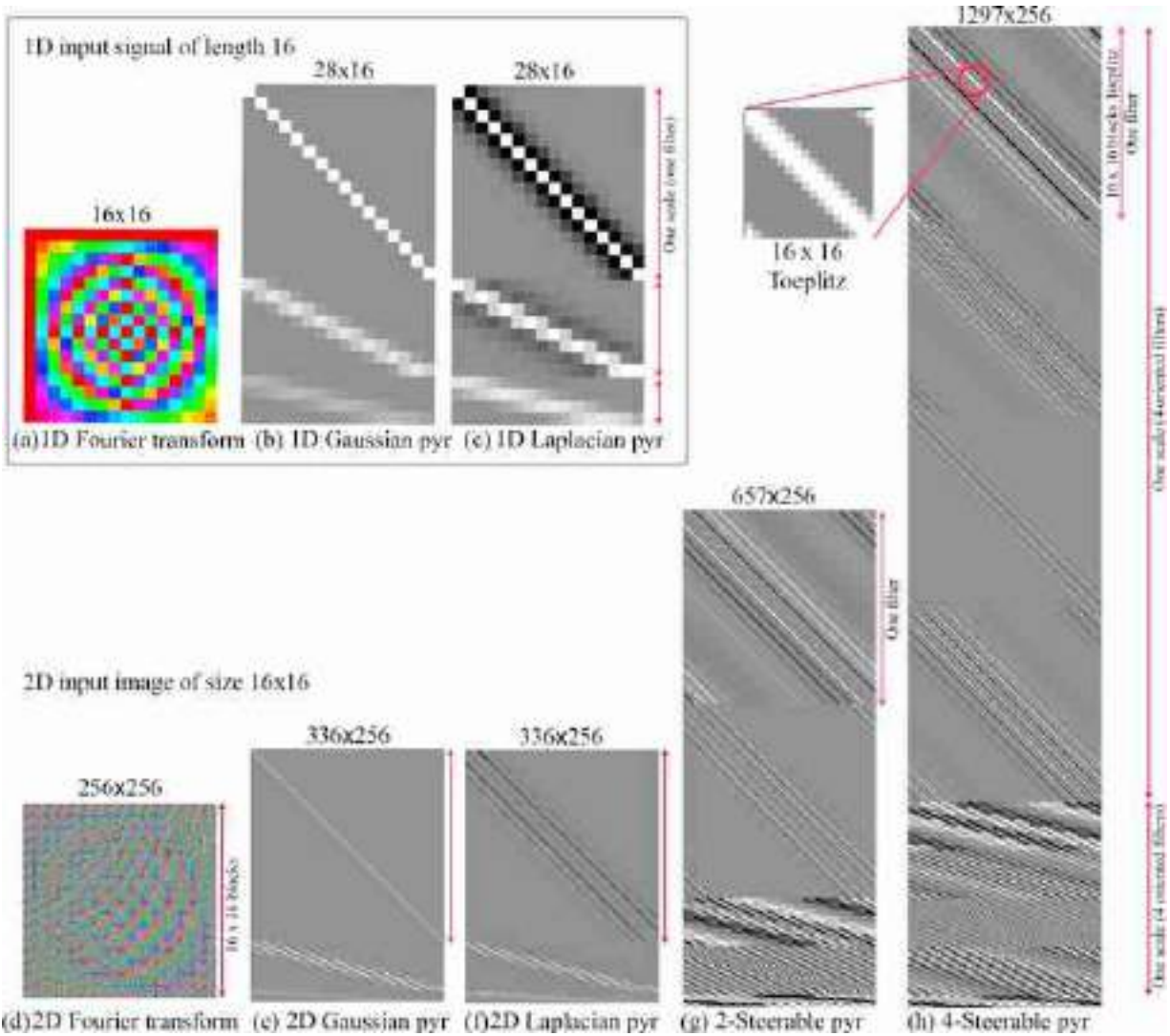


Figure 23.14: Visual comparison of linear transform image representations discussed in this chapter. All the transforms are visualized as a matrix where the number of rows is the size of the input, and the number of columns is the size of the output. All the transforms, except for the Fourier transform, are convolutional, revealed by the diagonal banding in the matrices.

23.8 Concluding Remarks

To recap briefly, the Fourier transform reveals spatial frequency content of the image wonderfully, but suffers from having no spatial localization. A Gaussian pyramid provides a multiscale representation of the image, useful for applying a fixed-scale algorithm to an image over a range of spatial scales. But it doesn't break the image into finer components than simply a range of low-pass filtered versions. The representation is overcomplete that is, there are more pixels in the Gaussian pyramid representation of an image than there are in the image itself.

The Laplacian pyramid reveals what is captured at one spatial scale of a Gaussian pyramid, and not seen at the lower-resolution level above it. Like the Gaussian pyramid, it is overcomplete. It is useful for various image manipulation tasks, allowing you to treat different spatial frequency bands separately.

The steerable pyramid adds orientation information into the representation, and the representation can be very overcomplete (the matrix is much taller than wide). The steerable pyramid has negligible aliasing artifacts and so can be useful for various image analysis applications.

In image processing applications (i.e., image compression, denoising, etc.) people have used other transforms. For instance, the Haar/wavelet/QMF pyramid [10] brings in some limited orientation analysis and different than the other pyramid representations, is complete rather than overcomplete. This helps it for image compression applications, but hurts it for others because each subband depends on information in other subbands to let it reconstruct the original image without artifacts. So if you alter one band without altering the corresponding other ones, you can easily introduce artifacts (although artifacts will also appear in overcompleted representations).

Another important framework for multiscale image analysis, not presented in this book, is **scale space**. For an in depth presentation of this topic we direct the reader to [299].

[9.](#) Downsampling is discussed in detail in chapter 21.

VII

NEURAL ARCHITECTURES FOR VISION

Building on the concepts introduced in the previous set of chapters, we will describe neural networks for vision. One way of understanding neural networks is to think of them as big signal processors, built as a succession of multiple stages (layers) of learned linear filters (convolutional units) followed by nonlinearities.

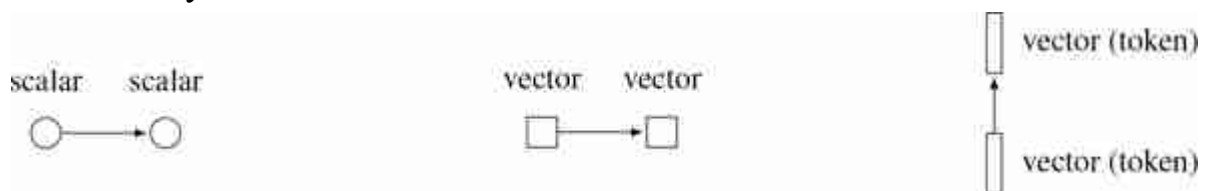
- **Chapter 24** introduces convolutional neural networks, which are very much like image pyramids, but with learned filters.
- **Chapter 25** covers recurrent networks, which enable memory and adaptation over time.
- **Chapter 26** describes transformers, a relatively new kind of architecture where the units are vector-valued **tokens** rather than scalar-valued neurons.

Notation

- In this part we will study architectures that can work on both images as well as other kinds of data. Because of this, we will denote inputs as $\mathbf{x}$ rather than ℓ , even when the input is an image.
- Some of the architectures we will encounter can operate over input tensors with variable shapes. For this reason, we will sometimes treat the input signal as a lower-case bolded variable regardless of its dimensionality: $\mathbf{x}_{\text{in}}$ can represent a 1D signal, a 2D image, 3D volumetric data, etc.
- For signals with multiple channels, including neural feature maps with multiple features at each coordinate, the first dimension of the tensor indexes over channel. For example, in $\mathbf{x} \in \mathbb{R}^{C \times N \times M \times \dots}$, where C is the number of channels of the signal.

- For transformers, we deviate from the previous point slightly, in order to match standard notation: a set of tokens (which will be defined in the transformers chapter) is represented by a $N \times d$ matrix, where d is the token dimensionality.

In this part we also need some new conventions for drawing neural nets. For multilayer perceptrons (chapter 12), we used circles to represent neurons, which are scalar nodes in a graph. Edges between neurons represent a $\mathbb{R} \rightarrow \mathbb{R}$ transformation (usually parameterized by a single weight associated with the edge). In the architectures we will presently see, the nodes of our networks may be multidimensional, and we draw them as squares or rectangles. In these cases, an edge between a node of dimensionality C_1 and a node of dimensionality C_2 represents a $\mathbb{R}^{C_1} \rightarrow \mathbb{R}^{C_2}$ transformation (which may be parameterized by multiple weights). The three node symbols we use are shown below:



A token is a vector of neurons used in a particular way, which will be defined in the transformers chapter. We give it a special symbol, a rectangle, to distinguish it from other kinds of vectors of neurons.

As shown above, sometimes we draw networks with the layers moving left to right and sometimes bottom to top. Both mean the same thing, and the direction in each figure is just chosen for visual clarity.

24 Convolutional Neural Nets

24.1 Introduction

The neural nets we saw in chapter 12 are designed to process generic data. But in many domains the data has special structure, and we can design neural net architectures that are better suited to exploiting that structure. **Convolutional neural nets**, also called **convnets** or **CNNs**, are a neural net architecture especially suited to the structure in visual signals.

The key idea of CNNs is to chop up the input image into little patches, and then process each patch *independently* and *identically*. The gist of this is captured in [figure 24.1](#):

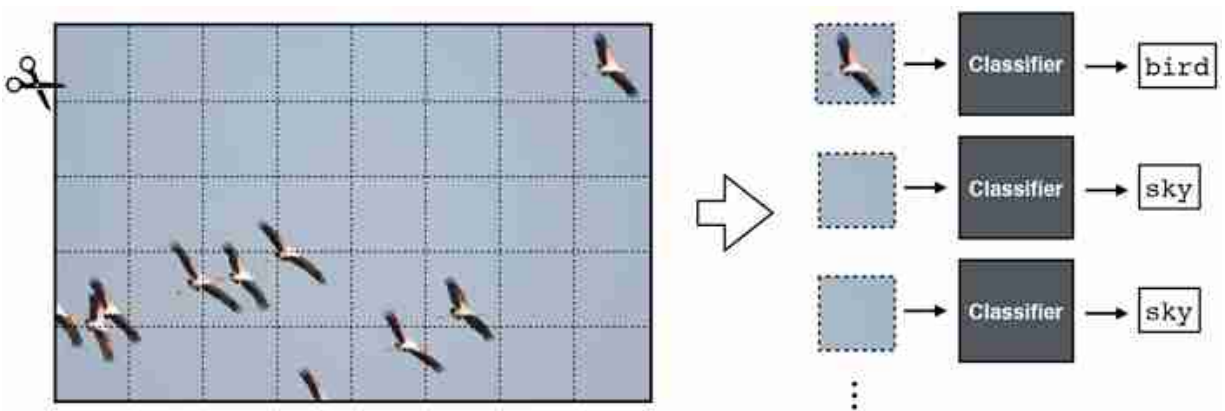
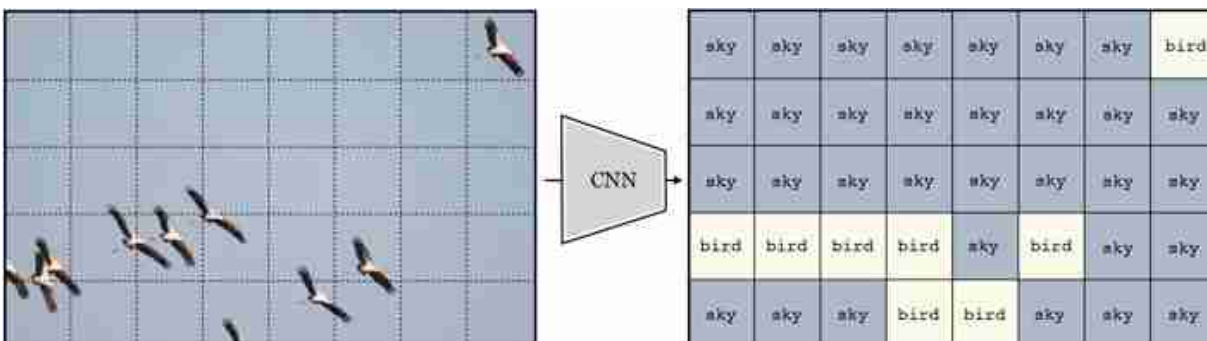


Figure 24.1: CNNs as patch processing. *Photo source:* Fredo Durand.

CNNs are also well suited to processing many other spatial or temporal signals, such as geospatial data or sounds. If there is a natural way to scan across a signal, processing each windowed region separately, then CNNs may be a reasonable choice.

Each patch is processed with a classifier module, which is a neural net. Essentially, this neural net scans across the patches in the input and classifies each. The output is a label *for each patch in the input image*. If we

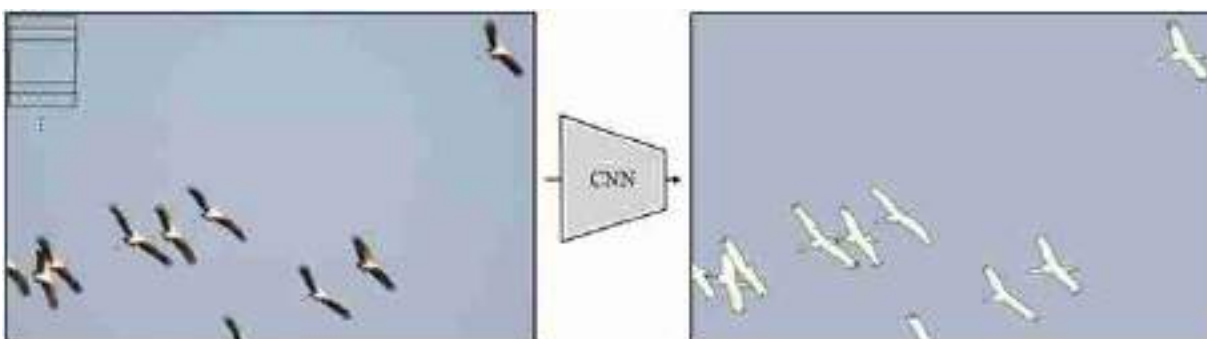
rearrange these predictions back into the shape of the input image and color code them, we get the below input-output mapping ([figure 24.2](#)):



[Figure 24.2](#): Input-output mapping of a CNN.

Notice that this is quite different than the neural nets we saw in chapter 12, which output a single prediction on the entire image; CNNs output a two-dimensional (2D) *array* of predictions.

We may also chop up the image into *overlapping* patches. If we do this densely, such that each patch is one pixel offset from the last, we get a full resolution image of predictions ([figure 24.3](#)):



[Figure 24.3](#): Dense input-output mapping.

Now that looks impressive! This CNN solved a task known as **semantic segmentation**, which is the task of assigning a class label to each pixel in an image. One reason CNNs are powerful is because they map an input image to an output image *with the same shape*, rather than outputting a

single label like in the nets we saw in previous chapters. CNNs can also be generalized to input and output other kinds of structures. The key property is that the output matches the topology of the input: an N-dimensional (ND) tensor of inputs will be mapped to an ND tensor of outputs.

Keeping in mind that chopping up and predicting is really all a CNN is doing, we will now dive into the details of how they work.

24.2 Convolutional Layers

CNNs are neural networks that are composed of **convolutional layers**. A convolutional layer transforms inputs $\mathbf{x}_{\text{in}}$ to outputs $\mathbf{x}_{\text{out}}$ by convolving $\mathbf{x}_{\text{in}}$ with one or more filters $\mathbf{w}$. A convolutional layer with a single filter looks like this:

$$\mathbf{x}_{\text{out}} = \mathbf{w} \star \mathbf{x}_{\text{in}} + b \quad \triangleleft \quad \text{conv} \quad (24.1)$$

where $\mathbf{w}$ is the kernel and b is the bias; $\theta = [\mathbf{w}, b]$ are the parameters of this layer.

In this chapter, we deviate slightly from our usual notation and use lowercase for convolutional filter $\mathbf{w}$, regardless of whether the kernel is a 1D array, a 2D array, or an ND array.

Recalling the definition of the operator $\star$ from chapter 15, we give here an example of a convolutional layer over a 2D array $\mathbf{x}_{\text{in}}$, using a square kernel of size $K \times K$:

$$x_{\text{out}}[n, m] = b + \sum_{k_1, k_2 = -K}^K w[k_1, k_2] x_{\text{in}}[n + k_1, m + k_2] \quad \triangleleft \quad \text{conv (expanded)} \quad (24.2)$$

“Convolutional” layers in deep nets are typically actually defined as cross-correlations ($\star$) and we stick to that convention in this book. We need not worry about the misnomer because whether you implement the layers with convolution or cross-correlation usually makes no difference for learning. This is because both *span an identical hypothesis space* (any cross-correlation can be converted to an equivalent convolution by flipping the filter horizontally and vertically).

As discussed in chapter 15, convolution is just a special kind of linear transform. Similarly, a convolutional layer is just a special kind of linear layer. It is a linear layer whose matrix $\mathbf{W}$ is Toeplitz. We can view it either

as a matrix or as a neural net, as shown in [figure 24.4](#), which shows the case of a one-dimensional (1D) convolution over a 1D signal $\mathbf{x}_{in}$, with zero bias.

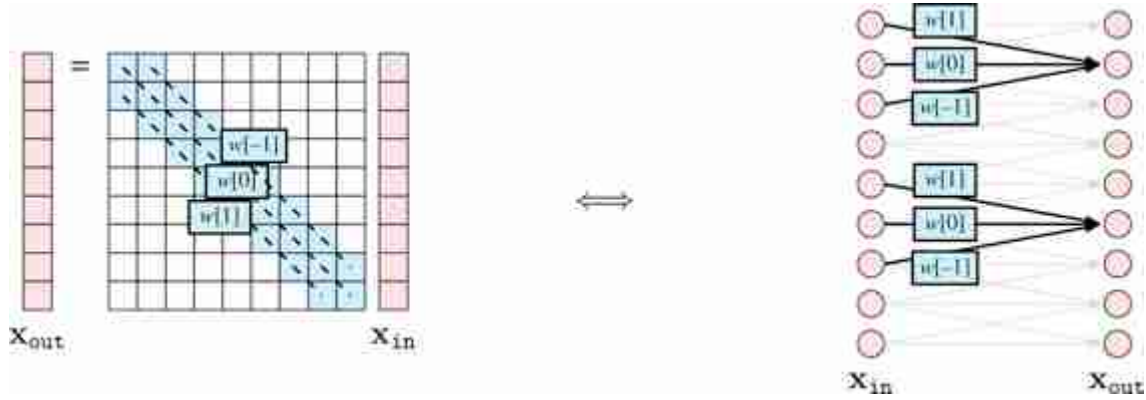


Figure 24.4: Two equivalent visualizations of a convolutional layer.

We already saw that convolutional filters are useful for image processing in parts IV and V. In those parts, we introduced a variety of hand-designed filter banks with useful properties. A CNN instead *learns* an effective filter bank.

24.2.1 Multi-Input, Multi-Output Convolutional Layers

In image processing, convolution usually refers to filtering a 1-channel signal and producing a 1-channel output, e.g., filtering a grayscale image and producing a scalar-valued response image. In neural networks, convolutional layers are more general, and typically map a multichannel input to a multichannel output. In this section we define how to handle multichannel inputs, then how to handle multichannel outputs, and then put them together to define the fully general convolutional layer.

Multichannel inputs Suppose we have an RGB image $\mathbf{x}_{in} \in \mathbb{R}^{3 \times N \times M}$. To apply a convolutional layer to such a multichannel image we simply use a multichannel filter $\mathbf{w} \in \mathbb{R}^{C \times K \times K}$, and filter each input channel with the corresponding filter channel, then sum the responses:

$$\mathbf{x}_{\text{out}} = \sum_c \mathbf{w}[c, :, :] \star \mathbf{x}_{\text{in}}[c, :, :] + b[c] \quad \triangleleft \quad \text{conv} \quad (\text{multichannel in}) \quad (24.3)$$

Multichannel outputs Above we saw a convolutional layer with just a single filter. More commonly each convolutional layer in a neural network will apply a set of filters, i.e. a **filter bank**.

We introduced filter banks in chapter 22. The difference here is that in CNNs the filter weights are learned.

If we have a bank of C filters $\mathbf{w}_0, \dots, \mathbf{w}_{C-1}$, and apply them to a grayscale input image $\mathbf{x}_{\text{in}} \in \mathbb{R}^{N \times M}$ we get C output images:

$$\mathbf{x}_{\text{out}}[0, :, :] = \mathbf{w}[0, :, :] \star \mathbf{x}_{\text{in}} + b[0] \quad (24.4)$$

$$\vdots$$

$$\mathbf{x}_{\text{out}}[C-1, :, :] = \mathbf{w}[C-1, :, :] \star \mathbf{x}_{\text{in}} + b[C-1] \quad (24.5)$$

Now $\mathbf{x}_{\text{out}}$ is an image with C channels. Each channel is the response of the input image to one of the filters.

We use the term “image” to refer to any 2D array of measurements or features. An image does not have to be a conventional photograph.

We call each of these channels a **feature map**, as it shows some features of the input, such as where the vertical edges are.

Multi-Input, Multi-Output Putting both of the above together, we can define a general convolutional layer that maps a signal with C_{in} input channels to a signal with C_{out} output channels. Here is what this looks like for an image $\mathbf{x}_{\text{in}} \in \mathbb{R}^{C_{\text{in}} \times N \times M}$, where c_2 indexes the output channel, with $c_2 \in \{0, \dots, C_{\text{out}} - 1\}$:

$$\mathbf{x}_{\text{out}}[c_2, :, :] = \sum_{c_1=1}^{C_{\text{in}}} \mathbf{w}[c_1, c_2, :, :] \star \mathbf{x}_{\text{in}}[c_1, :, :] + b[c_2] \quad \triangleleft \quad \text{conv} \quad (\text{multi-in-out}) \quad (24.6)$$

Notation for multichannel convolutions can get hard to keep track of, so let's spell out a few of the pieces here, which are also visualized in [figure 24.5](#):

- $\mathbf{x}_{\text{in}}[c_1, :, :]$ is the c_1 -th channel of the input signal.

- The filter bank is C_{out} filters, $[\mathbf{w}[:, 0, :, :], \dots, \mathbf{w}[:, C_{\text{out}} - 1, :, :]]$, each of which applies one convolutional filter per input channel and then sums the responses over all these filters.
- This convolutional layer maps inputs $\mathbf{x}_{\text{in}} \in \mathbb{R}^{C_{\text{in}} \times N \times M}$ to outputs $\mathbf{x}_{\text{out}} \in \mathbb{R}^{C_{\text{out}} \times N \times M}$.
- The filter bank is represented by a tensor $\mathbf{w} \in \mathbb{R}^{C_{\text{in}} \times C_{\text{out}} \times K \times K}$, where K is the (spatial, square) kernel size.

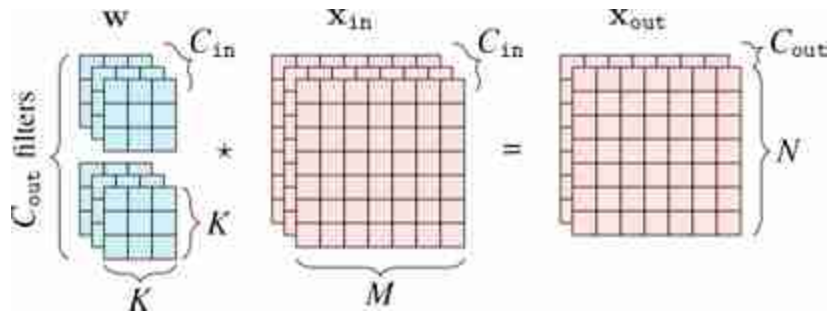


Figure 24.5: Multichannel convolution.

It's important to get comfortable with the shapes of the data and parameter tensors that get processed through different neural architectures. This is essential when designing and building these architectures, and when analyzing and debugging them. Let's go through an example with concrete numbers. Consider data $\mathbf{x}_{\text{in}}$, which is an RGB image of size 128×128 pixels. We will pass it through a convolutional layer that applies a bank of 3×3 filters (this refers to the spatial extent of the filters). We omit the bias terms for simplicity. The output ends up being a $96 \times 128 \times 128$ tensor, as shown in [figure 24.6](#).

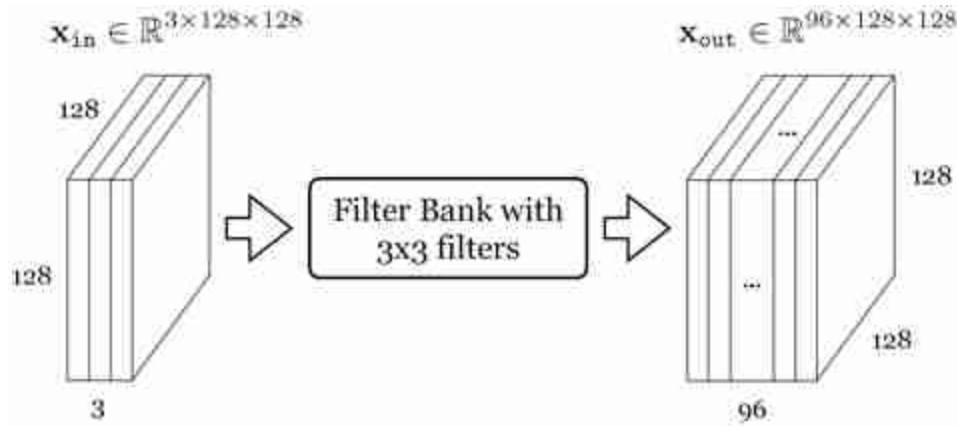


Figure 24.6: A convolutional layer that applies a bank of 3×3 filters. How many parameters does each filter have? How many filters are in the filter bank? *Source:* created by Jonas Wulff.

To check your understanding, you should be able to answer the following questions:

1. How many parameters does each filter have? (A) 9, (B) 27, (C) 96, (D) 864
2. How many filters are in the filter bank? (A) 3, (B) 27, (C) 96, (D) can't say

The answers are given in the footnote.¹⁰

24.2.2 Strided Convolution

Convolutional layers, as defined previously, maintain the spatial resolution of the signal they process. However, commonly it is sufficient, or even desirable, to output a lower resolution. This can be achieved with strided convolution:

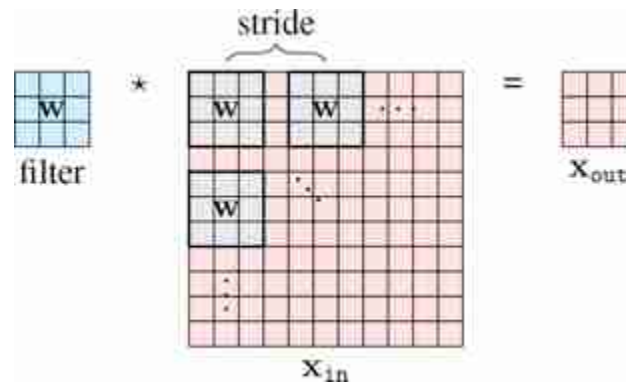
$$x_{out}[n, m] = b + \sum_{k_1, k_2=-K}^K w[k_1, k_2] x_{in}[s_n n - k_1, s_m m - k_2] \quad \triangleleft \text{conv (strided)} \quad (24.7)$$

where s_n and s_m are the strides in the vertical and horizontal directions, respectively.

Here and below, we define operations for the simplest case of convolution of a single square filter with a single channel 2D signal. All these operations can be straightforwardly extended for the multichannel in, multichannel out case, and for ND signals, and for non-square kernels. We leave it as an exercise for the reader to write out these variations as needed.

Commonly we use the same stride $s_n = s_m = s$. A convolution layer with these strides performs a mapping $\mathbb{R}^{M \times N} \rightarrow \mathbb{R}^{N/s_n \times M/s_m}$. In order to make this mapping well-defined, we require that N or M are divisible by s_n and s_m , respectively; if they are not, we may pad (or crop) the input until they are.

Strided convolution looks like this ([figure 24.7](#)):



[Figure 24.7](#): Strided convolution.

Strided convolutions can significantly reduce the computational cost and memory requirements when a neural network is large. However, strided convolution can decrease the quality of the convolution. Let's look at one concrete example where the kernel is the 2D Laplacian:

$$\mathbf{w} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (24.8)$$

As we saw in chapter 18, this filter detects boundaries on images. [Figure 24.8](#) shows an input image, and the result of strided convolution with the Laplacian kernel with strides 1, 2, and 4. The second row shows the magnitude of the discrete Fourier transforms (DFT).

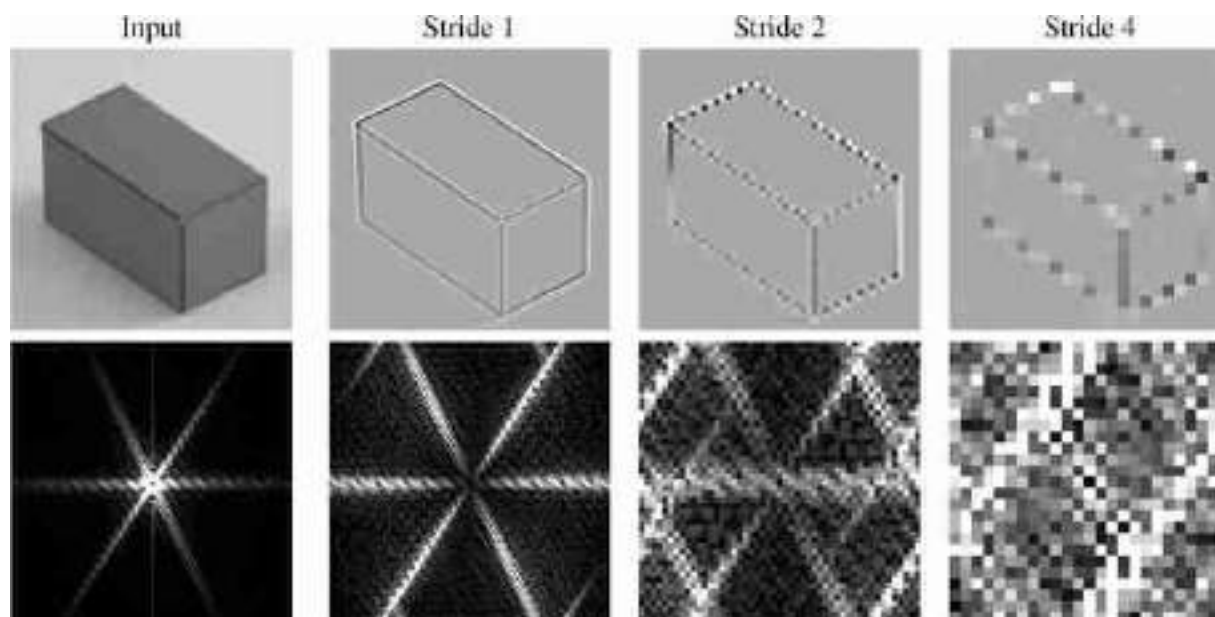


Figure 24.8: Strided convolution results and their Fourier transforms.

The result with stride 1 looks fine, and it is the output we would expect. However, stride 2 starts showing some artifacts on the boundaries, and stride 4 shows very severe artifacts, with some boundaries disappearing. The DFTs make the artifacts more obvious. In the stride 2 result we can see severe aliasing artifacts that introduce new lines in the Fourier domain that are not present in the DFT of the input image.

One can argue that these artifacts might not be important when the kernel is being learned. Indeed, the learning could search for kernels that minimize the artifacts due to aliasing as those probably increase the loss. Also, as each layer is composed of many channels, the set of learned kernels could learn to compensate for the aliasing produced by other channels. However, this reduces the space of useful kernels, and the learning might not succeed in removing all the artifacts.

24.2.3 Dilated Convolution

Dilated convolution is similar to strided convolution but spaces out the *filter* itself rather than spacing out where the filter is applied to the image:

$$x_{\text{out}}[n, m] = b + \sum_{k_1, k_2=-K}^K w[k_1, k_2] x_{\text{in}}[n - d_k k_1, m - d_k k_2] \quad \triangleleft \text{conv (dilated)} \quad (24.9)$$

Here we dilate by factor d_k in both spatial dimensions but we could choose a different dilation in each dimension. Or, we could even dilate in the channel dimension, if we were using a multichannel convolution, but this is uncommon.

An example of a dilated filter is visually shown in [figure 24.9](#):

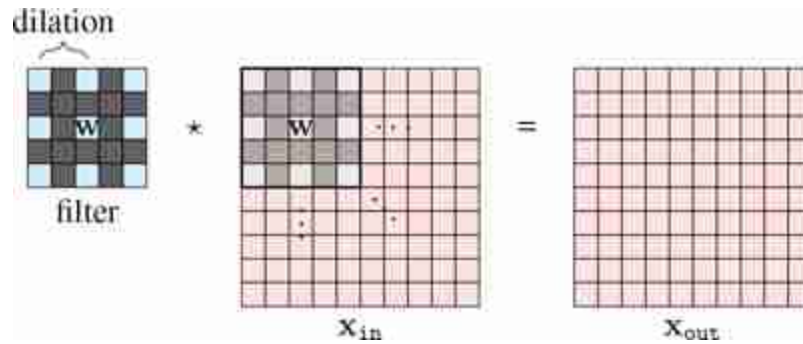


Figure 24.9: Dilated convolution. Dark gray cells have value 0.

As can be seen in the visualization, dilation is a way to achieve a filter with large kernel while only requiring a small number of weights. The weights are just spaced out so that a few will cover a bigger region of the image.

As was the case with strided convolution, dilation can also introduce artifacts. Let's look at one example in detail that illustrates the effect of dilation on a filter. Let's consider the blur kernel, $b_{2,2}$:

$$\mathbf{w} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad (24.10)$$

This filter blurs the input image by computing the weighted average of pixel intensities around each pixel location. But, dilation transforms this filter in ways that change the behavior of the filter, which does not behave as a blur filter any longer.

We saw that the 1D signal $[-1, 1, -1, \dots]$ convolved with $[1, 2, 1]$ outputs zero. However, check what happens when we convolve the input with the dilated kernel $[1, 0, 2, 0, 1]$.

The next figure shows the kernel with dilations $d_k = 1$, $d_k = 2$, and $d_k = 4$ together with the magnitude of the DFT of the three resulting kernels ([figure 24.10](#)).

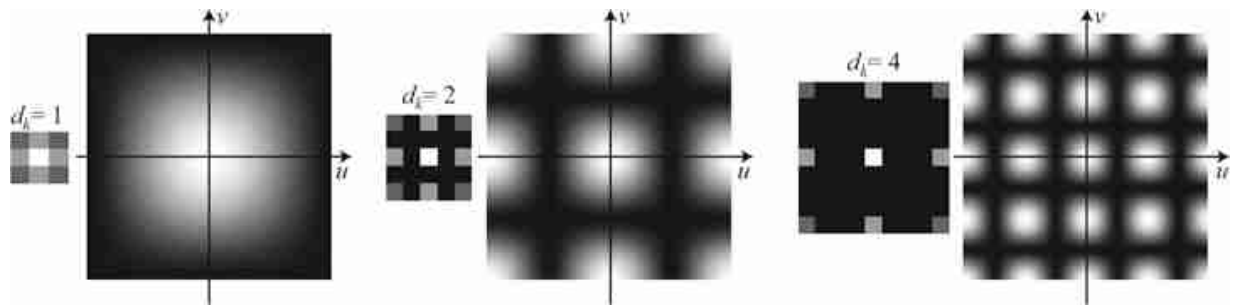


Figure 24.10: Dilated filters and their Fourier transforms.

When using the original binomial filter (which corresponds to $d_k = 1$) the DFT shows that the filter is a low-pass filter. When applying dilation ($d_k = 2$) the DFT changes and it is not unimodal anymore. It has now eight additional local maximum in high spatial frequencies. With $d_k = 4$, the DFT reveals an even more complex frequency behavior. [Figure 24.11](#) shows one input image and the result of the dilated convolutions with the blur kernel, $b_{2,2}$, with dilations $d_k = 1$, $d_k = 2$, and $d_k = 4$.

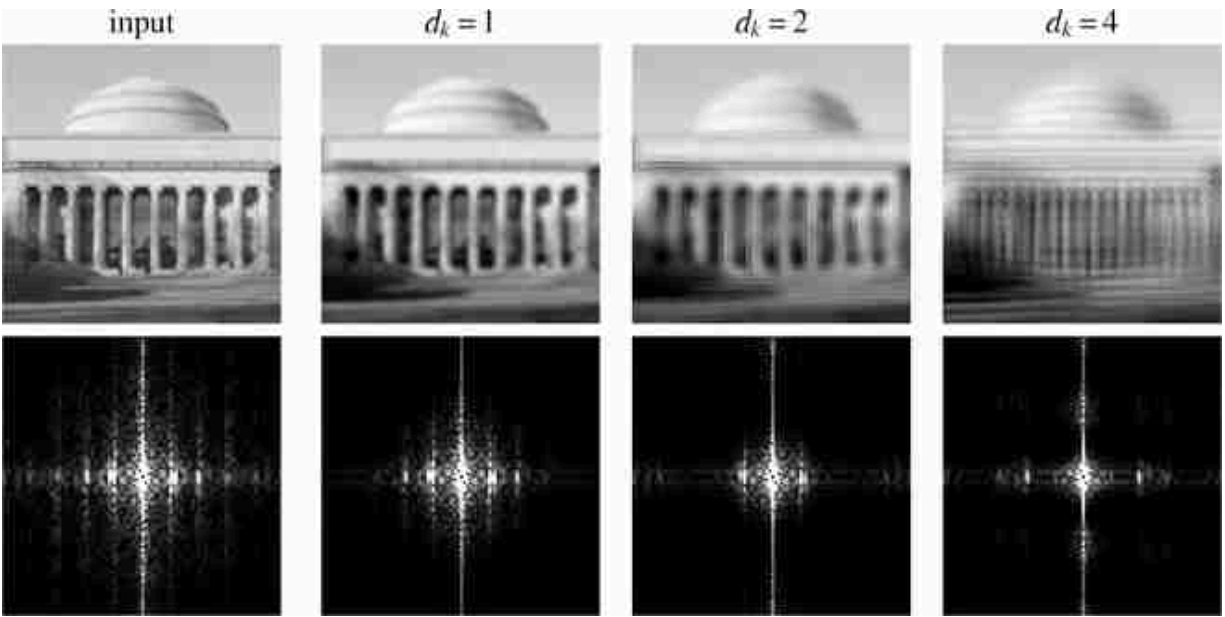


Figure 24.11: Result of the dilated convolutions with the blur kernel, $b_{2,2}$, with dilations $d_k = 1$, $d_k = 2$, and $d_k = 4$.

In summary, using dilation increases the size of the convolution kernels without increasing the computations (which is the original desired property) but it reduces the space of useful kernels (which is an undesired property).

There are ways in which dilation can be used to increase the family of useful filters. For instance, by composing three convolutions with $d_k = 1$, $d_k = 2$, and $d_k = 4$ together ([figure 24.12](#)), one can create a kernel that can switch during learning between high and low spatial frequencies and small and large kernels.

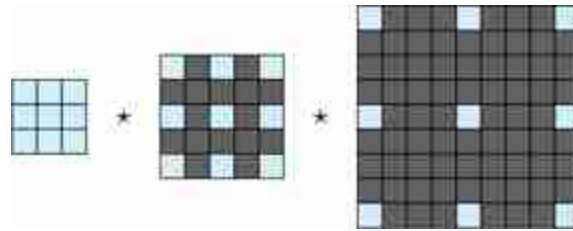


Figure 24.12: Convolving dilated filters creates a new filter that can measure effects at a mixture of frequencies.

This results in a kernel with a size of 9×9 (81 values) defined by 27 values. The relative computational efficiency increases when we cascade more filters with higher levels of dilation. [Figure 24.13](#) shows several multiscale kernels that can be obtained by the convolutions of three dilated kernels. Can you guess which kernels were used?

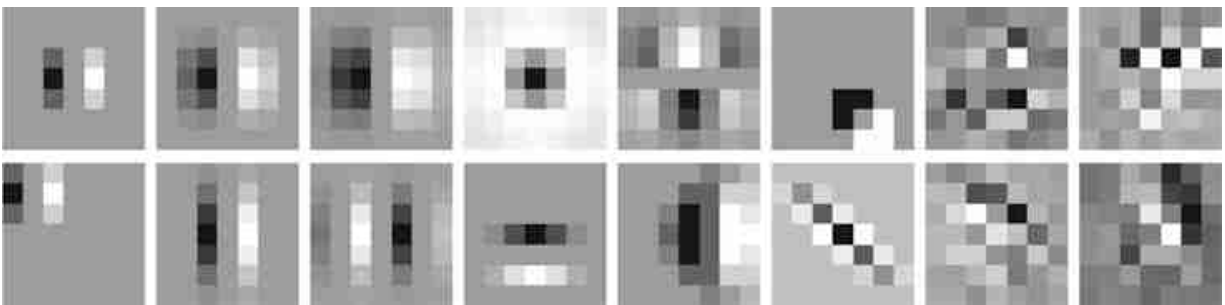


Figure 24.13: Example kernels that each result from convolving three filters.

As the figure shows, the cascade of three dilated convolutions can generate a large family of filters with different scales, orientations, shifts, and also other patterns such as corner detectors, long edge detectors, and curved edge detectors. The last four kernels show the result of convolving three random kernels, which provides further illustration of the diversity of kernels one can build. Each kernel is a 3×3 array sampled from a Gaussian distribution.

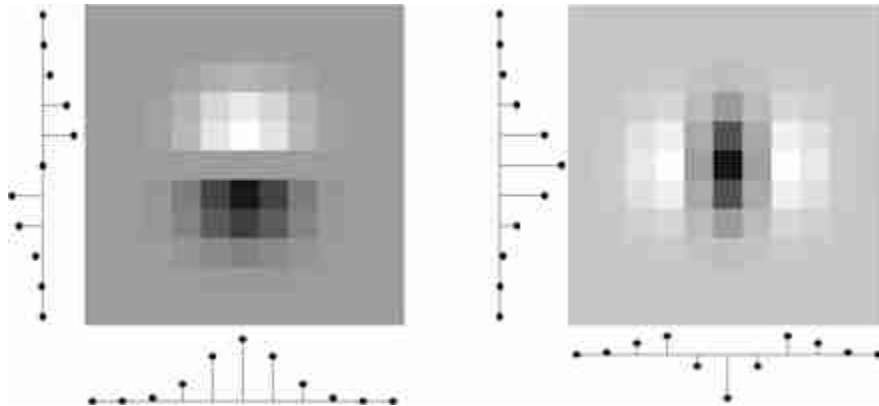
24.2.4 Low-Rank Filters

Dilation is one way to create a big filter that is parameterized by just a small number of weights, that is, a low-rank filter. This trick can be useful in many contexts where we know that good filters have low-rank structure.

Dilation uses this trick to make big kernels, which can capture long-range dependences.

Separable filters are another kind of low-rank filter that is useful in many applications (see chapter 16). We can create a convolutional layer with separable filters by simply stacking two convolutional layers in sequence, with no other layers in between. The first layer is a filter bank with $K \times 1$ kernels and the second uses $1 \times K$ kernels. The composition of these layers is equivalent to a single convolutional layer with $K \times K$ separable filters. Two examples of such separable filters are given below ([figure 24.14](#)):

When convolving one row and one column vector, $\mathbf{w} = \mathbf{u}^T \circ \mathbf{v}$, the result is the outer product: $w[n, m] = u[n] v[m]$.



[Figure 24.14](#): Two examples of separable filters.

Some important kernels are nonseparable but can be approximated by a linear combination of a small number of separable filters. For instance, the Gaussian Laplacian is nonseparable but can be approximated by a separable filter as shown here ([figure 24.15](#)):

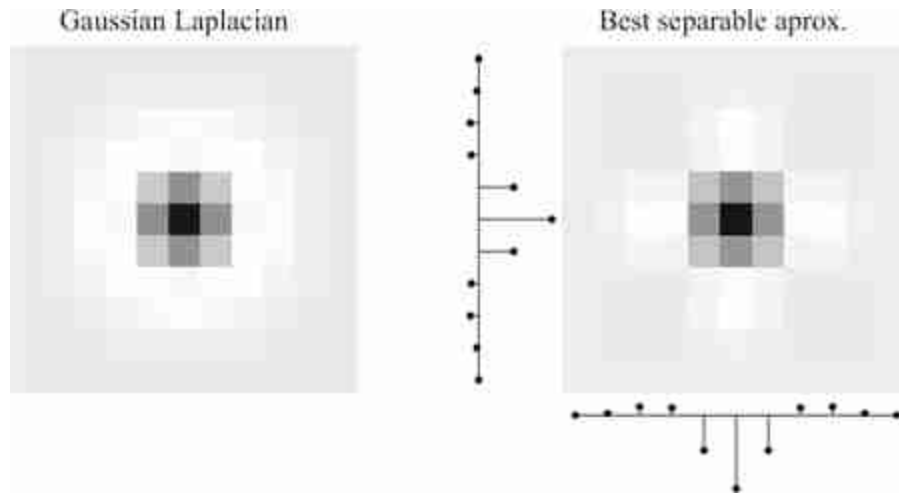


Figure 24.15: Approximating a Gaussian Laplacian filter as the outer product of two 1D filters.

The diagonal Gaussian derivative is another nonseparable kernel. When using a 3×3 kernel to approximate it we have:

$$\mathbf{w} = \begin{bmatrix} 0 & -2 & -2 \\ 2 & 0 & -2 \\ 2 & 2 & 0 \end{bmatrix} \quad (24.11)$$

But we know from chapter 18 that this kernel can be written as a linear combination of two separable kernels: $\mathbf{w} = \text{Sobel}_x + \text{Sobel}_y$, as defined in equation (18.22). In general, any $M \times N$ filter can be decomposed as a linear sum of $\min(N, M)$ separable filters. The separable filters can be obtained by applying the singular value decomposition (SVD) to the kernel array $\mathbf{w}$. The SVD results in three matrices, $\mathbf{U}$, $\mathbf{S}$ and $\mathbf{V}$, so that $\mathbf{w} = \mathbf{USV}^T$, where the columns of $\mathbf{U}$ and $\mathbf{V}$ are the separable 1D filters and the diagonal values of the diagonal matrix $\mathbf{S}$ are the linear weights. Computational benefits are only obtained when using small linear combinations for large kernels. Also, in a neural network, one could use only separable filters for all the units and the learning could discover ways of combining them in order to build more complex, nonseparable kernels.

24.2.5 Downsampling and Upsampling Layers

In chapter 23 we saw image pyramids and showed how they can be used for analysis and synthesis. CNNs can also be structured as analysis and synthesis pyramids, and this is a very powerful tool. To create a pyramid we just need to introduce a way of downsampling the signal during analysis

and upsampling during synthesis. In CNNs this is done with **downsampling and upsampling layers**.

Downsampling layers transform the input tensor to an output tensor that is smaller in the spatial dimensions: $\mathbb{R}^{N \times M} \rightarrow \mathbb{R}^{N/n_d \times M/n_m}$. We already saw one kind of downsampling layer, strided convolution, which is equivalent to convolution followed by subsampling. Another common kind of downsampling layer is **pooling**, which we will encounter in section 24.3.1.

Upsampling layers perform the opposite transformation, outputting a tensor that is larger in the spatial dimensions than the input: $\mathbb{R}^{N \times M} \rightarrow \mathbb{R}^{Nn_d \times Mn_m}$. One kind of upsampling layer can be made as the analogue of strided convolution. Strided convolution convolves then subsamples; this upsampling layer instead dilates the signal then convolves. Starting with a blank image of zeros, $\mathbf{h} = \mathbf{0}$, we set:

$$h[ns_n, ms_m] = x_{\text{in}}[n, m] \quad \Leftarrow \text{dilation} \quad (24.12)$$

$$\mathbf{x}_{\text{out}} = \mathbf{w} * \mathbf{h} + b \quad \Leftarrow \text{conv} \quad (24.13)$$

This equation applies for all integer values of $n \in \{1, \dots, N\}$ and $m \in \{1, \dots, M\}$.

Sometimes the combination of these two layers is called an UpConv layer or a deconvolution layer (but note that deconvolution has a different meaning in signal processing).

24.3 Nonlinear Filtering Layers

All the operations we have covered above are linear (or affine). It is also possible to define filters that are nonlinear. Like convolutional filters, these filters slide across the input tensor and process each window identically and independently, but the operation they perform is a nonlinear function of the local window.

24.3.1 Pooling Layers

Pooling layers are downsampling layers that summarize the information in a patch using some aggregate statistic, such as the patch's mean value, called **mean pooling**, or its max value, called **max pooling**, defined as follows:

$$x_{\text{out}}[i] = \max_{i \in \mathcal{N}(i)} x_{\text{in}}[i] \quad \triangleleft \quad \text{max pooling} \quad (24.14)$$

$$x_{\text{out}}[i] = \frac{1}{|\mathcal{N}|} \sum_{i \in \mathcal{N}(i)} x_{\text{in}}[i] \quad \triangleleft \quad \text{mean pooling} \quad (24.15)$$

The $\mathcal{N}(i)$ indicates the set of indices in the same patch as index i .

Like all downsampling layers, pooling layers can be used to reduce the resolution of the input tensor, removing high-frequency information in the signal. Pooling is also particularly useful as a way to achieve *invariance*. Convolutional layers produce outputs that are equivariant to translations of their input. Pooling is a way to convert equivariance into invariance. For example, suppose we have run a convolutional filter that detects vertical edges. The output is a response map that is large wherever there was a vertical edge in the input image. Now if we run a max pooling filter across this response map, it will coarsen the map, resulting in a large response anywhere *near* where there was a vertical edge in the input image. If we use a max pooling filter with large enough neighborhood N , the output will be invariant to the location of the edge in the input image.

Pooling can also be performed across channels, and this can be a way to achieve additional kinds of invariance. For example, suppose we have a convolutional layer that applies a filter bank of oriented edge detector filters, where each filter looks for edges at a different orientation. Now if we max pool across the channels output by this filter bank, the resulting feature map will be large wherever an edge of *any* orientation was found. Normally, we are not looking for edges but for more complicated patterns, but the same logic applies. First run a bank of filters that look for the pattern at k different orientations. Then pool across these k channels to detect the pattern regardless of its orientation. This can be a great way for a CNN to recognize objects even if they appear with various rotations within the image. Of course we usually do not hand-define this strategy but it is one the CNN can learn to use if given channelwise pooling layers.

24.3.2 Global Pooling Layers

One extreme of pooling is to pool over the entire spatial extent of the feature map. Global pooling is a function that maps a $C \times M \times N$ tensor into a vector of length C , where C is the number of channels in the input.

Global pooling is generally used in layers very close to the output. As before, global pooling can be **global average pooling**, averaging over all the responses of the feature map, or **global max pooling**, taking the max of the feature map.

Global pooling removes spatial information from each channel. However, spatial information about input features might be still be available within the output vector if different channels learn to be sensitive to features at different spatial positions.

24.3.3 Local Normalization Layers

Another kind of nonlinear filter is the **local normalization layer**. These layers normalize each activation in a feature map by statistics the adjacent activations within some neighborhood. There are many different choices for the type of normalization (L_1 norm, L_2 norm, standardization, etc.) and many different choices for the shape of the neighborhood, such as a square patch in the spatial dimensions, a set of channels, and so on. Each of these choices leads to different kinds of normalization filters with different names. One that is historically important but no longer frequently used is the **local response normalization**, or **LRN**, filter that was introduced in the AlexNet paper [279]. This filter has the following form:

$$x_{\text{out}}[c, n, m] = x_{\text{in}}[c, n, m] / \left(\gamma + \alpha \sum_{l=\max(1, c-l)}^{\min(C, c+l)} x_{\text{in}}[l, n, m]^2 \right)^{\beta} \quad \triangleleft \quad \text{LRN} \quad (24.16)$$

where α , β , γ , and l are hyperparameters of the layer. This layer normalizes each activation by the sum of squares of the activations in a window of adjacent *channels*.

Although local normalization is a common structure within the brain, it is not very frequently used in current neural networks, which more often use global normalization layers like batchnorm or layernorm (which we saw in chapter 12).

24.4 A Simple CNN Classifier

CNNs are deep nets that stack convolutional layers in a series, interleaved with nonlinearities. CNNs also frequently use downsampling and

upsampling layers, pooling layers, and normalization layers, as described above.

CNNs come in a large variety of architectures, each suited to a different kind of problem. We will see some of these architectures in section 24.11. For now we will focus on just one simple architecture that is suited to image classification. This architecture progressively downsamples the image until the last layer makes a single global prediction of the image label ([figure 24.16](#)):

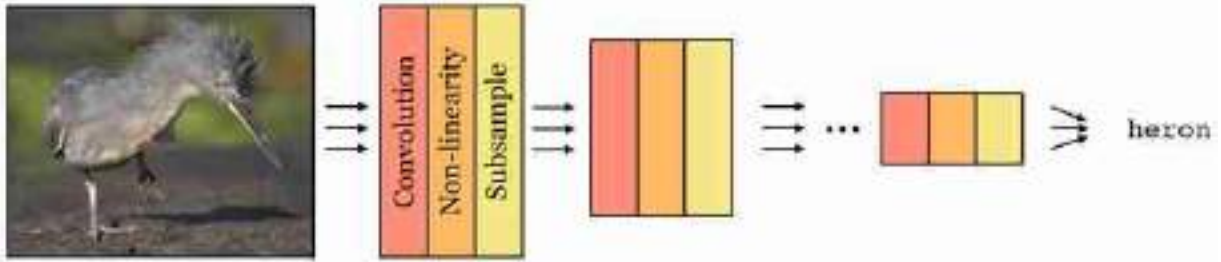


Figure 24.16: A CNN architecture for image classification. *Photo source:* Fredo Durand.

We will now walk through an example of such a classifier. Let $\mathbf{x} \in \mathbb{R}^{M \times N}$ be a black and white image. To process this image, we could use a simple CNN with two convolutional layers, defined as follows:

$$\mathbf{z}_1[c, :, :] = \mathbf{w}[c, :, :] * \mathbf{x} + b[c] \quad \triangleleft \text{conv}: [M \times N] \rightarrow [C \times M \times N] \quad (24.17)$$

$$h[c, n, m] = \max(\mathbf{z}_1[c, n, m], 0) \quad \triangleleft \text{relu}: [C \times M \times N] \rightarrow [C \times M \times N] \quad (24.18)$$

$$\mathbf{z}_2[c] = \frac{1}{NM} \sum_{n,m} h[c, n, m] \quad \triangleleft \text{gap}: [C \times M \times N] \rightarrow [C] \quad (24.19)$$

$$\mathbf{z}_3 = \mathbf{W}\mathbf{z}_2 + \mathbf{c} \quad \triangleleft \text{fc}: [C] \rightarrow [K] \quad (24.20)$$

$$y[k] = \frac{e^{-\tau \mathbf{z}_3[k]}}{\sum_{l=1}^K e^{-\tau \mathbf{z}_3[l]}} \quad \triangleleft \text{softmax}: [K] \rightarrow [K] \quad (24.21)$$

Note that these equations apply for all $c \in \{0, \dots, C-1\}$, $n \in \{0, \dots, N-1\}$ and $m \in \{0, \dots, M-1\}$.

This network has one convolutional layer with C channels followed by a relu layer. The next layer performs spatial global average pooling (gap), and each channel gets projected into a single number that contains the sum of the outputs of the relu. This results in a representation given by a vector

of length C . This vector is then processed by a **fully connected layer** (fc). A fully connected layer is simply another name for a linear layer that is full rank, that is, every output neuron is connected to every input neuron, and the mapping is described by a $K \times C$ matrix (plus a bias).

This neural net could be used to solve a K -way image classification problem (because the output is a K -way softmax for each input image). We could train it using gradient descent to find the parameters $\theta = [\mathbf{w}_1, \dots, \mathbf{w}_C, \mathbf{b}_1, \dots, \mathbf{b}_C, \mathbf{W}, \mathbf{c}]$ that optimize a cross-entropy loss over training data.

Such a network is also very easy to define in code, once we have a library of primitives for basic operations like convolution and softmax:

```
# first define parameterized layers
conv1 = nn.conv(channels_in=1, channels_out=C, kernel=k, stride=1)
fc1 = nn.fc(dim_in=C, dim_out=K)

# then run data through network
z1 = conv1(x)
h = nn.relu(z1)
z2 = nn.AvgPool2d(h)
z3 = fc1(z2)
y = nn.softmax(z3)
```

24.5 A Worked Example

In this section, we will analyze the simple network described in section 24.4, trained to discriminate between horizontal and vertical lines. Each subsection will tackle one aspect of the analysis that should be part of training any large system: (1) training and evaluation, (2) visualize and understand the network, (3) out-of-domain generalization, and (4) identifying vulnerabilities.

24.5.1 Training and Evaluation

Let's study one simple classification task. We design a simple image dataset that contains images with lines. The lines can be horizontal or vertical. Each image will contain only one type of line.

We want to design a CNN that will classify the image according to the orientation of the lines that it contains. We define the two output classes as: 0 (vertical) and 1 (horizontal). A few samples from the training set are shown in [figure 24.17](#).

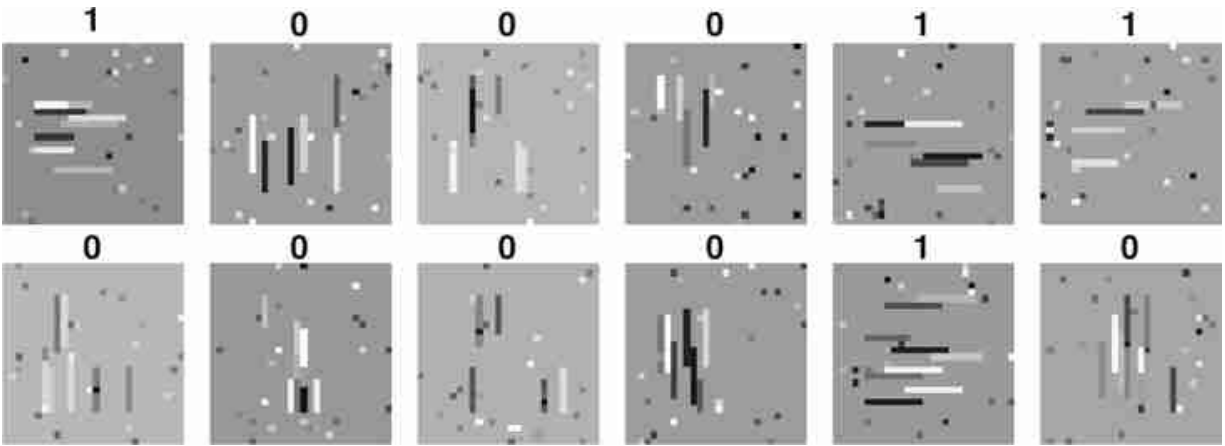


Figure 24.17: A sample of images from the training set. The training set defines the concept we want to learn. In this case we look for lines. But images of lines might not be enough to describe the concept of a line. What is a line? This lack of a precise definition will haunt us later.

To solve this problem we use the CNN defined before with two convolutional channels $C = 2$ in the first layer. Once we train the network, we can see that it has solve the task perfectly and that the output on the test set is 100 percent correct (there are only three errors out of 10,000 test images). Example images from the test set are shown in [figure 24.18](#).

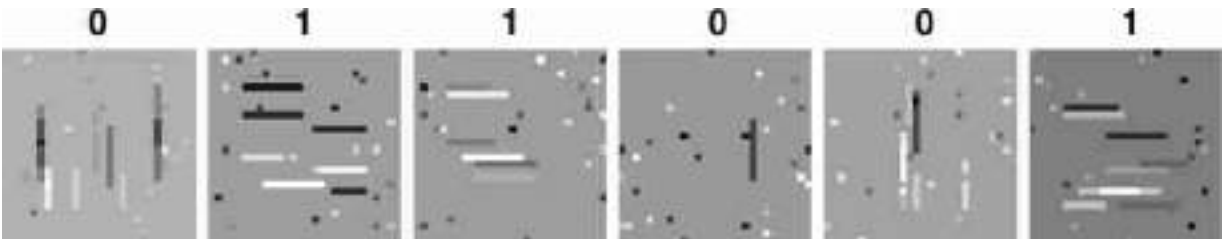


Figure 24.18: A sample of images from the test set. The predicted label is shown at the top.

24.5.2 Network Visualization

What has the network learned? How is it solving the problem? One important part of developing a system is to have tools to prove, understand, and debug it.

To understand the network it is useful to **visualize** the kernels. [Figure 24.19](#) shows the two learned 9×9 kernels. The first one looks like an horizontal derivative of a Gaussian filter (as we saw in chapter 18) and the second one looks like a vertical derivative of a Gaussian (maybe closer to a

second derivative). In fact, the DFT of each kernel shows that they are quite selective to a particular band on frequency content in the image.

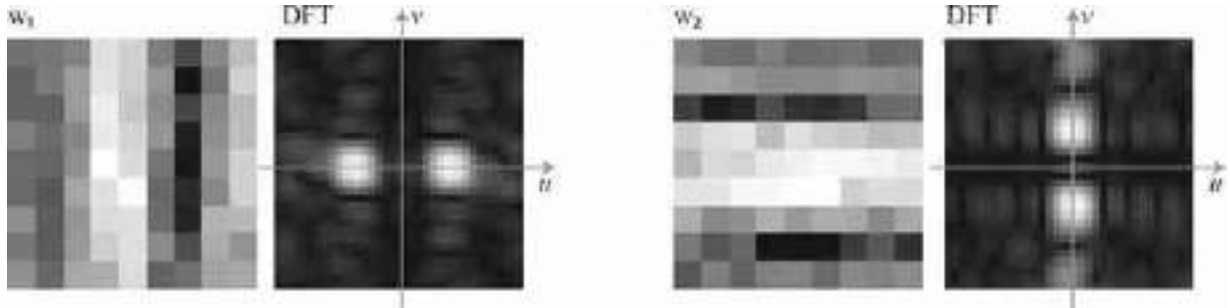


Figure 24.19: Visualization of the learned kernels.

The fully connected layer has learnt the weights:

$$\mathbf{W} = \begin{bmatrix} 2.83 & -2.36 \\ -0.60 & 1.14 \end{bmatrix} \quad (24.22)$$

This corresponds to two channel oppositions: the first feature is the vertical output minus the horizontal output, and the second feature computes the horizontal output minus the vertical one.

24.5.3 Out-of-Domain Generalization

What do we learn by analyzing how the trained network works? One interesting outcome is that can we predict how the network we defined before generalizes beyond the distribution of the training set, to **out-of-domain test samples**.

Another term for out-of-domain is **out-of-distribution**.

For instance, it seems natural to think that the network should still perform well in classifying whether the image contains vertical or horizontal structures even if they are not lines. We can test this hypothesis by generating images that match our idea of orientation. The following test images ([figure 24.20](#)) contain different oriented structures but no lines, and still captures our notion of what should be the correct generalization of the behavior.

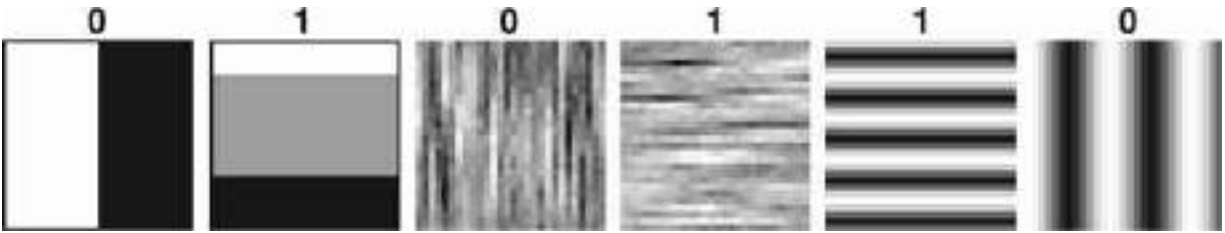


Figure 24.20: Out-of-domain test examples. The predicted label is shown at the top.

In fact, the network seems to perform correctly even with these new images that come from a distribution different from the training set.

24.5.4 Identifying Vulnerabilities

Does the network solve the task that we had in mind? Can we predict which inputs will make the output fail? Can we produce test examples that to us look right but for which the network produces the wrong classification output? The goal of this analysis is to identify weaknesses in the learned representation and in our training set (missing training examples, biases in our data, limitations of the architecture, etc.).

We saw that the output of the first layer does not really look for *lines*, instead it looks at where the energy is in the Fourier domain. So, we could fool the classifier by creating lines that for us look like vertical lines, but that have the energy content in the wrong side of the Fourier domain. We saw one trick to do this in chapter 16: modulation. If we multiply an image containing horizontal lines, by a sinusoidal wave, $\cos(\pi n/3)$, we can move the spectral content horizontally as shown in [figure 24.21](#).

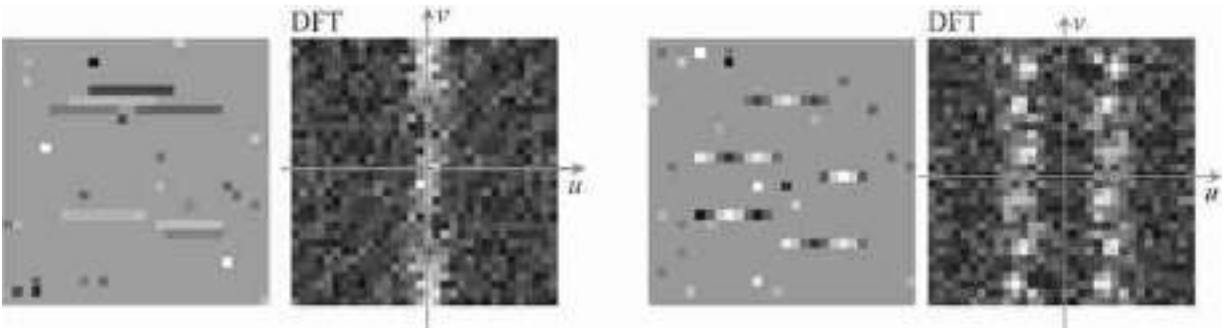


Figure 24.21: Creating a new out-of-domain test image obtained by multiplying an in-domain test image with horizontal lines (left image its DFT) with a sinusoidal wave. The resulting image and its DFT are shown on the right).

The lines still look horizontal to us, but their spectral content now is higher in the region that overlaps with the vertical line detector learned by the network. Indeed, when the network processes images that have lines with this *sinusoidal texture*, it produces the wrong classification results for all the images ([figure 24.22](#))!

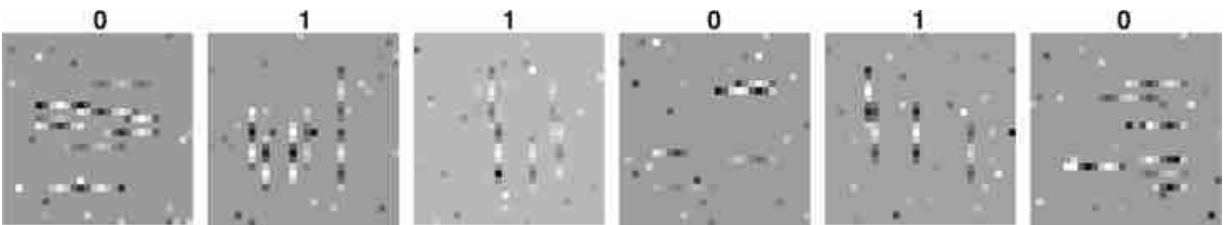


Figure 24.22: Classification results on out-of-domain test images created by modulation.

We have just designed an **adversarial example** manually! The question then could be as follows: If it is not detecting line orientations, what is it really detecting? Our analysis of the learned kernels had the answer.

For complex architectures, **adversarial examples** are obtained as an optimization problem: What is the minimal perturbation of an input that will produce the wrong output in the network?

One way of avoiding this would be to introduce these types of images in the training set and to repeat the whole process.

24.6 Feature Maps in CNNs

One of the most important concepts when working with CNNs is the feature map. A feature map can be a channel of the output of a conv layer (as we defined above) or it can refer to the entire stack of channels at some layer of a network. The idea is that these are *features* of the input data and the features are arranged in a *map* – an array that matches the shape of the input data. For images, feature maps are 2D spatial arrays, for videos they are 3D space-time arrays, and so forth.

[Figure 24.23](#) shows the interplay between feature maps and filter banks in a CNN:

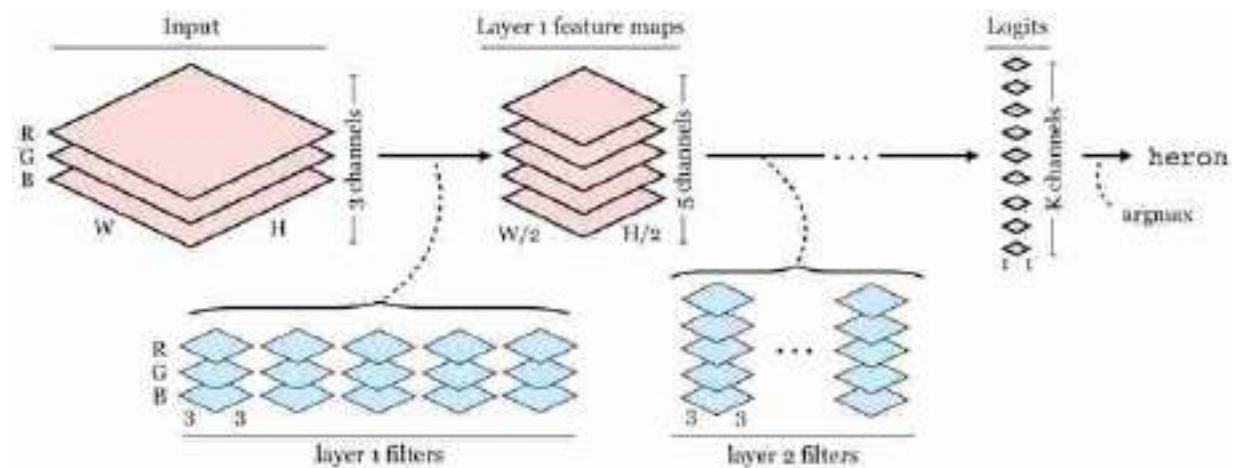


Figure 24.23: The interplay between feature maps and filters banks in a CNN. You can think of the input image itself as an R, G, B feature map and the output logits as a 1x1 resolution feature map with class logits as the channels.

The input to the network is an image and the output is a vector of logits. We can actually think of these inputs and outputs as feature maps as well: the input is just a feature map with red, green, and blue channels and the output is a 1x1 resolution feature map with class logits as the channels.

Now let's look at the feature maps in a real network, AlexNet [\[279\]](#). [Figure 24.24](#) shows what these look like after the first and second convolutional layer of the network:

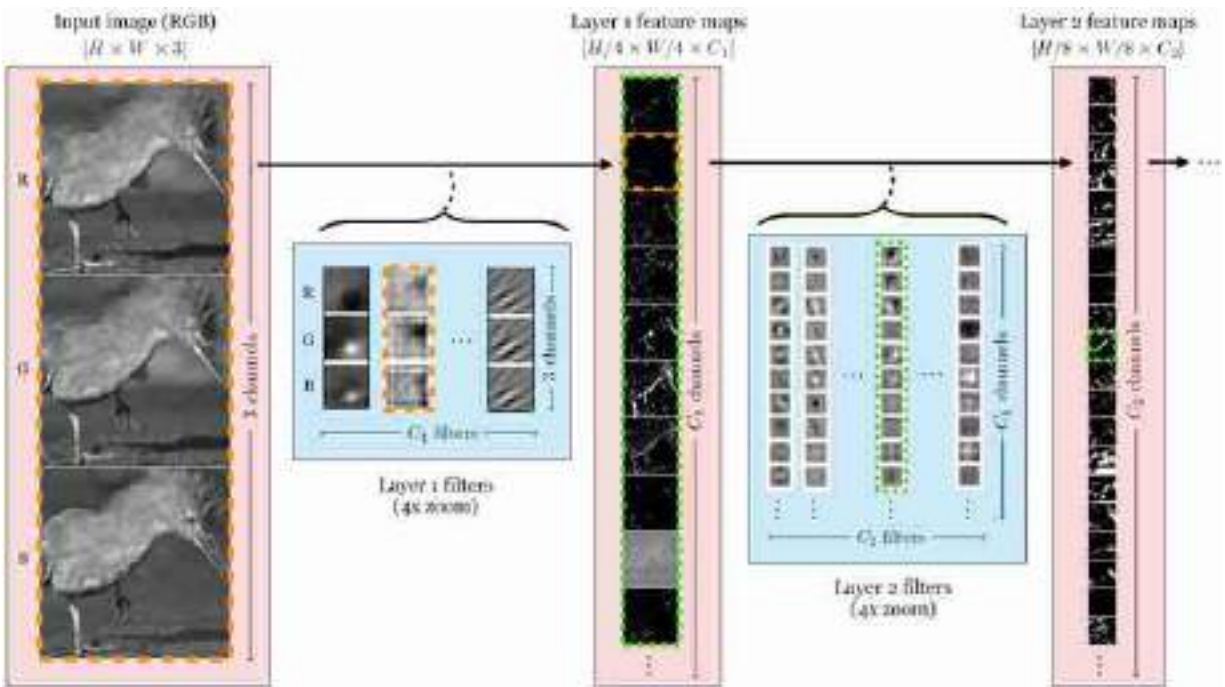


Figure 24.24: A selection of feature maps and filters in AlexNet. The layer 1 filter with the orange dashed border slides over the input image and produces the layer 1 feature map with the orange dashed border. The layer 2 filter with the green dashed border slides over the layer 1 feature maps and produces the layer 2 feature map with the green dashed border.

There are a few things to notice in this figure. First, the spatial resolution of the feature maps get lower as we go deeper into the network, and the number of channels increases. This is common in CNNs: each layer downsamples and adds channels to partially compensate for the reduction in resolution. Second, while on the first layer the feature maps are sensitive to basic patterns in the input image – edges, lines, etc – the maps become more abstracted as we go deeper. This is typical of image classifier networks: channels in the shallow layers capture basic image features and channels in the deeper layers increasingly correspond to class semantics (e.g., one channel might be a heatmap of where the “bird” pixels are).

Figure 24.25 shows one more way to visualize feature maps. Rather than plotting the channels as a column of grayscale images, we run PCA to reduce the channel dimensionality to 3. Then we can directly render each feature map as a color image, with red showing the first principle component of each layer's feature map, green the second, and blue the third. We show this for 5 layers in three common networks, AlexNet, VGG16 [444], and ResNet18 [196].

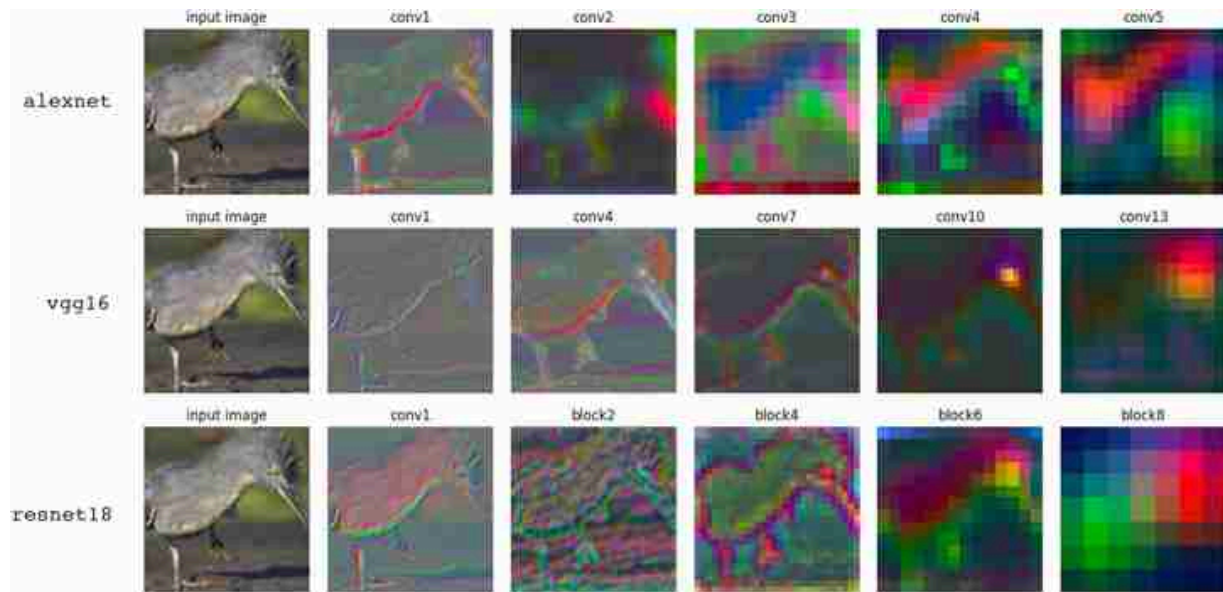


Figure 24.25: PCA visualization of feature maps in three convolutional networks. Because each of these networks has a different number of layers, we select 5 that are evenly spaced from the first to last convolutional feature map.

24.7 Receptive Fields

Receptive fields are another important concept when working with CNNs. In chapter 1, we learned about history of receptive fields in neuroscience. As a reminder, the receptive field of a neuron is the region of the input signal that the neuron is sensitive to, i.e. its support. In multilayer perceptrons (MLPs, chapter 12), the receptive field of each neuron is the entire input vector since MLPs use *fully* connected layers. In CNNs, on the other hand, each neuron only sees a portion of the input, since each output neuron on a conv layer is only connected to a subset of inputs to the conv layer, determined by the kernel size of the filter that produces that output.

The receptive fields of two example neurons in a CNN are shown below ([figure 24.26](#)):

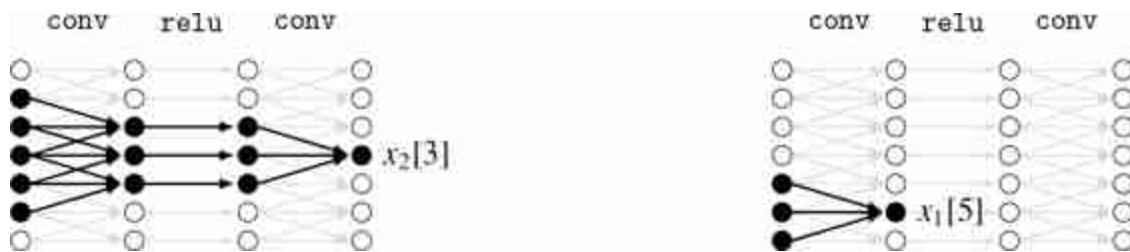


Figure 24.26: Receptive fields in a CNN. The black filled neurons are within the receptive fields of each labeled neuron (left: $x_2[3]$, right: $x_1[5]$).

Notice that the receptive field grows the deeper we go into the network. To understand why, consider a CNN without nonlinearities. Then the $l + 1$ -th layer is the composition of l convolutional filters. As we saw in section 15.4.1, composing filters results in a new filter with larger support (kernel size). The same happens in a CNN with pointwise nonlinearities, since pointwise operations do not affect receptive field (the outputs have the same receptive fields as the inputs). Further, whenever we have a downsampling layer by factor s , the receptive field of the output is s times larger than the receptive field of the input. Because of these properties, receptive field sizes can grow rapidly as we go deeper in CNNs. Generally we want that the final layer of the CNN has large enough receptive fields to see entire input image, so that output neurons are sensitive to *all* pixels in the input. This can be achieved with a gap layer, whose output will always have a receptive field size that covers the entire input.

24.8 Spatial Outputs

In section 24.4 we saw a CNN that outputs a single class probability vector for an image. What if we want to output a spatially varying map of predictions, like we discussed in the intro to this chapter? To achieve this, we can simply downsample less, so that the final layer of the CNN is a feature map that maintains higher spatial resolution. It is also important to remove any global pooling layers.

An example is given below:

$$\mathbf{z}_1[c_1, :, :] = \sum_{c=0}^c \mathbf{w}_1[c, c_1, :, :] * \mathbf{x} + b_1[c_1] \quad \triangleleft \text{conv}$$

$$[3 \times N \times M] \rightarrow [C_1 \times N \times M] \quad (24.23)$$

$$h[c_1, n, m] = \max(z_1[c_1, n, m], 0)$$

$$\triangleleft \text{relu}$$

$$[C_1 \times N \times M] \rightarrow [C_1 \times N \times M] \quad (24.24)$$

$$\mathbf{z}_2[c_2, :, :] = \sum_{c_1=0}^{C_1-1} \mathbf{w}_2[c_1, c_2, :, :] * \mathbf{x} + b_2[c_2] \quad \triangleleft \text{conv}$$

$$[C_1 \times N \times M] \rightarrow [C_2 \times N \times M] \quad (24.25)$$

$$y[k, n, m] = \frac{e^{-\tau z_2[k, n, m]}}{\sum_{l=1}^K e^{-\tau z_2[l, n, m]}} \quad \triangleleft \text{softmax}$$

$$[K \times N \times M] \rightarrow [K \times N \times M] \quad (24.26)$$

In [figure 24.27](#) we visualize this CNN (showing only a 1D slice of this 2D CNN):

Notation reminder: nodes that are squares indicate that they represent multiple channels (each is a vector of neurons)

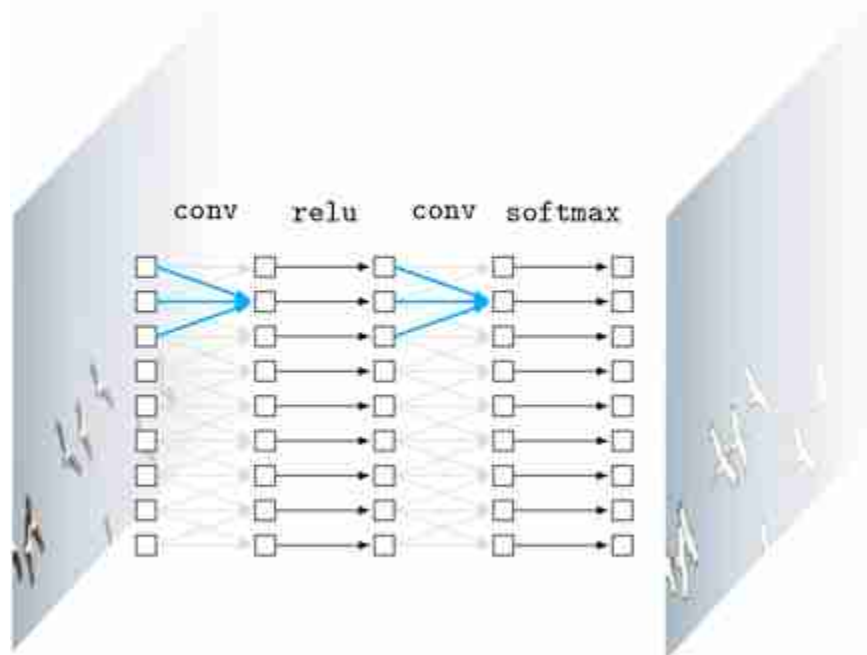


Figure 24.27: A 1D slice of a CNN that maps an image to an image. The input is the photo of size $3 \times N \times M$ and the output is a class probability map of size $K \times N \times M$; we visualize the corresponding label map on the right (per-pixel argmax over output probabilities). The blue arrows are the learnable parameters. The gray arrows share the weights of the blue arrows.

Although historically CNNs first became popular as image classifiers, this usage hides their real power. Rather than thinking of them as image-to-label architectures, think of them as *image-to-image* architectures.

More generally, CNNs are X -to- X architectures for any domain X over which translation can be defined.

24.9 CNN as a Sliding Filter

The core of CNNs are the convolutional layers, and in this section we will consider a CNN with only `conv` layers interleaved with pointwise nonlinearities. Such a CNN is sometimes called a **fully convolutional network** or **FCN** [305]. What we will show below is that a whole FCN is just another sliding image filter.

To see why, consider a CNN that processes a 1D signal, and outputs a feature map $\mathbf{x}_L$. Take two feature vectors in the output map, $\mathbf{x}_L[:, i]$ and $\mathbf{x}_L[:, j]$. The feature vector at location i is some function, F , of the input patch in

its receptive field, $\mathbf{x}_L[:, i] = F(\mathbf{x}_{in}[:, \text{RF}(i)])$, where RF returns the coordinates of the receptive field in the input image. It turns out that the feature vector at pixel j is produced by the *same* function, just applied to a different patch of the input: $\mathbf{x}_L[:, j] = F(\mathbf{x}_{in}[:, \text{RF}(j)])$.

This is easiest to understand with a visual proof, which we give in [figure 24.28](#) (pointwise nonlinearities are omitted for clarity):



Figure 24.28: A CNN is a non-linear filter. Edge colors indicate shared weights; two edges with the same color have the same weight. The colors demonstrate that the same function F is applied to each patch of input nodes.

To understand this property, first imagine the CNN has no pointwise nonlinearities. Then the entire CNN is just the composition of a sequence of convolutions, which itself is a convolution (by equation (15.16), convolving a signal with multiple filters in a row is equivalent to convolving the signal with a single equivalent filter). Therefore, a CNN with no nonlinearities is itself just a single big convolutional filter. The key property of such a system is that it processes each input patch independently and identically. Now notice that this key property is unchanged when we add pointwise nonlinearities, because they introduce no interaction between neurons or pixels (they are pointwise after all). Hence it follows that a complete CNN, made up only of convolutional layers and pointwise nonlinearities, is itself a nonlinear operator that applies the same transformation independently and identically to each patch of the input signal, i.e. a nonlinear filter!

This is why, in the intro to this chapter, we visualized a CNN as chopping up an image into patches and applying the same “classifier” function to each patch.

24.10 Why Process Images Patch by Patch?

As we have seen above, a fully convolutional CNN can be thought of a function that processes each patch of the input independently and identically.

Although some CNNs have non-convolutional layers, like `gap` and `fc` layers, what defines CNNs is the stack of fully convolutional layers, i.e. the convolutions and pointwise activations.

In this section we will discuss why these two properties are useful for image processing.

Property #1: Treating Patches as Independent This is a divide-and-conquer strategy. If you were to try to understand a complex problem, you might break it up into small pieces and solve each one separately. That's all a CNN is doing. We split up a big problem (i.e. “interpret this whole photo”) into a bunch of smaller problems (i.e. “interpret each small patch in the image”).

Why is this a good strategy?

1. The small problems are easier to solve than the original problem.
2. The small problems can all be solved in parallel.
3. This approach is *agnostic to signal length*, that is, you can solve an arbitrarily large problem just by breaking it down to bite size pieces and solving them “bird by bird” [281].

Chopping up into small patches like this is sufficient for many vision problems because the world exhibits **locality**: related things clump together, that is, within a single patch; far apart things can often be safely assumed to be independent.

Property #2: Processing Each Patch Identically For images, convolution is an especially suitable strategy because visual content tends to be *translation invariant*, and, as we learned in previous chapters, the convolution operator is also translation invariant.

Typically, objects can appear anywhere in an image and look the same, like the birds in the photo from [figure 24.1](#). This is because as the birds fly across the frame their position changes but their identity and appearance

does not. More generally, as a camera pans across a scene, the content shifts in position but is otherwise unchanged.

Because the visual world is roughly translation invariant, it is justified to process each patch the same way, regardless of its position (i.e., its translation away from some canonical center patch).

Translation invariant just means we process each patch identically, using the same function f . Some texts instead use the term **translation equivariant** to describe convolutions. This places emphasis on the fact that if we shift the input signal by some translation, then the output signal will get shifted by the same amount. That is, if f is a convolution, we have the property:

$$f(\text{translate}(x)) = \text{translate}(f(x)) \quad (24.27)$$

24.11 Popular CNN Architectures

We have now seen all the essential building blocks of CNNs. In this section we will treat these blocks like LEGOs and show how to piece them together to make a variety of useful architectures.

24.11.1 Encoder and Decoders

In chapter 23, we encountered image pyramids that include both an analysis pipeline, which converts an image into a multiscale representation of filter responses, and a synthesis pipeline, which reconstructs the image from the filter responses. Deep networks also may operate in either the analysis direction or the synthesis direction. In the context of deep networks, we call the analysis network an **encoder** and the synthesis network a **decoder**. An encoder maps from data to a representation of that data, which is usually lower dimensional, and a decoder performs the inverse operation, mapping from a representation back to the data.

Encoder and decoder networks will appear many times in this book, and can be made from many different architectures, including those that are not neural nets. In the context of CNNs, encoders are typically nets that take an image as input and then downsample it, layer by layer, until producing a much lower-dimensional feature map as output. A decoder is the opposite, taking a set of low-dimensional features as input, then upsampling them, layer by layer, until producing an image as the final output. An example of an encoder is an image classifier and an example of a decoder is an image

generator (covered in chapter 32). These two architectural patterns are shown in [figure 24.29](#).

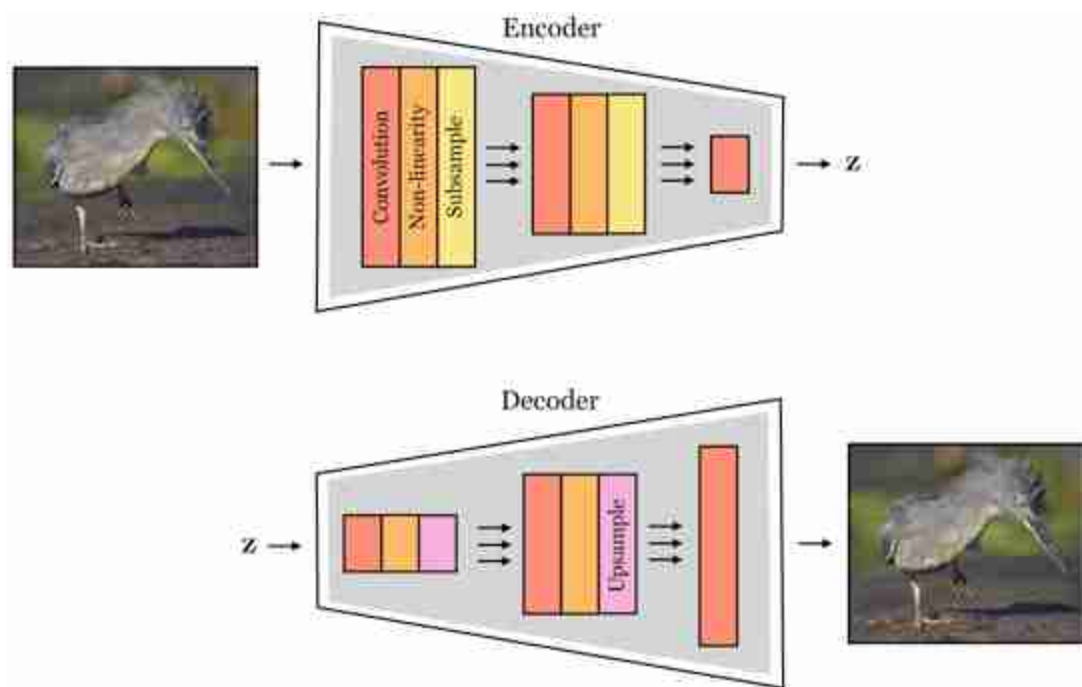


Figure 24.29: A convolutional encoder (top) and a convolutional decoder (bottom). The exact ordering of the operations (e.g., downsample before or after the non-linearity) is just an example and may vary in different encoder and decoder models. The z is a feature map or vector of neural activations, and is sometimes called an **embedding** (see chapter 30).

One powerful thing you can do with encoders and decoders is to put them together, forming an **encoder-decoder**. Such an architecture first encodes the input image into a low-dimensional representation, then decodes the representation back into an image output. This is therefore a suitable architecture for image-to-image problems, and it has a few advantages over the image-to-image CNNs we saw in section 24.8: 1) by downsampling, the internal feature maps are smaller, using less memory and computation, 2) encoder-decoders introduce an **information bottleneck** – i.e. the representation between the encoder and decoder is low-dimensional and can only transmit so much information – and this forces abstraction. This latter concept is one we will study in much greater detail in chapter 30, where we will see several benefits of compressing a

signal. A schematic of the encoder-decoder architecture is shown in [figure 24.30](#).

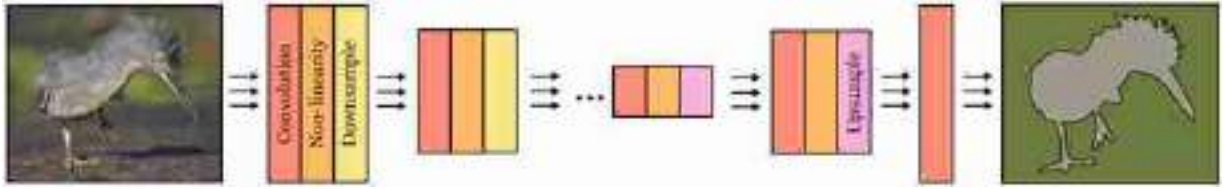


Figure 24.30: Encoder-decoder architecture. In this example, the input is an image and the output is a segmentation map.

24.11.2 U-Nets

Encoder-decoders force the signal to pass through a bottleneck, and although this can be a good thing (as discussed above), it also makes the task of the decoder rather difficult. In particular, the decoder may fail to be able to output high frequency details; for a semantic segmentation network, the consequence could be that the predicted label map is very coarse.

To circumvent this problem, we can add **skip connections** that shuttle information directly across blocks of layers in the net. A skip connection f is simply an identity pathway that connects two disparate layers of a net, $f(\mathbf{x}) = \mathbf{x}$.

Adding skip connections to an encoder-decoder results in an architecture known as a **U-Net** [410]. In this architecture, skip connections are arranged in a mirror pattern, where layer l is connected directly to layer $(L - l)$. The output of a skip connection must somehow be reintegrated into the network. U-nets do this by concatenating the activations from the prior layer to the activations on the later layer, along the channel dimension. This architecture can maintain the information-bottleneck of the encoder-decoder, with its incumbent benefits in terms of memory and compute efficiency and forced abstraction, while also allowing residual information to flow through the skip connections, thereby not sacrificing the ability to output high-frequency spatial predictions. U-Nets look a lot like the steerable pyramids from chapter 23, which also consist of a downsampling *analysis* path followed by an upsampling *synthesis* path, with skip connections in between mirror image stages in the pathways. The main difference is that the U-Net has *learned filters* and *nonlinearities*. A schematic of a U-net is given in [figure 24.31](#).

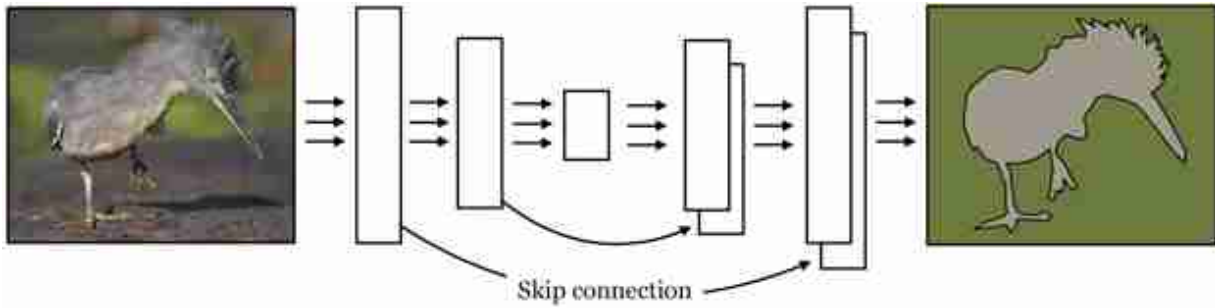


Figure 24.31: U-net architecture. Each block contains a series of layers. The skip connections concatenate activations.

24.11.3 ResNets

Another popular architecture that uses skip connections is called **Residual Networks** or **ResNets** [196].

See also Highway Networks [453], a related architecture that uses a form of skip connection but controlled by multiplicative gates.

In the context of ResNets, skip connections are called **residual connections**. This kind of skip connection is *added* to the output of a block of layers F :

$$\mathbf{x}_{\text{out}} = F(\mathbf{x}_{\text{in}}) + \mathbf{x}_{\text{in}} \quad \Leftarrow \text{residual block} \quad (24.28)$$

In a **residual block** like this, you can think of F as a *residual* that additively perturbs $\mathbf{x}_{\text{in}}$ to transform it into an improved $\mathbf{x}_{\text{out}}$. If $\mathbf{x}_{\text{out}}$ does not have the same dimensionality as $\mathbf{x}_{\text{in}}$, then we can add a linear mapping to convert the dimensions: $\mathbf{x}_{\text{out}} = F(\mathbf{x}_{\text{in}}) + \mathbf{W}\mathbf{x}_{\text{in}}$.

It is easy for a residual block to simply perform an identity mapping, it just has to learn to set F to zero. Because of this, if we stack many residual blocks in a row, it can end up that the net learns to use only a subset of them. If we set the number of stacked residual blocks to be very large then the net can essentially learn how deep to make itself, using as many blocks as necessary to solve the task. ResNets often exploit this fact by being very deep; for example, they may be hundreds of blocks deep. [Figure 24.32](#) depicts a 5 block deep ResNet.

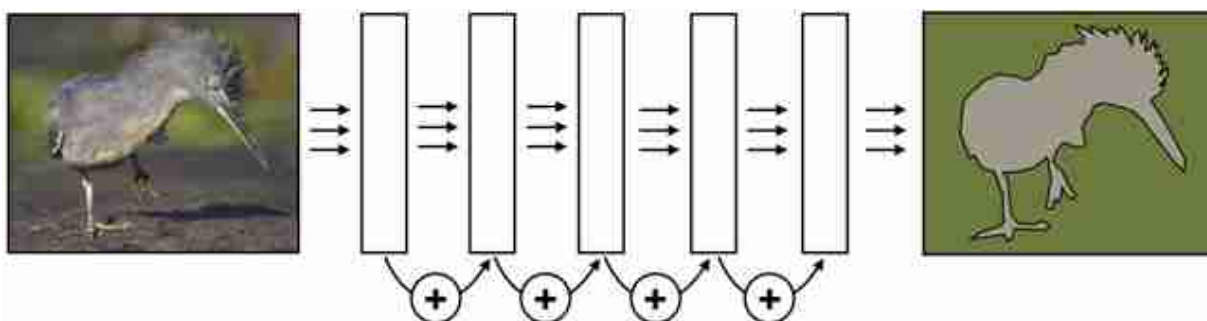


Figure 24.32: ResNet architecture. Each block contains a series of layers. The skip connections add activations from one block to the next.

24.12 Concluding Remarks

We will see in later chapters that several new kinds of models are recently supplanting CNNs as the most successful architectures for vision problems. One such architecture is the **transformer** (chapter 26). It may be tempting to think, “Why did we bother learning about CNNs then, if transformers are better!” The reason we cover CNNs is not because the exact architecture presented here will last, but because the underlying principles it embodies are ubiquitous in sensory processing. The two key properties mentioned in section 24.10 are in fact present in transformers and many other architectures beyond CNNs: transformers also include stages that process each patch identically and independently, but they interleave these stages with other stages that globalize information across patches. It comes down to preferences whether you want to call these newer architectures convolutional or not, and there is currently some debate about it in the community. For us it doesn't matter, because if we learn the principles we can recognize them in all the systems we encounter and need not get hung up on names.

10. The answers are 1-B, 2-C.

25 Recurrent Neural Nets

25.1 Introduction

RNNs are a neural net architecture for modeling sequences—they perform *sequential* computation (rather than parallel computation).

So far we have seen **feedforward** neural net architectures. These architectures can be represented by directed acyclical computation graphs. **Recurrent neural networks (RNNs)**, are neural nets with feedback connections.

A feedback connection defines a **recurrence** in the computational graph.

Their computation graphs have cycles. That means that the outputs of one layer can send a message back to earlier in the computational pipeline. To make this well-defined, RNNs introduce the notion of **timestep**. One timestep of running an RNN corresponds to one functional call to each layer (or computation node) in the RNN, mapping that layer's inputs to its outputs.

RNNs are all about adaptation. They solve the following problem: What if we want our computations in the past to influence our computations in the future? If we have a stream of data, a feedforward net has no way to adapting its behavior based on what outputs it previously produced.

To motivate RNNs, we will first consider a deficiency of convolutional neural nets (CNNs). Consider that we are processing a temporal signal with a CNN, like so:

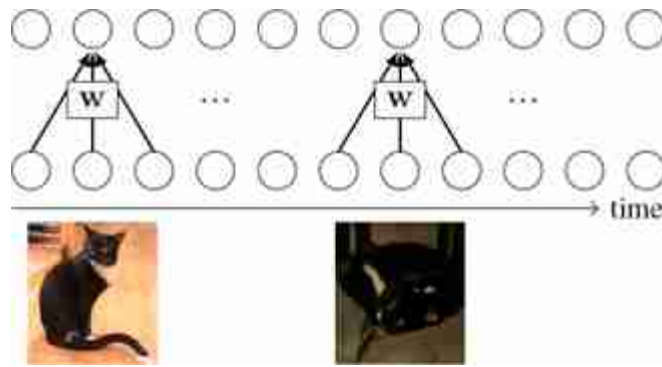


Figure 25.1: A CNN processing a video, with two example frames shown.

The filter w slides over the input signal and produces outputs. In this example, imagine the input is a video of your cat, named Mo. The output produced for the first set of frames is entirely independent from the output produced for a later frame, as long as the receptive fields do not overlap. This independence can cause problems. For instance, maybe it got dark outside as the video progressed, and a clear image of Mo became hard to discern as the light dimmed. Then the convolutional response, and the net's prediction, for later in the video cannot reference the earlier clear image, and will be limited to making the best inference it can from the dim image.

Wouldn't it be nice if we could inform the CNN's later predictions that earlier in the day we had a brighter view and this was clearly Mo? With a CNN, the only way to do this is to increase the size of the convolutional filters so that the receptive fields can see sufficiently far back in time. But what if we last saw Mo a year ago? It would be infeasible to extend our receptive fields a year back. How can we humans solve this problem and recognize Mo after a year of not seeing her?

The answer is memory. The recurrent feedback loops in an RNN are a kind of memory. They propagate information from timestep t to timestep $t + 1$. In other words, they remember information about timestep t when processing new information at timestep $t + 1$. Thus, by induction, an RNN can potentially remember information over arbitrarily long time intervals. In fact, RNNs are Turing complete: you can think of them as a little computer. They can do anything a computer can do.

Here's how RNNs arrange their connections to perform this propagation:

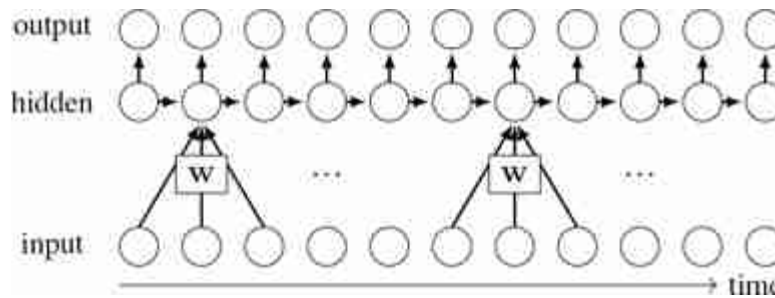


Figure 25.2: Basic RNN with one hidden layer.

The key idea of an RNN is that there are lateral connections, between hidden units, through time. The inputs, outputs, and hidden units may be multidimensional tensors; to denote this we will use $\square$, as $\bigcirc$ was reserved for denoting a single (scalar) neuron. Here is a simple 1-layer RNN:

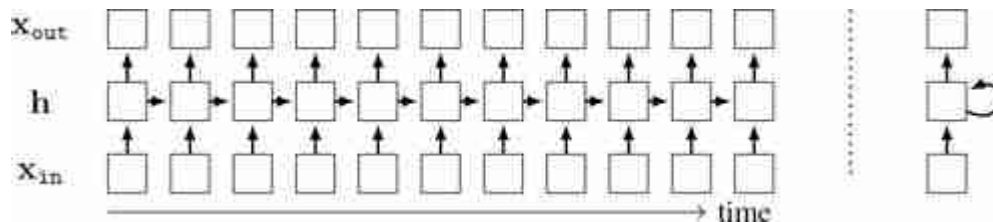


Figure 25.3: (left) RNN with multidimensional inputs, hidden states, and outputs. (right) the rolled-up version of this RNN, which can be unrolled for an unbounded number of steps.

To the left, is the **unrolled computation graph**. To the right, is the computation graph rolled up, with a feedback connection. Typically we will work with unrolled graphs because they are directed acyclic graphs (DAGs), and therefore all the tools we have developed for DAGs, including backpropagation, will extend naturally to them. However, note that RNNs can run for unbounded time, and process signals that have unbounded temporal extent, while a DAG can only represent a finite graph. The DAG depicted above is therefore a *truncated* version of the RNN's theoretical computation graph.

25.2 Recurrent Layer

A **recurrent layer** is defined by the following equations:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_{\text{in}}[t]) \quad \triangleleft \quad \text{update state based on input and previous state} \quad (25.1)$$

$$\mathbf{x}_{\text{out}}[t] = g(\mathbf{h}_t) \quad \triangleleft \quad \text{produce output based on state} \quad (25.2)$$

This is a layer that includes a state variable, $\mathbf{h}$. The operation of the layer depends on its state. The layer also updates its state every time it is called, therefore the state is a kind of memory.

The idea of a *time* in an RNN does not necessarily mean time as it is measured on a clock. Time just refers to *sequential computation*; the timestep t is an index into the sequence. The sequential computation could progress over a spatial dimension (processing an image pixel by pixel) or over the temporal dimension (processing a video frame by frame), or even over more abstracted computational modules.

The f and g can be arbitrary functions, and we can imagine a wide variety of recurrent layers defined by different choices for the form of f and g .

One common kind of recurrent layer is the **Simple Recurrent Layer (SRN)**, which was defined by Elman in “Finding Structure in Time” [116]. For this layer, f is a linear layer followed by a pointwise nonlinearity, and g is another linear layer. The full computation is defined by the following equations:

$$\mathbf{h}_t = \sigma_1(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_{\text{in}}[t] + \mathbf{b}) \quad (25.3)$$

$$\mathbf{x}_{\text{out}}[t] = \sigma_2(\mathbf{V}\mathbf{h}_t + \mathbf{c}) \quad (25.4)$$

where σ_1 and σ_2 are pointwise nonlinearities. In SRNs, $\tanh$ is the common choice for the nonlinearity, but relus and other choices may also be used.

25.3 Backpropagation through Time

Backpropagation is only defined for DAGs and RNNs are not DAGs. To apply backpropagation to RNNs, we unroll the net for T timesteps, which creates a DAG representation of the truncated RNN, and then do backpropagation through that DAG. This is known as **backpropagation through time (BPTT)**.

This approach yields exact gradients of the loss with respect to the parameters when T is equal to the total number of timesteps the RNN was run for. Otherwise, BPTT is an approximation that truncates the true computation graph. Essentially, BPTT ignores any dependencies in the computation graph greater than length T .

As an example of BPTT, suppose we want to compute the gradient of an output neuron at timestep five with respect to an input at timestep zero, that is, $\frac{\partial \mathbf{x}_{\text{out}}[5]}{\partial \mathbf{x}_{\text{in}}[0]}$. The forward pass (black arrows) and backward pass (red arrows) are shown in [figure 25.4](#), where we have only drawn arrows for the subpart of the computation graph that is relevant to the calculation of this particular gradient.

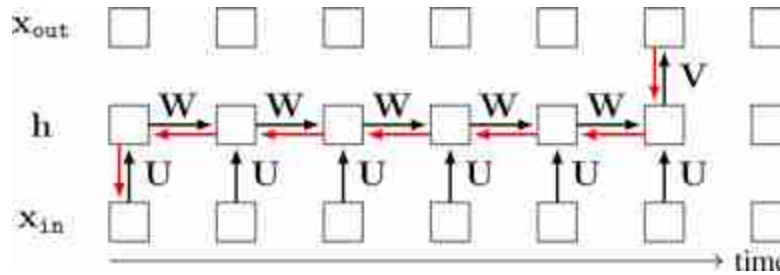


Figure 25.4: Backpropagation through time in an RNN.

What about the gradient of the total cost with respect to the parameters, $\frac{\partial J}{\partial [\mathbf{W}, \mathbf{U}, \mathbf{V}]}$? Suppose the RNN is being applied to a video and predicts a class label for every frame of the video. Then $\mathbf{x}_{\text{out}}$ is a prediction for each frame and a loss can be applied to each $\mathbf{x}_{\text{out}}[t]$. The total cost is simply the summation of the losses at each timestep:

$$\frac{\partial J}{\partial [\mathbf{W}, \mathbf{U}, \mathbf{V}]} = \sum_{t=0}^T \frac{\partial \mathcal{L}(\mathbf{x}_{\text{out}}[t], y_t)}{\partial [\mathbf{W}, \mathbf{U}, \mathbf{V}]} \quad (25.5)$$

The graph for backpropagation, shown in [figure 25.5](#), has a branching structure, which we saw how to deal with in chapter 14: gradients (red arrows) summate wherever the forward pass (black arrows) branches.

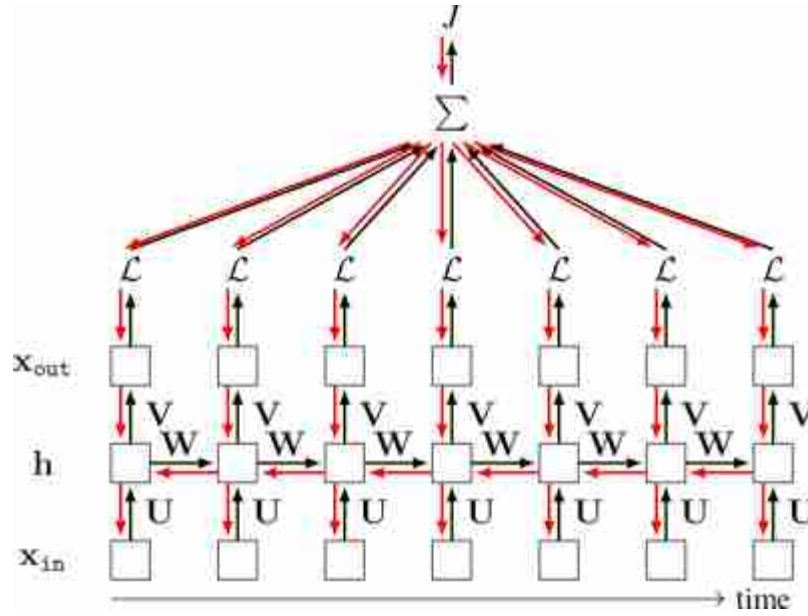


Figure 25.5: Backpropagation through time to minimize the sum of losses incurred at each timestep.

25.3.1 The Problem of Exploding and Vanishing Gradients

Notice that the we can get very large computation graphs when we run an RNN for many timesteps. The gradient computation involves a series of matrix multiplies that is $O(T)$ in length. For example, to compute our gradient $\frac{\partial x_{out}[5]}{\partial x_{in}[0]}$ in the example above involves the following product:

$$\frac{\partial x_{out}[t]}{\partial x_{in}[0]} = \frac{\partial x_{out}[t]}{\partial h_T} \frac{\partial h_T}{\partial h_{T-1}} \cdots \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x_{in}[0]} \quad (25.6)$$

Suppose we use an RNN with relu nonlinearities. Then each term $\frac{\partial h_i}{\partial h_{i-1}}$ equals $\mathbf{R}\mathbf{W}$, where $\mathbf{R}$ is a diagonal matrix with zeros or ones on the i -th diagonal element depending on whether the i -th neuron is above or below zero at the current operating point. This gives a product $\mathbf{R}_T\mathbf{W}\cdots\mathbf{R}_1\mathbf{W}$. In a worst-case scenario (for numerics), suppose all the relu are active; then we have a product of T matrices $\mathbf{W}$, which means the gradient is $\mathbf{W}^T$ (T is an exponent here, not a transpose). If values in $\mathbf{W}$ are large, the gradient will explode. If they are small, the gradient will vanish—basically, this system is not numerically well-behaved. One solution to this problem is to use RNN units that have better numerics, an example of which we will give in section 25.5 subsequently.

25.4 Stacking Recurrent Layers

A deep RNN can be constructed by stacking recurrent layers. Here is an example:

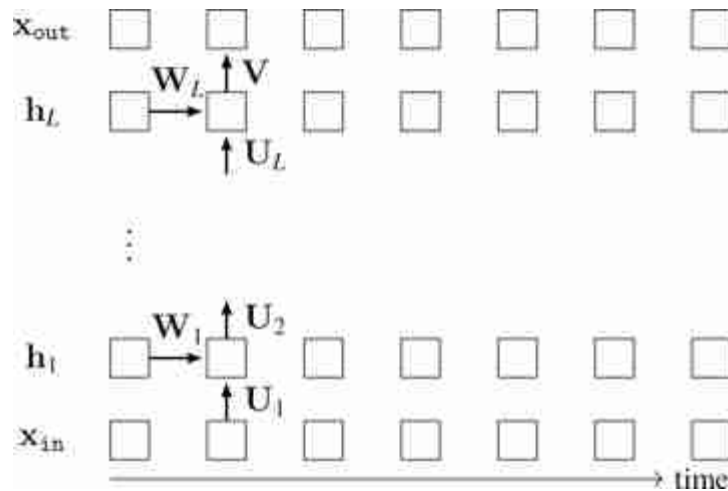


Figure 25.6: A deep RNN with multiple hidden layers.

To emphasize that the parameters are shared, we only show them once. Here is code for a two-layer version of this RNN:

```

# d_in, d_hid, d_out: input, hidden, and output dimensionalities
# x : input data sequence
# h_init: initial conditions of hidden state
# T: number of timesteps to run for

# first define parameterized layers
U1 = nn.fc(dim_in=d_in, dim_out=d_hid, bias=False)
U2 = nn.fc(dim_in=d_hid, dim_out=d_hid, bias=False)
W1 = nn.fc(dim_in=d_hid, dim_out=d_hid)
W2 = nn.fc(dim_in=d_hid, dim_out=d_hid)
V = nn.fc(dim_in=d_hid, dim_out=d_out)

# then run data through network
h1, h2 = h_init
for t in range(T):
    h1[t] = nn.tanh(W1(h1[t-1])+U1(x[t]))
    h2[t] = nn.tanh(W2(h2[t-1])+U2(h1[t]))
    x_out[t] = V(h2[t])

```

We set the bias to be “False” for U_1 and U_2 since the recurrent layer already gets a bias term from W_1 and W_2 .

25.5 Long Short-Term Memory

Long Short-Term Memory layers (LSTMs) are a special kind of recurrent module, that is designed to avoid the vanishing and exploding gradients problem described in section 25.3.1 [209].

The presentation of LSTMs in this section draws inspiration from a great blog post by Chris Olah [357].

An LSTM is like a little memory controller. It has a **cell state**, $\mathbf{c}$, which is a vector it can read from and write to. The cell state can record memories and retrieve them as needed. The cell state plays a similar role as the hidden state, $\mathbf{h}$, from regular RNNs—both record memories—but the cell state is built to store *persistent* memories that can last a long time (hence the name *long* short-term memory). We will see how next.

Like a normal recurrent layer, an LSTM takes as input a hidden state $\mathbf{h}_{t-1}$ and an observation $\mathbf{x}_{in}[t]$ and produces as output an updated hidden state $\mathbf{h}_t$. Internally, the LSTM uses its cell state to decide how to update $\mathbf{h}_{t-1}$.

First, the cell state is updated based on the incoming signals, $\mathbf{h}_{t-1}$ and $\mathbf{x}_{in}[t]$. Then the cell state is used to determine the hidden state $\mathbf{h}_t$ to output.

Different subcomponents of the LSTM layer decide what to *delete* from the cell state, what to *write* to the cell state, and what to *read* off the cell state.

One component picks the indices of the cell state to *delete*:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{h}_{t-1} + \mathbf{U}_f \mathbf{x}_{\text{in}}[t] + \mathbf{b}_f) \quad \triangleleft \quad \text{decide which indices to forget} \quad (25.7)$$

where $\mathbf{f}_t$ is a vector of length equal to the length of the cell state $\mathbf{c}_t$, and σ is the sigmoid function that outputs values in the range $[0, 1]$. The idea is that most values in $\mathbf{f}_t$ will be near either 1 (remember) or 0 (forget). Later $\mathbf{f}_t$ will be pointwise multiplied by the cell state to forget the values where $\mathbf{f}_t$ is zero.

Another component chooses what values to *write* to the cell state, and where to write them:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{h}_{t-1} + \mathbf{U}_i \mathbf{x}_{\text{in}}[t] + \mathbf{b}_i) \quad \triangleleft \quad \text{which indices to write to} \quad (25.8)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{h}_{t-1} + \mathbf{U}_c \mathbf{x}_{\text{in}}[t] + \mathbf{b}_c) \quad \triangleleft \quad \text{what to write to those indices} \quad (25.9)$$

where $\mathbf{i}_t$ is similar to $\mathbf{f}_t$ (it's a vector whose values that are nearly 0 or 1 of length equal to the length of the cell state) and it determines which indices of $\tilde{\mathbf{c}}_t$ will be written to the cell state. Next we use these vectors to actually update the cell state:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \triangleleft \quad \text{update the cell state} \quad (25.10)$$

Finally, given the updated cell state, we *read* from it and use the values to determine the hidden state to output:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{h}_{t-1} + \mathbf{U}_o \mathbf{x}_{\text{in}}[t] + \mathbf{b}_o) \quad \triangleleft \quad \text{which indices of cell state to use} \quad (25.11)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \triangleleft \quad \text{use these to determine next hidden state} \quad (25.12)$$

The basic idea is it should be easy, by default, not to forget. The cell state controls what is remembered. It gets modulated, but $\mathbf{f}_t$ can learn to be all ones so that information propagates forward through an identity connection from the previous cell state to the subsequent cell state. This idea is similar to the idea of skip connections and residual connections in U-Nets and ResNets (chapter 24).

25.6 Concluding Remarks

Recurrent connections give neural nets the ability to maintain a persistent state representation, which can be propagated forward in time or can be updated in the face of incoming experience. The same idea has been proposed in multiple fields: in control theory it is the basic distinction between open-loop and closed-loop control. Closed-loop control allows the

controller to change its behavior based on *feedback* from the consequences of its previous decisions; similarly, recurrent connections allow a neural net to change its behavior based on feedback, but the feedback is from downstream processing all within the same neural net. Feedback is a fundamental mechanism for making systems that are adaptive. These systems can become more competent the longer you run them.

26 Transformers

26.1 Introduction

Transformers are a recent family of architectures that generalize and expand the ideas behind convolutional neural nets (CNNs). The term for this family of architectures was coined by [487], where they were applied to language modeling. Our treatment in this chapter more closely follows the **vision transformers (ViTs)** that were introduced in [109].

Like CNNs, transformers factorize the signal processing problem into stages that involve independent and identically processed chunks. However, they also include layers that mix information across the chunks, called **attention layers**, so that the full pipeline can model dependencies between the chunks.

Transformers were originally introduced in the field of natural language processing, where they were used to model language, that is, sequences of characters and words. As a result, some texts present transformers as an alternative to recurrent neural nets (RNNs) for sequence modeling, but in fact transformer layers are *parallel* processing machines, like convolutional layers, rather than sequential machines, like recurrent layers.

26.2 A Limitation of CNNs: Independence between Far Apart Patches

CNNs are built around the idea of *locality*: different local regions of an image can safely be processed independently. This is what allows us to use filters with small kernels. However, very often, there is global information that needs to be shared across all receptive fields in an image. Convolutional layers are not well-suited to *globalizing* information since the only way they can do so is by either increasing the kernel size of their filters or stacking layers to increase the receptive field of neurons on deeper layers. [Figure 26.1](#) shows the inability of a shallow CNN to compare two input nodes (x_1 and x_7) that are spatially too far apart:

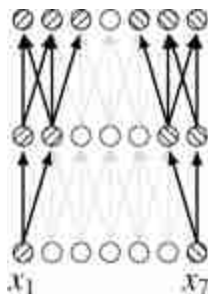


Figure 26.1: Consider a 2-layer CNN with kernel size 3, tasked to compare x_1 and x_7 . It can't do it: there are no neurons that are connected to both x_1 and x_7 . Hatch marks indicate which neurons are connected to x_1 and x_7 respectively.

How can we efficiently pass messages across large spatial distances? We already have seen one option: just use a fully connected layer, so that every output neuron after this layer takes input from every neuron on the layer before. However, fully connected layers have a ton of parameters (N^2 if their input and output are N -dimensional vectors), and it can take an exorbitant amount of time and data to fit all those parameters. Can we come up with a more efficient strategy?

26.3 The Idea of Attention

Attention is a strategy for processing global information efficiently, focusing just on the parts of the signal that are most salient to the task at hand. The idea can be motivated by attention in human perception. When we look at a scene, our eyes flick around and we attend to certain elements that stand out, rather than taking in the whole scene at once [508]. If we are asked a question about the color of a car in the scene, we will move our eyes to look at the car, rather than just staring passively. Can we give neural nets the same ability?

In neural nets, attention follows the same intuitive idea. A set of neurons on layer $l + 1$ may *attend* to a set of neurons on layer l , in order to decide what their response should be. If we “ask” that set of neurons to report the color of any cars in the input image, then they should direct their attention to the neurons on the previous layer that represent the color of the car. We will soon see how this is done, in full detail, but first we need to introduce a new data structure and a new way of thinking about neural processing.

26.4 A New Data Type: Tokens

We discussed that the main data structures in deep learning are different kinds of groups of neurons: channels, tensors, batches, and so on. Now we will introduce another fundamental data structure, **tokens**. A token is another kind of group of neurons, but there are particular ways we will operate over tokens that are different from how we operated over channels, batches, and the other groupings we saw before. Specifically, we will think of tokens as *encapsulated* groups of information; we will define operators over tokens, and these operators will be our only interface for accessing and modifying the internal contents of tokens. From a programming languages perspective, you can think of tokens as a new data *type*.

In this chapter we will only consider tokens whose internal content is a vector of neurons. A single token will therefore be represented by a column vector $\mathbf{t} \in \mathbb{R}^{d \times 1}$, which is also sometimes called the token's **code vector**.

26.4.1 Tokenizing Data

The first step to working with tokens is to *tokenize* the raw input data. Once we have done this, all subsequent layers will operate over tokens, until the

output layer, which will make some decision or prediction as a function of the final set of tokens. How can we tokenize an input image? Well, how did we “neuronize” an image for processing in a vanilla neural net? We simply represented each *pixel* in the image with a neuron (or three neurons, if it's a color image). To tokenize an image, we may simply represent each *patch of pixels* in the image with a token. The token vector is the vectorized patch (stacking the three color channels one after the other), or a lower-dimensional projection of the vectorized patch. With each patch represented by a token, the full image corresponds to an array of tokens. [Figure 26.2](#) shows what it looks like to tokenize a safari image in this way.

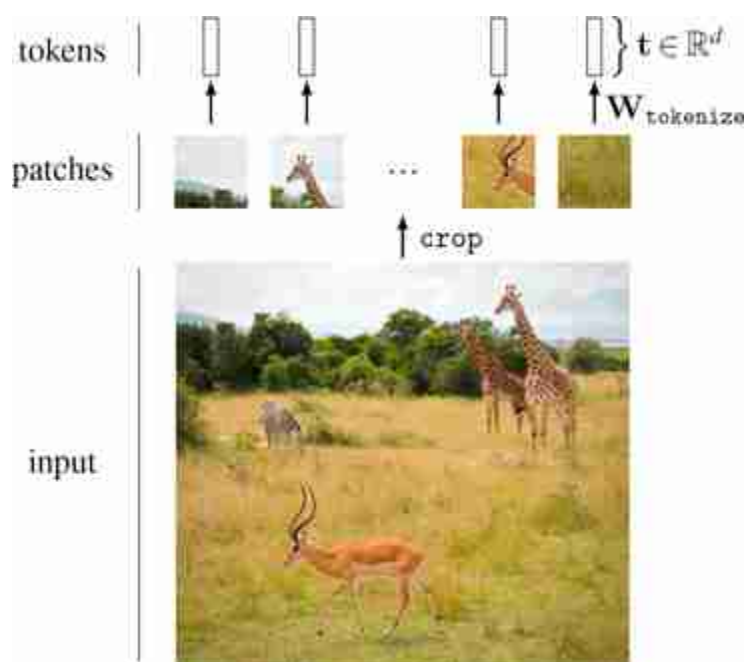


Figure 26.2: Tokenization: converting an image to a set of vectors. $\mathbf{W}_{\text{tokenize}}$ is a learnable linear projection from the dimensionality of the vectorized crops to d dimensions. This is just one of many possible ways to tokenize an image.

26.4.2 Data Structures and Notation for Working with Tokens

A sequence of tokens will be denoted by a matrix $\mathbf{T} \in \mathbb{R}^{N \times d}$, in which each token in the sequence, $\mathbf{t}_1, \dots, \mathbf{t}_N$, is transposed to become a row of the matrix:

$$\mathbf{T} = \begin{bmatrix} \mathbf{t}_1^T \\ \vdots \\ \mathbf{t}_N^T \end{bmatrix} \quad (26.1)$$

As we will see, transformers are invariant to permutations of the input sequence, so, as far as transformers are concerned, groups of tokens should be thought of as *sets* rather than ordered sequences.

Graphically, $\mathbf{T}$ is constructed from $\mathbf{t}_1, \dots, \mathbf{t}_N$ like this ([figure 26.3](#)):

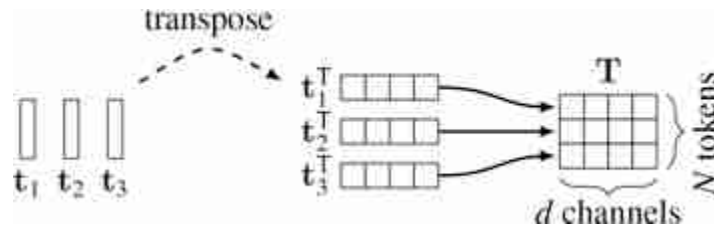


Figure 26.3: In this chapter, we will represent a set of tokens as a matrix whose rows are the token vectors.

The idea of this notation is that *tokens are to transformers as neurons are to neural nets*. Neural net layers operate over arrays of neurons; for example, an MLP takes as input a column vector $\mathbf{x}$, whose rows are scalar neurons. Transformers operate over arrays of tokens. A matrix $\mathbf{T}$ is just a convenient representation of 1D array of vector-value tokens.

Although we are only considering vector-valued tokens in this chapter, it's easy to imagine tokens that are any kind of structured group. We just need to define how basic operators, like summation, operate over these groups (and, ideally, in a differentiable manner).

Transformers consist of two main operations over tokens: (1) *mixing* tokens via a weighted sum, and (2) *modifying* each individual token via a nonlinear transformation. These operations are analogous to the two workhorses of regular neural nets: the linear layer and the pointwise nonlinearity.

26.4.3 Mixing Tokens

Once we have converted our data to tokens, we now need to define operations for transforming these tokens and eventually making decisions

based on them. The first operation we will define is how to take a *linear combination of tokens*.

A linear combination of tokens is not the same as a fully connected layer in a neural net. Instead of taking a weighted sum of scalar neurons, it takes a weighted sum of vector-valued tokens ([figure 26.4](#)):

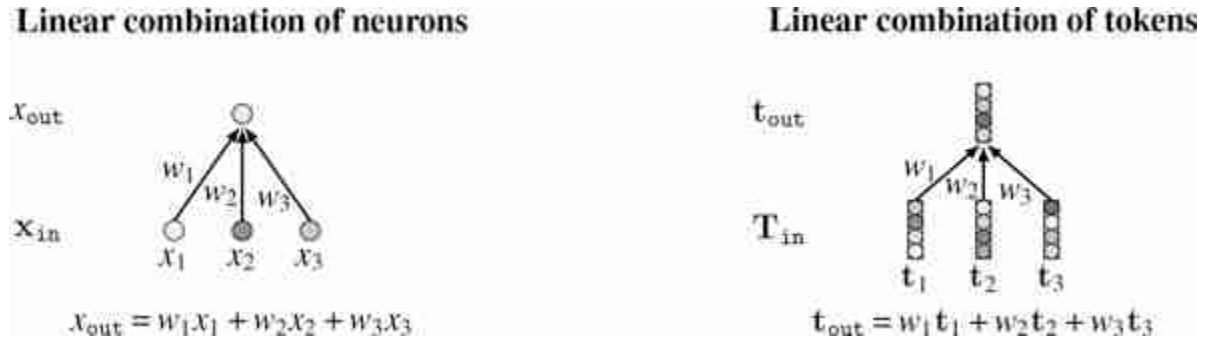


Figure 26.4: Linear combination of neurons versus tokens.

Notation: we use T_{out} when the output is multiple tokens and t_{out} when it is a single token.

The general form of these equations for multiple input and output neurons/tokens is:

$$x_{out}[i] = \sum_{j=1}^N w_{ij} x_{in}[j] \quad (26.2)$$

$$X_{out} = W X_{in} \quad \Leftarrow \text{linear combination of neurons} \quad (26.3)$$

$$T_{out}[i, :] = \sum_{j=1}^N w_{ij} T_{in}[j, :] \quad (26.4)$$

$$T_{out} = W T_{in} \quad \Leftarrow \text{linear combination of tokens} \quad (26.5)$$

As can be seen above, operations over tokens can be defined just like operations over neurons except that the tokens are vector-valued while the neurons are scalar-valued. Most layers we have encountered in previous chapters can be defined for tokens in an analogous way to how they were defined for neurons.

For example, we can define a fully connected layer (fc layer) over tokens as a mapping from N_1 input tokens to N_2 output tokens, parameterized by a

matrix $\mathbf{W} \in \mathbb{R}^{N_2 \times N_1}$ (and, optionally, by a set of token biases $\mathbf{b} \in \mathbb{R}^{N_2 \times d}$):

$$\mathbf{T}_{\text{out}} = \mathbf{W}\mathbf{T}_{\text{in}} + \mathbf{b} \quad \triangleleft \text{fc layer over tokens} \quad (26.6)$$

Our notation here, which represents a set of tokens as a matrix $\mathbf{T}$, transforms working with tokens into an exercise in matrix algebra. However, this notation is also somewhat limiting, as it only applies to *vector-valued tokens*. What if we want tokens that are tensor-valued, or tokens whose codes are elements of an abstract group such as $\text{SO}(3)$? There is not yet standard notation for working with tokens like this. As you read this chapter, try to think about how the operations we define for standard vector-valued tokens could be instead defined for other kinds of tokens.

26.4.4 Modifying Tokens

Linear combinations only let us linearly mix and recombine tokens, and stacking linear functions can only result in another linear function. In standard neural nets, we ran into the same problem with fully-connected and convolutional layers, which, on their own, are incapable of modeling nonlinear functions. To get around this limitation, we added *pointwise nonlinearities* to our neural nets. These are functions that apply a nonlinear transformation to each neuron *individually*, independently from all other neurons. Analogously, for networks of tokens we will introduce *tokenwise* operators; these are functions that apply a nonlinear transformation to each *token* individually, independently from all other tokens. Given a nonlinear function $F_\theta : \mathbb{R}^N \rightarrow \mathbb{R}^N$, a tokenwise nonlinearity layer, taking input $\mathbf{T}_{\text{in}}$, can be expressed as:

$$\mathbf{T}_{\text{out}} = \begin{bmatrix} F_\theta(\mathbf{T}_{\text{in}}[0,:]) \\ \vdots \\ F_\theta(\mathbf{T}_{\text{in}}[N-1,:]) \end{bmatrix} \quad \triangleleft \text{per-token nonlinearity} \quad (26.7)$$

Notice that this operation is generalization of the pointwise nonlinearity in regular neural nets; a `relu` layer is the special case where $F_\theta = \text{relu}$ and the layer operates over a set of neuron inputs (scalars) rather than token inputs (vectors):

$$\mathbf{x}_{\text{out}} = \begin{bmatrix} \text{relu}(x_{\text{in}}[0]) \\ \vdots \\ \text{relu}(x_{\text{in}}[N-1]) \end{bmatrix} \quad \triangleleft \text{per-neuron nonlinearity (relu)} \quad (26.8)$$

The F_θ may be any nonlinear function but some choices will work better than others. One popular choice is for F_θ to be a multilayer perceptron

(MLP); see chapter 12. In this case, F_θ has learnable parameters θ , which are the weights and biases of the MLP. This reveals an important difference between pointwise operations in regular neural nets and in token nets: `relu`, and most other neuronwise nonlinearities, have no learnable parameters, whereas F_θ typically does. This is one of the interesting things about working with tokens, the pointwise operations become expressive and parameter-rich.

26.5 Token Nets

We will use the term **token nets** to refer to computation graphs that use tokens as the primary nodes, rather than neurons.

Note that the terminology in this chapter is not standard. The term *token nets*, and some of the definitions we have given, are our own invention.

Token nets are just like neural nets, alternating between layers that mix nodes in linear combinations (e.g., fully connected linear layers, convolutional layers, etc.) and layers that apply a pointwise nonlinearity to each node (e.g., `relu`, per-token MLPs). Of course, since tokens are simply groups of neurons, every token net is itself also a neural net, just viewed differently—it is a net of subnets. In [figure 26.5](#), we show a standard neural net and a token net side by side, to emphasize the similarities in their operations.

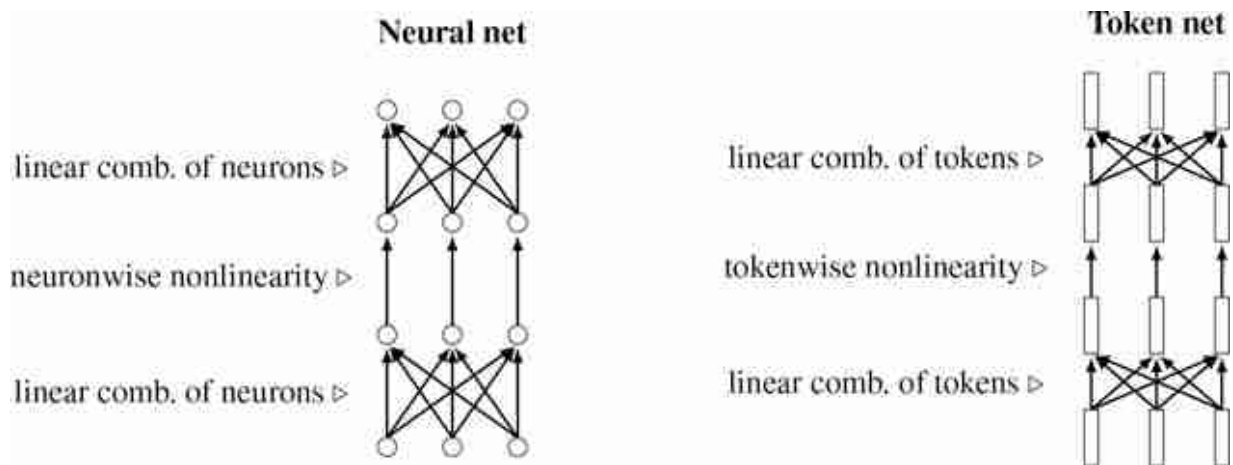


Figure 26.5: Neural nets versus token nets. The arrows here represent any functional dependency between the nodes (note that different arrows represent different types of functions).

26.6 The Attention Layer

Attention layers define a special kind of linear combination of tokens. Rather than parameterizing the linear combination with a matrix of free parameters $\mathbf{W}$, attention layers use a different matrix, which we call the attention matrix $\mathbf{A}$. The important difference between $\mathbf{A}$ and $\mathbf{W}$ is that $\mathbf{A}$ is *data-dependent*, that is, the values of $\mathbf{A}$ are a function the data input to the network. In addition, $\mathbf{A}$ typically only contains non-negative values, consistent with thinking of it as a matrix that allocates how much (non-negative) attention we pay to each input token. In the diagram below ([figure 26.6](#)), we indicate the data-dependency with the function labeled f , and we color the attention matrix red to indicate that it is constructed from *transformed data* rather than being free parameters (for which we use the color blue):

Here, we describe attention as fc layers with data-dependent weights. We could have instead described attention as a kind of **dynamic pooling**, which is mean pooling but using a weighted average where the weights are dynamically decided based on the input data.

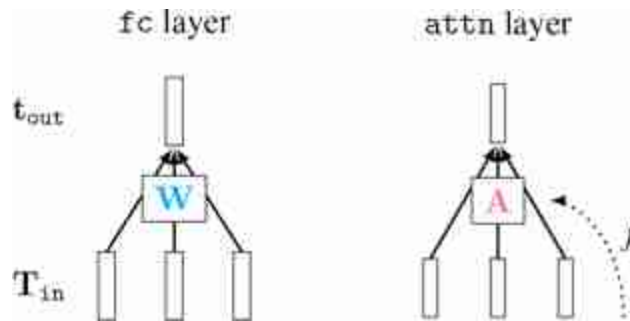


Figure 26.6: Fully-connected layers versus attention layers.

The equation for an attention layer is the same as for a linear layer except that the weights are a function of some other data (left unspecified for now but we will see concrete examples subsequently):

$$A = f(\dots) \quad \Leftarrow \text{attention} \quad (26.9)$$

$$T_{out} = AT_{in} \quad (26.10)$$

The key question, of course, is what exactly is f ? What inputs does f depend on and what is f 's mathematical form? Before writing out the exact equations, we will start with the intuition: f is a function that determines how much attention to apply to each token in T_{in} ; because this layer is just a weighted combination of tokens, f is simply determining the weights in this combination. The f can depend on any number of input signals that tell the net what to pay attention to.

As a concrete example, consider that we want to be able to ask questions about different objects in our safari example image, such as how many animals are in the photo. Then one strategy would be to attend to each token that represents an animal's head, and then just count them up. The f would take as input the text query, and would produce as output weights A that are high for the T_{in} tokens that correspond to any animal's head and are low for all other T_{in} tokens. If we train such a system to answer questions about counting animals, then the token code vectors might naturally end up encoding a feature that represents the number of animal heads in their receptive field; after all, this would be a solution that would solve our problem (it would minimize the loss and correctly answer the question). Other solutions might be possible, but we will focus on this intuitive solution, which we illustrate in [figure 26.7](#).

What's neat here is that attention gives us a way to make the layer dynamically change its behavior in response to different input questions; asking different questions results in different answers, as is visualized below in [figure 26.7](#).

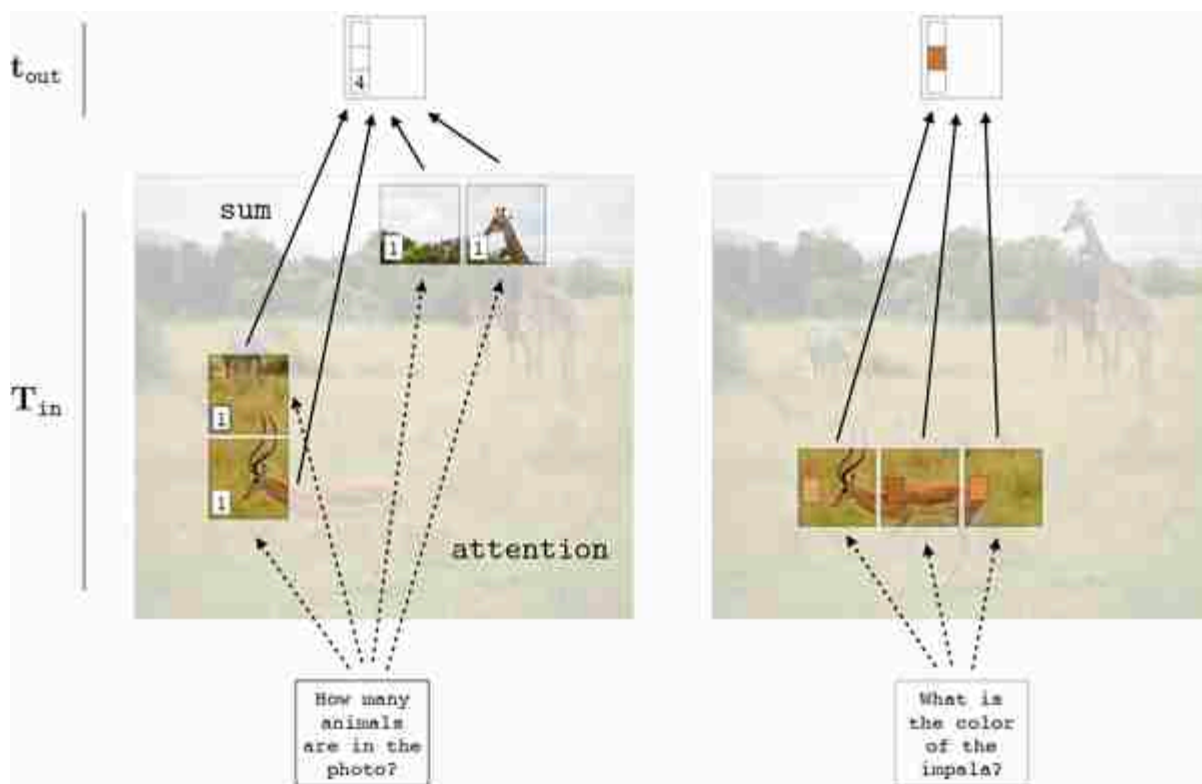


Figure 26.7: How attention can be allocated across different regions (tokens) in an image. The token code vectors consist of multiple dimensions and each can encode a different attribute of the token. To the left we show a dimension that encodes number of animal heads. To the right we show a different dimension that encodes color (or this could be three dimensions, coding RGB). The output token is a weighted sum over all the tokens attended to.

Let's walk through the logic of [figure 26.7](#). Here we are imagining a token representation that can answer two different kinds of questions, one about number and the other about color. The representation we have come up with (which learning could have arrived at) is to encode in one dimension of the token vector a constant of value 1, which will be used for counting up the number of attended tokens. In another set of dimensions we have the average RGB color of the patch the token represents. Note that tokens only directly represent image patches at the input to the network,

right after the tokenization step; at deeper layers of the network, the tokens may be more abstract in what they represent. Each text query elicits a different allocation of attention, and we will get to exactly how that process works later. For now just consider that the text query assigns a scalar weight to each token depending on how well that token's content matches the query's content. The output token, $\mathbf{t}_{\text{out}}$, is the sum of all the tokens weighted by the attention scalars. This scheme will arrive at a reasonable answer to the questions if the text query “How many animals are in this photo” gives attention weight 1 to just the tokens representing animal heads and the text query “What is the color of the impala” gives weight $\frac{1}{3}$ just to the impala tokens. Then the output vector in the former case contains the correct answer 4 in the dimension that represents number of attended tokens, and contains the RGB values for brownish in the dimensions that represent average patch color.

Keeping this intuitive picture in mind, we will now turn to the equations that define the attention allocation function f . We will focus on the particular version of f that appears in transformers, which is called **query-key-value attention**.

26.6.1 Query-Key-Value Attention

Transformers use a particular kind of attention based on the idea of queries, keys, and values.

The idea of queries, keys, and values comes from databases, where a database cell holds a *value*, which is retrieved when a *query* matches the cell's *key*. Tokens are like database cells and attention is like retrieving information from the database of tokens.

In query-key-value attention, each token is associated with a **query** vector, a **key** vector, and a **value** vector.

We define these vectors as linear transformations of the token's code vector, projecting to query/key/value vectors of length m . For a token $\mathbf{t}$, we have:

$$\mathbf{q} = \mathbf{W}_q \mathbf{t} \quad \triangleleft \text{query} \quad (26.11)$$

$$\mathbf{k} = \mathbf{W}_k \mathbf{t} \quad \triangleleft \text{key} \quad (26.12)$$

$$\mathbf{v} = \mathbf{W}_v \mathbf{t} \quad \triangleleft \text{value} \quad (26.13)$$

Here is a question to think about: Could you use other differentiable functions to compute the query, value, and key? Would that be useful?

In transformers, all inputs to the net are tokenized, so the textual question “How many animals are in the photo?” will also be represented as a token.

We do not cover them in this book, but methods from natural language processing can be used to transform text into a token, or into a sequence of tokens.

This token will submit its query vector, $\mathbf{q}_{\text{question}}$ to be matched against the keys of the tokens that represent different patches in the image; the similarity between the query and the key determines the amount of attention weight the query will apply to the token with that key. The most common measure of similarity between a query $\mathbf{q}$ and a key $\mathbf{k}$ is the dot product $\mathbf{q}^T \mathbf{k}$. Querying each token in $\mathbf{T}_{\text{in}}$ in this way gives us a vector of similarities:

$$\mathbf{s} = [s_1, \dots, s_N]^T = [\mathbf{q}_{\text{question}}^T \mathbf{k}_1, \dots, \mathbf{q}_{\text{question}}^T \mathbf{k}_N]^T \quad (26.14)$$

We then normalize the vector $\mathbf{s}$ using the softmax function to give us our attention weights $\mathbf{a} \in \mathbb{R}^{N \times 1}$, and finally, rather than applying $\mathbf{a}$ over token codes directly (i.e., taking a weighted sum over tokens), we take a weighted sum over token value vectors, to obtain $\mathbf{T}_{\text{out}}$:

$$\mathbf{a} = \text{softmax}(\mathbf{s}) \quad (26.15)$$

$$\mathbf{T}_{\text{out}} = \begin{bmatrix} a_1 \mathbf{v}_1^T \\ \vdots \\ a_N \mathbf{v}_N^T \end{bmatrix} \quad (26.16)$$

$\mathbf{v}_1$ is the value vector for $\mathbf{t}_1 = \mathbf{T}_{\text{in}}[0, :]$, and so forth.

[Figure 26.8](#) visualizes these steps.

We use the following color scheme here and later in this chapter:

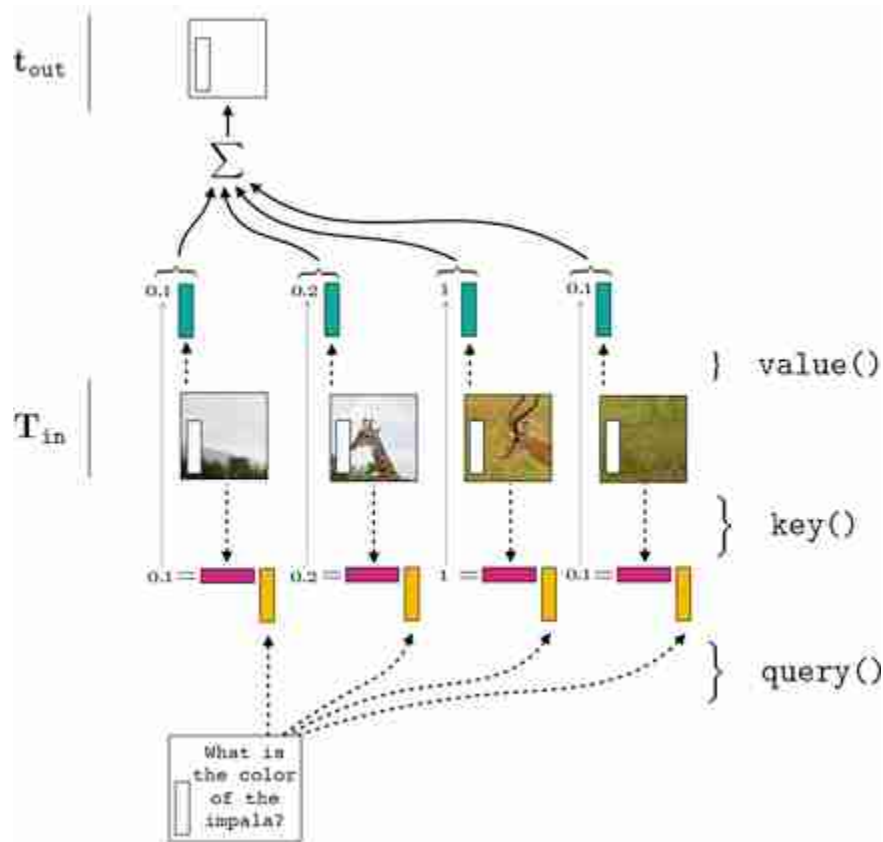


Figure 26.8: Mechanics of an attention layer. Queries from the question match keys from the tokens representing the impala; value vectors of the impala tokens then contribute the most to the sum that yields t_{out} 's code vector. (Softmax omitted in this example.)

26.6.2 Self-Attention

As we have now seen, attention is a general-purpose way of dynamically pooling information in one set of tokens based on queries from a different set of tokens. The next question we will consider is which tokens should be doing the querying and which should we be matching against? In the example from the last section, the answer was intuitive because we had a textual question that was asking about content in a visual image, so naturally the text gives the query and we match against tokens that

represent the image. But can we come up with a more generic architecture where we don't have to hand design which tokens interact in which ways?

Self-attention is just such an architecture. The idea is that on a self-attention layer, *all* tokens submit queries, and for each of these queries, we take a weighted sum over *all* tokens in that layer. If $\mathbf{T}_{\text{in}}$ is a set of N input tokens, then we have N queries, N weighted sums, and N output tokens to form $\mathbf{T}_{\text{out}}$. This is visualized below in [figure 26.9](#).

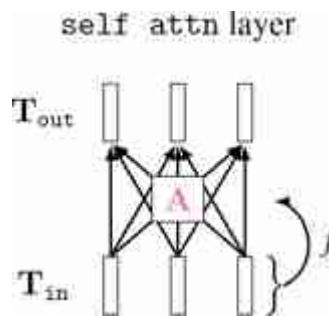


Figure 26.9: A self-attention layer.

To compute the query, key, and value for a set of input tokens, $\mathbf{T}_{\text{in}}$, we apply the same linear transformations to each token in the set, resulting in matrices $\mathbf{Q}_{\text{in}}, \mathbf{K}_{\text{in}} \in \mathbb{R}^{N \times m}$ and $\mathbf{V}_{\text{in}} \in \mathbb{R}^{N \times d}$, where each row is the query/key/value for each token:

Note that the query and key vectors must have the same dimensionality, m , because we take a dot product between them. Conversely, the value vectors must match the dimensionality of the token code vectors, d , because these are summed up to produce the new token code vectors.

$$\mathbf{Q}_{in} = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_q \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_q \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{in} \mathbf{W}_q^\top \quad \triangleleft \quad \text{query matrix} \quad (26.17)$$

$$\mathbf{K}_{in} = \begin{bmatrix} \mathbf{k}_1^\top \\ \vdots \\ \mathbf{k}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_k \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_k \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{in} \mathbf{W}_k^\top \quad \triangleleft \quad \text{key matrix} \quad (26.18)$$

$$\mathbf{V}_{in} = \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_N^\top \end{bmatrix} = \begin{bmatrix} (\mathbf{W}_v \mathbf{t}_1)^\top \\ \vdots \\ (\mathbf{W}_v \mathbf{t}_N)^\top \end{bmatrix} = \mathbf{T}_{in} \mathbf{W}_v^\top \quad \triangleleft \quad \text{value matrix} \quad (26.19)$$

Finally, we have the attention equation:

$$\mathbf{A} = f(\mathbf{T}_{in}) = \text{softmax}\left(\frac{\mathbf{Q}_{in} \mathbf{K}_{in}^\top}{\sqrt{m}}\right) \quad \triangleleft \quad \text{attention matrix} \quad (26.20)$$

$$\mathbf{T}_{out} = \mathbf{A} \mathbf{V}_{in} \quad (26.21)$$

where the softmax is taken within each row (i.e., over the vector of matches for each separate query vector, like in equation (26.14)). In expanded detail, here are the full mechanics of a self-attention layer ([figure 26.10](#)):

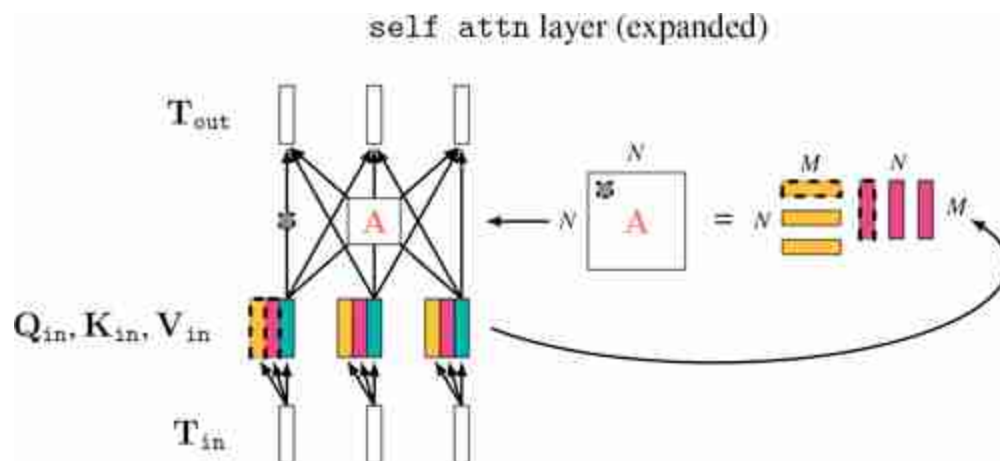


Figure 26.10: Self-attention layer expanded. The nodes with the dashed outline correspond to each other; they represent one query being matched against one key to result in a scalar similarity value, in the gray box, which acts as a weight in the weighted sum computed by $\mathbf{A}$.

This fully defines a self-attention layer, which is the kind of attention layer used in transformers. Before we move on though, let's think through the intuition of what self-attention might be doing.

Consider that we are processing the safari image, and our task is semantic segmentation (label each patch with an object class). [Figure 26.11](#) illustrates this scenario. We start by tokenizing the image so that each patch is represented by a token. Now we have a token, $\mathbf{t}_2$, that represents the patch of pixels around the torso of the impala. We wish to update this token via one layer of self-attention. Since the goal of the network is to classify patches, it would make sense to update $\mathbf{t}_2$ to get a better semantic representation of what's going on in that patch. One way to do this would be to attend to the tokens representing other patches of the impala, and use them to refine $\mathbf{t}_2$ into a more abstracted token vector, capturing the label *impala*. The intuition is that it's easier to recognize a patch given the context of other relevant patches around it. The refinement operation is just to sum over the token code vectors, which has the effect of reducing noise that is not shared between the three attended impala patches, which amplifies the commonality between them – the label *impala*. More sophisticated refinements could be achieved via multiple layers of self-attention. Further, the impala patch query could also retrieve information from the giraffe and zebra patches, as those patches provide additional context that could be informative (the animal in the query is more likely to be an impala if it is found near giraffes and zebras, since all those animals tend to congregate together in the same biome).

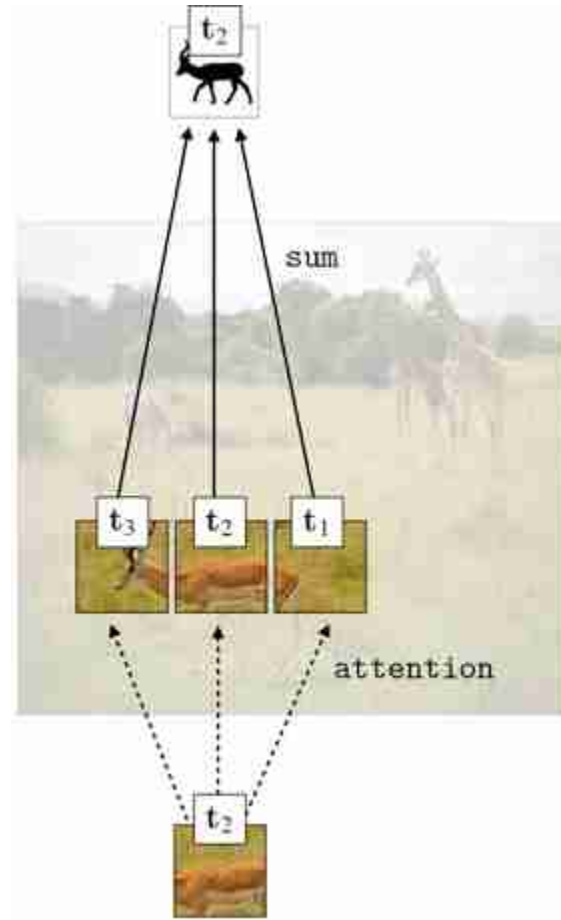


Figure 26.11: One way self-attention could be used to aggregate information across all patches containing the same object, and thereby arrive at a better representation of the object in t_2 , the query patch.

This is just one way self-attention could be used by the network. How it is actually used will be determined by the training data and task. What really happens might deviate from our intuitive story: tokens on hidden layers do not necessarily represent spatially localized patches of pixels. While the initial tokenization layer creates tokens out of local image patches, after this point attention layers can mix information across spatially distant tokens; note that $\mathbf{T}_{\text{out}}[0, :]$ does not necessarily represent the same spatial region in the image as $\mathbf{T}_{\text{in}}[0, :]$.

[Figure 26.12](#) gives an example of what self-attention maps can look like on the safari image. In this example, we are simply using patch color as the query and key features. Each attention map shows one row of $\mathbf{A}$ reshaped into the size of the input image.



Figure 26.12: Example of self-attention maps where each token is an image patch and the query and key vectors are both set to the mean color of the patch, normalized to be a unit vector.

26.6.3 Multihead Self-Attention

Despite their power, self-attention layers are still limited in that they only have one set of query/key/value projection matrices (namely, $\mathbf{W}_q$, $\mathbf{W}_k$, $\mathbf{W}_v$). These matrices define the notion of similarity that is used to match queries to keys. In particular, the similarity between two tokens i and j is measured as:

$$s_{ij} = \mathbf{q}_i^T \mathbf{k}_j \quad (26.22)$$

$$= (\mathbf{W}_q \mathbf{t}_i)^T \mathbf{W}_k \mathbf{t}_j \quad (26.23)$$

$$= \mathbf{t}_i^T \mathbf{W}_q^T \mathbf{W}_k \mathbf{t}_j \quad (26.24)$$

$$= \mathbf{t}_i^T \mathbf{S} \mathbf{t}_j \quad (26.25)$$

What this shows is that $\mathbf{W}_q$ and $\mathbf{W}_k$ define some matrix $\mathbf{S} = \mathbf{W}_q^T \mathbf{W}_k$ that modulates how we measure similarity (dot product) between $\mathbf{t}_i$ and $\mathbf{t}_j$. A single self-attention layer therefore measures similarity in just one way.

What if we want to measure similarity in more than one way? For example, maybe we want our net to perform some set of computations based on color similarity, another based on texture similarity, and yet another based on shape similarity? The way transformers can do this is with **multihead self-attention (MSA)**. This method simply consists of running k attention layers in parallel. All these layers are applied to the same input $\mathbf{T}_{in}$. This results in k output sets of tokens, $\mathbf{T}_{out}^1, \dots, \mathbf{T}_{out}^k$. To merge these outputs, we concatenate all of them and project back to the original dimensionality of $\mathbf{T}_{in}$. These steps are shown in the math below:

$$\mathbf{T}_{\text{out}}^i = \text{attn}^i(\mathbf{T}_{\text{in}}) \quad \text{for } i \in \{1, \dots, k\} \quad (26.26)$$

$$\mathbf{T}_{\text{out}} = \begin{bmatrix} \mathbf{T}_{\text{out}}^1[0, :] & \dots & \mathbf{T}_{\text{out}}^k[0, :] \\ \vdots & & \vdots \\ \mathbf{T}_{\text{out}}^1[N-1, :] & \dots & \mathbf{T}_{\text{out}}^k[N-1, :] \end{bmatrix} \quad \triangleleft \quad \bar{\mathbf{T}}_{\text{out}} \in \mathbb{R}^{N \times kv} \quad (26.27)$$

$$\mathbf{T}_{\text{out}} = \bar{\mathbf{T}}_{\text{out}} \mathbf{W}_{\text{MSA}} \quad \triangleleft \quad \mathbf{W}_{\text{MSA}} \in \mathbb{R}^{kv \times d} \quad (26.28)$$

where v is the dimensionality of the value vectors and d is the dimensionality of the code vectors of the output ([109] recommends setting $kv = d$). The matrix $\mathbf{W}_{\text{MSA}}$ merges all the heads; its values are learnable parameters. The other learnable parameters of MSA are the query, key, and value projections for each of the k attention heads.

Notice that here, unlike in the single-headed self-attention layers presented previously, the value vectors need not have the same dimensionality as the token code vectors, since we are applying the projection equation (26.28).

The basic reasoning here is quite simple: if self-attention layers are a good thing, why not just add more of them? We can add more *sequential* self-attention layers by building deeper transformers, or we can add more *parallel* self-attention layers by using MSA.

26.7 The Full Transformer Architecture

A full transformer architecture is a stack of self-attention layers interleaved with tokenwise nonlinearities. These two steps are analogous to linear layers interleaved with neuronwise nonlinearities in an MLP, as shown below ([figure 26.13](#)):

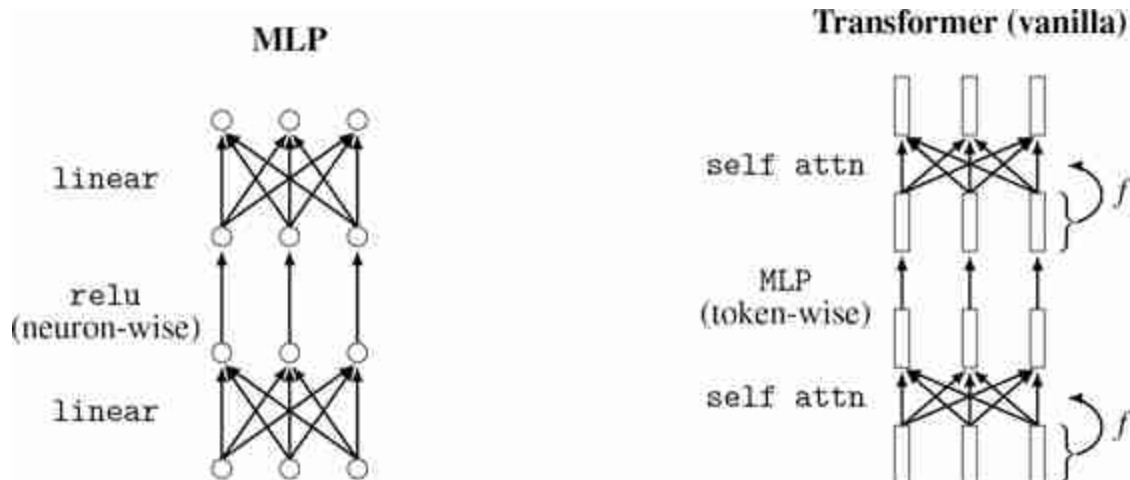


Figure 26.13: The basic transformer architecture versus an MLP.

Beyond this basic template, there are many variations that can be added, resulting in different particular architectures within the transformer family. Some common additions are normalization layers and residual connections. In [figure 26.14](#) we plot the ViT architecture from [\[109\]](#), showing where these additional pieces enter the picture.

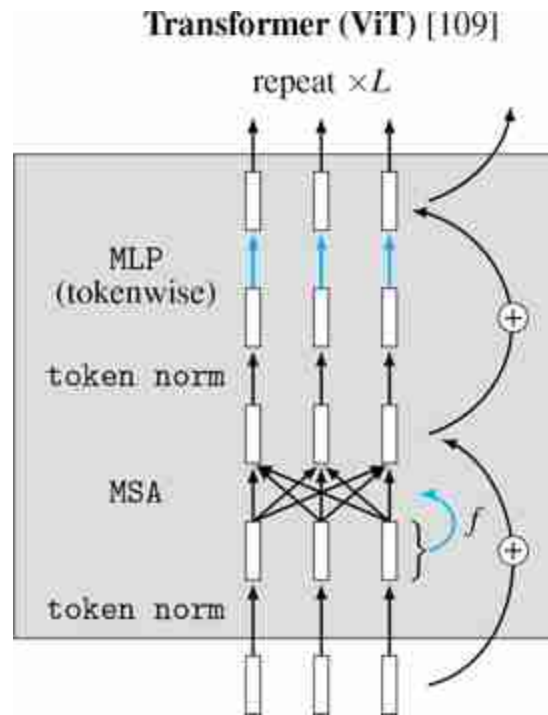


Figure 26.14: The ViT transformer architecture [109]. This set of layers forms a computational block, shaded in gray, that can be repeated L times for a depth L ViT. To clarify where the parameters live in this architecture, we have colored all the edges with learnable parameters in blue (note that the MSA merge, equation (26.28), is also learnable but not explicitly shown in this diagram).

This architecture uses layer normalization (section 12.7.3) before each attention layer and before each token-wise MLP layer. The normalization is done *within* each token (the token code vector is treated as a akin to a layer; each dimension of this vector is standardized by the mean and variance over all dimensions of this vector), so in figure 12.7.3 we refer to this layer as token norm. Notice that token norm is a tokenwise operation, just like our tokenwise MLP, but it performs a different kind of transformation and does not have learnable parameters. Residual connections are added around each group of layers.

Pseudocode for this a ViT (with single-headed attention) is given below:

```

# x : input data (RGB image)
# K : tokenisation patch size
# d : token/query/key/value dimensionality (setting these all as the same)
# L : number of layers
# W_q_T, W_k_T, W_v_T : transposed query/key/value projection matrices
# mlp: tokenwise mlps

# tokenize input image
T = tokenize(x,K) # 3 x H x W image --> N x d array of token code vectors

# run tokens through all L layers
for l in range(L):

    # attention layer
    Q, K, V = nn.matmul(nn.layer_norm(T), [W_q_T[l], W_k_T[l], W_v_T[l]])
    # nn.matmul does matrix multiplication
    A = nn.softmax(nn.matmul(Q,K.transpose()), dim=0)/sqrt(d)
    T = nn.matmul(A,V) + T # note residual connection

    # tokenwise mlp
    T = mlp[l](nn.layer_norm(T)) + T # note residual connection

# T now contains the output token representation computed by the transformer

```

The output of a transformer, as we have so far defined it, is a set of tokens $\mathbf{T}_{\text{out}}$. Often we want an output of a different format, such as a single vector of logits for image classification (section 9.7.3), or in the format of an image for image-to-image tasks (section 34.6). To handle these cases, we typically define a task-specific output layer that takes $\mathbf{T}_{\text{out}}$ as input and produces the desired format as output. For example, to produce a vector of logit predictions we could first sum all the token code vectors in $\mathbf{T}_{\text{out}}$ and then, using a single linear layer, project the resulting d -dimensional vector into a K -dimensional vector (for K -way classification).

26.8 Permutation Equivariance

An important property of transformers is that they are equivariant to permutations of the input token sequence. This follows from the fact that both tokenwise layers, F_θ , and attention layers, attn , are **permutation equivariant**:

$$F_\theta(\text{permute}(\mathbf{T}_{\text{in}})) = \text{permute}(F_\theta(\mathbf{T}_{\text{in}})) \quad (26.29)$$

$$\text{attn}(\text{permute}(\mathbf{T}_{\text{in}})) = \text{permute}(\text{attn}(\mathbf{T}_{\text{in}})) \quad (26.30)$$

where permute is a permutation of the order of tokens in $\mathbf{T}_{\text{in}}$ (i.e., permutes the rows of the matrix). This means that if you scramble (i.e. permute) the patches in the input image then apply attention, the output will

be unchanged up to a permutation of the original output. Since the full transformer architecture is just composition of these two types of layers (plus, potentially, residual connections and token normalization, which are also permutation equivariant), and because composing two permutation equivariant functions results in a permutation equivariant operation, we have:

$$\text{transformer}(\text{permute}(\mathbf{T}_{\text{in}})) = \text{permute}(\text{transformer}(\mathbf{T}_{\text{in}})) \quad (26.31)$$

This property is visualized in [figure 26.15](#).

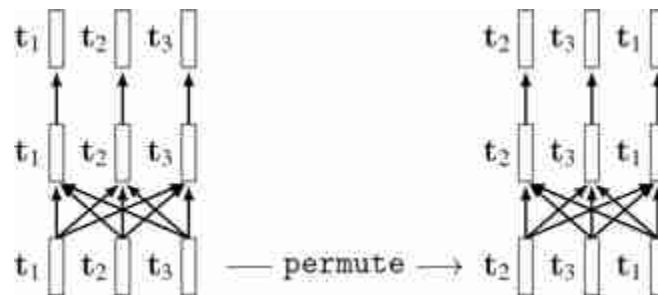


Figure 26.15: Transformers are permutation equivariant. For notational simplicity, we omit layer indices on the token variables here.

It is often useful to understand layers in terms of their invariances and equivariances. Convolutional layers are translation equivariant but not necessarily permutation equivariant whereas attention layers are both translation equivariant *and* permutation equivariant (since translation is a special kind of permutation, any permutation equivariant layer is also translation equivariant). Other layers can be catalogued similarly: global average pooling layers are permutation *invariant*, relu layers are permutation equivariant, per-token MLP layers are also permutation equivariant (but with respect to sets of tokens rather than sets of neurons), and so on.

A generally good strategy is to select layers that reflect the symmetries in your data or task: in object detection, translation equivariance makes sense because, roughly, a bird is a bird no matter where it appears in an image. Permutation equivariance might also make sense, for that same reason, but only to an extent: if you break up an image into small patches and scramble them, this could disrupt spatial layout that is important for recognition. We

will see in section 26.11 how transformers use something called positional codes to reinsert useful information about spatial layout.

26.9 CNNs in Disguise

Transformers provide a new way of thinking about data processing, and it may seem like they are very different from past architectures. However, as we have alluded to, they actually have many commonalities with CNNs. In fact, most (but not all) of the transformer architecture can be viewed as a CNN in disguise. In this section we will walk through several of the layers we learned about above, and see how they are in fact performing convolutions.

26.9.1 Tokenization

The first step in working with transformers is to tokenize the input. The most basic way to do this is to chop up the input image into non-overlapping patches of size $K \times K$, then convert these patches to vectors via a linear projection. You might already have noticed that this operation can be written as convolution; after all we said the whole idea of CNNs is to chop the signal into patches. In particular, this form of tokenization can be written as a convolutional layer with kernel size and stride both equal to K :

$$\mathbf{T}[n(N/K) + m, c_2] = b[c_2] + \sum_{c_1=1}^{C_{in}} \sum_{k_1, k_2=-K}^K w[c_1, c_2, k_1, k_2] x_{in}[c_1, Kn - k_1, Km - k_2] \quad \triangleq \quad \text{(tokenization)} \quad (26.32)$$

where, for RGB images, $\mathbf{x}_{in} \in \mathbb{R}^{3 \times N \times M}$, $C_{in} = 3$, and $C_{out} = d$ (the token dimensionality). This math assumes N and M are evenly divisible by K ; if they aren't then the input can be resized or padded until they are.

Although the equation starts to look complicated, it is just a `conv` operator with the following parameters:

```
T = conv(x_in, channels_in=3, channels_out=d, kernel=K, stride=K) # tokenize
```

26.9.2 Query-Key-Value Projections

Next let's look at the query, key, and value projections that are part of the attention layers. For simplicity, we will consider just the query projection, since key and value follow exactly the same pattern.

We wrote this operation as a matrix multiply $\mathbf{T}_{\text{in}} \mathbf{W}_q^\top$ (equation (26.17)). What this multiply is doing is applying the same linear transformation ($\mathbf{W}_q$) to each token vector (each row of $\mathbf{T}_{\text{in}}$). Applying the same linear operation to each element in a sequence is exactly what convolution does. Specifically, the query operation can be written as convolving the set of N d -channel tokens with a filter bank of m filters, with kernel size 1, producing a new set of N m -channel tokens. This equivalence is visualized below ([figure 26.16](#)):

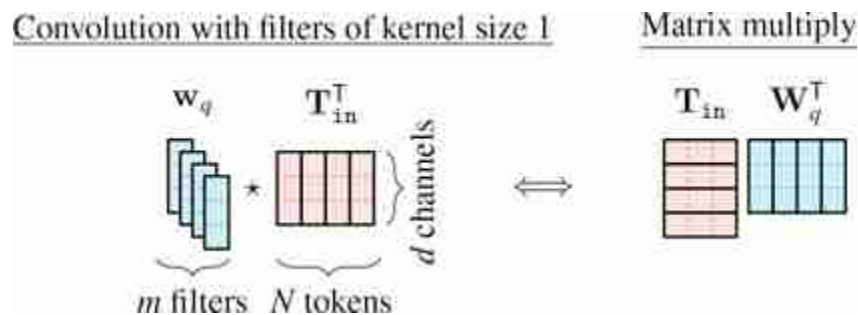


Figure 26.16: The query, key, and value projections in transformers can be written either as a convolution or a matrix multiply.

Therefore, the query, key, and value projections are all multichannel convolutions with kernels of size 1.

Convolution actually appears all over in linear algebra, and in fact *every matrix product can be written as a convolution!* Whenever you see a product $\mathbf{AB}$, you can think of it as the convolution of a multichannel filter bank $\mathbf{B}$ (one filter in each row; kernel size 1) with the signal $\mathbf{A}$ (time indexes rows, channels in the columns).

26.9.3 Tokenwise MLP

Next we will consider the token-wise MLP layer. A token-wise MLP applies the same MLP F_θ to each token in a sequence. The F_θ consists of linear layers and pointwise nonlinearities. For simplicity, we will assume no biases (as an exercise, this can be relaxed). The linear layers in F_θ all have the following form:

$$\mathbf{t}_{\text{out}} = \mathbf{W} \mathbf{t}_{\text{in}} \quad (26.33)$$

When we apply such a layer to each token in the sequence, we have:

$$\mathbf{T}_{\text{out}} = \mathbf{T}_{\text{in}} \mathbf{W}^T \quad (26.34)$$

Notice that this looks just like the query operation we covered in the previous section, equation (26.17). Therefore, the same result holds: the linear layers of the token-wise MLP can all be written as convolutions with kernel size 1.

Now the pointwise nonlinearities in the MLP are applied neuronwise, so these layers function identically to the pointwise nonlinearity in CNNs. This is the full set of layers in the MLP, and therefore we have that a token-wise MLP can be written as a series of convolutions interleaved with neuronwise-nonlinearities, i.e. a CNN.

26.9.4 The Similarities between CNNs and Transformers

As we have now seen, most layers in transformers are convolutional. These layers break up the signal processing problem into chunks, then process each chunk independently and identically.

Breaking up into chunks is such a fundamentally useful idea that it shows up in many different fields under different names. One general name for it is **factorizing** a problem into smaller pieces.

Some of the other operations in transformers – normalization layers, residual connections, etc – are also common in CNNs. So what is *different* between transformers and CNNs?

26.9.5 The Differences between CNNs and Transformers

26.9.5.1 CNNs can have kernels with non-unitary spatial extent

When we wrote them as convolutions, the query-key-value projections and token-wise MLPs *only used 1x1 filters*.

We use the term “1x1 filter” to refer to any filter whose kernel size is 1 in all its dimensions, regardless of whether the signal is one-dimensional, two-dimensional, three-dimensional, etc.

In fact it cannot be otherwise. If you used larger kernel it would break the permutation invariance property of transformers, since the output of the filters would depend on which token is next to which. This is one of the key differences between CNNs and transformers. CNNs use $K \times K$ filters, and

this makes it so adjacent image regions get processed together. Transformers use 1×1 filters which means the network has no architectural way of knowing about spatial structure (which token is next to which). For vision problems, where spatial structure is often crucially important, transformers can instead be given knowledge of position through the *inputs* to the network, rather than through the architectural structure. We will cover this idea in section 26.11.

26.9.5.2 Transformers have attention layers

Attention layers are *not* convolutional. They do not factor the processing into independent chunks but instead perform a global operation, in which all input tokens can interact. The linear combination of tokens that results is not a mixing operation found in CNNs and addresses the limitation of CNNs being myopic, with each filter only seeing information in its receptive field.

26.10 Masked Attention

Sometimes we want to restrict which tokens can attend to which. This can be done by *masking* the attention matrix, which just means fixing some of the weights to be zero. This can be useful for many settings, including learning features via masked autoencoding [192], cross-attention between different data modalities [497], and for sequential prediction [78]. To illustrate, we will describe the sequential prediction use case.

For simplicity, in this section we depict tokens with one-dimensional code vectors, but remember $\mathbf{T}$ would have d columns for d -dimensional code vectors.

A common problem is to predict the $(n + 1)$ -th token in a sequence given the previous n tokens. For example, we may be trying to predict tokens that represent the next frame in a video, the next word in a sentence, or the weather on the next day.

A simple way to model this prediction problem is with a linear layer:
 $\mathbf{y}_{n+1} = \mathbf{A}\mathbf{T}_{1:n}$.

The $T_{1:n}$ is shorthand for the sequence of tokens $\begin{bmatrix} t_1 \\ \vdots \\ t_n \end{bmatrix}$.

Here is what this looks like diagrammatically, and on the right is the layer shown as matrix multiplication:

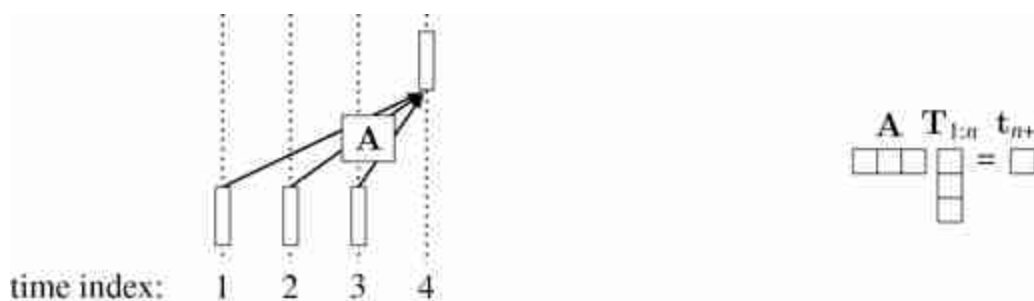


Figure 26.17: Masked prediction of time index 4 from time indices 1-3.

During training, we will give examples like this:

$$\{t_1, \dots, t_n\} \rightarrow t_{n+1} \quad (26.35)$$

$$\{t_1, \dots, t_{n-1}\} \rightarrow t_n \quad (26.36)$$

$$\{t_1, \dots, t_{n-2}\} \rightarrow t_{n-1} \quad (26.37)$$

and so on. We can make all these predictions at once with a single matrix multiply ([figure 26.18](#)):

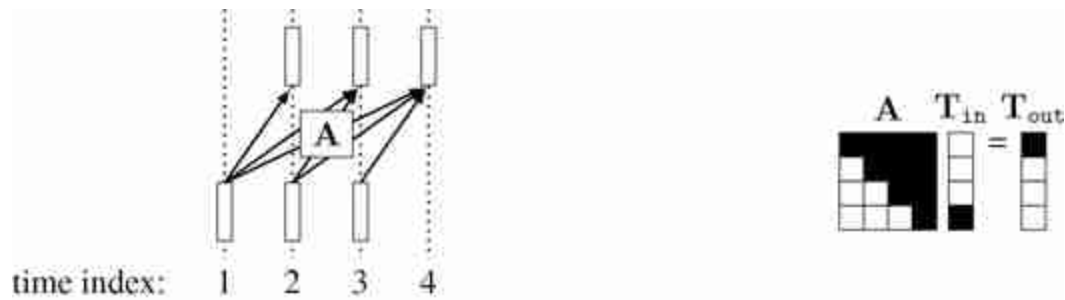


Figure 26.18: Masked attention to make multiple causal predictions at once. Black cells are masked; they are filled with zeros.

This way, one forward pass makes N predictions rather than one prediction. This is equivalent to doing a single next token prediction N times, but it all happens in a single matrix multiply, using the matrix shown on the right.

This kind of matrix is called *causal* because each output index i only depends on input indices j such that $j < i$. If $\mathbf{A}$ is an attention matrix, then this strategy is called **causal attention**. This is a masking strategy where each token can only attend to *previous* tokens in the sequence. This approach can dramatically speed up training because all the sub-sequence prediction problems (predict $\mathbf{t}_{n-1}$ given $\mathbf{T}_{1:n-2}$, predict $\mathbf{t}_n$ given $\mathbf{T}_{1:n-1}$, predict $\mathbf{t}_{n+1}$ given $\mathbf{T}_{1:n}$) are supervised at the same time.

This also works for transformers with more than one layer, where the masking strategy looks like shown in [figure 26.19](#).

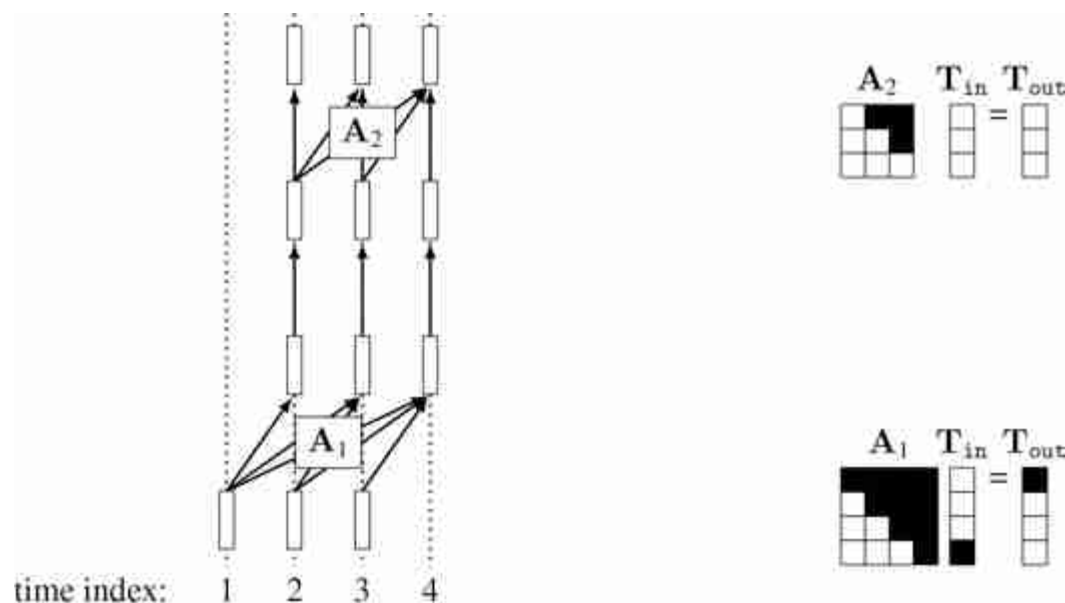


Figure 26.19: Multilayer masked attention achieves causal prediction with a deep net.

Notice that the output tokens on every layer l have the property that t_n^l only depends on $T_{1:n-1}^0$, where T^0 are the initial set of tokens input into the transformer. Also notice that, after the first layer, all subsequent layers can use causal attention that is not shifted in time, and the previous property is still maintained. Finally, notice that the subnetwork that predicts each subsequent output token overlaps substantially with the subnetworks that predict each previous token. That is, there is sharing of computation between all the prediction problems. We will see a more concrete application of this strategy when we get to autoregressive models in section 32.7.

26.11 Positional Encodings

Another idea associated with transformers is **positional encoding**. Operations over tokens in a transformer are permutation equivariant, which means that we can shuffle the positions of the tokens and nothing substantial changes (the only change is that the outputs get permuted). A consequence is that tokens do not know encode their position within the representation of the signal.

Note that masked attention layers are not permutation invariant because which tokens get masked depends on their ordering. Because of this, masked attention models do not necessarily need positional encodings in order to become sensitive to position [190].

Sometimes, however, we may wish to retain positional knowledge. For example, knowing that a token is a representation of the top region of an image can help us identify that the token is likely to represent sky. Positional encoding concatenates a code representing *position within the signal* onto each token. If the signal is an image, then the positional code should represent the x- and y-coordinates. However, it need not represent these coordinates as scalars; more commonly we use a periodic representation of position, where the coordinates are encoded as the vector of values a set of sinusoidal waves take on at each position:

$$\mathbf{p}_x = [\sin(x), \sin(x/B), \sin(x/B^2), \dots, \sin(x/B^P)]^T \quad (26.38)$$

$$\mathbf{p}_y = [\sin(y), \sin(y/B), \sin(y/B^2), \dots, \sin(y/B^P)]^T \quad (26.39)$$

$$\mathbf{p} = \begin{bmatrix} \mathbf{p}_x \\ \mathbf{p}_y \end{bmatrix} \quad (26.40)$$

where x and y are the coordinates of the token. This representation is visualized in [figure 26.20](#):

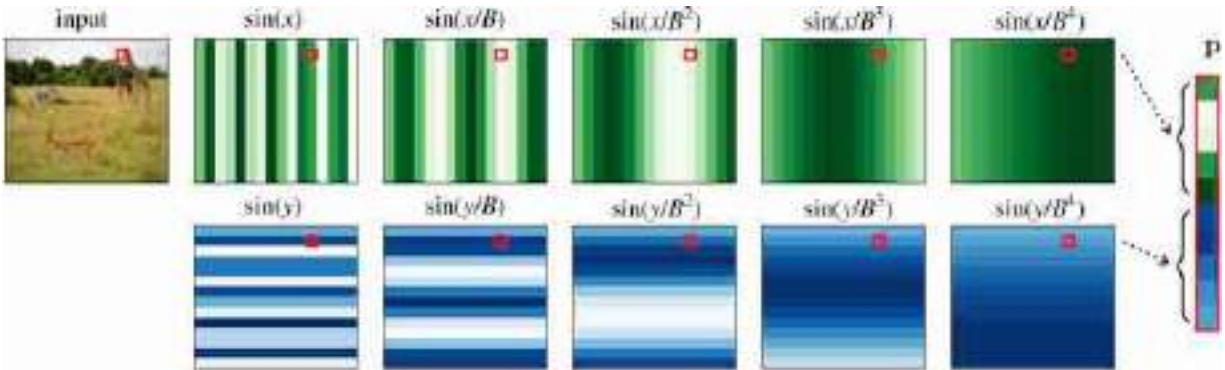


Figure 26.20: Positional codes.

Another strategy is to simply let the positional codes be *learned* by the model, which could potentially result in a better representation of space than sinusoidal codes [109].

While positional encoding is useful and common in transformers, it is not specific to this architecture. The same kind of encodings can be useful for CNNs as well, as a way to make convolutional filters that are conditioned on position, thereby applying a different weighted sum at each location in the image [303]. Positional encodings also appear in radiance fields, which we cover in chapter 45.

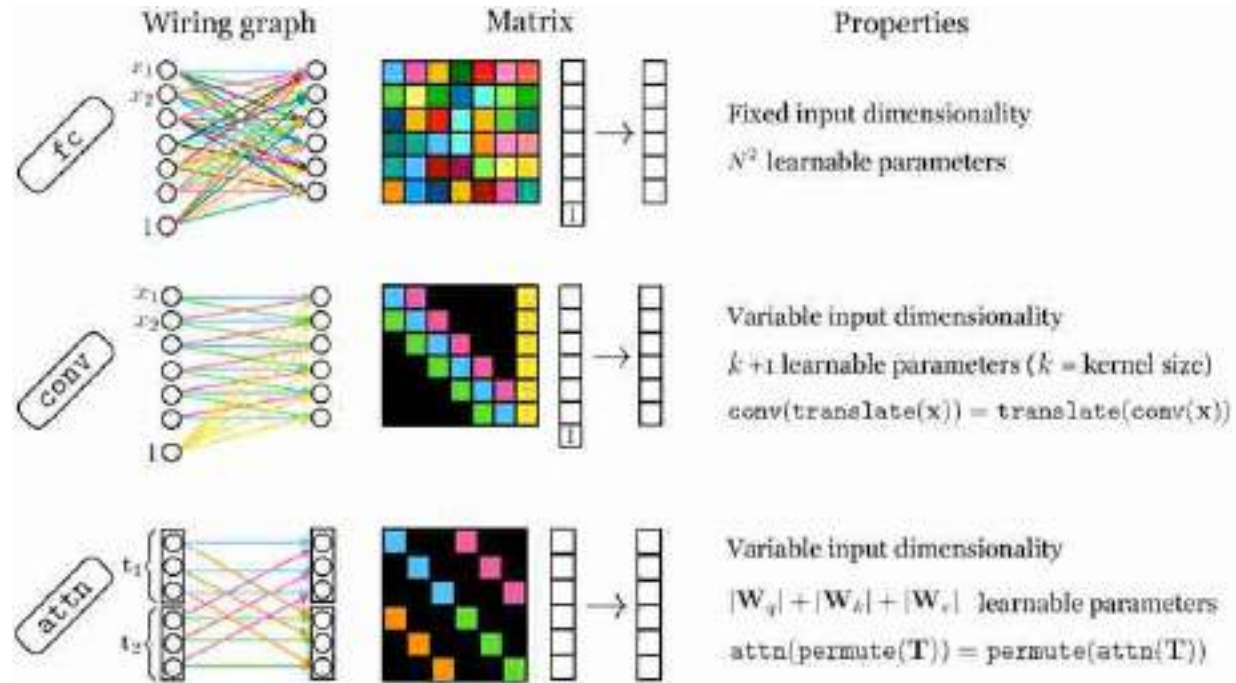


Figure 26.21: Comparing different kinds of layers that all represent an affine transformation between the inputs and outputs.

26.12 Comparing Fully Connected, Convolutional, and Self-Attention Layers

The shorthand “fc” is often used to indicate a fully-connected linear layer.

Many layers in deep nets are special kinds of affine transformations. Three we have seen so far are fully-connected layers (fc), convolutional layers (conv), and self-attention layers (attn). All these layers are alike in that their forward pass can be written as $\mathbf{X}_{\text{out}} = \mathbf{W}\mathbf{X}_{\text{in}} + \mathbf{b}$ for some matrix $\mathbf{W}$ and some vector $\mathbf{b}$. In conv and attn layers, $\mathbf{W}$ and $\mathbf{b}$ are determined as

some function of the input $\mathbf{X}_{in}$. In `conv` layers this function is very simple: just make a Toeplitz matrix that repeats the convolutional kernel(s) to match the dimensionality of $\mathbf{X}_{in}$. In `attn` layers the function that determines $\mathbf{W}$ is a bit more involved, as we saw above, and typically we don't use biases $\mathbf{b}$.

Each of these layers can be represented as a matrix, and examining the structure in these matrices can be a useful way to understand their similarities and differences. The matrix for an `fc` layer is full rank, whereas the matrices for `conv` and `attn` layers have low-rank structure, but different kinds of low-rank structure. In [figure 26.21](#), we show what these matrices look like, and also catalog some of the other important properties of each of these layers.

26.13 Concluding Remarks

As of this writing, transformers are the dominant architecture in computer vision and in fact in most fields of artificial intelligence. They combine many of the best ideas from earlier architectures—convolutional patchwise processing, residual connections, relu nonlinearities, and normalization layers—with several newer innovations, in particular, vector-valued tokens, attention layers, and positional codes. Transformers can also be considered as a special case of **graph neural networks (GNNs)**. We do not have a separate chapter on GNNs in this book since GNNs, other than transformers, are not yet popular in computer vision. GNNs are a very general class of architecture for processing *sets* by forming a graph of operations over the set. A transformer is doing exactly that: it takes an input set of tokens and, layer by layer, applies a network of transformations over that set until after enough layers a final representation or prediction is read out.

VIII

PROBABILISTIC MODELS OF IMAGES

These set of chapters will provide some insights about how simple image models can capture useful properties of natural images. The ideas described here will help us build some intuitions about how the generative models presented in the next part are so successful in modeling images.

- **Chapter 27** presents a sequence of probabilistic models describing images, and a noise removal algorithm corresponding to each model.
- **Chapter 28** introduces the problem of texture analysis and synthesis, and describes two approaches that will be the basic building blocks to understand for other generative models of images.
- **Chapter 29** presents a representation and inference algorithm appropriate for statistical relationships with a modular structure, as is common for many computer vision problems.

27 Statistical Image Models

27.1 Introduction

Building statistical models for images is an active area of research in which there has been remarkable progress. Understanding the world of images and finding rules that describe the pixel arrangements that are likely to be present in an image and which arrangements are not seems like a very challenging task. What we will see in this chapter is that some of the linear filters that we have seen in the previous chapters have remarkable properties when applied to images that allow us to put constraints on a model that aims to capture what real images look like.

First, we need to understand what we mean by a **natural image**. An image is a very high dimensional array of pixels $\ell [n, m]$ and therefore, the space of all possible images is very large. Even if you consider very small thumbnails of just 32×32 pixels and three color channels, with 8 bits per channel, there are more than $10^{7,398}$ possible images. Of course, most of them will just be random noise. [Figure 27.1](#) shows one image, with size 32×32 color pixels, sampled from the set of natural images, and on typical example from the set of all images. A typical sample from the set of all images is 32×32 pixels independently sampled from a uniform distribution.

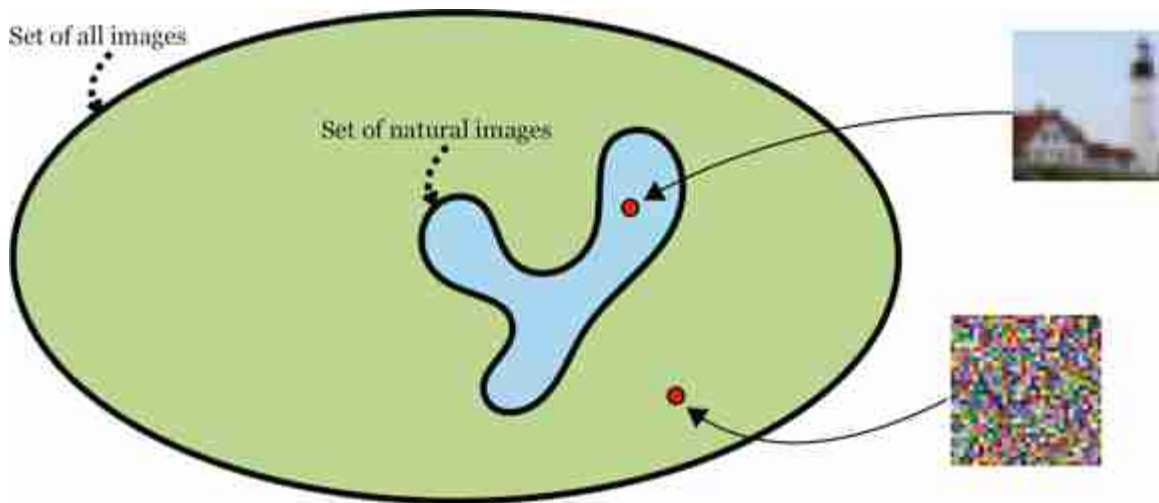


Figure 27.1: The space of natural images is just very small part of the space of all possible images. In the case of color images with 32×32 pixels, most of the space is filled with images that look like noise.

As natural images are quite complex, researchers have used simple visual worlds that retain some of the important properties of natural images while allowing the development of analytic tools to characterize them. In the last decade, researchers have developed large neural networks that learned generative models of images and we discuss these models more in depth in chapter 32. In this chapter we will focus on simpler image models as they will serve as motivation for some of the concepts that will be used later when describing generative models.

[Figure 27.2](#) shows images that belong to different visual worlds. Each world has different visual characteristics. We can clearly differentiate images as belonging to any of these eight worlds, even if those visual worlds look like nothing we normally see when walking around. Explaining what makes images in the set of real images ([figure 27.2\[h\]](#)) different to images in the other sets is not a simple task.

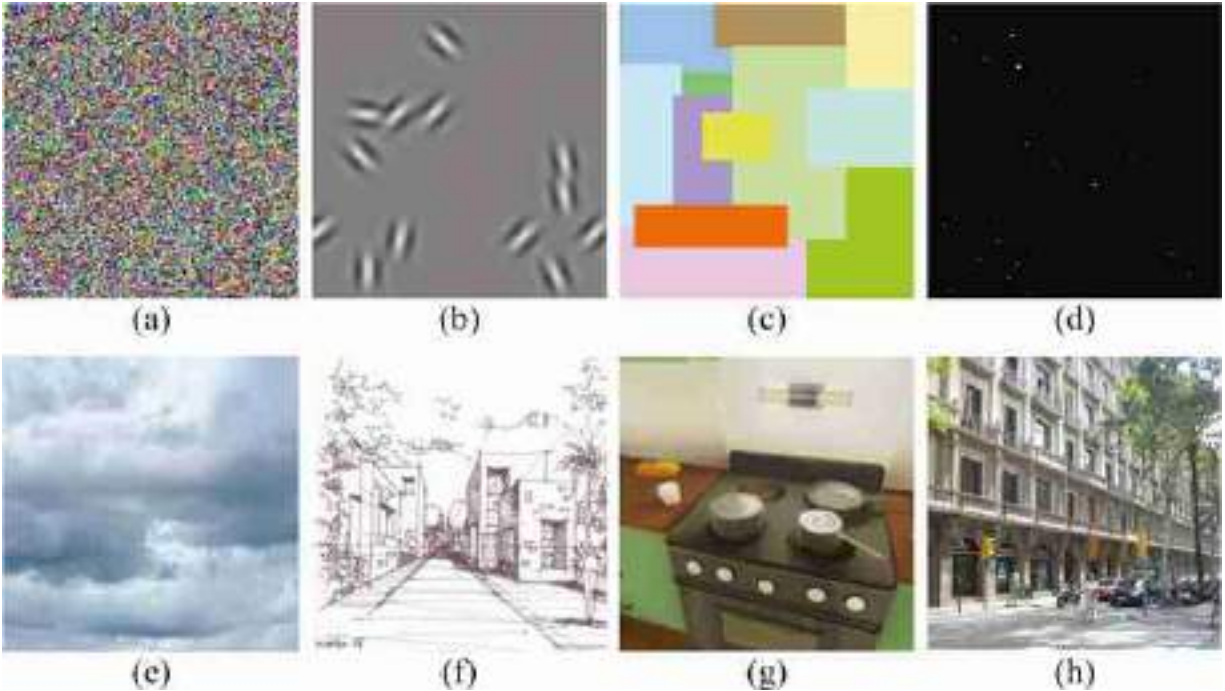


Figure 27.2: Different visual worlds, some real, some synthetic. (a) White noise. (b) Gabor patches. (c) Mondrian. (d) Stars. (e) Clouds. (f) Line drawing. (g) Computer graphic imagery (CGI). (h) A real street. All these worlds have different visual properties. There is even something that makes CGI images distinct from pictures of true scenes.

In this chapter we will talk, in some way or another, about all these visual worlds. The goal is to present a set of models that can be used to describe images with different properties. We will describe models that can be used to capture what makes each visual world unique. These models can be used to separate images into different causes, to remove noise from images, to predict the missing high-frequencies in blurry images, to fill in regions of missing image information due to occlusions, etc.

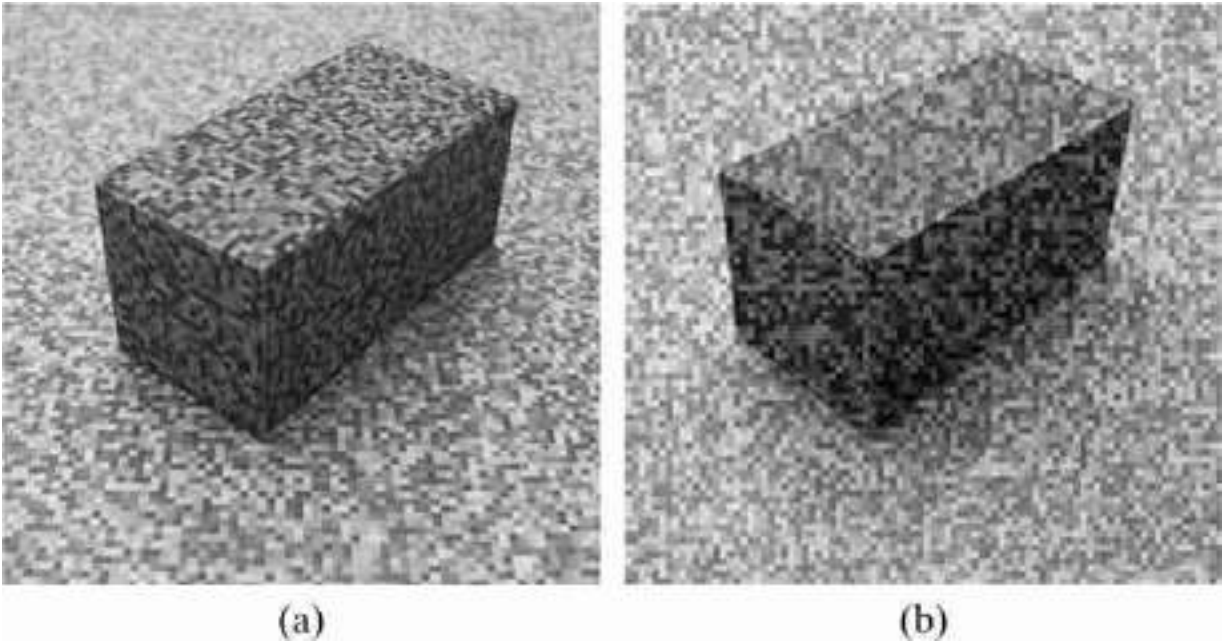


Figure 27.3: Telling noise from surface texture. Which one is which?

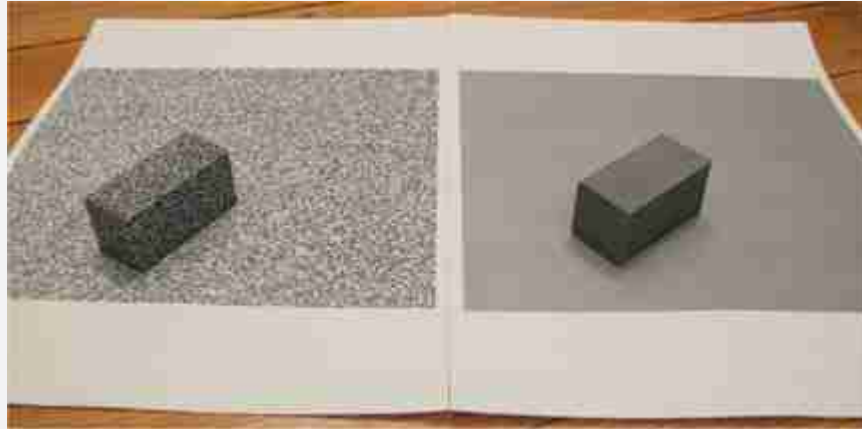
27.2 How Do We Tell Noise from Texture?

How do we tell noise from texture? [Figure 27.3](#) shows two scenes: one image is corrupted with additive stationary noise added on top of the image, the other image contains one object textured with a pattern that looks like noise. Can you tell which one is which? Where is the noise? Is it in the image or in the world?

Stationary image noise¹¹ is independent of the content of the scene, it is not affected by the object boundaries, it does not deform following the three-dimensional (3D) orientation of the surfaces, and it does not change frequency with distance between the camera and the scene. If the noise was really in the world, stationary noise would require a special noise distribution that conspires with the observer viewpoint to appear stationary.

Noise is an independent process from the image content, and our visual system does not perceive noise as a strange form of paint. One important task is image processing it image denoising. Image denoising consists in removing the noise from an image automatically. This is equivalent to being able to separate the image into two components, one looking like a clean uncorrupted picture and the second image containing only noise. Statistical image models try to do this and more.

The original images from [figure 27.3](#).



[Figure 27.3](#)(b) has noise added.

There is a significant interest in building image models that can represent whatever makes real photographs unique relative to, say, images that just contain noise. One way of building an image model is by having some procedural description, for instance a list of objects with locations, poses, and styles and a rendering engine, just as one would do when writing a program to generate CGI. One issue with this way of modeling images is that it will be hard to use it, and will require having an exhaustive list of all the things that we can find in the world. We will see later that the apparent explosion in complexity should not stop us.

However, there is another way of building image models. In the rest of this chapter we will study a very different approach that consists of building statistical models of images that try to capture the properties of small image neighborhoods. Statistical models are also foundational in other disciplines such as natural language modeling. Here we will review models of images of increasing complexity, starting with the simplest possible neighborhood: *the single pixel*.

27.3 Independent Pixels

The simplest image model will consist in assuming that all pixels are independent and their intensity value is drawn from some stationary distribution (i.e., the distribution of pixel intensities is independent of the location in the image). We can then write the probability, $p_{\theta}(\ell)$, of an image, $\ell[n, m]$ with size $N \times M$, as follows:

$$p_{\theta}(\ell) = \prod_{n=1}^N \prod_{m=1}^M p_{\theta}(\ell[n, m]) \quad (27.1)$$

We use p_{θ} to denote a parameterized probability model. The vector θ contains the parameters and can be fitted to data using maximum likelihood or other loss functions. We will see many variants of this probability model here and in the following chapters.

One way of assessing the accuracy a statistical image model is to sample from the model and view the images it produces. This will require knowing the distribution $p_{\theta}(\ell[n, m])$.

What we can do is to take one image from one of the visual worlds that we described in the introduction, and then estimate the model $p_{\theta}(\ell[n, m])$, as illustrated in [figure 27.4](#).

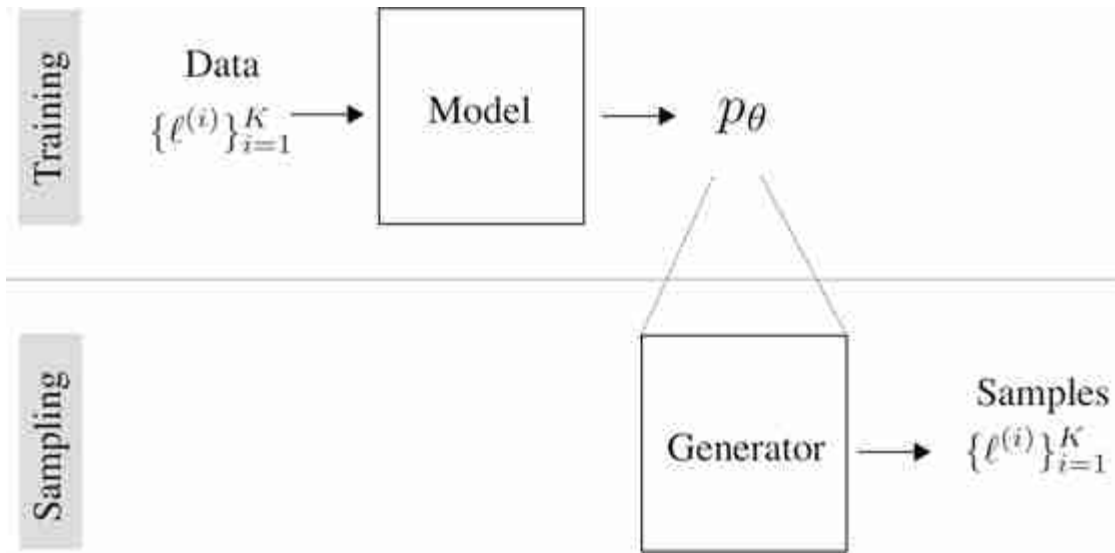


Figure 27.4: Fitting a statistical image model to a set of training images. Once the model is learned we can use the model to sample new images.

In the case of discrete gray-scale images with 256 gray levels, we can represent the distribution p_{θ} as vector of 256 values indicating the probability of each intensity value. In the case of considering color images with 256 levels on each channel (i.e., 8 bits per channel) we will have 256^3 bins in the color histogram.

The **histogram**, $h(c)$, of a gray scale image, $\ell[n, m]$, contains the number of times each intensity value, c , appears in the image:

$$h(c) = \sum_{n,m} \mathbb{1}(\ell[n, m] = c)$$

The length of $\mathbf{h}$ is equal to the number of distinct intensity values.

The parameters, θ , of the distribution are those 256 (or 256^3 for color) values. If we are given a set of images, we can estimate the vector θ as the average histogram of the images. We can also estimate the model from a single image. We can then sample new images based on equation (27.1). In order to sample a new image, we will sample each pixel independently using the intensity distribution given by the training set.

The procedure for four training images is shown in [figure 27.5](#). As illustrated in [figure 27.5](#), the process starts by computing the average color histogram for the four images. Then, we sample a new image by sampling every pixel from the multinomial defined by the p_θ .

In the multinomial model, the probability that the image in location (n, m) has the value k is $p_\theta(\ell[n, m] = k) = \theta_k$. In the case of color images, in this model, k will be one of the 256^3 possible color values.

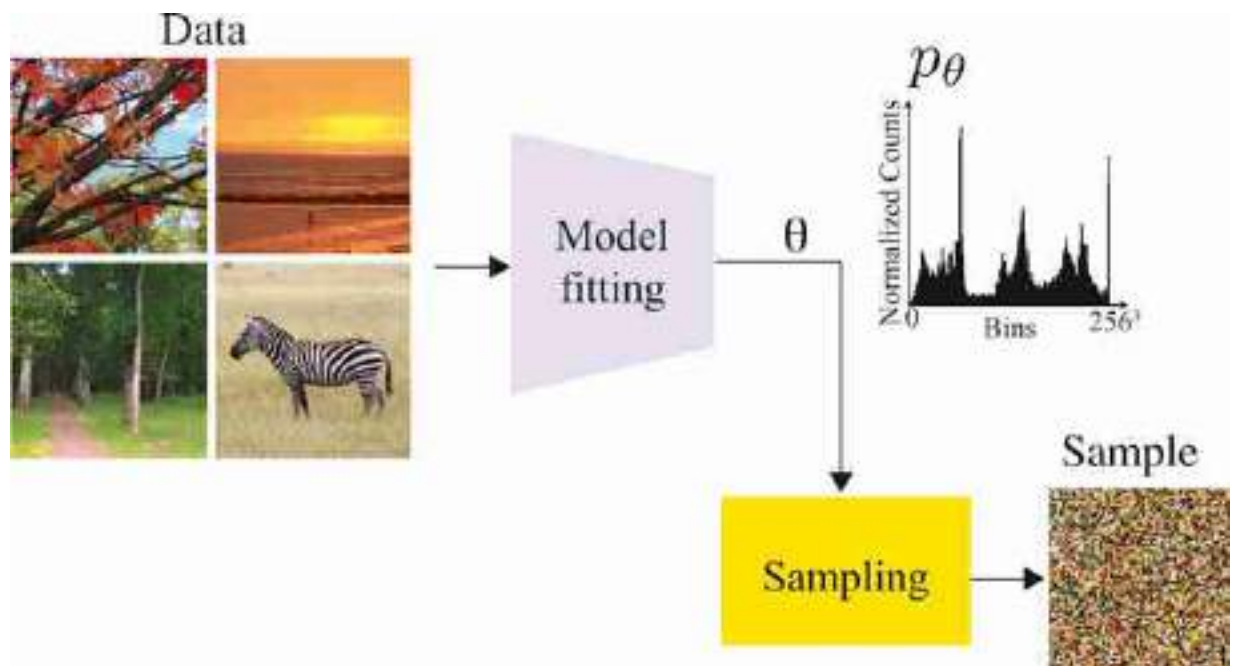


Figure 27.5: Estimation of the image model parameters using one image, and then sampling a new image by sampling pixels independently using the histogram of the training image.

[Figure 27.6](#) shows four pairs of images with matched color histograms. We can see that this simplistic model is somewhat appropriate for pictures of stars (although star images can have a lot more structure), but it miserably fails in reproducing anything with any similarity (besides the overall color) to any of the other images. The failure of color histograms as a generative model of images should not be surprising as treating pixels as independent observations is clearly not how images work.

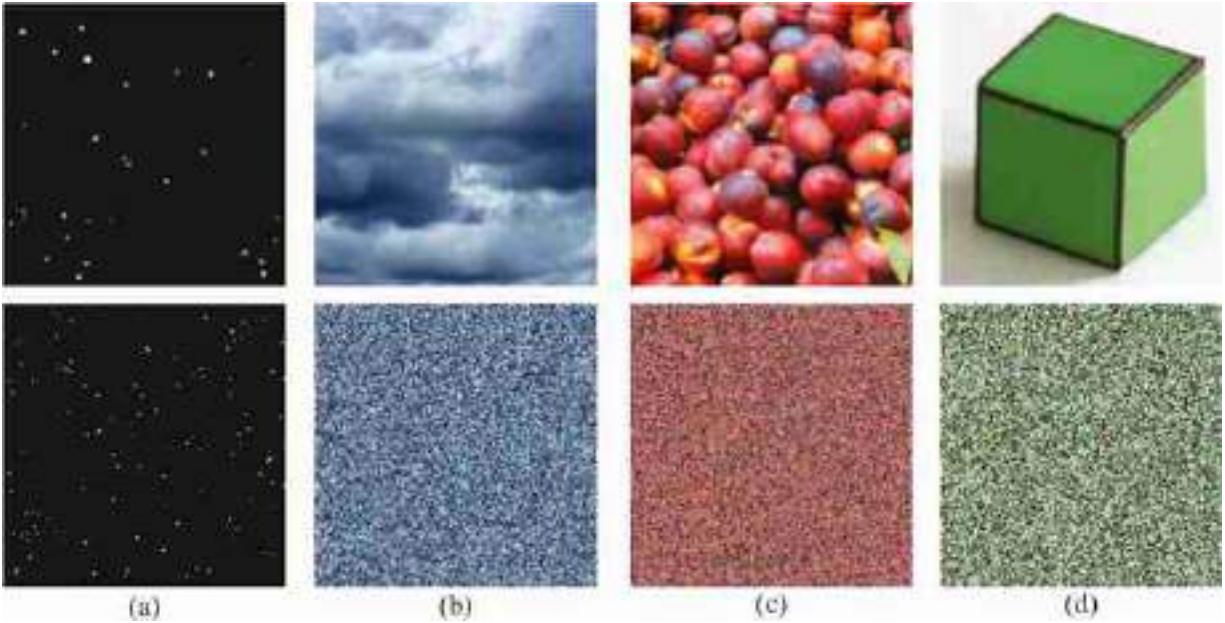


Figure 27.6: Examples of images and corresponding random samples with the same distribution of pixel intensities. Only the image with stars (a) has some visual similarity with the randomly sampled image.

As a generative model of images, this model is very poor. However, this does not mean that image histograms are not important. Manipulating image histograms is a very useful operation. Given two images ℓ_1 and ℓ_2 with histograms $\mathbf{h}_1$ and $\mathbf{h}_2$, we look for a pixelwise transformation $f(\ell[n, m])$ so that the histogram of the transformed image, $f(\ell_1[n, m])$, matches the histogram of ℓ_2 . There are many ways in which histograms can be transformed. One natural choice is a transformation that preserves the ordering of pixels intensities.

[Figure 27.7](#) shows examples of spheres that are reflecting some complex ambient illumination. This is an example that illustrates the perceptual importance of simple histogram manipulations. The two spheres on the left are the two original images. The ones on the right are generated by modifying the image histograms to match the histogram of the opposite image. The figure shows that by simply changing the image histogram we can transform how we perceive the material of each sphere: from shiny to matte [131]. The sphere was first rendered with two illumination models: from a campus photograph ([figure 27.7\[a\]](#)), and from pink noise ([figure 27.7\[b\]](#)). Pink noise is random noise with a $\frac{1}{\sqrt{1+w_x^2}}$ power spectrum, where w_x

and w_y are the spatial frequencies. In [figure 27.7\(c\)](#) and [figure 27.7\(d\)](#) the sphere histograms were swapped, creating a different sense of gloss in the sphere images. Note how by simply modifying the image histograms within the pixels of the sphere, the sphere seems to be made by a different type of material.

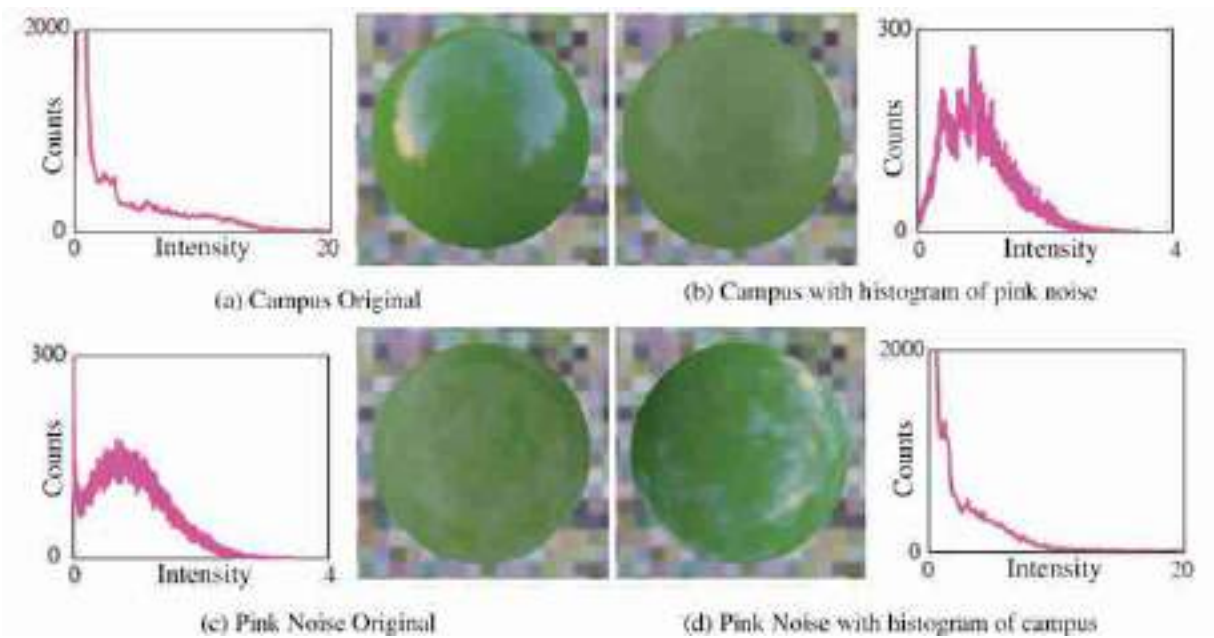


Figure 27.7: Perceptual importance of simple histogram manipulations. Figure from [\[131\]](#).

Image histograms carry useful information, but the histogram alone fails in capturing what makes images unique. We will need to build a more sophisticated image model.

27.4 Dead Leaves Model

In the quest to find what makes photographs of everyday scenes special, the next simplest image structure is a set of two pixels. Things get a lot more interesting now. If one plots the value of the intensity of one pixel as a function of the intensity value of the pixel nearby, the scatterplot produced by many pixel pairs ([figure 27.8](#)) is concentrated on the identity line. As we look at the correlation between pixels that are farther away, the correlation

value slowly decays. The correlation, C , between two pixels separated by offsets of Δn and Δm , is:

$$C(\Delta n, \Delta m) = \rho(\ell[n + \Delta n, m + \Delta m], \ell[n, m]) \quad (27.2)$$

where

$$\rho(a, b) = \frac{E(a - \bar{a}, b - \bar{b})}{\sigma_a \sigma_b}, \quad (27.3)$$

where σ_a is the standard deviation of the variable a , and $\bar{a}$ is the mean of a .

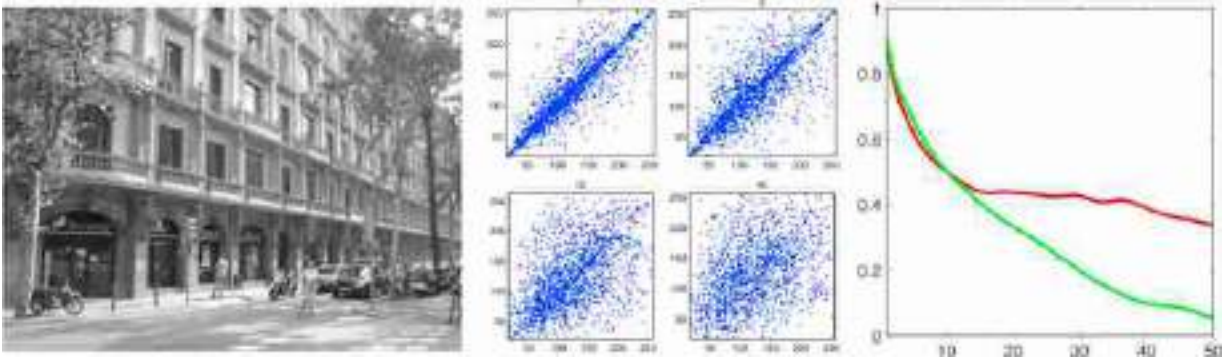


Figure 27.8: (center) Scatter plots of pairs of pixels intensities as a function of distance and (right) cross-correlation as a function for vertical (green) and horizontal (red) displacements for the street scene image.

The behavior of this correlation function when computed on natural images is very different from the one we would observe if the correlation was computed over white noise images.

There have been many efforts to model the minimal set of assumptions needed in order to produce image-like random processes. The **dead leaves model** is a simplified image formation model that tries to approximate some of the properties observed for natural images. This model was introduced in the 1960s by Matheron [323] and popularized by Ruderman in the 90's [416]. The model is better described by a procedure. An image is modeled as a set of shapes (dead leaves) that fall on a flat surface generating a random superposition (e.g., [figure 27.9](#)).

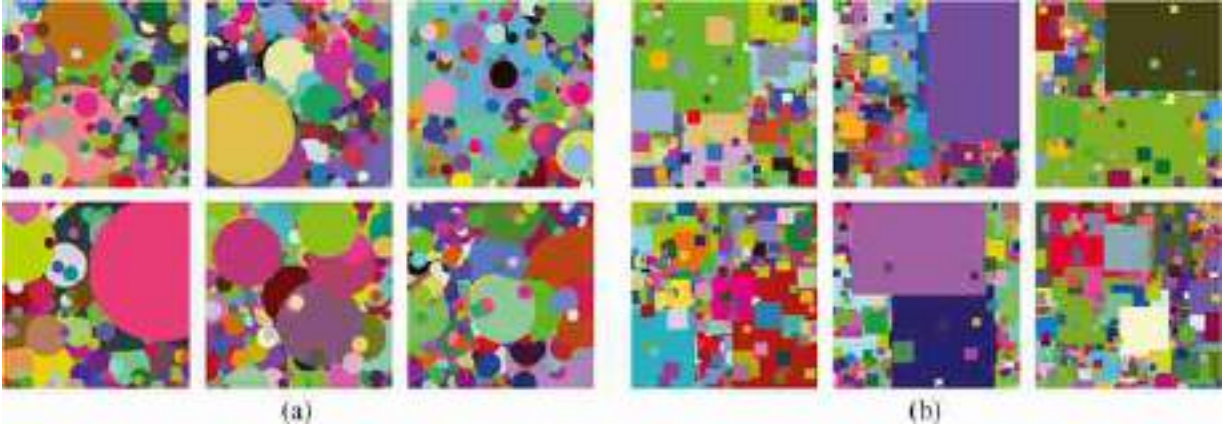


Figure 27.9: Images sampled from the dead leaves model, for different shapes of dead leaves. (a) Circles, and (b) Squares.

The dead leaves model produces very simple images that do not look realistic, but it is useful in explaining the distribution of albedos in natural images. This model is used by the Retinex algorithm [282] to separate an image into illumination and albedo variations, as we saw in chapter 18.

27.4.1 The Power Spectrum of Natural Images

The correlation function from equation (27.2) is related to the magnitude of the Fourier transform of an image.

The Fourier transform of the autocorrelation of a signal is the square of magnitude of its Fourier Transform. The square of the magnitude of the FT, $\|\mathcal{L}(u, v)\|^2$, is the power spectrum of a signal.

Interestingly, the one remarkable property of most natural images is that if you take the 2D Fourier transform of an image, its magnitude falls off as a power law in frequency:

$$\|\mathcal{L}(u, v)\| \simeq \frac{1}{w^\alpha} \quad (27.4)$$

where w denotes the radial spatial frequency ($w = \sqrt{u^2 + v^2}$), $\mathcal{L}(u, v)$ is the Fourier transform of the image $\ell[n, m]$, and $\alpha \simeq 1$. And this is true for all orientations (you can think of this as taking the Fourier transform and looking at the profile of a slice that passes by the origin).

The fact that most natural images follow equation (27.4) has been the focus of an extensive literature [127, 485], and it has been shown to be related to several properties of the early visual system [22, 23] which is

adapted to process natural images. We will see in the next section how this model can be used to solve a number of image processing tasks.

[Figure 27.10](#) shows three natural images and the magnitude of their FT (middle row), and compares them to a random image where each pixel is independently sampled from a uniform distribution. The frequency $(0, 0)$ lies in the middle of each FT. From the images in the middle row, we can see that the largest amplitudes concentrate in the low spatial frequencies for the natural images (we already discussed this in chapter 16). However, the random image has similar spectral content at all frequencies. In order to better appreciate the particular decay in the spectral amplitude with frequency, we can express the FT using polar coordinates, then calculate the average magnitude of the FT by aggregating the values over all angular directions. The result is shown in the bottom row of [figure 27.10](#). The plot also shows three additional curves corresponding to $1/w$, $1/w^{1.5}$, and $1/w^2$. The plots are normalized to start at 1. These images have size 512×512 . The frequency ranges used for the plot are $w \in [20, 256]$. As we can see from the middle row, there are differences on how spectral content is organized across orientations, but the decay along the radial frequency roughly follows a $1/w^{1.5}$ decay in these examples. And these property is shared across the four images despite their very distinct appearance.

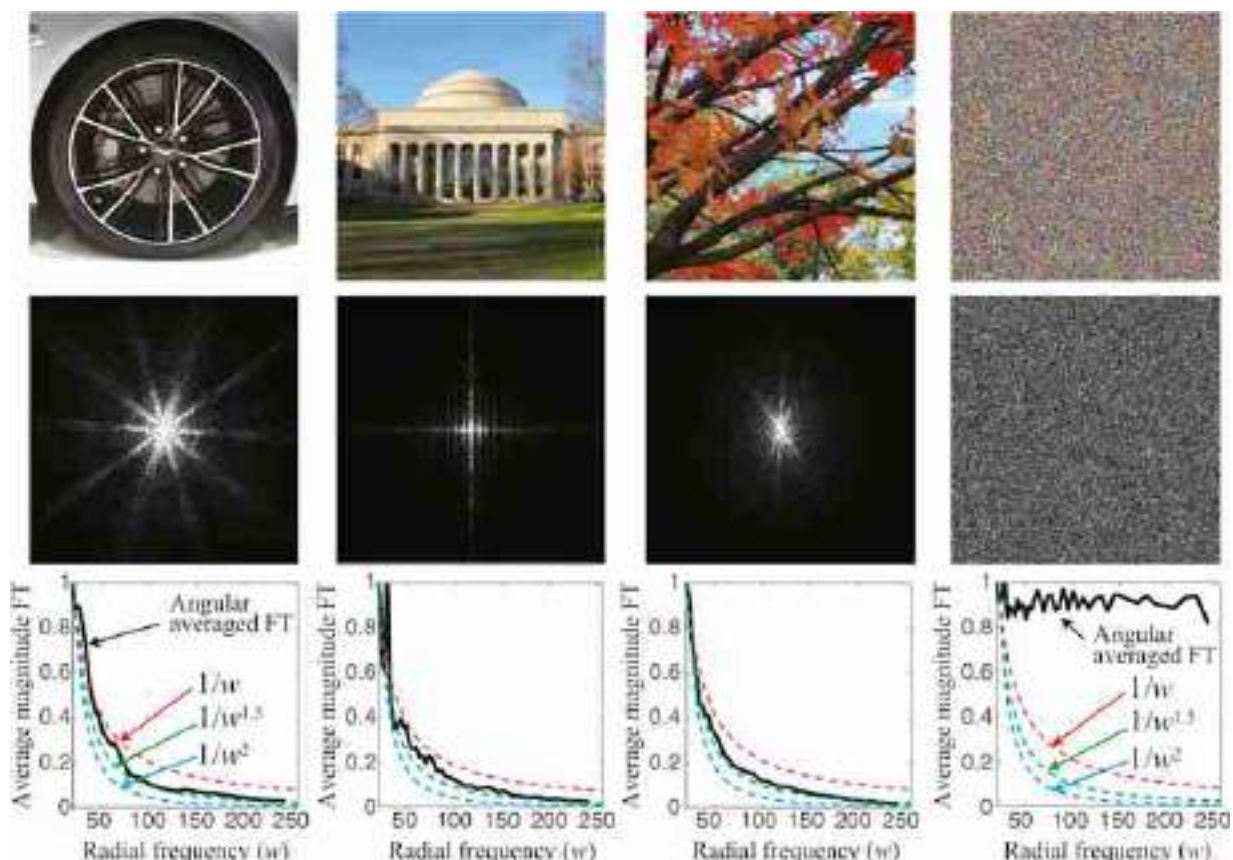


Figure 27.10: (top) Three natural images with size 512×512 pixels, and one random image. (middle) The magnitude of their Fourier transforms (FT), images are transformed to grayscale by averaging the three color channels. (bottom) Plot of the angular average of radial sections of the FT, compared with the curves that correspond to $1/w$, $1/w^{1.5}$, and $1/w^2$, where w is the radial frequency. We can see that the FT of the three natural images decays roughly with $1/w^\alpha$. The noise image is very different (has a flat magnitude).

We will now use this observation to build a better statistical model of images than using pixel histograms as we described in section 27.3.

27.5 The Gaussian Model

We want to write a statistical image model that takes into account the correlation statistics and the power spectrum of natural images (see [439] for a review). If the only constraint we have is the correlation function among image pixels, which mean that among all the possible distributions, the one that has the maximum entropy (thus, imposing the fewest other constraints) is the Gaussian distribution:

$$p(\ell) \propto \exp\left(-\frac{1}{2}\ell^T \mathbf{C}^{-1}\ell\right) \quad (27.5)$$

where $\mathbf{C}$ is the covariance matrix of all the image pixels. Note that this model no longer assumes independence across pixels. It accounts for the correlation of intensity values between different pixels, as discussed in previous sections. This model is also related to studies that use principal component analysis to study the typical components of natural images [183].

For stationary signals, the covariance matrix $\mathbf{C}$ has a circulant structure (assuming a tiling of the image) and can be diagonalized using the Fourier transform. The eigenvalues of the diagonal matrix correspond to the power (squared magnitude) of the frequency components of the Fourier transform. This is, the matrix $\mathbf{C}$ can be written as $\mathbf{C} = \mathbf{F}\mathbf{D}\mathbf{F}^T$. Where $\mathbf{F}$ is the matrix that contains the Fourier basis, as seen in chapter 16, and the diagonal matrix $\mathbf{D}$ has, along the diagonal, the values $1/w^\alpha$ for all radial frequencies w . The value of α can be set to a fix value of 1.5.

27.5.1 Sampling Images

Once a distribution is defined, we can use it to sample new images. First we need to specify the parameters of the distribution. In this case, we need to specify the covariance matrix $\mathbf{C}$. [Figure 27.11](#) shows the results of sampling images using the $1/w^\alpha$ model to fill the diagonal values of the matrix $\mathbf{D}$. These images are generated by making an image with a random phase and with a magnitude of the power spectrum following $1/(1 + w^\alpha)$ with $\alpha = 1.5$. We add 1 to the denominator term to avoid division by zero. The bottom row of [figure 27.11](#) generates color images by sampling each color channel independently.

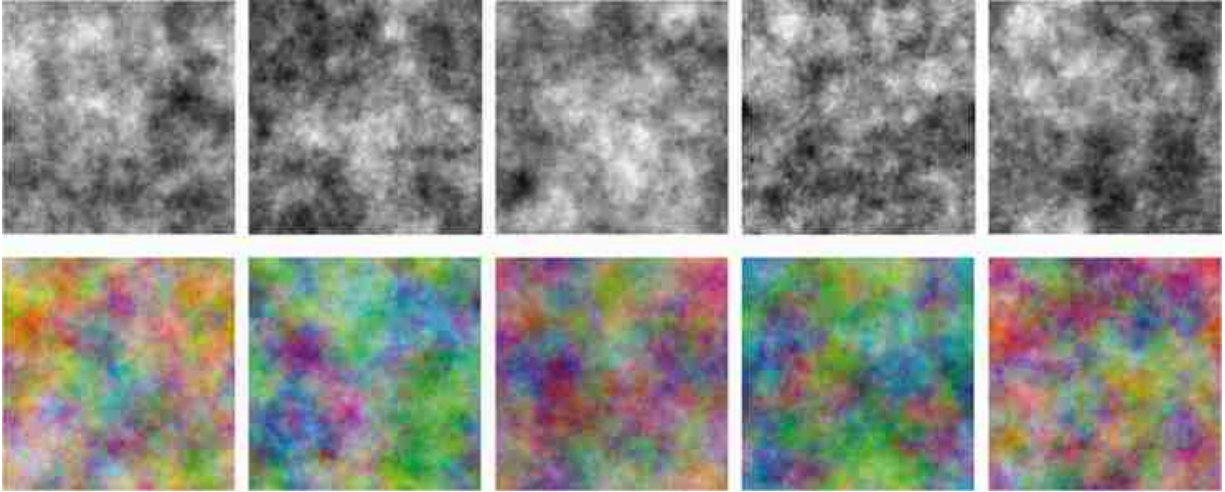


Figure 27.11: (top) Images are generated by making an image with a random phase and with a magnitude of the power spectrum following $1/(1 + w^\alpha)$ with $\alpha = 1.5$. (bottom) Same generative process applied to each color channel independently. The generated images look like clouds.

We can also sample new images by matching the parameter of individual natural images. [Figure 27.12](#) shows four examples. To process color images, we first look for a color space that decorrelates color information for each image. This can be done by applying principal component analysis (PCA) to the three color channels. This results in a 3×3 matrix that will produce three new channels that are decorrelated. Then, for each channel, we compute the FT and keep the magnitude and replace the phase with random noise. We finally compute the new generated image by doing the inverse FT and applying the inverse of the PCA matrix in order to return to the original color space. This results in images that keep a similar color distribution than the inputs.

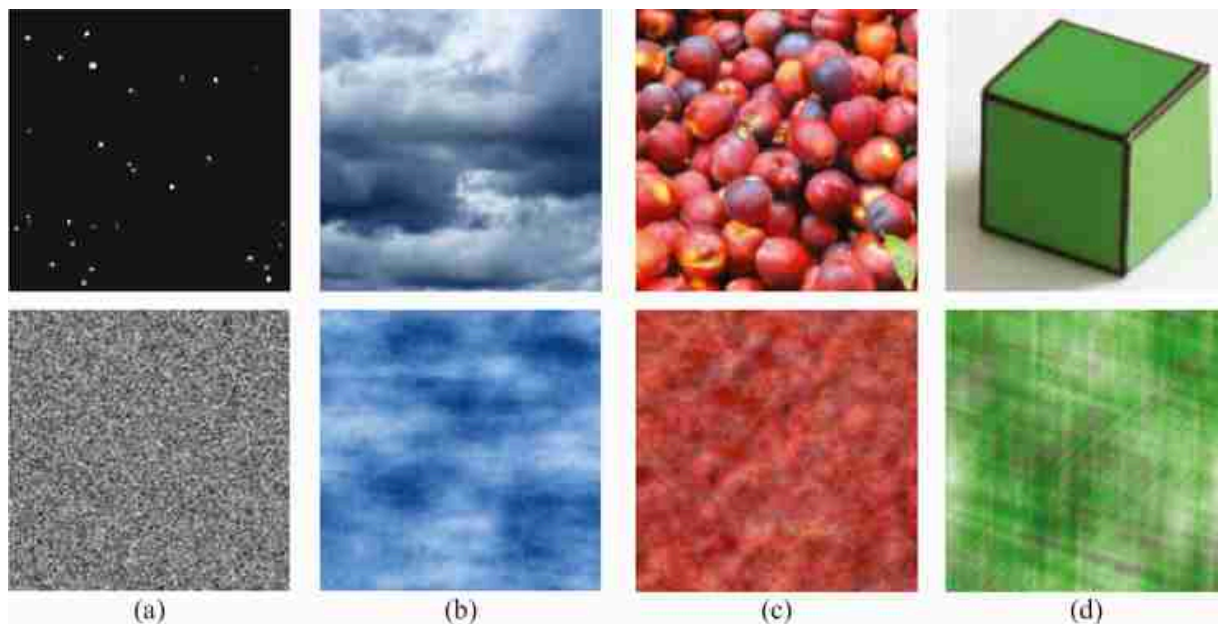


Figure 27.12: Examples of images and random samples with the same magnitude of the FT of the corresponding image. We use PCA in color space to find decorrelated color components. Then, each decorrelated color channel is sampled independently and the final image is created by returning to the original color space. Only the image with clouds (b) has some visual similarity with the randomly sampled image.

As shown in [figure 27.12](#), when fitting the Gaussian model to different real images, and then sampling from it, the result is always an image that looks like cloudy noise regardless of the image content. However, a few attributes of the original image are preserved including the size of the details present in the image and the dominant orientations.

[Figure 27.13](#) shows an example image where the sample from its Gaussian model produces a similar image, due to the strong oriented pattern present in the image. In this example, the input image is mostly one-dimensional (1D) as most of the variations occur across one orientation being constant along the perpendicular direction. This structure can be captured by the magnitude of the Fourier transform of the image.

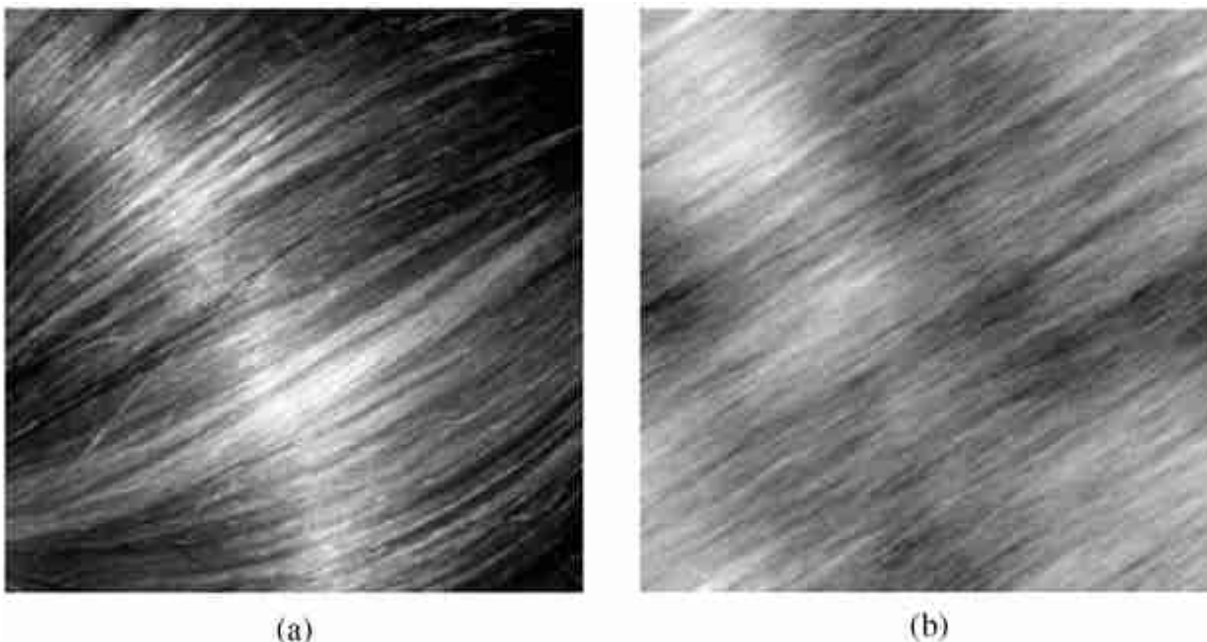


Figure 27.13: (a) Photograph of hair. (b) Random draw of Gaussian image model using covariance matrix fit from (a). For this unusual image, the model works well, but this is an exception for the Gaussian image model.

The Gaussian image model, despite being very simple, can be used to study some image processing tasks, including image denoising, which we will discuss next.

Devising processes that generate realistic textures has a long history and many applications in the movie industry. One example is Perlin noise developed by Ken Perlin in 1985. In the rest of this book, we will study in depth the image generation problem when talking about generative models.

27.5.2 Image Denoising under the Gaussian Model: Wiener Filter

As an example of how to use image priors for vision tasks, we will study how to do image denoising using the prior on the structure of the correlation of natural images. In this problem, we observe a noisy image ℓ_g corrupted with white Gaussian noise:

$$\ell_g[n, m] = \ell[n, m] + g[n, m] \quad (27.6)$$

The goal is to recover the uncorrupted image $\ell[n, m]$. The noise $g[m, n]$ is white Gaussian noise with variance σ_g^2 . The denoising problem can be

formulated as finding the $\ell[n, m]$ that maximizes the probability of the uncorrupted image, ℓ_g , called the maximum a posteriori, or MAP, estimate:

$$\max_{\ell} p(\ell | \ell_g) \quad (27.7)$$

In these equations we write the image as a column vector ℓ . This posterior density can be written as:

$$\max_{\ell} p(\ell | \ell_g) = \max_{\ell} p(\ell_g | \ell) p(\ell) \quad (27.8)$$

where the likelihood and prior functions are:

$$p(\ell_g | \ell) \propto \exp(-|\ell_g - \ell|^2 / \sigma_g^2) \quad (27.9)$$

$$p(\ell) \propto \exp\left(-\frac{1}{2} \ell^T \mathbf{C}^{-1} \ell\right) \quad (27.10)$$

The solution to this problem can be obtained in closed form:

$$\ell = \mathbf{C} (\mathbf{C} + \sigma_g^2 \mathbf{I})^{-1} \ell_g \quad (27.11)$$

This is just a linear operation. It can also be written in the Fourier domain as:

$$\mathcal{L}(v) = \frac{A/|v|^{2\alpha}}{A/|v|^{2\alpha} + \sigma_g^2} \mathcal{L}_g(v) \quad (27.12)$$

[Figure 27.14](#) shows the result of using the Gaussian image model for image denoising. Although there is a certain amount of noise reduction, the result is far from satisfactory. The Gaussian image model fails to capture the properties that make a natural image look real.

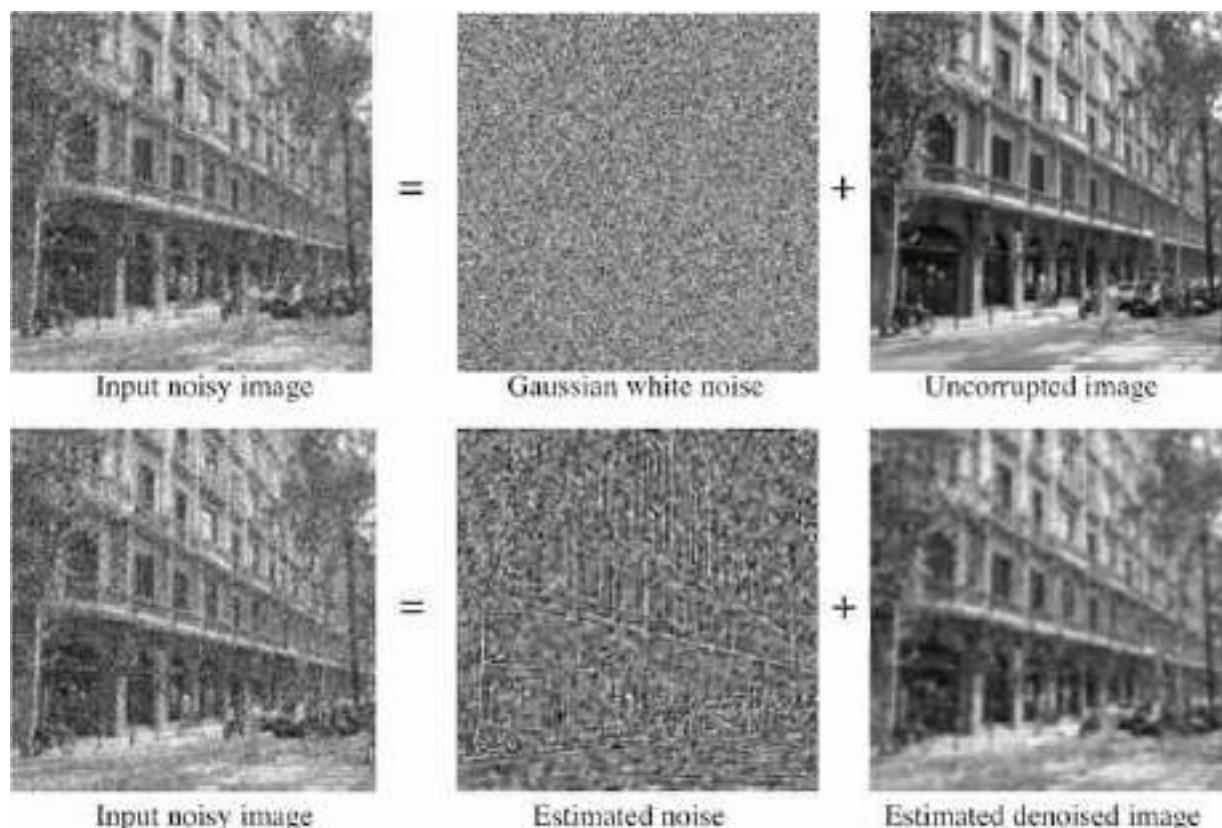


Figure 27.14: (Top row) Ground truth decomposition of image into Gaussian white noise and the image, uncorrupted by noise. (bottom row) Gaussian image model denoising results. Note that the estimated noise image shows residual spatial structure from the original image.

27.6 The Wavelet Marginal Model

As we showed previously, sampling from a Gaussian prior model generates images of clouds. Although the Gaussian prior does not give any image zero probability (i.e., by sampling for a very long time it is not impossible to sample the Mona Lisa), the most typical images under this prior look like clouds.

The Gaussian model captures the fact that pixel values are correlated and that such correlation decreases with distance. However, it fails to capture other very important properties of images, such as flat regions, edges, and lines. Here, we extend the Gaussian model in two ways. (1) We will use localized filters instead of sinusoids (i.e., Fourier basis) that span the entire image, as in the power spectrum model used in the previous section. This helps to capture local structure such as lines and edges. (2) Instead of

characterizing the outputs of those filters by a single number (e.g., the mean power), we measure the histogram shape of all the filter responses.

In the 1980s, a number of papers noticed a remarkable property of images: when filtering images with band-pass filters (i.e, image derivatives, Gabor filters, etc.) the output had a non-Gaussian distribution [96, 127]. [Figure 27.15](#) shows the histogram of the input image and the histogram of the output of applying the filters $[-1, 1]$ and $[-1, 1]^T$.

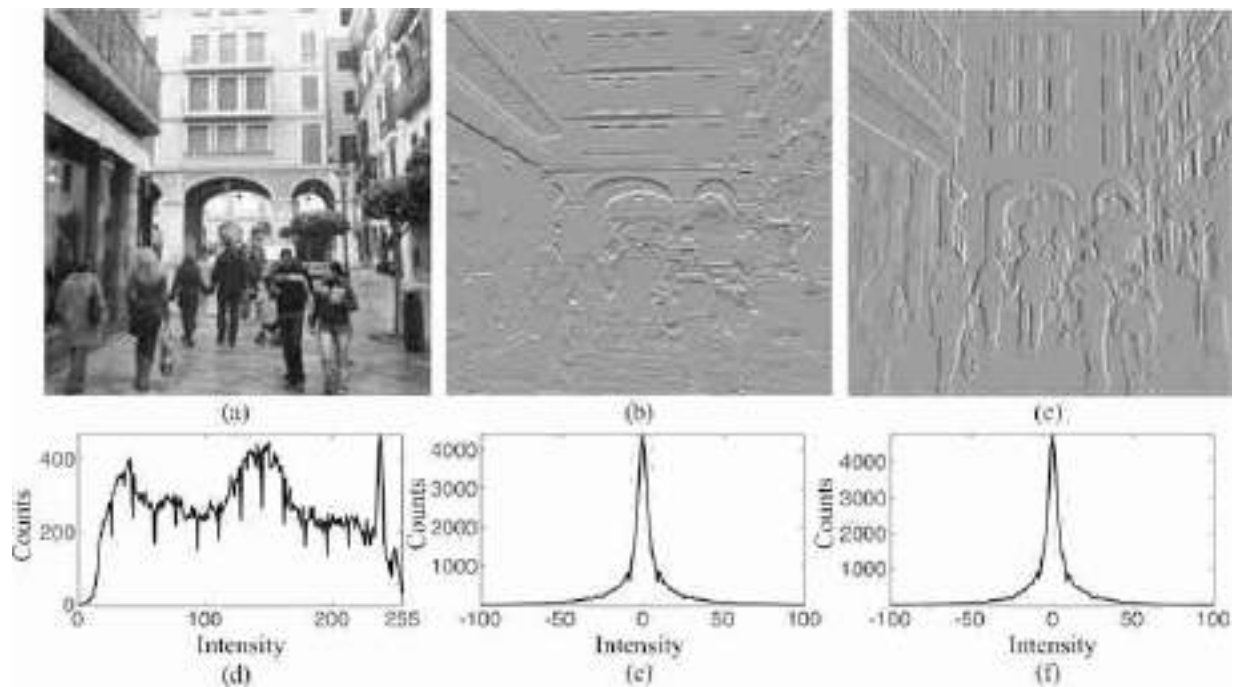


Figure 27.15: (a) Input image. (b) Horizontal and (c) vertical derivatives of the input image. (d) Histogram of pixel intensities of the input image. (e and f) Histograms of the two derivatives of the original image. Note that both histograms have non-Gaussian distributions, a characteristic of natural images.

First, the histogram of an unfiltered image is already non-Gaussian. However, the image histogram does not have any interesting structure and different images have different histograms. Something very different happens to the outputs of the filters $[-1, 1]$ and $[-1, 1]^T$. The histograms of the filter outputs are very clean (note that they are computed over the same number of pixels), have a unique maximum around zero, and seem to be symmetric.

Note that this distribution of the filter outputs goes against the hypothesis of the Gaussian model. If the Gaussian image model were correct, then any filtered image would also be Gaussian, as a filtering is just a linear operation over the image. [Figure 27.16](#) makes this point, showing histograms of band-pass filtered images (an image containing Gaussian noise, and two natural images). Note that the image subband histograms are significantly different than Gaussian.

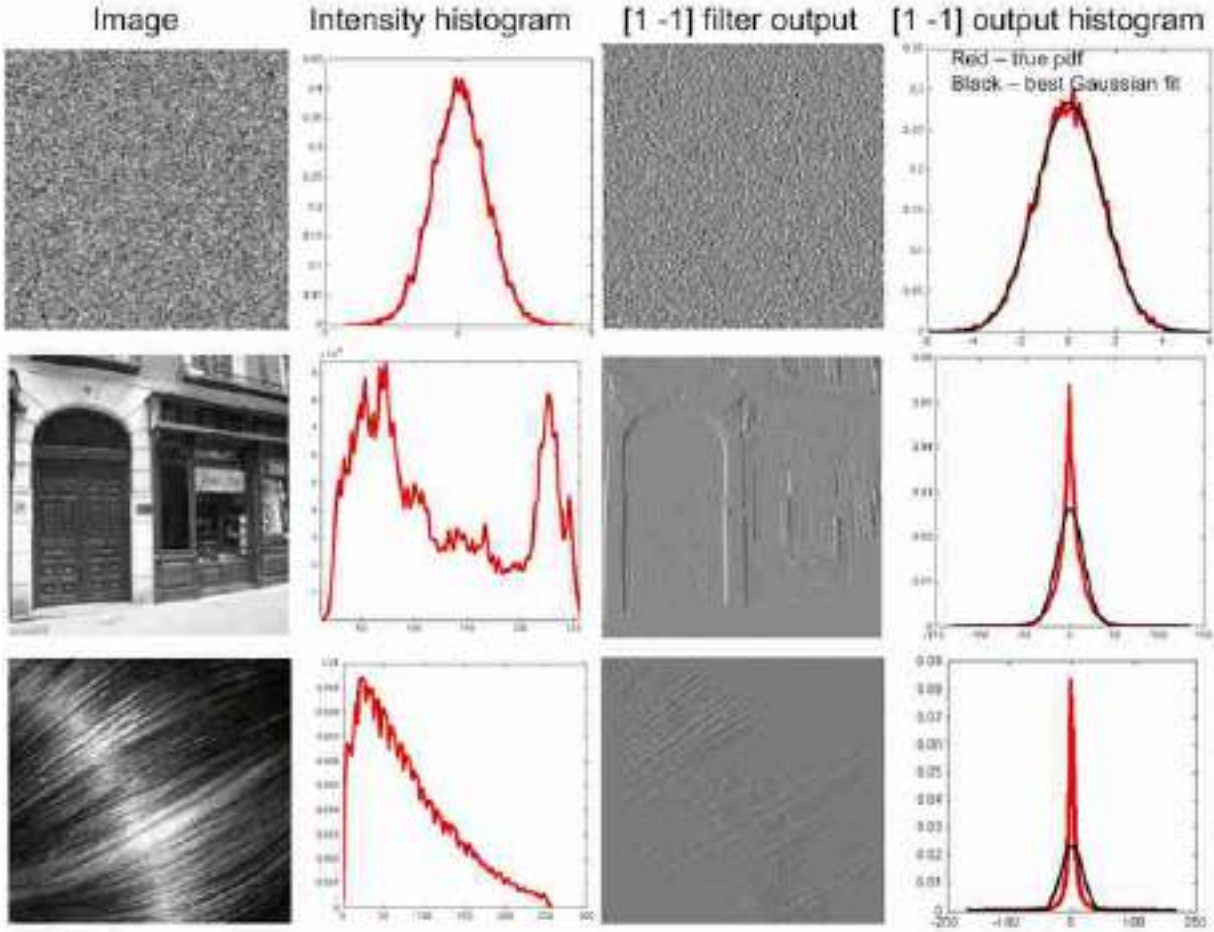


Figure 27.16: Comparison of histograms of images from different visual worlds, and band-pass filtered versions of those images. (top row) Gaussian noise. Band-pass filtered Gaussian noise is still Gaussian distributed. (middle and bottom rows) Two very different looking images, when band-pass filtered, have similar looking non-Gaussian, narrowly peaked histogram distributions. The black line shows the best Gaussian fit, a poor fit to the spiky histograms.

The distribution of intensity values in the image filtered by a band-pass filter h , $\ell_h = \ell \circ \mathbf{h}$, can be parameterized by a simple function:

$$p(\ell_h[n, m] = x) = \frac{\exp(-|x/s|^r)}{2(s/r)\Gamma(1/r)} \quad (27.13)$$

Where Γ is the Gamma distribution. This function, equation (27.13), known as the generalized Laplacian distribution, has two parameters: the exponent r , which alters the distribution's shape; and the scale parameter s , which influences its variance. This distribution fits the outputs of band-pass filters of natural images remarkably well. In natural images, typical values of r lie within the range $[0.4, 0.8]$.

The standard Laplacian distribution has the form:

$$p(x) = \frac{1}{2} \exp(-|x|)$$

[Figure 27.17](#) shows the shape of the distribution when changing the parameter r . Setting $r = 2$ gives the Gaussian distribution. With $r = 1$ we obtain the standard Laplacian distribution. When $r \rightarrow \infty$ the distribution converges to a uniform distribution in the range $[-s, s]$. And when $r \rightarrow 0$, the distribution converges to a delta function.

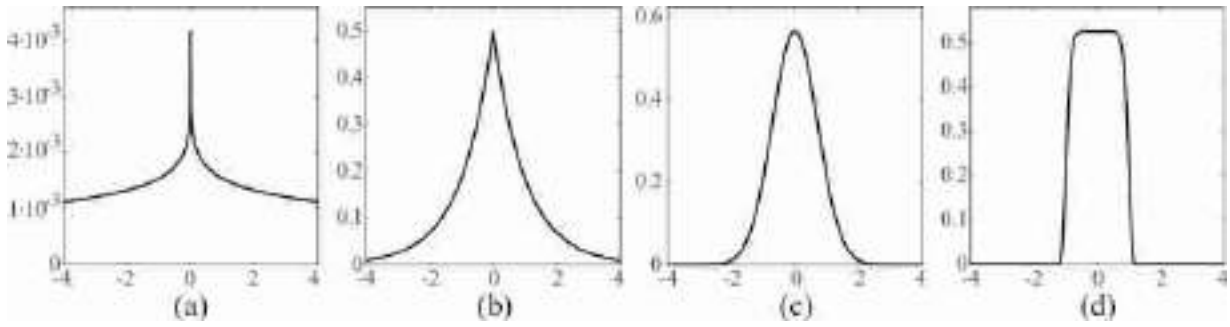


Figure 27.17: Generalized Laplacian distribution with $s = 1$ and (a) $r = 0.1$, (b) $r = 1$, (c) $r = 2$, and (d) $r = 10$.

Now that we have seen that image derivatives follow a special distribution for natural images, let's define a statistical image model that captures the distribution of image derivatives. We can use this prior model in synthesis and noise cleaning applications.

Let's consider a filter bank of K filters with the convolution kernels: $h_k[n, m]$ with $k = 0, \dots, K - 1$. The filter bank could be composed of several Gaussian derivatives with different orientations and orders, or Gabor filters, or steerable filters from the steerable pyramid, or any other collection of band-pass filters. If we assume that the output of each filter is independent from each other (clearly, this assumption is only an approximation) we can write the following prior model for natural images:

$$p(\ell) = \prod_{k=0}^{K-1} \prod_{n,m} p_k(\ell_k[n, m]) \quad (27.14)$$

This distribution is the product over all filters and all locations, of how likely is each filtered value according to the distribution of natural images

as modeled by the distributions p_k , where p_k is a distribution with the form from equation (27.13).

What is the most likely image under the model equation (27.14)? It is a constant image.

To see how this model works, let's look at a toy problem.

27.6.1 Toy Model of Image Inpainting

To build intuition about image structures that are likely under the wavelet marginal model, let's consider a simple 1D image of length $N = 11$ that has a missing value in the middle:

$$\ell = [1, 1, 1, 1, 1, ?, 0, 0, 0, 0, 0] \quad (27.15)$$

The middle value is missing because of image corruption or occlusion and we want to guess the most likely pixel intensity at that location. This task is called **image inpainting**. In order to guess the value of the missing pixel we will use the image prior models we have presented in this chapter.

We will denote the missing value with the variable a . Different values of a will produce different types of steps. For instance, setting $a = 1$ or $a = 0$ will produce a sharp step edge. Setting $a = 0.5$ will produce a smooth step signal. What is the value of a that maximizes the probability of this 1D image under the wavelet marginal model?

First, we need to specify the prior distribution. For this example we will use a model specified by a single filter ($K = 1$) with the convolution kernel $[-1, 1]$, and we will use the distribution given by equations (27.13) and (27.14). In this case, we can write the wavelet marginal model from equation (27.14) in close form. Applying the filter $[-1, 1]$ to the 1D signal and using mirror boundary conditions, results in:

$$\ell_0 = \ell \circ [-1, 1] = [0, 0, 0, 0, 0, 0, 1-a, a, 0, 0, 0] \quad (27.16)$$

Now, we can use equation (27.14) to obtain the probability of ℓ as a function of a :

$$p(\ell | a) = \prod_{n=0}^{N-1} p(\ell_0[n] | a) = \quad (27.17)$$

$$= \frac{\exp(-|(1-a)/s|^r) \exp(-|a/s|^r)}{(2s/r\Gamma(1/r))^N} = \quad (27.18)$$

$$= \frac{1}{(2s/r\Gamma(1/r))^N} \exp\left(-\frac{|1-a|^r + |a|^r}{s^r}\right) \quad (27.19)$$

Note that the first factor only depends on r and on the signal length N , and the second factor depends on r and the missing value a . We are interested in finding the values of a that maximize the $p(\ell | a)$ for each value of r . Therefore, only the second factor is important for this analysis. [Figure 27.18](#) plots the value of the second factor of equation (27.19) as we vary a and r . The red line represent the places where the function reaches a local maximum as a function of a for every value of r .

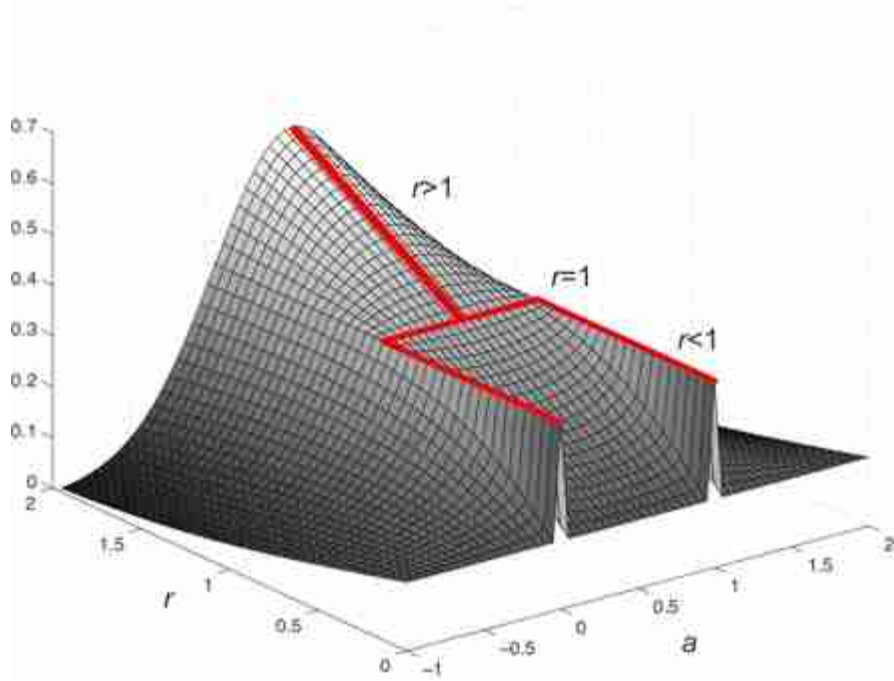


Figure 27.18: Best values of a , as a function of r , that maximize the probability of the 1D image $[1, 1, 1, 1, a, 0, 0, 0, 0]$ under the wavelet prior model. The optimal a changes when crossing the $r = 1$ value.

For $r = 2$ the best value of a is $a = 0.5$, and this solution is the best for any value of $r > 1$. For $r = 1$, any value $a \in [0, 1]$ is equally good, and $p(\ell)$

decreases for values of a outside that range. And for $r < 1$, the best solution is for $a = 0$ or $a = 1$. Therefore, values of $r < 1$ prefer sharp step edges. Note that $a = 0$ and $a = 1$ are both the same step edge translated by one pixel.

[Figure 27.19](#) shows the final result using two different values of r . When using $r = 2$, the result is a smooth signal (which is the same output we would have gotten if we had used the Gaussian image prior). The estimated value for a is the average of the nearby values. For $r = 0.5$, the result is a sharp step edge. The estimated value of a can be equal to any of the two nearby values. In the plot shown in [figure 27.19](#)(right) corresponds to taking the value $a = 1$.

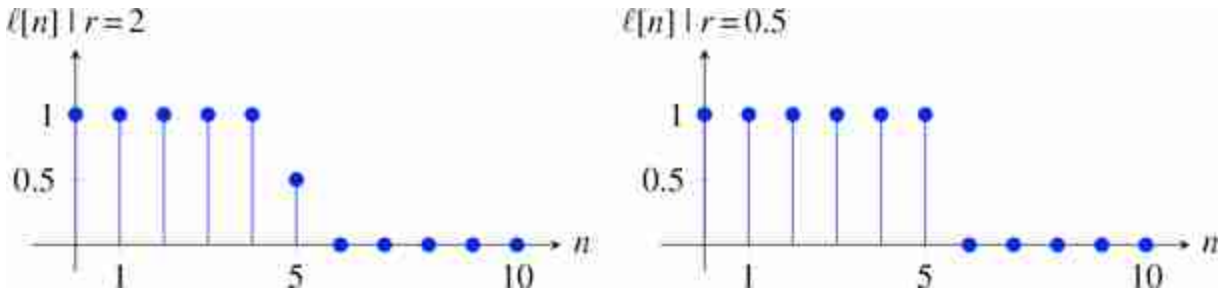


Figure 27.19: Reconstructed signal using (left) $r = 2$ and (right) $r = 0.5$. The estimated values of a are 0.5 and 1.

27.6.2 Image Denoising (Bayesian Image Estimation)

The highly kurtotic distribution of band-pass filter responses in images provides a regularity that is very useful for image denoising. If we assume zero-mean additive Gaussian noise of variance σ^2 , and the observed subband coefficient value $\hat{x} = \hat{\ell}_0[n, m]$, where $\ell_0[n, m]$ is the subband of the observed noisy input image, then the likelihood of any coefficient value, $x = \ell_0[n, m]$, is

$$p(\hat{x}|x) \propto \exp\left(-\frac{(x - \hat{x})^2}{2\sigma^2}\right), \quad (27.20)$$

where we are suppressing the normalization constants of equation (27.13) for simplicity. The prior probability of any coefficient value is the Laplacian distribution (assuming $r = 1$) of the subbands,

$$p(x) \propto \exp\left(-\frac{|x|}{s}\right). \quad (27.21)$$

By Bayes rule, the posterior probability is the product of those two functions,

$$P(x|\hat{x}) \propto \exp\left(-\frac{|x|}{s}\right) \exp\left(-\frac{(x-\hat{x})^2}{2\sigma^2}\right) \quad (27.22)$$

The plots of [figure 27.20](#) show how this works. The horizontal axis for each plot is the value at a pixel of an image subband coefficient. The blue curve, showing the Laplacian prior probability for the subband values, is the same for all rows. The red line is the Gaussian likelihood and it is centered in the observed coefficient, and it is proportional to the probability that the true value had any other coefficient value. The green line shows the posterior, equation (27.22), for several different coefficient observation values. [Figure 27.20\(a\)](#) shows the result when a zero subband coefficient is observed, resulting in the zero-mean Gaussian likelihood term (red), and the posterior (black) with a maximum at zero. In [figure 27.20\(b\)](#) an observation of 0.26 shifts the likelihood term, but the posterior still has a peak at zero. And in [figure 27.20\(c\)](#) an observed coefficient value of 1.22 yields a maximum posterior probability estimate of 0.9.

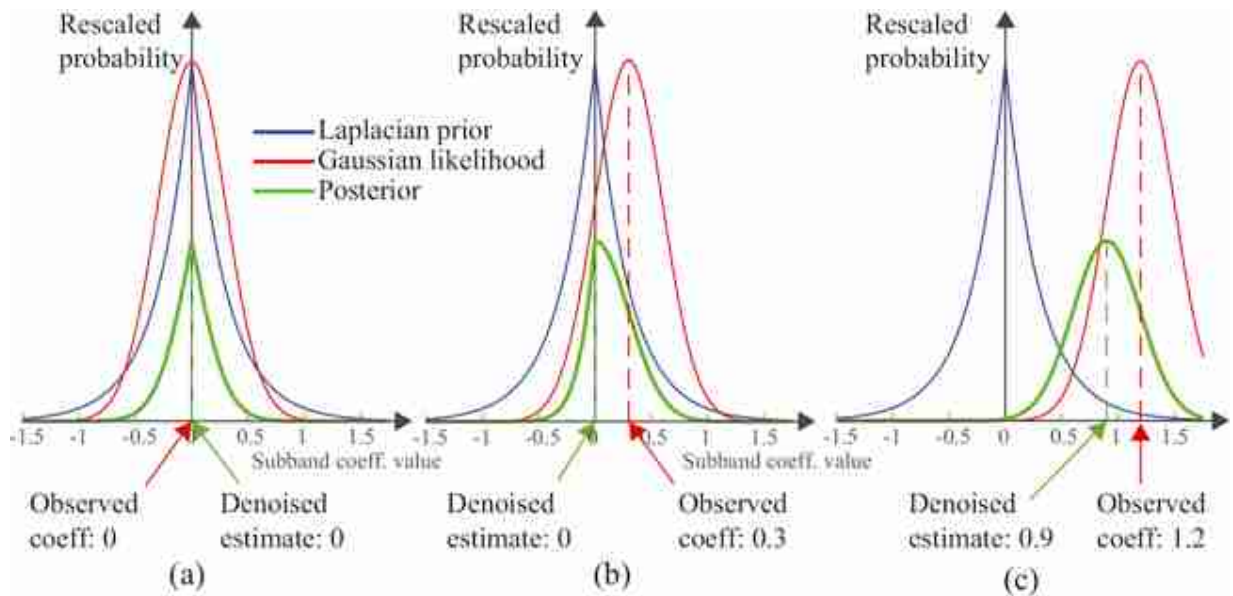


Figure 27.20: Showing the likelihood, prior, and posterior terms for the estimation of a subband coefficient from several noisy observations. (a) A zero subband coefficient is observed. (b) An observation of 0.26 shifts the likelihood term, but the posterior still has a peak at zero. (c) an observed coefficient value of 1.22 yields a maximum posterior probability estimate of 0.9.

[Figure 27.21](#) shows the resulting look-up table to modify the noisy observation to the MAP estimate for the denoised coefficient value. Note that this implements a **coring function** [440]: if a coefficient value is observed near zero, it is set to zero, based on the very high prior probability of zero coefficient values.

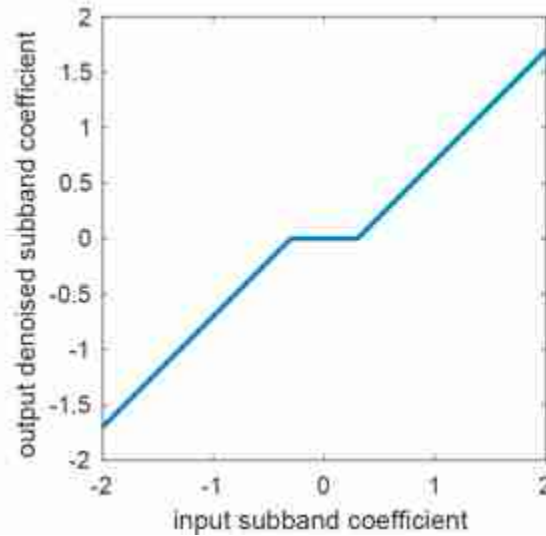


Figure 27.21: Input/output coring curve for maximum posterior denoising for the example of [figure 27.20](#).

For a more detailed analysis and examples of denoising using the Laplacian image prior, we refer the reader to the work of Eero Simoncelli [[440](#)].

27.7 Nonparametric Markov Random Field Image Models

The Gaussian and kurtotic wavelet image priors both involve modeling the image as a sum of statistically independent basis functions, Fourier basis functions, in the case of the Gaussian model, or wavelets, in the case of the wavelet image prior.

A powerful class of image models is known as **Markov random field** (MRF) models. They have a precise mathematical form, which we will describe later, in chapter 29. For now, we present the intuition for these models: if the values of enough pixels surrounding a given pixel are specified, then the probability distribution of the given pixel is independent of the other pixels in the image. To fit and sample from the MRF structure requires the mathematical tools of graphical models, to be introduced in chapter 29.

As we will see in the following chapter, Efros and Leung [115] introduced an algorithm for texture synthesis that exploits intuitions about the Markov structure of images to synthesize textures that are visually compelling. The intuition of the Efros and Leung texture generation model is that if you condition on a large enough region of nearby pixels around a center pixel, that effectively determines the value of the central pixel. We will study this algorithm in more detail in the next chapter devoted to texture analysis and synthesis (chapter 28). We will focus here on a similar algorithm for the problem of image denoising.

27.7.1 Denoising Model (Nonlocal Means)

Baodes, Coll, and Morel made use of that intuition to define the **nonlocal means** denoising algorithm [64]. The nonlocal means denoised image is a weighted sum of all other pixels in the image, weighted by the similarity of the local neighborhoods of each pair of pixels. More formally,

$$\ell_{\text{NLM}}[n, m] = \sum_{k, l} w_{n, m}[k, l] \ell[k, l] \quad (27.23)$$

where

$$w_{n, m}[k, l] = \frac{1}{Z_{n, m}} \exp \frac{-|\ell[N_{k, l}] - \ell[N_{n, m}]|^2}{h^2}, \quad (27.24)$$

where $N_{i, j}$ denotes an $h \times h$ region of pixels centered at i, j , and $Z_{n, m}$ is set so that the weights over the image sum to one, that is $\sum_{kl} w_{n, m}[k, l] = 1$.

[Figure 27.22](#) shows the weighting functions for two images. The parameter h is typically set to 10σ , where σ is the estimated image noise standard deviation. Within [figures 27.22\(a and b\)](#) at left is a grayscale image where the center position of interest to be denoised is marked with a white star. The right side of the figures shows for some value of h in equation (27.24) the weighting function $w_{n, m}[k, l]$ indicating which pixels are averaged together to yield the nonlocal mean value placed at the starred location in the denoised image. Note, for both [figures 27.22\(a and b\)](#) the sampling at image regions with similar local structure as the image position to be denoised.

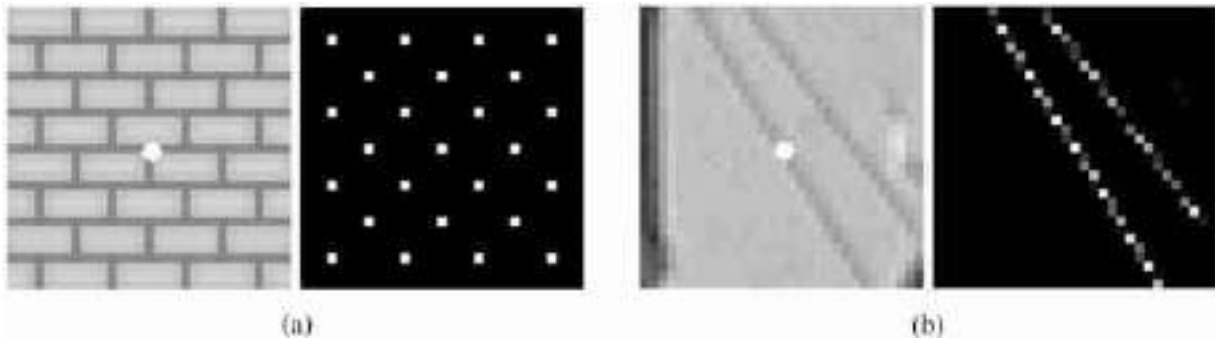


Figure 27.22: Regions of support for the nonlocal means denoising algorithm. *Source:* Image from [64].

[Figure 27.23](#) shows the result on a real image. On the left is the original image. The middle image is with additive Gaussian noise added, that is, $\sigma = 15$ for 0–255 image scale. On the right is the result, showing the image with nonlocal means noise removed.



Figure 27.23: Nonlocal means denoising algorithm results. (left) Original image without noise. (middle) Noisy image. (right) Denoised image. *Source:* Figure from [40].

27.8 Concluding Remarks

The models presented in this chapter do not try to understand the content of the pictures, but utilize low-level regularities of images in order to build a statistical model. Modeling these statistical regularities enable many tasks, including both image synthesis and image denoising.

We have seen some simple models that try to constrain the space of natural images. These models, despite their simplicity, are useful in several applications. However, they fail in providing a strong image model that can be used to sample new images. As illustrated in [figure 27.24](#) the sequence of models we have studied in this chapter, only help in reducing a bit the space of possible images.

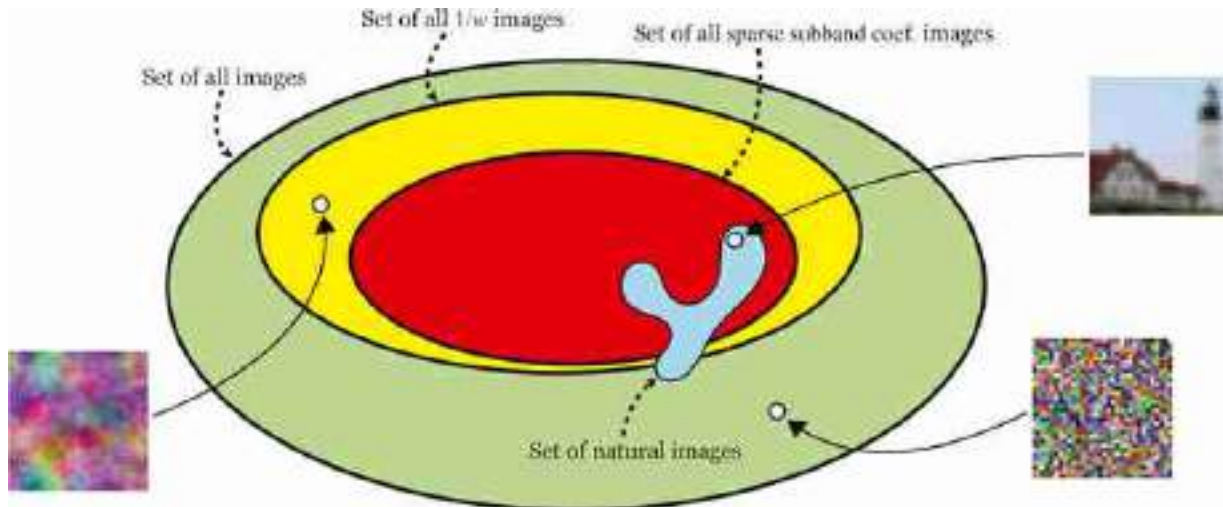


Figure 27.24: The space of natural images is just very small part of the space of all possible images. In the case of color images with 32×32 pixels, most of the space is filled with images that look like noise.

We will see stronger generative image models in chapter 32.

[11.](#) Additive stationary noise is noise with statistical properties independent of location that is added to an observation. Check section 27.5.2 for a more precise definition.

28 Textures

28.1 Introduction

The visual world is made of objects (chairs, cars, tables), surfaces (walls, floor, ceiling), and stuff (wood, grass, water). This chapter is about seeing stuff [8]. How do we build image representations that can capture what makes wood different from a wall of bricks?

Representing textures is a task, similar to color perception, that is intimately related to human perception. A texture is a type of image that is composed by a set of similar looking elements. A texture representation contains information about the statistics of its constituent elements, but not about the elements individually.

In this chapter we will introduce the problems of texture analysis and synthesis as a way to explore texture representations. The models presented here are precursors of more modern approaches using deep learning. But many of the concepts, and some of the intuitions about why these models might be successful, are better understood by exploring first simple yet powerful models.

Texture synthesis can be solved by a trivial algorithm as shown in [figure 28.1](#). But such an algorithm, although successful in practice, will give us little understanding on how human perception works and how to modify textures to create new ones. It will also not help us in understanding what representation can be useful to measure similarity between textures.

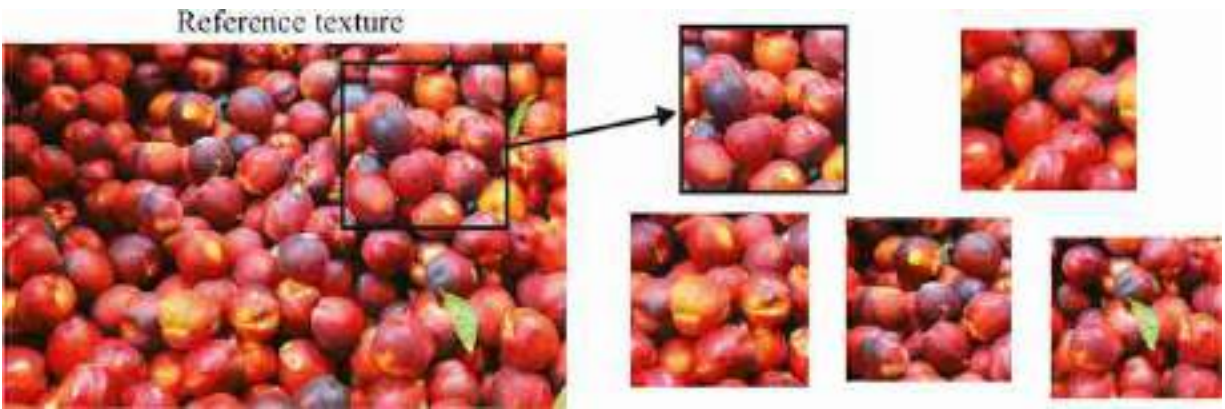


Figure 28.1: The infinite texture generation algorithm. If we had access to a very large image of the texture we want to generate, we could just crop pieces from it to create new images.

Before we dive into computational approaches to texture analysis, let's first build some intuitions about what might be plausible representations by looking into human perception.

28.2 A Few Notes About Human Perception

The study of texture perception is a large field in visual psychophysics and we will not try to review it here in detail. Instead, we will just describe three visual phenomena that will allow you getting a sense of the mechanisms underlying texture perception by your own visual system.

How many stones there are in this wall?



Most people will not care about the number of rocks in the wall, unless you are in the business of selling rocks. Most observers perceive this image as a rock texture. It appears to observers as a texture instead of a composition of countable rocks. They are indeed countable, but counting them requires a significant effort.

28.2.1 Perception of Sets

When an image is composed of one or a few distinct objects, we can easily pay attention to each of them individually. We can also count them in parallel, that is, we can tell if an image has one, two, three, or four elements, very quickly and the time it takes for us to say how many elements are in the display does not depend on the number of elements in the image. That is, if a display has only one element or four elements, we are equally fast at reporting that number ([figures 28.2](#)[a, b and c]).

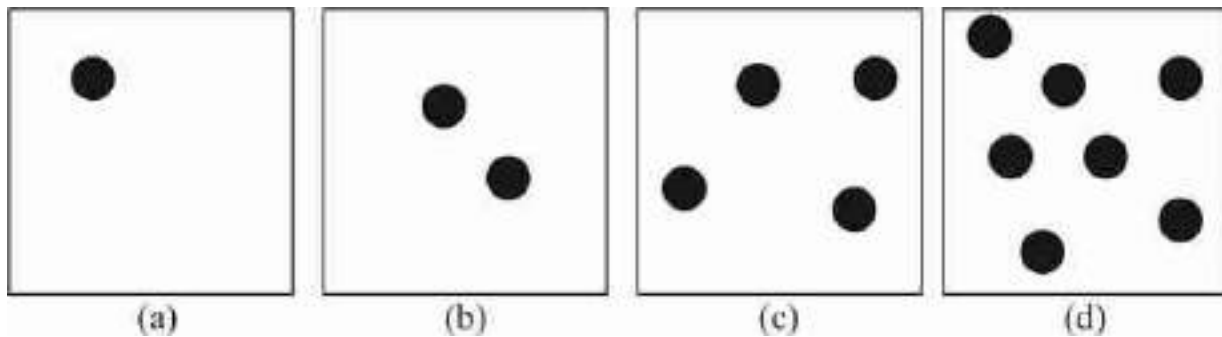


Figure 28.2: When looking at these images, we can count the number of circles at a glance if there are less than five circles. When an image has more than five items, we have to count them one by one.

But something interesting happens when images are composed of more than five similarly looking elements. If we want to count them we need to look at all of them one by one, and the time to count them grows linearly with the number of elements in the image, as illustrated in [figure 28.2](#).

The ability to know how many objects are in a display without counting them is called **subitizing** [[253](#)]. Subitizing only works when there are fewer than five elements in a display.

When there are more than four or five objects, we seem to pay attention only to a few of them and represent the rest of them as a set. The perception of sets is an important area of study in human visual psychophysics [[19](#)].

28.2.2 Crowding

[Figure 28.3](#) illustrates a curious visual phenomenon. Look at the central cross and try to read the letter on the left and right without moving your eyes.

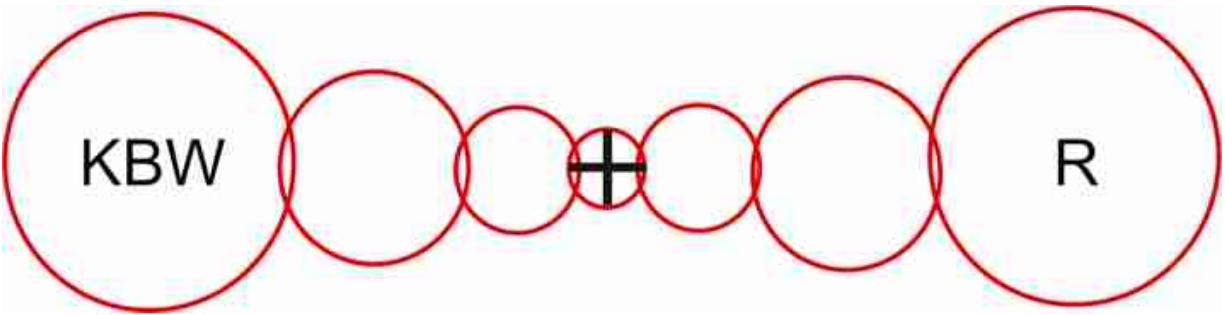


Figure 28.3: Crowding. If you look at the central cross, the letter R on the right can be recognized, however the letter B on the left is hard to read.

You will notice that the letter R on the right can be recognized easily while the letter B on the left is hard to read. The reason cannot be due to the low resolution of peripheral vision as both letters are at the same distance from the fixation location. The explanation of this phenomena is due to something called **crowding** [381].

As the letter B is surrounded by other letters, the features of the three letters get entangled as if the pooling window for image features was larger than the three letters together. While you fixate the cross, as you pay attention to the location of the B, you will notice that the features of the three letters are mixed. It is easy to see that it is text, but it is hard to bind the features together to perceive the individual letters. The circles in [figure 28.3](#) represent an approximation of the size of the visual regions over which visual features are pooled. Anything inside the region will suffer from crowding effects. Those regions are larger than what would be predicted by the loss of resolution due to the spacing between receptors in the periphery.

The main message here is for you to feel what it is to perceive what seems to be a statistical representation of the image features present in an image region.

28.2.3 Pre-Attentive Texture Discrimination

One of the first computational models of texture perception was due to Bela Julesz [240]. He proposed that textures are analyzed in terms of statistical relationships between fundamental texture elements that he called **textons**. It generally required a human to look at the texture in order to decide what those fundamental units were and his models were restricted to binary images formed by simple elements as shown in [figure 28.4](#).

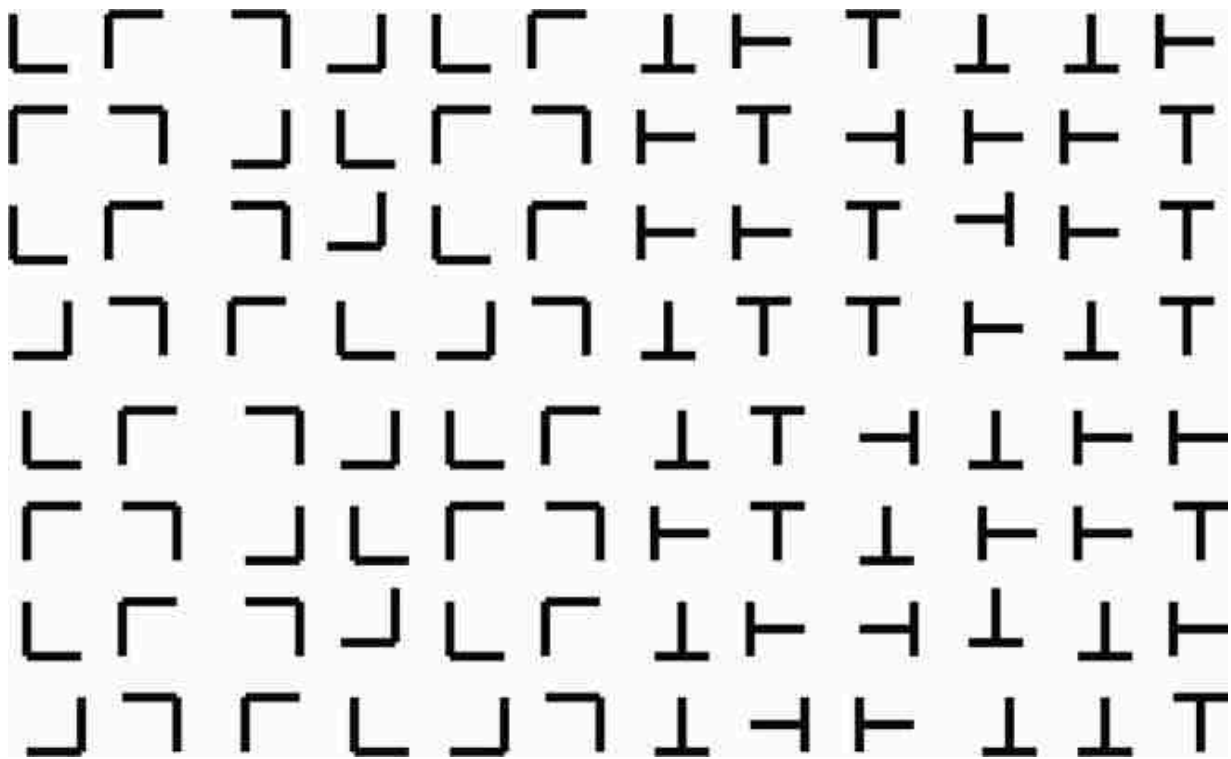


Figure 28.4: Texture discrimination using textons.

One interesting observation about [figure 28.4](#) is that it seems easy to see the boundary between the two textures in the image. Interestingly, not all textures can be easily discriminated pre-attentively (i.e., without making an effort). Bela Julesz and many others studied what makes two textures look similar or different in order to understand the nature of the visual representation used by the human visual system.

According to Bela Julesz, textons might be represented by features such as the number of junctions, terminators, corners, and intersections within the patterns. The challenge is that detecting those elements can be unreliable making the representation non-robust.

A different texture representation based on filter banks was introduced in parallel by Bergen and Adelson [46], and by Malik and Perona [315]. They showed that filters were capable of explaining several of the **pre-attentive texture discrimination** results, and that features based on filter outputs could be used to do texture-based image segmentation [315]. The texture model we will study in the next section is motivated by those works.

In the rest of this chapter we will study two texture analysis and synthesis approaches.

28.3 Heeger-Bergen Texture Analysis and Synthesis

The statistical characteristics of image subbands allow for powerful image synthesis and also denoising algorithms. Let's start with the problem of **texture synthesis**. The task is as follows: given a reference texture we want to build a process that outputs images with new random samples of the same texture, as shown in [figure 28.5](#). Texture synthesis is important for many computer graphics applications, and also as way of studying what representations of a texture can be useful for visual recognition.

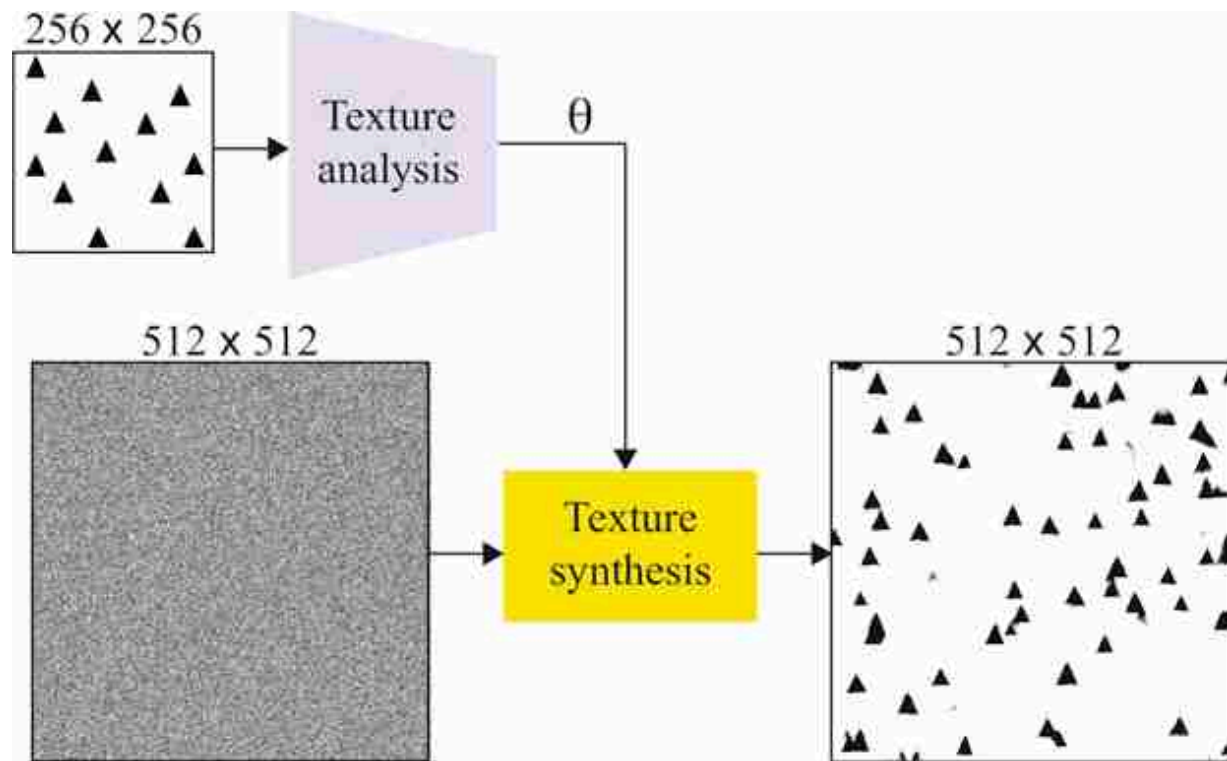


Figure 28.5: A reference texture image is transformed into a representation using a texture analysis (encoder). Then the texture synthesis procedure takes as input a random noise image of the size of the desired output texture and the parameters of the reference image θ .

An influential wavelet-based image synthesis algorithm is the Heeger-Bergen texture synthesis method published in 1995, with the following origin story [\[200\]](#): David Heeger heard Jim Bergen give a talk at a human vision conference about representations for image texture. Bergen

highlighted the value of measuring the mean-squared responses of filters applied to images for characterizing texture. He suggested that determining the full probability distribution of each filter's responses would be even more effective. He conjectured that if two textures produced the same distribution of filter responses for all filters, they would look identical to a human observer. Heeger disagreed and aimed to prove Bergen wrong by implementing the proposal using a steerable pyramid [441]. To Heeger's surprise, the first example he tried worked very well, and this led to Heeger and Bergen's influential paper on texture synthesis [202].

Texture generation has two stages depicted in [figure 28.5](#). **Texture analysis** of a reference texture extracts a texture representation, θ . **Texture synthesis** uses a representation, θ , to generate new random samples of the same texture. In the case of texture, it is difficult to precisely define what we mean by “same texture.” The perception of textures is an important topic in studies of visual human perception. In general, two textures will be perceived as similar when they seem to be generated by the same physical process.

An important research question is as follows: what is the minimal representation, θ , needed to generate textures that seem identical (i.e., appear as being originated by the same process) to human observers?

Texture synthesis is usually an iterative image synthesis procedure that begins with a random noise image and uses the parameters θ to produce a new improved image as shown in [figure 28.6](#). We will describe a similar algorithm in section 32.8 when talking about **diffusion models**.

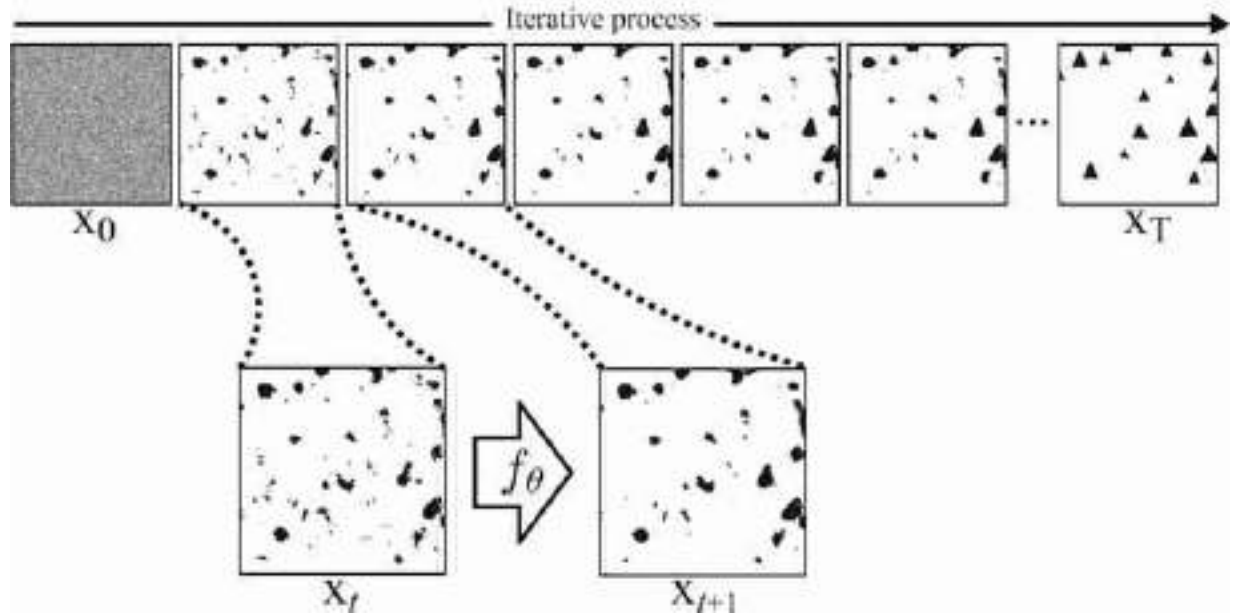


Figure 28.6: The steps of the Heeger-Bergen texture synthesis algorithm. The process starts with white noise input image. Each step takes as input the previous output, and it is modified by a function f_θ , where θ are the parameters describing a texture. At each step the output image x_t gets closer to the appearance of the reference texture ([figure 28.5](#)). The result of 500 iterations is shown in the right.

[Figure 28.7](#) shows the main idea behind the approach proposed by Heeger and Bergen to implement the analysis and synthesis functions. First, the reference texture is decomposed multiple orientation and scales as the outputs of many oriented filters over different spatial scales. The transform should represent all spatial frequencies of the image, so that all aspects of a texture can be synthesized, and the subbands should not be aliased, to avoid artifacts in the synthesized results. In the examples shown here we use a steerable pyramid [\[441\]](#) described in chapter 23 with six orientations and three scales. The final texture representation is the concatenation of the 18 subband histograms, the low-pass residual histogram and the input image histogram ([figure 28.7\[a\]](#)).

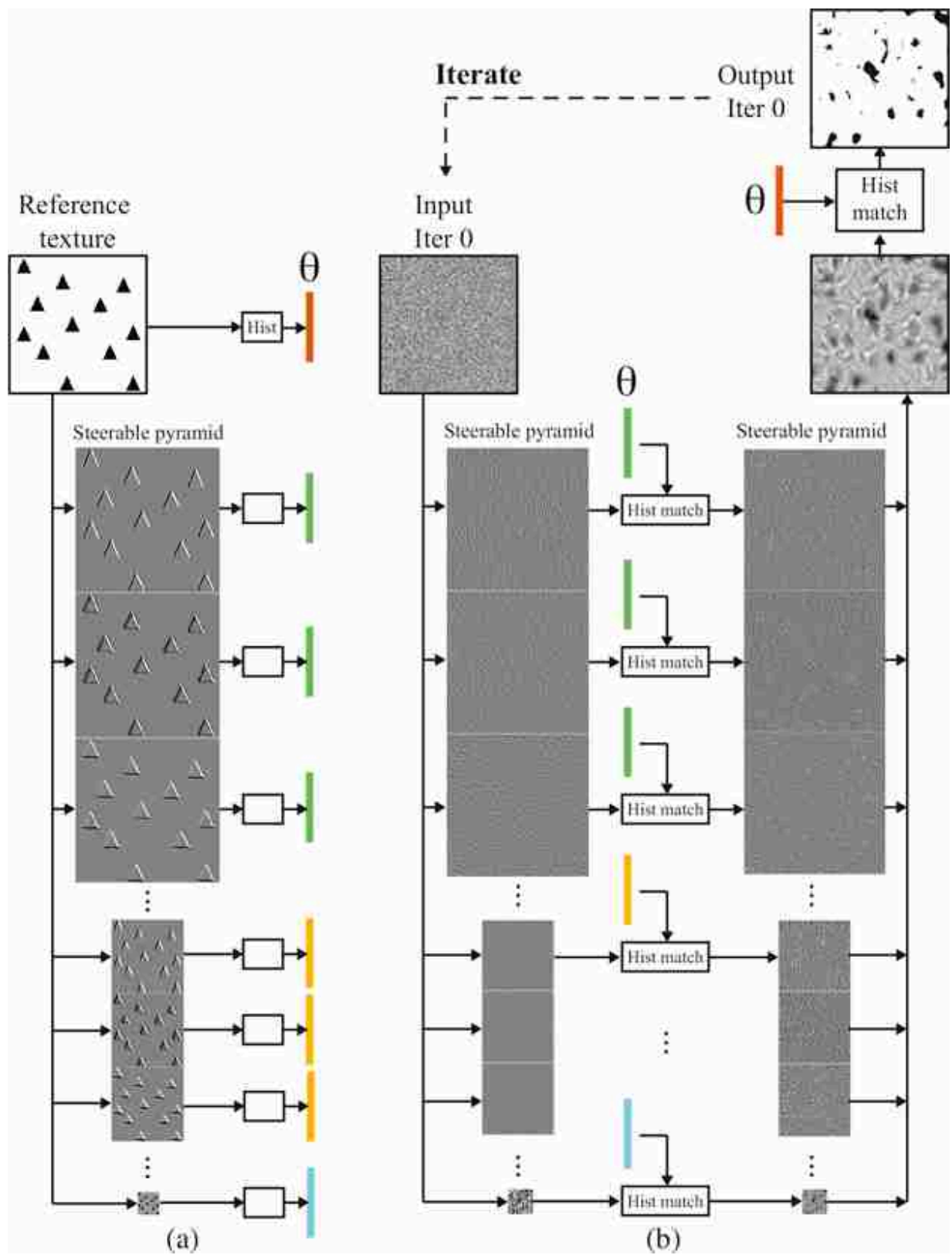


Figure 28.7: (a) Texture analysis (encoder) using a steerable pyramid with six orientations and three scales. The output representation is the concatenation of the 18 subband histograms, the low-pass residual histogram and the input image histogram. (b) Texture synthesis, only one iteration shown. At each iteration, the output is put back as input and the process is repeated N times. The diagram corresponds to the implementation of the function f_θ from [figure 28.6](#).

The crux of the algorithm is to alternate matching the intensity domain pixel histogram of the image, then to transform the resulting image into the transform domain and enforce a match, subband by subband, of the image histograms there.

Histogram matching is implemented by a pointwise monotonic nonlinearity.

[Figure 28.8](#) shows how one of the subbands gets transformed by the **histogram matching** procedure. The histogram of the subband of the reference texture has the Laplacian shape typical of natural images. Most of the values of the output of the oriented filter are zero, and only near the triangle boundaries the values are non-zero. In the first iteration, the texture synthesis process starts with white noise (each pixel is sampled independently from a Gaussian distribution). The subband of the input noise has also a Gaussian distribution (as a linear combination of Gaussian random variables also follows a Gaussian distribution). Histogram matching modifies the values of each subband pixel (the histogram matching function is a pixelwise nonlinearity that depends on the current and target pixel histograms). After histogram matching, the noise subband looks sparser, with many values set to zero. The same operation is done for all the subbands in the steerable pyramid.

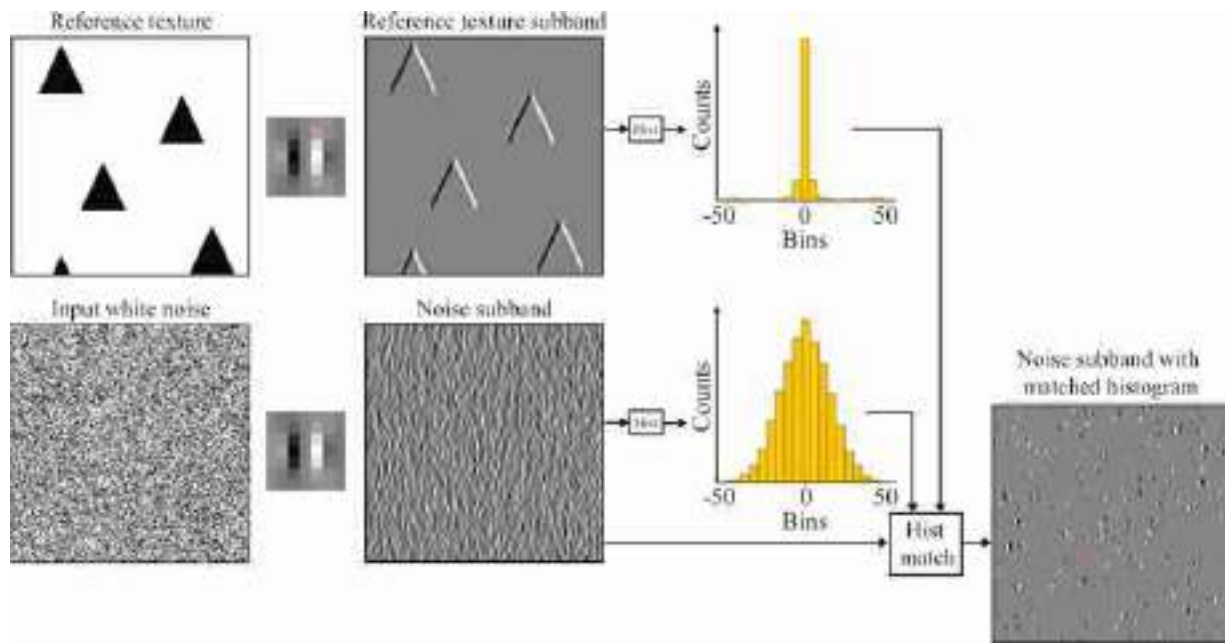


Figure 28.8: Histogram matching for one of the subbands. The same operation is done for all the subbands independently.

[Figure 28.7\(b\)](#) shows the result of applying this to all the subbands and then collapsing the pyramid to reconstruct an image. Then we also histogram match the output texture to match the histogram of the pixel values of the reference texture. The result is an image that starts showing some initial black blobs that, after repeating the same process multiple times, will become triangles, as shown in [figure 28.6](#). After several iterations of the histogram matching steps in both domains (subbands and image) the algorithm appears to converge and the result is the synthesized texture.

The results are often quite good. For images like the triangles of [figure 28.5](#), the algorithm works well. However, we note that correlations of filter responses across scale is not captured in this procedure, and long or large-scale structures are often depicted poorly. [Figure 28.9](#) shows two additional examples with more complex images. To process color images, we first do principal component analysis (PCA) in color space to find decorrelated color components, then we apply the texture analysis-synthesis to each decorrelated channel independently. The final image is produced by projecting back into the original color space. For the results in [figure 28.9](#),

the algorithm is run only for 15 iterations. Adding more iterations makes the images look worse.



Figure 28.9: Two examples of synthesized textures. Inputs have a size of 256×256 pixels, outputs are 512×512 . In these examples, the algorithm runs for 15 iterations, using a pyramid of six orientation and four scales.

Texture models using statistics of filter outputs are a precursor to diffusion models (section 32.8) and share a number of architectural features.

The Heeger and Bergen model was one of the first successful approaches that used filter outputs to represent arbitrary textures. Other models followed after that introducing stronger statistical representations capable of generating higher quality textures [392].

Parametric texture models represent a texture image with a small vector of parameters, usually around 1,000 dimensions [392].

These models, usually called **parametric texture models**, stayed popular until 2000. After that, **non-parametric texture models** became dominant (section 28.4) until generative models with neural networks emerged around 2014.

28.4 Efros-Leung Texture Analysis and Synthesis Model

Nonparametric Markov random field image models, introduced earlier in chapter 27, define the probability distribution of a pixel as dependent of neighborhood around the pixel. As we said before, if the values of enough

pixels surrounding a given pixel are specified, then the probability distribution of the given pixel is independent of the other pixels in the image. This image model can be used as a powerful texture synthesis model [115].

As with other texture synthesis algorithms, the input to the Efros-Leung algorithm [115] is an example of the texture to be synthesized, and the output is an image of more of that texture. It is an iterative algorithm as shown in [figure 28.10](#).

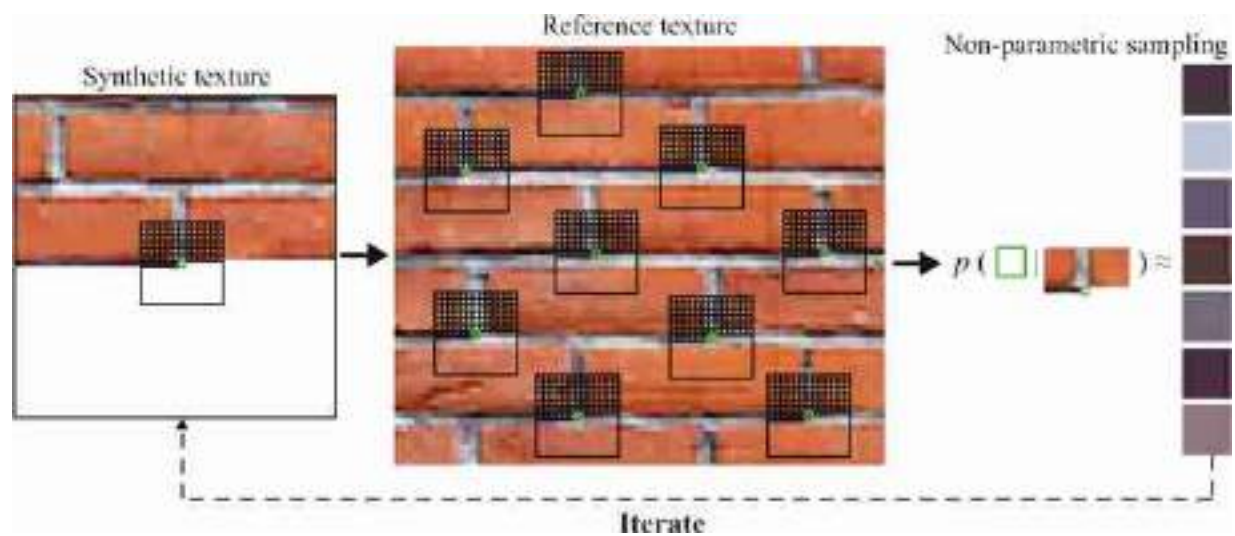


Figure 28.10: In the Efros-Leung algorithm [115], a new pixel in the synthetic texture is created by searching the pixel's neighborhood for similar parts in the reference texture. From N retrieved regions, the central pixels determine the color distribution of the new pixel, generated by randomly selecting one value.

At each iteration, we have synthesized pixels in the neighborhood of the pixel to be synthesized. We look through the input image for examples of similar configurations of pixel values as the current local neighborhood, having a sum of squared intensity differences from the local neighborhood lower than some threshold. Randomly select one of the matching instances, and copy the value of the pixel into the pixel to be synthesized. Repeat for all pixels to be synthesized. It can be natural to synthesize the pixels in a raster-scan ordering. Image pixels without sufficient neighboring pixels for context can be randomly sampled from the input texture.

In this nonparametric model, the representation of a texture is the reference image itself. There is not a low-dimensional representation as it is the case of parametric texture models. The advantage is that no information is lost in this representation.

[Figure 28.11](#) shows how the synthesized texture from the Efros-Leung algorithm [115] changes as a function of size of the context region. [Figure 28.11\(a\)](#) shows the source texture; [figure 28.11\(b\)](#) shows how a very small context region (the grid of [figure 28.10](#)) only covers a part of each circle and the synthesized texture doesn't show complete circles; [figure 28.11\(c\)](#) shows how a larger context region yields a synthesized texture that maintains the circles, but not the regularity of their spacing; and [figure 28.11\(d\)](#) shows how a still larger context region enforces all the spatial regularities of the input texture.

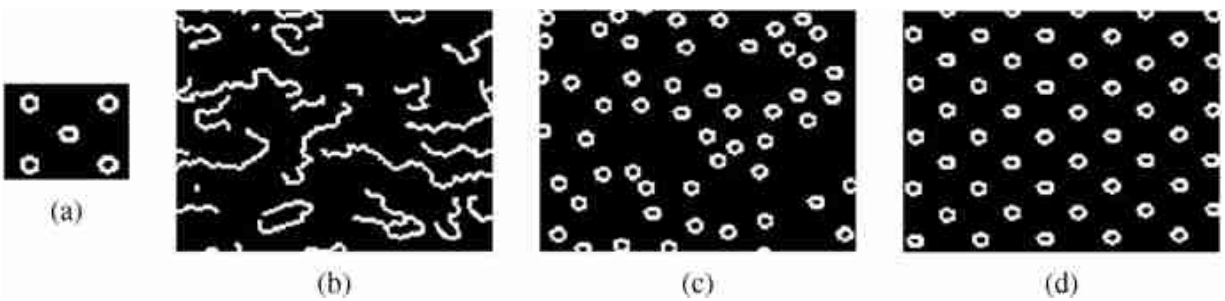


Figure 28.11: Synthesized texture from the Efros-Leung algorithm [115] as a function of size of the context region.

[Figure 28.12](#) shows additional image synthesis results. It is remarkable that such a simple model works so well. The results seem better than the ones shown in [figure 28.9](#); however, note that there can still be artifacts (such as the very big leaf that appear in the output). Increasing the neighborhood size can remove those issues. Also, we can see that some elements of the reference texture might appear multiple times in the output texture. Removing repetitions requires decreasing the neighborhood size.



Figure 28.12: Image synthesis results from the Efros-Leung algorithm. The small crops are used to synthesize the larger texture regions. Input images are 128×128 pixels, and the outputs are 256×256 pixels. The neighbourhood size is 17×17 pixels.

Embellishments have been developed that account for structures over scale [98] or which are more efficient by operating on patches [114, 35].

28.5 Connection to Deep Generative Models

In chapters 32 and 33 we will learn about generative models for image synthesis. These chapters cover models that generate images of all kinds, not just textures. However, they have strong similarities to the texture synthesis models we saw in this chapter. A few of connections are as follows:

- The Efros-Leung algorithm is an **autoregressive model**. Section 32.7 covers this class of models in more generality, and shows how to use them in conjunction with deep nets.
- The iterative optimization of the Heeger-Bergen algorithm is a type of denoising process: it starts with a noise image and little by little transforms it into an image of a texture. This general idea reappears in section 32.8 on **diffusion models**, which follow this same strategy.
- The Heeger-Bergen algorithm is also connected to **generative adversarial networks (GANs)**, which we will encounter in section 32.9. The objective of Heeger-Bergen is to make a texture that has the same histogram statistics as an exemplar. A GAN also makes images that have the same statistics as a set of exemplars; however, the statistics

we seek to match are learned (by a so-called discriminator neural net) rather than hand-defined to be subband statistics.

- More generally, texture modeling is about modeling a *distribution over patches*, while generative modeling is about modeling a *distribution over whole images*. The tools are similar in both cases but the scope differs in this way.

Be on the lookout for more connections; all generative models are intimately related to each other and it is often possible to frame any one generative model as a special case or extension of any other.

28.6 Concluding Remarks

There is an important relationship between texture models and human perception. One of the goals of many vision algorithms is to compute representations that reproduce image similarities relevant to humans. The representations might be the result of constraining the models to learn human preferences, or as a byproduct of unsupervised learning techniques that result in representations that correlate with human perception.

The methods described in this chapter are the basis to understand some of the latest image generative models.

29 Probabilistic Graphical Models

29.1 Introduction

Probabilistic graphical models describe joint probability distributions in a modular way that allows us to reason about the visual world even when we're modeling very complicated situations. These models are useful in vision, where we often need to exploit modularity to make computations tractable.

A probabilistic graphical model is a graph that describes a class of probability distributions sharing a common structure. The graph has nodes, drawn as circles, indicating the variables of the joint probability. It has edges, drawn as lines connecting nodes to other nodes. At first, we'll restrict our attention to a type of graphical model called undirected, so the edges are line segments without arrowheads.

In undirected graphical models, the edges indicate the conditional independence structure of the nodes of the graph. If there is no edge between two nodes, then the variables described by those nodes are independent, conditioned on the values of intervening nodes in any path between them in the graph. If two nodes do have a line between them, then we cannot assume they are independent. We introduce this through a set of examples.

29.2 Simple Examples

Here is the simplest probabilistic graphical model:



Figure 29.1: A graphical model with only one node.

This **graph** is just a single node for the variable x_1 . There is no restriction imposed by the graph on the functional form of its probability function. In keeping with notation we'll use later, we write that the probability distribution over x_1 , $p(x_1) = \phi_1(x_1)$.

Another trivial graphical model is this:



Figure 29.2: Two independent variables.

Here, the lack of a line connecting x_1 with x_2 indicates a lack of statistical dependence between these two variables. In detail, we condition on knowing the values of any connecting neighbors of x_1 and x_2 (in this case there are none) and thus x_1 and x_2 are statistically independent. Because of that independence, the joint probability depicted by this graphical model must be a product of each variable's marginal probability and thus must have the form: $p(x_1, x_2) = \phi_1(x_1)\phi_2(x_2)$.

Let's add a line between these two variables:



Figure 29.3: Two dependent variables.

By that graph, we mean that there may be a statistical dependency between the two variables. The class of probability distributions depicted

here is now more general than the one above, and we can only write that the joint probability as some unknown function of x_1 and x_2 , $p(x_1, x_2) = \phi_{12}(x_1, x_2)$. This graph offers no simplification from the most general probability function of two variables.

Here is a graph with some structure:

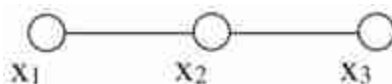


Figure 29.4: Three dependent variables.

This graph means that if we condition on the variable x_2 , then x_1 and x_3 are independent. A general form for $p(x_1, x_2, x_3)$ that guarantees such conditional independence is

$$p(x_1, x_2, x_3) = \phi_{12}(x_1, x_2)\phi_{23}(x_2, x_3), \quad (29.1)$$

Note the conditional independence implicit in equation (29.1): if the value of x_2 is given (indicated by the solid circle in the graph below), then the joint probability of equation (29.1) becomes a product of some function $f(x_1)$ times some other function $g(x_3)$, revealing the conditional independence of x_1 and x_3 . Graphically, we denote the conditioning on the variable x_2 with a filled circle for that node:



Figure 29.5: Conditioning on the variable x_2 is indicated by the filled circle.

We will exploit that structure of the joint probability to perform inference efficiently using an algorithm called belief propagation (BP).

A celebrated theorem, the **Hammersley-Clifford theorem** [182], tells the form the joint probability must have for any given graphical model. The joint probability for a probabilistic graphical model must be a product of

functions of each the **maximal cliques** of the graph. Here we define the terms.

A **clique** is any set of nodes where each node is connected to every other node in the clique. These graphs illustrate the clique property:

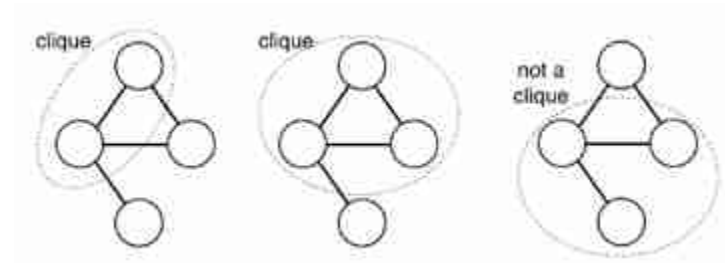


Figure 29.6: A clique is any set of nodes where each node is connected to every other node in the same clique.

A **maximal clique** is a clique that can't include more nodes of the graph without losing the clique property. The sets of nodes below form maximal cliques (left), or do not (right):

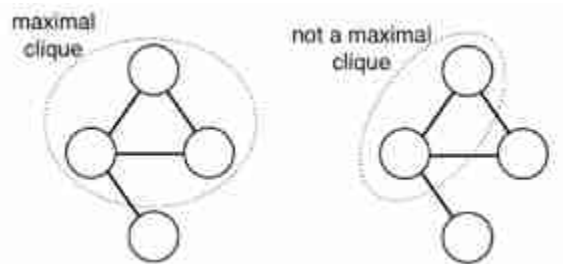


Figure 29.7: A maximal clique is the largest possible clique.

The **Hammersley-Clifford theorem** [182] states that a positive probability distribution has the independence structure described by a graphical model if and only if it can be written as a product of functions over the variables of each maximal clique:

$$p(x_1, x_2, \dots, x_N) = \prod_{x_c \in \mathcal{C}} \psi_c(x_c), \quad (29.2)$$

where the product is over all maximal cliques x_c in the graph, x_i .

In the example of graph in [figure 29.4](#), the maximal cliques are (x_1, x_2) and (x_2, x_3) , so the Hammersley-Clifford theorem says that the corresponding joint probability must be of the form of equation (29.1).

Now we examine some graphical model structures that are especially useful in vision. In perception problems, we typically have both observed and unobserved variables. The graph below shows a simple **Markov chain** [[151](#)] structure with three observed variables, shaded, labeled y_i , and three unobserved variables, labeled x_i :

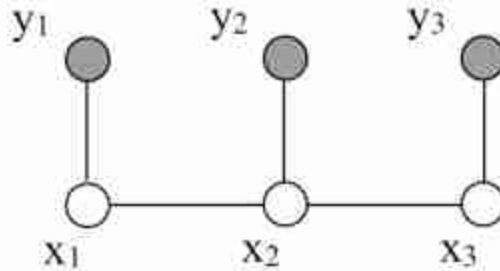


Figure 29.8: Markov chain with three observed variables, shaded, and three unobserved variables.

This is a chain because the variables form a linear sequence. It's a Markov chain structure because the hidden variables have the Markov property: conditional on x_2 , variable x_3 is independent of variable x_1 . The joint probability of all the variables shown here is $p(x_1, x_2, x_3, y_1, y_2, y_3) = \phi_{12}(x_1, x_2)\phi_{23}(x_2, x_3)\psi_1(y_1, x_1)\psi_2(y_2, x_2)\psi_3(y_3, x_3)$. Using $p(a, b) = p(a | b)p(b)$, we can also write the probability of the x variables conditioned on the observations y ,

$$p(x_1, x_2, x_3 | y_1, y_2, y_3) = \frac{1}{p(\mathbf{y})} \phi_{12}(x_1, x_2)\phi_{23}(x_2, x_3)\psi_1(y_1, x_1)\psi_2(y_2, x_2)\psi_3(y_3, x_3). \quad (29.3)$$

For brevity, we write $p(\mathbf{y})$ for $p(y_1, y_2, y_3)$. Thus, to form the conditional distribution, within a normalization factor, we simply include the observed variable values into the joint probability. For vision applications, we often use such Markov chain structures to describe events over time.

To capture relationships over space, a two-dimensional (2D) structure is useful, called a **Markov random field** (MRF) [[53](#)]:

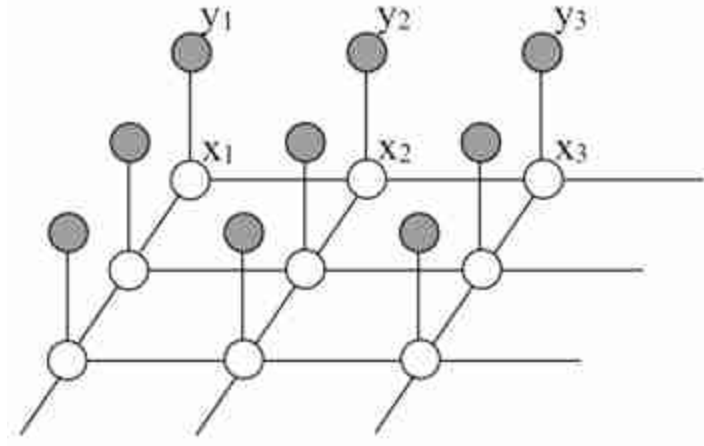


Figure 29.9: Two-dimensional Markov random field.

Then the joint probability over all the variables factorizes into this product:

$$p(\mathbf{x} | \mathbf{y}) = \frac{1}{p(\mathbf{y})} \prod_{(i,j)} \phi_{ij}(x_i, x_j) \prod_i \psi_i(x_i, y_i), \quad (29.4)$$

where the first product is over all spatial neighbors i and j , and the second product is over all nodes i .

An example of how Markov random field models are applied to images is depicted in [figure 29.10](#), which show a small image region and its context in the image ([figure 29.10\[a\]](#)). [Figure 29.10\(b\)](#) shows an MRF with each node corresponding to a pixel of the image region. The states of an MRF can be indicator variables of image segment membership for each pixel. The states of a hypothetical most-probable configuration of this MRF are shown as the color of each node in this illustration.

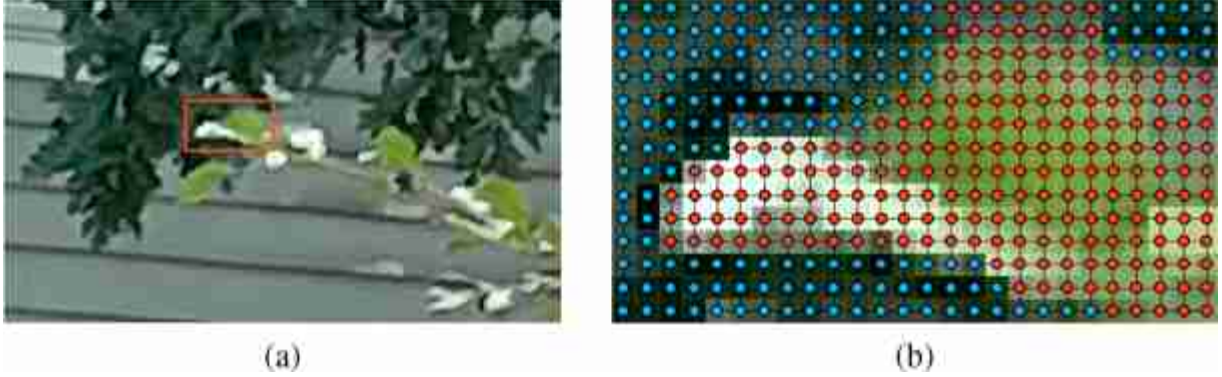


Figure 29.10: (a) Image to be segmented and a local region of (a) marked in red. (b) Visualization of an MRF of nodes corresponding to image pixels inside the marked region, with states indicating segment membership, for a hypothetical most probable configuration.

29.3 Directed Graphical Models

In addition to undirected graphical models, another type of graphical model is commonly used. **Directed graphical models** [274] describe factorizations of the joint probability into products of conditional probability distributions. Each node in a directed graph contributes a well-specified factor in the joint probability: the probability of its variable, conditioned all the variables originating arrows pointing into it. So this graph

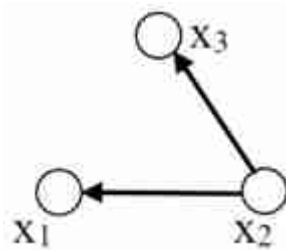


Figure 29.11: A directed graphical model with three variables.

denotes the joint probability,

$$p(x_1, x_2, x_3) = p(x_2)p(x_1 | x_2)p(x_3 | x_2) \quad (29.5)$$

The general rule for writing the joint probability described by a directed graph is this: Each node, x_n , contributes the factor $p(x_n | x_{\Xi})$, where Ξ is the set of nodes with arrows pointing in to node x_n . You can verify that equation

(29.5) follows this rule. Directed graphical models are often used to describe causal processes.

29.4 Inference in Graphical Models

Given a probabilistic graphical model and observations, we want to estimate the states of the unobserved variables. For example, given image observations, we may want to estimate the pose of the person. The **belief propagation** algorithm lets us do that efficiently.

Recall the Bayesian inference task: our observations are the elements of a vector, $\mathbf{y}$, and we seek to infer the probability $p(\mathbf{x} \mid \mathbf{y})$ of some scene parameters, $\mathbf{x}$, given the observations. By Bayes' theorem, we have

$$p(\mathbf{x} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \mathbf{x})p(\mathbf{x})}{p(\mathbf{y})} \quad (29.6)$$

The **likelihood term** $p(\mathbf{y} \mid \mathbf{x})$ describes how a rendered scene $\mathbf{x}$ generates observations $\mathbf{y}$. The **prior probability**, $p(\mathbf{x})$, tells the probability of any given scene $\mathbf{x}$ occurring. For inference, we often ignore the denominator, $P(\mathbf{y})$, called the **evidence**, as it is constant with respect to the variables we seek to estimate, $\mathbf{x}$.

The statistician Harold Jeffreys wrote, “Bayes’ theorem is to the theory of probability what Pythagorus’ theorem is to geometry” [236]

Typically, we make many observations, $\mathbf{y}$, of the variables of some system, and we want to find the the state of some hidden variable, $\mathbf{x}$, given those observations. The posterior probability, $p(\mathbf{x} \mid \mathbf{y})$, tells us the probability for any value of the hidden variables, $\mathbf{x}$. From this posterior probability, we often want some single *best* estimate for $\mathbf{x}$, denoted $\hat{\mathbf{x}}$ and called a point estimate.

Selecting the best estimate $\hat{\mathbf{x}}$ requires specifying a penalty for making a wrong guess. If we penalize all wrong answers equally, the best strategy is to guess the value of $\mathbf{x}$ that maximizes the posterior probability, $p(\mathbf{x} \mid \mathbf{y})$ (because any other explanation $\hat{\mathbf{x}}$ for the observations $\mathbf{y}$ would be less probable). That is called the **maximum a posteriori** (MAP) estimate.

But we may want to penalize wrong answers as a function of how far they are from the correct answer. If that penalty function is the squared distance in $\mathbf{x}$, then the point estimate that minimizes the average value of

that error is called the **minimum mean squared error** estimate (MMSE). To find this estimate, we seek the $\hat{\mathbf{x}}$ that minimizes the squared error, weighted by the probability of each outcome:

$$\hat{\mathbf{x}}_{MMSE} = \operatorname{argmin}_{\hat{\mathbf{x}}} \int_{\mathbf{x}} p(\mathbf{x} | \mathbf{y}) (\mathbf{x} - \hat{\mathbf{x}})' (\mathbf{x} - \hat{\mathbf{x}}) d\mathbf{x} \quad (29.7)$$

Differentiating with respect to $\mathbf{x}$ to solve for the stationary point, the global minimum for this convex function, we find

$$\hat{\mathbf{x}}_{MMSE} = \int_{\mathbf{x}} \mathbf{x} p(\mathbf{x} | \mathbf{y}) d\mathbf{x}. \quad (29.8)$$

Thus, the minimum mean square error estimate, $\hat{\mathbf{x}}_{MMSE}$, is the mean of the posterior distribution. If $\mathbf{x}$ represents a discretized space, then the marginalization integrals over $d_{\mathbf{x}}$ become sums over the corresponding discrete states of $\mathbf{x}$.

For a Gaussian probability distribution, the MAP estimate and the MMSE estimate are always the same, since the mean of the distribution is always its maximum value.

For now, we'll assume we seek the MMSE estimate. By the properties of the multivariate mean, to find the mean at each variable, or node, in a network, we can first find the marginal probability at each node, then compute the mean of each marginal probability. In other words, given $p(\mathbf{x} | \mathbf{y})$, we will compute $p(x_i | \mathbf{y})$, where i is the i -th hidden node. From $p(x_i | \mathbf{y})$ it is simple to compute the posterior mean at node i .

For the case of discrete variables, we compute the marginal probability at a node by summing over the states at all the other nodes,

$$p(x_i | \mathbf{y}) = \sum_{j \setminus i} \sum_{\mathbf{s}_j} p(x_1, x_2, \dots, x_i, \dots, x_N | \mathbf{y}), \quad (29.9)$$

where the notation $j \setminus i$ means “all possible values of j except for $j = i$.”

29.5 Simple Example of Inference in a Graphical Model

To gain intuition, let's calculate the marginal probability for a simple example. Consider the three-node Markov chain of [figure 29.12](#).

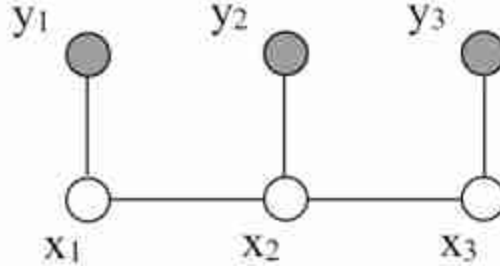


Figure 29.12: Same three-node Markov chain of [figure 29.8](#).

Vision tasks this can apply to include modeling the probability of a pixel belonging to an object edge, given the evidence, observations $\mathbf{y}$, at a point and two neighboring locations. The inferred states $\mathbf{x}$ could be a label indicating the presence of an edge.

We seek to marginalize the joint probability in order to find the marginal probability at node 1, $p(x_1 | \mathbf{y})$, given observations y_1, y_2 , and y_3 , which we denote as $\mathbf{y}$. We have, assuming the nodes have discrete states,

$$p(x_1 | \mathbf{y}) = \sum_{x_2} \sum_{x_3} p(x_1, x_2, x_3 | \mathbf{y}) \quad (29.10)$$

Here's the main point. If we knew nothing about the structure of the joint probability $p(x_1, x_2, x_3 | \mathbf{y})$, the computation would require $|x|^3$ computations: a double-sum over all x_2 and x_3 states must be computed for each possible x_1 state (we're denoting the number of states of any of the x variables as $|x|$). In the more general case, for a Markov chain of N nodes, we would need $|x|^N$ summations to compute the desired marginal at any node of the chain. Such a computation quickly becomes intractable as N grows.

But we can exploit the model's structure to avoid the exponential growth of the computation with N . Substituting the joint probability, from the graphical model, into the marginalization equation, equation (29.10), gives

$$p(x_1 | \mathbf{y}) = \frac{1}{p(\mathbf{y})} \sum_{x_2} \sum_{x_3} \phi_{12}(x_1, x_2) \phi_{23}(x_2, x_3) \psi_1(y_1, x_1) \psi_2(y_2, x_2) \psi_3(y_3, x_3) \quad (29.11)$$

This form for the joint probability reveals that not every variable is coupled to every other one. We can pass summations through variables they don't sum over, letting us compute the marginalization much more efficiently.

This will make only a small difference for this short chain, but it makes a huge difference for longer ones. So we write

$$p(x_1 | y) = \frac{1}{p(y)} \sum_{x_2} \sum_{x_3} \phi_{12}(x_1, x_2) \phi_{23}(x_2, x_3) \psi_1(y_1, x_1) \psi_2(y_2, x_2) \psi_3(y_3, x_3) \quad (29.12)$$

$$= \frac{1}{p(y)} \psi_1(y_1, x_1) \sum_{x_2} \phi_{12}(x_1, x_2) \psi_2(y_2, x_2) \sum_{x_3} \phi_{23}(x_2, x_3) \psi_3(y_3, x_3) \quad (29.13)$$

That factorization of equation (29.13) is the key step. It reduces the number of terms summed from order $|x|^3$ to order 2 $|x|^2$ for this chain, and for a chain of length N chain, from order $|x|^N$ to order $(N - 1) |x|^2$ —from exponential to linear dependence on N —a huge computational savings for large N .

The partial sums of equation (29.13) are named **messages** because they pass information from one node to another. We call the message from node 3 to node 2, $m_{32}(x_2) = \sum_{x_3} \phi_{23}(x_2, x_3) m_{63}(x_3)$. The other partial sum is the message from node 2 to node 1, $m_{21}(x_1) = \sum_{x_2} \phi_{12}(x_1, x_2) m_{52}(x_2) m_{32}(x_2)$. Note that the messages are always messages about the states of the node that the message is being sent to, that is, the arguments of the message m_{ij} are the states x_j of node j . The algorithm corresponding to equation (29.13) is called **belief propagation**.

Equation (29.13) gives us the marginal probability at node 1. To find the marginal probability at another node, we can write out the sums over variables needed for that node, pass the sums through factors in the joint probability that they don't operate on, to come up with an efficient reorganization of summations analogous to equation (29.13). We would find that many of the summations from marginalization at the node x_1 would need to be recomputed for the marginalization at node x_2 . That motivates storing and reusing the messages, which the belief propagation algorithm does in an optimal way.

29.6 Belief Propagation

For more complicated graphical models, we want to replace the manual factorization above with an automatic procedure for identifying the computations needed for marginalizations and to cache them efficiently. BP does that by identifying those reusable sums, that is, the messages.

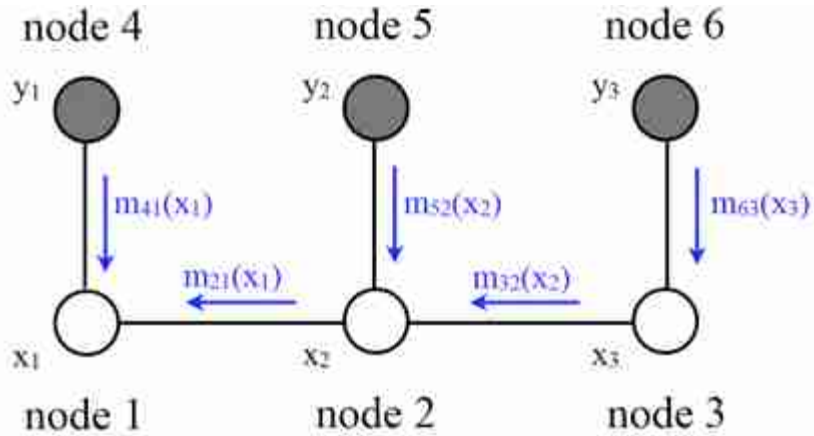


Figure 29.13: Summary of the messages (partial sums) for a simple belief propagation example.

29.6.1 Derivation of Message-Passing Rule

We'll describe belief propagation only for the special case of graphical models with pairwise potentials. The clique potentials between neighboring nodes are $\psi_{ij}(x_j, x_i)$. Extensions to higher-order potentials is straightforward. (Convert the graphical model into one with only pairwise potentials. This can be done by augmenting the state of some nodes to encompass several others, until the remaining nodes only need pairwise potential functions in their factorization of the joint probability.) You can find formal derivations of belief propagation in [239, 274].

Consider [figure 29.14](#), showing a section of a general network with pairwise potentials. There is a network of $N + 1$ nodes, numbered 0 through N and we will marginalize over nodes $x_1 \dots x_N$. [Figure 29.14\(a\)](#) shows the marginalization equation for a network of variables with discrete states. (For continuous variables, integrals replace the summations). If we assume the nodes form a tree, we can distribute the marginalization sum past nodes for which the sum is a constant value to obtain the sums depicted in [figure 29.14\(b\)](#).

We define a **belief propagation message**: A message m_{ij} , from node i to node j , is the sum of the probability over all states of all nodes in the subtree that leaves node i and does not include node j .

Referring to [figure 29.14\(a\)](#), shows the desired marginalization sum over a network of pairwise cliques. We can pass the summations over node states through nodes that are constant over those summations, arriving at the

factorization shown in [figure 29.14\(b\)](#). Remembering that the message from node j to node i is the sum over the tree leaving node j , we can then read-off the recursive belief propagation message update rule by marginalizing over the state probabilities at node j in [figure 29.14](#). As illustrated in [figure 29.14](#), messages $m_{j+1,j}$ and m_{kj} correspond to partial sums over nodes indicated in the graph: $m_{j+1,j} = \sum_{x_{j+1}, \dots, x_{k-1}}$ and $m_{kj} = \sum_{x_k, \dots, x_N}$. Marginalization over the states of all nodes leaving node j , not including node i , leads to equation (29.14), the belief propagation message passing rule.

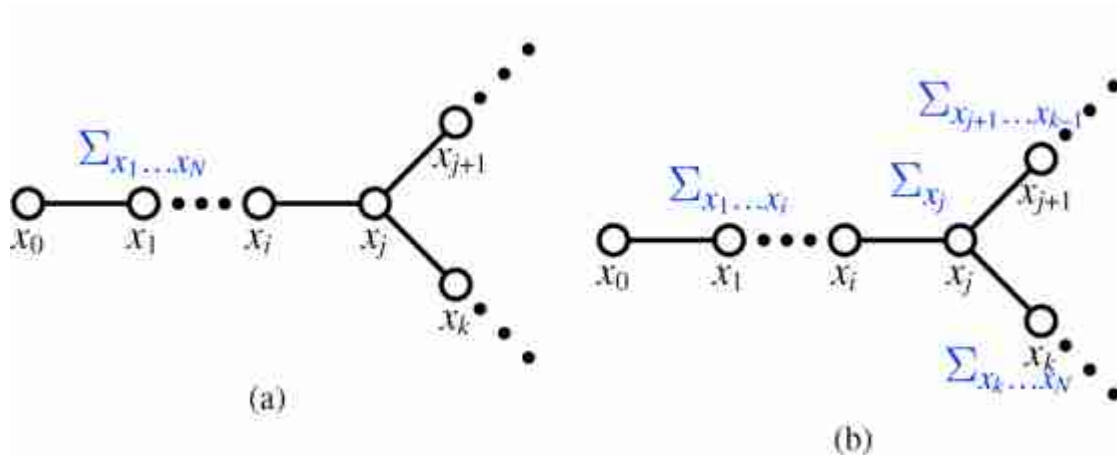


Figure 29.14: Example motivating belief propagation update rule. (a) Marginalization of a graph with no loops. (b) Shows how the partial sums at x_j distribute over nodes.

To compute the message from node j to node i :

1. Multiply together all messages (independent probabilities) coming in to node j , except for the message from node i back to node j ,
2. Multiply by the pairwise compatibility function $\psi_{ij}(x_i, x_j)$ (also independent of the other multiplicands).
3. Marginalize over the variable x_j .

These steps are summarized in this equation to compute the message from node j to node i :

$$m_{ji}(x_i) = \sum_{x_j} \psi_{ij}(x_i, x_j) \prod_{k \in \eta(j) \setminus i} m_{kj}(x_j) \quad (29.14)$$

where $\eta(j) \setminus i$ means “the neighbors of node j except for node i .” [Figure 29.15](#) shows this equation in a graphical form. Any local potential functions $\phi_j(x_j)$ are treated as an additional message into node j , that is, $m_{0j}(x_j) = \phi_j(x_j)$. For the case of continuous variables, the sum over the states of x_j in equation (29.14) is replaced by an integral over the domain of x_j .

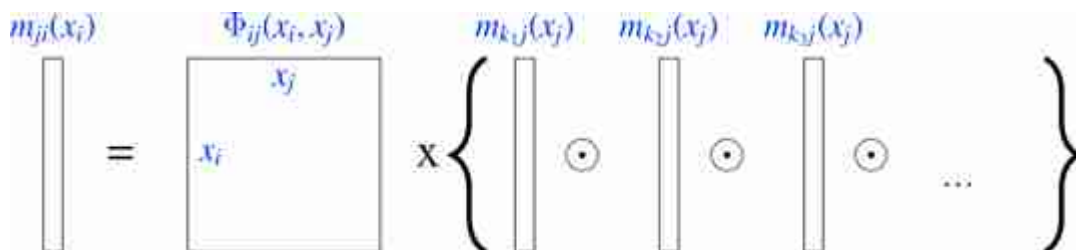


Figure 29.15: Pictorial depiction of belief propagation message passing rules of equation (29.14), where $\odot$ indicates elementwise multiplication (i.e., the Hadamard product).

As mentioned previously, BP messages are partial sums in the marginalization calculation. The arguments of messages are always the state of the node that the message is going to. (BP follows the “nosey neighbor rule” (from Brendan Frey, Univ. Toronto): Every node is a house in some neighborhood. Your nosey neighbor says to you, “Given what I’ve seen myself and everything I’ve heard from my neighbors, here’s what I think is going on inside your house.” (That metaphor especially makes sense if one has teenaged children.)

29.6.2 Marginal Probability

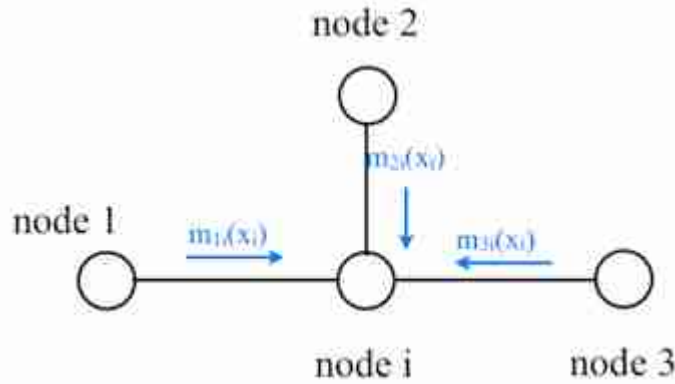


Figure 29.16: To compute the marginal probability at node i , we multiply together all the incoming messages at that node: $p_i(x_i) = \prod_{j \in \eta(i)} m_{ji}(x_i)$, including any local potential terms $\phi_i(x_i)$ as another message.

The marginal probability at a node i , equation (29.9), is the sum over the joint probabilities of all states except those of x_i . Because we assume the network has no loops, the conditional independence structure assures us that marginal probability at a node i is the product of the sum of all states of all nodes in each subtree connected to node i : Conditioned on node i , the probabilities within each subtree are independent. Thus the marginal probability at node i is the product of all the incoming messages:

$$p_i(x_i) = \prod_{j \in \eta(i)} m_{ji}(x_i) \quad (29.15)$$

(We include the local clique potential at node i , $\psi_i(x_i)$, as one of the messages in the product of all messages into node i).

29.6.3 Message Update Sequence

To find all the messages, how do we invoke the recursive BP update rule, equation (29.14)? We can apply equation (29.14) whenever all the incoming messages in the BP update rule are defined. If there are no incoming messages to a node, then its outgoing message is well-defined in the update rule. This lets us start the recursive algorithm.

A node can send a message whenever all the incoming messages it needs have been computed. We can compute the outgoing messages from leaf

nodes in the graphical model tree, since they have no incoming messages other than from the node to which they are sending a message, which doesn't enter in the outgoing message computation. Two natural message passing protocols are consistent with that rule: depth-first update, and parallel update. In depth-first update, one node is arbitrarily picked as the root. Messages are then passed from the leaves of the tree (leaves, relative to that root node) up to the root, then back down to the leaves. In parallel update, at each turn, every node sends every outgoing message for which it has received all the necessary incoming messages. [Figure 29.17](#) depicts the flow of messages for the parallel, synchronous update scheme and [Figure 29.18](#) shows the flow for the same network, but with a depth-first update schedule.

Note that when computing marginals at many nodes, we reuse messages with the BP algorithm. A single message-passing sweep through all the nodes lets us calculate the marginal at any node (using the depth-first update rules to calculate the marginal at the root node). A second sweep from the root node back to all the leaf nodes calculates all the messages needed to find the marginal probability at every node. It takes only twice the number of computations to calculate the incoming messages to every node, and thus the marginal probabilities everywhere, as it does to calculate the incoming messages for the marginal probability at a single node.

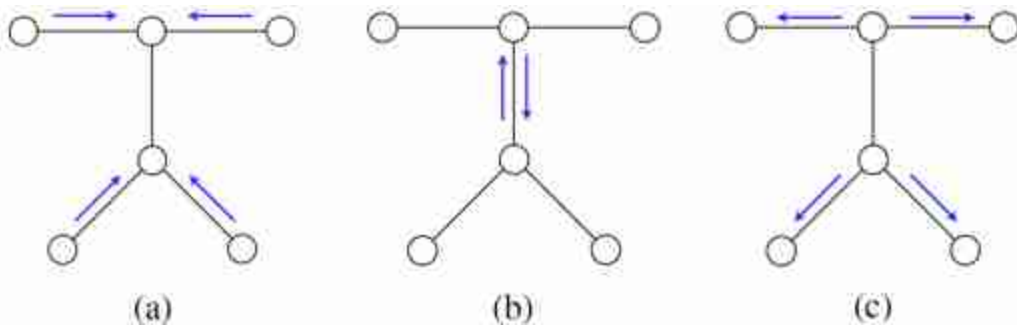


Figure 29.17: Synchronous parallel update schedule for BP message passing: Each node sends an outgoing message as soon as it receives the necessary incoming messages. Iterations: (a) one, (b) two, and (c) three.

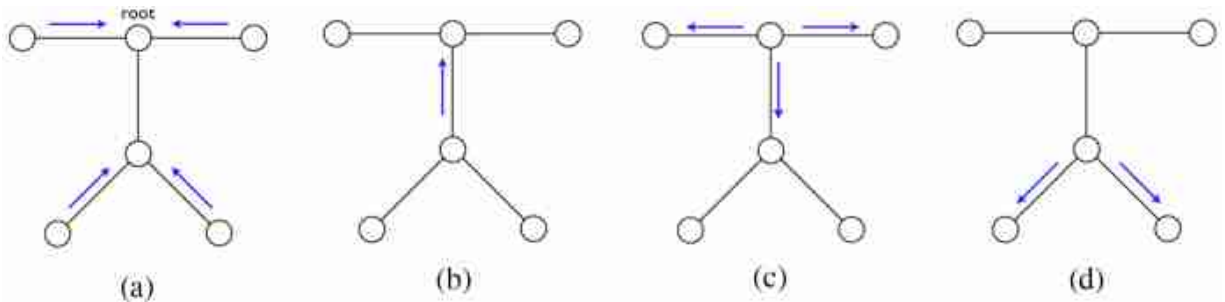


Figure 29.18: Depth-first BP message passing schedule. Select a root node. (a) Start at leaves; (b) proceed to root; (c), perform an outgoing sweep; and (d) compute remaining messages, ending at leaf nodes.

29.6.4 Example Belief Propagation Application: Stereo

Assume that we want to compute the depth from a stereo image pair, such as the pair shown in [figures 29.19\(a and b\)](#) and with their insets marked with the red rectangles, shown in [figures 29.19\(c and d\)](#). Assume that the two images have been rectified, as described in chapter 40, and consider the common scanline, marked in [figures 29.19\(c and d\)](#) with a black horizontal line. The luminance intensities of each scanline are plotted in [figures 29.19\(e and f\)](#). Most computer vision stereo algorithms [425] examine multiple scanlines to compute the depth at every pixel, but to illustrate a simple example of belief propagation in vision, we will consider the stereo vision problem while considering only one scanline pair at a time.

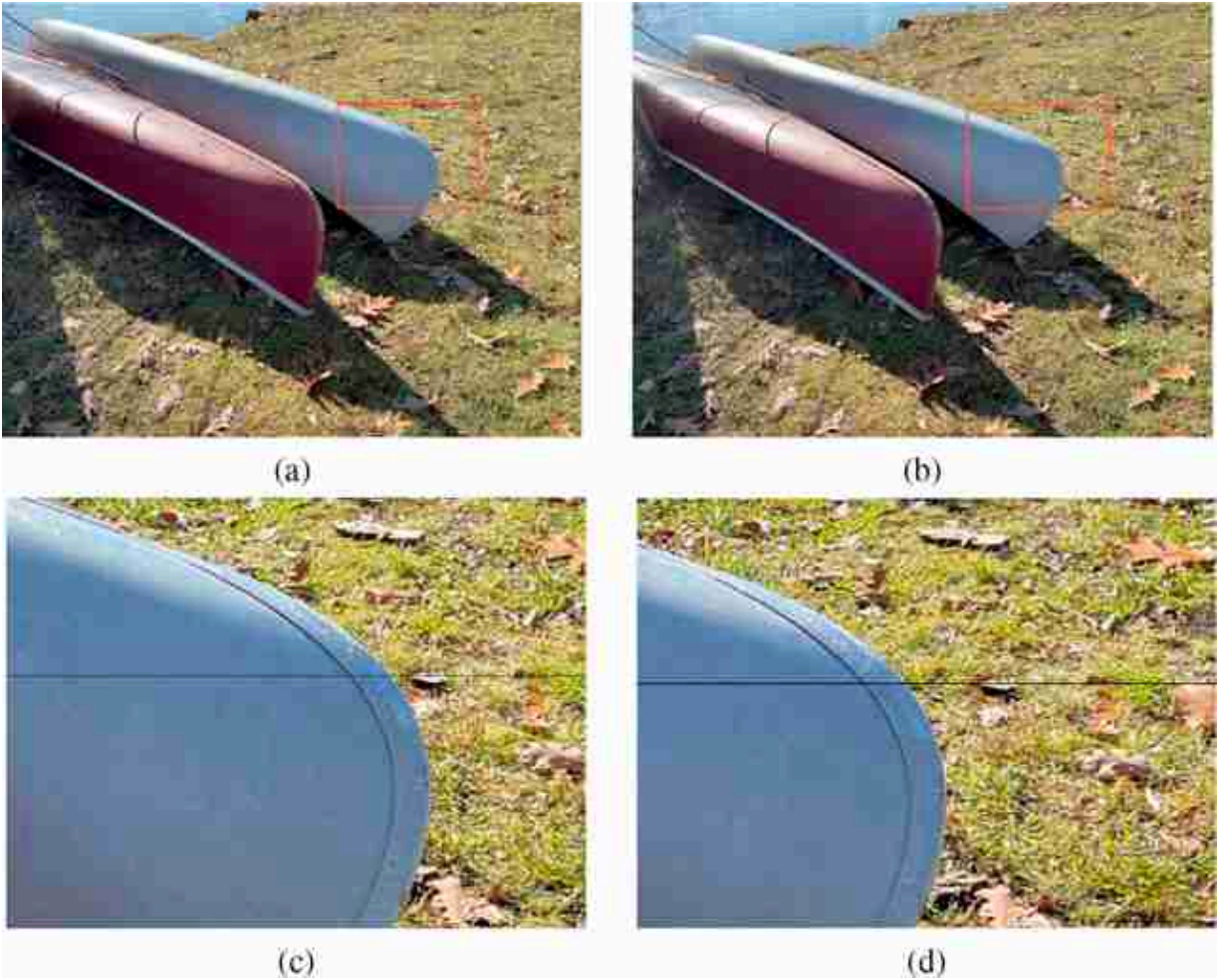


Figure 29.19: (a) Left and (b) right camera images. (c and d): insets showing areas of analysis.

The graphical model that describes this stereo depth reconstruction for a single scanline is shown in [figure 29.20](#).

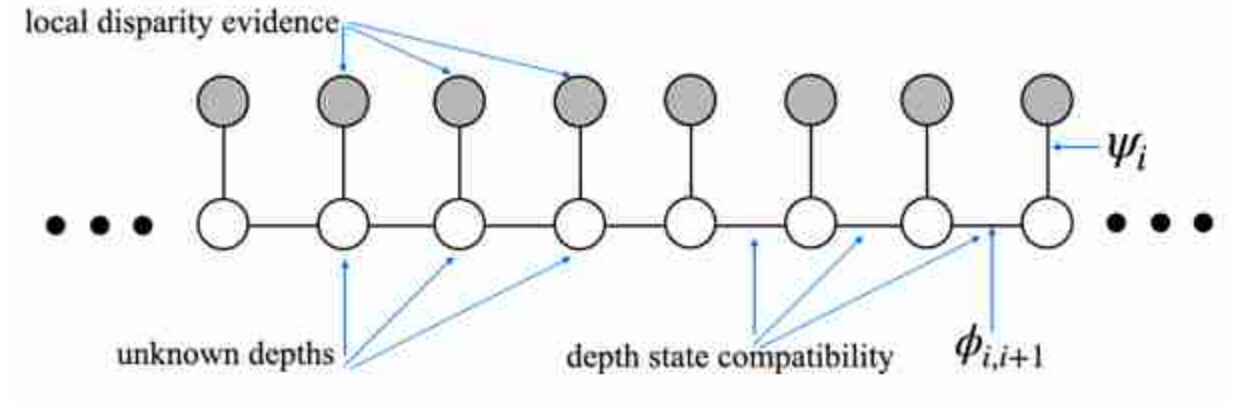


Figure 29.20: Graphical model for the posterior probability for stereo disparity offset between the left and right camera views from a stereo rig.

Each unfilled circle in the graphical model ([figure 29.20](#)) represents the unknown depth of each pixel in the scan line of one of the stereo images, in this example, the left camera image in [figure 29.19\(c\)](#). The open circles mean that the depth of each pixels is an unobserved variable. Often, we represent the depth as the **disparity** between the left and right camera images, see chapter 40.

The intensity values of the left and right camera scan lines can be used to compute the local evidence for the disparity offset, $d[i, j]$, of each right camera pixel corresponding to the left image pixel at each right camera position, $[i, j]$. For this example, we assume that each right camera pixel value, ℓ_r is that of a corresponding left camera value, ℓ_l , but with independent, identically distributed (IID) Gaussian random noise added. This leads to a simple formula for the local evidence, ψ_i , for the given depth value of any pixel,

$$\psi_i = p(d[i, j] | \ell_r, \ell_l) = k \exp \left(- \sum_{i=-10}^{i=10} \frac{(\ell_r[i, j] - \ell_l[i - d[i, j], j])^2}{2\sigma^2} \right) \quad (29.16)$$

For simplicity in this example, we discretize the disparity offsets into four different bins, each 45 disparity pixels wide. We sum the local disparity evidence within each depth bin, then normalize the evidence at each position to maintain a probability distribution. The resulting local evidence matrix, for each of the four depth states, at each left camera spatial position, is displayed in [figure 29.21\(c\)](#). The intensities in each column in the third-row image add up to one.

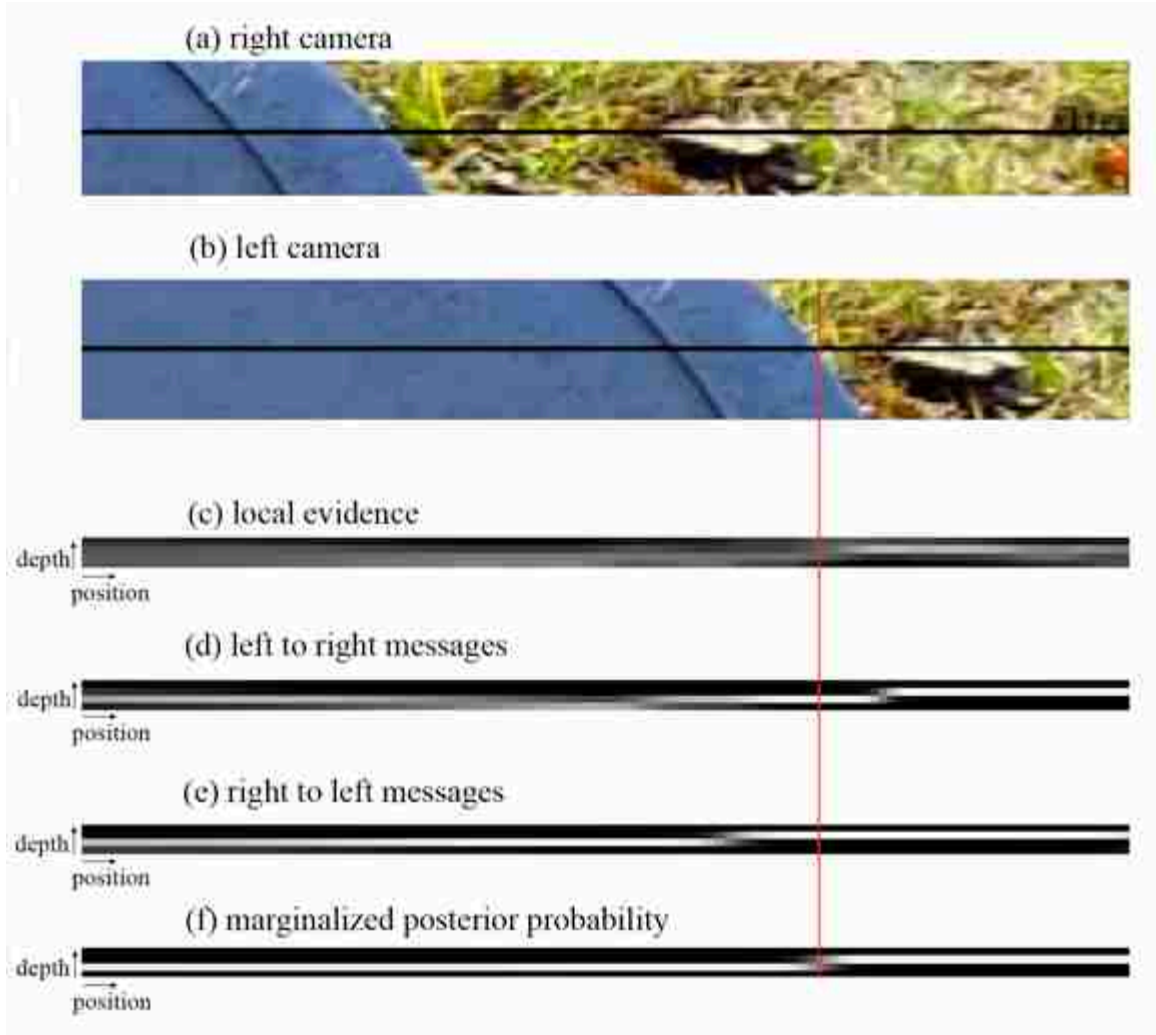


Figure 29.21: Belief propagation applied to graphical model, [figure 29.20](#), for the stereo problem. (a) Right and (b) left camera views. The black line shows the analyzed row. (c) Local evidence for each depth disparity at left camera. (d) Final rightward and (e) leftward belief propagation messages at each position. (f) Final marginalized posterior probability at each left camera pixel accurately finds the depth discontinuity.

The prior probability of any configuration of pixel disparity states is described by setting the compatibility matrices, $\phi[s_i, s_{i+1}]$ of the Markov chain of [figure 29.20](#). For this example, we use a compatibility matrix referred to as the Potts Model [\[510\]](#):

$$\phi[s_i, s_{i+1}] = \begin{cases} 1, & \text{if } s_i = s_{i+1} \\ \delta, & \text{otherwise} \end{cases} \quad (29.17)$$

For this example, we used $\delta = 0.001$

We then ran BP (equation [29.14]) in a depth-first update schedule. For this network, that involves two linear sweeps over all the nodes of the chain, first from left to right, then from right to left. Starting from the left-most node, we updated the rightward messages one node at a time, in a rightward sweep; then, starting from the right-most node, updated all the leftward messages one node at a time, in a leftward sweep. The results are in [figure 29.21](#)(d and e), respectively.

Once all the messages were computed, we used equation (29.15) to find the posterior marginal probability at each node. That involves, at each position, multiplying together all the incoming messages, including the local evidence. Thus, the values shown in [figure 29.21](#)(f) are the product, at each position, of [figure 29.21](#)(c, d and e). Note that for this scanline, in this image pair, there is a depth discontinuity at location of the red vertical mark in [figure 29.21](#), namely the canoe is closer to the camera than the ground beyond the canoe that appears to the right of the canoe. The local evidence for depth disparity, [figure 29.21](#)(c), is rather noisy, but the marginal posterior probability, after aggregating evidence along the scanline, accurately finds the depth discontinuity at the canoe boundary.

29.7 Loopy Belief Propagation

The BP message update rules only work to give the exact marginals when the topology of the network is that of a tree or a chain. In general, one can show that exact computation of marginal probabilities for graphs with loops depends on a graph-theoretic quantity known as the **treewidth** of the graph [274]. For many graphical models of interest in vision, such as 2D Markov random fields related to images, these quantities can be intractably large.

Message update rules are described locally, and one might imagine that it is a useful local operation to perform, even without the global guarantees of ordinary BP. It turns out that is true. Here is the algorithm, also known as **loopy belief propagation algorithm**:

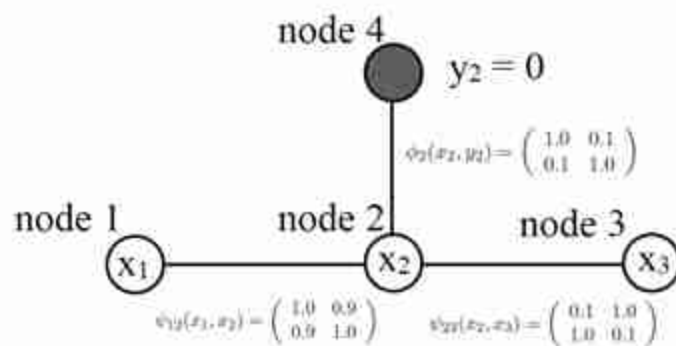
1. Convert graph to pairwise potentials.
2. Initialize all messages to all ones, or to random values between 0 and 1.
3. Run the belief propagation update rules of section 29.6.1 until convergence.

One can show that, in networks with loops, fixed points of the belief propagation algorithm (message configurations where the messages don't change with a message update) correspond to minima of a well-known approximation from the statistical physics community known as the Bethe free energy [513]. In practice, the solutions found by the loopy belief propagation algorithm can be quite good [346].

Instead of summing over the states of other nodes, we are sometimes interested in finding the $\mathbf{x}$ that maximizes the joint probability. The argmax operator passes through constant variables just as the summation sign did. This leads to an alternate version of the BP algorithm, equation (29.14), with the summation (of multiplying the vector message products by the node compatibility matrix) replaced with argmax. This is called the **max-product** version of belief propagation, and it computes an MAP estimate of the hidden states. Improvements have been developed over loopy belief propagation for the case of MAP estimation; see, for example, tree-reweighted belief propagation [275] and graph cuts [515].

29.7.1 Numerical Example of Belief Propagation

Here we work through a numerical example of belief propagation. To make the arithmetic easy, we'll solve for the marginal probabilities in the graphical model of two-state (0 and 1) random variables shown in [figure 29.22](#).



[Figure 29.22:](#) Undirected graphical model used in belief propagation example.

That graphical model has three hidden variables, and one variable observed to be in state 0. The compatibility matrices are given in the arrays

below (for which the state indices are 0, then 1, reading from left to right and top to bottom):

$$\psi_{12}(x_1, x_2) = \begin{bmatrix} 1.0 & 0.9 \\ 0.9 & 1.0 \end{bmatrix} \quad (29.18)$$

$$\psi_{23}(x_2, x_3) = \begin{bmatrix} 0.1 & 1.0 \\ 1.0 & 0.1 \end{bmatrix} \quad (29.19)$$

$$\psi_{42}(x_2, y_2) = \begin{bmatrix} 1.0 & 0.1 \\ 0.1 & 1.0 \end{bmatrix} \quad (29.20)$$

Note that in defining these potential functions, we haven't taken care to normalize the joint probability, so we'll need to normalize each marginal probability at the end (remember $p(x_1, x_2, x_3, y_2) = \psi_{42}(x_2, y_2)\psi_{23}(x_2, x_3)\psi_{12}(x_1, x_2)$, which should sum to 1 after summing over all states).

For this simple toy example, we can tell what results to expect by inspection, then verify that BP is doing the right thing. Node x_2 wants very much to look like $y_2 = 0$, because $\psi_{42}(x_2, y_2)$ contributes a large valued to the posterior probability when $x_2 = y_2 = 1$ or when $x_2 = y_2 = 0$. From $\psi_{12}(x_1, x_2)$ we see that x_1 has a very mild preference to look like x_2 . So we expect the marginal probability at node x_2 will be heavily biased toward $x_2 = 0$, and that node x_1 will have a mild preference for state 0. $\psi_{23}(x_2, x_3)$ encourages x_3 to be the opposite of x_2 , so it will be biased toward the state $x_3 = 1$.

Let's see what belief propagation gives. We'll follow the parallel, synchronous update scheme for calculating all the messages. The leaf nodes can send messages along their edges without waiting for any messages to be updated. For the message from node 1, we have

$$\begin{aligned} m_{12}(x_2) &= \sum_{x_1} \psi_{12}(x_1, x_2) \\ &= \begin{bmatrix} 1.0 & 0.9 \\ 0.9 & 1.0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.9 \\ 1.9 \end{bmatrix} = k_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} \end{aligned} \quad (29.21)$$

For numerical stability, we typically normalize the computed messages in equation (29.14) so the entries sum to 1, or so their maximum entry is 1, then remember to renormalize the final marginal probabilities to sum to 1. Here, we will normalize the messages for simplicity, absorbing the normalization into constants, k_i .

The message from node 3 to node 2 is

$$\begin{aligned}
m_{32}(x_2) &= \sum_{x_3} \psi_{32}(x_2, x_3) \\
&= \begin{bmatrix} 0.1 & 1.0 \\ 1.0 & 0.1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.1 \\ 1.1 \end{bmatrix} = k_2 \begin{bmatrix} 1 \\ 1 \end{bmatrix}
\end{aligned} \tag{29.22}$$

We have a nontrivial message from observed node y_2 (node 4) to the hidden variable x_2 :

$$\begin{aligned}
m_{42}(x_2) &= \sum_{x_4} \psi_{42}(x_2, y_2) \\
&= \begin{bmatrix} 1.0 & 0.1 \\ 0.1 & 1.0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1.0 \\ 0.1 \end{bmatrix}
\end{aligned} \tag{29.23}$$

where y_2 has been fixed to $y_2 = 0$, thus restricting $\psi_{42}(x_2, y_2)$ to just the first column.

Now we just have two messages left to compute before we have all messages computed (and therefore all node marginals computed from simple combinations of those messages). The message from node 2 to node 1 uses the messages from nodes 4 to 2 and 3 to 2:

$$\begin{aligned}
m_{21}(x_1) &= \sum_{x_2} \psi_{12}(x_1, x_2) m_{42}(x_2) m_{32}(x_2) \\
&= \begin{bmatrix} 1.0 & 0.9 \\ 0.9 & 1.0 \end{bmatrix} \left(\begin{bmatrix} 1.0 \\ 0.1 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 1.09 \\ 1.0 \end{bmatrix}
\end{aligned} \tag{29.24}$$

The final message is that from node 2 to node 3 (since y_2 is observed, we don't need to compute the message from node 2 to node 4). That message is:

$$\begin{aligned}
m_{23}(x_3) &= \sum_{x_2} \psi_{23}(x_2, x_3) m_{42}(x_2) m_{12}(x_2) \\
&= \begin{bmatrix} 0.1 & 1.0 \\ 1.0 & 0.1 \end{bmatrix} \left(\begin{bmatrix} 1.0 \\ 0.1 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 0.2 \\ 1.01 \end{bmatrix}
\end{aligned} \tag{29.25}$$

Now that we've computed all the messages, let's look at the marginals of the three hidden nodes. The product of all the messages arriving at node 1 is just the one message, $m_{21}(x_1)$, so we have (introducing constant k_3 to normalize the product of messages to be a probability distribution)

$$p_1(x_1) = k m_{21}(x_1) = \frac{1}{2.09} \begin{bmatrix} 1.09 \\ 1.0 \end{bmatrix} \tag{29.26}$$

As we knew it should, node 1 shows a slight preference for state 0.

The marginal at node 2 is proportional to the product of three messages. Two of those are trivial messages, but we'll show them all for completeness:

$$p_2(x_2) = k_4 m_{12}(x_2) m_{42}(x_2) m_{32}(x_2) \quad (29.27)$$

$$= k_4 \begin{bmatrix} 1 \\ 1 \end{bmatrix} \odot \begin{bmatrix} 1.0 \\ 0.1 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{1.1} \begin{bmatrix} 1.0 \\ 0.1 \end{bmatrix} \quad (29.28)$$

As expected, belief propagation reveals a strong bias for node 2 being in state 0.

Finally, for the marginal probability at node 3, we have

$$p_3(x_3) = k_5 m_{23}(x_3) = \frac{1}{1.21} \begin{bmatrix} 0.2 \\ 1.01 \end{bmatrix} \quad (29.29)$$

As predicted, this variable is biased toward being in state 1.

By running belief propagation within this tree, we have computed the exact marginal probabilities at each node, reusing the intermediate sums across different marginalizations, and exploiting the structure of the joint probability to perform the computation efficiently. If nothing were known about the joint probability structure, the marginalization cost would grow exponentially with the number of nodes in the network. But if the graph structure corresponding to the joint probability is known to be a chain or a tree, then the marginalization cost only grows linearly with the number of nodes, and is quadratic in the node state dimensions. The belief propagation algorithm enables inference in many large-scale problems.

29.8 Relationship of Probabilistic Graphical Models to Neural Networks

The probabilistic graphical models (PGMs) we studied in this chapter and neural networks (NNs) share some characteristics, but are quite different. Both share a graph structure, with nodes and edges, but the nodes and edges mean different things in each case. Below is a brief summary of the differences:

- **Nodes:** The nodes of a NN diagram typically represent a real-valued scalar activations. The nodes of a PGM represent a random variable, real or discrete-valued, and scalars or vectors.
- **Edges:** Edges in PGMs determine statistical dependencies. Edges in NNs determine which are involved in a linear summation to the next

layer.

- **Computation:** The big difference between PGMs and NNs lies in what they calculate. PGMs represent factorized probability distributions and pass messages that allow for the computation of probabilities, for example, the marginal probability at a node. In contrast, NNs compute results that minimize some expected loss or an energy function [[285](#), [286](#)]. That different goal, that is, minimizing an expected loss rather than calculating a probability, allows for more general computations by a neural network at the cost of output that may be less modular or less interpretable because they are not probabilities.

Both PGMs and NNs can have nodes organized into grid-like structures but also more generally in arbitrary graphs; see, for example, [[164](#), [263](#)].

29.9 Concluding Remarks

Probabilistic graphical models provide a modular representation for complex joint probability distributions. Nodes can represent pixels, scenes, objects, or object parts, while the edges describe the conditional independence structure. For tree-like graphs, belief propagation delivers optimal inference. In general graphs with loops, it can give a reasonable approximation.

IX

GENERATIVE IMAGE MODELS AND REPRESENTATION LEARNING

Vision is the problem of mapping visual data to representations. This section examines that mapping and its inverse—mapping representations to data—and forges a tight link between them. This part of the book covers some of the same topics as in other parts, but with a new set of tools that revolve primarily around deep neural nets. These tools are highly effective and provide a simple and unified framework for dealing with a large family of problems in computer vision.

- **Chapter 30** introduces the idea of representation learning, where the goal is to train a model that produces good representations of the raw data, such as vector embeddings.
- **Chapter 31** zooms in on the particular problem of identifying perceptual groups, which are an important kind of visual representation with a long history in vision science.
- **Chapter 32** describes generative models that synthesize images.
- **Chapter 33** forges a connection between representation learning and generative modeling, describing these as inverses of each other.
- **Chapter 34** extends generative models to the conditional setting, where some data is synthesized based on other data.

Notation

This part deals extensively with random variables and probability distributions. See the Notation section before chapter 1 for our conventions. A few reminders follow.

- X and Y are random variables, while x and y are realizations of those variables. X and Y are the domains of those variables.
- $p(X)$ is a distribution over X ; that is, $p(X)$ is a function over the domain X . Conversely, $p(\mathbf{x})$ is the probability of a realization $X = \mathbf{x}$. It is short for $p(X = \mathbf{x})$, and it is a scalar.

30 Representation Learning

30.1 Introduction

In this book we have seen many ways to represent visual signals: in the spatial domain versus frequency domain, with pyramids and filter responses, and more. We have seen that the choice of representation is critical: each type of representation makes some operations easy and others hard. We also saw that deep neural networks can be thought of as transforming the data, layer by layer, from one representation into another and another. If representations are so important, why not set our objective to be “come up with the best representation possible.” In this chapter, we will try to do exactly that.

This chapter is primarily an investigation of *objectives*. We will identify different objective functions that capture properties of what it means to be a good representation. Then we will train deep nets to optimize these objectives, and investigate the resulting learned representations.

30.2 Problem Setting

Before diving in, let us present the basic problem statement and the notation we will be using throughout this chapter.

The goal of representation learning is to learn to map from datapoints, $\mathbf{x} \in X$, to abstract representations, $\mathbf{z} \in Z$, as schematized in [figure 30.1](#):

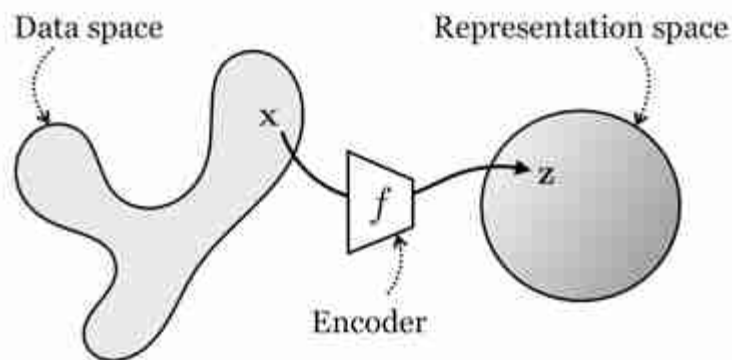


Figure 30.1: The goal in this chapter is to learn to map from datapoints to abstract representations, which are typically simpler and more useful than the raw data.

We call this mapping an **encoding** and learn an **encoder** function $f: X \rightarrow Z$. Typically both x and z are high-dimensional vectors, and z is called a **vector embedding** of x . The mapping f is trained so that z has certain desirable properties: common desiderata include that z be lower dimensional than x ; that the distribution of z values, that is, $p(z)$, has a simple structure (such as being the unit normal distribution); and that the dimensions of z be independent factors of variation, that is, the representation should be **disentangled** [43]. In this way Z is a simpler, more abstracted, or better organized representational space than X , as reflected in the shapes in [figure 30.1](#).

30.3 What Makes for a Good Representation?

A good representation is one that makes subsequent problem solving easier. Below we discuss several desiderata of a good representation.

30.3.1 Compression

A good representation is one that is parsimonious and captures just the essential characteristics of the data necessary for tasks of human interest. There are at least three senses in which compressed representations are desirable.

1. Compressed representations require less memory to store.

2. Compression is a way to achieve invariance to nuisance factors. Depending on what the representation will be used for, nuisances might include camera noise, lighting conditions, and viewing angle. If we can factor out those nuisances and discard them from our representation, then the representation will be more useful for tasks like object recognition where camera and lighting may change without affecting the semantics of the observed object.
3. Compression is an embodiment of *Occam's razor*: among competing hypotheses that all explain the data equally well, the simplest is most likely to be true. As we will see below, and in the next chapter, many representation learning algorithms seek simple representations from which you can regenerate the raw data (e.g., autoencoders). Such representations explain the data in the formal sense that they assign high likelihood to the data. This is the same sense to which the idea of Bayesian Occam's razor applies (section 11.3.2); see [313] for a mathematical treatment of this idea. Therefore, we can state Occam's razor for representation learning: among two representations that fit the data equally well, give preference to the more compressed.

This version of Occam's razor is also known as the **minimum description length principle** [177]

Many representation learning algorithms capture these goals by penalizing or constraining the complexity of the learned representation. Such a penalty must be paired with some other objective or else it will lead to a degenerate solution: just make the representation contain nothing at all. Usually we try to compress the signal in certain ways while preserving other aspects of the information it carries. *Autoencoders* and *contrastive learning*, which we will describe subsequently, are two representation learning algorithms based on the idea of compression.

30.3.2 Prediction

The whole purpose of having a visual system is to be able to take actions that achieve desirable future outcomes. Predicting the future is therefore very important, and we often want representations that act as a good substrate for prediction.

The idea of prediction can be generalized to involve more than just predicting the *future*. Instead we may want to predict the past (i.e., given when I'm seeing today, what happened yesterday?), or we may want to predict gaps in our measurements, a problem known as **imputation**. Filling in missing pixels in a corrupted image is an example of imputation, as is superresolving a movie to play at a higher framerate. We may even want to perform more abstract kinds of predictions, like predicting what some other agent is thinking about (a problem called theory of mind). In general, prediction can refer to the inference of any arbitrary property of the world given observed data. Facilitating the prediction of important world properties—the future, the past, mental states, cause and effect, and so on—is perhaps the defining property of a good representation.

Most representation learning algorithms in vision are about learning compressed encodings of the world that are also predictive of the future (and therefore a good substrate for decision making). Beyond just compression and prediction, some works have explored other goals, including that a representation be disentangled [43], interpretable [273], and actionable [449] (you can use it as an effective substrate for control). All these goals—compression, prediction, interpretability, and so on—are not in contrast to each other but rather overlap; for example, more compressed representations may be simpler and hence easier for a human to interpret.

30.3.3 Types of Representation Learners

Representation learning algorithms are mostly differentiated by the objective function they optimize, as well as the constraints on their hypothesis space. We explore a variety of common objectives in the following sections, and summarize how these relate to the core principles of compression and prediction in [table 30.1](#).

Table 30.1

One way to categorize different representation learning methods.

Learning Method	Learning Principle	Short Summary
Autoencoding	Compression	Remove redundant information
Contrastive	Compression	Achieve invariance to viewing transformations
Clustering	Compression	Quantize continuous data into discrete categories
Future prediction	Prediction	Predict the future
Imputation	Prediction	Predict missing data
Pretext tasks	Prediction	Predict abstract properties of your data

30.4 Autoencoders

Autoencoders are one of the oldest and most common kinds of representation learners [418, 31]. An autoencoder is a function that maps data back to itself (hence the “auto”), but via a low-dimensional representational **bottleneck**, as shown in [figure 30.2](#):

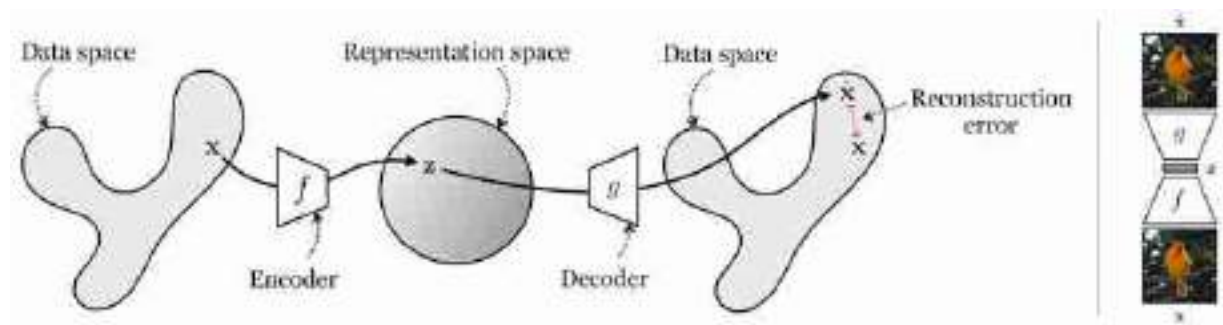


Figure 30.2: (left) An autoencoder maps from points in data space, to points in representation space, and back. (right) An example of running an autoencoder on an input image of a bird.

On the right of this figure is an example of an autoencoder applied to an image. At first glance this might not seem very useful: the output is the

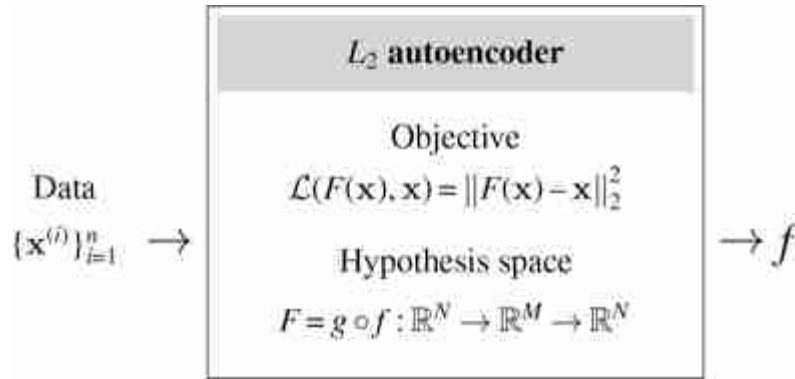
same as the input! The key trick is to impose constraints on the intermediate representation $\mathbf{z}$, so that it becomes useful. The most common constraint is compression: $\mathbf{z}$ is a low-dimensional, compressed representation of $\mathbf{x}$.

To be precise, an autoencoder F consists of two parts, an **encoder** f and a **decoder** g , with $F = g \circ f$. The encoder, $f : \mathbb{R}^N \rightarrow \mathbb{R}^M$, maps high-dimensional data $\mathbf{x} \in \mathbb{R}^N$ to a vector embedding $\mathbf{z} \in \mathbb{R}^M$. Typically, the key property is that $M < N$, that is, we have performed **dimensionality reduction** (although autoencoders can also be constructed without this property, in which case they are not doing dimensionality reduction but instead may place other explicit or implicit constraints on $\mathbf{z}$). The decoder, $g : \mathbb{R}^M \rightarrow \mathbb{R}^N$ performs the inverse mapping to f , and ideally g is exactly the inverse function f^{-1} . Because it may be impossible to perfectly invert f , we use a loss function to penalize how far we are from perfect inversion, and a typical choice is the squared error **reconstruction loss**, $\mathcal{L}_{\text{recon}}(\hat{\mathbf{x}}, \mathbf{x}) = \|\hat{\mathbf{x}} - \mathbf{x}\|_2^2$. Then the learning problem is:

$$f^*, g^* = \arg \min_{f, g} \mathbb{E}_{\mathbf{x}} \|g(f(\mathbf{x})) - \mathbf{x}\|_2^2 \quad (30.1)$$

The idea is to find a lower dimensional representation of the data from which we are able to reconstruct the data. An autoencoder is fine with throwing away any redundant features of the raw data but is not happy with actually losing information. The particular loss function determines what kind of information is preferred when the bottleneck cannot support preserving *all* the information and a hard choice has to be made. The L_2 loss decomposes as $\|g(f(\mathbf{x})) - \mathbf{x}\|_2^2 = \sum_i (g(f(\mathbf{x}))_i - x_i)^2$, that is, a sum over individual pixel errors. This means that it only cares about matching each individual pixel intensity $g(f(\mathbf{x}))_i$ to the ground truth x_i and does not directly penalize patch-level statistics of $\mathbf{x}$ (i.e., statistics $\phi(\mathbf{x})$ that do not factorize as a sum $\sum_i \psi_i(x_i)$ over per-pixel functions ψ_i for any possible set of functions $\{\psi_i\}$). Autoencoders can also be constructed using loss functions that penalize higher-order statistics of $\mathbf{x}$, and this allows, for example, penalizing errors in the reconstruction of edges, textures, and other perceptual structures beyond just pixels [448].

The learning diagram for the basic L_2 autoencoder looks like this:



The optimizer is arbitrary but a typical choice would be gradient descent. The functional form of f and g are typically deep neural nets. The output is a learned data encoder f . We also get a learned decoder g as a byproduct, which has its own uses, but, for the purpose of representation learning, is usually discarded and only used as scaffolding for training what we really care about, namely f .

Closely related to autoencoders are other dimensionality reduction algorithms like **principle components analysis (PCA)**. In fact, an L_2 autoencoder, for which both f and g are linear functions, learns an M -dimensional embedding that spans the same subspace as a PCA projection to M -dimensions [57].

Autoencoders may not seem like much at first glance, but they actually appear all over the place, and many methods in this book can be considered to be special kinds of autoencoders, if you squint. Two examples: the steerable pyramid from chapter 23 is an autoencoder, and the CycleGAN algorithm from chapter 32 is also an autoencoder. See if you can find more examples.

30.4.1 Experiment: Do Autoencoders Learn Useful Representations?

It is clear from the above that autoencoders will learn a *compressed* representation of the data, but do they learn a *useful* representation? Of course the answer to this question depends on what we will use the representation for. Let's explore how autoencoders work on a simple data domain, consisting just of colored circles, triangles, and squares. The data consists of 64,000 images, samples of which are shown in [figure 30.3](#).

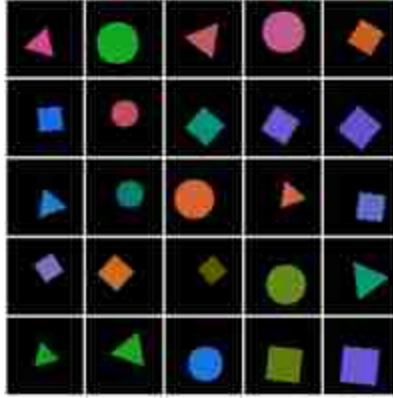


Figure 30.3: A sample from the toy dataset we will work with in this chapter.

Each shape has a randomized size, position, and rotation. Each shape's color is sampled from one of eight color classes (orange, green, purple, etc.) plus a small random perturbation.

For the autoencoder architecture we use a convolutional encoder and decoder, each with six convolutional layers interspersed with relu nonlinearities and a 128-dimensional bottleneck (i.e., $M = 128$). We train this autoencoder for 20,000 steps of stochastic gradient descent, using the Adam optimizer [262] with a batch size of 128.

After training, does this autoencoder obtain a good representation of the data? To answer this question, we need ways of evaluating the quality of a representation. There are many ways and indeed how to evaluate representations is an open area of research. But here we will stick with a very simple approach: see if the nearest neighbors, in representational space, are meaningful.

We can test this in two ways: (1) for a given query, visualize the images in the dataset whose embeddings are nearest neighbors to the query's embedding, and (2) measure the accuracy of a one-nearest-neighbor classifier in embedding space. Below, in [figure 30.4](#), we show both these analyses.

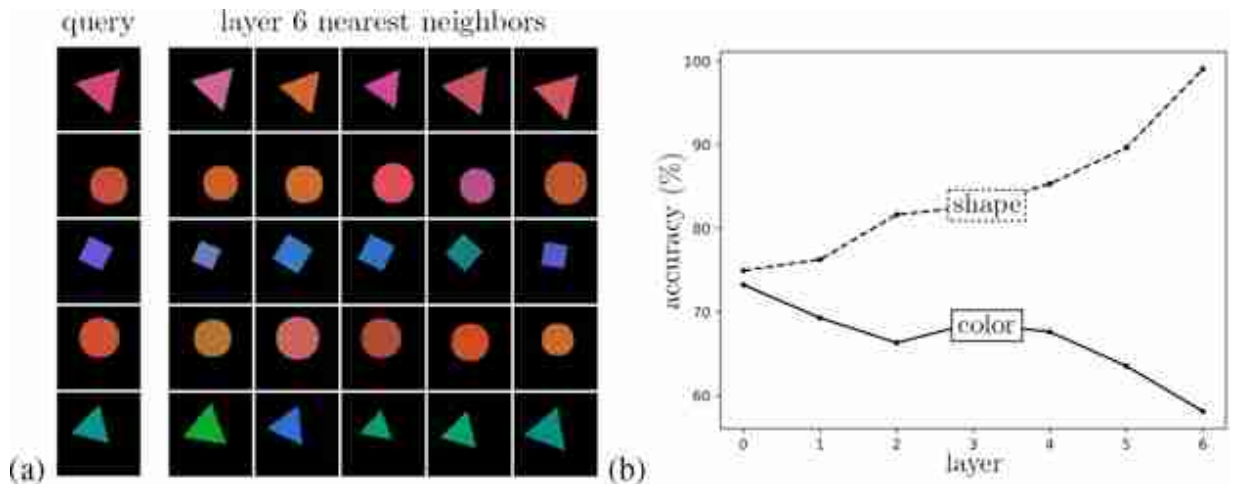


Figure 30.4: (a) Nearest neighbors, in autoencoder embedding space, to a set of query images. (b) Classification accuracy of a one-nearest-neighbor classifier of color and shape using the embeddings at each layer of the autoencoder's encoder.

Recall that every layer of a neural net can be considered as an embedding (representation) of the data. On the left we show the nearest neighbors to a set of query images, using the layer 6 embeddings of the data as the feature space in which to measure distance (i.e., nearness). Since we have a six-layer encoder, layer 6 is the bottleneck layer, the output of the full encoder f . Notice that the neighbors are indeed similar to the queries in terms of their colors, shapes, positions, and rotations; it seems the autoencoder produced a meaningful representation of the data!

Next we will probe a bit deeper, and ask, how effective are these embeddings at classifying key properties of the data? On the right we show the accuracy of a one-nearest-neighbor color classifier (between the eight color classes) and a shape classifier (circle vs. triangle vs. square) applied on embedding vectors at each layer of the autoencoder's encoder. The zeroth layer corresponds to measuring distance in the raw pixel space; interestingly, the color classifier does its best on this raw representation of the data. That's because the pixels are a more or less direct representation of color. Color classification performance gets worse and worse as we go deeper in the encoder. Conversely, shape is not explicit in raw pixels, so measuring distance in pixel-space does not yield good shape nearest neighbors. Deeper layers of the encoder give representations that are increasingly sensitive to shape similarity, and shape classification performance gets better. In general, there is no one representation that is

universally the best. Each is good at capturing some properties of the data and bad at capturing others. As we go deeper into an autoencoder, the embeddings tend to become more abstracted and therefore better at capturing abstract properties like shape and worse at capturing superficial properties like color. For many tasks—object recognition, geometry understanding, future prediction—the salient information is rather abstracted compared to the raw data, and therefore for these tasks deeper embeddings tend to work better.

30.5 Predictive Encodings

We have already seen many kinds of predictive learning, indeed almost any function can be thought of as making a prediction. In predictive representation learning, the goal is not to make predictions per se, but to use prediction as a way to train a useful representation. The way to do this is first encode the data into an embedding $\mathbf{z}$, then map from the embedding to your target prediction. This gives a composition of functions, just like with the autoencoder, that can be trained with a prediction task and yield a good data encoder. The prediction task is a **pretext task** for learning good representations.

Neuroscientists think the brain also uses prediction to better encode sensory signals, but focus on a different part of the problem. The idea of **predictive coding** states that the sensory cortex only transmits the difference between its predictions and the actual observed signal [225]. This chapter presents how to learn a representation that can make good predictions in the first place. Predictive coding focuses on one thing you can do with such a representation: use it to compress future signals by just transmitting the surprises.

Different kinds of prediction tasks have been proposed for learning good representations, and depending on the properties you want in your representation different prediction tasks will be best. Examples include predicting future frames in a video [402] and predicting the next pixel in an image given a sequence of preceding pixels [78]. It is also possible to use an image's semantic class as the prediction target. In that case, the prediction problem is identical to training an image classifier, but the goal is very different. Rather than obtaining a good classifier at the end, our goal is

instead to obtain a good image encoding (which is predictive of semantics) [106]. These three examples are visualized below ([figure 30.5](#)).

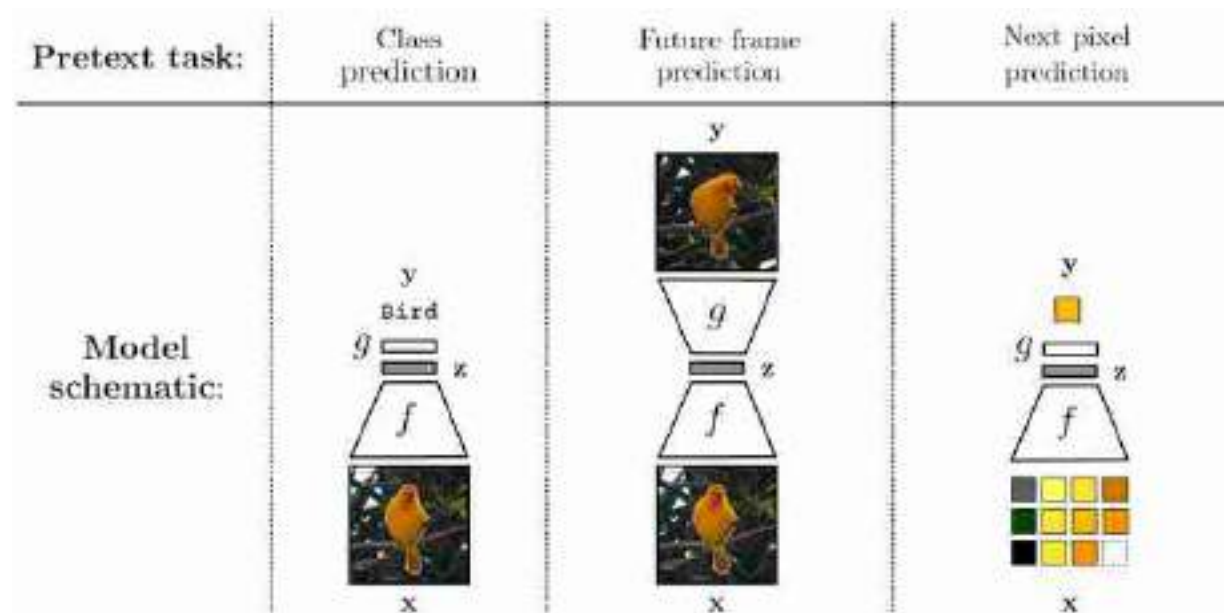
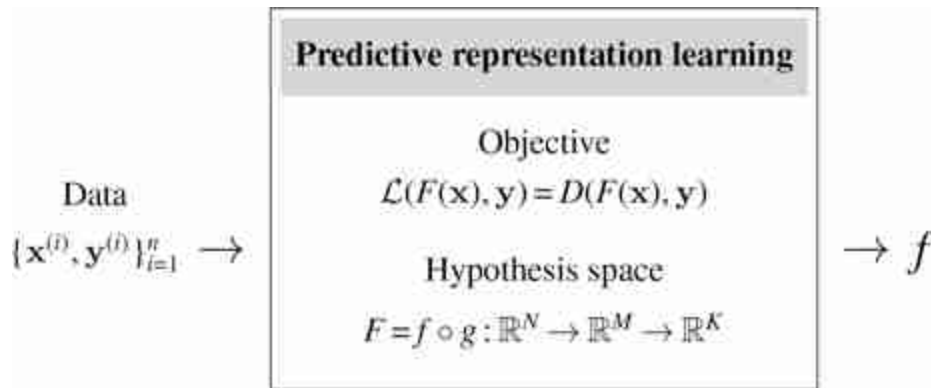


Figure 30.5: Examples of different pretext tasks.

These tasks look a lot like supervised learning. We avoid calling it “supervised” because that connotes that we have examples of input-output pairs on the target task. Here that would be input data and exemplar output representations. But we don't have that. The supervision in this setup is a pretext task that we hope induces good representations.

Let us now describe the predictive learning problem more formally. Let y be the prediction target. Then predictive representation learning looks like this:



where D is some distance function, for example, L_2 . Just like with the autoencoder, f and g are usually neural nets but may be any family of functions; often g is just a single linear layer. Unlike with autoencoders, there is no standard setting for the relative dimensionalities of N , M , and K ; instead it depends on the prediction task. If the task is image classification, then N will be the (large) dimensionality of the input pixels, M will usually be much lower dimensional, and K will be the number of image classes (to output K -dimensional class probability vectors).

30.5.1 Object Detectors Emerge from Scene-Level Supervision

The real power of these pretext tasks lies not in solving the tasks themselves but in acquiring useful image embeddings $\mathbf{z}$ as a byproduct of solving the pretext task. The amazing thing that ends up happening is that $\mathbf{z}$ -space may have *emergent structure* that was not explicit in either the raw training data nor the pretext task. As a case study, consider the work of [528]. They trained a convolutional neural net (CNN) to perform scene classification, which asks whether the image shows a living room, bedroom, kitchen, and so on. The net did wonderfully at that task, but that wasn't the point. The researchers instead peeled apart the net and looked at which input images were causing different neurons within the net to fire. What they found was that there were neurons, on hidden layers in the net, that fired selectively whenever the input image was of a specific object class. For example, one particular neuron would fire when the input was a staircase, and another neuron would fire predominantly for inputs that were rafts. The subsequent images ([figure 30.6](#)) show four of the top images that activate these two particular neurons. These particular neurons are on

convolutional layers, so really each is a filter response; the highlighted regions indicate where the feature map for that filter exceeds a threshold.

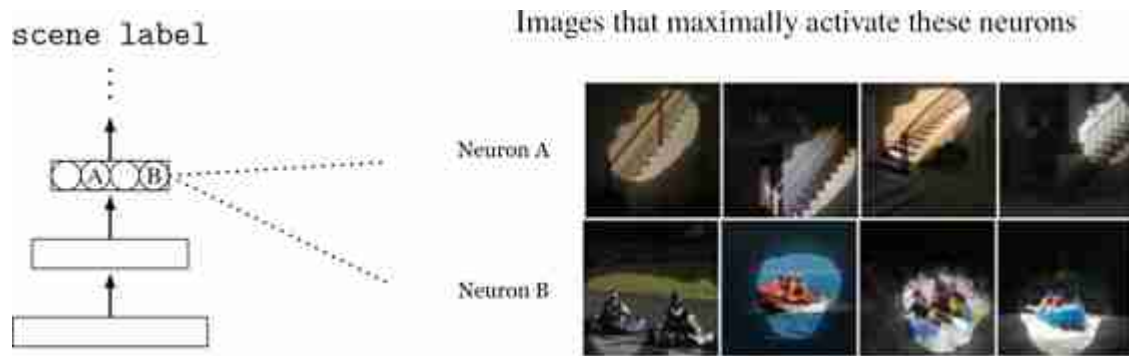
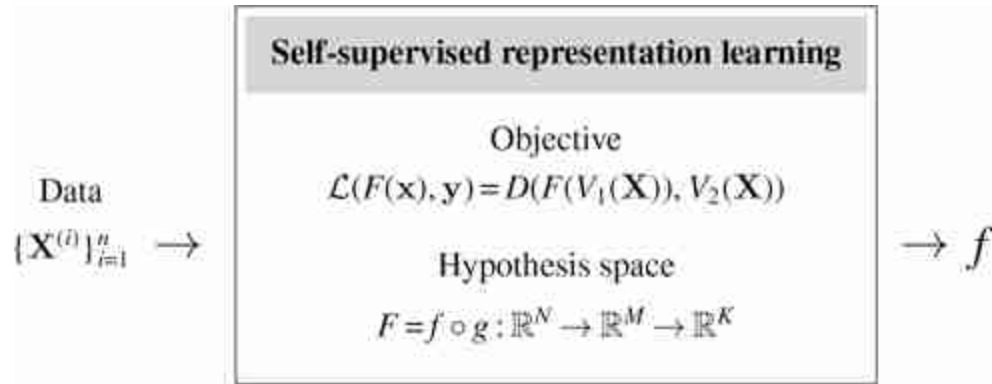


Figure 30.6: Visualizing two neural receptive fields in a scene classifier neural net. Images taken from [528].

What this shows is that *object* detectors, that is, neurons that selectively fire when they see a particular object class, emerge in the hidden layers of a CNN trained only to perform *scene* classification. This makes sense in retrospect—how else would the net recognize scenes if not first by identifying their constituent objects?—but it was quite a shock for the community to see it for the first time. It gave some evidence that the way we humans parse and recognize scenes may match the way CNNs also internally parse and recognize scenes.

30.6 Self-Supervised Learning

Predictive learning is great when we have good prediction targets that induce good representations. What if we don't have labeled targets provided to us? Instead we could try to cook up targets out of the raw data itself; for example, we could decide that the top right pixel's color will be the “label” of the image. This idea is called **self-supervision**. It looks like this:



where V_1 and V_2 are two different functions of the *full data tensor* $\mathbf{X}$. For example, V_1 might be the left side of the image $\mathbf{X}$ and V_2 could be the right side, so the pretext task is to predict the right side of an image from its left side. In fact, several of the examples we gave previously for predictive learning are of the self-supervised variety: supervision for predicting a future frame, or a next pixel, can be cooked up just by splitting a video into past and future frames, or splitting an image into previous and next pixels in a raster-order sequence.

30.7 Imputation

Imputation is a special case of self-supervised learning, where the prediction targets are missing elements of the input data. For example, predicting missing pixels is an imputation problem, as is colorizing a black and white photo (i.e., predicting missing color channels). [Figure 30.7](#) gives several examples of these imputation tasks.

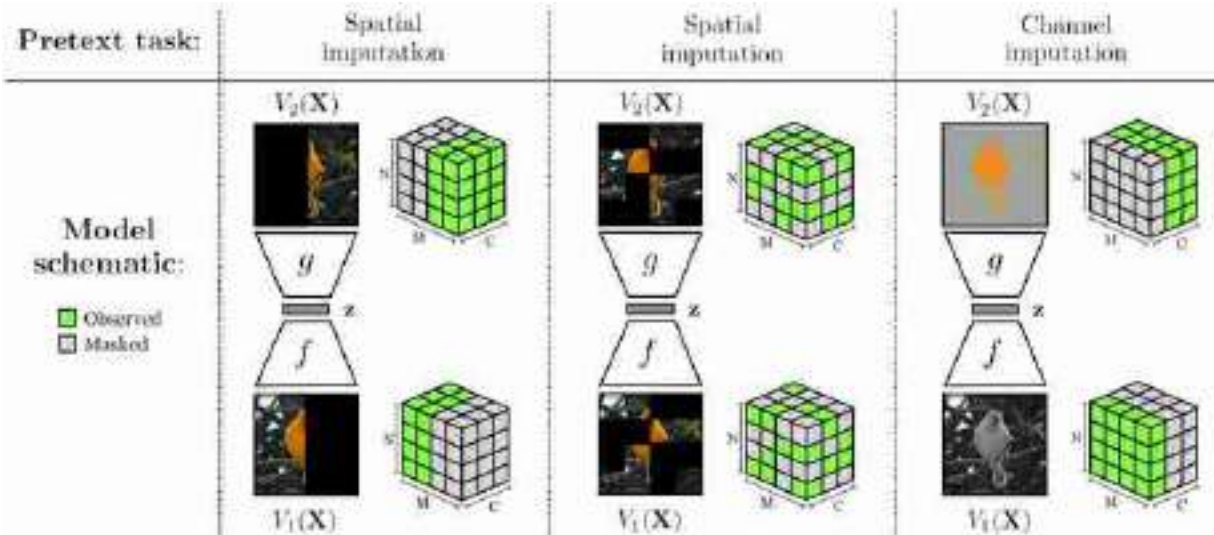


Figure 30.7: Many common pretext tasks are special cases of imputation on missing values in the data tensor.

Imputation—whether over spatial masks or missing channels—can result in effective visual representations [488, 380, 192, 523, 284, 524]. Notice that predicting future frames and next pixels (our examples from [figure 30.5](#)) are also imputation problems.

Above we described how object detectors emerge as a byproduct of training a net to perform scene classification. What do you think emerges as a byproduct of training a net to perform colorization?

It may surprise you to find that the answer is object detectors once again! This certainly surprised us when we saw the results in [figure 30.8](#), which are taken from [523].

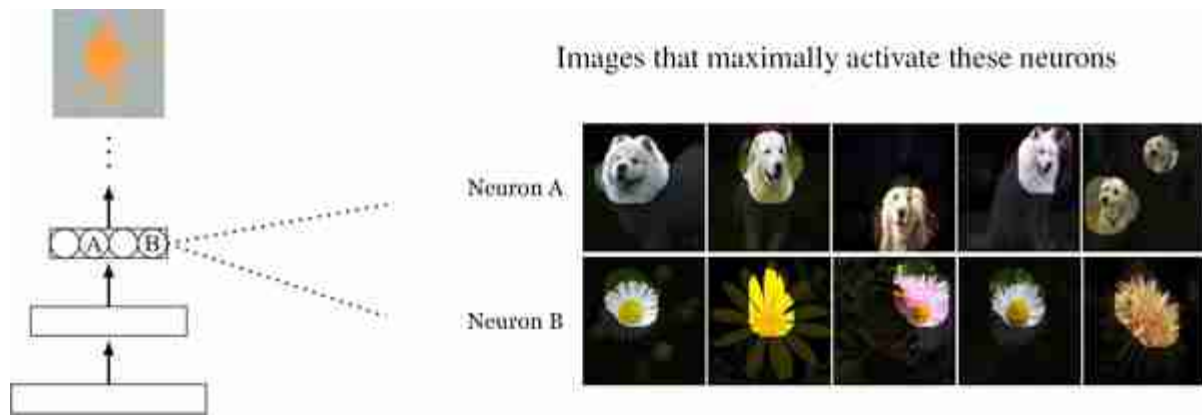


Figure 30.8: Visualizing two neural receptive fields in the colorization model from [523]. Images generated by Andrew Owens and Richard Zhang.

In fact, object detectors emerge in CNNs for just about any reasonable pretext task: scene recognition, colorization, inpainting missing pixels, and more. What may be going on is that these things we call “objects” are not just a peculiarity of human perception but rather map onto some fundamentally useful structure out there in the world, and any visual system tasked with understanding our world would arrive at a similar representation, carving up the sensory array into objects and other kinds of **perceptual groups**. This idea will be explored in greater detail in the next chapter.

30.8 Abstract Pretext Tasks

Other varieties of self-supervised learning set up more abstract prediction problems, rather than just aiming to predict missing data. For example, we may try to predict if an image has been rotated 90 degrees [276], or we may aim to predict the relative position of two image patches given their appearance [103]. These pretext tasks can induce effective visual representations because solving them requires learning about semantic and geometric regularities in the world, such as that clouds tend to appear near the top of an image or that the trunk of a tree tends to appear below its branches.

30.9 Clustering

One way to compress a signal is dimensionality reduction, which we saw an example of previously with the autoencoder. Another way is to quantize the signal into discrete categories, an operation known also as **clustering**.

Another name for clustering, more common in the representation learning literature, is **vector quantization**.

Mathematically, clustering is a function $f: \{\mathbf{x}^{(i)}\}_{i=1}^N \rightarrow \{1, \dots, k\}$, that is, a mapping from the members of a dataset $\{\mathbf{x}^{(i)}\}_{i=1}^N$ to k integer classes (k can potentially be unbounded). Representing integers with one-hot codes, clustering is shown in [figure 30.9](#).

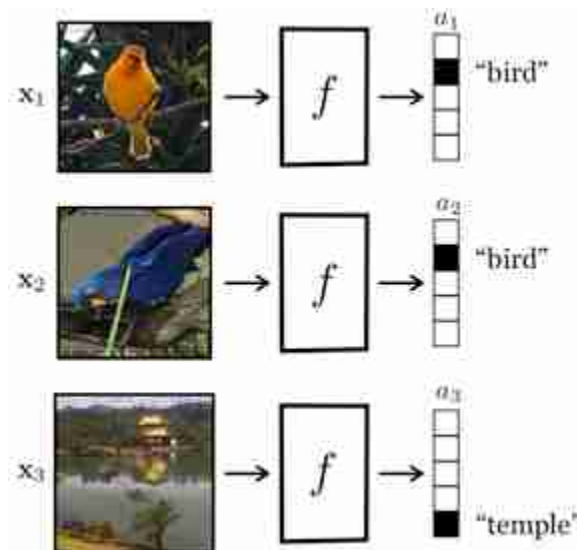


Figure 30.9: You can think of clustering as being just like image labeling, except that that labels are self-discovered rather than being predefined.

Clustering follows from the principle of compression: if we can well summarize a signal with just a discrete category label, then this summary can serve as a lighter weight and more abstracted substrate for further reasoning. You will already be familiar with clustering because we use it in our natural language everyday. For example, consider the words “antelope,” “giraffe,” and “zebra.” Those words are discrete category labels (i.e., integers) that summarize huge conceptual sets (just think of all the

individual lives and richly diverse personalities you are lumping together with the simple word “antelope”). Words, then, are clusters! They are mappings from data to integers, and clustering is the problem of making up new words for things.

Words, of course, are given additional structure when used in a language (grammar, connotations, etc.) beyond just being a set of clusters. The same kind of structure can be added on top of visual clusters.

Many clustering algorithms not only partition the data but also compute a representation of the data within each cluster; this representation is sometimes called a **code vector**. The most common code vector is the **cluster center**, μ , that is, the mean value of all datapoints assigned to the cluster. Clusters can also be represented by other statistics of the data assigned to them, such as the variance of this data or some arbitrary embedding vector, but this is less common. The set of cluster centers, $\{\mu_i\}_{i=1}^k$, is a representation of a whole *dataset*. They summarize the main modes of behavior in the dataset.

30.9.1 *K*-Means

There are many types of clustering algorithm but we will illustrate the basic principles with just one example, perhaps the most popular clustering algorithm of them all, **k-means**. *K*-means is a clustering algorithm that partitions the data into k clusters. Each datapoint $\mathbf{x}^{(i)}$ is assigned to a cluster indexed by an integer $a_i \in \{1, \dots, k\}$. Each cluster is given a code vector $\mathbf{z} \in \mathbb{R}^M$. The k -means objective is to minimize the distance between each datapoint and the code vector of the cluster it is assigned to:

$$J_{\text{kmeans}} = \sum_{i=1}^N \|\mathbf{z}_{a_i} - \mathbf{x}^{(i)}\|_2^2 \quad \Leftarrow \quad k\text{-means objective} \quad (30.2)$$

This way, the code vector assigned to each datapoint will be a good approximation to the value of that datapoint, and we will have a faithful, but compressed, representation of the dataset.

There are two free parameter sets to optimize over: the code vectors and the cluster assignments. The cluster assignments can be represented with a clustering function $f: \{\mathbf{x}^{(i)}\}_{i=1}^N \rightarrow \{1, \dots, k\}$:

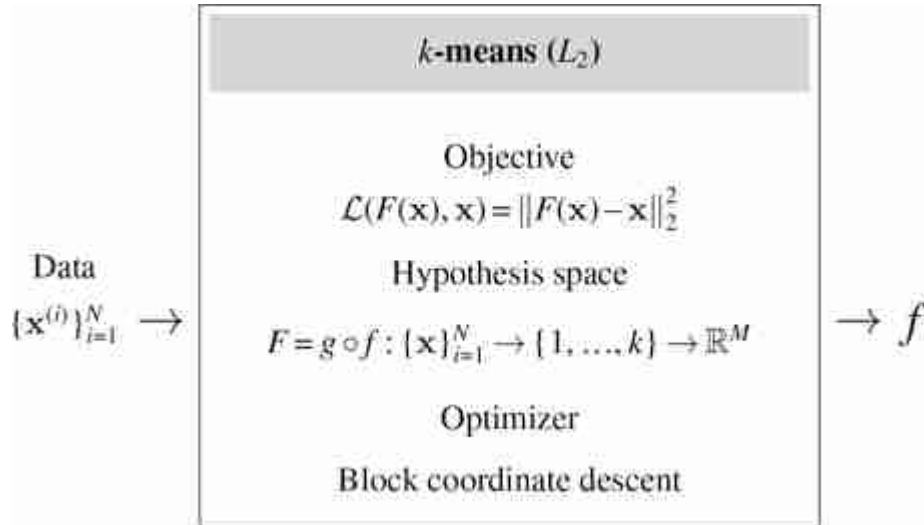
$$a_i = f(\mathbf{x}^{(i)}) \quad (30.3)$$

We will represent the code vectors for each cluster with a function $g : \{1, \dots, k\} \rightarrow \mathbb{R}^M$, where the data dimensionality is M :

$$\mathbf{z}_{a_i} = g(a_i) \quad (30.4)$$

Both f and g can be implemented as lookup tables, since for both the input is a countable set. The k -means algorithm amounts to just filling in these two lookup tables.

Now we are ready to present the full k -means algorithm, viewed as a learning algorithm. As you will see below, the learning diagram looks almost the same as for an autoencoder! The key differences are (1) the bottleneck is discrete integers rather than continuous vectors, and (2) the optimizer is slightly different (we will delve into it subsequently).



To recap, there are two functions we are optimizing over: the encoder f and the decoder g . The f is parameterized by a set of integers $\{a_i\}_{i=1}^N$, $a_i \in \{1, \dots, k\}$, which specify the cluster assignment for each datapoint. The decoder g is parameterized by a set of k code vectors $\{\mathbf{z}_j\}_{j=1}^k$, $\mathbf{z}_j \in \mathbb{R}^M$, one for each cluster, so we have $F(\mathbf{x}^{(i)}) = g(f(\mathbf{x}^{(i)})) = g(a_i) = \mathbf{z}_{a_i}$. The g is differentiable with respect to its parameters but f is not, because the parameters of f are discrete variables. This means that gradient descent will not be a suitable optimization algorithm (the gradient $\frac{\partial f}{\partial a_i}$ is undefined). Instead, we will use the optimization strategy described next.

30.9.1.1 Optimizing k -means

K -means uses an optimization algorithm called **block coordinate descent**. This algorithm splits up the optimization parameters into multiple subsets (blocks). Then it alternates between *fully* minimizing the objective with respect to each block of parameters. In k -means there are two parameter blocks: $\{a_i\}_{i=1}^N$ and $\{z_j\}_{j=1}^k$. So, we need to derive update rules that find the minimizer of the k -means objective with respect to each block. It turns out there are simple solutions for both:

$$a_i \leftarrow \arg \min_{j \in \{1, \dots, k\}} \|z_j - \mathbf{x}^{(i)}\|_2^2 \quad \Leftarrow \text{Assign datapoint to nearest cluster} \quad (30.5)$$

$$\mathbf{z}_j \leftarrow \frac{1}{N} \sum_{i=1}^N \mathbb{1}(a_i = j) \mathbf{x}^{(i)} \quad \Leftarrow \text{Set code vector to cluster center} \quad (30.6)$$

These steps are repeated until a fixed point is reached, which occurs when all the datapoints that are assigned to each cluster are closest to that cluster's code vector.

Why are these the correct updates? Equation (30.5) is straightforward: it's just a brute force enumeration of all k possible assignments, from which we select the one that minimizes the k -means objective for that datapoint, given the current set of code vectors $\{z_j\}_{j=1}^k$.

Equation (30.6) is slightly trickier. It finds the optimal codes $\{z_j\}_{j=1}^k$ given the current set of assignments $\{a_i\}_{i=1}^N$. To see why equation (30.6) is optimal, first rewrite the k -means objective as follows:

$$J_{\text{kmeans}} = \sum_{i=1}^N \|z_{a_i} - \mathbf{x}^{(i)}\|_2^2 \quad (30.7)$$

$$= \sum_{j=1}^k \sum_{i=1}^N \mathbb{1}(a_i = j) \|z_j - \mathbf{x}^{(i)}\|_2^2 \quad (30.8)$$

Looking at one code vector (the j -th) in isolation, we are seeking:

$$\arg \min_{z_j} \sum_{i' \text{ s.t. } a_{i'} = j} \|z_j - \mathbf{x}^{(i')}\|_2^2 \quad (30.9)$$

where i' enumerates all the datapoints for which $a_{i'} = j$. Recall that the point that minimizes the sum of squared distances from a dataset is the mean of the dataset. This yields our update in equation (30.6): just set each code to be the mean of all datapoints assigned to that code's cluster.

Now we can see where the name k -means comes from: the optimal code vectors are the cluster *means*. The algorithm is very simple: compute the cluster means given current data assignments; reassign each datapoint to nearest cluster mean; recompute the means; and so on until convergence. An example of applying this algorithm to a simple dataset of two-dimensional (2D) points is given in [figure 30.10](#).

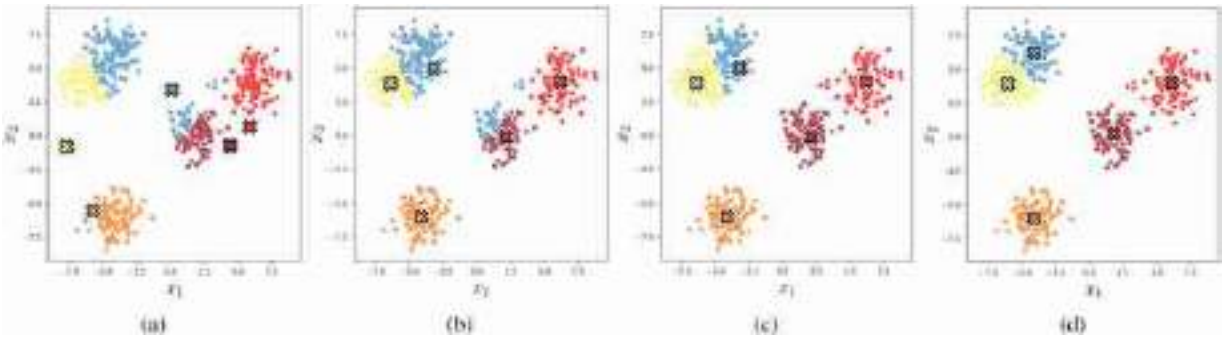


Figure 30.10: Iterations of k -means applied to a simple 2D dataset, with $k = 5$. The code vectors assigned to each cluster are marked with an x. (a) Initialization. (b) Update code vector to be cluster means. (c) Update assignments. (d) Converged solution (which occurs here after four updates of both codes and assignments).

30.9.2 K -Means from Multiple Perspectives

This section has presented k -means as a representation learning algorithm. In other texts you may encounter other views on k -means, for example, as a way of interpreting your data or as a simple generative model. These views are all complementary and all simultaneously true. Here we showed that k -means is like an autoencoder with a discrete bottleneck. In the next chapters we will encounter generative models, including one called a Gaussian mixture model (GMM). K -means can also be viewed as a vanilla form of a GMM. Later we will show that another kind of autoencoder, called a variational autoencoder (VAE), is a continuous version of a GMM. So we have a rich tapestry of relationships: autoencoders and VAEs are continuous versions of k -means and GMMs, respectively. GMMs and VAEs are formal probabilistic models that, respectively, extend k -means and autoencoders (which do not come with probabilistic semantics). This is just one set of connections that can be made. In this book, be on the lookout for more connections like this. It can be confusing at first to see multiple different perspectives on the same method: Which one is correct? But rarely is there

a single correct perspective. It is useful to identify the delta between each new model you encounter and all the models you already know. Usually there is a small delta with respect to *many* things you already know, and rarely is there no meaningful connection between any two arbitrary models. As an exercise, try picking a random concept on a random page in this book (or, for a challenge, any book on your bookshelf). What is the delta between that concept and k -means, or between that concept and a different concept on any other random page? In our experience, it will often be surprisingly small (but sometimes it takes a lot of effort to see the connection).

30.9.3 Clustering in Vision

In vision, the problem of clustering is related to the idea of **perceptual grouping**, which we cover in detail in chapter 31. We humans see the world as organized into different levels of perceptual structure: contours and surfaces, objects and events. These structures are groupings, or clusters, of related visual elements: a contour is a group of points that form a line, an object is a group of parts that form a cohesive, nameable whole, and so on. Algorithms like k -means, in a suitable feature space, can discover them.

30.10 Contrastive Learning

Dimensionality reduction and clustering algorithms learn compressed representations by creating an **information bottleneck**, that is, by constraining the number of bits available in the representation. An alternative compression strategy is to *supervise* what information should be thrown away. **Contrastive learning** is one such approach where a representation is supervised to be *invariant* to certain viewing transformations, resulting in a compressed representation that only captures the properties that are common between the different data **views**. Two different data views could correspond to two different cameras looking at the same scene or two different imaging modalities, such as color and depth, and we will see more examples subsequently.

Contrastive learning is actually more closely related to clustering than it may at first seem. Contrastive learning maps similar datapoints to similar embeddings. Clustering is just the extreme version of this where there are only k distinct embeddings and similar datapoints get mapped to the *exact same* embedding.

Learning invariant representations is a classic goal of computer vision. Recall that this was one of the reasons we used convolutional image filters: convolution is equivariant with camera translation, and invariance can then be achieved simply by pooling over filter responses. CNNs, through their convolutional architecture, bake translation invariance into the *hypothesis space*. In this section we will see how to incentivize invariances instead through the *objective* function.

The idea is to simply penalize deviations from the invariance we want. Suppose T is a transformation we wish our representation to be invariant to. Then we may use a loss of the form $\|f(T(\mathbf{x})) - f(\mathbf{x})\|_2^2$ to learn an encoder f that is invariant to T . We call such a loss an **alignment** loss [494].

That seems easy enough, but you may have noticed a flaw: What if f just learns to output the zero vector all the time? Trivial alignment can be achieved when there is representational collapse, and all datapoints get mapped to the same arbitrary vector.

Contrastive learning fixes this issue by coupling an alignment loss with a second loss that pushes apart embeddings of datapoints for which we do not want an invariant representation. The supervision for contrastive learning comes in the form of *positive pairs* and *negative pairs*. Positive pairs are two datapoints we wish to align in z -space; if we wish for invariance to T then a positive pair should be constructed as $\{\mathbf{x}, \mathbf{x}^+\}$ with $\mathbf{x}^+ = T(\mathbf{x})$. Negative pairs, $\{\mathbf{x}, \mathbf{x}^-\}$, are two datapoints that should be represented differently in z -space. Commonly, negative pairs are randomly constructed by sampling two datapoints independently and identically from the same data distribution, that is, $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ and $\mathbf{x}^- \sim p_{\text{data}}(\mathbf{x})$. Given such data pairings, the objective is to pull together the positive pairs and push apart the negative pairs, as illustrated in [figure 30.11](#).

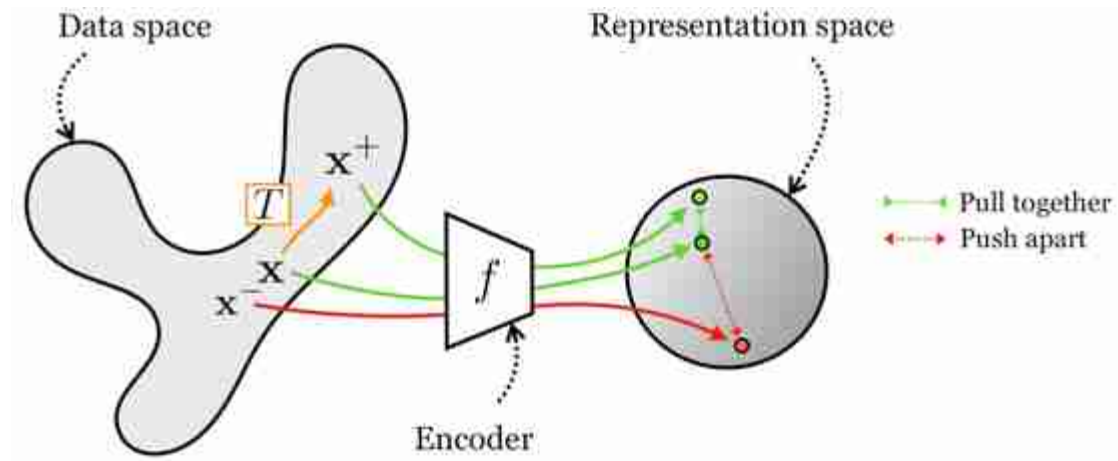


Figure 30.11: Contrastive learning.

This kind of contrastive learning results in an embedding that is invariant to a transformation T . Extending this to achieve invariance to a *set* of transformations $\{T_1, \dots, T_n\}$ is straightforward: just apply the same loss for each of $T_1, \dots, T_n$.

A second kind of contrastive learning is based on co-occurrence, where the goal is to learn a common representation of all co-occurring signals. This form of contrastive learning is useful for learning, for example, an audiovisual representation where the embedding of an image matches the embedding of the sound for that same scene. Or, returning to our colorization example, we can learn an image representation where the embedding of the grayscale channels matches the embedding of the color channels. In both these cases we are learning to align co-occurring sensory signals. This kind of contrastive learning is schematized in [figure 30.12](#).

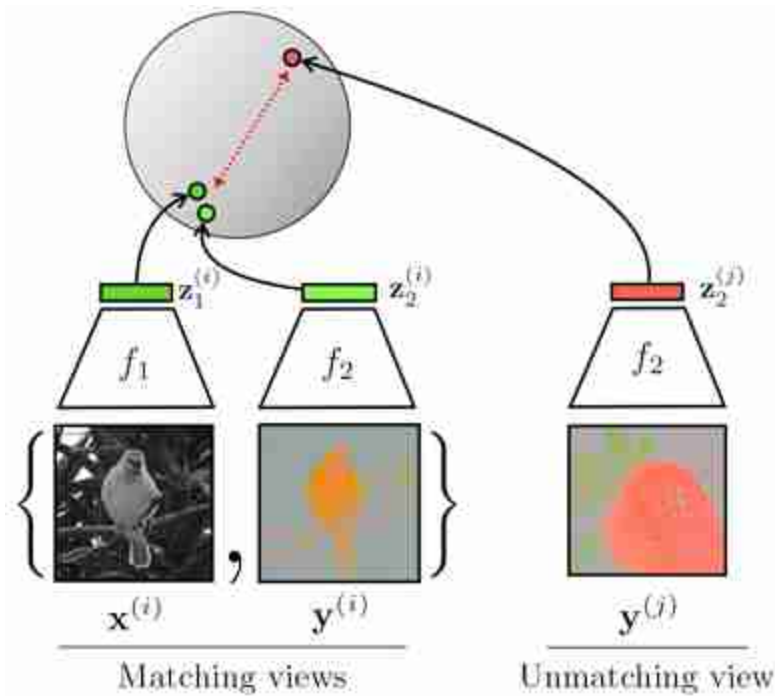
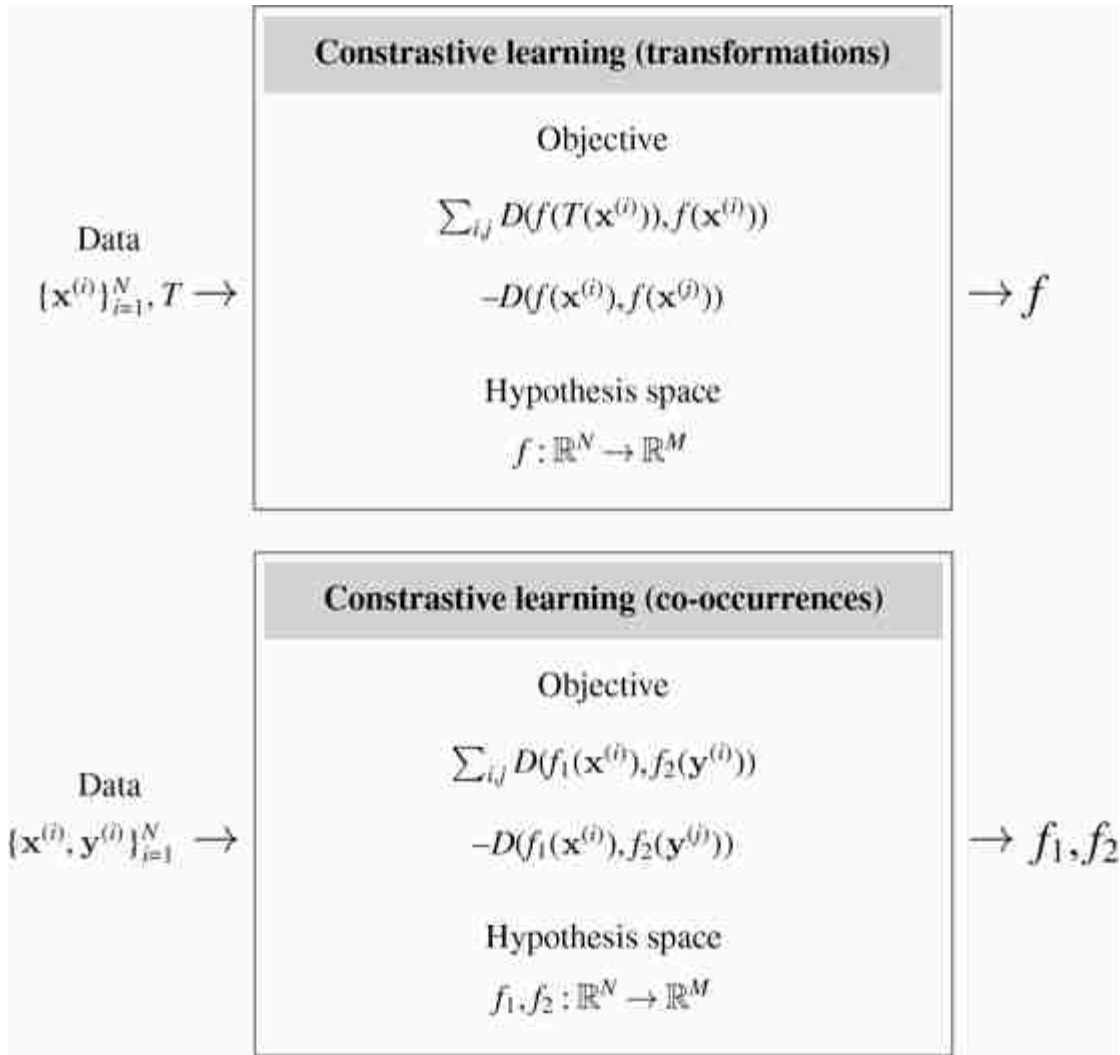


Figure 30.12: Contrastive learning from multiple views of the data. Figure inspired by [468].

In [figure 30.12](#), we refer to the two co-occurring signals—color and grayscale—as two different **views** of the total data tensor $\mathbf{X}$, just like we did in the previous sections: $\mathbf{x} = V_1(\mathbf{X})$, $\mathbf{y} = V_2(\mathbf{X})$. You can think of these views either as resulting from sensory co-occurrences or as two transformations of $\mathbf{X}$, where the transformation in the color example is channel dropping. Thus, the two kinds of contrastive learning we have presented are really one and the same: any two signals can be considered transformations of a combined total signal, and any signal and its transformation can be considered two co-occurring ways of measuring the underlying world.

Nonetheless, it is often easiest to conceptualize these two approaches separately, and next we give learning diagrams for each:



In these diagrams, D is a distance function. Above we give just one simple form for the contrastive objective; many variations have been proposed. Three of the most popular are (1) Hadsell et al.'s “contrastive loss” [180] (an older definition of the term, now overloaded with our more general notion of a contrastive loss being the broader family of any loss that pulls together positive samples and pushes apart negative samples), (2) the **triplet loss** [76], and (3) the **InfoNCE loss** [360]. Hadsell et al.'s contrastive loss and the triplet loss add the concept of a **margin** to the vanilla formulation: they only push/pull when the distance is less than a specified amount m (called the margin), otherwise points are considered far enough apart (or close enough together). The InfoNCE loss is a variation that treats establishing a contrast as a classification problem: it tries to move points apart until you can classify the positive sample, for a given anchor,

separately from all the negatives. The general formulation of these losses takes as input an anchor $\mathbf{x}$, a positive example $\mathbf{x}^+$, and one or more negative examples $\mathbf{x}^-$. The positive and negative may be defined based on transformations, cooccurrences, or something else. The full learning objective is to sum over many samples of anchors, positives, and negatives, producing a sampled set evaluated according to the losses as follows:

$$\mathcal{L}(\mathbf{x}, \mathbf{x}^+, \mathbf{x}^-) = \max(D(f(\mathbf{x}), f(\mathbf{x}^+)) - m_{\text{pos}}, 0) - \max(m_{\text{neg}} - D(f(\mathbf{x}), f(\mathbf{x}^-)), 0) \quad \triangleleft \text{Hadsell et al. "contrastive"} \quad (30.10)$$

$$\mathcal{L}(\mathbf{x}, \mathbf{x}^+, \mathbf{x}^-) = \max(D(f(\mathbf{x}), f(\mathbf{x}^+)) - D(f(\mathbf{x}), f(\mathbf{x}^-)) + m, 0) \quad \triangleleft \text{triplet} \quad (30.11)$$

$$\mathcal{L}(\mathbf{x}, \mathbf{x}^+, \{\mathbf{x}_i^-\}_{i=1}^N) = -\log \frac{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau}}{e^{f(\mathbf{x})^\top f(\mathbf{x}^+)/\tau} + \sum_i e^{f(\mathbf{x})^\top f(\mathbf{x}_i^-)/\tau}} \quad \triangleleft \text{InfoNCE} \quad (30.12)$$

Here m is a general margin parameter, and m_{pos} and m_{neg} are separate margins for the positive and negative pairs respectively.

Notice that the InfoNCE loss is a log softmax over a vector of scores $f_1(\mathbf{x})^\top f_2(c)/\tau$ with $c \in \{\mathbf{x}^+, \mathbf{x}_1^-, \dots, \mathbf{x}_N^-\}$; you can therefore think of this loss as corresponding to a classification problem where the ground truth class is $\mathbf{x}^+$ and the other possible classes are $\{\mathbf{x}_1^-, \dots, \mathbf{x}_N^-\}$ (refer to chapter 9 to revisit softmax classification).

30.10.1 Alignment and Uniformity

Wang and Isola [494] showed that the contrastive loss (specifically the InfoNCE form) encourages two simple properties of the embeddings: alignment and uniformity. We have already seen that alignment is the property that two views in a positive pair will map to the same point in embedding point, that is, the mapping is invariant to the difference between the views. **Uniformity** comes from the negative term, which encourages embeddings to spread out and tend toward an evenly spread, uniform distribution. Importantly, for this to work out mathematically, the embeddings must be *normalized*, that is, each embedding vector must be a unit vector. Otherwise, the negative term can push embeddings toward being infinitely far apart from each other. Fortunately, it is standard practice in contrastive learning (and many other forms of representation learning) to apply L_2 normalization to the embeddings. The result is that the embeddings will tend toward a uniform distribution over the surface of the M -

dimensional hypersphere, where M is the dimensionality of the embeddings. See theorem 1 in [494] for a formal statement of this fact.

A result of this analysis is we may explicit decompose contrastive learning into one loss for alignment and another for uniformity, with the following forms:

$$\mathcal{L}_{\text{align}}(f; \alpha) = \mathbb{E}_{(\mathbf{x}, \mathbf{x}^+) \sim p_{\text{pos}}} [\|f(\mathbf{x}) - f(\mathbf{x}^+)\|_2^\alpha] \quad (30.13)$$

$$\mathcal{L}_{\text{unif}}(f; t) = \log \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}, \mathbf{x}^- \sim p_{\text{data}}} [e^{-t\|f(\mathbf{x}) - f(\mathbf{x}^-)\|_2^2}] \quad (30.14)$$

$$\mathcal{L}(f; \alpha, t, \lambda) = \mathcal{L}_{\text{align}}(f; \alpha) + \lambda \mathcal{L}_{\text{unif}} \quad (30.15)$$

where p_{pos} is the distribution of positive pairs and α , t , and λ are hyperparameters of the losses.

30.10.2 Experiment: Designing Embeddings with Contrastive Learning

Using the alignment and uniformity objective defined previously, we will now illustrate how one can design embeddings with desired invariances. We will use the shape dataset described in section 30.4, and will use the same encoder architecture and optimizer as from that section (a CNN with six layers, Adam optimizer, 20,000 iterations of stochastic gradient descent, batch size of 128). In contrast to the autoencoder experiment, however, we will set the dimensionality of the embedding to $M = 2$, so that we can visualize it in a 2D plot.

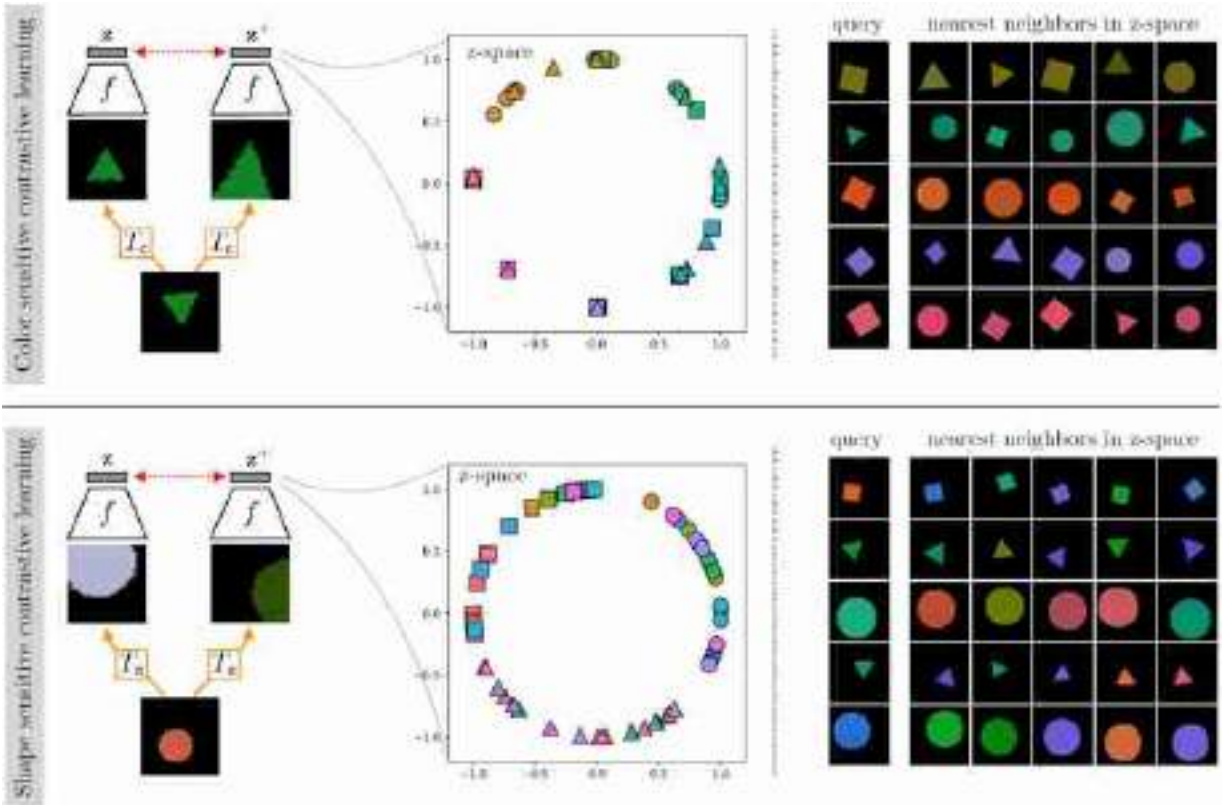


Figure 30.13: Contrastive learning on colored shapes using two different transformations for creating positive pairs. The choice of transformation controls which features the embedding becomes sensitive to and which it becomes invariant to.

Suppose we wish to obtain an embedding that is sensitive to color and invariant to shape. Then we should choose a view transformation T_c that preserves color while changing shape. A simple choice that turns out to work is for $T_c(x)$ to simply output a crop from the image x (this transformation does not really change the object's shape but still ends up resulting in shape-invariant embeddings because color is a much more obvious cue for the CNN to pick up on and use for solving the contrastive problem). The result of contrastive learning, using $L_{\text{align}} + L_{\text{unif}}$ on data generated from T_c is shown on the top row of [figure 30.13](#). Notice that the trained f maps colors to be clustered into the color classes of this toy data, and that these classes become spread out more or less uniformly across the surface of a circle in embedding space (a one-dimensional hypersphere, because the embeddings are 2D and normalized).

We can repeat the same experiment but with a view transformation T_s designed to be invariant to color. To do so we simply use the same transformation as for T_c (i.e., cropping) plus add a random shift in hue, brightness, and saturation. Training on views generated from T_s results in the embeddings on the bottom row of [figure 30.13](#). Now the embeddings become invariant to color but cluster shapes and spread out the three shape types to be roughly uniformly spaced around the hypersphere.

30.11 Concluding Remarks

This chapter, like much of this book, is about representations. Different representations make different problems easy, or hard. It is important to remember that every representation involves a set of tradeoffs: a good representation for one task may be a bad representation for another task. One of the current goals of computer vision research is to find *general-purpose representations*, and what this means is not that the representation will be good for all tasks but that it will be good for a wide variety of the tasks that humans care about, which is a very tiny subset of all possible tasks.

31 Perceptual Grouping

31.1 Introduction

The previous chapter discussed the importance of learning good visual representations, which can serve as the substrate for further vision algorithms. In this chapter we will continue our exploration of perceptual representations, but we will approach it from a slightly different school of thought, which is inherited from pre-deep learning computer vision and from human vision science. In this tradition, one particular kind of visual representation, the **perceptual group**, becomes the central object of interest; we will see what these are, how to find them, and how they are related to the representation learning algorithms from the last chapter.

First, what is a perceptual group? Look at the photo in [figure 31.1](#) and consider what you see. Not what raw pixel values you see, but what structures you see. What are some structures you can name?



Figure 31.1: A few kinds of perceptual organization in the human visual system.

What a camera sees is the raw RGB pixel values of the photo. But what your mind sees is rather different. You see contours and surfaces, objects and events.

Many of these perceptual structures can be thought of as *groups of primitive elements*: a contour is a group of points that form a line, an object

is a group of parts that form a cohesive whole, and so on. Because of this, understanding how humans, and machines, can identify meaningful perceptual groups has been a longstanding area of interest in both human and computer vision [372].

The content of this chapter can also go under the name **perceptual organization**, which is the study of how our vision system organizes the pixels into more abstracted structures, such as perceptual groups. The Gestalt psychologists were some of the first to study this problem [272].

This leads us to a central question: which primitives should be grouped? We will examine a few answers below, including the idea that primitives should be grouped based on *similarity* or based on *association*. We will also look at how natural groups can emerge as a byproduct of a downstream purpose.

31.2 Why Group?

Grouping is a special case of representation learning, and the reasons to group are the same as the reasons to do representation learning: groups are a good substrate for downstream analysis and processing. To get an intuition for why this is, consider how you might describe the scene around you. Go ahead and write down what you see. If you are in a cafe, you might write, “I’m sitting at a table with my laptop in front of me. A latte is next to the laptop; the foam and coffee are swirling into a spiral.”

Now let’s look at each of these words. Some refer to objects (e.g., “table”, “laptop”, “latte”). Others refer to spatial relationships (“in front”, “next to”), and still others refer to shapes (“spiral”) and events (“swirling”). These kinds of structures—objects, relationships, shapes, events—are the elements of perceptual organization, and you can think of perceptual organization as being a lot like describing an image in text. One of the hottest current trends in computer vision is to map images to text, then use the tools of natural language processing to reason about the image content. In this way text is treated as a powerful image representation, and indeed it has become clear that this kind of image representation—symbolic, discrete, semantic—can be exceedingly effective [13, 509, 178, 457].

Perceptual organization is all about trying to get similarly powerful representations, but doing so via bottom-up grouping rules, rather than by

mimicking human language. In other words, perceptual organization is about *discovering* language-like structures in the first place, from unsupervised principles rather than from human-language supervision. Why do we have the words we have, where did they come from? What exactly is an *object*? This chapter may start to provide some answers.

Although the analogy to text makes clear why perceptual organization is important, perceptual groups can also go beyond the limits of language. Some structures we see are hard to name but still clearly represent a coherent group in our heads. Consider, for example, the image in [figure 31.2](#) below.



[Figure 31.2:](#) How many objects do you see?

There are clearly two separate objects, but we don't have names for them. The next sections will cover a few different kinds of perceptual groups that are important in vision.

31.3 Segments

Segmentation is the problem of partitioning an image into regions that represent different coherent entities. There is no single definition of what is a coherent entity and it will vary depending on the task; a segment could correspond to a nameable object, or a image region made of a single material, or a physically distinct piece of geometry (see [figure 31.3](#)).

Segmentation can be framed as a clustering problem: assign each pixel in an image to one of k clusters. We will explore two approaches to solving this clustering problem: (1) assign similar pixels to the same cluster, (2)

assign two pixels to the same cluster if there is no boundary between them. We will start with the first.

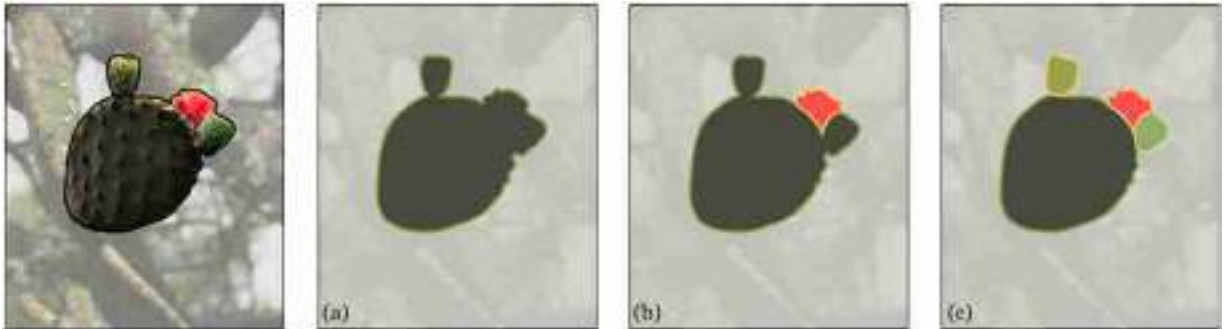


Figure 31.3: There are multiple ways you might segment any given image, and each way will highlight different world properties. This cactus branch could be viewed as (a) a single segment (the “branch”), (b) two segments (the “leaf” and the “flower”), or (c) four segments (the main “leaf,” two “buds,” and the “flower”).

31.3.1 K-Means Segmentation

K-means, which we covered in chapter 30, is one of the simplest and most useful clustering algorithms, so let's try it for segmentation.

K-means operates on a data matrix of N datapoints by M features per datapoint. Our first task is to convert the segmentation problem to that format. The simplest thing to do is let each pixel be a datapoint and the pixel's color be its feature vector. For ease of visualization, we will use the *ab* color value as the two-dimensional (2D) feature per pixel. An image represented this way can be plotted as a scatter plot and you can think of it as samples from a distribution over *ab* colors, as shown in [figure 31.4](#).

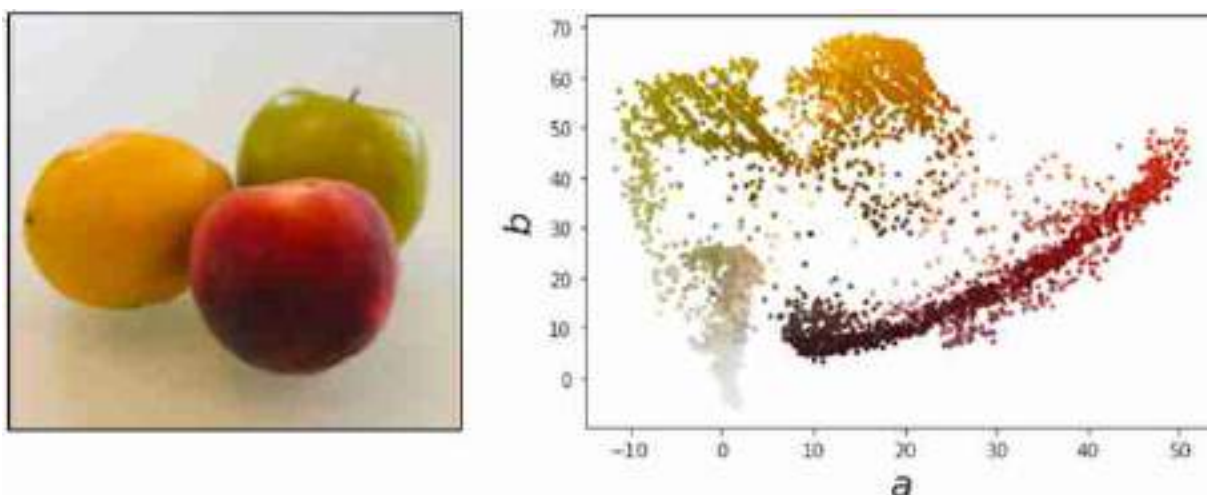


Figure 31.4: Different regions of coherent color in the image correspond to clusters in the scatter plot of all the ab -values of the image's pixels. In general, segmentation can be reduced to the problem of *clustering pixels* in some feature space (ab -color in this example, but it can be any pixel-wise feature).

K -means tries to find clusters in this scatter plot (and, if you prefer a probabilistic interpretation, it can be thought of as fitting a mixture of Gaussians to the datapoints, which aims to approximate the distribution from which these pixels were sampled). We can segment via k -means by simply assigning each pixel to the cluster assignment of its color value. Below we color each pixel in each cluster by the mean color of all pixels assigned to that cluster ([figure 31.5](#)).

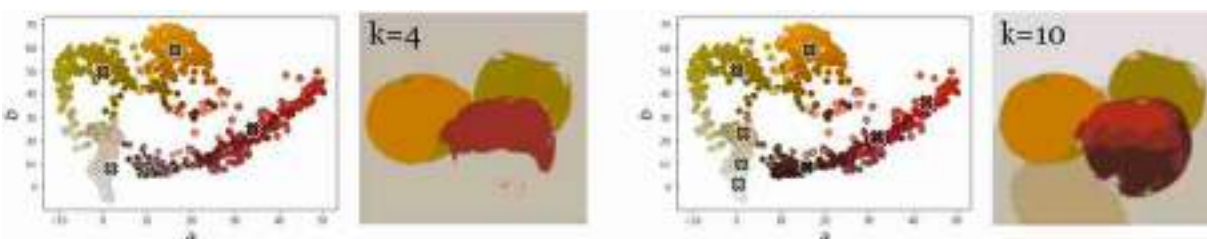


Figure 31.5: K -means segmentation on the fruits example.

Naturally, k -means on ab -values finds segments that correspond to regions of roughly a constant color. This simple approach can be surprisingly effective. notice how it segments a zebra from the background in [figure 31.6](#) (because the black and white stripes have roughly the same

ab-values). However, it fails to group more complex patterns that are not defined by color similarity, as in the beach photo in [figure 31.6](#).

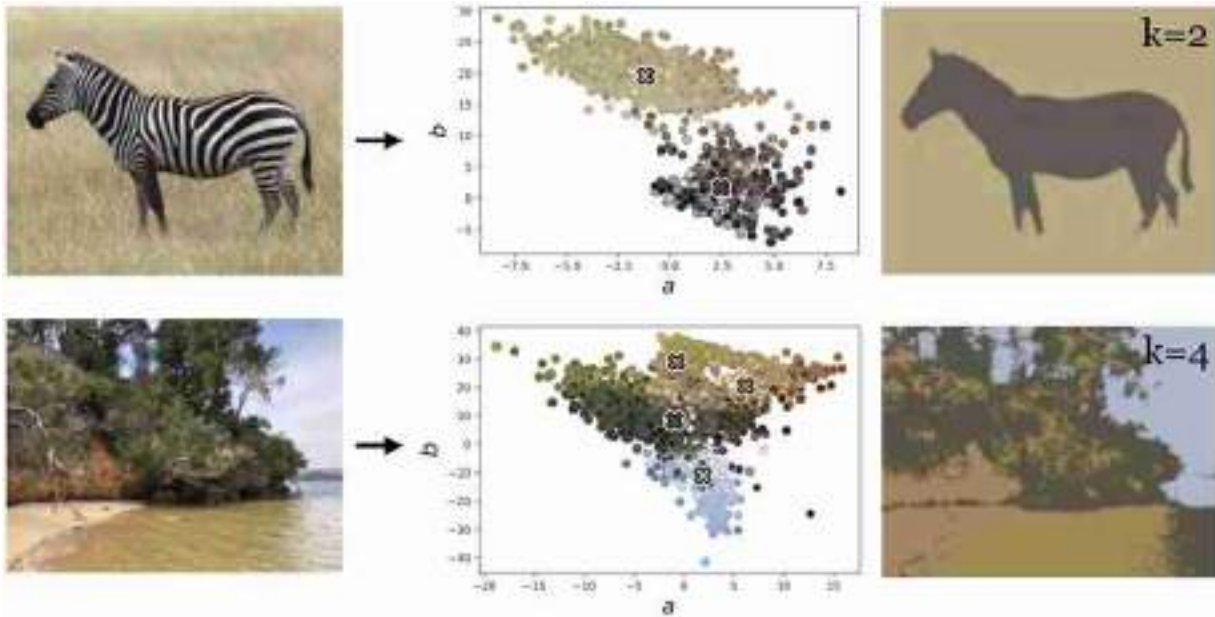


Figure 31.6: (top row): k -means segmentation succeeds on a seemingly hard image. *Photo source:* Fredo Durand. (bottom row): k -means fails on an image that, to our eye, looks no more complex.

31.3.2 Affinity-Based Segmentation

Other clustering methods are based on the idea of **affinity**, where we try to model a distance metric (the affinity) that tells us whether two items should be grouped. The distance metric usually measures how similar two patterns are, or how likely they are to be part of the same object.

For example, a simple affinity function over pixel pairs could just be the Euclidean distance between the color values of the two pixels. There are numerous ways we could use such an affinity to group pixels into segments, all of which share the general idea that pixels with high affinity should belong to the same group and pixels with low affinity should belong to two different groups. Methods that use this idea include those based on graph cuts [435], spectral clustering [18], and Markov random field (MRF)-based clustering (chapter 29).

We do not have time to cover all these methods and in fact they are no longer commonly used. Instead we will focus on a method you have seen earlier in the book that actually is in common use: contrastive learning

(which was covered in section 30.10). Contrastive learning is a perfect fit for affinity-based segmentation because contrastive methods learn from similarities (positive pairs) and differences (negative pairs), and this is what an affinity metric gives us! Contrastive methods learn an embedding in which datapoints with high affinity are nearby and datapoints with low affinity are far apart. Once we have such an embedding, clustering is easy in the embedding space, and pretty much any off-the-shelf clustering algorithm will work; for simplicity, we will stick with k -means in this book.

In this book we only present one way to do affinity-based clustering: first use affinities to supervise a embedding in which the data is nearly clustered, then read off the clusters using a simple quantization method like k -means. There are many other affinity-based methods but this simple method often works well.

If we define affinities in terms of color distance, then there's no point in *learning* such an embedding, because the raw colors themselves are already a great embedding under that notion of affinity. Affinity-based clustering only really becomes interesting when we have nontrivial affinities to model; for example, they could be given by human judgments about which pixels should go together in natural images.

Because human supervision is expensive and limiting, a popular alternative is to use self-supervised affinity measures, just like we saw in self-supervised contrastive learning. Popular contrastive learning methods like SimCLR [79], MoCo [193], and DINO [70], train a mapping from image patches to embedding vectors such that two patches that come from the same image embed near each other and patches from different images embed far apart. We can run these methods *densely* (in a sliding window fashion) across a single image to create a feature map of size $C \times N \times M$, where N and M are the spatial dimensions of the feature map and C is the dimensionality of the embedding vectors. This creates a new image where each pixel has a value (the embedding vector) that is similar to other pixels that the model thinks would be likely to appear in the same scene. In effect, we have learned an affinity metric based on natural patch co-occurrences, then used that affinity metric to obtain a feature map that reveals the coherent regions of an image. If we apply k -means to the pixels in this feature map, we can obtain an effective segmentation. This entire pipeline is shown in [figure 31.7](#), using DINO [70] as the particular contrastive embedding.

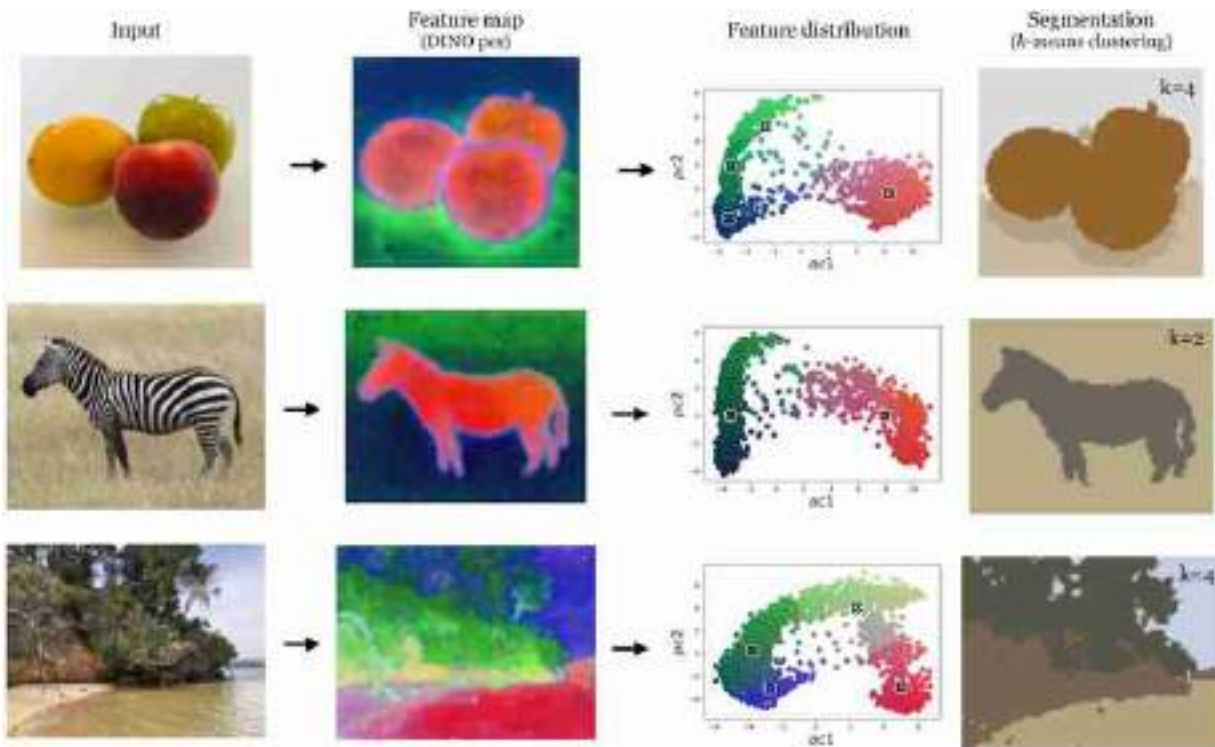


Figure 31.7: Image segmentation using k -means over DINO [70] features.

31.4 Edges, Boundaries, and Contours

A complementary problem to segmentation is boundary detection, because a partitioning of an image can be represented either by partition assignments (segments) or by the boundaries between partitions. Many vision algorithms organize images into different kinds of boundaries between perceptual structures. Early vision algorithms focused on **edge detection**, where the idea was that the edges in an image should form a good, compact representation of the image's content, because edges are relatively sparse, and they are stable with respect to nuisances like noise and lighting. Subsequent work identified connections between different kinds of edge structure and scene geometry; for example, an edge that represents an occluding contour indicates that there is a depth discontinuity at that location in the scene.

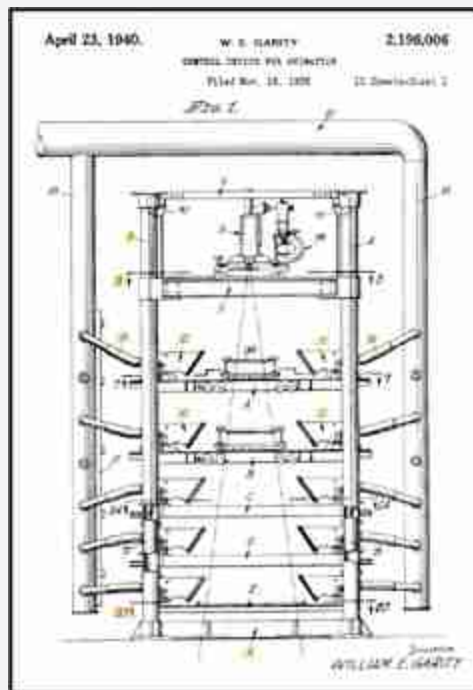
Chapter 2 described some simple edge detectors and showed their use in reconstructing scene geometry. Chapter 42 covers more recent approaches to inferring three-dimensional (3D) scene structure, which have largely superseded the methods that explicitly reasoned about edges.

Line drawings are another way of representing the edge structure in an image, and they seem to be of high interest to humans. This suggests that there may be something useful about representing images in terms of their edge structure. Although there is an active line of research in computer graphics on modeling line drawings, this work has not yet paid off in vision, and it remains unclear whether or not line drawings will one day prove as compelling to visually intelligent machines as they do to us humans.

31.5 Layers

If you have ever seen an early Disney cartoon (e.g., Snow White), you may have noticed a compelling illusion of depth created by parallax between different layers of the animated scene.

An schematic of a multiplane camera from William E. Garity's 1940 patent for a "Control Device for Animation."



Animators for these cartoons used a device called a multiplane camera, which worked as follows. A camera looked at a stack of glass planes. On each plane, the animators would draw a different **layer** of the scene. For example, on the back plane would be the sky, then there would be mountains on the next plane, then trees, and then the foreground characters.

During filming the planes could be shifted back and forth to create parallax where the foreground planes move by more quickly than the planes further back.

This device works because it is a rough approximation to the actual 3D structure of the world. In computer vision, **layer-based** representations like this have been popular as a way of capturing some aspects of depth without the expense of modeling a fully 3D world; sometimes layers and other related pseudo-3D representations are called **2.5D representations**.

Layers are not just an approximation to true depth, but also capture geometric structure that is *not* well-represented by depth. Consider a pile of papers on your desk. The depth difference between one page and the next might only be a millimeter—too small to show up on standard depth sensing cameras. But the layer structure is obvious. Look back at [figure 31.2](#): the depth *order* is obvious even though you might have no idea how far apart these objects are in metric space. Layers capture **ordinal** structure and often this is the structure we care about; for example, if we want to pick up a piece of paper in our pile, the main thing that matters is if it is on top.

31.6 Emergent Groups

It may seem like some of the topics in this chapter are out of fashion in modern computer vision. Edge detection and bottom-up segmentation are not the workhorses they once were. But this doesn't mean that perceptual organization is absent from current systems. These systems still see the world in terms of objects, contours, layers, and so forth, but those structures are emergent in their hidden layers rather than made explicit by human design. For example, in figure 30.8 we observed object detectors inside a neural net trained to do colorization.

31.7 Concluding Remarks

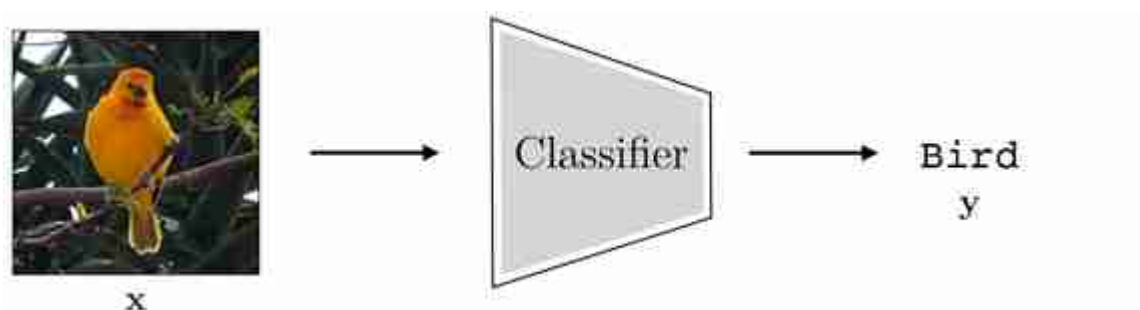
Perceptual organization aims to represent visual content in terms of modular and disentangled elements that make subsequent processing easier. In this way, perceptual organization shares the same goal as visual representation learning, although the former deals more with discrete structures whereas the latter has focused mostly on continuous representations. These two fields evolved somewhat separately, with perceptual organization having its

roots in psychology and representation learning coming more from machine learning, but these days the end result is converging in the form of vision systems that see the world in terms of objects, parts, relationships, layers, and many other useful structures.

32 Generative Models

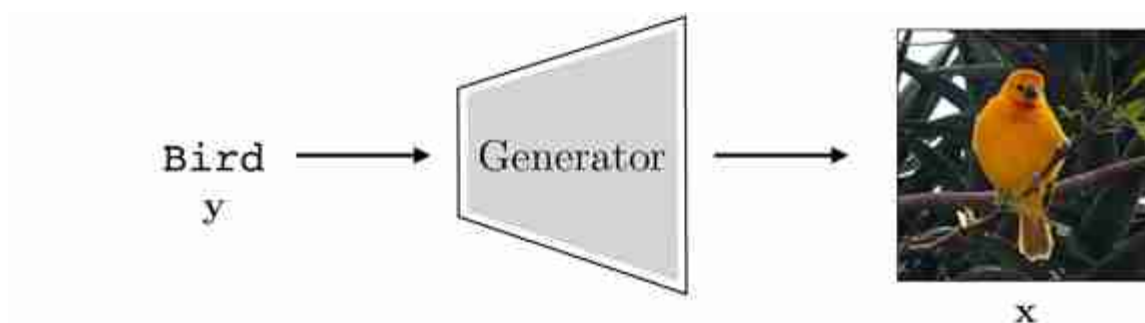
32.1 Introduction

Generative models perform (or describe) the synthesis of data. Recall the image classifier from chapter 9, shown again below ([figure 32.1](#)):



[Figure 32.1](#): A classifier maps images to labels.

A generative model does the opposite ([figure 32.2](#)):



[Figure 32.2](#): A generator maps labels (or other descriptions) to images.

Whereas an image classifier is a function $f: X \rightarrow Y$, a generative model is a function in the opposite direction $g: Y \rightarrow X$. Things are a bit different in this direction. The function f is **many-to-one**: there are many images that all

should be given the same label “bird.” The function g , conversely, is **one-to-many**: there are many possible outputs for any given input. Generative models handle this ambiguity by making g a **stochastic function**.

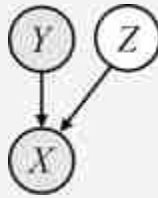
Although we describe y as a label here, generative models can in fact take other kinds of instructions as inputs, such as text descriptions of what we want to generate or hand-drawn sketches that we wish the model to fill in.

One way to make a function stochastic is to make it a deterministic function of a stochastic input, and this is how most of the generative models in this chapter will work. We define g as a function, called a **generator**, that takes a randomized vector z as input along with the label/description y , and produces an image x as output, that is, $g : Z \times Y \rightarrow X$. Then this function can output a different image x for each different setting of z . The generative process is as follows. First sample z from a prior $p(Z)$, then deterministically generate an image based on z and y :

$$z \sim p(Z) \quad (32.1)$$

$$x = g(z, y) \quad (32.2)$$

Graphical model for generating $X | Y, Z$:



This procedure is shown in [figure 32.3](#):

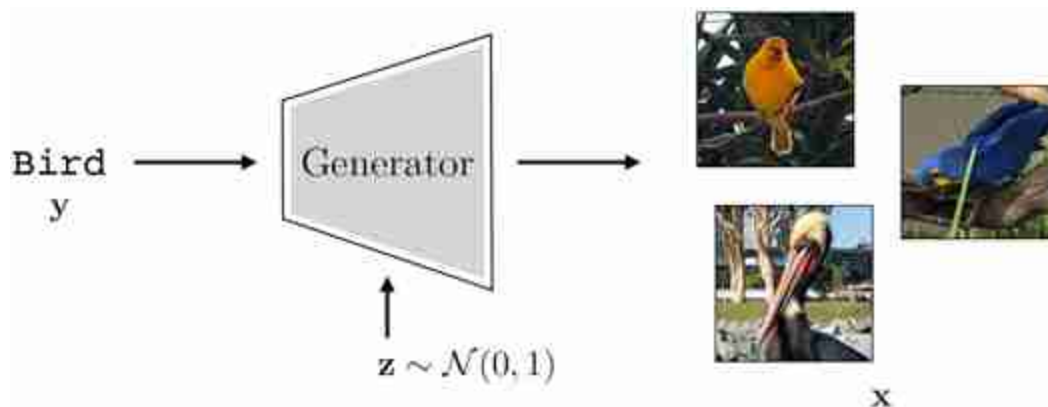


Figure 32.3: Making a generator stochastic by conditioning on a random variable.

Usually we draw z from a simple distribution such as Gaussian noise, that is, $p(Z) = \mathcal{N}(\mathbf{0}, \mathbf{1})$. The way to think of z is that it is a vector of **latent variables**, which specify all the attributes of the images other than those determined by the label. These variables are called latent because they are not directly observed in the training data (it just consists of images x and their labels y). In our example, y tells g to make a bird but the z -vector is what specifies exactly which bird. Different dimensions of z specify different attributes, such as the size, color, pose, background, and so on; everything necessary to fully determine the output image.

The z -vector is also sometimes called **noise**, but noise has a very different meaning here than in signal processing. In signal processing, noise is a nuisance to get rid of. In generative modeling, *the noise is the signal*; it determines exactly which image to generate.

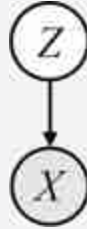
32.2 Unconditional Generative Models

Sometimes we wish to simply make up data from scratch; in fact, this is the canonical setting in which generative models are often studied. To do so, we can simply drop the dependency on the input y . This yields a procedure for making data from scratch:

$$\mathbf{z} \sim p(\mathbf{Z}) \quad (32.3)$$

$$\mathbf{x} = g(\mathbf{z}) \quad (32.4)$$

Graphical model for an unconditional generative model:



We call this an **unconditional generative model** because it is model of the unconditional distribution $p(X)$. Generally we will refer to unconditional generative models simply as generative models and use the term **conditional generative model** for a model of any conditional distribution $p(X | Y)$. Conditional generative models will be the focus of chapter 34; in the present chapter we will restrict our attention, from this point on, to unconditional models.

Why bother with (unconditional) generative models, which make up random synthetic data? At first this may seem a silly goal. Why should we care to make up images from scratch? One reason is *content creation*; we will see other reasons later, but content creation is a good starting point. Suppose we are making a video game and we want to automatically generate a bunch of exciting levels for the player the explore. We would like a procedure for making up new levels from scratch. Such **procedural graphics** have been successfully used to generate random landscapes and cities for virtual realities [377]. Suppose we want to add a river to a landscape. We need to decide what path the river should take. A simple program for generating the path could be “walk an increment forward, flip a coin to decide whether to turn left or right, repeat.”

Here is that program in pseudocode:

Algorithm 32.1: A simple generative model that draws images of rivers.

```
1 Input: Random vector of coin flips  $\mathbf{z} = [z_1, \dots, z_n]$  with each  $z_i \in \{0, 1\}$ 
2 Output: Picture of a river
3 start drawing at origin with heading =  $90^\circ$ 
4 for  $i = 1, \dots, n$  do
5   extend line 1 unit in current heading direction
6   if  $z_i == 1$  then
7     rotate heading  $10^\circ$  to the right
8   else
9     rotate heading  $10^\circ$  to the left
```

Here are a few rivers this program draws ([figure 32.4](#)):



Figure 32.4: Procedurally generated rivers.

This program relies on a sequence of coin flips to generate the path of the river. In other words, the program took a randomized vector (noise) as input, and converted this vector into an image of the path of the river. It's exactly the same idea as we described previously for generating images of birds, just this time the generator is a program that makes rivers ([figure 32.5](#)):

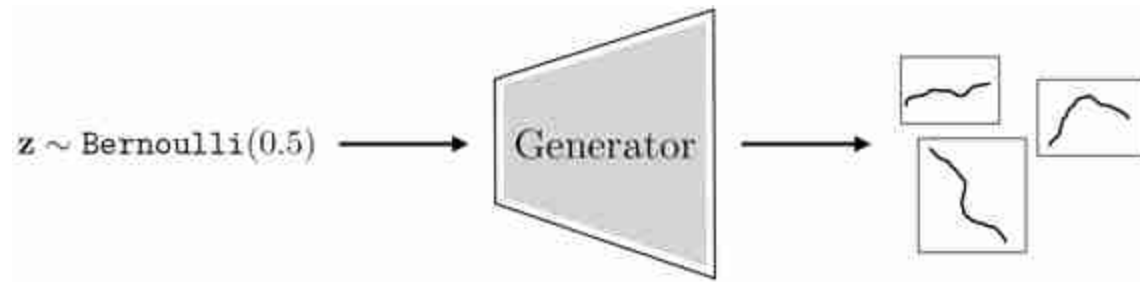


Figure 32.5: A generator that makes procedural rivers.

This generator was written by hand. Next we will see generative models that *learn* the program that synthesizes data.

32.3 Learning Generative Models

How can we learn to synthesize images that look realistic? The machine learning way to do this is to start with a training set of *examples* of real images, $\{\mathbf{x}^{(i)}\}_{i=1}^N$. Recall that in supervised learning, an *example* was defined as an $\{\text{input}, \text{output}\}$ pair; here things are actually no different, only the input happens to be the null set. We feed these examples to a learner, which spits out a generator function. Later, we may query the generator with new, randomized $\mathbf{z}$ -vectors to produce novel outputs, a process called **sampling** from the model. This two-stage procedure is shown in [figure 32.6](#):

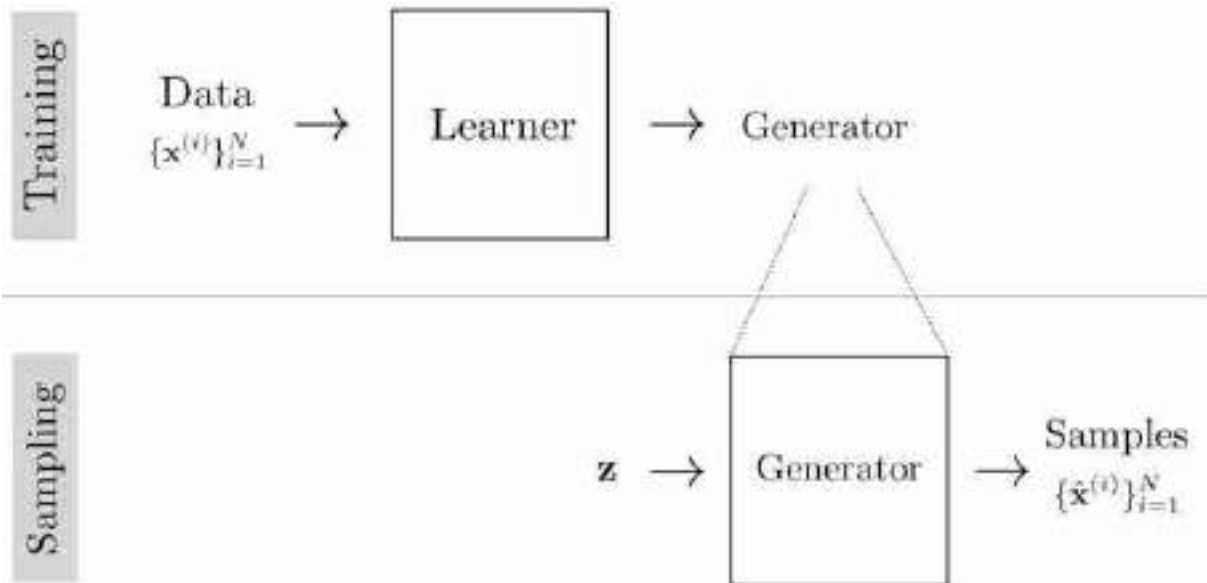


Figure 32.6: Learning and using a generator.

32.3.1 What's the Objective of Generative Modeling?

The objective of the learner is to create a generator that produces *synthetic* data, $\{\hat{\mathbf{x}}^{(i)}\}_{i=1}^N$, that looks like the *real* data, $\{\mathbf{x}^{(i)}\}_{i=1}^N$. There are a lot of ways of define “looks like” and they each result in a different kind of generative model. Two examples are:

1. Synthetic data looks like real data if matches the real data in terms of certain marginal statistics, for example, it has the same mean color as real photos, or the same color variance, or the same edge statistics.
2. Synthetic data looks like real data if it has high probability under a density model fit to the real data.

The first approach is the one we saw in chapter 27 on statistical image models, where synthetic textures were made that had the same filter response statistics as a real training example. This approach works well when we only want to match certain properties of the training data. The second approach, which is the main focus of this chapter, is better when we want to produce synthetic data that matches *all* statistics of the training examples.^{[12](#)}

To be precise, the goal of the deep generative models we consider in this chapter is to produce synthetic data that is *identically distributed* as the

training data, that is, we want $\mathbf{x} \sim p_{\text{data}}$ where p_{data} is the true process that produced the training data.

What if our model just memorizes all the training examples and generates random draws from this memory? Indeed, memorized training samples perfectly satisfy the goal of making synthetic data that looks real. A second, sometimes overlooked, property of a good generative model is that it be a simple, or smooth, function, so that it generalizes to producing synthetic samples that look like the training data but are not identical to it. A generator that only regurgitates the training data is overfit in the same way a classifier that memorizes the training data is overfit. In both cases, the true goal is to fit the training data and to do so with a function that generalizes.

32.3.2 The Direct Approach and the Indirect Approach

There are two general approaches to forming data generators:

1. Direct approach: learn the function $G : Z \rightarrow X$.
2. Indirect approach: learn a function $E : X \rightarrow \mathbb{R}$ and generate samples by finding values for $\mathbf{x}$ that score highly under this function.

In the generative modeling literature, the direct approach is sometimes called an *implicit* model. This is because the probability density is never explicitly represented. However, note that this “implicit” model explicitly describes the way data is generated. To avoid confusion, we will not use this terminology.

So far in this chapter we have focused on the direct approach, and it is schematized in [figure 32.6](#). Interestingly, the direct approach only became popular recently, with models like **generative adversarial networks (GANs)** and **diffusion models**, which we will see later in this chapter. The indirect approach is more classical, and describes many of the methods we saw in chapter 27. Indirect approaches come in two general flavors, density models and energy models, which we will describe next. Both follow the schematic given in [figure 32.7](#):

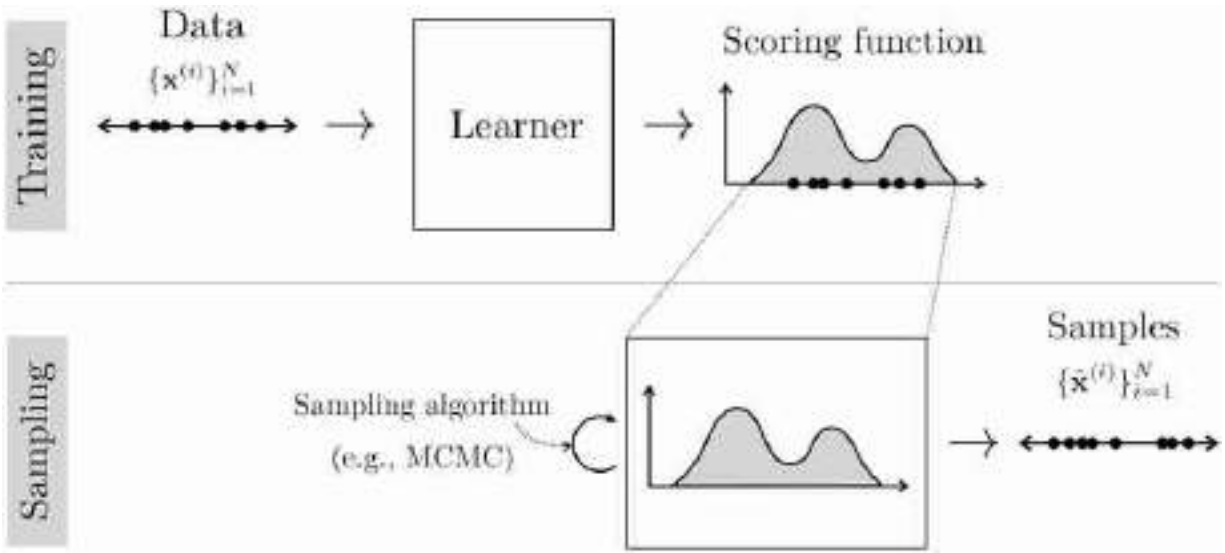


Figure 32.7: The indirect approach to generative modeling. The scoring function can be either a probability density or an energy function. The models we learned about in chapter 27 follow this recipe (see figures 27.4 and 27.5 in particular).

32.4 Density Models

Some generative models not only produce generators but also yield a probability density function p_θ , fit to the training data. This density function may play a role in training the generator (e.g., we want the generator to produce samples from p_θ), or the density function may be the goal itself (e.g., we want to be able to estimate the probability of datapoints in order to detect anomalies).

In fact, some generative models *only* produce a density p_θ and do not learn any explicit generator function. Instead, samples can be drawn from p_θ using a sampling algorithm, such as **Markov Chain Monte Carlo (MCMC)**, that takes a density as input and produce samples from it as output. This is a form of the indirect approach to synthesizing data that we mentioned above ([figure 32.7](#)).

32.4.1 Learning Density Models

The objective of the learner for a density model is to output a density function p_θ that is as close as possible to p_{data} . How should we measure closeness? We can define a **divergence** between the two distributions, D , and then solve the following optimization problem:

$$\arg \min_{p_\theta} D(p_\theta, p_{\text{data}}) \quad (32.5)$$

The problem is that we do not actually have the function p_{data} , we only have samples from this function, $\mathbf{x} \sim p_{\text{data}}$. Therefore, we need a divergence D that measures the distance¹³ between p_θ and p_{data} while only accessing samples from p_{data} . A common choice is to use the **Kullback-Leibler (KL) divergence**, which is defined as follows:

$$p_\theta^* = \arg \min_{p_\theta} \text{KL}(p_{\text{data}} \parallel p_\theta) \quad (32.6)$$

$$= \arg \min_{p_\theta} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[-\log \frac{p_\theta(\mathbf{x})}{p_{\text{data}}(\mathbf{x})} \right] \quad (32.7)$$

$$= \arg \max_{p_\theta} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log p_\theta(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log p_{\text{data}}(\mathbf{x})] \quad (32.8)$$

$$= \arg \max_{p_\theta} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log p_\theta(\mathbf{x})] \quad \leftarrow \text{dropped second term since no dependence on } p_\theta \quad (32.9)$$

$$\approx \arg \max_{p_\theta} \frac{1}{N} \sum_{i=1}^N \log p_\theta(\mathbf{x}^{(i)}) \quad (32.10)$$

where the final line is an empirical estimate of the expectation by sampling over the training dataset $\{\mathbf{x}^{(i)}\}_{i=1}^N$.

$\text{KL}(p_{\text{data}} \parallel p_\theta)$ is sometime called the *forward* KL divergence, and it measures the probability of the data under the model distribution. The *reverse* KL divergence, $\text{KL}(p_\theta \parallel p_{\text{data}})$, does the converse, measuring the probability of the model's samples under the data distribution; because we do not have access to the true data distribution, the reverse KL divergence usually cannot be computed.

Equation (32.9) is the *expected log likelihood of the training data under the model's density function*. Maximizing this objective is therefore a form of **max likelihood learning**. Pictorially we can visualize the max likelihood objective as trying to push up the density over each observed datapoint:

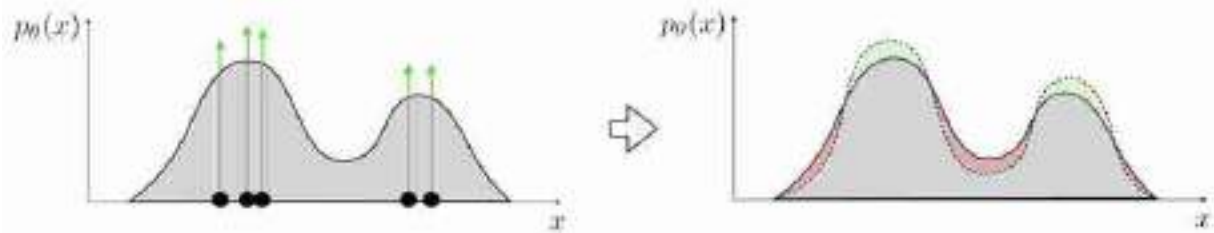


Figure 32.8: Fitting a max likelihood density model to data. The gray region holds a constant amount of mass; think of it as piles of dirt. To increase the amount of dirt at the locations of the green arrows you must remove dirt from other regions, indicated in red.

Remember that a probability density function (pdf) is a function $p_\theta : X \rightarrow [0, \infty)$ with $\int_{\mathbf{x}} p_\theta(\mathbf{x}) d\mathbf{x} = 1$ (i.e. it's normalized). To learn a pdf, we will typically learn the parameters of a family of pdfs. For example, in section 32.6, we will cover learning the parameters of the Gaussian family of pdfs. All members of such a the family are normalized nonnegative functions; this way we do not need to add an explicit constraint that the learned function have these properties, we are simply searching over a space of functions *all of which* have these properties. This means that whenever we push up density over datapoints, we are forced to sacrifice density over other regions, so implicitly we are removing density from places where there is no data, as indicated by the red regions in [figure 32.8](#).

In the next section we will see an alternative approach where the parametric family we search over need not be normalized.

32.5 Energy-Based Models

Density models constrain the learned function to be normalized, that is, $\int_{\mathbf{x}} p_\theta(\mathbf{x}) d\mathbf{x} = 1$. This constraint is often hard to realize. One approach is to learn an unnormalized function E_θ , then convert it to the normalized density $p_\theta = \frac{e^{-E_\theta}}{Z(\theta)}$, where $Z(\theta) = \int_{\mathbf{x}} e^{-E_\theta(\mathbf{x})}$ is the normalizing constant. The $Z(\theta)$ can be very expensive to compute and often can only be approximated.

The parametric form $\frac{e^{-E_\theta}}{Z(\theta)}$ is sometimes referred to as a **Boltzmann** or **Gibbs distribution**.

Notice that Z is a function of model parameters θ but not of datapoint $\mathbf{x}$, since we integrate over all possible data values.

Note that low energy $\Rightarrow$ high probability. So we minimize energy to maximize probability.

Energy-based models (EBMs) address this by simply skipping the step where we normalize the density, and letting the output of the learner just be E_θ . Even though it is not a true probability density, E_θ can still be used for many of the applications we would want a density for. This is because we can compare *relative probabilities* with E_θ :

$$\frac{p_\theta(\mathbf{x}_1)}{p_\theta(\mathbf{x}_2)} = \frac{e^{-E_\theta(\mathbf{x}_1)}/Z(\theta)}{e^{-E_\theta(\mathbf{x}_2)}/Z(\theta)} = \frac{e^{-E_\theta(\mathbf{x}_1)}}{e^{-E_\theta(\mathbf{x}_2)}} \quad (32.11)$$

Knowing relative probabilities is all that is required for sampling (via MCMC), for outlier detection (the relatively lowest probability datapoint in a set of datapoints is the outlier), and even for optimizing over a space of data to find the datapoint that is max probability (because $\arg \max_{\mathbf{x} \in X} p_\theta(\mathbf{x}) = \arg \max_{\mathbf{x} \in X} -E_\theta(\mathbf{x})$). To solve such a maximization problem, we might want to find the gradient of the log probability density with respect to $\mathbf{x}$; it turns out this gradient is identical to the gradient of $-E_\theta$ with respect to $\mathbf{x}$!

$$\nabla_{\mathbf{x}} \log p_\theta(\mathbf{x}) = \nabla_{\mathbf{x}} \log \frac{e^{-E_\theta(\mathbf{x})}}{Z(\theta)} \quad (32.12)$$

$$= -\nabla_{\mathbf{x}} E_\theta(\mathbf{x}) - \nabla_{\mathbf{x}} \log Z(\theta) \quad (32.13)$$

$$= -\nabla_{\mathbf{x}} E_\theta(\mathbf{x}) \quad (32.14)$$

In sum, energy functions can do most of what probability densities can do, except that they do not give normalized probabilities. Therefore, they are insufficient for applications where communicating probabilities is important for either interpretability or for interfacing with downstream systems that require knowing true probabilities.

A case where a density model might be preferable over an energy model is a medical imaging system that needs to communicate to doctors the likelihood that a patient has cancer.

32.5.1 Learning Energy-Based Models

Learning the parameters of an energy-based model is a bit different than learning the parameters of a density model. In a density model, we simply increase the probability mass over observed datapoints, and because the density is a normalized function, this implicitly pushes down the density assigned to regions where there is no observed data. Since energy functions are not normalized, we need to add an explicit negative term to push up energy where there are no datapoints, in addition to the positive term of pushing down energy where the data is observed. One way to do so is called **contrastive divergence** [206]. On each iteration of optimization, this method modifies the energy function to decrease the energy assigned to samples from the data (positive term) and to increase the energy assigned to samples from the model (i.e., samples from the energy function itself; negative term), as shown in [figure 32.9](#):

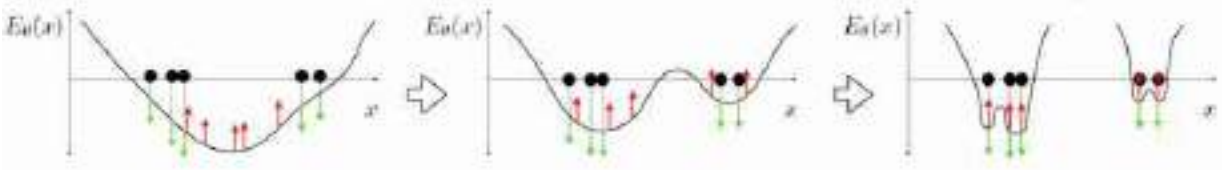


Figure 32.9: Fitting a max likelihood energy function to data, using contrastive divergence [206].

Once the energy function perfectly adheres to the data, samples from the model are identical to samples from the data and the positive and negative terms cancel out. This should make intuitive sense because we don't want the energy function to change once we have perfectly fit the data. It turns out that mathematically this procedure is an approximation to the gradient of the log likelihood function. Defining $p_\theta = \frac{e^{-E_\theta}}{Z(\theta)}$, start by decomposing the gradient of the log likelihood into two terms, which, as we will see, will end up playing the role of positive and negative terms:

$$\nabla_\theta \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log p_\theta(\mathbf{x})] = \nabla_\theta \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[\log \frac{e^{-E_\theta(\mathbf{x})}}{Z(\theta)} \right] \quad (32.15)$$

$$= -\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\nabla_\theta E_\theta(\mathbf{x})] - \nabla_\theta \log Z(\theta) \quad (32.16)$$

The first term is the positive term gradient, which tries to modify parameters to decrease the energy placed on data samples. The second term

is the negative term gradient; here it appears as an intractable integral, so our strategy is to rewrite it as an expectation, which can be approximated via sampling:

In equation (32.17) we use a very useful identity from the chain rule of calculus, which appears often in machine learning: $\nabla_x \log f(x) = \frac{1}{f(x)} \nabla_x f(x)$

$$\nabla_{\theta} \log Z(\theta) = \frac{1}{Z(\theta)} \nabla_{\theta} Z(\theta) \quad (32.17)$$

$$= \frac{1}{Z(\theta)} \nabla_{\theta} \int_{\mathbf{x}} e^{-E_{\theta}(\mathbf{x})} d\mathbf{x} \quad \triangleleft \text{definition of } Z \quad (32.18)$$

$$= \frac{1}{Z(\theta)} \int_{\mathbf{x}} \nabla_{\theta} e^{-E_{\theta}(\mathbf{x})} d\mathbf{x} \quad \triangleleft \text{exchange sum and grad} \quad (32.19)$$

$$= \frac{1}{Z(\theta)} - \int_{\mathbf{x}} e^{-E_{\theta}(\mathbf{x})} \nabla_{\theta} E_{\theta}(\mathbf{x}) d\mathbf{x} \quad (32.20)$$

$$= - \int_{\mathbf{x}} \frac{e^{-E_{\theta}(\mathbf{x})}}{Z(\theta)} \nabla_{\theta} E_{\theta}(\mathbf{x}) d\mathbf{x} \quad (32.21)$$

$$= - \int_{\mathbf{x}} p_{\theta}(\mathbf{x}) \nabla_{\theta} E_{\theta}(\mathbf{x}) d\mathbf{x} \quad \triangleleft \text{definition of } p_{\theta} \quad (32.22)$$

$$= -\mathbb{E}_{\mathbf{x} \sim p_{\theta}} [\nabla_{\theta} E_{\theta}(\mathbf{x})] \quad \triangleleft \text{definition of expectation} \quad (32.23)$$

Plugging equation (32.23) back into equation (32.16), we arrive at:

$$\nabla_{\theta} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log p_{\theta}(\mathbf{x})] = -\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\nabla_{\theta} E_{\theta}(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p_{\theta}} [\nabla_{\theta} E_{\theta}(\mathbf{x})] \quad (32.24)$$

Both expectations can be approximated via sampling: defining $\mathbf{x}^{(i)} \sim p_{\text{data}}$ and $\hat{\mathbf{x}}^{(i)} \sim p_{\theta}$, and taking N such samples, we have

$$-\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\nabla_{\theta} E_{\theta}(\mathbf{x})] + \mathbb{E}_{\mathbf{x} \sim p_{\theta}} [\nabla_{\theta} E_{\theta}(\mathbf{x})] \approx -\frac{1}{N} \sum_{i=1}^N \nabla_{\theta} E_{\theta}(\mathbf{x}^{(i)}) + \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} E_{\theta}(\hat{\mathbf{x}}^{(i)}) \quad (32.25)$$

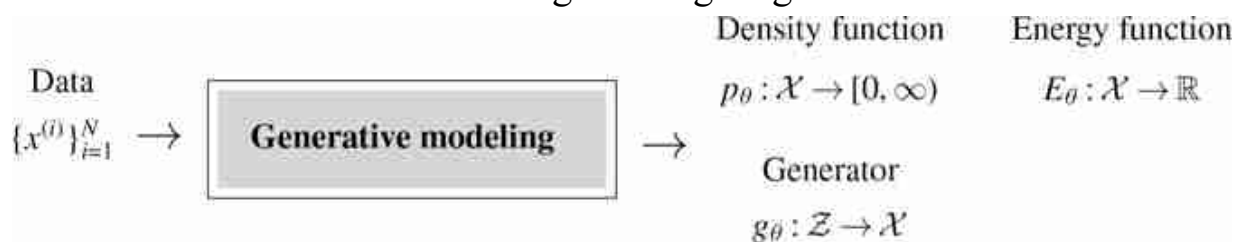
$$= \frac{1}{N} \nabla_{\theta} \sum_{i=1}^N (-E_{\theta}(\mathbf{x}^{(i)}) + E_{\theta}(\hat{\mathbf{x}}^{(i)})) \quad (32.26)$$

The last line should make clear the intuition: we establish a contrast between data samples and current model samples, then update the model to decrease this contrast, bringing the model closer to the data. Once $\mathbf{x}^{(i)}$ and $\hat{\mathbf{x}}^{(i)}$ are identically distributed, this gradient will be zero (in expectation); we have perfectly fit the data and no further updates should be taken.

Under our formalization of a learning problem as an objective, hypothesis space, and optimizer, contrastive divergence is an *optimizer*; it's an optimizer specifically built for max likelihood objectives over energy functions. Contrastive divergence tells us how to approximate the gradient of the likelihood function, which can then be plugged into any gradient descent method.

32.5.2 Comparing Different Kinds of Generative Models

Some generative models give density or energy functions, others give generator functions, and still others give both. We can summarize all these kinds of models with the following learning diagram:



Each of these families of model has its own advantages and disadvantages. Generators have latent variables $\mathbf{z}$ that control the properties of the generated image. Changing these variables can change the generated image in meaningful ways; we will explore this idea in greater detail in the next chapter. In contrast, density and energy models do not have latent variables that directly control generated samples.

While density/energy functions do not have latent variables, when we draw samples from them we use a *sampling algorithm* that does, necessarily, depend on stochastic factors, which can be considered latent variables of the sampler.

Conversely, an advantage of density/energy functions is that they provide scores related to the likelihood of data. These scores (densities or energies) can be used to detect anomalies and unusual events, or can be optimized to synthesize higher quality data.

Some of the generative models we will describe in this chapter and the next are categorized along these dimensions in table 37.1. Note that this is a rough categorization, meant to reflect the most straightforward usage of these models. With additional effort some of the $\mathbf{X}$'s can be converted to $\checkmark$'s; for example, one can do additional inference to sample from a density

model, or one can extract a lower-bound estimate of density from a variational autoencoder (VAE) or diffusion model.

Table 32.1

Three desirable properties in a generative model. No method achieves all three (without caveats). VAEs (see chapter 33) and GANs are good at representation learning (i.e., at finding a low-dimensional latent space). Autoregressive models are great if you want to estimate the likelihood of your data points. Energy-based models can be an especially efficient way to model an unnormalized density.

Method	Latents?	Density/Energy?	Generator?
Energy-based models	✗	✓ (energy only)	✗
Gaussian	✗	✓	✗
Autoregressive models	✗	✓	✓ (slow)
Diffusion models	✓ (high-dimensional)	✗	✓ (slow)
GANs	✓	✗	✓
VAEs	✓	✗	✓

Generative models can also be distinguished according to their objectives, hypothesis spaces, and optimization algorithms. Some classes of model, such as autoregressive models, refer to just a particular kind of hypothesis space, whereas other classes of model, such as VAEs, are much more specific in referring to the conjunction of an objective, a general family of hypothesis spaces, and a particular optimization algorithm.

In the next sections, and in the next chapter, we will dive into the details of exactly how each of these models works.

32.6 Gaussian Density Models

One of the simplest and most useful density models is the Gaussian distribution, which in one dimension (1D) is:

$$p_{\theta}(x) = \frac{1}{Z} e^{-(x-\theta_1)^2/(2\theta_2)} \quad (32.27)$$

For 1D Gaussians $Z = \frac{1}{\sqrt{2\pi\theta_2}}$ We prefer to write it as Z to emphasize its structural role as a normalization constant. Usually we will not be explicitly evaluating normalization constants, and will not require knowing their analytical form.

This density has two parameters θ_1 and θ_2 , which are the mean and variance of the distribution. The normalization constant Z ensures that the function is normalized. This is the typical strategy in defining density models: create a parameterized family of functions such that any function in the family is normalized. Given such a family, we search over the parameters to optimize a generative modeling objective.

For density models, the most common objective is max likelihood:

$$\mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\theta}(x)] \approx \frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x^{(i)}) \quad (32.28)$$

For a 1D Gaussian, this has a simple form:

$$\frac{1}{N} \sum_{i=1}^N \log \frac{1}{Z} e^{-(x^{(i)} - \theta_1)^2/(2\theta_2)} = -\log(Z) + \frac{1}{N} \sum_{i=1}^N (x^{(i)} - \theta_1)^2/(2\theta_2) \quad (32.29)$$

Optimizing with respect to θ_1 and θ_2 could be done via gradient descent or random search, but in this case there is also an analytical solution we can find by setting the gradient to be zero. For θ_1^* we have:

$$\frac{\partial (-\log Z + \frac{1}{N} \sum_{i=1}^N (x^{(i)} - \theta_1^*)^2/(2\theta_2^*))}{\partial \theta_1^*} = 0 \quad (32.30)$$

$$\frac{1}{N} \sum_{i=1}^N 2(x^{(i)} - \theta_1^*)/(2\theta_2^*) = 0 \quad (32.31)$$

$$\sum_{i=1}^N x^{(i)} - \sum_{i=1}^N \theta_1^* = 0 \quad (32.32)$$

$$\theta_1^* = \frac{1}{N} \sum_{i=1}^N x^{(i)} \quad (32.33)$$

For θ_2^* we need to note that Z depends on θ_2^* and in particular notice that $\frac{\partial (-\log Z)}{\partial \theta_2^*} = \frac{1}{2\theta_2^*}$:

$$\frac{\partial(-\log Z + \frac{1}{N} \sum_{i=1}^N (x^{(i)} - \theta_1^*)^2 / (2\theta_2^*))}{\partial \theta_2^*} = 0 \quad (32.34)$$

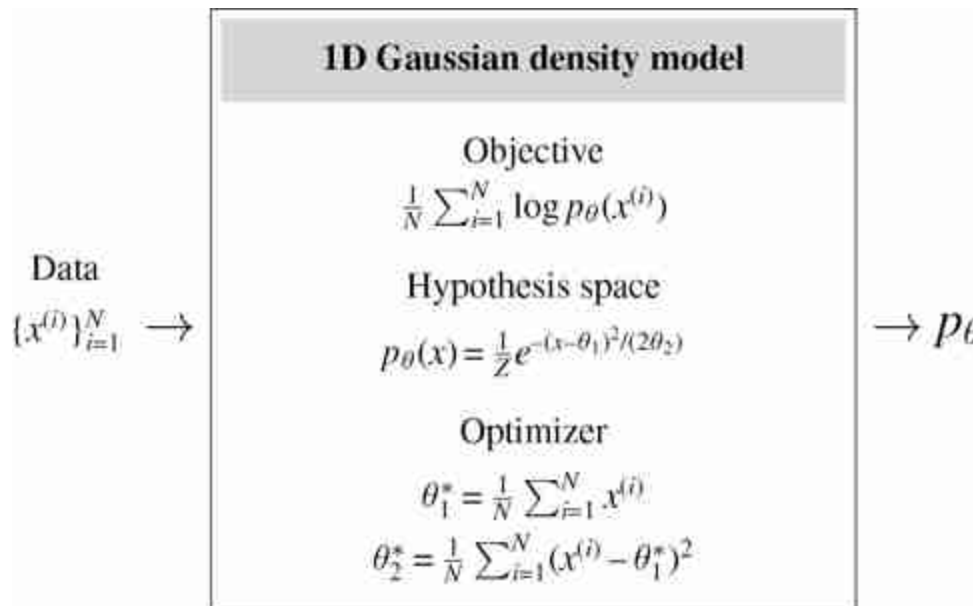
$$\frac{1}{2\theta_2^*} - \frac{1}{N} \sum_{i=1}^N 2(x^{(i)} - \theta_1^*)^2 / (2\theta_2^*)^2 = 0 \quad (32.35)$$

$$2\theta_2^* - \frac{1}{N} \sum_{i=1}^N 2(x^{(i)} - \theta_1^*)^2 = 0 \quad (32.36)$$

$$\theta_2^* = \frac{1}{N} \sum_{i=1}^N (x^{(i)} - \theta_1^*)^2 \quad (32.37)$$

You might recognize the solutions for θ_1^* and θ_2^* as the empirical mean and variance of the data, respectively. This makes sense: we have just shown that to maximize the probability of the data under a Gaussian, we should set the mean and variance of the Gaussian to be the empirical mean and variance of the data.

This fully describes the learning problem, and solution, for a 1D Gaussian density model. We can put it all together in the learning diagram below:



Gaussian density models are just about the simplest density models one can come up with. You may be wondering, do we actually use them for anything in computer vision, or are they just a toy example? The answer is that *yes* we do use them—in fact, we use them all the time. For example, in

least-squares regression, we are simply fitting a Gaussian density to the conditional probability $p(Y | X)$. If we want a more complicated density, we may use a mixture of multiple Gaussian distributions, called a **Gaussian mixture model (GMM)**, which we will encounter in the next chapter. It's useful to get comfortable with Gaussian fits because (1) they are a subcomponent of many more sophisticated models, and (2) they showcase all the key components of density modeling, with a clear objective, a parameterized hypothesis space, and an optimizer that finds the parameters that maximize the objective.

32.7 Autoregressive Density Models

A single Gaussian is a very limited model, and the real utility of Gaussians only shows up when they are part of a larger modeling framework. Next we will consider a recipe for building highly expressive models out of simple ones. There are many such recipes and the one we focus on here is called an **autoregressive model**.

The idea of an autoregressive model is to synthesize an image pixel by pixel. Each new pixel is decided on based on the sequence already generated. You can think of this as a simple sequence prediction problem: given a sequence of observed pixels, predict the color of the next one. We use the same learned function f_θ to make each subsequent prediction.

[Figure 32.10](#) shows this setup. We predict each next pixel from the partial image already completed. The red bordered region is the **context** the prediction is based on. This context could be the entire image synthesized so far or it could be a smaller region, like shown here. The green bordered pixel is the one we are predicting given this context. In this example, we always predict the bottom-center pixel in the context window. After making our prediction, we decide what color pixel to add to the image based on that prediction; we may add the color predicted as most likely, or we might sample from the predicted distribution so that we get some randomness in our completion. Then, to predict the next missing pixel we slide the context over by one and repeat.

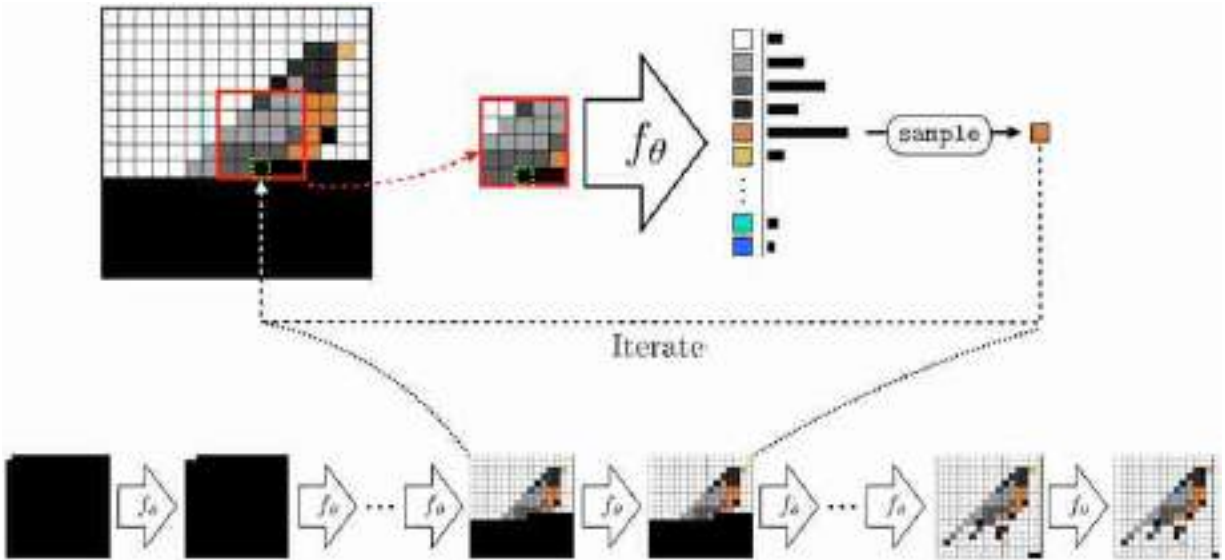


Figure 32.10: An autoregressive model, f_θ , that generates an image pixel by pixel. The black pixels are the remaining pixels to synthesize. Compare with the Efros-Leung model in figure 28.10.

As an exercise, think about how to design a convolutional neural network that can perform the prediction problem in [figure 32.10](#) (see the PixelCNN architecture in [\[484\]](#) for one solution).

You might have noticed that this setup looks very similar to the Efros-Leung texture synthesis algorithm in section 28.4, and indeed that was also an autoregressive model. The Efros-Leung algorithm was a nonparametric method that worked by stealing pixels from the matching regions of an exemplar texture. Now we will see how to do autoregressive modeling with a learned, parametric prediction function f_θ , such as a deep net.

These models can be easily understood by first considering the problem of synthesizing one pixel, then two, and so on. The first observation to make is that it's pretty easy to synthesize a single grayscale pixel. Such a pixel can take on 256 possible values (for a standard 8-bit grayscale image). So it suffices to use a categorical distribution to represent the probability that the pixel takes on each of these possible values. The categorical distribution is fully expressive: any possible probability mass function (pmf) over the 256 values can be represented with the categorical distribution. Fitting this categorical distribution to training data just amounts to counting how often we observe each of the 256 values in the training set pixels, and normalizing by the total number of training set

pixels. So, we know how to model one grayscale pixel. We can sample from this distribution to synthesize a random one-pixel image.

How do we model the distribution over a second grayscale pixel given the first? In fact, we already know how to model this; mathematically, we are just trying to model $p(\mathbf{x}_2 \mid \mathbf{x}_1)$ where $\mathbf{x}_1$ is the first pixel and $\mathbf{x}_2$ is the second. Treating $\mathbf{x}_2$ as a categorical variable (just like $\mathbf{x}_1$), we can simply use a softmax regression, which we saw in section 9.7.3. In that section we were modeling a K -way categorical distribution over K object classes, conditioned on an input image. Now we can use exactly the same tools to model a 256-way distribution over a the second pixel in a sequence conditioned on the first.

In this section we index the first pixel as $\mathbf{x}_1$ rather than $\mathbf{x}_0$.

What about the third pixel, conditioned on the first two? Well, this is again a problem of the same form: a 256-way softmax regression conditioned on some observations. Now you can see the induction: modeling each next pixel in the sequence is a softmax regression problem that models $p(\mathbf{x}_n \mid \mathbf{x}_1, \dots, \mathbf{x}_{n-1})$. We show how the cross-entropy loss can be computed in [figure 32.11](#). Notice that it looks almost identical to figure 9.9 from chapter 9.

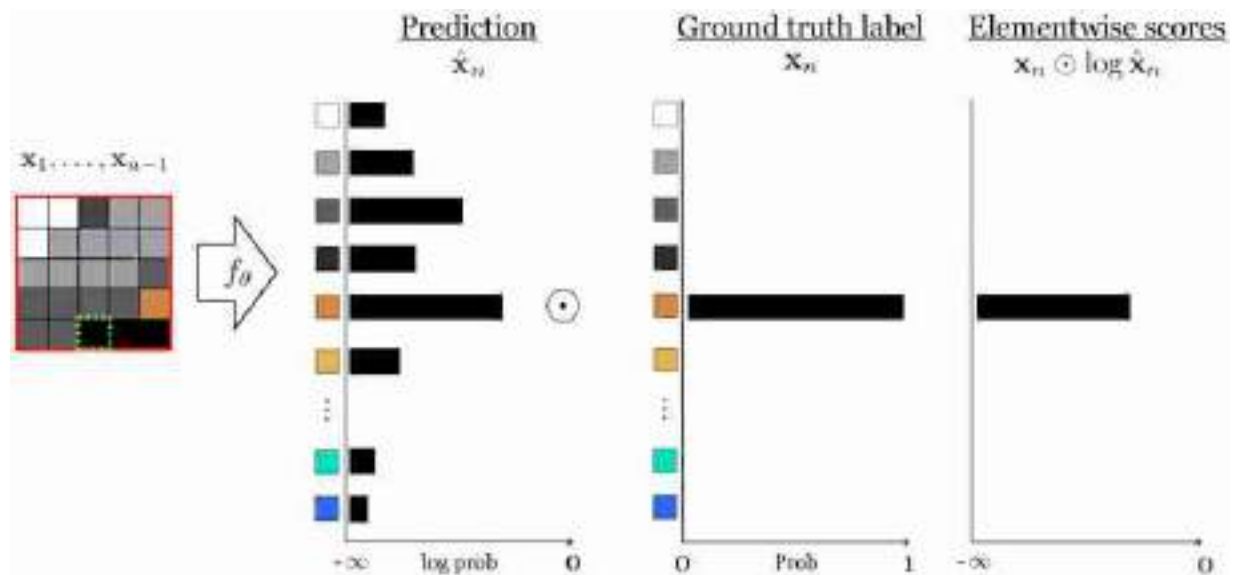


Figure 32.11: Autoregressive prediction as next-pixel-classification.

If we have color images we need to predict three values per-pixel, one for each of the red, green, and blue channels. One way to do this is to predict and sample the values for these three channels in sequence: first predict the red value as a 256-way classification problem, then sample the red value you will use, then predict the green value as a 256-way classification problem, and so on.

You might be wondering, how do we turn an image into a sequence of pixels? Good question! There are innumerable ways, but the simplest is often good enough: just vectorize the two-dimensional (2D) grid of pixels by first listing the first row of pixels, then the second row, and so forth. In general, any fixed ordering of the pixels into a sequence is actually valid, but this simple method is perhaps the most common.

So we can model the probability of each subsequent pixel given the preceding pixels. To generate an image we can sample a value for the first pixel, then sample the second given the first, then the third given the first and second, and so forth. But is this a valid model of $p(\mathbf{X}) = p(x_0, \dots, x_n)$, the probability distribution of the full set of pixels? Does this way of sequential sampling draw a valid sample from $p(\mathbf{X})$? It turns out it does, according to the **chain rule of probability**. This rule allows us to factorize *any* joint distribution into a product of conditionals as follows:

As a notational convenience, we define here that $p(\mathbf{x}_i | \mathbf{x}_1, \dots, \mathbf{x}_{i-1}) = p(\mathbf{x}_1)$ when $i = 1$.

$$p(\mathbf{X}) = p(\mathbf{x}_n | \mathbf{x}_1, \dots, \mathbf{x}_{n-1}) p(\mathbf{x}_{n-1} | \mathbf{x}_1, \dots, \mathbf{x}_{n-2}) \dots p(\mathbf{x}_2 | \mathbf{x}_1) p(\mathbf{x}_1) \quad (32.38)$$

$$p(\mathbf{X}) = \prod_{i=1}^n p(\mathbf{x}_i | \mathbf{x}_1, \dots, \mathbf{x}_{i-1}) \quad (32.39)$$

This factorization demonstrates that sampling from such a model can indeed be done in sequence because all the conditional distributions are independent of each other.

32.7.1 Training an Autoregressive Model

To train an autoregressive a model, you just need to extract supervised pairs of desired input-output behavior, as usual. For an autoregressive model of pixels, that means extracting sequences of pixels $\mathbf{x}_1, \dots, \mathbf{x}_{n-1}$ and corresponding observed next pixel $\mathbf{x}_n$. These can be extracted by traversing training images in raster order. The full training and testing setup looks like this ([figure 32.12](#)):

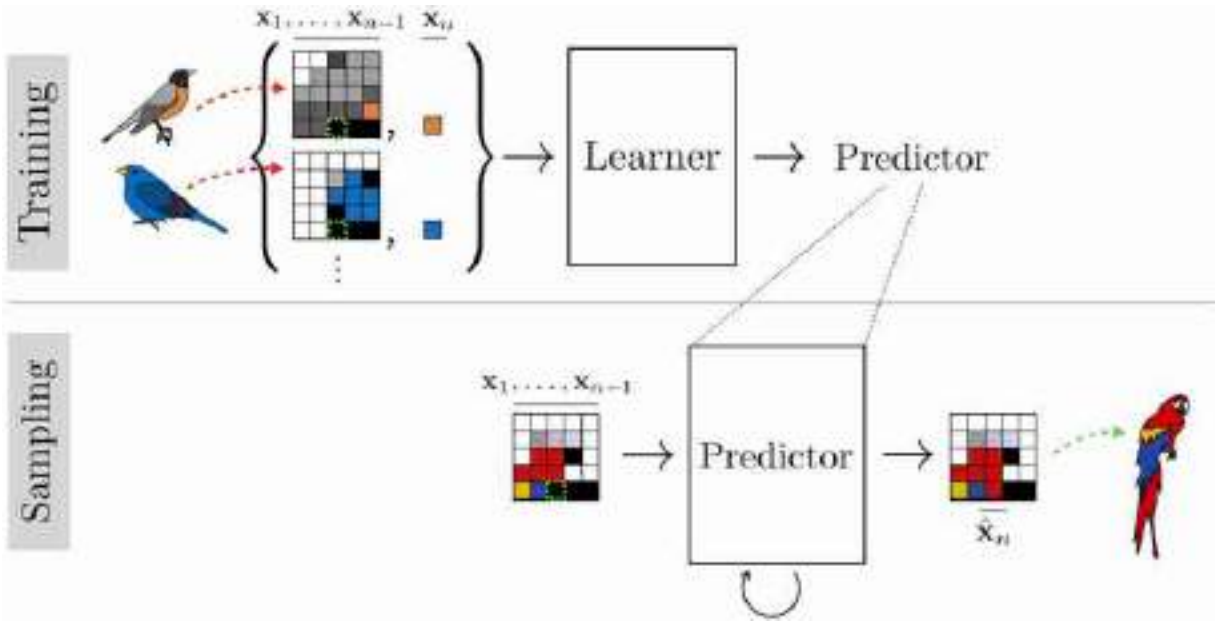


Figure 32.12: Training an autoregressive model, then sampling images from it.

It's worth noting here two different ways of setting up the training batches, one of which is much more efficient than the other. The first way is create a training batch that looks like this (we will call this a *type 1 batch*):

$$\text{input: } x_k^{(i)}, \dots, x_{k+n-1}^{(i)} \quad \text{target output: } x_{k+n}^{(i)} \quad (32.40)$$

$$\text{input: } x_l^{(j)}, \dots, x_{l+n-1}^{(j)} \quad \text{target output: } x_{l+n}^{(j)} \quad (32.41)$$

that is, we sample supervised examples ([sequence, completion] pairs) that each come from a different random starting location (indexed by k and l) in a different random image (indexed by i and j).

The other way to set up the batches is like this (we will call this a *type 2 batch*):

$$\text{input: } x_1^{(i)}, \dots, x_{n-1}^{(i)} \quad \text{target output: } x_n^{(i)} \quad (32.42)$$

$$\text{input: } x_2^{(i)}, \dots, x_n^{(i)} \quad \text{target output: } x_{n+1}^{(i)} \quad (32.43)$$

...

that is, the example sequences overlap. This second way can be much more efficient. The reason is because in order to predict x_n from $x_1, \dots, x_{n-1}$, we typically have to compute representations of $x_1, \dots, x_{n-1}$, and these same representations can be reused for predicting the next item over, that is, x_{n+1} .

from $\mathbf{x}_2, \dots, \mathbf{x}_n$. As a concrete example, this is the case in transformers with causal attention (section 26.10), which have the property that the representation of item $\mathbf{x}_n$ only depends on the items that preceded it in the sequence. What this allows us to do is *share computation between all our overlapping predictions*, and this is why it makes sense to use type 2 training batches. Notice these are the same kind of batches we described in the causal attention section (see equation (26.35)).

32.7.2 Sampling from Autoregressive Models

Autoregressive models give us an explicit density function, equation (32.39). To sample from this density we use **ancestral sampling**, which is the process described previously, of sampling the first pixel from $p(\mathbf{x}_1)$, then, conditioned on this pixel, sampling the second from $p(\mathbf{x}_2 | \mathbf{x}_1)$ and so forth. Since each of these densities is a categorical distribution, sampling is easy: one option is to partition a unit line segment into regions of length equal to the categorical probabilities and see where a uniform random draw along this line falls. Autoregressive models do not have latent variables $\mathbf{z}$, which makes them incompatible with applications that involve extracting or manipulating latent variables.

32.8 Diffusion Models

The strategy of autoregressive models is to break a hard problem into lots of simple pieces. **Diffusion models** are another class of model that uses this same strategy [450]. They can be easy to understand if we start by looking at what autoregressive models do in the *reverse* direction: starting with a complete image, they remove one pixel, then the next, and the next ([figure 32.13](#)):

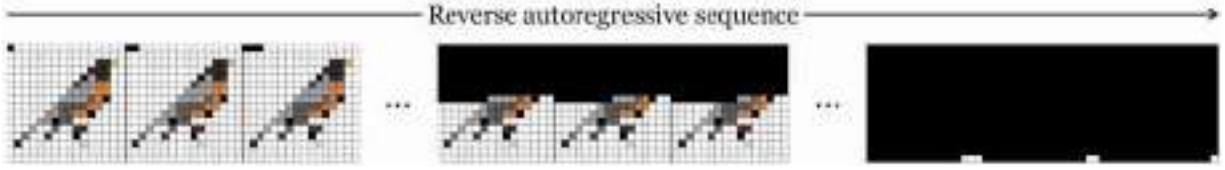


Figure 32.13: An autoregressive sequence in reverse is a corruption process that removes one pixel at a time.

This is a *signal corruption* process. The idea of diffusion models is that this is not the only corruption process we could have used, and maybe not the best. Diffusion models instead use the following corruption process: starting with an uncorrupted image, $\mathbf{x}_0$, they add noise ϵ_0 to this image, resulting in a noisy version of the image, $\mathbf{x}_1$. Then they repeat this process, adding noise ϵ_1 to produce an even noisier signal $\mathbf{x}_2$, and so on. Most commonly, the noise is isotropic Gaussian noise. [Figure 32.14](#) shows what that looks like:



Figure 32.14: Forward diffusion process.

After T steps of this process, the image $\mathbf{x}_T$ looks like pure noise, if T is large enough. In fact, if we use the following noise process, then $\mathbf{x}_T \sim N(\mathbf{0}, \mathbf{I})$ as $T \rightarrow \infty$ (which follows from equation 4 in [\[208\]](#)).

$$\mathbf{x}_t = \sqrt{(1 - \beta_t)}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon_t \quad (32.44)$$

$$\epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (32.45)$$

The β_t values should be ≤ 1 and can be fixed for all t (like in the previous equations) or can be set according to some schedule that changes the amount of noise added over time.

Now, just like autoregressive models, diffusion models train a neural net, f_θ , to *reverse* this process. It can be trained via supervised learning on

examples sequences of different images getting noisier and noisier. For example, as shown below in [figure 32.15](#), the sequence of images of the bird getting noisier and noisier can be flipped in time and thereby provide a sequence of *training examples* of the mapping $\mathbf{x}_t \rightarrow \mathbf{x}_{t-1}$, and these examples can be fit by a predictor f_θ .

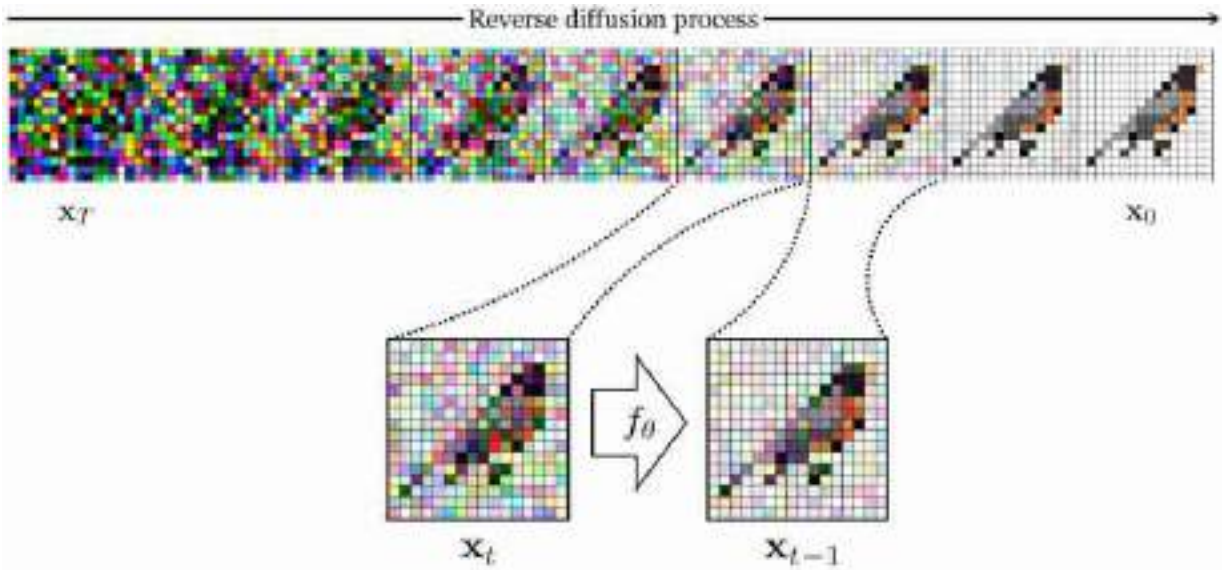


Figure 32.15: Reverse diffusion process. The forward process creates supervision to train the reverse process.

Notice the similarity between this reverse diffusion process and the Heeger-Bergen algorithm from chapter 28 (figure 28.6). Both algorithms start with a noise input and iteratively refine it until it looks like a real image.

We call f_θ a **denoiser**; it learns to remove a little bit of noise from an image. If we apply this denoiser over and over, starting with pure noise, the process should coalesce on a noise-less image that looks like one of our training examples. The steps to train a diffusion model are therefore as follows:

1. Generate training data by corrupting a bunch of images (forward process; noising).
2. Train a neural net to invert each step of corruption (reverse process; denoising).

As an additional trick, it can help to let the denoising function observe time step t as input, so that we have,

$$\hat{\mathbf{x}}_{t-1} = f_{\theta}(\mathbf{x}_t, t) \quad (32.46)$$

This can help the denoiser to make better predictions, because knowing t helps us know if we are looking at a normal scene that has been corrupted by our noise (if t is large this is likely) or a scene where the actual physical structure is full of chaotic, rough textures that happen to look like noise (if t is small this is more likely, because for small t we haven't added much noise to the scene yet). Algorithm 32.2 presents these steps in more formal detail.

In algorithm 32.2 we train the denoiser using a loss L , which could be the L_2 distance. In many diffusion models, f_{θ} is instead trained to output the parameters (mean and variance) as a Gaussian density model fit to the data. This formulation yields useful probabilistic interpretations (in fact, such a diffusion model can be framed as a variational autoencoder, which we will see in chapter 33). However, diffusion models can still work well without these nice interpretations, instead using a wide variety of prediction models for f_{θ} [32].

Algorithm 32.2: Training a diffusion model consists of first producing training pairs of the form {noisy image, less noisy image}. Then do supervised learning on these pairs.

```

1  Input: training data  $\{\hat{\mathbf{x}}^{(i)}\}_{i=1}^N$ 
2  Output: trained model  $f_{\theta}$ 
3  Generate training sequences via diffusion:
4  for  $i = 1, \dots, N$  do
5      for  $t = 1, \dots, T$  do
6           $\epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
7           $\mathbf{x}_t^{(i)} \leftarrow \sqrt{(1 - \beta_t)}\mathbf{x}_{t-1}^{(i)} + \sqrt{\beta_t}\epsilon_t$ 
8
9  Train denoiser  $f_{\theta}$  to reverse these sequences:
10  $\theta^* = \arg \min_{\theta} \sum_{i=1}^N \sum_{t=1}^T \mathcal{L}(f_{\theta}(\mathbf{x}_t^{(i)}, t), \mathbf{x}_{t-1}^{(i)})$ 
11 Return:  $f_{\theta^*}$ 

```

One useful trick for training diffusion models is to reparameterize the learning problem as predicting the noise rather than the signal [208]:

$$f_{\theta}(\mathbf{x}_t, t) = \epsilon_t \quad \triangleleft \text{first predict noise (32.47)}$$

$$\hat{\mathbf{x}}_{t-1} = \mathbf{x}_t + \epsilon_t \quad \triangleleft \text{then remove this noise (32.48)}$$

One way to look at diffusion models is that we want to learn a mapping from pure noise (e.g., $\mathbf{z} \sim N(\mathbf{0}, \mathbf{I})$) to data. It is very hard to figure out how to create structured data out of noise, but it is easy to do the reverse, turning data into noise. So diffusion models turn images into noise in order to *create the training sequences* for a process that turns noise into data.

32.9 Generative Adversarial Networks

Autoregressive models and diffusion models sample simple distributions step by step to build up a complex distribution. Could we instead create a system that directly, in one step, outputs samples from the complex distribution. It turns out we can, and one model that does this is the **generative adversarial network (GAN)**, which was introduced by [168].

Recall that the goal of generative modeling is to make synthetic data that looks like real data. We stated previously that there are many different ways to measure “looks like” and each leads to a different kind of generative model. GANs take a very direct and intuitive approach: synthetic data looks like real data if a classifier cannot distinguish between synthetic images and real images.

GANs consist of two neural networks, the **generator**, $g_{\theta} : Z \rightarrow X$, which synthesizes data from noise, and a **discriminator**, $d_{\phi} : X \rightarrow \Delta^1$, which tries to classify between synthesized data and real data.

Notice that d_{ϕ} has a form similar to an energy function, but, unlike energy functions, d_{ϕ} is not, in general, interpretable as an unnormalized probability density. Nonetheless, we can roughly think of a GAN as a type of energy-based generative model where we train another network g_{θ} to directly sample from the energy function rather than relying on MCMC to sample from the energy function.

The g_{θ} and d_{ϕ} play an adversarial game in which g_{θ} tries to become better and better at generating synthetic images while d_{ϕ} tries to become

better and better at detecting any errors g_θ is making. The learning problem can be written as a minimax game:

$$\arg \min_{\theta} \max_{\phi} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log d_{\phi}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}} [\log(1 - d_{\phi}(g_{\theta}(\mathbf{z})))] \quad (32.49)$$

Schematically, the generator synthesizes images that are then fed as input to the discriminator. The discriminator tries to assign a high score to generated images (classifying them as synthetic) and a low score to real images from some training set (classifying them as real), as shown in [figure 32.16](#).

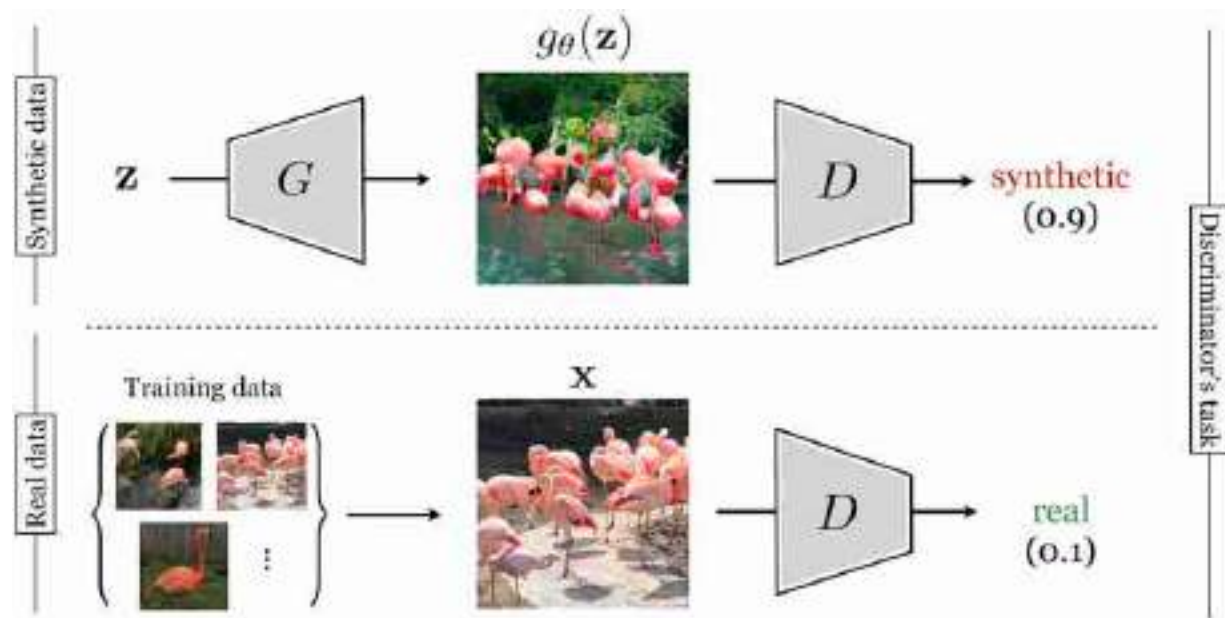


Figure 32.16: Architecture of a GAN being trained to generate images of flamingos. The synthetic image in this example is generated by BigGAN [61]. Notice the artifacts in the synthetic image; after more training, the discriminator could pick up on these and tell the generator to correct them.

Although we call this an adversarial game, the discriminator is in fact helping the generator to perform better and better, by pointing out its current errors (we call it an adversary because it tries to point out errors). You can think of the generator as a student taking a painting class and the discriminator as the teacher. The student is trying to produce new paintings that match the quality and style of the teacher. At first the student paints flat landscapes that lack shading and the illusion of depth; the teacher gives feedback: “This mountain is not well shaded; it looks 2D.” So the student

improves and corrects the error, adding haze and shadows to the mountain. The teacher is pleased but now points out a different error: “The trees all look identical; there is not enough variety.” The teacher and student continue on in this fashion until the student has succeeded at satisfying the teacher. Eventually, in theory, the student—the generator—produces paintings that are just as good as the teacher's paintings.

This objective may be easier to understand if we think of the objectives for g_θ and d_ϕ separately. Given a particular generator g_θ , d_ϕ tries to maximize its ability to discriminate between real and synthetic images (synthetic images are anything output by g_θ). The objective of d_ϕ is logistic regression between a set of real data $\{\mathbf{x}^{(i)}\}_{i=1}^N$ and synthetic data $\{\tilde{\mathbf{x}}^{(i)}\}_{i=1}^N$, where $\tilde{\mathbf{x}}^{(i)} = g_\theta(\mathbf{z}^{(i)})$.

Let the optimal discriminator be labeled d_ϕ^* . Then we have,

$$d_\phi^* = \arg \max_{\phi} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log d_\phi(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}} [\log(1 - d_\phi(g_\theta(\mathbf{z})))] \quad (32.50)$$

Now we turn to g_θ 's perspective. Given d_ϕ^* , g_θ tries to solve the following problem:

$$\arg \min_{\theta} \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}} [\log(1 - d_\phi^*(g_\theta(\mathbf{z})))] \quad (32.51)$$

Now, because the optimal discriminator d_ϕ^* depends on the current behavior of g_θ , as soon as we *change* g_θ , updating it to better fool d_ϕ^* , then d_ϕ^* no longer is the optimal discriminator and we need to again solve problem in equation (32.50). To optimize a GAN, we simply alternate between taking one gradient step on the objective in equation (32.51) and then K gradient steps on the objective in equation (32.50), where the larger the K , the closer we are to approximating the true d_θ^* . In practice, setting $K = 1$ is often sufficient.

32.9.1 GANs are Statistical Image Models

GANs are related to the image and texture models we saw in chapters 27 and 28. For example, in the Heeger-Bergen model (section 28.3; [202]), we synthesize images with the same statistics as a source texture. This can be phrased as an optimization problem in which we optimize image pixels until certain statistics of the images match those same statistics measured on a source (training) image. In the Heeger-Bergen model the objective is to match subband histograms. In the language of GANs, the loss that checks

for such a match is a kind of discriminator; it outputs a score related to the difference between a generated image's statistics and the real image's statistics. However, unlike a GAN, this discriminator is hand-defined in terms of certain statistics of interest rather than learned. Additionally, GANs amortize the optimization via learning. That is, GANs learn a mapping g_θ from latent noise to samples rather than arriving at samples via an optimization process that starts from scratch each time we want to make a new sample.

32.10 Concluding Remarks

Generative models are models that synthesize data. They can be useful for content creation—artistic images, video game assets, and so on—but also are useful for much more. In the next chapters we will see that generative models also learn good *image representations*, and conditional generative models can be viewed as a general framework for answering questions and making predictions. As Richard Feynman famously wrote, “What I cannot create, I do not understand.”¹⁴ By modeling creation, generative models also help us understand the visual world around us.

^{12.} In practice, we may not be able to match all statistics, due to limits of the model's capacity.

^{13.} The divergence need not be a proper distance *metric*, and often is not; it can be nonsymmetric, where $D(p, q) \neq D(q, p)$, and need not satisfy the triangle-inequality. In fact, a divergence is defined by just two properties: nonnegativity and $D(p, q) = 0 \Leftrightarrow p = q$.

^{14.} https://en.wikiquote.org/wiki/Richard_Feynman

33 Generative Modeling Meets Representation Learning

33.1 Introduction

This chapter is about models that unite the ideas of both generative modeling and representation learning. These models learn mappings both to and from data.

The intuition is that generative models map a simple base distribution (i.e., noise) to data, whereas representation learning maps data to simple underlying representations (**embeddings**). These two problems are, essentially, inverses of each other. Many algorithms explicitly treat them as inverse problems, where solving the problem in one direction can inform the solution in the other direction.

In chapter 12, we described neural nets as being a sequence of mappings from raw data to ever more abstracted representations, layer by layer. This perspective puts representation learning in the spotlight: deep learning is just representation learning! Let us now point out an alternative perspective: in backward order, deep nets are mappings from abstracted representations to ever more concrete representations of the data, layer by layer. This backward ordering is the direction in which deep generative networks work. This perspective puts the spotlight on generative modeling: deep learning is just generative modeling! Indeed both modeling directions are valid, and the full picture looks like this ([figure 33.1](#)):

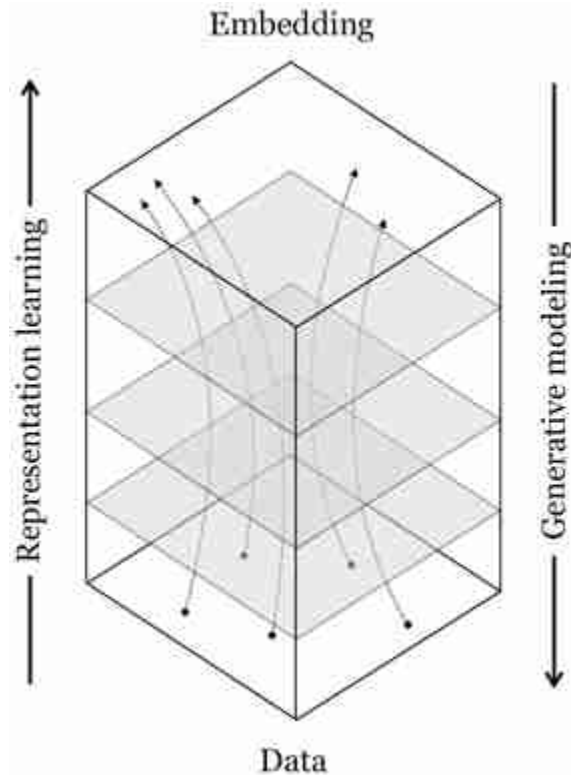


Figure 33.1: The relationship between representation learning and generative modeling. Here we label one side as “Data” and the other as “Embedding”; but what’s the precise difference between these two things? Why is an RGB image data while a 100-dimensional vector of neural activations is an embedding? This is a question for you to think about; there is no correct answer.

Moving backward through a net is also what backpropagation does, but it computes a different function: backpropagation computes the gradient ∇f whereas here we focus on the inverse f^{-1} .

33.2 Latent Variables as Representations

In chapter 32, we introduced generative models with latent variables $\mathbf{z}$. In that context, the role of the latent variable was to specify all unobserved factors that might affect the output of a model. For example, if the model predicts the color of a black and white photo, it is a mapping $g : \mathbf{x}, \mathbf{z} \rightarrow \mathbf{y}$, with $\mathbf{x}$ being the black and white input, $\mathbf{y}$ being the color output and $\mathbf{z}$ being any other information that needs to be known in order to make the mapping completely deterministic; for example, the color of a t-shirt which cannot be inferred solely from the black and white input. In the extreme case of

unconditional generative models, all properties of the generated images are controlled by the latent variables.

What we did not mention in chapter 32, but will focus on now, is that *latent variables are representations of the data*. In the case of an unconditional generative model, the latent variables are a *complete* representation of the data: all information in the data is represented in the latent variables.

Given this connection, this chapter will ask the following question: Are latent variables *good* representations of the data? And can they be combined with other representation learning algorithms?

33.3 Technical Setting

We will consider random variables $\mathbf{z}$ and $\mathbf{x}$ related as follows, with g being deterministic generator (i.e., decoder) and f being a deterministic encoder:

$$\mathbf{x} \sim p_{\text{data}} \quad \mathbf{z} \sim p_{\mathbf{z}} \quad (33.1)$$

$$\hat{\mathbf{z}} = f(\mathbf{x}) \quad \hat{\mathbf{x}} = g(\mathbf{z}) \quad (33.2)$$

where f and g will be trained so that $g \approx f^{-1}$, which means that $\hat{\mathbf{x}} \approx \mathbf{x}$ and $\hat{\mathbf{z}} \approx \mathbf{z}$. In figure 30.1, we sketched how representation learning maps from a data domain to a simple embedding space. We can now put that diagram side by side with the equivalent diagram for generative modeling. Notice again how they are just the same thing in opposite directions ([figure 33.2](#)):

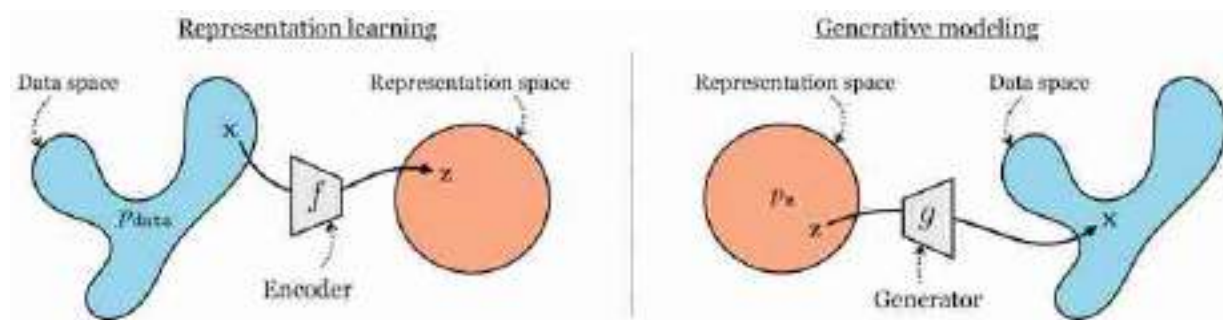


Figure 33.2: Generative modeling performs the opposite mapping from representation learning.

The critical thing in most generative models, which is not necessarily true for representation learning models, is that we assume we know the

distribution p_z , and typically it has a simple form such as a unit Gaussian. Knowing this distribution allows us to sample from it and then generate images via g .

One of the most important quantities we will measure is the data log likelihood function $L(\{\mathbf{x}\}_{i=1}^N, \theta)$, which measures the log likelihood of the data under the model p_θ :

$$L(\{\mathbf{x}^{(i)}\}_{i=1}^N, \theta) = \sum_{i=1}^N \log p_\theta(\mathbf{x}^{(i)}) \quad (33.3)$$

Many methods use a max likelihood objective, optimizing L with respect to θ . To compute the likelihood function, we need to compute $p_\theta(\mathbf{x})$. One way to express this function is as the **marginal likelihood** of $\mathbf{x}$, marginalizing over all unobserved latent variables $\mathbf{z}$:

$$p_\theta(\mathbf{x}) = \int_{\mathbf{z}} p_\theta(\mathbf{x} | \mathbf{z}) p_z(\mathbf{z}) d\mathbf{z} \quad (33.4)$$

The advantage of expressing $p_\theta(\mathbf{x})$ in this way is that, assuming we know p_z , the rest of the modeling problem is reduced to learning the conditional distribution $p_\theta(\mathbf{x} | \mathbf{z})$, which itself can be straightforwardly modeled using g . For example, we could model $p_\theta(\mathbf{x} | \mathbf{z}) = N(\mu = g(\mathbf{z}), \sigma = \mathbf{1})$, that is, just place a unit Gaussian distribution over $\mathbf{x}$ centered on $g(\mathbf{z})$.

The integral in equation (33.4) is expensive so most generative models either sidestep the need to explicitly calculate it, or approximate it. We will examine one such strategy next.

33.4 Variational Autoencoders

In the next sections we will examine the **variational autoencoder (VAE)** [261, 260], which is a model that turns an autoencoder into a generative model that can synthesize data.

33.4.1 The Decoder of an Autoencoder is a Data Generator

In chapter 30 we learned about autoencoders. These are models that learn an embedding that can be decoded to reconstruct the input data. You may already have noticed that the decoder of an autoencoder looks just like a

generator. It is a mapping from a representation of the data, $\mathbf{z}$, back to the data itself, $\mathbf{x}$. Given a $\mathbf{z}$, we can synthesize an image by passing it through the decoder of the autoencoder ([figure 33.3](#)):

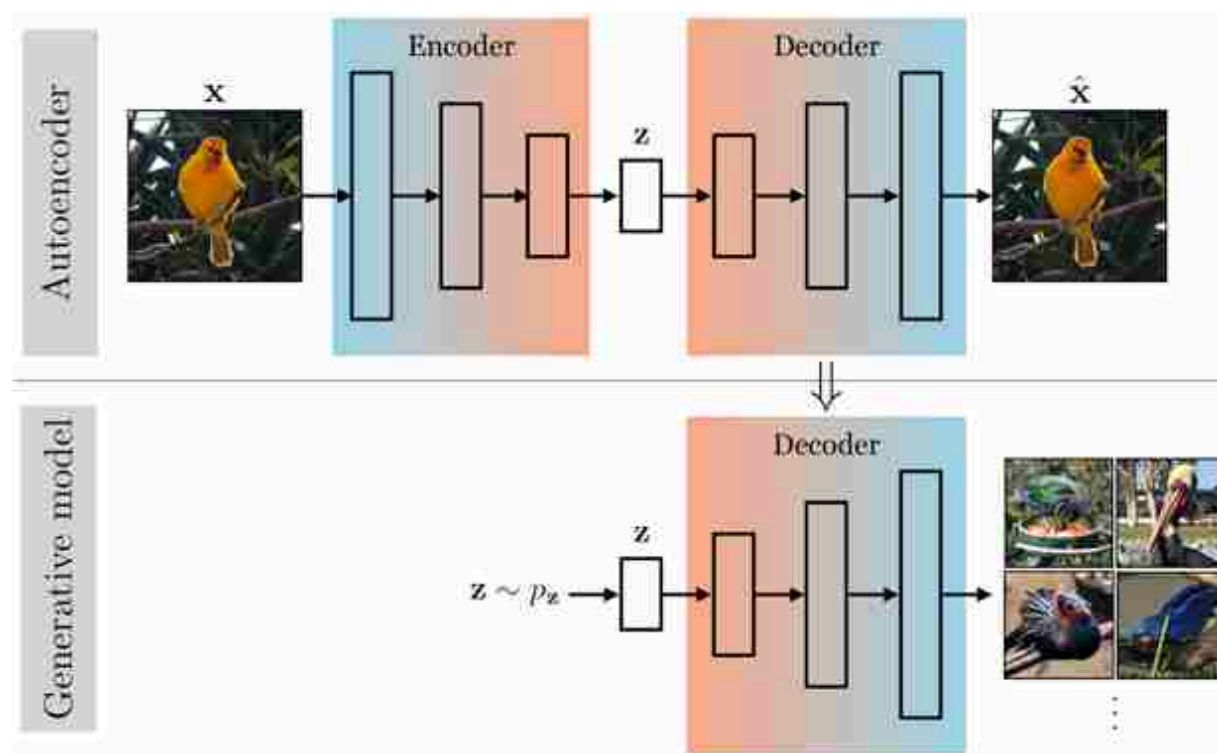


Figure 33.3: The relationship between an autoencoder and a generative model.

But how do we get this $\mathbf{z}$? One goal of generative modeling is to be able to make up random images from scratch. So we need a distribution from which we can sample different $\mathbf{z}$ from scratch, that is, we need $p_{\mathbf{z}}$. An autoencoder doesn't directly give us this. You might ask, what if, after training an autoencoder, you just sample a random $\mathbf{z}$, say from a unit Gaussian, and feed it through the decoder? The problem is that this sample might be very different from what the decoder was trained on, and it therefore might not map to a natural looking image. For example, maybe the encoder has learned to map all images to embeddings far from the origin; then a unit Gaussian $\mathbf{z}$ would be far out of distribution and the decoder's behavior could be arbitrary for this out-of-distribution input.

In general, the embedding space of an autoencoder might be just as complicated to model as the data space we started with, as indicated in

[figure 33.4](#):

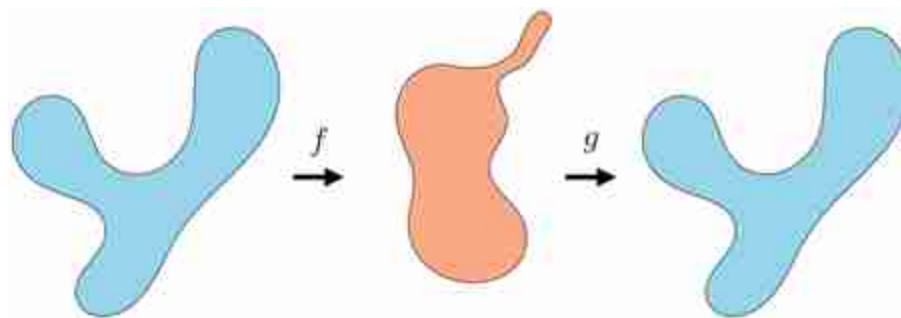


Figure 33.4: The latent space of an autoencoder can be just as complex as the data space.

VAEs are a way to turn an autoencoder into a proper generative model, which can be sampled from and which maximizes data likelihood under a formal probabilistic model. The trick is very simple: just take a vanilla autoencoder and (1) regularize the latent distribution to squish it into a Gaussian (or some other base distribution), (2) add noise to the output of the encoder. In code, it can be as simple as a two line change!

Okay, but seeing *why* this is the right and proper thing to do requires quite a bit of math. We will derive it now, using a different approach than in most texts. We think this approach makes it easier to intuit what is going on. See [\[260\]](#) for the more standard derivation.

33.4.2 The VAE Hypothesis Space

VAEs are max likelihood generative models, which maximize the likelihood function L in equation (33.4). What distinguishes them from other max likelihood models is their particular hypothesis space and optimization algorithm. We will first describe the hypothesis space.

Remember the Gaussian generative model from chapter 32 (i.e., fitting a Gaussian to data)? We stated that this model is too simple for most purposes, but can form the basis for more flexible density models, which work by combining a lot of simple distributions. We gave one example in chapter 32: autoregressive models, which model a complicated distribution as a product over many simple conditional distributions. VAEs follow a slightly different strategy: they model complicated distributions as *sums* of simple distributions.

In particular, VAEs are **mixture models**, and the most common kind of VAE is a **mixture of Gaussians**.

Mixture models are probability models of the form $P(\mathbf{x}) = \sum_i w_i p_i(\mathbf{x})$.

The mixture of Gaussians model is in fact classical model that represents a density as a weighted sum of Gaussian distributions:

$$p_{\theta}(\mathbf{x}) = \sum_{i=1}^k w_i \mathcal{N}(\mathbf{x}; \mu_i, \Sigma_i) \quad \triangleleft \text{mixture of Gaussians} \quad (33.5)$$

where the parameters are $\theta = \{\mu_i, \Sigma_i\}_{i=1}^k$, that is, the mean and variance of all Gaussians in the mixture. Unlike classical Gaussian mixture models, VAEs use an *infinite* mixture of Gaussians, that is, $k \rightarrow \infty$.

But wait, how can we parameterize an infinite mixture? We can't learn an infinite set of means and variances. The trick we will use is to make the mean and variance be *functions* of an underlying continuous variable.

You can think of a function as the *infinite set* of values a variable takes on over some domain.

The function the VAE uses is g_{θ} . For notational convenience, we decompose this function into g_{θ}^{μ} and g_{θ}^{Σ} to separately model the means and variances of the Gaussians in the infinite mixture. Next we need a continuous domain to integrate our mixture over, and as a simple choice, we will use the unit Gaussian distribution. Then our infinite mixture can be described as:

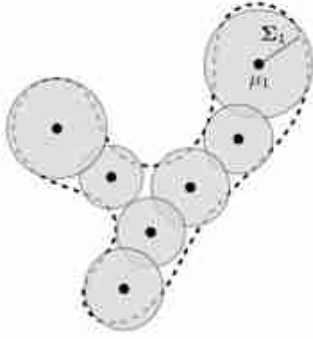
$$p_{\theta}(\mathbf{x}) = \int_{\mathbf{z}} \underbrace{\mathcal{N}(\mathbf{x}; g_{\theta}^{\mu}(\mathbf{z}), g_{\theta}^{\Sigma}(\mathbf{z}))}_{p_{\theta}(\mathbf{x} | \mathbf{z})} \underbrace{\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})}_{p_{\mathbf{z}}(\mathbf{z})} d\mathbf{z} \quad \triangleleft \text{VAE hypothesis space} \quad (33.6)$$

Notice that this equation—an infinite mixture of Gaussians parameterized by a function g_{θ} —has exactly the same form as the marginal likelihood in equation (33.4). What we have done is model an infinite mixture as an integral that marginalizes over a continuous latent variable.

You can think about this as transforming a base distribution $p_{\mathbf{z}}$ to a modeled distribution p_{θ} by applying a deterministic mapping g_{θ} and then putting a blip of Gaussian probability around each point in the range of this mapping. If you sample a few of the Gaussians in this infinite mixture, they might look like this ([figure 33.5](#)):

Finite mixture of Gaussians

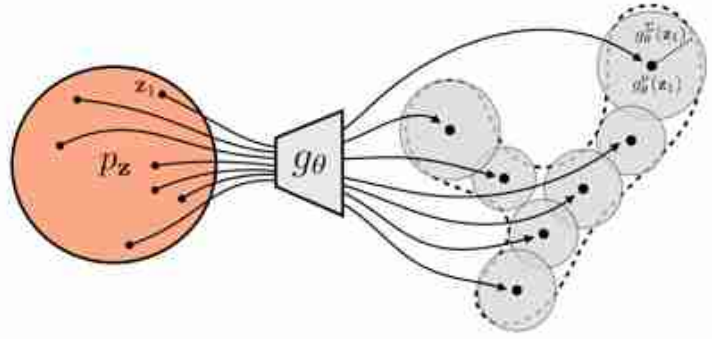
Parameters: $\{w_i, \mu_i, \Sigma_i\}_{i=1}^k$



$$p_{\theta}(\mathbf{x}) = \sum_{i=1}^k w_i \mathcal{N}(\mathbf{x}; \mu_i, \Sigma_i)$$

Infinite mixture of Gaussians (VAE)

Parameters: θ



$$p_{\theta}(\mathbf{x}) = \int_{\mathbf{z}} p_{\mathbf{z}}(\mathbf{z}) \mathcal{N}(\mathbf{x}; g_{\theta}^{\mu}(\mathbf{z}), g_{\theta}^{\Sigma}(\mathbf{z})) d\mathbf{z}$$

Figure 33.5: From a finite mixture of Gaussians to an infinite mixture.

While we have chosen Gaussians for $p_{\theta}(\mathbf{x} | \mathbf{z})$ and $p_{\mathbf{z}}(\mathbf{z})$ in this example, VAEs can also be constructed using other base distributions, even complicated ones. For example, we could make an infinite mixture of the autoregressive distributions we saw in chapter 32. In this sense, mixture models are metamodels, and their components can themselves be any of the density models we have learned about in this book, including other mixture models.

33.4.3 Optimizing VAEs

Wait, a whole section on optimization? Didn't this book say that general-purpose optimizers (like backpropagation) are often sufficient in the deep learning era? Yes. *But only if you can actually compute the objective and its gradient.* The issue here is that the VAE's objective is *intractable*. Its exact computation requires integrating over an infinite set of deep net forward passes. The difficulty of optimizing VAEs lies in the difficulty of approximating this intractable objective. Once we derive our approximation, optimization again will be easy: just apply backpropagation on this approximate loss.

With the objective and hypothesis given previously, we can now fully define the VAE learning problem:

$$\theta^* = \arg \max_{\theta} L(\{\mathbf{x}^{(i)}\}_{i=1}^N, \theta) \quad (33.7)$$

$$= \arg \max_{\theta} \sum_{i=1}^N \log \underbrace{\int_{\mathbf{z}} \overbrace{\mathcal{N}(\mathbf{x}^{(i)}; g_{\theta}^{\mu}(\mathbf{z}), g_{\theta}^{\Sigma}(\mathbf{z}))}^{p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z})} \overbrace{\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})}^{p_{\mathbf{z}}(\mathbf{z})} d\mathbf{z}}_{p_{\theta}(\mathbf{x}^{(i)})} \quad (33.8)$$

33.4.3.1 Trick 1: Approximating the objective via sampling

The integral for $p_{\theta}(\mathbf{x}^{(i)})$ does not necessarily have a closed form since g_{θ} may be an arbitrarily complex function. Therefore, we need to approximate this integral numerically. The first trick we will use is a *Monte Carlo estimate* of this integral:

$$p_{\theta}(\mathbf{x}) = \int_{\mathbf{z}} p_{\theta}(\mathbf{x}|\mathbf{z}) p_{\mathbf{z}}(\mathbf{z}) d\mathbf{z} \quad (33.9)$$

$$= \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [p_{\theta}(\mathbf{x}|\mathbf{z})] \quad (33.10)$$

$$\approx \frac{1}{M} \sum_{j=1}^M p_{\theta}(\mathbf{x}|\mathbf{z}^{(j)}), \quad \mathbf{z}^{(j)} \sim p_{\mathbf{z}} \quad (33.11)$$

We could stop here, as the learning problem is now written in a closed differentiable form:

$$\theta^* = \arg \max_{\theta} \frac{1}{M} \sum_{i=1}^N \log \sum_{j=1}^M \overbrace{\mathcal{N}(\mathbf{x}^{(i)}; g_{\theta}^{\mu}(\mathbf{z}^{(j)}), g_{\theta}^{\Sigma}(\mathbf{z}^{(j)}))}^{p_{\theta}(\mathbf{x}^{(i)}|\mathbf{z}^{(j)})} \quad (33.12)$$

As long as g_{θ} is a differentiable neural net, we can proceed with optimization via backpropagation. In practice, on each iteration of backpropagation, we would collect a batch of random samples $\{\mathbf{x}^{(i)}\}_{i=1}^{B_1} \sim p_{\text{data}}$ from the training set and a batch of random latents $\{\mathbf{z}^{(j)}\}_{j=1}^{B_2} \sim p_{\mathbf{z}}$ from the unit Gaussian distribution (with B_1 and B_2 being batch sizes). Then we would pass each of the $\mathbf{z}$ samples through our net g_{θ} , which yields a Gaussian under which we can evaluate each $\mathbf{x}$ sample. After evaluating and summing up the log probabilities, we would run a backward pass to update the parameters θ .

In [figure 33.6](#) we show what this process looks like at three checkpoints during training. Here we use an isotropic Gaussian model, that is, we parameterize the covariance as $\Sigma = \sigma \mathbf{I}$, where $\sigma = g_{\theta}^{\sigma}(\mathbf{z})$ is a scalar.

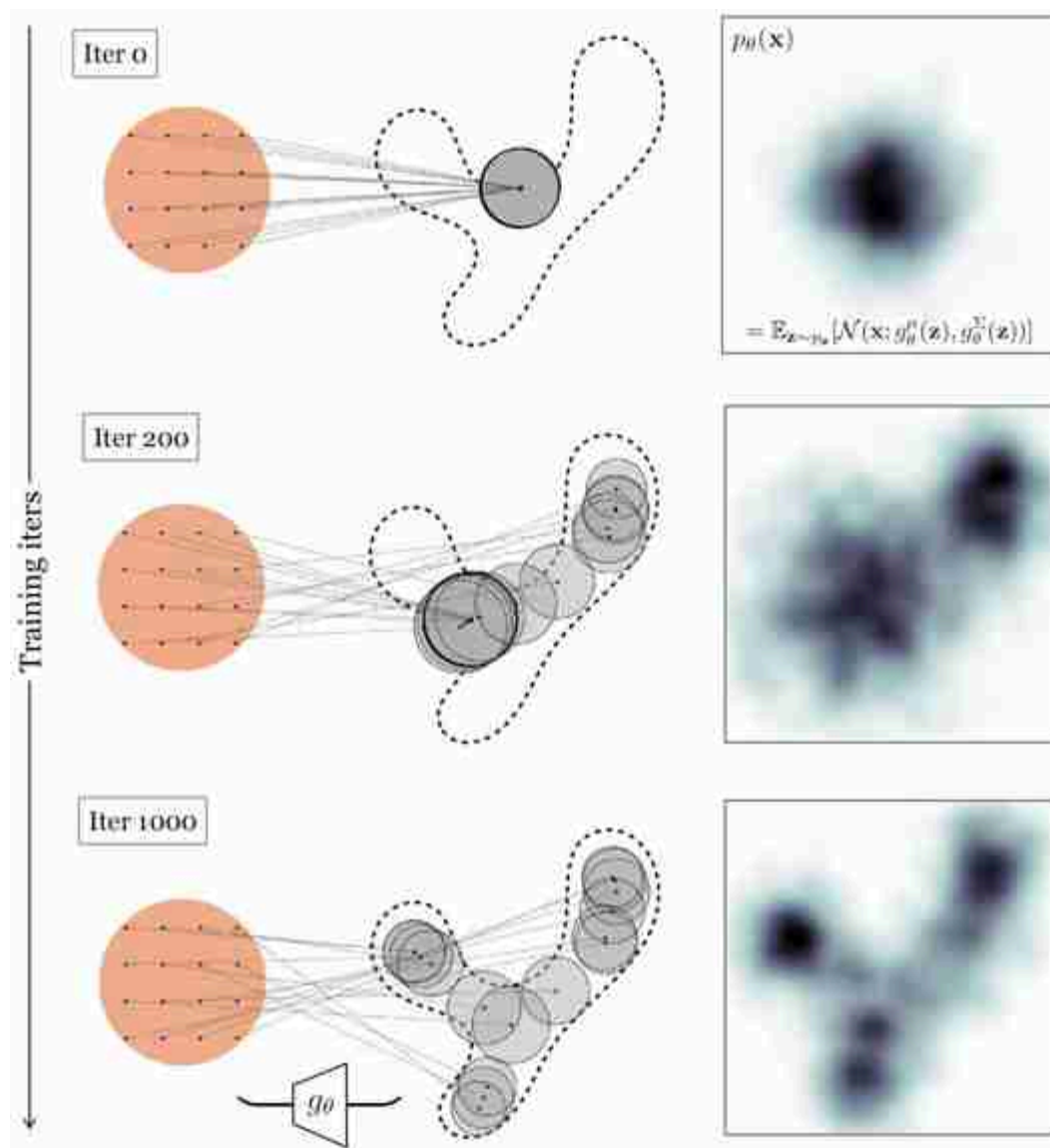


Figure 33.6: Fitting an infinite mixture of Gaussians whose means and variances are parameterized by a generator function g_θ .

33.4.3.2 Trick 2: Efficient approximation via importance sampling

The previous trick works decently for modeling low-dimensional distributions. Unfortunately, this approach does not scale well to high-dimensions. The reason is that in order for our Monte Carlo estimate of the integral to be accurate, we may need many samples from p_z , and the higher the dimensionality of $\mathbf{z}$, the more samples we will typically need.

Can we come up with a more efficient way of approximating the integral in equation (33.9)? Let's start by writing out the sum from equation (33.11)

more explicitly:

$$p_{\theta}(\mathbf{x}) \approx \frac{1}{M}(p_{\theta}(\mathbf{x} | \mathbf{z}^{(1)}) + p_{\theta}(\mathbf{x} | \mathbf{z}^{(2)}) + p_{\theta}(\mathbf{x} | \mathbf{z}^{(3)}) + \dots) \quad (33.13)$$

In general, some of the terms $p_{\theta}(\mathbf{x} | \mathbf{z}^{(j)})$ will be larger than others. In fact, in our example in [figure 33.6](#), *most* of these terms are near zero. This is because, to maximize likelihood, the model spread out the Gaussians so that each places high density on a different part of the data distribution. A datapoint $\mathbf{x}$ will only have substantial probability under the Gaussians whose means are near $\mathbf{x}$.

Consider the example in [figure 33.7](#), where we are trying to estimate the probability of the point $\mathbf{x}$ (blue circle).

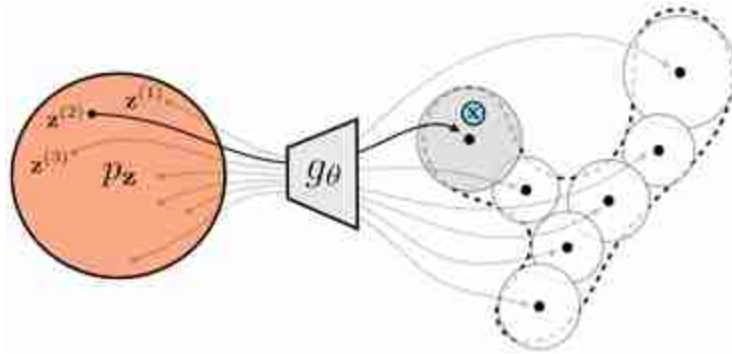


Figure 33.7: To estimate $p_{\theta}(\mathbf{x})$ we only need to consider the Gaussian components (gray circles) that place significant probability on $\mathbf{x}$.

The mixture components are shaded according to the probability they assign to $\mathbf{x}$. Almost all are so far from $\mathbf{x}$ that we have:

$$p_{\theta}(\mathbf{x}) \approx \frac{1}{M}(0 + p_{\theta}(\mathbf{x} | \mathbf{z}^{(2)}) + 0 + \dots) \quad (33.14)$$

If we had *only* sampled $\mathbf{z}^{(2)}$, we would have had almost as good an estimate!

This brings us to the second trick of VAEs: when approximating the likelihood integral for $p_{\theta}(\mathbf{x})$, try to only sample $\mathbf{z}$ values that place high probability on $\mathbf{x}$, that is, those $\mathbf{z}$ values for which $p_{\theta}(\mathbf{x} | \mathbf{z})$ is large. Then, a few samples will suffice to well approximate the entire expectation. This trick is called **importance sampling**. It is a general trick for approximating expectations. Rather than sampling $\mathbf{z}^{(i)} \sim p_{\mathbf{z}}$, we sample from some other

density $\mathbf{z}^{(i)} \sim q_{\mathbf{z}}$, and multiply by a correction factor $\frac{p_{\mathbf{z}}(\mathbf{z})}{q_{\mathbf{z}}(\mathbf{z})}$ to account for the fact that we are sampling from a biased distribution:

$$p_{\theta}(\mathbf{x}) = \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}} [p_{\theta}(\mathbf{x} | \mathbf{z})] = \int_{\mathbf{z}} p_{\mathbf{z}}(\mathbf{z}) p_{\theta}(\mathbf{x} | \mathbf{z}) d\mathbf{z} = \int_{\mathbf{z}} q_{\mathbf{z}}(\mathbf{z}) \frac{p_{\mathbf{z}}(\mathbf{z})}{q_{\mathbf{z}}(\mathbf{z})} p_{\theta}(\mathbf{x} | \mathbf{z}) d\mathbf{z} \quad (33.15)$$

$$= \mathbb{E}_{\mathbf{z} \sim q_{\mathbf{z}}} \left[\frac{p_{\mathbf{z}}(\mathbf{z})}{q_{\mathbf{z}}(\mathbf{z})} p_{\theta}(\mathbf{x} | \mathbf{z}) \right] \quad (33.16)$$

Using the intuition we developed previously, the distribution q we would really like to sample from is the one whose samples maximize $p_{\theta}(\mathbf{x} | \mathbf{z})$. It turns out that the optimal distribution is $q^* = p_{\theta}(Z | \mathbf{x})$.¹⁵ This distribution minimizes the expected error between a Monte Carlo estimate of the expectation and its true value (i.e., minimizes the variance of our Monte Carlo estimator). The intuition is that $p_{\theta}(Z | \mathbf{x})$ is precisely a prediction of which $\mathbf{z}$ values are most likely to have generated the observed $\mathbf{x}$. The optimal importance sampling way to estimate the likelihood of a datapoint will therefore look like this:

$$p_{\theta}(\mathbf{x}) \approx \frac{1}{M} \sum_{j=1}^M \frac{p_{\mathbf{z}}(\mathbf{z}^{(j)})}{p_{\theta}(\mathbf{z}^{(j)} | \mathbf{x})} p_{\theta}(\mathbf{x} | \mathbf{z}^{(j)}) \quad \triangleleft \text{Optimal importance sampling} \quad (33.17)$$

$$\mathbf{z}^{(j)} \sim p_{\theta}(Z | \mathbf{x})$$

Visually, rather than sampling from all over $p_{\mathbf{z}}$ and wasting samples on regions that place nearly zero likelihood on the data, we focus our sampling on just the region that places high likelihood on the data, and we can get then away with far fewer samples to well approximate the data likelihood, as indicated [figure 33.8](#).

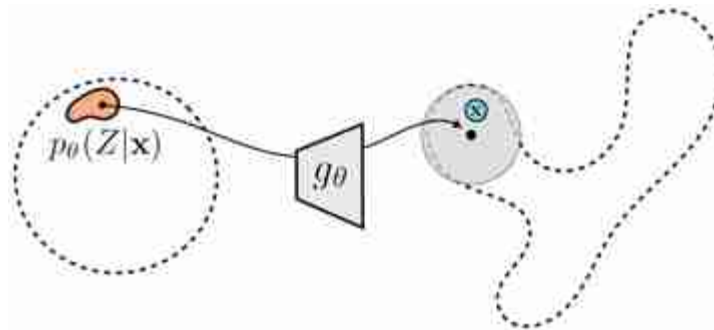


Figure 33.8: Optimal importance sampling estimates $p_{\theta}(\mathbf{x})$ by drawing samples from $p_{\theta}(Z | \mathbf{x})$.

33.4.3.3 Trick 3: Variational inference to approximate the sampling distribution

Now we know what distribution we *should* be sampling $\mathbf{z}$ from: $p_\theta(Z | \mathbf{x})$. The only remaining problem is that this distribution may be complicated and hard to sample from.

Remember, we have defined simple forms for only $p_\theta(X | \mathbf{z})$ and $p_{\mathbf{z}}$ (both are Gaussians), but this does not mean $p_\theta(Z | \mathbf{x})$ has a simple form.

Sampling from arbitrary distributions is a standard topic in statistics, and many algorithms have been proposed, including the Markov chain Monte Carlo methods we encountered in previous chapters. VAEs use a strategy called **variational inference**.

The name *variational* comes from the “calculus of variations,” which studies functionals (functions of functions). Integrals of probability densities are functionals. Variational inference is commonly (but not always) used to approximate densities expressed as the integral of some other densities, hence functionals, hence the name variational.

The idea of variational inference is to approximate an intractable density p by finding the nearest density in a tractable family q_ψ , parameterized by ψ . In VAEs, we approximate our ideal importance density $p_\theta(Z | \mathbf{x})$ with a q_ψ in a family we can efficiently sample from; the most common choice is to again use a Gaussian, conditioned on $\mathbf{x}$: $q_\psi = \mathcal{N}(f_\psi^\mu(\mathbf{x}), f_\psi^\Sigma(\mathbf{x}))$. The f_ψ is a function that maps from $\mathbf{x}$ to parameters of a distribution over $\mathbf{z}$; in other words, f_ψ is a probabilistic encoder! This function is shown next in [figure 33.9](#):

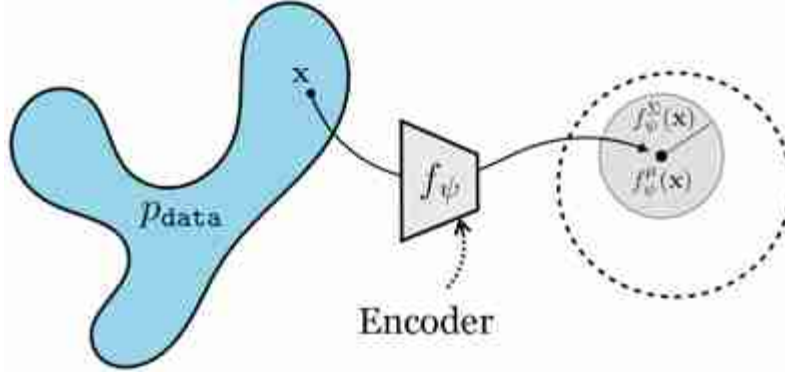


Figure 33.9: A VAE's encoder, f_ψ , models $p(Z | \mathbf{x})$ as a Gaussian parameterized by $f_\psi(\mathbf{x})$.

It will turn out that f indeed plays the role of the encoder in the VAE, while g plays the role of the decoder.

We want q_ψ to be the best approximation of $p_\theta(Z | \mathbf{x})$, so our goal will be to choose the q_ψ that minimizes the Kullback-Leibler (KL) divergence between these two distributions. We could have chosen other divergences or notions of approximation, but we will see that using the KL divergence yields some nice properties. We will call the objective for q_ψ as J_q , which we will define as the negative KL-divergence, so our goal is to maximize this quantity. Using the definition of KL-divergence, we can expand this objective as follows:

$$J_q(\mathbf{x}, \psi) = -\text{KL}(q_\psi(Z | \mathbf{x}) \| p_\theta(Z | \mathbf{x})) \quad (33.18)$$

$$= -\mathbb{E}_{\mathbf{z} \sim q_\psi(Z | \mathbf{x})} [\log q_\psi(\mathbf{z} | \mathbf{x}) - \log p_\theta(\mathbf{z} | \mathbf{x})] \quad (33.19)$$

$$= -\mathbb{E}_{\mathbf{z} \sim q_\psi(Z | \mathbf{x})} [\log q_\psi(\mathbf{z} | \mathbf{x}) - \log p_\theta(\mathbf{x} | \mathbf{z}) - \log p_\theta(\mathbf{z}) + \log p_\theta(\mathbf{x})] \quad (33.20)$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\psi(Z | \mathbf{x})} [-\log q_\psi(\mathbf{z} | \mathbf{x}) + \log p_\theta(\mathbf{x} | \mathbf{z}) + \log p_\theta(\mathbf{z})] - \log p_\theta(\mathbf{x}) \quad (33.21)$$

The last line follows from the fact that $\log p_\theta(\mathbf{x})$ is a constant with respect to the distribution we are taking expectation over.

The learning problem for q_ψ is to maximize J_q , over all images in our dataset, $\{\mathbf{x}^{(i)}\}_{i=1}^N$, with respect to parameters ψ . Notice that the term $\log p_\theta(\mathbf{x})$ is constant with respect to these parameters, so we can drop that term, yielding:

$$\psi^* = \arg \max_{\psi} \frac{1}{N} \sum_{i=1}^N J_q(\mathbf{x}^{(i)}, \psi) \quad (33.22)$$

$$= \arg \max_{\psi} \frac{1}{N} \sum_{i=1}^N \underbrace{\mathbb{E}_{\mathbf{z} \sim q_{\psi}(\mathbf{z} | \mathbf{x}^{(i)})} [-\log q_{\psi}(\mathbf{z} | \mathbf{x}^{(i)}) + \log p_{\theta}(\mathbf{x}^{(i)} | \mathbf{z}) + \log p_{\mathbf{z}}(\mathbf{z})]}_J \quad (33.23)$$

Here we have defined a new cost function, J (the term inside the sum in equation (33.23)), whose maximizer with respect to ψ is the same as the maximizer for J_q .

Now, let us now recall our learning problem for p_{θ} , which is to maximize data log likelihood. Using importance sampling to estimate data likelihood (equation [33.16]), and using q_{ψ} as our sampling distribution, we have that the objective for p_{θ} is to maximize the following objective J_p with respect to θ :

$$J_p(\mathbf{x}, \theta) = \log \mathbb{E}_{\mathbf{z} \sim q_{\psi}(\mathbf{z} | \mathbf{x})} \left[\frac{p_{\mathbf{z}}(\mathbf{z})}{q_{\psi}(\mathbf{z} | \mathbf{x})} p_{\theta}(\mathbf{x} | \mathbf{z}) \right] \quad (33.24)$$

$$\theta^* = \arg \max_{\theta} \frac{1}{N} \sum_{i=1}^N J_p(\mathbf{x}^{(i)}, \theta) \quad (33.25)$$

We now have a differentiable objective for both ψ and θ ; the objective for ψ is expressed as an expectation and can be optimized by taking a Monte Carlo sample from that expectation. We *could* also try using Monte Carlo to approximate the objective for θ , but this would yield a biased estimator of θ , since equation (33.24) has the log outside the expectation.

The expression $\log \frac{1}{N} \sum_i x_i$ is not an unbiased estimator of $\log \mathbb{E}[x]$, hence a Monte Carlo estimate is not the best choice for equation (33.24).

That might be okay (as the number of samples N goes to infinity, the bias goes to zero), but we can do better. To get around this issue, VAEs adopt the following strategy: rather than maximizing J_p with respect to p_{θ} , they maximize a lower-bound to J_p , which is expressed purely as an expectation and yields unbiased Monte Carlo estimates. The particular lower-bound used is in fact J : the same objective we used for optimizing ψ in equation (33.23)!

The fact that J is a lower-bound on J_p follows from Jensen's inequality:

$$J_p = \log p_\theta(\mathbf{x}) \quad (33.26)$$

$$= \log \mathbb{E}_{\mathbf{z} \sim q_\psi(Z | \mathbf{x})} \left[\frac{p_{\mathbf{z}}(\mathbf{z})}{q_\psi(\mathbf{z} | \mathbf{x})} p_\theta(\mathbf{x} | \mathbf{z}) \right] \quad (33.27)$$

$$\geq \mathbb{E}_{\mathbf{z} \sim q_\psi(Z | \mathbf{x})} \left[\log \left(\frac{p_{\mathbf{z}}(\mathbf{z})}{q_\psi(\mathbf{z} | \mathbf{x})} p_\theta(\mathbf{x} | \mathbf{z}) \right) \right] \quad \triangleleft \text{ Jensen's inequality} \quad (33.28)$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\psi(Z | \mathbf{x})} \left[-\log q_\psi(\mathbf{z} | \mathbf{x}) + \log p_\theta(\mathbf{x} | \mathbf{z}) + \log p_{\mathbf{z}}(\mathbf{z}) \right] \quad (33.29)$$

$$= J \quad \triangleleft \text{ VAE objective} \quad (33.30)$$

$$\Rightarrow J \leq J_p \quad (33.31)$$

This way our learning problem for both θ and ψ share the same objective (which also saves computation) and can be stated simply as:

$$\theta^*, \psi^* = \arg \max_{\theta, \psi} \frac{1}{N} \sum_{i=1}^N J(\mathbf{x}^{(i)}, \theta, \psi) \quad (33.32)$$

The VAE objective, J , is often called the **Evidence Lower Bound** or **ELBO**, because it is a lower-bound on the data log-likelihood (i.e. J_p , which equals $\log p_\theta(\mathbf{x})$).

$\log p_\theta(\mathbf{x})$ is sometimes referred to as the **evidence** for $\mathbf{x}$.

Using the definition of KL-divergence, we can also rewrite J as follows:

$$J = \mathbb{E}_{\mathbf{z} \sim q_\psi(Z | \mathbf{x})} \left[\log p_\theta(\mathbf{x} | \mathbf{z}) \right] - \text{KL}(q_\psi(Z | \mathbf{x}) \| p_{\mathbf{z}}) \quad \triangleleft \text{ VAE objective} \quad (33.33)$$

In this form, the VAE objective is presented as the sum of two terms. The first term measures the data log likelihood when the latent variables are sampled from q_ψ and the second term measures the gap between q_ψ and the distribution of latent variables, $p_{\mathbf{z}}$, which we actually should have been sampling from to obtain the correct estimate of $p_\theta(\mathbf{x})$.

33.4.4 Connection to Autoencoders

You may have noticed that in the previous sections we made use of both an encoder f_ψ (which parameterizes $q_\psi(Z | \mathbf{x})$) and a decoder g_θ (which parameterizes $p_\theta(X | \mathbf{z})$); it looks like we are using the two pieces of an autoencoder but what's the exact connection?

We will derive the connection for a simple VAE on one-dimensional (1D) data with 1D latent space, that is, $x \in \mathbb{R}$, $z \in \mathbb{R}$. First let us define

shorthand notation for the means and variances of the Gaussians parameterized by the encoder and decoder:

$$\mu_z = f_\psi^\mu(x), \quad \sigma_z^2 = f_\psi^\Sigma(x) \quad (33.34)$$

$$\mu_x = g_\theta^\mu(z), \quad \sigma_x^2 = g_\theta^\Sigma(z) \quad (33.35)$$

Then the distributions involved in the VAE are as follows:

$$p_z = \mathcal{N}(0, 1) \quad (33.36)$$

$$q_\psi(Z|x) = \mathcal{N}(\mu_z, \sigma_z^2) \quad (33.37)$$

$$p_\theta(X|z) = \mathcal{N}(\mu_x, \sigma_x^2) \quad (33.38)$$

As shown in equation (33.33), the VAE learning problem seeks to maximize the following objective:

$$\frac{1}{N} \sum_{i=1}^N \left(\underbrace{\mathbb{E}_{z \sim q_\psi(Z|x^{(i)})} [\log p_\theta(x^{(i)}|z)]}_{\text{likelihood term}} - \underbrace{\text{KL}(q_\psi(Z|x^{(i)}) \| p_z)}_{\text{KL term}} \right) \quad (33.39)$$

On each step of optimization, we compute this objective over a batch of datapoints, and then apply backpropagation to update the parameters to increase the objective.

For each datapoint x , the KL term can be computed in closed form since it is between two normal distributions (see the appendix of [261] for a derivation):

$$\text{KL}(q_\psi(Z|x) \| p_z) = \text{KL}(\mathcal{N}(\mu_z, \sigma_z^2) \| \mathcal{N}(0, 1)) \quad (33.40)$$

$$= \frac{1}{2} (\mu_z^2 + \sigma_z^2 - \log(\sigma_z^2) - 1) \quad (33.41)$$

The other term, the likelihood term, will be approximated by sampling. For each datapoint x , we will take just a single sample from this expectation, as this is often sufficient in practice. To do so, first we encode x to parameterize $q_\psi(Z|x)$. Then we sample a z from $q_\psi(Z|x)$. Finally we decode this z to parameterize $p_\theta(X|z)$, and we measure the probability this distribution places on our observed input x , as shown below:

$$\mu_z, \sigma_z^2 = f_\psi(x) \quad \Leftarrow \text{Encode } x \text{ (33.42)}$$

$$z \sim \mathcal{N}(\mu_z, \sigma_z^2) \quad \Leftarrow \text{Sample } z \text{ (33.43)}$$

$$\mu_x, \sigma_x^2 = g_\theta(z) \quad \Leftarrow \text{Decode } z \text{ (33.44)}$$

$$\log p_\theta(x|z) = \log \mathcal{N}(x; \mu_x, \sigma_x^2) \quad (33.45)$$

$$= \log \frac{1}{\sigma_x \sqrt{2\pi}} - \frac{\overbrace{(x - \mu_x)^2}^{\text{reconstruction error}}}{2\sigma_x^2} \quad \Leftarrow \text{Measure likelihood} \quad (33.46)$$

In other words, we encode, then decode, then measure the **reconstruction error** between our original input and the output of the decoder; this looks just like an autoencoder, as shown in [figure 33.10](#)!

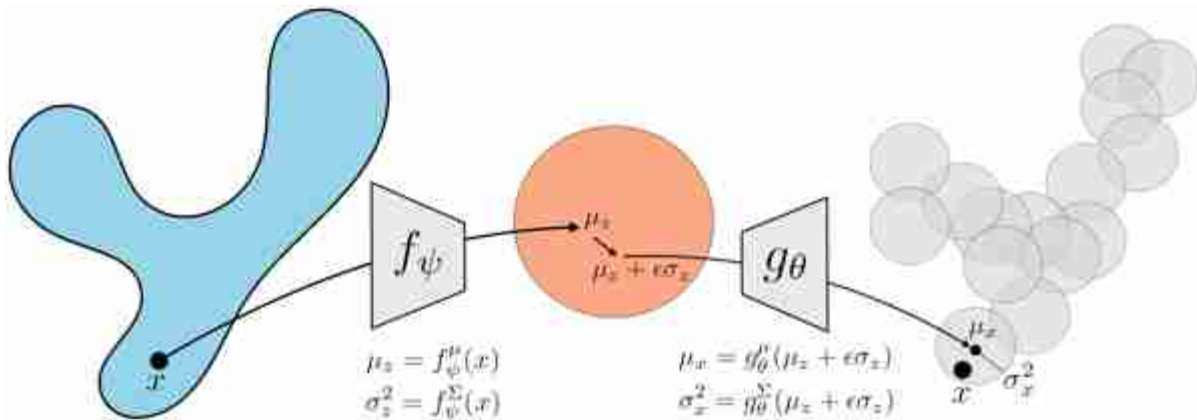


Figure 33.10: To evaluate the likelihood a VAE places on a datapoint x , we encode x into z -space and then decode back and compute the reconstruction error. This corresponds to one importance sample for approximating the likelihood function.

The only differences from an autoencoder are that 1) we sample a stochastic z from the output of the encoder, 2) the reconstruction error is scaled and offset by the predicted variance of the Gaussian likelihood model, and 3) we add to this term the KL loss defined previously.

Difference #1 is worth remarking on. To train the VAE, we need to backpropagate through the sampling step in equation (33.43). How can we backpropagate through the sampling operation? The way to do this turns out to be quite simple: we reparameterize sampling from $\mathcal{N}(\mu_z, \sigma_z^2)$ as follows:

$$\epsilon \sim \mathcal{N}(0, 1) \quad (33.47)$$

$$z = \mu_z + \epsilon \sigma_z \quad (33.48)$$

This step is known as the **reparameterization trick**, as it reparameterizes a stochastic function (sampling from a Gaussian parameterized by a neural net) to be a deterministic transformation of a fixed noise source (the unit Gaussian). To optimize the parameters for the encoder, we only need to backpropagate through μ_z and σ_z , which are deterministic functions of x , and therefore we have sidestepped the need to handle backpropagation through a stochastic function.

Putting all the terms together, the objective we are maximizing can now be written as:

$$\frac{1}{N} \sum_{i=1}^N \left(\log \frac{1}{\sigma_{x^{(i)}} \sqrt{2\pi}} - \overbrace{\frac{(x^{(i)} - \mu_{x^{(i)}})^2}{2\sigma_{x^{(i)}}^2}}^{\text{reconstruction error}} - \underbrace{\frac{1}{2}(\mu_{z^{(i)}}^2 + \sigma_{z^{(i)}}^2 - \log(\sigma_{z^{(i)}}^2) - 1)}_{\text{KL term}} \right) \quad (33.49)$$

$x^{(i)}$ is the i -th datapoint in our training set and $z^{(i)}$ is its encoding.

The KL term encourages the encodings $z^{(i)}$ to be near a unit Gaussian distribution. To get an intuition for the effect of this term, consider the case where we fix $\sigma_{z^{(i)}}$ to be 1; this is still a valid model for $q_\psi(Z|x)$, just with lower capacity because it has fewer free parameters. In this simple case, the KL term reduces to $\frac{1}{2}(\mu_{z^{(i)}}^2 - 1) \propto \mu_{z^{(i)}}^2$. The effect of this term is therefore to encourage the encodings $\mu_{z^{(i)}}$ to be *as close to zero as possible*. In other words, the KL term squashes the distribution of latents ($q_\psi(Z)$) to be near the origin. Minimizing the reconstruction error, on the other hand, requires that the latents do not collapse to the origin; this term wants them to be as spread out as possible so as to preserve information about the inputs $x^{(i)}$ that they encode. The tension between these two terms is what causes the VAE to work. While a standard autoencoder may produce an arbitrary latent distribution, with gaps and tendrils of density (as we saw in [figure 33.4](#)), a VAE produces a tightly packed latent space which can be densely sampled from.

These effects can be seen in [figure 33.11](#), where we show three checkpoints of optimizing a VAE. As in the infinite mixture of Gaussians example shown previously, we again assume an isotropic Gaussian model for the decoder, and here also assume that model for the encoder.

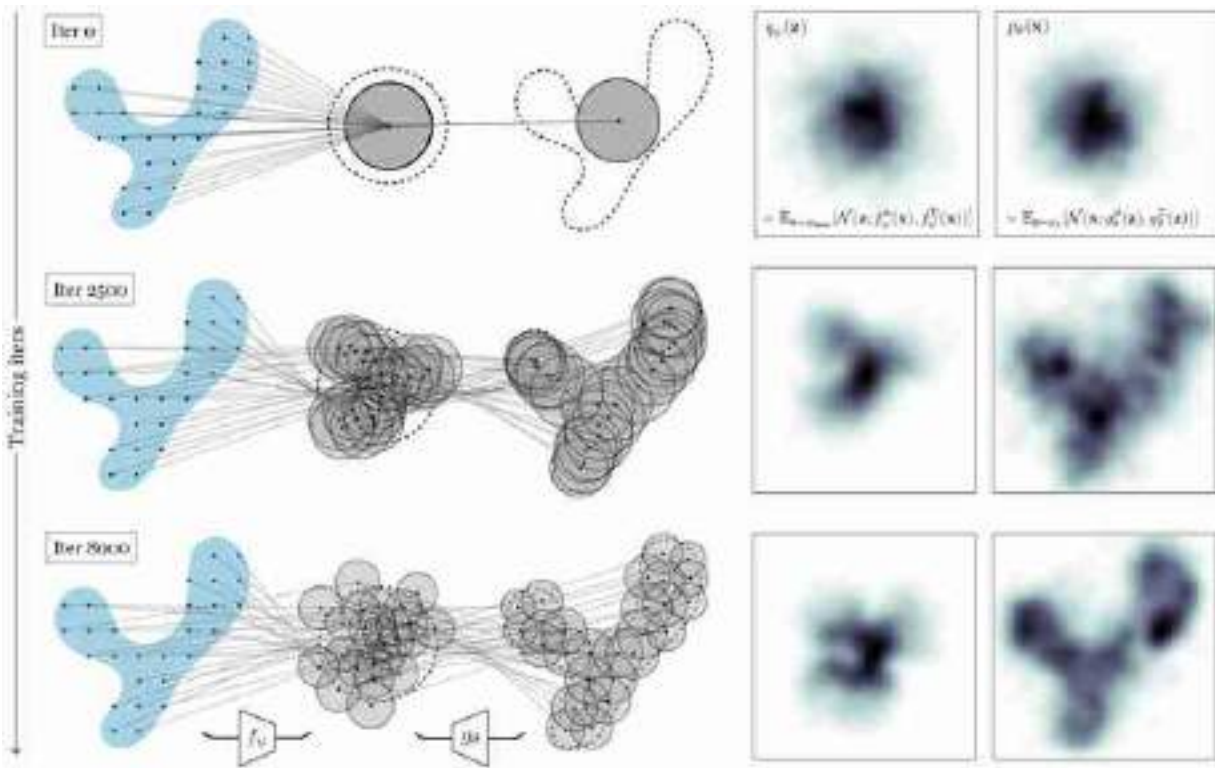


Figure 33.11: Training a VAE to model the blue distribution. The Gaussian components spread out to tile both the embedding space and the data space.

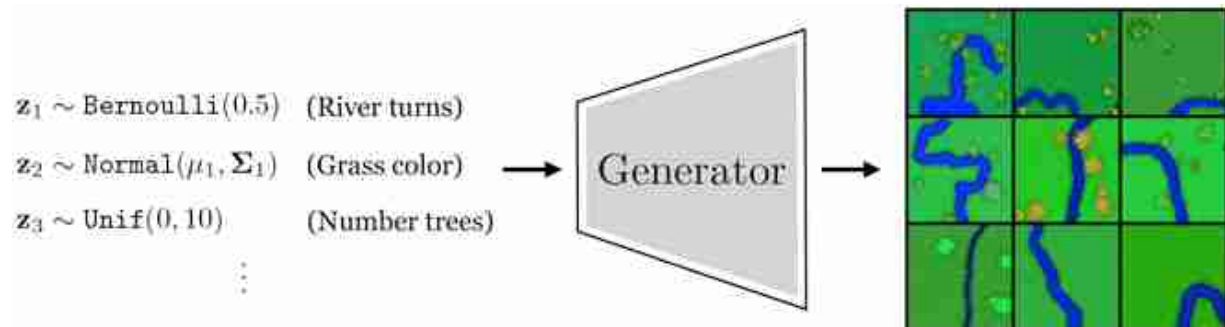
33.5 Do VAEs Learn Good Representations?

One perspective on VAEs is that they are a way to train a generative model p_θ . From this perspective, the encoder is just scaffolding for learning a decoder. However, the encoder can also be useful as an end in itself, and we might instead think of the decoder as scaffolding for training an encoder. This was the perspective presented by autoencoders, after all, and the VAE encoder comes with the same useful properties: it maps data to a low-dimensional latent code that preserves information about the input. In fact, from a representation learning perspective, VAEs even go beyond autoencoders. Not only do VAEs learn a compressed embedding, the embeddings may also have other desirable properties depending on the prior p_z . For example, if $p_z = N(0, 1)$, as is common, then the loss encourages that the dimensions of the embedding are independent, a property called **disentanglement**.

Disentanglement means that we can vary one dimension of the embedding at time, and just one independent factor of variation in the generated images will change. For example, one dimension might control the direction of light in a scene and another dimension might control the intensity of light.

33.5.1 Example: Learning a VAE for Rivers

Suppose we have a dataset of aerial views of rivers. We wish to fit this data with a VAE so that (1) we can generate new rivers, and (2) we can identify the underlying latent variables that explain river appearance. In this example we will use data for which we know the true data generating process, which is simply a Python script that procedurally synthesizes cartoon images of rivers given input noise (this is a more elaborate version of the script we saw in algorithm 32.1 of chapter 32). The script takes in random values that control the attributes of the scene (the grass color, the heading of the river, the number of trees, etc.) and generates an image with these attributes, as shown in [figure 33.12](#).



[Figure 33.12](#): A toy generative model hand-coded in Python.

Training a VAE on this data ([figure 33.13](#)) learns to recreate the generative process with a *neural net* (rather than a Python script) and maps zero-mean unit-variance *Gaussian noise* to images (rather than taking as input the noise types the script uses).

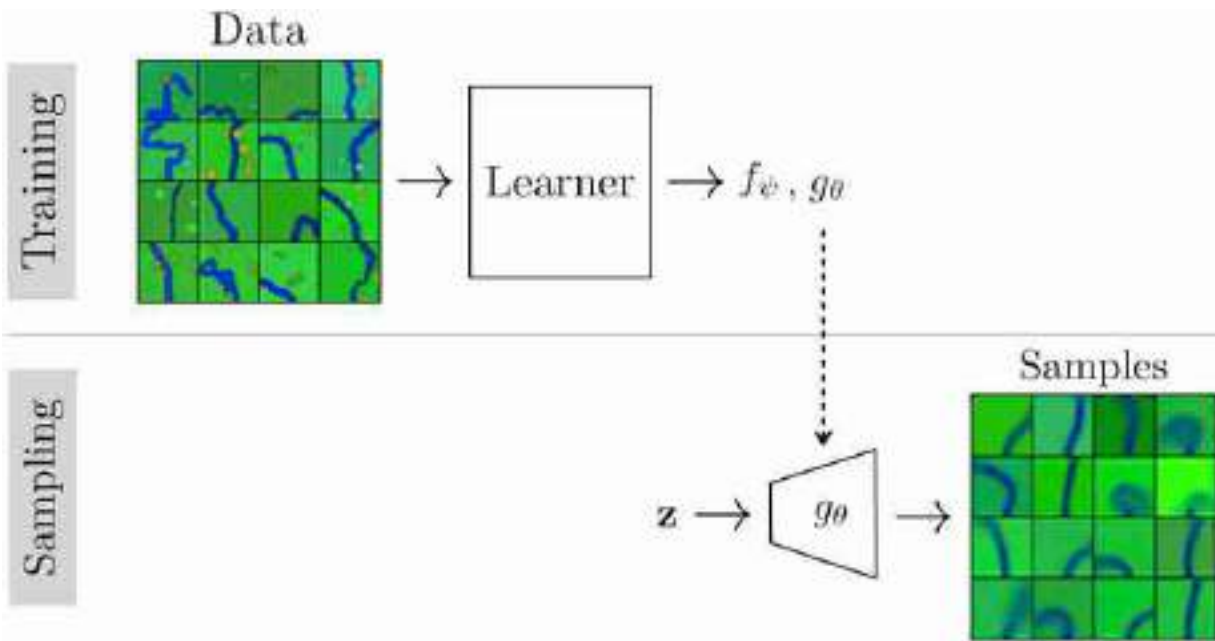


Figure 33.13: Fitting a VAE to the rivers dataset

Did the VAE uncover the *true* latent variables that generated the data, that is, did it recover latent dimensions corresponding to the attribute values that were the inputs to the Python script? We can examine this by generating a set of images that walk along two latent dimensions of the VAE's z -space, shown in [figure 33.14](#).

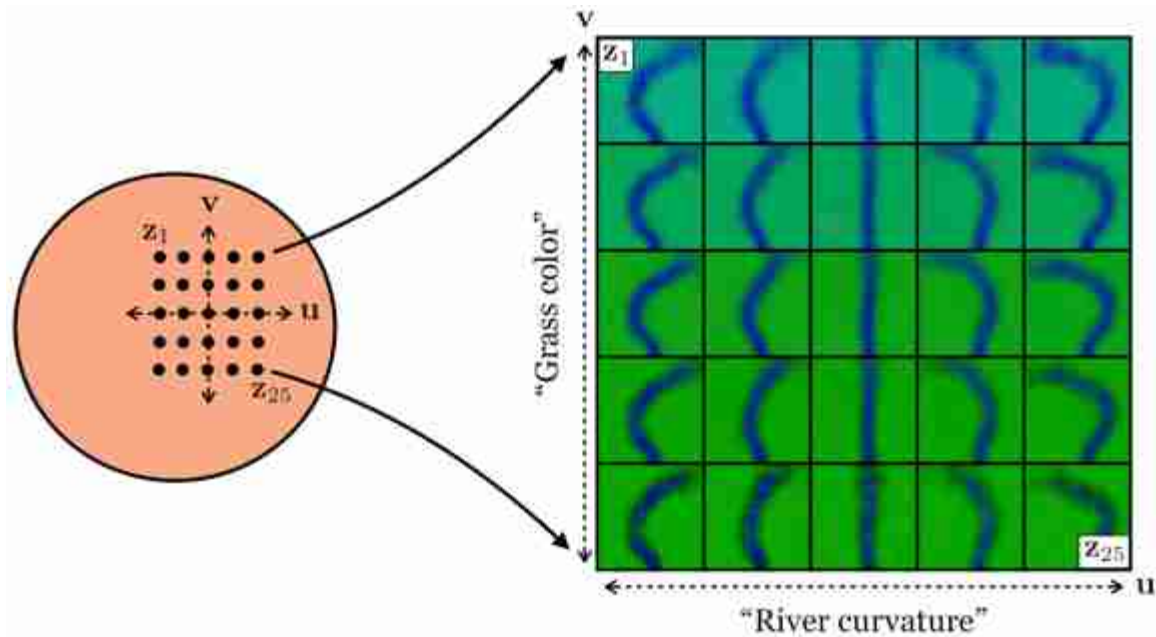


Figure 33.14: Walking along two axes of latent space generates images that show variation in two distinct attributes, demonstrating that this model is, to a degree, disentangled.

One of the latent dimensions seems to control the grass color, and another controls the river curvature! These two latent dimensions are disentangled in the sense that varying the latent dimension that controls color has little effect on curvature and varying the latent dimension that controls curvature has little effect on color. Indeed, grass color was one of the attributes of the true data generating process (the Python script), and the VAE recovered it. However, interestingly there was no single input to the script that controls the overall river curvature, instead the curves are generated by a vector of Bernoulli variables that rotate the heading left and right as the river extends (using the same algorithm as in algorithm 32.1 of chapter 32). The VAE has discovered a latent dimension that somehow summarizes a more global mode of behavior (i.e., bend left or bend right) than is explicit in the Python script. It is important to realize that VAEs, and most representation learning methods, do not necessarily recover the true causal mechanisms that generated the data but rather might find other mechanisms that can equivalently explain the data.

A formal name for this issue is the **nonidentifiability** of the true parameters that generated a dataset.

To summarize this section, we have seen that a VAE can be considered two things:

- An efficient way to optimize an infinite mixture of Gaussians generative model.
- A way to learn a low-dimensional, disentangled representation that can reconstruct the data.

33.6 Generative Adversarial Networks Are Representation Learners Too

In chapter 32 we covered generative adversarial networks (GANs), which, like VAEs, train a mapping from latent variables to synthesized data, $g : \mathbf{z} \rightarrow \mathbf{y}$. Do GANs also learn meaningful and disentangled latents?

To see, let us repeat the experiment of examining latent dimensions of a generative model, but this time with GANs. Here we will use a powerful GAN, called BigGAN [61], that has been trained on the ImageNet dataset [419]. Here are images generated by walking along two latent dimensions of this GAN ([figure 33.15](#)):

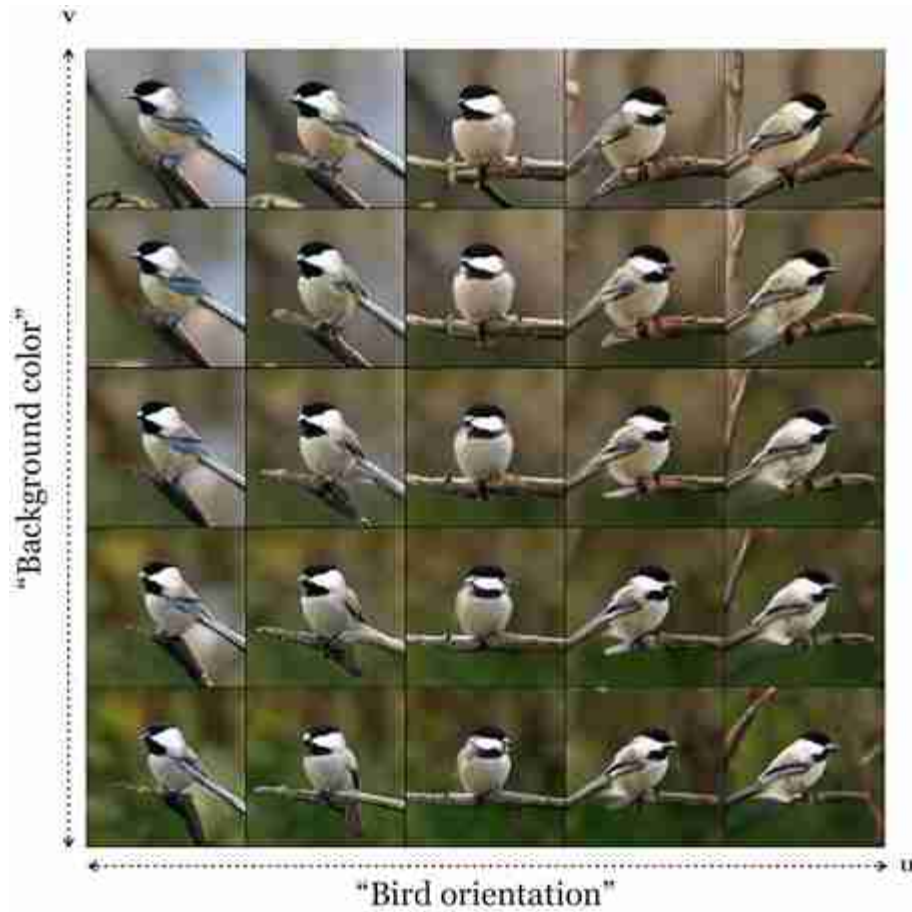


Figure 33.15: Walking in in two orthogonal directions in BigGAN [61] latent space.

Just like with the VAE trained on cartoon rivers, the GAN has also discovered disentangled latent variables; in this case they seem to control background color and the bird's orientation.

This makes sense: structurally, the GAN generator is very similar to the VAE decoder. In both cases, they map a low-dimensional random variable $\mathbf{z}$ to data, and typically $p_{\mathbf{z}} = N(0, 1)$. That means that the dimensions of $\mathbf{z}$ are a priori independent (disentangled). In both models the goal is roughly the same: create synthetic data that has all the properties of real data. It should therefore come as no surprise that both models learn latent representations with similar properties. Indeed, these are just two examples of a large class of models that map low-dimensional latents from a simple (high entropy) distribution to high-dimensional data from a more structured (low entropy) distribution, and we might expect all models in this family to lead to similarly useful representations of the data.

33.7 Concluding Remarks

In this chapter we have seen that representation learning and generative modeling are intimately connected; they can be viewed as inverses of each other. This view also reveals an important property of the latent variables in generative models. These variables are like noise in that they are random variables with simple prior distributions, but they are not like our common sense understanding of noise as an unimportant nuisance. In fact, the latent variables can act as a powerful *representation* of the data. You may prefer to think of them as the underlying control knobs that generate the data. A user can spin these knobs randomly to get a random image, or they can tune the knobs navigate the natural image manifold in a systematic way, and arrive at the image they want.

[15.](#) See chapter 9, section 1 of [\[368\]](#) for a proof that $q^*(\mathbf{z}) \propto |p_\theta(\mathbf{x} | \mathbf{z})|p_{\mathbf{z}}(\mathbf{z})$, from which it then follows that $q^*(\mathbf{z}) \propto p_\theta(\mathbf{x} | \mathbf{z})p_{\mathbf{z}}(\mathbf{z}) = p_\theta(\mathbf{x}, \mathbf{z}) \propto p_\theta(\mathbf{z} | \mathbf{x})$, yielding our result.

34 Conditional Generative Models

34.1 Introduction

In the preceding chapters, we learned about two uses of generative models: (1) as a way to synthesize realistic but novel data, and (2) as a way to learn representations of the data. Now we will introduce a third use, which is arguably the most widespread use of generative models: *as a way to solve prediction problems*.

34.2 A Motivating Example: Image Colorization

To motivate this use case, consider the following problem. We wish to colorize a black and white photo, that is, we wish to predict the color of each pixel in a black and white photo.

Now, we already have seen some tools we could apply to this problem. First we will try to solve it with least-squares regression. Second, with softmax classification. We will see that both approaches fall short of a good solution. This will motivate the necessity of conditional generative models, which turn out to solve the colorization problem quite nicely, and can produce nearly photorealistic results.

34.2.1 The Failure of Point Prediction: Multimodal Distributions

We could formulate the problem as least-squares regression: train a function to output a vector of real-valued numbers, representing the red, green, and blue values of every pixel in the image, then penalize the squared distance between these values and the ground truth values.

This kind of regression fits a function $f: X \rightarrow Y$. For every input $\mathbf{x} \in X$, the output is a single **point prediction** $\hat{\mathbf{y}} \in Y$, that is, we output just a *single* prediction of what the value of $\mathbf{y}$ should be.

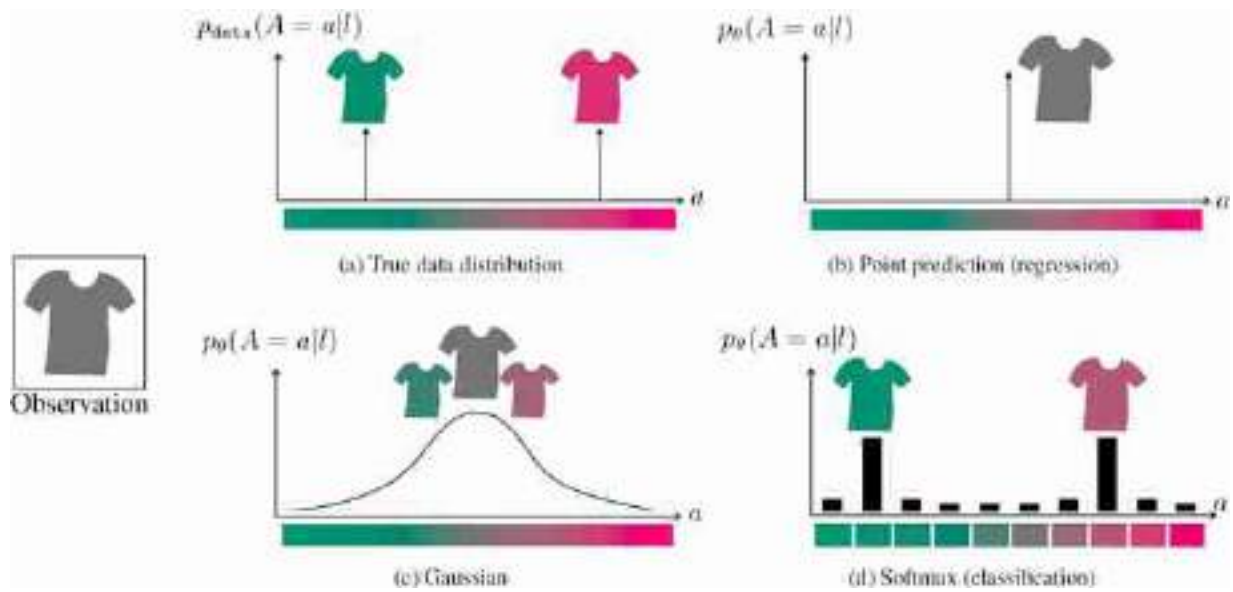


Figure 34.1: Different kinds of predictive distributions.

What if there are multiple equally valid answers? Taking a color image and making it grayscale is a many-to-one mapping: for any given grayscale value, there are many different color values that could have projected to that value of gray. This means that the colorization problem is fundamentally ambiguous, and there may be multiple valid color solutions for any grayscale input. A point estimate is bound to be a poor inference in this case, since a point estimate can only predict one of the multiple possible solutions.

[Figure 34.1](#) shows several kinds of predictions one could make given an observation of a grayscale t-shirt. The question is straightforward: What is the color of the shirt? To unify different kinds of prediction models, we will represent them all as outputting a distribution over the possible colors of the shirt. Let A be a random variable that represents the a value of the shirt in *lab* color space. Our input is just the l value of the shirt. Our predictions will be represented as $p_\theta(A = a|l)$. The particular shirt we are looking at comes in two colors, either teal or pink. The true data distribution, p_{data} , is therefore two delta functions, one on teal and the other on pink.

Least-squares regression results in a model that predicts the *mean* of the data distribution. Therefore, the point prediction output by a least-squares regressor will be that the shirt is gray. The probability density associated with this prediction is shown in [figure 34.1\(b\)](#).

A point prediction does not necessarily come with probabilistic semantics, but here we will interpret a prediction of $\hat{a} = f_{\theta}(I)$ as implying a predictive distribution over A that has the form $p_{\theta}(A = a \mid I) = \delta(a - \hat{a})$, where δ is the Dirac delta function.

This prediction is entirely wrong! It splits the difference between the two true possibilities and comes out with something that has *zero* probability under the data distribution. As discussed in chapter 29, we could have done regression with a different loss function (using the L_1 loss for example, rather than L_2), and we would have come up with a different solution. But we will never get the correct solution. Because the true distribution has two equally probable modes, and a single point prediction can only ever represent one mode.

We can do better by predicting a *distribution* rather than a point estimate. An example is shown in [figure 34.1\(c\)](#), where we predict a Gaussian distribution. In fact, the least-squares regression model can be described as outputting the mean of a max likelihood Gaussian fit to $p_{\theta}(A \mid I)$. Predicting the distribution then just requires also predicting the variance of the Gaussian. This gives us a better sense of the possible colors the shirt could be, but it is still a unimodal distribution, while the data is bimodal.

Naturally, then, we may want to predict a more expressive distribution; a mixture of two Gaussians could, for example, capture the bimodal nature of the data. An easy way to predict a distribution is to predict the parameters of some parametric family of distributions. This is precisely what we did with the Gaussian fit: the parameters of a 1D Gaussian are its mean and variance, so if $\hat{\mathbf{y}} \in \mathbb{R}^2$ then $\hat{\mathbf{y}}$ suffices to parameterize a 1D Gaussian. A mixture of N Gaussians just requires outputting $3N$ numbers: the mean, variance, and weight of each Gaussian in the mixture. The loss function could then just be the data likelihood under the mixture of Gaussians parameterized by $\hat{\mathbf{y}} \in \mathbb{R}^{3N}$.

One of the most common choices for an expressive, multimodal family of distributions is the categorical distribution, Cat . This distribution applies only to discrete random variables, so to use it, the first thing we have to do is quantize our possible output values. [Figure 34.1\(d\)](#) shows an example, where we have quantized the a -value into ten bins. Then the categorical distribution is simply a ten-dimensional vector of nonnegative numbers that sum to 1 (i.e., a probability mass function). The nice thing about this

parameterization is that *all* probability mass functions over k classes are members of the family $\text{Cat}(k)$. In other words, this is the most expressive distribution possible over a discrete random variable! That's great because it means we can use it to represent predictions with any number of modes (well, up to k , the resolution of our quantized data).

In fact, we have already seen one super important use of the categorical distribution: *softmax regression* (section 9.7.3). Now you might have a better understanding of why classification is such a ubiquitous modeling tool: it models a maximally expressive predictive distribution over a quantized decision space.

Remember that *softmax regression* is a way to solve a *classification* problem. It is called regression since we predict a set of continuous numbers (a probability distribution), then take the *argmax* of this set to perform the classification.

34.2.2 Classification to the Rescue?

Classification solves some of the deficiencies of point prediction. However, it comes with several new problems of its own:

- It incurs quantization error.
- It may require a number of classes exponential in the data dimensionality.

The first is not such a big issue. You can see its effect in [figure 34.1\(d\)](#). The true colors are teal and bright pink but the quantization lowers the color resolution and we cannot know if the prediction is that the shirt is dull pink or bright pink as both fall in the same bin. This is why the pink shirt here appears slightly duller than in the data distribution (we colored each bin with the floor of its range). But generally this isn't such a big deal. We can always increase the resolution by increasing k . Most image formats only represent 256 possible values for a , so if we set $k = 256$ then we incur no quantization error.

Why 256 values? Because each channel in standard image formats is stored as an array of `uint8s`.

The second issue is much more severe. If we are predicting the a -value of a single pixel, then the classification approach says there are k possible values it could take on. This approach can be extended to predicting ab -

values: we come up with a list of discrete color classes, like red, orange, turquoise, and so forth. But to tile the two-dimensional space of ab -values will require k^2 color classes, if we wish to retain a resolution of k for both a and b . Nonetheless, we can still do this using a reasonable number of classes, and it might look like as shown in [figure 34.2](#).

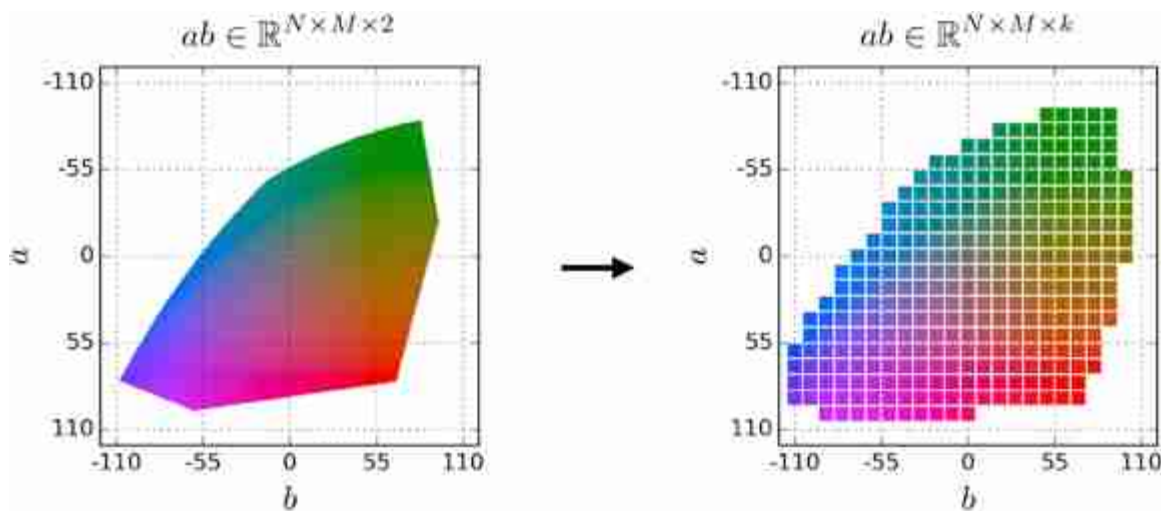


Figure 34.2: The funny shape of the *lab* color gamut is because not every ab value maps to a valid pixel color. When working with predictions over *lab* color space, we may map ab values that fall outside the gamut (valid range) to the nearest in-gamut value.

The real problem comes about when predicting more than one pixel. To quantize the ab -values of N pixels requires k^{2N} classes, again assuming we want a resolution of k for the a and b value of each pixel. For a 256×256 resolution image, the number of classes required is astronomical (for $k = 10$, the number is a one followed by over 100,000 zeros).

Quantizing on a grid doesn't scale to high-dimensions, but more intelligent quantization methods can work. These more intelligent methods are called **vector quantization** or **clustering**, and we cover them in chapter 30.

34.2.3 The Failure of Independent Predictions: Joint Structure

To circumvent this curse of dimensionality, we can turn to factorization: rather than treating the whole configuration of pixels as a class, make *independent* predictions for each pixel in the image. From a probabilistic

inference perspective, the corresponds to the following factorization of the joint distribution:

$$p_{\theta}(\mathbf{ab}|\mathbf{l}) = \prod_{n=1}^N \prod_{m=1}^M p_{\theta}(ab[n, m, :] | \mathbf{l}) \quad (34.1)$$

Note the similarity between this image model and the “independent pixels” model we saw in equation (27.1) in chapter 27. The present model can represent images with more structure because rather than assuming the pixels are all completely independent (i.e., marginally independent), it only assumes the pixel colors are *conditionally* independent, conditioned on the luminance image (which provides a great deal of structure).

The underlying assumption of this factorization is one of conditional independence: each pixel's ab value is considered to be conditionally independent from all other pixels' ab values, given the observed luminance (of all pixels). This is a very common assumption in image modeling problems: in fact, *whenever* you use least-squares regression for a multidimensional prediction, you are implicitly making this same assumption. To see this, suppose you are predicting a vector $\mathbf{y}$ from an observed vector $\mathbf{x}$, and your prediction is $\hat{\mathbf{y}} = f(\mathbf{x})$. Then, we can write the least-squares objective (L_2) as:

$$-\|\hat{\mathbf{y}} - \mathbf{y}\|_2^2 = \sum_i -(\hat{y}_i - y_i)^2 \quad (34.2)$$

$$= \log \prod_i \phi(\hat{y}_i, y_i) \quad (34.3)$$

The loss factorizes as a product over pairwise potentials. Therefore, by the Hammersley-Clifford theorem (chapter 29), the L_2 loss implies a probability distribution that treats all true values y_i as independent from one another, given all the predictions $\hat{y}_i$. The predictions are a function of just the input $\mathbf{x}$, so the implication is that all the true values y_i are independent from one another given the observation $\mathbf{x}$. Therefore, we have arrived at our conditional independence assumption: each dimension's predicted value is assumed to be independent of each other dimension's predicted values, given the observed input. This is a huge assumption and rarely true of prediction problems in computer vision!

So, whether you are using per-pixel classification, or least-squares regression, you are implicitly fitting a model that assumes independence

between all the output pixels, conditioned on the input. This is called **unstructured prediction**.

This causes problems. Let's return to our t-shirt example. We will use a per-pixel color classifier and see where it fails. Since the data distribution has two equally probable modes—teal and pink—the classifier will learn to place roughly equal probability mass on these two modes. As training data and time go to infinity, the classifier should recover the exact data distribution, but with finite data and time it will only be approximate, and so we might have a case where for some luminance values the classifier places 51 percent chance on teal and for others it places 49 percent on teal. Then if, as we scan across pixels in the shirt we are observing, the luminance changes very slightly, the model predictions might wiggle back and forth between 49 percent and 51 percent teal. As our application is to colorize the photo, at some point we need to make a hard decision and output a single color for each pixel. Doing so in the present case will cause chaotic transitions from predicting pink (where $p(\text{teal}) < 0.5$) and teal (where $p(\text{teal}) > 0.5$). An example of this kind of prediction flipping is shown in [figure 34.3](#).

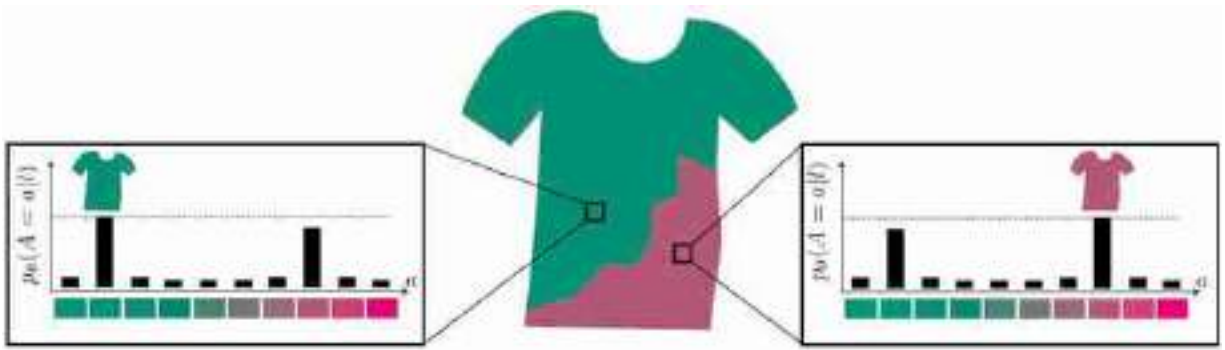


Figure 34.3: Color flipping can arise from a smooth underlying predictive distribution, on top of which independent choices are made.

A real example of color flipping from [523]. The model is unsure whether the shirt, and the background, are blue or red, so it chaotically alternates between these two options.



34.3 Conditional Generative Models Solve Multimodal Structured Prediction

In the previous section, we learned that standard approaches to prediction are insufficient for making the kinds of predictions we usually care about in computer vision.

Conditional generative models are a general family of prediction methods that:

- Model a multimodal distribution of predictions, and
- Model joint structure.

Methods that model joint structure in the output space are called **structured prediction** methods—they don't factorize the output into independent potentials conditioned on the input. Conditional generative modeling is a structured prediction approach that models a full distribution of possibilities over the joint configuration of outputs.

34.3.1 Relationship to Conditional Random Fields

The conditional random fields (CRFs) from chapter 29 are one kind of model that fits this definition. Now we will see some other ones. The big difference is that CRFs make predictions via *thinking slow* [244]: given a query observation, you run belief propagation or another iterative inference algorithm to arrive at a prediction. The conditional generative models we will see in this section *think fast*: they do inference through a single forward pass of a neural net. Sometimes the thinking fast approach is referred to as **amortized inference**, where the idea is that the cost of inference is amortized over a training phase. This training phase learns a direct mapping from inputs to outputs that approximates the solution we would have gotten if we did exact inference on that input.

34.4 A Tour of Popular Conditional Models

We saw a bunch of unconditional generative models in the previous chapters. How can we make each conditional? It is usually pretty straightforward. This is because if you can model an arbitrary distribution over a random variable, $p(Y)$, then you can certainly model the conditional distribution $p(Y | X = \mathbf{x})$ —it's just another arbitrary distribution. Of course, we typically care about modeling $p(Y | X = \mathbf{x})$ for *all* possible settings of $\mathbf{x}$. We could, but don't want to, fit a separate generative model for each $\mathbf{x}$. Instead we want neural nets that take a query $\mathbf{x}$ as input and produce an output that models or samples from $p(Y | X = \mathbf{x})$. We will briefly cover how to do this for several popular models:

In this chapter, Y is the image we are generating and X is the data we are conditioning on. Note that this is different than in the previous generative modeling chapters (chapters 32 and 33), where X was the image we were generating, unconditionally.

34.4.1 Conditional Generative Adversarial Networks

We can make a generative adversarial network (GAN; section 32.9) conditional simply by adding $\mathbf{x}$ as an input to both the generator and the discriminator:

$$\arg \min_{\theta} \max_{\phi} \mathbb{E}_{\mathbf{z}, \mathbf{x}, \mathbf{y}} [\log d_{\phi}(\mathbf{x}, g_{\theta}(\mathbf{x}, \mathbf{z})) + \log(1 - d_{\phi}(\mathbf{x}, \mathbf{y}))] \quad (34.4)$$

What this does is change the job of the discriminator from asking “is the output real or synthetic?” to asking “is the input-output *pair* real or synthetic?” An input-output pair can be considered synthetic for two possible reasons:

1. The output looks synthetic.
2. The output does not match the input.

If both reasons are avoided, then it can be shown that the produced samples are *iid* with respect to the true conditional distribution of the data $p_{\text{data}}(Y | \mathbf{x})$ (this follows from the analogous proof for unconditional GANs in [168], since that proof applies to modeling any arbitrary distribution over Y , including $p_{\text{data}}(Y | \mathbf{x})$ for any $\mathbf{x}$).

34.4.2 Conditional Variational Autoencoders

Recall that a variational autoencoder (VAE; section 33.4) is an infinite mixture model which makes use of the following identity:

$$p_{\theta}(\mathbf{x}) = \int_{\mathbf{z}} p_{\theta}(\mathbf{x} | \mathbf{z}) p_{\mathbf{z}}(\mathbf{z}) d\mathbf{z} \quad (34.5)$$

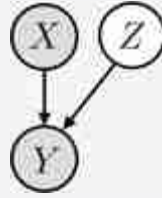
Analogously, we can define any conditional distribution as the marginal over some latent variable:

$$p_{\theta}(\mathbf{y} | \mathbf{x}) = \int_{\mathbf{z}} p_{\theta}(\mathbf{y} | \mathbf{z}, \mathbf{x}) p_{\mathbf{z}}(\mathbf{z} | \mathbf{x}) d\mathbf{z} \quad (34.6)$$

In **conditional VAEs (cVAEs)**, we restrict our attention to latent variables $\mathbf{z}$ that are independent of the inputs we are conditioning on, so we have:

$$p_{\theta}(\mathbf{y} | \mathbf{x}) = \int_{\mathbf{z}} p_{\theta}(\mathbf{y} | \mathbf{z}, \mathbf{x}) p_{\mathbf{z}}(\mathbf{z}) d\mathbf{z} \quad \triangleleft \quad \text{cVAE likelihood model} \quad (34.7)$$

This corresponds to this graphical model:



The idea is that $\mathbf{z}$ should only encode bits of information about $\mathbf{y}$ that are *independent* from whatever $\mathbf{x}$ already tells us about $\mathbf{y}$. For example, suppose that we are trying to predict the motion of a billiard ball bouncing around in a video sequence. We are given a single frame $\mathbf{x}$ and asked to predict the next frame $\mathbf{y}$. Only knowing $\mathbf{x}$ we can't know whether the ball will move up, down, diagonally, and so on, but we *can* know what color the ball will be in the next frame (it must be the same as the previous frame) and the rough position on the screen of the ball (it can't have moved too far). Therefore, the only missing information about $\mathbf{y}$, given we know $\mathbf{x}$, is the velocity of the ball. Naturally, then, if the model learns to interpret $\mathbf{z}$ as coding for velocity, we would have a perfect prediction $p_{\theta}(\mathbf{y} | \mathbf{z}, \mathbf{x})$ (one that places max likelihood on the observed next frame $\mathbf{y}$), and marginalizing over all the possible $\mathbf{z}$ values would place max likelihood on the data ($p_{\theta}(\mathbf{y} | \mathbf{x})$). This is what ends up happening in a cVAE (or, to be precise, it is one solution that maximizes the cVAE objective; it is not guaranteed to be the solution that is arrived at, but it is a good model of what tends to happen in practice). [Figure 34.4](#) shows this scenario.

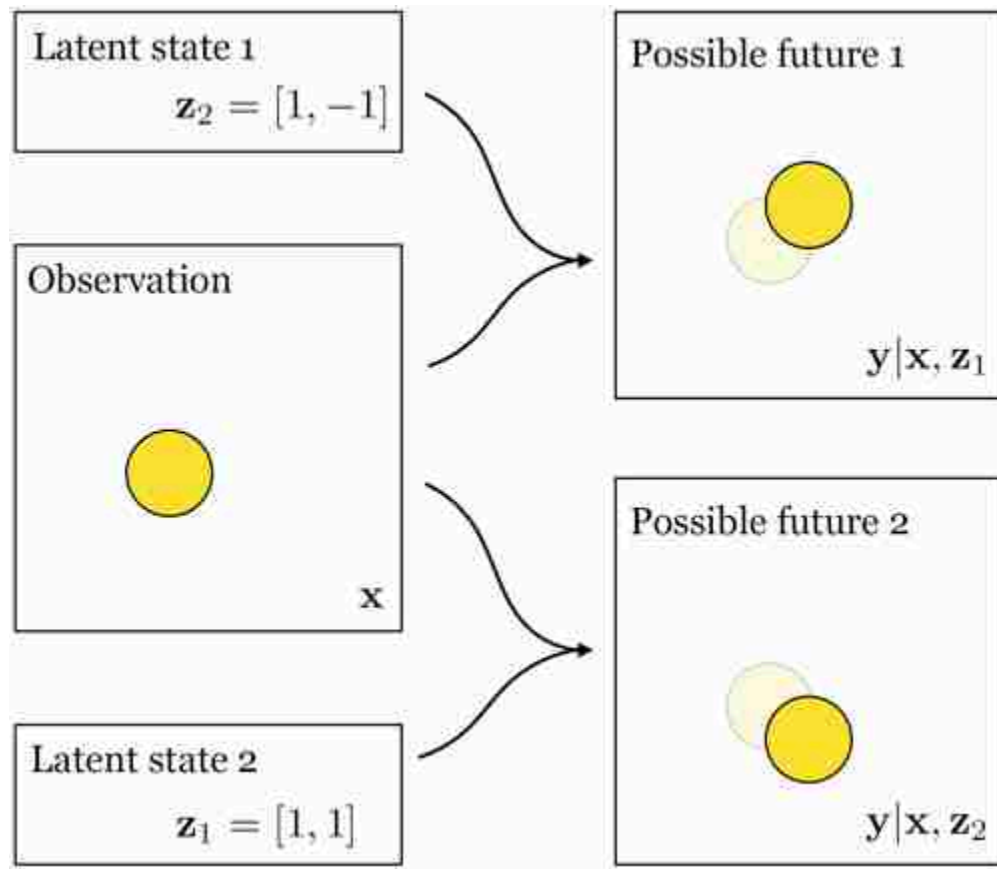


Figure 34.4: A scenario where a yellow ball is moving across a plane. The observation, x , is a static frame. From that observation, we know what will be the color and rough position of the ball in the next frame, y , but we don't know what direction it will have moved, because the velocity of the ball is unobserved. Therefore, velocity is a latent variable and one solution to the cVAE objective will be to encode in the model's latent variables (z) the velocity of the ball, as is depicted here.

Just like regular VAEs, cVAEs also have an encoder, which acts to predict the optimal importance sampling distribution $p_\theta(Z | x, y)$. In practice, this means that a cVAE can be trained just like a regular VAE except that the encoder takes the conditioning information, x , as input (in addition to y), and the decoder also takes in x (in addition to z). [Figure 34.5](#) depicts this setting.

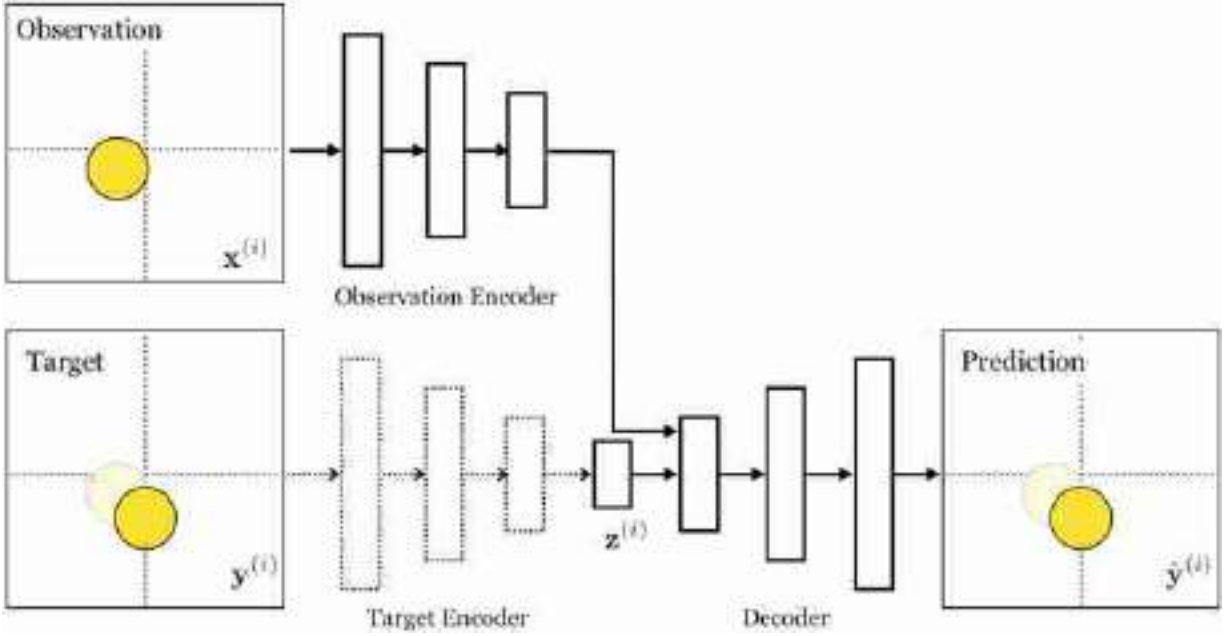


Figure 34.5: cVAE architecture. The dotted lines indicate that the *target encoder* is only used during training; at test time, usage follows the solid path.

34.4.3 Conditional Autoregressive Models

Autoregressive models (section 32.7) are already modeling a sequence of conditional distributions. So, to condition them on some inputs $\mathbf{x}$, we can simply concatenate $\mathbf{x}$ as a prefix to the sequence of y values we are modeling, yielding the new sequence: $[x_1, \dots, x_m, y_1, \dots, y_n]$. The probability model factorizes as:

$$p_{\theta}(\mathbf{y} | \mathbf{x}) = \prod_{i=1}^n p_{\theta}(y_i | y_1, \dots, y_{i-1}, x_1, \dots, x_m) \quad (34.8)$$

Each distribution in this product is a prediction of the next item in a sequence given the previous items, which is no different than what we had for unconditional autoregressive models. Therefore, the tools for modeling unconditional autoregressive distributions are also appropriate for modeling conditional autoregressive distributions. From an implementation perspective, the same exact code will handle both cases, just depending on whether you prefix the sequence or not.

34.4.4 Conditional Diffusion Models

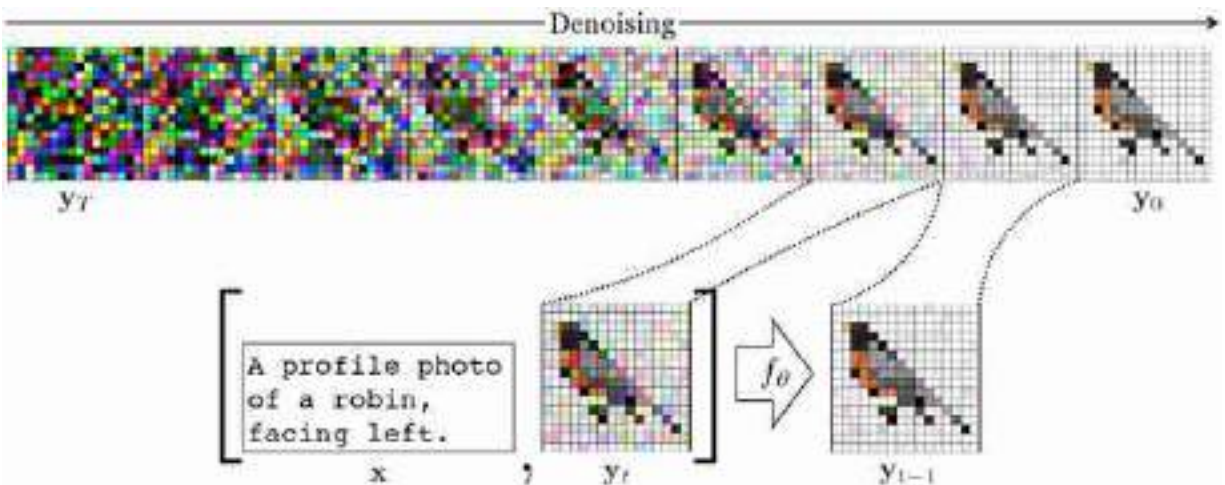
Diffusion models (section 32.8) are quite similar to autoregressive models and they can be made conditional in a similar way. All we need to do is

concatenate the conditioning variables, $\mathbf{x}$, into the input to the denoising function:

$$\hat{\mathbf{y}}_{t-1} = f_{\theta}(\mathbf{y}_t, t, \mathbf{x}) \quad (34.9)$$

Both diffusion models and autoregressive models convert generative modeling into a sequence of supervised prediction problems. To make them conditional is therefore just as easy as conditioning a supervised learner on more observations. If the original training pairs are $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$, we can condition on additional paired observations $\mathbf{c}$ by augmenting the pairs to become $\{[\mathbf{x}^{(i)}, \mathbf{c}^{(i)}], \mathbf{y}^{(i)}\}_{i=1}^N$.

An example where we condition on text is shown in [figure 34.6](#). The intuition is that if we give f_{θ} a description of the image as an additional input, then it can do a better job at solving its prediction task. Because of this, f_{θ} will become sensitive to the text command, and if you give different text it will denoise toward a different image—an image that matches that text! In chapter 51 we will describe in more detail a particular neural architecture for text-to-image synthesis, which is based on this intuition.



[Figure 34.6:](#) Text-conditional diffusion model.

34.5 Structured Prediction in Vision

Whenever you want to make a prediction of an *image*, conditional generative models are a suitable modeling choice. But how often do we really want to predict images? Yes, image colorization is one example, but

isn't that more of a graphics problem? Most problems in vision are about predicting labels or geometry, right?

Well yes, but what does label prediction look like? The input is an image and the output is label map. In image classification the output might just be a single class, but more generally, in object detection and semantic segmentation, we want a label for each part of the image. The target output in these problems is high-dimensional and structured. Or consider geometry estimation: the output is a depth map, or a voxel grid, or a mesh, and so on. All these are high-dimensional structured objects. The tool we need for solving these problems is structured prediction, and conditional generative models are therefore a good choice. In the next sections we will see two important families of structured prediction methods used in vision.

34.6 Image-To-Image Translation

Image-to-image problems are mapping problems where the input is an image and the output is also an image, where we will here think of an image as any array of size $N \times M \times C$ (height by width by number of channels). These problems are very common in computer vision. Examples include colorization, next frame prediction, depth map estimation, and semantic segmentation (a per-pixel label map is also an image, just with K channels, one for each label, rather than three channels, one for each color channel). One way to think about all these problems is as *translating* from one view of the data to another; for example, semantic segmentation is a translation from viewing the world in terms of its colors to viewing the world in terms of its semantics. This perspective yields the problem of **image-to-image translation** [232]; just like we can translate from English to French, can we translate from pixels to semantics, or perhaps from a photographic depiction of scene to a painting of that same scene?

34.6.1 Image-to-Image Translation with a Conditional GAN

One approach to image-to-image translation is to use a conditional GAN, as was popularized in the “pix2pix” paper [232] (whose method we will follow here). To illustrate this approach, we will look at the problem of translating a facade layout map into a photo. In this setting, the conditioning information (input) is a layout map showing where all the architectural

elements are positioned on the facade (i.e., a semantic segmentation image of the facade, with colors indicating where the windows, doors, and so on are located), and the output is a matching photo. The generator therefore maps an image to an image so we will implement it with an image-to-image neural architecture. This could be a convolutional neural network (CNN), or a transformer; following the pix2pix paper we will use a U-Net (a CNN with accordion-shaped skip connections; see section 24.11.2). The discriminator maps the output image to a real-versus-synthetic score (a scalar), so for the discriminator we will use a regular CNN (just like the ones used for image classification). Here is the full architecture ([figure 34.7](#)):

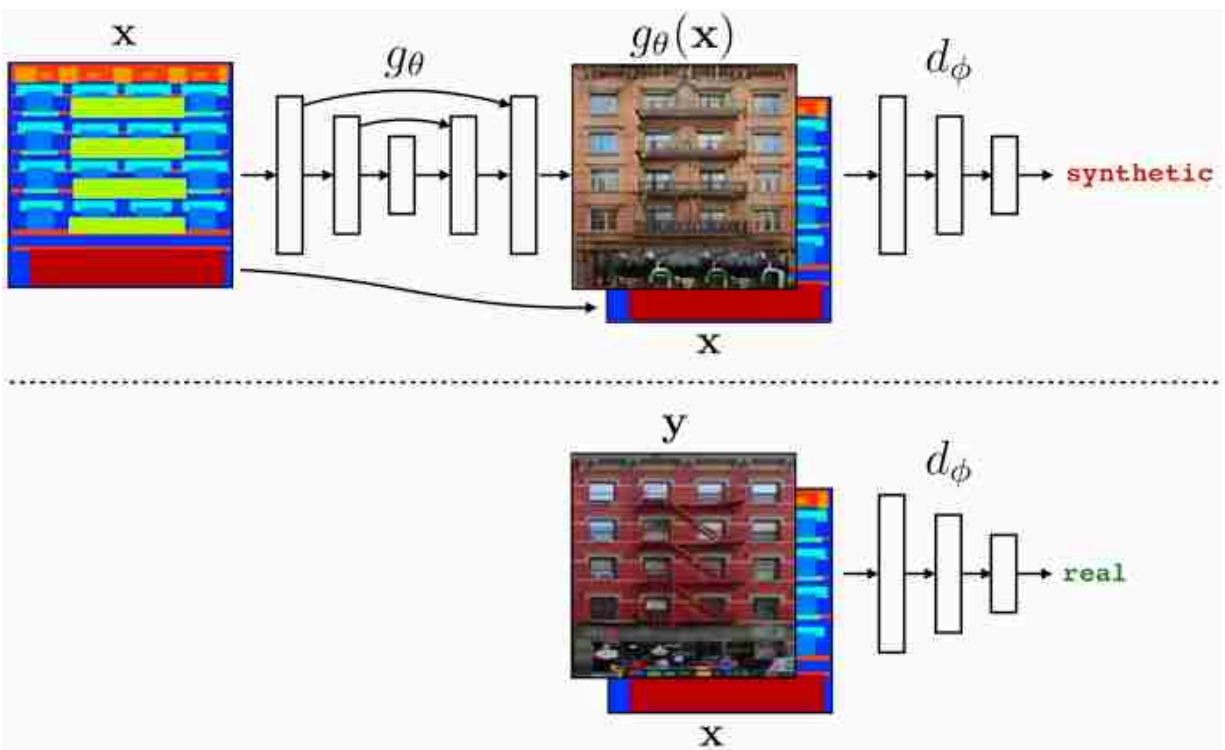
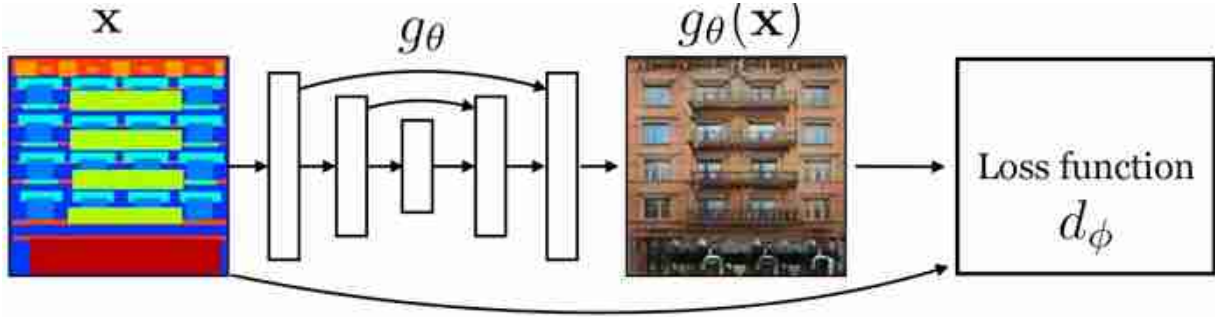


Figure 34.7: The pix2pix [232] model applied to translating a facade layout map into a photo of the facade. The model was trained on the CMP Facades Database [479] and the input and ground truth images shown here are from that dataset.

Notice that we have omitted the noise inputs here; instead the generator only takes as input the image x . It turns out that in this setting the noise is not really a necessary input, and many implementations omit it. This is

because the input image $\mathbf{x}$ has enough entropy by itself, and you don't necessarily need additional noise to create variety in the outputs of the model. The downside of this approach is that you only get one prediction $\mathbf{y}$ for each input $\mathbf{x}$.

One way to think about this setup is that it is a regression problem with a *learned loss function* d_ϕ , a view shown in [figure 34.8](#).



[Figure 34.8](#): The discriminator of a GAN as a learned loss function.

This loss function adapts to the structure of the data and the behavior of the generator to penalize just the relevant errors the generator is currently making. It can help to also add a conventional regression loss, such as the L_1 loss, to stabilize the optimization process, yielding the following objective:

$$\arg \min_G \max_D \mathbb{E}_{\mathbf{z}, \mathbf{x}, \mathbf{y}} [\log d_\phi(\mathbf{x}, g_\theta(\mathbf{x}, \mathbf{z})) + \log(1 - d_\phi(\mathbf{x}, \mathbf{y})) + \|g_\theta(\mathbf{x}) - \mathbf{y}\|_1] \quad (34.10)$$

Thinking of d_ϕ as a learned loss function raises new questions: What does this loss function penalize, and how can we control its properties. One of the main levers we have is the architecture of d_ϕ ; different architectures will have the capacity, and/or inductive bias, to penalize different kinds of errors. One popular architecture for image-to-image tasks is a **PatchGAN** discriminator [232], in which we score each *patch* in the output image as real or fake and take the average over patch scores as our total loss:

$$d_\phi(\mathbf{x}, \mathbf{y}) = \frac{1}{NM} \sum_{i=0}^N \sum_{j=0}^M d_\phi^{\text{patch}}(\mathbf{x}[i:i+k, j:j+k], \mathbf{y}[i:i+k, j:j+k]) \quad (34.11)$$

where $k \times k$ is the patch size. Notice that this operation is equivalent to the sliding window action of CNN, so d_ϕ^{patch} can simply be implemented as a

CNN that outputs a $2 \times N \times M$ feature map of real-versus-synthetic scores. Naturally, patches can be sampled at different strides and resolutions, depending on the exact architecture of the CNN d_{ϕ}^{patch} .

The PatchGAN strategy has two advantages over just scoring the whole image as real or synthetic with a classifier architecture: (1) it can be easier to model if a patch is real or synthetic than to model if an entire image is real or synthetic (d_{ϕ}^{patch} needs fewer parameters), (2) there are more patches in the training data than there are images. These two properties give PatchGAN discriminators a statistical advantage over whole image discriminators (fewer parameters fit to more data). The disadvantage is that the PatchGAN only has the architectural capacity to penalize errors that are observable within a single patch. This can be seen by training models with different discriminator patch sizes and observing the results. We show this in [figure 34.9](#).



Figure 34.9: Varying the receptive field (patch size) of the convolutional discriminator affects what kinds of structure the discriminator enforces. These results are from [232].

A 1×1 discriminator can only observe a single pixel at a time and cannot penalize errors in joint pixel statistics such as edge structure, hence the blurry results. Larger receptive fields can enforce higher order patch realism, but cannot model structure larger than the patch size (hence the tiling artifacts that occur with a period roughly equal to the patch size). Quality degrades with the 286×286 discriminator possibly because this discriminator has too hard a task given the limited training data (in any given training set, there are fewer examples of 286×286 regions than there are of, say, 70×70 regions).

34.6.2 Unpaired Image-to-Image Translation

In the preceding section we saw how to solve image-to-image translation by treating it just like a supervised regression problem, except using a learned loss function given by a GAN discriminator. This approach works great when we have lots of training data pairs $\{\mathbf{x}, \mathbf{y}\}$ to learn from. Unfortunately, very often, paired training data is hard to obtain. For example, consider the task of translating a photo into a Cezanne painting ([figure 34.10](#)):

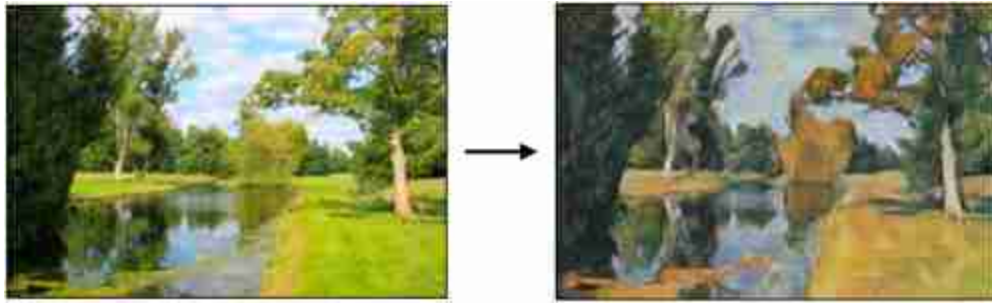


Figure 34.10: A style transfer example from the CycleGAN paper [531]. *Input photo source:* Alexei A. Efros.

The task depicted here is to predict: What would it have looked like if Cezanne had stood at this riverbank and painted it?

How could this be done? We can't have solved it with paired training data because, in this setting, paired data is extremely hard to come by. We would have to resurrect Cezanne and ask him to paint for us a bunch of new scenes, one for each photo in our desired training set. But is all that effort really necessary? You and I, as humans, can certainly imagine the answer to the previous question. We know what Cezanne's style looks like and can reason about the changes that would have to occur to make the photo match his style. Dabs of paint would have to replace the photographic pixels, and the colors should take on more pastel tones. We can imagine the necessary stylistic changes because we have seen many Cezanne paintings before, even though we didn't see them paired with a photograph of the exact same scenes. Let's now see how we can give a machine this same ability. We call this setting the **unpaired translation** setting, and distinguish it from **paired translation** as indicated in [figure 34.11](#).

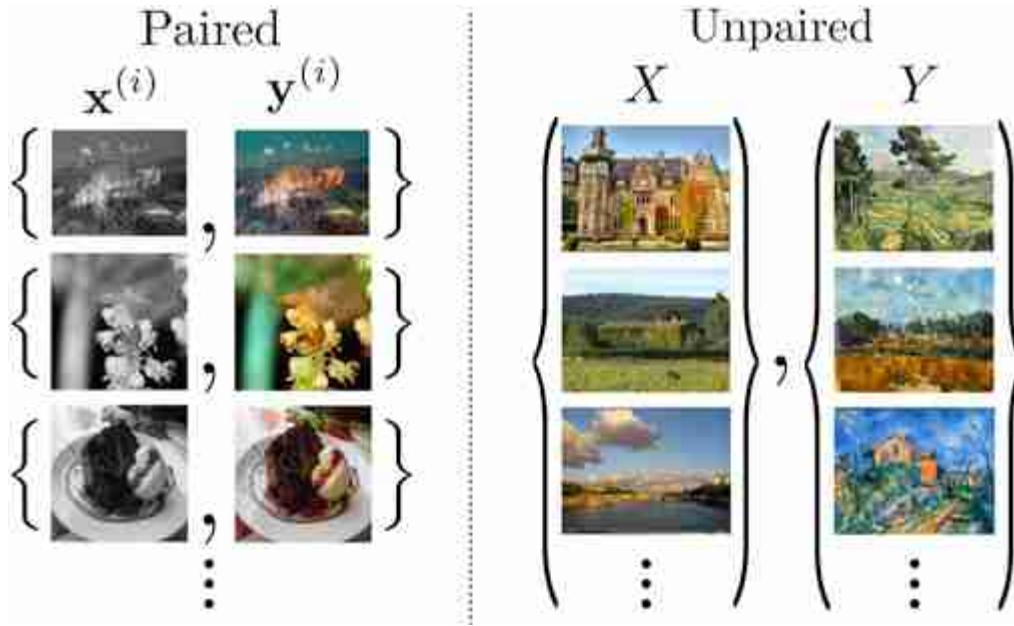


Figure 34.11: (left) An example of paired image-to-image translation (colorization) versus (right) unpaired translation (photo to Cezanne). Figure adapted from [531].

It turns out that GANs have a very useful property that makes them well-suited to solving this task. GANs train a mapping from a noise distribution to a data distribution $p(Z) \rightarrow p(X)$. They do this without knowing the pairing between $\mathbf{z}$ and $\mathbf{x}$ values in the training data. Instead this pairing is *emergent*. Which $\mathbf{z}$ -vector corresponds to which $\mathbf{x}$ is not predetermined but self-organizes to satisfy the GAN objective (all $\mathbf{z}$ -vectors should map to realistic images, and, collectively they should map to the data distribution).

Now consider training a GAN to map from $\mathbf{x}$ to $\mathbf{y}$ values:

$$\arg \min_{\theta} \max_{\phi} \mathbb{E}_{\mathbf{x}, \mathbf{y}} [\log d_{\phi}(g_{\theta}(\mathbf{x})) + \log(1 - d_{\phi}(\mathbf{y}))] \quad (34.12)$$

Note that this is a regular GAN objective rather than a conditional GAN, except that we have relabeled the variables compared to equation (32.49), with $\mathbf{y}$ playing the role of $\mathbf{x}$ and $\mathbf{x}$ playing the role of $\mathbf{z}$.

Such a GAN would self-organize so that the outputs are realistic $\mathbf{y}$ values and so that different inputs map to different outputs (collectively they must map to the data distribution over possible $\mathbf{y}$ values). No paired supervision is required to achieve this.

Such a GAN may indeed learn the correct mapping from $\mathbf{x}$ to $\mathbf{y}$ values—that's one of the solutions that minimizes the loss—but there are many

symmetries in this learning problem, that is, many different mappings achieve equivalent loss. For example, consider that we permute the mapping: if $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}$ is the true mapping then consider we instead recover $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i+1)}\}$ after learning (circularly shifting at the boundary). This mapping achieves equal loss to the true mapping, under the standard GAN objective (equation [34.12]). This is because any permutation in the mapping does not affect the output *distribution*, and the GAN objective only cares about the output distribution.

Put another way, the GAN discriminator is different than a normal loss function. Rather than checking if the output matches a target *instance* (that $g_{\theta}(\mathbf{x}^{(i)})$ matches $\mathbf{y}^{(i)}$), it checks if the output is part of an admissible set, that is, the set of things that look like realistic $\mathbf{y}$ values. This property makes GANs perfect for working with unpaired data, where we don't have instance-level supervision but only have set-level supervision.

We call this **unpaired** learning rather than unsupervised because we still have supervision in the form of labels indicating which *set* each datapoint belongs to. That is, we know whether each image is an $\mathbf{x}$ or a $\mathbf{y}$.

So, a regular GAN objective, applied to mapping from X to Y , solves the unpaired translation problem, but it does not distinguish between the correct mapping and the many other mappings that achieve the same output distribution. To break the symmetry we need to add additional constraints or inductive biases. One that works especially well is **cycle-consistency**, which was introduced in this context by CycleGAN [531], Dual-GAN [514], and DiscoGAN [257], which are all essentially the same model. The idea of cycle-consistency is that if we translate from X to Y and then translate back (from Y to X) we should arrive where we started. Think about translating from English to French, for example. If we translate “apple” to French, “apple” → “pomme,” and then translate back, “pomme” → “apple,” we arrive where we started.¹⁶ We can check the quality of a translation system, for a language we are unfamiliar with, by using this trick. If translating back does not return the word we started with then something went wrong with the translation model. This is because we expect language translation to be roughly one-to-one: for each word in English there is usually an equivalent word in French. Cycle-consistency losses are

regularizers that encourage a learned mapping to be roughly one-to-one. The cycle-consistency losses from CycleGAN are simply:

$$\mathcal{L}_{cyc}(\mathbf{x}; g_\theta, f_\psi) = \|\mathbf{x} - f_\psi(g_\theta(\mathbf{x}))\|_1 \quad (34.13)$$

$$\mathcal{L}_{cyc}(\mathbf{y}; g_\theta, f_\psi) = \|\mathbf{y} - g_\theta(f_\psi(\mathbf{y}))\|_1 \quad (34.14)$$

Adding this loss to equation (34.12) yields the complete CycleGAN objective:

$$\arg \min_{\theta, \psi} \max_{\phi} \mathbb{E}_{\mathbf{x}, \mathbf{y}} [\log d_\phi^Y(g_\theta(\mathbf{x})) + \log(1 - d_\phi^Y(\mathbf{y})) + \|\mathbf{x} - f_\psi(g_\theta(\mathbf{x}))\|_1 + \quad (34.15)$$

$$\log d_\phi^X(f_\psi(\mathbf{y})) + \log(1 - d_\phi^X(\mathbf{x})) + \|\mathbf{y} - g_\theta(f_\psi(\mathbf{y}))\|_1] \quad (34.16)$$

This model translates from domain X to domain Y and back, such that the outputs in domain Y look realistic according to a domain Y discriminator, and vice versa for domain X , *and* the mappings are cycle-consistent: one complete cycle from X to Y and back should arrive where it started (and, again, vice versa). A diagram for this whole process is given in [figure 34.12](#):

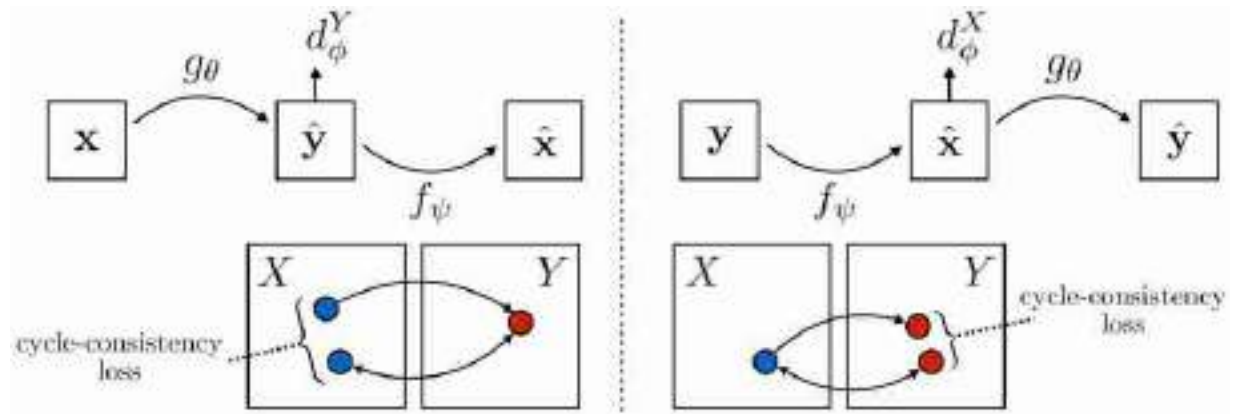


Figure 34.12: CycleGAN schematic. Figure derived from [531].

The cycle-consistency losses encourage that the translation is a one-to-one function, and thereby reduce the number of valid solutions. This is a good idea when the true solution is indeed one-to-one, as is often the case in translation problems. However, other symmetries still exist and may confound methods like CycleGAN: if we permute the correct X -to- Y mapping and apply the inverse permutation to the Y -to- X mapping, then CycleGAN will still be satisfied but the mapping will now be incorrect.

34.7 Concluding Remarks

Many problems in vision can be phrased as structured prediction, and conditional generative models are a great approach to these problems. They provide a unified and simple solution to a diverse array of problems. A current trend in computer vision (and all of artificial intelligence) is to replace bespoke systems for specific kinds of structured prediction—object detectors, image captioning systems, and so on—with general-purpose conditional generative models. This is one reason why this book does not focus much on special-purpose systems that solve specific problems in vision—because the trend is toward general-purpose modeling tools.

[16.](#) In natural language processing, the idea of cycle-consistency is known as **backtranslation**.

X

CHALLENGES IN LEARNING-BASED VISION

Learning-based vision comes with a unique set of challenges. This approach to vision derives rules from *data*, and therefore requires tools for understanding where data can go wrong. This part of the book first presents several failure modes of data-driven methods, and then provides tools for mitigating these failures.

- **Chapter 35** introduces the problem of dataset bias and distribution shift. We also encounter the issue of adversarial examples. These problems all come down to a gap between how the model is trained and how it will be used.
- **Chapter 36** presents one way of dealing with this gap: train on a more diverse distribution of data.
- **Chapter 37** presents a second way of dealing with the gap: adapt your models to bridge the gap.

35 Data Bias and Shift

35.1 Introduction

The goal of learning is to acquire rules from past data that work well on future data. In the theory of machine learning, it is almost always assumed that the future is *identically distributed* as the past. In reality, it is almost always the case that this is not true. Many of the failures of machine learning in industry and in society are due to this discrepancy between theory and practice.

What happens when a learned system is deployed on data that is unlike what it was trained on? In this chapter we will look just at what can go wrong, and in later chapters we will consider how we can mitigate these effects.

As a case study we will look at the paper “Colorful Image Colorization” [523], which one of the authors of this book co-authored in 2016, so we don't feel too bad pointing out its flaws. In that paper, we trained a CNN to colorize black and white photos, and the results looked amazing ([figure 35.1](#)).



Figure 35.1: Each pair of images shows the grayscale input image and, to its right, the output of the automatic colorization system. Source: [523].

This was the teaser figure from our paper, showing sixteen examples of grayscale input images and the output of our system. Looks almost perfect, right? Well that's what people must have thought, because shortly after releasing our code, a user of the website Reddit¹⁷ turned it into a bot called ColorizeBot¹⁸ that would colorize black and white photos posted on Reddit [24]. In [figure 35.2](#) are two of the more popular results.

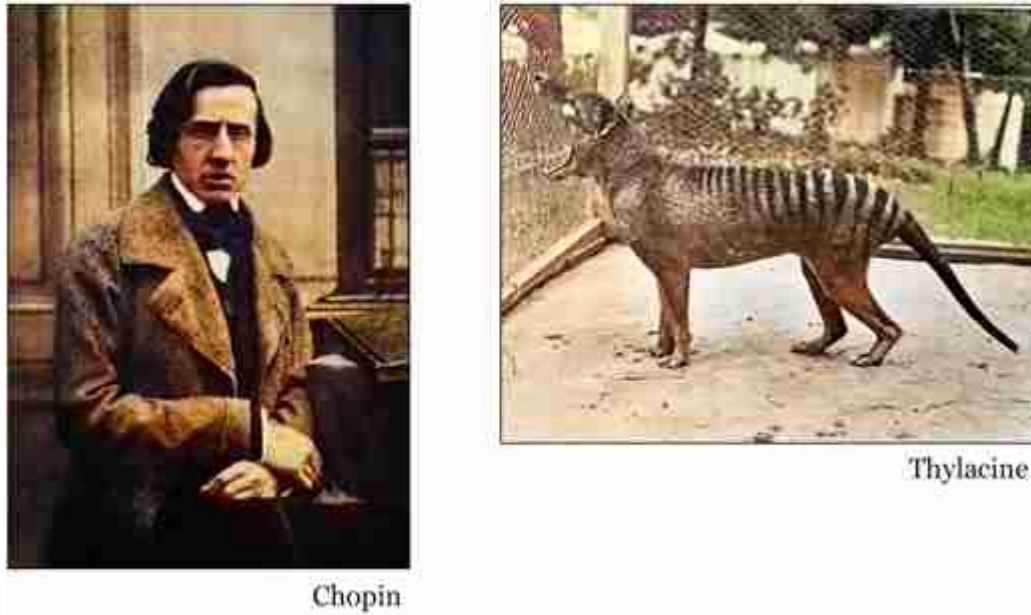


Figure 35.2: Two results of Reddit's ColorizeBot [24]. Original images are (left) Chopin, by Louis-Auguste Bisson, 1849, (right) and Benjamin the Thylacine, 1933. The bot ran the model from [523] on the original black and white photos.

The results were much worse than what we had shown in our paper. The bot turned most photos sepia-toned, and would frequently add ugly smudges of yellow and red. You might think, well the Reddit user must have implemented our method incorrectly, but that isn't it; to our knowledge, the code is correct. Why then, did it work so poorly?

The answer is data bias. We trained our colorization net on ImageNet. Reddit users tested it on famous historical photos, extinct animals, and old pictures of their family members. ImageNet is mostly photos of animals, along with a few other object categories. It does not contain many photos of scenes, and especially underrepresents humans, compared to how often people take photos of humans. In fact, a large percent of ImageNet photos are just dogs (around 12 percent). You can get an idea of the degree of the bias by looking at how our net colorizes the dogs in [figure 35.3](#).



Figure 35.3: (left) Colorization results, using a model trained on ImageNet (example made by Richard Zhang; input black and white photos from ImageNet [419]). (right) Examples of dog images in ImageNet [419].

It adds a pinkish blob underneath their mouths! Why? Because not only is ImageNet full of dogs, it is full of dogs with their tongues out. So the net learned that it's a good strategy to put down a blob of pink whenever it detects a dog mouth, regardless of whether that mouth is open or closed.

Our net did not work well because the training data (ImageNet) was categorically different than the test data (photos uploaded on Reddit). This is true in terms of the content of the images but also in terms of the photographic style of the images. ImageNet is almost exclusively made up of photos taken with modern digital cameras. Reddit users were more interested in old photos taken with analog cameras, full of photographic grain and covered in dust and scratches.

35.2 Out-Of-Distribution Generalization

To formalize what's going on, we will return to a simple regression problem, shown below ([figure 35.4](#)).

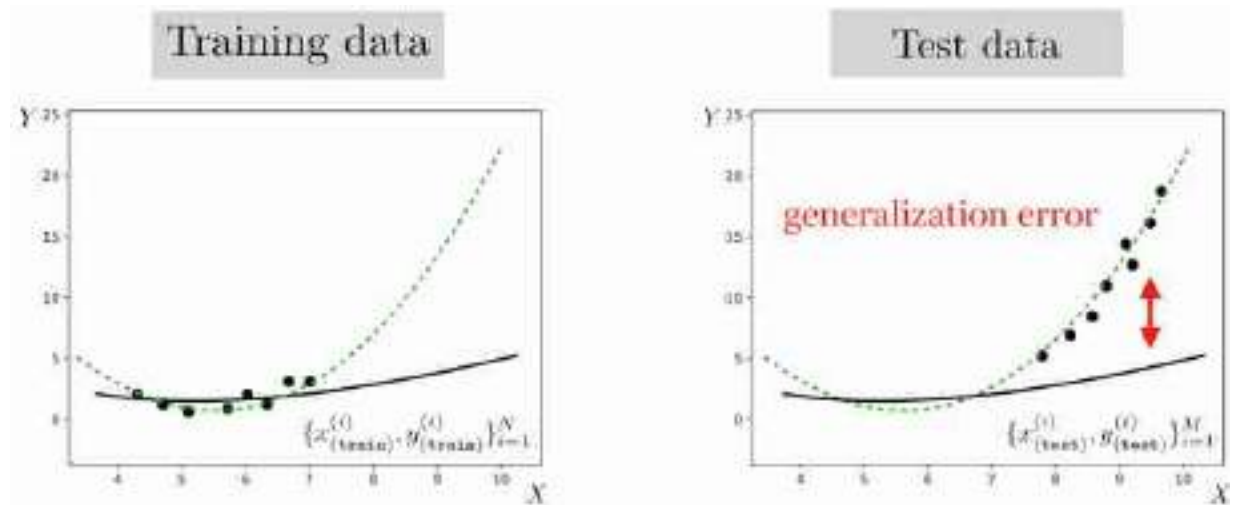


Figure 35.4: Generalization error can be arbitrarily bad when there is distribution shift between training and test.

The green line indicates the true data generating process, p_{data} . The training data is sampled from the flat part of the curve, whereas the test data is sampled from a different region, where the curve slopes sharply up. The solid black line is the least squares fit on the training data. It is a great fit for that part of the function but not for other parts of the data space.

The same thing is happening in the colorization example: the training data is sampled from one part of the total data space and the test data is sampled from another part. There is a **domain gap** between the training distribution and test distribution, which we indicate with the red arrows in [figure 35.5](#).

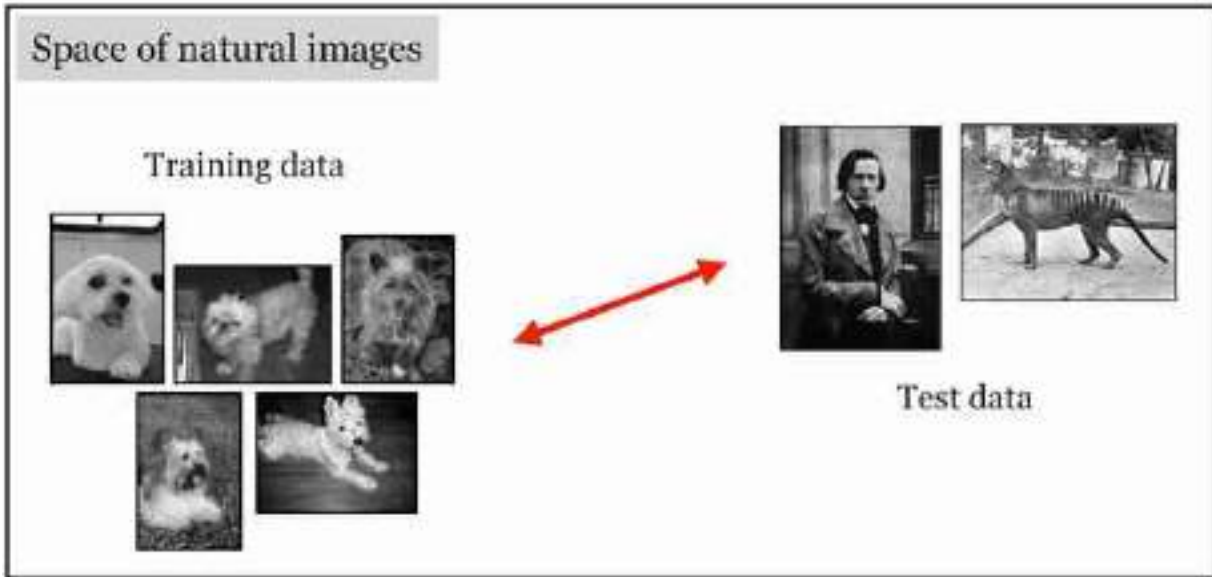


Figure 35.5: Domain gap between training data and test data in the ColorizeBot example.

Aside from colorizing photos on Reddit, how often do situations like this come up in the real world? *All the time*. Suppose you train a weather prediction system on data from the winter months, then test it in the summer. Or train it in 2020 and test it in 2030. What would happen if there were global warming? Suppose you trained a predictive model at the beginning of the COVID-19 pandemic, based on the observed behavior at that time. What happens when people adjust their behavior? What happens if a vaccine is developed, or if a variant emerges. Financial advisors offer a disclaimer like “past performance is no guarantee of future results.” Why do they have to say this? In classical statistical learning theory, past performance does provide a (probabilistic) guarantee over future results. In all these cases we are confounded by a shifting data distribution. Shifting data is the rule, not the exception.

35.3 A Toy Example

Let's start by looking at toy example. In this toy example, we will consider that we have access to two datasets, each one with different data distributions. We will study the effect of training on one dataset and testing on another.

Observations (x, y) are sampled from a distribution where x is sampled from a Gaussian and y are noisy observations of the function f applied to x :

$$x \sim \mathcal{N}(\mu, \sigma) \quad (35.1)$$

$$y = f(x) + \beta \epsilon \quad (35.2)$$

$$\epsilon \sim \mathcal{N}(0, 1) \quad (35.3)$$

The unknown function f has the form:

$$f_{\theta}(x) = \theta_0 x + \theta_1 x^2 \quad \triangleleft \text{ hypothesis space} \quad (35.4)$$

In the first dataset, the Gaussian parameter's are $\mu_A = -1.5$, and the standard deviation is $\sigma_A = 1.5$. In the second dataset, the Gaussian has $\mu_B = 2.5$ and $\sigma_B = 0.5$.

For each dataset we will sample two sets of examples. One set of examples will be the training set, and the other set of examples will be the test set, which we will use to measure the generalization error.

Learning is done by optimizing the loss over the corresponding training set:

$$J(\theta; \{x^{(i)}, y^{(i)}\}_{i=1}^N) = \frac{1}{N} \sum_i \|f_{\theta}(x^{(i)}) - y^{(i)}\|_2^2 + \lambda \|\theta\|_2^2 \quad \triangleleft \text{ objective} \quad (35.5)$$

[Figure 35.6](#) shows training and test examples from both datasets and illustrates the effect of training on one dataset and testing on another. Each plot shows the training samples (circles), used to fit the model, and test samples (triangles) used to measure the generalization error. The diagonal plots show the training and test set corresponding to distribution A, [figure 35.6](#)(top-left), and distribution B, [figure 35.6](#)(bottom-right). In each case, the fitted model is shown as a dashed line.

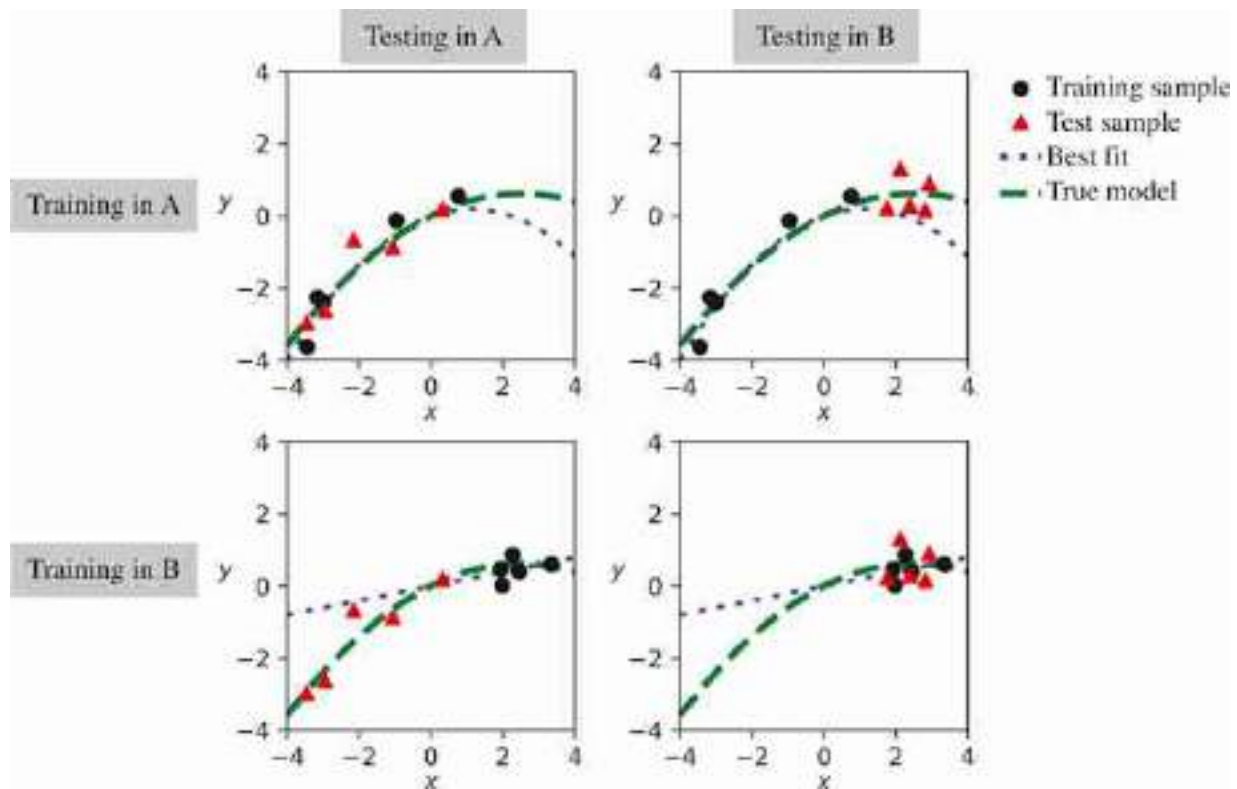


Figure 35.6: Training and testing on different datasets. In this illustration, all datasets have 5 samples.

When training and testing is done using samples from the same distribution (A to A, or B to B) the learned model (best fit) passes near the test samples and the error is small. However, when training and test are coming from different distributions, the error can be large. [Table 35.1](#) shows the generalization error made with different combinations of training and set sets as shown in [figure 35.6](#).

Table 35.1

Generalization error on the test sets A and B, when training on the training samples from datasets A and B.

		Test set	
		A	B
Training set	A	0.18	0.7
	B	1.93	0.23

The diagonal elements correspond to the error obtained when training on the training examples of dataset A and testing on the test examples of dataset A, and when training and testing on dataset B.

As shown in [table 35.1](#), the error becomes larger for the elements off-diagonal. However, [table 35.1](#) only gives a partial picture about what is happening because it gives the error only for dataset size equal to 5. What happens if we add more data?

It is interesting in general to look at the performance as a function of training data. If we add more data, does performance improve when training and testing on different datasets? Or does it saturate? The answer will depend on the form of the data distributions and the hypothesis space. But we can check what happens in this toy example.

[Figure 35.7](#) plots the generalization error (vertical axis) when evaluating on dataset A as a function of the amount of training data (horizontal axis).

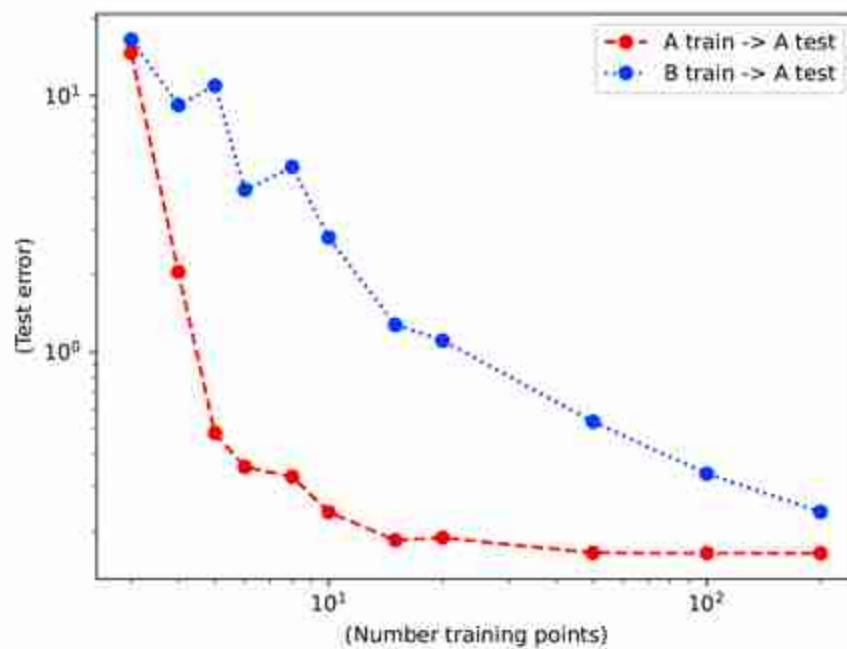


Figure 35.7: Generalization error as a function of number of training examples. Test set is dataset A. Each curve shows the performance when training with dataset A and with dataset B (same distributions as in [figure 35.6](#)).

Interestingly, both training sets (A and B) succeed in reducing the generalization error on the test set of dataset A when more training data is

available. In this example, even the biased dataset B is still able to train a successful model of dataset A. However, the reduction of the error is slower than when training on dataset A. Examples from dataset B are less valuable than examples from dataset A [471]. We need approximately 10 times more examples from dataset B in order to reach a particular performance. Another way of saying this is that the value of a training example from dataset B when evaluating on dataset A is around 1/10 of the value of a sample from A.

In this toy example, we created two datasets that had a mean shift and a standard-deviation shift. Shifts in the distributions represented by different datasets are always present.

35.4 Dataset Bias

The first thing is to notice that bias is present in all datasets. Even datasets that try to cover the same aspect of the world, they collect data with different styles and strategies that will reflect on the statistics of the dataset. In the case of image datasets for computer vision, the goal is to represent the visual world with all its complexity and diversity. Despite that there is a single visual world, different computer vision datasets offer a different, and biased, sample of that world.

In the paper [471], the authors propose a game that we reproduce in [figure 35.8](#). The name of the game is *Name That Dataset!* The figure shows three random images from 12 famous computer vision datasets and the goal of the game is to match the images with each dataset name. The point of the game is to show that it is easy to recognize the *style* of images and to identify the dataset they belong to. If the datasets were unbiased samples of the same visual world it would be impossible to win the game, yet researchers with experience on these datasets can get around 75 percent of the matches right [471].



Figure 35.8: Let's play the *Name That Dataset!* game. Modified from [471]. The game consists in associating each set of three images with the name of the dataset, from the list on the right, they belong to.

Answer key for [figure 35.8](#): 1) Caltech-101, 2) UIUC, 3) MSRC, 4) TinyImages, 5) ImageNet, 6) PASCAL VOC, 7) LabelMe, 8) SUNS-09, 9) 15 Scenes, 10) Corel, 11) Caltech-256, 12) COIL-100.

One can also train a classifier to play the *Name That Dataset!* game automatically. We can do that by selecting a random subset of images from each dataset that we will use to train a classifier, and then we will test on new unseen images. Indeed, as shown in [471], a classifier can play the game quite successfully.

35.4.1 The Value of Training Examples

In [table 35.2](#), each number represents the value of one training example from one dataset when used to test on another. A value of 0.25 means that you need 4 times more training examples than when using a sample derived

from the same testing distribution. These results are from 2011 [471] and computer vision datasets have evolved since then.

Table 35.2

“Market Value” for a “car” sample across datasets. Modified from [471].

	LabelMe market	PASCAL mkt.	ImageNet mkt.	Caltech101 mkt.
1 LabelMe is worth	1 LabelMe	0.26 Pascal	0.31 ImageNet	0 Caltech
1 Pascal is worth	0.50 LabelMe	1 Pascal	0.88 ImageNet	0 Caltech
1 ImageNet is worth	0.24 LabelMe	0.40 Pascal	1 ImageNet	0 Caltech
1 Caltech101 is worth	0.23 LabelMe	0 Pascal	0.28 ImageNet	1 Caltech

35.5 Sources of Bias

There are many reasons why datasets are biased. It is important when designing a new dataset to study first the potential sources of biases associated with each data collection procedure so as to mitigate those biases as much as possible. Collecting a diverse and an unbiased dataset usually translates to improved performance for the downstream task. We will list here some sources of bias, but this list is far from exhaustive.

35.5.1 Photographer Bias

The photographer bias is probably one of the most common biases in image databases, and surprisingly difficult to get rid of it.

In a beautiful study, Steve Palmer et al [371] showed participants photographs of objects with different orientations in order to measure what orientations they preferred. For each object there were 12 different orientations. Participants had consistent preferences for particular viewpoints for each object. Interestingly, those preferences also correlated with how easy it was to recognize each object. In another experiment from the same study, participants were shown one picture and their task was to

classify the object as quickly as possible from a fixed set of twelve known possible classes. The average response time and accuracy were measured for each of the 12×12 images. The speed and accuracy are a measure of how easy is to recognize an object under a particular viewpoint. The study showed that people recognized objects better when presented in a canonical orientation. [Figure 35.9](#) shows the preferred viewpoint for the 12 objects studied.



Figure 35.9: Canonical viewpoints for 12 objects. Figure from [\[371\]](#).

The views that are recognized faster also match the viewpoints that are preferred when taking a picture of those objects [\[371\]](#). Such photographer preferred viewpoint introduces biases on how objects are captured, and this bias makes the statistics of object pose in photographs different from the statistics of how objects are actually seen. The photographer bias exists for objects and scenes (composed of many objects). When taking pictures of scenes, there are biases on the preferred viewpoint, camera orientation (e.g., the horizon is usually inside the picture frame), object locations (such as the one third rule), object arrangements, positions of shadows, types of

occlusions, position of the sun, time of day, weather, and so on. All of those factors are taken in consideration (consciously or not) by photographers when deciding which pictures to take.

When building a benchmark, especially when it is small, it is easy to introduce another photographer bias which is to take pictures that “feel” easier to recognize. This might be the right thing to do at first, when debugging a system, but, if abused, has the risk of leading us in the wrong direction when choosing which approach works best.

These biases also apply to sequences. When filming, the user will tend to record people in certain configurations, or have the camera position in a specific orientation which respect to the main direction of the action, etc. Movie directors also use a number of techniques to capture shots that produce special viewpoint distributions.

35.5.2 Collection Bias

There are many potential data sources, all with different biases; Egocentric videos, internet images, vacation photographs, video captured from a moving vehicle, etc. The potential biases from each data source can be quite significant, and it's often straightforward to identify the capture technique merely by examining the images in a dataset.

[Figure 35.10](#) shows the first 22 images returned by Google for the query “mug.” It looks far from a diverse and unbiased collection of images. As you can appreciate these pictures of mugs look very different from the visual settings in which one usually encounters mugs (inside cabinets, on top of a table, inside the dishwasher, or dirty inside the sink). Instead the images that appear in the search are taken against a flat background, mostly just white, and under a very canonical point of view. For instance, there are no views directly from above. Other biases can be a bit subtle, for example, most of the handles appear on the right side, and most of the mugs are fully visible, without any occlusions from hands or other objects, and the only two cases of occlusions from mugs occluded by another mug.



Figure 35.10: Pictures of mugs returned by a Google image search. There are many types of biases present in this small image collection.

The example of the mug, although a bit of a caricature, helps to understand some of the biases that come from the particular collection protocol used to collect images. In this case, images on the internet have a particular type of bias (at least of the ones that get returned under specific searches).

[Figure 35.11](#) shows images returned when looking for “horse.” Interestingly many of the pictures are similar to the horse canonical orientation shown in [figure 35.9](#).



Figure 35.11: Pictures of horses returned by a Google image search. There are many types of biases present in this small image collection.

35.5.3 Annotation Bias

When annotating images, image annotators will choose which images to annotate and which labels to use. This will also correlate with viewpoint preferences and other photographer biases as those images might be easier

to annotate and segment, and this has the risk of compounding with the photographer bias.

Annotations will require making language choices and that are an important source of bias, especially social bias.

35.5.4 Social Bias

This type of bias can affect both the images and the associated labels. Social bias refers to biases associated to how groups of people might be reflected differently in the data. Social biases can be originated by the data collection protocol or by existing social biases. Images can be captured in certain world regions, or contain stereotypes. Labels can reflect social assumptions and biases. An example of social bias is a dataset of images and labels that associates doctors with males and nurses with females.

These types of biases, besides impacting the overall model performance, can have a negative social impact once they are deployed. It is important to be aware of those biases, and incorporate techniques to mitigate their effects. Now it is a good moment for the reader to revisit chapter 4.

35.5.5 Testing and Benchmark Bias

One usually thinks about the biases in the training set, but biases in the test set can have an even more negative impact on the final system behavior than the training bias. The test set is what researchers and engineers use to decide which algorithm is best, or what data collection is more effective for training. However, there might be little correlation between the performance measured on the test set and the final performance of the system during deployment.

In particular, if we compare two training datasets, one biased and another less biased, by evaluating the performance of a system trained on them using a biased test dataset, we could reach the conclusion that the biased dataset is better.

Remember that “bias” is a relative term between two datasets, or between a dataset and the real world, or a desired distribution. We only have access to the real world via datasets (which inevitably are biased).

35.6 Adversarial Shifts

The data shifts described previously may be disconcerting, but at least they are not malicious. Unfortunately, things can get worse. Quite often the data shift is not just random but adversarial. This may come up whenever our test setting is under the control of a competing agent, for example, someone trying to hack our system (computer security), another animal trying to eat us (evolution), or a trader trying to outperform us on the stock market (finance). Worse still, the neural nets we have seen in this book are very easy to fool by an adversary.

35.6.1 Adversarial Attacks

This was famously demonstrated by [461], who showed that one can add imperceptible noise to an image that fools neural net classifiers despite looking completely innocuous to a human. In fact, the noise pattern can be selected to force the network to produce any output the attacker desires. These are called **adversarial attacks** because we suppose the attacker, who adds the noise, is an adversary that wants to fool or manipulate your system. The adversary can do this by simply optimizing the noise pattern to cause the network to make a particular output, while restricting the noise to be very subtle (within an small r -radius ball around the original image). For example, to cause an image classifier f to think the cat in [figure 35.12](#) is an ostrich, the attacker could find the noise pattern via the following optimization:

$$\arg \max_{\epsilon} H(y_{\text{ostrich}}, f(x + \epsilon)), \quad \text{subject to} \quad \|\epsilon\| < r \quad (35.6)$$

where H is the cross-entropy loss and y_{ostrich} is a one-hot code for the ostrich class.

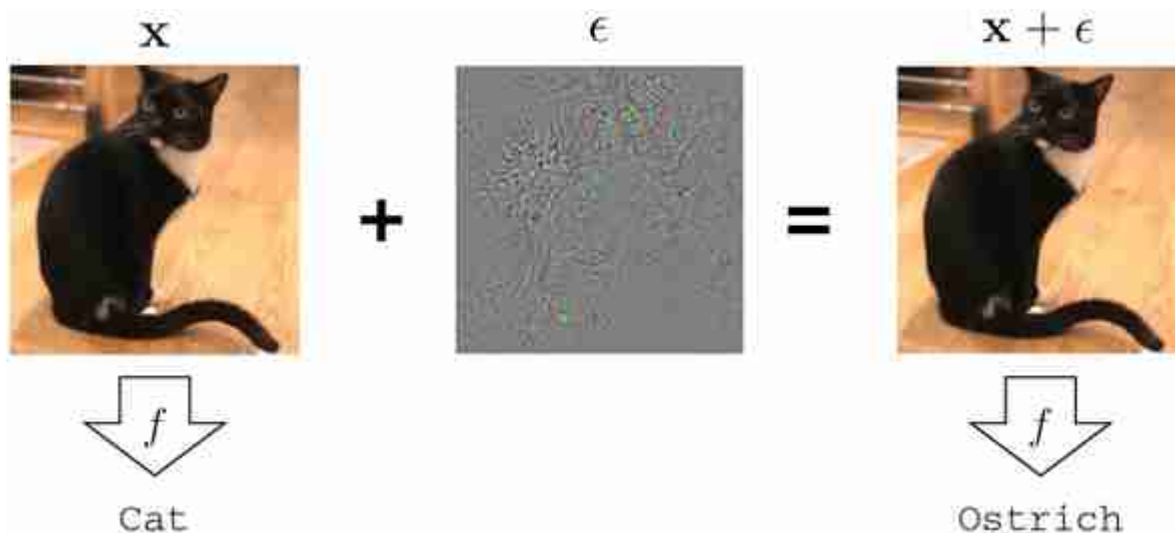


Figure 35.12: An adversarial attack that adds subtle changes to the cat photo to make a neural net classify it instead as an ostrich. The noise image color values are scaled 20x for visualization.

This type of attack required that the attacker has access to your system (the neural net f) so that they can optimize the noise pattern ϵ to fool the system. Unfortunately, if you want to deploy your system to the public, then you have to give people access to it and that makes it vulnerable to attack.

In chapter 12 we described a simple neural network for line classification and showed how, by accessing the architecture and the learned filters, we could manually design input examples that would get wrongly classified. Similarly, in chapter 21 we showed how aliasing could also create vulnerabilities in a network. For large systems, adversarial attacks are not manually designed, instead they are the result of an optimization procedure that optimizes the noise pattern ϵ so that it has a small amplitude (i.e., it is imperceptible) and makes the system fail with high confidence.

35.6.2 Defenses

Adversarial analysis is worst case analysis. Remember that we typically train our systems to minimize empirical risk, defined as the *mean* error over the training data. Because we only focus on the mean, the *worst case* might be arbitrarily bad. Alternative approaches try to minimize worst-case error, and this can result in systems that are more **adversarially robust**. In

chapter 36 we will discuss how **adversarial training** can be used to increase the robustness of the models to adversarial attacks.

35.7 Concluding Remarks

One way to reduce the issues of dataset bias and shift is to implement a rigorous data collection procedure that tries to minimize the biases present in the data. Although it is possible to prevent (or reduce) certain forms of bias, others will remain challenging because it is hard to collect the necessary data or because we are not aware of it. In many cases, deployment will be done in specific settings that might be unknown a priori. Therefore, bias is here to stay and it is important to study forms to reduce its methods in the system performance.

There are two general ways of dealing with dataset bias and shift. The first is called domain randomization, or data augmentation. The second is called transfer learning, or model adaptation. The next two chapters are about these options.

17. <http://www.reddit.com>

18. http://www.reddit.com/user/pm_me_your_bw_pics

36 Training for Robustness and Generality

36.1 Introduction

In the previous chapter, we saw that performance of a vision system can look quite good on a training set but fail dramatically when deployed in the real world, and the reason is often distribution shift. How can we train models that are more robust to these kinds of shifts?

This chapter presents several strategies that all have the same goal: broaden the training distribution so that the test data just looks like more of the training data. The following sections present two ways of doing this.

36.2 Data Augmentation

Data augmentation augments a dataset by adding random transformations of each datapoint. For example, below we show some common data augmentations applied to an example image ([figure 36.1](#)):



Figure 36.1: A few common types of data augmentation.

Data augmentation might not seem very glamorous, but it is one of the most important tools at our disposal. Machine learning is data-driven intelligence, and the bigger the data the better. Data augmentation is like a philosopher's stone of machine learning: it converts lead (small data) into gold (big data).

When you do this for every image in a dataset, you are essentially mapping from a smaller dataset to a larger dataset. We know that more data is usually better, so this is a way to simulate the effect of adding more real data to your dataset.



The way data augmentation works is that for each $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}$ pair in your original dataset, you add M new synthetic pairs $\{T(\mathbf{x}^{(i)}; \theta^{(j)}), \mathbf{y}^{(i)}\}_{j=1}^M$, where T is a data transformation function with stochastic parameters $\theta \sim p_\theta$. As an example, T might be the `crop` function, and then θ would be a set of coordinates specifying where to apply the crop. We would randomly select different coordinates for each of the M crops we generate.

Notice that we did not apply any transformation to the $\mathbf{y}^{(i)}$ values. The assumption we are making is that the target outputs $\mathbf{y}$ are **invariant** to the augmentations of the inputs $\mathbf{x}$. Let $y(\mathbf{x})$ be the true $\mathbf{y}$ for an input $\mathbf{x}$; then we are assuming that,

$$y(\mathbf{x}) = y(T(\mathbf{x}, \theta)) \quad \forall \theta \quad \Leftrightarrow \quad \mathbf{y} \text{ invariant to } T \quad (36.1)$$

For example, if our task is scene classification then we would want to only apply augmentations that do not affect the class label of the scene. We could use mirror augmentation, since the mirroring a scene does not change its class. In contrast, if the task were optical character recognition, then mirror augmentation would not be a good idea. That's because the mirror image of the character `b` looks just like the character `d`. Character recognition is not mirror invariant. In other words, for the task of character recognition, we do not have that $y(\text{mirror}(\mathbf{x})) = y(\mathbf{x})$. The same is true for molecular data that exhibits so-called *chiral* structure: a molecule may behave very differently from its mirror image.

To handle the case where $y(\mathbf{x}) \neq y(T(\mathbf{x}, \theta))$, we may use a more advanced kind of data augmentation, where we transform the $\mathbf{y}$ values at the same time as we transform the $\mathbf{x}$ values, using transforms T_X and T_Y . This results in an augmented dataset of the form $\{T_X(\mathbf{x}^{(i)}; \theta^{(j)}), T_Y(\mathbf{y}^{(i)}; \theta^{(j)})\}_{j=1}^M$. For this to make sense, we require that T_X and T_Y express an **equivariance** between the two domains, which means,

$$T_Y(y(x), \theta_Y) = y(T_X(x, \theta_X)) \quad \forall \theta \quad \Leftarrow \quad y \text{ equivariant with respect to } T_X, T_Y \quad (36.2)$$

This kind of data augmentation is standard in settings where $\mathbf{y}$ has spatial or temporal structure that is aligned with the structure in $\mathbf{x}$, like if $\mathbf{x}$ is an image and $\mathbf{y}$ is a label map as in the problem of semantic segmentation. Then, if we do random crop augmentation, we need to make sure to apply the same crops to both $\mathbf{x}$ and $\mathbf{y}$: $\{\text{crop}(\mathbf{x}^{(i)}; \theta^{(i)}), \text{crop}(\mathbf{y}^{(i)}; \theta^{(i)})\}_{i=1}^M$.

Sometimes our data is not sampled from a static *dataset* but instead is generated by a simulator. In this setting we may apply an idea analogous to data augmentation called **domain randomization**. Here, rather than adding random transformations to points in a dataset, we instead randomize the parameters of the simulator, so that each interaction with the simulation will look slightly different, for example, the lighting conditions may be randomized. The effect is the same as with data augmentation: you convert a narrow set of experiences (the simulation with one kind of lighting condition) to a much broader set of experiences to learn from (i.e., the simulation with many different lighting conditions).

Why does data augmentation work? It *broadens* the distribution of training data, and this can reduce the gap between the training data and the test data. This effect is illustrated in [figure 36.2](#). In this example, we have training image from one dataset (Caltech256 [[175](#)]) and test images from a different dataset (CIFAR100 [[278](#)]). The test images look quite a bit different from the training data; they are lower resolution and contain a different set of object categories. We say that the test data are **out-of-distribution** relative to the training data. Data augmentation adds variation to the training data, which broadens the training distribution. The result is that the test data become more in-distribution, compared to the training data. If we apply enough data augmentation, then the test data will look just like more random samples from the training distribution!

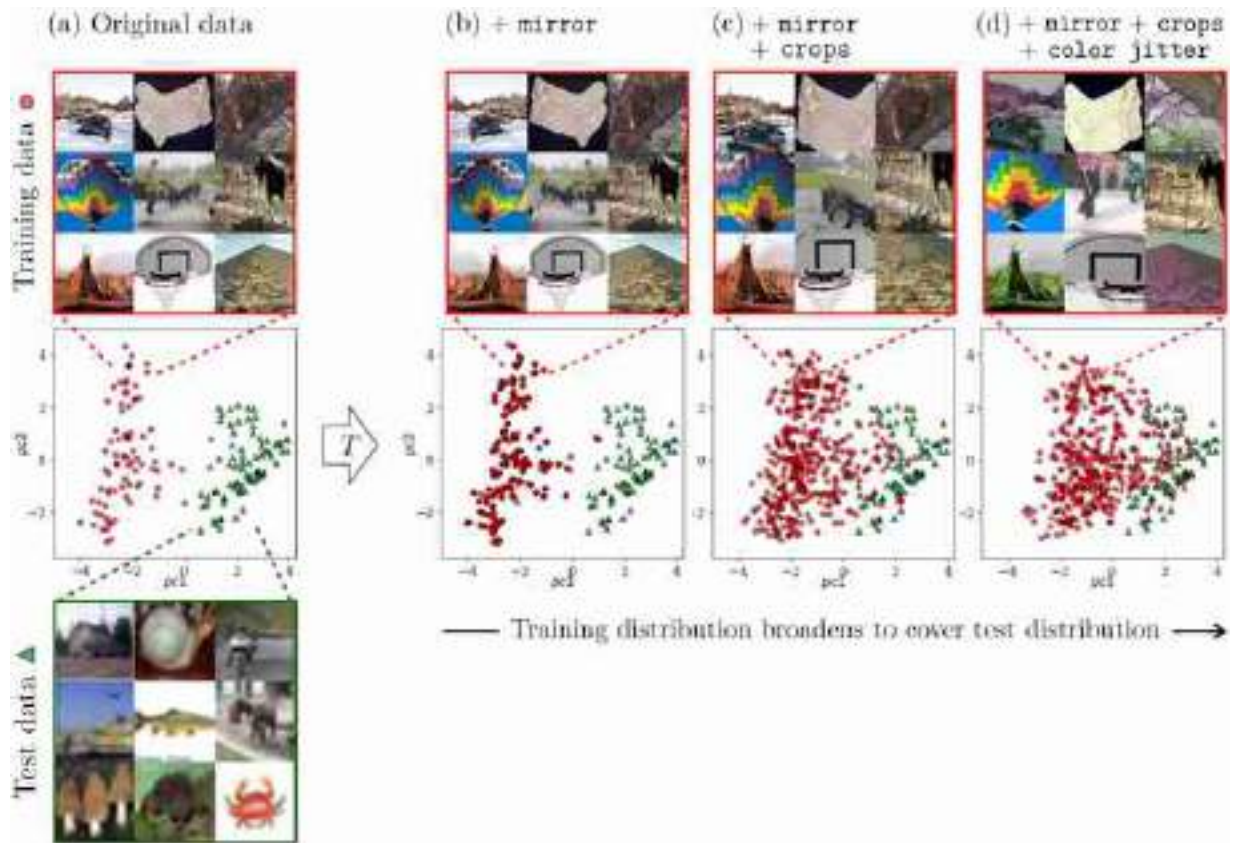


Figure 36.2: Data augmentation broadens the training distribution so that it might better cover the test cases. (a) Training data are from Caltech256 [175] and test data are from CIFAR100 [278]. The scatter plots show these data in a 2D feature space [the first two principle components (pcs) of CLIP [394]]. (b) Training data after `mirror` augmentations (random horizontal flips). (c) The same plus crops (crop then rescale to the original size). (d) The same plus `color jitter` (random shifts in color and contrast).

36.2.1 Learned Data Augmentation

Simple transformations like mirror flipping or cropping can be very effective while also being easy to implement by hand. After learning, the model will be invariant (or sometimes equivariant, if the y values are transformed too) with respect to these transformations. But these are just a few of all possible transformations that we might want to be invariant to. Imagine if we are building a pedestrian detector. Then we would probably like it to be invariant to pose variation in the pedestrian. This can be achieved by augmenting our data with random pose variations. But to do so may be very hard to code by hand. Instead we can use learning algorithms to *learn* how to augment our data and achieve the invariances we would like.

One way to do this is to use generative models. Latent variable generative models like generative adversarial networks (GANs) or variational autoencoders (VAEs) are especially suitable, because the latent variables can be used to control different factors of variation in the generated data (see chapter 33). Given a datapoint $\mathbf{x}$, an encoder $f_\psi : X \rightarrow Z$, and a generator $g_\theta : Z \rightarrow X$, we can create an augmented copy of $\mathbf{x}$ as,

$$\mathbf{x} = g_\theta(f_\psi(\mathbf{x}) + \mathbf{w}) \quad \triangleleft \text{generative data augmentation via latent manipulation} \quad (36.3)$$

where $\mathbf{w}$ is a small perturbation to the latent variable $\mathbf{z}$.

GANs do not necessarily come with encoders f_ψ , but for applications that need one, like we have here, you can simply train an encoder via supervised learning on pairs $\{g_\theta(\mathbf{z}), \mathbf{z}\}$, solving $\arg \min_\psi \mathbb{E}_z[L(f_\psi(g_\theta(\mathbf{z})), \mathbf{z})]$. See [107] for discussion of this method and more advanced techniques.

If we do this for a variety of settings of $\mathbf{w}$, then we will get a set of output images that are all slight variations of one another, just like we saw previously in figure 33.15. One simple approach is to just use Gaussian perturbations in latent space, that is, $\mathbf{w} \sim N(0, \sigma^2)$ for some perturbation scale given by σ . Algorithm 36.1 formalizes this approach (see [73] for a representative work that roughly follows this recipe, or [25] for an alternative approach to generative data augmentation).

Algorithm 36.1: Generative data augmentation (genaug).

```

1 Input: Generative model  $g_\theta : Z \rightarrow X$  and its (approximate) inverse  $f_\psi : X \rightarrow Z$ ,
  perturbation scale parameter  $\sigma$ , Dataset  $X \triangleq \{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$ ,
  hyperparameter  $k$ 
2 Output: Augmented dataset  $\tilde{X} \triangleq \{\tilde{\mathbf{x}}^{(i)}, \tilde{\mathbf{y}}^{(i)}\}_{i=1}^{kN}$ 
3 for  $i = 1, \dots, N$  do
4   for  $j = 1, \dots, k$  do
5      $\mathbf{z}^{(i)} \leftarrow f_\psi(\mathbf{x}^{(i)})$ 
6      $\mathbf{w}_j \sim \mathcal{N}(0, \sigma^2)$ 
7      $\tilde{X} \leftarrow \{g_\theta(\mathbf{z}^{(i)} + \mathbf{w}_j), \mathbf{y}^{(i)}\}$ 

```

There are some subtleties when working with learned data augmentation. You might be tempted to train g_θ on the same dataset you are trying to

augment, that is, train g_θ on X , then use g_θ to augment X . This can have interesting regularizing effects, but it introduces an uncomfortable circularity, which can be understood as follows. We want to create augmented data for training some function f . Let the learner for f be denoted as the function L , which is a mapping from input data to output learned function f .

It can be confusing to think of a learning *algorithm* as a *function*, but that's what it is; as we pointed out in chapter 9, a learner is a function that outputs a function.

Without data augmentation we have $f = L(X)$, and with data augmentation we have $f = L(\bar{X})$. But now, when g_θ is trained on X , then $\bar{X}$ is just a function of X ; in particular, if we use algorithm 36.1, it is $\bar{X} = \text{genaug}(X)$. So, we can define a new learner $\bar{L} = L \circ \text{genaug}$ where we have $f = \bar{L}(X) = L(\text{genaug}(X)) = L(\bar{X})$. This demonstrates that there exists a learning algorithm, $\bar{L}$, that uses unaugmented data and is just as good as our learning algorithm that used augmented data.

A clearer gain can be achieved by using a g_θ pretrained on external data. For example, if we have a general purpose image generative model, trained on millions of images (such as [397] or [409]), then we can use it to augment a small dataset for a new specific task we come across.

36.3 Adversarial Training

With data augmentation, the learning problem looks like this:

$$\arg \min_f \mathbb{E}_{\mathbf{x}, \mathbf{y}} [\mathbb{E}_{\theta \sim p_\theta} [\mathcal{L}(f(T(\mathbf{x}; \theta)), \mathbf{y})]] \quad (36.4)$$

Here we are applying random transformations to the data, and we want our learner to do well in expectation over these random transformations. Instead of doing random transformations, we could instead consider augmenting with *worst case* transformations, applying those transformations that incur the highest loss possible for a given training example:

$$\arg \min_f \mathbb{E}_{\mathbf{x}, \mathbf{y}} [\max_{\theta \in \Theta} [\mathcal{L}(f(T(\mathbf{x}; \theta)), \mathbf{y})]] \quad (36.5)$$

This idea is known as **adversarial training**. One of its effects is that it can increase the robustness of the learned function f to perturbations that may occur at test time (like those we saw in the previous chapter).

In the context of adversarial robustness, one form of equation (36.5) is to set T to be an epsilon-ball attack, that is, $T(\mathbf{x}) = \mathbf{x} + \epsilon$:

$$\arg \min_f \mathbb{E}_{\mathbf{x}, \mathbf{y}} [\max_{\epsilon: \|\epsilon\| \leq \epsilon} [\mathcal{L}(f(\mathbf{x} + \epsilon), \mathbf{y})]] \quad (36.6)$$

This can increase robustness against the kinds of epsilon-ball attacks we saw in the previous chapter.

Adversarial training has many other uses as well, beyond the scope of the current chapter: it is used for training generative adversarial networks (chapter 32), for training agents to explore in reinforcement learning [379], and for training agents that compete against other agents [438]. This general approach is also known as **robust optimization** in the optimization literature.

36.4 Toward General-Purpose Vision Models

The lesson from this chapter is that if you train on more and more data, you can achieve better coverage over all the test cases you might encounter. This raises a natural question: Rather than inventing new augmentation and robustness methods, why not just collect ever more data? This approach is becoming increasingly dominant, and a major focus of current efforts is on just scaling the training data size and diversity.

36.4.1 Multitask training

Because we have finite data for any given task, it can help to share data between tasks. Suppose we have two tasks, A and B . How can we borrow data from task B to increase the data available for task A ? In line with the theme of this chapter, one approach is to not just broaden the *data* but to also broaden the *task*. The idea is to define a new task, AB , for which tasks A and B are special cases. Then we can train AB on both the data for task A and for task B . An example of a metatask like AB is data imputation (see chapter 33). One example of data imputation is to predict missing color channels; this is the colorization problem, call this task A . Another data imputation problem is predicting a chunk of pixels masked out of the image; this is the inpainting problem, which we can call task B . We can train a joint colorization–inpainting model if we simply set up both as data

imputation and train a general data imputation model. This model can be trained on both colorization training data and inpainting training data.

Data imputation is an incredibly general framework. Can you think of other vision problems that can be formulated as data imputation? Or, here is a harder question: Can you think of any vision problems that cannot be formulated this way?

In the next chapter, we will see several more examples of how to make use of data from task B for task A .

36.5 Concluding Remarks

A grand challenge of computer vision is to make algorithms that work in *general*, no matter the setting in which you deploy them. For example, we would like object detectors that will recognize a cat on the internet, a cat in your house, a cat on Mars, an upside down cat, and so on. One of the simplest ways to achieve this is to train for generality: train your system on as much data, as diverse data, and as many tasks as possible. This approach is, perhaps, the main reason we now have computer vision systems that really work in practice, whereas a decade ago we didn't.

37 Transfer Learning and Adaptation

37.1 Introduction

A common criticism of deep learning systems is that they are data hungry. To learn a new concept (e.g., “is this a dog?”) may require showing a deep net thousands of labeled examples of that concept. Conversely, humans can learn to recognize a new kind of animal after seeing it just once, an ability known as **one-shot learning**.

Correspondingly, **few-shot learning** refers to learning from just a few examples, and the **zero-shot** setting refers to when a model works out of the box on a new problem, with zero adaptation whatsoever.

Humans can do this because we have extensive prior knowledge we bring to bear to accelerate the learning of new concepts. Deep nets are data hungry when they are trained from *scratch*. But they can actually be rather data efficient if we give them appropriate prior knowledge and the means to use their priors to accelerate learning. Transfer learning deals with how to use prior knowledge to solve new learning problems quickly.

Transfer learning is an alternative to the ideas we saw last chapter, where we simply trained on a broad distribution of data so that most test queries happen to be things we have encountered before. Because it is usually impossible to train on *all* queries we might encounter, we often need to rely on transfer learning to adapt to new kinds of queries.

37.2 Problem Setting

Suppose you are working at a farm and the drones that water the plants need to be able to recognize whether the crop is wheat or corn. Using the machinery we have learned so far, you know what to do: gather a big dataset of aerial photos of the crops and label each as either wheat or corn, then, train your favorite classifier to perform the mapping $f_{\theta} : X \rightarrow$

{wheat, corn}. It works! The crop yield is plentiful. Now a new season rolls around and it turns out the value of corn has plummeted. You decide to plant soybeans instead. So, you need a new classifier; what should you do?

One option is to train a new classifier from scratch, this time on images of either wheat or soybeans. But you already know how to recognize wheat; you still have last year's classifier f_θ . We would like to make use of f_θ to accelerate the learning of this year's new classifier. Transfer learning is the study of how to do this.

Transfer learning algorithms involve two parts:

1. **Pretraining**: how to acquire prior knowledge in the first place.
2. **Adaptation**: how to use prior knowledge to solve a new task.

In our example, the pretraining phase was training the wheat versus corn classifier, and the adaptation phase is updating that classifier to instead recognize wheat versus soybeans. This is visualized below in [figure 37.1](#).

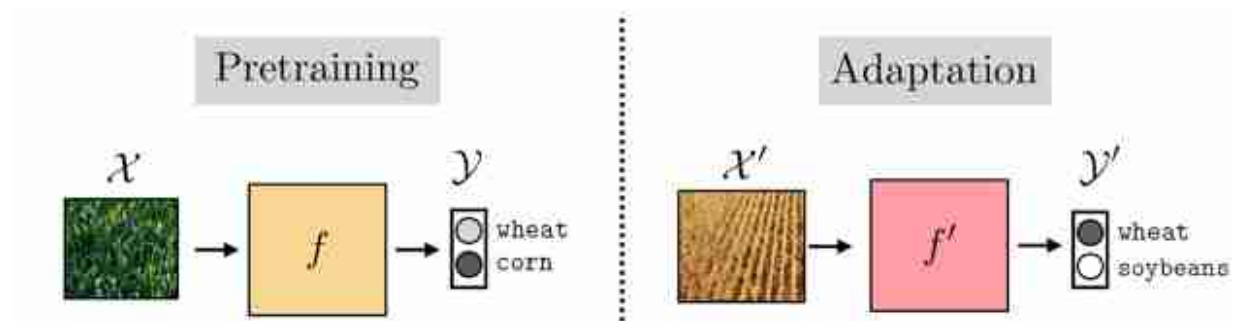


Figure 37.1: Transfer learning consists of two phases: first we pretrain a model on one task and then we adapt that model to perform a new task.

The pretraining phase could just be regular training, which we only call *pretraining* in retrospect, once we decide to start adapting to a new problem. Or we can specifically design pretraining algorithms that are *only* meant as a warm up for the real event. Representation learning algorithms (chapter 30) are of this latter variety.

In the next sections we will describe specific kinds of transfer learning algorithms, which handle each of these stages in different ways. Then, in section 37.9, we will reflect on the landscape of choices we have made, and we will see that transfer learning is better understood as a collection of

tools, which can be combined in innumerable ways, rather than as a set of specific named algorithms.

37.3 Finetuning

There are many ways to do transfer learning. We will start with perhaps the simplest: when you encounter a new task, just keep on learning *as if nothing happened*.

Finetuning consists of initializing a new classifier with the parameter vector from the pretrained model, and then training it as usual. In other words, finetuning is making small (fine) adjustments (tuning) to the parameters of your model to adapt it to do something slightly different than what it did before, or even something a lot different.

There are three stages: (1) pretrain $f: X \rightarrow Y$, (2) initialize $f' = f$, and (3) finetune $f': X' \rightarrow Y'$. The full algorithm is written in algorithm 37.1, with f and f' indicated as just a continually learning f_θ with different iterates of θ as learning progresses.

Finetuning is simple and works well. But there is one tricky bit we still need to deal with: What if the structure of f_θ is incompatible with the new problem we wish to finetune on?

For example, suppose f is a neural net and θ are the weights and biases of this net. Then the previous algorithm will only work if the dimensionality of X matches the dimensionality of X' , and the same for Y and Y' . This is because, for our neural net model, f_θ must have the same shape inputs and outputs regardless of the values θ takes on. What if we start with our net trained to classify between wheat and corn and now want to finetune it to classify between wheat, corn, and soybeans? The dimensionality of the output has changed from two classes to three.

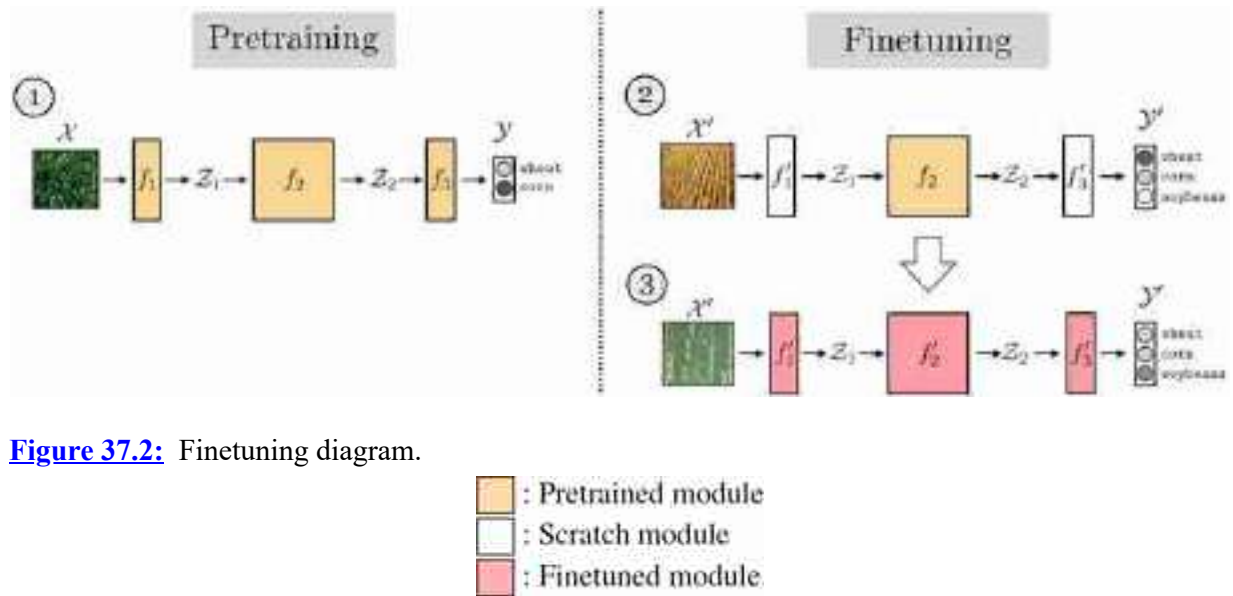
Algorithm 37.1: Finetuning. Using gradient descent to train one model and then finetuning to produce a second model.

```

1 Input: initial parameter vector  $\theta^0$ , data  $\{x^{(i)}, y^{(i)}\}_{i=1}^N$ ,  $\{x'^{(i)}, y'^{(i)}\}_{i=1}^M$ , learning
   rates  $\eta_1$  and  $\eta_2$ 
2 Output: trained models  $f_{\theta^N}$  and  $f_{\theta^{N+M}}$ 
3 Pretraining: for  $k = 1, \dots, K_1$  do
4    $J = \mathbb{E}_{x,y}[\mathcal{L}(f_{\theta^{k-1}}(x), y)]$ 
5    $\theta^k \leftarrow \theta^{k-1} - \eta_1 \nabla_{\theta} J$ 
6 Finetuning: for  $k = 1, \dots, K_2$  do
7    $J = \mathbb{E}_{x',y'}[\mathcal{L}(f_{\theta^{N+k-1}}(x'), y')]$ 
8    $\theta^{k+N} \leftarrow \theta^{k-1+N} - \eta_2 \nabla_{\theta} J$ 

```

To handle this it is common to cut off the last layer of the network and replace it with a new final layer that outputs the correct dimensionality. If the input dimensionality changes, you can do the same with the first layer of the network. Both these changes are shown next, in [figure 37.2](#).



To be precise, let $f: X \rightarrow Y$ decompose as $f = f_1 \circ f_2 \circ f_3$, with $f_1: X \rightarrow Z_1$, $f_2: Z_1 \rightarrow Z_2$, and $f_3: Z_2 \rightarrow Y$. Now we wish to learn a function $f': X' \rightarrow Y'$,

which decomposes as $f' = f'_1 \circ f'_2 \circ f'_3$, with $f'_1 : X \rightarrow Z_1$, $f'_2 : Z_1 \rightarrow Z_2$, and $f'_3 : Z_2 \rightarrow Y'$. The finetuning approach here would be to first learn f ; then, in order to learn f' , we initialize f'_2 with the parameters of f_2 and initialize f'_1 and f'_3 randomly (i.e., train these from scratch).

37.3.1 Finetuning Subparts of a Network

[Figure 37.2](#) shows that you don't have to finetune all layers in your network but can selectively finetune some and initialize others from scratch. In fact this strategy can be applied quite generally, where we initialize some parts of a computation graph from pretrained models and initialize others from scratch. There are many reasons why you might want to do this, or not. Sometimes you might have pretrained models that can perform just certain parts of what your full system needs to do. You can use these pretrained models and compose them together with layers initialized from scratch. Other times you may want to *freeze* certain layers to prevent them from forgetting their prior knowledge [\[522\]](#).

An example of both these use cases is given in [figure 37.3](#). In this example we imagine we are making a system to detect cars in street images. We have four modules that work together to achieve this. The `featurize` module converts the input image `x` into a feature map (for example, this could be DINO [\[70\]](#), which we saw in section 31.3.2). This module is pretrained and frozen; the idea is we have a good generic feature extractor that will work for many problems, and we do not need to update it for each new problem we encounter. Next we have two modules that take these feature maps as input and extract different kinds of useful information from them: (1) `depth_estimator` predicts the depth map of the image (geometry), (2) `dense_classifier` predicts a class label per-pixel (semantics). In our example, these two models are pretrained on generic images but we need to finetune them to work on the kind of street images we will be applying our detector to. Finally, given the depth and class predictions, we have a final module, `detector`, that searches for the image region that has the geometry and semantics of a car. This module outputs a bounding box `y` indicating where it thinks the car is.

We will use the symbol  to indicate a frozen module and the symbol  to indicate a module that is updated during the adaptation phase.

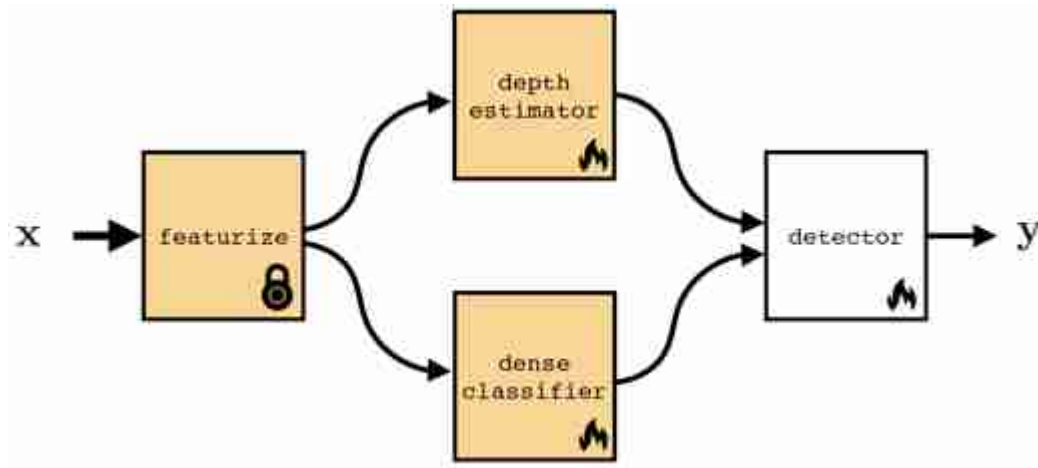


Figure 37.3: Finetuning certain subsets of a computation graph. Here we show some modules that have been pretrained (those with a shaded background) and one that is trained from scratch at finetuning time.

Within a single layer, it is also possible to update just a subspace of the full weight matrix. This can be an effective way to save memory (fewer weight gradients to track during backpropagation) or to regularize the amount of adaptation you are performing. One popular way to do so is to apply a low-rank update to the weight matrix [224].

37.3.2 Heads and Probes

As discussed previously, a model can be adapted by changing some or all of its parameters, or by adding entirely new modules to the computation graph. One important kind of new module is a **read out** module that takes some features from the computation graph as input and produces a new prediction as output. When these modules are linear, they are called **linear probes**. These modules are relatively lightweight (few parameters) and can be optimized with tools from linear optimization (where there are often closed form solutions). Because of this, linear probes are very popular as a way of repurposing a neural net to perform a new task. These modules are also useful as a way of assessing what kind of knowledge each feature map in

the original network represents, in the sense that if a feature map can linearly classify some attribute of the data, then that feature maps knows something about that attribute. In this usage, linear probes are probing the knowledge represented in some layer of a neural net.

37.4 Learning from a Teacher

When we first encountered supervised learning in chapter 9, we described it as *learning from examples*. You are given a dataset of example inputs and their corresponding outputs and the task is to infer the relationship that can explain these input-output pairs:

$$\{x^{(i)}, y^{(i)}\}_{i=1}^N \rightarrow \boxed{\text{Learning from examples}} \rightarrow f_\theta$$

This kind of learning is like having a very lazy teacher, who just gives you the answer key to the homework questions but does not provide any explanations and won't answer questions. Taking that analogy forward, could we learn instead from a more instructive teacher?

One way of formalizing this setting is to consider the teacher as a well-trained model $t : X \rightarrow Y$ that maps input queries to output answers. The student (the learner) observes both data $\{x^{(i)}\}_{i=1}^N$ and has access to the teacher model t . The student can learn to imitate the teacher's outputs or can learn to match *how the teacher processes the queries*, matching intermediate activations of the teacher model.

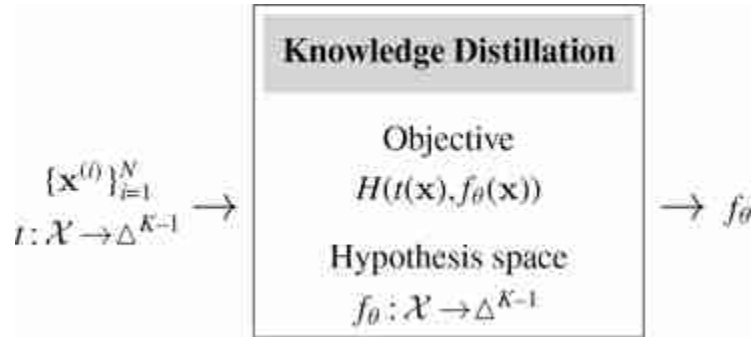
$$\{x^{(i)}\}_{i=1}^N, t : \mathcal{X} \rightarrow \mathcal{Y} \rightarrow \boxed{\text{Learning from a teacher}} \rightarrow f_\theta$$

Learning from examples can be considered a special case of learning from a teacher. It is the case where the teacher is the world and our only access to the teacher is observing outputs from it, that is, the examples y that match each input x .

37.4.1 Knowledge distillation

Knowledge distillation is a simple and popular algorithm for learning a classifier from a teacher. We assume that the teacher is a well-trained classifier that outputs a $K - 1$ -dimensional probability mass function (pmf),

$t : X \rightarrow \Delta^{K-1}$ (see section 9.7.3). The student, a model $f_\theta : X \rightarrow \Delta^{K-1}$, simply tries to imitate the teacher's output pmf: for each input x that the student sees, its goal is to output the same pmf vector as the teacher outputs for x . The full algorithm is given in the diagram below.



This section describes knowledge distillation as it was defined in [205], which is the paper that introduced the term. Note that the term is now often used to refer to the whole family of algorithms that have extended this original method. See [171] for a survey.

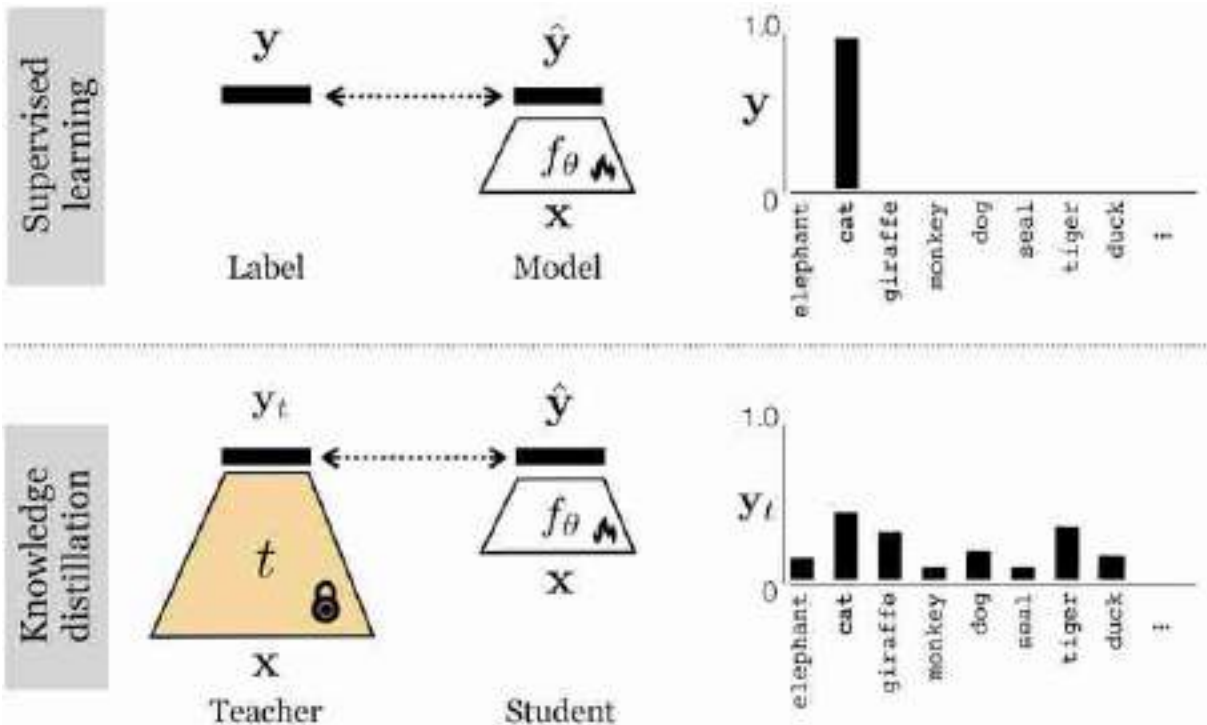


Figure 37.4: Comparing knowledge distillation to supervised learning from labels.

Notice that this learning problem is very similar to softmax regression on labeled data, and a diagram comparing the two is given in [figure 37.4](#). The difference is that in knowledge distillation the targets are not one-hot (like in standard label prediction problems) but rather are more *graded*. The teacher outputs a probability vector that reflect the teacher's belief as to the class of the input, rather than the ground truth class of the input. These beliefs may assign partial credit to classes that are similar to the ground truth, and this can actually make the teacher *more* informative than learning from ground truth labels.

Think of it this way: the ground truth label for a domestic cat is “domestic cat” but the pmf vector output by a teacher, looking at a domestic cat, is more like “this image looks 90 percent like a domestic cat, but also 10 percent like a tiger.” That would be rather informative if the domestic cat in the image is a particularly large and fearsome looking one. The student not only gets to learn that this image is showing a domestic cat but also that this particular domestic cat is tiger-like. That tells the student something about both the class “domestic cat” and the class “tiger.” In this way the teacher can provide more information about the image than a one-hot label

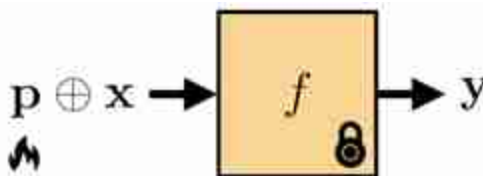
would, and in some cases this extra information can accelerate student learning [206].

37.5 Prompting

Prompting edits the *input data* to solve a new problem. Prompting is very popular in natural language processing (NLP), where text-to-text systems can have natural language prompts prepended to the string that is input to the model. For example, a general-purpose language model can be prompted with [Make sure to answer in French, X], for some arbitrary input X. Then the output will be a French translation of whatever the model would have responded to when queried with X.

The same can be done for vision systems. First, prompting methods from NLP can be directly used for prompting the text inputs to vision-language models [394] (section 51.3). Second, prompting can also be applied to visual inputs. Just like we can prepend a textual input with text, we can prepend a visual input with visual data, yielding a method that may be called **visual prompting** [28, 33, 237]. This can be done by concatenating pixels, tokens, or other visual formats to a vision model's inputs. This requires a model that takes variable-sized inputs; luckily this includes most neural net architectures, such as convolutional neural nets (CNNs), recurrent neural nets (RNNs), and transformers. Or, we can combine the visual prompt with the input query in other ways, such as by addition.

Prompting is visualized in [figure 37.5](#), where $\oplus$ represents concatenation, addition, or potentially other ways of mixing prompts and input signals.



[Figure 37.5](#): Prompting: a prompt p is combined with the input x to change the output y .

Let's now look at several concrete ways of prompting a vision system. One simple way is to directly edit the input pixels, and this approach is

shown in [figure 37.6](#):

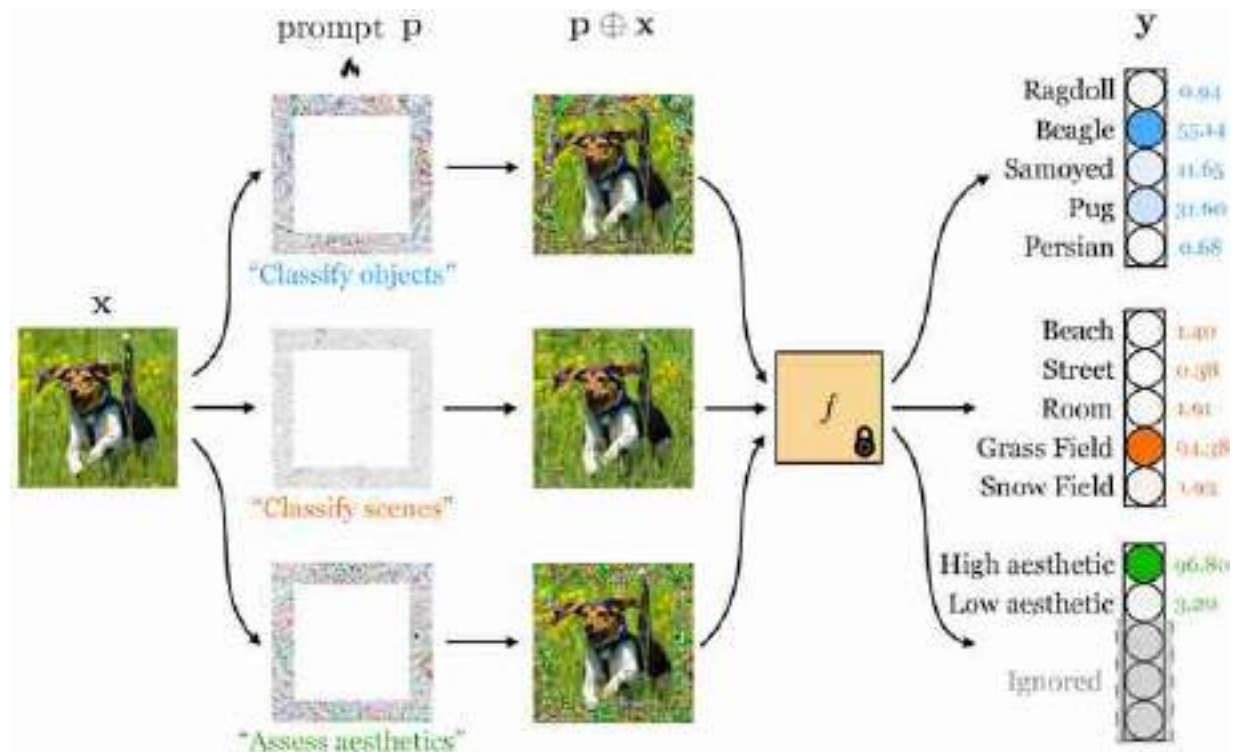


Figure 37.6: A prompt can be made out of pixels: a prompt image (which looks like a border of noise here) is added to the input image in order to change the model's behavior. Three different prompts are shown, one that adapts the model to perform scene classification, a second for aesthetics classification, and a third for object classification. Modified from [28].

Here, the prompt is an image p that is added to the input image x . The prompt looks like noise, but it is actually an optimized signal that changes the model's behavior. The top prompt was optimized to make the model predict the dog's species and the bottom prompt was optimized to make the model predict the dog's expression. Mathematically, this kind of prompting is very similar to the adversarial noise we covered in section 35.6.1.

Because of its similarity to adversarial examples, early papers on prompting sometimes referred to it as **adversarial reprogramming** [117].

However, here we are not adding noise that *hurts* performance but instead adding noise that *improves* performance on a new task of interest. Given a set of M training examples that we would like to adapt to, $\{x^{(i)}, y^{(i)}\}_{i=1}^M$, we learn a prompt via the following optimization problem:

$$\arg \min_{\mathbf{p}} \sum_{i=1}^M \mathcal{L}(f(\mathbf{x}^{(i)} + \mathbf{p}), \mathbf{y}^{(i)}) \quad (37.1)$$

where L is the loss function for our problem.¹⁹

Rather than adding a prompt in pixel space, it is also possible to add a prompt to an intermediate feature space. Let's look next at one way to do this. Jia, Tang, *et al.* [237] proposed to use a *token* as the prompt for a vision transformer architecture [109] (see chapter 26 for details on transformers). This new token is concatenated to the input of the transformer, which is a set of tokens representing image patches ([figure 37.7](#)):

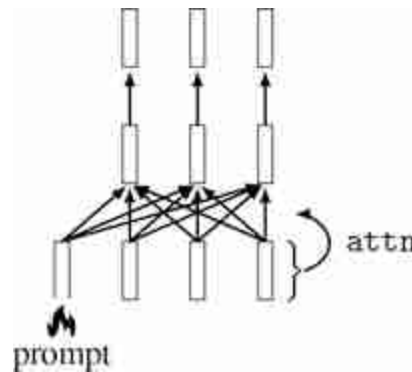


Figure 37.7: Prompting a transformer with a learnable input token. The prompt token is mixed into the network via attention; therefore *no* parameters of the original network have to be updated (see chapter 26).

The token is a d -dimensional code vector and this vector is optimized to change the behavior of the transformer to achieve a desired result, such as increased performance on a new dataset or task. The attention mechanism works as usual except now all the input tokens also attend to the prompt. The prompt thereby modulates the transformation applied to the input tokens on the first layer, and this is how it affects the transformer's behavior.

It is also possible to train a model to be *promptable*, so that at test time it adapts better to user prompts. An advantage of this approach is that we can train a model to understand human-interpretable prompts. For example, we could train the model so that its behavior can be adapted based on text instructions or based on user-drawn strokes (a representative example of both of these use cases can be found in [264]). In fact, we have already seen

many such systems in this book: any system that interactively adapts to user inputs can be considered a promptable system. In particular, many promptable systems use the tools of conditional generative modeling (chapter 34) to train systems can be conditioned on user text instructions (e.g., [398], [409], which we cover in chapter 51).

37.6 Domain Adaptation

Often, the data we train on comes from a different distribution, or domain, than the data we will encounter at test time. For example, maybe we have trained a recognition system on internet photos and now we want to deploy the system on a self-driving car. The imagery the car sees looks very different than everyday photos you might find on the internet. **Domain adaptation** refers to methods for adapting the training domain to be more like the test domain, or vice versa.

The first option is to adapt the test data, $\{\mathbf{x}_{\text{test}}^{(i)}, \mathbf{y}_{\text{test}}^{(i)}\}_{i=1}^N$ to look more like the training data. Then f should work well on both the training data and the test data. Or we can go the other way around, adapting the training data to look like the test data, *before* training f on the adapted training data. In either case, the trick is to make the distribution of test and train data identical, so that a method that works on one will work just as well on the other.

Commonly, we only have access to labels at training time, but may have plentiful unlabeled inputs $\{\mathbf{x}'_i\}_{i=1}^N$ at testing time. This setting is sometimes referred to as **unsupervised domain adaptation**. Here we cannot make use of test labels but we can still align the test inputs to look like the training inputs (or vice versa). One way to do so is to use a generative adversarial network (GAN; chapter 32) that translates the data in one domain to look identically distributed as the data in the other domain, as has been explored in e.g., [480, 531, 211]. A simple version of this algorithm is given in algorithm 37.2.

You may notice that domain adaptation is very similar to prompting. Both edit the model's inputs to make the model perform better. However, while prompting's objective is to improve or steer model performance, domain adaptation's objective is to make the inputs look more familiar. This means domain adaptation is applicable even when we don't have access to the model or even know what the task will be.

Algorithm 37.2: Unpaired domain adaptation via adversarial translation. Here we use a GAN to translate from one domain to the other. This algorithm could be improved by adding cycle-consistency constraints (see CycleGAN from chapter 34, [531]) as was done in [211].

- ¹ **Input:** training data $\{x_{\text{train}}^{(i)}, y_{\text{train}}^{(i)}\}_{i=1}^N$, test data, $\{x_{\text{test}}^{(j)}\}_{j=1}^M$
 - ² **Output:** trained model F
 - ³ **Train predictor on train:** $f = \arg \min_f \mathbb{E}_{x_{\text{train}}, y_{\text{train}}} [\mathcal{L}(f(x_{\text{train}}), y_{\text{train}})]$
 - ⁴ **Train translator X_{test} to X_{train} :**
 $G = \arg \min_g \max_d \mathbb{E}_{x_{\text{train}}} [\log d(x_{\text{train}})] + \mathbb{E}_{x_{\text{test}}} [\log(1 - d(g(x_{\text{test}})))]$
 - ⁵ return $F = f \circ g$
-

37.7 Generative Data

Generative models (chapter 32) produce synthetic data that looks like real data. Could this ability be useful for transfer learning? At first glance it may seem like the answer is no: Why use fake data if you have access to the real thing?

However, **generative data** (i.e., synthetic data sampled from a generative model) is not just a cheap copy of real data. It can be *more* data, and it can be *better* data. It can be *more* data because we can draw an unbounded number of samples from a generative model. These samples are not just copies of the training datapoints but can interpolate and remix the training data in novel ways (chapter 32). Generative models can also produce *better* data because the data comes coupled with a procedure that generated it, and this procedure has structure that can be exploited to understand and manipulate the data distribution.

One such structure is latent variables (chapter 33). Suppose you have generated a training set of outdoor images with a latent variable generative model. You want to train an object classifier on this data. You notice that the images never show a cow by a beach, but you happen to live in northern California where the cows love to hang out on the coastal bluffs. How can you adapt your training data to be useful for detecting these particular cows? Easy: just edit the latent variables of your model to make it produce images of cows on beaches. If the model is pretrained to produce all kinds

of images, it may have the ability to generate cows on beaches with just a little tweaking.

It is currently a popular area of research to use large, pretrained generative models as data sources for many applications like this [234, 197, 62]. In this usage, you are transferring knowledge about *data*, to accelerate learning to solve a new problem. The function being learned can be trained from scratch, but it is leveraging the generative model's knowledge about what images look like and how the world works.

37.8 Other Kinds of Knowledge that Can Be Transferred

This chapter has covered a few different kinds of knowledge that can be pretrained and transferred. We have not aimed to be exhaustive. As an exercise, you can try the following: (1) download the code for your favorite learning algorithm, (2) pick a random line, (3) and figure out how you can pretrain whatever is being computed on that line. Maybe you picked the line that defines the loss function; that can be pretrained! The term for this is a **learned loss** (e.g., [223]). Or maybe you picked the line `optim.SGD...` that defines the optimizer: the optimizer can also be pretrained. This is called a **learned optimizer** (e.g., [16]). It's currently less common to *adapt* modules like these (they are usually just pretrained and then held fixed), but in principle there is no reason why you can't. Maybe you, reader, can figure out a good way to do so.

37.9 A Combinatorial Catalog of Transfer Learning Methods

Many of the methods we introduced previously follow a similar format:

- Pretrain one component of the system.
- Adapt that component and/or other components.

In [table 37.1](#), we present the relationship these methods by framing them all as empirical risk minimization with different components optimized during pretraining and/or during adaptation.

Table 37.1

Different kinds of transfer learning presented as empirical risk minimization (ERM). *Notes:* The $\oplus$ symbol indicates concatenation or addition (or other kinds of signal combination). Domain adaptation does not map neatly to ERM so we omit it here.

Method	Learning pretrained / adapted / both	Inference
Generative data	$\arg \min_{\theta} \mathbb{E}_{x,y}[L(f_{\theta}(x), y)]$	$f_{\theta}(x)$
Distillation	$\arg \min_{\theta} \mathbb{E}_x[L(f_{\theta}(x), t(x))]$	$f_{\theta}(x)$
Prompting	$\arg \min_{\epsilon} \mathbb{E}_{x,y}[L(f_{\theta}(x \oplus \epsilon), y)]$	$f_{\theta}(x \oplus \epsilon)$
Finetuning	$\arg \min_{\theta} \mathbb{E}_{x,y}[L(f_{\theta}(x), y)]$	$f_{\theta}(x)$
Probes / heads	$\arg \min_{\phi} \mathbb{E}_{x,y}[L(h_{\phi}(f_{\theta}(x)), y)]$	$h_{\phi}(f_{\theta}(x))$
Learned loss	$\arg \min_{\theta} \mathbb{E}_{x,y}[L(f_{\theta}(x), y)]$	$f_{\theta}(x)$
Learned optimizer	$\arg \min_{\theta} \mathbb{E}_{x,y}[L(f_{\theta}(x), y)]$	$f_{\theta}(x)$

37.10 Sequence Models from the Lens of Adaptation

There is one more important class of adaptation algorithms: any model of a sequence that updates its behavior based on the items in the sequence it has already seen. We can consider such models as *adapting* to the earlier items in the sequence. CNNs, transformers, and RNNs all have this ability, among many other models, because they can all process sequences and model dependences between earlier items in the sequence and later decisions. However, the ability of sequence models to handle adaptation is clearest with RNNs, which we will focus on in this section.

RNNs update their hidden states as they process a sequence of data. These changes in hidden state change the behavior of the mapping from input to output. Therefore, RNNs *adapt* as they process data. Now consider that learning is the problem of adapting future behavior as a function of past data. RNNs can be run on sequences of frames in a movie or on sequences of pixels in an image. They can also be run on sequences of training points in a training dataset! If you do that, then the RNN is doing *learning*, and learning such an RNN can be considered metalearning. For example, let's see how we can make an RNN do supervised learning. Supervised learning

is the problem of taking a set of examples $\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^N$ and inferring a function $f: \mathbf{x} \rightarrow \mathbf{y}$. Without loss of generality, we can define an ordering over the input set, so the learning problem is to take the sequence $\{\mathbf{x}^{(1)}, \mathbf{y}^{(1)}, \dots, \mathbf{x}^{(n)}, \mathbf{y}^{(n)}\}$ as input and produce f as output.

The goal of supervised learning is that after training on the training data, we should do well on any random test query. After processing the training sequence $\{\mathbf{x}^{(1)}, \mathbf{y}^{(1)}, \dots, \mathbf{x}^{(n)}, \mathbf{y}^{(n)}\}$, an RNN will have arrived at some setting of its hidden state. Now if we feed the RNN a new query $\mathbf{x}^{(n+1)}$, the mapping it applies, which produces an output $\mathbf{y}^{(n+1)}$, can be considered the learned function f . This function is defined by the parameters of the RNN along with its hidden state that was learned from the training sequence. Since we can apply this f to any arbitrary query $\mathbf{x}^{(n+1)}$, what we have is a learned function that operates just like a function learned by any other learning algorithm; it's just that in this case the learning algorithm was an RNN.

Another name for this kind of learning, which is emergent in a sequence modeling problem, is **in-context learning**.

37.11 Concluding Remarks

Transfer learning and adaptation are becoming increasingly important as we move to ever bigger models that have been pretrained on ever bigger datasets. Until we have a truly universal model, which can solve any problem zero-shot, adaptation will remain an important topic of study.

[19.](#) Here we require that the output space Y is the same between the pretrained model and the prompted model. If we want to adapt to perform a task with a different output dimensionality then we can add a learned projection layer f' after the pretrained f , just like we did in [figure 37.2](#).

XI

UNDERSTANDING GEOMETRY

Let's face it, the world is three-dimensional (3D), and vision systems capture only two-dimensional (2D) images. Understanding the projection of the 3D world into 2D images, and how to reverse this projection to recover the 3D structure of the scene, is one of the most important topics in the study of vision (both natural vision and computer vision). Geometry is therefore a fundamental tool in computer vision.

This collection of chapters will cover many aspects of 3D vision:

- **Chapter 38** introduces homogeneous and heterogeneous coordinate systems and how to use them to model geometric transformations.
- **Chapter 39** describes camera models (intrinsic and extrinsic camera parameters) and camera calibration.
- **Chapter 40** describes how to recover 3D information from stereo images.
- **Chapter 41** describes homographies and the application to build image panoramas.
- **Chapter 42** describes how to recover 3D information using only a single image.

The material in this set of chapters will come handy as soon as you have to deal with 3D scenes (i.e., always).

38 Representing Images and Geometry

38.1 Introduction

Before we dive into the material of this chapter, let's start by questioning the way in which we have been representing images up to now. In most of the chapters, we have represented images as ordered arrays of pixel values, each pixel described by its grayscale value (or three arrays for color images). We will call this representation the **ordered array**.

$$\ell = \begin{bmatrix} \ell[1,1] & \dots & \dots \\ \vdots & \ell[n,m] & \vdots \\ \dots & \dots & \ell[N,M] \end{bmatrix} \quad (38.1)$$

Each value is a sample on a regular spatial grid. In this notation s represents the pixel intensity at one location. This is the representation we are most used to and the typical representation used when taking a signal processing perspective. It makes it simpler to describe operations such as convolution.

However, an image can be represented in different ways, making explicit certain aspects of the information present on the input. What else could we do to represent an image? Another very different, but equivalent, representation is to encode an image as a collection of points, indicating its color and location explicitly. In this case, an image is represented as a **set of pixels**:

$$\{[\ell_i, x_i, y_i] : i\} \quad (38.2)$$

where ℓ_i is the pixel intensity (or color) recorded at location (x_i, y_i) . This representation makes the geometry explicit. It might seem like a trivial representation but it makes some operations easy (and others complex). For instance, we can apply geometric transformations easily by directly working with the spatial coordinates. Imagine you want to translate an image, described by equation (38.2), to the right by one pixel, you can do that easily by simply creating the new image defined by the set of translated pixels: $\{[\ell_i, x_i + 1, y_i] : i\}$.

Representing images as sets of points is very common in geometry-based computer vision. We will use this representation extensively in the following chapters.

Although both previous representations might seem equivalent, the set representation makes it easy to deal with other image geometries where the points are not on a regular array.

That representation can also be extended by representing geometry in different ways such as using homogeneous coordinates, or positional encoding. We will discuss homogeneous coordinates in this chapter.

The previous two representations are not the only ways in which we can represent images. Another option is to represent an image as a continuous function whose input is a location (x, y) and its output is an intensity, or a color, ℓ :

$$\ell(x, y) = f_{\theta}(x, y) \quad (38.3)$$

This image representation is commonly used when we want to make image priors more explicit. The function f_{θ} is parameterized by the coefficients θ . This function becomes especially interesting when the parameters θ are different than the original pixel values. They will have to be estimated from the original image. But once learned, the function f should be able to take as input continuous spatial variables.

These three representations induce different ways of thinking about architectures to process the visual input. These representations are not opposed and can be used simultaneously.

The ordered array of pixels is the format that computers take as input when visualizing images. However, a set of pixels is the most common format when thinking about three-dimensional (3D) geometry and image formation. Therefore, it is always important to be familiar with how to transform any representation into an ordered array.

We will start by introducing homogeneous coordinates, an important tool that will simplify the formulation of perspective projection.

38.2 Homogeneous and Heterogeneous Coordinates

Homogeneous coordinates represent a vector of length N by a vector of length $N + 1$. The transformation rule from heterogeneous to homogeneous

coordinates is simply to add an additional coordinate, 1, to the heterogeneous coordinates as shown here:

$$\begin{pmatrix} x \\ y \end{pmatrix} \rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (38.4)$$

It is the same if the vector has three dimensions:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad (38.5)$$

We will refer to conventional Cartesian coordinate descriptions of a point in 2D, such as (x, y) , or 3D, such as (x, y, z) , as **heterogeneous coordinates** and write with rounded brackets in this chapter. We denote their corresponding **homogeneous coordinate** representations by square bracketed vectors.

August Ferdinand Möbius, a German mathematician and astronomer, introduced homogeneous coordinates in 1827. He also cocreated the famous Möbius strip.

The homogeneous coordinates have the additional rule that all (non-zero) scalings of a homogeneous coordinate vector are equivalent. For example, to represent a 2D point, we have (transforming from heterogeneous to homogeneous coordinates),

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \lambda x \\ \lambda y \\ \lambda \end{bmatrix} \quad (38.6)$$

for any non-zero scalar, λ . To go from homogeneous coordinates back to the heterogeneous representation of the point, we divide all the entries of the homogeneous coordinate vector by their last dimension:

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \rightarrow \begin{pmatrix} x/w \\ y/w \end{pmatrix} \quad (38.7)$$

[Figure 38.1](#) illustrates how homogeneous and heterogeneous coordinates relate to each other geometrically. A point in homogeneous coordinates is scale invariant (any point within the line that passes through the origin translates into the same point in heterogeneous coordinates). We can already

see that this is closely related to the operation performed by perspective projection.

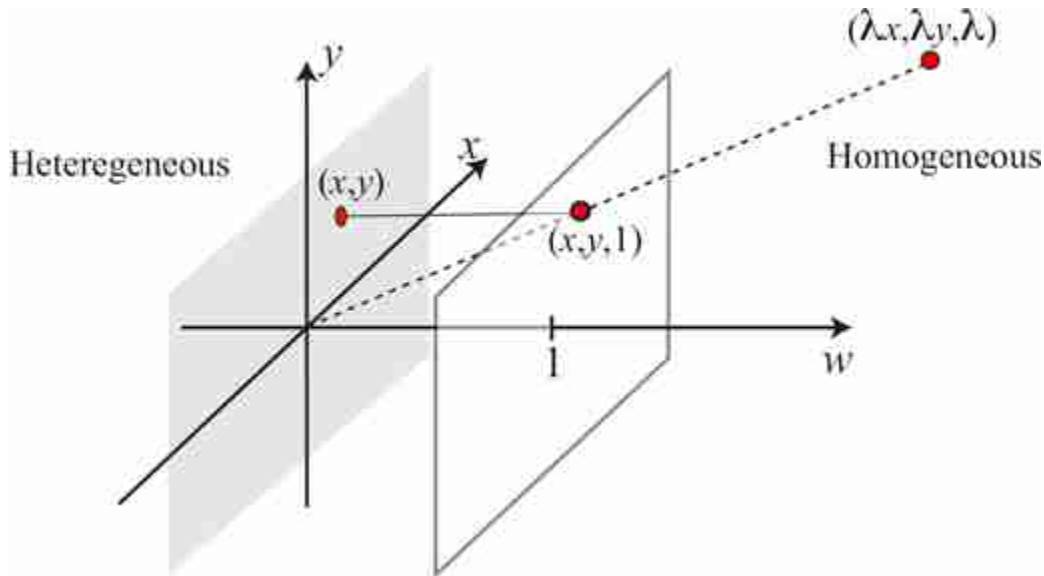


Figure 38.1: Transformation rule between heterogeneous and homogeneous coordinates in 2D. All the points along the dotted line correspond to the same point in heterogeneous coordinates. Note that the points (x, y) and $(x, y, 1)$ live in different spaces.

It is important to mention that while you can add two points in heterogeneous coordinates, you can not do the same in homogeneous coordinates!

Using homogeneous coordinates, we can place a point at infinity by setting $w = 0$. This is called the **ideal point**.

38.3 2D Image Transformations

One important operation in computer vision (and in computer graphics) is geometric transformations of shapes. Some of the common transformations we'll want to represent include translation, rotation, scaling, and shearing (see [figure 38.2](#)). These transformations can be written as affine transformations of the coordinate system. The mathematical description of these transformations becomes surprisingly simple when using homogeneous coordinates, and they will become the basis for more

complex operations such as 3D perspective projection and camera calibration as we will see in later sections.

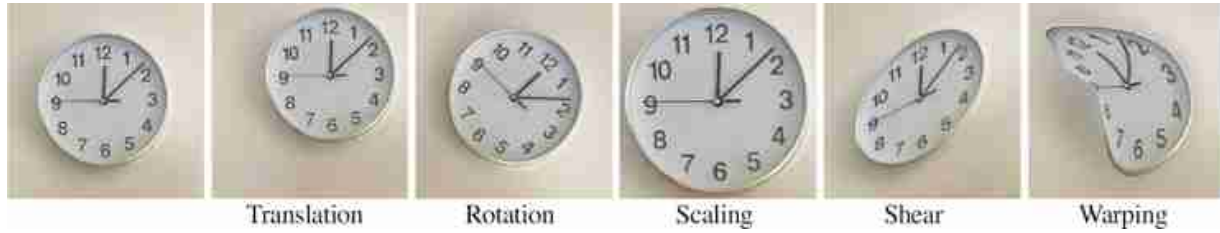


Figure 38.2: 2D geometric transformations. The final transformation is an arbitrary warping with a more complex deformation field.

We'll explore the use of homogeneous coordinates for describing 2D geometric transformations first, then see how they can easily represent 3D perspective projection.

38.3.1 Translation

Consider a translation by a 2D vector, (t_x, t_y) as shown in [figure 38.3](#). We'll denote the coordinates after the transformation with a prime. In heterogeneous coordinates, we have

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} \quad (38.8)$$

We can write this translation in homogeneous coordinates by the product:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (38.9)$$

What is interesting is that we have transformed an addition into a product by a matrix, $\mathbf{T}$, and a vector, $\mathbf{p}$, in homogeneous coordinates. Rewriting the equation above as:

$$\mathbf{p}' = \mathbf{T}\mathbf{p}, \quad (38.10)$$

with the homogeneous matrix, $\mathbf{T}$, defined as

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad (38.11)$$

For calculations in homogeneous coordinates, both matrices and vectors are only defined up to an arbitrary (non-zero) multiplicative scaling factor.

To cascade two translation operations $\mathbf{t}$ and $\mathbf{s}$, in heterogeneous coordinates, we have

$$\mathbf{p}' = \mathbf{p} + \mathbf{t} + \mathbf{s} \quad (38.12)$$

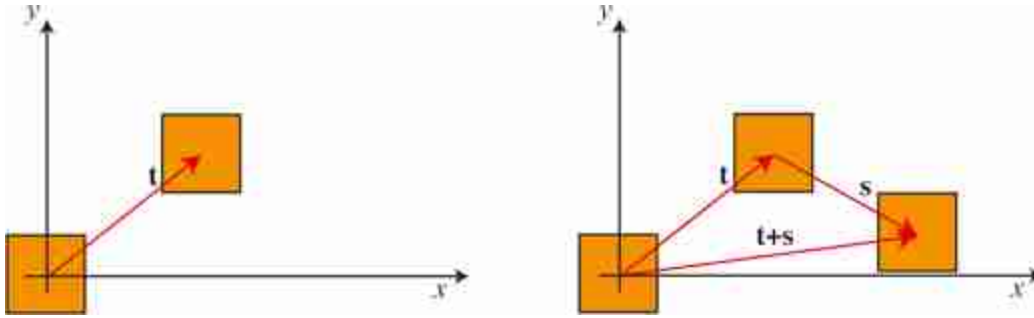


Figure 38.3: (left) Translation. (right) Composition of two consecutive translations.

In homogeneous coordinates, we cascade the corresponding translation matrices:

$$\mathbf{p}' = \mathbf{S}\mathbf{T}\mathbf{p}, \quad (38.13)$$

where the homogeneous translation matrix, $\mathbf{S}$, corresponding to the offset $\mathbf{s}$, is:

$$\mathbf{S} = \begin{bmatrix} 1 & 0 & s_x \\ 0 & 1 & s_y \\ 0 & 0 & 1 \end{bmatrix} \quad (38.14)$$

You can check that:

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & s_x \\ 0 & 1 & s_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x + s_x \\ 0 & 1 & t_y + s_y \\ 0 & 0 & 1 \end{bmatrix} \quad (38.15)$$

In summary, in homogeneous coordinates a translation becomes a product with a matrix, and chaining translations can be done by multiplying the matrices together. The benefits of using the homogeneous coordinates will become more obvious later.

38.3.2 Scaling

Scaling the x -axis by s_x and the y -axis by s_y , shown in [figure 38.4](#), yields the transformation matrix in homogeneous coordinates,

$$\mathbf{S} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (38.16)$$

The scaling matrix is a diagonal matrix.

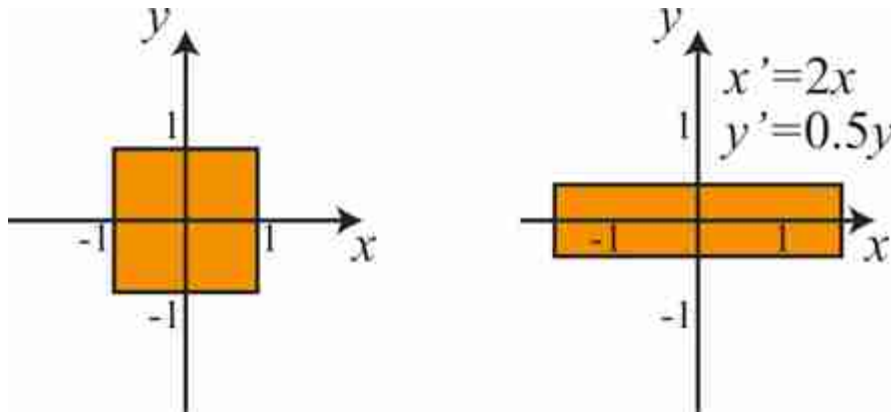


Figure 38.4: Non-uniform or anisotropic scaling. In this example the y -axis is compressed and the x -axis is expanded.

Uniform scaling is obtained when $s_x = s_y$, otherwise the scaling is nonuniform or anisotropic. After uniform scaling, all of the angles are preserved. In all cases, parallel lines remain parallel. Areas are scaled by the determinant of the scaling matrix.

38.3.3 Rotation

For a rotation by an angle θ , [figure 38.5](#), we simply have the matrix in homogeneous coordinates:

$$\mathbf{R} = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (38.17)$$

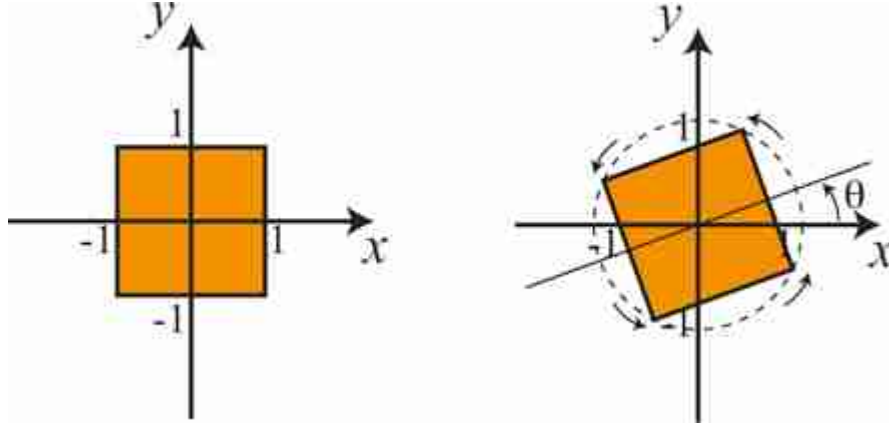


Figure 38.5: Rotation by an angle θ counterclockwise around the origin.

As in the case of the translation, a rotation in homogeneous coordinates is a product

$$\mathbf{p}' = \mathbf{R}\mathbf{p}. \quad (38.18)$$

Also, as we should expect, chaining two rotations with angles α and β , is the same than applying a rotation with an angle $\alpha + \beta$. You can check that multiplying the two rotation matrices you get the right transformation.

The determinant of a rotation matrix is 1 and the matrix is orthogonal, that is the transpose is equal to the inverse: $\mathbf{R}^T = \mathbf{R}^{-1}$. The inverse is also a rotation matrix. The distance between any point and the origin does not change after a rotation.

In heterogeneous coordinates, the transformation can also be written in the same way but using only the upper-left 2×2 matrix. Representing rotations in homogeneous coordinates has no benefit with respect to heterogeneous coordinates. But the advantage is that now both rotation and translation are written in the same way! They are both products of a matrix times a vector, so that they can be combined as we will discuss in section 38.3.5.

If the angle of rotation is very small, then we can approximate the rotation matrix by its Taylor development (for small x , $\sin(x) \approx x$ and $\cos(x) \approx 1$):

$$\mathbf{R} \simeq \begin{bmatrix} 1 & \theta & 0 \\ -\theta & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (38.19)$$

For small angles a rotation becomes a shear, which we will discuss next. In general, for all angles, a rotation can be written as two shears and a scaling.

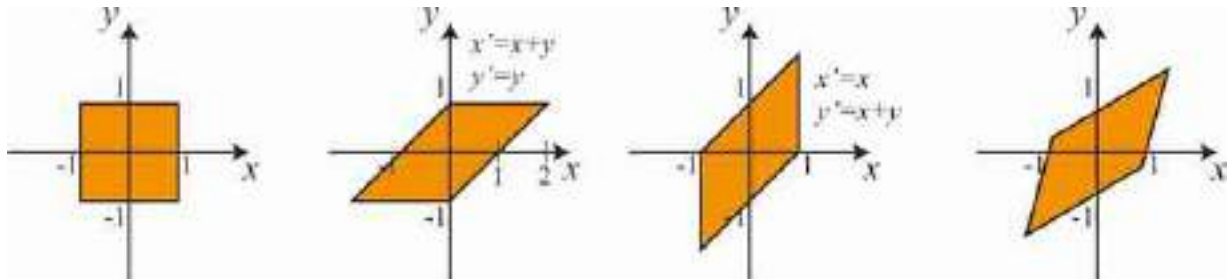
Rotations in 3D become more complex as there are multiple possible parametrizations.

38.3.4 Shearing

Shearing involves scale factors in the off-diagonal matrix locations, as shown in the matrix, $\mathbf{Q}$:

$$\mathbf{Q} = \begin{bmatrix} 1 & q_x & 0 \\ q_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (38.20)$$

[Figure 38.6](#) shows examples with an horizontal shear ($q_x = 1, q_y = 0$), a vertical shear ($q_x = 0, q_y = 1$), and an arbitrary shear.



[Figure 38.6:](#) Three examples of shear transformation.

In the example of the horizontal shear, the points are displaced along horizontal lines by displacement proportional to the y coordinate, $x' = x + q_y y$.

In a shear, lines are mapped to lines, parallel lines remain parallel, (non-zero) angles between lines change.

38.3.5 Chaining Transformations

As we have seen, homogeneous coordinates allows using all the four different transformations as matrix multiplications. We can now build complex transformations by combining these four transformations:

$$\mathbf{p}' = \begin{bmatrix} 1 & q_x & 0 \\ q_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \mathbf{p} \quad (38.21)$$

In heterogeneous coordinates the translation will have to be modeled as an addition breaking the homogeneity of this equation. The transformations described in this section are summarized in [figure 38.7](#).

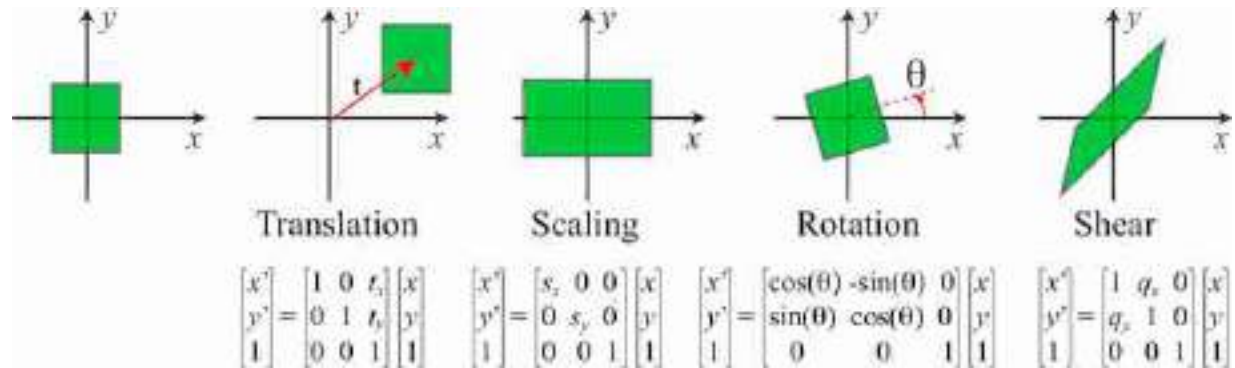


Figure 38.7: Summary of 2D geometric transformations and their formulation in homogeneous coordinates.

As matrix operation are noncommutative, the order in which operations are done is important (i.e., it is not the same to rotate with respect to the origin and then translate, as it is to translate and then rotate). All of the geometric transformations we have described are relative to the origin. If you want to rotate an image around an arbitrary central location, then you need to first translate to put that location at the origin, then rotate and then translate back.

$$\mathbf{p}' = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -t_x \\ 0 & 1 & -t_y \\ 0 & 0 & 1 \end{bmatrix} \mathbf{p} \quad (38.22)$$

Chaining transformations in such a way is a very important tool in computer graphics and we will also use it extensively as we dive deeper into geometry.

When chaining rotations and translations only we will have a **euclidean transformation** (lengths and angles between lines are preserved). A **similarity transform** is the result of chaining a rotation, translation and scaling (with uniform scaling, $s_x = s_y$). In this case angles are preserved but not lengths. Chaining all transformations results in an **affine transformation**. Each set of transformations forms a group.

38.3.6 Generic 2D Transformations

In general, chaining translations, scalings, rotations and shears will result in a generic matrix with the form:

$$\mathbf{p}' = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \mathbf{p} \quad (38.23)$$

Any transformation with that form (6 degrees of freedom) is an affine transformation. An affine transformation has the property that parallel lines will remain parallel after the transformation; however, lengths and non-zero angles might change.

As the last row of the transformation is $[0, 0, 1]$, one could be tempted to drop it and go directly from homogeneous to heterogeneous, using the top 2×3 matrix, but this will only work if the input vector has a 1 in the third component.

What happens if we have 9 degrees of freedom?

$$\mathbf{p}' = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \mathbf{p} \quad (38.24)$$

In fact we only have 8 degrees of freedom as a global scaling of the matrix does not change the homogeneous coordinates. The set of transformations described by this full matrix becomes more general than the transformations described in the previous sections. The additional degrees of freedom include **elations** and **projective transformations**.

38.3.7 Geometric Transformations as Convolutions

In chapter 15 we showed how certain geometric transformations can be written as convolutions (such as the translation) while others cannot (such as rotations, scalings, shears, etc.). However, things change when adding geometry explicitly to the image representation!

Once geometry is added to the representation, all of the transformations we discussed before can be implemented as one-dimensional (1D) convolutions over the locations as shown in the diagram in [figure 38.8](#).

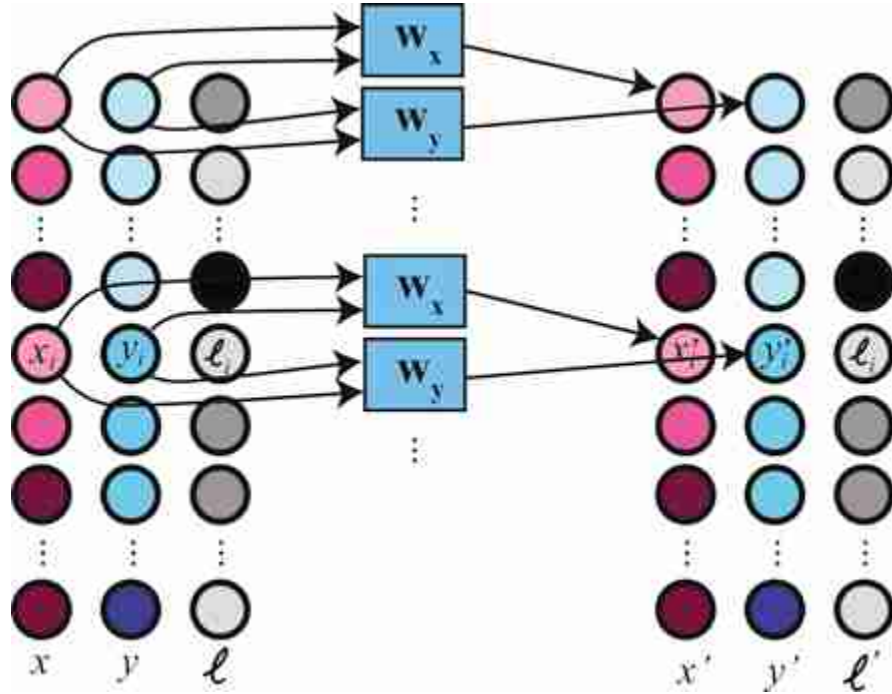


Figure 38.8: Geometric transformation as a convolution. The input and output signals are represented as pixel sets with explicit geometry. The convolution kernels are w_x and w_y , corresponding to one dimensional convolutions as they only mix input features within the same input vector. The output intensity values are the same as the input.

In the diagram ([figure 38.8](#)), each input element is a vector (x_i, y_i, ℓ_i) where x_i, y_i are the pixel location, and ℓ_i is the pixel intensity at that location. The convolution kernels are: $w_x = [\cos(\theta), \sin(\theta)]$ and $w_y = [-\sin(\theta), \cos(\theta)]$. The weights are the same for all inputs. The output is also represented using position explicitly: (x', y', ℓ') .

However, note that to perform a convolution on the output intensity will require translating the position encoding back into an image on a rectangular grid. For instance, after a rotation, the locations for the intensity values will change and the pixels will not lie on a rectangular grid anymore. The convolution kernels for the intensity channel will have to be transformed too.

38.4 Lines and Planes in Homogeneous Coordinates

One interesting application of homogeneous coordinates is to use it to describe lines and planes and perform operations with them. In 2D, the equation of a line is $ax + by + c = 0$, which can be written in homogeneous coordinates as

$$ax + by + c = 0 \rightarrow [a \quad b \quad c] \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = 0 \quad (38.25)$$

In homogeneous coordinates, the equation of a line is the dot product:

$$l^T p = 0 \quad (38.26)$$

where $l^T = [a, b, c]$. Therefore, a point p , belongs to the line when l and p are perpendicular.

This representation of the line is in homogeneous coordinates because it is scale invariant. This is, $[a, b, c]$ is the same line as $[a/c, b/c, 1]$. Therefore, it is also useful to describe the equation of the line with $l^T = n_x, n_y, -d$ where (n_x, n_y) is the normal to the line and d is the distance to the origin.

Using homogeneous coordinates makes obtaining geometric properties of points and lines very easy. Given two points p_1 and p_2 in homogeneous coordinates, the line that passes by both points is the cross product ([figure 38.9](#)):

$$l = p_1 \times p_2 \quad (38.27)$$

This is because if the line passes by both points, it has to verify that $l^T p_1 = 0$ and $l^T p_2 = 0$. That is, the vector l has to be perpendicular to both p_1 and p_2 . The cross product between p_1 and p_2 gives a vector that is perpendicular to both.

Following a similar argument, you can show that given two lines l_1 and l_2 the intersection point between them is the cross product ([figure 38.9](#)):

$$p = l_1 \times l_2 \quad (38.28)$$

The coordinates of p computed that way will be given in homogeneous coordinates. So you need to divide by the third component in order to get the actual point coordinates in heterogeneous coordinates.

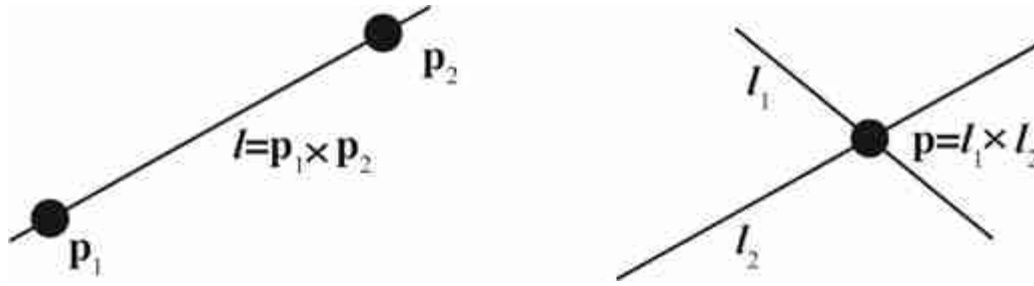


Figure 38.9: Using homogeneous coordinates to get the line that passes by two points, and to obtain the intersection of two lines. Both operations are analogous.

If three 2D points are colinear, then the determinant of the matrix formed by concatenating the three vectors, in homogeneous coordinates, as columns is equal to zero: $\det([\mathbf{p}_1 \ \mathbf{p}_2 \ \mathbf{p}_3]) = 0$. If three lines intersect in the same point we have a similar relationship: $\det([l_1 \ l_2 \ l_3]) = 0$.

It is also interesting to point out that a 3D vector can be interpreted as a 2D line or as a 2D point in homogeneous coordinates.

We can also do something analogous to represent planes in 3D. The equation of a 3D plane is $aX + bY + cZ + d = 0$, which can be written in homogeneous coordinates as:

$$\pi^T \mathbf{P} = 0 \quad (38.29)$$

where $\pi = [a, b, c, d]^T$ are the plane parameters and $\mathbf{P} = [X, Y, Z, 1]^T$ are the 3D point homogeneous coordinates.

Representing 3D lines with homogeneous coordinates is not that easy and the reader can consult other sources [187] to learn more about representing geometric objects in homogeneous coordinates.

38.5 Image Warping

Now that we have seen how to describe simple geometric transformations to pixel coordinates, we need to go back to the representation of the image as samples on a regular grid. This will require applying image interpolation.

The first algorithm that usually comes to mind when transforming an image is to take every pixel in the original image represented as $[\ell_i, x_i, y_i]$, apply the transformation, $\mathbf{M}$, to the coordinates, and record the pixel color into the resulting coordinates in the target image (figure 38.10). As coordinates might result in non-integer values, we can simply round the

result to the closest pixel coordinate (i.e., nearest neighbor interpolation as we discussed in section 21.4.1). This algorithm is called **forward mapping**. It is an intuitive way of warping an image but it is really not a good idea. We will have all sorts of artifacts such as missing values and aliasing as shown in [figures 38.10 and 38.11](#).

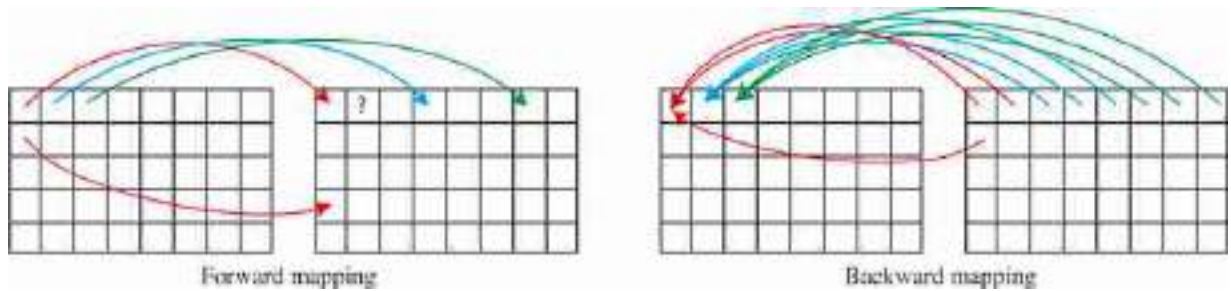


Figure 38.10: Comparison between forward and backward mapping using nearest neighbor interpolation. Forward mapping will produce missing values.

The best approach is to use what is called **backward mapping** which consists of looping over all the pixels of the target image and applying the inverse geometric transform, $\mathbf{M}^{-1}$; we then use interpolation (as described in section 21.4.1 in chapter 21) to get the correct color value ([figure 38.10](#)). This process guarantees that there will be no missing values (unless the coordinates go outside the frame of the input image) and there will be no aliasing if the interpolation is done correctly. To avoid aliasing, blurring of the input image might be necessary if the density of pixels in the target image is lower than in the input image. [Figures 38.10 and 38.11](#) compare forward and backward mapping.

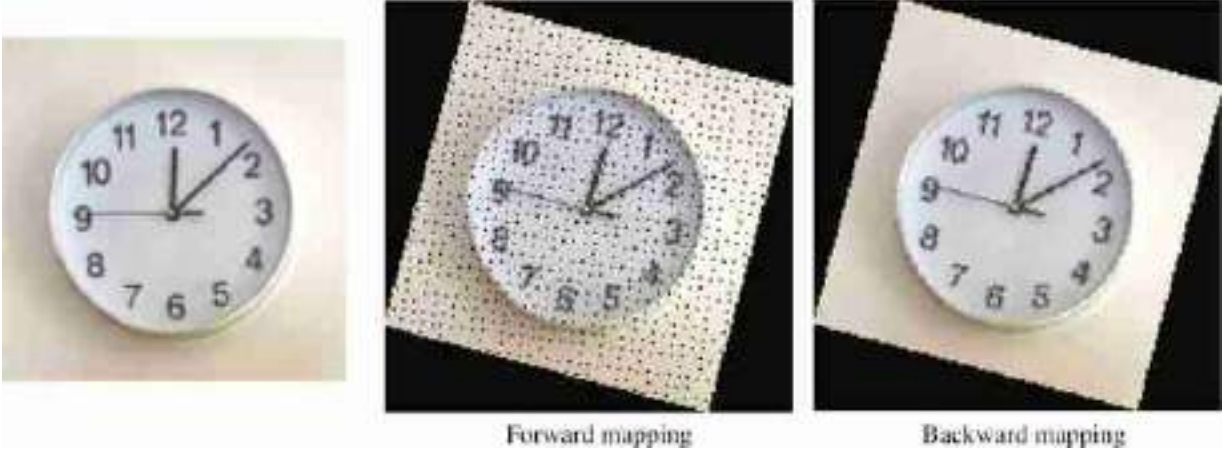


Figure 38.11: Comparison between forward and backward mapping. Forward mapping produces many artifacts. In both cases we use nearest neighbor interpolation.

To achieve high image quality warping it is important to choose a high quality interpolation filter such as bilinear, bicubic, or Lanczos. MIP-mapping [504] is another popular technique for high quality and efficient interpolation. MIP-mapping relies on a multiscale image pyramid to efficiently compute the best neighborhood structure needed to perform the interpolation at each location, which can be very useful when warping an image onto a curved surface.

Image warping can be applied to arbitrary geometric transformations and not just the ones described in this section.

38.6 Implicit Image Representations

An image is an array of values, $\ell \in \mathbb{R}^{N \times M \times 3}$, where each value is indexed as $\ell[n, m]$ when n, m take only on discrete values. We can say that interpolation is a way of transforming the discrete image into a continuous signal: $\ell(x, y)$.

An implicit image representation via a function f_θ trained to reproduce the image pixels is a function such that,

$$\ell(x, y) = f_\theta(x, y) \quad (38.30)$$

where now x, y can take on any real value.

For this representation to work better, the input location is usually first transformed using **positional encoding**, and the final network is more

complex than the one shown here. We will discuss this type of representations more in depth in chapter 45.

38.6.1 Interpolation

In the case of nearest neighbors or bilinear interpolation, the parameters of the interpolation function θ is the input image itself. For example, using a functional form, nearest neighbors interpolation can be written as:

$$\ell(x, y) = \ell[\text{round}(x), \text{round}(y)] \quad (38.31)$$

What is really interesting about thinking about interpolation in this way is that we can now extend the space of possible functions $f_\theta(x, y)$ to include other functional forms. For instance, this function could be implemented by a neural network that will take as input the two image coordinate values x and y and will output the intensity value at that location. The training set for the neural network is the image itself, and it will consist of the input-output pairs $[(x_i, y_i); \ell(x_i, y_i)]$ (i.e., location as input and intensities/colors as output). During training the neural network will memorize the image. The training will contain only values at discrete positions but in test time we can use any continuous input values. For this formulation to work, the neural network should be able to generalize to non-integer coordinate values, that is, it should be able to interpolate between samples.

38.6.2 Image Warping with Implicit Representations

Once the neural network, $f_\theta(x, y)$, has learned to reproduce the image, we can reconstruct the original image or apply transformations to it. Image warping is then implemented by simply applying the inverse geometric transformation to the discrete coordinates of the output grid and use the functional representation of the image to get the interpolated values:

$$\ell(x, y) = f_\theta(\mathbf{M}^{-1}(x, y, 1)^T) \quad (38.32)$$

In this equation the input location is written in homogeneous coordinates, so the function f will first have to divide by the third component to translate the input back to heterogeneous coordinates.

The example in [figure 38.12](#) shows an image encoded by a sinusoidal representation network (SIREN) [445] and then reconstructed with a rotation of 45 degrees and a scaling by 0.5 along both dimensions.

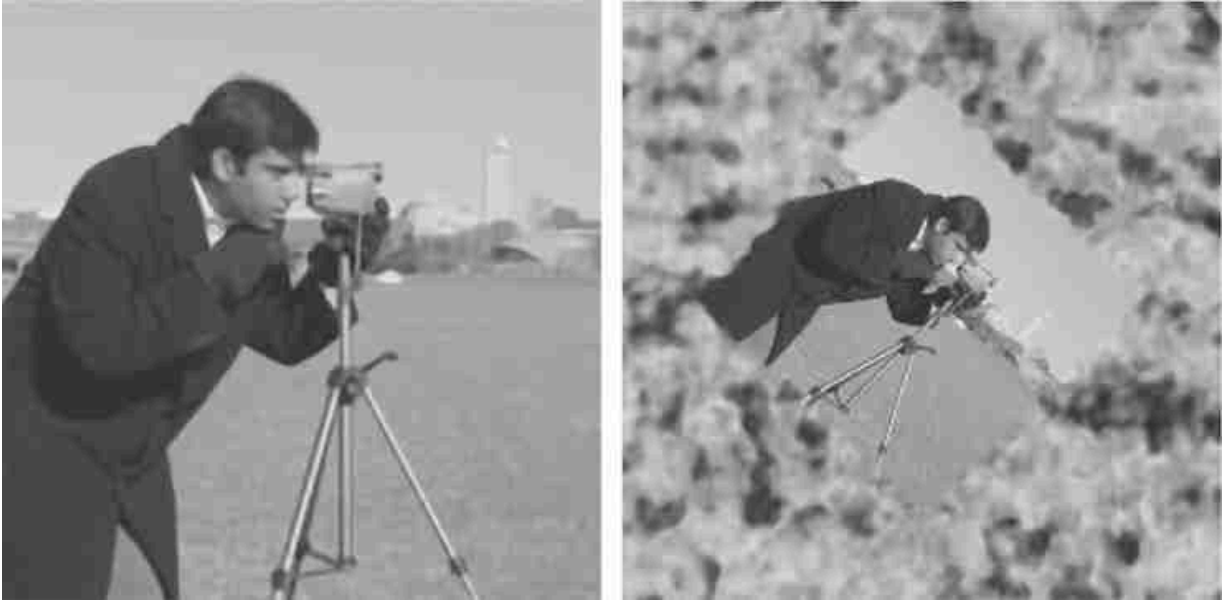


Figure 38.12: (left) Image encoded by SIREN. (right) Image reconstructed with a rotation of 45 degrees and a scaling by 0.5 along both dimensions.

From this result we can make a few observations. First, we can see that, due to the transformation, there is aliasing in the sampled image (aliasing is most visible in the leg of the tripod). One way of avoiding aliasing would be by sampling the output on a finer grid and then blurring and downsampling the result to the desired image size. The second observation is that the way the boundary is extended is not like any of the methods that we studied in chapter 15; instead the image is padded by some form of noise that smoothly extends the image without adding strong image derivatives.

We can also study the reverse problem where we have two images (one is a transformed version of the other) and the goal is to identify the transformation $\mathbf{M}$. This problem is called **image alignment**.

The spatial transformer network [233] is an example of using parametric image transformations inside a neural network. Such transformation can be helpful during learning to align the training examples into a canonical pose.

38.7 Concluding Remarks

Homogeneous coordinates are extensively used in computer vision and computer graphics. They allow simplifying the computation of geometric

image transformations. Therefore, many libraries in vision and graphics assume that the user is knowledgeable about the different coordinate systems.

One of the most important uses is in the formulation of perspective projection. We will devote the next chapter to describing the image formation process and camera models using homogeneous coordinates.

Representing images as collections of pixels with an explicit representation of geometry has a long history and is at the center of many modern methods for 3D image representation.

39 Camera Modeling and Calibration

39.1 Introduction

In chapter 5 we described the image formation process assuming that the camera was the central element and we placed the origin of the world coordinates system in the pinhole. But, as we will be interested in interpreting the scene in front of the camera, other systems of reference might be more appropriate. For instance, in the simple world model of chapter 2 we placed the origin of the world coordinates system on the ground, away from the camera. What formulation of the image formation process will allow us changing coordinate systems easily? This is what we will study in this chapter.

In the sketch shown below ([figure 39.1](#)), a standing person is holding a camera and taking a picture with it of someone sitting at a table (sorry for our drawing skills but hopefully this description compensates for our poor artistry). We will be answering questions about the scene based on the picture taken by the camera: for instance, how high and large is the table? We will also answer questions about the camera, like how high is the camera above the floor?



Figure 39.1: Camera-centric and world-centric camera coordinate systems.

In settings where we have multiple cameras we will have to be able to transform the coordinates frames between cameras. And finally, to translate the two-dimensional (2D) images into the underlying three-dimensional (3D) scene, we need to know how 2D points relate to 3D world coordinates, which is called calibrating the cameras.

In this chapter we will show how to use homogeneous coordinates to describe a camera projection model that is more general than what we have done until now. This formulation will be key to building most of the approaches we will study from now until the end of this book; therefore it is important to be familiar with it. It is also often used both in classic computer vision approaches and in deep learning architectures.

But before we move into the material, we are sure you would like to see the picture that the standing character from the previous sketch took. Here it is:



Figure 39.2: The picture taken by the standing character from [figure 39.1](#).

39.2 3D Camera Projections in Homogeneous Coordinates

Let's start this chapter with the most important application of homogeneous coordinates for us: describing perspective projection. For now, we will continue assuming that the origin of the world coordinates system is at the pinhole location and that the Z -axis corresponds to the optical axis (i.e., it is perpendicular to the image plane).

As we have already discussed (see [figure 39.3](#) to refresh your memory), the perspective projection equations are nonlinear and involve a division by the point's Z -location. That is, a 3D point with coordinates $[X, Y, Z]^T$ appears in the image at coordinates $[x, y]^T$ where $x = fX/Z$ and $y = fY/Z$ (f is the focal length). This division complicates algebraic relations. Projective geometry and homogeneous coordinates are tools that simplify those algebraic relations, and thus simplify many of the equations that involve geometric transformations.

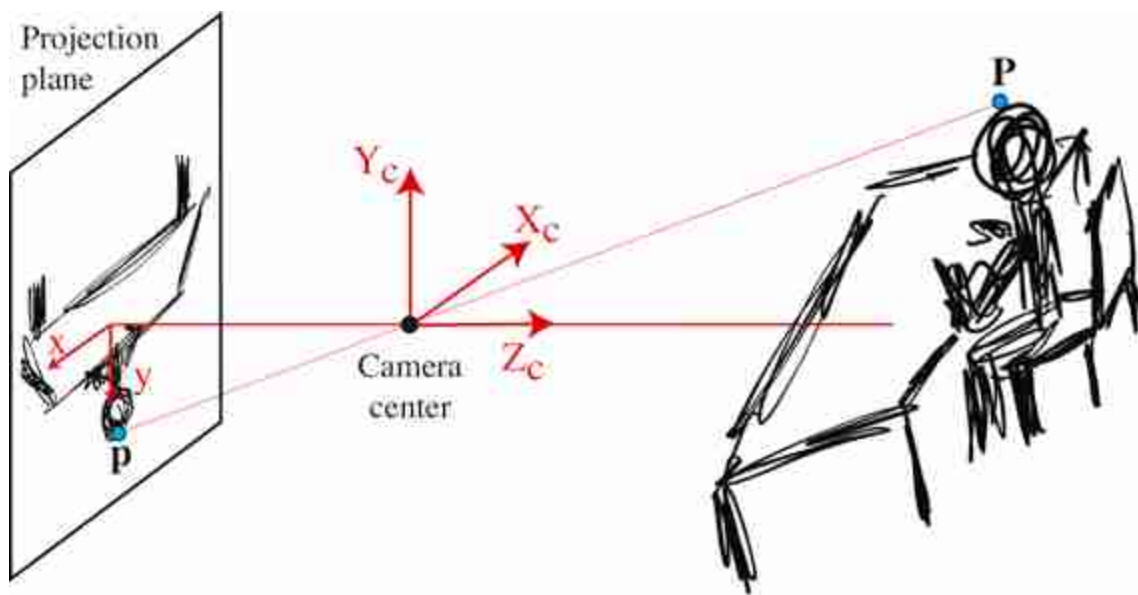


Figure 39.3: Perspective projection. Remember that from similar triangles, we have $x/f = X/Z$ and $y/f = Y/Z$.

To write points in 3D space in homogeneous coordinates, we add a fourth coordinate to the three coordinates of Euclidean space, as described

in chapter 38. Using homogeneous coordinates, camera projections can be written in a simple form as a matrix multiplication. Consider a coordinate system with the origin fixed at the center of projection of the camera and the 3D point in homogeneous coordinates, $\mathbf{P} = [X, Y, Z, 1]^T$.

We can verify that the matrix, $\mathbf{K}$, shown below, multiplied by the vector describing the 3D point, $\mathbf{P}$, returns the position of its perspective projection (equation [5.7]):

$$\mathbf{K}\mathbf{P} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \begin{bmatrix} fX \\ fY \\ Z \end{bmatrix} = \lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (39.1)$$

Transforming the product vector from homogeneous coordinates to heterogeneous coordinates yields

$$\begin{bmatrix} fX \\ fY \\ Z \end{bmatrix} \rightarrow \begin{pmatrix} fX/Z \\ fY/Z \end{pmatrix} = \begin{pmatrix} cx \\ y \end{pmatrix} = \mathbf{p} \quad (39.2)$$

showing that the matrix $\mathbf{K}$, when multiplied by the homogeneous coordinates of a 3D point, $\mathbf{P}$, renders the homogeneous coordinates of the perspective projection of that 3D point, $\mathbf{p}$. This formulation is one of the biggest benefits of using homogeneous coordinates. It allows writing perspective projection as a linear operator in homogeneous coordinates.

Because homogeneous coordinates, and therefore also the transformation matrices, are only defined up to an arbitrary uniform (non zero) scale factor, the perspective projection transformation matrix is equivalently written as

$$\mathbf{K} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1/f & 0 \end{bmatrix} \quad (39.3)$$

Also, in this particular case, it is fine to drop the last column of $\mathbf{K}$ and use the heterogeneous coordinates form of $\mathbf{P}$. However, it is interesting to keep the homogeneous formulation, as it will allow us to work with more general forms of camera models, as we will see next.

39.2.1 Parallel Projection

We can use homogeneous coordinates to describe a wide variety of camera models. For instance, the projection matrix for an orthographic projection (equation [5.10]), $\mathbf{K}$, is

$$\mathbf{K} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad (39.4)$$

as can be verified by multiplying by a 3D point in homogeneous coordinates.

39.3 Camera-Intrinsic Parameters

Now that we have seen how 3D to 2D projection can be effectively modeled using homogeneous coordinates and simple vector computations, let's apply these tools to model cameras.

We want to construct a camera model that captures the image formation process. Cameras translate the world into pixels, but the relationship between the 3D scene and the 2D image captured by the camera is not always straightforward. To start with, the world is measured in units of meters while points in images are measured in pixels. Factors such as geometric distortions can influence this relationship, and we need to account for these elements in our camera model.

39.3.1 From Meters to Pixels

[Figure 39.4](#) illustrates the image formation process and the image sampling by the sensor. We need to account for the perspective projection of the point, $\mathbf{P}$, described in camera coordinates, to pixel coordinates in the sensor plane of the camera.

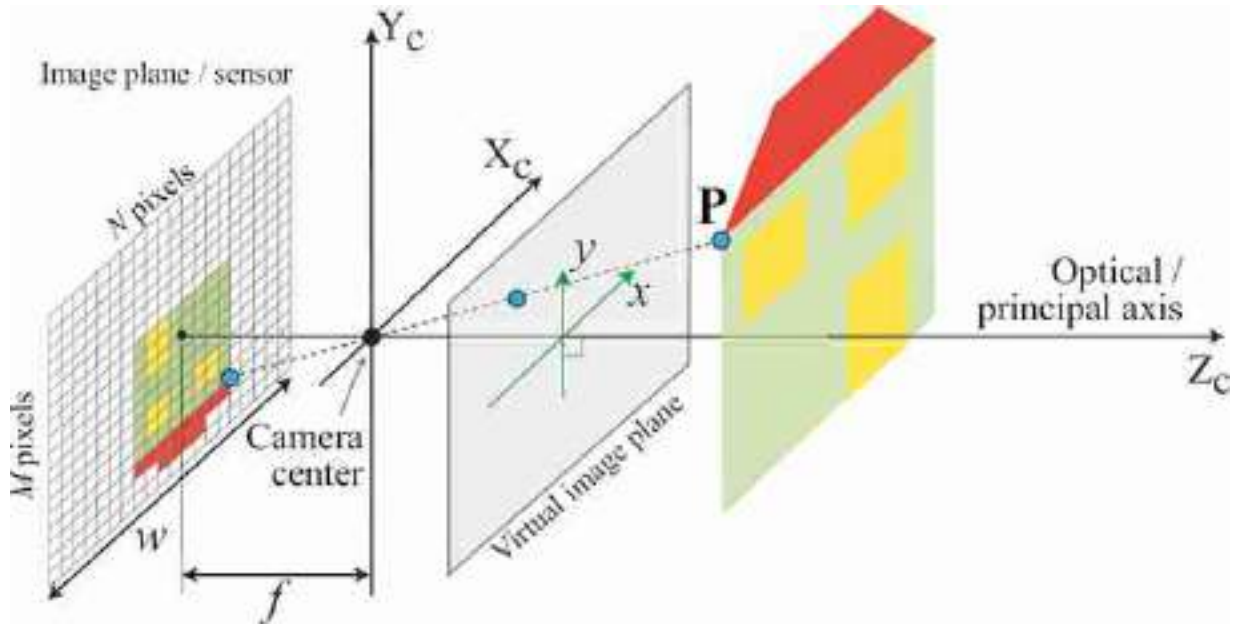


Figure 39.4: An image is projected into the sensor. World coordinates are transformed into pixels at the sensor. The focal length is f , and the physical width of the sensor is w . The sensor has $N \times M$ pixels.

The units of the position in pixel space may be different than those of the world coordinates, which scales the focal length, f , in equation (39.3) to some other value. Let's call that value a :

$$\mathbf{K} = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (39.5)$$

where the constant a is related to the physical camera parameters in the following way:

$$a = f N / w \quad (39.6)$$

where f is the focal length (in meters), w is the width of the sensor in meters, and N is the image width in pixels. The ratio N/w is the number of pixels per meter in the sensor.

Also, the image center might not be the pixel $[0, 0]^T$. If the optical axis coincides with the image center, we need to apply a translation so that a 3D point on the optical axis (i.e., $X = Y = 0$) projects to $[c_x, c_y]^T$. For example, for an image of size $N \times M$ we will have $c_x = N/2$ and $c_y = M/2$:

$$\mathbf{K} = \begin{bmatrix} a & 0 & c_x & 0 \\ 0 & a & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (39.7)$$

It is also possible that the pixels are not square, in which case we have to account for different scaling along both axis:

$$\mathbf{K} = \begin{bmatrix} a & 0 & c_x & 0 \\ 0 & b & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (39.8)$$

Then, the final transformation from 3D coordinates, $[X, Y, Z]^T$, to image pixels, $[n, m]^T$, has the form:

$$\begin{bmatrix} a & 0 & c_x & 0 \\ 0 & b & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \rightarrow \begin{pmatrix} aX/Z + c_x \\ bY/Z + c_y \end{pmatrix} = \begin{pmatrix} n \\ m \end{pmatrix} \quad (39.9)$$

The signs of a and b can be positive or negative. With the convention used in this book, where the camera optical axis coincides with the Z -axis and the camera looks in the direction of positive Z , the sign of a has to be negative. If n and m are indices into an array, then the image origin is located at the top-left of the array, which means the signs of a and b will be negative. This is the convention used by Python and OpenCV. [Figure 39.5](#) shows two different common conventions.

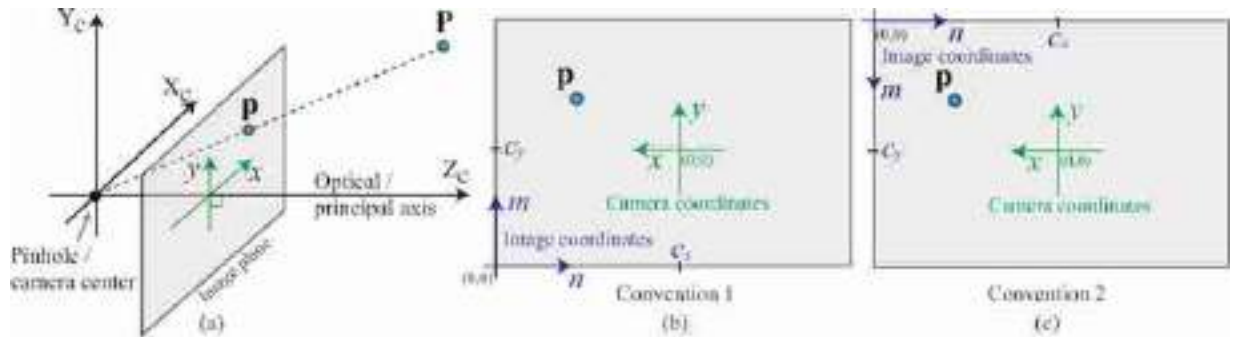


Figure 39.5: (a) 3D representation of the image plane. (b and c) Two different conventions for the image coordinate systems. In this book we have been using (b).

39.3.2 From Pixels to Rays

If you have a calibrated camera, can we derive the equation of the ray that leaves the image at one point? Answering this question, will allow us

answering the following one: If we know Z for an image pixel with camera-coordinates x, y , can get the point world-coordinates X and Y ?

We can relate the point in the image plane, $\mathbf{p}$, with the 3D point $\mathbf{P}$ in world-coordinates. To do this, we will first play a little trick: We will express the 2D point $\mathbf{p}$ in 3D coordinates. We can do this by realizing that the point $\mathbf{p}$ is contained in the image plane which is located at a distance f of the origin along the Z -axis ([figure 39.6](#)). Therefore, in 3D, the 2D point $\mathbf{p}$ is a point located at coordinates $Z_c = f$, where f is the focal length, $X_c = x$, and $Y_c = y$, as illustrated in [figure 39.6](#).

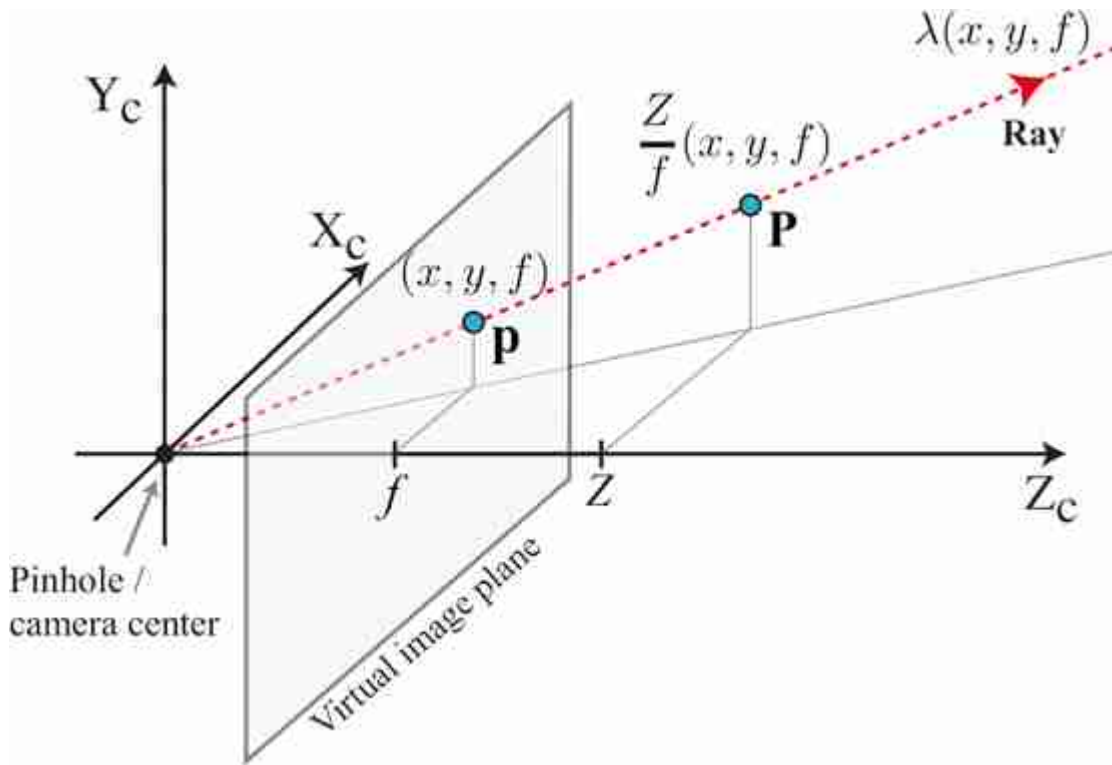


Figure 39.6: The 3D point $\mathbf{P}$ is obtained by scaling the point $\mathbf{p} = (x, y, f)$ with a scaling factor Z/f . The ray that passes by the point $\mathbf{p}$ is the line defined by $\lambda(x, y, f)$, with λ being a positive real number.

The ray that connects the camera center and the point $\mathbf{p}$ is $\lambda(x, y, f)$, with λ being a positive real number. Any 3D point along this line will project into the same image point $\mathbf{p}$ under perspective projection.

As shown in [figure 39.6](#) the 3D point $\mathbf{P}$ is obtained by scaling the ray that passes by $\mathbf{p}$. The scaling factor is $\lambda = Z/f$:

$$\frac{z}{f} \begin{pmatrix} x \\ y \\ f \end{pmatrix} = \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \quad (39.10)$$

We should use a instead of f if we are using coordinates in pixels. We will revisit this again in chapter 43.

39.3.3 A Simple, although Unreliable, Calibration Method

Everything we described up to here, relies on a series of parameters (i.e., focal length, sensor size) that will be unknown in general. Before we continue, let's try to see if we can find out what those parameters are in a real setting.

Let's start by describing a simple and intuitive method for camera calibration illustrated in [figure 39.7](#). The method we will describe is not very accurate, but it provides be a good sanity check before doing more precise camera calibrations. We will describe a more accurate method in section 39.7.

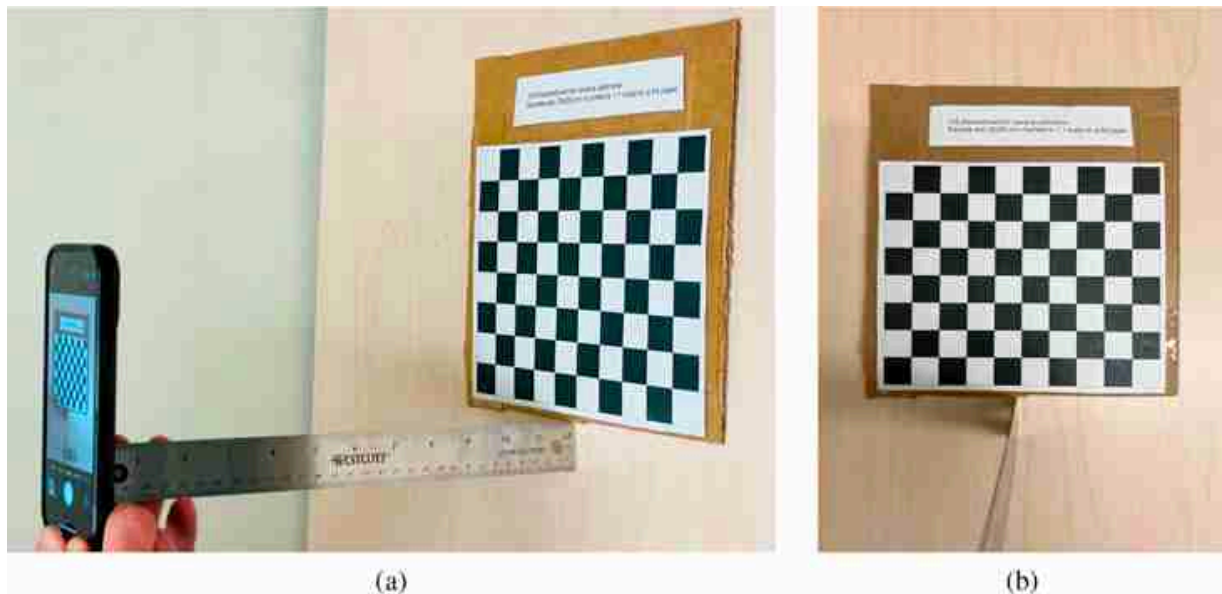


Figure 39.7: A simple calibration setting.

To calibrate a camera we need the following three ingredients:

- An object of known size: In this setting, [figure 39.7\(a\)](#), the object is a 8×10 chessboard pattern printed so that each square exactly measures 2×2 cm. The total width is $W = 20$ cm. This chessboard is a standard calibration target [526].
- The distance between the camera and the object: We use a ruler to measure that the distance between the camera and the chessboard as shown in [figure 39.7\(a\)](#). The distance is approximately $Z = 31$ cm.
- A picture of the object: [Figure 39.7\(b\)](#) shows the picture taken by the camera shown in [figure 39.7\(a\)](#). The camera plane is parallel to the chessboard. From the picture, we can measure the width of the chessboard in pixels, which is $L = 2,002$ pixels.

These three quantities, W , L and Z , are related via the parameter a of the projection matrix by the equation: $a = ZL/W$, which results in:

$$a = \frac{31 \times 2,002}{20} = 3,103.1 \quad (39.11)$$

The picture size is $4,032 \times 3,024$ pixels which we can use to get the other two parameters of the camera projection matrix: $c_x = 3,024/2$ and $c_y = 4,032/2$. Putting all together we get the projection matrix:

$$\mathbf{K} = \begin{bmatrix} 3,103.1 & 0 & 1,512 & 0 \\ 0 & 3,103.1 & 2,016 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (39.12)$$

Would it be possible to infer the intrinsic camera matrix from the camera technical specifications alone? Can we directly estimate the parameter a directly from the camera specs? The picture is taken with the wide-angle lens of an iPhone 13 pro. The focal length is 5.7 mm, the sensor size is 7.6×5.7 mm, and the image size is $4,032 \times 3,024$ pixels. This gives as an intrinsic parameter of:

$$a = \frac{4,032 \times 5.7}{7.6} = 3,024. \quad (39.13)$$

It works! (Does it?) Or at least it is close. But a lot of things happen inside a camera and estimating these values from hardware specs might be difficult.

But is this quality for the calibration enough?

39.3.4 Other Camera Parameters

Unfortunately, there are other important aspects of a camera that require precise calibration. In some rare cases, pixels might not be square, or might

be arranged along non-perpendicular axes, which requires introducing a skew parameter in $\mathbf{K}$. Radial distortion introduced by lenses is often an important and frequent source of issues that requires calibration. When there is radial distortion, straight lines will appear curved. In that case, it is very important to correct for the distortion. Standardized tools for camera calibrations take into account all these different aspects of the camera in order to build an accurate camera model. A more in-depth description of these tools and methods can be found here [[526](#), [525](#)].

39.4 Camera-Extrinsic Parameters

In all our previous examples, we have been assuming that the camera origin was located at the origin of the world coordinate system with the optical axis aligned with the world Z -axis. In practice, there will be many situations where placing the camera at a special position will be more convenient. For instance, in the simple vision system we studied in chapter 2 the camera center was placed above the Z -axis. Let's now study a more general setup where the camera is placed at an arbitrary location away from the world coordinate system.

In the examples shown in [figure 39.8](#), we are interested in placing the world coordinate system so that the origin is on the ground and the axes Z_w and X_w are contained on the ground plane, and parallel to the two axis defined by the table. The axis Y_w is perpendicular to the ground. Let's say that our camera has its camera center displaced by vector $\mathbf{T}$ and the axes are rotated by $\mathbf{R}^T$ with respect to the world-coordinates system, as shown in [figure 39.8](#). Precisely, this is done by first placing the camera at the origin of the world-coordinates system, then rotating the camera with the rotation matrix $\mathbf{R}^T$ and finally translating it by $\mathbf{T}$ (the order of these two operations matters). We use the inverse of $\mathbf{R}$ (equal to its transpose) to simplify the derivations later.

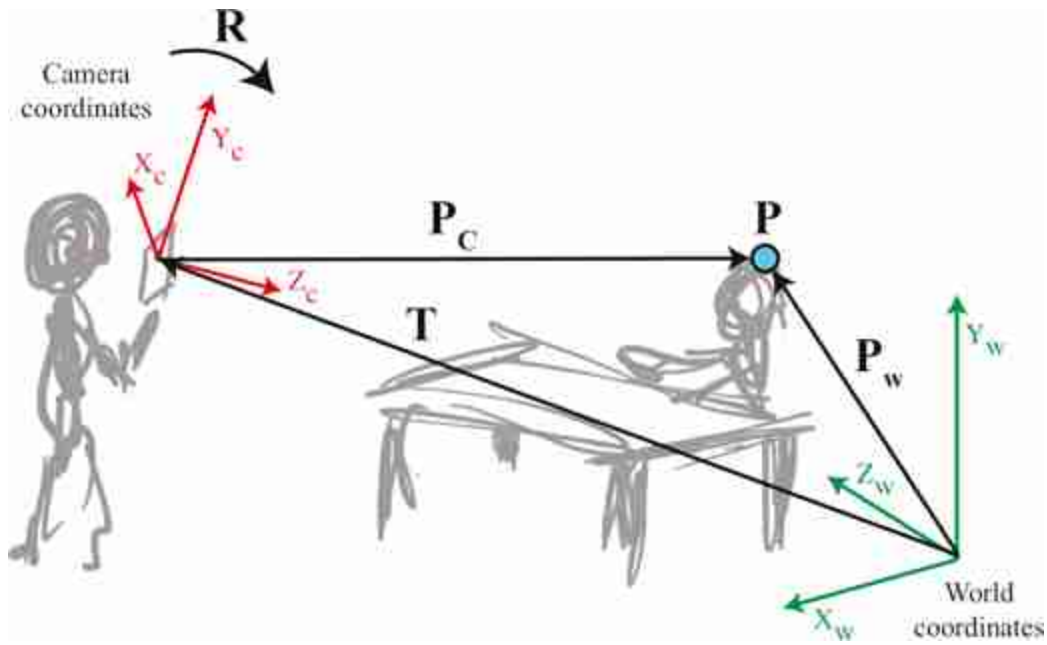


Figure 39.8: World- and camera-coordinate systems. A 3D point expressed in world coordinates, $\mathbf{P}_w$, can be expressed in the camera-coordinate frame, $\mathbf{P}_c$, by applying the translation, $\mathbf{T}$, and rotation, $\mathbf{R}$, to the point coordinates.

We are now given a point in the world $\mathbf{P}_w$, with its coordinates are described in terms of the world-coordinate system. Our goal is to find how that point projects into the image captured by the camera. To do this, first we need to change the coordinate system and express the point with respect to the camera coordinate system. Once we have done that change of coordinate system, we can use the intrinsic camera model K to project the point into the image plane.

We need to initially translate, and subsequently rotate, the camera so that points in the world, $\mathbf{P}_w$, are expressed in a coordinate system that originates at the camera center and is rotated to align with the camera. Note that this order is reversed from how we defined the camera coordinate system earlier in this section. Using heterogeneous coordinates we can describe these two transformations as:

$$\mathbf{P}_c = \mathbf{R}(\mathbf{P}_w - \mathbf{T}) = \mathbf{R}\mathbf{P}_w - \mathbf{R}\mathbf{T} \quad (39.14)$$

where we use the 3×3 matrix, $\mathbf{R}$, to rotate the camera so that the camera's coordinates are parallel to those of the world. The vector $\mathbf{T}$ is the position of the camera in world coordinates. The rotation $\mathbf{R}$ in this equation is the

inverse of the rotation matrix of the camera coordinate system with respect to the world coordinates.

Let's use now homogeneous coordinates to describe the transformation from world coordinates to camera coordinates. To do so, we first translate the coordinate system by $-\mathbf{T}$ using the homogeneous matrix, $\mathbf{M}_1$:

$$\mathbf{M}_1 = \begin{bmatrix} 1 & 0 & 0 & -T_x \\ 0 & 1 & 0 & -T_y \\ 0 & 0 & 1 & -T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (39.15)$$

Then, we use the 3×3 matrix, $\mathbf{R}$, to rotate the camera so that the camera's coordinates are parallel to those of the world:

$$\mathbf{M}_2 = \begin{bmatrix} R_1 & R_2 & R_3 & 0 \\ R_4 & R_5 & R_6 & 0 \\ R_7 & R_8 & R_9 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (39.16)$$

Using these two matrices we have

$$\mathbf{P}_C = \mathbf{M}_2 \mathbf{M}_1 \mathbf{P}_W \quad (39.17)$$

where $\mathbf{P}_W$ and $\mathbf{P}_C$ are the 3D coordinates of the same point ([figure 39.8](#)), written in world and camera coordinates, respectively. Sometimes, this transformation is written by making the product of the two matrices explicit:

$$\mathbf{M}_2 \mathbf{M}_1 = \begin{bmatrix} R_1 & R_2 & R_3 & -\mathbf{r}_1 \mathbf{T} \\ R_4 & R_5 & R_6 & -\mathbf{r}_2 \mathbf{T} \\ R_7 & R_8 & R_9 & -\mathbf{r}_3 \mathbf{T} \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R} & -\mathbf{R}\mathbf{T} \\ \mathbf{0}^T & 1 \end{bmatrix} \quad (39.18)$$

where $\mathbf{r}_i$ represents the row i of the rotation matrix $\mathbf{R}$.

Substituting equation (39.18) into equation (39.17) we can now translate a 3D point expressed in world coordinates into camera coordinates:

$$\mathbf{P}_C = \begin{bmatrix} \mathbf{R} & -\mathbf{R}\mathbf{T} \\ \mathbf{0}^T & 1 \end{bmatrix} \mathbf{P}_W \quad (39.19)$$

The camera parameters, $\mathbf{R}$ and $\mathbf{T}$, that relate the camera to the world are called the **extrinsic camera parameters**. These parameters are external to the camera.

We have seen now how to transform a point described in the world-coordinate system into the camera-coordinates system. We can now use the intrinsic camera parameters to project the point into the image plane. In the

next section we will see how to combine both sets of parameters to get a full camera model.

39.5 Full Camera Model

The concatenation of the extrinsic and intrinsic transformation matrices yields the desired transformation matrix, which will transform world coordinates to rendered pixel coordinates:

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \mathbf{K} \mathbf{M}_2 \mathbf{M}_1 \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (39.20)$$

Converting back to heterogeneous from homogeneous coordinates, the 2D pixel indices of a 3D point $[X, Y, Z]^T$ are $[x, y]^T = [x'/w, y'/w]^T$. We can make all the matrices in equation (39.20) explicit and write:

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & 0 & c_x & 0 \\ 0 & a & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} R_1 & R_2 & R_3 & 0 \\ R_4 & R_5 & R_6 & 0 \\ R_7 & R_8 & R_9 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -T_X \\ 0 & 1 & 0 & -T_Y \\ 0 & 0 & 1 & -T_Z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (39.21)$$

The right-most matrix multiplications reflect the camera's extrinsic parameters, and the left-most matrix reflects the intrinsic parameters and the perspective projection. Note that column of zeros in the left-most matrix means that, without loss of generality, equation (39.21) can be written in slightly simpler form as:

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & 0 & c_x \\ 0 & a & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R_1 & R_2 & R_3 \\ R_4 & R_5 & R_6 \\ R_7 & R_8 & R_9 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -T_X \\ 0 & 1 & 0 & -T_Y \\ 0 & 0 & 1 & -T_Z \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (39.22)$$

This is usually written as:

$$\mathbf{p} = \mathbf{K} [\mathbf{R} | -\mathbf{RT}] \mathbf{P}_w \quad (39.23)$$

By replacing $\mathbf{K}$ with different intrinsic parameters we can build models for different camera types such as orthographic cameras, weak-perspective model, affine cameras, and so on. [Figure 39.9](#) summarizes this section.

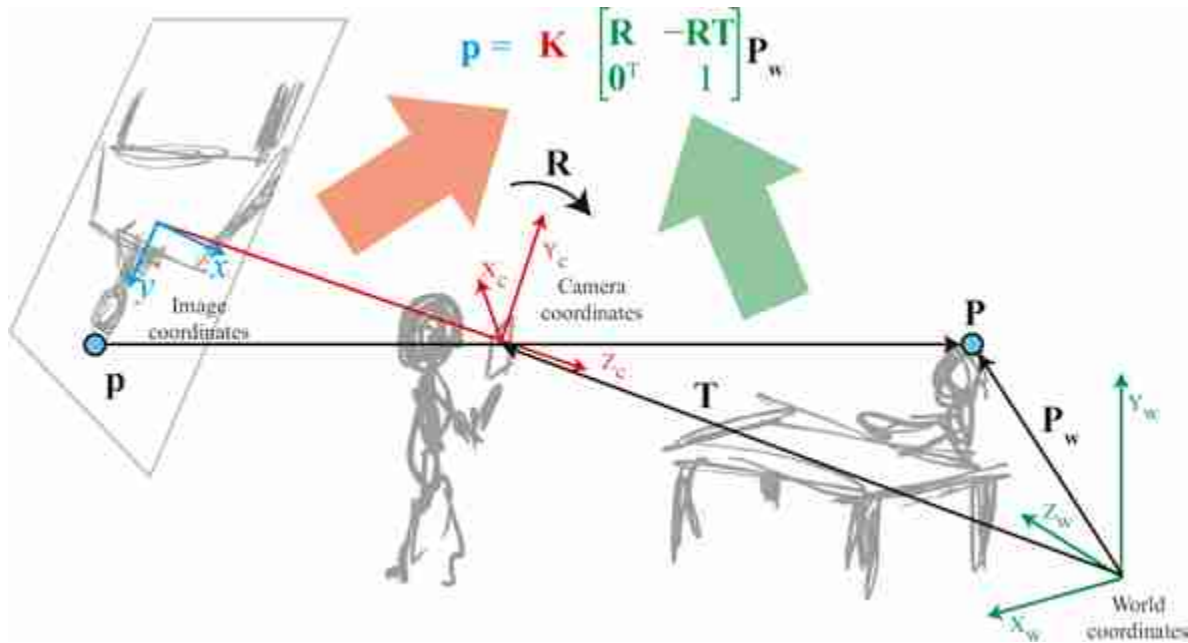


Figure 39.9: Projection of a point into the image plane. This summary puts together [figure 39.3](#) and [figure 39.8](#). First, we change the world-coordinates system, in which the point is expressed, into the camera-coordinates system, using the extrinsic camera model, and then we project it into the camera plane using the intrinsic camera model.

In the rest, we will usually refer to the camera projection matrix, which includes both intrinsic as extrinsic parameters as:

$$M = K \begin{bmatrix} R & -RT \\ 0^T & 1 \end{bmatrix} \quad (39.24)$$

39.6 A Few Concrete Examples

Let's write down the full camera model for a few concrete scenarios that are also of practical relevance. The four scenarios we will consider have growing complexity as shown in [figure 39.10](#).

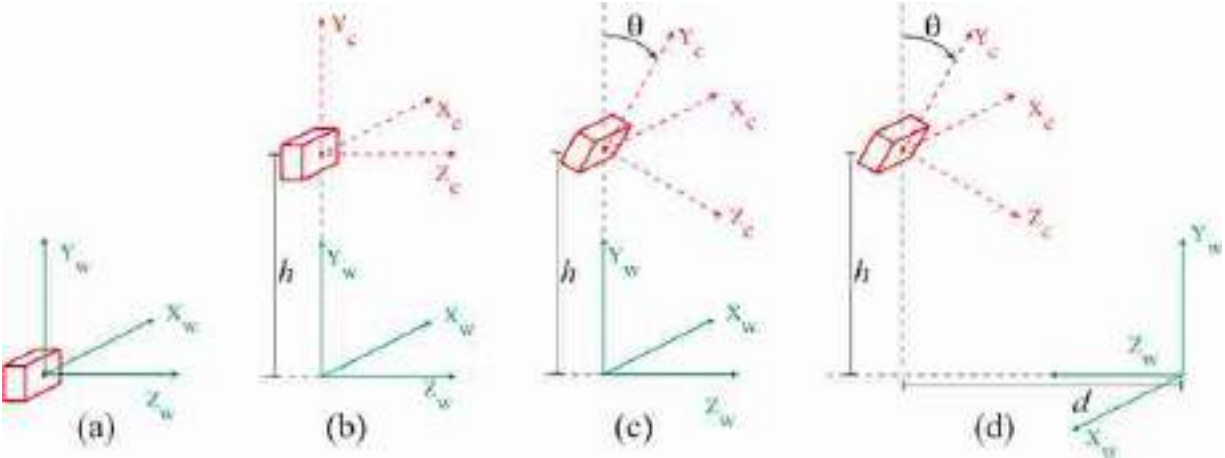


Figure 39.10: Four examples of camera poses respect to the world-coordinates system with increasing complexity. In the text we derive the projection matrix for each scenario.

Example 1. We will start with a camera located at the origin of the world-coordinate systems, as shown in [figure 39.10\(a\)](#). In this case, both coordinate systems are identical and the camera projection matrix is:

$$\mathbf{M} = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (39.25)$$

We have set $c_x = c_y = 0$, which assumes that the center of the image are the coordinates $(x, y) = (0, 0)$. This will simplify the equations for the following examples.

Example 2. The second scenario we will consider is shown in [figure 39.10\(b\)](#). This scenario corresponds to a case where a person is holding a camera, pointing toward the horizon in such a way that the optical axis of the camera is parallel to the ground. In this case we place the world-coordinate frame with its origin on the ground plane right below the camera. The camera is at a height h from the ground, measured in meters.

$$\mathbf{M} = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -h \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a & 0 & -ah \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (39.26)$$

Going back to heterogeneous coordinates we have that a 3D point at location $P_W = [X, Y, Z]^T$ in world coordinates projects to the image coordinates:

$$x = a \frac{X}{Z} \quad (39.27)$$

$$y = a \frac{Y - h}{Z} \quad (39.28)$$

In this scenario, if you hold the camera parallel to the ground at the height of your eyes and take a picture of a person standing in front of you with a similar height, their eyes will project near the middle portion of the vertical axis of the picture (their eyes will be located at $Y \simeq h$; therefore, they will project to $y \simeq 0$, which is the center of the picture). And this will be true regardless of how far away they are. This is illustrated in [figure 39.11](#). The horizon line is located at the points where $Z \rightarrow \infty$, which are also at $y = 0$ (we will talk more about the horizon line in the following chapter).



Figure 39.11: If you hold the camera parallel to the ground at the height of your eyes and take a picture of a person standing in front of you with a similar height, their eyes will project near the middle portion of the vertical axis of the picture.

Example 3. Let's now consider that the camera is tilted by looking downward with an angle θ , as shown in [figure 39.10\(c\)](#). The horizontal camera axis remains parallel to the ground, but now the optical axis points toward the ground if $\theta > 0$.

Let's first write down the camera-extrinsic parameters. The angle θ here corresponds to a rotation around the X_c -axis, thus we can write:

$$\begin{bmatrix} \mathbf{R} & -\mathbf{RT} \\ \mathbf{0}^\top & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & \sin(\theta) & 0 \\ 0 & -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -h \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (39.29)$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & \sin(\theta) & -h \cos(\theta) \\ 0 & -\sin(\theta) & \cos(\theta) & h \sin(\theta) \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (39.30)$$

Putting both intrinsic and extrinsic camera parameters together we get the following projection matrix:

$$\mathbf{M} = \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a \cos(\theta) & a \sin(\theta) & -ah \cos(\theta) \\ 0 & -\sin(\theta) & \cos(\theta) & h \sin(\theta) \end{bmatrix} \quad (39.31)$$

Going back to heterogeneous coordinates we have:

$$x = a \frac{X}{\sin(\theta)(h - Y) + \cos(\theta)Z} \quad (39.32)$$

$$y = a \frac{\cos(\theta)(Y - h) + \sin(\theta)Z}{\sin(\theta)(h - Y) + \cos(\theta)Z} \quad (39.33)$$

Let's look at three special cases according to the camera angle θ :

- For $\theta = 0$, we recover the projection equations from the previous example. The horizon line is an horizontal line that passes by the center of the image.
- For $\theta = 90$ degrees (this is when the camera is looking downward), we get $x = aX/(h - Y)$ and $y = aZ/(h - Y)$. In this case, the equations are analogous to the standard projection equation with the distance to the camera being $h - Y$ and Z playing the role of Y .
- For arbitrary angles θ , the horizon line corresponds to the y location for a point in infinity, $Z \rightarrow \infty$. Using equation (39.33), the horizon line is located at $y = a \tan(\theta)$.

The sketch in [figure 39.12](#) shows a person taking a picture of two people standing at different distances from the camera. If you take a picture of two people with a similar height to you, h , standing in front of the camera. Then, we can set $Y = h$ in the previous equations, their eyes will be located at the position $y = a \tan(\theta)$ in the picture, which is independent of Z and the same vertical location as the horizon line.

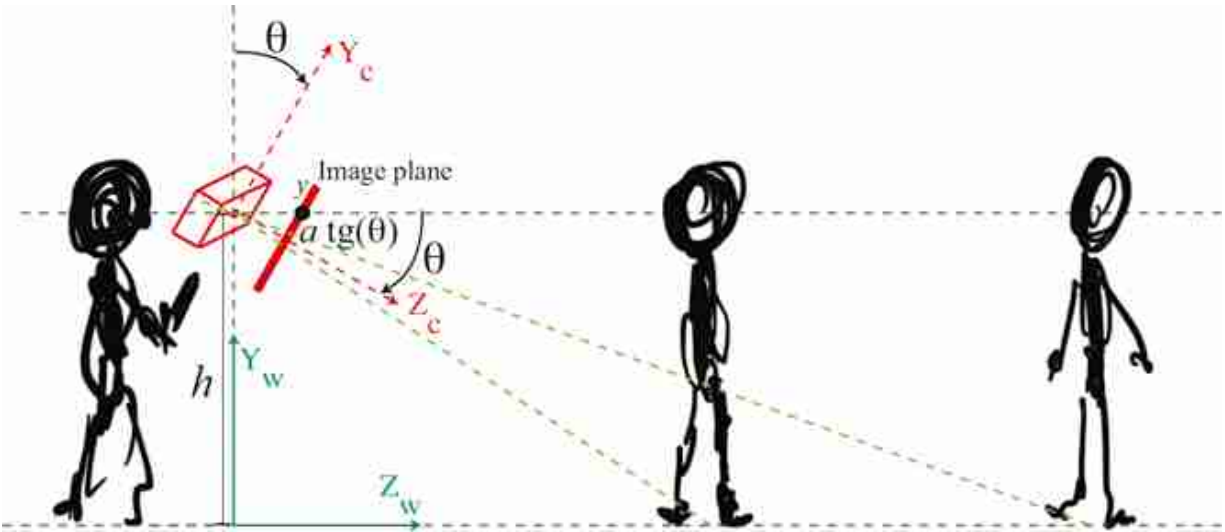


Figure 39.12: Sketch showing a person taking a picture of two people standing at different distances from the camera. Their eyes will project to the same image row regardless of their distance to the camera.

Note that if a is known (i.e., if the camera is calibrated), then we can infer θ by locating the horizon line in the image. Analogously, if the angle of the camera, θ , is known, we could estimate a .

The two pictures in [figure 39.13](#) are taken with two different camera angles. In both pictures, the camera's x -axis is kept parallel to the ground. The right and left pictures were taken at the same location with the camera tilted upward and downward, respectively. The horizontal red lines show the approximate location of the horizon line in each image and coincides also with the vertical location where the heads of the people walking appear on the picture.



Figure 39.13: The two pictures taken with two different camera angles. The red line indicates the position of the horizon line estimated as the horizontal location in which the two vanishing lines (blue) intersect.

These pictures were taken with the iPhone 13 pro, which we calibrated before. The intrinsic camera parameter we got is $a \approx 3,103$, which we can use to compute the angle θ . On the right, the location of the horizon line is at $y_h = -798$, with the center of the picture being the coordinates $(0, 0)$. The angle is $\theta = \arctan(y_h/a) = 14.6$ degrees. For the picture on the right the horizon line is at $y_h = 1,129$, resulting in an estimated camera angle of $\theta = 20$ degrees. Although we did not precisely measured the angle of the camera when we were taking these two pictures, both angles seem about right. We leave it to the reader to take two similar pictures to the ones in the example above, and get a precise measurement of the camera angle using a protractor and check if the estimated angle from the horizon line matches your measurement.

Example 3 can be useful to model typical imaging conditions. It results in a simple model with three parameters (h , a , and θ).

Example 4. The setting shown in shown in [figure 39.10\(d\)](#) is like the one we used in the simple visual system (chapter 2). Let's see if we recover the same equations!

Let's start computing the rotation matrix. Now we have rotation along two axes. We have a rotation around the X_w -axis with an angle $\theta_X = \theta$ and a rotation along the Y_w -axis with a $\theta_Y = 180$ degrees rotation.

$$\begin{aligned}
\mathbf{R} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta_X) & \sin(\theta_X) \\ 0 & -\sin(\theta_X) & \cos(\theta_X) \end{bmatrix} \begin{bmatrix} \cos(\theta_Y) & 0 & \sin(\theta_Y) \\ 0 & 1 & 0 \\ -\sin(\theta_Y) & 0 & \cos(\theta_Y) \end{bmatrix} \\
&= \begin{bmatrix} -1 & 0 & 0 \\ 0 & \cos(\theta_X) & -\sin(\theta_X) \\ 0 & -\sin(\theta_X) & -\cos(\theta_X) \end{bmatrix}
\end{aligned} \tag{39.34}$$

It is important to note that this form of dealing with rotations is more complex than it seems. Once we apply a rotation along one axis, the following rotation will be affected. That is, the order in which these rotations are executed matters. In this example, it works if we rotate along the Y -axis first. In general, it is better to use other conventions to write the rotation matrix.

The projection matrix is obtained by using both intrinsic and extrinsic camera parameters together, resulting in:

$$\begin{aligned}
\mathbf{M} &= \begin{bmatrix} a & 0 & 0 & 0 \\ 0 & a & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & \cos(\theta_X) & -\sin(\theta_X) & 0 \\ 0 & -\sin(\theta_X) & -\cos(\theta_X) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -h \\ 0 & 0 & 1 & -d \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} -a & 0 & 0 & 0 \\ 0 & a \cos(\theta) & -a \sin(\theta) & -ah \cos(\theta) + ad \sin(\theta) \\ 0 & -\sin(\theta) & -\cos(\theta) & h \sin(\theta) + d \cos(\theta) \end{bmatrix}
\end{aligned} \tag{39.35}$$

And going back to heterogeneous coordinates we have:

$$x = -a \frac{X}{\sin(\theta)(h - Y) + \cos(\theta)(d - Z)} \tag{39.36}$$

$$y = a \frac{\cos(\theta)(Y - h) + \sin(\theta)(d - Z)}{\sin(\theta)(h - Y) + \cos(\theta)(d - Z)} \tag{39.37}$$

This setting is similar to the one we used when studying the simple visual system from chapter 2. However, you will notice that the equations obtained are different. The reason is that in the simple visual system projection model we made two additional simplifying assumptions. The first simplifying assumption was that the use of parallel projection, while the previous equations use perspective projection. To get the right set of equations we should use the $\mathbf{K}$ matrix that corresponds to the parallel projection camera model, as shown in equation (39.4), which results in:

$$\mathbf{M} = \begin{bmatrix} -a & 0 & 0 & 0 \\ 0 & a \cos(\theta) & -a \sin(\theta) & -ah \cos(\theta) + ad \sin(\theta) \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (39.38)$$

The second constraint we had in the simple world chapter is that the origin of the world coordinates was the point where the camera optical axis intersected the ground plane, this puts a particular restriction between the values of d , h , and θ . Concretely, the constraint is, $\tan(\theta) = h/d$. Therefore, $d \sin(\theta) - h \cos(\theta) = 0$. In heterogeneous coordinates this results in:

$$\bar{x} = -aX \quad (39.39)$$

$$\bar{y} = a \cos(\theta)Y - a \sin(\theta)Z \quad (39.40)$$

These equations are similar to equation (2.2) from chapter 2 (with $a = 1$). One remaining difference is that x has the opposite sign. This is because we are using here a different convention for the sign of the x -axis of the image coordinates. This is fine, and in fact, books and conference publications use a wide set of conventions, and you should be ready to adapt the formulation to each setting.

As you can see, working with heterogeneous coordinates can be tedious. However, in most cases we will work directly with the camera matrix $\mathbf{K}$ and/or the projection matrix $\mathbf{M}$ without needing to derive their equations. These matrices will be obtained during the camera calibration process.

39.7 Camera Calibration

A camera is said to be **calibrated** if we know the transformations relating 3D world coordinates to pixel coordinates within the camera. Those transformations involve two types of parameters: (1) parameters that depend on where the camera is physically located in the world, and (2) parameters that are a function of the camera itself. These two types of parameters are called extrinsic and intrinsic parameters, respectively.

39.7.1 Direct Linear Transform

Let's assume that we know the exact locations of several 3D points in our scene in world coordinates and we also know the locations in which those 3D points project into the image. Can we use them to recover the intrinsic and extrinsic camera parameters?

If we have six correspondences, then, under certain conditions, we can recover the projection matrix $\mathbf{M}$ by solving a linear system of equations.

For each pair of corresponding pixels, $\mathbf{p}_i$, and 3D points, $\mathbf{P}_i$, we have the following relationship:

$$\mathbf{p}_i = \mathbf{M} \mathbf{P}_i = \begin{bmatrix} \mathbf{m}_0^\top \\ \mathbf{m}_1^\top \\ \mathbf{m}_2^\top \end{bmatrix} \mathbf{P}_i \quad (39.41)$$

where $\mathbf{M}$ is a 3×4 projection matrix. To simplify the derivation that comes next, it is convenient to write the matrix $\mathbf{M}$ in terms of its three rows using the vectors $\mathbf{m}_0^\top$, $\mathbf{m}_1^\top$, and $\mathbf{m}_2^\top$. With this notation, the heterogeneous coordinates of $\mathbf{p}$ are:

$$x_i = \frac{\mathbf{m}_0^\top \mathbf{P}_i}{\mathbf{m}_2^\top \mathbf{P}_i} \quad (39.42)$$

$$y_i = \frac{\mathbf{m}_1^\top \mathbf{P}_i}{\mathbf{m}_2^\top \mathbf{P}_i} \quad (39.43)$$

By rearranging the terms, we get the following two linear equations on the parameters of the matrix $\mathbf{M}$:

$$\mathbf{P}_i^\top \mathbf{m}_0 - x_i \mathbf{P}_i^\top \mathbf{m}_2 = 0 \quad (39.44)$$

$$\mathbf{P}_i^\top \mathbf{m}_1 - y_i \mathbf{P}_i^\top \mathbf{m}_2 = 0 \quad (39.45)$$

The same two equations written in matrix form are:

$$\begin{bmatrix} -\mathbf{P}_i^\top & \mathbf{0} & x_i \mathbf{P}_i^\top \\ \mathbf{0} & -\mathbf{P}_i^\top & y_i \mathbf{P}_i^\top \end{bmatrix} \begin{bmatrix} \mathbf{m}_0 \\ \mathbf{m}_1 \\ \mathbf{m}_2 \end{bmatrix} = \mathbf{0} \quad (39.46)$$

Now, if we stack together all the equations derived from N correspondences we get the following homogeneous linear system of equations:

$$\begin{bmatrix} -\mathbf{P}_1^\top & \mathbf{0} & x_1 \mathbf{P}_1^\top \\ \mathbf{0} & -\mathbf{P}_1^\top & y_1 \mathbf{P}_1^\top \\ \vdots & \vdots & \vdots \\ -\mathbf{P}_N^\top & \mathbf{0} & x_N \mathbf{P}_N^\top \\ \mathbf{0} & -\mathbf{P}_N^\top & y_N \mathbf{P}_N^\top \end{bmatrix} \begin{bmatrix} \mathbf{m}_0 \\ \mathbf{m}_1 \\ \mathbf{m}_2 \end{bmatrix} = \mathbf{0} \quad (39.47)$$

This can be summarized as:

$$\mathbf{A} \mathbf{m} = \mathbf{0} \quad (39.48)$$

where $\mathbf{A}$ is a $2N \times 12$ matrix. The vector $\mathbf{m}$ contains all the elements of the matrix $\mathbf{M}$ stacked as a column vector of length 12. Note that $\mathbf{m}$ only has 11

degrees of freedom as the results do not change for a global scaling of all the values.

Rearranging the vector $\mathbf{m}$ into the matrix $\mathbf{M}$ is a potential source of bugs. Try first simulating some 3D points and project them with a known $\mathbf{M}$ matrix, and check that you recover the same values (up to a scale factor).

As the measurements of the points in correspondence will be noisy, it is generally useful to use more than six correspondences. Therefore, the solution will not be 0. The solution is the vector that minimizes the residual $\|\mathbf{A}\mathbf{m}\|^2$, which can be obtained as the smallest eigenvector of the matrix $\mathbf{A}^T\mathbf{A}$. Once we estimate $\mathbf{m}$ we can rearrange the terms into the projection matrix $\mathbf{M}$. This method to obtain the projection matrix is called the **direct linear transform** (DLT) algorithm. This method requires that not all of the N points lie in a single plane [187].

The term $\|\mathbf{A}\mathbf{m}\|^2$ doesn't have any particular physical meaning, but minimizing it provides a good first guess that can be improved by other methods as we will discuss in section 39.7.4.

39.7.2 Recovering Intrinsic and Extrinsic Camera Parameters

Once $\mathbf{M}$ is estimated, we would like to recover the intrinsic camera parameters, and also camera translation and rotation. Can it be done? It turns out that the answer is yes!

Let's say that you start with a 3×4 matrix $\mathbf{M}$ obtained by calibrating the camera using DLT (or any other calibration method), and you want to extract the matrices $\mathbf{K}$, $\mathbf{R}$, and $\mathbf{T}$. It is possible to decompose the matrix $\mathbf{M}$ such that it can be written as:

$$\mathbf{M} = [\mathbf{KR} \mid -\mathbf{KRT}] \quad (39.49)$$

It turns out that, due to the characteristics of those three matrices, one can find such decomposition. The first step is to recover $\mathbf{T}$. This can be done by seeing that the matrix $\mathbf{M}$ can be written as the concatenation of a 3×3 matrix, $\mathbf{B}$, and 1×3 column vector $\mathbf{b}$; that is, $\mathbf{M} = [\mathbf{B} \mid \mathbf{b}]$. The first matrix is $\mathbf{B} = \mathbf{KR}$, and the vector is $\mathbf{b} = -\mathbf{KRT}$. Therefore, we can estimate the camera translation as:

$$\mathbf{T} = -\mathbf{B}^{-1}\mathbf{b} \quad (39.50)$$

The next step is to decompose $\mathbf{B}$ into the product of two matrices $\mathbf{KR}$. These two matrices are very special. The matrix $\mathbf{K}$ is an upper triangular matrix by construction (in the case of perspective projection), and $\mathbf{R}$ is an orthonormal matrix. To estimate $\mathbf{K}$ and $\mathbf{R}$ we can use the QR decomposition of the matrix $\mathbf{B}^{-1}$. The QR decomposition decomposes a matrix in the product of a unitary matrix and an upper triangular one, which is the reverse order of what we want. This is why we need to use the inverse of $\mathbf{B}$.

The decomposition obtained with the QR decomposition is not unique. In fact, we can change the sign of both matrices and still get the same decomposition. Also, changing the sign of column i in $\mathbf{K}$ and the sign of the column i in $\mathbf{R}$ also results in the same product. As we know that the diagonal elements of the $\mathbf{K}$ have to be positive we can change the sign of the columns of $\mathbf{K}$ with a negative entry in the diagonal element and also change the sign of the corresponding row on the rotation matrix $\mathbf{R}$.

This method to recover the camera parameters might be quite sensitive to noise, but it can be used to initialize other methods based on minimizing the **reprojection error**.

39.7.3 Multiplane Calibration Method

A popular method for camera calibration is to use a planar calibration chart and to take multiple pictures of it by placing the chart in different locations and orientations relative to a fixed camera. This method was introduced by Zhang's in 1999 [[526](#)] and it is still commonly used.

39.7.4 Nonlinear Optimization by Minimizing Reprojection Error

The reprojection error is illustrated in [figure 39.14](#). The reprojection error is the difference between the estimated image coordinates of a 3D point and its ground-truth image coordinates.

We are given 3D points, $\mathbf{P}_i$, and their corresponding 2D projections, $\mathbf{p}_i$. We start with some estimated camera parameters $\mathbf{K}$, $\mathbf{R}$, and $\mathbf{T}$. We can use these parameters to project the 3D points and get estimated 2D locations, $\hat{\mathbf{p}}_i$, of their projections. The reprojection error is the distance between those points, as shown in [figure 39.14](#).

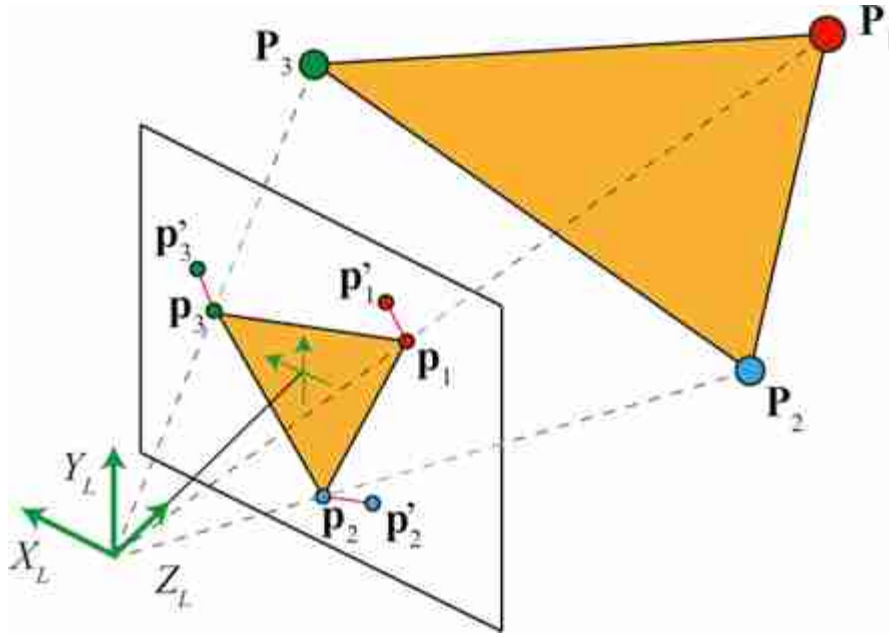


Figure 39.14: Reprojection error indicated by the red lines on the image plane.

The reprojection error can be evaluated with the loss:

$$\sum_{i=1}^N \|p_i - p'_i\|^2 = \sum_{i=1}^N \|p_i - \pi(K [R | -RT] P_i)\|^2 \quad (39.51)$$

where $\pi()$ is the function that transforms homogeneous coordinates to heterogeneous coordinates. This function can also incorporate other camera parameters to account for geometric distortion introduced by the camera optics.

The reprojection loss (equation [39.51]) can be minimized by gradient descent, optimizing the three matrices, K , R , and T , directly. For the minimization to converge it is usually initialized with the solution obtained using DLT.

39.7.5 A Toy Example

The last few pages were full of math, and although everything seems sound, it is good to remain suspicious about what subtleties might be hidden behind the math that might make everything break. Can we run a simple test to see if the theory meets the practice?

We wanted to try it out so here is what we did. We started by taking a picture of one office and measure some distances as shown in [figure 39.15](#).

The goal is to get real 3D coordinates measured in the world and use them to calibrate the camera.

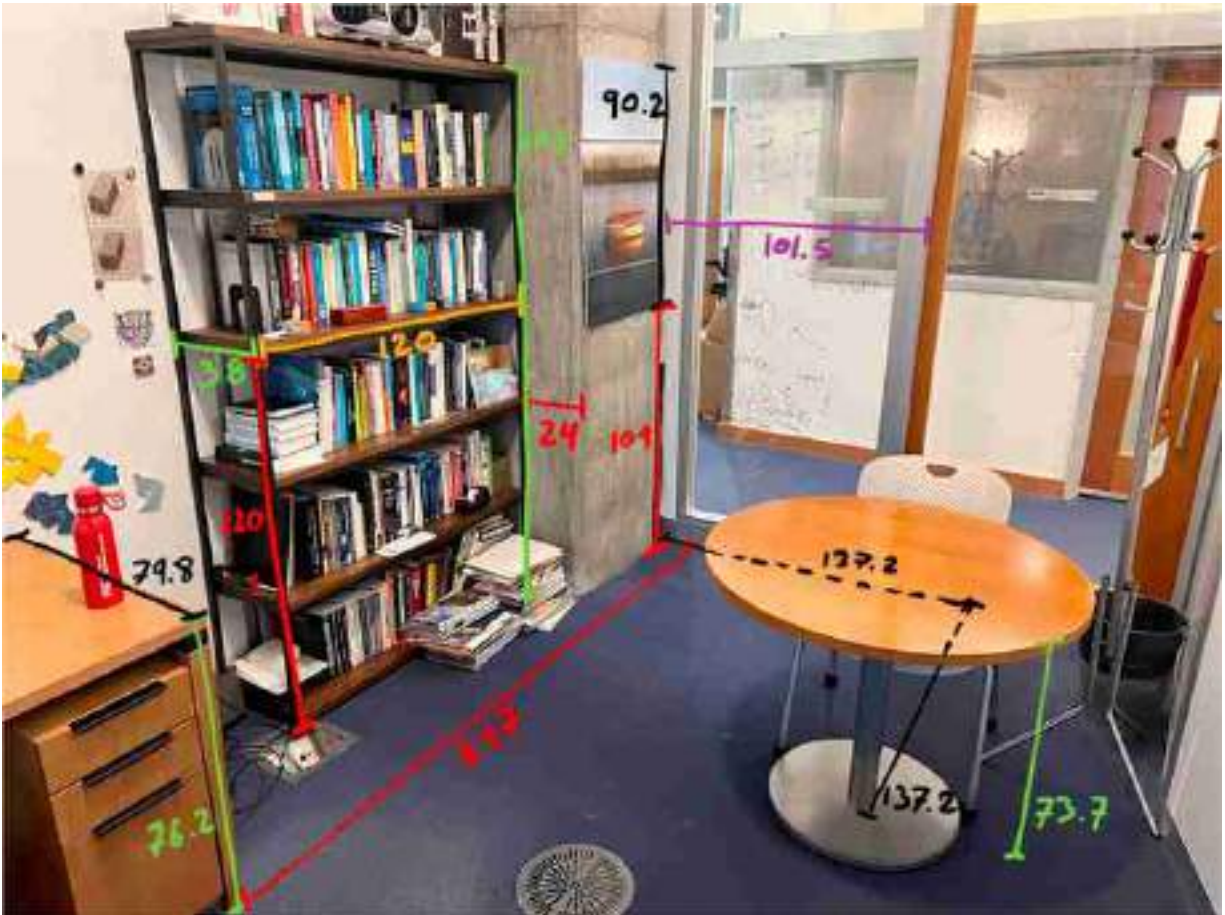


Figure 39.15: Office picture and real distances between a sparse set of points measured in centimeters.

We took a picture, [figure 39.15](#), with the same iPhone we used in section 39.3.3, holding the camera at eye height (which is around 67 in, or 170 cm above the ground). The camera was looking downward at an angle of around 15 degrees with respect to the vertical. Would we be able to recover these extrinsic camera parameters, together with the intrinsic camera parameters, using the theory described in this chapter?

First we need to decide where we will place the origin of the world coordinates and the axis direction. As shown in the following picture, we place the origin on the ground floor, in the corner between the column and

the glass wall in the back. Now we can get a list of 3D points from the office measurements and their corresponding 2D coordinates. The following figure shows a list of easily identifiable 3D points and their corresponding image coordinates. The image coordinates are approximated also and are collected manually. We use Adobe Photoshop to read the pixel coordinates.

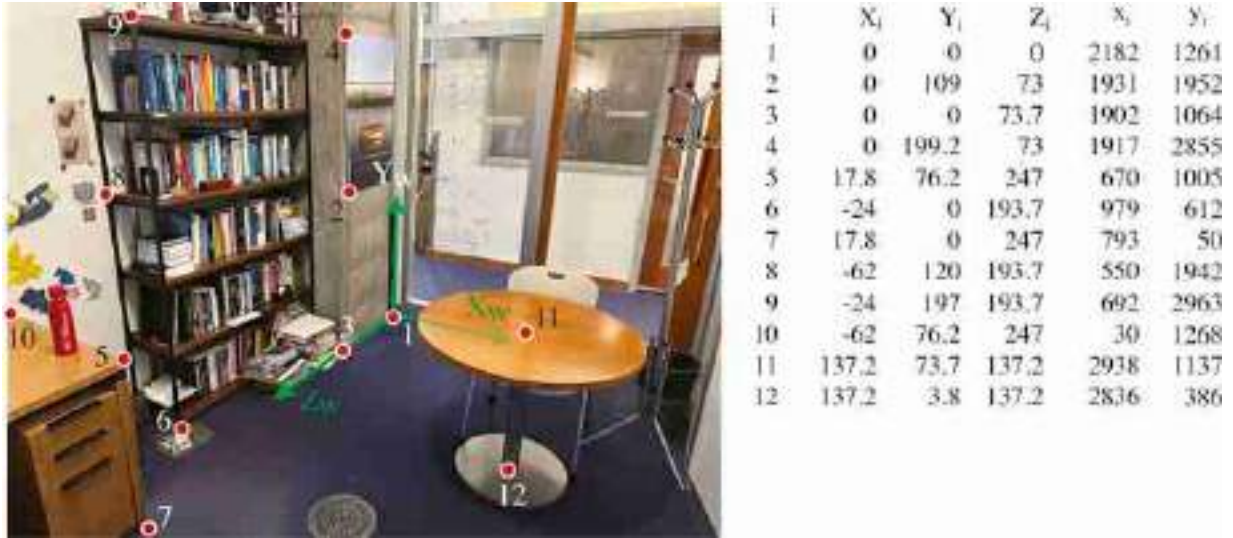


Figure 39.16: Office picture and a table with the 3D world coordinates of 12 points, extracted using the measurements from [figure 39.15](#), and their corresponding 2D image coordinates measured in pixels.

Photoshop has the image coordinates origin on the top left. Our convention has been to put the origin on the bottom and the x -axis point from right to left. Therefore, we need to change the coordinates first (or just deal with it later). It is just simple to replace the image coordinates by $y_i \leftarrow 3,024 - y_i$ and $x_i \leftarrow 4,032 - x_i$.

In order to estimate the projection matrix we need a minimum of eight correspondences. Here we have 12, which will help to reduce the noise (and we assure you there is a lot of noise on those manual measurements). Applying the DLT algorithm we recover the following projection matrix:

$$\mathbf{M} = \begin{bmatrix} -8.1 & -1.8 & -0.2 & 1,845.5 \\ -3 & 5.4 & -4.8 & 1,262.5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (39.52)$$

The matrix has been scaled so that the bottom-right element is a 1. Using this matrix, the mean reprojection error is 12.3 pixels, which is reasonably small for an image of size $4,032 \times 3,024$ pixels. That is, the reprojection error is < 0.5 percent of the image height.

We now need to recover the intrinsic and extrinsic camera parameters. Using equation (39.50) we recover first the translation vector which gives us $\mathbf{T} = (182.3; 171.8; 347.6)$. From our definition of the world-coordinates frames, the middle element of $\mathbf{T}$ is the camera height with respect to the ground plane. The value of 171.8 cm is very close to our initial guess! Given a scene with known measurements and a picture, one can find out details about the camera and also about the observer.

A picture reveals information about the camera and the photographer, even when there is no one on the picture. It is important to keep this in mind when thinking about privacy.

Let's now see if we can recover the camera angle and the intrinsics. Using the QR decomposition as described in section 39.7.2, we get

$$\mathbf{K} = \begin{bmatrix} 2,960 & -24.9 & 1,979.7 \\ 0 & 3,019 & 1,433.6 \\ 0 & 0 & 1 \end{bmatrix} \quad (39.53)$$

The values are very close to the ones from equation (39.12) which were obtained using a calibration pattern and also from the camera technical specifications. The off-diagonal elements, which would account for a skew pixel shape in the camera, are small relative to the focal length and probably just due to the annotation noise. Also, both scaling factors are very similar, and their difference might just be due to noise. The camera is likely to have square pixels. The last column is the location of the optical image center, as we defined the image coordinates with the origin in the bottom right instead of that in the image center, and is quite close to what one would expect: $[W/2, H/2, 1]$.

Now let's look at the camera orientation. The rotation matrix obtained with the QR decomposition is

$$\mathbf{R} = \begin{bmatrix} -0.8576 & -0.0162 & 0.5141 \\ 0.1928 & -0.9368 & 0.2921 \\ -0.4769 & -0.3496 & -0.8064 \end{bmatrix} \quad (39.54)$$

The viewing direction of the camera can be obtained as $\mathbf{R}^T[0, 0, 1]^T$ (this is the direction of the Z -axis in the camera-coordinates system).

Another potential bug when extracting the camera orientation is to use the rotation matrix instead of its inverse. We defined $\mathbf{R}^T$ as the rotation needed to move the world coordinate axes to be aligned with the camera-coordinate axes.

The angle of the camera with respect to the vertically can be computed in multiple ways. Here is one:

$$\begin{aligned} \arccos([0, 1, 0]\mathbf{R}^T[0, 1, 0]^T) &= \arccos(R[2, 2]) \\ &= \arccos(-0.9368) \\ &= 20.5 \text{ degrees} \end{aligned} \quad (39.55)$$

Here we just computed the camera Y -axis and then computed the dot product with a unit vector along the Y -axis of the world-coordinates. This angle is also quite close to the approximate value that we gave at the beginning of the section.

It is always important to visualize everything to make sure that all is correct. [Figure 39.17](#) shows a visualization of the annotated 3D points and the inferred camera location and orientation. The figure shows three different viewpoints.

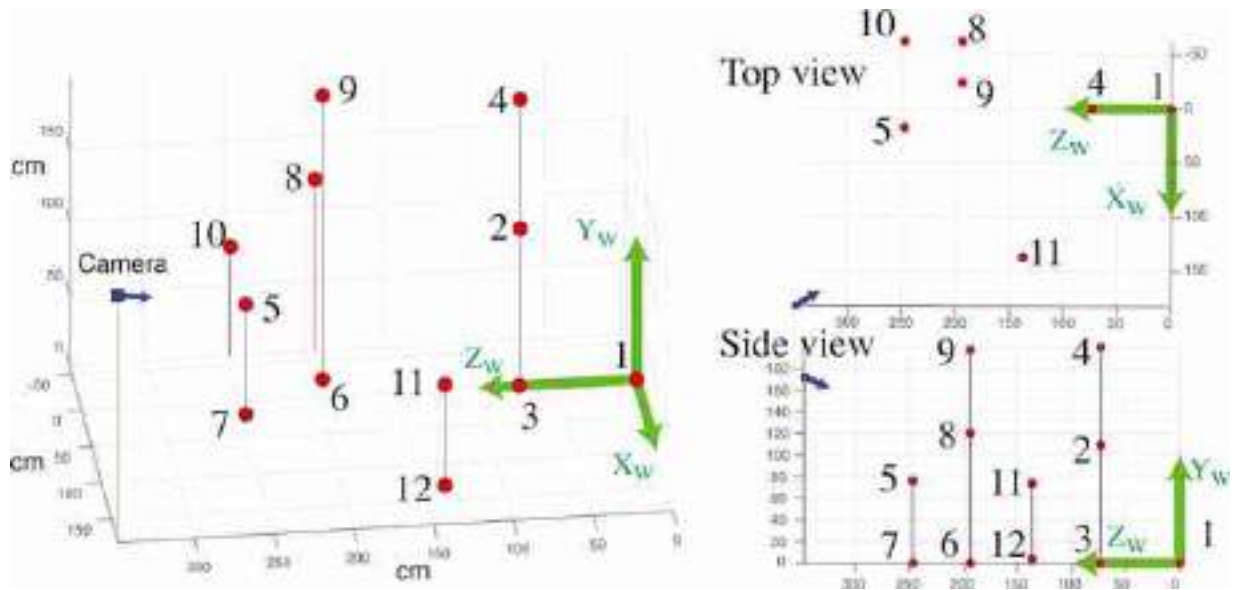


Figure 39.17: Inferred camera location for the office picture. The figure shows three different viewpoints.

39.8 Concluding Remarks

In this chapter we have developed a model for describing how points in the 3D world project into the image. For a more in depth study of the topics described in this chapter, the reader should consult specialized books such as [477] and [187].

There are a number subtleties involved in computing the quantities we have presented in this chapter. Although there are packages that will calibrate a camera and compute the projection matrix for you, it is useful to be familiar with how these methods work as they play an important role when building unsupervised learning methods. Unsupervised learning often demands familiarity with the constraints of the vision problem, which become useful when trying to design the loss functions that do not require human annotated data.

But the goal of vision is to interpret the scene, that is, to recover the 3D scene from images. We will discuss in the next chapters how to do this.

40 Stereo Vision

40.1 Introduction

[Figure 40.1](#)(a) shows a **stereo anaglyph** of the ill-fated ocean liner, the Titanic, from [329]. This was taken with two cameras, one displaced laterally from the other. The right camera's image appears in red in this image, and the left camera image is in cyan. If you view this image with a red filter covering the left eye, and a cyan filter covering the right ([figure 40.1](#)[b]) then the right eye will see only the right camera's image (analogously for the left eye), allowing the ship to pop-out into a three-dimensional (3D) stereo image. Note that the relative displacement between each camera's image of the smokestacks changes as a function of their depth, which allows us to form a 3D interpretation of the ship when viewing the image with the glasses ([figure 40.1](#)[b]).



Figure 40.1: (a) Stereo anaglyph of the Titanic [329]. The red image shows the right eye's view, and cyan the left eye's view. (b) When viewed through red/cyan stereo glasses the cyan variations appear in the left eye and the red variations appear to the right eye, creating a perception of 3D.

In this chapter, we study how to compute depth from a pair of images from spatially offset cameras, such as those of [figure 40.1](#). There are two

parts to the depth computation, usually handled separately: (1) analyzing the geometry of the camera projections, which allows for triangulating depth once image offsets are known; and (2) calculating the offset between matching parts of the objects depicted in each image. Different techniques are used in each part. We first address the geometry, asking where points matching those in a first image can lie in the second image. A manipulation, called **image rectification**, is often used to place the locus of potential matches along a pixel row. Then, assuming a rectified image pair, we will address the second part of the depth computation, computing the offsets between matching image points.

40.2 Stereo Cues

How can we estimate depth from two pictures taken by two cameras placed at different locations? Let's first gain some intuition about the cues that are useful to estimate distances from two views captured at two separate observation points.

40.2.1 How Far Away Is a Boat?

To give an intuition on how depth from stereo vision works, let's first look at a simple geometry problem of practical use: How do we estimate how far away a boat is from the coast?

Sailors can use tricks to estimate distances using methods related to the ones presented in this chapter. These techniques can be useful when the electronics on the boat are out.

We can solve this problem in several ways. We will briefly discuss two methods. The first one uses a single observation point but will use the horizon line as a reference. The second method will use two observation points. Both methods are illustrated in [figure 40.2](#).

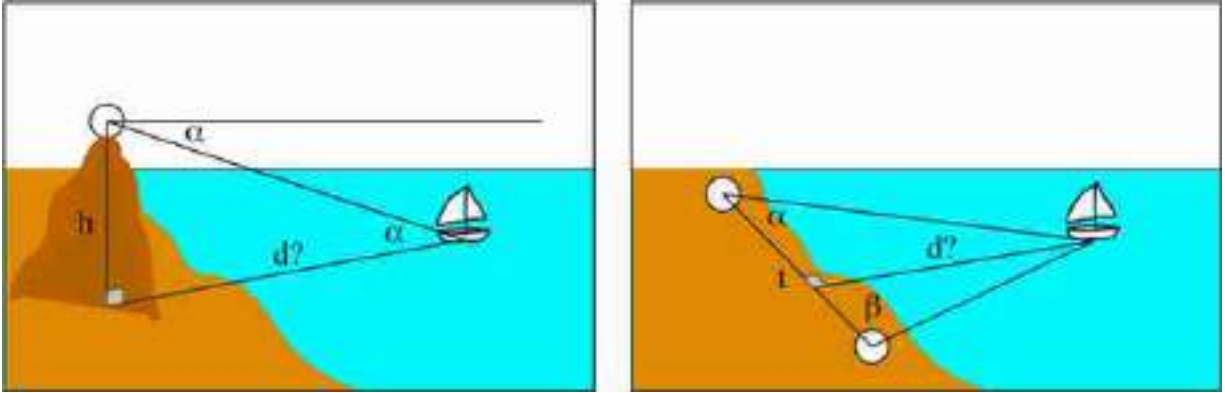


Figure 40.2: Two methods to estimate the distance of a boat from the coast. (left) The first method uses a single observation point, with knowledge of the observer's height above the water. (right) The second method uses two observation points.

The first method measures the location of the boat relative to the horizon line. An observer is standing on top of an elevated point (a mountain, tower, or building) at a known height, h . The observer measures the angle α between the horizontal (obtained by pointing toward the horizon) and the position of the boat. We can derive the boat distance to the base of the point of observation, d by the simple trigonometric relation:

$$d = \frac{h}{\tan(\alpha)} \quad (40.1)$$

This method is not very accurate and might require high observation points, which might not be available, but it allows us to estimate the boat position with a single observation point, if the observer's height above the water is known. It also requires incorporating the earth curvature when the boat is far away.

The second method requires two different observation points along the coast separated by a distance t . At each point, we measure the angles, α and β , between the direction of the boat and the line connecting the two observation points. Then we can apply the following relation:

$$d = t \frac{\sin(\alpha) \sin(\beta)}{\sin(\alpha + \beta)} \quad (40.2)$$

This method is called **triangulation**. For it to work we need a large distance t so that the angles are significantly different from 90 degrees.

40.2.2 Depth from Image Disparities

When observing the world with two eyes we get two different views of the world ([figure 40.1](#)). The relative displacement of features across the two images (**parallax effect**) is related to the distance between the observer and the observed scene. Just as in the example of estimating the distance to boats in the sea, our brain uses disparities across the two eye views to measure distances to objects.

Free fusion consists in making the eyes converge or diverge in order to fuse a stereogram without needing a stereoscope. Making it work requires practice. Try it in [figure 40.3](#)

Our brain will use many different cues such as the ones presented in the previous chapter. But image disparity provides an independent additional cue. This is illustrated by the **random dot stereograms** ([figure 40.3](#)) introduced by Bela Julesz [[241](#)]. Random dot stereograms are a beautiful demonstration that humans can perceive depth using the relative displacement of features across two views even in the absence of any other depth cues. The random images contain no recognizable features and are quite different from the statistics of the natural world. But this seems not to affect our ability to compute the displacements between the two images and perceive 3D structure.

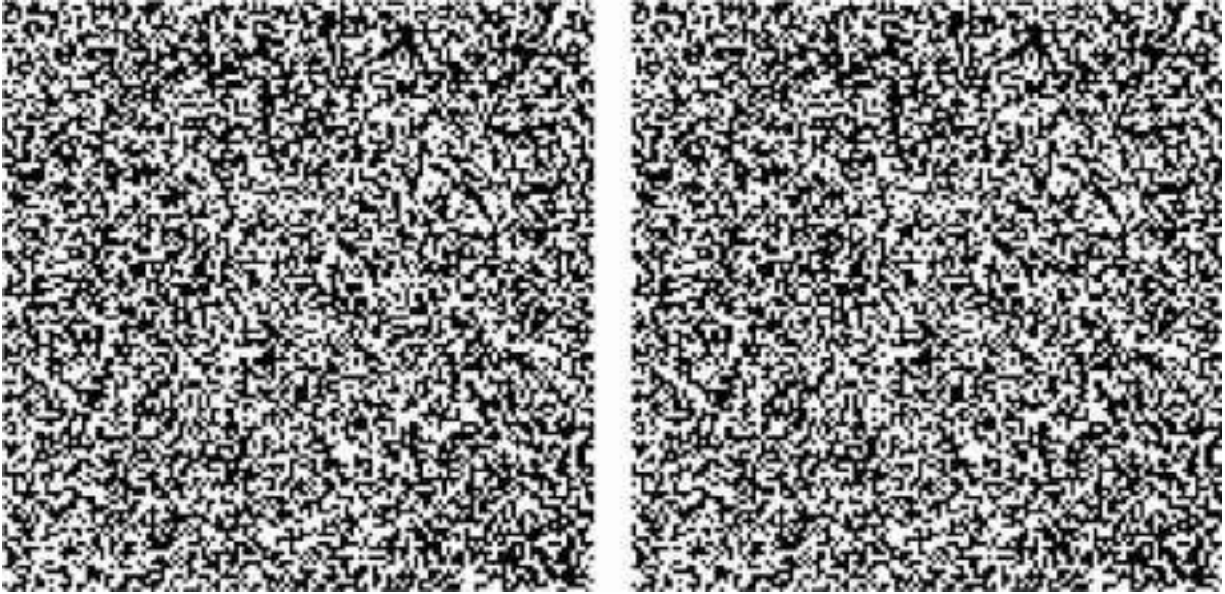


Figure 40.3: Random dot stereogram. Both images are almost identical. The difference is that there is a square in the middle that is displaced by a few pixels left-right between the two images. When each image is seen by one eye you should see a square floating in front of a flat background. You can use a stereoscope or free fusion to see it.

40.2.3 Building a Stereo Pinhole Camera

Stereo cameras generally are built by having two cameras mounted on a rigid rig so that the camera parameters do not change over time.

We can also build a **stereo pinhole camera**. As we discussed in chapter 5, a pinhole camera is formed by a dark chamber with a single hole in one extreme that lets light in. The light gets projected into the opposite wall, forming an image. One interesting aspect of pinhole cameras is that you can build different types of cameras by playing with the shape of the pinhole. So, let's make a new kind of pinhole camera! In particular, we can build a camera that produces anaglyph images, like the one in [figure 40.1](#), in a single shot by making two pinholes instead of just one.

We will transform the pinhole camera into an anaglyph pinhole camera by making two holes and placing different color filters on each hole ([figure 40.4](#)). This will produce an image that will be the superposition of two images taken from slightly different viewpoints. Each image will have a different color. Then, to see the 3D image we can look at the picture by placing the same color filters in front of our eyes. In order to do this, we used color glasses with blue and red filters.

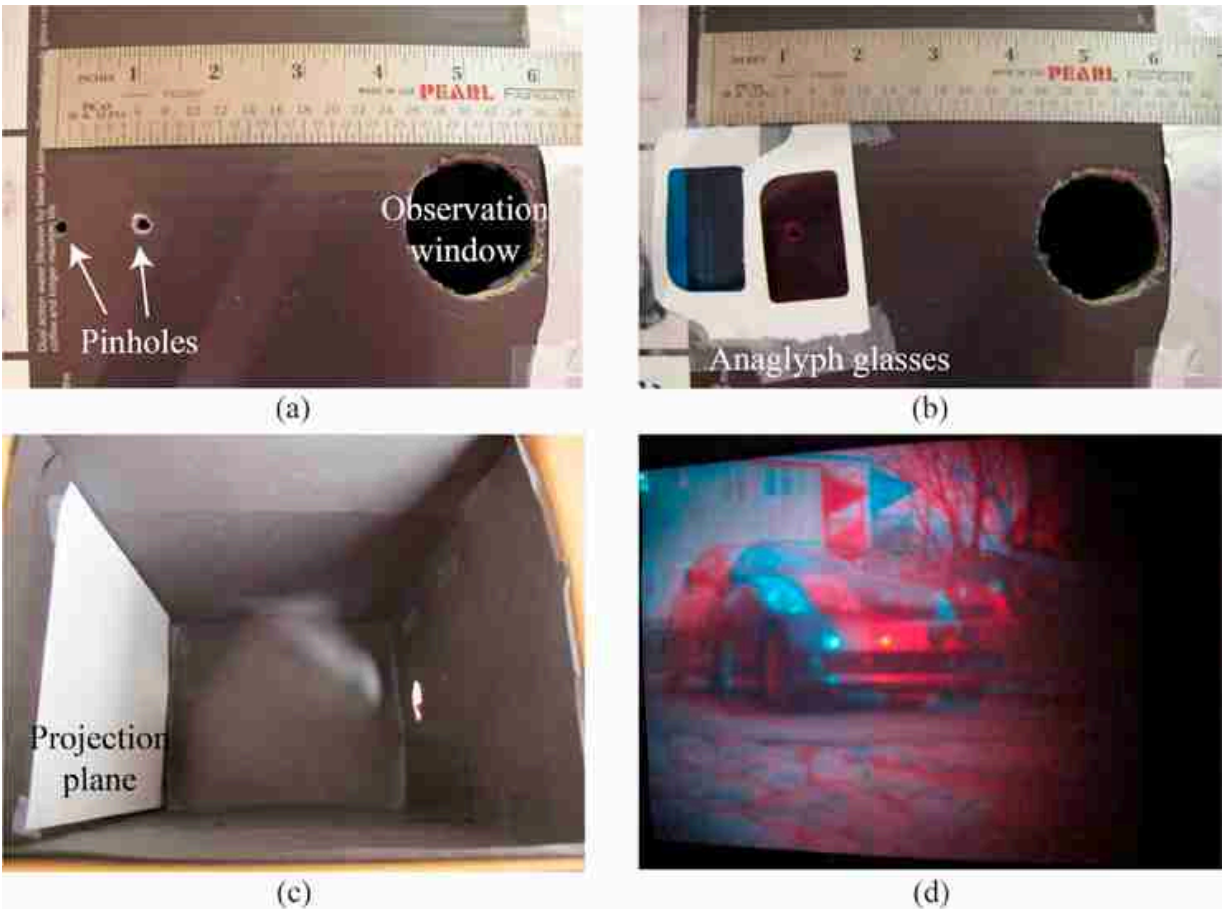


Figure 40.4: One way of playing with pinhole cameras is to build an anaglyph pinhole camera. The anaglyph pinhole camera captures stereo images by projecting an anaglyph image on the projection plane.

We used one of the anaglyph glasses to get the filters and placed them in front of the two pinholes. [Figure 40.4](#) shows two pictures of the two pinholes ([figure 40.4\[a\]](#)) and the filters positioned in front of each pinhole ([figure 40.4\[b\]](#)). [Figure 40.4\(d\)](#) is a resulting anaglyph image that appears on the projection plane ([figure 40.4\[c\]](#)); this picture was taken by placing a camera in the observation hole seen at the right of [figure 40.4\(c\)](#). [Figure 40.4\(d\)](#) should give you a 3D perception of the image when viewed with anaglyph glasses.

Here are experimental details for making the stereo pinhole camera. You will have to experiment with the distance between the two pinholes. We tried two distances: 3 in and 1.5 in. We found it easier to see the 3D for the 1.5 in images. Try to take good quality images. Once you have placed the color filters on each pinhole, you will need longer exposure times in order

to get the same image quality than with the regular pinhole camera because the amount of light that enters the camera is smaller. If the images are too blurry, it will be hard to perceive 3D. Also, as you increase the distance between the pinholes, you will increase the range of distances that will provide 3D information because the parallax effect will increase. But if the images are too far apart, the visual system will not be able to fuse them and you will not see a 3D image when looking with the anaglyph glasses.

Let's now look into how to use two views to estimate the 3D structure of the observed scene.

40.3 Model-Based Methods

Let's now make more concrete the intuitions we have built in the previous sections. We will start describing some model-based methods to estimate 3D from stereo images. Studying these models will help us understand the sources of 3D information, the constraints, and limitations that exist.

40.3.1 Triangulation

The task of stereo is to triangulate the depth of points in the world by comparing the images from two spatially offset cameras.

If more than two cameras are used, the task is referred to as multiview stereo.

To triangulate depth, we need to describe how image positions in the stereo pair relate to depth in the 3D world.

We will start with a very simple setting where both cameras are identical (same intrinsic parameters) and one is translated horizontally with respect to the other so that their optical axes are parallel, as shown in [figure 40.5](#). We also will assume calibrated cameras. The geometry shown in [figure 40.5](#) reveals the essence of depth estimation from the two cameras. Let's assume that there is one point, $\mathbf{P}$, in 3D space and we know the locations in which it projects on each camera, $[x_L, y_L]^T$ for the left camera and $[x_R, y_R]^T$ for the right camera, as shown in [figure 40.5\(a\)](#). Because the two cameras are aligned horizontally, the y -coordinates for the projection in both cameras will be equal, $y_L = y_R$. Therefore, we can look at the top view of the setting

as shown in [figure 40.5\(b\)](#). We consider the depth estimate within a single image row.

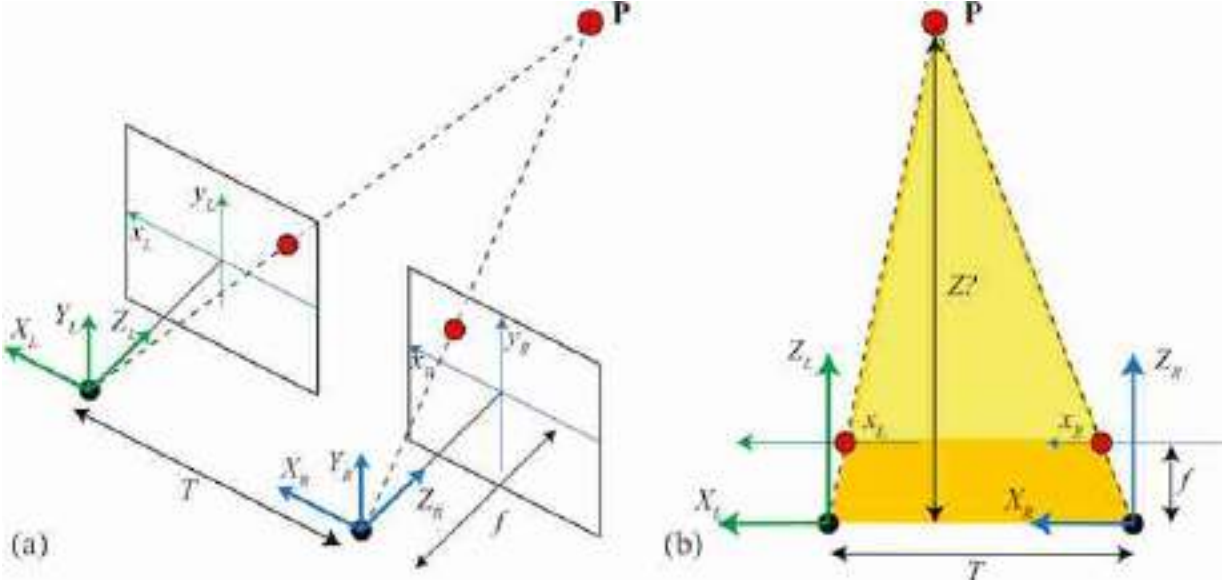


Figure 40.5: Simple stereo. (a) Two cameras, of identical focal length, f , and separated by an offset, T , image the point P onto the two-dimensional (2D) points $[x_L, y_L]^T$ and $[x_R, y_R]^T$ at each camera. (b) The similarity of the two triangles in leads to equation (40.4) for the depth, Z , of the point P .

The point P , for which we want to estimate the depth, Z , appears in the left camera at position x_L and in the right camera at position x_R .

To distinguish 2D image coordinates from 3D world coordinates, for this chapter, we will denote 2D image coordinates by lowercase letters, and 3D world coordinates by uppercase letters.

A simple way to triangulate the depth, Z , of a point visible in both images is through similar triangles. The two triangles (yellow triangle and yellow+orange triangle) shown in [figure 40.5\(b\)](#) are similar, and so the ratios of their height to base must be the same. Thus we have:

$$\frac{T + x_R - x_L}{Z - f} = \frac{T}{Z} \quad (40.3)$$

Solving for Z gives:

$$Z = \frac{fT}{x_L - x_R} \quad (40.4)$$

The difference in horizontal position of any given world point, $\mathbf{P}$, as seen in the left and right camera images, $x_L - x_R$, is called the **disparity**, and is *inversely proportional to the depth* Z of the point $\mathbf{P}$ for this configuration of the two cameras—parallel orientation. The task of inferring depth of any point in either image of a stereo pair thus reduces to measuring the disparity everywhere.

However, we are far from having solved the problem of perceiving 3D from two stereo images. One important assumption we have made is that we know the two corresponding projections of the point $\mathbf{P}$. But in reality the two images will be complex and we will not know which points in one camera correspond to the same 3D point on the other camera. This requires solving the **stereo matching** problem.

40.3.2 Stereo Matching

First, let's examine the task visually. [Figure 40.6](#) shows two images of an office, taken approximately 1 m apart. The locations of several objects in [figure 40.6\(a\)](#) are shown in [figure 40.6\(b\)](#), revealing some of the disparities that we seek to measure automatically. As the reader can appreciate by comparing the stereo images by eye, one can compute the disparity by first localizing a feature point in one image and then finding the corresponding point in the other image, based on the visual similarity of the local image region. To do this automatically is the crux of the stereo problem.

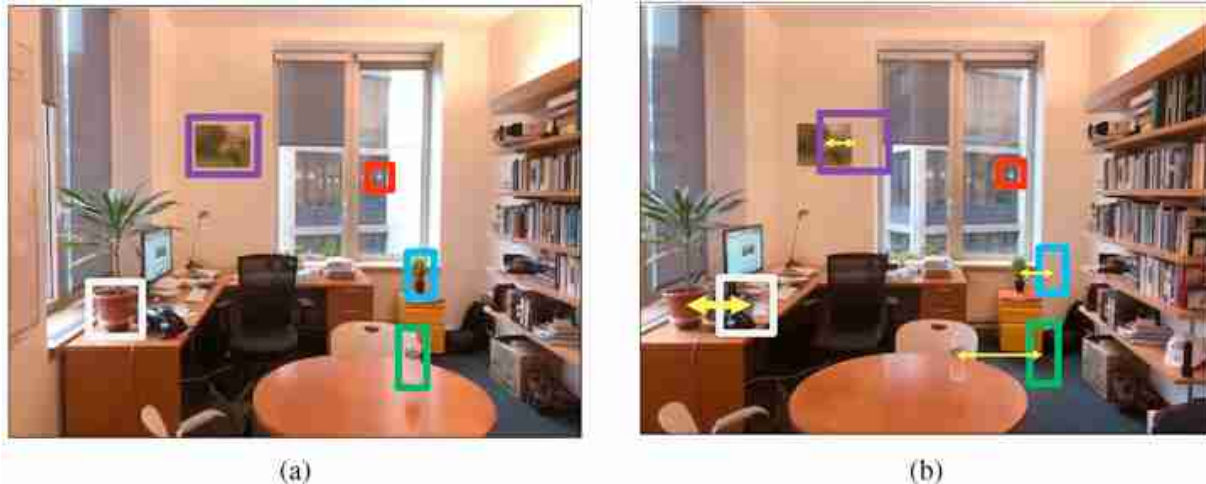


Figure 40.6: Two cameras, displaced from each other laterally, photograph the same scene, resulting in images (a) and (b). Colored rectangles show some identifiable features in image (a), and where those features appear in image (b). Arrows show the corresponding displacements, which reveal depth through equation (40.4).

A naive algorithm for finding the disparity of any pixel in the left image is to search for a pixel of the same intensity in the corresponding row of the right image. [Figure 40.7](#), from [220], shows why that naive algorithm won't work and why this problem is hard. [Figure 40.7\(a\)](#) shows the intensities of one row of the right-eye image of a stereo pair. The squared difference of the intensity differences from offset versions of the same row in the rectified left-eye image of the stereo pair is shown in [figure 40.7\(b\)](#), for a range of disparity offsets, plotted vertically, in a configuration known as the **disparity space image** [425]. The disparity corresponding to the minimum squared error matches are plotted in red. Note the disparities of the best pixel intensity matches show much more disparity variations than would result from the smooth surfaces depicted in the image; the local intensity matches don't reliably indicate the disparity. Many effects lead to that unreliability, including: lack of texture in the image, which leads to noisy matches among the many nearly identical pixels; image intensity noise; specularities in the image which change location from one camera view to another; and scene points appearing in one image but not another due to occlusion. [Figure 40.7\(d\)](#) shows an effective approach to remove the matching noise: blurring the disparity space image using an edge-preserving filter [220]. The best-matched disparities are plotted in red, and compare

well with the ground truth disparities ([figure 40.7\[f\]](#)), as labeled in the dataset [\[425\]](#).

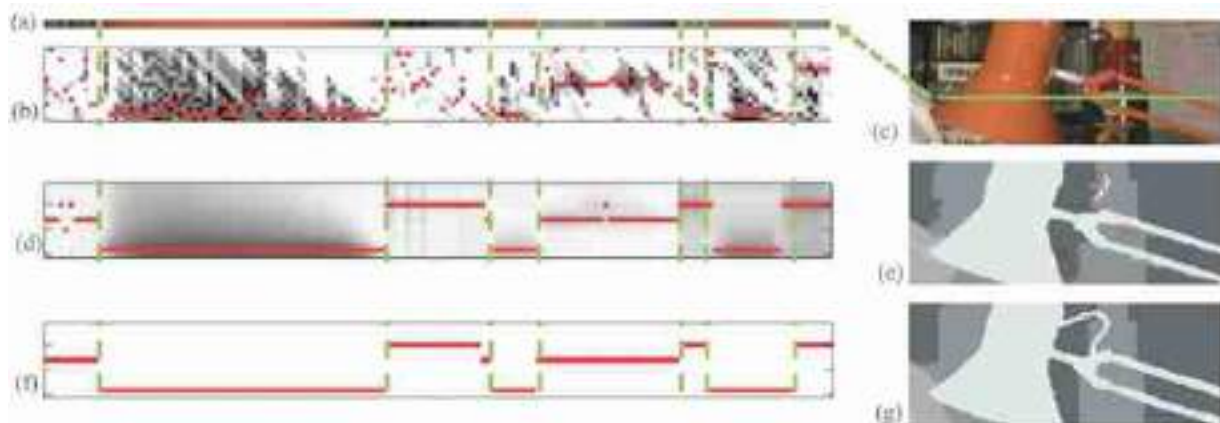


Figure 40.7: Issues with intensity-based stereo matching. Dataset from [\[425\]](#). Figure modified from [\[220\]](#).

The task of finding disparity at each point is often broken into two parts: (1) finding features, and matching the features across images, and (2) interpolating between, or adjusting, the feature matches to obtain a reliable disparity estimate everywhere. These two steps are often called matching and filtering, because the interpolation can be performed using filtering methods.

We will describe next a classical method to find key points and extract image descriptors to find image correspondences. In chapter 44 we will describe other learning-based approaches.

40.3.2.1 Finding image features

Good image features are positions in the image that are easy to localize, given only the image intensities. The **Harris corner detector** [\[186\]](#) identifies image points that are easy to localize by evaluating how the sum of squared intensity differences over an image patch change under a small translation of the image. If the squared intensity changes quickly with patch translation in every direction, then the region contains a good image feature.

To derive the equation of the Harris corner detector we will compute image derivatives. Therefore, we will write an image as a continuous function on x and y . To compute the final quantities in practice we will

approximate the derivatives using their discrete approximations as discussed in chapter 18. Let the input image be $\ell(x, y)$, and let the small translation be Δx in x and Δy in y . Then, the squared intensity difference, $E(\Delta x, \Delta y)$, induced by that small translation, summed over a small image patch, P , is

$$E(\Delta x, \Delta y) = \sum_{(x,y) \in P} (\ell(x, y) - \ell(x + \Delta x, y + \Delta y))^2 \quad (40.5)$$

We expand $\ell(x + \Delta x, y + \Delta y)$ about $\ell(x, y)$ in a Taylor series:

$$\ell(x + \Delta x, y + \Delta y) \approx \ell(x, y) + \frac{\partial \ell}{\partial x} \Delta x + \frac{\partial \ell}{\partial y} \Delta y \quad (40.6)$$

Substituting the above into equation (40.5) and writing the result in matrix form yields

$$E(\Delta x, \Delta y) \approx [\Delta x \ \Delta y] \mathbf{M} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (40.7)$$

where

$$\mathbf{M} = \sum_{(n,m) \in P} \begin{bmatrix} (\frac{\partial \ell}{\partial x})^2 & \frac{\partial \ell}{\partial x} \frac{\partial \ell}{\partial y} \\ \frac{\partial \ell}{\partial x} \frac{\partial \ell}{\partial y} & (\frac{\partial \ell}{\partial y})^2 \end{bmatrix} \quad (40.8)$$

The smallest eigenvalue, $\lambda_{\min}$ of the matrix, $\mathbf{M}$, indicates the smallest possible change of image intensities under translation of the patch in any direction. Thus, to detect good corners, one can identify all points for which $\lambda_{\min} > \lambda_c$, for some threshold λ_c . The Harris corner detector has been widely used for identifying features to match across images, although they were later supplanted by the scale-invariant feature transform (SIFT) [309], based on maxima over a scale of Laplacian filter outputs. [Figures 40.8\(a and b\)](#) show the result of the Harris feature detector to the images of [figure 40.6](#).

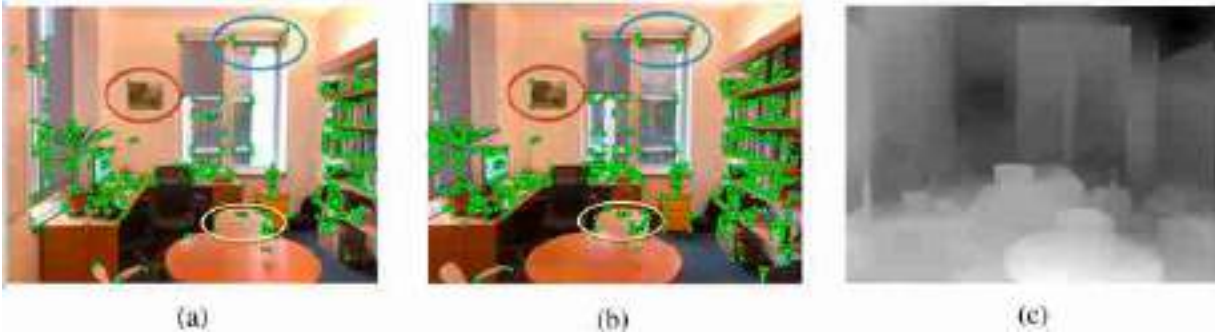


Figure 40.8: The feature-based stereo approach, illustrated for the stereo pair of [figure 40.6](#). (a and b) Feature points. (c) Depth image.

[Figures 40.8](#)(a and b) reveal the challenges of the feature matching task: (1) not every feature is marked across both images, and (2) we need to decide which pairs of image features go together. Feature points are found, independently, in each of the stereo pair images ([figures 40.8](#)[a and b]). Detected Harris feature points [[186](#)] are shown as green dots. It is especially visible within the marked ellipses that not the same set of feature points is marked in each image. [Figure 40.8](#)(c) shows a depth image resulting from a matched and interpolated set of feature points.

40.3.2.2 Local image descriptors

Matching corresponding image features require a local description of the image region to allow matching like regions across two images. SIFT [[309](#)] uses histograms of oriented filter responses to create an image descriptor that is sensitive to local image structure, but that is insensitive to minor changes in lighting or location which may occur across the two images of a stereo pair.

[Figure 40.9](#) shows a sketch of the use of oriented edges and SIFT descriptors in image analysis. The top row of [figure 40.9](#)(a) shows two images of a hand under different lighting conditions. Despite that both images look quite different in a pixel intensity representation, they look quite similar in a local orientation, depicted in the bottom row by line segment directions. SIFT image descriptors [[309](#)] exploit that observation. Taking histograms ([figure 40.9](#)[b]) of local orientation over local spatial regions further allow for image descriptors to be robust to small image translations [[145](#), [309](#)].

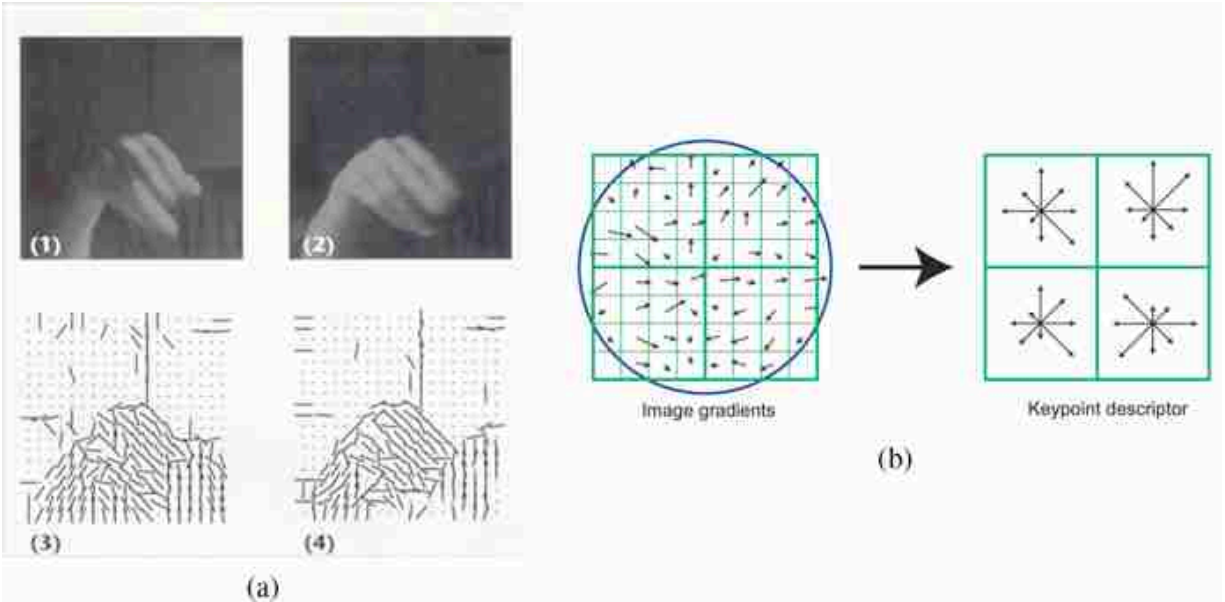


Figure 40.9: Orientation-based image descriptors. (a) Oriented features. Reprinted from [145]. (b) SIFT image descriptors [309].

40.3.2.3 Interpolation between feature matches

Methods to interpolate depth between the depths of matched points include probabilistic methods such as belief propagation, discussed in chapter 29. Under assumptions about the local spatial relationships between depth values, optimal shapes can be inferred provided the spatial structure over which inference is performed is a chain or a tree. Note that [207] uses a related inference algorithm, dynamic programming, which has the same constraints on spatial structure for exact inference. Hirschmuller's algorithm, called semiglobal matching (SGM), applies the one-dimensional processing over many different orientations to approximate a global solution to the stereo shape inference problem.

40.3.3 Constraints for Arbitrary Cameras

Until now we have considered only the case where the two cameras are parallel, but in general the two images will be produced by cameras that can have arbitrary orientations. Note that disparity of the smokestacks in [figure 40.1\(a\)](#) increases with depth, rather than decreasing as described by equation (40.4). That is because the cameras that took the stereo pair were not in the same orientation, related only by a translation.

Let's assume we have two calibrated cameras: we know the intrinsic and extrinsic parameters for both. They are related by a generic translation, $\mathbf{T}$,

and a rotation, $\mathbf{R}$, such that their optical axis are not parallel. Let's now consider that we see one point in the image produced by camera 1 and we want to identify where is the corresponding point in the image from camera 2. The following sketch ([figure 40.10](#)) shows the setting of the problem we are trying to solve.

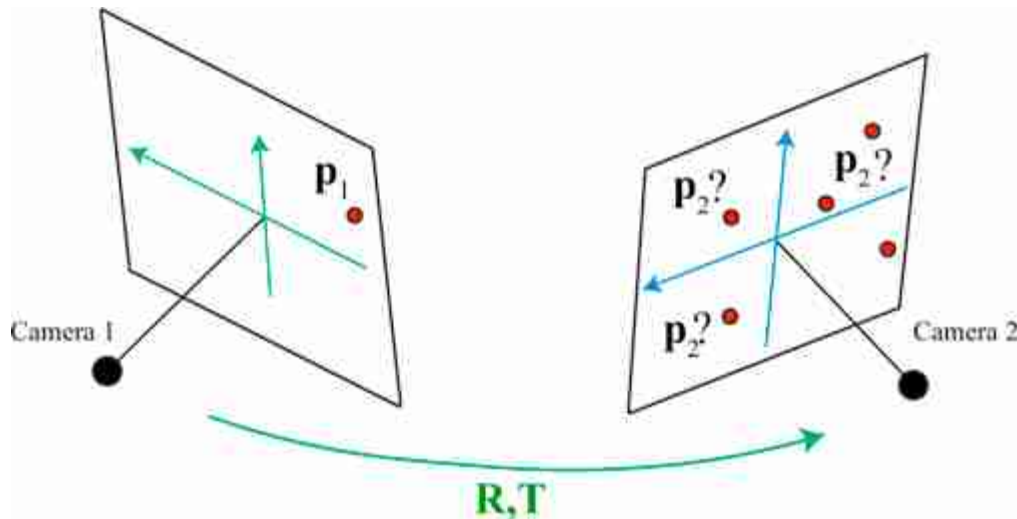


Figure 40.10: How can we identify the specific point $\mathbf{p}_2$ on camera 2 that represents the projection of the same 3D point as $\mathbf{p}_1$ on camera 1?

Are there any geometric constraints that can help us to identify candidate matches? We know that the observed point in camera 1, $\mathbf{p}_1$, corresponds to a 3D point that lies within the ray that connects the camera 1 origin and $\mathbf{p}_1$ (shown in red in [figure 40.11](#)):

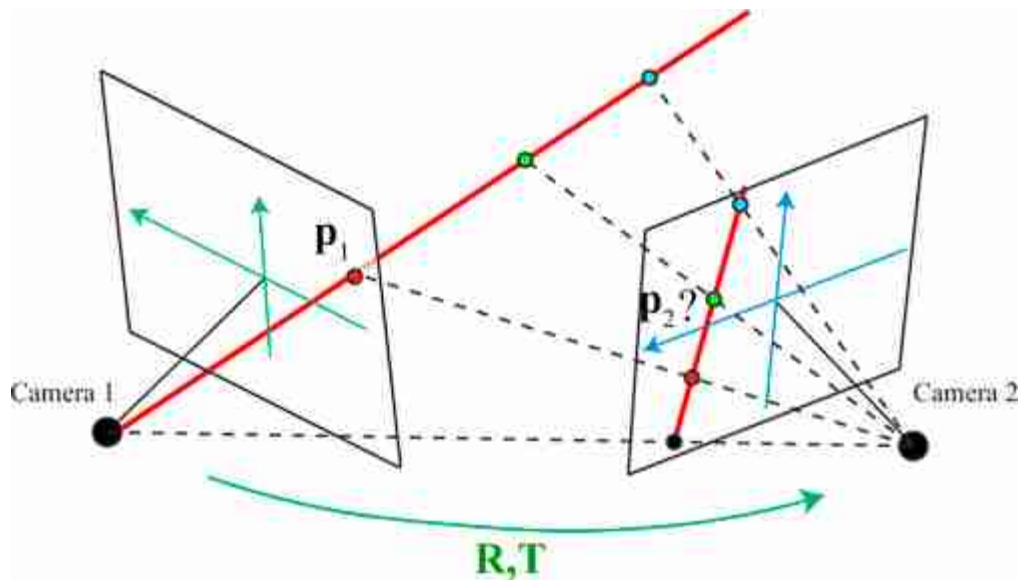


Figure 40.11: The point p_1 corresponds to a 3D point that lies within the ray that connects the camera 1 origin and p_1 . The point p_2 is constrained to be within the projection of that line on camera 2.

Therefore, p_2 is constrained to be within the line that results of projecting the ray into camera 2. Remember that a line in 3D projects into a line in 2D. There are two points that are of particular interest in the line projected into camera 2. The first point is the projection of the camera center of camera 1 into the camera plane 2 (the point could lay outside the camera view but in this example it happens to be visible). The second point is the projection of p_1 itself into camera 2. As we know all the intrinsic and extrinsic camera parameters, those two points should be easy to calculate. And these two points will define the equation of the line (there are many other ways to think about this problem).

The following figure shows the geometry of a stereo pair. Given a 3D point, P , in the world, we define the following objects:

- The **epipolar plane** is the plane that contains the two camera origins and a 3D point, P .
- The **epipolar lines** are the intersection between the epipolar plane and the camera planes. When the point P moves in space, the epipolar plane and epipolar lines change.
- The **epipoles** are the intersection points between the line that connects both camera origins and the camera planes. The epipoles' location does

not depend on the point P . Therefore, all the epipolar lines intersect at the same epipoles.

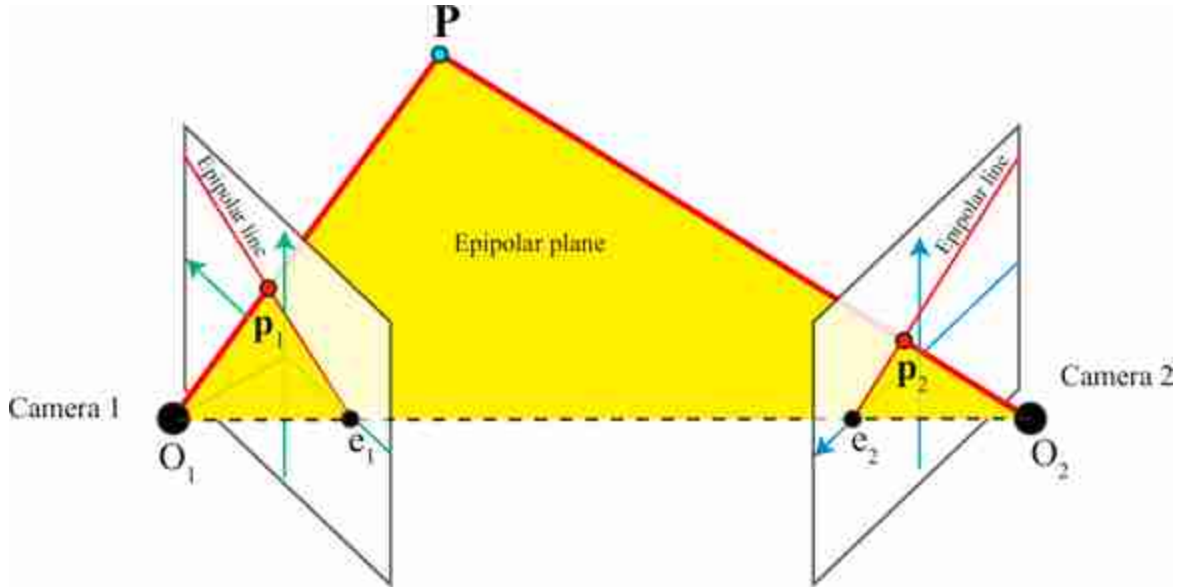


Figure 40.12: Geometry of a stereo pair and terminology. The figure shows the epipolar plane, epipolar lines, and epipoles (e_1 and e_2), for a given point P . As we move the point P , only the epipoles remain constant.

In the example shown in [figure 40.12](#), as both cameras are only rotated along their vertical axis and there is no vertical displacement between them, the epipoles are contained in the x axes of both camera coordinate systems. The epipolar line in camera 2, corresponds to the projection of the line O_1P into the camera plane. Therefore, given the image point p_1 , we know that the match should be somewhere along the epipolar line.

The epipolar line in camera 2 that corresponds to p_1 is uniquely defined by the camera geometries and the location of image point p_1 . So, when looking for matches of a point in one camera, we only need to look along the corresponding epipolar line in the other camera. The same applies for finding matches in camera 1 given a point in camera 2.

In the next section we will see how to find the epipolar lines mathematically.

40.3.4 The Essential and Fundamental Matrices

We will follow here the derivation that was proposed by Longuet-Higgins, in 1981 [306]. Let's consider a 3D point, $\mathbf{P}$, viewed by both cameras. The coordinates of this 3D point $\mathbf{P}$ are $\mathbf{P}_1 = [X_1, Y_1, Z_1]^T$ when expressed in the coordinates system of camera 1, and $\mathbf{P}_2 = [X_2, Y_2, Z_2]^T$ when expressed in the coordinates system of camera 2. Both systems are related by (using heterogeneous coordinates):

$$\mathbf{P}_2 = \mathbf{R}\mathbf{P}_1 + \mathbf{T} \quad (40.9)$$

The 3D point $\mathbf{P}_1$ projects into the camera 1 at the coordinates $\mathbf{p}_1 = f\mathbf{P}_1/Z_1$ where $\mathbf{p}_1$ is expressed also in 3D coordinates $\mathbf{p}_1 = [fX_1/Z_1, fY_1/Z_1, f]^T$. Without losing generality we will assume $f = 1$.

Taking the cross product of both sides of equation (40.9) with $\mathbf{T}$ we get:

$$\mathbf{T} \times \mathbf{P}_2 = \mathbf{T} \times \mathbf{R}\mathbf{P}_1 \quad (40.10)$$

as $\mathbf{T} \times \mathbf{T} = 0$. And now, taking the dot product of both sides of equation (40.10) with $\mathbf{P}_2$:

$$\mathbf{P}_2^T \mathbf{T} \times \mathbf{P}_2 = \mathbf{P}_2^T \mathbf{T} \times \mathbf{R}\mathbf{P}_1 \quad (40.11)$$

The left side is zero because $\mathbf{T} \times \mathbf{P}_2$ results in a vector that is orthogonal to both $\mathbf{T}$ and $\mathbf{P}_2$, and the dot product with $\mathbf{P}_2$ is zero (dot product of two orthogonal vectors is zero). Therefore, $\mathbf{P}_1$ and $\mathbf{P}_2$ satisfy the following relationship:

$$\mathbf{P}_2^T (\mathbf{T} \times \mathbf{R}) \mathbf{P}_1 = 0 \quad (40.12)$$

The cross product of two vectors $\mathbf{a} \times \mathbf{b}$ can be written in matrix form as:

$$\mathbf{a} \times \mathbf{b} = \mathbf{a}_\times \mathbf{b}$$

The special matrix $\mathbf{a}_\times$ is defined as:

$$\mathbf{a}_\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}$$

This matrix $\mathbf{a}_\times$ is rank 2. In addition, it is a skew-symmetric matrix (i.e., $\mathbf{a}_\times = -\mathbf{a}_\times^T$). It has two identical singular values and the third one is zero.

The matrix $\mathbf{E}$ is called the **essential matrix** and it corresponds to:

$$\mathbf{E} = \mathbf{T} \times \mathbf{R} = \mathbf{T}_\times \mathbf{R} \quad (40.13)$$

This results in the relationship:

$$\mathbf{P}_2^T \mathbf{E} \mathbf{P}_1 = 0 \quad (40.14)$$

If we divide equation (40.14) by $Z_1 Z_2$ we will obtain the same relationship but relating the image coordinates:

$$\mathbf{p}_2^T \mathbf{E} \mathbf{p}_1 = 0 \quad (40.15)$$

This equation relates the coordinates of the two projections of a 3D point into two cameras. If we fix the coordinates from one of the cameras, the equation reduces to the equation of a line for the coordinates of the other camera, which is consistent with the geometric reasoning that we made before.

The essential matrix is a 3×3 matrix with some special properties:

- $\mathbf{E}$ has five degrees of freedom instead of nine. To see this, let's count degrees of freedom: the rotation and translation have three degrees of freedom each, this gives six; then, one degree is lost in equation (40.15) as a global scaling of the matrix does not change the result. This results in five degrees of freedom.
- The essential matrix has rank 2. This is because $\mathbf{E}$ is the product of a rank 2 matrix, $\mathbf{T}_\times$, and a rank 3 matrix, $\mathbf{R}$.
- The essential matrix has two identical singular values and the third one is zero. This is because $\mathbf{E}$ is the product of $\mathbf{T}_\times$, which has two identical singular values, and the rotation matrix $\mathbf{R}$ that does not change the singular values.

As a consequence, not all 3×3 matrices correspond to an essential matrix which is important when trying to estimate $\mathbf{E}$.

40.3.4.1 The fundamental matrix

We have been using camera coordinates, $\mathbf{p}$. If we want to use image coordinates, in homogeneous coordinates, $\mathbf{c} = [n, m, 1]^T$, we need to also incorporate the intrinsic camera parameters (which will also incorporate pixel size and change of origin). Incorporating the transformation from camera coordinates, $\mathbf{p}$, to image coordinates, $\mathbf{c}$, the equation relating image coordinates in both cameras has the same algebraic form as in equation (40.15):

$$\mathbf{c}_2^T \mathbf{F} \mathbf{c}_1 = 0 \quad (40.16)$$

where $\mathbf{F}$ is the **fundamental matrix**.

The fundamental and essential matrices are related via the intrinsic camera matrix, namely:

$$\mathbf{F} = \mathbf{K}^{-\top} \mathbf{E} \mathbf{K}^{\top} \quad (40.17)$$

The fundamental matrix has rank 2 (like the essential matrix) and 7 degrees of freedom (out of the 9 degrees of freedom, one is lost due to the scale ambiguity and another is lost because the determinant has to be zero).

40.3.4.2 Estimation of the essential/fundamental matrix

Given a set of matches between the two views we can estimate either the essential or the fundamental matrix by writing a linear set of equations. Any two matching points will satisfy $\mathbf{p}_2^{\top} \mathbf{E} \mathbf{p}_1 = 0$ and we can use this a linear set of equations for the coefficients of $\mathbf{E}$. However, the matrix $\mathbf{E}$ has a very special structure that allows more efficient and robust methods to estimate the coefficients. The same applies for the fundamental matrix.

As both matrices are rank 2, the minimization the linear set of equations can be done using a constrained minimization or directly optimizing a rank 2 matrix. You can also delete the third singular value $\mathbf{F}$, and, for $\mathbf{E}$, you can delete the third singular value and set the first and second singular values to be equal.

Once $\mathbf{E}$ is estimated, it can be decomposed to recover the translation (skew-symmetric matrix) and rotation (orthonormal matrix) between the two cameras using the singular value decomposition methods.

40.3.4.3 Finding the epipoles

As the epipoles, $\mathbf{e}_1$ and $\mathbf{e}_2$, belong to the epipolar lines, they satisfy

$$\mathbf{p}_2^{\top} \mathbf{E} \mathbf{e}_1 = 0 \quad (40.18)$$

$$\mathbf{e}_2^{\top} \mathbf{E} \mathbf{p}_1 = 0 \quad (40.19)$$

and, as the epipoles are the intersection of all the epipolar lines, both relationships shown previously have to be true for any points $\mathbf{p}_1$ and $\mathbf{p}_2$. Therefore,

$$\mathbf{E} \mathbf{e}_1 = 0 \quad (40.20)$$

$$\mathbf{e}_2^{\top} \mathbf{E} = 0 \quad (40.21)$$

As the matrix $\mathbf{E}$ is likely to be noisy, we can compute the epipoles as the eigenvectors with the smallest eigenvalues of $\mathbf{E}$ and $\mathbf{E}^{\top}$.

40.3.4.4 Epipolar lines: The game

In order to build an intuition of how epipolar lines work we propose to play a game. [Figure 40.13](#) shows a set of camera pairs and a set of epipolar lines. Can you guess what camera pair goes with what set of epipolar lines?²⁰

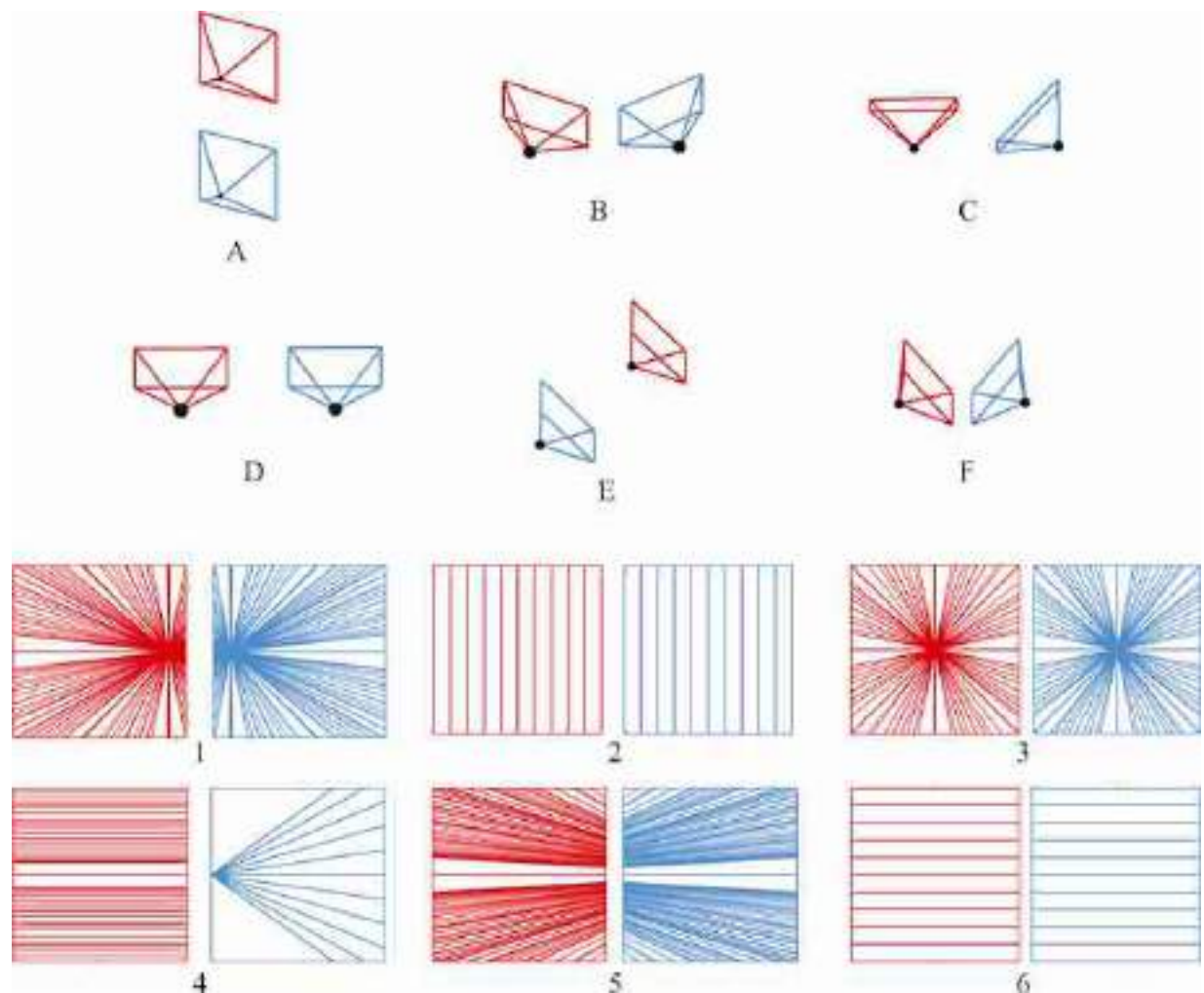


Figure 40.13: Epipolar game. The figure shows six camera pair arrangements, and six sets of epipolar lines. Can you identify which camera pairs correspond to each set of epipolar lines?

To solve the game, visualize mentally the cameras and think about how rays from the origin of one camera will be seen from the other camera. The epipole corresponds to the location where one camera sees the origin of the other camera.

Observe that most of the sets of epipolar lines shown in the figure are symmetric across the two cameras, but it is not always true. What conditions are needed to have symmetric epipolar lines across both cameras? What camera arrangements will break the symmetry? If you were to place two cameras in random locations, do you expect to see symmetric epipolar lines?

It is also useful to use the equations defining the epipolar lines and deduce the equations for the epipolar lines for some of the special cases shown in [figure 40.13](#).

When building systems, it is crucial to visualize intermediate results in order to find bugs in code. This requires knowing what the results should look like. The game from [figure 40.13](#) helps building that understanding.

40.3.5 Image Rectification

As described previously, one only needs to search along epipolar lines to find matching points in the second image. If the sensor planes of each camera are coplanar, and if the pixel rows are colinear across the two cameras, then the epipolar lines will be along image scanlines, simplifying the stereo search.

Using the homography transformations (chapter 41), we can warp observed stereo camera images taken under general conditions to synthesize the images that would have been recorded had the cameras been aligned to yield epipolar lines along scan rows.

Many configurations of the two cameras satisfy the desired image rectification constraints [521]: (1) that epipolar lines are along image scanlines, and (2) that matching points are within the same rows in each camera image. One procedure that will satisfy those constraints is as follows [503]. First, rotate each camera to be parallel with each other and with optical axes perpendicular to the line joining the two camera centers. Second, rotate each of the cameras about its optical axis to align the rows of each camera with those of the other. Third, uniformly scale one camera's image to be the same size as to the other, if necessary. The resulting image pair will thus satisfy the two requirements above for image rectification. Image rectification uses image warping with bilinear (or other) interpolation as described in section 21.4.2.

Other algorithms are optimized to minimize image distortions [521] and may give better performance for stereo algorithms. We assume camera

calibrations are known, but other rectification algorithms can use weaker conditions, such as knowing the fundamental matrix [187, 390]. Many software packages provide image rectification code [307].

40.4 Learning-Based Methods

As is the case for most other vision tasks, neural network approaches now dominate stereo methods. The two stages of a stereo algorithm, matching and filtering, can be implemented separately by neural networks [516], or together in a single, end-to-end optimization ([519, 75, 327, 255]).

40.4.1 Output Representation

One important question is how we want to represent the output. We are ultimately interested in estimating the 3D structure of the scene. We can estimate the disparity or the depth. Disparity might be easier to estimate as it is independent of the camera parameters. It is also easier to model noise, as we can assume that noise is independent of disparity. However, when using depth, estimation error will be larger for distant points.

Another interesting property of disparity is that for flat surfaces, the second-order derivative along any spatial direction, $\mathbf{x}$, is zero:

$$\frac{\partial^2 d}{\partial \mathbf{x}^2} = 0. \quad (40.22)$$

This equality does not hold for depth. This property allows introducing a simple regularization term that favors planarity.

To translate the returned disparity for a given pixel $d[n, m]$ into depth values $z[n, m]$, we need to invert the disparity, that is

$$z[n, m] = \frac{1}{d[n, m]} \quad (40.23)$$

Now we need to recover the world coordinates for every pixel. Assuming a pinhole camera and using the depth, $z[n, m]$, and the intrinsic camera parameters, $\mathbf{K}$, we obtain the 3D coordinates for a pixel with image coordinates $[n, m]$ in the camera reference frame as

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = z[n, m] \times \mathbf{K}^{-1} \times \begin{bmatrix} n \\ m \\ 1 \end{bmatrix} \quad (40.24)$$

Note that $\mathbf{K}$ are the intrinsic camera parameters for the reference image.

40.4.2 Two-Stage Networks

Given two images, $\mathbf{v}_1$ and $\mathbf{v}_2$, the disparity at each location can be computed by a function, that is:

$$\hat{\mathbf{d}} = f_{\theta}(\mathbf{v}_1, \mathbf{v}_2) \quad (40.25)$$

The objective is that the estimated disparities, $\hat{\mathbf{d}}$, should be close to ground truth disparities, $\mathbf{d}$. One example of training set are the KITTI dataset [157] and the scene flow dataset [327], which contain ground truth disparities from calibrated stereo cameras. One typical loss is:

$$J(\theta) = \sum_i |\mathbf{d} - f_{\theta}(\mathbf{v}_1, \mathbf{v}_2)|^2 \quad (40.26)$$

This is optimized by gradient descent as usual.

When matching and filtering are implemented separately, one common formulation is to use one neural network to extract features from both images, $g(\mathbf{i}_1)$ and $g(\mathbf{i}_2)$, and one neural network to compute the matching cost and estimate the disparities at every pixel.

$$f(\mathbf{v}_1, \mathbf{v}_2) = c(g(\mathbf{v}_1), g(\mathbf{v}_2)) \quad (40.27)$$

A common approach, depicted in block diagram form in [figure 40.14](#), is the following:

- Rectify the stereo pair so that scanlines in each image depict the same points in the world.
- Extract features from each image, using a pair of networks with shared weights.
- Form a 3D cost volume indicating the local visual evidence of a match between the two images for each possible pixel position and each possible stereo disparity. (This 3D cost volume can be referenced to the H and V positions of one of the input images.)
- Train and apply a 3D convolutional neural net (CNN) to aggregate (process) the costs over the 3D cost volume in order to estimate a single, best disparity estimate for each pixel position. This performs the function of the regularization step of nonneural-network approaches, such as SGM [207], but the performance is better for the neural network approaches.

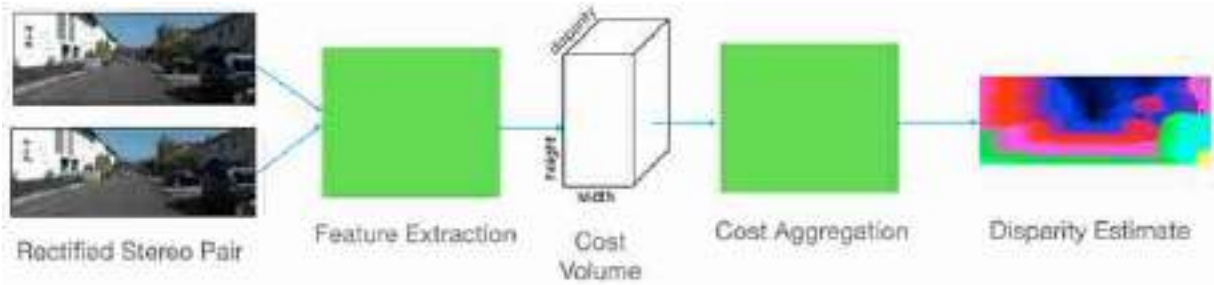


Figure 40.14: Block diagram of CNN stereo processing, abstracted from the block diagrams of [75, 519, 255].

The performance of methods following the block diagram of [figure 40.14](#) surpassed that of methods with handcrafted features. More recent work has addressed the problem of poor generalization to out-of-training-domain test examples [520]. Training and computing the 3D neural network to process the cost volume can be expensive in time and memory and recent work has obtained both speed and performance improvement through developing an iterative algorithm to avoid the 3D filtering [301].

40.5 Evaluation

The Middlebury stereo web page [425] provides a comprehensive comparison of many stereo algorithms, compared on a common dataset, with pointers to algorithm descriptions and code. One can use the web page to evaluate the improvement of stereo reconstruction algorithms over time. The advancement of the field over time is very impressive.

40.6 Concluding Remarks

Accurate depth estimation from stereo pairs remains an unsolved problem. The results are still not reliable. In order to get accurate depth estimates, most methods require many images and the use of multiview geometry methods to reconstruct the 3D scene.

Stereo matching is often also formulated as a problem of estimating the image motion between two views captured at different locations. In chapter 46 we will discuss motion estimation algorithms.

We have only provided a short introduction to stereo vision, for an in depth study of the geometry of stereo pairs the reader can consult specialized books such as [[187](#)] and [[122](#)].

[20.](#) Answer for the game in [figure 40.13](#): A-2, B-5, C-4, D-6, E-3, F-1.

41 Homographies

41.1 Introduction

In chapter 38 we discussed several types of geometric transformations of two-dimensional (2D) images: translations, rotations, skewing, scalings, and we saw that all could be modeled as the product between the coordinates of a point in the input image described using homogeneous coordinates and a 3×3 matrix. Now that we have discussed camera models, we are well equipped to present a more general geometric image transformation that opens the door to many applications: the homography.

As we discussed in the previous chapter 40, if we capture a scene with two cameras from different viewpoints, corresponding points across both images are constrained by the epipolar constrain. In general, there is not a simple geometric transformation that maps pixels from one image into the pixels of the other image. To find that mapping we will need to know the three-dimensional (3D) location of each point as well as the relative camera translation and rotation between both images.

However, there are a few scenarios where we can find a simple transformation that allows warping one image into another image corresponding to a new camera location without requiring knowing the 3D scene structure: this will happen when the scene is planar (as in [figure 41.1](#)) or when the two cameras are related just by a rotation ([figure 41.4](#)). In those two cases the coordinates of corresponding points across the two images are related by a **homography**.

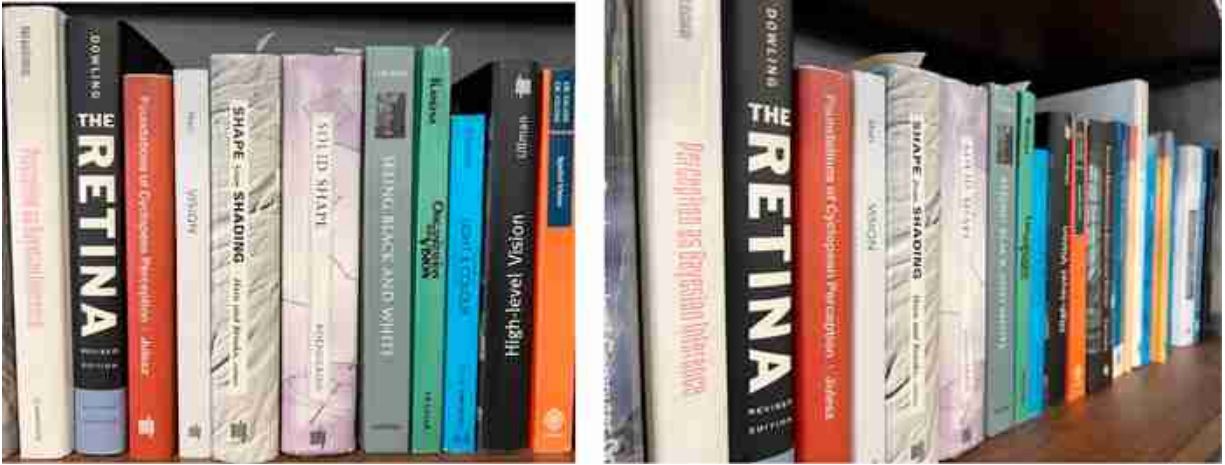


Figure 41.1: These two images are (approximately) related by an homography.

Homographies can be used in applications such as perspective correction, augmented reality, image stitching, creating bird's-eye views, correcting keystone distortions for projected images, and many others.

One popular tool available in many cameras and photography editing tools is the option to combine multiple overlapping images into a large panorama. The **homography** is what is behind those tools.

41.2 Homography

Let's start with a formal definition of the homography. A homography (or projective transformation) is a geometric transformation that preserves straight lines. The homography is a function h that maps points into points, $\mathbf{p}' = h(\mathbf{p})$, with the property that if points $\mathbf{p}_1$, $\mathbf{p}_2$, and $\mathbf{p}_3$ are colinear, then the transformed points $h(\mathbf{p}_1)$, $h(\mathbf{p}_2)$, and $h(\mathbf{p}_3)$ are also colinear. Such a function, $\mathbf{p}' = h(\mathbf{p})$, can be written in homogeneous coordinates as a product with a matrix:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (41.1)$$

This can be written in the short form:

$$\mathbf{p}' = \mathbf{H}\mathbf{p} \quad (41.2)$$

where $\mathbf{p}$ and $\mathbf{p}'$ correspond to 2D points in homogeneous coordinates, and $\mathbf{H}$ is a 3×3 matrix.

To prove that colinear points remain colinear after the homography we can use the equation of a line in homogeneous coordinates, $\mathbf{l}^T \mathbf{p} = 0$. From equation (41.2), we have that $\mathbf{p} = \mathbf{H}^{-1} \mathbf{p}'$. Replacing the last equality into the equation of the line gives $(\mathbf{H}^{-T} \mathbf{l})^T \mathbf{p}' = 0$, which corresponds to the equation of the line for the projected points $\mathbf{p}'$.

In a general homography, angles and lengths are not preserved. Therefore, parallel lines might not remain parallel, and lines of identical length might have different lengths after a homography. [Figure 41.2](#) shows how a planar grid gets transformed after applying a homography.

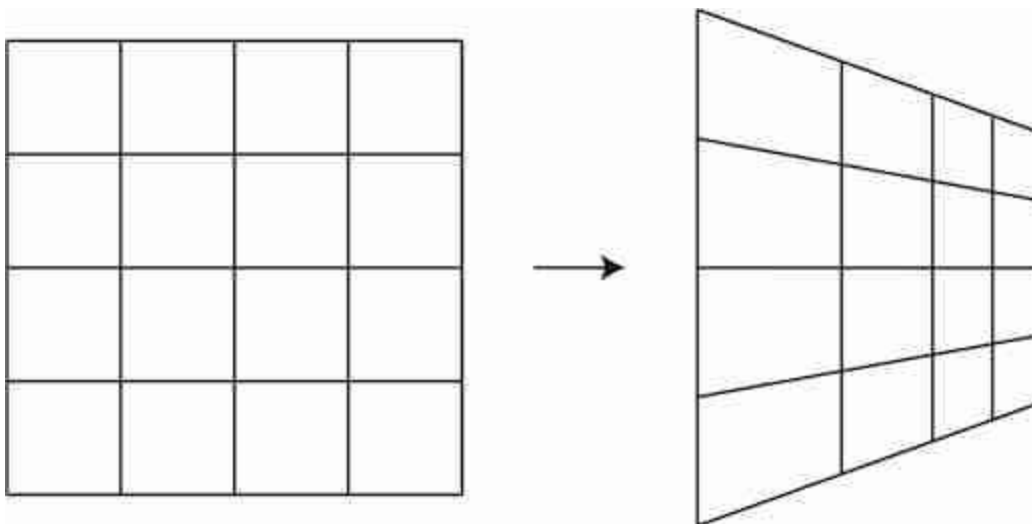


Figure 41.2: Homography. Colinear points remain colinear after transformation. Angles between lines are not preserved.

Let's start discussing in which scenarios the coordinates of points between two images are related by a homography.

41.2.1 Camera Rotation

Consider the following three images shown in [figure 41.3](#).



Figure 41.3: Three pictures taken by rotating the camera while trying to keep the camera center fixed.

The three images are taken from the same point by rotating the camera. There is no translation between the three camera positions as the photographer took the three images by keeping the camera origin static when moving the camera. Under this condition, the coordinates of the corresponding points across each pair of images are related by a homography independently of how far they are from the camera.

Here, we show that the mapping between feature point locations in two cameras differing only in their 3D orientation, as shown in [figure 41.4](#), is a 3×3 matrix transformation of their 2D positions written in homogeneous coordinates, that is, a homography.

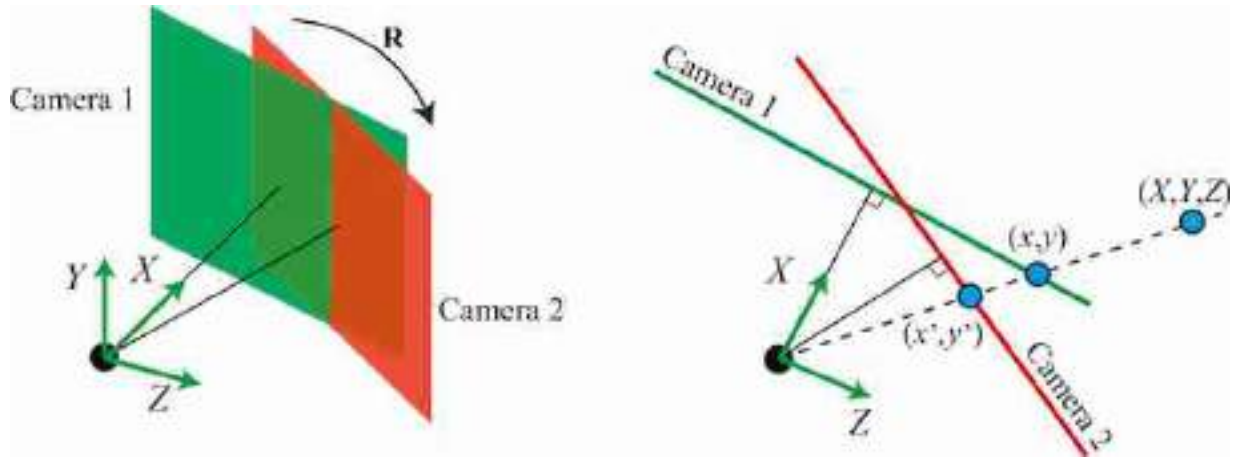


Figure 41.4: Camera rotation generates two images related by a homography.

We start by setting the world coordinate system aligned with the coordinate system of camera 1, as shown in [figure 41.4](#). Therefore, the projection of a 3D point, $\mathbf{P} = [X, Y, Z]^T$, into the camera 1 plane gives, setting the translation vector to be zero:

$$\lambda_1 \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{I} & \mathbf{0} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (41.3)$$

We can now project the same 3D point into camera 2. Setting the translation vector to be zero in equation (39.22), and writing the 3×3 rotation matrices for camera 2 as $\mathbf{R}$, we have for the observed position of a common 3D point observed from camera 2:

$$\lambda_2 \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{0} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{KR} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (41.4)$$

As both cameras see the same 3D point, $\mathbf{P} = [X, Y, Z]^T$, we can invert the two projection equations (41.10) and (41.4) to get the following relationship:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \lambda_1 \mathbf{K}^{-1} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \lambda_2 \mathbf{R}^{-1} \mathbf{K}^{-1} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \quad (41.5)$$

Observe that equation (41.5) does not mean that we can recover the 3D coordinates of a point from the image coordinates. We know that this is impossible, so what is the issue? The subtlety is that the equations are written as functions of λ_1 and λ_2 , which we do not know when only given image coordinates. Therefore, the equations only specify the equation of a ray when looking at all possible values of λ_1/λ_2 .

Using the last equality, we can establish the following relationship between the corresponding image points, in homogeneous coordinates:

$$\lambda_2/\lambda_1 \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{K} \mathbf{R} \mathbf{K}^{-1} \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \quad (41.6)$$

We can write the relationship between corresponding points as

$$\lambda \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (41.7)$$

where $\mathbf{H}$ is a 3×3 matrix. Thus, camera rotation modifies the locations of the image points by a homography. The relationship also holds if the two cameras have different intrinsic camera parameters. This concludes the proof.

Under certain conditions, a homography predicts the position of points when viewed with another camera from another position. We just discussed one condition, when the two cameras differ in their position by a rotation about a common center of projection. We will now consider another case.

41.2.2 Planar Surface

A second case is when two cameras observe a 2D plane [187]. In that case, the coordinates of corresponding points across the two camera views, for points inside the plane, are related by a homography ([figure 41.5](#)).



Figure 41.5: Two pictures of a building facade taken from different positions. The coordinates of corresponding points on the planar facade are related by a homography.

To prove this, we can first show that the coordinates inside a plane relate to the coordinates in the image plane by a homography independently of relative position between the plane and the camera. Therefore, the relationship between points across two images will also be related by a homography.

We start by placing the world-coordinate system on the plane so that the plane is defined by $Z = 0$ as shown in the [figure 41.6](#). Note that the origin can be placed at any arbitrary location in the world.

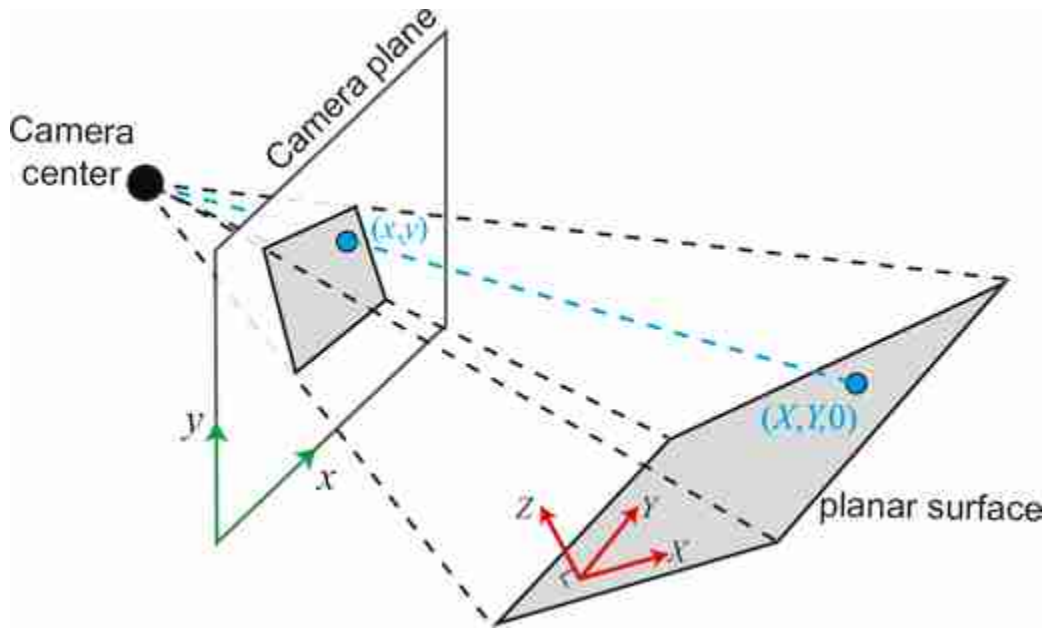


Figure 41.6: Geometry of a the projection of a planar scene. The origin of the world-coordinates system is placed inside the plane and the Z-axis is perpendicular to it.

The proof is similar to what we did in the previous section. We start writing the projection equation

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{K} [\mathbf{R} \quad \mathbf{t}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (41.8)$$

Note that we are not using the same notation for the translation vector as we used in equation (39.17). You can obtain the same by setting $\mathbf{t} = -\mathbf{RT}$, where $\mathbf{R}$ and $\mathbf{T}$ are the translation and rotation of the camera with respect to the world-coordinate system. As we can put the world-coordinate system in any arbitrary location, we chose the world-coordinates system to be so that points in the plane have coordinates $Z = 0$ as shown in [figure 41.6](#). Therefore, we can write:

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{K} [\mathbf{R} \quad \mathbf{t}] \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix} = \mathbf{K} [\mathbf{c}_1 \quad \mathbf{c}_2 \quad \mathbf{t}] \begin{bmatrix} X_1 \\ Y_1 \\ 1 \end{bmatrix} \quad (41.9)$$

Where c_i is the column i of the rotation matrix $\mathbf{R}$. As the last expression contains the product of two 3×3 matrices, we can write

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \mathbf{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} \quad (41.10)$$

This last result shows that the coordinates of points on the plane are related to the corresponding points in the image plane by a homography.

As equation (41.10) is true for any camera, the relationship between corresponding points in the two cameras will also be related by a homography when the points correspond to points in a plane in the 3D space.

A homography is a stronger constraint than the epipolar constraint between corresponding points across two camera views. However, the homography only applies under certain conditions as we have seen, while the epipolar constraint always holds.

41.3 Creating Image Panoramas

Let's now study a popular application of the homography: stitching images to create large panoramas. To create a large panorama from multiple pictures, the images need to be taken by a rotating camera without translation.

To stitch together images, one needs to estimate the homography relating the images to be stitched together. While one could estimate the rotation and intrinsic camera matrices within equation (41.6) to compute the homography, it is usually simpler to follow the procedure below.

We will start assuming we have some initial correspondences between each pair of images as shown in [figure 41.7](#). The correspondences in this example are computer using **speeded up robust features** (SURF) [[41](#)].



Figure 41.7: Two overlapping images and some found correspondences using SURF descriptors. A random set of four correspondences are highlighted.

41.3.1 Direct Linear Transform Algorithm

Given a set of point locations, and their correspondences across the two cameras, we can compute the homography relating the locations of imaged points within the two cameras. To estimate the homography we can use the direct linear transform (DLT) algorithm, as we did for the camera calibration problem in section 39.7. This will give us a linear set of equations for the parameters of the homography matrix with the form:

$$\mathbf{A}\mathbf{h} = \mathbf{0} \quad (41.11)$$

where $\mathbf{A}$ is a $2N \times 9$ matrix, when given N corresponding pairs. The vector $\mathbf{h}$ contains all the elements of the matrix $\mathbf{H}$ stacked as a column vector of length 9. Note that $\mathbf{h}$ only has 8 degrees of freedom as the results do not change for a global scaling of all the values. Therefore we will need at least four correspondences to estimate the homography between two sets of corresponding points, as the ones shown in [figure 41.7](#). As before, the solution is the vector that minimizes the residual $\|\mathbf{A}\mathbf{h}\|^2$. The result is given by the eigenvector of the matrix $\mathbf{A}^T \mathbf{A}$ with the smallest eigenvalue.

In practice, to get accurate results, it is useful to also minimize the reprojection error after you initialize with the DLT solution since $\|\mathbf{A}\mathbf{h}\|^2$ is

meaningless geometrically.

41.3.2 Robust Model Fitting: Random Sampling with Consensus

To perform the image stitching in the section above, we had to find the homography parameters that best described the transformation from one image to another. Because the detection of the feature points can be noisy, the homography fitting needs to be robust against inevitable feature mismatches between images. A good algorithm to fit models to data robustly is RANSAC [128], which stands for **random sampling with consensus**.

RANSAC, introduced in 1981 by Fischler and Bolles [128], has been a workhorse in computer vision ever since [532].

The procedure for RANSAC is simple, and the name contains most of the steps of the algorithm. First, randomly select a sufficient set of datapoints to fit the parameters of some model. This could be the parameters that define a line, some other structure, or a homography. Then, compute the model parameters from the randomly sampled set of points. Compute the inliers in the dataset, that is, the datapoints that fit the model (or achieve consensus) to within some tolerance. Repeat that procedure some number of times, N . Compute a final model from the set of inlier points corresponding to the largest number of inlier datapoints.

Figure 41.8 shows a simple instantiation of this algorithm for the case of robustly fitting a straight line model to a collection of datapoints (figure 41.8[a]). First, a number of datapoints, sufficient to uniquely specify the model parameters, are drawn from the dataset. For this problem, that is two points. Two randomly selected points are marked in green in figure 41.8(b). After finding the line that passes through those two points, the number of **inliers** (datapoints within ϵ of the model) are determined. For the line in figure 41.8(b), the number of inliers is three. This process is repeated until some desired number of samplings, S , have been drawn. The output model is fitted from all the inlier datapoints corresponding to the model that led to the most inlier datapoints.

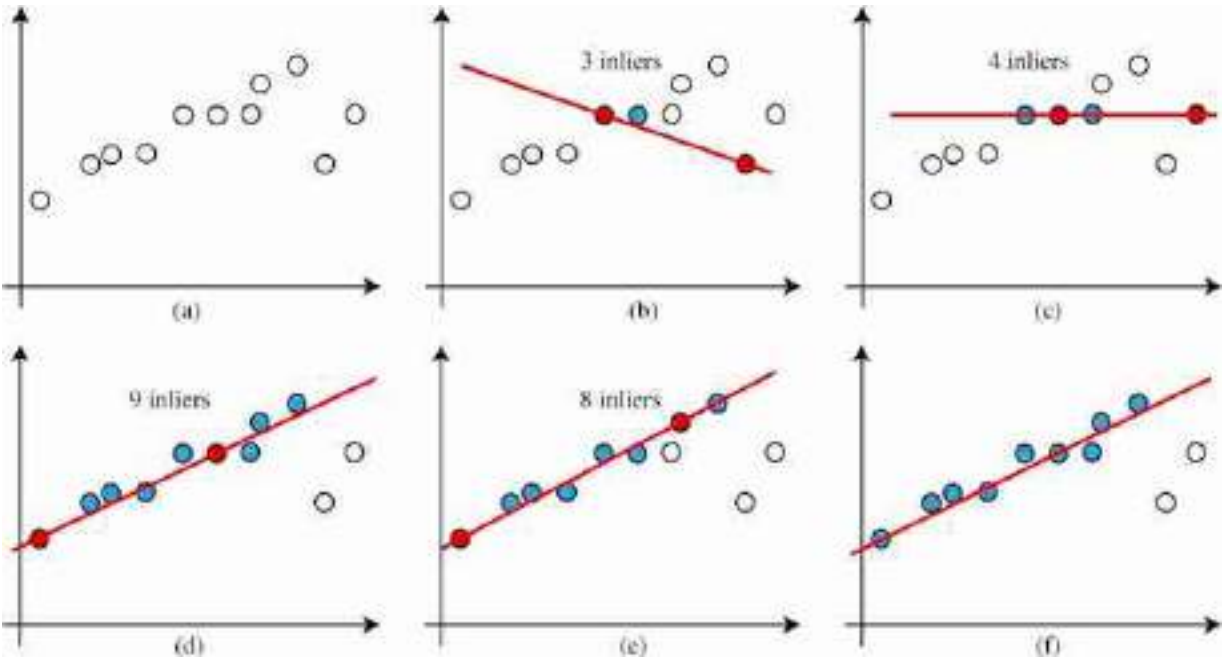


Figure 41.8: RANSAC applied to robust estimation of line parameters. (a) Datapoints for which robust line estimation is sought. (b) Two points (in blue) are selected at random, sufficient to estimate line model parameters. The number of inliers is computed for this model. (c - e) Repeat k times. (f) Finally, select the model with the maximum number of inliers.

The paper [128] derives a heuristic for the number of samples, S , required to be assured a good model fit. Let n be the number of datapoints required to specify the model, and let w be (an estimate of) the probability that any selected datapoint is within the error tolerance of the model. For [figure 41.8\(f\)](#), there are 2 outliers out of 11 points, so $w = 0.18$ (in general, this value needs to be estimated). The w^n is the probability that all the selected points are inliers, and $1 - w^n$ is the probability that at least one selected point is an outlier. The probability that in k trials only bad models are selected is then $(1 - w^n)^k$. If p is the probability of selecting the correct model, we have

$$1 - p = (1 - w^n)^k \quad (41.12)$$

Taking the logarithm of both sides lets us solve for k , the number of RANSAC iterations required to find a good fit, with probability p :

$$k = \frac{\log(1-p)}{\log(1-w^n)} \quad (41.13)$$

For the example of [figure 41.8](#), for 95 percent accuracy, $k = \frac{\log(0.05)}{\log(1-0.18^2)} = \frac{-1.3}{-0.0143} = 91$. So with 91 trials, we would have 95 percent confidence of finding the correct model through RANSAC. Note that in practice you should select points without replacement to avoid degenerate fits, but the previous analysis assumed that points are selected with replacement. This is all legitimate when the number of points selected is small relative to the set of available points.

In the case of the homography estimation, we need eight equations to use the DLT algorithm to estimate $\mathbf{H}$. As each point contributes two equations, we need four corresponding points. We can sample multiple randomly selected sets of four points and use RANSAC to estimate the homography between two images.

41.3.3 Image Stitching

Using DLT and RANSAC we can compute the homography between each pair of images in figure and align them with respect to a reference image (in this example we pick the middle image as a reference). We can use the estimated homographies to warp all of the images into a single camera view as shown in [figure 41.9](#).



Figure 41.9: Panoramic image composed by the three overlapping images from [figure 41.3](#). Only the middle picture is unmodified. The other two are warped using the estimated homographies.

41.4 Concluding Remarks

Homographies are an important class of geometric transforms between images and they have lots of applications. Homographies can be used to take measures on a plane given a reference measure. Although we have focused on computing homographies between images using correspondences, one can compute homographies in other ways. For instance, to produce a bird's-eye view of a plane one can compute the needed homography using the horizon line to get the camera rotation and, for an uncalibrated camera, we can use vanishing points to get the intrinsic camera parameters. The homography can also be extracted using a neural networks trained to regress the homography given two input images [2].

42 Single View Metrology

42.1 Introduction

The goal of this chapter is to introduce a set of techniques that allows recovering three-dimensional (3D) information about the scene from a single image like the picture of the office we used before ([figure 42.1](#)).



Figure 42.1: Is it possible to recover three-dimensional (3D) scene information from a single two-dimensional (2D) image? How tall is the desk? How wide is the bookshelf? What do we need to know about the scene in order to be able to answer those questions?

Single view metrology relies on understanding the geometry of the image formation process, the rules of perspective, and other visual cues present in the image to answer questions about the 3D structure of the scene. This is in contrast to other methods, like stereo vision, structure from motion or multiview geometry, which utilize multiple images to reconstruct 3D information. The reader should consult specialized monographs for a deeper analysis such as the book on multiview geometry by Hartley and Zisserman [187].

There are many scenarios where recovering 3D information from a single view is important. There are applications from which only one view is available, such as when looking at paintings or TV, when analyzing photographs, and so on. Even when we have stereo vision, stereo is only

effective when the disparity is large enough which limits the range of distances for which stereo can be used reliably.

We could just start by diving into learning-based methods and train a huge neural network to solve the task, but that would be no fun (we will do it later, no worries). Instead, we will start by using a model based single-view metrology. There will be no learning involved. Instead we will derive cues for 3D estimation from first principles.

42.2 A Few Notes About Perception of Depth by Humans

The image in [figure 42.2](#), recreated from Mezzanotte and Biederman [49], displays a flat set of lines that elicit a powerful 3D perception. Even more striking is that we perceive far more than mere 3D geometric figures (which is already intriguing, considering we are looking at a flat piece of paper). We discern buildings, people, a car in the street, the sky between the structures, and even what appears to be the entrance to a movie theater on the left sidewalk. Yet, the sidewalk is simply one line!

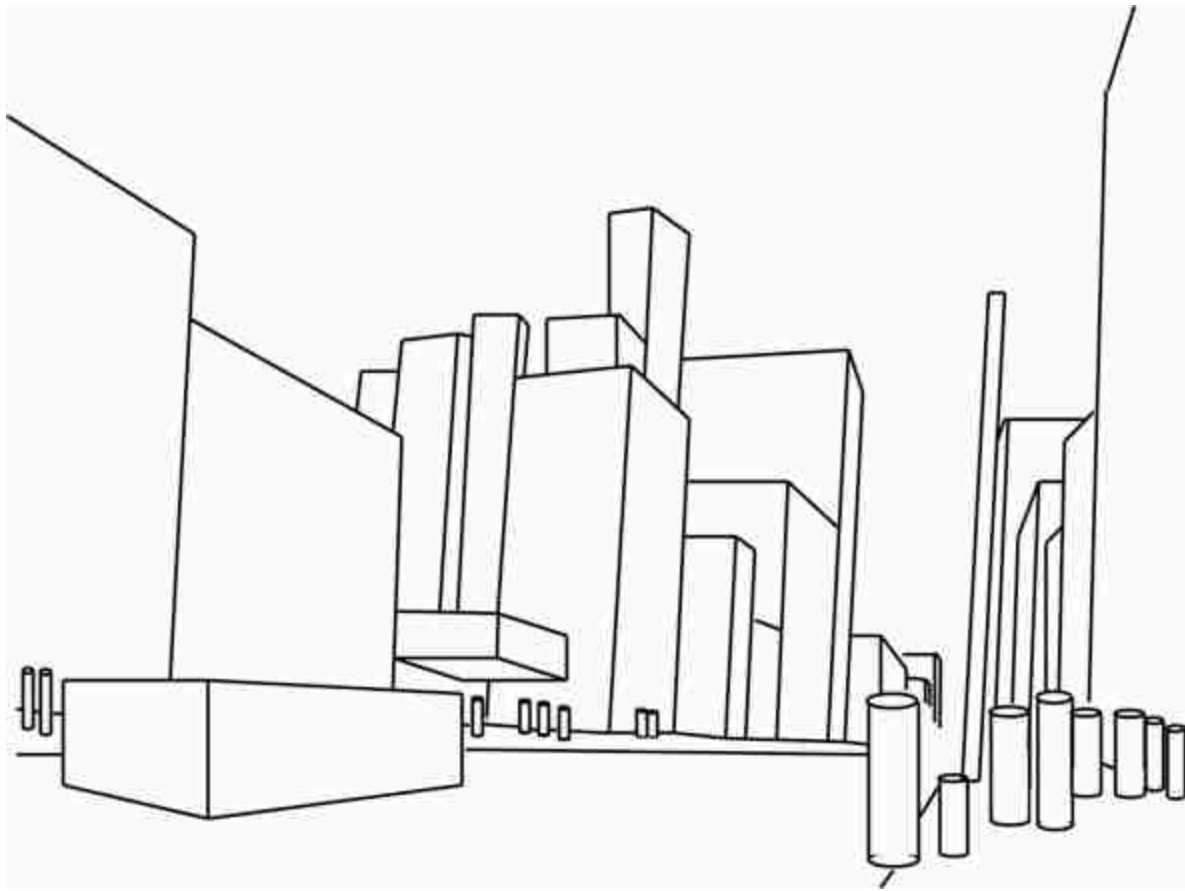


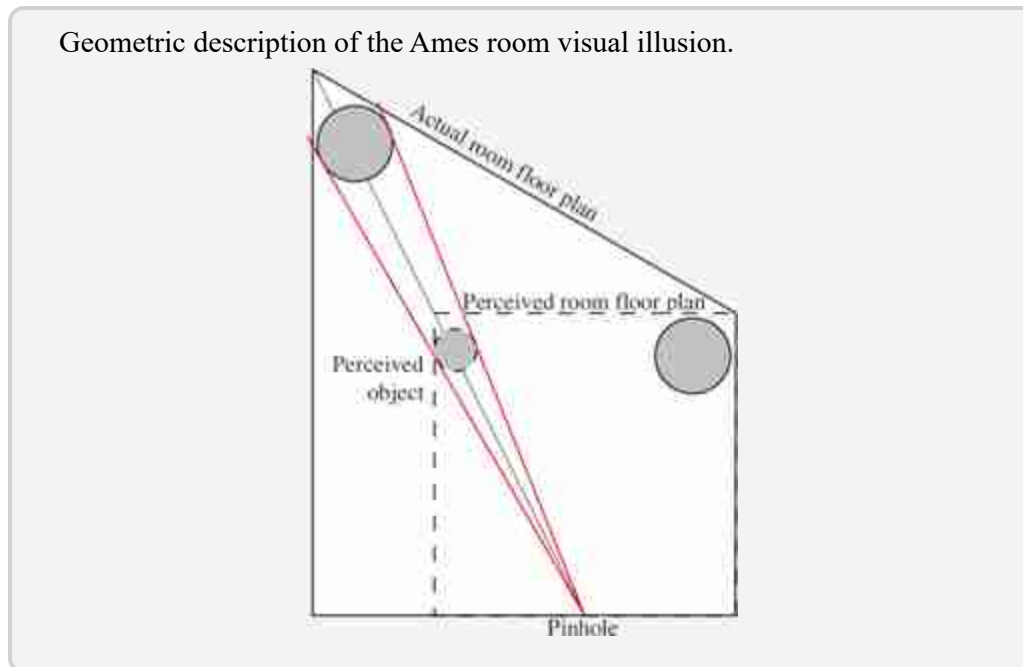
Figure 42.2: The image shows how scene understanding is possible from simple 3D cues and context. This sketch is a recreation of a real photograph shown in [49]. *Source:* Mezzanotte and Biederman.

We perceive everything we see in the sketch as 3D, even when we know that it's merely a 2D image. Our 3D processing is ingrained that we cannot easily switch off. That said, we are aware that it is just a collection of lines and geometric figures drawn on a flat piece of paper, but the 3D interpretation transforms the way in which we perceive the sketch. The influence of the 3D perceptual mechanisms is better appreciated when looking at visual illusions.

3D visual illusions are remarkably powerful. One captivating 3D illusion is the **Ames room**.

The Ames room was invented by Adelbert Ames, director of research at the Dartmouth Eye Institute, in 1946. Since then, illusions based on Ames' ideas have been utilized by comedians to create amusing effects.

The Ames room is an irregular room constructed in such a way that when you look at it with one eye from a particular vantage point, the room appears to be perfectly rectangular. This is achieved by making the part of the wall that is farther away from the viewer larger so that it appears fronto-parallel.



The best way to understand this illusion is by building it and experiencing it by yourself. You can find the instructions in [figure 42.3](#). When you look through the aperture on the wall indicated by “cut” the room will appear square and smaller than it actually is, and it will seem as if you are looking inside the room from the middle of it despite that the opening is closer to the right wall.

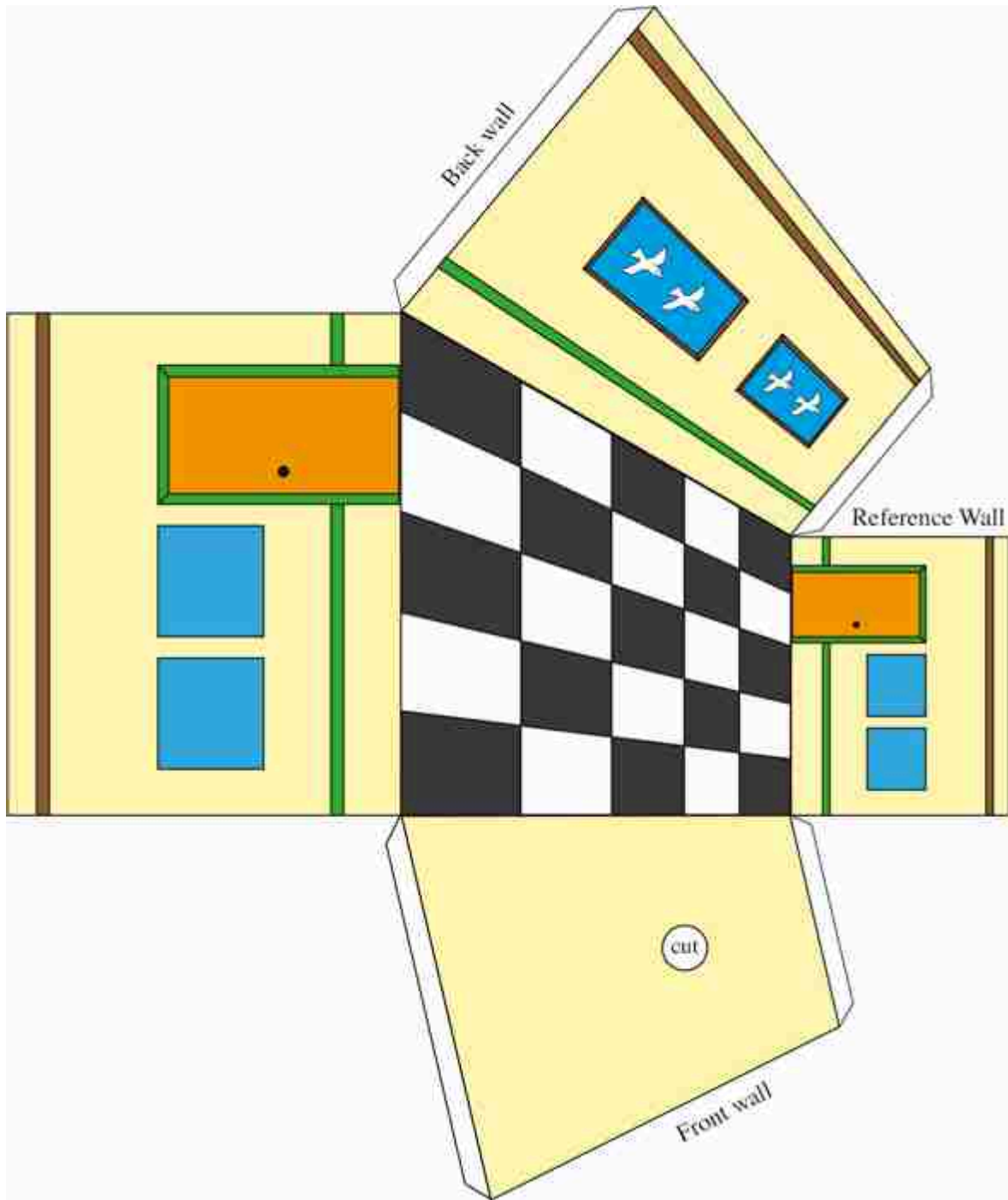


Figure 42.3: Make your own Ames room. Take a picture of this page and print it as large as you can (A4 or letter size will be sufficient). When you look through the aperture on the *front wall*, the room will appear square and smaller than it actually is. The shape of the *reference wall* is the only one that will appear unchanged, while all other planes will appear smaller than their actual size. You can change the room's content by drawing on the *reference wall* and then warping the image onto the other walls using homography. The floor pattern results from a homography with a square floor plan.

This illusion is especially remarkable when the room is large enough for people to walk in. You will notice that as people walk along the back wall, they appear to change in height. Your perception will disregard all of your (very strong) prior expectations about how that person should appear. [Figure 42.4](#) illustrates this phenomenon with our cut-out version of the Ames room.

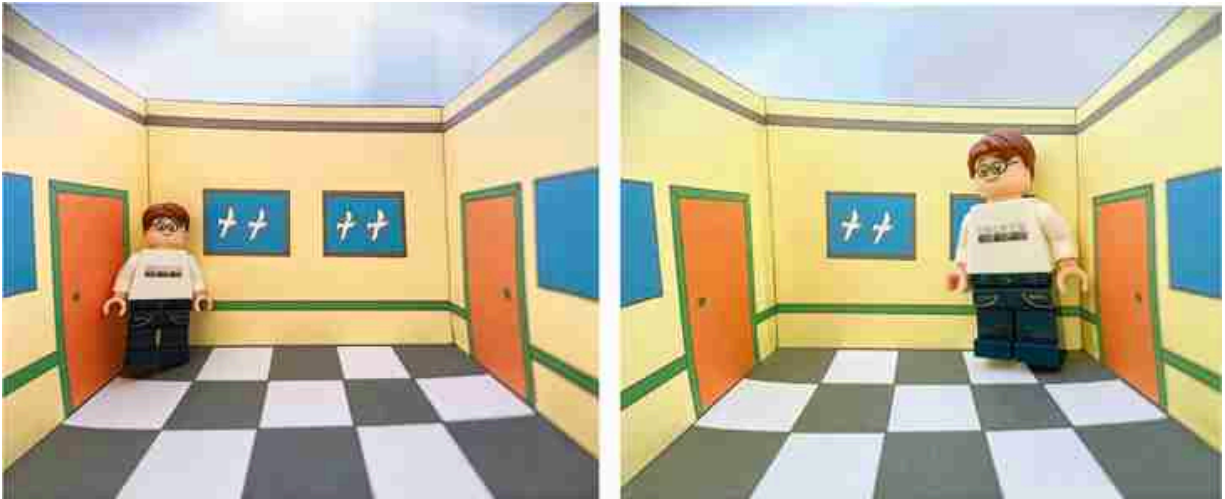


Figure 42.4: If you place an object along the back wall of the Ames room shown in [figure 42.3](#), its perceived size will change. Also note that both doors appear to be similar in size despite being quite different in reality.

Clearly, a single image contains very strong cues about the 3D scene structure. In the following sections we will study linear perspective, arguably the most powerful among all pictorial cues used for 3D interpretation from a 2D image.

42.3 Linear Perspective

Linear perspective uses parallel and orthogonal lines, vanishing points, the ground plane and the horizon line to infer the 3D scene structure from a 2D image.

The discovery of linear perspective is attributed to the Italian architect Filippo Brunelleschi in 1415.

As we will see in the following sections, linear perspective relies on a few assumptions about the scene and, as a consequence, linear perspective is not a general cue that is useful in all scenes, but it is very powerful in most human-made environments.

42.3.1 Projection of 3D Lines

In perspective projection, 3D lines project into 2D lines in the camera plane. Let's start writing the parametric vector form of a 3D line using heterogeneous coordinates. We can parameterize a 3D line with a point, $\mathbf{P}_0 = [X_0, Y_0, Z_0]^T$, and a direction vector, $\mathbf{D} = [D_X, D_Y, D_Z]^T$, as:

$$\mathbf{P}_t = \begin{bmatrix} X_0 + tD_X \\ Y_0 + tD_Y \\ Z_0 + tD_Z \end{bmatrix} \quad (42.1)$$

The 3D point $\mathbf{P}_t$ moves along a line when the independent variable t goes from $-\infty$ to ∞ . A camera located in the world coordinates origin will observe a 2D line resulting from projecting the 3D line into the camera plane as shown in the sketch shown in [figure 42.5](#).

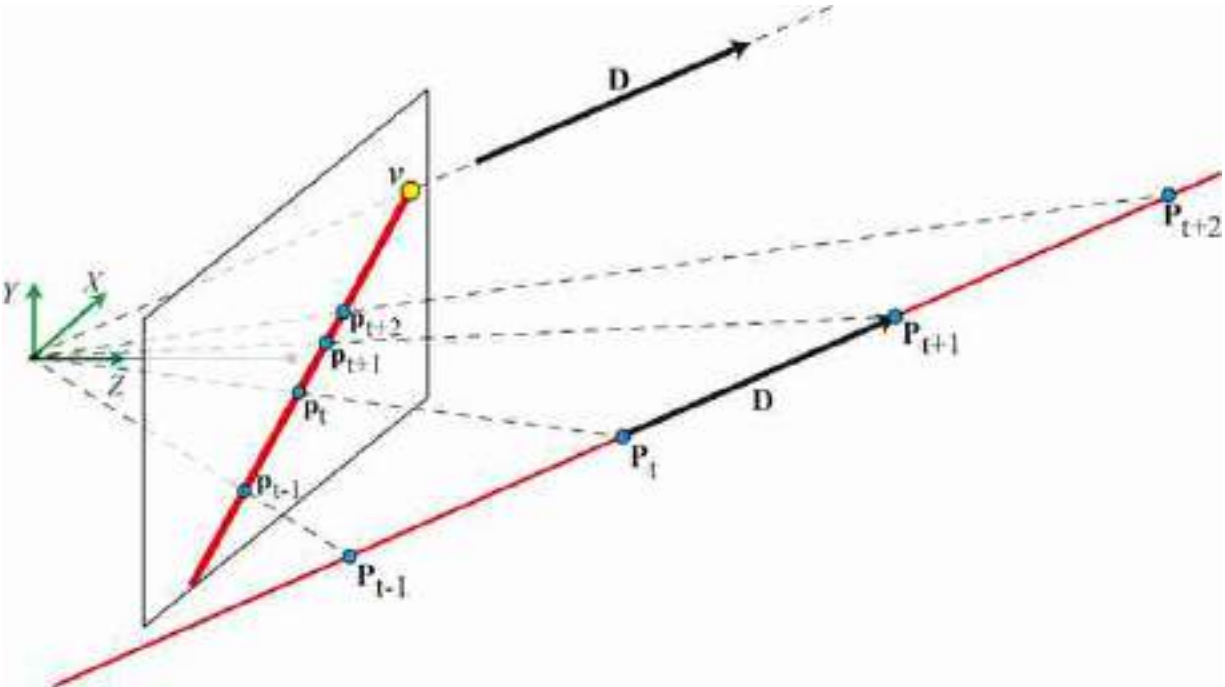


Figure 42.5: Projection of a 3D line onto the camera plane. The 3D line and the camera center form a plane (the camera center is the world-coordinates origin). The intersection of this plane with the camera plane is the 2D line that appears in the image. The image point v is the vanishing point of the line.

We can get the equation of the 2D line that appears in the image by using the perspective projection equations, resulting in:

$$\mathbf{p}_t = \begin{bmatrix} x_t \\ y_t \end{bmatrix} = \begin{bmatrix} f \frac{X_0 + tD_X}{Z_0 + tD_Z} \\ f \frac{Y_0 + tD_Y}{Z_0 + tD_Z} \end{bmatrix} \quad (42.2)$$

One subtle point about what is visible by the camera: Remember that when we plot the image plane we are using the virtual camera plane, but the real “image” is being formed behind the pinhole. So, in the example shown in [figure 42.5](#), the points along the line with positive Z -value might be visible even when they are behind the virtual camera plane.

42.3.2 Vanishing Points

Even though a line in 3D space extends infinitely, its 2D projection doesn't. According to equation (42.2), in the limit when t goes to infinity, the line converges to a finite point called the vanishing point:

$$\mathbf{v} = \lim_{l \rightarrow \infty} \mathbf{p}_l = \begin{bmatrix} f \frac{D_x}{D_z} \\ f \frac{D_y}{D_z} \end{bmatrix} \quad (42.3)$$

The vanishing point, $\mathbf{v} = [v_x, v_y]^T$, only depends on the direction vector, $\mathbf{D}$, thus all parallel lines in 3D project into non-parallel 2D images that converge to the same vanishing point in the image. If $D_z = 0$ then the 3D line is parallel to the camera plane and the vanishing point is in infinity.

[Figure 42.6](#) shows a set of parallel 3D lines and their 2D image projections. Each group of parallel lines converges to a different vanishing point. Lines that are parallel to the camera plane remain parallel as they converge to a vanishing point in infinity.

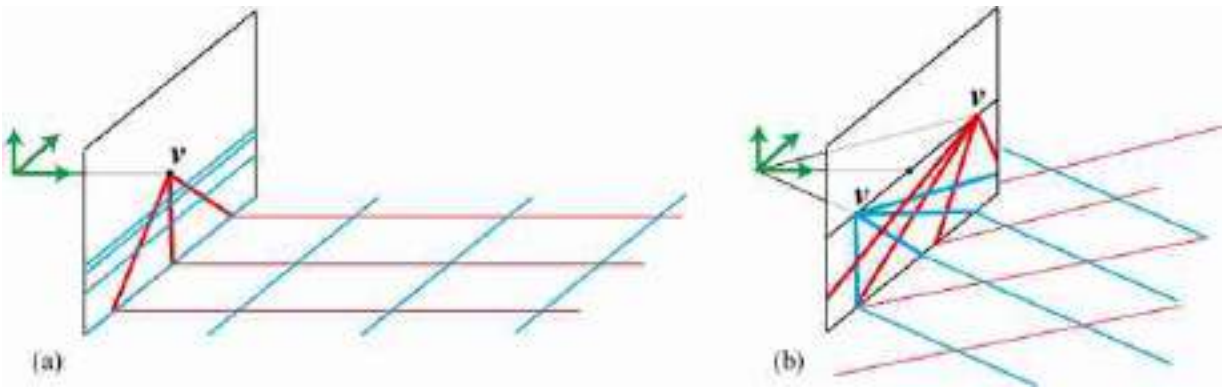


Figure 42.6: Parallel 3D lines converge to the same vanishing point. (a) The red lines are parallel to the optical axis and converge to a vanishing point in the center. (b) Two sets of parallel lines contained in an horizontal plane, parallel to the optical axis. Their vanishing points are on an horizontal image line.

All the sets of parallel lines inside a plane converge to a set of co-linear vanishing points. These points lie along the plane's horizon line.

Vanishing points are very useful to recover the missing depth of the scene as we will see later. The human visual system also makes use of vanishing points for 3D perception. Take, for example, the *leaning tower visual illusion* [258] as depicted in [figure 42.7](#). Here, a pair of identical images of the Leaning Tower of Pisa are placed side by side. Interestingly, despite being identical, they create an illusion of leaning with different angles. The image on the right appears to have a greater tilt than the one on

the left. One explanation of this illusion is that the lines of each tower converge at a different vanishing point. If both towers were parallel in 3D, all the lines would converge to the same 2D vanishing point. As they do not, our brain interprets that the towers should have different 3D orientations. This process is automatic and we can not shut it down despite the fact that we know both images are identical.

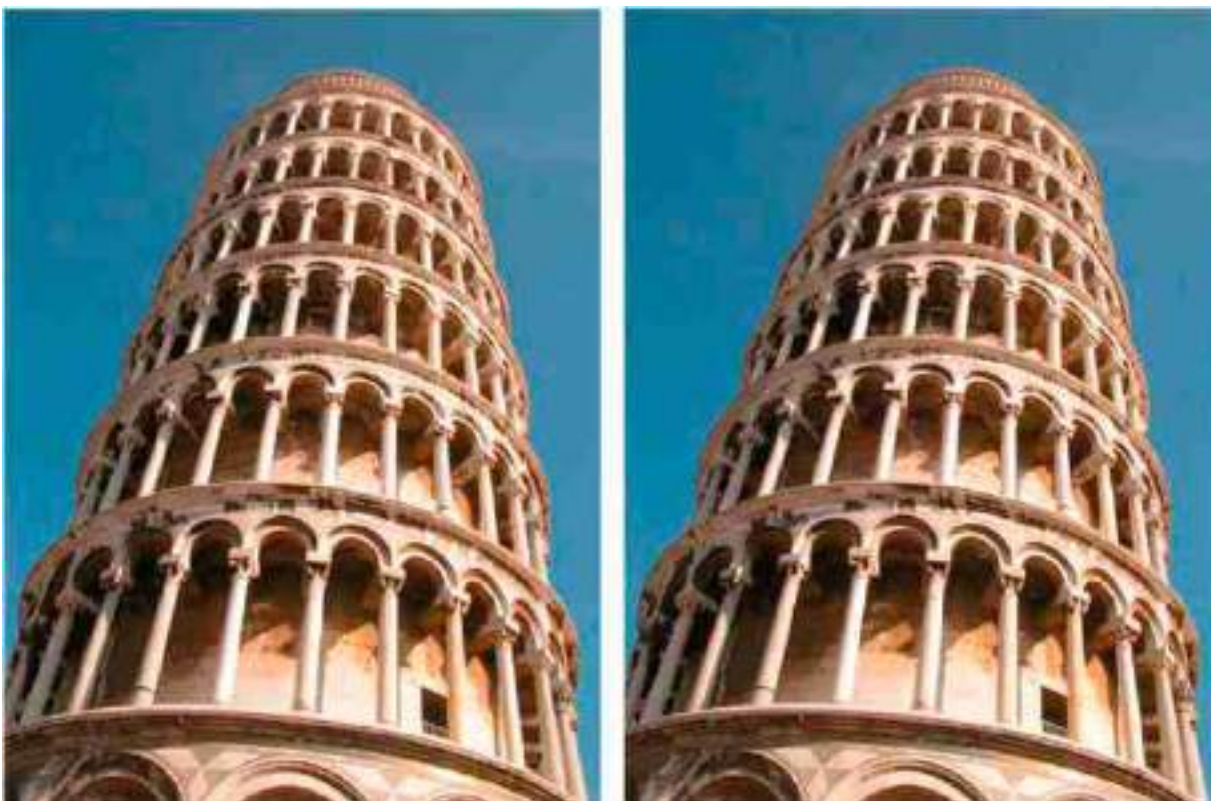


Figure 42.7: The *leaning tower illusion* [258] involves two identical pictures placed side by side. The picture on the right appears to be more tilted than the one on the left.

Homogeneous coordinates offer an alternative way of working with vanishing points. Let's start by writing the parametric vector form of a 3D line using homogeneous coordinates:

$$\mathbf{P}_t = \begin{bmatrix} X_0 + tD_X \\ Y_0 + tD_Y \\ Z_0 + tD_Z \\ 1 \end{bmatrix} = \begin{bmatrix} X_0/t + D_X \\ Y_0/t + D_Y \\ Z_0/t + D_Z \\ 1/t \end{bmatrix} \quad (42.4)$$

The first term is equivalent to the writing of the line in equation (42.1). As homogeneous coordinates are invariant to a global scaling, we can divide the vector by t which results in the second equality.

In the limit $t \rightarrow \infty$ the fourth coordinate of equation (42.4) converges resulting in:

$$\mathbf{P}_\infty = \begin{bmatrix} D_X \\ D_Y \\ D_Z \\ 0 \end{bmatrix} = \begin{bmatrix} \mathbf{D} \\ 0 \end{bmatrix} \quad (42.5)$$

Note that here we are representing a point at infinity with finite homogeneous coordinates. This is another of the advantages of homogeneous coordinates.

Homogeneous coordinates allow us to represent points at infinity with finite coordinates by setting the last element to 0.

The vanishing point, in homogeneous coordinates, is the result of projecting $\mathbf{P}_\infty$ using the camera model, $\mathbf{M}$:

$$\mathbf{v} = \mathbf{M}\mathbf{P}_\infty = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{T} \end{bmatrix} \begin{bmatrix} \mathbf{D} \\ 0 \end{bmatrix} = \mathbf{K}\mathbf{R}\mathbf{D} \quad (42.6)$$

The last equality is obtained by using equation (39.17), and it shows that the vanishing point coordinates do not depend on camera translation. Only a camera rotation will change the location of the vanishing points in an image. If a scene is captured by two identical cameras translated from each other, their corresponding images will have the same vanishing points.

Both equation (42.3) and equation (42.6) are equivalent if we replace $\mathbf{K}$ by the appropriate perspective camera model.

If instead of perspective projection, we have parallel projection, then all the vanishing points will project at infinity in the image plane.

42.3.3 Horizon Line

Any 3D plane becomes a half-plane in the image and has an horizon line. The horizon line is the line that passes by all the vanishing points of all the

lines contained in the plane as shown in [figure 42.8](#).

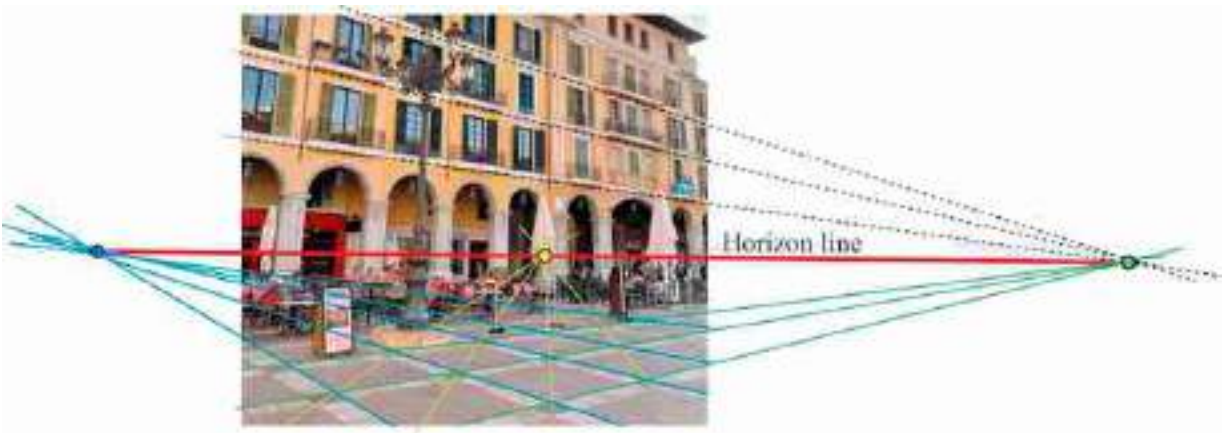


Figure 42.8: All parallel lines contained in a plane converge to a set of vanishing points. All those vanishing points are contained in the **horizon line** of the plane. Parallel planes have the same horizon line.

As parallel lines converge to the same vanishing point, parallel planes also have the same horizon line. In [figure 42.8](#), the lines in the building facade that are parallel to the ground also converge to a vanishing point that lies on the horizon line. In this particular examples, the lines in the facade also happen to be parallel to one of the axis of the floor tiles, converging to the same vanishing point on the right side of the figure.

The ground plane is a special plane because it supports most objects. This is different than a wall. Both are planes, but the ground has an ecological importance that a wall does not have. The point of contact between objects and the ground plane informs about its distance. The distance between the contact point and the horizon line is related to distance between the object and the camera. Objects further away have a contact point projecting onto a higher point in the image plane.

[Figure 42.9](#) illustrates how to detect the horizon line using homogeneous coordinates. If lines l_1 and l_2 , in homogeneous coordinates, correspond to the 2D projection of 3D parallel lines, we can obtain the vanishing point as their intersection (cross-product, $l_1 \times l_2$). If we now have another set of parallel lines, contained in the same plane, l_3 and l_4 , we can get another vanishing point by computing their cross-product. Finally, the horizon line

is the line that passes through both points that can be computed as the cross-product of the vanishing points in homogeneous coordinates.

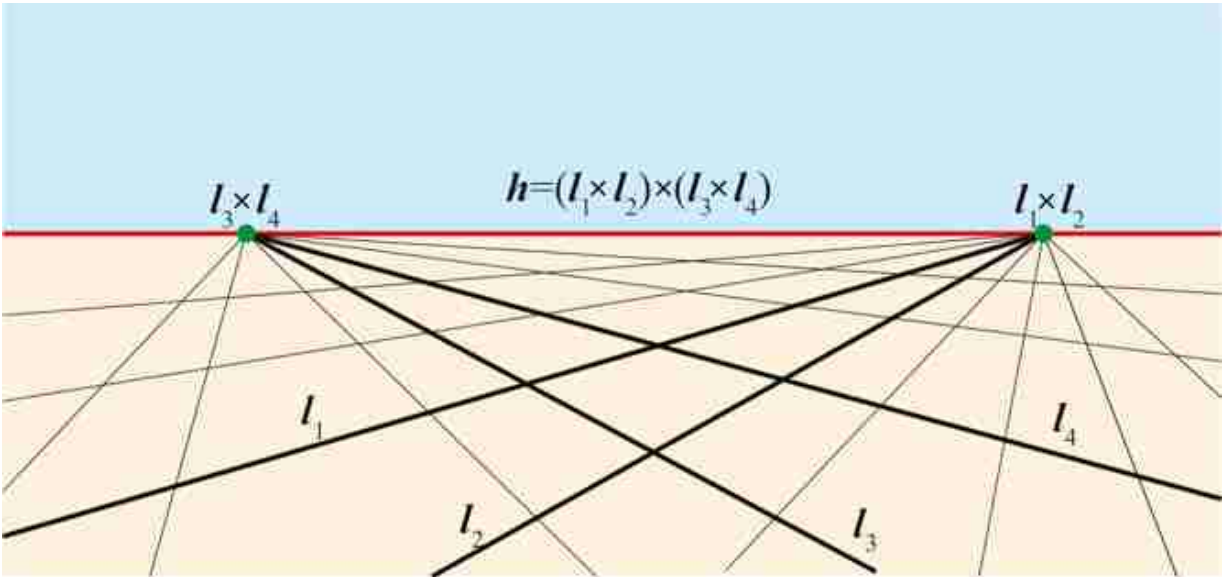


Figure 42.9: Using parallel lines, in homogeneous coordinates, to compute vanishing points and the horizon line.

42.3.4 Detecting Vanishing Points

[Figure 42.10](#) shows the office scene and three vanishing points computed manually. In this scene there are three dominant orientations along orthogonal directions. The three vanishing points are estimated by selecting pairs of long edges in the image corresponding to parallel 3D lines in the scene and finding their intersection.

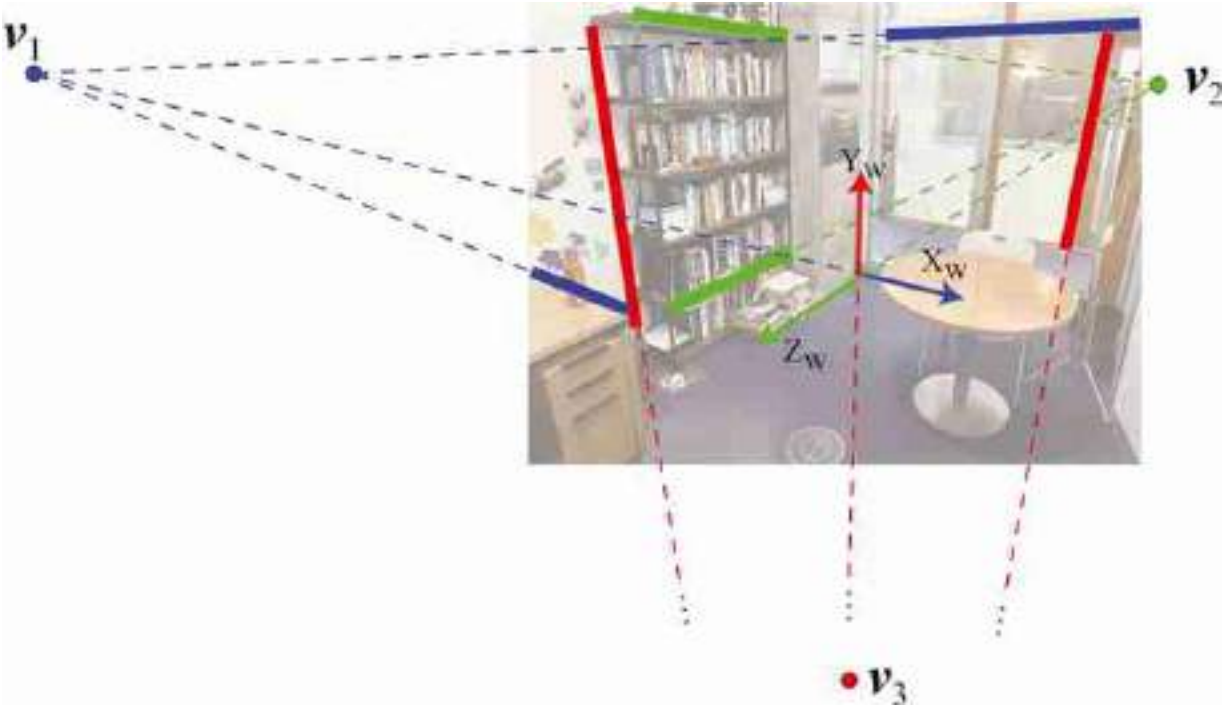


Figure 42.10: Most object boundaries in this photograph are oriented parallel to one of the world-coordinates reference frame (the world-coordinates frame is chosen in this way on purpose). All the parallel 3D lines converge on the image plane in a vanishing point. In this picture there are three main vanishing points, each of them related to one of the world-coordinates frame.

Vanishing points can be extracted automatically from images by first detecting image edges and their orientations (like we did in chapter 2) and then using RANSAC (section 41.3.2) to group the edges: every pair of edges will vote for a vanishing point and the point with the largest support will be considered a vanishing point.

There are also learning-based approaches for vanishing point detection using graphical models [84], or deep learning based approaches such as DeepVP [74] and NeurVPS [530].

42.4 Measuring Heights Using Parallel Lines

Let's now use what we have seen up to now to measurement distances in the world from a single image. We will discuss in this section how to make those measurements using three vanishing points, or just the horizon line and one vanishing point. We will not use camera calibration or the

projection matrix. For a deeper analysis we refer the reader to the wonderful thesis of A. Criminisi [85].

Before we can make any 3D measurements we need to study an important invariant of perspective projection: the cross-ratio.

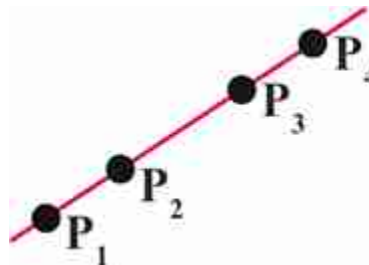
42.4.1 Cross-Ratio

As we have discussed before, perspective projection transforms many aspects of the 3D world when projecting into the 2D image: angles between lines are not preserved, the relative lengths between lines is not preserved, parallel 3D lines do not project into 2D parallel lines in the image (unless they are also parallel to the camera plane), and so on. Fortunately, some geometric properties are preserved under perspective projection. For instance, straight lines remain straight. Are there other quantities that are preserved? The cross-ratio is one of those quantities.

The cross-ratio of four collinear points is defined by the expression:

$$CR(P_1, P_2, P_3, P_4) = \frac{|P_3 - P_1|}{|P_3 - P_2|} \frac{|P_4 - P_2|}{|P_4 - P_1|} \quad (42.7)$$

where $|P_i - P_j|$ is the euclidean distance between points P_i and P_j , using heterogeneous coordinates.



The cross-ratio is considered the most important projective invariant. Two sets of points P and Q related by a projective transformation, as shown in [figure 42.11](#), have the same cross-ratio (regardless of the choice of origin O or scale factor). That is:

$$CR(P_1, P_2, P_3, P_4) = CR(Q_1, Q_2, Q_3, Q_4) \quad (42.8)$$

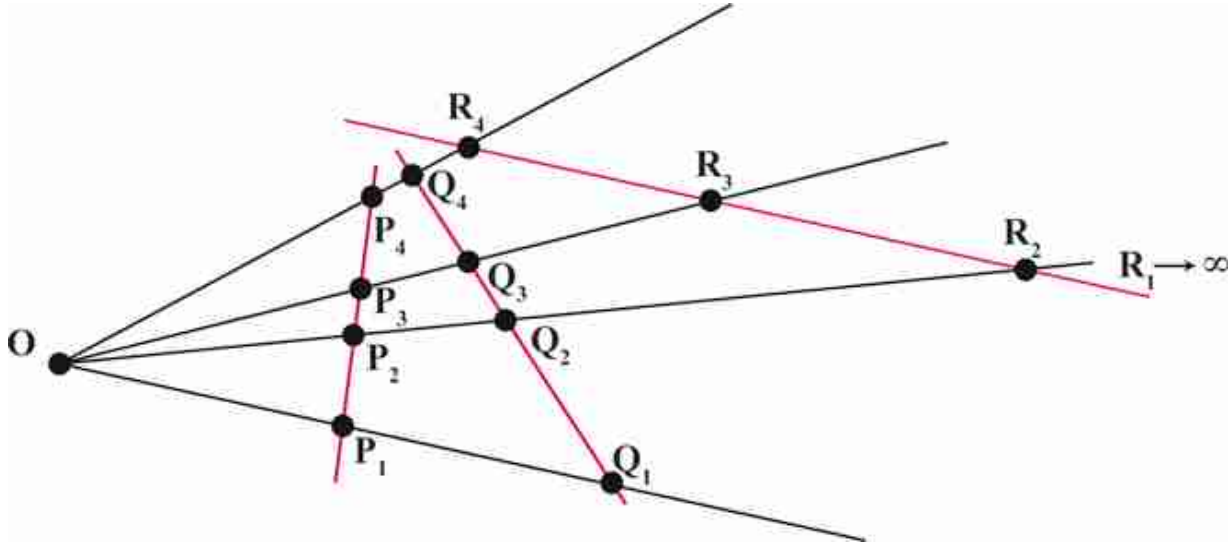


Figure 42.11: The set of points **P** are related to the points **Q** (and **R**) by the cross-ratio. The point **R**₁ is in infinity because the line connecting the points **R** is parallel to the line passing by **O**, **P**₁, and **Q**₁.

The order in which these points are considered matters in the calculation. There are $4! = 24$ possible orderings resulting in six different values. If two sets of points, related by a projection, have the same ordering, then their cross ratio will be preserved. We do not provide the proof for the cross-ratio invariant here.

When one of the points is at infinity, all the terms that contain that point are cancelled out (this can also be seen by taking a limit). In the case of the points **R** from [figure 42.11](#) we get that cross-ratio is:

$$CR(R_1, R_2, R_3, R_4) = \frac{|R_4 - R_2|}{|R_3 - R_2|} \quad (42.9)$$

This value is equal to the cross-ratio of the points **P** and **Q**.

To gain some intuition about the cross-ratio invariant, let's look at an empirical demonstration. [Figure 42.12](#) shows a ruler seen under different view points. The blue dots correspond to same ruler locations with $P_1 = 0$ in, $P_2 = 6$ in, $P_3 = 8$ in, and $P_4 = 10$ in (we are making these measurements in inches, but note that the units do not affect the cross-ratio as it is scale invariant). Using those measurements, the cross-ratio is: $(8 \times 4)/(2 \times 10) =$

1.6. The same cross-ratio is obtained if we measure the positions of the four points using image coordinates and measuring distances in the image plane.

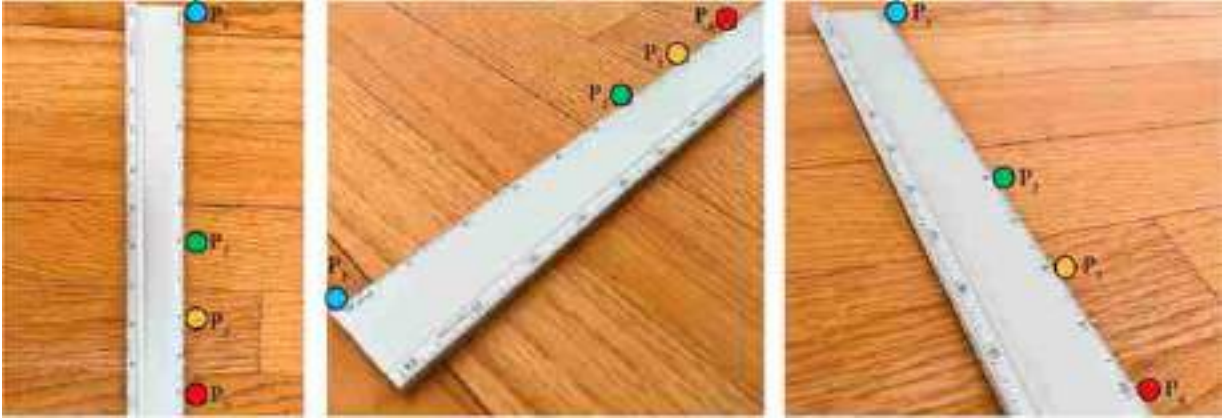


Figure 42.12: Empirical demonstration of the cross-ratio invariant. You can check that measuring distances between the three dots on each image and computing the cross-ratio results in the value 1.6 for the three images.

42.4.2 Measuring the Height of an Object

The problem we will address in this section is how to measure the height of one object given the height of a reference object in the scene. As shown in [figure 42.13](#), the height of the bookshelf is $h_{\text{bookshelf}} = 197$ cm (in this example we will make our measurements using centimeters, but again, the choice of units is not important as long as all the measurements are consistent). Can we use this information together with the geometry of the scene to estimate the height of the desk, h_{desk} ?

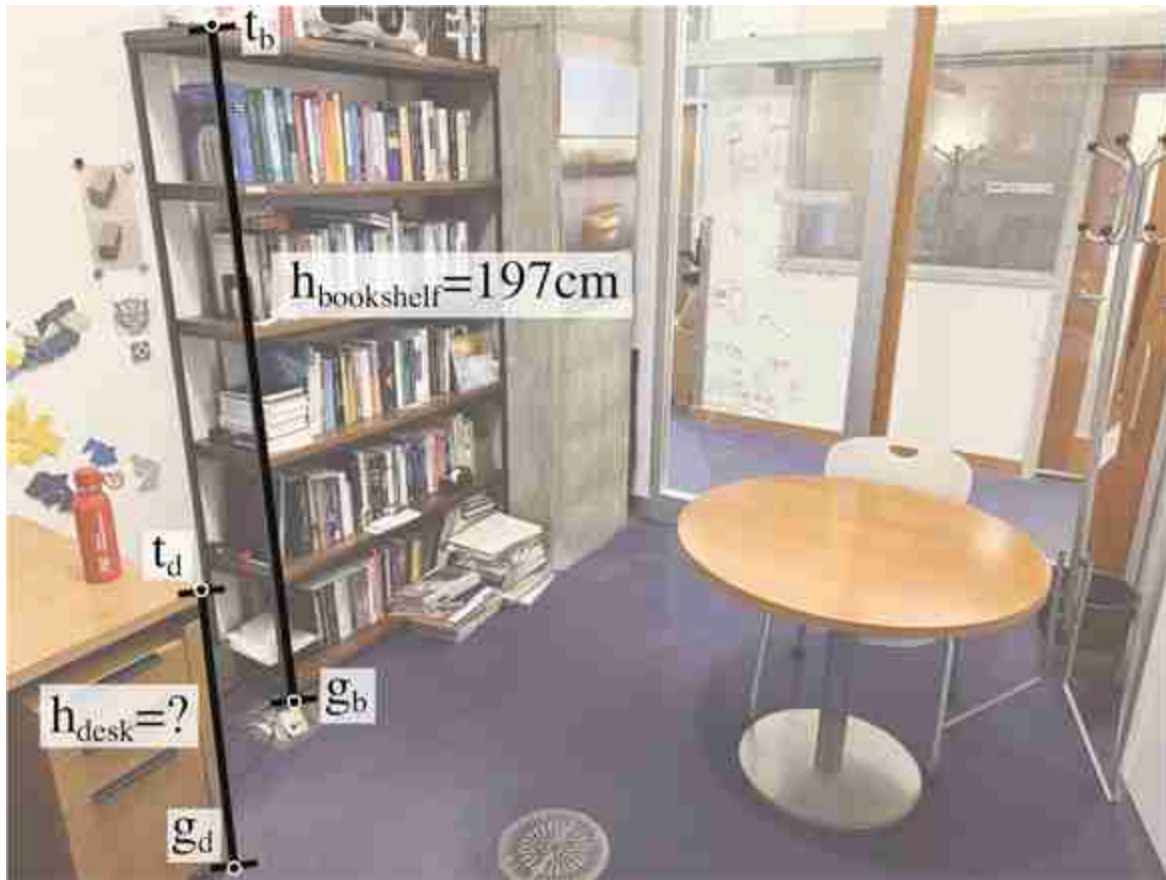


Figure 42.13: Given the height of a reference object, the location of the horizon line and one vanishing point, how can we estimate the height of other objects in the scene?

Due to perspective projection, we can not directly compare the relative height of the desk with respect to the height of the bookshelf using distances measured over the image. In order to be able to compare 3D distances using 2D image distances both objects need to be put in correspondence first. The bookshelf height is the distance between the 3D points that correspond to the 2D locations of the bottom of the bookshelf, $\mathbf{g}_b$, and the top of the bookshelf, $\mathbf{t}_b$. Similarly, the desk height is the distance between the 3D points that correspond to $\mathbf{g}_d$ and $\mathbf{t}_d$. We can not use directly the points $\mathbf{g}_b$ and $\mathbf{t}_b$, and $\mathbf{g}_d$ and $\mathbf{t}_d$ as they are given in image coordinates. In order to be able to use distances in the image domain we need to first translate the points defining the desk height on top of the bookshelf, and then use the cross-ratio invariant to relate the ratio between image distances with the ratios of 3D heights.

In the rest, all the vectors refer to image coordinates in homogeneous coordinates. Let's start by projecting the desk height onto the bookshelf. To do this we need to follow several steps:

- Estimate the line that passes by the ground points $\mathbf{g}_d$ and $\mathbf{g}_b$. Using homogeneous coordinates, the line can be computed using the cross-product:

$$\mathbf{l}_1 = \mathbf{g}_d \times \mathbf{g}_b \quad (42.10)$$

- Compute the intersection of the line $\mathbf{l}_1$ with the horizon line $\mathbf{h}$ as:

$$\mathbf{a} = \mathbf{l}_1 \times \mathbf{h} \quad (42.11)$$

As the line $\mathbf{l}_1$ is on the ground plane, the point $\mathbf{a}$ is the vanishing point of line $\mathbf{l}_1$.

- Compute the line that passes by $\mathbf{a}$ and the top of the desk $\mathbf{t}_d$ as:

$$\mathbf{l}_2 = \mathbf{a} \times \mathbf{t}_d \quad (42.12)$$

Lines $\mathbf{l}_1$ and $\mathbf{l}_2$ are parallel on 3D and vertically aligned.

- Finally we can compute where line $\mathbf{l}_2$ intersects with the bookshelf in the image plane which in this case will also correspond to an intersection of the corresponding 3D lines. We first compute the line that connects the bottom and top points of the bookshelf $\mathbf{l}_3 = \mathbf{g}_b \times \mathbf{t}_b$, and now we obtain the intersection with $\mathbf{l}_2$ as:

$$\mathbf{b} = \mathbf{l}_2 \times \mathbf{l}_3 \quad (42.13)$$

We can write all the previous steps into a single equation:

$$\mathbf{b} = (((\mathbf{g}_d \times \mathbf{g}_b) \times \mathbf{h}) \times \mathbf{t}_d) \times (\mathbf{g}_b \times \mathbf{t}_b) \quad (42.14)$$

Point $\mathbf{b}$ is the location where the top of the desk will be if we physically rearrange the furniture to bring the desk and the bookshelf into contact. The segment between points $\mathbf{b}$ and $\mathbf{g}_b$ is the projection of the desk height on the bookshelf. However, due to the foreshortening effect induced by perspective projection, we cannot simply rely on the ratio of image distances to estimate the desk's true height. Instead, we will utilize the cross-ratio invariant to make this estimation.

As J. J. Gibson pointed out, the ground plane is very useful for all kinds of 3D scene measurements.

In order to use the cross-ratio invariant we need two sets of four colinear points related by a projective geometry. [Figure 42.15](#) shows a sketch of the

setup we have in [figure 42.14](#).

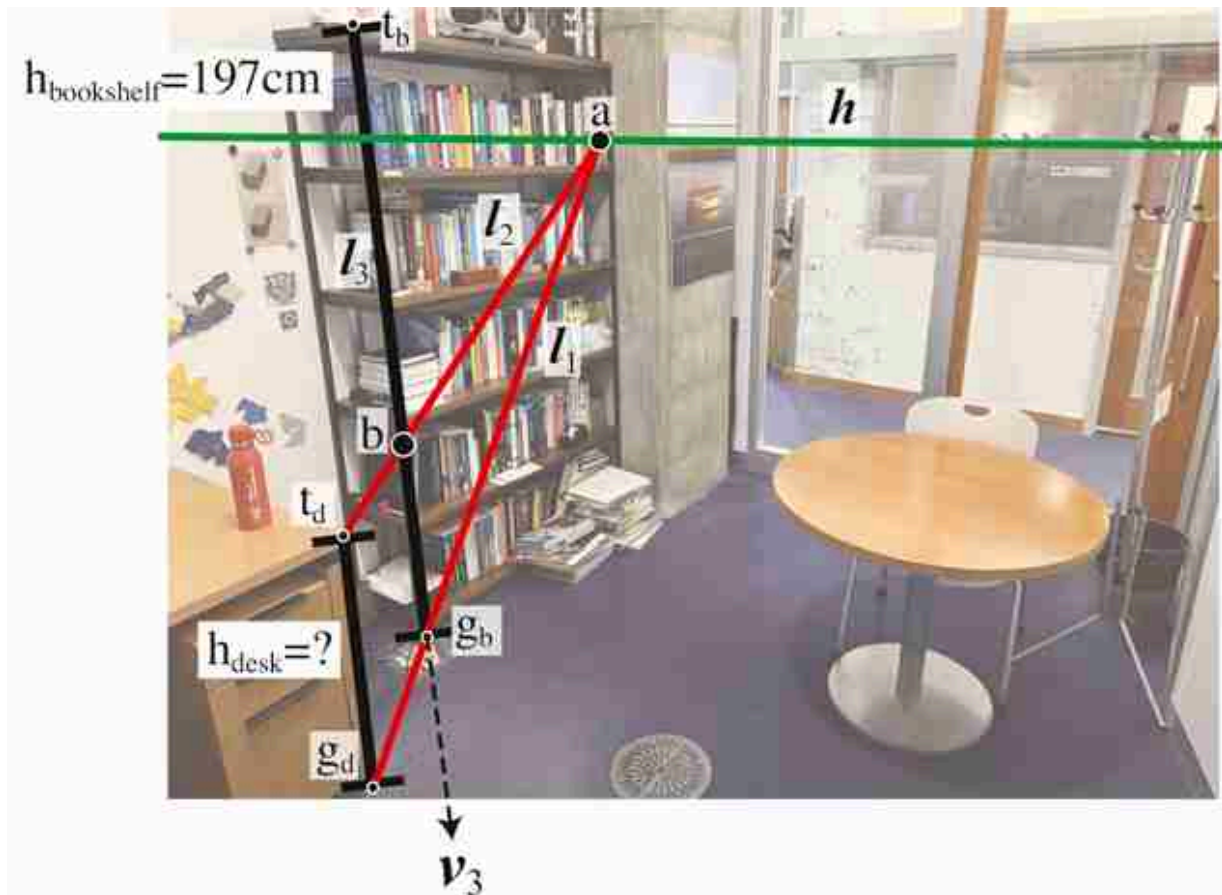


Figure 42.14: Projection of desk height onto the bookshelf. We start with the bottom and top image coordinates of points along vertical edges of the bookshelf: $\mathbf{t}_b = [702 \ 2,958]^T$ $\mathbf{g}_b = [987 \ 618]^T$ and the desk: $\mathbf{t}_d = [679 \ 1,018]^T$ $\mathbf{g}_d = [793 \ 59]^T$ We compute the points $\mathbf{a} = [4,471 \ 2,486]^T$ and $\mathbf{b} = [2,237 \ 2,765]^T$ as described in the text.

In the image plane, we have the four image points $\mathbf{t}_b$, $\mathbf{b}$, $\mathbf{g}_b$ and $\mathbf{v}_3$. Those four 2D image points correspond to the 3D locations given by the top of the bookshelf, desk height, ground and infinity (which is the location of the vertical vanishing point in 3D).

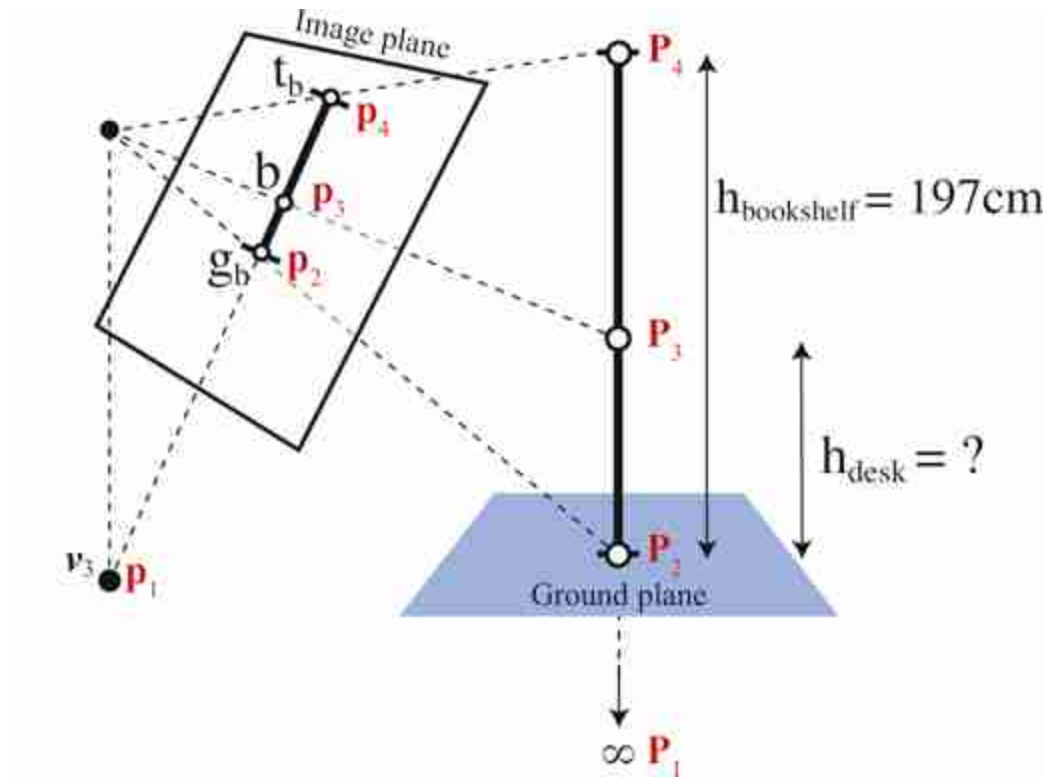


Figure 42.15: Sketch of the setting from [figure 42.14](#). There are two sets of colinear points related by central projection. Their cross-ratios are identical.

The rest of calculations requires computing distances between points, which only works when using heterogeneous coordinates (or homogeneous coordinates if the last component is equal to 1 for all points). The cross-ratio invariant between both sets of four points gives as the equality:

$$CR(P_1, P_2, P_3, P_4) = CR(p_1, p_2, p_3, p_4) \quad (42.15)$$

The left side of the equality corresponds to the distances between the 3D points and the right side to the distances of the image points measured in pixel units. As P_1 is in infinity (as it corresponds to the vanishing point), the left side ratio only has two terms, resulting in the ratio between the bookshelf and desk heights. To compute the hand side we need the coordinates of the corresponding four image points. Those are provided in [figure 42.14](#). Replacing all those values in equation (42.15) we obtain:

$$\frac{h_{\text{bookshelf}}}{h_{\text{desk}}} = \frac{\|\mathbf{b} - \mathbf{v}_3\| \|\mathbf{t}_b - \mathbf{g}_b\|}{\|\mathbf{b} - \mathbf{g}_b\| \|\mathbf{t}_b - \mathbf{v}_3\|} \quad (42.16)$$

The only unknown value in this equality is the height of the desk, h_{desk} , which results in $h_{\text{desk}} \approx 73.1$ cm, which is close to the actual height of the desk (which is around 76 cm).

Projective invariants are measures that do not change after perspective projection. The cross-ratio is the most important projective invariant.

42.4.3 Height Propagation to Supported Objects

The method described in the previous section relies on using vertical lines in contact with the ground plane to propagate information from one point in space to another in order to make measurements. The method can not work if we can not establish the vertical projection of a point into the ground plane (or any other plane of reference). But once we have estimated the height of objects that are in contact with the ground, we can use them to propagate 3D information to other objects not directly on top of the ground. We can estimate the height of objects that are not in contact with the ground if they are on top of objects of known height. As an example, let's estimate the red bottle's height, which we can do after we estimated the height of the desk.

[Figure 42.16](#) shows how to estimate the height of the bottle.

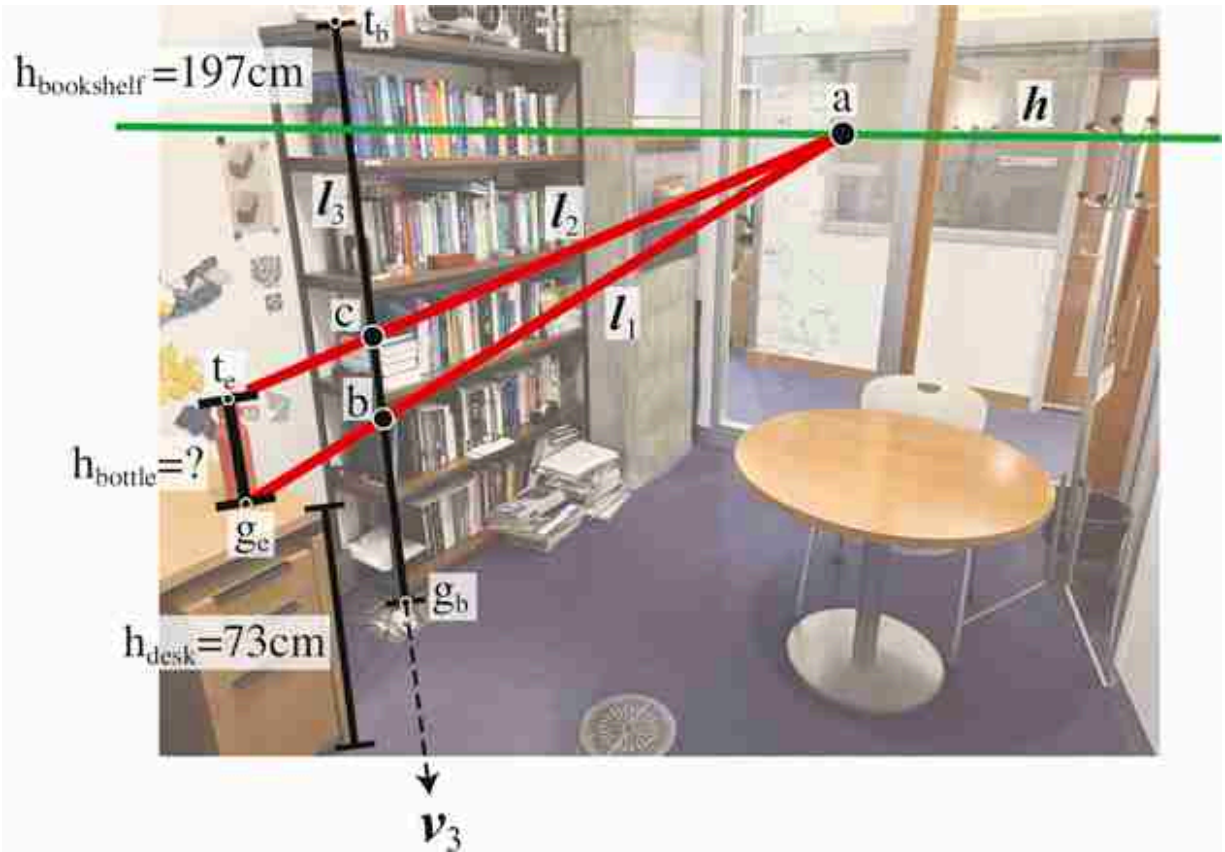


Figure 42.16: Estimating the height of the red bottle. For an object not in the ground we can propagate information from the objects that support it.

As before, we will project the bottle height on the reference line on the corner of the bookshelf. To do this we need to find two vertically aligned parallel 3D lines that connect the bottle's top and bottom points with the bookshelf. The first line, l_1 , is the line connecting the bottom of the bottle, g_e , and the point b . These two points are at the same height from the ground plane as b corresponds to the top of the desk and g_e is the point of the bottle that is in contact with the top of the desk. The intersection of the line l_1 with the horizon line h give us point a . Connecting point a with the top of the bottle, point t_e , gives us the second line, l_2 . The intersection of this line with the reference bookshelf line is the point c . The distance between the point c and the ground in 3D is the sum $h_{\text{bookshelf}} + h_{\text{desk}}$. Writing everything into a compact equation results in:

$$\mathbf{c} \equiv (((\mathbf{g}_c \times \mathbf{b}) \times \mathbf{h}) \times \mathbf{t}_c) \times (\mathbf{g}_b \times \mathbf{t}_b) \quad (42.17)$$

Note that this equation relies on having computed $\mathbf{b}$ first.

Using the cross-ratio invariant we arrive to the following equation:

$$h_{\text{bottle}} = h_{\text{bookshelf}} \frac{\left| \frac{\mathbf{c} - \mathbf{g}_b}{\mathbf{c} - \mathbf{v}_3} \right| \left| \frac{\mathbf{t}_b - \mathbf{v}_3}{\mathbf{t}_b - \mathbf{g}_b} \right|}{\left| \frac{\mathbf{c} - \mathbf{g}_b}{\mathbf{c} - \mathbf{v}_3} \right| \left| \frac{\mathbf{t}_b - \mathbf{v}_3}{\mathbf{t}_b - \mathbf{g}_b} \right|} - h_{\text{desk}} \quad (42.18)$$

This last equation shows how the height of the bottle is estimated by propagating information from the ground via its supporting object, the desk. Learning based approaches that estimated 3D from single images will have to perform such propagation implicitly. The result that we get is $h_{\text{bottle}} \simeq 27.6$ cm which is close to the real height of 25.5 cm.

This procedure highlights the importance of the correct parsing of the **supported-by hierarchy** between objects in the scene.

A **scene graph** is a representation of a scene using a graph where the nodes correspond to objects, and the edges encode the relationship between them. One important relationship is **supported-by**.

42.5 3D Metrology from a Single View

In the previous section we showed how to use a reference object to measure other objects. Let's now discuss a more general framework to locate 3D points [85].

The office picture from [figure 42.17](#) shows the projection of the 3D coordinates frame into the image plane. The axes directions are aligned with the three dominant orthogonal orientations present in the scene and are aligned with the image vanishing points. We will show how can we use the world-coordinates in order to extract 3D object locations from a single image.

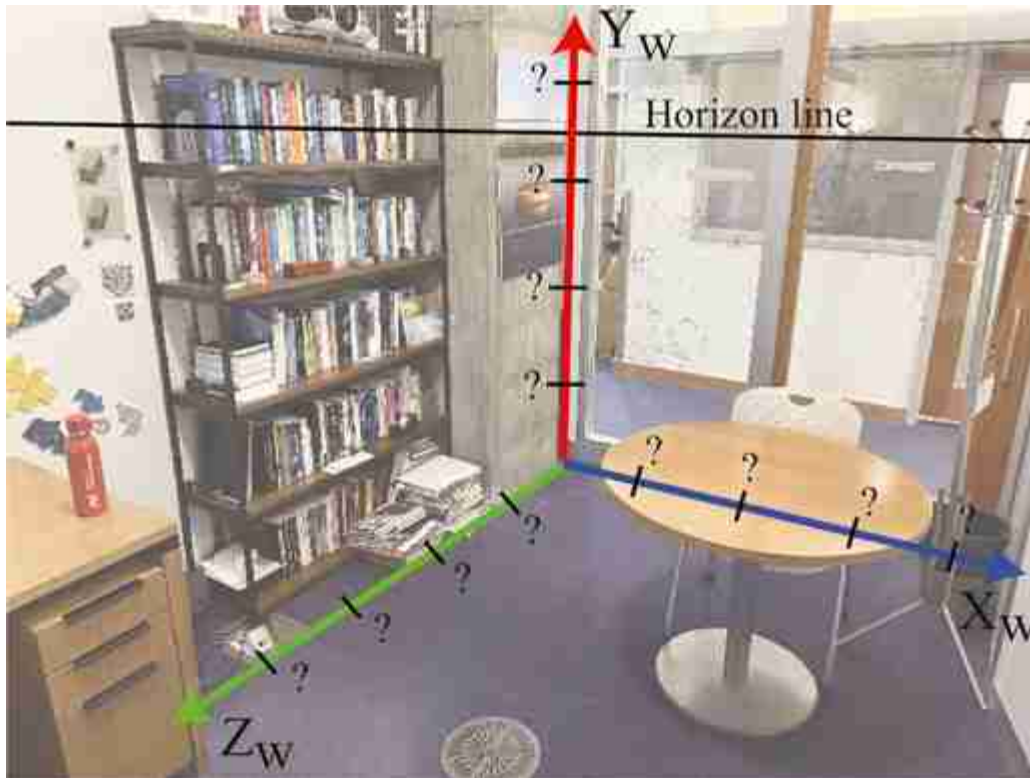


Figure 42.17: Can we find out where to put the ticks in the world-coordinate axes as seen by the camera so that they correspond to calibrated measurements? Simply placing them at evenly spaced intervals along the three axes in the image won't be correct due to perspective projection.

In this section we will describe first how to calibrate the projected world-coordinate axes in the image plane and then we will show how to transport points into the 3D world axes in order to measure object sizes and points locations.

42.5.1 Calibration of the Projected World Axis

Where should we position the tick marks that correspond to 1, 2, 3, ... meters from the origin on each axis in [figure 42.17](#)? Simply placing them at evenly spaced intervals along the axis in the image won't be correct. Due to geometric distortion introduced by perspective projection, the tick marks are not evenly spaced within the image plane. This distortion affects each axis in a distinct manner.

Let's start assuming we know the location of some arbitrary 3D distance to the origin on each axis that we will denote by α_X , α_Y and α_Z . The question is, where do we place the ticks for $k\alpha_X$, $k\alpha_Y$ and $k\alpha_Z$ for all k ? We will use

the cross-ratio to calibrate the projection of the world coordinate system in the image plane. For doing this we will need the location of the horizon line of a reference plane (i.e. the ground plane), the position of the orthogonal vanishing point and the real measure of one object in the scene.

As shown in [figure 42.18\(a\)](#) we can use the cross-ratio to estimate the location of the points $Y = k\alpha_X$ where α_X is an arbitrary constant as shown in [figure 42.18\(b\)](#), and the corresponding image location r , for $k = 1$, is chosen arbitrarily as shown in [figure 42.18\(a\)](#). Due to perspective projection, the image projection of the point $Y = k\alpha_X$ will not be evenly spaced in the image. In this particular example we chose the orientation of the world axis in image coordinates to be aligned with the location of the vanishing points.

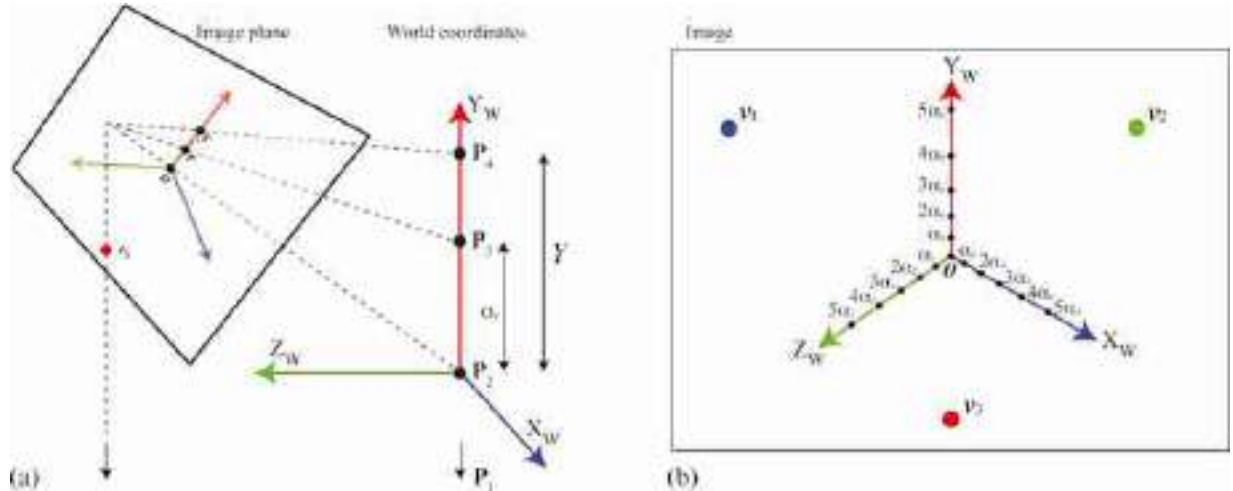


Figure 42.18: Using the cross-ratio invariant to project the world coordinates into the image using only the vanishing points.

In order to calibrate the axis we need to know the dimensions of an object in the scene. We need measurements along the three axis. In this example, we used the bookshelf height (197 cm) and the radius of the table base (50 cm). Using these reference measurements we can solve for the image locations of the ticks $\alpha_X = \alpha_Y = \alpha_Z = 1$ m.

[Figure 42.19](#) shows the final calibrated axes with ticks placed each 50 cm along each of the three axes.

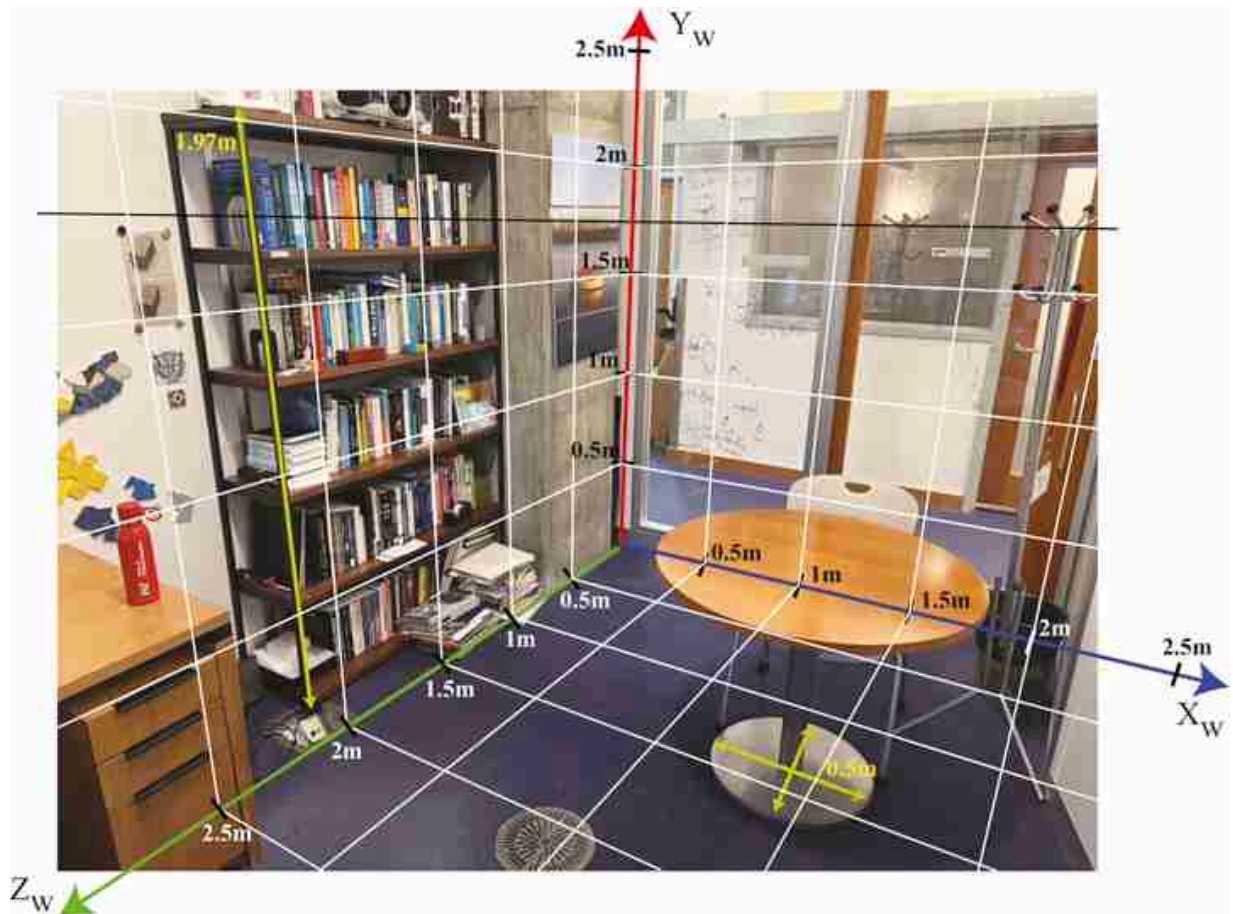


Figure 42.19: Calibrated world axes. Each axis is calibrated using the measurement of a scene element parallel to the world-coordinate axis. In this example, we used the bookshelf height (197 cm) and the radius of the table base (50 cm). From the calibrated axes, you can see that the office is around 250 cm wide.

From this calibrated system we can directly read the height of the camera. The location of the camera coincides with the location of the horizon line because, in this example, the ground plane is horizontal. It seems that the center of the camera is approximately 163 cm above the ground.

There are many ways in which we can arrive to the same result.

42.5.2 Locating a 3D Point

In the absence of any other information, we can not recover the 3D location of a single point P from a single view as illustrated in [figure 42.20\(a\)](#). But real world scenes are composed by a multitude of objects providing context, which allows solving the 3D estimation problem even with just a single

image. In the previous section we showed how to estimate the height of an object, here we show how to estimate the 3D coordinates of a point.

The basic procedure is illustrated in [figure 42.20\(b\)](#). The trick consists in using context (what object is the point part of, where is the horizon line of the scene and where are the vanishing points) to estimate 3D from a single image. It suffices to know that the point $\mathbf{P}$ belongs to an object that is in contact with the ground and the geometry of this object provides the information to localize the projection of the point $\mathbf{P}$ on the ground plane, which we denote with $\mathbf{G}$.

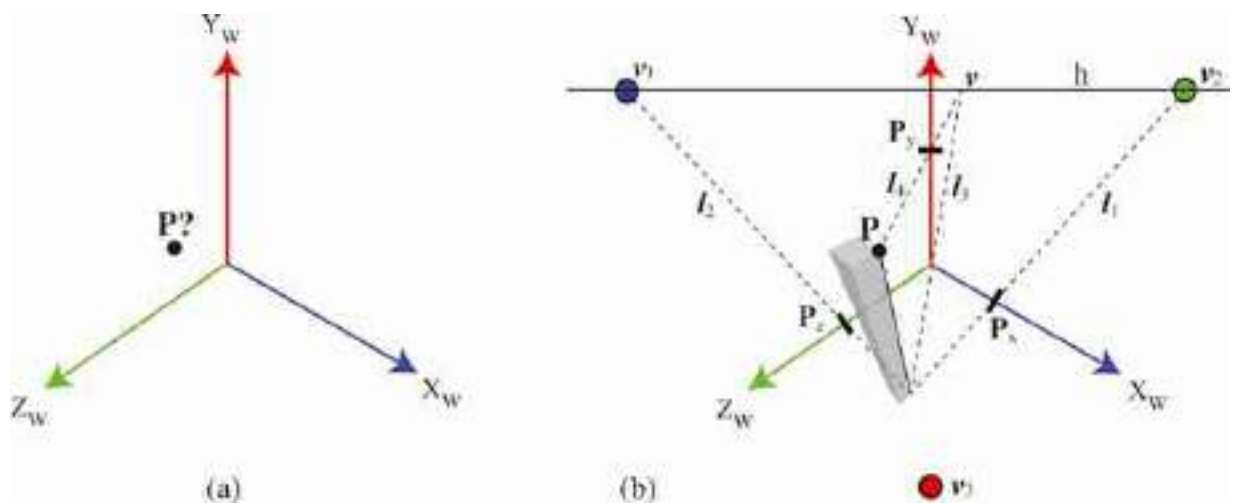


Figure 42.20: (a) In the absence of any other information, we can not recover the 3D location of point $\mathbf{P}$. (b) If we are given a way to project the point $\mathbf{P}$ into the ground, then we can read out the coordinates of the point by projecting into the world axis.

Once we know the image location of the projection of the 3D point $\mathbf{P}$ to the ground, $\mathbf{G}$, we can project this point to the world-coordinate axis. To estimate the Y -coordinate we use a similar method as the one described in the previous section. We first find the line, l_3 , that passes by the point $\mathbf{G}$ and the origin, and then find the intersection with the horizon line, giving the point $\mathbf{v}$. Next, find the line, l_4 , that passes by $\mathbf{v}$ and $\mathbf{P}$. The intersection of l_4 with the Y_w -axis gives the coordinate P_y . The P_x and P_z coordinates can be obtained by connecting $\mathbf{G}$ with the vanishing points $\mathbf{v}_1$ and $\mathbf{v}_2$.

Here we have been using the ground plane as a reference. But the same idea can be applied if we know the projection of the point $\mathbf{P}$ into another

plane with a known horizon line.

In summary, we can recover the location of the 3D point from a single image by propagating information from the calibrated world-coordinate axis using the vanishing points that are aligned with the axis.

42.6 Camera Calibration from Vanishing Points

In the previous chapters, we have seen different methods to calibrate a camera. We saw that we could do it by taking pictures of a calibration pattern, and we also showed that we can extract both the intrinsics and extrinsics from a set of 3D points and their image correspondences. Here we will see a different method that uses only the vanishing points to extract the camera parameters; note that this method will only work under some conditions.

The first step consists in finding the vanishing points in the image. Let's use the office picture from [figure 42.10](#). In this particular scene there are many parallel lines in 3D, and they also happen to be parallel to the three directions we used to define the world coordinates system. Therefore, the three vanishing points are also aligned with the world coordinate axis.

Let's assume that we have three orthogonal vanishing directions. That is the vectors $\mathbf{D}_1$, $\mathbf{D}_2$, and $\mathbf{D}_3$ are orthogonal in 3D. From equation (42.6), the vanishing points measured on the image are the result of projecting $\mathbf{D}_1$, $\mathbf{D}_2$, and $\mathbf{D}_3$ into the image using the unknown camera projection matrix. We will use the vanishing points to derive a set of constraints on the projection matrix. To do this, we start by inverting equation (42.6):

$$\mathbf{D}_i = \mathbf{R}^{-1} \mathbf{K}^{-1} \mathbf{v}_i \quad (42.19)$$

Because the vectors $\mathbf{D}_i$ are orthogonal, we have that $\mathbf{D}_i^T \mathbf{D}_j = 0$ for $i \neq j$. Therefore, we can write:

$$\mathbf{D}_i^T \mathbf{D}_j = \mathbf{v}_i^T \mathbf{K}^{-T} \mathbf{R}^{-T} \mathbf{R}^{-1} \mathbf{K}^{-1} \mathbf{v}_j \quad (42.20)$$

As $\mathbf{R}$ is an orthonormal matrix, we have that $\mathbf{R}^{-T} \mathbf{R}^{-1} = \mathbf{I}$, resulting in a relationship that only depends on the intrinsic camera parameters $\mathbf{K}$:

$$\mathbf{D}_i^T \mathbf{D}_j = \mathbf{v}_i^T \mathbf{K}^{-T} \mathbf{K}^{-1} \mathbf{v}_j = 0 \quad (42.21)$$

It is useful to define the following matrix:

$$\mathbf{W} = \mathbf{K}^{-T} \mathbf{K}^{-1} \quad (42.22)$$

This is also known as the **conic matrix** and it has lots of different properties [187].

Its inverse has a simple structure:

$$\mathbf{K}^{-1} = \begin{bmatrix} \frac{1}{a} & 0 & -\frac{c_x}{a} \\ 0 & \frac{1}{a} & -\frac{c_y}{a} \\ 0 & 0 & 1 \end{bmatrix}$$

[-1.5in]

If we assume that the skew parameter of the intrinsic camera matrix is zero, then the matrix $\mathbf{W}$ has a very particular structure and it has only four different values:

$$\mathbf{W} = \begin{bmatrix} a & 0 & b \\ 0 & a & c \\ b & c & d \end{bmatrix} \quad (42.23)$$

Using equation (42.21), we can derive a linear constraint on the values a, b, c, d for each pair of vanishing points. Defining the vector $\mathbf{w} = [a \ b \ c \ d]^T$, we can rewrite equation (42.21) into a system of linear equations that has the form:

$$\mathbf{A}\mathbf{w} = 0 \quad (42.24)$$

The linear equation can be solved with the singular value decomposition (SVD) and the result is the eigenvector with the smallest eigenvalue.

Once we have $\mathbf{W}$, we can find $\mathbf{K}$ by using the Cholesky factorization, which decomposes a matrix as the product of an upper triangular matrix and its transpose. The last step is to normalize the result by normalizing all the matrix values so that the value on the bottom-right side is 1. In our running example of the office picture, we get the following solution for $\mathbf{K}$:

$$\mathbf{K} = \begin{bmatrix} 3,054.6 & 0 & 1,999.4 \\ 0 & 3,054.6 & 1,528.3 \\ 0 & 0 & 1 \end{bmatrix} \quad (42.25)$$

The Cholesky factorization of an Hermitian positive-definite matrix, $\mathbf{A}$, is $\mathbf{A} = \mathbf{B}\mathbf{B}^T$, where $\mathbf{B}$ is a lower triangular matrix with positive diagonal values. The factorization is unique.

We can now compare this result with the one we obtained when we (1) used the physical camera parameters (section 39.3.3); (2) used a calibration pattern (section 39.3.3); and (3) used a collection of 3D points and the

corresponding image locations (section 39.7.5). We can see that in all cases we get similar results, although, not identical.

42.7 Concluding Remarks

In this chapter we have shown how to use geometric image features (vanishing points, horizon line) and projective invariants (such as the cross-ratio) to estimate 3D scene properties from a single image. However, most of the steps required some manual intervention. All of those steps can be performed in a fully automatic way by training detectors to localize the full extent of an object as we will discuss in chapter 50 and detecting points of contact between objects.

One of the key insights from this chapter is that a single images contain a lot of information that, in most cases, is enough to estimate metric 3D information from a single 2D image. In the next chapter we will discuss learning-based methods for 3D estimation from images.

43 Learning to Estimate Depth from a Single Image

43.1 Introduction

In the previous chapters we studied the image formation process and how to estimate the three-dimensional (3D) scene structure from images. We used the mathematical formulation of perspective projection, together with some hypothesis, in order to recover the missing third dimension: we could use multiple images or a single image using the rules of single view metrology.

There are many other cues that we have ignored that also inform us about the 3D structure of the scene. Instead of building precise models of how to integrate them, here we will describe a learning-based approach that bypasses modeling and instead learns from data how to recover the 3D scene structure from a single image. We will formulate depth estimation as a regression problem.

The trick consists in how to get the training data. We will discuss supervised and unsupervised methods. But before that, let's revise what cues are present in single images that reveal the 3D scene structure.

43.2 Monocular Depth Cues

Before delving into the subject of this chapter, let's quickly review some of the image features that are used by the human visual system to interpret the 3D structure of a picture.

For a greater understanding of depth cues, take an art class.

Shading refers to the changes in image intensity produced by changes in surface orientation. The amount of light reflected in the direction of the viewer by a surface illuminated by a directional light source is a function of the surface albedo, the light intensity, and the relative angle between the light direction and the surface normal orientation, as we discussed in

section 5.2. **Shape from shading** [[38](#), [142](#), [217](#), [382](#)] studies how to recover shape by inverting the rendering equations.

Another source of 3D information are **texture gradients**. As far things appear smaller than closer things, by measuring the relative size between textural elements that appear in different image regions we can get a coarse estimate of the 3D scene structure (see [figure 43.1\[a\]](#)). Many methods using texture gradient cues have been proposed over time [[506](#)].

Another source of information about the 3D scene structure are shadows and interreflections. We saw in chapter 3, figure 3.11 an example of how interreflections can inform us about the location of a ball. The use of shadows and interreflections by the human visual system has been the focus of lots of studies [[266](#), [314](#)]. There are also computational approaches that use shadows and interreflections to recover 3D [[350](#), [251](#)].

When looking at large landscapes, like a mountain range, mountains that are far away have lower contrast due to dispersion by the atmosphere (see [figure 43.1\[b\]](#)). This effect gets accentuated when there is fog. Therefore, contrast can be used as a cue to infer relative distances between objects. This depth cue is called **aerial perspective** or atmospheric perspective. One example using haze to infer depth is in [[195](#)].

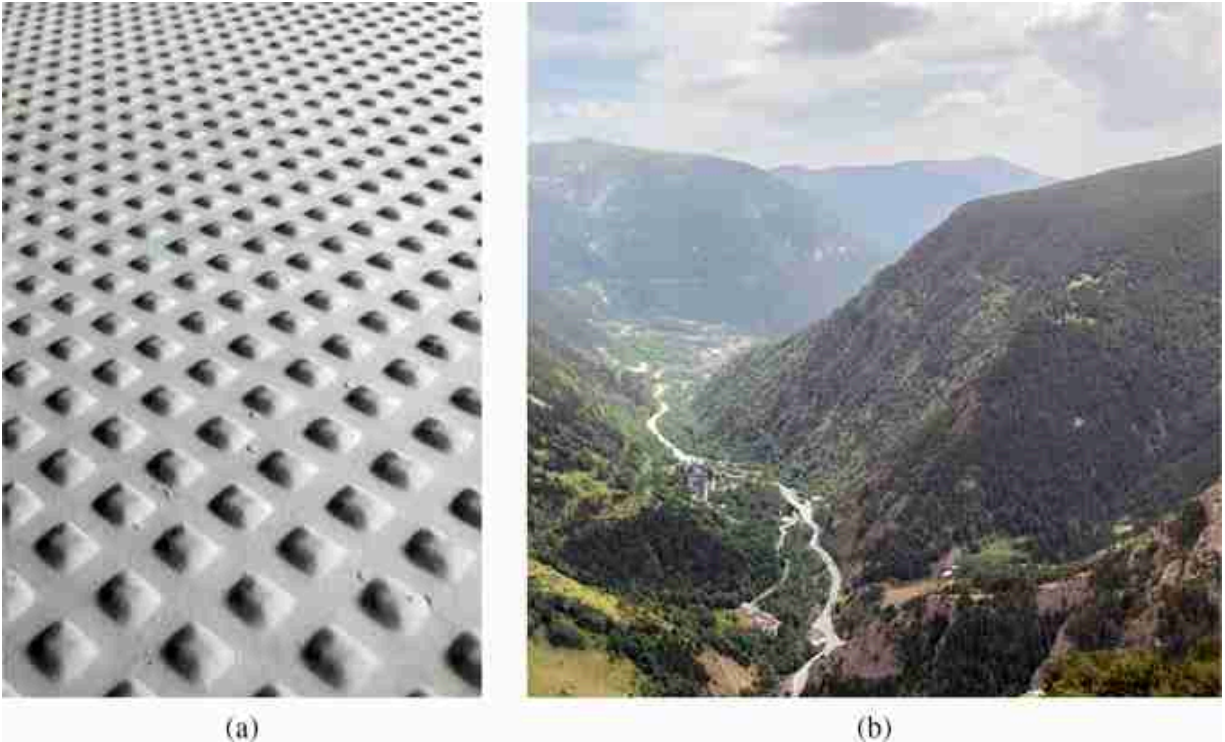


Figure 43.1: (a) This image contains multiple 3D cues: shading effects (bump shapes), texture gradients (relative size between bumps), and linear perspective (due to the regular arrangement of the bumps). (b) Aerial perspective. Objects far in the background have lower contrast than objects closer to the camera.

Depth can also be estimated by detecting **familiar objects** with known sizes. One example is people detection. Objects with known sizes can be used to estimate absolute depth while many of the cues that we discussed before can only give relative depth measurements. One example of using object sizes to reason about 3D geometry is [213].

And finally, **linear perspective** provides strong cues for 3D reconstruction as we already studied in chapter 42.

43.3 3D Representations

When thinking about learning based approaches, it is always good to question the representation for the input and the output. Choosing the right representation can dramatically impact the performances of the final system (even when using the same training set and the same architecture). 3D information can be represented in many different ways (i.e., disparity maps,

depth maps, planes, 3D point clouds, voxels, implicit functions, surface normals, meshes) and there are methods for each one of those representations.

In the rest of this chapter we will use depth maps (or disparity maps which is related to the inverse of the depth as we saw in chapter 40).

Most depth sensors capture aligned RGB images, $\ell[n, m]$, and dense depth maps, $z[n, m]$, as shown in [figure 43.2](#). Given a dense depth map, we can translate the depth into a 3D point cloud if we know the intrinsic camera parameters, $\mathbf{K}$, of the sensor.

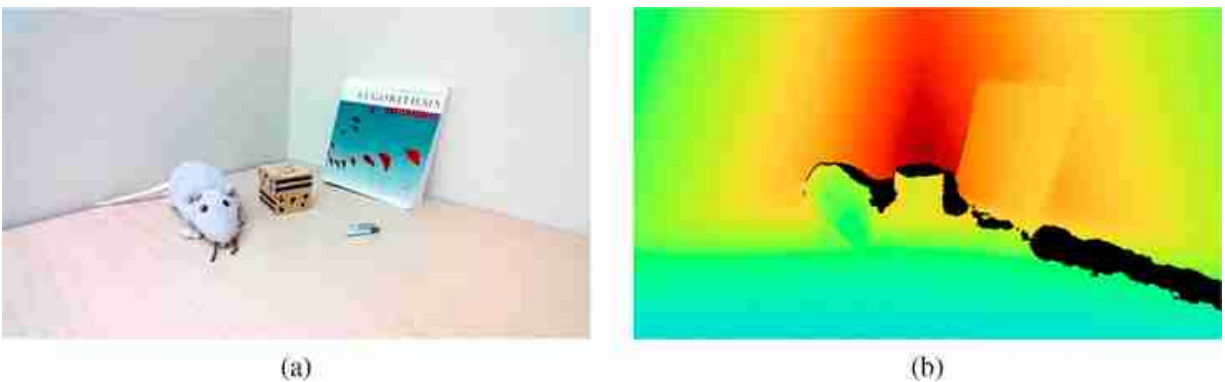
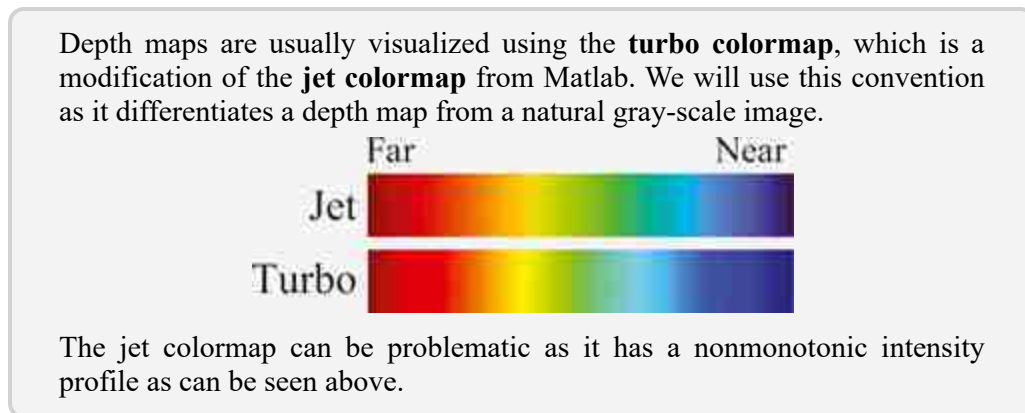


Figure 43.2: Image and depth captured by an Azure Kinect sensor. (a) RGB image; and (b) corresponding dense depth map. Both images have the same size of 1280×720 pixels.



[Figure 43.3](#) illustrates how the depth map translates into a point cloud where each pixel is mapped to a 3D point.

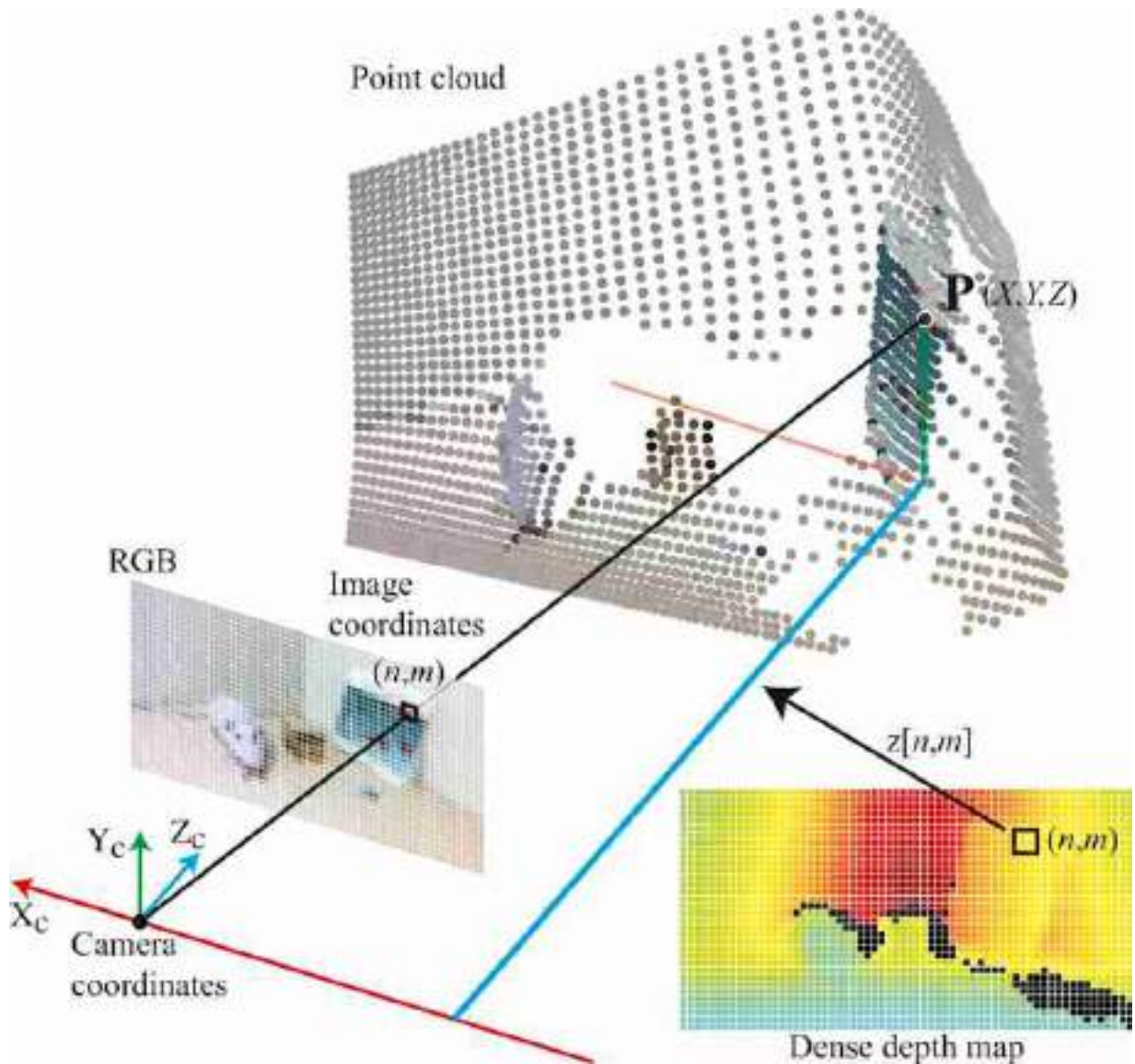


Figure 43.3: Translating the depth map into a point cloud. This requires a calibrated camera (i.e., knowing the intrinsic camera parameters). The sketch shows how one pixel (n, m) is mapped into a 3D point, P , by using the depth, $z[n, m]$, at that location. Colored lines are parallel to the respective axis direction in the camera coordinates system.

Using homogeneous coordinates, the equation translating a depth map into a 3D point cloud is (for each pixel):

$$P = z(p)K^{-1}p \quad (43.1)$$

where p are the pixel coordinates of a point, $z(p)$ is the depth measured at that location, and P are the final 3D point coordinates. In heterogeneous

coordinates, and making the pixel locations explicit, equation (43.1) can be rewritten as:

$$\begin{aligned} X[n, m] &= (n - c_x) \frac{z[n, m]}{a} \\ Y[n, m] &= (m - c_y) \frac{z[n, m]}{a} \\ Z[n, m] &= z[n, m] \end{aligned} \tag{43.2}$$

where a , c_x and c_y are the intrinsic camera parameters (in pixel units). This writing shows that the 3D point coordinates are three images $X[n, m]$, $Y[n, m]$, and $Z[n, m]$ with the same size as the input image.

In a point cloud, there is no notion of grouping of 3D points. 3D points are isolated and are not connected to each other. Note that translating a depth map (or a point cloud) into a mesh requires solving the perceptual organization problem first. A mesh connects points with triangles, but only makes sense to connect points that belong to the same surface or that are in contact.

43.4 Supervised Methods for Depth from a Single Image

Previous computer vision approaches for 3D reconstruction from single images focused on developing the formulation to extract depth using one or few of the cues we discussed in the previous section, and can only be applied to a restricted set of images. Learning-based methods instead have the potential to learn to extract many cues from images, decide which cues are more relevant in each case, and combine them to provide depth estimates. There were a number of early models that applied learning for recovering 3D scene structure [424, 212] but in this chapter we will focus on methods using deep learning although the approaches described here are independent of the particular learning architecture used.

In the supervised setting we will assume that we have a training set that contains N images, $\ell^{(i)}[n, m]$, and their associated depth maps, $z^{(i)}[n, m]$. In this formulation, depth estimation becomes a pixelwise regression problem. Usually $\mathbf{z}^{(i)}$ and $\boldsymbol{\ell}^{(i)}$ are images of the same size. Our goal is to estimate depth at each pixel, $\hat{z}^{(i)}[n, m]$, from a single image using a learnable function, h_θ :

$$\hat{\mathbf{z}}^{(i)} = h_{\theta}(\ell^{(i)}) \quad (43.3)$$

where the parameters θ are learned by minimizing the loss over a training set:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(\hat{\mathbf{z}}^{(i)}, \mathbf{z}^{(i)}) \quad (43.4)$$

43.4.1 Collecting Training Data

Coming up with efficient and scalable data collection strategies is crucial in this task. Several methods for supervised learning from single images differ mostly on how they collect the data. This simplest data capture method is using a depth sensor such as an RGB-D camera or a light detection and ranging (LiDAR) device.

LiDAR (Light Detection and Ranging) is a device that calculates distances by measuring the time taken for emitted light to return after reflecting off a surface (time of flight). A LiDAR produces a range image by scanning multiple direction. The output is a point cloud.

Two examples of datasets are the NYUv2 dataset [348], captured using a handheld Microsoft Kinect sensor while walking indoors, and the KITTI dataset [157], collected from a moving vehicle with a LiDAR scan.

Other methods consist of using stereo pairs for training [512] or multiview geometry methods that provide more accurate 3D estimates. MegaDepth [296] uses scenes captured with multiple cameras to get good 3D reconstructions using structure from motion techniques. This allows creating a dataset of images and associated depth maps that can be used to train depth from single image methods.

43.4.2 Loss

Once we have a dataset, the next steps consist in using an appropriate loss to optimize the parameters of the depth estimator. One simple loss is the L2 loss between the estimated depth, $\hat{\mathbf{z}}$ and the ground truth depth, $\mathbf{z}$.

$$\mathcal{L}(\hat{\mathbf{z}}, \mathbf{z}) = \sum_{n,m} (\hat{z}[n, m] - z[n, m])^2 \quad (43.5)$$

This loss assumes that it is possible to estimate the real depth of each point, which might not be possible in general due to the scale ambiguity. Often, depth may only be estimated up to a global scaling factor. The **scale-**

invariant loss addresses this by eliminating the global scale factor from the calculation:

$$\mathcal{L}(\hat{z}, z) = \sum_{n,m} (\hat{z}[n, m] - z[n, m] - \alpha)^2 \quad (43.6)$$

where $\alpha = \frac{1}{NM} \sum_{n,m} \hat{z}[n, m] - z[n, m]$. This loss compares the estimated and ground truth depth at each pixel independently. In order to get a more globally consistent reconstruction one can also introduce in the loss the depth derivatives, although ground truth derivatives might be noisy depending on how the depth has been collected. Optimizing depth reconstruction might over emphasize the errors for points that large depth values. One way to mitigate this is to optimize the reconstruction of the log of the depth, $\log(z[n, m])$, or the disparity, which corresponds to the inverse of the depth, $1/z[n, m]$. A more complete set of losses are reviewed in [400, 401].

43.4.3 Architecture

The function, h_θ , is usually implemented with a neural network as shown in [figure 43.4](#). The architecture illustrated in the figure is a encoder-decoder with skip connections. The input is an red-green-blue (RGB) image and the output is a depth map. The architecture shown is very generic and there is nothing specific to the fact that it will be trained to uniquely estimate depth, but one could consider adding other constraints such as a preference for planar shapes, or introducing other structural priors. However, most current approaches will use a fairly generic architecture and let the data and the loss decide what the best function is.

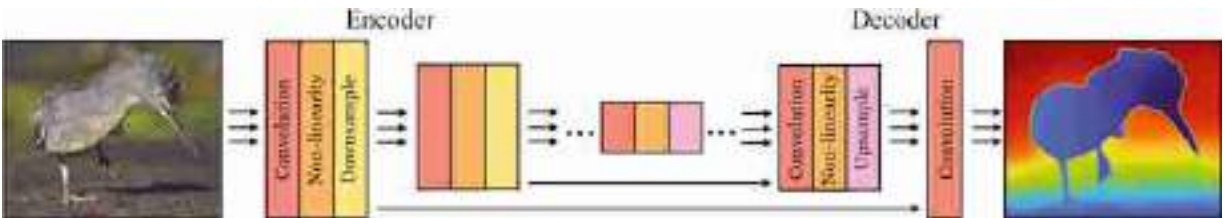


Figure 43.4: Typical architecture for depth estimation from a single image [512].

Existing methods rely on training on massive amounts of data to achieve good generalization across a wide set of images. [Figure 43.5](#) shows the

result of applying the method from [401] on the office scene from figure 42.1 in the previous chapter.



Figure 43.5: (a) Office scene. (b) Estimated Z component by the approach from [400, 401].

Just looking at the depth map from [figure 43.5\(b\)](#) might seem like the estimates are good. But it is hard to tell how good the estimation is with this visualization. The best way of appreciating the quality of a 3D reconstruction is by translating the depth map into a point cloud and then reconstructing the scene under different view points. [Figure 43.6](#) shows two different view points for the office scene from [figure 43.5](#). Points near regions of depth discontinuities are removed to improve the clarity of the visualization. Points near depth discontinuities are often wrong and have depth values that are the average of the values on each side of the discontinuity.

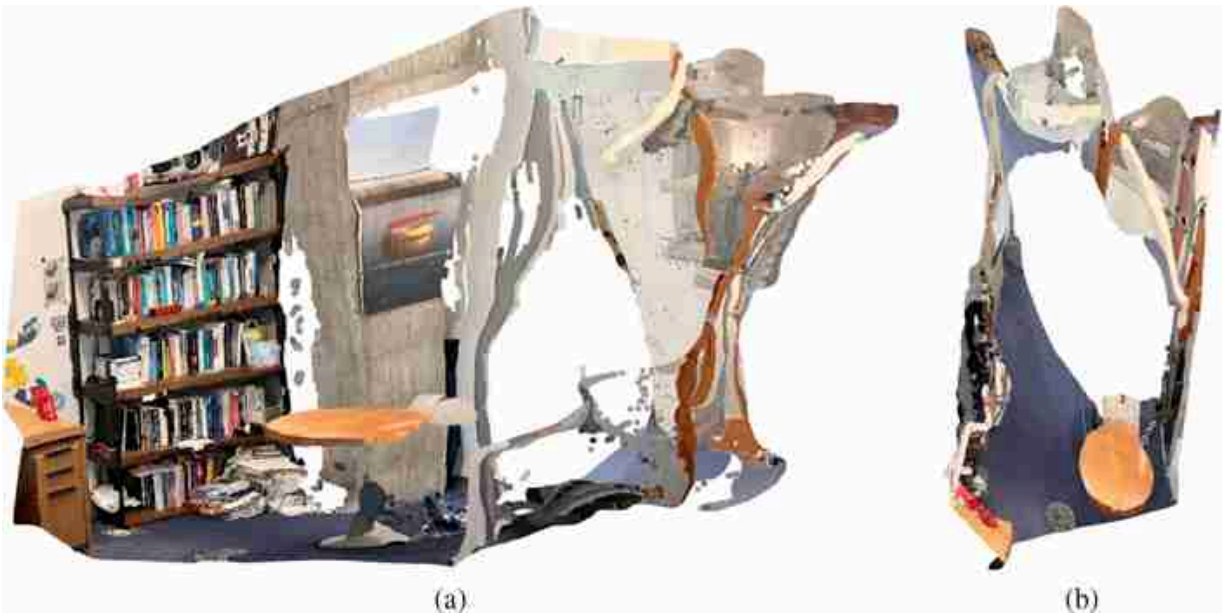


Figure 43.6: Reconstructed point cloud of the office scene with two different view points. Results are good, but not great.

The point cloud has an arbitrary scaling, so we cannot directly use it to make 3D measurements. But if we are given one real distance we can calibrate the 3D reconstruction.

What cues are being used by a learning-based model to infer depth from a single image? What has the network learned? We discussed in chapter 42 the importance of the ground plane and the support-by relationships to make 3D measurements. Is there a representation inside the network of the ground plane? Are surfaces and object represented? If you train a system, how would you try to interpret the learned representation?

43.5 Unsupervised Methods for Depth from a Single Image

Although supervised techniques can be very effective, unsupervised techniques are more likely to eventually reach higher accuracy as they can learn from much larger amounts of data and improve over time. But unsupervised techniques pose a grander intellectual challenge: How can a system learn to perceive depth from a camera without supervision? We will assume that the camera is calibrated (i.e., the intrinsic camera parameters are known).

There are a number of approaches one could imagine. For instance, we could use some of the single view metrology techniques introduced in the previous chapter in order to generate supervisory data. A different, and likely simpler, approach would be to use motion and temporal consistency as a supervisory signal [529]. Let's explore how this method works.

As a camera moves in the world, the pixel values between two images captured at two different time instants are related by the 3D scene structure and the relative camera motion between the two frames. Therefore, we can formulate the problem as an image prediction problem. Let's say we have a function f_1 that can estimate depth from a single image, and a function f_2 that takes as input two frames, ℓ_1 and ℓ_2 , and estimates the relative camera positions (the rotation and translation matrices, $\mathbf{R}$ and $\mathbf{T}$) between the two frames, as shown in [figure 43.7](#):

$$\hat{z} = f_1(\ell) \quad (43.7)$$

$$[\hat{\mathbf{R}}, \hat{\mathbf{T}}] = f_2(\ell_1, \ell_2) \quad (43.8)$$

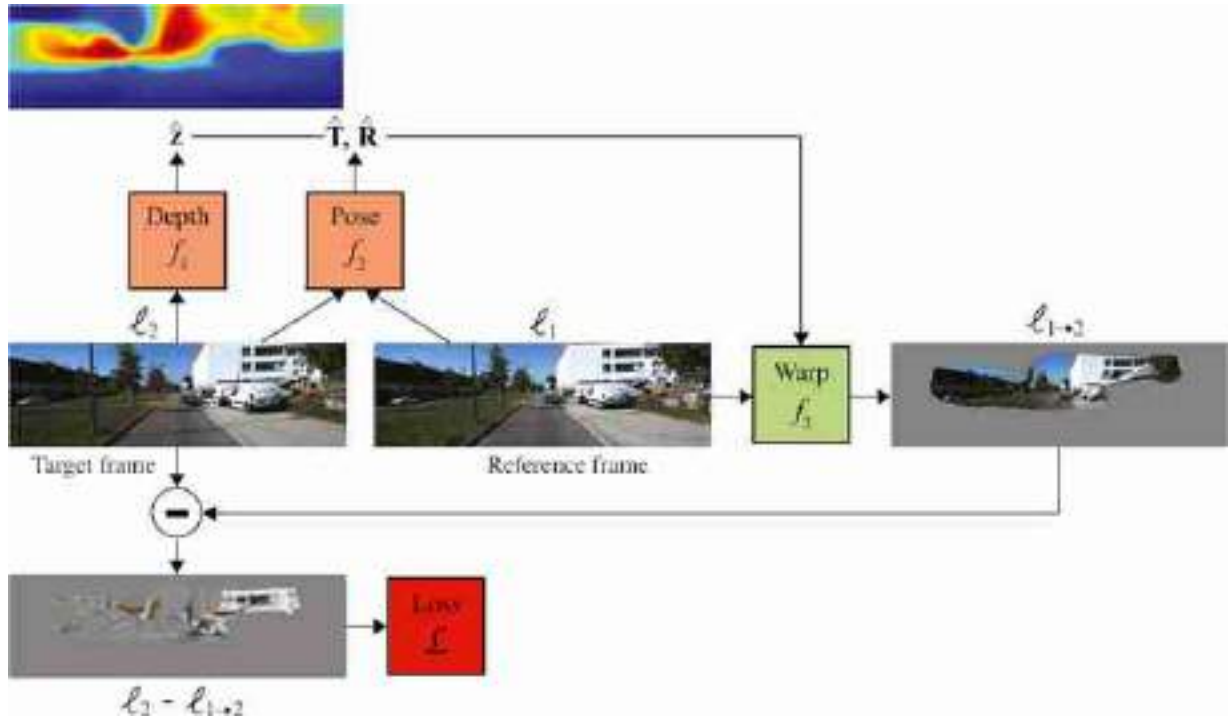


Figure 43.7: System for unsupervised learning of camera motion and depth [529]. The goal is to learn the parameters of the functions f_1 and f_2 by minimizing the reconstruction error. The function f_3 is deterministic.

If the estimations of the depth and camera motion were correct, then we could warp the pixels in ℓ_1 into the pixels of ℓ_2 as depth and camera motion is all with need to precisely predict how the corresponding 3D points will move between the two frames. Warping is a deterministic function (f_3 in [figure 43.7](#)) that takes as input a reference frame, the depth map, and the relative camera motion matrices and outputs an image:

$$\ell_{1 \rightarrow 2} = f_3(\ell_1, z, R, T) \quad (43.9)$$

The function f_3 is built with the tools we have studied in the previous chapters. In section 38.5 we described the procedure for warping an image using backward warping. Here we will use the same procedure.

The goal is to warp a **reference frame** into a **target frame**. We want to look over all pixel locations, $\mathbf{x}$, over the target frame ℓ_2 , and compute the corresponding location on the reference frame ℓ_1 . For each pixel on the target frame, we will follow the next steps (see [figure 43.8](#) for the geometric interpretation of all the steps):

- Express the image coordinates of the pixel in the target frame in homogeneous coordinates, $\mathbf{p}$.
- Given the depth at that location, recover the 3D world coordinates of the target point, $\mathbf{P}$. To do this we will use the intrinsic camera matrix, assumed to be known, and the depth at that point: $\mathbf{P} = z(\mathbf{p})\mathbf{K}^{-1}\mathbf{p}$. As we do not have access to ground truth depth, we will use the function f_1 to estimate depth: $\hat{z} = f_1(\ell_2)$. The coordinates of the point $\mathbf{P}$ are expressed with respect to the camera coordinates of ℓ_2 .
- We now use the camera rotation and translation matrices to change the camera coordinates to the coordinates defined by the reference frame, ℓ_1 . This is done as $\mathbf{P}' = \mathbf{M}\mathbf{P}$ where the matrix $\mathbf{M}$ contains the translation and rotation matrices.
- Finally, we project the point $\mathbf{P}'$ into the image coordinates of the reference frame $\mathbf{p}' = \mathbf{K}\mathbf{P}'$.
- The resulting coordinates $\mathbf{p}'$ are translated back into heterogeneous coordinates.

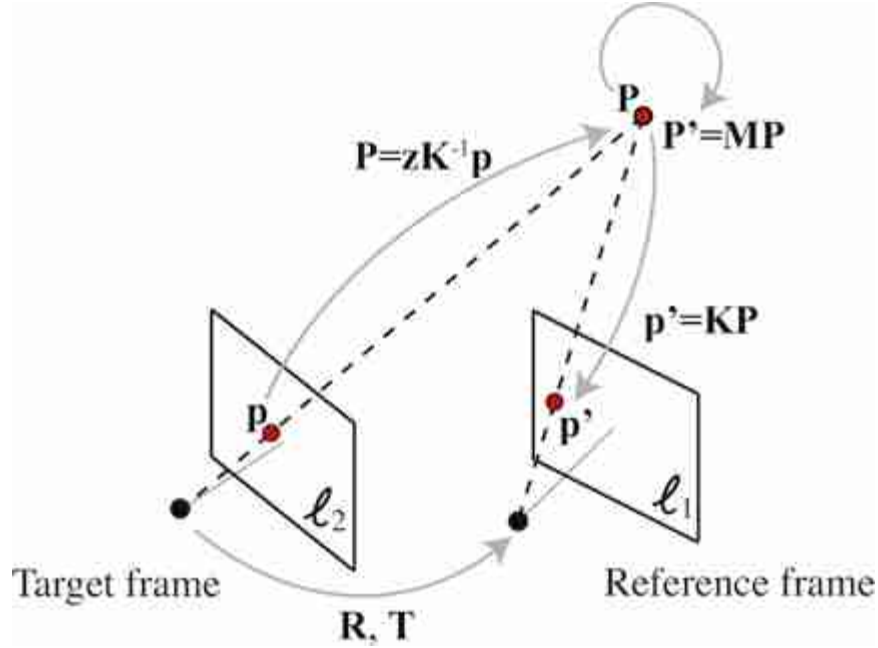


Figure 43.8: Geometry between two images captured by a moving camera. The coordinates of corresponding points between the two frames are constrained by the 3D world coordinates of the point the relative camera motion.

Putting all these steps together, ignoring the transformations between homogeneous and heterogeneous coordinates, the reconstruction of ℓ_2 results in:

$$\ell_{1 \rightarrow 2}(p) = \ell_1 \left(KM_z^{-1}(p) K^{-1} p \right) \quad (43.10)$$

where $\ell_{1 \rightarrow 2}$ is the result of warping image ℓ_1 into ℓ_2 . As the coordinates of the point p' will not be discrete, we use bilinear interpolation to estimate the pixel value.

If the reconstruction is correct, we expect the difference $|\ell_{1 \rightarrow 2}(p) - \ell_2(p)|$ to be very small. There will be some errors due to pixels that are invisible in one frame but not the other, changes in illumination, or due to the presence to moving objects in the scene.

The functions f_1 and f_2 are unknown and we want to learn them in an unsupervised way. If we have very long video sequences captured with a moving camera (assuming all objects are static) we can optimize the parameters of the functions f_1 and f_2 to minimize the reconstruction error.

The functions f_1 and f_2 can be neural networks and their parameters can be obtained by running back-propagation over the loss:

$$\mathcal{L} = \sum_{n,m} v(n, m) (\ell_1[n, m] - \ell_2[n, m]) \quad (43.11)$$

As only the pixels with coordinates $\mathbf{p}'$ inside the image can be reconstructed by warping, we compute the loss only over the valid set of pixels. This is done using the mask $v(n, m)$, which is 1 for valid pixels and 0 for pixels that lie outside the visible part of the reference frame. The mask could also exclude pixels that fall in moving objects as those will have displacements that cannot be predicted by the motion of the camera alone.

Not all camera motions provide enough information to learn about the 3D world structure. As we discussed in chapter 41, if all the videos contain only sequences captured by a rotating camera, the points are related by a homography regardless of the 3D structure. Therefore, to learn depth from single images, we need to have complex camera motions inside environments with rich 3D structure.

To model the motion of objects we need to introduce new concepts. We will do that later when studying motion estimation in chapter 48.

43.6 Concluding Remarks

In this chapter we have reviewed some of the key components of a system trained to estimate depth from single images. The methods are similar to other regression problems and most of the challenge consists of identifying how to collect the data needed for training. Supervised methods are more reliable than unsupervised methods for now. Unsupervised methods rely on learning to estimate depth using the constraints present between camera motion, 3D, and photometric consistency. Unsupervised learning allow a system to learn from a very diverse set of data (as the only thing needed is video captured by a moving camera), and to adapt to new environments where ground truth supervision is not available.

44 Multiview Geometry and Structure from Motion

44.1 Introduction

As humans, we know how it feels to look at the world with one eye. We can easily experience this by closing one eye, or when looking at a picture. The world clearly appears to us as three-dimensional (3D), but there is something that still feels somewhat flat. We also know what it is to look at the world with two eyes. Objects and scenes appear and feel 3D. We sense the volume of the space. In fact, even when we look with both of our eyes at a flat surface we feel it is 3D as we can feel how far the surface is and what is its orientation. But we do not know what it is to look at the world with three or more eyes. We can try to make guesses about what new sensations we might feel, but we are likely to fall short in truly understanding how it feels to see world through many eyes. It's like someone with one eye trying to envision seeing the world with two; we can make guesses, but our comprehension will be limited.

Looking at an object simultaneously with many eyes surrounding the object, we will not only feel its 3D but also its wholeness in a way hard to imagine with only two eyes. For us, only one side of the object is visible at any given time. We can turn around an object to see all of its sides, but that will not compare with the simultaneous perception of all its sides.

Multiview geometry studies the 3D reconstruction of an object or a scene when captured from N cameras (with $N > 2$).

44.2 Structure from Motion

In 1971, Gunnar Johansson created a series of beautiful videos by recording the movements of a person with a set of lights attached to few of the body joints (e.g., elbows, knees, feet, head, and hands as illustrated in [figure 44.1\[b\]](#)).

Gunnar Johansson was a Swedish psychologist working on gestalt laws of motion perception. He introduced the term of **biological motion** to describe the typical motion of biological organisms.

Johansson [238] demonstrated that by watching the motion of only a few of those points, no more than 12, it was possible to clearly recognize the person actions. By showing only moving points, Johansson dissociated the perception of form/shape from the motion pattern. Johansson's videos were very surprising at the time and researchers were stunned by how easy was to recognize many types of actions from just a few moving points. Since then, Johansson's early demonstrations have been used to argue about the importance of reliably tracking a few points over time.

One major advance that allowed writing a mathematical formulation of the problem of structure from motion (SFM) was the **rigidity assumption** introduced by Shimon Ullman in 1979 [482] where he writes: “Any set of elements undergoing a two dimensional transformation which has a unique interpretation as a rigid body moving in space should be interpreted as such a body in motion.” Since then, the 3D reconstruction of rigid objects from camera motion has been a rich area of work. Most of the works start by representing the sequence using as input the temporal trajectories of a sparse set of points (**sparse SFM**). [Figure 44.1\(a\)](#) shows one frame from the nested cylinders sequence. In this sequence, there are a set of points placed on the surface of two nested cylinders. The cylinders are rotating. When viewing the sequence it is easy to see both cylinders, even though when seeing any single frame (as shown in [figure 44.1\[a\]](#)) it is hard to see the cylinders or even to say which points are attached to the same surface.

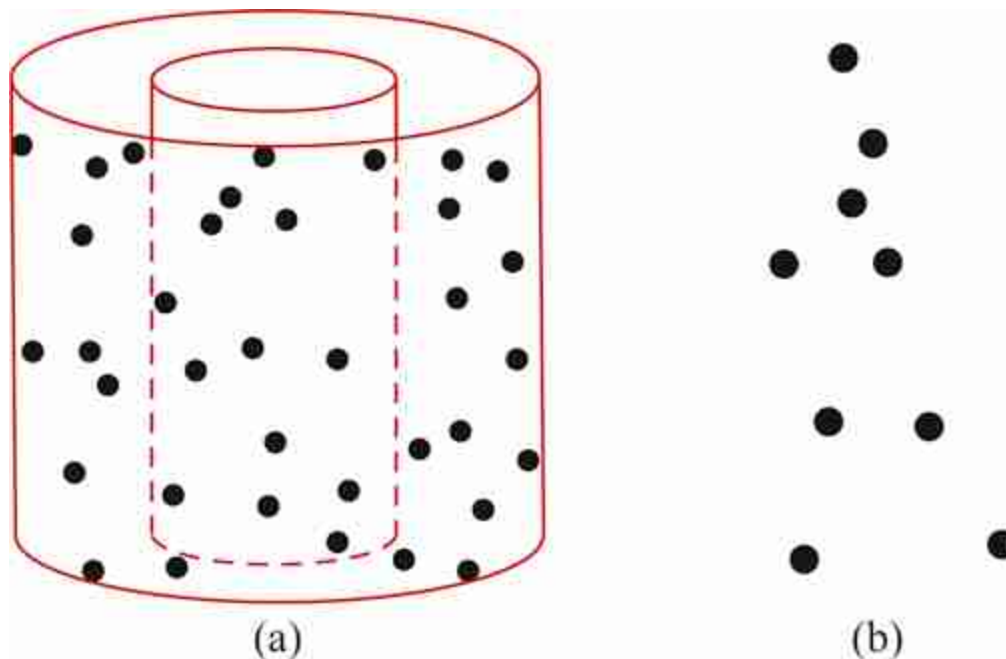


Figure 44.1: Early illustrations on the perceptual power of motion. (a) Nested cylinders by Shimon Ullman, 1979, illustrated how rigid motion elicits a strong 3D interpretation. (b) One frame from the point-light walker by Gunnar Johansson, as an early illustration of biological motion perception from few moving keypoints.

One of the first algorithms for structure from motion for rigid bodies was introduced by Tomasi and Kanade [470]. For simplicity, their approach assumed parallel projection (orthographic camera) instead of perspective projection. The algorithm took as input a video sequence captured by a moving camera looking at a static scene. The video was first processed by extracting keypoints and tracking them over time. The input to the reconstruction algorithm was the keypoint trajectories and the output was the 3D locations of those points with respect to a reference world-coordinate system (usually defined by the first frame in the video). The model introduced by Tomasi and Kanade was later extended by Sturm and Triggs [456] to handle perspective projection.

One of the key problems in SFM is to find correspondences across multiple images. Finding image correspondences is a topic that we have already mentioned in multiple places (e.g., chapter 40 and 41) and that will be mentioned again (chapter 48).

SIFT [309] made image matching more reliable and structure from motion started to work much better under difficult conditions such as where

views are captured by cameras at very different locations, which is a more difficult image matching than when the views are frames from one sequence.

The task of SFM became more general, and instead of focusing on one moving camera, SFM was applied to the problem of reconstructing the 3D scene even from multiple images taken by different cameras and making no assumption about the placement of the cameras. The work by Snavely et al. [447] was one of the first to produce spectacular registrations and camera pose estimates of images taken with many different cameras. They showed how one could reconstruct famous buildings from pictures taken by tourists.

In the last decade, deep learning now provides a more robust set of features for finding image correspondences, which is one of the key elements of the SFM algorithms.

44.3 Sparse SFM

Let's start defining the notation and the formulation of the problem we want to solve; given a set of M images with overlapping content, we want to recover the 3D structure of the scene and the location of the camera for each image. [Figures 44.2 and 44.3](#) show two examples of sets of images. [Figure 44.2](#) shows 12 images of an object taken by a camera moving around the object. [Figure 44.3](#) shows eight images taken inside a plaza. Both sets of images represent very different scenarios that will be reconstructed with the same set of techniques.

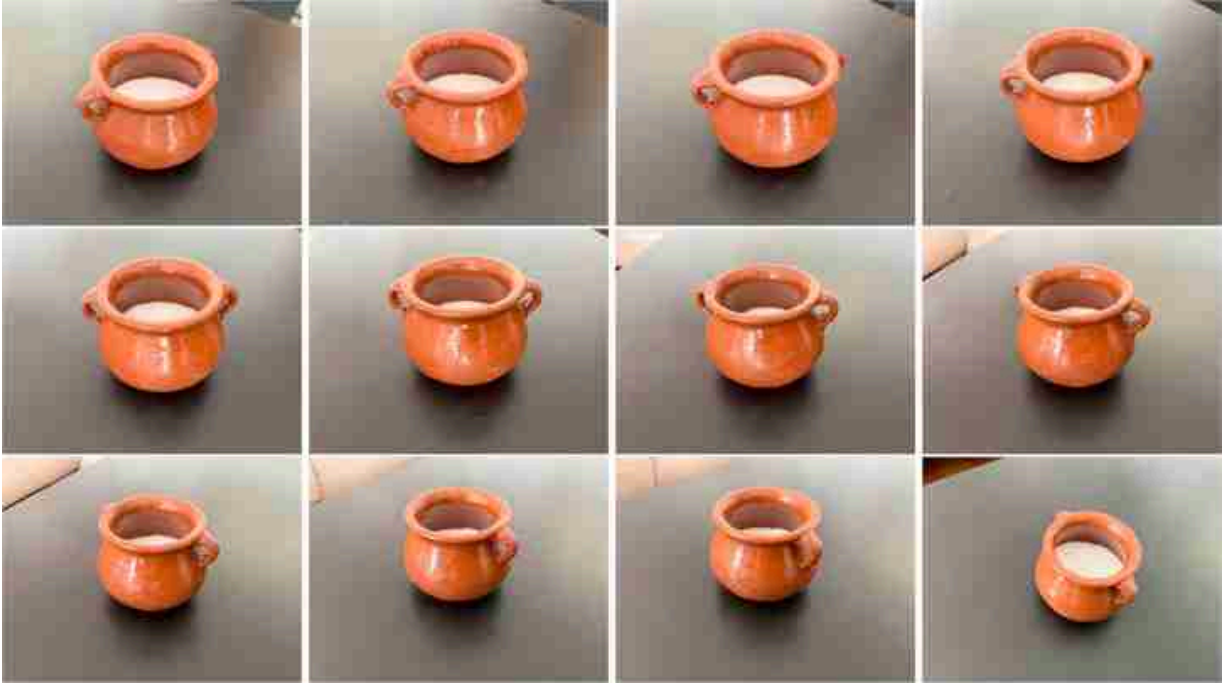


Figure 44.2: Twelve viewpoints of a sugar bowl we had at home. Although it is not a fancy object, it is useful to start with a simple shape to check what happens on every step. The bowl has a rough texture that might also allow for enough keypoints to be present along its surface. The images have $3,024 \times 4,032$ pixels.



Figure 44.3: Eight pictures taken in a plaza in Palma de Mallorca, Spain. The images have $3,024 \times 4,032$ pixels.

44.3.1 Problem Formulation

In the sparse SFM problem, we consider that in the scene there are N distinct points (with N smaller than the number of pixels on each image)

that are matched across M different views (we will consider later that not all points may be visible across all images). We denote the N 3D points as $\mathbf{P}_i$.

Notation reminder: we use capital letters for quantities measured in 3D world coordinates and lowercase for image coordinates.

For each of the M views (it could be different cameras or different pictures taking by a moving camera), we denote the projection of point $\mathbf{P}_i$ as seen by camera j by $\mathbf{p}_i^{(j)}$. Each view has camera parameters $\mathbf{K}^{(j)}$, $\mathbf{R}^{(j)}$, and $\mathbf{T}^{(j)}$. If all the images are captured by the same camera, then the matrices $\mathbf{K}^{(j)}$ will be the same. In general it is not necessary to make that assumption.

The goal of the SFM problem is, given the set of corresponding 2D points $\mathbf{p}_i^{(j)}$, recover 3D points $\mathbf{P}_i$, the intrinsic camera parameters, $\mathbf{K}^{(j)}$, and the extrinsic camera parameters $\mathbf{R}^{(j)}$ and $\mathbf{T}^{(j)}$, for each camera j .

We will next study what is the minimal number of observations that we need to have any hope of being able to solve the problem. Let's assume that we have calibrated cameras, that is, we know the intrinsic camera parameters for each image, $\mathbf{K}^{(j)}$. We will first count the number of unknowns that we have. We have two sets of unknowns: the 3D coordinates of the N points and the extrinsic camera parameters for the M images.

This results in $3N$ unknowns for the N 3D points. For the camera parameters, we assume that the first camera is at the origin. Therefore, we need to figure out the camera matrices for $M - 1$ cameras. The rotations can be specified with three numbers, which results in $3(M - 1)$ unknowns. The translations (up to a scale factor) result in $3(M - 1) - 1$ unknowns. This gives a total of $3N + 6(M - 1) - 1$ unknowns.

If we have the image coordinates of the N points across the M images, we would have $2NM$ observations (in reality we will have less as some points will only be visible in a subset of the images). So, there is a solution when,

$$2NM \geq 3N + 6(M - 1) - 1 \quad (44.1)$$

Note that if $M = 1$, only points captured by one camera are available, then we can not solve the problem. If cameras are uncalibrated, this will add $3M$ additional unknowns (the exact number of additional unknowns will depend on the camera type).

In the previous description, we are making the assumption that every point is visible in every view (so one can run fundamental matrix estimation, triangulation, and other tasks of each of the $M - 1$ cameras with respect to the first view). However, if the visibility criteria isn't met, for each new image we are adding in to SFM, we will need to triangulate it with respect to its closest image – when doing so, we may have a slight numerical error, and this new frame may end up having an ever-so-slightly different scale factor with respect to the reconstruction. This accumulates over time, causing **scale drift**.

44.3.2 Reprojection Error

We already introduced the **reprojection error** in section 39.7.4 when talking about camera calibration. Here the formulation is similar but we optimize over a different set of unknowns. The reprojection error is the sum of euclidean distances (in heterogeneous coordinates) between the observed image points $\mathbf{p}_i^{(j)}$ and the estimated projected points $\hat{\mathbf{p}}_i^{(j)}$

$$\sum_{j=1}^M \sum_{i=1}^N \left\| \mathbf{p}_i^{(j)} - \hat{\mathbf{p}}_i^{(j)} \right\|^2 \quad (44.2)$$

The estimated projected points are obtained using the estimated 3D points locations $\mathbf{P}_i^{(j)}$ and the estimated camera matrices $\mathbf{M}^{(j)}$,

$$\sum_{j=1}^M \sum_{i=1}^N \left\| \mathbf{p}_i^{(j)} - \pi \left(\mathbf{M}^{(j)} \mathbf{P}_i^{(j)} \right) \right\|^2 \quad (44.3)$$

where $\pi()$ is the function that transforms a vector described with homogeneous coordinates into the corresponding vector in heterogeneous ones.

We can make the camera model explicit in equation (44.3) by replacing the camera matrix, $\mathbf{M}^{(j)}$. By using the intrinsic and extrinsic camera matrices we get:

$$\sum_{j=1}^M \sum_{i=1}^N \left\| \mathbf{p}_i^{(j)} - \pi \left(\mathbf{K}^{(j)} \left[\mathbf{R}^{(j)} \mid -\mathbf{R}^{(j)} \mathbf{T}^{(j)} \right] \mathbf{P}_i^{(j)} \right) \right\|^2 \quad (44.4)$$

To have the final error we still need to add one more factor. We add a **visibility** term, $v_i^{(j)}$, to account for 3D points that are only be visible in a subset of the views. We will use the binary variable $v_i^{(j)}$ to indicate if point i

is seeing from camera j . For all the points that are not visible in a view we set $v_i^{(j)} = 0$, that is, their reprojection error is ignored. This can be written as:

$$\sum_{j=1}^M \sum_{i=1}^N v_i^{(j)} \left\| \mathbf{p}_i^{(j)} - \pi \left(\mathbf{K}^{(j)} \left[\mathbf{R}^{(j)} \mid -\mathbf{R}^{(j)} \mathbf{T}^{(j)} \right] \mathbf{P}_i^{(0)} \right) \right\|^2 \quad (44.5)$$

and this gives us the final reprojection error equation. One common simplification is to assume that all the cameras have the same intrinsic parameters, which means that $\mathbf{K}^{(j)} = \mathbf{K}$. This is the case when the views have been captured by a moving camera with a fixed focal length. The optimization of the reprojection error is also called **bundle adjustment**.

The term **bundle adjustment** refers to the process of optimizing the “bundle of rays” connecting the camera centers and the coordinates of the 3D points.

When optimizing equation (44.5) there are a number of ambiguities that cannot be recovered. Those include a global translation and rotation of the world-coordinate system, the global scale, and a projective ambiguity when $\mathbf{K}$ is unknown.

There are two main challenges in SFM. The first challenge is that we need reliable matching points, and the second challenge is optimizing equation (44.5), which is difficult due to local minima and to noisy matches.

44.3.3 Finding Matches across Images

When describing stereo vision in chapter 40 we had to find matching points across two views in order to recover depth, but we did not spend much time on the problem of matching points across images. Matching features across views in a stereo pair is challenging, but it is easy in comparison with the challenge of matching points in a multiview setting. Now we will have to match points across views that might be very different, with little overlap and with dramatic camera motions. Therefore, we will need a more robust way to match points.

44.3.3.1 Interest point detector

The goal of finding image features is to locate image locations that are likely to be stable under different sets of geometric transformations (translation, scaling, rotation, skew, and projective), and illumination changes. These stable regions are called **interest points**.

Initially, **interest points** were called **moving elements**. A moving element was any image point that could be identified and followed over time such as a corner, a line termination point, or a texture element [482].

When talking about stereo matching we described a simple procedure to detect interest points: the Harris corner detector [186]. Corners are examples of image regions that are likely to be distinct from the image surroundings and also be stable under geometric transformations (i.e., corners remain corners under a number of transformations). There are a number of other important detectors of regions of interest, in addition to the Harris corner detector, such as local extrema of the Laplacian pyramid across space and scale [335, 309], Harris-Laplace [334], and maximally stable regions [322]. For an in depth description of interest point detectors and their evaluation we refer the reader to the work of Cordelia Schmid et al. [426] and Mikolajczyk [335].

More recently, the problem of detecting points of interest has been formulated as a learning problem. Given a set of examples of points of interest manually annotated on images, we can train a classifier to automatically label image patches as being centered on a keypoint or not. Some examples of learning-based approaches are Quad-Networks [423], and SuperPoint [101].

[Figure 44.4](#) illustrates a general formulation for detecting keypoints using a classifier. We start by training a classifier using hand-annotated images with keypoints. This can be done by generating simple synthetic images where keypoints are unambiguously defined. The training set can be extended by running an initial keypoint detector on images and then augmenting the images with random geometric transformations. Then we can use the transformed keypoint coordinates together with the transformed images as new ground truth to train a more robust classifier. This is the approach proposed by SuperPoint [101].

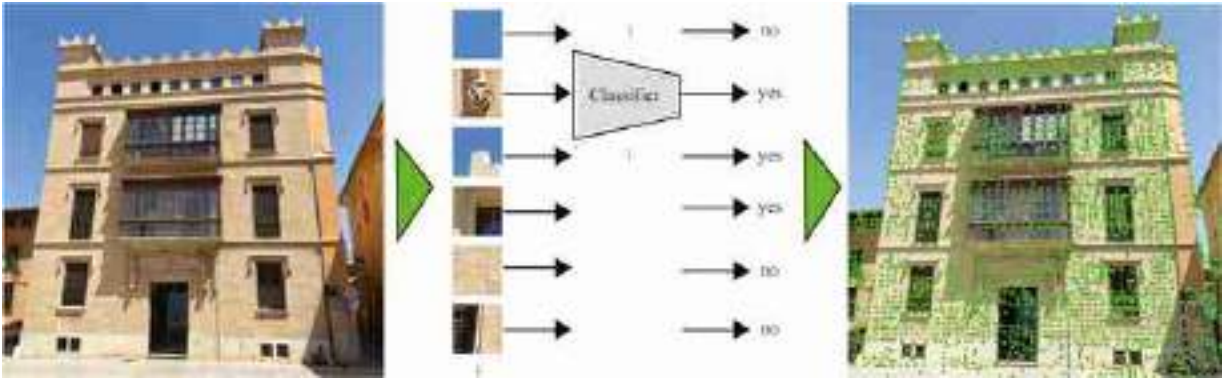


Figure 44.4: Keypoint detector formulated as a patch classification problem. The classifier labels the patch as containing a keypoint in the center or not.



Figure 44.5: All the detected keypoints in three views from the sugar bowl. But the images look rather textureless; what is the keypoint detector finding interesting in these images?

[Figure 44.5](#) shows the resulting detected keypoints by SuperPoint on three viewpoints of the sugar bowl. The images shown in [figure 44.2](#) seem to lack texture. Interestingly, when we zoom into the images (which are very high-resolution) we can see that the surface contains many irregularities that are detected by the keypoints classifier ([figure 44.6\[a\]](#)). If the surface of the bowl was smooth and textureless, the method would fail to detect any interesting points, and the whole pipeline for 3D reconstruction would fail.

The keypoints detected on the plaza scene, shown in [figure 44.6\(b\)](#), correspond to more obvious keypoints, such as window corners, bricks, and other building details, than the ones detected on the sugar bowl.

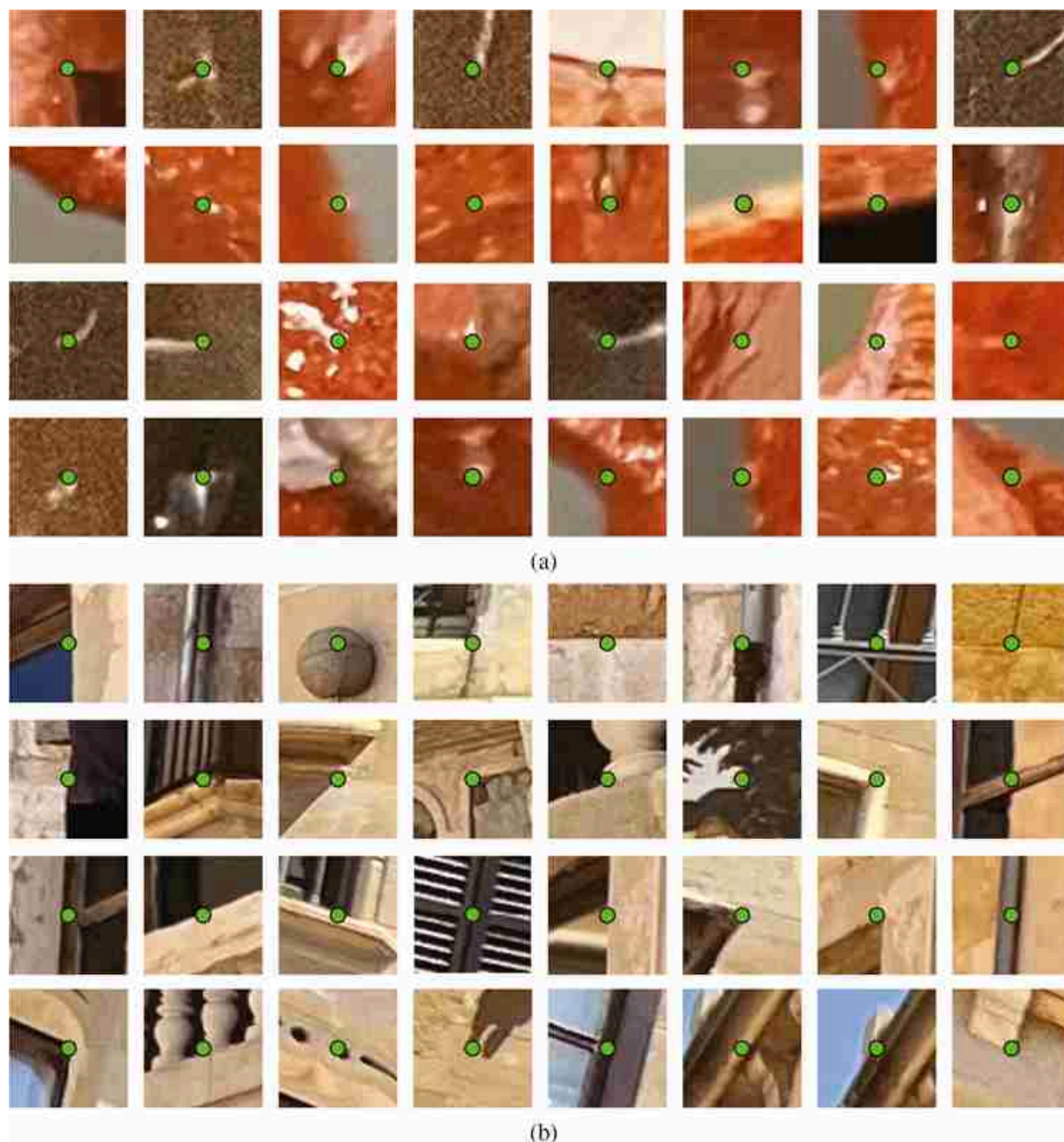


Figure 44.6: Each patch shows a 65×65 image crop centered on a detected keypoint. The keypoint detector finds image regions with strong spatial changes. Those regions are also likely to be found on images of the same object but under different viewpoints. (a) Sugar bowl, [figure 44.2](#). (b) Plaza, [figure 44.3](#).

The advantage of hand-engineered detectors is that they are derived from first principles and do not require any training data. However, learning-

based detectors can leverage unsupervised learning to achieve state of the art performance.

44.3.3.2 Local image descriptors

Once we have trained a keypoint detector that finds image regions that are distinct and likely to be stable under geometric transformations, we run the detector in the set of images we want to match. We need now to extract at each location a descriptor that will allow finding the same keypoint across the different images.

In chapter 40 we described SIFT [309] as a local image descriptor. Other classical descriptors are SURF [41], and ORB [415]. For a review of several hand-engineered descriptors and their evaluation, we refer the reader to the work of Krystian Mikolajczyk and Cordelia Schmid [333]. [Figure 44.7](#) shows three interest points in the sugar bowl and the three associated descriptors. In this example, each descriptor is a vector of length 256.

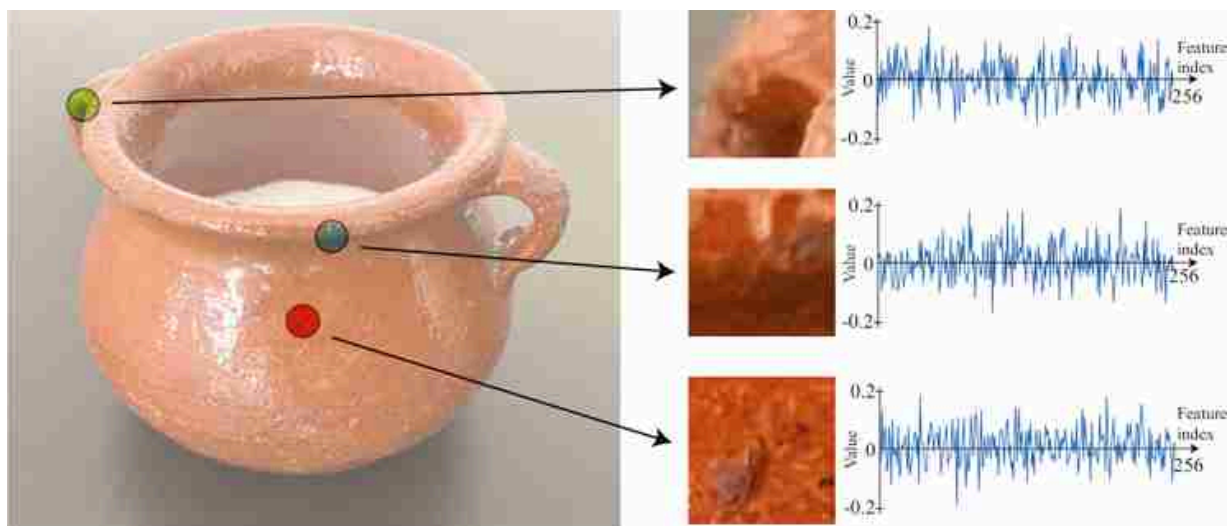


Figure 44.7: Examples of descriptors for three keypoints. For each keypoint, the figure shows the corresponding image patch and the computed descriptor. Each descriptor has a length of 256 values.

As shown in [figure 44.8](#), the keypoint detector and descriptor finds image regions and its matches even under different viewpoints and illumination conditions. The top row of [figure 44.8](#)(a) shows the location of a matching keypoint across six views of the sugar bowl (this keypoint was

not detected in the remaining six images). The bottom row shows crops of size 129×129 pixels around the keypoint. The larger crops allow better getting a sense of the context of each keypoint. [Figure 44.8\(b\)](#) shows the same for the plaza scene. In this case the keypoint detects the same window corner across multiple views.

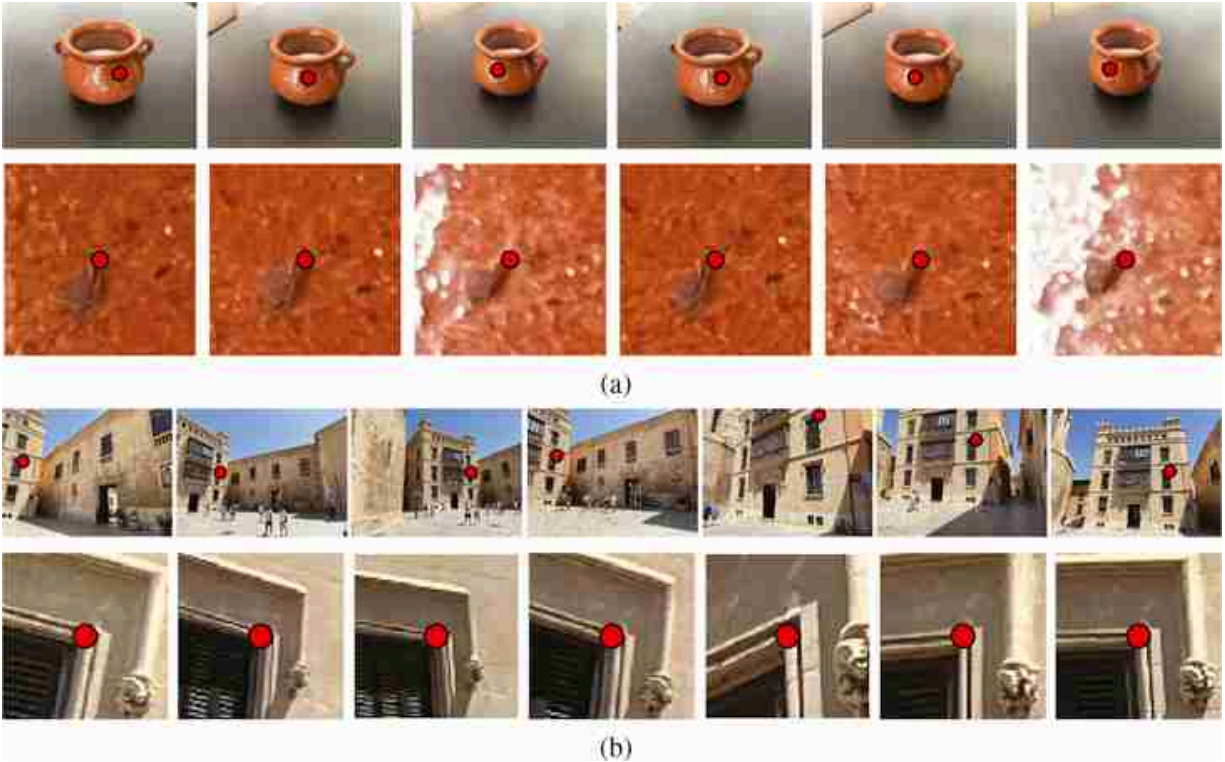


Figure 44.8: Matching keypoints across different viewpoints. For each subfigure, the top row shows the location of a matching keypoint across several views. The bottom row shows crops of size 129×129 pixels around each the keypoint.

We perform brute-force matching, that is, match features from one image with features from every other image because we have a small number of images. For the actual matching shown in this section, we use SuperGlue, another graph network-based matcher.

44.3.4 Optimization

The matching procedure should have given as a set of image points $\mathbf{p}_i^{(j)}$ and visibility indicators $v_i^{(j)}$. Each location will also have a feature, $\mathbf{f}_i^{(j)}$, associated to it.

There are many different ways in which one can attempt to minimize the loss from equation (44.5). One standard approach is to do it incrementally starting with only a pair of images. We use the matches between those two images to compute the fundamental and essential matrices, while accounting for outliers using RANSAC. We can then compute an initial 3D reconstruction of the matched points using triangulation as we have already discussed in chapter 40. Then, we add another image and we repeat the same process until we have incorporated all the images. This process will result in a first set of approximated camera poses and 3D locations for a subset of the detected keypoints.

To find the final set of 3D points, the camera poses and the 3D locations of the keypoints are then further refined by bundle adjustment (camera intrinsics are refined too, if needed). The reprojection error loss is usually rewritten using a robust cost function (typically the Huber kernel).

[Figures 44.9 and 44.10](#) show the final 3D reconstructions and recovered camera locations for the two sets of images that we have used in this chapter.



Figure 44.9: 3D reconstruction of the sugar bowl. The figure shows two different viewpoints and the recovered camera locations. (a) Bird's view. (b) Side view.

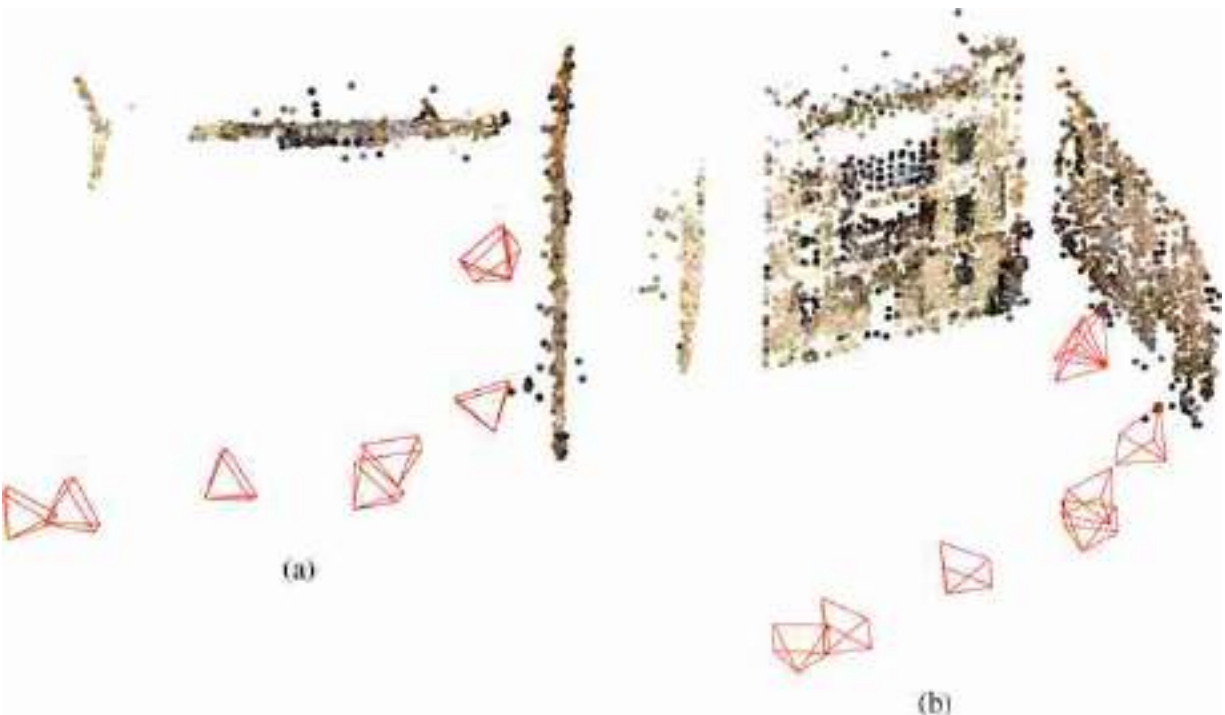


Figure 44.10: 3D reconstruction of the plaza. The figure shows two different viewpoints and the recovered camera locations. (a) Bird's view. (b) Arbitrary view.

SFM is predominantly performed in one of two modes: the incremental mode; and the batch model offline or batch-mode SFM approaches, which

compute the scene structure after processing all input images. In this mode, bundle adjustment will only need to be run once, albeit across a potentially large number of cameras and 3D points. This mode allows leverages images from all available viewpoints right from the outset. However, if new images are added the entire SFM pipeline needs to be rerun.

Online, or incremental-mode SFM approaches attempt to estimate scene structure and camera parameters as and when a new image arrives. In this mode, bundle adjustment is run periodically, whenever a sufficient number of points have been newly added. Incremental-mode SFM is often used in applications where all images are not available a priori, or where real-time performance is desired.

44.4 Concluding Remarks

Multiview reconstruction is an active field of research with a wide range of applications in vision, graphics and healthcare. While a sparse SFM pipeline results in accurate 3D reconstruction, several applications (such as robotics and computer graphics) require dense 3D scene representations. The goal of dense SFM, not covered here, is to reconstruct every pixel from every image, as opposed to the sparse SFM systems described previously that only reconstruct a subset of pixels.

In this chapter, we focused on the most popular variant of SFM used today — feature-based, or sparse, SFM. In sparse SFM, the 3D reconstruction is computed only for a small subset of the image pixels — distinct points that may be detected reliably and repeatably across camera pose variations. SFM approaches have also attempted to leverage other kinds of features, such as lines, planes, or objects. Point-based features remain the most popular choice due to their geometric simplicity, and the availability of a wide range of point feature detectors and descriptors.

In dense SFM, the 3D reconstruction is computed for every (or most) of the image pixels. The resulting 3D reconstructions are rich and visually informative, and are useful in applications like robot navigation or mixed reality. Dense SFM approaches usually optimize photometric error, that is, the difference in grayscale or red-green-blue (RGB) intensities of corresponding pixels across images. This is in contrast to the point feature-based methods we looked at in this chapter that minimize a 3D reprojection error. However, these dense SFM techniques tend to be much slower than

the sparse SFM methods, and often require more assumptions about the scene or the images, such as the scene being Lambertian (i.e., the grayscale or RGB intensity of a 3D point is invariant to the viewing direction) or the images having a substantially smaller baseline motion compared to the range of motions a point feature-based SFM approach can handle. We will talk about dense optimization of the photometric error when discussing motion estimation in part XII.

45 Radiance Fields

45.1 Introduction

In this chapter we will come full circle. Starting from the Greek's model of optics (chapter 1) that postulated that light is emitted by the eyes and travels in straight lines, touching objects in order to produce the sensation of sight (extramission theory), radiance fields will use that analogy to model scenes from multiple images. Here, geometry-based vision and machine learning will meet.

Before diving into radiance fields, we will review Adelson and Bergen's **plenoptic function**; a radiance field can be considered as a way to represent a portion of the plenoptic function.

45.1.1 The Plenoptic Function

We first discussed the plenoptic function when we introduced the challenge of vision in chapter 1.

As stated in the work by Adelson and Bergen [12], the plenoptic function tries to answer the following question: “We begin by asking what can potentially be seen. What information about the world is contained in the light filling a region of space?”

Given a scene, the plenoptic function is a full description of all the light rays that travel across the space ([figure 45.1](#)). The plenoptic function tells us the light intensity of a light ray passing through the three-dimensional (3D) point (X, Y, Z) from the direction given by the angles (ψ, ϕ) , with wavelength λ , at time t :

$$L(X, Y, Z, \psi, \phi, \lambda, t) \tag{45.1}$$

In this chapter we will ignore time and we will only use three color channels instead of the continuous wavelength. We can represent the plenoptic function with the parametric form L_θ where the parameters θ have to be adapted to represent each scene.

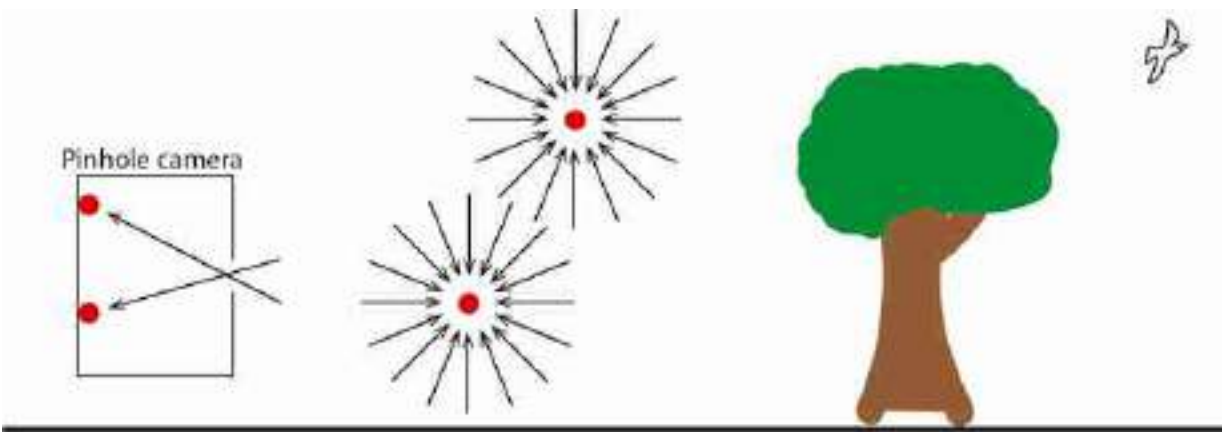


Figure 45.1: Plenoptic function [12]. The figure shows a slice of the plenoptic function at four locations. Two of the locations are in free space, and two other locations are inside a pinhole camera.

An image gets formed by summing all the light rays that reach each sensor on the camera (that is, we sum over a section of the plenoptic function). [Figure 45.1](#) shows an illustration of the plenoptic function at four points, two of them are inside a pinhole camera and are used to form an image of what is outside. The plenoptic function inside the pinhole camera has most of the values set to zero and only specific directions at each location have non-zero values.

In chapter 1 we discarded the plenoptic function by simply saying, “Although recovering the entire plenoptic function would have many applications, fortunately, the goal of vision is not to recover this function.” But we did not really give any argument as to why simplifying the goal of vision was a good idea. What if we now we change our minds and decide that one useful goal of vision is to actually recover that function entirely? Can it be done? How?

If we had access to the plenoptic function of a scene, we would be able to render images from all possible viewpoints within that scene. One attempt at this is the **lumigraph** [170] which extracts a subset of the plenoptic function from a large set of images showing different viewpoints of an object. Another approach, **light field rendering** [293], also uses many images to get a continuous representation of the field of light in order to be able to render new viewpoints (with some restrictions). This chapter will mostly focus on a third approach to modeling portions of the plenoptic function, where we model a scene with a **radiance field** that assigns a color and volumetric density to each point in 3D space.

45.2 What Is a Radiance Field?

A radiance field is a representation of part of the plenoptic function, based on the idea that the optical content of a scene can be modeled as a cloud of colorful particles with different levels of transparency. Such a representation can be rendered into images using volume rendering, which we will learn about in section 45.4. Radiance fields come in a variety of configurations but we will stick with the definition from Mildenhall et al. [336], which popularized this representation. We will presently define a radiance field L as a mapping from the coordinates in a scene to color and **density** (i.e. transparency) values of the scene content at those coordinates. The coordinates can be two-dimensional (2D) (X, Y) for a 2D world, or 3D (X, Y, Z) for a 3D world. They may also contain the angular dimensions $(\psi$ and $\phi)$ that represent the viewing angle from which we are looking at the scene. Representing color using r, g, b values, and representing density with a scalar σ , a radiance field is therefore a function:

$$L: X, Y, Z, \psi, \phi \rightarrow r, g, b, \sigma \quad \triangleleft \text{radiance field} \quad (45.2)$$

The angular coordinates ψ and ϕ allow that an object's color be different depending on the angle we look at it, which is the case for shiny surfaces and many other materials. We will use the terms L^c and L^σ to denote the subcomponents of L that output color and density values respectively:

$$L^c: X, Y, Z, \psi, \phi \rightarrow r, g, b \quad (45.3)$$

$$L^\sigma: X, Y, Z, \psi, \phi \rightarrow \sigma \quad (45.4)$$

45.2.1 A Running Example: Seeing in Flatland

For simplicity, we will study radiance fields for a 2D world. Radiance fields in 3D are the same thing just with one more dimension. This 2D world is like the one in *Flatland*, which is a book by Edwin Abbott [3] where the characters are triangles, squares, and other simple shapes that inhabit a 2D plane. Here is what their world looks like ([figure 45.2](#)):

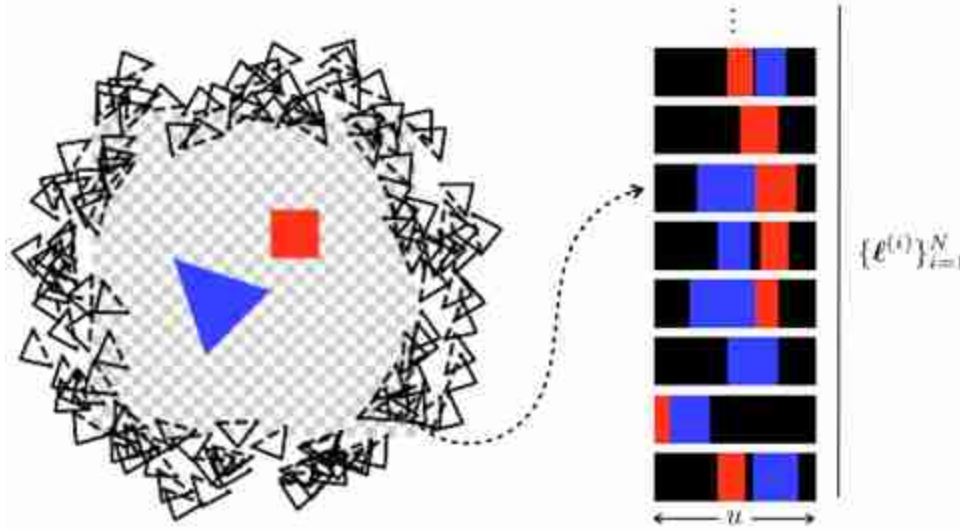


Figure 45.2: The scene we will model. The circle of black triangles on the left are the cameras. The 1D images they see are shown on the right. These images are denoted as $\{\ell^{(i)}\}_{i=1}^N$.

On the left is a top-down image of the world, but inhabitants of Flatland cannot see this. Instead they see the images on the right, which are one-dimensional (1D) renderings of the 2D world. The circle of black triangles on the left are cameras that result in the 1D photos seen on the right. These show what our Flatland citizens would see if they look at the scene from different angles.

A radiance field for this scene is given in [figure 45.3](#). This figure shows how each possible input coordinate (X, Y) gets mapped to a corresponding output color L^c and density L^σ .

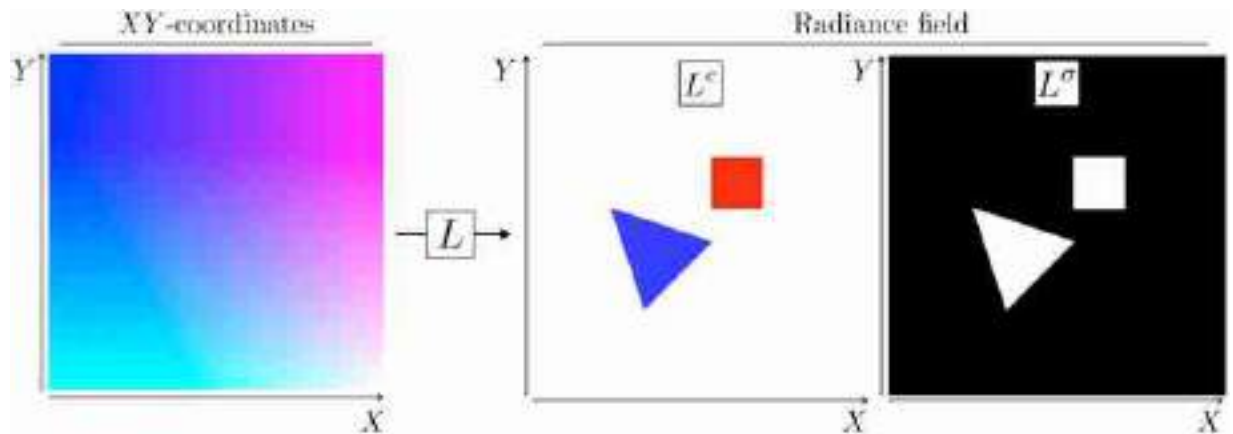


Figure 45.3: How a radiance field maps coordinates to colors/densities. (left) Input (X, Y) coordinates, visualized with X -values in the green channel and Y -values in the blue channel. (right) Radiance field components L^c (colors) and L^σ (densities) rendered at each of these coordinates.

45.2.2 Aside: Representing Scenes with Fields

Radiance fields are an example of **vector fields**, which are functions that assign a vector to each position in space. For example, we might have a vector field $f: X, Y, Z \rightarrow v_1, v_2$, where X, Y and Z are coordinates and v_1 and v_2 are some values. Let's take a moment to consider field-based representations more broadly, since they turn out to be a very important way of representing scenes. Essentially, fields are a way to represent functions that vary over space, and fields appear all over the place in this book. For example, every image is a field: it is a mapping from pixel coordinates to color values. The images we usually deal with in computer vision are discrete fields (the input coordinates are discrete-valued) but in this chapter we will instead deal with continuous fields (the input coordinates are continuous-valued). Unlike regular images, fields can also be more than two-dimensional and can even be defined over non-Euclidean geometries like the surface of a sphere. Because they are such a general-purpose object, fields are used as a scene representation in many contexts. Some examples are the following:

- **Pixel** images, $f: x, y \rightarrow r, g, b$, are fields with the property that the input domain is discrete, or, equivalently, the field is piecewise constant over little squares that tile the space.
- **Voxel** fields, $f: X, Y, Z \rightarrow \mathbf{v}$ are discrete fields that can represent values $\mathbf{v}$ like spatial occupancy, flux, or color in a volume. They are like 3D

pixels. Like pixels, they have the special property that they are piecewise constant over cubes the size of their resolution.

- **Optical flow fields**, $f: x, y, t \rightarrow u, v$, measure motion in a video in terms of how scene content flows across the image plane. We encountered these fields in chapter 47.
- **Signed distance functions (SDFs)** [90], $f: X, Y, Z \rightarrow d$, represent the distance to the closest surface to point $[X, Y, Z]^T$. This is a useful representation of geometry.

Notation reminder: we use capital letters for world coordinates and lowercase letters for image coordinates.

As you read this chapter, keep in mind these other kinds of fields, and think about how the methods you learn next could be used to model other kinds of fields as well.

45.3 Representing Radiance Fields With Parameterized Functions

Continuous fields are powerful because they have infinite resolution. But this means that we can't represent a continuous field explicitly; we cannot record the value at all *infinite* possible positions. Instead we may use parameterized functions to represent continuous fields. These functions take as input any continuous position and tell us the value of the field at that location. Sometimes this is called an **implicit representation** of the field, since we don't explicitly store the values of the field at all coordinates. Instead we just record a *finite* set of parameters of a function that can tell us the value of the field at any coordinate. Recall that we learned about implicit image representations in section 38.6.

Implicit representations are also used in variational autoencoders (VAEs) where we represent an infinite set (an infinite mixture of Gaussians) via a parameterized function from a continuous input domain (see chapter 33). You can consider a VAE to be a continuous field mapping from latent variables to images!

There are many different parameterized functions L_θ that could represent our target radiance field L . We could use a neural net (e.g., [336]), or a mixture of Gaussians (e.g., [256]), or any number of other function

approximators. The important property we seek is that the field L_θ is a differentiable function of the parameters θ , hence we can use gradient-based optimization to find a setting of the parameters that yields a desirable field (we will use this property when we fit a radiance field to images). Whatever function family we use, it will take as input coordinates and produce colors and densities as output ([figure 45.4](#)). Because we will use differentiable functions for this, we can think of it just like a module in a differentiable computation graph ([figure 45.4](#)). Later we will put together multiple such modules and use backpropagation to optimize the whole system to fit the field to a set of images.

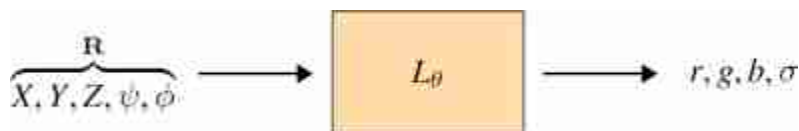


Figure 45.4: Module for a parameterized radiance field. We use $\mathbf{R}$ to denote the vector of coordinates L_θ takes as input.

45.3.1 Neural Radiance Fields (NeRFs)

We will focus our attention on one very popular way of parameterizing L_θ : **Neural radiance fields (NeRFs)** [[336](#)]. NeRFs model the radiance field L with a neural network L_θ . The neural network architecture in the original NeRF is a multilayer perceptron (MLP), but other architectures could be used as well. For the MLP formulation, evaluating the color-density at a single coordinate corresponds to a single forward pass through an MLP.

For fitting and rendering a NeRF, as we will see below, we will not actually need to evaluate L_θ at all locations on a grid. Instead we sample positions along rays.

Often, however, we want to evaluate the color-density at multiple coordinate locations. To create an explicit representation of the field's values at a grid of coordinates, we can query the field on this grid. In this usage, the NeRF L_θ looks like a convolutional neural network (CNN) with 1×1 filters, as this is the architecture that results from applying an MLP to each position in a grid of inputs. We show this usage below, in [figure 45.5](#).

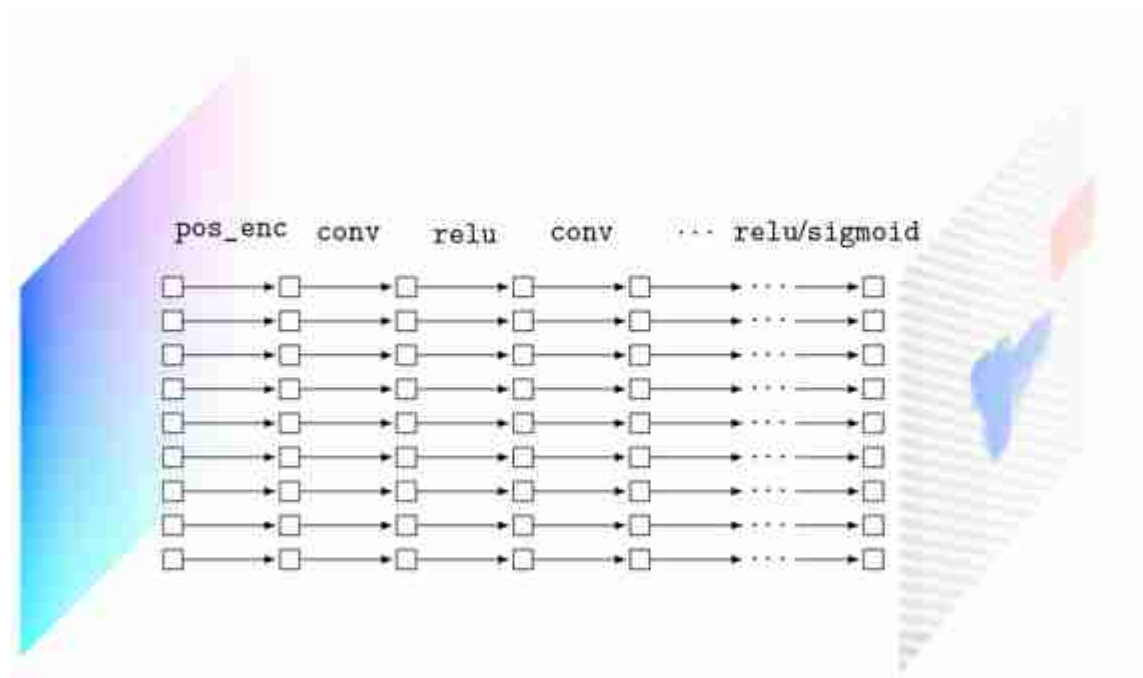


Figure 45.5: NeRF architecture for computing the values at each position in an entire radiance field, sampled on a grid. The `pos_enc` refers to positional encoding. You can consider this architecture be a CNN with 1x1 filters, or as an MLP applied to each input coordinate vector.

As in the original NeRF [336], the output layer is `relu` for producing σ (which needs to be nonnegative) and `sigmoid` for producing r , g , b values (which fall in the range $[0, 1]$).

NeRF also includes a positional encoding layer to transform the raw coordinates into a Fourier representation. In fact, we use the same positional encoding scheme as is used in transformers (see section 26.11). Figure 45.6 shows how this scheme translates the XY -coordinate field of Flatland to positional encodings:

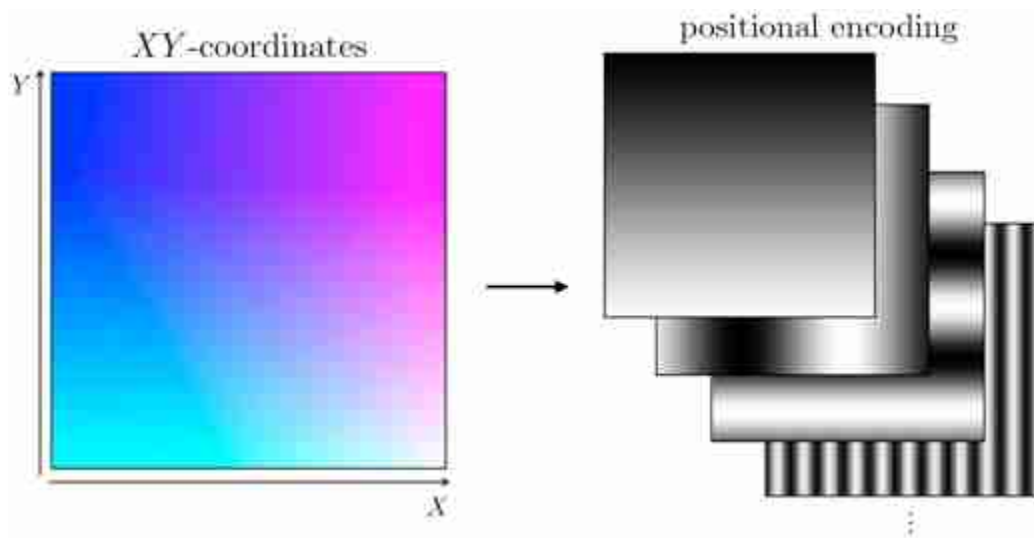


Figure 45.6: The first layer of NeRF applies positional encoding to the input coordinate values. Here we show the resulting positional codes for all possible input coordinate values (X, Y) within some range.

Fourier positional encodings are very effective for NeRFs because they help the network to model high frequencies variations, which are abundant in radiance fields [463]. Why do such encodings help model high frequencies? Essentially, in regular Cartesian coordinates, the location $(0, 0)$ is much closer to $(1, 1)$ than it is to $(100, 100)$, but this isn't necessarily the case with Fourier positional codes. Because of this, with Cartesian coordinates as inputs, an MLP will be biased to assign similar values to $(0, 0)$ and $(1, 1)$ and different values to $(0, 0)$ and $(100, 100)$. Conversely, with Fourier positional codes as input, this bias will be reduced. This makes it so the MLP, taking Fourier positional codes as input, has an easier time fitting high frequency functions that change in value rapidly, as in the case of assigning very different output values to the locations $(0, 0)$ and $(1, 1)$. See [463] for a more thorough analysis of this phenomenon.

45.3.2 Other Ways of Parameterizing Radiance fields

In principle L_θ could be other kinds of functions, including those that are not neural nets. The important property it needs to have is that it be differentiable, so that we can optimize its parameters to fit a given scene. One alternative is to parameterize the radiance fields using a voxel-based representation, as was explored in, e.g., [147]. Another alternative is to use

a mixture of Gaussians [256]. In this case, each Gaussian represents an semi-transparent ellipsoid of some color $\mathbf{c}$ and some density σ . When many such Gaussians overlap, their sum can approximate scene content very well.

45.4 Rendering Radiance Fields

Radiance fields can be useful for many tasks in scene modeling, as they represent both appearance and geometry (density) of content in 3D. However, the main purpose they were introduced for is **view synthesis**. This is the problem of rendering what a scene looks like from different camera viewpoints. In our Flatland example from [figure 45.2](#), this is the problem of rendering the set of 1D images, $\{\ell^{(D)}\}_{D=1}^N$, shown on the left of that figure. In this section, we will see how you can solve view synthesis once you have a radiance field for a scene.

For this purpose, we will use **volume rendering**, which is the strategy that methods like NeRFs use for rendering images from radiance fields (although other rendering methods could be possible). Volume rendering works by taking an integral of the radiance values along each camera ray. Think of it like we are looking into a hazy volume of colored dust. The integral adds up all the color values along the camera ray, weighted by a value related to the density of the dust. A cartoon visualization of the process is given in [figure 45.7](#), and several real results of rendering radiance fields are shown in [figure 45.12](#). But before we get to those, let us go step by step through the math of volume rendering.

The volume rendering equation is:

$$\ell(r) = \int_{t_0}^{t_f} \alpha(t) \overbrace{L^\sigma(r(t))}^{\text{density}} \overbrace{L^c(r(t), \mathbf{D})}^{\text{color}} dt \quad (45.5)$$

$$\alpha(t) = \exp \left(- \int_{t_0}^t \underbrace{L^\sigma(r(t))}_{\text{density}} dt \right) \quad (45.6)$$

Here we model the color as being dependent on the direction $\mathbf{D}$ but the density as being direction-independent. This is a modeling choice and happens to work well because view-dependent color effects are abundant in scenes (e.g., specularities) but view-dependent density effects are less common.

where $r(t)$ gives the coordinates, in the radiance field, of a point t distance along the ray, $\mathbf{D}$ is the direction of the ray (a unit vector), and t_n and t_f are the near and far cutoff points, respectively (we only integrate over the region of the ray within an interval sufficiently large to capture the scene content of interest). To render an image, we can apply this equation to the ray that passes through each pixel in the image. This procedure maps a radiance field L to an image ℓ as viewed by some camera.

This model is based on the idea that the volume is filled with colored particles. When we trace a ray out of the camera into the scene, it may potentially hit one of these colored particles. If it does, that is the color we will see. However, for any given increment along the ray, there is also a chance we will not hit a particle. The chance we hit a particle within an increment is modeled by the density function L^σ , which represents the differential probability of hitting a particle as we move along the ray. The chance that we have not hit a particle all the way up to distance t is then given by α , which is a cumulative integral of the densities along the ray. The integral in equation (45.5) averages over the colors of all the particles we might hit along the ray, weighted by the probability of hitting them.

This may seem a bit strange, but note that there is a special case that might be more intuitive to you. If the particle density is zero, then we have free space, which contributes nothing to the integral. If the particle density is infinite, we have a solid object, and after the ray intersects such an object, α immediately goes to zero and the ray essentially terminates at the surface of the object. Putting these two cases together, if we have a scene with solid objects and otherwise empty space, *then volume rendering reduces to measuring the color of the first surface hit by each ray exiting the camera*. This simple model, called **ray casting**, is in fact how many basic renderers work, such as in the standard rasterization pipeline [437].

This is all to say that volume rendering is a simple generalization of ray casting. One advantage it has is that it can model translucency, and media like gases, fluids, and thin films that let some photons pass through and reflect back others. We will see an even bigger advantage in section 45.5, where we will find that volume rendering gives good gradients for fitting radiance fields to images.

45.4.1 Computing the Volume Rendering Integral

The volume rendering integral has no simple analytical form. It depends on the function L , which may be arbitrarily complex; think about how complex this function will have to be to represent all the objects around you, including their geometry, colors, and material properties. Because of this, we must use numerical methods to approximate equation (45.5).

One way to do this is to approximate the integral as a discrete sum. A particularly effective approximation is the following, which is called a quadrature rule (see [325, 326] for justification of this choice of approximation):

$$\ell(r) \approx \sum_{i=1}^T \alpha_i (1 - e^{-L^\sigma(\mathbf{R}_i)\delta_i}) L^c(\mathbf{R}_i, \mathbf{D}) \quad (45.7)$$

$$\alpha_i = \exp\left(-\sum_{j=1}^{i-1} L^\sigma(\mathbf{R}_j)\delta_j\right) \quad (45.8)$$

$$\delta_i = t_{i+1} - t_i \quad (45.9)$$

where we have now replaced our continuous radiance field from equation (45.5) with a discrete vector of samples from the radiance field, $\{\mathbf{R}_1, \dots, \mathbf{R}_T\}$, where $\mathbf{R}_t = r(t)$ is the world coordinate of a point distance t along the ray r .

[Figure 45.7](#) visualizes this procedure:

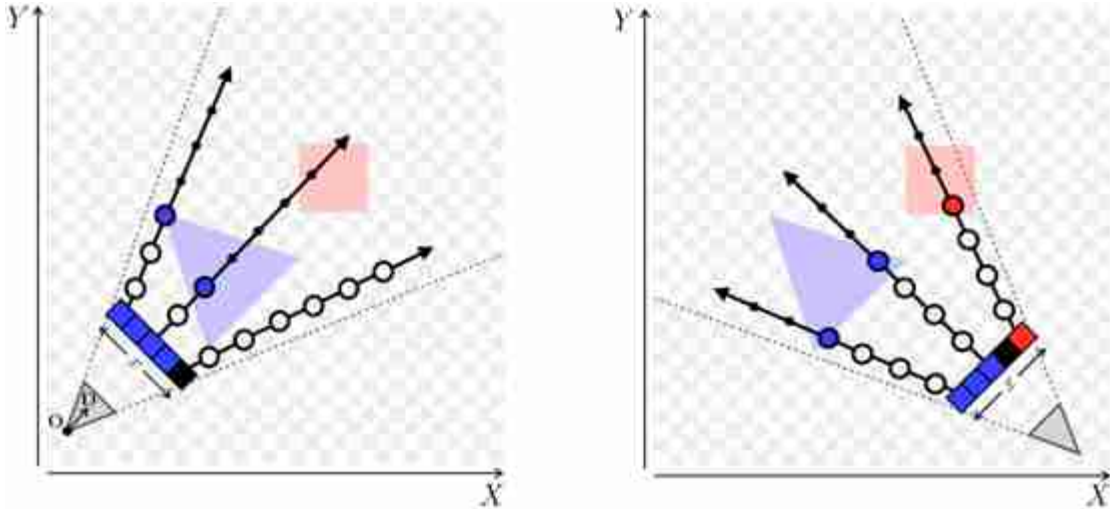


Figure 45.7: Volume rendering of our Flatland radiance field.

Here we have a 2D radiance field of our Flatland scene, and we are looking at it from above. We show how two cameras (gray triangles in corners) view this scene. The camera sensors are depicted as a row of five pixels, indexed by coordinate x . We send a ray outward from the camera origin through each pixel to query the scene. Each ray gets sampled at the points that are drawn as circles. The size of the circle indicates the α value and the color/transparency of the circle indicates the color/density of the radiance field at that point. In this scene, we have solid objects, so the density is infinite within the objects and as soon as a ray hits an object, all remaining points become occluded from view (α goes to zero for the remaining points, as indicated by the circles shrinking in size).

In this figure, we took samples at evenly spaced increments along the ray. Instead, as suggested in [336], we can do better by chopping the ray into T evenly spaced intervals, then sampling one point at uniform within each of these intervals. The effect of this is that all continuous coordinates will get sampled with some probability, and therefore if we run the approximation over and over again (like we will when using it in an optimization loop; see section 45.3.1) we will eventually take into account every point in the continuous space (they will all be supervised during fitting a radiance field to explain a scene).

Following this strategy, for a ray whose origin is $\mathbf{O}$ and whose direction is $\mathbf{D}$, we compute sampled coordinates $\{\mathbf{R}_1, \dots, \mathbf{R}_T\}$ as follows:

$$\mathbf{R}_i = \mathbf{O} + t_i \mathbf{D} \quad (45.10)$$

$$t_i \sim \mathcal{U}[i-1, i] * \frac{t_f - t_n}{T} + t_n \quad (45.11)$$

The distribution $U[a, b]$ is the uniform distribution over the interval $[a, b]$.

Now we can put all our pieces together as a series a computational modules that define the full volume rendering pipeline. Later in the chapter, we will combine these modules with other modules (including neural nets) to create a computation graph that can be optimized with backpropagation.

Our task is to compute the image ℓ that will be seen by a specified camera viewing the scene. We need to find the color of each pixel in this image. To find the color of a pixel, $\ell[n, m, :]$ ²¹, at camera coordinate n, m , the first step is to find the ray (origin $\mathbf{O}$ and direction $\mathbf{D}$) that passes through this pixel. This can be done using the methods we have encountered in this book for modeling camera optics and geometry, which we will not repeat here (see section 39.3.2). The key pieces of information we need to solve this task are the camera origin, $\mathbf{O}_{\text{cam}}$, and the camera matrix, $\mathbf{K}_{\text{cam}}$, that describes the mapping between pixel coordinates and world coordinates (steps for constructing $\mathbf{K}_{\text{cam}}$ are given in section 55.2.2). We apply this mapping, then compute the unit vector from the origin to the pixel to get the direction $\mathbf{D}$ ([figure 45.8](#)):

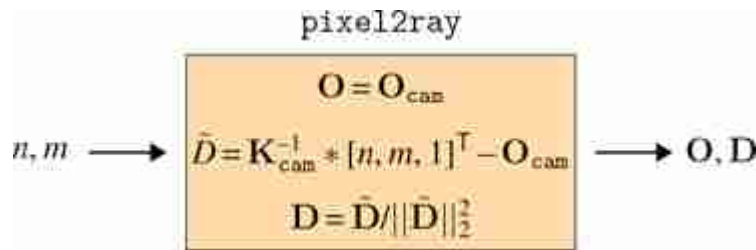


Figure 45.8: Module for mapping from pixel coordinates to the world coordinates of a ray through that pixel.

Next we sample world coordinates along the ray, as described previously ([figure 45.9](#)):

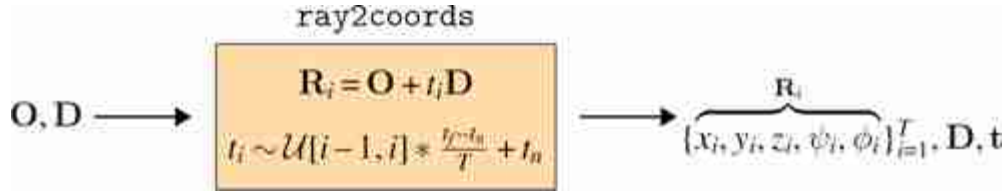


Figure 45.9: Module for sampling coordinates along a ray.

Finally, we use the volume rendering integral (using the quadrature rule approximation given previously) to compute the color of the pixel ([figure 45.10](#)):

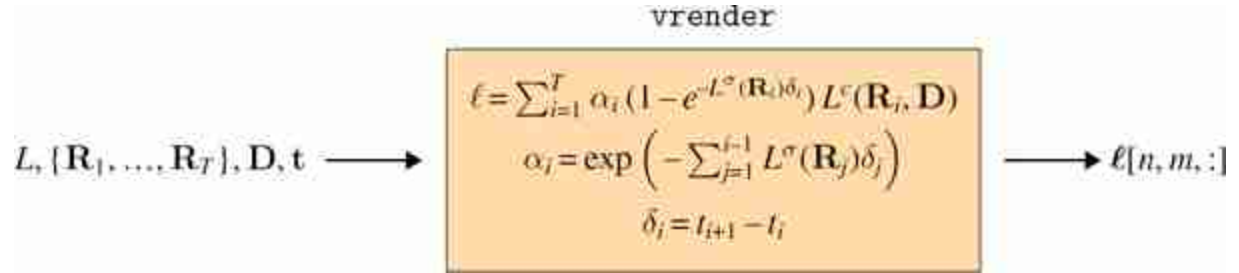


Figure 45.10: Module for volume rendering of a single ray.

This pipeline gives a mapping from pixel coordinates to color values: $n, m \rightarrow r, g, b$. Our task in the next section will be to explain a set of images as being the result of volume rendering of a radiance field, that is, we want to infer the radiance field from the photos.

45.5 Fitting a Radiance Field to Explain a Scene

In this section our goal is to find a radiance field that can explain a set of observed images. This is the task our denizens of Flatland have to solve as they walk around their world and try to come up with a representation of the scene they are seeing. This is the task of vision. Rendering radiance fields into images (view synthesis) is a graphics problem. We are now moving on to the vision problem, which is the inverse problem and is where

we really wanted to get: inferring the radiance field that renders to an observed set of images.

Concretely, our task is to take as input a set of images, along with the camera parameters for the cameras that captured those images. We will produce a parameterized radiance field as output. The full fitting procedure therefore maps $\{\ell^{(i)}, \mathbf{K}_{\text{cam}}^{(i)}, \mathbf{O}_{\text{cam}}^{(i)}\}_{i=1}^N \rightarrow L_\theta$.

Our objective is that if we render that radiance field, using each of the input cameras, the rendering will match the input images as closely as possible.

45.5.1 Computation Graph for Rendering an Image

Given L_θ , we will render an image simply using volume rendering as described previously. The entire computation graph is shown in [figure 45.11](#). Notice that all the modules are differentiable, which will be critical for our next step, fitting the parameters to data.

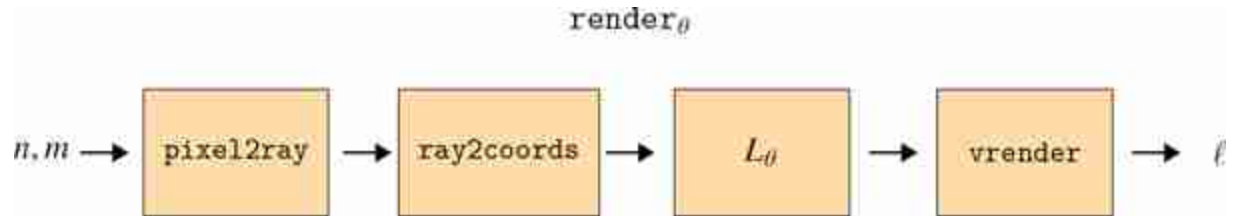


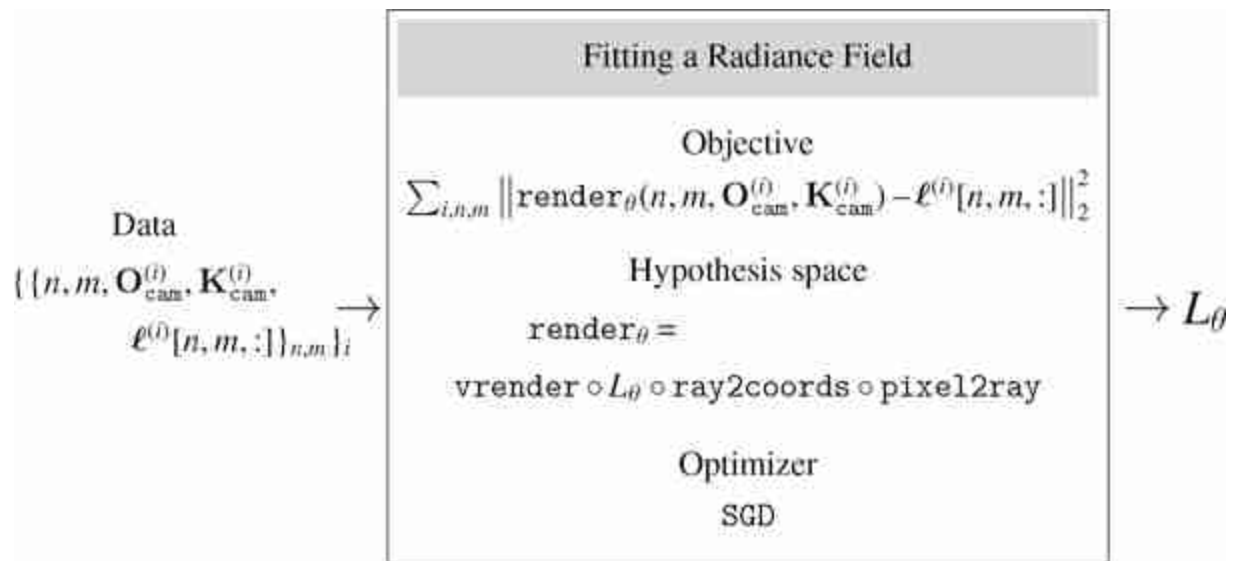
Figure 45.11: Rendering a particular camera's pixel at coordinate n, m .

Note that the camera parameters also need to be input into `pixel2ray` and that `vrender` also takes as input the direction $\mathbf{D}$ and sampling points $\mathbf{t}$ computed by `ray2coords`.

45.5.2 Optimizing the Parameters of this Computation Graph

Fitting a L_θ to explain the images just involves optimizing the parameters θ to minimize the reconstruction error between the images of the radiance field rendered through `render_theta` and the observed input images.

We can phrase this procedure as a learning problem and show it as the diagram below:



Fitting a radiance field to volume rendered images is very much like solving a magic square puzzle. We try to find the radiance field (the square) that integrates to the observed images (the row and column sums of the square).

From this perspective, radiance fields such as NeRF are a clever hypothesis space for constraining the mapping from inputs to outputs of a learned rendering system. Of course we could have used a much less constrained hypothesis space such as a big transformer that directly maps from input pixel coordinates to output colors, without the other modules like `ray2coords` or `vrender`. However, such an unconstrained approach would be much less sample efficient and would require far more data and parameters to learn a good rendering solution (refer back to chapter 11 to recall why a more constrained hypothesis space can reduce the required amount of training data to find a good solution). However, such a solution could also be more general purpose, as it could handle optical effects that are not well captured by radiance fields.

In fact, there are two products you can get out of a fit radiance field, one for the graphics community and one for the vision community. The graphics product is a system that can render a scene from novel viewpoints. This works by just applying `renderθ` ([figure 45.11](#)) to new cameras with new viewpoints. The second product is an inferred radiance field, which tells us something about what is where in the scene. For example, the density of this radiance field can tell us the geometry of the scene, as we will see in the following example of fitting a radiance field to our Flatland scene.

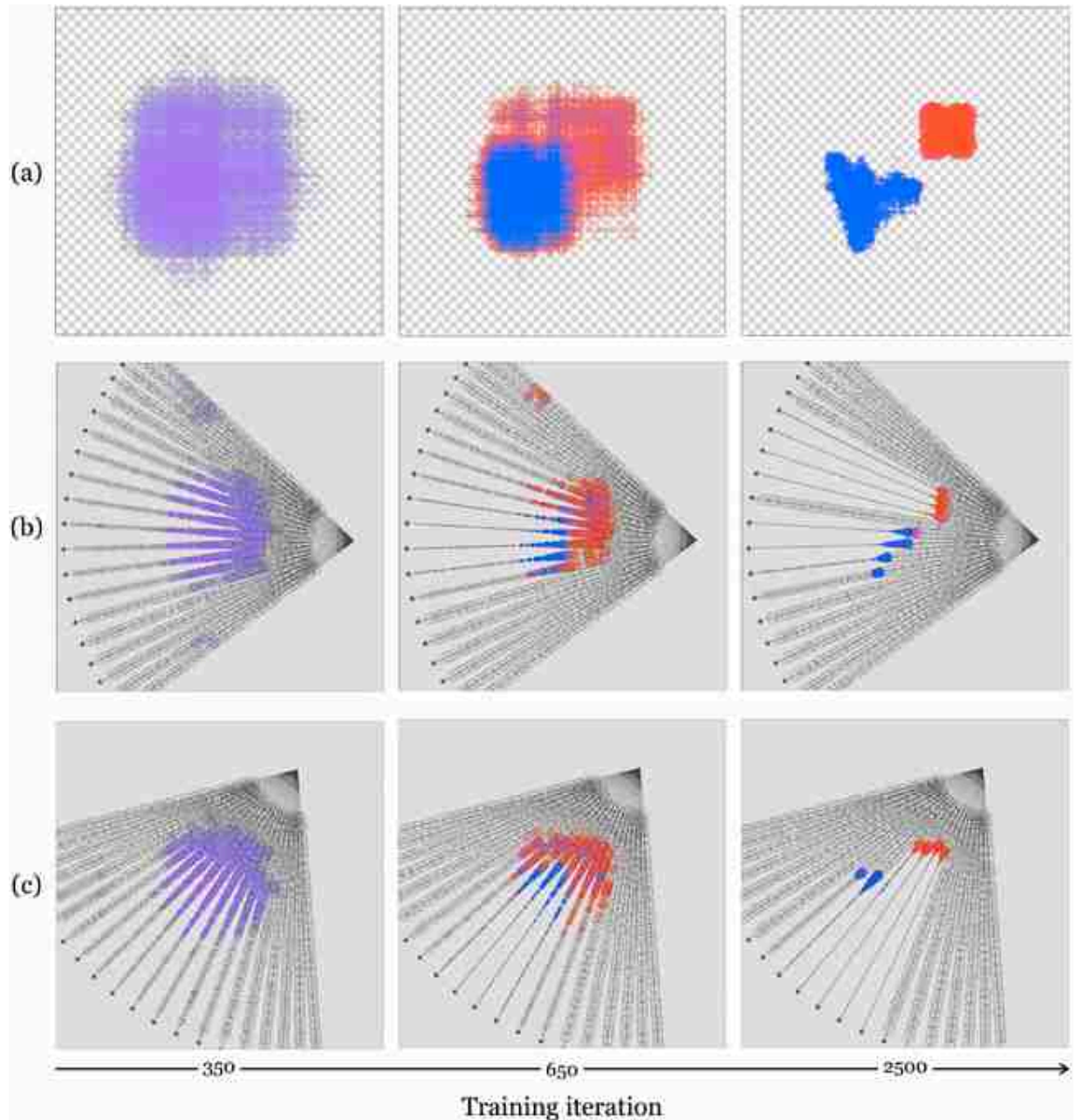


Figure 45.12: Iterations of fitting a radiance field to Flatland. All of these are top-down views of the world (we are looking at Flatland from above, which is a view the inhabitants cannot see). (a) The radiance field visualized as an image with color equal to the color of the field at each position and transparency proportional to the density of the field at each position. (b and c) The volume rendering process for two different cameras looking at the radiance field. The circle colors and transparencies again show the color and density of the field, and the circle size shows the α value as we walk along each camera ray. Small circles mean those points are more occluded; i.e. there is a low probability of the ray reaching them and they contribute very little to the volume rendering integral.

In [figure 45.12](#), we show several iterations of the fitting process. The top

row shows the radiance field as seen from above. This is a perspective our Flatlanders cannot directly see but can infer, just like we on Earth cannot directly see the full shape of a 3D (X , Y , Z) radiance field but can infer it from images. The bottom rows show volume rendering from two different cameras. The small circles along the rays are samples of the radiance field. Their size is the α value at that location along the ray, and their color and opacity are the color and density values of that location in the radiance field. Notice that initially we have a hazy cloud of stuff in the middle of the scene and over iterations it congeals into solid objects in the shapes of our Flatland denizens.

After around 2,500 steps of gradient descent, we have arrived at a fairly accurate representation of the scene. At this point, the objects' densities are high enough that volume rendering essentially amounts to just ray casting (finding the color of the first surface we hit). So why did we use volume rendering? Not because we care about volumetric effects; in this scene there are none. Rather because it makes it possible for optimization to find the ray casting solution. With volume rendering, we have the property that small changes to the scene content yield small changes to our loss; that is, we have small but non-zero gradients. Ray casting, in contrast, is an all or none operation, and therefore small changes to scene content can yield drastic changes to the rendered images (an occluder can suddenly appear as the object that was previously in view of a pixel). This makes gradient-based optimization difficult with ray casting, but achievable with volume rendering. This is the second, and arguably main benefit of volume rendering that we alluded to previously (the first being the ability to model translucency and subsurface effects).

45.6 Beyond Radiance Fields: The Rendering Equation

Radiance fields are built to support volume rendering, but volume rendering has certain limitations. One major limitation of is that volume rendering does not model multi-bounce optical effects, where a photon hits a surface, bounces off, and then hits another surface, and so on. This leads to issues with how radiance fields represent shadows and reflections. Instead of a shadow being the result of a physical process where photons are blocked

from illuminating one part of the scene, in radiance fields shadows get painted onto scene elements: L^c gets a darker color value in shadowed parts of the scene. If we want to change the lighting of our scene, we therefore need to update the colors in the radiance field to account for the new shadow locations, and with the radiance field representation, there is no simple way to do this (we may have to run our fitting procedure anew, this time fitting the radiance field to new images of the newly lit scene). The same is true for reflections: a radiance field will represent reflected content as painted onto a surface, rather than modeling the reflection in the physically correct way, as due to light bouncing off the surface.

Fortunately, other, more general, rendering models exist that better handle multi-bounce effects. Perhaps the most general is **the rendering equation**, which was introduced by Kajiya [245] in the field of computer graphics. The goal was to introduce a new formalism for image rendering by directly modeling the light scattering off the surfaces composing a scene. The rendering equation can be written as,

$$L(\mathbf{x}, \mathbf{x}') = g(\mathbf{x}, \mathbf{x}') \left[e(\mathbf{x}, \mathbf{x}') + \int_S \rho(\mathbf{x}, \mathbf{x}', \mathbf{x}'') L(\mathbf{x}', \mathbf{x}'') d\mathbf{x}'' \right] \quad (45.12)$$

Here $\mathbf{x}$, $\mathbf{x}'$, and $\mathbf{x}''$ represents a vector of world coordinates, e.g., $\mathbf{x} = [X, Y, Z]$.

where $L(\mathbf{x}, \mathbf{x}')$ is the intensity of light ray that passes from point $\mathbf{x}'$ to point $\mathbf{x}$ (this is analogous to the plenoptic function but with a different parameterization). The function $e(\mathbf{x}, \mathbf{x}')$ is the light emitted from $\mathbf{x}'$ to $\mathbf{x}$, and can be used to represent light sources. The function $\rho(\mathbf{x}, \mathbf{x}', \mathbf{x}'')$ is the intensity of light scattered from $\mathbf{x}''$ to $\mathbf{x}$ by a patch of surface at location $\mathbf{x}'$ (this is related to the bidirectional reflectance distribution function).

The term $g(\mathbf{x}, \mathbf{x}')$ is a visibility function and encodes the geometry of the scene and the occlusions present. If points $\mathbf{x}$ and $\mathbf{x}'$ are not visible from each other, the function is 0, otherwise it is $1/\|\mathbf{x} - \mathbf{x}'\|^2$, modeling how energy propagates from each point.

In words of Kajiya, “The equation states that the transport intensity of light from one surface point to another is simply the sum of the emitted light and the total light intensity which is scattered toward $\mathbf{x}$ from all other surface points.” Integrating equation (45.12) can be done using numeric methods and has been the focus of numerous studies in computer graphics.

While this rendering model is more powerful than the one radiance fields use, it is also more costly to compute. In the future, as hardware improves, we may see a movement away from radiance fields and toward models that more fully approximate the full rendering equation.

45.7 Concluding Remarks

Radiance fields try to model the light content in a scene. The ultimate goal is to model the full plenoptic function, that is, all physical properties of all the photons in the scene. Along the way to this goal, many simplified models have been proposed, and radiance fields are just one of them. Methods for modeling and rendering radiance fields build upon many of the topics we have seen earlier in this book, such as multiview geometry, signal processing, and neural networks. They appear near the end of this book because they rest upon almost all the foundations we have by now built up.

[21](#). In this chapter, we represent RGB images as $N \times M \times 3$ dimensional arrays.

XII

UNDERSTANDING MOTION

Images are observed by a moving camera recording a dynamic world, but we have devoted little space to discuss the analysis of image sequences. We will focus on understanding motion in this set of chapters. This part is composed of four chapters:

- **Chapter 46** introduces the problem of motion estimation and provides a very simple approach to estimate motion across two frames of a video.
- **Chapter 47** explains the image formation process and how the three-dimensional (3D) motion in the scene projects into a sequence of two-dimensional (2D) images.
- **Chapter 48** goes deeper into optical flow estimation, describing classical methods for motion estimation.
- **Chapter 49** finally introduces supervised and unsupervised learning-based methods for motion estimation.

Notation

We will continue using the same notation as in the previous chapters:

- Moving points: $\mathbf{P}(t)$ for 3D points, and $\mathbf{p}(t)$ for 2D points. Where t is time.
- Temporal derivatives: to simplify the equations, we will use the dot notation for temporal derivatives, $\dot{\mathbf{P}}(t) = \partial \mathbf{P} / \partial t$.

46 Motion Estimation

46.1 Introduction

An important task in both human and computer vision is to model how images (and the underlying scene) change over time. Our visual input is constantly moving, even when the world is static. Motion tells us how objects move in the world, and how we move relative to the scene. It is an important grouping cue that lets us discover new objects. It also tells us about the three-dimensional (3D) structure of the scene.

Look around you and write down how many things are moving and what are they doing. Take note of the things that are moving because you interact with them (such as this book or your computer) and the things that move independently of you.

The first observation you might make is that not much is happening. Nothing really moves. Most of the world is remarkably static, and when something moves it attracts our attention. However, motion perception becomes extremely powerful as soon as the world starts to move. Our visual system can form a detailed representation of moving objects with complex shapes. Even in front of a static image, we form a representation of the dynamics of an object, as shown in the photograph in [figure 46.1](#).



Figure 46.1: Even from a static picture we form a rich representation of the dynamics of the scene.
Source: Photograph by Fredo Durand.

Looking at the power of that static image to convey motion, one wonders if seeing movies is really necessary. From the notes you took about what moves around you, probably you deduced that the world is, most of the time, static.

And yet, biological systems need motion signals to learn. Hubel and Wiesel [501] observed that a paralyzed kitten was not capable of developing its visual system properly. The human eye is constantly moving with saccades and microsaccades. Even when the world is static, the eye is a moving camera that explores the world.

Motion tells us about the temporal evolution of a 3D scene, and is important for predicting events, perceiving physics, and recognizing actions. Motion allows us to segment objects from the static background, understand events, and predict what will happen next. Motion is also an important grouping cue that our visual system uses to understand what parts of the image are connected. Similarly moving scene points are likely to belong to the same object. For example, the movement of a shadow accompanying an object, or various parts of a scene moving in unison—even when the connecting mechanism is concealed—strongly suggests that they are physically linked and form a single entity.

Motion estimation between two frames in a sequence is closely related to disparity estimation in stereo images. A key difference is that stereo images

incorporate additional constraints, as only the camera moves—imagine a stereo pair as a sequence with a moving camera while everything else remains static. The displacements between stereo images respect the epipolar constraint, which allows the estimated motions to be more robust. In contrast, optical flow estimation doesn't assume a static world.

Disparity from stereo and optical flow estimations are closely related. Stereo benefits from the epipolar constraint to make estimation easier. For a rectified stereo pair the vertical component of motion between the stereo frames is zero.

Another distinction is that optical flow generally presumes small displacements between consecutive frames due to the short time gap between them. In stereo images, feature displacements tend to be larger. Despite these differences, the remaining steps are similar, and the same architectures can address both tasks.

46.2 Motion Perception in the Human Visual System

The eye is constantly moving, fixating different scene locations every 300 ms. Therefore, the nature of the input signal to the brain is a sequence of ever-changing visual information. It is not a surprise then that motion perception is a key component of visual perception.

Only when the eye tracks a moving object can it move continuously. Otherwise, the eye jumps from one location to another in saccades. Try moving your eyes smoothly and you will notice that you cannot. However, if you look at your finger you will see that you can follow its motion smoothly.

What do we know about the human perception of motion? Much is known and many advances from cognitive psychology have impacted imaging technology. One example is movies. The fact that we learned to create the illusion of continuous motion by displaying a sequence of static images (a phenomenon called **apparent motion**) was a remarkable discovery.

There are a number of visual illusions associated with motion perception that are intriguing and offer a window into how motion perception is implemented in the brain. One famous illusion is the **waterfall illusion**.

When looking at a constant motion (such as a waterfall) our brain adapts to the motion in such a way that if, immediately after adaptation, we look at a static texture we will see it drifting in the opposite direction. The waterfall illusion was already known to the Greeks and was reported by Aristotle. Another remarkable and surprising visual illusion is that it is possible to create static images that produce the sensation of motion. One beautiful example is the **Rotating Snakes** visual illusion created by cognitive psychologist Akiyoshi Kitaoka. [Figure 46.2](#) shows an example a motion-inducing visual illusion, using very simple elements to produce the illusion. The effect becomes more intense as we move our eyes to explore different parts of the bicycle.



Figure 46.2: Motion-induced visual illusion, after [\[344\]](#). The illusion becomes stronger when viewed peripherally rather than looking directly at the image. Changing the contrast of this image can change the direction of perceived motion.

The opposite visual illusion can also be achieved: perceiving no motion when there is movement. By creating sequences with isoluminant and textureless patterns [\[451\]](#), an observer can perceive them as perfectly still, even though they are moving. This illusion requires precise calibration and

only works for the specific observer the system is tuned for; other observers will still see motion. The illusion can be thought of as an adversarial attack on a single observer's motion estimation system.

What mechanisms does the brain use to translate the sequence of images projected on the retina into actual motion in the 3D scene? This question has long been studied by neuroscientists and psychologists.

In area V1 of the visual cortex, most visual neurons respond to moving stimuli and exhibit selectivity to specific motion directions. A motion-selective neuron responds strongly when an edge moves across its receptive field in a particular orientation, and its response diminishes as the motion deviates from the preferred direction. These motion-selective cells project to other specialized areas, with the middle temporal area (area MT) and the medial superior temporal area (area MST) playing significant roles in motion processing. The exact functions and roles of these areas are not yet fully understood. This organization suggests that motion is processed by specialized visual pathways, indicating a modular architecture for the visual system.

One of the early computational models of motion perception was proposed by Hassenstein and Reichardt [[189](#)] when studying the motion detectors in the fly's visual system. Another computational model of human motion perception is the energy model proposed by Adelson and Bergen [[9](#)] and briefly discussed in chapter 22. In this chapter we will focus on motion estimation algorithms developed by the computer vision community without trying to follow biologically plausible mechanisms.

46.3 Matching-Based Motion Estimation

Let's get our hands dirty quickly by trying to estimate how pixels move in a video. Let's consider two frames of a video sequence that contains a few moving objects, as shown in [figure 46.3](#).



Figure 46.3: Two frames of a video sequence captured from a moving car driving along a busy street in Palma de Mallorca (Spain).

These two frames belong to a sequence captured from a moving car driving along a busy street. In this sequence there are cars moving on both sides of the road, some moving away from the camera and others moving toward it. How can we compute the motion between the two frames? One way of representing the motion is by computing the displacement for each pixel between the two frames.

Under this formulation, the task of motion estimation consists of finding, for each pixel in frame 1, the location of the corresponding pixel in frame 2. Just as in the chapter on stereo matching, we need first to define how we will compare pixels to find **correspondences**.

Using the color of each individual pixel will be insufficient as many pixels are likely to have very similar colors. Instead we will represent each pixel by a color patch of size $C = 3 \times (2s + 1) \times (2s + 1)$ centered on each pixel. That is, for pixel in location (n, m) in image ℓ , we will use the patch $\ell[n - s : n + s, m - s : m + s]$ to represent the local appearance in that location. Small values of s will result in small patches that might be less distinctive ($s = 0$ corresponds to use individual pixels), while large values of s will result in large discriminative patches but might fail if there is sufficient geometric distortion between the two frames due to motion. By representing each pixel with a patch, we are transforming the image into a feature map of size $3 \times (2s + 1) \times (2s + 1)$. As in chapter 31, we could also use other patch embeddings such as DINO or SIFT descriptors. But since the image transformation between two consecutive frames is usually small,

using red-green-blue (RGB) patches can work well. Another constraint we can use to simplify the matching is to assume that motion will be small between the two frames so that we only need to look for matches inside a small neighborhood of size $L \times L$ in the second frame around the original pixel location. To compute distances between two patches we will use the Euclidian distance between them. We will then compute the motion at each pixel in the first frame by searching for the patch with the smallest distance in the second frame. We can implement this with the following algorithm:

Algorithm 46.1: Patch matching motion estimation. The algorithm starts by chopping the two frames into overlapping patches. Then, for every patch from the first frame, we compute the distance to all the nearby patches in frame 2. Finally, for each input patch we select the closest patch from frame 2 and we record the relative displacement between the two patches. The pseudocode can be rearranged to be more memory efficient.

```

1  Input frames:  $\ell_1, \ell_2 \in \mathbb{R}^{N \times M \times 3}$ , Output motion flow:  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{N \times M}$ 
2  Parameters:  $L$  = Maximum displacement,  $s$  = Patch size parameter
3  for  $i = 0, \dots, M-1$  do
4      for  $j = 0, \dots, N-1$  do
5           $\mathbf{r}_1[i, j] = \ell_1[i-s:i+s, j-s:j+s]$    $\triangleleft$  Chop up frame 1 into patches
6           $\mathbf{r}_2[i, j] = \ell_2[i-s:i+s, j-s:j+s]$    $\triangleleft$  Chop up frame 2 into patches
7  for  $i = L, \dots, M-L-1$  do
8      for  $j = L, \dots, N-L-1$  do
9          for  $d_i = -L, \dots, L$  do
10             for  $d_j = -L, \dots, L$  do
11                  $C[i, j][d_i, d_j] = \text{dist}(\mathbf{r}_2[i + d_i, j + d_j], \mathbf{r}_1[i, j])$    $\triangleleft$  Compute matching cost
12 for  $i = L, \dots, M-L$  do
13     for  $j = L, \dots, N-L$  do
14          $\mathbf{u}[i, j] = \arg \min(\min(C[i, j], \text{axis} = 1))$    $\triangleleft$  Estimate displacement x
15          $\mathbf{v}[i, j] = \arg \min(\min(C[i, j], \text{axis} = 0))$    $\triangleleft$  Estimate displacement y

```

Note that padding must be applied if we want to compute the output optical flow near the image boundaries. Algorithm 46.1 could be written more compactly but we prefer this form for its clarity. The following

images shows the matching result for one input location. In this example, $s = 5$ (patch size of 11×11 pixels), and $L = 16$ (search window of size 33×33 pixels).

In the example shown in [figure 46.4](#), the input patch is centered on the car logo. The matching cost displays the distance using a reversed grayscale map, with smaller values appearing as brighter spots. In this case, the search identifies a unique match. However, it is worth noting that the best matching patch, although correctly detecting the same logo, is not identical to the input patch. This discrepancy can be attributed to factors such as nondiscrete motion and the slight enlargement of the logo as the car approaches the camera.

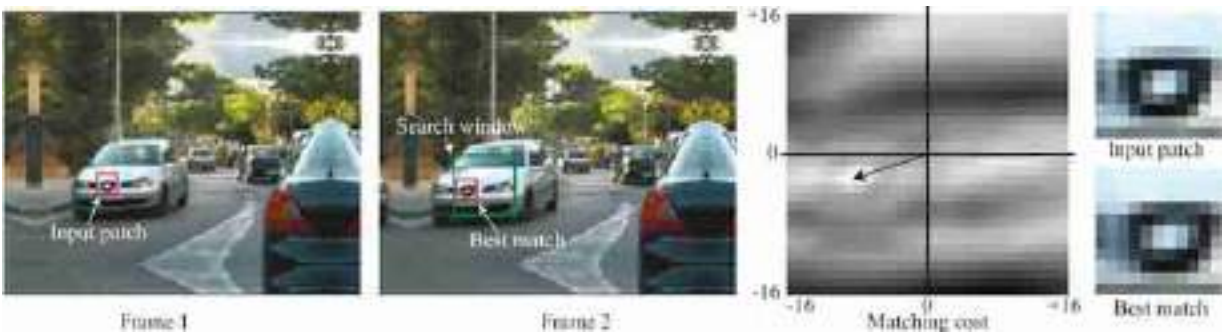


Figure 46.4: Two frames and best match for an input patch from frame 1 within frame 2. Search is done only within a small neighborhood.

The patch size used to represent each pixel is the most important parameter in this algorithm. [Figure 46.5](#) shows the effect of the choice of the parameter s on the estimated optical flow.

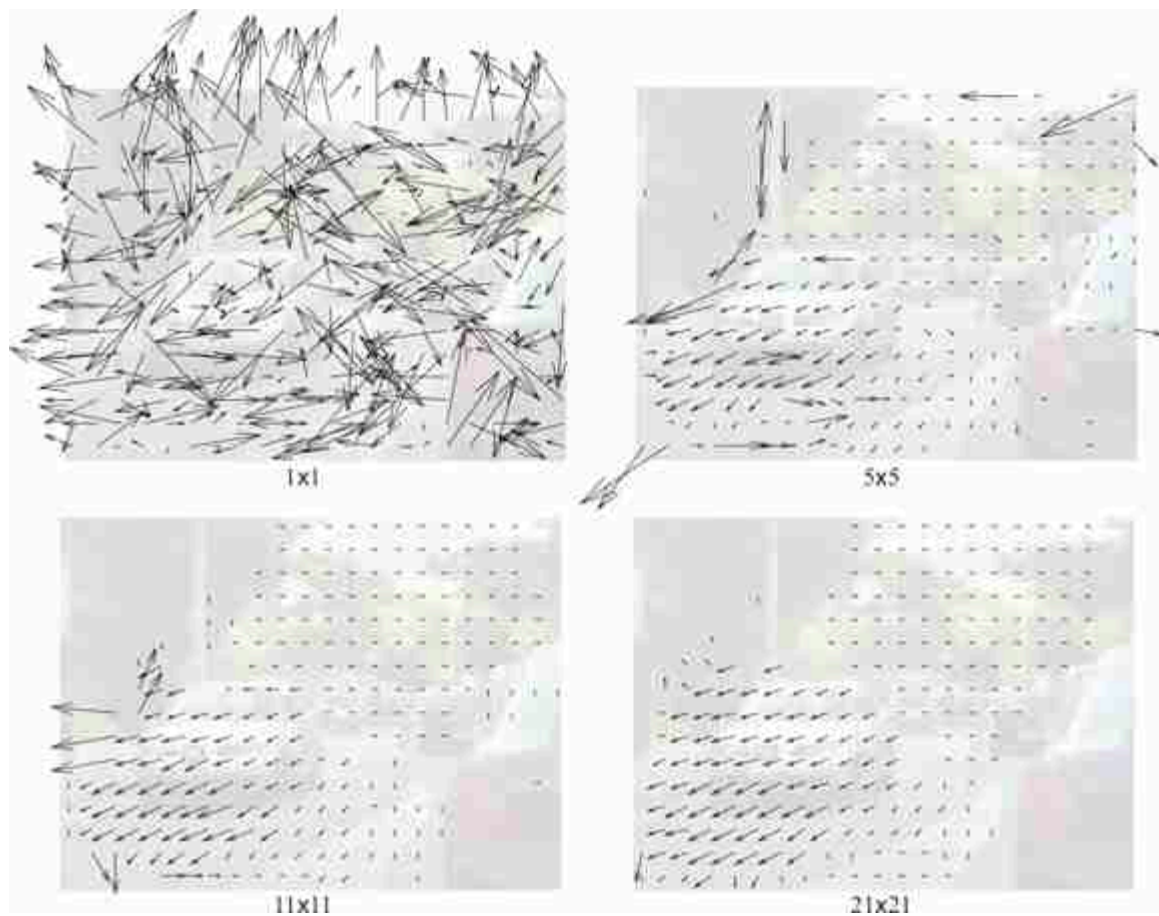


Figure 46.5: Effect of the choice of the patch size parameter, s , on the estimated optical flow. When the patch size is just one pixel ($s = 0$), the approach fails as there are many similar pixels that correspond to different parts of the scene. Only when the patches are large enough, the image matches correspond to the same scene elements. Large patch sizes are necessary. But too large patches lead to oversmoothing.

When using single pixels to represent each location, the matching fails in detecting true correspondences and the estimated motion field is very noisy. Just making the patches 5×5 pixels is already capable of detecting many correct matches and the motion field seems to mostly capture the true motion between the two frames. Further increasing the patch size eliminates some of the errors. However, using very big patches also introduces new problems. In this example we can see how the motion of the gray car extends over the road. This is due to patches overlapping with the car on top, and as the road is mostly uniform, the motion of the car propagates to all the nearby pixels. One of the challenges in this algorithm is that the ideal value of s will depend on the sequence.

This approach has many shortcomings. To start, we assumed that motion is discrete (it can only take on integer values). Therefore, the current approach does not compute displacements of pixels to subpixel accuracy that might be important if we need precision or if motions are very small. We could improve the approach by interpolating the cost function or by computing subpixel displacements using bilinear or bicubic interpolation, but it would become more computationally expensive. In addition, the patch matching method gives poor results near motion discontinuities such as object edges.

One advantage of this algorithm is that motion is computed in a way that is independent of the objects present in the scene. We did not make any assumption about what objects are moving or how. We did not introduce any grouping cues (as in chapter 31) to presegment the image into candidate objects. Therefore, we could use the computed motion as new cue for grouping.

46.4 Does the Human Visual System Use Matching to Estimate Motion?

While researchers aren't sure of the precise computations involved in human motion processing, some experiments can distinguish between classes of algorithms that the visual system may use. The previous method is an example of **pattern matching methods** [9], which are often based on image correlations. A second class of motion algorithms are based on **spatiotemporal filtering** [9] and their principles were briefly described in chapter 22. Spatiotemporal filtering uses the responses of velocity-tuned filters to estimate the motion.

Adelson and Bergen proposed a beautiful motion illusion that distinguishes between two classes of motion algorithms that might be used by the visual system ([figure 46.6](#)). The illusion involves temporal filtering, motion processing, and aliasing and thus provides a good review of the material in this chapter and also chapters 20 and 19.

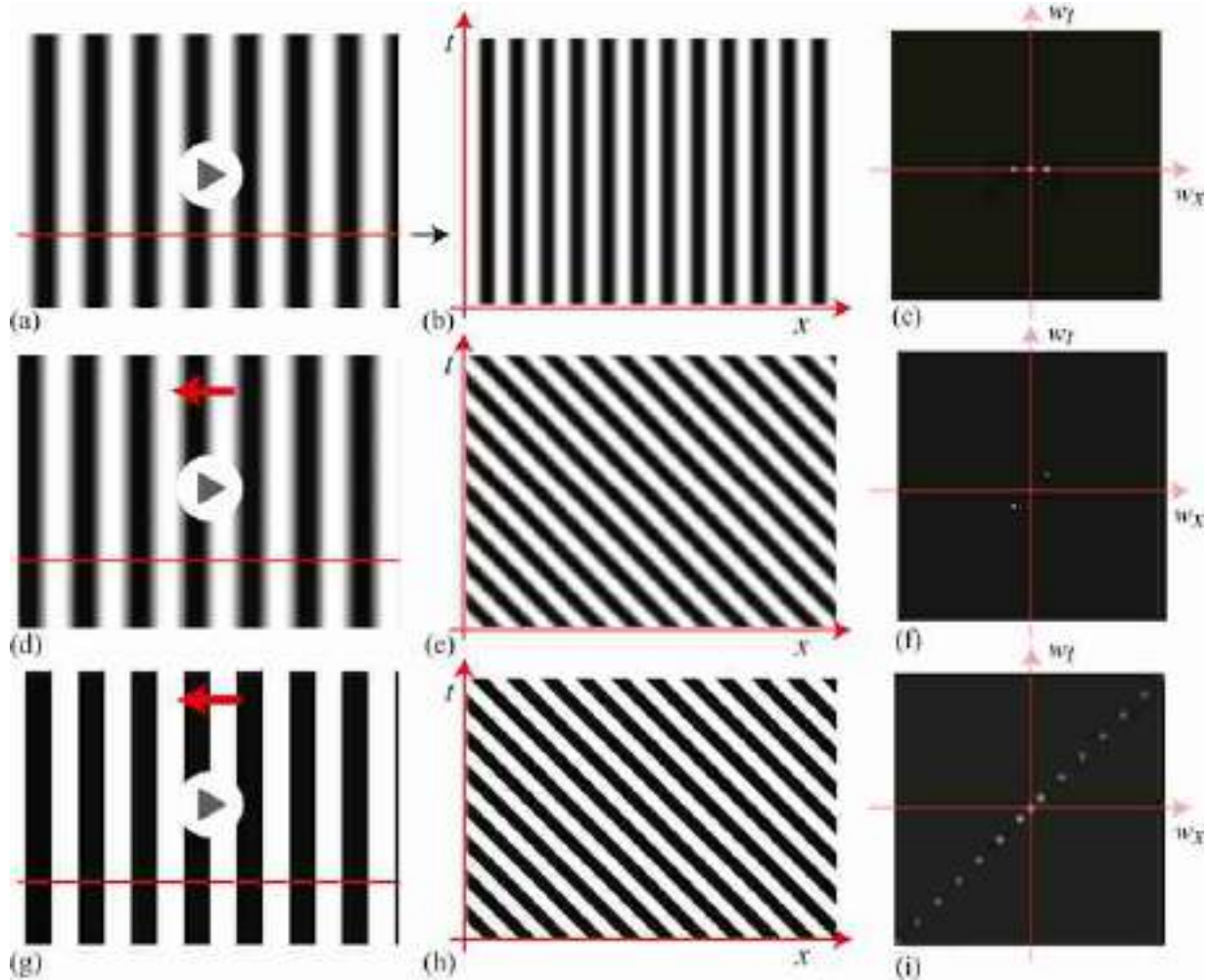


Figure 46.6: Space-time signals, building toward the fluted square-wave motion illusion. The first row shows a stationary sine wave. (a) Movie of a motionless sine wave. (b) Space-time plot shows only vertical structure. (c) Spatiotemporal Fourier transform shows all energy on the zero temporal frequency axis because nothing is moving. (d) The second row shows a moving sine wave. (e) In the space-time plot, speed corresponds to local orientation. (f) The Fourier transform energy is sheared according to the sine wave's speed. (g–i) The third row shows a moving square wave. The additional harmonics required to form a square wave are visible in (i) the spatiotemporal Fourier transform.

The signals, and magnitudes of their space-time Fourier transforms, are developed in [figures 46.6](#) and [46.7](#), building-up from simpler signals. The three rows of [figure 46.6](#) show a stationary sinusoid, a moving sinusoid, and a moving square wave. The spatiotemporal Fourier transform of the stationary sinusoid, $f(x, y, t) = \cos(\pi\omega x)$, is $(\delta(w_x + \omega) + \delta(w_x - \omega))\delta(w_y)\delta(w_t)$. We have added a constant bias to the sinusoid to avoid negative intensity values, leading to an impulse at the center of the Fourier

transform, as shown in [figure 46.6\(c\)](#). The resulting three colinear impulses are along the temporal frequency, $w_t = 0$ line. A space-time plot of the signal ([figure 46.6\[b\]](#)) shows only vertical structures, indicating no motion.

A moving sinusoid has a similar Fourier transform magnitude ([figure 46.6\[f\]](#)) but with the spatiotemporal energies along a line perpendicular to the moving structures in the spatiotemporal signal ([figure 46.6\[e\]](#)). A moving square wave is similar, but the extra harmonics needed to construct the square wave visible in the Fourier transform ([figure 46.6\[i\]](#)).

Continuing the development of the illusion, [figure 46.7\(c\)](#) shows the Fourier transform and space-time plot of a square wave moving in 1/4 period jumps each time increment. This signal can be formed from [figure 46.6\(h\)](#) by applying a periodic sample-and-hold function, resulting in the spectrum of [figure 46.6\(i\)](#) replicated over temporal frequencies, and multiplied by a sinc function over temporal frequency. The resulting Fourier transform magnitude is shown in [figure 46.7\(c\)](#).

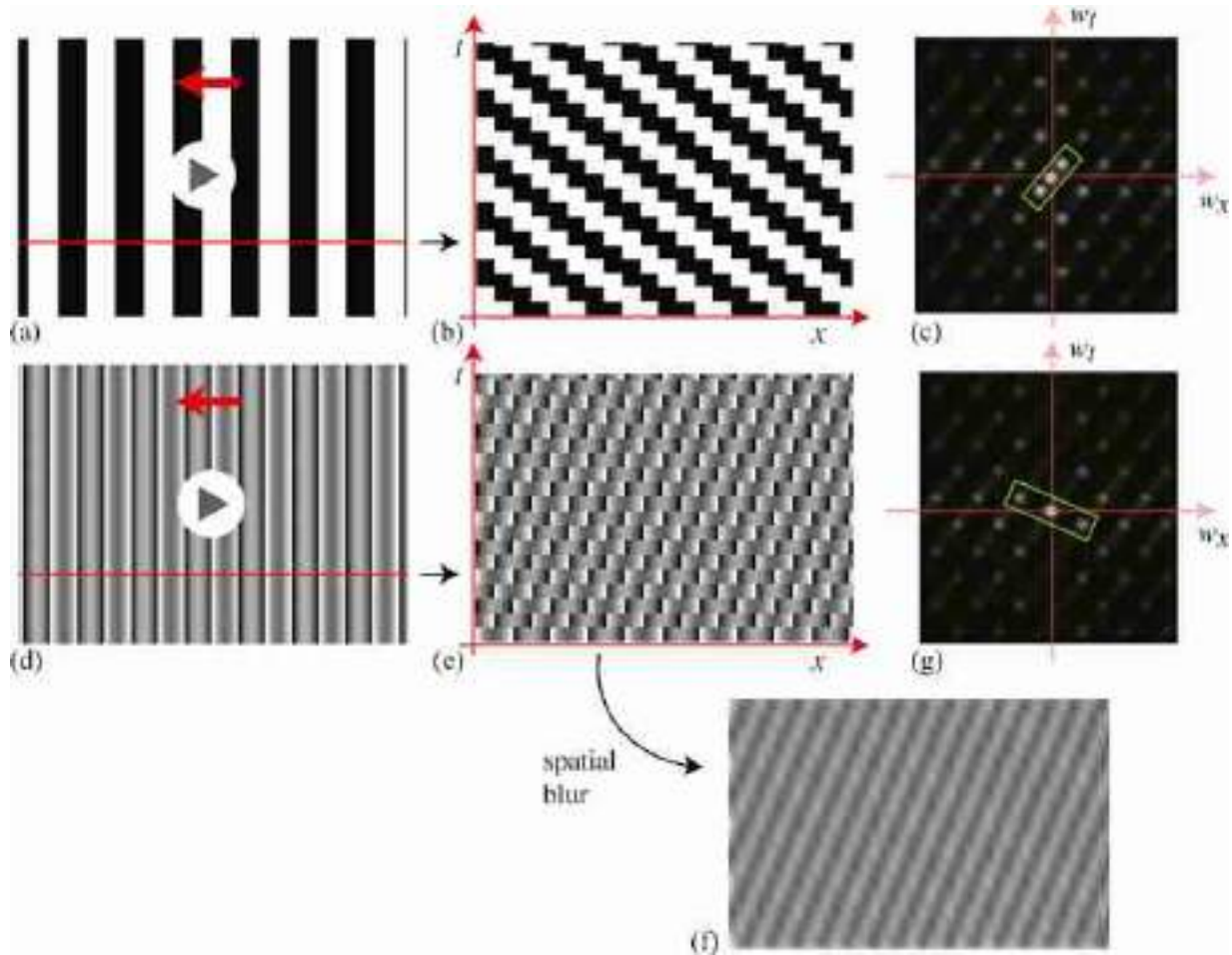


Figure 46.7: Derivation of the fluted square-wave motion illusion, continued from [figure 46.6](#). Top row shows that the square wave moves in $1/4$ wavelength jumps, instead of continuously. This staggered motion generates the additional spatiotemporal frequencies shown in (c). The lowest spatiotemporal frequency (green rectangle) still indicates motion to the left. Second row shows that if we remove the lowest spatial frequency sine wave of the square wave, creating a *fluted square wave*, then the lowest spatio-temporal frequency now moves in the other direction. This is also visible from (e) the space time plot and especially in (f) the spatiotemporally low-pass filtered version.

Because orientation in space-time tells motion direction (section 19.2) the space-time plot of [figure 46.7\(b\)](#) shows that the motion should be perceived to the left. This will be consistent with the behavior of velocity tuned filters (section 22.4.2) responding to the lowest spatiotemporal frequency impulses shown in [figure 46.7\(c\)](#). However, if we remove the lowest spatial frequency sinusoid from the signal, the result is shown in [figure 46.7\(e\)](#), with spatiotemporal Fourier transform shown in [figure 46.7\(g\)](#). Now the lowest spatiotemporal frequency cosine wave is oriented in the other direction. This opposite slope is also visible in the spatial

domain, in the space-time plot of [figure 46.7\(e\)](#), and especially if we apply a low-pass filter, resulting in [figure 46.7\(f\)](#).

The signal of the second row of [figure 46.7](#) poses a conundrum. It can be argued that the signal moves to the left, just as does the signal of row 1 of [figure 46.7](#). The pattern match, that is, the minimum correlation signal indeed moves to the left. But the vision system examining the orientation of the lowest spatiotemporal frequency components of the signal in [figure 46.7\(g\)](#), or looking at the dominant orientations in the space-time plots of [figures 46.7\(e and g\)](#), would find a signal moving to the right! Videos showing each signal are available on the book's web page. These illusions give support to the spatiotemporal energy models for human motion processing.

46.5 Concluding Remarks

In this section we have introduced a conceptually simple approach to compute the motion in sequences. But are the estimated patch displacements meaningful? Do they correspond in any way to the motion in the 3D world?

We have computed motion between two frames before really understanding the motion formation process (i.e., how a camera looking at a moving 3D scene produces two-dimensional [2D] sequences). How should the correct motion look? And what do we want to do with it? So, before we move into more sophisticated motion estimation algorithms, let's revisit the image formation process and examine how motion on the image plane emerges from the perspective projection of a dynamic 3D scene into a moving camera.

47 3D Motion and Its 2D Projection

47.1 Introduction

As objects move in the world, or as the camera moves, the projection of the dynamic scene into the two-dimensional (2D) camera plane produces a sequence of temporally varying pixel brightness. Before diving into how to estimate motion from pixels, it is useful to understand the image formation process. Studying how three-dimensional (3D) motion projects into the camera will allow us to understand what the difference is between a moving camera or a moving object and what types of constraints one might be able to use to estimate motion.

47.2 3D Motion and Its 2D Projection

A 3D point will follow a trajectory $\mathbf{P}(t) = (X(t), Y(t), Z(t))$, in camera coordinates ([figure 47.1](#)). As the point moves, it has an instantaneous 3D velocity of $\dot{\mathbf{P}} = (\dot{X}(t), \dot{Y}(t), \dot{Z}(t))$. The projection of this point into the image plane location is $\mathbf{p}(t) = (x(t), y(t))$ and its projection will move with the 2D instantaneous velocity $\dot{\mathbf{p}} = (\dot{x}(t), \dot{y}(t))$, where all the derivatives are done with respect to time, t .

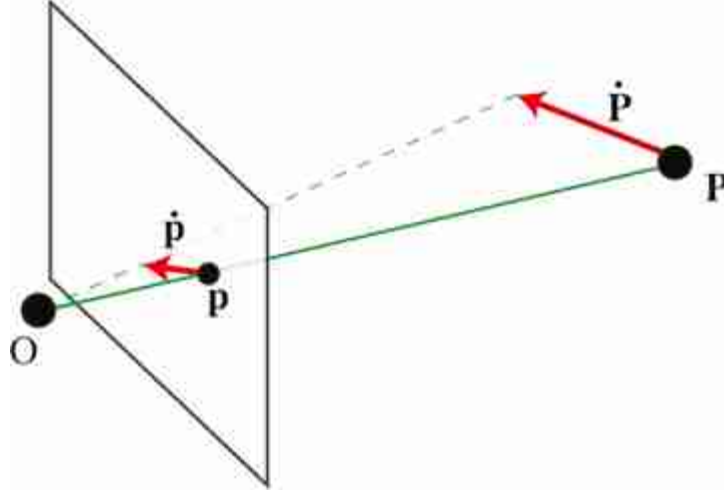


Figure 47.1: A 3D point, P , moving in the world projects a 2D moving point, p , into the camera plane.

Using the equations of perspective projection $x = fX/Z$ and $y = fY/Z$ (assuming that the camera is at the origin of the world-coordinate system), we can derive the equations of how the instantaneous velocity in the camera plane relates to the point motion in 3D world coordinates:

$$\dot{x} = f \frac{\dot{X}Z - \dot{Z}X}{Z^2} = \frac{f\dot{X} - \dot{Z}x}{Z} \quad (47.1)$$

$$\dot{y} = f \frac{\dot{Y}Z - \dot{Z}Y}{Z^2} = \frac{f\dot{Y} - \dot{Z}y}{Z} \quad (47.2)$$

The second expression is obtained by using $x = fX/Z$ and $y = fY/Z$, which removes the dependency on the world coordinates X and Y . Note that f is the focal length. In many derivations, the notation is simplified by setting the focal length $f = 1$. Here we will keep it to make explicit which factors depend on the camera parameters and which ones do not. We can write the last two equations in matrix form as:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} f & 0 & -x \\ 0 & f & -y \end{bmatrix} \begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix} \quad (47.3)$$

This expression reveals a number of interesting properties of the optical flow and how it relates to motion in the world. For instance, points that move parallel to the camera plane ($\dot{Z} = 0$) will project to a motion parallel to the motion in 3D but with a magnitude that will be inversely proportional to the distance Z :

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{f}{Z} \begin{bmatrix} \dot{X} \\ \dot{Y} \end{bmatrix} \quad (47.4)$$

For points moving parallel to the Z axis ($\dot{X} = \dot{Y} = 0$) we get:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = -\frac{\dot{Z}}{Z} \begin{bmatrix} x \\ y \end{bmatrix} \quad (47.5)$$

Points at the same distance Z from the camera, moving away or towards the camera ($\dot{Z} \neq 0$, and $\dot{X} = \dot{Y} = 0$), with the same velocity will project into points moving at different velocities on the image plane.

[Figure 47.2](#) illustrates the geometry of the projection of 3D motion into the camera for points moving parallel to the camera plane ([figure 47.2](#)[left]) and parallel to the optical axis of the camera ([figure 47.2](#)[right]). The arrows at the image plane show the imaged scene velocities.

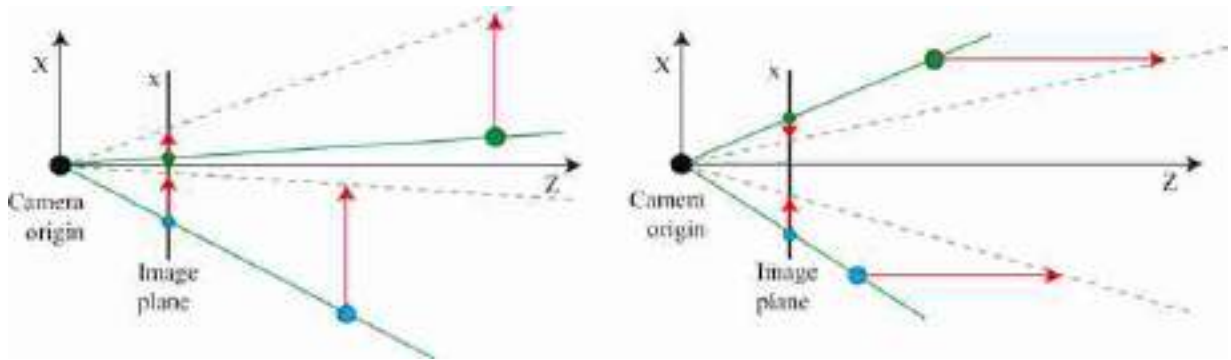


Figure 47.2: (left) Geometry of the projection of 3D motion into the camera for points moving parallel to the camera plane, and (right) parallel to the optical axis of the camera.

Let's examine a few scenarios in a bit more detail to gain some familiarity with the relationship between 3D motion and the projected 2D motion field.

47.2.1 Vanishing Point

Let's consider a point moving in a straight line in 3D with constant velocity over time: $\dot{\mathbf{P}} = (V_X, V_Y, V_Z)^T$. At each time instant, t , the point location will be $\mathbf{P}(t) = (X + V_X t, Y + V_Y t, Z + V_Z t)^T$, and its 2D projection:

$$x(t) = f \frac{X + V_X t}{Z + V_Z t} \quad (47.6)$$

$$y(t) = f \frac{Y + V_Y t}{Z + V_Z t} \quad (47.7)$$

If $\dot{Z} = 0$, then the projected point will move with constant velocity over time, as shown in equation (47.4). If $\dot{Z} \neq 0$, then, as time goes to infinity, the point will converge to a **vanishing point**:

$$\lim_{t \rightarrow \infty} x(t) = f \frac{V_X}{V_Z} = x_{\infty} \quad (47.8)$$

$$\lim_{t \rightarrow \infty} y(t) = f \frac{V_Y}{V_Z} = y_{\infty} \quad (47.9)$$

The following sketch ([figure 47.3](#)) shows Gibson's bird (see figure 1.9) flying away from the camera along a straight line. In the camera the bird gets smaller as it flies away until it disappears at the vanishing point. In this drawing the vanishing point is within the view of the camera.

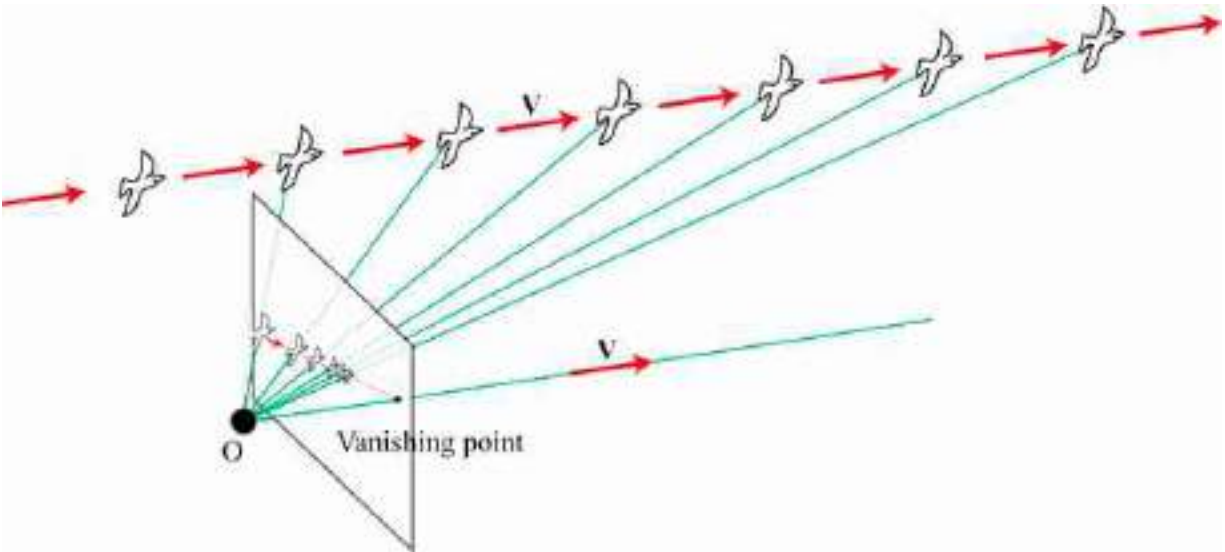


Figure 47.3: Projection onto the camera plane of the sequence produced by a bird flying away. The bird will vanish at the **vanishing point**.

The vanishing point is the location $\mathbf{p}_{\infty} = (x_{\infty}, y_{\infty})^T$ where the moving point slowly converges to. The location of the vanishing point is independent of the point location at time $t = 0$, and it only depends on the 3D velocity vector, $\mathbf{V}$. Therefore, if the scene contains multiple points at

different locations moving with the same velocity, they will converge to the same vanishing point.

47.2.2 Camera Translation

Let's now assume that the scene is static and that only the camera is moving. In this case, all the observed motion in the image will be due to the motion of the camera.

Let's assume the camera is moving in a straight line with a velocity $\dot{\mathbf{T}} = \mathbf{V} = (V_X, V_Y, V_Z)^T$. The translation of the camera after time t will be $\mathbf{T} = \mathbf{V}t$. A point in space $\mathbf{P} = (X, Y, Z)^T$, will move with velocity, relative to the camera, equal to $\dot{\mathbf{P}} = -\mathbf{V}$. The moving camera is equivalent to the case where all the scene points move relative to the camera with the same velocity.

The 2D motion field by using equation (47.3) is:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} -f & 0 & x \\ 0 & -f & y \end{bmatrix} \begin{bmatrix} V_X \\ V_Y \\ V_Z \end{bmatrix} \quad (47.10)$$

We can also express the same relationship by making the contribution of the camera coordinates more explicit. We can do this by rearranging the terms in equation (47.10), resulting in:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = -\frac{f}{Z} \begin{bmatrix} V_X \\ V_Y \end{bmatrix} + \frac{V_Z}{Z} \begin{bmatrix} x \\ y \end{bmatrix} \quad (47.11)$$

Equation (47.10) gives a generic expression for the observed motion in the camera plane produced by a moving camera undergoing a translation (we will see later what happens if we also have camera rotation). But let's first look at a few specific scenarios with a camera following simple translation trajectories (and no rotations).

47.2.2.1 Lateral camera motion

Consider a camera translating laterally, as shown in [figure 47.4](#). This will happen if you are looking through a side window of a car at the scene passing by. In this case, the forward velocity is zero, $V_Z = 0$.



Figure 47.4: Lateral camera motion parallel to the camera plane.

Under lateral camera motion, using equation (47.4), we have the following relationship between the velocity of a 3D point and the apparent velocity of its projection in the image plane:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = -\frac{f}{Z} \begin{bmatrix} V_X \\ V_Y \end{bmatrix} \quad (47.12)$$

The motion field depends on the depth at each location Z as illustrated in [figure 47.5](#). Objects close to the camera will appear moving faster than objects farther away. Objects that are very far will appear as not moving. This parallax effect is the same one used in stereo vision to recover depth. The 2D motion in the image plane is in the opposite direction to the camera motion.

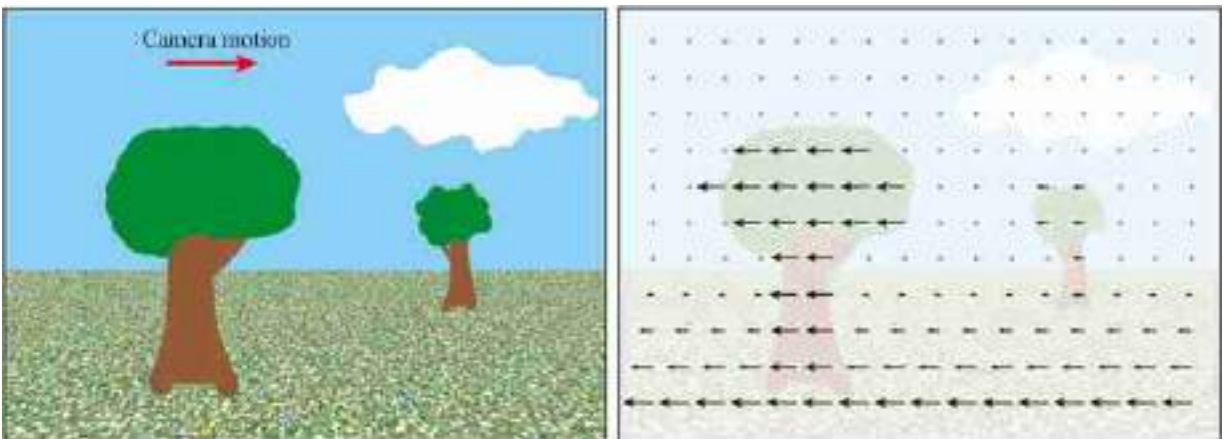


Figure 47.5: Sketch of the motion field under lateral camera motion. Objects close to the camera will appear to be moving faster than objects farther away. Objects that are very far (like the cloud) will appear to be nearly stationary.

47.2.2.2 Camera forward motion and focus of expansion

For a camera moving forward, as illustrated in [figure 47.6](#), note that $V_X = V_Y = 0$. In this case, the motion is only along the Z -axis, $V_Z \neq 0$.

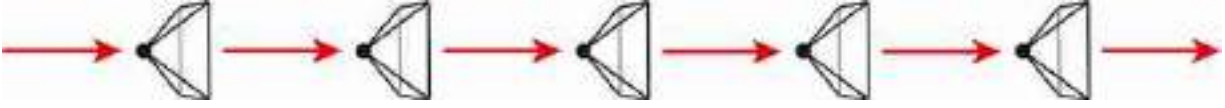


Figure 47.6: Camera moving forward, along the camera axis.

Using equation (47.5), we get:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{V_Z}{Z} \begin{bmatrix} x \\ y \end{bmatrix} \quad (47.13)$$

Equation (47.13) provides a few interesting insights. First, the rate of expansion does not depend on the focal length f . Second, the observed motion only depends on the ratio V_Z/Z , which is the inverse of the **time to contact**. The time to contact, V_Z/Z , is the time it will take the camera to reach the object located a distance Z when moving at velocity V_Z .

For a camera moving in an arbitrary direction, that is, with $V_X \neq 0$, $V_Y \neq 0$, and $V_Z \neq 0$, using equation (47.3) and equation (47.9) we get:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{V_Z}{Z} \begin{bmatrix} x - x_\infty \\ y - y_\infty \end{bmatrix} \quad (47.14)$$

The observed motion is zero at the **focus of expansion**, (x_∞, y_∞) .

[Figure 47.7](#) illustrates the apparent motion field for a camera moving toward the center of a wall. Points near the center (which will be the point of impact) appear stationary, while points in the periphery appear to move faster and away from the center. The whole wall expands over time.

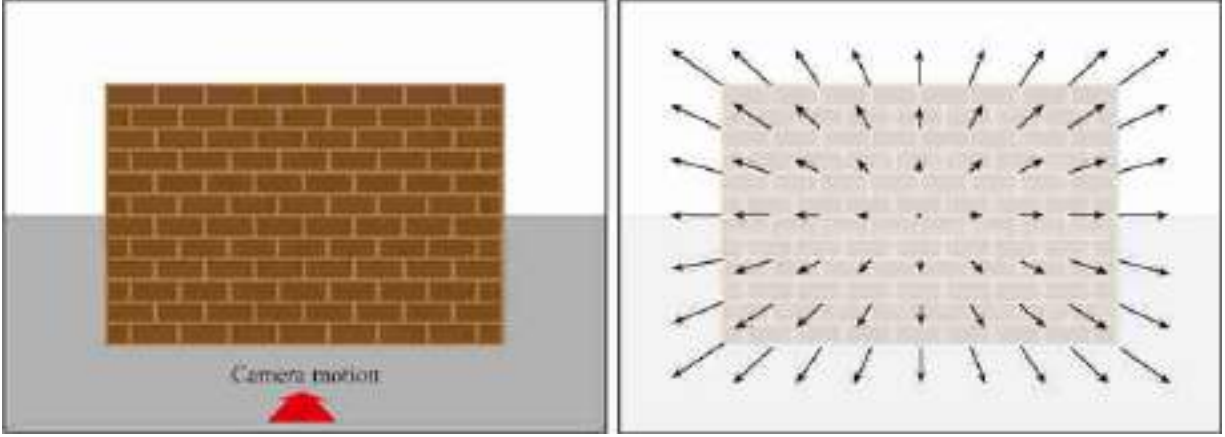


Figure 47.7: Sketch of the motion field when the camera approaches a planar surface. The motion field indicates the rate of expansion of the image, and it is a function of the time to contact. In this example, the focus of expansion is on the center.

47.2.3 Camera Rotation

Let's consider a general camera motion undergoing both translation and rotation. To compute the motion field with a compact expression, we will do a number of simplifications assuming a small motion between consecutive frames. After a small time interval, Δt , the camera will move, generating a displacement in the points with respect to the camera coordinate system equal to:

$$\mathbf{P}_{t+\Delta t} = -\mathbf{T}_{\Delta t} + \mathbf{R}_{\Delta t} \mathbf{P}_t \quad (47.15)$$

where $\mathbf{T}_{\Delta t}$ is the camera translation and $\mathbf{R}_{\Delta t}$ is the camera rotation that took place over that time interval, Δt . The velocity of a 3D point with respect to the camera will be:

$$\dot{\mathbf{P}} = \frac{\mathbf{P}_{t+\Delta t} - \mathbf{P}_t}{\Delta t} = -\mathbf{V} + \frac{\mathbf{R}_{\Delta t} - \mathbf{I}}{\Delta t} \mathbf{P}_t \quad (47.16)$$

To derive the rotation, we consider the **Euler angles** ([figure 47.8](#)) and decompose the rotation using rotations along the three axes (yaw, pitch, roll):

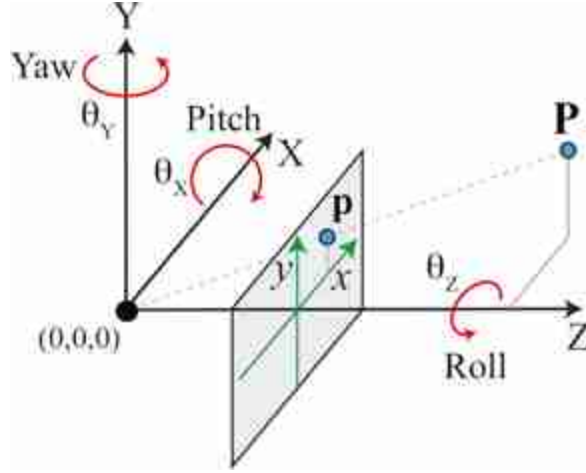


Figure 47.8: Rotation expressed by Euler angles (yaw, pitch, roll).

Each angle measures the rotation along the camera-coordinate axes. Using this representation of the rotation, the rotation matrix can be written as:

$$\mathbf{R}_{\Delta t} = \begin{bmatrix} \cos \theta_Z & \sin \theta_Z & 0 \\ -\sin \theta_Z & \cos \theta_Z & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta_Y & 0 & -\sin \theta_Y \\ 0 & 1 & 0 \\ \sin \theta_Y & 0 & \cos \theta_Y \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_X & \sin \theta_X \\ 0 & -\sin \theta_X & \cos \theta_X \end{bmatrix} \quad (47.17)$$

In this equation, the sign of the angles are chosen to reflect that a rotation of the camera is equivalent to the opposite rotation of the 3D point.

For a small Δt , the angles will be small, and we can approximate the trigonometric functions, $\cos$ and $\sin$, by $\cos \alpha \approx 1$ and $\sin \alpha \approx \alpha$. We can also approximate the product $\sin \alpha \sin \beta \approx 0$ as it will result in a second-order term.

$$\mathbf{R}_{\Delta t} \approx \begin{bmatrix} 1 & \theta_Z & 0 \\ -\theta_Z & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -\theta_Y \\ 0 & 1 & 0 \\ \theta_Y & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & \theta_X \\ 0 & -\theta_X & 1 \end{bmatrix} \approx \begin{bmatrix} 1 & \theta_Z & -\theta_Y \\ -\theta_Z & 1 & \theta_X \\ \theta_Y & -\theta_X & 1 \end{bmatrix} \quad (47.18)$$

$$\mathbf{P}_{t+\Delta t} - \mathbf{P}_t = -\mathbf{T}_{\Delta t} + (\mathbf{R}_{\Delta t} - \mathbf{I})\mathbf{P}_t = -\mathbf{T}_{\Delta t} - \begin{bmatrix} 0 & -\theta_Z & \theta_Y \\ \theta_Z & 0 & -\theta_X \\ -\theta_Y & \theta_X & 0 \end{bmatrix} \mathbf{P}_t \quad (47.19)$$

The last term corresponds to the cross product in matrix form (note that we changed the sign of the matrix to make the cross product form more obvious). Therefore, we can rewrite the previous expression as:

$$\mathbf{P}_{t+\Delta t} - \mathbf{P}_t = -\mathbf{T}_{\Delta t} - \boldsymbol{\theta} \times \mathbf{P}_t \quad (47.20)$$

where $\boldsymbol{\theta} = (\theta_X, \theta_Y, \theta_Z)$. Substituting this expression into equation (47.16), we get the expression of the motion of a 3D point:

$$\dot{\mathbf{P}} = -\mathbf{V} - \mathbf{W} \times \mathbf{P}_t = - \begin{bmatrix} V_X \\ V_Y \\ V_Z \end{bmatrix} - \begin{bmatrix} -W_Z Y + W_Y Z \\ W_Z X - W_X Z \\ -W_Y X + W_X Y \end{bmatrix} \quad (47.21)$$

where $\mathbf{W}$ is the angular velocity $\mathbf{W} = (W_X, W_Y, W_Z)$. Now we are ready to compute the 2D motion field by using equation (47.3):

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = -\frac{1}{Z} \begin{bmatrix} f & 0 & -x \\ 0 & f & -y \end{bmatrix} \begin{bmatrix} V_X \\ V_Y \\ V_Z \end{bmatrix} - \frac{1}{Z} \begin{bmatrix} f & 0 & -x \\ 0 & f & -y \end{bmatrix} \begin{bmatrix} -W_Z Y + W_Y Z \\ W_Z X - W_X Z \\ -W_Y X + W_X Y \end{bmatrix} \quad (47.22)$$

This expression can be rewritten as, using $x = fX/Z$ and $y = fY/Z$:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} -f & 0 & x \\ 0 & -f & y \end{bmatrix} \begin{bmatrix} V_X \\ V_Y \\ V_Z \end{bmatrix} + \frac{1}{f} \begin{bmatrix} xy & -f^2 - x^2 & fy \\ f^2 + y^2 & -xy & -fx \end{bmatrix} \begin{bmatrix} W_X \\ W_Y \\ W_Z \end{bmatrix} \quad (47.23)$$

This is the expression we were looking for. It relates the 2D motion field with the camera velocity and rotation. The matrices are only a function of the intrinsic camera parameters (focal length, f) and the camera coordinates. Note that this expression is only valid for small displacements.

In the previous section we saw what happens when there is no rotation; now we can focus on the case when there is only camera rotation, that is $V_X = V_Y = V_Z = 0$,

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{1}{f} \begin{bmatrix} xy & -f^2 - x^2 & fy \\ f^2 + y^2 & -xy & -fx \end{bmatrix} \begin{bmatrix} W_X \\ W_Y \\ W_Z \end{bmatrix} \quad (47.24)$$

The first thing to notice is that the 2D motion field does not depend on the 3D scene structure, Z , and it is only a function of the rotational velocity and the camera parameters. Therefore, under camera rotation we can not learn anything about the scene by observing the motion field. The only thing we can learn from the 2D motion field is about the motion of the camera.

Let's consider first rotation along the camera optical axis, that is $W_X = W_Y = 0$. In this case the motion field is:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \begin{bmatrix} y \\ -x \end{bmatrix} W_Z \quad (47.25)$$

The 2D motion field at each location (x, y) will point in the orthogonal direction to the vector that connects that point with the origin ([figure 47.9](#)). The motion field does not depend on the focal length, f .

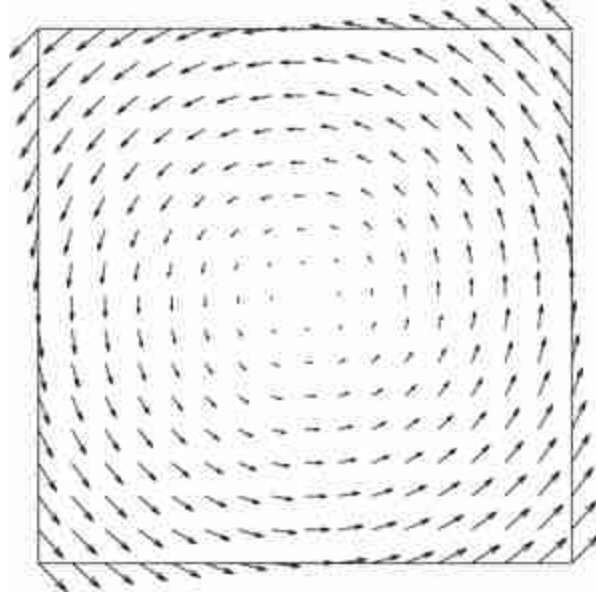


Figure 47.9: Motion field for a rotating camera around the optical axis, $W_X = W_Y = 0$.

If $W_Z = 0$, then the rotation along W_X or W_Y will produce similar 2D motion fields, so let's consider $W_X = 0$. We have:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = -\frac{1}{f} \begin{bmatrix} f^2 + x^2 \\ xy \end{bmatrix} W_Y \quad (47.26)$$

In this case, the focal length f will have a strong effect on the appearance of the motion field. For very large f , we can approximate the 2D motion field by $\dot{x} \approx fW_Y$ and $\dot{y} \approx 0$. The resulting motion field is approximately constant across the entire image and looks like lateral translation motion. For very small f , the motion field will be similar to the one produced by a homography and it will be very different to a lateral camera motion. The flows shown in [figure 47.10](#) correspond to $f = 1/3$, $f = 1$, and $f = 3$.

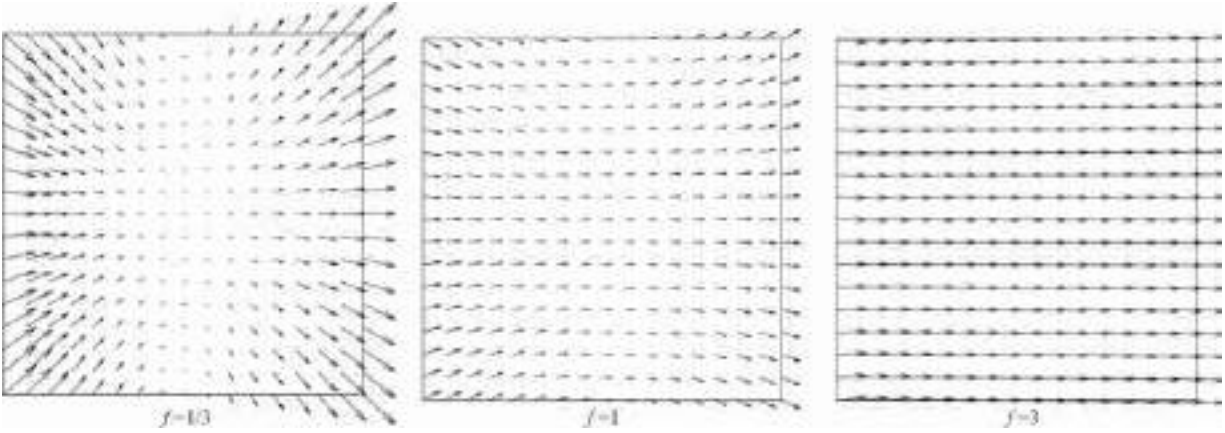


Figure 47.10: Motion flows corresponding to a rotation around the Y -axis. (left) $f = 1/3$. (middle) $f = 1$. (right) $f = 3$.

Camera rotation around the Y -axis does not inform about the scene structure, but it informs about the camera parameters. The magnitude of W_Y only affects the scaling of the motion vectors, but it does not change their orientation.

In the case of camera rotation, for a large angle (or a large Δt), the relationship between the image at time t and the image at time $t + \Delta t$ is a homography.

47.2.4 Motion under Varying Focal Length

Before moving into motion estimation, let's consider one final scenario: a static camera observing a static scene, but the focal length changes over time. What will the motion field be? In this setting, despite that there is not motion in the scene, the focal length of the camera changes over time, producing motion in the image plane. If the focal length increases, it will seem as if we are zooming into the scene. Would it be similar to a forward motion?

Starting from the perspective projection equation,

$$\begin{bmatrix} x \\ y \end{bmatrix} = \frac{f}{Z} \begin{bmatrix} X \\ Y \end{bmatrix} \quad (47.27)$$

If we compute the temporal derivative where only f varies over time, we get:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \frac{\dot{f}}{Z} \begin{bmatrix} X \\ Y \end{bmatrix} = \frac{\dot{f}}{f} \begin{bmatrix} x \\ y \end{bmatrix} \quad (47.28)$$

The last expression is obtained by using equation (47.27). As the equation shows, changing the focal length only results in a scaling of the projected image on the image plane. It does not create any parallax. As the sensor has finite size, changing the focal length results in a zoom and a crop. The motion field does not depend on the 3D scene structure. Therefore, images taken by a pinhole camera from the same viewpoint but with different focal lengths do not provide depth information about the scene. This is an example where there is a 2D motion field even when there is no motion in the scene.

47.3 Concluding Remarks

As we did in chapter 5, in this chapter we have focused on formulating the problem of image formation: How does the 3D motion in the world appear once it is projected on the image plane?

But the goal of vision is to inverse this projection and recover the 3D scene structure. In the upcoming chapters, we will proceed on the path begun in chapter 46, and we will study how to estimate motion from pixels.

48 Optical Flow Estimation

48.1 Introduction

Now that we have seen how a moving three-dimensional (3D) scene (or camera) produces a two-dimensional (2D) motion field on the image, let's see how can we measure the resulting 2D motion field using the recorded images by the camera. We want to measure the 2D displacement of every pixel in a sequence.

Unfortunately, we do not have a direct observation of the 2D motion field either, and not all the displacements in image intensities correspond to 3D motion. In some cases, we can have scene motion without producing changes in the image, such as when the camera moves in front of a completely white wall; in other cases, we will see motion in the image even when there is not motion in the scene such as when the illumination source moves.

In the previous chapter we discussed a matching-based algorithm for motion estimation but it is slow and assumes that the motion is only on discrete pixel locations. In this chapter we will discuss gradient-based approaches that allow for estimating continuous displacement values. These methods introduce many of the concepts later used by learning-based approaches that employ deep learning.

48.2 2D Motion Field and Optical Flow

Before we discuss how to estimate motion, let's introduce a new concept: optical flow.

James J. Gibson, presented in chapter 1, introduced the concept of optical flow in 1947 [\[159\]](#).

Optical flow is an approximation to the 2D motion field computed by measuring displacement of image brightness ([figure 48.1](#)). The ideal optical flow is defined as follows: given two images ℓ_1 and $\ell_2 \in \mathbb{R}^{N \times M \times 3}$, the

optical flow $[\mathbf{u}, \mathbf{v}] \in \mathbb{R}^{N \times M \times 2}$ indicates the relative position of each pixel in ℓ_1 and the corresponding pixel in ℓ_2 . Note that optical flow will change if we reverse time. This definition assumes that there is a one-to-one mapping between two frames. This will not be true if an object appears in one frame, or when it disappears behind occlusions. The definition also assumes that one motion explains all pixel brightness changes. That assumption can be violated for many reasons, including, for example, if transparent objects move in different directions, or if an illumination source moves.

[Figure 48.1](#) shows two frames and the optical flow between them. This visualization using a color code was introduced in [\[29\]](#). In this chapter we will use arrows instead as it provides a more direct visualization and it is sufficient for the examples we will work with.



Figure 48.1: Two frames of a sequence, ground-truth optical flow (color coded), and the color code to read the vector at each pixel.

48.2.1 When Optical Flow and Motion Flow Are Not the Same

There are a number of scenarios where motion in the image brightness does not correspond to the motion of the 3D points in the scene. Here there are some examples where it is unclear how motion should be defined:

- Rotating Lambertian sphere with a static illumination will produce no changes in the image. If the sphere is static, and the light source moves, we will see motion in spite of the sphere being static.
- Moving in front of a textureless wall produces no change on the image.
- Waves in water: waves appear to move along the surface but the actual motion of the water is up and down (well, it is even more complicated than that).

- A rotating mirror will produce the appearance of a faster motion. And this will happen in general with any surface that has some specular component.
- A camera moving in front of a specular planar surface will not produce a motion field corresponding to a homography.

Motion estimation should not just measure pixel motion, it should also try to assign to each source of variation in the image the physical cause for that change. The models we will study in this chapter will not attempt to do this.

48.2.2 The Aperture Problem

One classical example of the limitations of motion estimation from images is the **aperture problem**. The aperture problem happens when observing the motion of a one-dimensional (1D) structure larger than the image frame, as shown in [figure 48.2](#). If none of the termination points are in view within the observation window (i.e., the aperture), it is impossible to measure the actual image motion. Only the component orthogonal to the moving structure can be measured.

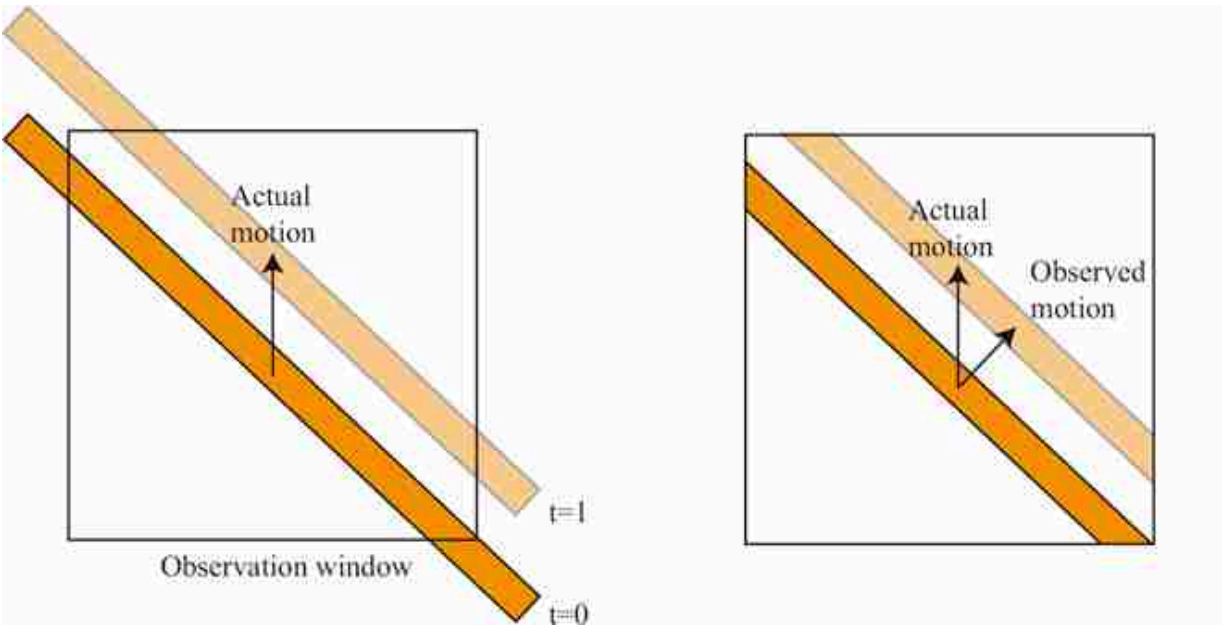
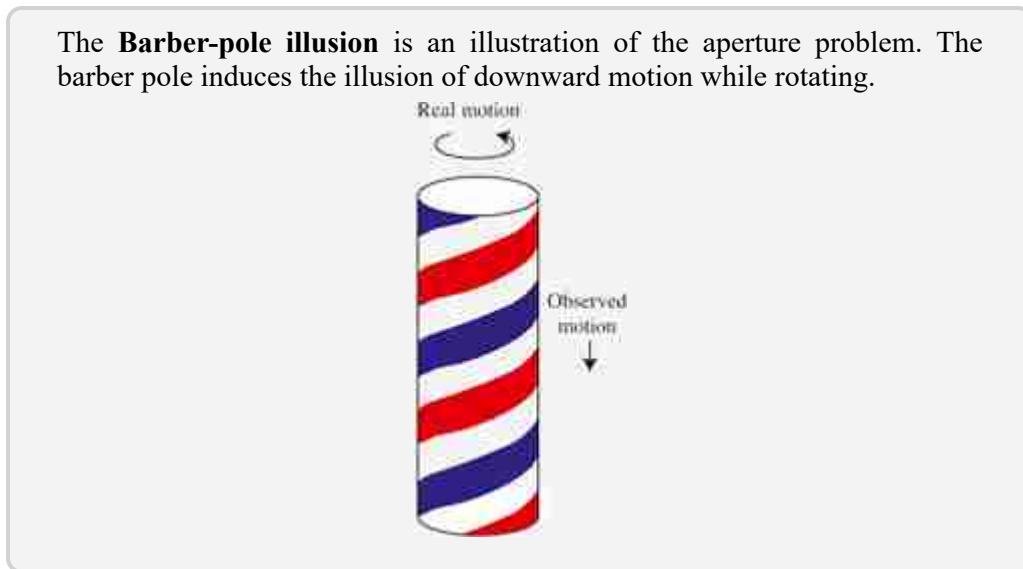


Figure 48.2: Aperture problem when observing the motion of a one-dimensional (1D) structure larger than the image frame. The actual motion of the bar is upward, but the perception, when vision is limited to what is visible within the observation window, appears as if the motion of the bar is in the direction perpendicular to the bar.



48.2.3 Representation of Optical Flow

We can model motion over time as a sequence of **dense optical flow** images:

$$[u(x, y, t), v(x, y, t)] \quad (48.1)$$

The quantities $u(x, y, t)$ and $v(x, y, t)$ indicate that a pixel at image coordinates (x, y) at time t moves with velocity (u, v) . The problem with this formulation is that we do not have an explicit representation of the trajectory followed by points in the scene over time. As time t changes, the location (x, y) might correspond to different scene elements.

An alternative representation is to model motion as a **sparse set of moving points** (tracks):

$$\{\mathbf{P}_t^{(i)}\}_{i=1}^N \quad (48.2)$$

This has the challenge that we have to establish the correspondence of the same scene point over time (tracking). The appearance of a 3D scene point will change over time due to perspective projection and other variations such as illumination changes over time. It might be difficult to do this on a dense array. Therefore, most representations of motion as sets of moving points use a sparse set of points.

Both of those motion representations have limitations, and depending on the applications, one representation may be preferred over the other. Choosing the right representation might be challenging in certain situations. For instance, what representation would be the most appropriate to describe the flow of smoke or water over time? (We do not know the answer to this question, and the answer might depend on what do we want to do with it.)

In the rest of this chapter we will use the dense optical flow representation.

48.3 Model-Based Approaches

Let's now describe a set of approaches for motion estimation that are not based on learning. Motion estimation equations will be derived from first principles and will rely on some simple assumptions.

In section 46.3 we discussed matching-based optical flow estimation. We will discuss now gradient-based methods. These approaches rely on the brightness constancy assumption and use the reconstruction error as the loss to optimize.

48.3.1 Brightness Constancy Assumption

The **brightness constancy assumption** says that as a scene element moves in the world, the brightness captured by the camera from that element does not change over time. Mathematically, this assumption translates into the following relationship, given a sequence $\ell(x, y, t)$:

$$\ell(x+u, y+v, t+1) = \ell(x, y, t) \quad (48.3)$$

where u and v are the element displacement over one unit of time and are also a function of pixel location, $u(x, y)$ and $v(x, y)$, but we will drop that dependency to simplify the notation. The previous relationship is equivalent to saying that $\ell(x, y, t+1) = \ell(x-u, y-v, t)$. To make sure that the reader remains onboard without getting confused by the indices and how the translation works, here is a simple toy example of a translation with $(u = 1, v = 0)$:

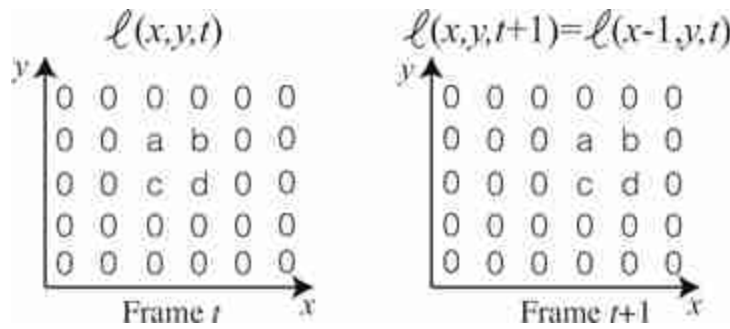


Figure 48.3: Translation to the right of a simple 6×6 size image.

The constant brightness assumption only approximately holds in reality. For it to be exact, we should have a scene with Lambertian objects illuminated from a light source at infinity, with no occlusions, no shadows, and no interreflections. Few real scenes check any of those boxes. This equation assumes that all the pixels in $\ell(x, y, t)$ are visible in $\ell(x, y, t+1)$, but in reality, some pixels might be occluded in the first frame and new pixels might appear around the image boundaries and behind occlusions.

48.3.2 Gradient-Based Optical Flow Estimation

The most popular version of a gradient-based method for optical flow estimation was introduced by Lucas and Kanade in 1981 [311].

Let's start describing the method in words, and we will see next how it translates into math. We will start by approximating the change between two consecutive frames by a linear equation using a Taylor approximation. This linear approximation combined with the constant brightness assumption will result in a linear constraint for the optical flow at each pixel. We will then derive a big system of linear equations for the entire image that, when solved, will result in an estimated optical flow for each image pixel. Let's now see, step by step, how this algorithm works.

If the motion (u, v) is small in comparison to how fast the image ℓ changes spatially, we can use a first-order Taylor expansion of the image $\ell(x + u, y + v, t + 1)$ around (x, y, t) :

$$\ell(x + u, y + v, t + 1) \simeq \ell(x, y, t) + u\ell_x + v\ell_y + \ell_t + \text{h. o. t.} \quad (48.4)$$

Combining equations (48.3) and (48.4), and ignoring higher order terms, we arrive at the **gradient constraint equation**:

$$u\ell_x + v\ell_y + \ell_t = 0 \quad (48.5)$$

This equation constrains the motion (u, v) in location (x, y) to be along a line perpendicular to the image gradient $\nabla\ell = \ell_x, \ell_y$ at that location. This is the same relationship that we discussed before when describing the aperture problem. This is not enough to estimate motion and we will need to add additional constraints. The second assumption that we will add is that the motion field is constant (or smoothly varying) over an extended image region. By solving for the gradient constraints over an image patch, we hope there will be only a unique velocity that will satisfy all the equations. We can implement this constraint in a different way. One simple way of implementing this constraint is by summing over a neighborhood using a weighting function $g(x, y)$. If $g(x, y)$ is a Gaussian window centered in the origin, the optical flow, (u, v) , at the location (x, y) can be estimated by minimizing:

$$\mathcal{L}(u, v) = \sum_{x', y'} g(x' - x, y' - y) \left| u(x, y)\ell_x(x', y', t) + v(x, y)\ell_y(x', y', t) + \ell_t(x', y', t) \right|^2 \quad (48.6)$$

The previous equation is a bit cumbersome because we wanted to make explicit the spatial variables, (x', y') , over which the sum is being made and the factors that are function of the location (x, y) at which the flow is computed. From now, to simplify the derivation, we will drop all the arguments and write the loss in a single location as:

$$\mathcal{L}(u, v) = \sum g |u\ell_x + v\ell_y + \ell_t|^2 \quad (48.7)$$

where only u and v are constant.

The solution that minimizes this loss can be obtained by computing where the derivatives of the loss with respect to u and v are equal to zero:

$$\frac{\partial \mathcal{L}(u, v)}{\partial u} = \sum g (u\ell_x^2 + v\ell_x\ell_y + \ell_x\ell_t) = 0 \quad (48.8)$$

$$\frac{\partial \mathcal{L}(u, v)}{\partial v} = \sum g (u\ell_x\ell_y + v\ell_y^2 + \ell_y\ell_t) = 0 \quad (48.9)$$

We can write the previous two equations in matrix form at each location (x', y') as:

$$\begin{bmatrix} \sum g\ell_x^2 & \sum g\ell_x\ell_y \\ \sum g\ell_x\ell_y & \sum g\ell_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum g\ell_x\ell_t \\ \sum g\ell_y\ell_t \end{bmatrix} \quad (48.10)$$

We will have an equivalent set of equations for each image location. The solution at each location can be computed analytically as $\mathbf{u} = \mathbf{A}^{-1}\mathbf{b}$ where $\mathbf{A}$ is the 2×2 matrix of equation (48.10). The motion at location x, y will only be uniquely defined if we can compute the inverse of $\mathbf{A}$. Note that the matrix $\mathbf{A}$ is a function of the image structure around location (x, y) . If the rank of the matrix is 1, we can not compute the inverse and the optical flow will be constrained along a 1D line, this is the aperture problem. This will happen if the image structure is 1D inside the region of analysis that will be defined by the size of the Gaussian window $g(x, y)$.

In order to implement this approach we can make use of convolutions in order to compute all the quantities efficiently in a compact algorithm 48.1:

Algorithm 48.1: Gradient-based optical flow estimation using two input frames.

```

1 Input:  $\ell_1, \ell_2 \in \mathbb{R}^{N \times M \times 3}$ , Output:  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^{N \times M}$ 
2 Parameters:  $g$  = integration window
3 Compute:  $\ell_x, \ell_y$ , and  $\ell_t$ 
4 Compute A:  $A_{1,1} = \ell_x^2 \circ g, A_{1,2} = \ell_x \ell_y \circ g, A_{2,2} = \ell_y^2 \circ g$ 
5 Compute b:  $b_1 = \ell_x \ell_t \circ g, b_2 = \ell_y \ell_t \circ g$ 
6 for  $i = 0, \dots, M-1$  do
7   for  $j = 0, \dots, N-1$  do
8      $u[i, j], v[i, j] = \text{inv}(\mathbf{A}[i, j])\mathbf{b}[i, j]$ 

```

To see how optical flow is computed from gradients, let's consider a simple sequence with two moving squares as shown in [figure 48.4](#) where the top square moves with a velocity of $(0, 0.5)$ pixels/frame and the bottom one moves at $(-0.5, -0.5)$ pixels/frame (the figure shows frames 0 and 10 to make the motion more visible).

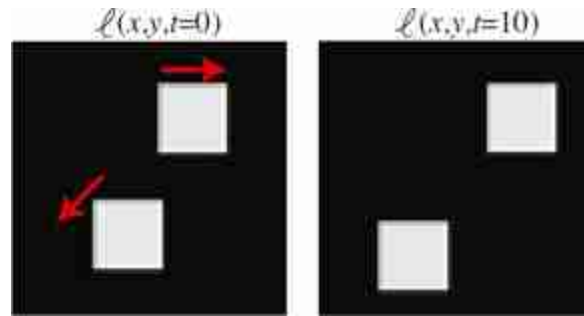
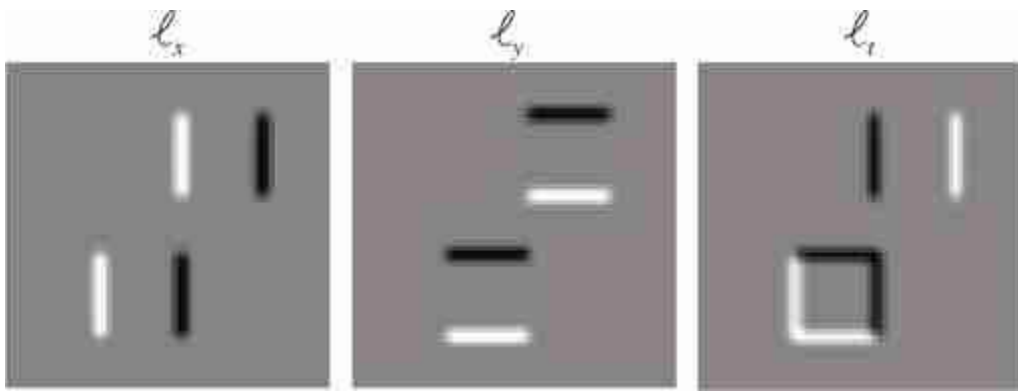


Figure 48.4: Toy sequence with two moving squares. The red arrows indicate the direction of motion of each square.

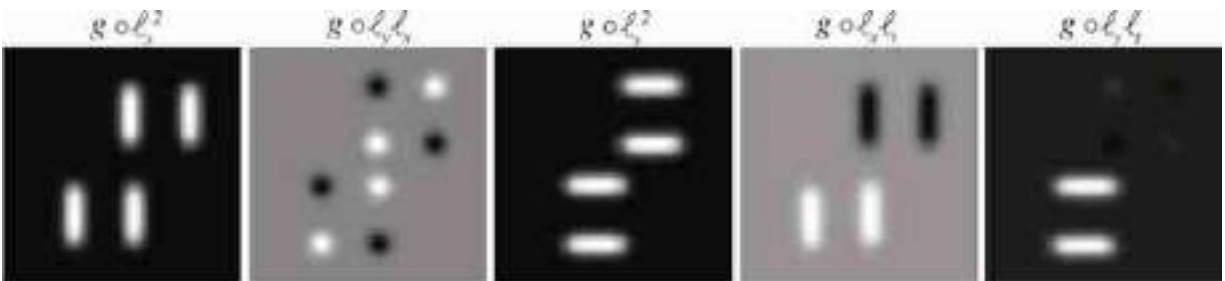
To apply the gradient-based optical flow algorithm we need to compute the gradients along x , y , and t . In practice, the image derivatives, ℓ_x and ℓ_y , are computed by convolving the image with Gaussian derivatives (see chapter 18). In the experiments here we first blur the two input frames with a Gaussian of $\sigma = 1$, approximated with a five-tap kernel (i.e., a kernel with size 5×5 values). For the spatial derivatives we use the kernel $[1, -8, 0, 8,$

$-1]/12$ for the x -derivative and its transposed for the y -derivative. The temporal derivative can be computed as the difference of two consecutive blurred frames. Choosing the appropriate filters to compute the derivatives is critical to get correct motion estimates (the three derivatives need to be centered at the same spatial and temporal location in the $x - y - t$ volume). For the moving squares sequence, the spatial and temporal derivatives are shown in [figure 48.5](#).



[Figure 48.5](#): Spatial and temporal derivatives for the sequence from [figure 48.4](#).

The next step consists of computing ℓ_x^2 , $\ell_x \ell_y$, ℓ_y^2 , $\ell_x \ell_t$, and $\ell_y \ell_t$ and blurring them with the Gaussian kernel, g , which will then be used to build the matrix $\mathbf{A}$ at each pixel. [Figure 48.6](#) shows the results.



[Figure 48.6](#): Computation of all the products between derivatives from [figure 48.5](#).

In order to compute the optical flow, we need to compute the matrix inverse $\mathbf{A}^{-1}$. This matrix depends only on the spatial derivatives and thus is

independent of the motion present in the scene. If we compute optical flow at each pixel, the result looks like the image in [figure 48.7](#).



Figure 48.7: Estimated optical flow for the sequence in [figure 48.5](#).

We can see that the result seems to be wrong for the motion estimated near the center of the side of each square. The inverse can only be computed in image regions with sufficient texture variations (i.e., near the corners). As in the Harris corner detector (see section 40.3.2.1), the eigenvector with the smallest eigenvalue indicates the direction in which the image has the smallest possible change under translations. Regions with a small minimum eigenvalue are regions with a 1D image structure and will suffer from the aperture problem. To identify regions where the motion will be reliable, we can use the following quantity (proposed by Harris), which relates to the conditioning of the matrix $\mathbf{A}$:

$$R = \det(\mathbf{A}) - \lambda \text{Tr}(\mathbf{A})^2 \quad (48.11)$$

where $\lambda = 0.05$ (which is within the range of values used in the Harris corner detector). [Figure 48.8](#) shows R and the estimated optical flow in the regions with $R > 2$. Harris proposed this formulation to avoid the computation of the eigenvalues at each pixel because it is computational expensive.

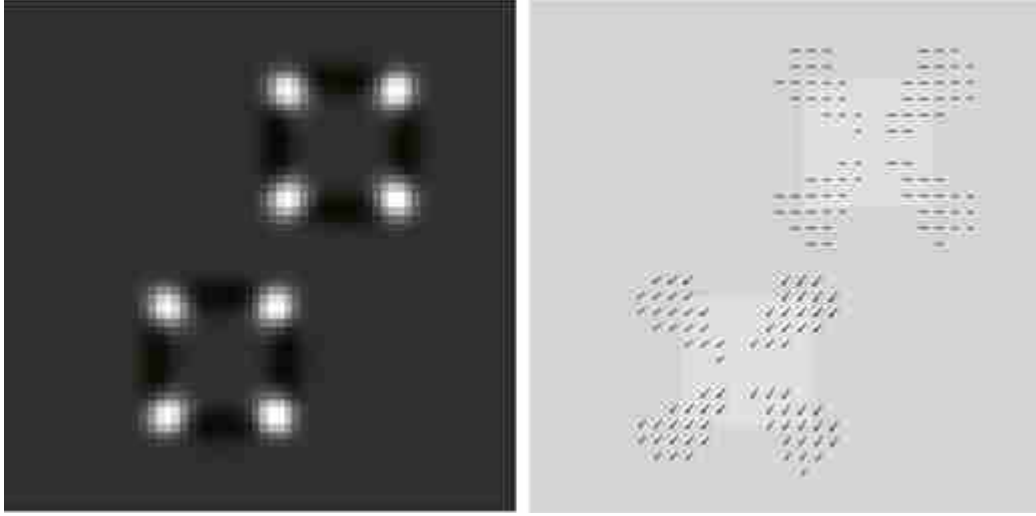


Figure 48.8: Estimated optical flow in the regions with $R > 2$ (around *good features to track* [436]).

The regions for which $R > 2$ will increase by making the apertures larger, which is achieved by using a Gaussian filter, g , with larger σ . However, this will result in smoother estimated flows and the estimated motion will not respect the object boundaries.

One advantage of this approach over the matching-based algorithm is that the estimated flow is not discrete, but it does not work well if the displacement is large, as the Taylor approximation is not valid anymore. One solution to the problem of large displacements is to compute a Gaussian pyramid for the input sequence. The low-resolution scales make the motion smaller and the gradient-based approach will work better.

48.3.3 Iterative Refinement for Optical Flow

The gradient-based approach is efficient but it only provides an approximation to the motion field because it ignores higher order terms in the Taylor expansion in equation (48.4). A different approach consists of directly minimizing the **photometric reconstruction** error:

$$L(\ell_1, \ell_2, \mathbf{u}, \mathbf{v}) = \sum_{x,y} \left| \ell_1(x+u, y+v, t+1) - \ell_2(x, y, t) \right|^2 \quad (48.12)$$

We can now run gradient descent on this loss. Running only one iteration would be similar to the gradient-based optical flow algorithm described in the previous section. However, adding iterations will provide a more accurate estimate of optical flow. At each iteration n , the estimated optical

flow will be used to compute the warped frame $\ell_1(x + u_n, y + v_n, t + 1)$ and we will compute an update, Δu_n and Δv_n , of the optical flow: $u_{n+1} = u_n + \Delta u_n$ and $v_{n+1} = v_n + \Delta v_n$. To improve the results, the optical flow estimation is done on a Gaussian pyramid. First, we run a few iterations on the lowest resolution scale of the pyramid (where the motion will be the smallest). The estimated motion is then upsampled and used as initialization at the next level. We iterate this process until arriving at the highest possible resolution. This process is shown in [figure 48.9](#).

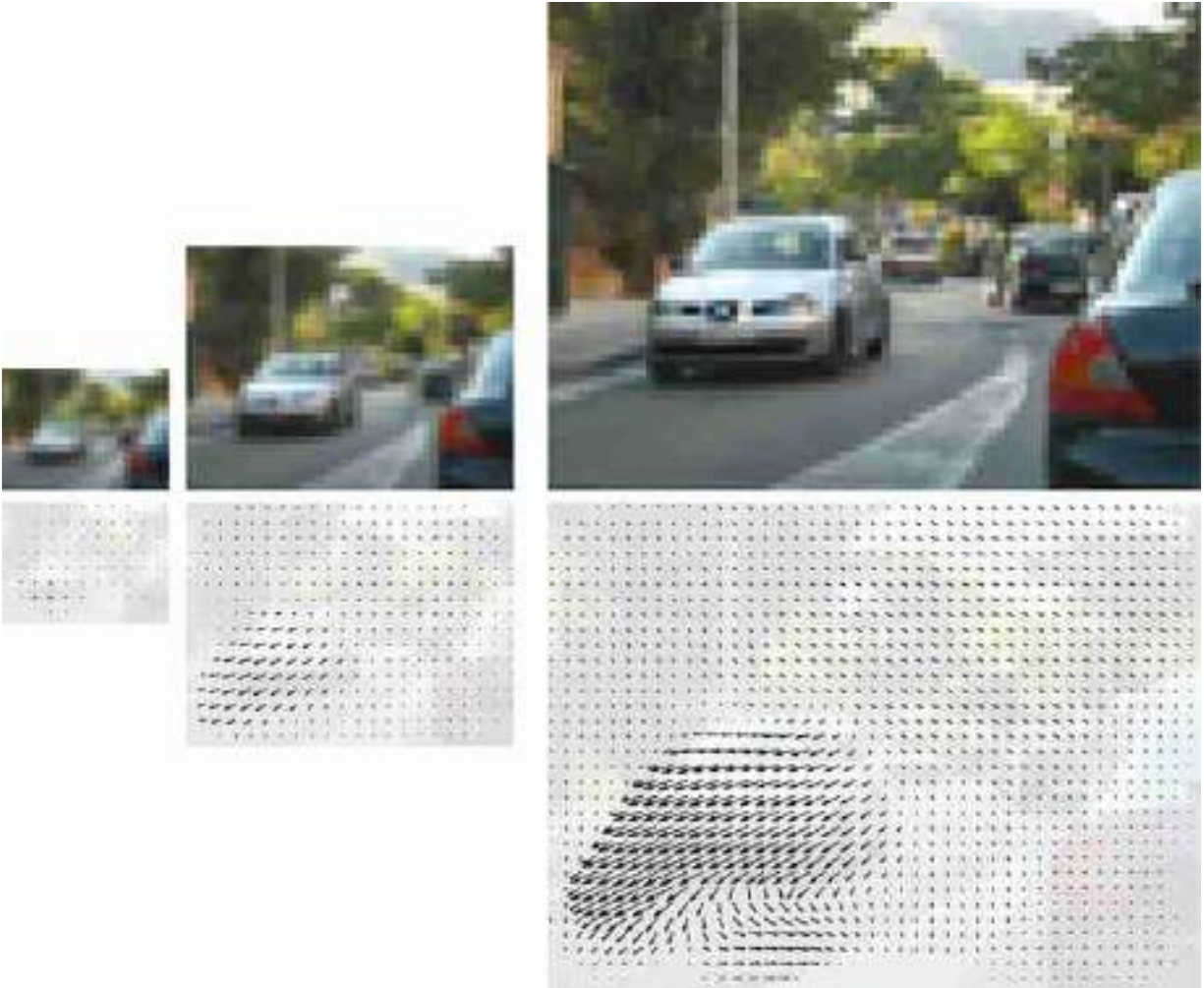


Figure 48.9: Multiscale iterative refinement for optical flow. Optical flow estimation is done on a Gaussian pyramid. (left) First, we run a few iterations on the lowest resolution scale of the pyramid (where the motion will be the smallest). The estimated motion is then upsampled and used as the initialization at the next level. (right) We iterate this process until arriving at the highest possible resolution.

[Figure 48.10](#) compares the optical flow computed using the gradient-based algorithm (i.e., one iteration) and the multiscale iterative refinement approach. Note how the gradient-based approach underestimates the motion of the left car. The displacement between consecutive frames is close to four pixels and that makes the first-order Taylor approximation very poor. The multiscale method is capable of estimating large displacements.

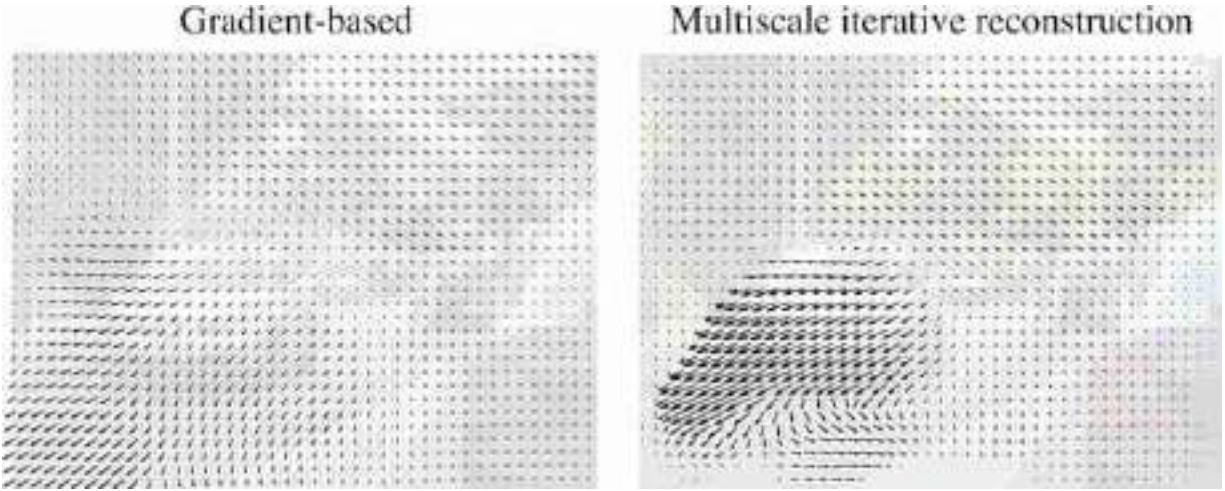


Figure 48.10: Comparison between the optical flow estimated using the gradient-based algorithm and the multiscale iterative refinement approach.

The photometric reconstruction can incorporate a regularization term penalizing fast variations on the estimated optical flow:

$$\mathcal{L}(\mathbf{u}, \mathbf{v}) = L(\ell_1, \ell_2, \mathbf{u}, \mathbf{v}) + \lambda R(\mathbf{u}, \mathbf{v}) \quad (48.13)$$

This problem formulation was introduced by Horn and Schunck in 1981 [218].

There are several popular regularization terms. One penalizes large velocities (**slow prior**):

$$R(\mathbf{u}, \mathbf{v}) = \sum_{x,y} (u(x, y))^2 + (v(x, y))^2 \quad (48.14)$$

Another penalizes variations on the optical flow (**smooth prior**):

$$R(\mathbf{u}, \mathbf{v}) = \sum_{x,y} \left(\frac{\partial u}{\partial x} \right)^2 + \left(\frac{\partial u}{\partial y} \right)^2 + \left(\frac{\partial v}{\partial x} \right)^2 + \left(\frac{\partial v}{\partial y} \right)^2 \quad (48.15)$$

The photometric loss plays an important role in unsupervised learning-based methods for optical flow estimation as we will discuss later.

48.3.4 Layer-Based Motion Estimation

Until now we have not made use of any of the properties of the motion field derived from the 3D projection of the scene. One way of incorporating some of that knowledge is by making some strong assumptions about the moving scene. If the scene is composed of rigid objects, then we can

assume that the motion field within each object will have the form described by equation (47.23).

In this case, instead of a moving camera we have rigid moving objects (which is equivalent). The 2D motion field can then be represented as a collection of superimposed layers, each layer containing one object and occluding the layers below. Each layer will be described by a different set of motion parameters. The parametric motion field can be incorporated into the gradient-based approach described previously. The motion parameters can then be estimated iteratively using an expectation-maximization (EM) style algorithm. At each step we will have to estimate, for each pixel, which layer it is likely to belong to, and then estimate the motion parameters of each layer. The idea of using layers to represent motion was first introduced by Wang and Adelson in 1994 [[493](#)].

48.4 Concluding Remarks

Motion estimation is an important task in image processing and computer vision. It is used in video denoising and compression. In computer vision it is a key attribute to understand scene dynamics and 3D structure. Despite being studied for a long time, accurate optical flow remains challenging, even when using state-of-the-art deep-learning techniques.

The approaches presented here require no training. In the next chapter, we will study several learning-based methods for motion estimation. The approaches presented in this chapter will become useful when exploring unsupervised learning methods.

49 Learning to Estimate Motion

49.1 Introduction

We have discussed in the previous sections a number of model-based methods for motion estimation. If these models describe the equations of motion based from first principles, why is that we need learning based methods at all? The reason is that the models make a number of assumptions that are not always true. Also, there are other sources of information that can reveal properties about motion that cannot be modeled but that can be learned.

Causes of modeling errors include failure of the brightness constancy assumption; the presence of occlusions, shadows and changes in illumination; new structures appearing due to changes in the resolution as a result of motion, deformable surfaces; and so on. Many of the motion computations involved approximations such as approximating the derivatives with finite size discrete convolutions. There could also be other motion-relevant cues present in the image, such as monocular depth cues that provide information about the three-dimensional (3D) scene structure and the presence of familiar objects for which we can have strong priors about their motion. These could include that buildings do not move, walls are solid and usually featureless, people are deformable, trees leaves have huge number of occlusions, and so on. Those semantic properties can be implicitly exploited by a learning-based model.

49.2 Learning-Based Approaches

Learning-based approaches rely on many of the concepts we introduced in the previous chapters. We will differentiate between two big families of models: supervised models that learn to estimate motion using a database of training examples, and unsupervised models that learn to estimate motion without training data.

49.2.1 Supervised Models for Optical Flow Estimation

The simplest formulation for learning to estimate optical flow is when we have available a dataset of image sequences with associated ground truth optical flow. Researchers have used synthetic data [68], using lidar [157] or human annotations [302] to build datasets with ground truth motion. In previous approaches, ground truth data could be used for evaluation; however, we will use it here to train a model to predict motion directly from the input frames.

49.2.1.1 Architectures

As in the case of stereo, we can train a function to estimate optical flow from two frames:

$$[\hat{\mathbf{u}}, \hat{\mathbf{v}}] = h_{\theta}(\ell_1, \ell_2) \quad (49.1)$$

One of the first approaches to use this formulation with neural networks was FlowNet [108]. The architecture is simple.

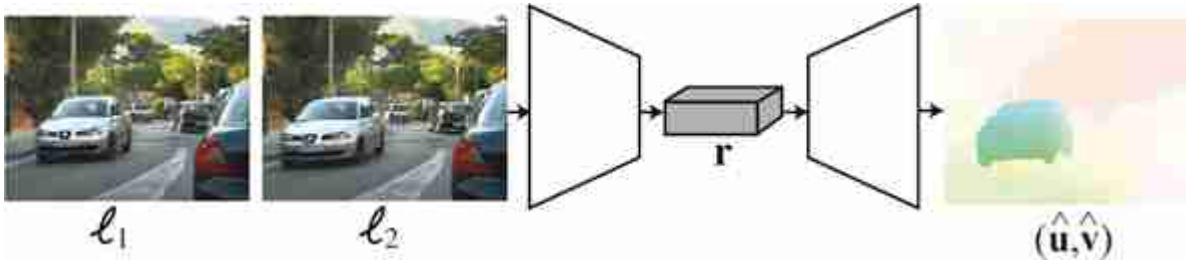


Figure 49.1: In FlowNet the direct approach estimates optical flow directly from a pair of frames.

The direct approach depicted in [figure 49.1](#) learns to estimate optical flow directly from a pair of frames. This architecture makes no assumptions about which architectural priors are needed to compute optical flow from images. The architecture is trained end-to-end using ground truth optical flow.

Another common approach, depicted in the block diagram shown in [figure 49.2](#), is to define an architecture that follows the same steps as traditional approaches:

- Extract features from each image using a pair of networks with shared weights. This can be done by a feature pyramid [297].

- Form a 3D cost volume indicating the local visual evidence of a match between the two images for each possible pixel position. This 3D cost volume can be referenced to the H and V positions of one of the input images (generally the first frame is the reference frame).
- Train and apply a CNN to aggregate (process) the costs over the cost volume in order to estimate a single best optical flow for each pixel position.
- Use a coarse-to-fine estimation procedure where optical flow estimated at a coarse scale is used to warp the features at a finer scale to compute a refined cost volume. Then, estimate an update to the optical flow to warp the features and the next finer level of the pyramid.

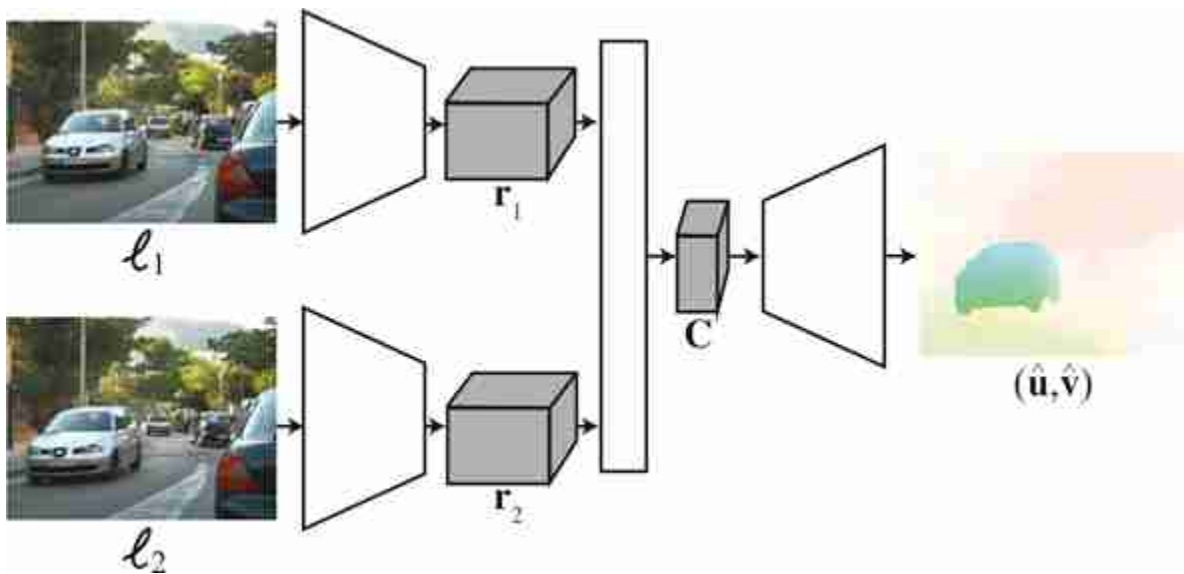


Figure 49.2: Motion estimation. (1) Extract features from each image; (2) compute a 3D cost volume; and (3) aggregate the cost volume in order to estimate the best optical flow for each pixel.

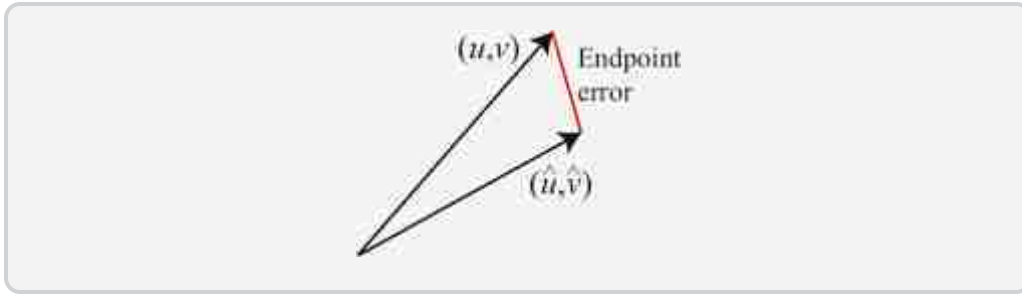
Other variations over this architecture incorporate some of the concepts we studied before, such as coarse-to-fine refinement, matching, and smoothing. Different approaches will differ in some of the details of how each step is implemented and how training takes place. The main building blocks can be implemented with convolutional neural networks or transformers. The main difference between the matching-based and gradient-based methods described earlier is that instead of using predefined

functions, the architectures are trained end-to-end to minimize the optical flow error when compared with ground truth data.

49.2.1.2 Loss functions

In supervised optical flow estimation, the most common loss is the **endpoint error**, which is the average, over the whole image, of the distance between the estimated optical flow vector, $(\hat{u}, \hat{v})$ and the ground-truth vector, (u, v) :

$$\mathcal{L}(\hat{u}, \hat{v}, u, v) = \sum_{n,m} (\hat{u}[n, m] - u[n, m])^2 + (\hat{v}[n, m] - v[n, m])^2 \quad (49.2)$$



The sum is over all the pixels in the image. Each pixel has an estimated optical flow $(\hat{u}[n, m], \hat{v}[n, m])$.

49.2.1.3 Handling occlusions

One of the challenges for estimating optical flow is that, as objects move, they will occlude some pixels from the background and reveal new ones. Therefore, together with the estimated flow it is also convenient to detect occluded pixels. If we have ground truth data, we can train a function to estimate optical flow and the occlusion map from two frames:

$$[\hat{u}, \hat{v}, \hat{o}] = h_{\theta}(\ell_1, \ell_2) \quad (49.3)$$

49.2.1.4 Training set

The biggest challenge of using a supervised model for motion estimation is that ground truth data is very hard to collect. This is probably one of the main limitations of these approaches. There are some small existing datasets, although this might change in a few years.

The largest existing datasets are synthetic 3D scenes with moving objects that can be rendered, which will give us perfect ground truth data to train the regression function. There are several examples of existing datasets such as this, like the Middelbury dataset [425], which contains six real and synthetic sequences with ground truth optical flow. The optical

flow for the real sequences was obtained by tracking hidden fluorescent texture. The KITTI dataset [157] contains real motion recorded from a moving car. The MPI Sintel [68] contains synthetic sequences made with great effort to make the scenes look realistic. Finally, the Flying Chairs dataset is an interesting synthetic dataset that consists of pasting the image of a random number of chairs over a background image [108]. Motion is created by applying different affine transformations to the background and the chairs. These sequences are easy to generate and pay little attention to their realism. This makes it possible to generate a very large number of sequences for training, allowing for competitive performance when used to train a neural network.

49.2.2 Unsupervised Learning of Optical Flow

Collecting ground truth data is the Achilles heel for learning-based approaches. This is particularly true for optical flow as it can not be recorded directly. Ground truth data optical flow can be obtained on synthetic data only, and for real data one needs to create specific recording scenarios that allow inferring accurate optical flow or relying on noisy human annotations [302]. As a consequence, real data collection is expensive and nonscalable.

Is it possible to learn to estimate optical flow by just looking at movies without using ground truth data?

Unsupervised methods for training an optical flow model will make some assumptions about dynamic image formation. Those assumptions will be similar to the ones we have presented all along this chapter: (1) when the motion is due only to camera motion, the optical flow will have to fit the equations of the projected motion that provide constraints that can be used to train a model; (2) we can assume that the appearances of objects and surfaces in the scene do not change due to motion (brightness constancy assumption); and (3) we can expect the optical flow to be smooth over regions, although with sharp variations along occlusion boundaries.

One typical formulation consists in learning to predict the displacement from frame 1 to frame 2, so that if we warp frame 1 we minimize the reconstruction error of frame 2. This is achieved by using the photometric loss:

$$L_{photo}(\ell_1, \ell_2, \mathbf{u}, \mathbf{v}) = \sum_{x,y} |\ell_2(x + \hat{u}, y + \hat{v}) - \ell_1(x, y)|^2 \quad (49.4)$$

where now $[\hat{\mathbf{u}}, \hat{\mathbf{v}}] = h_{\theta}(\ell_1, \ell_2)$.

The learning is done by searching over the parameter space for the parameters θ that minimize the photometric loss over a large collection of videos. The photometric loss can also be replaced by the L1 or other robust norms. If the network also predicts occlusions, the photometric loss can include a weight that cancels the contribution of occluded pixels to the loss.

The network can also take as input multiple frames and not just two.

49.3 Concluding Remarks

Supervised and unsupervised learning-based methods are now the state-of-the-art in motion estimation. But an accurate solution is still missing. One important question is, do we really need learning in order to solve this problem? Should we abandon the derivation of physically motivated algorithms for motion estimation that require no training? Our answer is that we should pursue both directions of work.

XIII

UNDERSTANDING VISION WITH LANGUAGE

Visual understanding consists of inferring scene properties from images. Some properties might refer to mid-level scene attributes such as motion or depth, while others may relate to high-level features like semantic segmentation.

In this part we will focus on the semantics of visual processing; that is, the association between visual stimuli and meaning. The goal is to infer from visual input, what is the signification of what we see. Therefore, there is a strong connection between semantic visual processing and natural language processing.

- **Chapter 50** describes how learn to recognize and localize objects in images, assigning words to them.
- **Chapter 51** explores the role of language as a representation of the visual world and its connection with vision systems.

50 Object Recognition

50.1 Introduction

There are many tasks that researchers use to address the problem of recognition. Although the ultimate goal is to tell what something it is by looking at it, the way that you will answer the question will change the approach that you will use. For instance, you could say, “That is a chair” and just point at it, or we could ask you to indicate all of the visible parts of the chair, which might be a lot harder if the chair is partially occluded by other chairs producing a sea of legs. It can be hard to precisely delineate the chair if we cannot tell which legs belong to it. In other cases, identifying what a chair is might be difficult, requiring understanding of its context and social conventions ([figure 50.1](#)).



Figure 50.1: What is a chair? The definition is based on affordances (i.e., something you can sit on) and on social conventions (i.e., some surfaces you can sit on are meant to be used to support other objects than yourself). Affordances are likely to be directly accessible through vision, while social conventions might not be.

In this chapter we will study three tasks related to the recognition of objects in images (classification, localization, and segmentation). We will introduce the problem definitions and the formulation that lays the foundations for most existing approaches.

50.2 A Few Notes About Object Recognition in Humans

Object perception is very rich and diverse area of interdisciplinary research. Many of the approaches in computer vision are rooted on hypothesis formulated in trying to model the mechanism used by the human visual system. It is useful to be familiar with the literature in cognitive science and neuroscience to understand the origin of existing approaches and to gain insights on how to build new models. Here we will review only a few findings, and we leave the reader with the task of going deeper into the field.

50.2.1 Recognition by Components

One important theory of how humans represent and recognize objects from images is the recognition-by-components theory proposed by Irving Biederman in 1987 [48]. He started motivating his theory with the experiment illustrated in [figure 50.2](#). Can you recognize the object depicted in the drawing? Does it look familiar? The first observation one can make when looking at the drawing in [figure 50.2](#) is that it does not seem to be any object we know, instead it seems to be a made-up object that does not correspond to anything in the world. The second observation is that the object seems to be decomposable into parts and that different observers will probably make the same decomposition. The figure can be separated into parts by breaking the object in “regions of sharp concavity”. And the third observation is that the object resembles other known objects. Does it look like an ice cream or hot-dog cart to you?

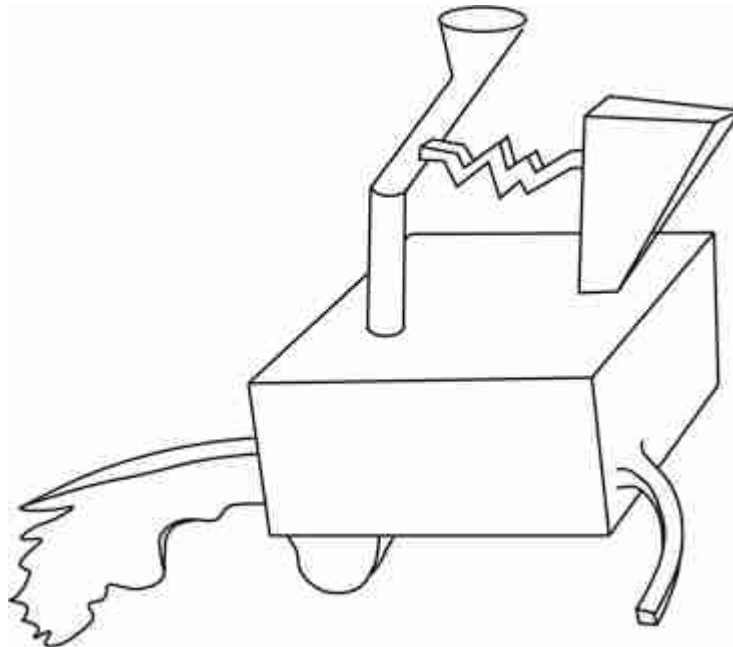


Figure 50.2: What is this? This drawing from Biederman [48], shows a novel object. After looking at this drawing we can conclude the following: (1) we have never seen this object before, (2) it can be decomposed into parts that most people will agree with, and (3) it resembles some familiar objects. Maybe it looks like an ice cream or hot-dog cart.

Biederman postulated that objects are represented by decomposing the object into a small set of geometric components (e.g., blocks, cylinders,

cones, and so on.) that he called **Geons**. He derived 36 different Geons that we can discriminate visually. Geons we defined by non-accidental attributes such as symmetry, colinearity, cotermination, parallelism, and curvature. An object is defined by a particular arrangement of a small set of Geons. Object recognition consists in a sequence of stages: (1) edge detection, (2) detection of non-accidental properties, (3) detection of Geons, and (4) matching the detected Geons to stored object representations.

One important aspect of his theory is that objects are formed by compositing simpler elements (Geons), which are shared across many different object classes. **Compositionality** in the visual world is not as strong as the one found in language, where a fix set of words are composed to form sentences with different meanings. In the visual world, components shared across different object classes will have different visual appearances (e.g., legs of tables and chairs share many properties but are not identical). Geons are great to represent artificial objects (e.g., tables, cabinets, and phones) but fail when applied to represent stuff (e.g., grass, clouds, and water) or highly textured objects such as trees or food.

Decomposing an object into parts has been an important ingredient of many computer vision object recognition systems [[124](#), [129](#), [496](#)].

50.2.2 Invariances

Object recognition in humans seems to be quite robust to changes in viewpoint, illumination, occlusion, deformations, styles, and so on. However, perception is not completely invariant to those variables.

One well studied property of the human visual system is its invariance to image and 3D rotations. As we discussed in chapter 35, studies in human visual perception have shown that objects are recognized faster when they appear in canonical poses.

In a landmark study, Roger N. Shepard and Jaqueline Metzler [[432](#)] showed to participants pairs of drawings of three-dimensional objects ([figure 50.3](#)). The pair of drawings could show the same object with a mirror or the same object with an arbitrary 3D rotation. The task consisted in deciding if the two objects were identical (up to a 3D rotation) or mirror images of each other. During the experiment, they recorded the success rate and the reaction time (how long did participant took to answer). The result showed that participants had a reaction time proportional to the rotation angle between the two views. This supported the idea that participants were

performing a **mental rotation** in order to compare both views. Whether humans actually perform mental rotations or not still remains controversial.

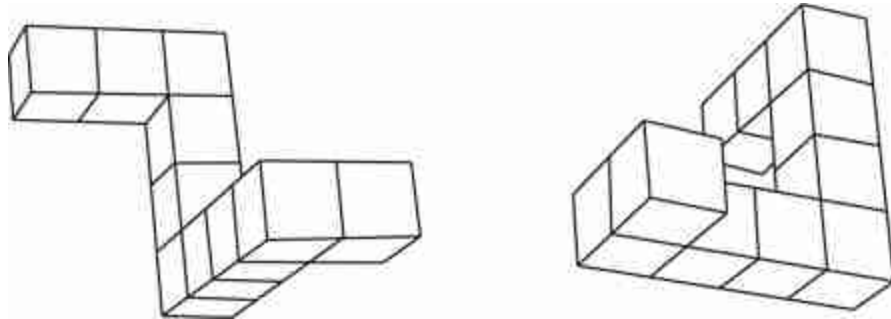


Figure 50.3: Two figures related by a 3D rotation. Modified from [\[432\]](#).

Michael Tarr and Steven Pinker [\[464\]](#) showed that similar results were obtained on a learning task. When observers learn to recognize a novel object from a single viewpoint they are able to generalize to new viewpoints but they do that at a cost: recognition time increases with the angle of rotation as if they had to mentally rotate the object in order to compare it with the view viewed during training. When trained with multiple viewpoints, participants recognized equally quickly all the familiar orientations.

The field of cognitive science has formulated a number hypothesis about the mechanisms used for recognizing objects: geometry based models, view-based templates, prototypes, and so on. Many of those hypothesis have been the inspiration for computer vision approaches for object recognition.

50.2.3 Principles of Categorization

When working on object recognition, one typical task is to train a classifier to classify images of objects into categories. A category represents a collection of equivalent objects. Categorization offers a number of advantages. As described in [\[412\]](#) “It is to the organism's advantage not to differentiate one stimulus from others when that differentiation is irrelevant for the purposes at hand.” Eleanor Rosch and her collaborators proposed that categorization occurred at there different levels of abstraction:

superordinate level, basic-level, and subordinate level. The following example, from [411], illustrates the three different levels of categorization:

Table 50.1

Three levels of categorization. Superordinate level, basic level and subordinate.

Superordinate	Basic Level	Subordinate
Furniture	Chair	Kitchen chair
		Living-room chair
	Table	Kitchen table
		Dining-room table
	Lamp	Floor lamp
		Desk lamp
Tree	Oak	White oak
		Red oak
	Maple	Silver maple
		Sugar maple
	Birch	River birch
		White birch

Superordinate categories are very general and provide the a high degree of abstraction. Basic level categories correspond to the most common level of abstraction. The subordinate level of categorization is the most specific one. Two objects that belong to the same superordinate categories share fewer attributes than two objects that belong to the same subordinate category.

Object categorization into discrete classes has been a dominant approach in computer vision. One example is the ImageNet dataset that organizes object categories into the taxonomy provided by WordNet [123].

However, as argued by E. Rosch [411], “natural categories tend to be fuzzy at their boundaries and inconsistent in the status of their constituent members.” Rosch suggested that categories can be represented by **prototypes**. Prototypes are the clearest members of a category. Rosch hypothesized that humans recognize categories by measuring the similarity to prototypes. By using prototypes, Rosch [411] suggested that it is easier to

work with continuous categories, as the categories are defined by the “clear cases rather than its boundaries.”

But object recognition is not as straightforward as a categorization task and using categories can result in a number of issues. One example is dealing with the different affordances that an object might have as a function of context. In language, a word or a sentence can have a standing meaning and an occasion meanings. An standing meaning corresponds to the conventional meaning of an expression. The occasion meaning is the particular meaning that the same expression might have when used in a specific context. Therefore, when an expression is being used, it has an occasion meaning that differs from its conventional meaning. The same thing happens with visual objects as illustrated in [figure 50.4](#).



Figure 50.4: The same object can have different occasion meanings depending on the context. (a) A box. (b) A box used as a table.

Avoiding categorization all together has been also the focus on some computer vision approaches to image understanding like **the Visual Memex** by T. Malisiewicz and A. Efros [\[316\]](#).

We will now study a sequence of object recognition models with outputs of increasing descriptive power.

50.3 Image Classification

In section 50.3 we will repeat material already presented in the book. Read this section as if it were written by Pierre Menard, from the short story *Pierre Menard, Author of the Quixote* by Jorge Luis Borges [55]. This character rewrote *Don Quixote* word for word, but the same words did not mean the same thing, because the context of the writing (who wrote it, why they wrote it, and when they wrote it) was entirely different.

Image classification is one of the simplest tasks in object recognition. The goal is to answer the following, seemingly simple, question: Is object class c present anywhere in the image $\mathbf{x}$? We also covered image classification as a case study of machine learning in section 9.7.3. Even though we will use the same words now, and even the same sentences, you should read them in a different way. The same words will mean something different. Before, we emphasized the problem of learning; now we are focusing on the problem of object understanding. So, when we say $\mathbf{y} = f(\mathbf{x})$, before the focus of our attention was f , now the focus is $\mathbf{y}$.

Image classification typically assumes that there is a closed vocabulary of object classes with a finite number of predefined classes. If our vocabulary contains K classes, we can loop over the classes and ask the same question for each class. This task does not try to localize the object in the image, or to count how many instances of the object are present.

50.3.1 Formulation

We can answer the previous question in a mathematical form by building a function that maps the input image $\mathbf{x}$ into an output vector $\hat{\mathbf{y}}$:

$$\hat{\mathbf{y}} = f(\mathbf{x}) \tag{50.1}$$

The output, $\hat{\mathbf{y}}$, of the function will be a binary response: 1 = *yes*, the object is present; and 0 = *no*, the object is not present.



Figure 50.5: Image classification: Is there a car in this image?

In general we want to classify an image according to multiple classes. We can do this by having as output a vector $\hat{\mathbf{y}}$ of length K , where K is the number of classes. Component c of the vector $\hat{\mathbf{y}}$ will indicate whether class c is present or absent in the image. For instance, the training set will be composed of examples of images and the corresponding class indicator vectors as shown in [figure 50.6](#) (in this case, $K = 3$).

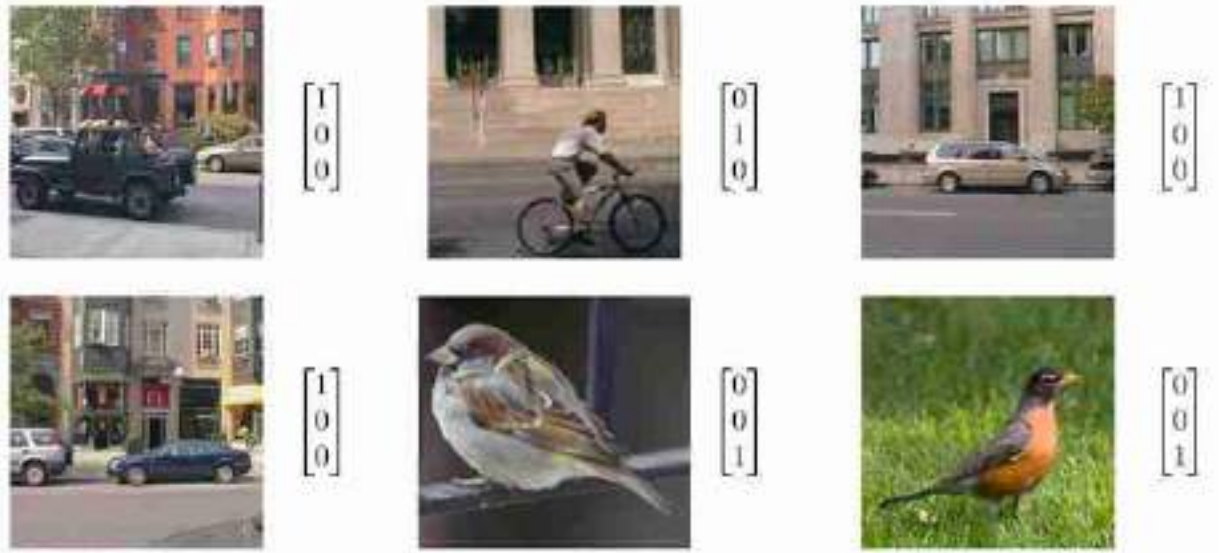


Figure 50.6: Training set for image classification with $K = 3$ classes.

We want to enrich the output so that it also represents the uncertainty in the presence of the object. This uncertainty can be the result of a function f_θ that does not work very well, or from a noisy or blurry input image $\mathbf{x}$ where the object is difficult to see. One can formulate this as a function f_θ that takes as input the image $\mathbf{x}$ and outputs the probability of the presence of each of the K classes. Letting $Y_c \in \{0, 1\}$ be a binary random variable indicating the presence of class c , we model uncertainty as,

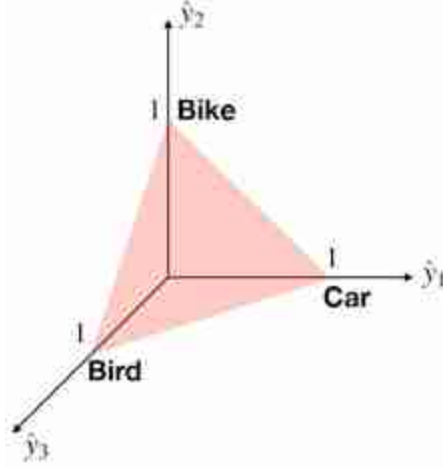
$$p(Y_c = 1 | \mathbf{x}) = \bar{y}_c \quad (50.2)$$

50.3.1.1 Exclusive classes

One common assumption is that only one class, among the K possible classes, is present in the image. This is typical in settings where most of the images contain a single large object. Multiclass classification with exclusive classes results is given in the following constraint:

$$\sum_{c=1}^K \hat{y}_c = 1 \quad (50.3)$$

where $0 > \hat{y}_c > 1$. In the toy example with three classes shown previously, the valid solutions for $\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \hat{y}_3]^T$ are constrained to lie within a **simplex**.



Under this formulation, the function f is a mapping $f: \mathbb{R}^{N \times M \times 3} \rightarrow \Delta^{K-1}$ from the set of red-green-blue (RGB) images to the $(K - 1)$ -dimensional simplex, Δ^{K-1} .

The function f is constrained to belong to a family of possible functions. For instance, f might belong to the space of all the functions that can be built with a neural network. In such a case the function is specified by the network parameters $\theta: f_\theta$. When using a neural net, the vector $\hat{\mathbf{y}}$ is usually computed as the output of a softmax layer.

This yields the softmax regression model we saw previously in section 9.7.3. Given the constraint of equation (50.3), the output of f_θ can be interpreted as a probability mass function over K classes:

$$p_\theta(Y | \mathbf{x}) = \hat{\mathbf{y}} = f_\theta(\mathbf{x}) \quad (50.4)$$

where Y is the random variable indicating the single class per image. This relates to our previous model in equation (50.2) as:

$$p(Y_c = 1 | \mathbf{x}) = p_\theta(Y = c | \mathbf{x}) \quad (50.5)$$

50.3.1.2 Multilabel classification

When the image can have multiple labels simultaneously, the classification problem can be formulated as K binary classification problems (where K is the number of classes).

In such a case, the function f_θ can be a neural network shared across all classes ([figure 50.7](#)), and the output vector $\hat{\mathbf{y}}$, of length K , is computed using a sigmoid nonlinearity for each output class. In this case the output can still be interpreted as the probability $\hat{y}_c = p_\theta(Y_c = 1 | \mathbf{x})$, but without the constraint that the sum across classes is 1.

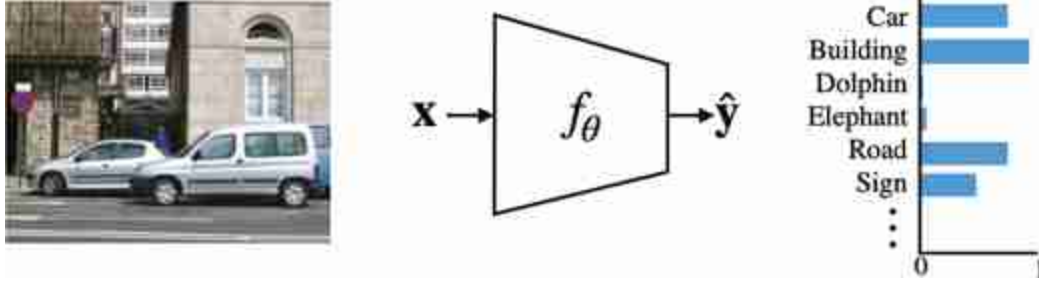


Figure 50.7: Multilabel image classification system.

50.3.2 Classification Loss

In order to learn the model parameters, we need to define a loss function that will capture the task that we want to solve. One natural measure of classification error is the **misclassification error**. The errors that the function f makes are the number of misplaced ones (i.e., the misclassification error) over the dataset, which we can write as:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{t=1}^T \sum_{c=1}^K \mathbb{1}(\hat{y}_c^{(t)} \neq y_c^{(t)}), \quad (50.6)$$

where T is the number of images in the dataset.

However, it is hard to learn the function parameters using gradient descent with this loss as it is not differentiable. Finding the optimal parameters of function f_θ requires defining a loss function that we will be tractable to optimize during the training stage.

50.3.2.1 Exclusive classes

If we interpret the function f as estimating the probability $p(Y_c = 1 \mid \mathbf{x}) = \hat{y}_c$, with $\sum_c \hat{y}_c = 1$. The likelihood of the ground truth data is:

$$\prod_{t=1}^T \prod_{c=1}^K (\hat{y}_c^{(t)})^{y_c^{(t)}} \quad (50.7)$$

where $y_c^{(t)}$ is the ground truth value of Y_c for image t , and the first product loops over all T training examples while the second product loops over all the classes K . We want to find the parameters θ that maximize the likelihood of the ground truth labels for whole training image. As we have seen in chapter 24, maximizing the likelihood corresponds to minimizing the **cross-entropy loss** (which we get by just taking the negative log of the equation [50.7]), resulting in a total classification loss:

$$\mathcal{L}_{cls}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{t=1}^T \sum_{c=1}^K y_c^{(t)} \log(\hat{y}_c^{(t)}) \quad (50.8)$$

The cross-entropy loss is differentiable and it is commonly used in image classification tasks.

50.3.2.2 Multilabel classification

If classes are not exclusive, the likelihood of the ground truth data is:

$$\prod_{t=1}^T \prod_{c=1}^K (\hat{y}_c^{(t)})^{y_c^{(t)}} (1 - \hat{y}_c^{(t)})^{1 - y_c^{(t)}} \quad (50.9)$$

Bernoulli distribution: The binary random variable Y_c takes the value 1 with probability a and the value 0 with probability $1 - a$. The probability of a realization y can be written as $p(Y_c = y) = a^y (1 - a)^{1-y}$.

where the outputs $\hat{y}_c$ are computed using a sigmoid layer. Taking the negative log of the likelihood gives the **multiclass binary cross-entropy loss**:

$$\mathcal{L}_{cls}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{t=1}^T \sum_{c=1}^K y_c^{(t)} \log(\hat{y}_c^{(t)}) + (1 - y_c^{(t)}) \log(1 - \hat{y}_c^{(t)}) \quad (50.10)$$

50.3.3 Evaluation

Once the function f_θ has been trained, we have to evaluate its performance over a holdout test set. There are several popular measures of performance.

Classification performance uses the output of the classifier, f , as a continuous value, the score $\hat{y}_c$, for each class. We can rank the predicted classes according to that value. The **top-1** performance measures the percentage of times that the true class-label matches the highest scored prediction, $\hat{y} = \underset{c}{\operatorname{argmax}}(\hat{y}_c)$, in the test set:

$$\text{TOP-1} = \frac{100}{T} \sum_{t=1}^T \mathbb{I}(\hat{y}^{(t)} = y^{(t)}) \quad (50.11)$$

where $y^{(t)}$ is the ground truth class of image t . This evaluation metric only works for the exclusive class case since we assume a single ground truth class per image. For the multilabel case, we may instead check if each label's is predicted correctly.

In some benchmarks (such as ImageNet), researchers also measure the **top-5** performance, which is the percentage of times that the true label is within the set of five highest scoring predictions. Top-5 is useful in settings where the labels might be ambiguous, or where multiple labels for one image might be possible. In the case of multiple labels, ideally, the test set should specify which labels are possible and evaluate only using those. The top-5 measure is less precise and it is used when the test set only contains one label despite that multiple labels might be correct.

For multiclass prediction problems it is often useful to look at what is the structure of mistakes made by the model. The **confusion matrix** summarizes both the overall performance of the classifier and also the percentage of times that two classes are confused by the classifier. For instance, the following matrix shows the evaluation of a three-way classifier. The diagonal elements are the same as the top-1 performance for each class. The off-diagonal elements show the confusions. In this case, it seems that the classifier confuses cats as dogs the most.

Table 50.2

Confusion matrix for a three-way classification task. The numbers are percentages.

		Predicted class			
		Cat	Car	Dog	
True class	Cat	80	5	15	$\rightarrow \Sigma = 100$
	Car	2	95	3	$\rightarrow \Sigma = 100$
	Dog	4	0	96	$\rightarrow \Sigma = 100$

The elements of the confusion matrix are:

$$C_{i,j} = 100 \frac{\sum_{t=1}^N \mathbb{1}(y^{(t)} = j) \mathbb{1}(y^{(t)} = i)}{\sum_{t=1}^N \mathbb{1}(y^{(t)} = i)} \quad (50.12)$$

where $C_{i,j}$ measures the percentage of times that true class i is classified as class j .

In this toy example ([table 50.2](#)), we can see that 80 percent of the cat images are correctly classified, while 5 percent of cat images are classified as car images, and 15 percent are classified as pictures of dogs.

50.3.4 Shortcomings

The task of image classification is plagued with issues. Although it is a useful task to measure progress in computer vision and machine learning, one has to be aware of its limitations when trying to use it to produce a meaningful description of a scene, or when developing a classifier for consumption in the real world.

One very important shortcoming of this task is that it assumes that we can actually answer unambiguously the question *does the image contains object c ?* But what happens if class definitions are ambiguous or class boundaries are soft? Even in cases where we believe that the boundaries might be well defined, it is easy to find images that will challenge our assumption, as shown in [figure 50.8](#).



Figure 50.8: Which of these images contains a car? This is a simple question that does not have a simple answer.

Which of these images of [figure 50.8](#) contains a car? A simple question that does not have a simple answer. The top three images contain cars, although with increasing difficulty. However, for the three images on the second row, it is not clear what the desired answer is. We can clearly recognize the content of those images, but it feels as if using a single word for describing those images leaves too much ambiguity. Is a car under construction already a car? Is the toy car a car? And if we play with food and make a car out of watermelon, would that be a car? Probably not. But if a vision system classifies that shape as a car, would that be a mistake like any other?

Although it is not clearly stated in the problem definition, language plays a big role in object classification. The objects in [figure 50.9](#) are easily recognizable to us, but if we ask the question “Is there a fruit in this picture?,” the answer is a bit more complex than it might appear at first glance.



Figure 50.9: A pepper is a fruit according to botanics, and it is a vegetable according to the culinary classification.

What if the object is present in the scene but invisible in the image? What if there are infinite classes? In our formulation, $\hat{\mathbf{y}}$ is a vector of a fixed length. Therefore, we can only answer the question “is object class c present in the image?” for a finite set of classes. What if we want to be able to answer an infinite set of questions? The next sections introduce more sophisticated formulations of object recognition that address some of these questions.

Is it possible to classify an image without localizing the object? How can we answer the question “Is object class c present in the image?” without localizing the object? One possibility is that the function f has learned to localize the object internally, but the information about the location is not being recorded in the output. Another possibility is that the function has learned to use other cues present in the image (biases in the dataset). As it is not trying to localize an object, the function f will not get penalized if it uses shortcuts such as only recognizing one part of the object (e.g., such as the head of a dog, or car wheels), or if it makes use of contextual biases in the dataset (e.g., all images with grass and blue skies have horses, or all streets have cars) or detects unintended correlations between low-level image properties and the image content (e.g., all images of insects have a blurry background). As a consequence, image classification performance could mislead us into believing that the classifier works well and that it has learned a good representation of the object class it classifies.

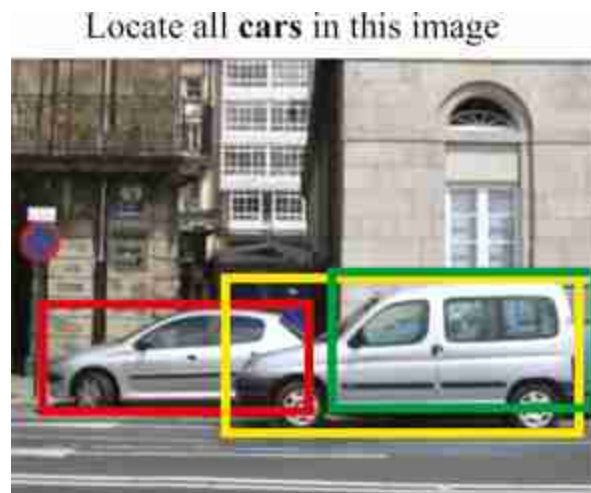
50.4 Object Localization

For many applications, saying that an object is present in the image is not enough. Suppose that you are building a visual system for an autonomous vehicle. If the visual system only tells the navigation system that there is a person in the image it will be insufficient for deciding what to do. Object localization consists of localizing where the object is in the image. There are many ways one can specify where the object is. The most traditional way of representing the object location is using a **bounding box** ([figure 50.10](#)), that is, finding the image-coordinates of a tight box around each of the instances of class c in the image $\mathbf{x}$.

50.4.1 Formulation

How you look at an object does not change the object itself (whether you see it or feel it, etc.). This induces translation and scale invariance. Let's build a system that has this property.

We will formulate object detection as a function that maps the input image $\mathbf{x}$ into a list of bounding boxes $\mathbf{b}_i$ and the associated classes to each bounding box encoded by a vector $\hat{\mathbf{y}}_i$, as illustrated in [figure 50.10](#).



[Figure 50.10](#): Car localization using bounding boxes. Each instance is shown with a different color.

The class vector, $\hat{\mathbf{y}}_i$, has the same structure as in the image classification task but now it is applied to describe each bounding box. Bounding boxes

are usually represented as a vector of length 4 using the coordinates of the two corners, $\mathbf{b} = [x_1, y_1, x_2, y_2]$, or with the center coordinates, width, and height, $\mathbf{b} = [x_c, y_c, w, h]$. Our goal is a function f that outputs a set of bounding boxes, $\mathbf{b}$, and their classes $\mathbf{y}$:

$$\{\hat{\mathbf{y}}_i, \hat{\mathbf{b}}_i\} = f(\mathbf{x}) \quad (50.13)$$

Most approaches for object detection have three main steps. In the first step, a set of candidate bounding boxes are proposed. In the second step, we loop over all proposed bounding boxes, and for each one, we apply an classifier the image patch inside the bounding box. In the third and final step, the selected bounding boxes are postprocessed to remove any redundant detections.

50.4.1.1 Window scanning approach

In its simplest form, the problem of object localization is posed as a binary classification task, namely distinguishing between a single object class and background class. Such a classification task can be turned into a detector by sliding it across the image (or image pyramid), and classifying each local window as shown in [figure 50.11](#).



Figure 50.11: Window scanning approach using a multiscale image pyramid.

The window scanning algorithm is the following:

Algorithm 50.1: Scanning window approach for object localization.

```
1 Input: Image; Output: Object bounding boxes and classes,  $\{\hat{\mathbf{b}}_i, \hat{\mathbf{y}}_i\}_{i=1}^B$ 
2  $\{\mathbf{b}_i\}_{i=1}^{B'} \leftarrow$  list all bounding boxes (all locations, scales, aspect ratios, ...)
3 for  $i = 1, \dots, B'$  do
4    $\hat{\mathbf{y}}_i \leftarrow$  Classification of content in box  $\mathbf{b}_i$ 
5  $\{\hat{\mathbf{b}}_i, \hat{\mathbf{y}}_i\}_{i=1}^B \leftarrow$  Thresholding and nonmaximum suppression
```

In this approach, location and translation invariance are achieved by the bounding box proposal mechanism. We can add another invariance, such as rotation invariance, by proposing rotated bounding boxes.

50.4.1.2 Selective search

The window scanning approach can be slow as the same classifier needs to be applied to tens of thousands of image patches. Selective search makes the process more efficient by proposing an initial set of bounding boxes that are good candidates to contain an object ([figure 50.12](#)). This proposal mechanism is performed by a selection mechanism simpler than the object classifier. This approach was motivated by the strategy used by the visual system in which attention is first directed toward image regions likely to contain the target [[507](#), [476](#), [268](#)]. This first attentional mechanism is very fast but might be wrong. Therefore, a second, more accurate but also more expensive, processing stage is required in order to take a reliable decision. The advantage of this **cascade** of decisions is that the most expensive classifier is only applied to a sparse set of locations (the ones selected by the cheap attentional mechanism) dramatically reducing the overall computational cost.

The algorithm for selective search only differs from the window scanning approach on how the list of candidate bounding boxes is generated. Some approaches also add a bounding box refinement step. The overall approach is described in algorithm 50.2.

Algorithm 50.2: Selective search approach for object localization.

```
1 Input: Image; Output: Object bounding boxes and classes,  $\{\hat{\mathbf{b}}_i, \hat{\mathbf{y}}_i\}_{i=1}^B$ 
2  $\{\tilde{\mathbf{b}}_i\}_{i=1}^B \leftarrow$  Bounding box proposals
3 for  $i = 1, \dots, B$  do
4    $\hat{\mathbf{y}}_i \leftarrow$  Classification of content in box  $\tilde{\mathbf{b}}_i$ 
5    $\hat{\mathbf{b}}_i \leftarrow$  Bounding box refinement
6  $\{\hat{\mathbf{b}}_i, \hat{\mathbf{y}}_i\}_{i=1}^B \leftarrow$  Thresholding and nonmaximum suppression
```

This algorithm has four main steps. In the first step, the algorithm uses an efficient window scanning approach to produce a set of candidate bounding boxes. In the second step, we loop over all the candidate bounding boxes, we crop out the box and resize it to a canonical size, and then we apply an object classifier to classify the cropped image as containing the object we are looking for or not. In the third step, we refine the bounding box for the crops that are classified as containing the object. And finally, in the fourth step, we remove low-scoring boxes and we use **nonmaximum suppression** (NMS) to discard overlapping detections likely to correspond to the same object, so as to output only one bounding box for each instance present in the image.

The nonmaximum suppression algorithm takes as input a set of object bounding boxes and confidences, $S = \{\tilde{\mathbf{b}}_i, \tilde{\mathbf{y}}_i\}_{i=1}^B$ and outputs a smaller set removing overlapping bounding boxes. It is an iterative algorithm as follows: (1) Take the highest confidence bounding box from the set S and add it to the final set S^* . (2) Remove from S the selected bounding box and all the bounding boxes with an IoU larger than a threshold. (3) Go to step 1 until S is empty.

Each step can be implemented in several different ways, giving rise to different approaches. The whole pipeline is summarized in [figure 50.12](#).



Figure 50.12: A first classifier selects candidate bounding boxes. A second, stronger, classifier makes the final detections (e.g., cars). Nonmaximum suppression (NMS) removes overlapping detections.

Bounding box proposal (represented as f_0 in [figure 50.13](#)) can be implemented in several ways (e.g., using image segmentation [\[481\]](#), a neural network [\[404\]](#), or a window scanning approach with a low-cost classifier). The classification and bounding box refinement, f_1 , can be implemented by a classifier and a regression function.

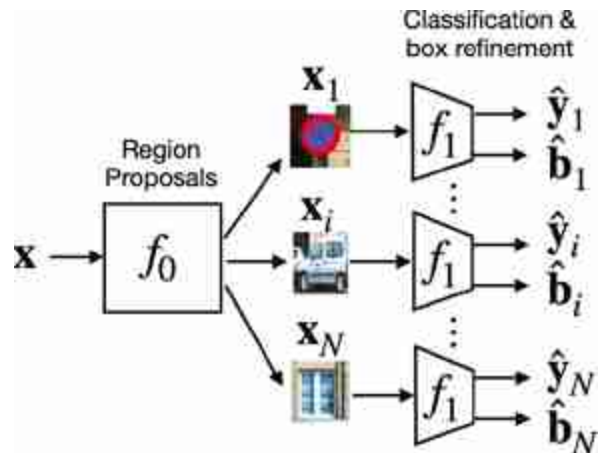


Figure 50.13: Sketch of the selective search architecture. The image is first broken into candidate regions. Then each region is processed individually using a classifier and a regression.

50.4.1.3 Cascade of classifiers

Trying to localize an object in an image is like finding a needle in a haystack: the object is usually small and might be surrounded by a complex background. Selective search reduced the complexity of the search by dividing the search in two steps: a first, fast, and cheap classification function that detects good candidate locations; and a second, slow, and

expensive classification function capable of accurately classifying the object and that only needs to be applied in a subset of all possible locations and scales. Cascade of classifiers pushes this idea to the limit by dividing the search in a sequence of classifiers of increasing computational complexity and accuracy.

The algorithm for the cascade of classifiers is:

Algorithm 50.3: Cascade of classifiers for object localization.

```

1 Input: Image,  $\mathbf{x}$ ; Output: Object bounding boxes and classes,  $\{\hat{\mathbf{b}}_i, \hat{\mathbf{y}}_i\}_{i=1}^B$ 
2  $S_0 = \{\mathbf{b}_i\}_{i=1}^{\theta_0} \leftarrow$  Initial set of bounding box proposals
3 for  $j = 1, \dots, \text{Levels}$  do
4   for  $\mathbf{b}_i \in S_{j-1}$  do
4      $\hat{\mathbf{y}}_i = f_j(\mathbf{x}, \mathbf{b}_i) \leftarrow$  Classification of content in box  $\mathbf{b}_i$ 
4      $S_j = \{\mathbf{b}_i \text{ such that } \hat{\mathbf{y}}_i > \theta_j\} \leftarrow$  Keep high scoring bounding boxes
5
6
7  $\{\hat{\mathbf{b}}_i, \hat{\mathbf{y}}_i\}_{i=1}^B$  Nonmaximum suppression

```

Cascades of classifiers became popular in computer vision when Paul Viola and Michael Jones [489] introduced it in 2001 with a ground-breaking real-time face detector based on a cascade of boosted classifiers. In parallel, Fleuret and Geman [132] also proposed a system that performs a sequence of binary tests at each location. Each binary test checks for the presence of a particular image feature. Early versions of this strategy were also inspired by the game “Twenty Questions” [158].

In a cascade, computational power is allocated into the image regions that are more likely to contain the target object while regions that are flat or contain few features are rejected quickly and almost no computations are allocated in them. The following figure, from [132], shows a beautiful illustration of how a cascaded classifier allocates computing power in the image when trained to detect faces. The intensity shown in the heat map is proportional to the number of levels in the cascade applied to each location.

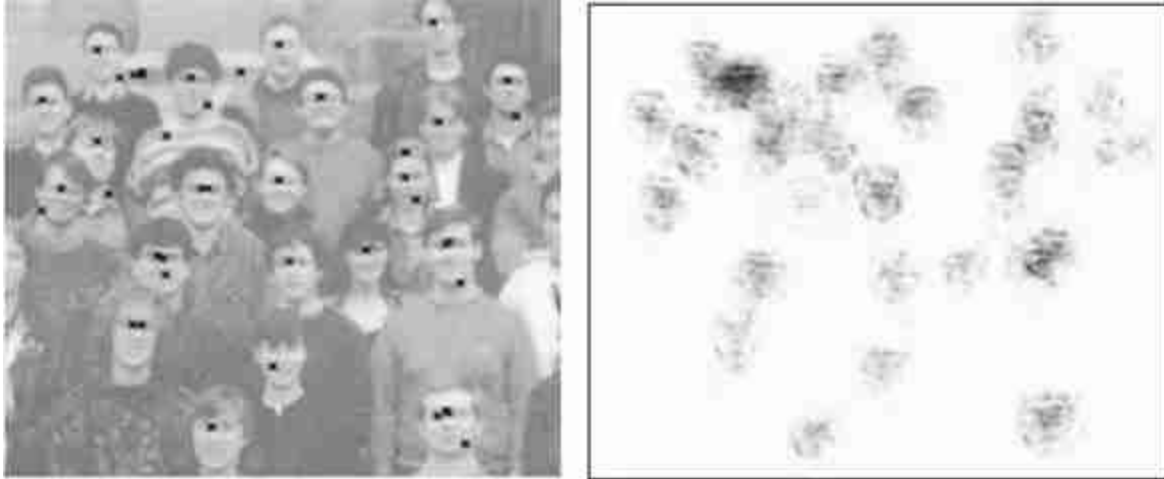


Figure 50.14: (left) Input image with the output of a face detector from 1999. (right) The heat map reports the computational cost at each location. Most of the computation was allocated into the wrong detections in the top left corner. Figure from [\[132\]](#).

It is interesting to point out that the scanning window approach (*Levels* = 1) and the selective search procedure (*Levels* = 2) are special cases of this algorithm. The cascade of classifiers usually does not have a bounding box refinement stage as it already started with a full list of all bounding boxes, but it could be added if we wanted to add new transformations not available in the initial set (e.g., rotations).

50.4.1.4 Other approaches

Object localization is an active area of research and there are a number of different formulations that share some elements with the approaches we shared previously. One example is YOLO [\[403\]](#), which makes predictions by looking at the image globally. It is not as accurate as some of the scanning methods but it can be computationally more efficient. There are many other approaches that we will not summarize here as the list will be obsolete shortly. Instead, we will continue focusing on general concepts that should help the reader understand other approaches.

50.4.2 Object Localization Loss

The object localization loss has to take into account two complementary tasks: classification and localization.

- Classification loss (L_{cls}): For each detected bounding box, does the predicted label matches the ground truth at that location?
- Localization loss (L_{loc}): How close is the detected location to the ground truth object location?

For each image, the output of our object detector is a set of bounding boxes $\{\mathbf{b}_i, \hat{\mathbf{y}}_i\}_{i=1}^B$, where B is likely to be larger than the number of ground truth bounding boxes in the training set. The first step in the evaluation is to associate each detection with a ground truth label so that we have a set $\{\mathbf{b}_i, \mathbf{y}_i\}_{i=1}^B$. For each of the predicted bounding boxes that overlaps with the ground truth instances, we want to optimize the system parameters to improve the predicted locations and labels. The remaining predicted bounding boxes that do not overlap with ground truth instances are assigned to the background class, $\mathbf{y}_i = 0$. For those bounding boxes, we want to optimize the model parameters in order to reduce their predicted class score, $\hat{\mathbf{y}}_i$. This process is illustrated in the following figure. The detector produces a set of bounding boxes for candidate car locations. Those are compared with the ground truth data. Each detected bounding box is assigned one ground truth label (indicated by the color) or assigned to the background class (indicated in white). Note that several detections can be assigned to the same ground truth bounding box.

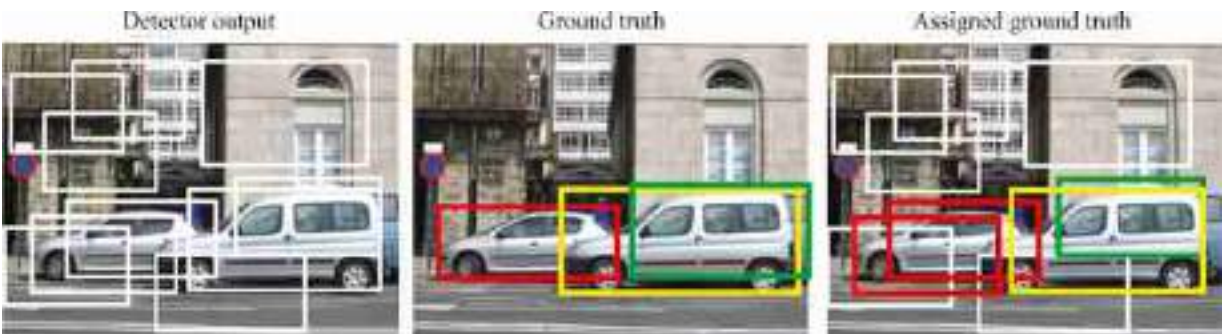


Figure 50.15: (left) Detector output (using a low threshold to generate many detections). (middle) Ground truth bounding boxes (one color per instance). (right) Detector outputs that overlap with ground truth annotations, which are color coded.

Now that we have assigned detections to ground truth annotations, we can compare them to measure the loss. For the classification loss, as each

bounding box can only have one class, we can use the cross-entropy loss:

$$\mathcal{L}_{\text{cls}}(\hat{y}_i, y_i) = - \sum_{c=1}^K y_{c,i} \log(\hat{y}_{c,i}) \quad (50.14)$$

where K is the number of classes.

Let's now focus on the second part; how do we measure the localization loss $\mathcal{L}_{\text{loc}}(\hat{\mathbf{b}}_i, \mathbf{b}_i)$? One typical measure of similarity between two bounding boxes is the **Intersection over Union** (IoU) as shown in the following drawing:

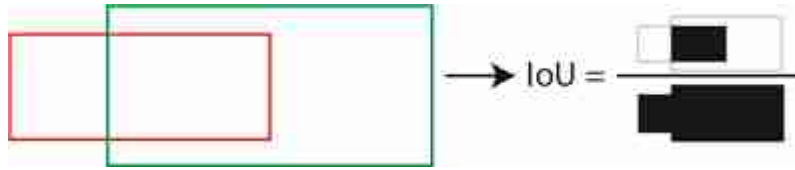


Figure 50.16: Sketch of the computation of intersection over union for two bounding boxes.

The IoU is a quantity between 0 and 1, with 0 meaning no overlap and 1 meaning that both bounding boxes are identical. As the IoU is a similarity measure, the loss is defined as:

$$\mathcal{L}_{\text{loc}}(\hat{\mathbf{b}}, \mathbf{b}) = 1 - \text{IoU}(\hat{\mathbf{b}}, \mathbf{b}). \quad (50.15)$$

The IoU is translation and scale invariant, and it is frequently used to evaluate object detectors. A simpler loss to optimize is the L2 regression loss:

$$\mathcal{L}_{\text{loc}}(\hat{\mathbf{b}}, \mathbf{b}) = (\hat{x}_1 - x_1)^2 + (\hat{x}_2 - x_2)^2 + (\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 \quad (50.16)$$

The L2 regression loss is translation invariant, but it is not scale invariant. The L2 loss is larger for big bounding boxes. The next graph ([figure 50.17](#)) compares the IoU loss, and the L2 regression loss for two square bounding boxes, with an area equal to 1, as a function of the relative x -displacement.

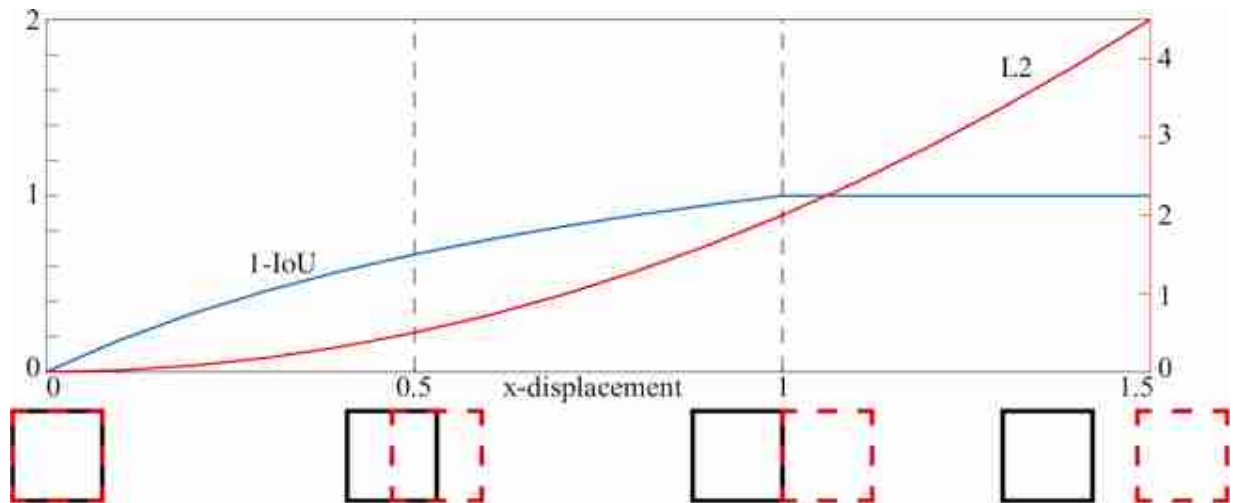


Figure 50.17: Comparison of the the IoU loss and the L2 regression loss for two square bounding boxes as a function of the relative x-displacement.

The IoU loss becomes 1 when the two bounding boxes do not overlap, which means that there will be no gradient information when running backpropagation.

Now we can put together the classification and the localization loss to compute the overall loss:

$$\mathcal{L}(\{\hat{\mathbf{b}}_i, \hat{\mathbf{y}}_i\}, \{\mathbf{b}_i, \mathbf{y}_i\}) = \mathcal{L}_{\text{cls}}(\hat{\mathbf{y}}_i, \mathbf{y}_i) + \lambda \mathbb{I}(\mathbf{y}_i \neq 0) \mathcal{L}_{\text{loc}}(\hat{\mathbf{b}}_i, \mathbf{b}_i) \quad (50.17)$$

where the indicator function $\mathbb{I}(\mathbf{y}_i \neq 0)$ sets to zero the location loss for the bounding boxes that do not overlap with any of the ground truth bounding boxes. The parameter λ can be used to balance the relative strength of both losses. This loss makes it possible to train the whole detection algorithm end-to-end. It is possible to train the localization and the classification stages independently and some approaches follow that strategy.

50.4.3 Evaluation

There are several ways of evaluating object localization approaches. The most common approach is measuring the average precision-recall.

Just as we did when defining the localization loss, we need to assign detection outputs to ground truth labels. We can do this in several ways, and they can result in different measures of performance. The methodology introduced in the PASCAL challenge used the following procedure.

Assign detections to ground truth labels: For each image, sort all the detections by their score, $\hat{y}_i$. Then, loop over the sorted list in decreasing order. For each bounding box, compute the IoU with all the ground truth bounding boxes. Select the ground truth bounding box with the highest IoU. If the IoU is larger than a predefined threshold, (a typical value is 0.5) mark the detection as correct and remove the ground truth bounding box from the list to avoid double counting the same object. If the IoU is below the threshold, mark the detection as incorrect and do not remove the ground truth label. Repeat this operation until there are no more detections to evaluate. Mark remaining ground-truth bounding boxes as missed detections.

Precision-recall curve measures the performance of the detection as a function of the decision threshold. As each bounding box comes with a confidence score $\hat{y}_i$, we need to use a threshold, β , to decide if an object is present at the bounding box location $\mathbf{b}_i$ or not. Given a threshold β , the number of detections is $\sum_i \mathbb{1}(\hat{y}_i > \beta)$ and the number of correct detections is $\sum_i \mathbb{1}(\hat{y}_i > \beta) \times y_i$. From these two quantities we compute the **precision** as the percentage of correct detections:

$$Precision(\beta) = \frac{\sum_i \mathbb{1}(\hat{y}_i > \beta) \times y_i}{\sum_i \mathbb{1}(\hat{y}_i > \beta)} \quad (50.18)$$

The precision only gives a partial view on the detector performance as it does not account for the number of ground truth instances that are not detected (misdetections). The **recall** measures the proportion of ground truth instances that are detected for a given decision threshold β :

$$Recall(\beta) = \frac{\sum_i \mathbb{1}(\hat{y}_i > \beta) \times y_i}{\sum_i y_i} \quad (50.19)$$

Both, the precision and recall, are quantities between 0 and 1. High values of precision and recall correspond to high performance. The next graph ([figure 50.18](#)) shows the precision-recall curve as a function of β (decision threshold).

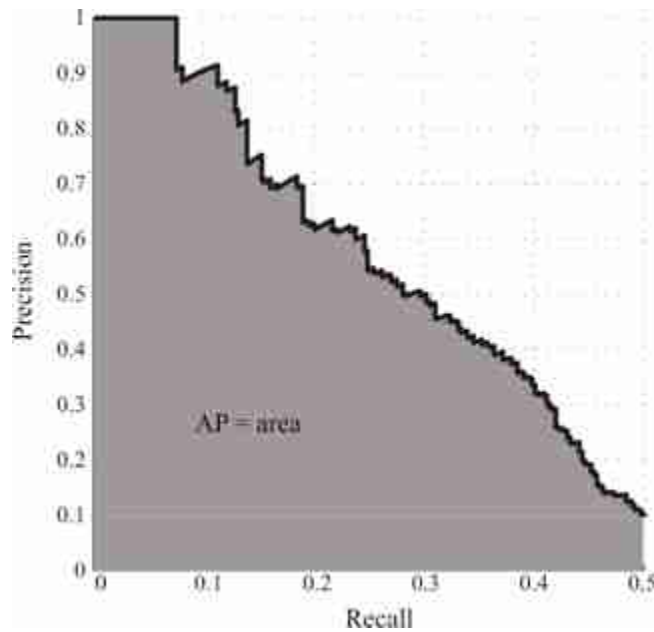


Figure 50.18: Example of a precision-recall curve. This is a standard plot to evaluate detection algorithms. It is usually summarized by the area under the curve. In this example, note that recall stops around 0.5. This is because this algorithm fails to detect the other 50 percent of the test examples for all thresholds β .

The precision-recall curve is non-monotonic. The average precision (AP) summarizes the entire curve with one number. The AP is the area under the precision-recall curve, and it is a number between 0 and 1.

50.4.4 Shortcomings

Bounding boxes can be very powerful in many applications. For instance, digital cameras detect faces and use bounding boxes to encode their location. The camera uses the pixels inside the bounding box to automatically set the focus and exposure time.

But using bounding boxes to represent the location objects will not be appropriate for objects with long and thin structures or for regions that do not have well-defined boundaries ([figure 50.19](#)). It is useful to differentiate between **stuff** and **objects**. Stuff refers to things that do not have well-defined boundaries such as grass, water, and sky (we already discussed this in chapter 28). But the distinction between stuff and objects is not very sharp. In some images, object instances might be easy to separate like the two trees in the left image below, or become a texture where instances cannot be detected individually (as shown in the right image). In cases with

lots of instances, bounding boxes might not be appropriate and it is better to represent the region as “trees” than to try to detect each instance.



Figure 50.19: Bounding boxes are not an appropriate representation of the location of trees. This is true for many object classes with extended shapes that are hard to approximate with a box.

Even when there are few instances, bounding boxes can provide an ambiguous localization when two objects overlap because it might not be clear which pixels belong to each object.

Bounding boxes are also an insufficient object description if the task is robot manipulation. A robot will need a more detailed description of the pose and shape of an object to interact with it.

Let's face it, localizing objects with bounding boxes does not address most of the shortcomings present in the image classification formulation. In fact, it adds a few more.

50.5 Class Segmentation

An object is something localized in space. There are other things that are not localized such as fog, light, and so on. Not everything is well-described by a bounding box (stuff, wiry objects, etc.). Instead we can try to classify each pixel in an image with an object class. Per-pixel classification of object labels, illustrated in [figure 50.20](#), is referred to as **semantic segmentation**.

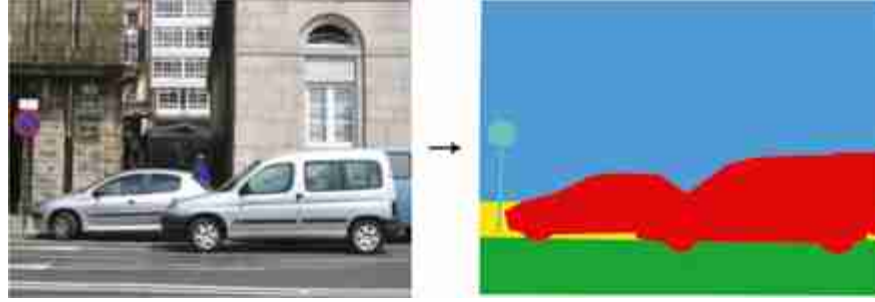


Figure 50.20: Semantic segmentation. Every pixel is annotated with a semantic tag (red = car, blue = building, green = road, yellow = sidewalk, and cyan = sign).

In the most generic formulation, semantic segmentation takes an image as input, $\mathbf{x}(n, m)$, and it outputs a classification vector at each location $\mathbf{y}(n, m)$:

$$\hat{\mathbf{y}}[n, m] = f(\mathbf{x}[n, m]) \quad (50.20)$$

There are many ways in which such a function can be implemented. Generally the function f is a neural network with an encoder-decoder structure ([figure 50.21](#)), first introduced in [27].

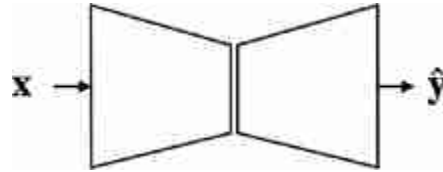


Figure 50.21: Sketch of the encoder-decoder architecture for semantic segmentation.

Another formulation consists of implementing a window-scanning approach where the function f scans the image and takes as input an image patch and it outputs the class of the central pixel of each patch. This is like a convolutional extension of the image classifier that we discussed in section 50.3 and several architectures are variations over this same theme.

Training this function requires access to a dataset of segmented images. Each image in the training set will be associated with a ground truth segmentation, $y_c^{(t)}[n, m]$ for each class c . The multiclass semantic segmentation loss is the cross-entropy loss applied to each pixel:

$$\mathcal{L}_{seg}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{t=1}^T \sum_{c=1}^K \sum_{n,m} y_c^{(t)} [n, m] \log(\hat{y}_c^{(t)} [n, m]) \quad (50.21)$$

where the sum is over all the training examples, all classes, and all pixel locations.

This loss might focus too much on the large objects and ignore small objects. Therefore, it is useful to add a weight to each class that normalizes each class according to its average area in the training set.

In order to evaluate the quality of the segmentation, we can measure the percentage of correctly labeled pixels. However, this measure will be dominated by the large objects (sky, road, etc.). A more informative measure is the average IoU across classes. The IoU is a measure that is invariant to scale and will provide a better characterization of the overall segmentation quality across all classes.

50.5.1 Shortcomings

In this representation we have lost something that we had with the bounding boxes: this representation cannot count instances. The semantic segmentation representation, as formulated in this section, cannot separate two instances of the same class that are in contact; these will appear as a single segment with the same label.

Another important limitation is that each pixel is labeled as belonging to a single class. This means it cannot deal with transparencies. In such a cases, we would like to be able to provide multiple labels to each pixel and also provide a sense of depth ordering.

50.6 Instance Segmentation

We can combine the ideas of object detection and semantic segmentation into the problem of instance segmentation. Here the idea is to output a set of localized objects, but rather than representing them as bounding boxes we represent them as pixel-level masks as shown in [figure 50.22](#).

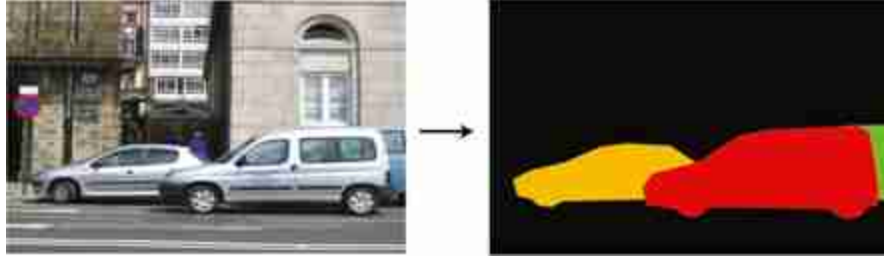


Figure 50.22: Instance segmentation. Each instance of the class car is assigned to a different color.

The difference from semantic segmentation is that if there are K objects of the same type, we will output K masks; conversely, semantic segmentation has no way of telling us how many objects of the same type there are, nor can it delineate between two objects of the same type that overlap; it just gives us an aggregate region of pixels that are all the pixels of the same object type.

One approach to instance segmentation is to follow the object localization pipeline but, after the bounding boxes have been proposed and labeled, feed the cropped image within each bounding box to a binary semantic segmentation function that simply predicts, for each pixel in box $\hat{b}_i$ whether it belongs to the object or not. This way we get an instance mask for each box. Such an approach was introduced in [194]. The architecture is illustrated in [figure 50.23](#).

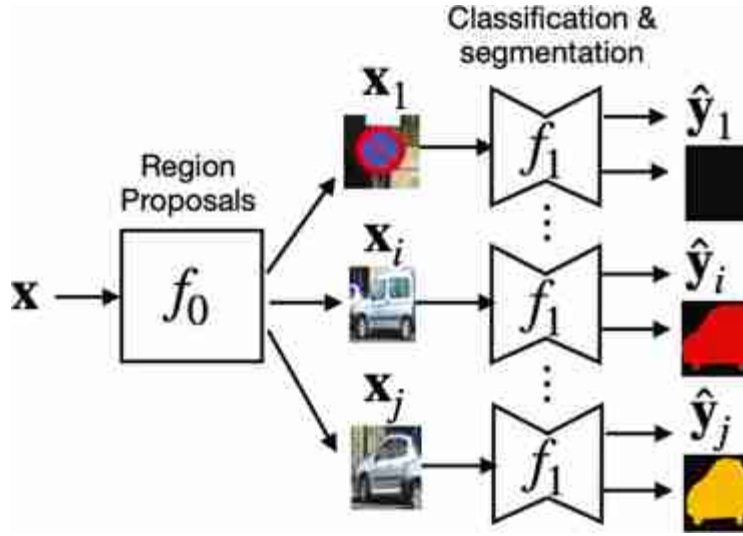


Figure 50.23: Architecture for instance segmentation. This architecture combines several of the components we have seen previously. The image is first broken into overlapping regions, and then class segmentation is applied to each region individually.

As before, f_0 is a low-cost detector that proposes regions likely to contain instances of the target object. The function f_1 performs classification and segmentation of the instance:

$$[\hat{y}_i, \hat{s}_i] = f_1(\mathbf{x}, \mathbf{b}_i) \quad (50.22)$$

where $\hat{y}_i$ is the binary label confidence, and $\hat{s}_i [n, m]$ is the instance segmentation mask.

The loss can be written as a combination of the classification, localization, and segmentation losses that we discussed before:

$$\mathcal{L}(\{\hat{\mathbf{b}}_i, \hat{y}_i\}, \{\mathbf{b}_i, y_i\}) = \mathcal{L}_{\text{cls}}(\hat{y}_i, y_i) + \mathbb{1}(y_i \neq 0) \left(\lambda_1 \mathcal{L}_{\text{loc}}(\hat{\mathbf{b}}_i, \mathbf{b}_i) + \lambda_2 \mathcal{L}_{\text{seg}}(\hat{\mathbf{s}}_i, \mathbf{s}_i) \right) \quad (50.23)$$

The first classification loss corresponds to the classification of the crop as containing the object or not; if the object is present, the second term measures the accuracy of the bounding box, and the third term measures whether the segmentation, $\hat{s}_i$, inside each bounding box matches the ground truth instance segmentation, s_i . The parameters λ_1 and λ_2 control how much weight each loss has on the final loss.

What is interesting about this formulation is that it can be extended to provide a rich representation of the detected objects. For instance, in addition to the segmentation mask, we could also regress the $u - v$

coordinates for each pixel using an instance-centered coordinate frame, or shape maps [165]. We can also output a segmentation of the parts.

50.6.1 Shortcomings

This representation is more complete than any of the previous ones as it combines the best of all of them. It can classify, localize, and count objects and it can also deal with overlapping objects as it will produce a different segmentation mask for each instance. In some aspects, this representation might be trying to do too much; sometimes precise instance segmentation might be too challenging, even for humans.

However, this object representation still suffers from limitations due to its reliance on language in order to define the class.

50.7 Concluding Remarks

[Figure 50.24](#) shows a pictorial summary of the different methods we have seen in this chapter to represent objects in images. The four different approaches we have described use a common set of building blocks: image classifiers, region proposals, pixelwise classifiers, and bounding box regression.

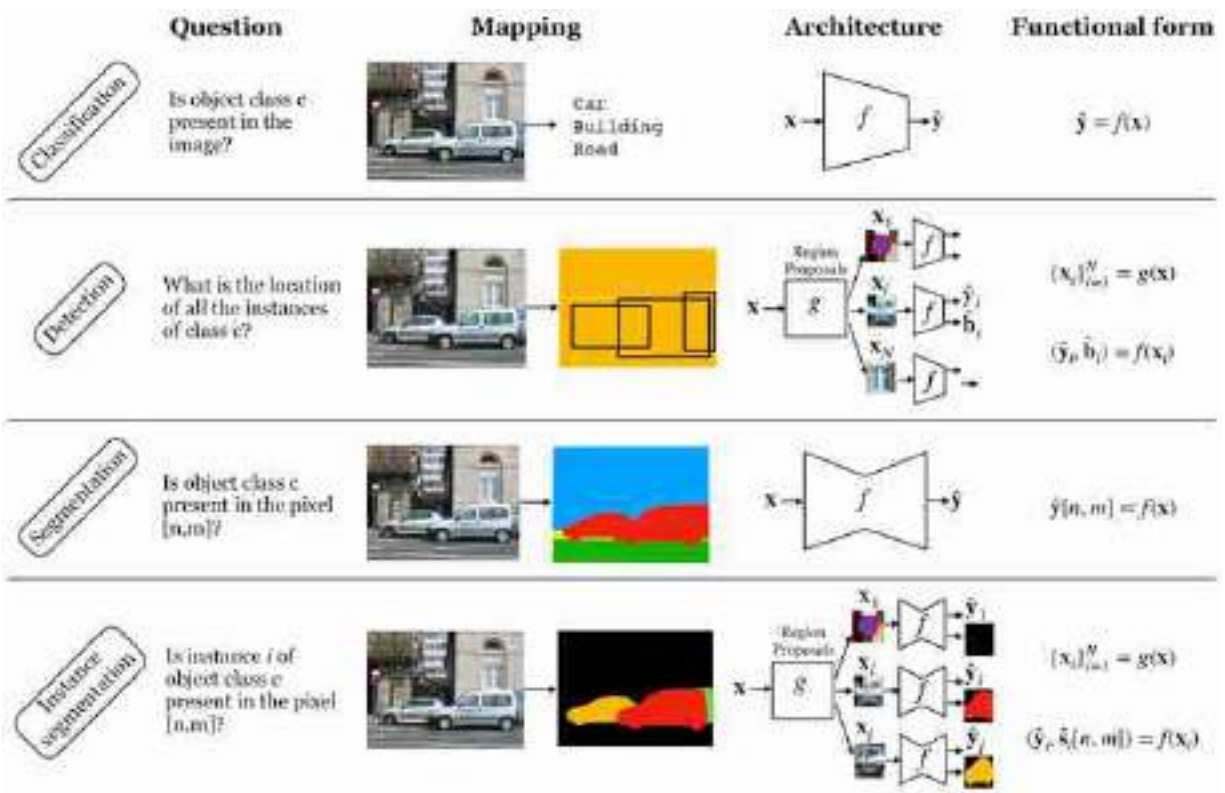


Figure 50.24: A family of object recognition definitions.

However, the representations we have described in this chapter feel very incomplete. If you are building a robot or an autonomous vehicle, such object representations are very hard to act on. A robot planning to grasp an object will need an accurate three-dimensional (3D) modeling of the object as well as an understanding of some of its physical properties in order to plan for the best grasping points. An autonomous vehicle will also need an accurate 3D description of the locations of objects in the world, together with a characterization of their dynamics. Therefore, object segmentation and classification only provide one part of the information needed.

The techniques we have described in this chapter rely on language to define what an object is (e.g., image tagging, captioning, etc.). What else can we do besides associating an object with a word? Here there are other questions that an object recognition system could try to answer:

- Is this piece of matter an object? could I find other instances that might belong to the same class or is it just a random arrangement of things with little chances of being found somewhere else?

- What is its 3D shape?
- Is it resting on the ground? How is it supported?
- What are its mechanical properties? Is it soft? Does it have parts? Is it rigid or deformable?
- How will it react to external actions? Can I grasp the object? How?
- What might this object be useful for? If I have to solve a task, can I use this object?
- Is it alive? Can I eat it?
- What is the physical process that created it? Is it artificial or natural?
- How did the object arrive to its current location?
- Is it stable? What will it do next?

Those are the questions that a scientist would ask themselves when observing a new natural phenomenon, and a computer vision system should behave as a scientist when confronted with a new visual stimulus. A system should formulate hypotheses about its visual input, devise experiments and perform interventions to answer them. One can rely on language to answer some or all of those questions, but language does not need to be the only internal representation used.

51 Vision and Language

51.1 Introduction

The last few decades have seen enormous improvements in both computer vision and natural language processing. Naturally, there has been great interest in putting these two branches of artificial intelligence (AI) together.

At first it might seem like vision has little to do with language. This is what you might conclude if you see the role of language as just being about communication. However, another view on language is that it is a representation of the world around us, and this representation is one the brain uses extensively for higher-level cognition [[133](#)]. Visual perception is also about representing the world around us and forming mental models that act as a substrate for reasoning and decision making. From this perspective, both language and vision are trying to solve more or less the same problem.

It turns out this view is quite powerful, and indeed language and vision can help each other in innumerable ways. This chapter is about some of the ways language can improve vision systems, and also how vision systems can support language systems.

51.2 Background: Representing Text as Tokens

In order to understand **vision-language models (VLMs)** we first need to understand a bit about **language models**. There are many ways to model language and indeed there is a whole field that studies this, called **natural language processing (NLP)**. We will present just one way, which follows almost exactly the same format as modern computer vision systems: tokenize the text, then process it with a transformer. In chapter 26 we saw how to apply transformers to images and now we will see how to apply them to text.

Language models analyze or synthesize languages, including human languages like English or Chinese, programming languages like Python and C++, and scripting languages like Latex or HTML. Some communities reserve the term for models of *human* languages, but we will use the term in a broader sense and include other kinds of languages too.

The amazing thing about transformers is that they work well for a wide variety of data types: in fact, the same architectures that we saw in chapter 26 also works for text processing. To apply transformers to a new data modality may require nothing more than figuring out how to represent that data as numbers.

So our first question is, how can we represent text as numbers? One option, which we have already seen, is to use a word-level encoding scheme, mapping each word in our vocabulary to a unique one-hot code. This requires K -dimensional one-hot codes for a vocabulary of K different words.

A disadvantage of this approach is that K has to be very large to cover a language like English, which has a large vocabulary of over 100,000 possible words. This becomes even worse for computer languages, like Python. Programmers routinely name variables using made up words, which might never have been used before. We can't define a vocabulary large enough to cover all the strange strings programmers might come up with.

Instead, we could use a different strategy, where we assign a one-hot code to each *character* in our vocabulary, rather than to each unique word. If we represent all alphanumeric characters and common punctuation marks in this way, this might only require a few dozen codes. The disadvantage here is that representing a single word now requires a sequence of one-hots rather than a single one-hot code, and we will have to model that sequence.

A happy middle ground can be achieved using **byte-pair** encodings [150]. In this approach, we assign a one-hot code to each small chunk of text. The chunks are common substrings found in text, like “th” or “ing.” This strategy is popular in language models like the **generative pretrained transformer** (GPT) series [63] and the **contrastive language-image pretraining** (CLIP) model [394] that we will learn about in section 51.3.

Once we have converted text into a sequence of one-hot codes, the next step is to convert these into tokens via a projection, $\mathbf{W}_{\text{tokenize}}$, from the one-hot dimensionality (K) to a target dimensionality for token code vectors

(d). [Figure 51.1](#) shows these steps for converting an input string to tokens. For simplicity, we show word-level one-hot encoding rather than byte-pair encoding.

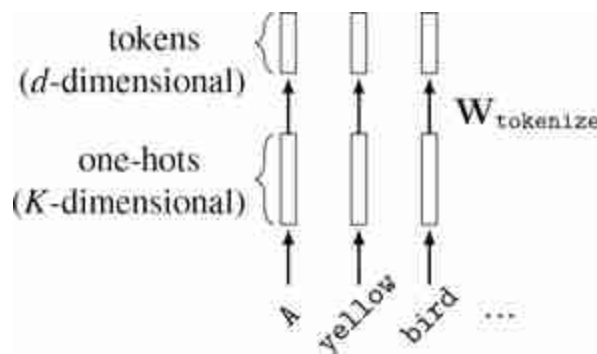


Figure 51.1: Tokenizing text. Note that these steps can be implemented with a lookup table that simply stores the token code vector for each unique item in our vocabulary.

These steps map text to tokens, which can be input into transformers. To handle the output side, where we produce text as an output, we need to convert tokens back to text. The most common way to do this is to use an autoregressive model that predicts each next item in our vocabulary (word/character/byte-pair) given the sequence generated so far. Each prediction step can be modeled as a softmax over all K possible items in our vocabulary. We will see a specific architecture that does this in section 51.4.2.

51.3 Learning Visual Representations from Language Supervision

In chapter 30, we saw that one approach of learning visual representations is to set up a pretext task that requires arriving at a good representation of the scene. In this section we will revisit this idea, using language association as a pretext task for representation learning.

In fact this is not a new idea: historically, one of the most popular pretext tasks has been image classification, where an image is associated with one of K nouns. In a sense, this is already a way of supervising visual representations from language. Of course, human languages are much more

than just a set of K nouns. This raises the question, could we use richer linguistic structure to supervise vision?

The answer is yes! And it works really well. Researchers have tried using all kinds of language structures to supervise vision systems, including adjectives and verbs [231], words indicating relationships between objects [277], and question-answer pairs [17]. What seems to work best, once you reach a certain scale of data and compute, is to just use *all* the structure in language, and the simplest way to do this is to train a vision system that associates images with free-form natural language.

This is the idea of the highly popular CLIP system [394].²² CLIP is a contrastive method (see section 30.10) in which one data view is the image and the other is a caption describing the image. Specifically, CLIP is a form of contrastive learning from co-occurring visual and linguistic views that is formulated as follows:

CLIP

Objective

$$\left(-\log \frac{e^{f_\ell(\mathcal{E})^\top f_t(\mathbf{t}^+) / \tau}}{e^{f_\ell(\mathcal{E})^\top f_t(\mathbf{t}^+) / \tau} + \sum_i e^{f_\ell(\mathcal{E})^\top f_t(\mathbf{t}_i^-) / \tau}} + \right. \\ \left. -\log \frac{e^{f_\ell(\mathbf{t})^\top f_\ell(\mathcal{E}^+) / \tau}}{e^{f_\ell(\mathbf{t})^\top f_\ell(\mathcal{E}^+) / \tau} + \sum_i e^{f_\ell(\mathbf{t})^\top f_\ell(\mathcal{E}_i^-) / \tau}} \right) / 2$$

Hypothesis space

$$f_\ell: \mathbb{R}^{3 \times N \times M} \rightarrow \mathbb{S}^{d_\ell}$$

$$f_t: \mathbb{R}^{T \times d_t} \rightarrow \mathbb{S}^{d_t}$$

Data

 $\{\mathbf{t}^{(i)}, \mathcal{E}^{(i)}\}_{i=1}^N \rightarrow$

$\rightarrow f_\ell, f_t$

where ℓ represents an image, $\mathbf{t}$ represents text, $\mathbb{S}^{d_z}$ represents the space of unit vectors of dimensionality d_z (i.e., the surface of the $[d_z - 1]$ -dimensional hypersphere), f_ℓ is the image encoder, f_t is the text encoder, image inputs are represented as $[3 \times N \times M]$ pixel arrays, text inputs are represented as d_t dimensional tokens, and d_z is the embedding dimensionality. Notice that the objective is a symmetric version of the InfoNCE loss defined in equation (30.12). Also notice that f_ℓ and f_t both output unit vectors (which is achieved using L_2 -normalization before

computing the objective); this ensures that the denominator cannot dominate by pushing negative pairs infinitely far apart.

[Figure 51.2](#) visually depicts how CLIP is trained. First we sample a batch of N language-image pairs, $\{\mathbf{t}^{(i)}, \mathbf{e}^{(i)}\}_{i=1}^N$ ($N = 6$ in the figure). Next we embed measure the dot product between all these text strings and images using a language encoder f_t and an image encoder f_ℓ , respectively. This produces a set of text embeddings $\{\mathbf{z}_t^{(i)}\}_{i=1}^N$ and a set of image embeddings $\{\mathbf{z}_\ell^{(i)}\}_{i=1}^N$. To compute the loss, we take the dot product $\mathbf{z}_t^{(i)} \cdot \mathbf{z}_\ell^{(j)}$ for all $i, j \in \{1, \dots, 6\}$. Terms for which $i \neq j$ are the negative pairs and we seek to minimize these dot products (denominator of the loss); terms for which $i = j$ are the positive pairs and we seek to maximize these dot product (they appear in both the numerator and denominator of the loss).

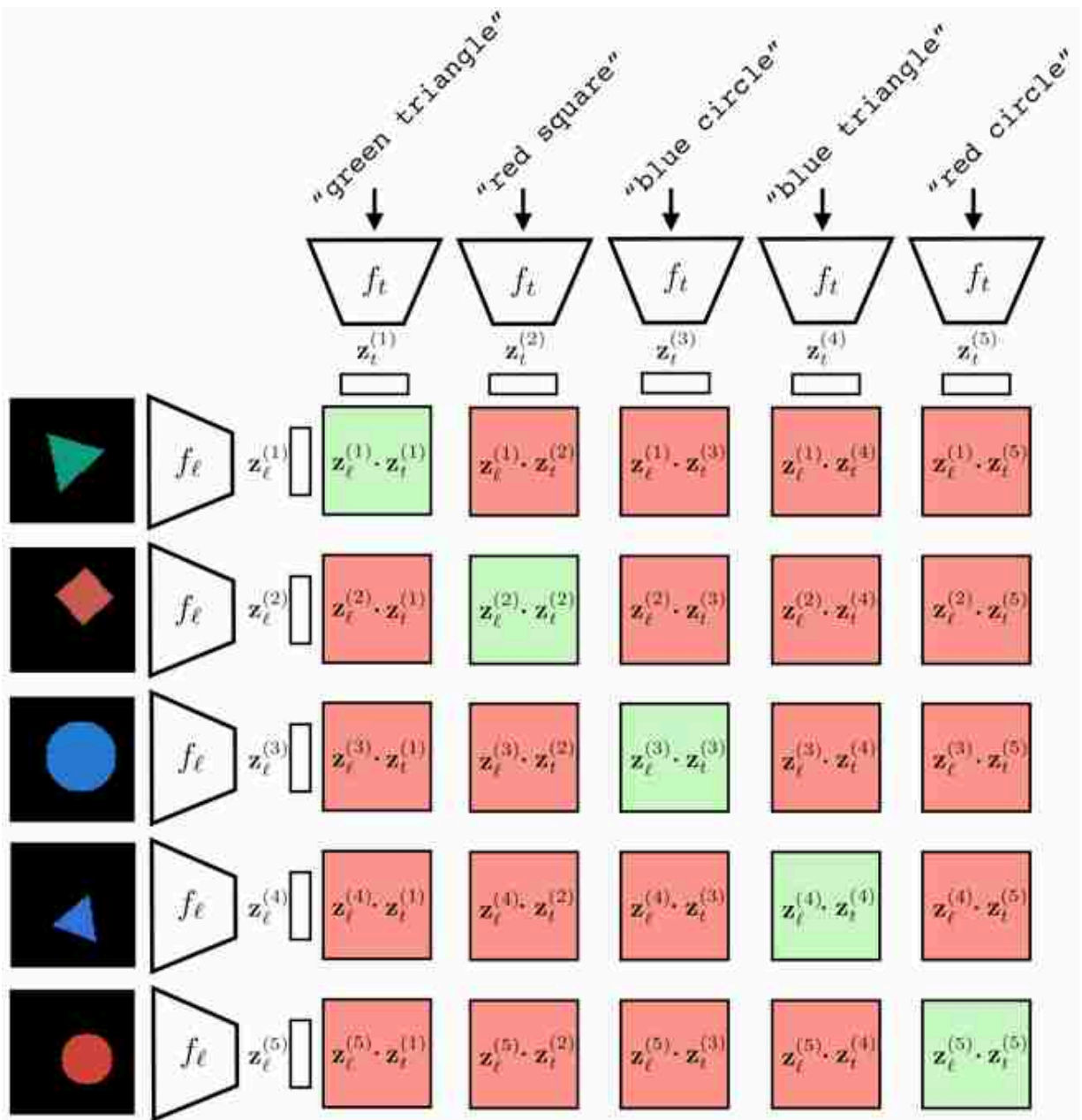


Figure 51.2: CLIP training for one batch of six language-image training examples. Green boxes are positive pairings (we seek to increase their value); these are the six training pairs. Red boxes are negative pairings (we seek to decrease their value); these are all other possible pairings within the batch. Inspired by figure 1 of [394].

After training, we have a text encoder f_t and an image encoder f_ℓ that map to the same embedding space. In this space, the angle between a text embedding and an image embedding will be small if the text matches the matches.

The dot product between unit vectors is proportional to the angle between the vectors.

[Figure 51.3](#) shows how a trained CLIP model maps data to embeddings. This figure is generated in the same way as figure 13.9: for each encoder we reduce dimensionality for visualization using **t-distributed Stochastic Neighbor Embedding (t-SNE)** [\[312\]](#). For the image encoder, we show the visual content using icons to reflect the color and shape depicted in the image (we previously used the same visualization in section 30.10.2). It's a bit hard to see, but one thing you can notice here is that for the image encoder, the early layers group images by color and the later layers group more by shape. The opposite is true for the text encoder. Why do you think this is? One reason may be that color is a superficial feature in pixel-space while shape requires processing to extract, but in text-space, color words and shape words are equally superficial and easy to group sentences by.

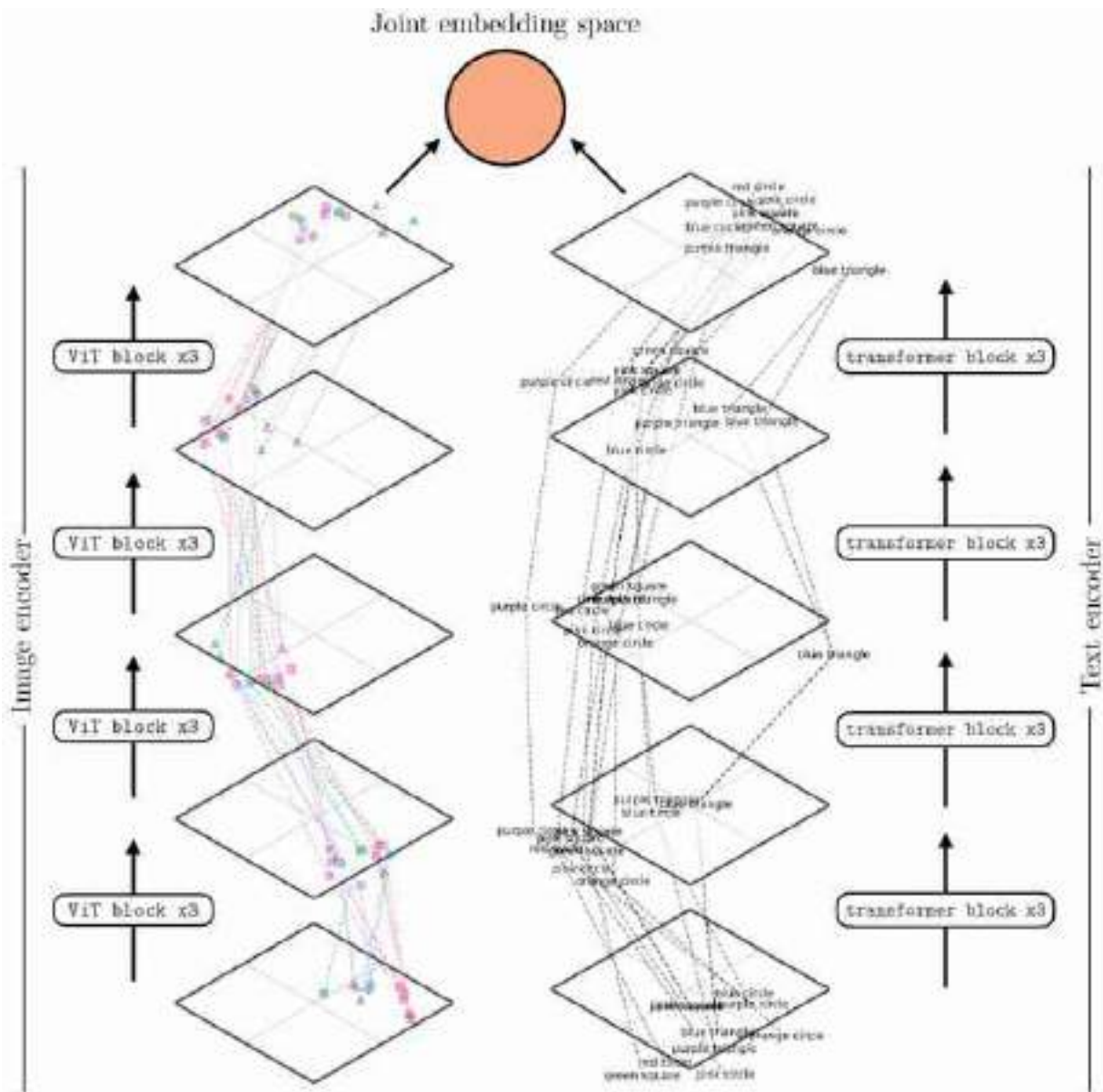


Figure 51.3: CLIP [394] mapping diagram, using the same technique as described in chapter 13. To reduce dimensionality we apply t-SNE [312] separately for the text encoder and the image encoder. Within each encoder, we run t-SNE jointly across all shown layers.

After passing the data through these two encoders, the final stage is to normalize the outputs and compute the alignment between the image and text embeddings. [Figure 51.4](#) shows this last step. After normalization, all the embeddings line on the surface of a hypersphere (the circle in the figure). Notice that the text descriptions are embedded near the image embeddings (icons) that match that description. It's not perfect but note that

this is partially due to limitations in the visualization, which projects the 768-dimensional CLIP embeddings into a 2D visualization. Here we use kernel principle component analysis [427] on the embeddings, with a cosine kernel, and remove the first principle component as that component codes for a global offset between the image embeddings and text embeddings.

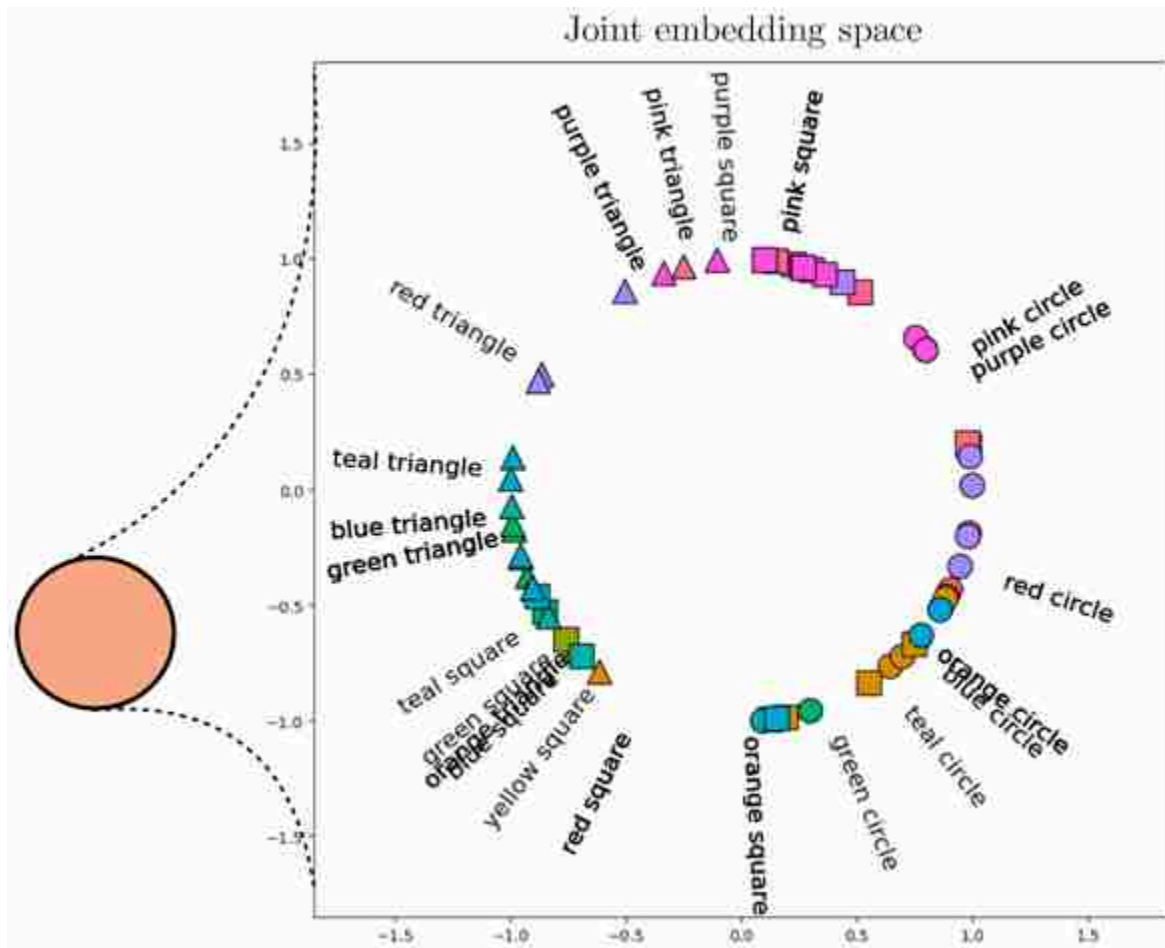


Figure 51.4: CLIP [394] joint embedding.

Using language as one of the views may seem like a minor variation on the contrastive methods we saw previously, but it's a change that opens up a lot of new possibilities. CLIP connects the domain of images to the domain of language. This means that many of the powerful abilities of language become applicable to imagery as well. One ability that the CLIP paper showcased is making a novel image classifier on the fly. With language, you can describe a new conceptual category with ease. Suppose I want a

classifier to distinguish striped red circles from polka dotted green squares. These classes can be described in English just like that: “striped red circles” versus “polka dotted green square.” CLIP can then leverage this amazing ability of English to compose concepts and construct an *image* classifier for these two concepts.

Here's how it works. Given a set of sentences $\{\mathbf{t}^a, \mathbf{t}^b, \dots\}$ that describe images of classes a, b , and so on,

1. Embed each sentence into a $\mathbf{z}$ -vector, $\mathbf{z}_t^a = f_t(\mathbf{t}^a), \mathbf{z}_t^b = f_t(\mathbf{t}^b)$, and so on.
2. Embed your query image into a $\mathbf{z}$ -vector, $\mathbf{z}_t^q = f_t(\ell^q)$.
3. See which sentence embedding is closest to your image embedding; that's the predicted class.

These steps are visualized in [figure 51.5](#). The green outlined dot product is the highest, indicating that the query image will be classified as class a , which is defined as “striped red circles.”

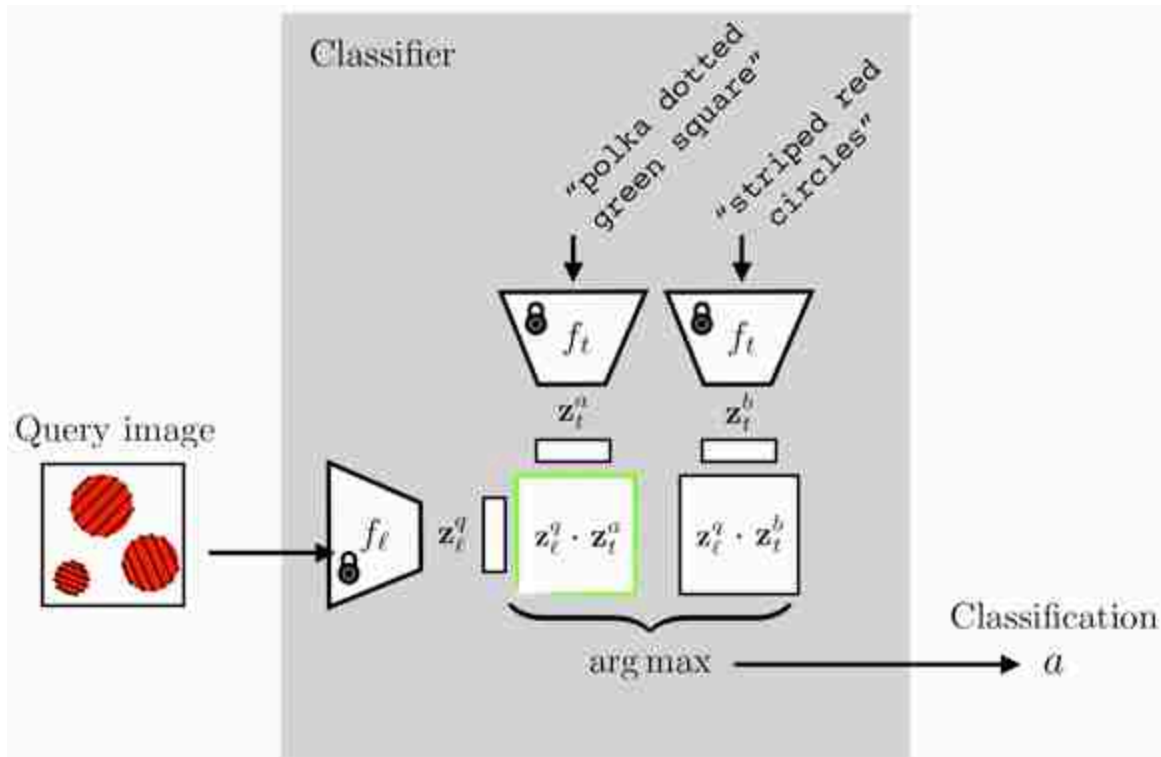


Figure 51.5: Making a “striped red circles” (class a) versus “polka dotted green square” (class b) classifier using a trained CLIP. The CLIP model parameters are frozen and not updated. Instead the classifier is constructed by embedding the text describing to the two classes and seeing which is closer to the embedding of the query image. Inspired by figure 1 of [394].

[Figure 51.6](#) gives more examples of creating custom binary classifiers in this way. The two classes are described by two text strings (“triangle” versus “cricle”; “purple” versus “teal”; and “arrowhead” versus “ball”). The embeddings of the text descriptions are the red and green vectors. The classifier simply checks where an image embedding is closer, in angular distance, to the red vector or the green vector. This approach isn't limited to binary classification: you can add more text vectors to partition the space in more ways.

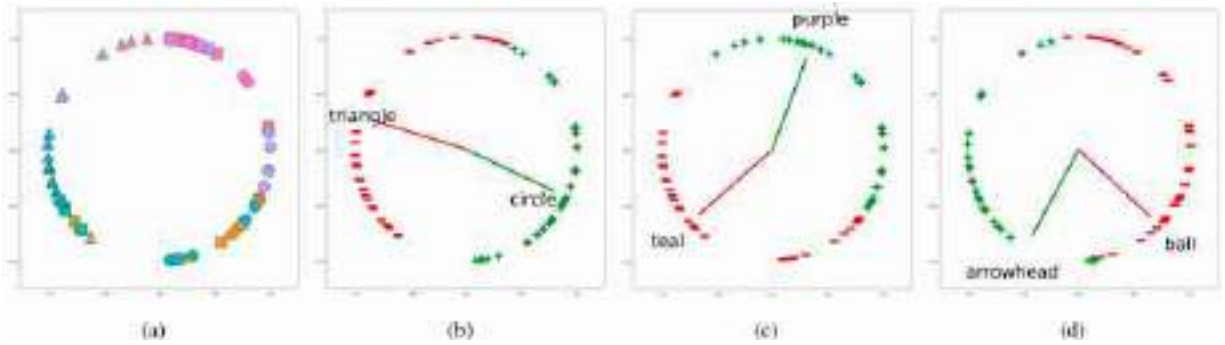


Figure 51.6: Making custom classifiers using CLIP. (a) Image embeddings in joint embedding space, (b) triangle-circle classifier, (c) purple-teal classifier, and (d) arrowhead-ball classifier. It's a bit like a modern astrolabe where you turn dials to read off the prediction of interest.

51.4 Translating between Images and Text

In chapter 34, we saw how to translate between different image domains. Could we be even more ambitious and translate between entirely different data *modalities*? It turns out that this is indeed possible and is an area of growing interest. For any type of data X , and any other type Y , there is, increasingly, a method $X2Y$ (and $Y2X$) that translates between these two data types. Go ahead and pick your favorite data types— X, Y could be images and sounds, music and lyrics, amino acid sequences and protein geometry—then Google “ X to Y with a deep net” and you will probably find some interesting work that addresses this problem. $X2Y$ systems are powerful because they forge links between disparate domains of knowledge, allowing tools and discoveries from one domain to become applicable to other linked domains.

One approach to solving X -to- Y translation problems is to do it in a modular fashion by hooking up an *encoder* for domain X to a decoder for domain Y . This approach is popular because powerful encoder and decoder architectures, and pretrained models, exist for many domains. If we have an encoder $f: X \rightarrow Z$ and a decoder $g: Z \rightarrow Y$, then we can create a translator simply by function composition, $g \circ f$.

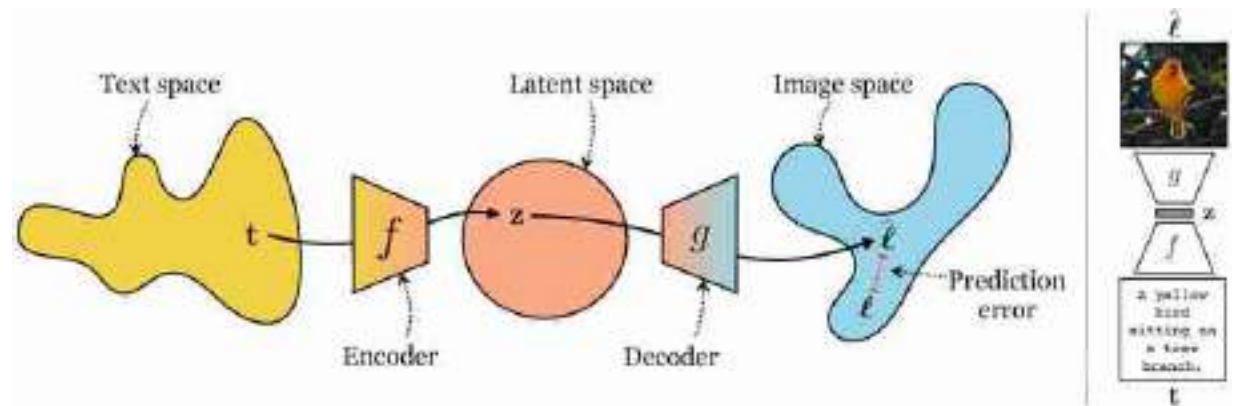
Of course, the output of our encoder might not be interpretable as a proper input to the decoder we selected. For example, suppose f is a text encoder and g is an image decoder and we have $\ell = g(f(t))$, where ℓ is a generated image from the text input (t). Then nothing guarantees that ℓ semantically matches t .

To *align* the encoder and decoder we can follow several strategies. One is to simply finetune $g \circ f$ on paired examples of the target translation problem, $\{\mathbf{t}^{(i)}, \ell^{(i)}\}_{i=1}^N$. Another option is to freeze f and g and instead train a translation function in latent space, $h : Z \rightarrow Z$, so that $g \circ h \circ f$ correctly maps each training point $\mathbf{t}^{(i)}$ to the target output $\ell^{(i)}$. See [409] for an example of this latter option.

In the next sections, we will look at a few specific architectures for solving this problem of text-to-image translation as well as the reverse, image-to-text.

51.4.1 Text-to-Image

One amazing ability of modern generative models is to synthesize an image just from a text description, a problem called **text-to-image**. This can be achieved using any of the conditional generative models we saw in chapter 34. We simply have to find an effective way to use text as the conditioning information to these models. [Figure 51.7](#) shows how this problem looks as a mapping from the space of text to the space of images:



[Figure 51.7:](#) Text-to-image.

On the right is an example of such a system translating input text $\mathbf{t}$ to an output image ℓ that matches what the text describes. Recall the similar diagrams in the chapters on representation learning and generative modeling and think about how they relate (see figures 30.2 and 33.2). This new problem is not so different from what we have seen before!

Now let's build some concrete architectures to solve this mapping. We will start with a conditional variational autoencoder (cVAE; section 34.4.2). This was the architecture used by the DALL-E 1 model [398], which was one of the first text-to-image models that caught the public's imagination.

A cVAE for text-to-image can be trained just like a regular cVAE; the special thing is just that the encoder is a text encoder, that is, a neural net that maps from text to a vector embedding. [Figure 51.8](#) shows what the training looks like; it's conceptually the same as figure 34.5, which we saw previously.

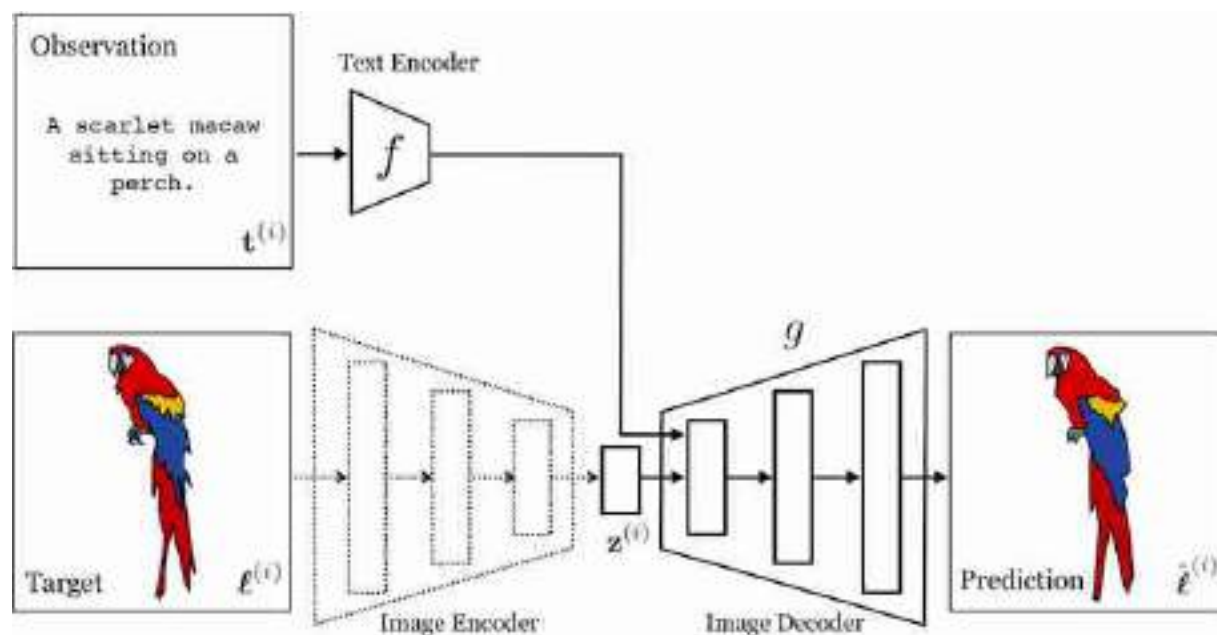


Figure 51.8: Training a cVAE to perform text-to-image. The dotted lines indicate that the image encoder is only used during training; at test time, usage follows the solid path. In this cartoon we draw the output as slightly distorted to reflect that the autoencoder path might not be able to perfectly reconstruct the input.

After training this model, we can sample from it by simply discarding the image encoder and instead inputting a random Gaussian vector to the decoder, in addition to providing the decoder with the text embedding. [Figure 51.9](#) shows how to sample an image in this way.

This is how text-to-image generation can be done with a cVAE. We could perform the same exercise for any conditional generative model. Let's do one more, a conditional diffusion model. This is the approach followed

by the second version of the DALL-E system, DALL-E 2 [\[397\]](#), as well as other popular systems like the one described in [\[409\]](#). The difficulty we will focus on here is as follows: how do you most effectively insert the text embeddings into the the decoder, so that it can make best use of this information? [Figure 51.10](#) shows one way to do this, which is roughly how it is done in [\[409\]](#): we insert text embedding vectors into each denoising U-Net, and even into multiple layers of each U-Net. This makes it very easy for the diffusion model to condition all of its decisions on the text information.

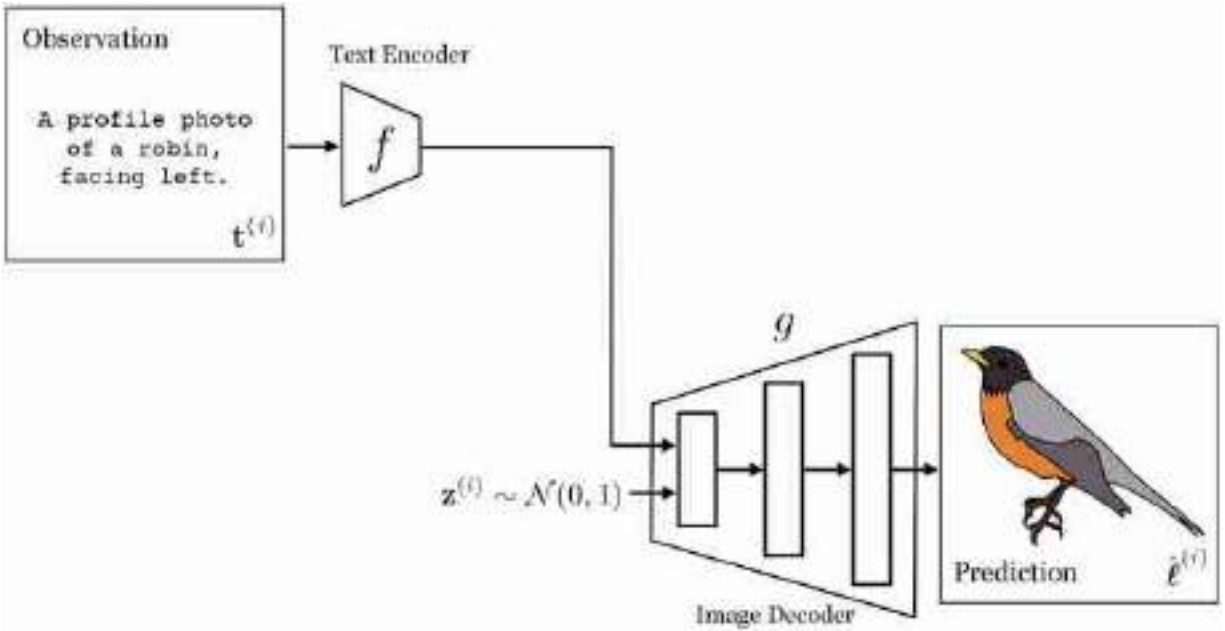


Figure 51.9: Sampling from a trained cVAE.

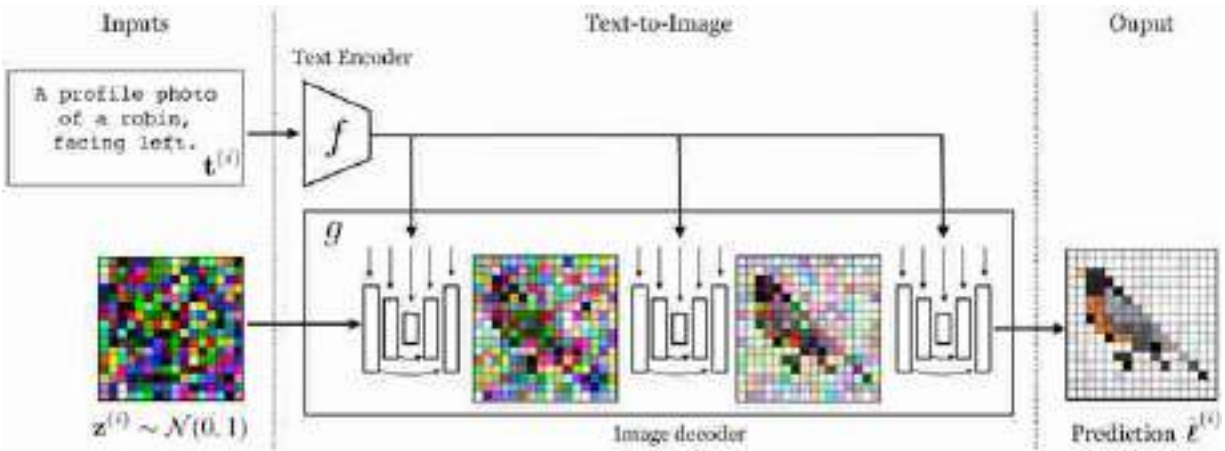


Figure 51.10: A text-to-image diffusion model.

Notice that [figure 51.10](#) is actually very similar to [figure 51.9](#). A conditional diffusion model is in fact very similar to a cVAE. At sampling time, both take as input text and Gaussian noise. The VAE uses a network that directly maps these inputs to an image, in a single forward pass. The diffusion model, in contrast, maps to an image via a chain of denoising steps, each of which is a forward pass through a neural network. See [\[259\]](#) for a discussion of the connection between diffusion models and VAEs.

51.4.2 Image-to-Text

Naturally, we can also translate in the opposite direction. We just need to use an *image encoder* (rather than a text encoder) and a *text decoder* (rather than a text encoder). In this direction, the problem can be called **image-to-text** or **image captioning**. This kind of translation is very useful because text is a ubiquitous and general-purpose interface between almost all domains of human knowledge (we use language to describe pretty everything we do and everything we know). So, if we can translate images to text then we link imagery to many other domains of knowledge.

As an example of an image-to-text system, we will use a transformer to map the input image to a set of tokens, and we will then feed these tokens as input to an autoregressive text decoder, which is also implemented with a transformer. See [\[295\]](#) for an example of a system that uses this general approach. [Figure 51.11](#) shows this architecture.

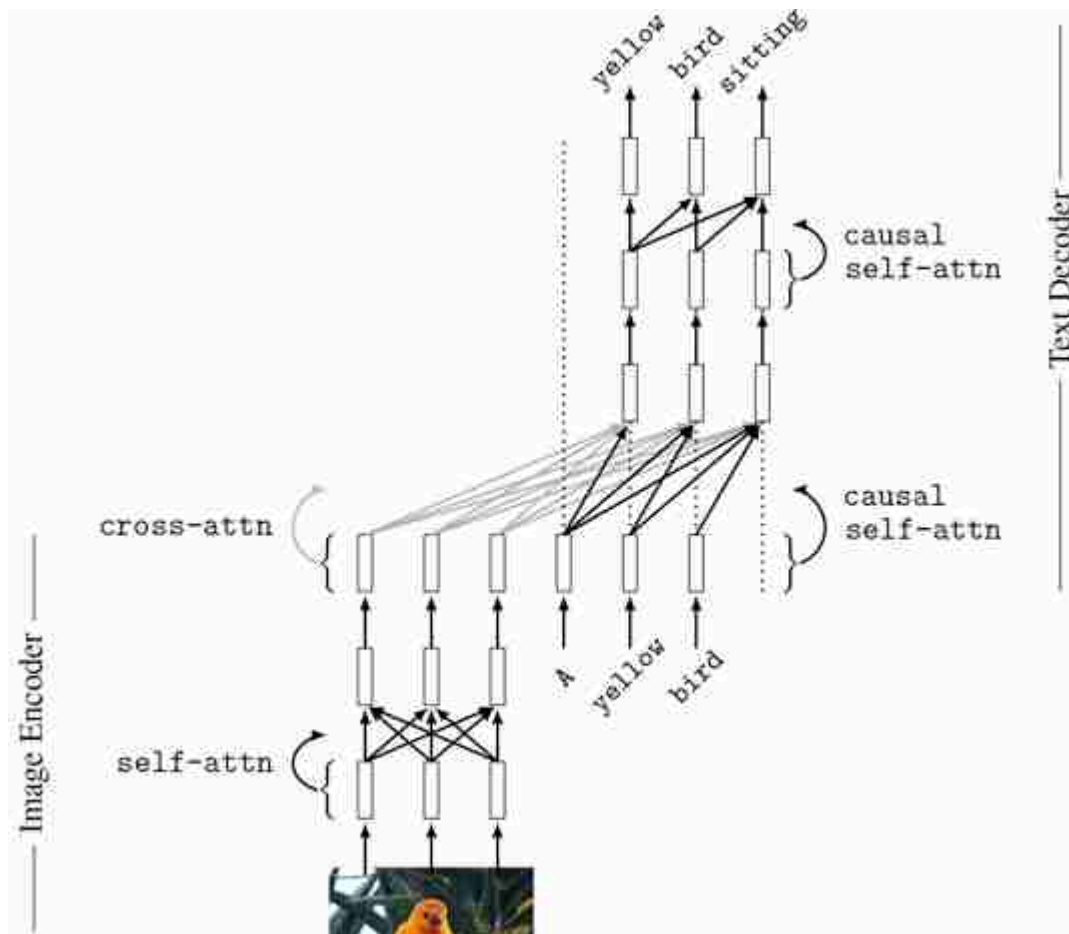


Figure 51.11: Image-to-text using a small transformer with cross-attention. The edges in gray are cross-attention edges. Many variations of this architecture are possible. For example, each layer of the text encoder could cross-attend to the output image tokens. Or, the image encoder could compress all its input tokens into a single output token that represents the entire photo; this output token would then get cross-attended by the text decoder. This approach could save memory and computation time.

There is one special layer to point out, the **cross-attention** layer (**cross-attn**). This layer addresses the question of how to insert image tokens into a text model. The answer is quite simple: just let the text tokens attend to the image tokens. The image tokens will emit their value vectors which will be mixed with the text token value vectors according to the attention weights to produce the next layer of tokens in the text decoder.

Here are the equations for a cross-attention layer:

$$\mathbf{Q}_t = \mathbf{T}_t \mathbf{W}_{t,q}^\top \quad (51.1)$$

$$\mathbf{K}_\ell = \mathbf{T}_\ell \mathbf{W}_{\ell,k}^\top \quad (51.2)$$

$$\mathbf{V}_\ell = \mathbf{T}_\ell \mathbf{W}_{\ell,v}^\top \quad (51.3)$$

$$\mathbf{A}_{\text{cross}} = \text{softmax}\left(\frac{\mathbf{Q}_t \mathbf{K}_\ell^\top}{\sqrt{m}}\right) \quad (51.4)$$

$$\mathbf{T}_{\text{out,cross}} = \mathbf{A}_{\text{cross}} \mathbf{V}_\ell \quad (51.5)$$

where variables subscripted with t are associated with text tokens and variables subscripted with ℓ are associated with image tokens.

Then, the tokens output by cross-attention can be combined with tokens output by self-attention by weighted summation:

$$\mathbf{T} = w_{\text{cross}} \mathbf{T}_{\text{out,cross}} + w_{\text{self}} \mathbf{T}_{\text{out,self}} \quad (51.6)$$

where $\mathbf{T}_{\text{out,self}}$ is regular self-attention on the input text token $\mathbf{T}_t$, and w_{cross} and w_{self} are optional learnable weights.

The text decoder is an autoregressive model (section 32.7) that uses causal attention (section 26.10) to predict the next word in the sentence given the previous words generated so far. In this strategy, the input to the text decoder is the tokens representing the previous words in the training data caption that describes the image. At sampling time, the model is run sequentially, where for each next word generated it autoregressively repeats, with that word being added to the input sequence of tokens. The text tokens themselves are produced by tokenizing the input words, for example, via byte-pair encodings or word-level encodings (the latter is shown in the figure).

51.5 Text as a Visual Representation

One way of thinking about image-to-text is as yet another visual encoder, where text is the visual representation. Just like we can represent an image in terms of its frequencies (Fourier transform), its semantics and geometry (scene understanding), or its feature embeddings (DINO [70], CLIP [394], and others), we can also represent it in terms of a textual description of its content.

In [figure 51.12](#) we show an image represented in several ways (the depth map is produced by MiDaS [401] and the text is produced by BLIP2 [295]):

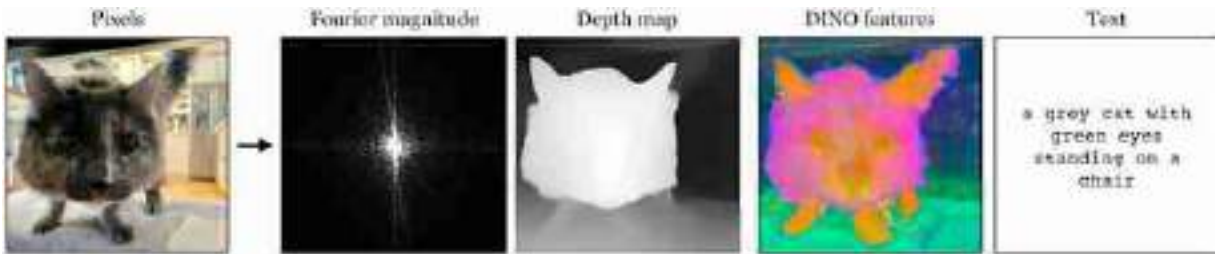


Figure 51.12: Four representations computed from the input on the left. The input image itself (its pixel values) is in fact a fifth representation of the scene.

Think about the relative merits of each of these representations, and what advantages or disadvantages text might have over these other kinds of visual representation.

Text is a powerful visual representation because it interfaces with language models. This means that after translating an image into text we can use language models to reason about the image contents.

51.6 Visual Question Answering

As AI systems become more general purpose, one goal is to make a vision algorithm that is fully general, so that you can ask it any question about an image. This is akin to a Turing test [478] for visual intelligence. **Visual question answering (VQA)** is one formulation of this goal [17]. The problem statement in VQA is to make a system that maps an image and a textual question to a textual answer, as shown in [figure 51.13](#).



Figure 51.13: An example VQA system.

By now you have seen several ways to build image and text encoders as well as decoders, and how to put them together. As an exercise, think about how you might implement the VQA system shown here, using the tools from elsewhere in this chapter and book. See if you can modify the transformer architecture in [figure 51.11](#) to solve this problem.

In this chapter, we have already seen all the tools necessary to create a VQA system; you just need a text encoder (for the question), an image encoder (for the image you are asking the question about), and a text decoder (for the answer). These can be hooked together using the same kinds of tools we have seen earlier in this chapter. The critical piece is just to acquire training data of VQA examples, each of which takes the form of a tuple $\{[\text{question}, \text{image}], \text{answer}\}$. For a representative example of how to implement a VQA model using transformers, see [\[295\]](#).

51.7 Concluding Remarks

From the perspective of vision, language is a perceptual representation. It is a way of describing “what is where”^{[23](#)} and much more in a low-dimensional and abstracted format that interfaces well with other domains of human knowledge. There are many theories of how human language evolved, and what it is useful for: communication, reasoning, abstract thought. In this chapter, we have suggested another: language is, perhaps, the result of perceptual representation learning over the course of human evolution and cultural development. It's one of the main formats brains have arrived at as the best way to represent the “blooming, buzzing”^{[24](#)} sensory array around us.

[22.](#) The idea of visual learning by image captioning actually has a long history before CLIP. One important early paper on this topic is [\[365\]](#).

[23.](#) “To know what is where by looking” is how David Marr defined the vision in his seminal book on the subject [\[317\]](#).

[24.](#) This phrasing is from a quote by William James (https://en.wikiquote.org/wiki/William_James).

XIV

ON RESEARCH, WRITING AND SPEAKING

The following chapters contain general advice for students and researchers. This advice is not limited to computer vision, nor is it limited to researchers. It is our personal perspective on skills that a good computer vision practitioner should have. However, as in all advice, you should always get multiple opinions and choose the one that helps you become the most successful student, researcher, or engineer. See, e.g., the excellent videos from this 2018 workshop [[375](#)] from the Computer Vision and Pattern Recognition Conference (CVPR).

While there are many topics important to a researcher's career that we don't cover (how to review papers, mentor and advise, and manage collaborations), we hope that these chapters will be useful to both students, researchers, and engineers.

- **Chapter 52** presents actionable tips for doing research, especially for those just starting out.
- **Chapter 53** gives our tips for writing and organizing your research manuscripts.
- **Chapter 54** provides some tips for giving engaging talks and managing nervousness. Feel free to read these chapters at any moment and in any order.

52 How to Do Research

52.1 Introduction

The jump from problem sets into research can be hard. Sometimes we see students who ace their classes but struggle with their research. In little bites, here is what we think is important for succeeding in research as a graduate student. This chapter is written as advice from research advisor to a graduate student (so we use the first person in the text), but we hope that this advice will be useful for anyone learning how to create and debug research or engineering projects.

52.2 Research Advice

The first piece of advice can go on a bumper sticker: **“Slow down to speed up.”** In classes, the world is rigged. There's a simple correct answer and the problem is structured to let you come to that answer. You get feedback with the correct answer within a day after you submit anything.

Research is different. No one tells you the right answer, and we may not know if there is a right answer. We don't know if something doesn't work because there's a silly mistake in the program or because a broad set of assumptions is flawed.

How do you deal with that? Take things slowly. Verify your assumptions. Understand the thing, whatever it is—the program, the algorithm, or the proof. As you do experiments, only change one thing at a time, so you know what the outcome of the experiment means.

It may feel like you're going slowly, but you'll be making much more progress than if you flail around, trying different things, but not understanding what's going on.



Figure 52.1: Research advice for a bumper sticker.

Please don't tell me "it doesn't work." Of course, it doesn't work. If there's a single mistake in the chain, the whole thing won't work, and how could you possibly go through all those steps without making a mistake somewhere? What I want to hear instead is something like, "I've narrowed down the problem to step B. Until step A, you can see that it works, because you put in X and you get Y out, as we expect. You can see how it fails here at B. I've ruled out W and Z as the cause."

"This sounds like hard work." Yes. It's no longer about being smart. By now, everyone around you is smart. In graduate school, it's the *hard workers* who pull ahead. This happens in sports, too. You always read stories about how hard the great players work, being the first ones out to practice, the last ones to leave, and so on.

A co-author, who generally works harder than I do, tells me I should add comments here about the importance of taking time off from work and maintaining a good life/work balance. That is true: you should work at a pace you can sustain, and you should refresh yourself by making time for the things that matter more than work, such as relationships, service, and relaxation. Perhaps the point is to protect your time so you will have enough time to do the research well.

"How do I get myself to work hard enough to do research well?" It all plays out if you love what you're doing. You become good at it because you

spend time at it and you do that because you enjoy it. So pick something to work on that you can love. If you're not the type who falls in love with a problem, then just know that working hard is what you have to do to succeed at research.

The above isn't completely true. Beyond working hard, there's also *steering*. We're like boats. We need motors—that's the part about working hard. But we also need a rudder for steering—that means stepping back periodically to make sure we're working on the right thing. On the topic of steering, I find time management books to be very helpful. They teach you how to spend your time solving the right problems.

Toy models. There's a concept I want a simple phrase for, and maybe you can help me think up a good name. It's “the simplest toy model that captures the main idea” (TSTMTCTMI). Anyway, simple toy models always help me. With a good one, you can build up intuition about what matters, which is a big advantage in research.

Here's an example. The color constancy problem is to estimate surface reflectance colors when we only get to observe the wavelength-by-wavelength product of the each surface reflectance spectrum and the unknown illuminant spectrum. A toy model for that problem is to try to estimate the scalars a and b from only observing their product, $y = a \times b$. There's a surprising richness even to this simple problem, and thinking about it allows you to think through loss functions and other aspects of Bayesian decision theory ([figure 52.2](#)). I co-authored a paper that discusses $y = a \times b$ for much of the manuscript [[143](#)]. Another toy model is to consider, as a proxy for complicated shaded surfaces, a single bump. You get the idea.

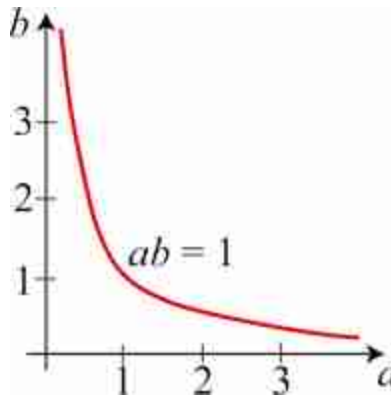


Figure 52.2: A toy model for the color constancy problem: $y = a \times b$. The plot shows the possible solutions for a and b when $y = 1$.

Having the intuitions from working with toy problems gives you a big advantage in the research, because you can figure out what will work by thinking it through with your toy model.

Strategies for success. Here is a parable, as told by my friend Yair Weiss. There is a weak and a strong graduate student. They are both asked by their advisor to try a particular approach to solving a research problem. The weak student does exactly what the advisor has asked. But the advisor's solution fails, and the student reports that failure. The strong student starts doing what the advisor has asked, sees that it doesn't work, looks around within some epsilon ball of the original proposal to find what does work, and reports that solution.



Figure 52.3: The parable of the two students.

Sometimes it's useful to think that everyone else is completely off-track. This lets you do things that no one else is doing. It's best not to be too vocal about that. You can say something like, “Oh, I just thought I'd try out this direction.”

It's also sometimes useful to remember that many smart people have worked on this and related problems and written their thoughts and results down in papers. Don't be caught flat-footed with a large body of closely related literature that you aren't familiar with.

How a business school might talk about your research. You have a brand: you. There are many impressions you want to build up about your brand: that person always does great work, they have good ideas, they give great talks, they write wonderful software. Promote your brand. Build up a great reputation for yourself by consistently behaving in the way you'd like to be thought of.

Cultivate your strengths and play to those strengths. Some possible strengths include being broad, being creative, being a great implementer, or being great at doing theory.

Please don't report to me and say, “This instance doesn't work.” Why doesn't it work? Why should it work? Is there a simpler case we can make work? Do you think it's a general issue that affects all problems of this

category? Can you think of what's not working? Can you contort things to make an example that does work? At the very least, can you make it fail worse, so we understand some aspects of the system?

Progress. I love to hear about progress when I meet with students, but note that I have a very general notion of progress. Progress can include things like “I've shown why this doesn't work,” “I've simplified the task to get it to start working,” or “I spent the whole time reading because I know I have to understand this before I can make any progress.”

Please don't hide from me. Let's talk. I like it when you track me down and insist that we talk, for example, if I've been traveling.

On collaboration. Science is generally a team sport. What matters in a collaboration is whether the work is insightful, foundational, or impactful. It doesn't matter that many people were involved. It is much better to be one of 10 contributors to a great paper than to be the sole author on an unimportant paper. The relationships formed through collaborations can become friendships that last over your career.

On authorship. What names should be on a paper? Everyone who contributed to the paper. Contributions can be through experimentation, contributing a seminal idea that the paper builds on, or helping with the writing. Sometimes an authorship contribution can include trying out a research direction that didn't work. If I'm on the fence about whether someone contributed enough to be an author, I usually ask the person themselves, and go with whichever outcome they prefer.

52.3 Concluding Remarks

For a presentation to the visiting admitted MIT computer science graduate students, I emailed all MIT Computer Science and Artificial Intelligence Laboratory (CSAIL) researchers and faculty members, asking “Please send me what you think is the most important quality for success in graduate school.” I compiled their responses (along with photos of the responders) into slides that are available online:

<http://people.csail.mit.edu/billf/talks/10minFreeman2013.pdf>

I think it's a lot of good advice about research.

One final note about doing research. We hope you love it. We certainly do. The research community is a community of people who are passionate about what they do, and we welcome you to it!

53 How to Write Papers

53.1 Introduction

An important part of research is communicating your results to others, so we include our thoughts about how to write good papers. A video presentation of material related to this chapter is [\[146\]](#).

Many graduate students we know feel it is important to coauthor many papers. While the number of papers a student has can be a rough measure of research productivity, it is our experience as mentors, faculty search committee members, and industrial research managers that *only the good papers count*. (We acknowledge that this is not true everywhere, and some institutions simply count papers. But you should still strive to make all your papers good!) This emphasis is shown in [figure 53.1](#), a plot of made-up data that summarizes our impression, formed over many years, of the relationship between paper quality and its impact on one's career. Only the really good papers matter for your career. So it makes sense to learn how to write the best possible paper from any given piece of research work.

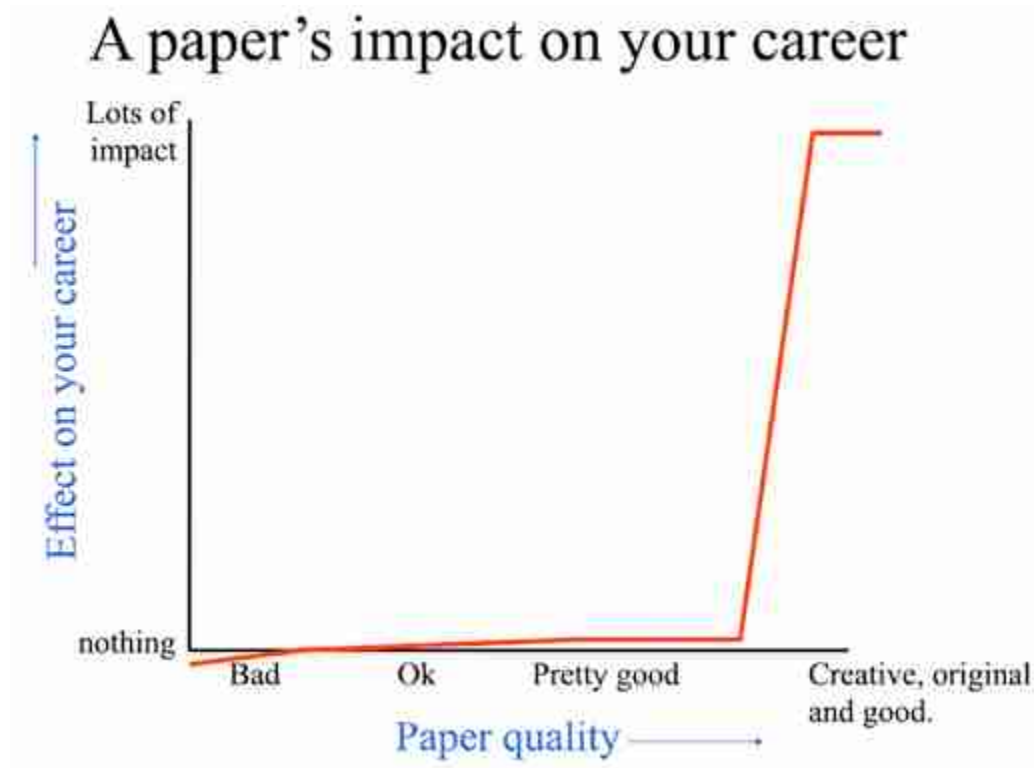


Figure 53.1: Plot of conjectured data showing the impact of a paper on one's career, as a function of paper quality.

53.2 Organization

The Ph.D. thesis advisor of two of us, Prof. Edward Adelson, wrote some good advice in response to a student's question of how to write a good paper [5]:

- Start by stating which problem you are addressing, keeping the audience in mind. They must care about it, which means that sometimes you must tell them why they should care about the problem.
- Then state briefly what the other solutions are to the problem, and why they aren't satisfactory. If they were satisfactory, you wouldn't need to do the work.
- Then explain your own solution, compare it with other solutions, and say why it's better.
- At the end, talk about related work where similar techniques and experiments have been used, but applied to a different problem.

That structure fits well with a progression of section headings typical of many conference papers. As an example, here are the headings from paper [125], coauthored by one of us. The structure is useful for many research papers:

1. Introduction
2. Related work
3. Main idea
4. Algorithm
 - 4.1 Estimating the blur kernel
 - 4.1.1 Multiscale approach
 - 4.1.2 User supervision
 - 4.2 Image reconstruction
5. Experiments
 - 5.1 Small blurs
 - 5.2 Large blurs
 - 5.3 Images with significant saturation
6. Discussion

53.2.1 The Paper's Introduction

Regarding paper introductions, Kajiya [246] writes, “You must make your paper easy to read. You've got to make it easy for anyone to tell what your paper is about, what problem it solves, why the problem is interesting, what is really new in your paper (and what isn't), why it's so neat. And you must do it up front. In other words, you must write a dynamite introduction.”

53.2.2 Main Idea

When appropriate to the paper, it can be very helpful to show a simple example that captures the main idea of the paper. Here is a figure from a different paper [442] that conveys the main idea very simply. The paper's main idea was that wavelets lacked important desirable features for an image representation. [Figure 53.2](#) shows the failure of a wavelet representation to form a translation invariant representation. The top row shows two versions of the same signal, differing only in a one-sample translation. The bottom three rows show the coefficients of the high-, mid-,

and low-frequency bands of a wavelet representation of that signal. The figure points out in a simple way the drawback of a wavelet representation with aliased subbands that was the focus of the paper: as the signal translates, the signal representation energy moves to different frequency bands and changes form within the mid-frequency band.

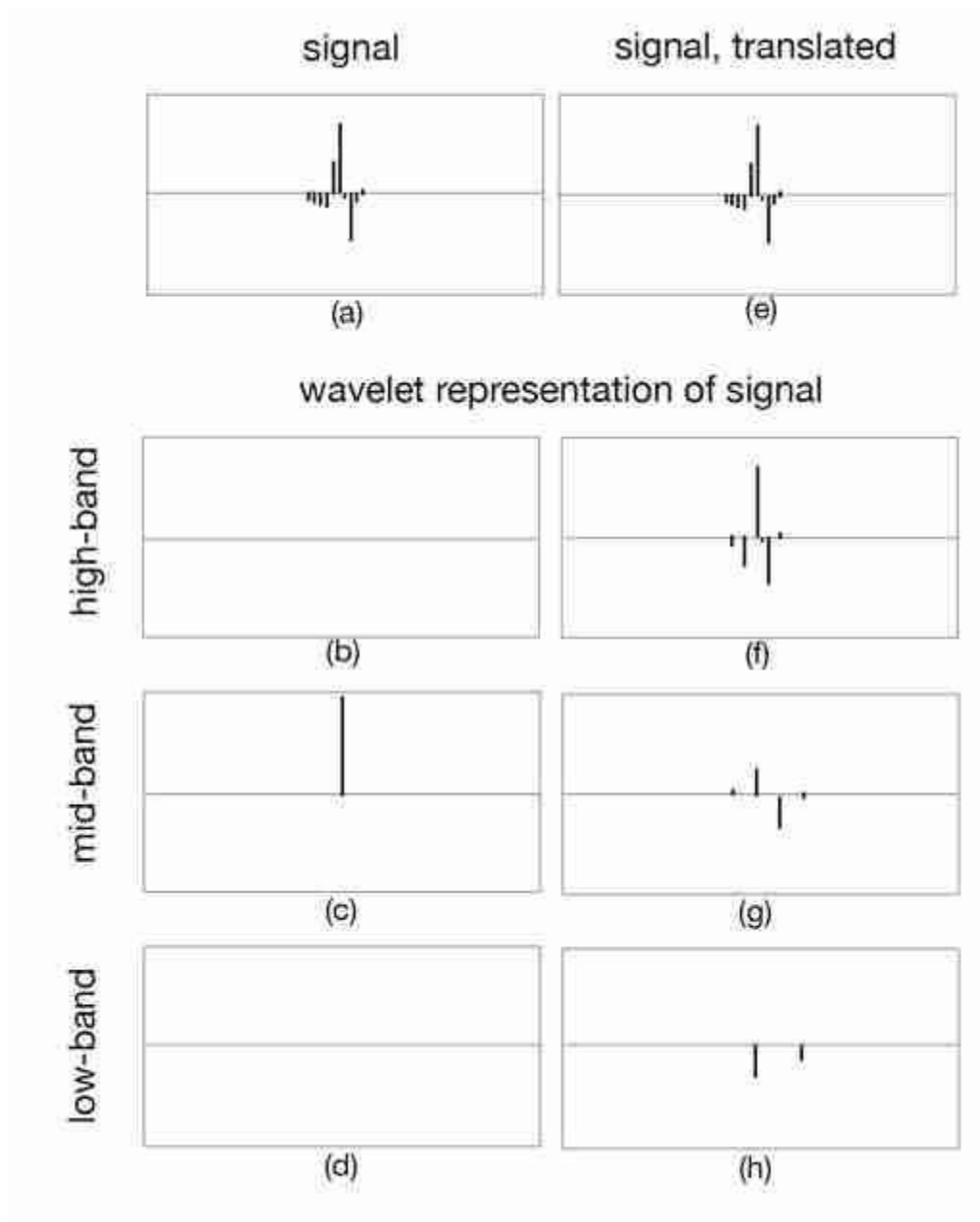


Figure 53.2: Effect of translation on the wavelet representation of a signal (caption and figure after [442]). (a) An input signal. (b-d) The high-, mid-, and low-frequency coefficients within a wavelet subband decomposition. (e) The same input signal, translated one sample to the right. (f-h) The decomposition of the shifted signal into the same three bands. Note the drastic change in the transform coefficients induced by the shift.

53.2.3 Experimental Results

In earlier days of computer vision research, good ideas could be presented with only plausibility arguments to support them. However, modern standards require that assertions in a paper be backed up with experimental

evidence. Readers will be expecting a table quantitatively comparing the performance of your algorithm with that of the previous state-of-the-art. If you are proposing a new task for which there aren't previous benchmarks, you should still compare with an algorithm that solves a related task, noting that the algorithm was designed for a different task.

Psychophysical study		Loudness		Centroid	
Algorithm	Labeled <i>real</i>	Err.	<i>r</i>	Err.	<i>r</i>
Full system	40.01% $\pm$ 1.66	0.21	0.44	3.85	0.47
- Trained from scratch	36.46% $\pm$ 1.68	0.24	0.36	4.73	0.33
- No spacetime	37.88% $\pm$ 1.67	0.22	0.37	4.30	0.37
- Parametric synthesis	34.66% $\pm$ 1.62	0.21	0.44	3.85	0.47
- No RNN	29.96% $\pm$ 1.55	1.24	0.04	7.92	0.28
Image match	32.98% $\pm$ 1.59	0.37	0.16	8.39	0.18
Spacetime match	31.92% $\pm$ 1.56	0.41	0.14	7.19	0.21
Image + spacetime	33.77% $\pm$ 1.58	0.37	0.18	7.74	0.20
Random impact sound	19.77% $\pm$ 1.34	0.44	0.00	9.32	0.02

(a) Model evaluation

Figure 53.3: A prototypical table of results, from [369] Rows are different algorithms, or ablations of a favored algorithm. Columns are datasets or tasks, and the numbers indicate performance, with the best performances given in bold.

53.2.4 How to End the Paper

A cautionary note on how *not* to end the paper. Conference papers are often written under deadline pressure, and there is inevitably a list of results that the authors wanted to obtain that have been left uncompleted. You should resist the temptation to describe the items that were left unfinished in a section on “Future Work.” It is hard to imagine a weaker way to end the paper than to enumerate all the things the paper doesn't accomplish, providing a summary of where it falls short.

Instead, the paper can finish with a review what has been presented, emphasizing the contributions. How has the world changed, now that the reader has read this paper? What new research directions have been opened up; what can we do now that we couldn't do before?

53.3 General Writing Tips

Donald Knuth, the author of the classic book series, *The Art of Computer Programming*, wrote [267], “Perhaps the most important principle of good writing is to keep the reader uppermost in mind: What does the reader know so far? What does the reader expect next and why?”

Our mental image is that of a great host, who anticipates every need of their house guest: after you arrive, they say, “Can I take your coat? Would you like something to drink?” At every moment, they know what you might be wanting and how to help you find it.

53.3.1 Use Fewer Words

In *The Elements of Style* [455], Strunk and White write, “Vigorous writing is concise. A sentence should contain no unnecessary words, a paragraph no unnecessary sentences, for the same reason that a drawing should have no unnecessary lines and a machine no unnecessary parts.”

To make this point in the context of scientific writing, the following is a paragraph that one of the authors of this textbook wrote with a student.

The underlying assumption of this work is that the estimate of a given node will only depend on nodes within a patch: this is a locality assumption imposed at the patch-level. This assumption can be justified in case of skin images since a pixel in one corner of the image is likely to have small effect on a different pixel far away from itself. Therefore, we can crop the image into smaller windows, as shown in Figure 5, and compute the inverse J matrix of the cropped window. Since the cropped window is much smaller than the input image, the inversion of J matrix is computationally cheaper. Since we are inferring on blocks of image patches (i.e., ignoring pixels outside of the cropped window), the interpolated image will have blocky artifacts. Therefore, only part of xMAP is used to interpolate the image, as shown in Figure 5.

Original text: 149 words.

While the text above conveys the desired technical information, it describes things in a roundabout way. The word count is 149. Below, the paragraph has been rewritten to just 88 words without changing its meaning.

We assume local influence—that nodes only depend on other nodes within a patch. This condition often holds for skin images, which have few long edges or structures. We crop the image into small windows, as shown in Fig. 5, and compute the inverse J matrix of each small window. This is much faster than computing the inverse J matrix for the input image. To avoid artifacts from the block processing, only the center region of xMAP is used in the final image, as shown in Fig. 5.

Rewritten text conveying the same information: 88 words

Note that writing more concise text helps your paper in two ways. First, researchers are typically fighting against a page limit, particularly in conference submissions, so tighter text lets the author say more or add another figure. Second, concise text is usually clearer and easier to understand.

53.3.2 Title, Figure Captions, and Equations

While we may imagine our reader reading every word of our paper, in our current era of many published papers, most of your readers will probably only skim your paper. We suspect that the engagement of readers with your paper will look something as shown in [figure 53.4](#).

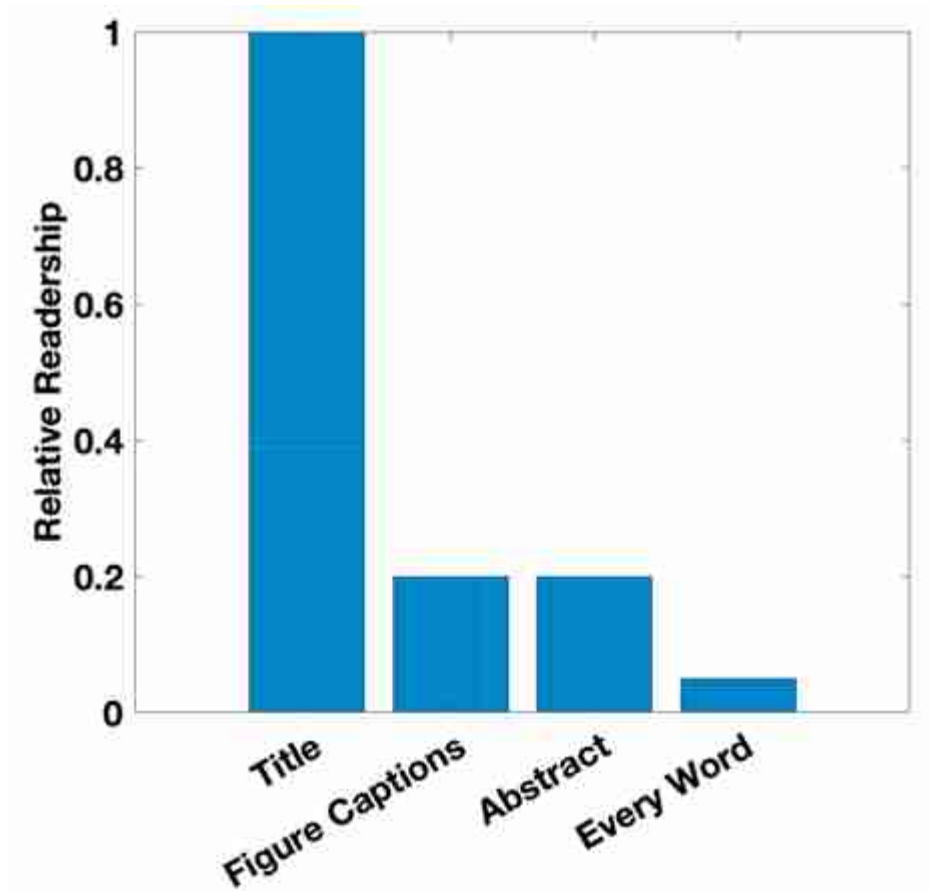


Figure 53.4: The conjectured readership of a computer science paper (conjectured relative amounts). Many people will read the title, but far fewer will read the abstract or look at the figures. Fewer still will read every word of the paper. You should allow readers to access your paper at each level of engagement.

If only because that is where most of your readership lies, you should write your paper so that readers will learn from it, even at those reduced levels of engagement. The title should convey the top-level message of the paper. The figures and their captions should be self-contained and tell the story of the paper. The abstract should provide a good summary.

A good title can generate attention and interest for the paper. “What makes Paris look like Paris” [104] is a delightful paper, and the title lets the reader see that right away. In contrast, one of us coauthored a paper called “Shiftable Multi-scale Transforms.” While that title is appropriate, it's not catchy. After the paper was in print, we realized that we should have added the subtitle, “What's wrong with wavelets?” — a catchy phrase that tells the main point of the paper.

Because you can assume that many of the readers of your figure captions will not be reading the full paper, *the captions should be self-contained*. They should point out what readers are supposed to notice about the figure, to help both the full-paper reader and the figures-only reader. [Figure 53.5](#) shows an example.

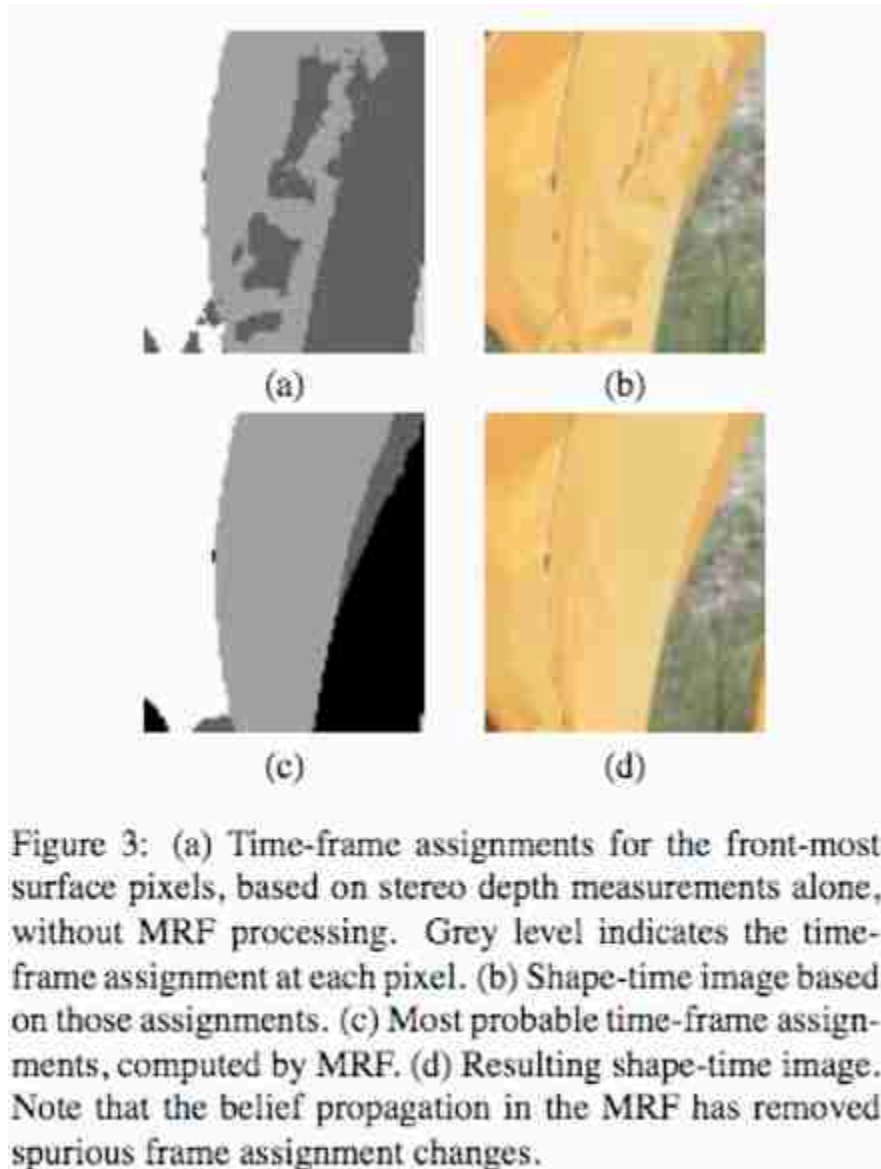


Figure 53.5: Figure and caption from [144]. The caption should be understandable by itself, and should point to what the reader should take away from the figure.

Equations interposed with text can pose a challenge to the reader. Does the reader stop to work through each equation, or continue reading as quickly as if the equation were words? Knuth [267] and Mermin [331] both give advice on writing with equations. Knuth writes, “*Many readers will skim over formulas on their first reading of your exposition. Therefore, your sentences should flow smoothly when all but the simplest formulas are replaced by ‘blah’ or some other grunting noise.*”

Mermin adds, “*When referring to an equation identify it by a phrase as well as a number. No compassionate and helpful person would herald the arrival of Eq. (7.38) by saying ‘inserting (2.47 and (3.51 into (5.13)...’ when it is possible to say ‘inserting the form (2.47 of the electric field E and the Lindhard form (3.51) of the dielectric function ϵ into the constitutive equation (5.13)...’*”

53.3.3 Tone

The tone of your writing is important. You may feel pressure to oversell, hide drawbacks, and disparage others’ work. It is in both your short-term and your long-term interest not to succumb to that pressure! If your work has a shortcoming, it is much better that you point it out than to have someone else do so later.

Papers written from the point of view that “we’re all good researchers, doing our best” are much more pleasant to read than papers written from the standpoint that everyone else is an idiot. Note the delightful sentences written by Efros [114] when he was a graduate student, “*A number of papers to be published this year, all developed independently, are closely related to our work. The idea of texture transfer based on variations of [6] has been proposed by several authors (in particular, see the elegant paper by Hertzmann et al. in these proceedings). Liang et al. propose a real-time patch-based texture synthesis method very similar to ours. The reader is urged to review these works for a more complete picture of the field.*” This generous tone wins over the reader and promotes long-lasting friendships with people who study the same problems you do.

You should develop a reputation for being clear and reliable. Always convey an accurate impression of the performance of an algorithm. If something doesn’t work in important cases, let the readers know. In [125], we noted that our deblurring algorithm didn’t work for a case where we had been sure it should work for a maximum a posteriori (MAP) estimation of the deblurred image. Some years later we learned, and described in [291], that we had been wrong to believe that MAP estimation of the deblurred image was feasible. We were glad we had described our counterintuitive result years before.

53.3.4 Author List

The quality of the papers you write matters so much more than your positions within the author lists of the papers. If adding more authors will make a paper better, then you should add more authors. If a collaborator feels they should be an author, but you yourself are not sure, we find it is generally a good idea to trust the collaborator and to add them as a coauthor. It's much better to be one of many authors on a great paper than to be one of just a few authors on a mediocre paper.

53.3.5 Avoiding Rejection

With the task of rejecting at least three-quarters of the submissions, conference area chairs, who make the acceptance decisions, are grasping for reasons to reject a paper. Here's a summary of reasons that are commonly used, and it is best not to provide the area chair with any of these easy reasons to reject your paper:

- Do the authors not deliver what they promise?
- Are important references missing (and therefore one suspects the authors not up on the state of the art for this problem)?
- Are the results too incremental (too similar to previous work)?
- Are the results believable, with sufficient tests to support the claims?
- Is the paper poorly written?
- Are there mistakes or incorrect statements?

As an area chair, the very good and the very bad papers are easy to deal with, and most of the decision-making effort goes to the borderline papers. One of us writes, “I find that much of my time is spent dealing with two kinds of borderline papers: cockroaches and puppies with six toes.”

“Cockroaches” have no exciting contributions, but the reviews are ok and the results show incremental improvement to the state of the art. They're hard to kill (hence the nickname), and maybe two-thirds are accepted as posters, and one-third are rejected. As an author, it's best to work harder for the fresh results, to bring papers like this out of the cockroach category.

At the other end of the spectrum of borderline papers are “puppies with six toes.” These are delightful papers, but with an easy-to-see flaw. The flaw may not matter (like six toes on a puppy) but makes the paper easy to reject, even though the paper may be fresh and wonderful. Maybe two-

thirds of those papers get rejected, and one-third maybe accepted as posters. As an author, it may be better to wait for another submission cycle with a paper like this, to remove the easy to spot flaw. Then the paper might receive an oral presentation at a subsequent conference submission cycle.



Figure 53.6: Borderline papers from an area chair's perspective. (a) The cockroach represents a mundane, hard-to-reject paper with no glaring issues [121]. (b) The six-toed puppy represents delightful paper with some minor, easily spotted flaw, leading to potential rejection [221].

53.4 Concluding Remarks

The approaches to writing and editing described previously all take time: good writing involves lots of rewriting. We find that the last days before a paper submission deadline are often better spent improving the structure, presentation, and clarity of the paper than in trying to obtain last-minute improvements to the results.

54 How to Give Talks

54.1 Introduction

Giving good talks is important for people who use the material of this book—for students, researchers, and engineers. One might think, “It is the work itself that really counts. Giving a talk about it is secondary.” But the ability to give a good talk is like having a big serve in tennis—by itself, it doesn't win the game for you, but it sure helps. And the very best tennis players all have great serves. So we include in this chapter material that we have found helpful in giving talks.

There are many other sources for giving talks, and you should access several of them to pick out the tips that work best for you. Prof. Patrick Winston's talk about giving talks is great [[505](#)], and another good source is this book [[214](#)]. This chapter has heuristics that have worked well for us.

54.1.1 A Taxonomy of Talks

There are different kinds of talks. Conference and workshop talks may range in length between 3 minutes for a “fast-forward” talk to 45 or 60 minutes for a keynote address. You may be one of many speakers, sometimes speaking in different rooms at the same time. Sometimes you will visit a university or company and speak in a special seminar. (Here [[15](#)] is a list of tips regarding faculty job talks from the MIT faculty.)

It is very often the case that the audience will have a wide range of familiarity with the work and topic that you will present about. In a small seminar to a competing research group, there may be many people who are very familiar with your work. But even in such a small seminar, and especially to a larger group, there may be many people who don't know your area, nor perhaps not even your broad area. So you will to include material to let the uninitiated not be lost. Even people who are very familiar with the topic will appreciate hearing material they already understand presented clearly and well.

The tips in the chapter apply to talks of all lengths. However, one class of talks is so short as to be a special case: the very short talk.

54.2 Very Short Talks (2 – 10 Minutes)

Short talks are often advertisements, designed to persuade the listener to read your paper, or to listen to a longer version of the talk. Rather than trying to convey many details about your work, you should aim to have the audience remember the main idea behind your work, and to share your excitement about the work.

For five minute talks describing students' final project class presentations, we suggest that the students cover these points:

1. What problem did you address?
2. Why is it interesting?
3. Why is it hard?
4. What was the key to your approach?
5. How well did it work?

That structure can be used for other short talks, too. For very short talks, the time needed to go through the whole thing is minimal, and there is no excuse not to practise the talk, from start to finish, several times. Next, we cover points that we feel relate to all types of technical talks, regardless of length.

54.3 Preparation



Figure 54.1: The key to a good talk: preparation and practice.

We believe the keys to both a good talk and to overcoming nerves are the same: prepare and practice. Think through your talk. It's almost always better to give a talk from notes than to read it from a script—the audience feels you're speaking to them so it's easier for them to pay attention. But for people giving a talk in a language that they are not comfortable with, writing out a script ahead of time may be a better solution. There may be parts of the talk that you feel are difficult to get through. For those parts, even for a native speaker, it may help to write out what you plan to say. When you give the talk, you may ignore the script, but having written it out once will make it easier to say.

Think through the talk and find the story—how one part relates to the next. If you can't find that story, you may want to reorder the presentation to create a good story.

When you can, you should visit the room you'll be speaking in to identify any issues that may come up. Will you be blocking anyone as you stand at the lectern to give the talk? Will there be someone there to help you

set up? You should decide how to position yourself so that you can see your presentation, and also engage well with the audience.

54.4 Nervousness

One of us writes, “I used to be a nervous public speaker. I still find the hours before giving a presentation to be stressful. I'm always reviewing my talk to try to improve it or to track down the answers to questions I think someone might ask me. The more I prepare, and the better I know the lecture material, the less stressed I am.”

A great cure for nervousness is to practice the talk aloud. Practice by yourself. Practice in front of your friends.

One approach [\[214\]](#) to calming nervousness is to remind yourself, “Get over it. They're not there to see you, they're there to hear the information. Just convey the information to them.”

Some find this quote, attributed to Dr. Rob Gilbert [\[162\]](#), to be helpful: “It's all right to have butterflies in your stomach. Your job is to get them to fly in formation.”

54.5 Your Distracted Audience

Before giving a talk, your mental image of the audience may be of many people, listening to your every word. In reality, on most every speaking situation, your audience is a collection of people who are checking their social media or email, looking on their laptops for other talks to attend, or are hungry or fidgety. How does one speak to such an audience? We have to practise and prepare in order to engage the audience.

54.6 Ways to Engage the Audience

In the middle of Patrick Winston's talk about public speaking [\[505\]](#), he asks the audience the question, “Can you think of a technique to get the audience more engaged in the talk?” The answer, of course, is to ask them questions.

Bill Hoogterp [\[214\]](#) expands on that: “The audience is like a sheep dog, always wanting to be working. Give the audience something to work on while they're listening to you. ‘Four’ pushes people back, while ‘two plus

two’ draws them in.” You might pose a sub-problem to them that you had to address in your work, to get them thinking about how to solve that problem. Then they’ll be more receptive to hear your solution.

54.6.1 Layer Your Talk

It’s often easier for the audience to listen to your talk if you layer it—provide verbal cues for new concepts and transitions between sections of the talk. This helps even the attentive listener to follow the structure of your talk. It keeps the distracted listener from being completely lost. The distracted listener, only listening for the verbal headings of your talk, may hear, “The probability of an observation has three terms to it. ... blah blah blah ... So that gives us the objective function we want to optimize. Now, how do we find the optimal value? There are two approaches you can take. ... blah blah blah So now, with these tools in hand, we can apply this methods to real images. ... blah blah blah ...” Your verbal cues can help even a distracted listener follow your main points.

You can also make your talk easier to follow if you add verbal dynamics—variations in speed or intensity. Find a part of the talk that is particularly special to you and let that show through. One of us writes, “When I gave talks about image deblurring, I would emphasize one aspect that I particularly enjoy: ‘I love this problem; it’s beautifully underdetermined. There are many ways we can explain a blurry image. It could be that that’s what was there in the world— we took a sharp picture of a world that happened to look blurry. Or we took a blurred image of a sharp world.’ ” The audience loves to watch you be excited about something!

54.6.2 People Like to See a Good Fight

As described by Adelson [6], the audience loves to watch a good fight. You can set up a fight between two competing conjectures. For example, you might say, “The flat earth theory predicts that ships will appear on the horizon as small versions of the complete ship. Under that theory, you’d expect approaching ships to appear as in [figure 54.2\(a\)](#). Conversely, the round earth theory predicts that the top of the sails will appear first, then gradually the rest of the ship below it, resulting in approaching ships looking as in [figure 54.2\(b\)](#). The audience waits with anticipation as you show them the result from your experiment, [figure 54.2\(c\)](#), thus revealing the winning theory.

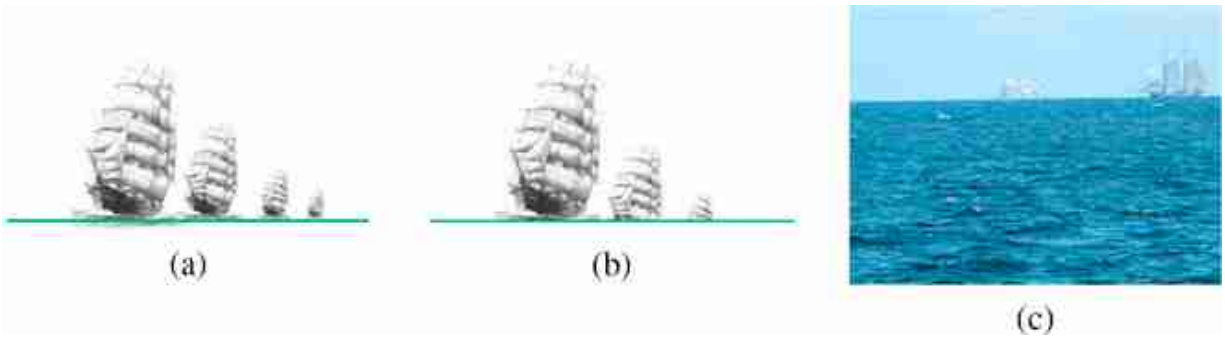


Figure 54.2: (a) The appearance of ships approaching a port, from the flat-earth theory. (b) Their appearance as predicted by the round-earth theory. (c) The experimental evidence: a photograph of ships approaching a port [387].

54.7 Show Yourself to the Audience

In a lovely video, the actor Alan Alda describes the importance of connecting with the audience in scientific presentations [14]. Alda noted the improvement in scientific speaking of volunteers after they engaged in improvisational theater exercises. After the exercises, the volunteers were primed to engage and connect with others, and their scientific talks sparkled.

Here's what we think an audience wants from a technical talk: (a) To have one part follow from another and make sense. (b) To learn a few things. (c) To connect with the speaker, to share their excitement for the topic. They want to watch you love something!

54.7.1 How to End the Talk

How should you end your talk? A common, but awkward, way is to end by asking, “Are there any questions?” The audience wants to clap at the end of a talk, but with that ending, they don't know whether to applaud or to ask a question, and they're likely to give only scattered applause [6]. Better is to close with “Thank you.” The audience then knows it's their time to applaud, and they will. Then you can ask for questions.

We note that Patrick Winston disagrees with ending a talk with “thank you” [505]. He didn't like the implication that the audience was doing you some favor for attending your talk, and preferred to remind the audience of how he has kept his earlier promise to them regarding what they would learn after they listened to his talk.

54.8 Concluding Remarks

Prepare and practice your talks! As you give the talk, let the audience see how much you enjoy what you've worked on. Follow-up with the references in this chapter to find the collection of talk tips that works best for you. Preparation and rehearsal are the best cures we know of for nervousness about giving a talk.

XV

CLOSING REMARKS

The purpose of this book was to cover foundational concepts that, in many cases, have already passed the test of time, and others that we believe are likely to pass it in the future. These concepts should help the reader to understand future advancements beyond what has been covered here.

With all the material presented in this book, we have now the tools to revisit the simple visual system we explored in chapter 2 and try to solve it in a different way to what we proposed earlier, which will conclude this book.

55 A Simple Vision System— Revisited

55.1 Introduction

The goal of this last chapter is to embrace the optimism of the 2020s, when we think that large-scale pretrained models can do it all, and use a pretrained end-to-end vision system in order to solve the simple visual world that we introduced in chapter 2.

In 2020, a number of big pretrained models (DALL-E [\[398\]](#), GPT, chatGPT [\[361\]](#), AutoPilot) that were solving a wide set of tasks (natural language processing, image generation, object classification, depth perception, programming, etc.) emerged. These models are trained with datasets that cover a wide set of diverse domains that improve their generalizations and require massive amounts of computation. They can be used as is, with little to no fine tuning, or as components of more complex systems. Researchers have shown many creative ways of connecting these building blocs to produce astonishing results. However, a close analysis shows that there is still an important gap to bridge in order to build reliable and robust systems. Is the field of computer vision in the 2020s naively optimistic like in the 1960s? It does not seem to be the case, but only time will tell.

In this chapter we will revisit the simple-world problem we analyzed in chapter 2 with a hand-crafted approach by looking for an appropriate high-performing pretrained model and applying it out-of-the-box to solve our task.

55.2 A Simple Neural Network

When we introduced the simple world in chapter 2, our goal was to recover the three-dimensional (3D) structure of the scene from a single image ([figure 55.1](#)).

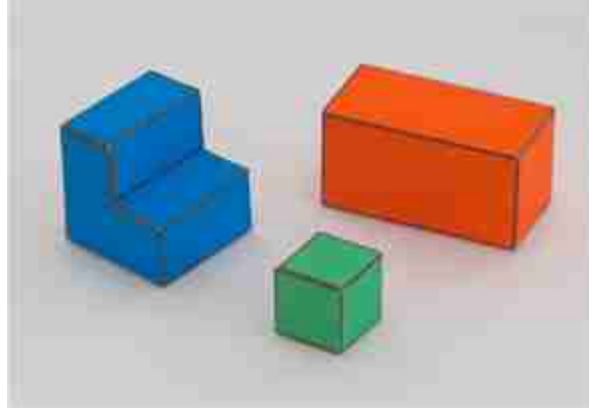


Figure 55.1: Image from the simple world.

We derived an algorithm to recover the 3D scene structure from the first principles: we started studying the image formation process, we then reasoned where the information might be preserved, how to build a representation that makes the information explicit, and finally how to integrate all the evidence left in the image in order to recover the 3D information that was lost during the image formation process.

In this chapter, we will instead download a pretrained model that has been trained to solve the 3D perception problem from single images. The current state of the art is MiDaS [400, 401]. The system is a neural network trained end-to-end using a diverse set of real images that had ground truth depth information and also using stereo images.

Maybe the neural net is not so simple, but at the end, from a user's perspective, it is just a function call. Before running the system out of the box, it is worth understanding how it was trained, what the output means, and how it can be adapted to our task.

55.2.1 How Was It Trained?

This model has been trained using a mix of four datasets with real images. The images below ([figure 55.2](#)) show one sample from each dataset alongside their ground-truth depth.



Figure 55.2: Images and depth maps from the training set. *Source:* Image from [401].

As the model is trained with a diverse collections of data sources, there is an ambiguity on the overall scale, camera parameters, and stereo baselines. Therefore, the loss is defined in a way that it can be invariant to all those ambiguities. The invariant loss makes the training more robust, but it also makes the output less convenient to use.

To deal with the ambiguities mentioned before, the model is trained to predict disparity (inverse depth up to scale and shift). This means that we can not directly use the output. We need to convert it first into world coordinates.

55.2.2 Adapting the Output

MiDaS returns disparity values, up to an unknown constant (d_0), but what we need are world coordinates (X, Y, Z) for each pixel. To perform this transformation, we will use what we learned in chapter 39. We will need to know the intrinsic and extrinsic camera parameters and also decide the value of d_0 .

To translate the returned disparity for a given pixel $d[n, m]$ into depth values $z[n, m]$, we need to invert the disparity after adding the constant:

$$z[n, m] = \frac{1}{d[n, m] + d_0} \quad (55.1)$$

Now we need to recover the world coordinates for every pixel. Assuming a pinhole camera and using the depth $z[n, m]$ and the camera matrix $\mathbf{K}$, we obtain the 3D coordinates for a pixel with image coordinates (n, m) in the camera reference frame as,

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = z[n, m] \times \mathbf{K}^{-1} \times \begin{bmatrix} n \\ m \\ 1 \end{bmatrix} \quad (55.2)$$

The camera parameters will have to account for the focal length, image size, camera center, and the camera rotation with respect to the world coordinates:

$$\mathbf{K} = \mathbf{M}\mathbf{R} \quad (55.3)$$

The intrinsic camera parameters (using perspective projection) are:

$$\mathbf{M} = \begin{bmatrix} a & 0 & n_0 \\ 0 & a & m_0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (55.4)$$

The extrinsic camera parameters will model the 3D rotation, which is mainly around the X -axis (we will ignore the translation of the origin):

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix}. \quad (55.5)$$



In order to apply equation (55.2), we need to define certain parameters: d_0 , n_0 , m_0 , a , and θ . Although we could determine these parameters using a calibration image, in this case, we'll opt for a simpler heuristic approach:

- d_0 : We will set this value to 0.
- n_0, m_0 : Defines the camera center point, where $n_0 = N/2$ and $m_0 = M/2$, when $N \times M$ is the input image size.
- a : Accounts for the image size and the camera focal length. One standard setting is to make $a = N$, where N is the image width.
- θ : Represents the angle between the optical camera axis and the ground plane. It would be possible to estimate this angle accurately by

determining the direction of the surface normal in pixels corresponding to the ground and measuring the angle with respect to the vertical axis. However, for our purposes, we will simply set it to $\theta = \pi/4$, as this is approximately the angle at which the pictures were taken.

Now we are ready to use equation (55.2) and the output of the system.

55.3 From 2D Images to 3D

Let's start running the model on the same input image that we used as a running example in chapter 2. We just need to apply the neural network to the input image, obtain the output disparity, and finally transform the disparity into world coordinates. We do not have to detect edges or any of the other features that we computed in chapter 2. Instead, the neural network has learned to compute whatever features it deems necessary to solve the task. We will not retrain the network using images from our simple visual world.

Just as we did in chapter 2, to show that the algorithm for 3D interpretation gives reasonable results, we can re-render the inferred 3D structure from different viewpoints.

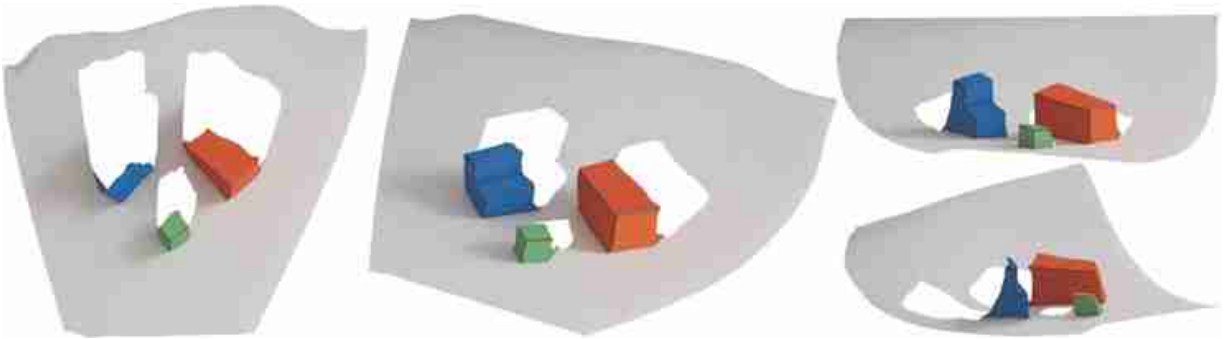


Figure 55.3: 3D reconstruction using a pretrained model. The images show multiple view points of the reconstructed 3D scene from [figure 55.1](#).

It is quite remarkable that it works somewhat well even though this system has been trained with natural images, which are quite dissimilar to the simple world we are using here. However, despite the surprising outputs, it is clear we cannot avoid a number of errors. For instance, the

ground is not recovered as a flat surface; instead it seems to be concave. The cubes' edges are not straight and the faces are twisted instead of flat.

We can compare this result with the 3D images recovered using the algorithm we defined in chapter 2:

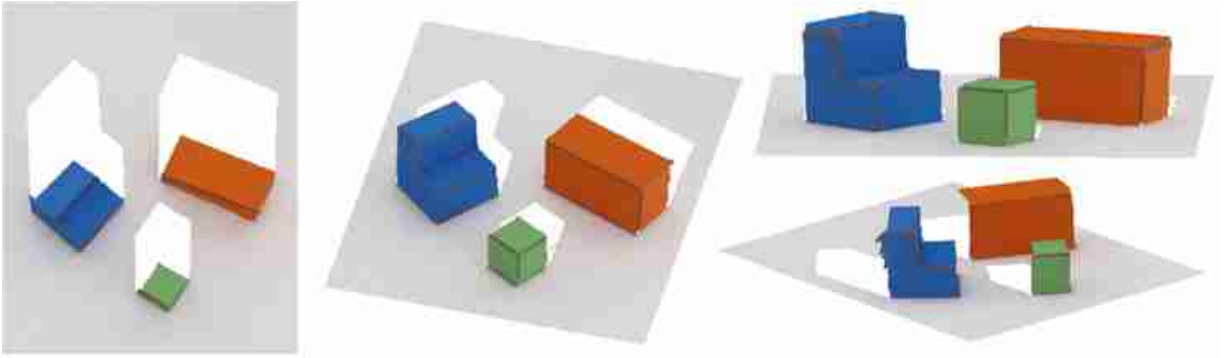


Figure 55.4: 3D reconstruction using the simple visual system from chapter 2 for the same input image as in [figure 55.3](#).

However, while the learned-based model may have lost accuracy in this example that fits our simple world assumptions, it has gained in robustness and generalization. The following figure compares the results obtained running MiDaS (the learning-based approach) or the simple world model (a hand-crafted approach).

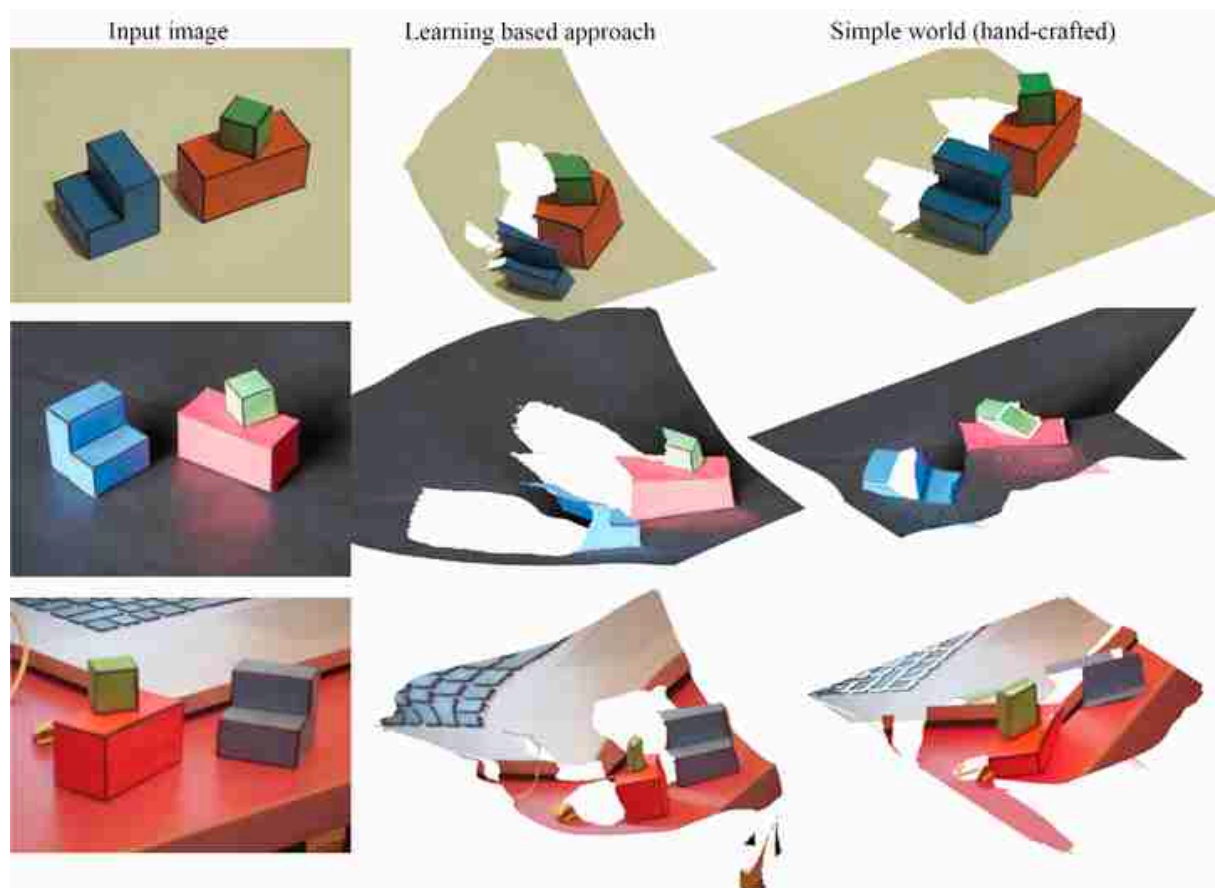


Figure 55.5: Out of domain generalization. For the pretrained model, these images might still be within the domain of the training, as it contained a very diverse set of natural images. The simple system model only works for the first example and breaks down for the rest, producing completely wrong reconstructions.

When the images fit the assumptions of the world model, the hand-crafted approach can produce beautiful results. However, it is very fragile, and poor illumination can break the output of the edge detectors, which results in a cascade of errors that propagate to the output. Conversely, the learning-based approach seems to produce results of good (although never perfect) quality regardless of the conditions. The hand-crafted approach completely fails even if we make a small change in the scene such as changing the ground plane from white to black.

55.4 Large Language Model-Based Scene Understanding

Another option is to use GPT4-v [362] to interpret the scene. GPT4-v can take as inputs images, and it allows asking questions about images. [Figure 55.6](#) shows the result of asking GPT4-v to describe a simple world image.

This method is remarkably accurate, and now you know how it works. At one level Gpt4-v is a transformer trained on massive data, using an autoregressive modeling objective (chapter 26). On another level, you can understand what rules that transformer may have learned: it detects edges in its first few layers, uses filters that act like wavelets tiling Fourier space, and it reasons about perspective geometry to find vanishing points and estimate shape (as we have discussed along many chapters in this book).

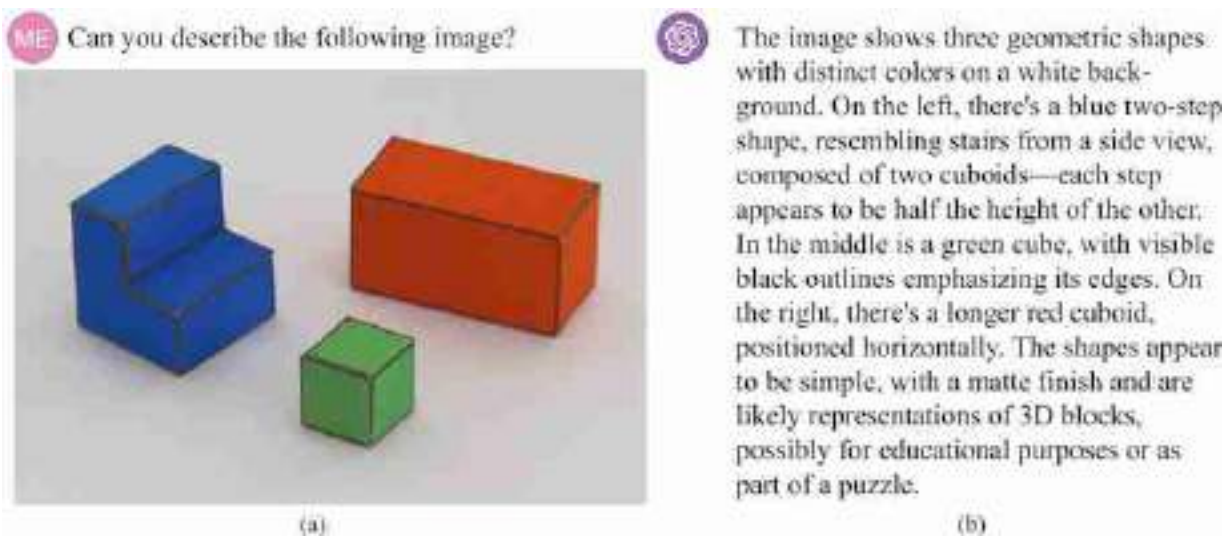


Figure 55.6: (a) Input image and text prompt to ChatGPT. (b) Output by ChatGPT (Gpt4-v, version from Nov 21, 2023) when asked to describe the image from the simple world.

But it's not all solved. Because if we reconstruct the image from this text, it looks not quite right: [figure 55.7](#). What is missing? How would you improve it. As the field moves forward quickly, it is likely that this particular example will be fixed, but there will be other failures. And some of those failures will give clues to what is the next fundamental question that will need to be addressed.

ME Render a new image using the same three geometric shapes but with different positions.

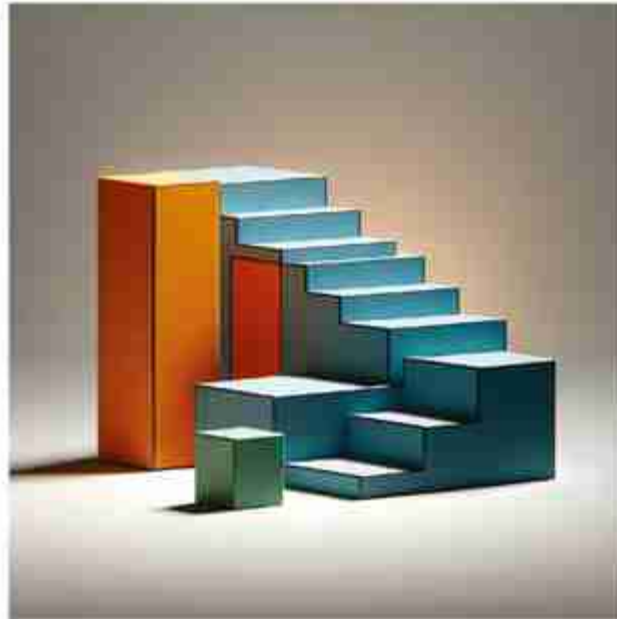


Figure 55.7: Output by ChatGPT when asked to render a new image with the same elements as in [figure 55.6](#). This conversation is a continuation of the same chat from [figure 55.6](#).

55.5 Unsolved Solved Computer Vision Problems

Computer vision is making amazing progress, but beware of the **unsolved solved computer vision problems**. For some vision task, it might appear as if there is one method that perfectly resolves the task. It might appear as if some foundational problems in computer vision are solved. It might often seem as if it is not possible to make progress regarding certain questions. But more often than not, it is just an illusion, and the solved problem was not really solved at all.

And even when a problem was genuinely solved (and probably is not), it's important to remember that for every solved problem, there are many unsolved problems lurking in the background.

When everything seems to be solved, remember that advances are the result of exploring new directions, with an open mind, and that creativity is a powerful tool.

55.6 Concluding Remarks

What have we learned in this chapter? We learned that using a large pretrained model kind of works; it is not as clean as the results we had in chapter 2, but it works in situations where our simple heuristic method was breaking. And we can expect performances of future pretrained models to keep improving.

With the hand-crafted model presented in chapter 2, we had a comprehensive understanding of the mechanisms, assumptions, and limitations of our system. Another interesting advantage of the hand-crafted model is that it required no learning and no training data. Once the model was written down it was ready to work. It worked really well when the conditions were met, but it was fragile and unreliable as soon as we deviated from those conditions. Our hand-crafted model did not generalize outside the space defined by our assumptions. When the system failed, it was easy to point out exactly what went wrong with it, but knowing what was wrong was not helping in making it work better.

Conversely, the pretrained system for 3D reconstruction seems to have generalized to our setting, at least to some degree, and it is able to deal with configurations that were not part of our modeling assumptions. The pretrained model is robust to changes on illumination and does not rely on fragile operations such as edge detection or figure-ground segmentation. But somehow, equivalent operations might be taking place inside the network. Information about 3D is still mostly contained in the edges, and information has to be propagated from the edges and junctions to the flat surfaces. How does the pretrained model work exactly? Has it learned some of the heuristics we introduced by hand, or is it using other ones?

GPT4-v seems to be capable of recognizing the geometric figures and their spatial configuration but the integration with the rendering engine seems fragile and was not able to create a new image with the same elements (although it is very close!). But why did it fail?

The precise answers to these questions do not matter right now. The field is evolving quickly and many of those problems will be addressed over time. What matters is that you should challenge existing approaches, and find what the relevant questions are for whatever new systems you encounter.

With learning-based systems we find ourselves doing experiments similar to the ones performed by psychophysics in trying to understand the mechanisms implemented by the human visual system.



Can you generate an image that could go well with this conclusion?



The generated image illustrates a side-by-side comparison of two types of computer vision models, aligning with the themes discussed in the book's conclusion. On one side, there's a hand-crafted model depicted as a transparent machine with visible gears and cogs, processing simple geometric shapes. On the other side, a complex neural network represents a pretrained model, dealing with a variety of complex images. This visual comparison effectively captures the contrast between the two approaches in computer vision.



Bibliography

- [1] Creative Commons Attribution CC BY NC SA. 2015. URL: <https://asknature.org/strategy/pupil-enables-clear-vision-in-extreme-light-conditions/>.
- [2] S. A. Abbas and A. Zisserman. “A Geometric Approach to Obtain a Bird's Eye View From an Image.” In: *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*. 2019, pp. 4095–4104.
- [3] E. Abbott. *Flatland*. Broadview Press, 2009.
- [4] user Acdx. *CIE 1931 XYZ Color Matching Functions*. GNU Free Documentation License. https://en.wikipedia.org/wiki/CIE_1931_color_space.2009.
- [5] E. H. Adelson. Personal Communication. 1992.
- [6] E. H. Adelson. Personal Communication. 1995.
- [7] E. H. Adelson. “Lightness Perception and Lightness Illusions.” In: *The New Cognitive Neurosciences*. Ed. by M. Gazzaniga. Cambridge, MA: MIT Press, 2000, pp. 339–351.
- [8] E. H. Adelson. “On Seeing Stuff: The Perception of Materials by Humans and Machines.” In: *Proceedings of SPIE 4299* (June 2001). DOI: 10.1117/12.429489.
- [9] E. H. Adelson and J. R. Bergen. “Spatiotemporal Energy Models for the Perception of Motion.” In: *Journal of the Optical Society of America A* 2.2 (1985), pp. 284–299.
- [10] E. H. Adelson and E. P. Simoncelli. “QMF Pyramids: A New Class of Orthogonal Pyramid Transform.” In: *Optical Society of America, Annual Meeting*. Vol. A4-13. 1987.
- [11] E. H. Adelson. *Checkershadow Illusion*. 1995.
- [12] E. H. Adelson and J. R. Bergen. “The Plenoptic Function and the Elements of Early Vision”. In: *Computational Models of Visual Processing*. Ed. by M. S. Landy and A. J. Movshon. Cambridge, MA: MIT Press, 1991, pp. 3–20.
- [13] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, et al. “Flamingo: A Visual Language Model for Few-Shot Learning.” In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 23716–23736.
- [14] A. Alda. “Alan Alda on Improvisation for Communication of Science.” In: (2014). MIT McGovern Institute talk. URL: <https://www.youtube.com/watch?v=j4XgjkXDxss>.
- [15] S. Amarasinghe and D. Montgomery. *Faculty Job Talks: Tips from the Faculty*. 2022. URL: <https://www.eecs.mit.edu/career-opportunities-at-eecs/faculty-job-talks-tips-from-the-faculty/>.
- [16] M. Andrychowicz, M. Denil, S. Gomez, M. W. Hoffman, D. Pfau, T. Schaul, B. Shillingford, and N. De Freitas. “Learning to Learn by Gradient Descent by Gradient Descent.” In: *Advances in Neural Information Processing Systems*. Vol. 29. 2016.
- [17] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh. “VQA: Visual Question Answering.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2015, pp. 2425–2433.

- [18] P. Arbelaez, M. Maire, C. Fowlkes, and J. Malik. “Contour Detection and Hierarchical Image Segmentation.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.5 (2010), pp. 898–916.
- [19] D. Ariely. “Seeing Sets: Representation by Statistical Properties.” In: *Psychological Science* 12.2 (2001), pp. 157–162.
- [20] Aristotle. *On Sense and the Sensible*. Translated by J. I. Beare. 350 BC. URL: <http://classics.mit.edu/Aristotle/sense.html>.
- [21] S. E. J. Arnold, S. Faruq, V. Savolainen, P. W. McOwan, and L. Chittka. “FReD: the Floral Reflectance Database— A Web Portal for Analysis of Flower Colour.” In: *PLoS ONE* 5.12 (2011), e14287.
- [22] J. J. Atick and A. N. Redlich. “Towards a Theory of Early Visual Processing.” In: *Neural Computation* 2.3 (Sept. 1990), pp. 308–320.
- [23] J. J. Atick and A. N. Redlich. “What Does the Retina Know about Natural Scenes?” In: *Neural Computation* 4.2 (1992), pp. 196–210.
- [24] A. Avni. 2016. URL: <http://www.whatimade.today/our-frst-reddit-bot-coloring-b-2/>.
- [25] S. Azizi, S. Kornblith, C. Saharia, M. Norouzi, and D. J. Fleet. *Synthetic Data from Diffusion Models Improves Imagenet Classification*. 2023. arXiv: 2304.08466.
- [26] J. L. Ba, J. R. Kiros, and G. E. Hinton. *Layer Normalization*. 2016. arXiv: 1607.06450.
- [27] V. Badrinarayanan, A. Handa, and R. Cipolla. “SegNet: A Deep Convolutional Encoder-Decoder Architecture for Robust Semantic Pixel-Wise Labelling.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 39 (2015), pp. 2481–2495.
- [28] H. Bahng, A. Jahanian, S. Sankaranarayanan, and P. Isola. *Exploring Visual Prompts for Adapting Large-Scale Models*. 2022. arXiv: 2203.17274.
- [29] S. Baker, S. Roth, D. Scharstein, M. J. Black, J. Lewis, and R. Szeliski. “A Database and Evaluation Methodology for Optical Flow.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2007, pp. 1–8.
- [30] G. Balakrishnan, Y. Xiong, W. Xia, and P. Perona. “Towards Causal Benchmarking of Bias in Face Analysis Algorithms.” In: *European Conference on Computer Vision*. 2020.
- [31] D. H. Ballard. “Modular Learning in Neural Networks.” In: *Proceedings of the National Conference on Artificial Intelligence*. Vol. 647. 1987, pp. 279–284.
- [32] A. Bansal, E. Borgnia, H.-M. Chu, J. S. Li, H. Kazemi, F. Huang, M. Goldblum, J. Geiping, and T. Goldstein. *Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise*. 2022. arXiv: 2208.09392.
- [33] A. Bar, Y. Gandelsman, T. Darrell, A. Globerson, and A. Efros. “Visual Prompting Via Image Inpainting.” In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 25005–25017.
- [34] G. Barber. “The Viral App That Labels You Isn't Quite What You Think”. In: *Wired* (2019).
- [35] C. Barnes, E. Shechtman, A. Finkelstein, and D. B. Goldman. “PatchMatch: A Randomized Correspondence Algorithm for Structural Image Editing.” In: *ACM SIGGRAPH: Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques*. 2009.
- [36] S. Barocas, M. Hardt, and A. Narayanan. *Fairness and Machine Learning*. fairmlbook.org, 2019.

- [37] J. T. Barron. “Convolutional Color Constancy.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2015.
- [38] J. T. Barron and J. Malik. “Shape, Illumination, and Reflectance from Shading.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 37 (2015), pp. 1670–1687.
- [39] H. G. Barrow and J. M. Tenenbaum. “Recovering Intrinsic Scene Characteristics from Images.” In: *Computer Vision Systems*. Ed. by A. R. Hanson and E. M. Riseman. New York: Academic Press, 1978, pp. 3–26.
- [40] A. Baudes, B. Coll, and J.-M. Morel. “Non-local Means Denoising.” In: *Image Processing On Line*. Vol. 1. 2011.
- [41] H. Bay, T. Tuytelaars, and L. Van Gool. “SURF: Speeded Up Robust Features.” In: *European Conference on Computer Vision*. 2006, pp. 404–417.
- [42] M. Belkin, D. Hsu, S. Ma, and S. Mandal. *Reconciling Modern Machine Learning and the Bias-Variance Trade-Off*. 2018. arXiv: 1812.11118.
- [43] Y. Bengio, A. Courville, and P. Vincent. “Representation Learning: A Review and New Perspectives.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8 (2013), pp. 1798–1828.
- [44] R. Benjamin. *Race After Technology*. Polity, 2019.
- [45] C. L. Bennett, C. Gleason, M. K. Scheuerman, J. P. Bigham, A. Guo, and A. To. “‘It’s Complicated’: Negotiating Accessibility and (Mis)Representation in Image Descriptions of Race, Gender, and Disability.” In: *CHI 2021*. 2021.
- [46] J. R. Bergen and E. H. Adelson. “Visual Texture Segmentation and Early Vision.” In: *Nature* 333 (1988), pp. 363–364.
- [47] H.-G. Beyer and H.-P. Schwefel. “Evolution Strategies— a Comprehensive Introduction.” In: *Natural Computing* 1.1 (2002), pp. 3–52.
- [48] I. Biederman. “Recognition by Components - a Theory of Human Image Understanding.” In: *Psychological Review* 94.2 (1987).
- [49] I. Biederman. “On Processing Information from a Glance at a Scene: Some Implications for a Syntax and Semantics of Visual Processing.” In: *Proceedings of the ACM/SIGGRAPH Workshop on User-Oriented Design of Interactive Graphics Systems*. 1976, pp. 75–88.
- [50] T. O. Binford. “Visual Perception by Computer.” In: *Proceedings of the IEEE Conference on Systems and Control (Miami, FL)*. 1971.
- [51] A. Birhane and V. U. Prabhu. “Large Image Datasets: A Pyrrhic Win for Computer Vision?” In: *IEEE/CVF Winter Conference on Applications of Computer Vision*. 2021.
- [52] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer-Verlag, 2006.
- [53] A. Blake, P. Kohli, and C. Rother. *Markov Random Fields for Vision and Image Processing*. Cambridge, MA: MIT Press, 2011.
- [54] C. Bleasdale. 2015. URL: https://en.wikipedia.org/wiki/The_dress.
- [55] J. L. Borges. “Pierre Menard, Author of the Quixote.” In: *Ficciones* (1939).
- [56] K. L. Bouman, V. Ye, A. B. Yedidia, F. Durand, G. W. Wornell, A. Torralba, and W. T. Freeman. “Turning Corners into Cameras: Principles and Methods.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2018.

- [57] H. Boulard and Y. Kamp. “Auto-Association by Multilayer Perceptrons and Singular Value Decomposition.” In: *Biological Cybernetics* 59.4 (1988), pp. 291–294.
- [58] C. B. Boyer. “Aristotelian References to the Law of Reflection.” In: *Isis* 36.2 (1946), pp. 92–95.
- [59] D. H. Brainard and W. T. Freeman. “Bayesian Color Constancy.” In: *Journal of the Optical Society of America A* 14.7 (1997), pp. 1393–1411.
- [60] D. H. Brainard and A. C. Hurlbert. “Colour Vision: Understanding #TheDress.” In: *Current Biology* 25 (2015), R551–R554.
- [61] A. Brock, J. Donahue, and K. Simonyan. “Large Scale GAN Training for High Fidelity Natural Image Synthesis.” In: *International Conference on Learning Representations* (2019).
- [62] T. Brooks, A. Holynski, and A. A. Efros. “Instructpix2pix: Learning to Follow Image Editing Instructions.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2023.
- [63] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. “Language Models Are Few-Shot Learners.” In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 1877–1901.
- [64] A. Buades, B. Coll, and J.-M. Morel. “A Non-Local Algorithm for Image Denoising.” In: *cvpr*. Vol. 2. 2005, 60–65 vol. 2.
- [65] J. Buolamwini and T. Gebru. “Intersectional Accuracy Disparities in Commercial Gender Classification”. In: *Proceedings of Machine Learning Research Conference on Fairness, Accountability, and Transparency*. Vol. 81. 2018, pp. 1–15.
- [66] P. J. Burt and E. H. Adelson. “The Laplacian Pyramid as a Compact Image Code.” In: *IEEE Transactions on Communications* 31.4 (1983), pp. 532–540.
- [67] H. E. Burton. “The Optics of Euclid.” In: *J. Opt. Soc. Am.* 35.5 (1945), pp. 357–372.
- [68] D. J. Butler, J. Wulff, G. B. Stanley, and M. J. Black. “A Naturalistic Open Source Movie for Optical Flow Evaluation.” In: *European Conference on Computer Vision*. Ed. by A. Fitzgibbon et al. Part IV, LNCS 7577. Springer-Verlag, Oct. 2012, pp. 611–625.
- [69] J. F. Canny. “A Computational Approach to Edge Detection.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 8.6 (1986), pp. 679–698.
- [70] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin. “Emerging Properties in Self-Supervised Vision Transformers.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021, pp. 9650–9660.
- [71] P. Cavanagh. “Vision is Getting Easier Every Day.” In: *Perception* 24 (1996), pp. 1227–1232.
- [72] J. G. Cavazos, P. J. Phillips, C. D. Castillo, and A. J. O’Toole. “Accuracy Comparison Across Face Recognition Algorithms: Where Are We on Measuring Race Bias?” In: *IEEE transactions on biometrics, behavior, and identity science* 3 (1 2021), pp. 101–111.
- [73] L. Chai, J.-Y. Zhu, E. Shechtman, P. Isola, and R. Zhang. “Ensembling with Deep Generative Views.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2021, pp. 14997–15007.
- [74] C.-K. Chang, J. Zhao, and L. Itti. “DeepVP: Deep Learning for Vanishing Point Detection on 1 Million Street View Images.” In: 2018, pp. 1–8.
- [75] J.-R. Chang and Y.-S. Chen. “Pyramid Stereo Matching Network.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2018.

- [76] G. Chechik, V. Sharma, U. Shalit, and S. Bengio. “Large Scale Online Learning of Image Similarity Through Ranking.” In: *Journal of Machine Learning Research* 11.3 (2010).
- [77] A. Chehikian and J. Crowley. “Fast Computation of Optimal Semi-Octave Pyramids.” In: *Scandinavian Conference on Image Analysis* (1991), pp. 18–27.
- [78] M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan, and I. Sutskever. “Generative Pretraining from Pixels.” In: *International Conference on Machine Learning*. 2020, pp. 1691–1703.
- [79] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. “A Simple Framework for Contrastive Learning of Visual Representations.” In: *International Conference on Machine Learning*. 2020, pp. 1597–1607.
- [80] CIE. *CIE 1931 XYZ Color Matching Functions*. 1931. URL: https://commons.wikimedia.org/wiki/File:CIE1931_RGBCMF.png.
- [81] A. L. Cohen. “Anti-Pinhole Imaging.” In: *Optica Acta: Intl. J. of Optics* 29.1 (1982).
- [82] J. W. Cooley and J. W. Tukey. “An Algorithm for the Machine Calculation of Complex Fourier Series.” In: *Mathematics of Computation* 19 (1965), pp. 297–301.
- [83] C. Cortes and V. Vapnik. “Support-Vector Networks.” In: *Machine Learning* 20 (1995), pp. 273–297.
- [84] J. Coughlan and A. L. Yuille. “Manhattan World: Compass Direction from a Single Image by Bayesian Inference.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 1999, pp. 941–947.
- [85] A. Criminisi. “Accurate Visual Metrology from Single and Multiple Uncalibrated Images.” PhD thesis. 1999.
- [86] G. Csurka, C. Bray, C. Dance, and L. Fan. “Visual Categorization with Bags of Keypoints.” In: *Workshop on Statistical Learning in Computer Vision, ECCV* (2004), pp. 1–22.
- [87] M. L. Cummings. “Automation Bias in Intelligent Time Critical Decision Support Systems.” In: *AIAA Third Intelligent Systems Conference*. 2004.
- [88] C. A. Curcio, K. R. Sloan, R. E. Kalina, and A. E. Hendrickson. “Human Photoreceptor Topography.” In: *Journal of Comparative Neurology* 292.4 (1990), pp. 497–523.
- [89] C. A. Curcio, K. R. Sloan, O. S. Packer, A. Hendrickson, and R. E. Kalina. “Distribution of Cones in Human and Monkey Retina: Individual Variability and Radial Asymmetry.” In: *Science* 236 4801 (1987), pp. 579–82.
- [90] B. Curless and M. Levoy. “A Volumetric Method for Building Complex Models from Range Images.” In: *Proceedings of the 23rd annual conference on Computer graphics and interactive techniques*. 1996, pp. 303–312.
- [91] G. Cybenko. “Approximation by Superpositions of a Sigmoidal Function.” In: *Mathematics of Control, Signals and Systems* 2.4 (1989), pp. 303–314.
- [92] B. d’Alessandro, C. O’Neil, and T. LaGatta. *Conscientious Classification: A Data Scientist's Guide to Discrimination-Aware Classification*. 2017.
- [93] N. Dalal and B. Triggs. “Histograms of Oriented Gradients for Human Detection.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2005.
- [94] D. J. Dalmotas, R. M. Hurley, and A. German. “Air Bag Deployments Involving Restrained Occupants.” In: *SAE Transactions* 104.6 (1985), pp. 1507–1512.

- [95] T. Darrell and E. Simoncelli. “On the use of ‘Nulling’ Filters to Separate Transparent Motions.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 1993.
- [96] J. Daugman. “Entropy Reduction and Decorrelation in Visual Coding by Oriented Neural Receptive Fields.” In: *IEEE Transactions on Biomedical Engineering* 36.1 (1989), pp. 107–114.
- [97] R. De Valois, K. De Valois, and O. U. Press. *Spatial Vision*. Oxford psychology series. Oxford University Press, 1988.
- [98] J. S. DeBonet and P. Viola. “Texture Recognition Using a Non-Parametric Multi-Scale Statistical Model.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 1998.
- [99] Deglr6328. *Blue Sky Spectrum*. <https://en.wikipedia.org/>, File:Spectrum_of_blue_sky.png. GNU Free Documentation License. 2006.
- [100] E. Denton, B. Hutchinson, M. Mitchell, T. Gebru, and A. Zaldivar. “Image Counterfactual Sensitivity Analysis for Detecting Unintended Bias.” In: *Computer Vision and Pattern Recognition Workshop*. 2019.
- [101] D. DeTone, T. Malisiewicz, and A. Rabinovich. “SuperPoint: Self-Supervised Interest Point Detection and Description.” In: *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 2018, pp. 337–33712.
- [102] T. DeVries, I. Misra, C. Wang, and L. van der Maaten. “Does Object Recognition Work for Everyone?” In: *Computer Vision and Pattern Recognition Workshop*. 2019.
- [103] C. Doersch, A. Gupta, and A. A. Efros. “Unsupervised Visual Representation Learning by Context Prediction.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2015, pp. 1422–1430.
- [104] C. Doersch, S. Singh, A. Gupta, J. Sivik, and A. A. Efros. “What Makes Paris Look like Paris?” In: *ACM Transactions on Graphics* 31.4 (2012), 101:1–101:9.
- [105] P. Doherty. Video Demonstration by Paul Doherty. 2023. URL: <https://www.exploratorium.edu/snacks/cd-spectroscope>.
- [106] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell. “Decaf: A deep convolutional activation feature for generic visual recognition.” In: *International Conference on Machine Learning*. PMLR. 2014, pp. 647–655.
- [107] J. Donahue, P. Krähenbühl, and T. Darrell. “Adversarial Feature Learning.” In: *International Conference on Learning Representations*. 2017.
- [108] A. Dosovitskiy, P. Fischer, E. Ilg, P. Häusser, C. Hazirbas, V. Golkov, P. v. d. Smagt, D. Cremers, and T. Brox. “FlowNet: Learning Optical Flow with Convolutional Networks.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2015, pp. 2758–2766.
- [109] A. Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.” In: *International Conference on Learning Representations* (2021).
- [110] R. O. Duda and P. E. Hart. “Use of the Hough Transformation to Detect Lines and Curves in Pictures.” In: *Communications of the ACM* 15.1 (1972), pp. 11–15.
- [111] C. Dwork and A. Roth. “The Algorithmic Foundations of Differential Privacy.” In: *Foundations and Trends in Theoretical Computer Science* 9.3–4 (2014), pp. 211–407.

- [112] C. Dwork, M. Hardt, T. Pitassi, O. Reingold, and R. Zemel. “Fairness Through Awareness.” In: *ITCS '12: Proceedings of the Third Innovations in Theoretical Computer Science Conference*. 2012, pp. 214–226.
- [113] C. Dwork, N. Kohli, and D. Mulligan. “Differential Privacy in Practice: Expose your Epsilons!” In: *Journal of Privacy and Confidentiality* 9.2 (2019).
- [114] A. A. Efros and W. T. Freeman. “Image Quilting for Texture Synthesis and Transfer.” In: *ACM SIGGRAPH: Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques*. 2001, pp. 341–346.
- [115] A. A. Efros and T. K. Leung. “Texture Synthesis by Non-Parametric Sampling.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 1999.
- [116] J. L. Elman. “Finding Structure in Time.” In: *Cognitive science* 14.2 (1990), pp. 179–211.
- [117] G. F. Elsayed, I. Goodfellow, and J. Sohl-Dickstein. *Adversarial Reprogramming of Neural Networks*. 2018. arXiv: 1806.11146.
- [118] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. “The PASCAL visual object classes (VOC) challenge.” In: *International Journal of Computer Vision* 88 (2010), pp. 303–338.
- [119] P. Fara. “Newton Shows the Light: a Commentary on Newton (1672) ‘A Letter . . . Containing his New Theory About Light and Colours. . . ’” In: *Philosophical Transactions of the Royal Society* (2015).
- [120] M. Farrell and C. Haynes. *Straw Camera*. 2017. URL: <https://strawcamera.com/>.
- [121] A. Fashion. *Fun World Adult Cockroach Costume*. 2021. URL: <https://www.amazon.com/Fun-World-Costumes-Cockroach-Costume/dp/B0038ZQYRC>.
- [122] O. Faugeras. *Three-Dimensional Computer Vision: A Geometric Viewpoint*. Cambridge, MA: MIT Press, 1993.
- [123] C. Fellbaum, ed. *WordNet: An Electronic Lexical Database*. Cambridge, MA: MIT Press, 1998.
- [124] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan. “Object Detection with Discriminatively Trained Part-Based Models.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 32.9 (2010), pp. 1627–1645.
- [125] R. Fergus, B. Singh, A. Hertzmann, S. Roweis, and W. T. Freeman. “Removing Camera Shake from a Single Image.” In: *ACM Transactions on Graphics* 25.3 (2006), pp. 787–794.
- [126] R. Fergus, P. Perona, and A. Zisserman. “Weakly Supervised Scale-Invariant Learning of Models for Visual Recognition.” In: *International Journal of Computer Vision* 71.3 (2007), pp. 273–303.
- [127] D. J. Field. “Relations Between the Statistics of Natural Images and the Response Properties of Cortical Cells.” In: *Journal of the Optical Society of America A* 4.12 (1987), pp. 2379–2394.
- [128] M. A. Fischler and R. C. Bolles. “Random Sample Consensus: a Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography.” In: *Communications of the ACM* 24.6 (1981), pp. 381–395.
- [129] M. A. Fischler and R. A. Elschlager. “The Representation and Matching of Pictorial Structures.” In: *IEEE Transactions on Computers* C-22.1 (1973), pp. 67–92.

- [130] D. Fleet and A. Jepson. “Computation of Normal Velocity from Local Phase Information.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 1989, pp. 379–386.
- [131] R. W. Fleming, R. O. Dror, and E. H. Adelson. “Surface Reflectance Estimation Under Unknown Natural Illumination.” In: *Journal of Vision* (2001). DOI: 10.1167/1.3.43.
- [132] F. Fleuret and D. Geman. “Coarse-to-Fine Face Detection.” In: *International Journal of Computer Vision* 41.1–2 (2001), pp. 85–107.
- [133] J. A. Fodor. *The Language of Thought*. Vol. 5. Cambridge, MA: Harvard University Press, 1975.
- [134] P. Foret, A. Kleiner, H. Mobahi, and B. Neyshabur. “Sharpness-Aware Minimization for Efficiently Improving Generalization.” In: *International Conference on Learning Representations*. 2021.
- [135] D. A. Forsyth and J. Ponce. *Computer Vision - A Modern Approach, Second Edition*. Pitman, 2012.
- [136] J. B. J. Fourier. *Théorie Analytique de la Chaleur*. Cambridge Library Collection. Cambridge University Press, 2009.
- [137] D. H. Freedman. *Wrong: Why Experts* Keep Failing Us—and How to Know When Not to Trust Them*. New York: Little, Brown Co., 2010.
- [138] W. T. Freeman. “The Generic Viewpoint Assumption in a Framework for Visual Perception.” In: *Nature* 368.6471 (1994), pp. 542–545.
- [139] W. T. Freeman and E. H. Adelson. “Steerable Filters for Early Vision, Image Analysis, and Wavelet Decomposition.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 1990, pp. 406–415.
- [140] W. T. Freeman and E. H. Adelson. “The Design and Use of Steerable Filters.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 13.9 (1991), pp. 891–906.
- [141] W. T. Freeman, E. H. Adelson, and D. J. Heeger. “Motion without Movement.” In: *ACM SIGGRAPH: Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques*. 1991, pp. 27–30.
- [142] W. T. Freeman, E. H. Adelson, and A. P. Pentland. “Shape-from-Shading Analysis with Bumples and Shadelets.” In: *Investigative Ophthalmology and Visual Science (ARVO)* (1990), p. 410.
- [143] W. T. Freeman and D. H. Brainard. “Bayesian Decision Theory, the Maximum Local Mass Estimate, and Color Constancy.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 1995, pp. 210–217.
- [144] W. T. Freeman and H. Zhang. “Shapetime Photography.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2003.
- [145] W. T. Freeman et al. “Computer Vision for Interactive Computer Graphics.” In: *IEEE Computer Graphics and Applications* 18.3 (1998), pp. 42–53.
- [146] W. T. Freeman. “How to Write Good Papers.” In: CVPR 2020 workshop, How to Write a Good Review, organized by Laura Leal-Taixé and Torsten Sattler <https://www.youtube.com/watch?v=W1zPtTt43LI>. 2020.
- [147] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa. “Plenoxels: Radiance Fields Without Neural Networks.” In: *Proceedings of the IEEE/CVF Computer Vision*

- and Pattern Recognition. 2022, pp. 5501–5510.
- [148] K. Fukushima. “Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position.” In: *Biological Cybernetics* 36.4 (1980), pp. 193–202.
 - [149] D. Gabor. “Theory of Communication.” In: *Journal of the Institution of Electrical Engineers - Part I: General* 94 (1946), pp. 58–58.
 - [150] P. Gage. “A New Algorithm for Data Compression.” In: *The C Users Journal* 12.2 (1994), pp. 23–38.
 - [151] P. A. Gagniuc. *Markov Chains: From Theory to Implementation and Experimentation*. John Wiley & Sons., 2017.
 - [152] G. Galileo. *Sidereus Nuncius, or The Sidereal Messenger*. Translated by Albert Van Helden. Chicago: University of Chicago Press, 2015.
 - [153] Gardenia. URL: <https://www.gardenia.net/plants/plant-family/hepatica-liverleaf>.
 - [154] C. Garvie, A. Bedoya, and J. Frankle. “The Perpetual Line-Up.” In: (2019). URL: <https://www.perpetuallineup.org/>.
 - [155] T. Gebru and E. Denton. *CVPR Tutorial on Fairness, Accountability, Transparency, and Ethics in Computer Vision*. 2020.
 - [156] T. Gebru and E. Denton. *CVPR Workshop: Beyond Fairness: Towards a Just, Equitable, and Accountable Computer Vision*. 2021. URL: <https://sites.google.com/view/beyond-fairness-cv/>.
 - [157] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun. “Vision Meets Robotics: The KITTI Dataset.” In: *The International Journal of Robotics Research* 32.11 (2013), pp. 1231–1237.
 - [158] D. Geman, B. Jedynak, P. Robotique, and P. Syntim. “Shape Recognition and Twenty Questions.” In: *Proceedings Reconnaissance des Formes et Intelligence Articielle*. 1994, pp. 21–37.
 - [159] J. Gibson. *Motion Picture Testing and Research*. Aviation Psychology Program Research Reports no. 7. U.S. Government Printing Office, 1947.
 - [160] J. J. Gibson. *The Ecological Approach to Visual Perception*. Boston: Houghton Mifflin, 1979.
 - [161] J. J. Gibson. *The Senses Considered as Perceptual Systems*. Boston: Houghton Mifflin, 1966.
 - [162] R. Gilbert. *Quotes*. URL: <https://www.quotes.net/quote/13310>.
 - [163] A. Gilchrist. *Seeing Black and White*. Oxford University Press, 2006.
 - [164] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl. “Neural Message Passing for Quantum Chemistry.” In: *International Conference on Machine Learning*. 2017, pp. 1263–1272.
 - [165] G. Gkioxari, J. Johnson, and J. Malik. “Mesh R-CNN.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2019, pp. 9784–9794.
 - [166] M. Glickstein. “Golgi and Cajal: The Neuron Doctrine and the 100th Anniversary of the 1906 Nobel Prize.” In: *Current Biology* 16.5 (2006), R147–R151.
 - [167] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016.

- [168] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. “Generative Adversarial Nets.” In: vol. 27. 2014.
- [169] M. Gorkani and R. Picard. “Texture Orientation for Sorting Photos ’at a glance.’” In: *Proceedings of 12th International Conference on Pattern Recognition*. Vol. 1. 1994, 459–464 vol.1.
- [170] S. J. Gortler, R. Grzeszczuk, R. Szeliski, and M. F. Cohen. “The Lumigraph.” In: *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*. 1996, pp. 43–54.
- [171] J. Gou, B. Yu, S. J. Maybank, and D. Tao. “Knowledge Distillation: A Survey.” In: *International Journal of Computer Vision* 129 (2021), pp. 1789–1819.
- [172] G. Granlund and H. Knutsson. *Signal Processing for Computer Vision*. New York, NY: Springer, 1995.
- [173] G. H. Granlund. “In Search of a General Picture Processing Operator.” In: *Computer Graphics, Image Proc.* 8 (1978), pp. 155–173.
- [174] G. H. Granlund. “In Search of a General Picture Processing Operator.” In: *Computer Graphics and Image Processing* 8.2 (1978), pp. 155–173.
- [175] G. Griffin, A. Holub, and P. Perona. “Caltech-256 object category dataset.” In: (2007).
- [176] P. Grother, M. Ngan, and K. Hanaoka. *Face Recognition Vendor Test (FRVT). Part 3: Demographic Effects*. NISTIR 8280. 2019.
- [177] P. D. Grünwald. *The Minimum Description Length Principle*. Cambridge, MA: MIT Press, 2007.
- [178] T. Gupta and A. Kembhavi. “Visual programming: Compositional visual reasoning without training.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2023, pp. 14953–14962.
- [179] D. Ha, A. Dai, and Q. V. Le. “Hypernetworks.” In: *International Conference on Learning Representations* (2016).
- [180] R. Hadsell, S. Chopra, and Y. LeCun. “Dimensionality reduction by learning an invariant mapping.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. Vol. 2. 2006, pp. 1735–1742.
- [181] F. Hamidi, M. K. Scheuerman, and S. M. Branham. “Gender Recognition or Gender Reductionism?: The Social Implications of Embedded Gender Recognition Systems.” In: *CHI ’18: Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*. 2018.
- [182] J. M. Hammersley and P. Clifford. *Markov Fields on Finite Graphs and Lattices*. 1971. URL: <http://www.statslab.cam.ac.uk/~grg/books/hammfest/hamm-cliff.pdf>.
- [183] P. Hancock, R. Baddeley, and L. Smith. “The Principal Components of Natural Images.” In: *Network: Computation in Neural Systems* 3 (1970).
- [184] M. Hardt. *MLSS 2020, Tübingen*. 2020.
- [185] L. D. Harmon and B. Julesz. “Masking in Visual Recognition: Effects of Two-Dimensional Filtered Noise.” In: *Science* 180.4091 (1973), pp. 1194–1197.
- [186] C. Harris and M. Stephens. “A Combined Corner and Edge Detector.” In: *Proc. of Fourth Alvey Vision Conference*. 1988, pp. 147–151.

- [187] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. 2nd. Cambridge, UK: Cambridge University Press, 2004.
- [188] H. K. Hartline. “The Response of Single Optic Nerve Fibers of the Vertebrate Eye to Illumination of the Retina.” In: *American Journal of Physiology-Legacy Content* 121.2 (1938), pp. 400–415.
- [189] V. Hassenstein and W. Reichardt. “System Theoretical Analysis of Time, Sequence and Sign Analysis of the Motion Perception of the Snout-Beetle *Chlorophanus*.” In: *Z. Naturforsch. B* 11 (1956), pp. 513–524.
- [190] A. Haviv, O. Ram, O. Press, P. Izsak, and O. Levy. “Transformer Language Models without Positional Encodings Still Learn Positional Information.” In: *Findings of the Association for Computational Linguistics: EMNLP*. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, 2022, pp. 1382–1390.
- [191] J. Hays and A. A. Efros. “Scene Completion Using Millions of Photographs.” In: *ACM Transactions on Graphics* 26.3 (2007), p. 4.
- [192] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. “Masked Autoencoders are Scalable Vision Learners.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2022, pp. 15979–15988.
- [193] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick. “Momentum Contrast for Unsupervised Visual Representation Learning.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2020, pp. 9729–9738.
- [194] K. He, G. Gkioxari, P. Dollár, and R. Girshick. “Mask R-CNN.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2017, pp. 2961–2969.
- [195] K. He, J. Sun, and X. Tang. “Single Image Haze Removal Using Dark Channel Prior.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2009, pp. 1956–1963.
- [196] K. He, X. Zhang, S. Ren, and J. Sun. “Deep Residual Learning for Image Recognition.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2016, pp. 770–778.
- [197] R. He, S. Sun, X. Yu, C. Xue, W. Zhang, P. Torr, S. Bai, and X. Qi. *Is Synthetic Data from Generative Models Ready for Image Recognition?* 2022. arXiv: 2210.07574.
- [198] D. O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Psychology Press, 2005.
- [199] E. Hecht. *Optics*. 5th ed. Hoboken, NJ: Pearson, 2016.
- [200] D. Heeger. Personal Communication. 1995.
- [201] D. J. Heeger and E. P. Simoncelli. “Model of Visual Motion Sensing.” In: *Spatial Vision in Humans and Robots*. Ed. by L. Harris and M. Jenkin. Cambridge Univ. Press, 1992.
- [202] D. J. Heeger and J. R. Bergen. “Pyramid-Based Texture Analysis/Synthesis.” In: *Computer Graphics Proceedings*. 1995, pp. 229–238.
- [203] H. V. Helmholtz. *Treatise on Physiological Optics*. Vol. III. Trans. from the 3rd German Edition. Edited by J. P. C. Southall. New York: Dover, 1962.
- [204] H. von Helmholtz. *Helmholtz's Treatise on Physiological Optics*. Optical Society of America, 1925.
- [205] G. Hinton, O. Vinyals, and J. Dean. *Distilling the Knowledge in a Neural Network*. 2015. arXiv: 1503.02531.

- [206] G. E. Hinton. “Training Products of Experts by Minimizing Contrastive Divergence.” In: *Neural Computation* 14.8 (2002), pp. 1771–1800.
- [207] H. Hirschmüller. “Stereo Processing by Semi-Global Matching and Mutual Information.” In: 30.2 (2007), pp. 328–341.
- [208] J. Ho, A. Jain, and P. Abbeel. “Denoising Diffusion Probabilistic Models.” In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 6840–6851.
- [209] S. Hochreiter and J. Schmidhuber. “Long Short-Term Memory.” In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [210] H. Hofer, J. Carroll, J. Neitz, M. Neitz, and D. R. Williams. “Organization of the Human Trichromatic Cone Mosaic.” In: *The Journal of Neuroscience* 25.42 (2005), pp. 9669–9679.
- [211] J. Hoffman, E. Tzeng, T. Park, J.-Y. Zhu, P. Isola, K. Saenko, A. Efros, and T. Darrell. “Cycada: Cycle-Consistent Adversarial Domain Adaptation.” In: *International Conference on Machine Learning*. PMLR. 2018, pp. 1989–1998.
- [212] D. Hoiem, A. A. Efros, and M. Hebert. “Automatic Photo Pop-Up.” In: *ACM Transactions on Graphics* 24.3 (2005), pp. 577–584.
- [213] D. Hoiem, A. A. Efros, and M. Hebert. “Putting Objects in Perspective.” In: *International Journal of Computer Vision* 80.1 (2008), pp. 3–15.
- [214] W. Hoogterp. *Your Perfect Presentation*. McGraw-Hill Education eBooks, 2014.
- [215] J. J. Hopfield. “Neural Networks and Physical Systems with Emergent Collective Computational Abilities.” In: *Proceedings of the National Academy of Sciences* 79.8 (1982), pp. 2554–2558.
- [216] B. K. P. Horn. *Robot Vision*. Cambridge, MA: MIT Press, 1986.
- [217] B. K. P. Horn and M. J. Brooks, eds. *Shape from Shading*. Cambridge, MA: MIT Press, 1989.
- [218] B. K. P. Horn and B. G. Schunck. “Determining Optical Flow.” In: *Artificial Intelligence* 17 (1981), pp. 185–203.
- [219] B. K. P. Horn. “Understanding Image Intensities.” In: *Artificial Intelligence* 8.2 (1977), pp. 201–231.
- [220] A. Hosni, C. Rhemann, M. Bleyer, C. Rother, and M. Gelautz. “Fast Cost-Volume Filtering for Visual Correspondence and Beyond.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.2 (2013), pp. 504–511.
- [221] S. J. A. Hospital. 2021. URL: <http://www.sanjoseanimalhospital.com/puppy-and-kitten-packages>.
- [222] P. V. C. Hough. “Machine Analysis of Bubble Chamber Pictures.” In: *International Conference on High Energy Accelerators and Instrumentation, CERN, 1959*. 1959, pp. 554–556.
- [223] R. Houthooft, R. Y. Chen, P. Isola, B. C. Stadie, F. Wolski, J. Ho, and P. Abbeel. “Evolved Policy Gradients.” In: *Advances in Neural Information Processing Systems*. 2018.
- [224] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. “LoRa: Low-Rank Adaptation of Large Language Models.” In: *International Conference on Learning Representations*. 2022.
- [225] Y. Huang and R. P. N. Rao. “Predictive coding.” In: *Wiley Interdisciplinary Reviews: Cognitive Science* 2.5 (2011), pp. 580–593.

- [226] D. H. Hubel and T. N. Wiesel. "Receptive Fields of Single Neurones in the Cat's Striate Cortex." In: *The Journal of Physiology* 148.3 (1959), pp. 574–591. DOI: 10.1113/jphysiol.1959.sp006308.
- [227] D. H. Hubel and T. N. Wiesel. "Receptive Fields, Binocular Interaction and Functional Architecture in the Cat's Visual Cortex." In: *J. Physiol.* 160 (1962), pp. 106–154.
- [228] B. Hutchinson and M. Mitchell. "50 Years of Test (Un)fairness: Lessons for Machine Learning." In: *FAT* '19: Proceedings of the Conference on Fairness, Accountability, and Transparency*. 2019, pp. 49–58.
- [229] A. Huxley. *Brave New World*. Chatto and Windus, 1932.
- [230] S. Ioffe and C. Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift." In: *International Conference on Machine Learning*. 2015, pp. 448–456.
- [231] P. Isola, J. J. Lim, and E. H. Adelson. "Discovering States and Transformations in Image Collections." In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2015.
- [232] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros. "Image-to-Image Translation with Conditional Adversarial Networks." In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2017.
- [233] M. Jaderberg, K. Simonyan, A. Zisserman, and k. kavukcuoglu koray. "Spatial Transformer Networks." In: *Advances in Neural Information Processing Systems*. 2015, pp. 2017–2025.
- [234] A. Jahanian, X. Puig, Y. Tian, and P. Isola. "Generative Models as a Data Source for Multiview Representation Learning." In: *International Conference on Learning Representations*. 2022.
- [235] W. H. Jefferys and J. O. Berger. "Ockham's Razor and Bayesian Analysis." In: *American Scientist* 80.1 (1992), pp. 64–72.
- [236] H. Jeffreys. *Scientific Inference*. 3rd ed. Cambridge University Press, 1973.
- [237] M. Jia, L. Tang, B.-C. Chen, C. Cardie, S. Belongie, B. Hariharan, and S.-N. Lim. "Visual Prompt Tuning." In: *European Conference on Computer Vision*. 2022, pp. 709–727.
- [238] G. Johansson. "Visual Perception of Biological Motion and a Model for Its Analysis." In: *Perception & Psychophysics* 14.2 (1973), pp. 201–211.
- [239] M. I. Jordan, ed. *Learning in Graphical Models*. Cambridge MA: MIT Press, 1998.
- [240] B. Julesz. "Textons, the Elements of Texture Perception, and Their Interactions." In: *Nature* 290.5802 (1981), pp. 91–97.
- [241] B. Julez. *Foundations of Cyclopean Perception*. University of Chicago Press, 1971.
- [242] M. Kac. "Can One Hear the Shape of a Drum?" In: *American Mathematical Monthly* (1966).
- [243] J. Kahn. "HireVue Drops Facial Monitoring Amid AI Algorithm Audit." In: *Fortune* (2021).
- [244] D. Kahneman. *Thinking, fast and slow*. Macmillan, 2011.
- [245] J. T. Kajiya. "The Rendering Equation." In: *ACM SIGGRAPH: Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques*. 1986, pp. 143–150.
- [246] J. Kajiya. "How to Get Your SIGGRAPH Paper Rejected." In: *SIGGRAPH Papers Chair*. 1993.

- [247] M. E. Kalderon. *Form Without Matter: Empedocles and Aristotle on Color Perception*. Oxford, UK: Oxford University Press, 2015.
- [248] E. R. Kandel, J. H. Schwartz, and T. M. Jessell, eds. *Principles of Neural Science*. Third. New York: Elsevier, 1991.
- [249] G. Kanizsa. *Organization in Vision: Essays on Gestalt Perception*. New York: Praeger Publishers, 1979.
- [250] N. G. Kanwisher, J. McDermott, and M. M. Chun. “The Fusiform Face Area: A Module in Human Extrastriate Cortex Specialized for Face Perception.” In: *The Journal of Neuroscience* 17 (1997), pp. 4302–4311.
- [251] A. Karnieli, O. Fried, and Y. Hel-Or. “DeepShadow: Neural Shape From Shadow.” In: *European Conference on Computer Vision*. 2022, pp. 415–430.
- [252] T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen, and T. Aila. “Alias-Free Generative Adversarial Networks”. In: *Advances in Neural Information Processing Systems*. 2021.
- [253] E. L. Kaufman and M. W. Lord. “The Discrimination of Visual Number.” In: *The American Journal of Psychology* 62.4 (1949), pp. 498–525.
- [254] M. Kearns and A. Roth. *The Ethical Algorithm: The Science of Socially Aware Algorithm Design*. Oxford University Press, 2020.
- [255] A. Kendall, H. Martirosyan, S. Dasgupta, P. Henry, R. Kennedy, A. Bachrach, and A. Bry. “End-to-End Learning of Geometry and Context for Deep Stereo Regression.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2017.
- [256] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. “3D Gaussian Splatting for Real-Time Radiance Field Rendering.” In: *ACM Transactions on Graphics* 42.4 (2023), pp. 1–14.
- [257] T. Kim, M. Cha, H. Kim, J. K. Lee, and J. Kim. “Learning to Discover Cross-Domain Relations with Generative Adversarial Networks.” In: *International Conference on Machine Learning*. 2017, pp. 1857–1865.
- [258] F. A. A. Kingdom, A. Yoonessi, and E. Gheorghiu. “The Leaning Tower Illusion.” In: *The Oxford Compendium of Visual Illusions*. Oxford University Press, July 2017.
- [259] D. Kingma, T. Salimans, B. Poole, and J. Ho. “Variational Diffusion Models.” In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021, pp. 21696–21707.
- [260] D. P. Kingma, M. Welling, et al. “An Introduction to Variational Autoencoders.” In: *Foundations and Trends in Machine Learning* 12.4 (2019), pp. 307–392.
- [261] D. P. Kingma and M. Welling. “Auto-Encoding Variational Bayes.” In: *International Conference on Learning Representations* (2014).
- [262] D. P. Kingma and J. Ba. *Adam: A Method for Stochastic Optimization*. 2014. arXiv: 1412.6980.
- [263] T. Kipf and M. Welling. *Semi-Supervised Classification with Graph Convolutional Networks*. 2017. arXiv: 1609.02907.
- [264] A. Kirillov et al. “Segment Anything”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023, pp. 4015–4026.
- [265] B. F. Klare, M. J. Burge, J. C. Klontz, R. W. V. Bruegge, and A. K. Jain. “Face Recognition Performance: Role of Demographic Information.” In: *IEEE Trans. on Information Forensics and Security* 7.6 (2012), pp. 1789–1801.

- [266] D. C. Knill, P. Mamassian, and D. Kersten. “Geometry of Shadows.” In: *Journal of the Optical Society of America A* 14.12 (1997), pp. 3216–3232.
- [267] D. E. Knuth, T. L. Larrabee, and P. M. Roberts. *Mathematical Writing*. Series Number 14. Mathematical Association of America Notes, 1989.
- [268] C. Koch and S. Ullman. “Shifts in Selective Visual Attention: Towards the Underlying Neural Circuitry.” In: *Human Neurobiology* 4.4 (1985), pp. 219–227.
- [269] J. J. Koenderink. “Image Structure.” In: *Mathematics and Computer Science in Medical Imaging*. Ed. by M. A. Viergever and A. E. Todd-Pokropek. Berlin: Springer-Verlag, 1988, pp. 67–103.
- [270] J. J. Koenderink. *Solid Shape*. Cambridge, MA: MIT Press, 1990.
- [271] J. J. Koenderink and A. J. van Doorn. “Representation of Local Geometry in the Visual System.” In: *Biological Cybernetics* 55 (1987), pp. 367–375.
- [272] K. Koffka. *Principles of Gestalt Psychology*. London: Routledge, 1935.
- [273] P. W. Koh, T. Nguyen, Y. S. Tang, S. Musmann, E. Pierson, B. Kim, and P. Liang. “Concept Bottleneck Models.” In: *International Conference on Machine Learning*. 2020, pp. 5338–5348.
- [274] D. Koller and N. Friedman, eds. *Probabilistic Graphical Models*. Cambridge MA: MIT Press, 2009.
- [275] V. Kolmogorov. “Convergent Tree-Reweighted Message Passing for Energy Minimization.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 28.10 (2006).
- [276] N. Komodakis and S. Gidaris. “Unsupervised Representation Learning by Predicting Image Rotations.” In: *International Conference on Learning Representations*. 2018.
- [277] R. Krishna et al. “Visual Genome: Connecting Language and Vision Using Crowdsourced Dense Image Annotations.” In: *International Journal of Computer Vision* 123.1 (2017), pp. 32–73.
- [278] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. Tech. rep. University of Toronto, Toronto, 2009.
- [279] A. Krizhevsky, I. Sutskever, and G. E. Hinton. “Imagenet Classification with Deep Convolutional Neural Networks.” In: vol. 25. 2012, pp. 1097–1105.
- [280] S. W. Kuffler. “Discharge Patterns and Functional Organization of Mammalian Retina.” In: *Journal of Neurophysiology* 16.1 (1953), pp. 37–68.
- [281] A. Lamott. *Bird by Bird*. Bantam Doubleday Dell Publishing Group, 1980.
- [282] E. H. Land. “Recent Advances in Retinex Theory and Some Implications for Cortical Computations: Color Vision and the Natural Image.” In: *Proceedings of the National Academy of Sciences of the USA* 80 (1983), pp. 5163–5169.
- [283] E. H. Land and J. J. McCann. “Lightness and Retinex Theory.” In: *Journal of the Optical Society of America* 61 1 (1971), pp. 1–11.
- [284] G. Larsson, M. Maire, and G. Shakhnarovich. “Learning Representations for Automatic Colorization.” In: *European Conference on Computer Vision*. Springer. 2016, pp. 577–593.
- [285] Y. LeCun. *A Tutorial on Energy-Based Learning*. 2006. URL: <http://www.cs.toronto.edu/~vnair/ciar/lecun1.pdf>.
- [286] Y. LeCun. *Energy-Based Models: The Cure Against Bayesian Fundamentalism*. 2007. URL: <https://www.mit.edu/~9.520/spring07/Courses/lecun-20070502-mit.pdf>.

- [287] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. “Backpropagation Applied to Handwritten Zip Code Recognition.” In: *Neural Computation* 1.4 (1989), pp. 541–551.
- [288] J. D. Lee, M. Simchowitz, M. I. Jordan, and B. Recht. “Gradient Descent Only Converges to Minimizers.” In: *Conference on Learning Theory*. 2016, pp. 1246–1257.
- [289] V. Lepetit and P. Fua. “Keypoint Recognition Using Randomized Trees.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 28.9 (2006), pp. 1465–1479.
- [290] T. Leung, M. Burl, and P. Perona. “Finding Faces in Cluttered Scenes Using Random Labeled Graph Matching.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 1995, pp. 637–644.
- [291] A. Levin, Y. Weiss, F. Durand, and W. T. Freeman. “Efficient Marginal Likelihood Optimization in Blind Deconvolution.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2011.
- [292] M. Levoy. *Optics I: Lenses and Apertures*. Stanford University Course. 2010. URL: <https://graphics.stanford.edu/courses/cs178-13/lectures/optics1-09apr13.pdf>.
- [293] M. Levoy and P. Hanrahan. “Light Field Rendering.” In: *ACM SIGGRAPH: Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques*. 1996, pp. 31–42.
- [294] F.-F. Li, M. Andreeto, M. Ranzato, and P. Perona. *Caltech 101*. 2022.
- [295] J. Li, D. Li, S. Savarese, and S. Hoi. “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models.” In: *International Conference on Machine Learning*. 2023.
- [296] Z. Li and N. Snavely. “MegaDepth: Learning Single-View Depth Prediction from Internet Photos.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2018.
- [297] T. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie. “Feature Pyramid Networks for Object Detection.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2017, pp. 936–944.
- [298] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár. “Microsoft COCO: Common Objects in Context.” In: *European Conference on Computer Vision*. 2014, pp. 740–755.
- [299] T. Lindeberg. *Scale-Space Theory in Computer Vision*. Kluwer Academic Publishers, 1994.
- [300] R. Linsker. “Self-Organization in a Perceptual Network.” In: *Computer* 21.3 (1988), pp. 105–117.
- [301] L. Lipson, Z. Teed, and J. Deng. “RAFT-Stereo: Multilevel Recurrent Field Transforms for Stereo Matching.” In: *International Conference on 3D Vision (3DV)*. 2021.
- [302] C. Liu, W. T. Freeman, E. H. Adelson, and Y. Weiss. “Human-Assisted Motion Annotation.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2008, pp. 1–8.
- [303] R. Liu, J. Lehman, P. Molino, F. P. Such, E. Frank, A. Sergeev, and J. Yosinski. *An Intriguing Failing of Convolutional Neural Networks and the Coordconv Solution*. 2018. arXiv: 1807.03247.
- [304] M. S. Livingstone and D. H. Hubel. “Anatomy and Physiology of a Color System in the Primate Visual Cortex.” In: *The Journal of neuroscience*. 1984.
- [305] J. Long, E. Shelhamer, and T. Darrell. “Fully Convolutional Networks for Semantic Segmentation.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*.

2015, pp. 3431–3440.

- [306] H. C. Longuet-Higgins. “A Computer Algorithm for Reconstructing a Scene from Two Projections.” In: *Nature* 293 (1981), pp. 133–135.
- [307] C. Loop and Z. Zhang. “Computing Rectifying Homographies for Stereo Vision.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. Vol. 1. 1999, pp. 125–131.
- [308] I. Loshchilov and F. Hutter. “SGDR: Stochastic Gradient Descent with Warm Restarts.” In: *International Conference on Learning Representations*. 2017.
- [309] D. G. Lowe. “Distinctive Image Features from Scale-Invariant Keypoints.” In: *International Journal of Computer Vision* 60.2 (2004), pp. 91–110.
- [310] D. G. Lowe. *Perceptual Organization and Visual Recognition*. Kluwer Academic Publishers, 1985.
- [311] B. D. Lucas and T. Kanade. “An Iterative Image Registration Technique with an Application to Stereo Vision.” In: *Proceedings of Imaging Understanding Workshop*. 1981, pp. 121–130.
- [312] L. v. d. Maaten and G. Hinton. “Visualizing Data Using t-SNE.” In: *Journal of Machine Learning Research* 9.Nov (2008), pp. 2579–2605.
- [313] D. J. MacKay. *Information Theory, Inference and Learning Algorithms*. Cambridge University Press, 2003.
- [314] C. Madison, W. Thompson, D. Kersten, P. Shirley, and B. Smits. “Use of Interreflection and Shadow for Surface Contact.” In: *Perception & Psychophysics* 63.2 (2001), pp. 187–194.
- [315] J. Malik and P. Perona. “Preattentive Texture Discrimination with Early Vision Mechanisms.” In: *Journal of the Optical Society of America A* 7 (1990), pp. 923–931.
- [316] T. Malisiewicz and A. A. Efros. “Beyond Categories: The Visual Memex Model for Reasoning About Object Relationships.” In: *Advances in Neural Information Processing Systems*. 2009.
- [317] D. Marr. *Vision: a Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman, San Francisco, 1982.
- [318] D. Marr. *Vision*. Cambridge, MA: MIT Press, 2010.
- [319] D. C. Marr and E. Hildreth. “Theory of Edge Detection.” In: *Proceedings of the Royal Society B* 207 (1980), pp. 187–217.
- [320] J. Martens and R. Grosse. “Optimizing Neural Networks with Kronecker-Factored Approximate Curvature.” In: *International Conference on Machine Learning*. 2015, pp. 2408–2417.
- [321] R. Martens. “Optics: Paralipomena to Witelo, and Optical Part of Astronomy. Johannes Kepler. Translated by William H. Donahue.” In: *Isis* 92.3 (2001), pp. 607–608.
- [322] J. Matas, O. Chum, M. Urban, and T. Pajdla. “Robust Wide Baseline Stereo from Maximally Stable Extremal Regions.” In: *Image and Vision Computing* 22 (Sept. 2004), pp. 761–767.
- [323] G. Matheron. *Random Sets and Integral Geometry*. Wiley, 1975.
- [324] W. Matusik, H. Pfister, M. Brand, and L. McMillan. “Directional Reflectance and Emissivity of an Opaque Surface.” In: *ACM Transactions on Graphics* 22 (3 2002).
- [325] N. Max. “Optical Models for Direct Volume Rendering.” In: *IEEE Transactions on Visualization and Computer Graphics* 1.2 (1995), pp. 99–108.

- [326] N. Max and M. Chen. *Local and Global Illumination in the Volume Rendering Integral*. Tech. rep. Lawrence Livermore National Lab.(LLNL), Livermore, CA (United States), 2005.
- [327] N. Mayer, E. Ilg, P. Hausser, P. Fischer, D. C. abd A. Dosovitskiy, and T. Brox. “A Large Dataset to Train Convolutional Networks for Disparity, Optical Flow, and Scene Flow Estimation.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2016.
- [328] J. J. McCann, S. P. McKee, and T. H. Taylor. “Quantitative Studies in Retinex Theory: a Comparison Between Theoretical Predictions and Observer Responses to the ‘Color Mondrian’ Experiments.” In: *Vision Research* 16 (1976), pp. 445–458.
- [329] J. McManus. *Real Titanic 3D*. 2022. URL: <http://www.realtitanic3d.com/>.
- [330] C. Mead. *Analog VLSI and Neural Systems*. Boston: Addison-Wesley Longman Publishing, 1989.
- [331] N. D. Mermin. “What's Wrong with These Equations?” In: *Physics Today* (1989).
- [332] R. M. Mersereau. “The Processing of Hexagonally Sampled Two-Dimensional Signals.” In: *Proceedings of the IEEE* 67.6 (1979), pp. 930–949.
- [333] K. Mikolajczyk and C. Schmid. “A Performance Evaluation of Local Descriptors.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 27.10 (2005), pp. 1615–1630.
- [334] K. Mikolajczyk and C. Schmid. “Indexing Based on Scale Invariant Interest Points.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. Vol. 2. 2001, p. 525.
- [335] K. Mikolajczyk and C. Schmid. “An Affine Invariant Interest Point Detector.” In: *European Conference on Computer Vision*. Ed. by A. Heyden, G. Sparr, M. Nielsen, and P. Johansen. 2002, pp. 128–142.
- [336] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. “Nerf: Representing Scenes as Neural Radiance Fields for View Synthesis.” In: *European Conference on Computer Vision*. Springer. 2020, pp. 405–421.
- [337] M. Minnaert. *Light and Color in the Outdoors*. New York: Springer New York, 1993. ISBN: 1-4612-2722-4.
- [338] M. Minnaert. *Light and Color in the Outdoors*. New York: Springer, 2012.
- [339] M. Minsky and S. Papert. *Perceptrons*. Cambridge, MA: MIT Press, 1969.
- [340] B. Moghaddam and A. P. Pentland. “Probabilistic Visual Learning for Object Representation.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 19.7 (1997), pp. 696–710.
- [341] P. Mozur. “One Month, 500,000 Face Scans”. In: *New York Times* (2019).
- [342] S. Mullainathan. “Biased Algorithms Are Easier to Fix Than Biased People.” In: *New York Times* (2019).
- [343] J. Mundy. “Object Recognition in the Geometric Era: A Retrospective.” In: vol. 4170. 2006, pp. 3–28.
- [344] I. Murakami, A. Kitaoka, and H. Ashida. “Artificial Image Oscillation Enhances the Rotating Snakes Illusion.” In: *Journal of Vision* 6.6 (2010), p. 551.
- [345] C. J. Murphy and H. C. Howland. “On the Gekko Pupil and Scheiner's Disc.” In: *Vision Research* 26.5 (1986), pp. 815–817.
- [346] K. Murphy, Y. Weiss, and M. I. Jordan. “Loopy Belief Propagation for Approximate Inference: An Empirical Study.” In: *Proceedings of the Fifteenth Conference on Uncertainty in Artificial*

- Intelligence*. 1999, pp. 467–475.
- [347] K. P. Murphy. *Probabilistic Machine Learning: An introduction*. Cambridge, MA: MIT Press, 2022.
- [348] P. K. Nathan Silberman Derek Hoiem and R. Fergus. “Indoor Segmentation and Support Inference from RGBD Images.” In: *European Conference on Computer Vision*. 2012.
- [349] B. Navez. Self-photographed, CC BY-SA 3.0, 2014. URL: <https://commons.wikimedia.org/w/index.php?curid=855487>.
- [350] S. K. Nayar, K. Ikeuchi, and T. Kanade. “Shape from Interreflections.” In: *International Journal of Computer Vision* 6.3 (1991), pp. 173–195.
- [351] L. Necker. “Observations on Some Remarkable Optical Phaenomena seen in Switzerland; and on an Optical Phaenomenon which Occurs on Viewing a Figure of a Crystal or Geometrical Solid.” In: *London and Edinburgh Philosophical Magazine and Journal of Science* 5.1 (2005), pp. 329–337.
- [352] S. A. Nene, S. K. Nayar, and H. Murase. *Columbia Object Image Library (COIL-20)*. Tech. rep. Department of Computer Science, Columbia University, 1996.
- [353] Y. Nesterov. “A Method for Unconstrained Convex Minimization Problem with the Rate of Convergence $O(1/k^2)$.” In: *Doklady an USSR*. Vol. 269. 1983, pp. 543–547.
- [354] A. Y. Ng, M. I. Jordan, and Y. Weiss. “On Spectral Clustering: Analysis and an Algorithm.” In: *Advances in Neural Information Processing Systems*. 2001, pp. 849–856.
- [355] F. E. Nicodemus. “Directional Reflectance and Emissivity of an Opaque Surface.” In: *Applied Optics* 4 (7 1965), pp. 767–775.
- [356] S. U. Noble. *Algorithms of Oppression*. NYU Press, Inc., 2018.
- [357] C. Olah. *Understanding LSTM Networks*. 2015.
- [358] C. Olah, A. Mordvintsev, and L. Schubert. “Feature Visualization.” In: *Distill* 2.11 (2017), e7.
- [359] A. Oliva and A. Torralba. “Modeling the Shape of the Scene: A Holistic Representation of the Spatial Envelope.” In: *International Journal of Computer Vision* 42(3) (2001), pp. 145–175.
- [360] A. v. d. Oord, Y. Li, and O. Vinyals. *Representation Learning with Contrastive Predictive Coding*. 2018. arXiv: 1807.03748.
- [361] OpenAI. *GPT-4 Technical Report*. 2023. arXiv: 2303.08774.
- [362] OpenAI. *GPT-4V(ision) System Card*. 2024.
- [363] A. Oppenheim and J. Lim. “The Importance of Phase in Signals.” In: *Proceedings of the IEEE* 69.5 (1981), pp. 529–541.
- [364] A. V. Oppenheim, A. S. Willsky, and S. H. Nawab. *Signals and Systems, 2nd ed.* Hoboken, NJ: Prentice-Hall, 1996.
- [365] V. Ordonez, G. Kulkarni, and T. Berg. “Im2text: Describing Images Using 1 Million Captioned Photographs.” In: *Advances in Neural Information Processing Systems*. Vol. 24. 2011.
- [366] M. Oren and S. K. Nayar. “Generalization of Lambert's Reflection Model.” In: *ACM SIGGRAPH: Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques*. 1994, pp. 239–246.
- [367] G. Orwell. *1984*. Prabhat Prakashan, 1948.

- [368] A. B. Owen. *Monte Carlo Theory, Methods and Examples*. 2013. URL: <https://artowen.su.domains/mc/>.
- [369] A. Owens, P. Isola, J. McDermott, A. Torralba, E. H. Adelson, and W. T. Freeman. “Visually Indicated Sounds.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2016.
- [370] I. Palmer. “Rethinking Perceptual Organization: The Role of Uniform Connectedness.” In: *Psychonomic Bulletin & Review*. 1.1 (1994).
- [371] S. Palmer, E. Rosch, and P. Chase. “Canonical Perspective and the Perception of Objects.” In: *International Symposium on Attention and Performance (Attention and performance IX)*. (1981), pp. 135–151.
- [372] S. E. Palmer. *Vision Science: Photons to Phenomenology*. Cambridge, MA: MIT Press, 1999.
- [373] Pantone. *Munsell USDA Frozen French Fry Standard*. 2020. URL: <https://www.pantone.com/products/munsell/munsell-usda-frozen-french-fry-standard>.
- [374] S. Papert. *The Summer Vision Project*. MIT AI Memo 100. Massachusetts Institute of Technology, Project Mac, 1966.
- [375] D. Parikh and D. Batra. *CVPR18 Workshop Panel: How to be a Good Citizen of the CVPR Community*. 2018.
- [376] S. Paris, P. Kornprobst, J. Tumblin, and F. Durand. *Bilateral Filtering: Theory and Applications*. Now Publishers, 2009.
- [377] Y. I. H. Parish and P. Müller. “Procedural Modeling of Cities.” In: *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*. 2001, pp. 301–308.
- [378] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. “Pytorch: An Imperative Style, High-Performance Deep Learning Library.” In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019.
- [379] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell. “Curiosity-Driven Exploration by Self-Supervised Prediction.” In: *International Conference on Machine Learning*. 2017, pp. 2778–2787.
- [380] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros. “Context Encoders: Feature Learning by Inpainting.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2016, pp. 2536–2544.
- [381] D. G. Pell, P. Cavanagh, R. Desimone, B. Tjan, and A. Treisman. “Crowding: Including Illusory Conjunctions, Surround Suppression, and Attention.” In: *Journal of Vision* 7.2 (2007), p. 1.
- [382] A. P. Pentland. “Linear Shape from Shading.” In: *International Journal of Computer Vision* 1.4 (1990), pp. 153–162.
- [383] P. Perona. “Deformable Kernels for Early Vision.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 17.5 (1995), pp. 488–499.
- [384] P. Perona and J. Malik. “Detecting and Localizing Edges Composed of Steps, Peaks and Roofs.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 1990.
- [385] P. Perona and J. Malik. “Scale-Space and Edge Detection Using Anisotropic Diffusion.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 12.7 (1990), pp. 629–639.
- [386] B. T. Phong. “Illumination for Computer Generated Pictures.” In: *Commun. ACM* 18.6 (1975), pp. 311–317.

- [387] M. Photos. Flickr. URL: <http://www.flickr.com/photos/mnsomero/2738807250/>.
- [388] Plato. Trans. by B. Jowett. 360 BCE. URL: <https://classics.mit.edu/Plato/timaeus.html>.
- [389] T. Poggio, V. Torre, and C. Koch. “Computational Vision and Regularization Theory.” In: *Nature* 317.26 (1985), pp. 314–139.
- [390] M. Pollefeys, R. Koch, and L. V. Gool. “A Simple and Efficient Rectification Method for General Motion.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 1999, pp. 496–501.
- [391] B. T. Polyak. “Some Methods of Speeding Up the Convergence of Iteration Methods.” In: *USSR Computational Mathematics and Mathematical Physics* 4.5 (1964), pp. 1–17.
- [392] J. Portilla and E. P. Simoncelli. “A Parametric Texture Model Based on Joint Statistics of Complex Wavelet Coefficients.” In: *International Journal of Computer Vision* 40.1 (2000), pp. 49–71.
- [393] S. Prince. *Computer Vision: Models Learning and Inference*. Cambridge University Press, 2012.
- [394] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, et al. “Learning Transferable Visual Models from Natural Language Supervision.” In: *International Conference on Machine Learning*. 2021, pp. 8748–8763.
- [395] V. V. Ramaswamy, W. T. Freeman, F.-F. Li, P. Perona, A. Torralba, and O. Russakovsky. *The Future of Computer Vision Datasets*. Computer Vision and Pattern Recognition Workshop. 2021.
- [396] V. V. Ramaswamy, S. S. Y. Kim, and O. Russakovsky. “Fair Attribute Classification through Latent Space De-biasing.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2021.
- [397] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen. *Hierarchical Text-Conditional Image Generation with Clip Latents*. 2022. arXiv: 2204.06125.
- [398] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever. “Zero-Shot Text-to-Image Generation.” In: *International Conference on Machine Learning*. 2021, pp. 8821–8831.
- [399] S. Ramón y Cajal. “La Rétine des Vertébrés.” In: *Cellule* 9 (1893), pp. 119–255.
- [400] R. Ranftl, A. Bochkovskiy, and V. Koltun. “Vision Transformers for Dense Prediction.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2021.
- [401] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun. “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-Shot Cross-Dataset Transfer.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.3 (2022).
- [402] A. Recasens, P. Luc, J.-B. Alayrac, L. Wang, F. Strub, C. Tallec, M. Malinowski, V. Patraucean, F. Altché, M. Valko, et al. “Broaden Your Views for Self-Supervised Video Learning.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2021).
- [403] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. “You Only Look Once: Unified, Real-Time Object Detection.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2016, pp. 779–788.
- [404] S. Ren, K. He, R. B. Girshick, and J. Sun. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.” In: *Advances in Neural Information Processing Systems*. 2015, pp. 91–99.

- [405] R. A. Rescorla. “A Theory of Pavlovian Conditioning: Variations in the Effectiveness of Reinforcement and Non-Reinforcement.” In: *Classical Conditioning, Current Research and Theory* 2 (1972), pp. 64–69.
- [406] L. G. Roberts. *Machine Perception of Three-Dimensional Solids*. Outstanding Dissertations in the Computer Sciences. New York: Garland Publishing, 1963.
- [407] A. Rodríguez-Muñoz and A. Torralba. *Aliasing is a Driver of Adversarial Attacks*. 2022. arXiv: 2212.11760.
- [408] P. Rogaway. “The Moral Character of Cryptographic Work?” In: *International Association for Cryptologic Research* (2015).
- [409] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. “High-Resolution Image Synthesis with Latent Diffusion Models.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2022, pp. 10684–10695.
- [410] O. Ronneberger, P. Fischer, and T. Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation.” In: *International Conference on Medical Image Computing and computer-Assisted Intervention*. Springer. 2015, pp. 234–241.
- [411] E. Rosch. *Principles of categorization*. De Gruyter Mouton, 1978, pp. 27–48.
- [412] E. Rosch, C. B. Mervis, W. D. Gray, D. M. Johnson, and P. Boyes-Braem. “Basic Objects in Natural Categories.” In: *Cognitive Psychology* 8 (1976), pp. 382–439.
- [413] F. Rosenblatt. “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain.” In: *Psychological review* 65.6 (1958), p. 386.
- [414] H. Rowley, S. Baluja, and T. Kanade. “Neural Network-Based Face Detection”. In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 1996, pp. 203–208.
- [415] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski. “ORB: An Efficient Alternative to SIFT or SURF.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2011, pp. 2564–2571.
- [416] D. L. Ruderman. “Origins of Scaling in Natural Images.” In: *Vision Research* 37.23 (1997), pp. 3385–3398.
- [417] D. E. Rumelhart and J. L. McClelland, eds. *Parallel Distributed Processing*. Cambridge, MA: MIT Press, 1986.
- [418] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. *Learning Internal Representations by Error Propagation*. Tech. rep. California Univ San Diego Inst for Cognitive Science, 1985.
- [419] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, et al. “Imagenet Large Scale Visual Recognition Challenge.” In: *International Journal of Computer Vision* 115.3 (2015), pp. 211–252.
- [420] B. C. Russell, A. Torralba, K. P. Murphy, and W. T. Freeman. “LabelMe: A Database and Web-based Tool for Image Annotation.” In: *International Journal of Computer Vision* 77 (2008), pp. 157–173.
- [421] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever. *Evolution Strategies as a Scalable Alternative to Reinforcement Learning*. 2017. arXiv: 1703.03864.
- [422] P. Sattigeri, S. C. Hoffman, V. Chenthamarakshan, and K. R. Varshney. “Fairness GAN: Generating Datasets with Fairness Properties Using a Generative Adversarial Network.” In: *Intl. Conf. on Learning Representations (ICLR) workshop*. 2019.

- [423] N. Savinov, A. Seki, L. Ladicky, T. Sattler, and M. Pollefeys. “Quad-Networks: Unsupervised Learning to Rank for Interest Point Detection.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2017, pp. 3929–3937.
- [424] A. Saxena, S. H. Chung, and A. Y. Ng. “3-D Depth Reconstruction from a Single Still Image.” In: *International Journal of Computer Vision* 76.1 (2008), pp. 53–69.
- [425] D. Scharstein and R. Szeliski. “A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms.” In: *International Journal of Computer Vision* 47 (2002). <https://vision.middlebury.edu/stereo/>.
- [426] C. Schmid, R. Mohr, and C. Bauckhage. “Evaluation of Interest Point Detectors.” In: *International Journal of Computer Vision* 37.2 (2000), pp. 151–172.
- [427] B. Schölkopf, A. Smola, and K.-R. Müller. “Nonlinear Component Analysis as a Kernel Eigenvalue Problem.” In: *Neural computation* 10.5 (1998), pp. 1299–1319.
- [428] M. Schrimpf, J. Kubilius, M. J. Lee, N. A. R. Murty, R. Ajemian, and J. J. DiCarlo. “Integrative Benchmarking to Advance Neurally Mechanistic Models of Human Intelligence.” In: *Neuron* (2020).
- [429] C. E. Shannon. “A Mathematical Theory of Communication.” In: *The Bell System Technical Journal* 27.3 (1948), pp. 379–423.
- [430] S. Shapin. “A Theorist of (Not Quite) Everything.” In: *The New York Review of Books* (2019).
- [431] R. N. Shepard. *Mind Sights: Original Visual Illusions, Ambiguities, and Other Anomalies, with a Commentary on the Play of Mind in Perception and Art*. New York: W.H. Freeman and Co., 1990.
- [432] R. N. Shepard and J. Metzler. “Mental Rotation of Three-Dimensional Objects.” In: *Science* 171.3972 (1971), pp. 701–703.
- [433] C. S. Sherrington. “Observations on the Scratch-Reflex in the Spinal Dog.” In: *The Journal of physiology* 34.1-2 (1906), pp. 1–50.
- [434] J. Shi and J. Malik. “Normalized Cuts and Image Segmentation.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.8 (2000), pp. 888–905.
- [435] J. Shi and J. Malik. “Normalized Cuts and Image Segmentation.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.8 (2000), pp. 888–905.
- [436] J. Shi and Tomasi. “Good Features to Track.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 1994, pp. 593–600.
- [437] P. Shirley, M. Ashikhmin, and S. Marschner. *Fundamentals of Computer Graphics*. AK Peters/CRC Press, 2009.
- [438] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al. “Mastering the Game of Go with Deep Neural Networks and Tree Search.” In: *Nature* 529.7587 (2016), pp. 484–489.
- [439] E. P. Simoncelli. “Statistical Modeling of Photographic Images.” In: *Handbook of Image and Video Processing*. Academic Press, 2005, pp. 431–441.
- [440] E. P. Simoncelli and E. H. Adelson. “Noise Removal via Bayesian Wavelet Coring.” In: *International Conference on Image Processing*. 1996, pp. 379–382.
- [441] E. P. Simoncelli and W. T. Freeman. “The Steerable Pyramid: a Flexible Architecture for Multi-Scale Derivative Computation.” In: *International Conference on Image Processing*. 1995.

- [442] E. P. Simoncelli, W. T. Freeman, E. H. Adelson, and D. J. Heeger. “Shiftable Multi-Scale Transforms.” In: *IEEE Transactions on Information Theory* 2.38 (1992), pp. 587–607.
- [443] E. P. Simoncelli and E. H. Adelson. “Subband Image Coding with Hexagonal Quadrature Mirror Filters.” In: *Picture Coding Symposium*. 1990.
- [444] K. Simonyan and A. Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition.” In: *International Conference on Learning Representations*. 2015.
- [445] V. Sitzmann, J. N. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein. “Implicit Neural Representations with Periodic Activation Functions.” In: *Advances in Neural Information Processing Systems*. 2020.
- [446] A. Smith. *Alhacen's Theory of Visual Perception: A Critical Edition, with English Translation and Commentary, of the First Three Books of Alhacen's De Aspectibus, the Medieval Latin Version of Ibn Al-Haytham's Kitab Al-Manazir*. v. 91, pt. 4. American Philosophical Society, 2001.
- [447] N. Snavely, S. M. Seitz, and R. Szeliski. “Photo Tourism: Exploring Photo Collections in 3D.” In: *ACM SIGGRAPH: Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques*. 2006, pp. 835–846.
- [448] J. Snell, K. Ridgeway, R. Liao, B. D. Roads, M. C. Mozer, and R. S. Zemel. “Learning to Generate Images with Perceptual Similarity Metrics.” In: *International Conference on Image Processing*. IEEE. 2017, pp. 4277–4281.
- [449] S. Soatto. “Actionable Information in Vision.” In: *Machine Learning for Computer Vision*. Springer, 2013, pp. 17–48.
- [450] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli. “Deep Unsupervised Learning Using Nonequilibrium Thermodynamics.” In: *International Conference on Machine Learning*. 2015, pp. 2256–2265.
- [451] G. Sperling, S.-H. Lyu, C.-H. Tseng, and Z.-L. Lu. “The Motion Standstill Illusion.” In: *The Oxford Compendium of Visual Illusions*. Oxford University Press, July 2017.
- [452] L. Spillmann. “Receptive Fields of Visual Neurons: The Early Years.” In: *Perception* 43 (2014), pp. 1145–1176.
- [453] R. K. Srivastava, K. Greff, and J. Schmidhuber. *Highway Networks*. 2015. arXiv: 1505.00387.
- [454] R. M. Steinman, Z. Pizlo, and F. J. Pizlo. “Phi Is Not Beta, and Why Wertheimer's Discovery Launched the Gestalt Revolution.” In: *Vision Research* 40.17 (2000), pp. 2257–2264.
- [455] W. Strunk and E. B. White. *The Elements of Style*. Boston: Allyn and Bacon, 1999.
- [456] P. Sturm and B. Triggs. “A Factorization Based Algorithm for Multi-image Projective Structure and Motion.” In: *European Conference on Computer Vision*. 1996, pp. 709–720.
- [457] D. Surís, S. Menon, and C. Vondrick. “ViperGPT: Visual Inference via Python Execution for Reasoning.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023.
- [458] I. Sutskever. Accessed: April 6, 2019. URL: <https://twitter.com/ilyasut/status/1114658175272095744?s=20>.
- [459] I. Sutskever, J. Martens, G. Dahl, and G. Hinton. “On the Importance of Initialization and Momentum in Deep Learning.” In: *International Conference on Machine Learning*. 2013, pp. 1139–1147.

- [460] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT press, 2018.
- [461] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. “Intriguing Properties of Neural Networks.” In: 2014.
- [462] R. Szeliski. *Computer Vision Algorithms and Applications*. 2nd ed. Springer, 2022.
- [463] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, and R. Ng. “Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains.” In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 7537–7547.
- [464] M. J. Tarr and S. Pinker. “Mental Rotation and Orientation-Dependence in Shape Recognition.” In: *Cognitive Psychology* 21 (1989), pp. 233–282.
- [465] M. Telgarsky. “Benefits of Depth in Neural Networks.” In: *Conference on Learning Theory*. PMLR. 2016, pp. 1517–1539.
- [466] M. Telgarsky. *Deep Learning Theory*. Lecture notes. 2021.
- [467] J. J. Thomson. “The Trolley Problem.” In: *The Yale Law Journal* 94.6 (1985), pp. 1395–1415.
- [468] Y. Tian, D. Krishnan, and P. Isola. “Contrastive Multiview Coding.” In: *European Conference on Computer Vision*. Springer. 2020, pp. 776–794.
- [469] K. Tieu and P. Viola. “Boosting Image Retrieval.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2000.
- [470] C. Tomasi and T. Kanade. “Shape and Motion from Image Streams under Orthography: a Factorization Method.” In: *International Journal of Computer Vision* 9.2 (1992), pp. 137–154.
- [471] A. Torralba and A. Efros. “Unbiased Look at Dataset Bias.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2011.
- [472] A. Torralba. “How Many Pixels Make an Image?” In: *Visual Neuroscience* 26.1 (2009), pp. 123–131.
- [473] A. Torralba, R. Fergus, and W. T. Freeman. “80 Million Tiny Images: A Large Data Set for Nonparametric Object and Scene Recognition.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 30.11 (2008), pp. 1958–1970.
- [474] A. Torralba and W. T. Freeman. “Accidental Pinhole and Pinspeck Cameras.” In: *International Journal of Computer Vision* 110.2 (2014), pp. 92–112.
- [475] J. Traer and J. H. McDermott. “Statistics of Natural Reverberation Enable Perceptual Separation of Sound and Space.” In: *Proceedings of the National Academy of Sciences* 113.48 (2016), E7856–E7865.
- [476] A. M. Treisman and G. Gelade. “A Feature-Integration Theory of Attention.” In: *Cognit Psychol* 12.1 (Jan. 1980), pp. 97–136.
- [477] E. Trucco and A. Verri. *Introductory Techniques for 3-D Computer Vision*. USA: Prentice Hall PTR, 1998.
- [478] A. M. Turing. *Computing Machinery and Intelligence*. Springer, 2009.
- [479] R. Tyleček and R. Šára. “Spatial Pattern Templates for Recognition of Objects with Regular Structure.” In: *Proceedings German Conference on Pattern Recognition*. Saarbrücken, Germany, 2013.

- [480] E. Tzeng, J. Hoffman, K. Saenko, and T. Darrell. “Adversarial Discriminative Domain Adaptation.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2017, pp. 7167–7176.
- [481] J. R. R. Uijlings, K. E. A. van de Sande, T. Gevers, and A. W. M. Smeulders. “Selective Search for Object Recognition.” In: *International Journal of Computer Vision* 104.2 (2013), pp. 154–171.
- [482] S. Ullman and S. Brenner. “The Interpretation of Structure from Motion.” In: *Proceedings of the Royal Society of London. Series B. Biological Sciences* 203.1153 (1979), pp. 405–426.
- [483] S. Ullman. *High-level Vision*. Cambridge, MA: MIT Press, 2000.
- [484] A. Van den Oord, N. Kalchbrenner, L. Espeholt, O. Vinyals, A. Graves, et al. “Conditional Image Generation with Pixelcnn Decoders.” In: *Advances in Neural Information Processing Systems*. Vol. 29. 2016.
- [485] A. van der Schaaf and J. van Hateren. “Modelling the Power Spectra of Natural Images: Statistics and Information.” In: *Vision Research* 36.17 (1996), pp. 2759–2770.
- [486] Vanessa Ezekowitz. *Eye Cone Responses*. GNU Free Documentation License. Data from Stockman, MacLeod, Johnson. 1993. URL: https://commons.wikimedia.org/wiki/File:Cones_SMJ2_E.svg.
- [487] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention Is All You Need.” In: *Advances in Neural Information Processing Systems*. 2017, pp. 5998–6008.
- [488] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol. “Extracting and Composing Robust Features with Denoising Autoencoders.” In: *International Conference on Machine Learning*. 2008, pp. 1096–1103.
- [489] P. Viola and M. Jones. “Rapid Object Detection Using a Boosted Cascade of Simple Classifiers.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2001.
- [490] D. L. Waltz. *Generating Semantic Descriptions From Drawings of Scenes with Shadows*. PhD Thesis, Artificial Intelligence Lab Memo. Massachusetts Institute of Technology, 1972.
- [491] B. Wandell. *Foundations of Vision*. Sinauer Assoc., 1995.
- [492] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell. *Fully Test-Time Adaptation by Entropy Minimization*. 2020. arXiv: 2006.10726.
- [493] J. Y. A. Wang and E. H. Adelson. “Representing Moving Images with Layers.” In: *IEEE Transactions on Image Processing* 3.5 (1994), pp. 625–638.
- [494] T. Wang and P. Isola. “Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere.” In: *International Conference on Machine Learning*. 2020, pp. 9929–9939.
- [495] Z. Wang, K. Qinami, I. C. Karakozis, K. Genova, P. Nair, K. Hata, and O. Russakovsky. “Towards Fairness in Visual Recognition: Effective Strategies for Bias Mitigation.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2020.
- [496] M. Weber, M. Welling, and P. Perona. “Towards Automatic Discovery of Object Categories.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. Vol. 2. 2000, pp. 101–108.
- [497] X. Wei, T. Zhang, Y. Li, Y. Zhang, and F. Wu. “Multi-Modality Cross Attention Network for Image and Sentence Matching.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern*

- Recognition*. 2020, pp. 10941–10950.
- [498] Y. Weiss. “Deriving Intrinsic Images from Image Sequences.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. Vol. 2. 2001, pp. 68–75.
 - [499] Y. Weiss and E. H. Adelson. *Slow and Smooth: A Bayesian Theory for the Combination of Local Motion Signals in Human Vision*. Tech. rep. A.I. Memo No. 1624. MIT, 1998.
 - [500] M. Wertheimer. “Experimentelle Studien Über das Sehen von Bewegung.” In: *Zeitschrift für Psychologie* 61 (1912).
 - [501] T. N. Wiesel. “The Postnatal Development of the Visual Cortex and the Influence of Environment.” In: *Nature* 299 (1982), pp. 583–591.
 - [502] Wikipedia. 2021. URL: https://en.wikipedia.org/wiki/Hermann_von_Helmholtz.
 - [503] Wikipedia. 2021. URL: https://en.wikipedia.org/wiki/Image_rectification.
 - [504] L. Williams. “Pyramidal Parametrics.” In: *ACM SIGGRAPH: Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques*. 1983, pp. 1–11.
 - [505] P. Winston. *How To Speak*. 2016. URL: <https://vimeo.com/101543862>.
 - [506] A. P. Witkin. “Recovering Surface Shape and Orientation from Texture.” In: *Artificial Intelligence* 17 (1981), pp. 17–45.
 - [507] J. M. Wolfe. “Guided Search 4.0: Current Progress with a Model of Visual Search.” In: *Integrated Models of Cognitive Systems*. Oxford University Press, 2007.
 - [508] J. M. Wolfe. “Visual Attention.” In: *Seeing* (2000), pp. 335–386.
 - [509] C. Wu, S. Yin, W. Qi, X. Wang, Z. Tang, and N. Duan. *Visual ChatGPT: Talking, Drawing and Editing with Visual Foundation Models*. 2023. arXiv: 2303.04671.
 - [510] F.-Y. Wu. “The Potts Model.” In: *Rev. Mod. Phys.* 54.1 (1982), pp. 235–268.
 - [511] Y. Wu and K. He. “Group Normalization.” In: *European Conference on Computer Vision*. 2018, pp. 3–19.
 - [512] K. Xian, C. Shen, Z. Cao, H. Lu, Y. Xiao, R. Li, and Z. Luo. “Monocular Relative Depth Perception with Web Stereo Data Supervision.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2018, pp. 311–320.
 - [513] J. S. Yedidia, W. T. Freeman, and Y. Weiss. “Generalized Belief Propagation.” In: *Advances in Neural Information Processing Systems*. Vol. 13. 2001, pp. 689–695.
 - [514] Z. Yi, H. Zhang, P. Tan, and M. Gong. “Dualgan: Unsupervised Dual Learning for Image-to-Image Translation.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2017, pp. 2849–2857.
 - [515] R. Zabih and V. Komogorov. “What Energy Functions Can Be Minimized via Graph Cuts?” In: *European Conf. Computer Vision*. Vol. 26. 2004, pp. 147–159.
 - [516] J. Zbontar and Y. LeCun. “Computing the Stereo Matching Cost with a Convolutional Neural Network.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2015, pp. 1592–1599.
 - [517] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. “Understanding Deep Learning (Still) Requires Rethinking Generalization.” In: *Communications of the ACM* 64.3 (2021), pp. 107–115.

- [518] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. *Understanding Deep Learning Requires Rethinking Generalization*. 2016. arXiv: 1611.03530.
- [519] F. Zhang, V. Prisacariu, R. Yang, and P. H. Torr. “GA-Net: Guided Aggregation Net for End-to-End Stereo Matching.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2019, pp. 185–194.
- [520] F. Zhang, X. Qi, R. Yang, V. Prisacariu, B. Wah, and P. Torr. “Domain-Invariant Stereo Matching Networks.” In: *European Conference on Computer Vision*. 2019.
- [521] L. Zhang, B. Curless, A. Hertzmann, and S. M. Seitz. “Shape and Motion Under Varying Illumination: Unifying Structure from Motion, Photometric Stereo, and Multi-View Stereo.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2003.
- [522] L. Zhang, A. Rao, and M. Agrawala. “Adding Conditional Control to Text-to-Image Diffusion Models.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2023.
- [523] R. Zhang, P. Isola, and A. A. Efros. “Colorful Image Colorization.” In: *European Conference on Computer Vision*. Springer. 2016, pp. 649–666.
- [524] R. Zhang, P. Isola, and A. A. Efros. “Split-Brain Autoencoders: Unsupervised Learning by Cross-Channel Prediction.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2017, pp. 1058–1067.
- [525] Z. Zhang. “A Flexible New Technique for Camera Calibration.” In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.11 (2000), pp. 1330–1334.
- [526] Z. Zhang. “Flexible Camera Calibration by Viewing a Plane from Unknown Orientations.” In: *Proceedings of the Seventh IEEE International Conference on Computer Vision*. Vol. 1. 1999, pp. 666–673.
- [527] J. Zhao, T. Wang, M. Yatskar, V. Ordonez, and K.-W. Chang. “Men Also Like Shopping: Reducing Gender Bias Amplification Using Corpus-Level Constraints.” In: *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2017.
- [528] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba. “Object Detectors Emerge in Deep Scene CNNs.” In: *International Conference on Learning Representations* (2015).
- [529] T. Zhou, M. Brown, N. Snavely, and D. G. Lowe. “Unsupervised Learning of Depth and Ego-Motion from Video.” In: *Proceedings of the IEEE/CVF Computer Vision and Pattern Recognition*. 2017, pp. 6612–6619.
- [530] Y. Zhou, H. Qi, J. Huang, and Y. Ma. “NeurVPS: Neural Vanishing Point Scanning via Conic Convolution.” In: *Advances in Neural Information Processing Systems*. 2019.
- [531] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros. “Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks.” In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2017.
- [532] A. Zisserman. *25 Years of RANSAC*. CVPR workshop, 2006. URL: <https://cmp.felk.cvut.cz/ransac-cvpr2006>.

Index

L_2 normalization, 188
0-1 loss, 147
2.5D representations, 556

Activation function, 176, 187
Adversarial attacks, 636
Adversarial example, 420
Adversarial robustness, 637
Adversarial training, 643
Aerial perspective, 758
Affine transformation, 666
Affinity-based segmentation, 553
Affordance, 17
Algorithmic Fairness, 67
Aliasing, 328
Ames room, 733
Amortized inference, 609
Ancestral sampling, 576
Aperture, 100
Aperture problem, 824
Apparent motion, 802
Approximation error, 164
Attention
 Attention layers, 445
 Causal attention, 460
 Cross-attention, 881
 Masked attention, 458
 Multihead self-attention, 452
 Query-Key-Value, 447
 Self-attention, 449
Autoencoders, 530
Autogressive model, 572
Autoregressive model
 Conditional, 612
 Context, 572

Backpropagation, 199

- Backpropagation through time, 433
- Backward mapping, 670
- Barber-pole illusion, 825
- Batch normalization, 187
- Batch processing, 185
- Batch size, 159
- Bayes' theorem, 510
- Bias, 176
- Biological motion, 769
- Block coordinate descent, 539
- Block world, 35
- Bounding box, 855
- BRDF, 77
 - Lambertian, 78
 - Phong model, 78
- Brightness constancy assumption, 826
- Bundle adjustment, 773
- Byte-pair encoding, 870

Camera

- Accidental camera, 83
- Corner camera, 112
- Edge camera, 111
- Light field cameras, 64
- Orthographic camera, 87
- Paper bag camera, 82
- Pinhole camera, 59, 79
- Pinspeck camera, 112
- Soda straw camera, 87
- Stereo pinhole, 704

Camera obscura, 8

Campbell and Robson chart, 271

Categorization, 846

Checkerboard-pattern noise, 285

Checkpoint, 195

Classification, 146

Classifier cascade, 856

Clique, 507

Clustering, 537

- Cluster center, 538

Code vector, 440, 538

Color

- CIE chromaticity coordinates, 131
- Lab space, 131
- Primary colors, 128

- Color histogram, 471
- Computation graph, 199
- Confusion matrix, 852
- Conic matrix, 754
- Constant brightness assumption, 316
- Contrast sensitivity function, 271, 306
- Contrastive divergence, 568
- Contrastive language-image pretraining, 870, 871
- Contrastive learning, 542
 - Alignment and uniformity, 545
 - Contrastive loss, 544
 - InfoNCE loss, 544
 - Triplet loss, 544
- Convolutional layer, 404
 - Dilated, 409
 - Strided, 407
- Convolutional neural nets, 403
- Cross-entropy loss, 147
- Crowding, 495
- Cycle-consistency, 619

- Data augmentation, 639
 - Generative, 642
 - Learned, 641
- Dataset bias, 68, 630
 - Annotation bias, 635
 - Collection bias, 634
 - Photographer bias, 632
 - Social bias, 635
 - Testing and benchmark bias, 635
- DC gain, 267, 277
- Decoder, 530
- Deep learning, 184
- Delta train, 330
- Dense optical flow, 825
- Density model, 565
- Depth of field, 100
- Diffusion model, 576
 - Conditional, 612
- Dimensionality reduction, 530
- Direct linear transform, 693
- Directed acyclic graph, 212
- Discrete sinc, 353
- Discriminator, 579
- Disentangled representation, 528, 598

- Disparity, 707
- Domain adaptation, 653
- Domain gap, 626
- Domain randomization, 640
- Downsampling layer, 413
- Dynamic pooling, 445

- Ecological optics, 16

- Edge

 - detection, 41
 - orientation, 42
 - strength, 42
 - types, 40

- Edge detection, 555

- Embedding, 193

- Empirical risk minimization, 141

- Encoder, 528, 530

- Encoder-decoder, 428

- Endpoint error, 837

- Energy-based models, 567

- Epipolar line, 713

- Epipolar plane, 713

- Epipole, 713

- Essential matrix, 714

- Euclidean transformation, 666

- Euler angles, 818

- Euler's formula, 246

- Evidence lower bound, 594

- Evolution strategies, 157

- Exploding gradient, 155

- Extramission theory, 5

- f-number, 100

- Fast Fourier transform, 248

- Feature map, 406

- Few-shot learning, 645

- Filter

 - [1, 2, 1], 284

 - Anti-aliasing, 340

 - Anticausal, 318

 - Binomial filter, 283

 - Box filter, 276

 - Causal, 318

 - Derivative, 288

- Gabor filter, 365
- Gaussian filter, 279
 - Discrete, 280
- Infinite impulse response, 319
- Laplacian, 302
- Nulling filter, 324
- Quadrature phase, 369
- Spatiotemporal, 318
- Spatiotemporal Gaussian, 319
- Stability, 319
- Triangular filter, 279
- Velocity-tuned, 381
- zero-phase, 285
- Filter bank, 372, 406
- Finetuning, 646
- First-order optimization, 152
- Focal length, 95
- Focal plane, 98
- Focus of expansion, 817
- Forward mapping, 670
- Fully connected layer, 416
- Fully convolutional network, 425
- Fundamental matrix, 715

- Gaussian derivative
 - Hermite polynomial, 295
 - Second order derivative, 295
- Gaussian mixture model, 587
- Generalization error, 164
- Generative adversarial network, 579
 - Conditional, 609
- Generative data, 654
- Generative models, 559, 603
 - Unconditional, 561
- Generative pretrained transformer, 870
- Generator, 560
- Generic view, 45
- Geometric optics, 89
- Geons, 844
- Gestalt psychology, 11, 43
 - Grouping laws, 12
- Gradient clipping, 158
- Gradient constraint equation, 827
- Gradient descent, 151
- Graph, 505

Hamming window, 355
Harris corner detector, 709
Hebbian learning, 184
Hexagonal sampling, 339
Hidden layer, 178
Histogram matching, 500
Horizon line, 739
Hypercolumn, 22
Hypernetwork, 211
Hypothesis space, 140

Ideal point, 661
Image captioning, 881
Image correspondence, 804
Image histogram, 471
Image-to-image translation, 614
 Paired, 614
 Unpaired, 617
Image-to-text, 881
Imaging, 77
Implicit representation, 787
Importance sampling, 591
Impulse, 230
Impulse train, 330
Imputation, 536
In-context learning, 656
Index of refraction, 91
Information bottleneck, 428, 542
Interest point, 774
Intersection over union, 860
Intrinsic images decomposition, 313
Intromission theory, 5
Invertible transform, 387

K-means, 538
K-means segmentation, 551
Kajiya's rendering equation, 797
Knowledge distillation, 649
Kullback-Leibler divergence, 565

Lanczos interpolation, 337
Language models, 869
LASSO regression, 166

- Latent variables, 560, 584
- Layer normalization, 188
- Layer-based representations, 556
- Learned function, 141
- Learning rate, 152
- Lens
 - concave, 102
 - convex, 97
 - thin, 93
- Lensmaker's Formula, 95
- Levels of abstraction, 846
- Light field, 108
- Light field rendering, 784
- Light ray, 77
- Lightness perception, 13
- Likelihood, 142, 510
- Linear layer, 176
- Linear least-squares regression, 142
- Linear probe, 649
- Local image descriptor, 777
- Local response normalization, 415
- Logits, 148
- Long Short-Term Memory, 436
- Loss function, 141
- Low-pass residual, 391
- Lumigraph, 784

- Margin, 544
- Marginal likelihood, 585
- Markov chain, 508
- Markov random field, 508
- Max likelihood learning, 566
- Max-product algorithm, 520
- Maxima cliques, 507
- Maximal clique, 507
- Maximum a posteriori, 510
- Maximum a posteriori learning, 142
- Maximum likelihood learning, 142
- Mental rotation, 845
- Message, 512
- Metalearning, 149
- Mexican hat wavelet, 302
- Minimum description length principle, 528
- Minimum mean squared error, 510
- MIP-mapping, 671

Misclassification error, 851
Mixture model, 587
Moire pattern, 343
Momentum, 153
Motion-inducing visual illusion, 803
Multilayer perceptron, 178
Multiresolution, 388
Multiscale image pyramid, 386
Multiscale oriented representation, 396
Multitask training, 643

Natural language processing, 869
Neocognitron, 29
Network depth, 184
Network width, 184
Neural radiance fields, 787
Nonaccidental image properties, 45
Nonidentifiability, 600
Nonlocal means, 489
Nonmaximum suppression, 857
Nonparametric texture model, 502
Normalization layers, 187
Nyquist limit, 334
Nyquist theorem, 330

Objective function, 140
Occam's razor, 165
One-hot code, 147
One-shot learning, 645
One-sided derivative, 155
Optical flow, 786, 823
Optimizer, 140
Oriented energy, 378
Orthographic projection, 86
Out-of-distribution generalization, 419, 625, 640
Overcomplete representation, 393
Overfitting, 161
Overparameterization, 141

Papers, 893
Parallax, 703
Parallel projection, 36
Parameter sharing, 214

- Parametric texture model, 501
- Pascal's triangle, 283
- Patch matching, 805
- PatchGAN, 616
- Perceptron, 28, 176
- Perceptual grouping, 541, 549
- Perceptual organization, 11, 550
 - Amodal completion, 14
 - Illusory contours, 15
 - Modal completion, 14
- Perlin noise, 480
- permutation equivariance, 455
- Perspective projection, 36, 85
- Photometric reconstruction loss, 831
- Photoreceptors, 19
- Pink noise, 473
- Pixel, 53
- Pixels, 786
- Plenoptic function, 3, 783
- Point prediction, 603
- Pooling layer, 413
 - Global pooling, 415
 - Max pooling, 414
 - Mean pooling, 414
- Positional encoding, 460
- Postactivation unit, 178
- Preactivation unit, 178
- Precision-recall curve, 862
- Predictive coding, 533
- Pretext task, 533
- Pretraining, 645
- Principle components analysis, 531
- Prior probability, 510
- Priors, 166
- Probability mass function, 147
- Procedural graphics, 561
- Program induction, 145
- Projective transformation, 667
- Prompting, 651
- Prototypes, 846

- Radial frequency, 246
- radiance field, 784
- Random dot stereograms, 703
- RANSAC, 728

- Ray casting, 790
- Read out module, 649
- Receptive field, 20
- Receptive fields, 423
- Reconstruction loss, 530
- Rectified linear unit, 181
- Recurrent layer, 433
 - Simple recurrent layer, 433
- Recurrent neural network, 431
- Refraction, 91
- Regularization, 165
- Reinforcement learning, 140
- Reparameterization trick, 596
- Reprojection error, 694, 773
- Research, 887
- Residual connections, 429
- ResNet, 429
- Retina, 20
- Retinex algorithm, 311
- Reward function, 140
- Ridge regression, 166
- Rigidity assumption, 770
- Roberts cross operator, 290, 300

- Sampling, 563
- Scale invariance, 386
- Scale space, 400
- Scale-invariant loss, 762
- Scene graph, 749
- Segmentation, 551
- Selective search, 856
- Self-inverting transform, 388
- Self-supervised learning, 535
- Semantic segmentation, 863
- semantic segmentation, 404
- Shading, 757
- Shape from shading, 757
- SIFT, 710
- Signal processing, 219
- Signals
 - Analytic signal, 370
 - Band-limited, 329
 - Continuous, 219
 - DC value, 222
 - Discrete, 219

- Energy, 222
- Finite energy, 222
- Finite length, 221
- Infinite energy, 222
- Infinite length, 221
- Periodic, 222
- Sampling, 219
- Signed distance function, 786
- Similarity transform, 666
- Simplex, 147
- Sinc function, 335
- Sine series, 242
- Skip connections, 429
- Slow and smooth assumption, 328
- Snell's Law, 91
- Sobel-Feldman operator, 300
- Softmax, 148
- Softmax regression, 149
- Specular surfaces, 78
- Steering equation, 374
- Stereo anaglyph, 701
- Stochastic gradient descent, 159
- Strided convolution, 345
- Structured prediction, 608
- Subitizing, 495
- Supervised learning, 139
- SURF, 727
- Surrogate loss, 157
- Systems
 - Equivariant, 226
 - Linear, 223
- t-Distributed Stochastic Neighbor Embedding, 873
- Talks, 903
- Telescope, 103
- Tensor, 185
- Testing phase, 137
- Text-to-image, 877
- Textons, 496
- Texture analysis, 497
- Texture gradients, 757
- Texture perception, 494
- Texture synthesis, 496
- The trolley problem, 72
- Time to contact, 817

Tokens, 440
Training phase, 137
Transformers, 439
Translation equivariance, 427
Translation invariance, 427
Trichromacy, 9

U-Net, 428
Underfitting, 161
Universal approximation theorem, 182
Unstructured prediction, 607
Unsupervised learning, 140
Upsampling layer, 413

Validation dataset, 164
Vanishing gradient, 155
Vanishing point, 736, 815
Variational autoencoder, 586
 Conditional, 609
Variational inference, 592
Vasarely visual illusion, 306
Vector embedding, 528
Vector fields, 786
Vector quantization, 537
View synthesis, 789
Views (representation learning), 544
Virtual camera plane, 84
Vision-language models, 869
Visual cortex, 22
Visual memex, 847
Visual psychophysics, 270
Visual question answering, 883
Volume rendering, 790
Voxels, 786

Weight decay, 166
Weight tying, 214
Window scanning, 855

YOLO, 859

Zero-crossings, 305

Zeroth-order optimization, 152

Adaptive Computation and Machine Learning series

Francis Bach, editor

Bioinformatics: The Machine Learning Approach, Pierre Baldi and Søren Brunak, 1998

Reinforcement Learning: An Introduction, Richard S. Sutton and Andrew G. Barto, 1998

Graphical Models for Machine Learning and Digital Communication, Brendan J. Frey, 1998

Learning in Graphical Models, edited by Michael I. Jordan, 1999

Causation, Prediction, and Search, second edition, Peter Spirtes, Clark Glymour, and Richard Scheines, 2000

Principles of Data Mining, David J. Hand, Heikki Mannila, and Padhraic Smyth, 2000

Bioinformatics: The Machine Learning Approach, second edition, Pierre Baldi and Søren Brunak, 2001

Learning Kernel Classifiers: Theory and Algorithms, Ralf Herbrich, 2002

Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond, Bernhard Schölkopf and Alexander J. Smola, 2002

Introduction to Machine Learning, Ethem Alpaydın, 2004

Gaussian Processes for Machine Learning, Carl Edward Rasmussen and Christopher K. I. Williams, 2006

Semi-Supervised Learning, edited by Olivier Chapelle, Bernhard Schölkopf, and Alexander Zien, 2006

The Minimum Description Length Principle, Peter D. Grünwald, 2007

Introduction to Statistical Relational Learning, edited by Lise Getoor and Ben Taskar, 2007

Probabilistic Graphical Models: Principles and Techniques, Daphne Koller and Nir Friedman, 2009

Introduction to Machine Learning, second edition, Ethem Alpaydın, 2010

Machine Learning in Non-Stationary Environments: Introduction to Covariate Shift Adaptation, Masashi Sugiyama and Motoaki Kawanabe, 2012

Boosting: Foundations and Algorithms, Robert E. Schapire and Yoav Freund, 2012

Foundations of Machine Learning, Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalker, 2012

Machine Learning: A Probabilistic Perspective, Kevin P. Murphy, 2012

Introduction to Machine Learning, third edition, Ethem Alpaydın, 2014

Deep Learning, Ian Goodfellow, Yoshua Bengio, and Aaron Courville, 2017

Elements of Causal Inference: Foundations and Learning Algorithms, Jonas Peters, Dominik Janzing, and Bernhard Schölkopf, 2017

Machine Learning for Data Streams, with Practical Examples in MOA, Albert Bifet, Ricard Gavaldà, Geoffrey Holmes, Bernhard Pfahringer, 2018

Reinforcement Learning: An Introduction, second edition, Richard S. Sutton and Andrew G. Barto, 2018

Foundations of Machine Learning, second edition, Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalker, 2019

Introduction to Natural Language Processing, Jacob Eisenstein, 2019

Introduction to Machine Learning, fourth edition, Ethem Alpaydın, 2020

Knowledge Graphs: Fundamentals, Techniques, and Applications, Mayank Kejriwal, Craig A. Knoblock, and Pedro Szekely, 2021

Probabilistic Machine Learning: An Introduction, Kevin P. Murphy, 2022

Machine Learning from Weak Supervision: An Empirical Risk Minimization Approach, Masashi Sugiyama, Han Bao, Takashi Ishida, Nan Lu, Tomoya Sakai, and Gang Niu, 2022

Introduction to Online Convex Optimization, second edition, Elad Hazan, 2022

Distributional Reinforcement Learning, Marc G. Bellemare, Will Dabney, and Mark Rowland, 2023

Probabilistic Machine Learning: Advanced Topics, Kevin P. Murphy, 2023

Foundations of Computer Vision, Antonio Torralba, Phillip Isola, and William T. Freeman, 2024